cadence®

Tempus Stylus Text Command Reference Manual

Product Version 25.13
April 2026

© 2026 Cadence Design Systems, Inc. All rights reserved.
Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as expressly permitted below, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, including automated processes, training or services involving a large language model, foundation model, deep machine learning, generative artificial intelligence, or any other similar process or technology, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, customers are granted permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used solely for personal, informational, and noncommercial purposes;
2. The publication may not be modified in any way;
3. Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and
4. Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information. Cadence is committed to using respectful language in our code and communications. We are also active in the removal and/or replacement of inappropriate language from existing content. This product documentation may however contain material that is no longer considered appropriate but still reflects long-standing industry terminology. Such content will be addressed at a time when the related software can be updated without end-user impact.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Contents

About This Manual 7
Audience 7
Conventions Used in This Manual 7
Related Documents 9
Tempus Product Documentation 9
1 10
Advanced Timing Tcl Scripting Commands 10
Overview 11
all_* Commands 11
Examples 11
get_* Commands 11
Examples 12
property Commands 12
Examples 12
collection Commands 12
Examples 13
add_to_collection 13
all_clocks 14
all_connected 15
all_fanin 16
all_fanout 18
all_inputs 20
all_instances 21
all_outputs 22
all_registers 23
append_to_collection 25
compare_collections 27
copy_collection 28
define_property 29
filter_collection 30
foreach_in_collection 32
get_activity 34

get_arcs 37
get_cells 38
get_clocks 40
get_designs 41
get_generated_clocks 42
get_lib_arcs 43
get_lib_cells 44
get_lib_pg_pins 46
get_lib_pins 47
get_libs 49
get_nets 50
get_object_name 51
get_path_groups 52
get_pg_nets 53
get_pg_pins 54
get_pins 55
get_ports 57
get_power 58
get_property 60
index_collection 157
query_objects 158
range_collection 159
remove_from_collection 161
report_object_property_names 162
report_property 163
sizeof_collection 165
sort_collection 166
parse_proc_arguments 167
define_proc_arguments 169
is_collection_object 171

2 173

ART Based Flow Commands 173

get_module_view 173
set_module_view 175

3 179

Basic Database Access Tcl Commands and Attributes 179

add_thru_substrate_insts 180
compress_special 182
convert_dbu_to_um 183
convert_um_to_dbu 184
create_cell_obs 185
db_hinst_connection 186
decompress_special 186
define_attribute 187
delete_cell_obs 190
delete_floating_nets 191
delete_obj 192
delete_thru_substrate_insts 193
get_computed_shapes 194
get_db 197
get_db Category Attributes 208
get_edge 209
get_obj_in_area 211
get_rect 216
get_selectively_expanded_modules 218
get_transform_shapes 219
get_via_pillars 221
init_hier_cell 222
is_attribute 223
report_floating_nets 224
report_metal_stack 226
report_net_wires 228
  Node Information 228
  Edge Information 228
report_via_def 230
reset_db 232
set_db 234
set_db Category Attributes 236
set_selectively_expanded_modules 238
set_via_pillars 243
sort_db 245
tech Category Attributes 247
uniquify 248

write_preserves 251
4 253
Check Commands and Attributes 253
check_ac_limit Category Attributes 253
check_ac_limits 271
check_power_vias 272
create_marker 281
delete_markers 283
gui_show_markers 284
read_markers 288
report_markers 291
5 295
Clock Tree Debugger Commands 295
gui_ctd_close 295
gui_ctd_get_title 297
gui_ctd_get_window_id 298
gui_ctd_highlight 299
gui_ctd_open 303
gui_ctd_write_view 307
gui_ctd_set_title 308
gui_ctd_show_skew_group 310
gui_ctd_write_histogram 312
gui_ctd_zoom 313
6 317
Delay Calculation Commands and Attributes 317
create_aging_stress_profile 317
delaycal Category Attributes 320
get_aging_activity 338
get_aging_analysis_mode 339
report_aging_activity 341
report_aging_stress_profile 343
report_delay_calculation 344
set_aging_activity 357
set_aging_analysis_mode 360
write_exclude_nets 364

write_set_load 365
7 367
Distributed MMMC Commands 367
distribute_command 368
distribute_design_views 370
distribute_read_db 373
distribute_views 376
get_distributed_variables 382
reset_servers 384
set_distributed_mmmc_mode 386
set_distributed_variables 389
8 392
Flowkit Commands and Attributes 392
check_flow 393
create_flow 395
create_flow_step 397
define_feature 399
delete_flow_config 401
edit_flow 402
end_flow 405
flow Category Attributes 405
flowtool Category Attributes 417
get_feature 420
get_flow_config 422
init_flow 423
is_feature 424
is_flow 425
read_flow 426
report_flow 427
reset_flow 429
run_flow 432
schedule_flow 435
set_feature 437
set_flow_config 438
set_flowkit_read_db_args 439
set_flowkit_write_db_args 442

write_flow 447
write_flow_template 449
9 452
General Commands and Attributes 452
alias 453
check_ccr 455
decrypt 456
define_proc 457
define_variables 462
delete_dangling_ports 463
delete_floating_constants 464
deselect_obj 466
encrypt 467
eval_legacy 468
get_common_ui_map 469
get_legacy 470
is_common_ui_mode 471
Message Commands 472
get_message 473
report_messages 475
set_message 477
pop_snapshot_stack 479
print_message 480
program Category Attributes 482
push_snapshot_stack 483
rename_obj 485
rename_snapshot 488
report_area 489
report_dangling_ports 493
report_hidden_usage 494
report_obj Category Attributes 496
report_obj_display_limit 496
resume 496
select_obj 497
set_layer_preference 498
set_log_post_cmd 500

set_proc_verbose 501
suspend 502
system Category Attributes 503
time_info 510
uniquify_filename 512
Utility Commands 513
check_netlist 515
convert_legacy_to_common_ui 518
create_metric_page 518
create_metric_table 519
create_metric_table_heading 520
create_snapshot 521
define_metric 523
delete_metric 528
delete_metric_page 529
delete_metric_run 530
delete_metric_table 531
delete_metric_table_heading 532
get_all_layers 533
get_metric 534
get_metric_alias 536
get_metric_config 537
get_metric_definition 538
gui_bind_key 539
gui_move_cursor 540
metric Category Attributes 541
puts 547
read_metric 548
read_metric_config 549
redirect 550
report_disk 552
report_metric 554
report_netlist_statistics 555
report_obj 556
reset_metric_config 558
reset_metric_run_id 560
run_metric_category 561

set_compress_level 562
set_license_check 563
set_limited_access_feature 565
set_metric 566
set_metric_alias 567
set_metric_header 568
set_preference 569
source 570
update_inst_name 571
update_metric 572
update_metric_definition 574
view_log 579
write_metric 580
write_metric_config 582
read_metric_html_package 583
get_metric_header 584
get_preference 585
delete_metric_header 586
10 588
GUI Commands 588
clear_all_rulers 590
create_gui_shape 591
create_ruler 594
display_temperature_result 595
gui_add_marker 595
gui_add_ui 597
gui_attach_to_cursor 602
gui_clear_highlight 603
gui_create_user_bundle_net 605
gui_delete_objs 607
gui_delete_ui 608
gui_delete_user_bundle_net 609
gui_deselect 610
gui_dim_foreground 611
gui_find_ui 612
gui_fit 617

gui_get_edge_number 618
gui_get_objects_from_schematic 621
gui_get_schematic 623
gui_get_ui 624
gui_get_visible_bumps 632
gui_get_visible_net_groups 633
gui_get_visible_nets 634
gui_highlight 635
gui_highlight_pin 639
gui_highlight_pin_connection 642
gui_list_marker 646
gui_open_schematic 647
gui_pan 650
gui_pan_center 652
gui_pan_page 653
gui_redraw 654
gui_remove_marker 655
gui_report_ruler 656
gui_schematic_add_selected 658
gui_schematic_load_view 659
gui_schematic_write_view 660
gui_select 661
gui_select_highlighted 663
gui_set_obj_color 664
gui_set_power_rail_display 667
gui_set_power_rail_layers_nets 679
gui_set_tool 685
gui_set_ui 687
gui_set_visible_bumps 695
gui_set_visible_net_groups 697
gui_set_visible_nets 698
gui_show_edge_number 699
gui_show_user_bundle_nets 701
gui_view_box 703
gui_write_picture 704
gui_zoom 707
read_power_rail_results 710

report_power_rail_results 718
view_last 727
view_next 728
write_to_gif 729
11 731
ILM and FlexILM Commands and Attributes 731
clear_active_logic_view 731
commit_module_model 733
create_active_logic_view 735
create_ilm_from_files 736
flatten_ilm 739
get_ilm_type 740
ilm Category Attributes 741
read_ilm 744
report_ilm_status 745
reset_ilm 747
set_ilm 748
set_ilm_type 749
unflatten_ilm 751
update_partition 752
write_ilm 754
12 758
Import and Export Commands and Attributes 758
add_route_vias Category Attributes 759
check_design 760
design Category Attributes 764
init_design 768
init Category Attributes 771
read_db 774
read_def 779
read_instance_voltage 783
read_libs 786
read_mmmc 788
read_netlist 791
read_physical 793
read_physical Category Attributes 797

read_rcdb 799
read_sdc 800
relink_db_files 801
set_cell_power_domain 803
set_library_unit 805
set_power_def_files 807
set_power_lib_files 808
write_db 810
write_db Category Attributes 816
write_def 818
write_def Category Attributes 827
write_mmmc 827
write_netlist Category Attributes 829
write_via_defs 830
check_design Category Attributes 833
reset_sdc 834
switch_top_cell 836
13 838
Interactive ECO Commands and Attributes 838
connect 838
connect_hpin 842
connect_pin 844
create_hport 847
create_module 850
create_port 851
delete_inst 853
disconnect 854
disconnect_hpin 856
disconnect_pin 857
gui_show_buffer_tree 859
update_inst 860
14 864
Low Power Commands and Attributes 864
add_pg_netlist_power_intent 864
check_power_domains 866
clone_msv_gate 869

commit_power_intent 871
cut_power_domain_by_overlaps 872
delete_power_intent 873
get_cpf_user_attributes 874
power_intent Category Attributes 875
read_power_intent 880
report_isolation 882
report_partition_boundary_ports 884
report_power_domains 885
report_shifters 892
report_voltage 895
set_lib_binding_by_voltage 897
update_power_domain 899
write_power_intent 904

15 907

Manual ECO Commands and Attributes 907

connect_net 907
create_inst 909
create_hinst 913
create_hnet 915
create_net 918
delete_nets 920
delete_routes 922
disconnect_net 923
eco_add_repeater 924
eco_delete_repeater 933
eco_update_cell 938
eco Category Attributes 947
read_eco 956
write_eco 958

16 960

Multiple-CPU Processing Commands 960

check_multi_cpu_usage 961
get_distributed_hosts 964
get_multi_cpu_usage 966
monitor_distributed_hosts 968

report_resource 970
set_distributed_hosts 974
set_multi_cpu_usage 981
17 989
OpenAccess Commands and Attributes 989
compare_oa_cellview 989
create_oa_lib 992
create_oa_lib_property 995
get_oa_default_rule_lib 997
get_oa_design_data 1000
oa_read_write Category Attributes 1002
read_oa 1018
register_trigger 1021
unlock_oa_design 1022
update_oa_lib 1023
write_lef_library 1026
write_oa_techfile 1029
18 1032
Machine Learning Commands 1032
init_ml 1032
write_ml_design 1035
write_ml_library 1038
write_ml_timing_graph 1040
write_ml_timing_path 1042
19 1047
Placement Commands and Attributes 1047
add_fillers Category Attributes 1047
gui_clear_scan_chains 1051
gui_clear_spare_cells 1052
gui_show_scan_chains 1053
gui_show_spare_cells 1054
place Category Attributes 1055
report_congestion 1062
set_cell_edge_spacing 1064
set_cell_edge_type 1067

report_cell_edge_type 1070
report_density_map 1071
report_pin_density_map 1073
write_physical_context_data 1076
read_physical_context_data 1077
reset_physical_context_data 1078
20 1080
Power Calculation Commands and Attributes 1080
encrypt_thermal_file 1080
get_default_switching_activity 1082
get_dynamic_power_simulation 1083
merge_vtm 1084
power Category Attributes 1087
propagate_activity 1153
query_power_data 1155
read_activity_file 1158
read_activity_map_file 1165
read_power_db 1167
read_temperature 1168
report_inst_power 1169
report_power 1170
report_unannotated_nets 1189
reset_power_activity 1191
set_default_switching_activity 1192
set_dynamic_power_simulation 1196
set_instance_temperature_file 1197
set_power 1198
set_power_analysis_temperature 1206
set_power_include_file 1207
set_power_output_dir 1208
set_switching_activity 1209
set_twf_attribute 1212
set_virtual_clock_network_parameters 1216
write_power_constraints 1217
write_tcf 1218
analyze_ir_paths 1219

21 1222
Power Route Category Attributes 1222
route_special Category Attributes 1222
22 1234
RC Extraction Commands and Attributes 1234
extract_parasitics 1234
extract_rc Category Attributes 1236
read_parasitics 1250
report_net_parasitics 1254
report_rcdb 1257
report_unit_parasitics 1259
write_parasitics 1262
write_rcdb 1265
23 1267
Signal Integrity Commands and Attributes 1267
analyze_jitter 1269
check_noise 1274
define_bundled_bus 1277
define_glitch_holding_resistance 1279
define_parameter 1280
get_glitch_threshold 1283
get_multi_input_switching_mode 1284
get_via_variation_mode 1285
merge_noise_reports 1286
report_bundled_bus 1287
report_double_clocking 1290
report_glitch_thresholds 1293
report_instance_cdb 1296
report_jitter 1298
report_mutex_condition 1301
report_noise 1303
report_noise_lib_pin 1319
report_noise_propagation 1321
report_parameter 1324
report_voltage_scaling 1325

report_voltage_swing 1327
reset_glitch_derate 1329
reset_noise_lib_pin 1330
set_advanced_variation_mode 1331
set_annotated_glitch 1332
set_cdb_binding 1335
set_glitch_derate 1336
set_glitch_threshold 1337
set_model_priority 1340
set_mutex_condition 1341
set_noise_lib_pin 1342
set_outbound_report 1343
set_quiet_aggressor 1344
set_via_variation_mode 1346
si Category Attributes 1348
update_glitch 1364
write_noise_eco_nets 1365
write_spice_deck 1366
24 1384
System Commands - Basic Tool-Tcl Commands 1384
gui_hide 1384
gui_show 1385
help 1386
man 1390
tempus 1391
25 1399
Tempus ECO Commands and Attributes 1399
opt_signoff 1400
opt_signoff Category Attributes 1403
read_hierarchical_def 1457
read_partition 1463
write_eco_opt_db 1464
26 1465
Timing Analysis Commands and Attributes 1465
check_boundary_model_consistency 1471

check_ipd_logic 1472
check_ldb_version 1476
check_timing 1477
check_timing_library 1491
check_timing_library_consistency 1494
commit_boundary_model_constraints 1495
compare_timing_context 1496
create_analysis_view 1501
create_basic_path_groups 1503
create_boundary_model 1505
create_constraint_mode 1507
create_delay_corner 1509
create_glitch_boundary_model 1514
create_ipd_configuration 1517
create_library_set 1519
create_opcond 1523
create_parasitic_coordinate_transform 1525
create_pg_net_group 1529
create_property_alias 1531
create_rc_corner 1532
create_timing_condition 1537
delete_analysis_view 1539
get_capacitance_unit 1540
get_clock_network_objects 1541
get_clock_to_clock_association 1542
get_constant 1544
get_interactive_constraint_modes 1546
get_propagated_clock 1547
get_resistance_unit 1548
get_socv_rc_variation_factor 1549
get_socv_reporting_nsigma_multiplier 1550
get_time_unit 1551
gui_show_timing_map 1552
list_libraries 1553
merge_timing_reports 1554
read_boundary_model 1557
read_ipd_scope 1560

read_path_descriptions 1561
read_sdf 1563
read_spef 1569
read_timing_context 1575
read_twf 1577
report_analysis_coverage 1579
report_analysis_summary 1589
report_analysis_views 1594
report_annotated_assertions 1598
report_annotated_check 1602
report_annotated_delay 1608
report_annotated_parasitics 1612
report_aocv_derate 1617
report_boundary_model_constraints 1621
report_case_analysis 1622
report_cell_instance_timing 1628
report_clock_gating_check 1633
report_clock_network_transitions 1636
report_clock_propagation 1638
report_clock_sense 1641
report_clock_timing 1642
report_clocks 1660
report_constraint 1670
report_cppr 1692
report_critical_insts 1698
report_critical_nets 1703
report_critical_pins 1705
report_critical_pins 1705
report_design 1706
report_fanin 1709
report_fanout 1713
report_freq_violation 1716
report_inactive_arcs 1720
report_instance_library 1725
report_ipd_configuration 1727
report_ipd_crossings 1730
report_lib_arcs 1734

report_min_pulse_width 1739
report_mode 1745
report_nets 1747
report_parasitic_coordinate_transforms 1752
report_path_based_aocv_derate 1753
report_path_exceptions 1756
report_path_groups 1760
report_pg_net_group 1762
report_ports 1764
report_reference 1769
report_slack_histogram 1771
report_socv_interconnect_variation 1775
report_socv_library 1777
report_statistical_timing_derate_factors 1781
report_timing 1785
report_timing_context_clock_mapping 1844
report_timing_derate 1846
report_timing_lib 1852
report_timing_robustness 1855
report_timing_summary 1857
report_wire_load 1861
reset_instance_library 1863
reset_path_adjust 1864
reset_path_adjust_group 1865
reset_sdf_assertions 1866
reset_timing_derate 1867
reset_wire_load_mode 1869
reset_wire_load_model 1870
reset_wire_load_selection_group 1871
select_boundary_model_constraints 1872
set_analysis_view 1873
set_boundary_model_clock_mapping 1876
set_boundary_model_view_mapping 1877
set_default_view 1878
set_instance_library 1880
set_interactive_constraint_modes 1881
set_io_thresholds 1883

set_ipd_rail_voltage 1886
set_max_cap_per_freq 1888
set_max_cap_per_freq_transition 1891
set_max_transition_per_freq 1894
set_multi_input_switching_mode 1897
set_opcond 1900
set_path_adjust 1903
set_path_adjust_group 1905
set_socv_constraint_config 1907
set_socv_rc_variation_factor 1909
set_socv_reporting_nsigma_multiplier 1910
set_table_style 1912
set_threshold_voltage_group 1920
set_timing_context_clock_mapping 1922
set_timing_context_view_mapping 1924
set_timing_derate 1925
set_timing_library 1936
set_vt_skew_config 1938
set_vt_skew_derate 1940
set_wire_load_mode 1942
set_wire_load_model 1945
set_wire_load_selection_group 1947
timing_enable_boundary_model_eco 1949
timing Category Attributes 1949
update_analysis_view 2068
update_constraint_mode 2070
update_delay_corner 2072
update_io_latency 2077
update_library_set 2078
update_model_context 2080
update_opcond 2081
update_rc_corner 2082
update_timing 2086
update_timing_condition 2087
write_boundary_model 2088
write_global_slack_report 2089
write_ipd_scope 2094

write_ldb 2097
write_loop_break_sdc 2099
write_model_context 2100
write_path_descriptions 2102
write_sdc 2105
write_sdf 2108
write_timing_context 2134
write_timing_windows 2136
write_twf 2139
27 2142
Distributed Static Timing Analysis Commands 2142
distribute_launch_deferred_clients 2143
distribute_partition 2144
distribute_print_client_usage 2146
distribute_print_partition_stats 2147
distribute_start_clients 2149
28 2153
Timing Constraint Commands 2153
Path Exception Priorities - Overview 2157
Priority Rules on the Basis of Path Exception Type 2157
Priority Rules on the Basis of Path Exception Command Reference Points 2157
Priority Rules on the Basis of -start/-end (only for set_multicycle_path) 2158
Path Exception Priority Rules Table 2158
Examples of Path Exception Priorities 2159
Bidirectional Pin Defaults 2160
Defaults for Pin (or Port) Constraints 2160
Defaults for Port Constraints 2161
Cycle Accounting Rules - Data Checks 2161
General Single-Cycle Timing 2162
Zero-Cycle Timing 2162
Zero-Cycle Timing with Multi-Cycle Adjust 2163
set_multicycle_path -start 2164
create_clock 2165
create_generated_clock 2168
current_design 2173
current_instance 2174

get_current_constraint_mode 2175
group_path 2176
reset_annotated_check 2178
reset_annotated_delay 2180
reset_annotated_transition 2181
reset_aocv_stage_weight 2183
reset_case_analysis 2184
reset_clock 2185
reset_clock_exclusivity 2187
reset_clock_gating_check 2188
reset_clock_groups 2189
reset_clock_latency 2191
reset_clock_sense 2194
reset_clock_transition 2195
reset_clock_tree_latency 2198
reset_clock_uncertainty 2199
reset_data_check 2202
reset_disable_clock_gating_check 2204
reset_disable_timing 2205
reset_drive 2208
reset_driving_cell 2209
reset_generated_clock 2210
reset_ideal_latency 2212
reset_ideal_network 2213
reset_ideal_transition 2214
reset_input_delay 2215
reset_load 2217
reset_max_capacitance 2218
reset_max_fanout 2219
reset_max_time_borrow 2220
reset_max_transition 2221
reset_min_capacitance 2222
reset_min_fanout 2223
reset_min_pulse_width 2224
reset_min_transition 2225
reset_mode 2226
reset_output_delay 2228

reset_path_exception 2231
reset_path_group 2238
reset_pll_timing 2239
reset_propagated_clock 2239
reset_property 2242
reset_pulse_clock_max_transition 2243
reset_pulse_clock_max_width 2244
reset_pulse_clock_min_transition 2245
reset_pulse_clock_min_width 2246
reset_resistance 2247
reset_sense 2248
set_active_clocks 2249
set_annotated_check 2250
set_annotated_delay 2252
set_annotated_transition 2255
set_aocv_interface_path_offset 2257
set_aocv_stage_weight 2259
set_aocv_thresholds 2260
set_case_analysis 2261
set_clock_exclusivity 2263
set_clock_gating_check 2265
set_clock_groups 2268
set_clock_latency 2271
set_clock_sense 2274
set_clock_transition 2277
set_clock_uncertainty 2279
set_data_check 2282
set_disable_clock_gating_check 2284
set_disable_timing 2285
set_dont_touch 2287
set_dont_touch_network 2289
set_dont_use 2290
set_drive 2291
set_driving_cell 2293
set_false_path 2295
set_fanout_load 2299
set_ideal_latency 2300

set_ideal_network 2302
set_ideal_transition 2304
set_input_delay 2306
set_input_transition 2310
set_load 2312
set_logic_one 2315
set_logic_zero 2316
set_max_capacitance 2317
set_max_delay 2319
set_max_fanout 2325
set_max_time_borrow 2327
set_max_transition 2328
set_min_capacitance 2331
set_min_delay 2334
set_min_fanout 2341
set_min_pulse_width 2343
set_min_transition 2345
set_mode 2348
set_multicycle_path 2350
set_normalized_driver_waveform_lib 2355
set_output_delay 2356
set_pll_timing 2359
set_propagated_clock 2362
set_property 2364
set_pulse_clock_max_transition 2365
set_pulse_clock_max_width 2367
set_pulse_clock_min_transition 2369
set_pulse_clock_min_width 2371
set_resistance 2373
set_sense 2375
write_annotated_transition 2378
write_load 2381
29 2384
Timing Debug Commands 2384
analyze_paths_by_bottleneck 2385
analyze_paths_by_clock_domain 2388

analyze_paths_by_critical_false_path 2390
analyze_paths_by_default_path_group 2392
analyze_paths_by_drv 2394
analyze_paths_by_hier_port 2397
analyze_paths_by_hierarchy 2399
analyze_paths_by_view 2404
create_histogram_png 2405
create_path_category 2406
delete_path_category 2415
gui_gtd_highlight_timing_report 2416
gui_open_gtd 2417
load_failure_report 2418
read_path_categories 2418
read_timing_debug_report 2420
write_category_summary 2423
write_histogram_view 2425
write_path_categories 2426
write_path_list_summary 2427
write_text_timing_report 2428
zoom_highlighted_timing_path 2431
30 2433
Timing Modeling Commands 2433
Extracted Timing Model - ETM 2434
check_timing_model 2435
merge_model_timing 2445
report_timing_model 2447
write_timing_model 2454
Prototype Timing Model - PTM 2458
create_ptm_constraint_arc 2460
create_ptm_delay_arc 2462
create_ptm_drive_type 2464
create_ptm_generated_clock 2466
create_ptm_load_type 2468
create_ptm_model 2469
create_ptm_path_type 2470
create_ptm_port 2472

report_ptm_model 2474
save_ptm_model 2477
set_ptm_design_rule 2478
set_ptm_global_parameter 2479
set_ptm_port_drive 2480
set_ptm_port_load 2481
Index 2483

About This Manual

The Cadence® Stylus Implementation System family of products provides an integrated solution for an RTL-to-GDSII design flow. This manual describes the syntax for the Stylus Implementation System (Stylus) text commands.

The chapters are presented alphabetically by general command functionality. Within each chapter, the text commands appear alphabetically.

Audience

This manual is written for experienced designers of digital integrated circuits. Such designers must be familiar with design planning, placement and routing, block implementation, chip assembly, and design verification. Designers must also have a solid understanding of UNIX and Tcl/Tk programming.

Conventions Used in This Manual

This section describes the typographic and syntax conventions used in this manual.

`text`	Indicates text that you must type exactly as shown. For example: `analyze_connectivity -analyze all`
`text`	Indicates information for which you must substitute a name or value. In the following example, you must substitute the name of a specific file for *`configfile`*: `wroute filename` *`configfile`*
text	Indicates the following: • Text found in the graphical user interface (GUI), including form names, button labels, and field names • Terms that are new to the manual, are the subject of discussion, or need special emphasis • Titles of manuals

`[ ]`	Indicates optional arguments. In the following example, you can specify none, one, or both of the bracketed arguments: `command [-arg1] [-arg2 value]`
`[ | ]`	Indicates an optional choice from a mutually exclusive list. In the following example, you can specify any of the arguments or none of the arguments, but you cannot specify more than one: `command [arg1 | arg2 | arg3 | arg4]`
`{ | }`	Indicates a required choice from a mutually exclusive list. In the following example, you must specify one, and only one, of the arguments: `command {arg1 | arg2 | arg3}`
`{[ ] [ ]}`	Indicates a required choice of one or more items in a list. In the following example, you must choose one argument from the list, but you can choose more than one: `command {[arg1] [arg2] [arg3]}`
`{ }`	Indicates curly braces that must be entered with the command syntax. In the following example, you must type the curly braces: `command arg1 {x y}`
. . .	Indicates that you can repeat the previous argument.
. . .	Indicates an omission in an example of computer output or input.

Command - Subcommand	Indicates a command sequence, which shows the order in which you choose commands and subcommands from the GUI menu. In the following example, you choose *Power* from the menu, then *Power Planning* from the submenu, and then *Add Rings* from the displayed list: *Power - Power Planning - Add Rings* This sequence opens the *Add Rings* form.

Related Documents

For more information about the Tempus family of products, see the following documents. You can access these and other Cadence documents with the Cadence Help documentation system.

Tempus Product Documentation

- *Stylus Common UI Database Object Information*
  Provides information on database objects.
- *Stylus Common UI Text Command Reference*
  Describes the Tempus text commands, including syntax and examples.

For a complete list of documents provided with this release, see the Cadence Help online documentation system.

1

Advanced Timing Tcl Scripting Commands

- add_to_collection
- all_clocks
- all_connected
- all_fanin
- all_fanout
- all_inputs
- all_instances
- all_outputs
- all_registers
- append_to_collection
- compare_collections
- copy_collection
- define_property
- filter_collection
- foreach_in_collection
- get_activity
- get_arcs
- get_cells
- get_clocks
- get_designs
- get_generated_clocks
- get_lib_arcs
- get_lib_cells
- get_lib_pg_pins
- get_lib_pins
- get_libs
- get_nets
- get_object_name
- get_path_groups
- get_pg_nets
- get_pg_pins
- get_pins
- get_ports
- get_power
- get_property
- index_collection
- query_objects
- range_collection

- remove_from_collection
- report_object_property_names
- report_property
- sizeof_collection
- sort_collection
- parse_proc_arguments
- define_proc_arguments
- is_collection_object

Overview

The following lists the Advanced Timing Tcl Scripting commands. For each category of similar commands, we explain what they are used for followed by some simple examples.

all_* Commands

The `all_*` commands return a collection of all the items of a specific type. Several of the commands provide options to filter the collection based on certain properties:

- `all_clocks`
- `all_connected`
- `all_fanin`
- `all_fanout`
- `all_inputs`
- `all_instances`
- `all_outputs`
- `all_registers`

Examples

- To set all the clocks to propagated mode, use the following command:
  ```
  set_propagated_clock [all_clocks]
  ```
- To report the worst timing path that begins at a register and ends at an output port, use the following command:
  ```
  report_timing -from [all_registers] -to [all_outputs]
  ```
- To store a collection of pins that fanout from a specific port, use the following command:
  ```
  set clk1Fanout [all_fanout -from clk1 -pin_levels 1]
  ```

get_* Commands

The `get_*` commands return a collection of items filtered by name pattern or object type. Additionally, you can use the `-filter` option for most of the `get_*` commands to filter the collection by property value. See the get_property command for a list of all the properties.

The `filter_expression` for the `-filter` option can be created by combining several object properties using the following relational and logical operators: `>`, `<`, `==`, `!=`, `<=`, `>=`, `&&`, `||`, `=~`, `!~`, `AND`, and `OR`. You also can use parentheses to create expressions. The `get_*` commands include the following:

- `get_arcs`
- `get_cells`
- `get_clocks`
- `get_designs`
- `get_generated_clocks`
- `get_lib_arcs`
- `get_lib_cells`

- `get_lib_pins`
- `get_libs`
- `get_nets`
- `get_object_name`
- `get_path_groups`
- `get_pins`
- `get_ports`

Examples

- To return a collection of all input ports (same as running `[all_inputs]`), use the following command:
  ```
  get_ports -filter "direction==in"
  ```
- To return a collection of pins that match `*/D`, use the following command:
  ```
  get_pins -hier -filter "hierarchical_name =~ */D"
  ```
- To return a collection of the libraries a cell belongs to, use the following command:
  ```
  get_lib_cells INVXL
  ```

property Commands

The report_object_property_names command lists the properties for the different object types. This command, without arguments, will list all the properties for all object types. Use the `-type` option to list properties for a specific object type. Legal option types are: `cell`, `clock`, `design`, `lib`, `lib_cell`, `lib_pin`, `lib_timing_arc`, `net`, `path_group`, `pin`, `port`, `timing_arc`, `timing_path`, or `timing_point`.

Examples

- The get_property command returns the value of the specified property. To report the clock period of clk1, use the following command:
  ```
  get_property [get_clocks clk1] period
  ```
- The report_property command returns the properties and their values for a specified object. To report the properties for clock clk1, use the following command:
  ```
  report_property [get_clocks clk1]
  ```
- For timing related properties such as properties for timing arcs, paths, and points run `report_timing -collection`. This will return the results in a collection of timing paths.
  - To report timing and store the top 10 paths as a collection in the variable `$paths`, use the following command:
    ```
    set paths [report_timing -collection -max_paths 10]
    ```
  - To filter `$paths` to include only those paths with slack between -0.806ns and 0ns, use the following command:
    ```
    set newPaths [filter_collection $paths {(slack > -0.806 && slack < 0.0)}]
    ```

collection Commands

Once you have the collection defined you can manipulate, filter, cycle through, and query it using the collection related commands:

- `add_to_collection`
- `append_to_collection`
- `compare_collections`
- `copy_collection`
- `filter_collection`
- `foreach_in_collection`
- `index_collection`
- `query_objects`
- `remove_from_collection`

- `sizeof_collection`
- `sort_collection`

Examples

- To list the contents of a collection, use the following command:
  ```
  query_objects $collectionName
  ```
- To cycle through the items of a collection, use the following command:
  ```
  foreach_in_collection iCell [get_cells *] {
  Puts "Cell = [get_property $iCell hierarchical_name]"
  }
  ```

add_to_collection

```
add_to_collection
base_collection
second_collection_or_list
[-unique]
```

Creates a new collection by adding objects from one collection to another collection. The following rules apply:

- If all of the objects in the base collection are of the same type, only the objects in the second collection that are the same type as those in the base collection are added to the new collection. If you specify an object list instead, all of the objects in the list must be the same type as those in the base collection.
- If the base collection contains different types of objects, all objects in the second collection are added. In this situation, you cannot specify an object list.

If the software finds no objects to add, an empty collection is returned.

Parameters

base_collection	Specifies the base collection of objects. Objects from the second collection or object list are added to this collection to create the new collection.
second_collection_or_list	Specifies the second object collection, or a list of objects.
-unique	Removes any duplicate objects from the new collection.

Examples

- The following command returns a collection which contains both the output ports as well as the input ports:
  ```
  tempus> add_to_collection [all_outputs] [all_inputs]
  ```

  Output:
  ```
  CLK4 CLK3 CLK2 CLK1 IN4 IN3 IN2 IN1 OUT4 OUT3 OUT2 OUT1 OUT5
  ```

  Here, the input and output ports are as follows:

  ```
  tempus> all_inputs
  CLK4 CLK3 CLK2 CLK1 IN4 IN3 IN2 IN1
  ```

  ```
  tempus> all_outputs
  OUT4 OUT3 OUT2 OUT1 OUT5
  ```

all_clocks

```
  all_clocks
[-help]
```

Creates a collection of all the clocks in the current design. If no clocks were defined, an empty string `""` is returned.

To filter clocks, use the `get_clocks` command to create a collection of clocks matching a specific pattern. These clocks can be assigned to a variable or passed using another command.

Parameters

`-help`	Prints out the command usage.

Examples

- The following example applies the `set_propagated_clock` command to all the clocks in the design:

  ```
  set_propagated_clock [all_clocks]
  ```

Related Commands

- `create_clock`
- `get_clocks`
- `set_propagated_clock`
- `all_inputs`

all_connected

```
all_connected
[-help]
[-leaf]
obj
```

Returns a collection of all of the objects connected to the specified net, pin, or port.

Note: If you specify a pin or port object, the `all_connected` command will return the net connected to that pin or port. If you specify a net object, this command will return ports, pins, or hpins connected to that net.

Parameters

`-help`	Prints out the command usage.
`-leaf`	Returns all instance pins connected to the specified net. When specified, only the flattened connection points (leaf pins) for the specified net are reported. *Default:* Returns hierarchical ports connected to the specified net.
obj	Specifies the net, pin, or port for which collection of connected objects is to be reported. You can specify an object name, or a single-object collection.

Examples

- The following command returns a collection of all of the objects connected to the pin `b1_Sin1`:

  ```
  all_connected Sin1
  ```

  The software returns the following information:

  ```
  Sin1
  ```

- The following command returns a collection of all of the objects connected to a pin contained in the single collection object i_block1/b1_Sin1:

  ```
  all_connected [get_pins {i_block1/b1_Sin1}]
  ```

  The software returns the following information:
  Sin1

- The following command returns a collection of all of the hierarchical ports connected to the net `net_1`:

  ```
  all_connected [get_nets net_1]
  ```

  The software returns the following information:

  ```
  TL2/l2_in TL1/l1_out
  ```

- The following command returns a collection that contains only the flattened connection points for the net `net_1`:

  ```
  all_connected -leaf [get_nets net_1]
  ```

  The software returns the following information:

  ```
  TL2/L1/RX/D TL1/RX/Q
  ```

all_fanin

```
all_fanin
[-help]
[-hpin]
[-only_cells]
[-startpoints_only]
-to {collection | object_list}
[-trace_through {case_disable |user_disable | all | clocks | loop_snipped | latches}]
[-view view_name]
[-levels value | -pin_levels value]
[ > | >> ]
```

Returns a collection of pins, ports, or cells that exist in the fanin cone of the specified objects.

Note: The `all_fanin` command does not work on netlist traversal but on timing graph traversal.

Parameters

`-help`	Prints out the command usage.
`> | >>`	Writes a list of the returned collection to the specified file. You can add the `.gz` extension to the file name to generate a compressed report. **Note:** The file name must be the last parameter specified.
`-hpin`	Returns the hierarchical pins along with the flat pins.
`-levels value`	Specifies the number of gate levels to go back from a specific pin object to create the collection.
`-only_cells`	Returns a collection of cells in the fanin zone.
`-pin_levels value`	Specifies the number of delay arcs to go back from a specific pin object to create the collection.
`-startpoints_only`	Returns a collection only of valid startpoints (sequential cells and input ports) in the fanin cone.
`-trace_through {case_disable | user_disable | all | clocks | loop_snipped | latches}`	You can specify one or more of the following options: • `case_disable`: Allows traversal of arcs disabled using the `set_case_analysis` command. The arcs disabled using the `set_disable_timing` command are not traversed. • `user_disable`: Allows traversal of arcs disabled using the `set_disable_timing` command, but arcs disabled using `set_case_analysis` command are not traversed. • `all`: Allows traversal through all the arcs. • `clocks`: Allows traversal through pins for which clock constraints, such as, `create_clock`, `create_generated_clock`, are applied. The pins that are disabled using the `set_case_analysis` and `set_disable_timing` commands are not traversed. • `loop_snipped`: Allows trace through the loop snipped arcs. • `latches`: Allows traversal of arcs of the latch cells.
`-to {collection | object_list}`	Specifies the objects for which to return the collection. You can specify a list of objects, or a collection of objects.
`-view view_name`	Specifies the view name for which fan cone traversal is to be performed.

Examples

- The following command returns the collection of cells in the fanin zone of the output port OUT1:
  ```
  all_fanin -to OUT1 -only_cells
  ```
- The following command returns all the valid startpoints (sequential elements and/or input ports) in the fanin zone of OUT1, allowing the traversal through arcs disabled by `set_case_analysis` and `set_disable_timing`:
  ```
  all_fanin -to OUT1 -startpoints_only -trace_through all
  ```

Related Commands

- all_fanout

all_fanout

```
all_fanout
[-help]
[-endpoints_only]
[-hpin]
[-only_cells]
[-trace_through {case_disable | user_disable | all | clocks | loop_snipped | latches}]
[-view view_name]
[-levels value | -pin_levels value]
{-from {collection | object_list} }
[ > | >> ]
```

Returns a collection of pins, ports, or cells that exist in the fanout cone of the specified objects.

Note: The `all_fanout` command does not work on netlist traversal but on timing graph traversal.

Parameters

`-help`	Prints out the command usage.
`-endpoints_only`	Returns a collection of valid endpoints (sequential elements or output ports) in the fanout cone.
`-hpin`	Returns hierarchical pins along with the flat pins.
`> | >>`	Writes a list of the returned collection elements to the specified file. You can add the `.gz` extension to the file name to generate a compressed report. **Note:** The file name must be the last parameter specified.
`-from {`*collection* `|` *object_list*`}`	Specifies the objects for which to return the collection. You can specify a list of objects, or a collection of objects.
`-levels` *value*	Specifies the number of gate levels to go forward from a specific pin object to create the collection.
`-only_cells`	Returns a collection only of the cells in the fanout zone.
`-pin_levels` *value*	Specifies the number delay arcs to go forward from a specific pin object to create the collection.
`-trace_through {case_disable |user_disable | all | clocks | loop_snipped | latches}`	Allows traversal through disabled arcs. You can specify one of the following options: • `case_disable`: Arcs which are disabled using the `set_case_analysis` command and through physical constant are traversed. The arcs disabled using `set_disable_timing` are not traversed. • `user_disable`: Arcs disabled using `set_disable_timing` command are traversed but those disabled using `set_case_analysis` command are not traversed. • `all`: Enables trace through all the arcs. • `clocks`: Allows traversal through pins for which clock constraints (such as, `create_clock`, `create_generated_clock`) are applied. The pins that are disabled using the `set_case_analysis` and `set_disable_timing` commands are not traversed. • `loop_snipped`: Allows trace through the loop snipped arcs. • `latches`: Allows traversal of arcs of the latch cells.
`-view` *view_name*	Specifies the view name for which fan cone traversal is to be performed.

Examples

- The following command returns the collection of cells in the fanout zone of the input port IN1:
  ```
  all_fanout -from IN1 -only_cells
  ```
- The following command returns all the valid endpoints (sequential elements and/oroutput ports) in the fanout zone of IN1, allowing the traversal through arcs disabled by `set_case_analysis` and `set_disable_timing`:
  ```
  all_fanout -from IN1 -endpoints_only -trace_through all
  ```

Related Commands

- all_fanin

all_inputs

```
all_inputs
[-clock list_of_clocks]
[-no_clocks]
[-edge_triggered]
[-level_sensitive]
```

Returns a collection of input or inout ports in the current design.

To filter inputs, use the `get_ports` command to create a collection of ports matching certain criteria. You can assign these ports to a variable or pass them using another command. You can limit the contents of the collection to ports with input delay relative to a specific clock.

Parameters

`-help`	Prints out the command usage.
`-clock` *`list_of_clocks`*	Considers or limits the search to ports that have input delay relative to a specified list of clocks.
`-edge_triggered`	Returns a collection of input ports (specified with the `set_input_delay -clock` constraint), that have edge-triggered input delay.
`-level_sensitive`	Returns a collection of input ports (specified with the `set_input_delay -level_sensitive` constraint), that have level-sensitive input delay.
`-no_clocks`	Returns primary input ports that do not have `create_clock` or `create_generated_clock` constraints.

Examples

- The following command returns the names all of the input ports with input delay relative to an `ideal_clk`:
  ```
  all_inputs -clock ideal_clk
  ```
- The following example lists all input ports in the current design:
  ```
  tempus > all_inputs
  ```
- The following example sets the drive value of all the input ports on the current design to 10:
  ```
  tempus > set_drive 10.0 [all_inputs]
  ```
- The following example marks with a multicycle value of 0 to the all paths from inputs having level-sensitive input delay relative to PHI1 to level-sensitive registers clocked by CDNSCLK:
  ```
  tempus > set_multicycle_path 0 \
        -from [all_inputs -clock CDNSCLK -level_sensitive] \
        -to [all_registers -data_pins -clock CDNSCLK]
  ```

Related Commands

- `all_outputs`
- `get_ports`
- `report_ports`
- `set_input_delay`

all_instances

```
all_instances
obj
[-hierarchical]
```

Returns a collection of all instances available in a design, or all instances of a specified library cell existing in the current design.

Note: You cannot use the `all_instances` command on the top level of the design (`all_instances [get_design top]`).

However, you can use the command for hierarchical modules inside the top level. For example, if `SUB` is a hierarchical module in the top level of the design, you can specify the following command to return all instances of that module:
`all_instances [get_design SUB]`.

Parameters

`-help`	Prints out the command usage.
`-hierarchical`	Specifies that the patterns should be matched hierarchically. If the hierarchical name of an instance at a particular hierarchical level matches the patterns, it is included in the result.
obj	Specifies the name of a design or a library cell in the current design. You can specify a pattern for the design or library cell name, or a single collection of a design or library cell.

Examples

- The following command returns a collection of all the instances of the library cell BUF:

  ```
  tempus> all_instances BUF
  ```

- The following returns a collection of all instances of the sub-module `subblock`:

  ```
  tempus > all_instances [get_design subblock]


  subblock_inst1 subblock_inst2
  ```

all_outputs

```
all_outputs
[-clock list_of_clocks
[-edge_triggered]
[-level_sensitive]
```

Creates a collection of all output ports in the current design. You can limit the contents of a collection to ports with output delay relative to a specific clock. You can assign these ports to a variable or pass them using another command.

Parameters

`-help`	Prints out the command usage.
`-clock list_of_clocks`	Considers only output ports with an output delay relative to a specific collection or a list of clocks.
`-edge_triggered`	Returns a collection of output ports with the `set_output_delay -clock` constraint.
`-level_sensitive`	Returns a collection of output ports with the `set_output_delay -level_sensitive` constraint.

Examples

- The following command returns a collection of output ports with an output delay relative to ideal_clk:
  ```
  all_outputs -clock ideal_clk
  ```

Related Commands

- `all_inputs`
- `get_ports`
- `report_ports`
- `set_output_delay`

all_registers

```
all_registers
[-clock {clock_list}]
[-cells]
[-rise_clock {clock_list}]
[-fall_clock {clock_list}]
[-flops | -edge_triggered]
[-no_hierarchy]
[-latches | -level_sensitive]
[-macros]
[-master_slave]
[-data_pins]
[-clock_pins]
[-output_pins]
[-async_pins]
[-slave_clock_pins]
```

Returns a collection of instances and pins of flip-flops, latches, or sequential macros in the design. A cell is considered sequential if the library cell defines trigger or timing check arcs.

Note: The following options can be used only after building the timing graph:

- `-clock`
- `-rise_clock`
- `-fall_clock`

Parameters

`-help`	Prints out the command usage.
`-async_pins`	Returns a collection of async preset or clear pins.
`-cells`	Creates a collection of cells. This is the default option.
`-clock {clock_list}`	Specifies the name of a clock, a list of clocks, or a clock collection. The `all_registers` command returns a collection of registers clocked by the specified the clock list.
`-clock_pins`	Returns a collection of register clock pins. You can use other parameters to further filter this collection.
`-data_pins`	Returns a collection of register data input pins. You can use other parameters to further filter this collection.
`-fall_clock {clock_list}`	Specifies the name of a clock, a list of clocks, or a clock collection. The `all_registers` command creates a collection of registers opened by the falling edge of the clocks specified in the clock list.
`-flops | -edge_triggered`	Returns a collection of edge-triggered sequential instances. Both the `-flops` and `-edge_triggered` parameters return the same collection. The `-flops` parameter is provided for backward compatibility.
`-latches | -level_sensitive`	Returns a collection of level-sensitive sequential instances. Both `-latches` and `-level_sensitive` parameters return the same collection. The `-latches` parameter is provided for backward compatibility.
`-macros`	Returns sequential instances that are neither flip-flops nor latches.
`-master_slave`	Returns a collection of master-slave register cells.
`-no_hierarchy`	Searches the current design hierarchy level; does not descend the hierarchy into sub-modules.
`-output_pins`	Returns a collection of register output pins.
`-rise_clock {clock_list}`	Specifies the name of a clock, a list of clocks, or a clock collection. The `all_registers` command creates a collection of registers opened by the rising edge of clocks specified in the clock list.
`-slave_clock_pins`	Returns a collection of slave clock pins of the master-slave registers. You can specify the slave clock pins as `clocked_on_also` in the library.

Example

- The following command reports the 10 worst paths going to registers in the design:

```
report_timing -to [all_registers] -max_paths 10
```

- The following script uses the `all_registers` command along with other TCL commands to get the number of flip-flops in the design:

```
set flopName [all_registers -flops]
set flopCount [sizeof_collection $flopName]
puts "Number of flops in the design is $flopCount"
```

Related Commands

- `get_cells`
- `get_pins`

append_to_collection

```
append_to_collection
[-help]
var_name
second_collection_or_list
[-unique]
```

Appends objects from one collection to another collection. The following rules apply:

- If all of the objects in the base collection are of the same type, only the objects in the second collection that are the same type as those in the base collection are appended to the base collection.
- If the base collection contains different types of objects, all objects in the second collection are appended to it. In this situation, you cannot specify an object list.

If the software finds no objects to add, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
var_name	Specifies a pointer to the base collection of objects. Objects in the second collection or object list are appended to this collection. If you specify a pointer that does not refer to an existing base collection, the software creates a new collection that uses the pointer as a reference.
second_collection_or_list	Specifies the second collection of objects, or an object list.
`-unique`	Retains any duplicate objects that already exist in the base collection, but does not append duplicate objects from the second collection to the base collection.

Examples

- The following command appends the nets in the specified collection to the collection `newVar`:

  ```
  append_to_collection newVar [get_nets {CLK1}]
  ```

- The following command appends the specified nets to the collection `newVar`:

  ```
  append_to_collection newVar {Sin1 CLK1 Sout1}
  ```

- You can use the following commands to create another collection:

  ```
  tempus 6> set base_clock [get_clocks sbw_clk]
  0xe50
  tempus 7> set top_clock [get_clocks aclk]
  0xe51
  tempus 8> set base_top_clock ""
  tempus 13> append_to_collection base_top_clock $base_clock
  0xe54
  tempus 14> append_to_collection base_top_clock $top_clock
  0xe54
  tempus 15> get_clocks $base_top_clock
  sbw_clk aclk
  0xe55
  tempus 16> get_clocks $base_clock
  sbw_clk
  0xe56
  tempus 17> get_clocks $top_clock
  aclk
  0xe57
  ```

  Note: If a new collection variable is defined with an existing collection, then both point to the same object set. Any modifications made into either collection will be visible in both cases - because they reference each other.

Related Commands

- range_collection
- compare_collections

compare_collections

`compare_collections` *`collection1 collection2`* `[-order_dependent]`

Compares two collections, and returns a value of `0` if all of the objects contained in both collections are the same. If the objects contained in both collections are different, the software returns a value of `1`.

Parameters

`collection1`	Specifies the first collection.
`collection2`	Specifies the second collection.
`-order_dependent`	Specifies that the order of the objects in both collections must be the same for the collections to be considered the same.

Related Commands

- `append_to_collection`
- `range_collection`
- `filter_collection`

copy_collection

```
copy_collection
[-help]
base_collection
```

Returns a collection that is an exact copy of the specified base collection.

Parameters

`-help`	Prints out the command usage.
`base_collection`	Specifies the base collection of objects.

Related Commands

- `append_to_collection`
- `range_collection`
- `filter_collection`

define_property

```
define_property
[-help]
-object_type string
-type data_type
[-lower_bound val]
[-upper_bound val]
[-string_values {values}]
property_name
```

Defines a new property.

Design objects can be associated with user-defined commands that can be used for analysis and debugging purposes.

You can use the `define_property` command if a define statement is not used for the specified property value(s). To read property values from the library, the command must be issued before the timing library is read.

Parameters

`-help`	Prints out the command usage.
`-lower_bound` *val*	Specifies a lower range - integer, float, or double.
-object_type string	Specifies the type of object on which the property is to be defined.
property_name	Specifies the name of the property.
`-string_values` *values*	Specifies the values that can be defined for the property.
`-type` *data_type*	Specifies the type of property.
`-upper_bound` *val*	Specifies a higher range - integer, float, or double.

Examples

- The following example defines property having object type `pin` and type of property `int`:
  ```
  define_property -object_type pin -type int slk
  ```
- The following example defines a property on a library arc:
  ```
  define_property -object_type lib_arc -type int libArcProperty
  ```
- The following example defines `aocv_weight` property on a library arc:
  ```
  define_property -type float -object_type lib_arc aocv_weight
  ```

Related Commands

- `get_property`
- `set_property`
- `reset_property`

filter_collection

```
filter_collection
[-help]
base_collection
{filter_expression}
[-nocase]
[-quiet]
[-regexp]
```

Returns a new collection of objects that were filtered from the specified collection of objects based on user-specified criteria (*filter_expression*). The original collection remains unmodified.

The following rules apply:

- The *filter_expression* can be created by combining several object properties using the following relational and logical operators: >, <, ==, !=, <=, >=, &&, ||, =~, !~, AND, OR. You also can use parentheses to create expressions.
- You can use the defined and undefined existence operators to determine whether an attribute is defined for an object.
- All design object properties supported through the get_property command can be used for filtering the base collection.

Note: You can perform filtering operations by specifying the -filter parameter with the get_* commands.

Parameters

-help	Prints out the command usage.
base_collection	Specifies the collection of objects from which to filter objects for the new collection.
filter_expression	Specifies the criteria with which to filter objects. The software adds any objects in the base collection that match *filter_expression* to the new collection. See "Examples" on how to create filter expressions.
-nocase	Specifies that pattern matching is not case sensitive. **Note:** You must specify -regexp in order to use this parameter.
-quiet	Suppresses all the error and warning messages that are generated when the filter_collection command is run.
-regexp	Treats the specified patterns as regular expression patterns. *Default:* Treats the specified pattern as a wildcard.

Examples

- The following command returns a single-object collection containing the pin with the hierarchical name lat_3/Q:
  ```
  filter_collection [get_pins] {hierarchical_name == lat_3/Q}
  ```
- The following command returns a collection of pins that have a maximum slew greater than 0.1:
  ```
  filter_collection [get_pins] {slew_max_rise > 0.1}
  ```
- The following commands create a new collection containing all those paths from the path collection with a slack value greater than -0.806 and less than 0.0:
  ```
  set paths [report_timing -collection -max_paths 10]
  set newPaths [filter_collection $paths {(slack > -0.806) && (slack < 0.0)}]
  ```
- The following command returns a collection of object names that do not match with “*clk*”:
  ```
  filter_collection $abc {full_name !~ *clk*}
  out in1 in2 in3 select TopBus[2] TopBus[1] TopBus[0] bidi
  0x53
  ```
- The following command returns a collection of object names that are not “*clk*” only:

```
filter_collection $abc {full_name != *clk*}
out clkl clk2 in1 in2 in3 select TopBus[2] TopBus[1] TopBus[0] bidi
0x59
```

- The following command returns a collection of object names that match with "*clk*":

```
filter_collection $abc {full_name =~ *clk*}
clkl clk2
0x5f
```

- The following command creates a collection of pins with `user_constant_value`:

```
set constant_pins [filter_collection [get_pins] {defined(user_constant_value}]
```

Related Commands

- append_to_collection
- copy_collection
- range_collection

foreach_in_collection

```
foreach_in_collection
[-help]
var_name
collection
body
```

Iterates through all objects in the specified collection and executes the commands in the specified script body.

Parameters

`-help`	Prints out the command usage.
`body`	Specifies the script of commands to execute during each iteration.
`collection`	Specifies the collection of objects through which to iterate.
`var_name`	Specifies the iterator variable. During each iteration, *`var_name`* is set to a collection of exactly one object. Any command that accepts collections can accept *`var_name`* as a value.

Examples

- The following command prints out the hierarchical names of all the cells in a design:

```
foreach_in_collection iCell [get_cells *] {
puts "Cell = [get_property $iCell hierarchical_name]"

    }
    Cell = ck_0
    Cell = ck_1
    Cell = ck_2
    Cell = ck_4
```

- The following example shows a more complex use of the `foreach_in_collection` command:

```
set cellCollection [get_cells temp1_reg]
set libCellCollection [get_lib_cells -of_objects $cellCollection]
set libArc [get_lib_arcs -of_objects $libCellCollection]

foreach_in_collection arc $libArc {
set fpin [get_property $arc from_lib_pin]

set tpin [get_property $arc to_lib_pin]
set sense [get_property $arc timing_type]

set from_pin_name [get_property $fpin name]
set to_pin_name [get_property $tpin name]

puts [format "%25s -> %25s %-20s" $from_pin_name $to_pin_name $sense]
}
```

```
   CK -> D           setup_rising
   CK -> D           hold_rising
   CK -> CK          minimum_period
   CK -> CK          minimum_period
   CK -> CK          min_pulse_width

   foreach_in_collection.txt
   CK -> CK          min_pulse_width
   RN -> RN          min_pulse_width
   CK -> RN          recovery_rising
   CK -> RN          removal_rising
   CK -> Q           falling_edge
   RN -> Q           clear
```

Related Commands

- append_to_collection
- copy_collection
- range_collection

get_activity

```
get_activity
[-help]
[-out_file filename]
[-net netname]
[-pin pinname]
[-port portname]
[-summary]
[-tcl_list]
[-list_of_nets_based_on_driver {primary_inputs | sequential | combinational | memory | ICG}]
[-list_of_nets_based_on_source_of_activity_info {activity_file|set_switching_activity|propagated|default}]
[-report_average_switching_activity]
[-report_cell_group_activity_summary]
[-inst inst_name]
```

Determines the source of activity for the specified nets/pins/ports. The power analysis engine has various sources from where activities can be annotated from, such as switching activity specification (input/sequential/global/clock gate), clock definitions/constants through SDC/TWF, activity files (VCD/TCF/SAF), user defined activity and/or propagated values.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_activity` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_activity`
`-inst inst_name`	
	Specifies the top/hierarchy instance or the leaf level instance name for which average activity is to be reported.
`-list_of_nets_based_on_driver {primary_inputs | sequential | combinational | memory | ICG}`	
	Specifies to output the list of nets based on the specified net driver. You can specify one or more net drivers at a time, as shown below: `get_activity -list_of_nets_based_on_driver primary_inputs|combinational|memory` To specify multiple drivers, you can use / (means 'or') or **&** (means 'and') between two drivers, without a space.
`-list_of_nets_based_on_source_of_activity_info {activity_file|set_switching_activity|propagated|default}`	
	Specifies to output the list of nets based on the specified activity source information.
`-net netname`	Specifies the name of the net. **Note:** You can use the `get_nets` command to determine the activity source for a collection of nets.
`-out_file filename`	Specifies the output file into which the activity sources are to be written. The format of the report is: <*net_name/pin_name*> <*activity_source*> <*duty_cycle*> <*transition_density*>
`-pin pinname`	Specifies the name of the pin. **Note:** You can use the `get_pins` command to determine the activity source for a collection of pins.
`-port portname`	Specifies the name of the port. **Note:** You can use the `get_ports` command to determine the activity source for a collection of ports.
`-report_average_switching_activity`	
	Specifies to report the average activity of the given instance/hierarchy/top-level.
`-report_cell_group_activity_summary`	
	Specifies to give a summary of cell group activity.

`-summary`	Generates a detailed activity annotation report of the specified nets, pins, and ports. This report contains the following information: • Primary Inputs • Sequential Outputs • Memory/Macro Outputs • Tristate Outputs **Note:** Use the `get_activity -summary` command to report activity annotation summary without printing information for nets/pins/ports.
`-tcl_list`	Produces the report in the Tcl list format instead of a tabular format. The `-tcl_list` and `-out_file` parameters are mutually exclusive; you cannot specify them together.

The activity report for pins is displayed in the following format:

```
<Pin Name> <Source of the Activity Value> <Duty Cycle> <Toggles/Sec>
```

The report fields are described below:

- `Pin Name`: Name of the specified pins.
- `Source of the Activity Value`: Source of activity for the specified nets, pins, or ports.
- `Duty Cycle`: Probability of signal at 1.
- `Toggles/Sec`: Number of toggles divided by the total simulation time.

Examples

- The following example returns the activity sources of all nets that match the specified pattern and writes the output information to a file named `nets.rpt`:

  ```
  get_activity -net [get_nets n*] -out_file nets.rpt
  ```

  The following is the example output of the nets.rpt file:

  ```
  n_14 user_defined_activity 0.5 6e+07
  n_11 user_defined_activity 0.5 6e+07
  n_25 user_defined_activity 0.5 6e+07
  n_13 user_defined_activity 0.5 6e+07
  n_26 user_defined_activity 0.5 6e+07
  n_9 user_defined_activity 0.5 6e+07
  n_27 user_defined_activity 0.5 6e+07
  n_23 user_defined_activity 0.5 6e+07
  n_20 user_defined_activity 0.5 6e+07
  n_16 user_defined_activity 0.5 6e+07
  ```

- The following example returns the activity source of the pin `i_4/A1` and writes the output information to a power report file named `pin.rpt`:

  ```
  get_activity -pin i_4/A1 -out_file pin.rpt
  ```

- The following command report the average activity of all nets:

  ```
  get_activity -report_average_switching_activity
  ```

  The following is the example output:

  ```
  Avg Switching activity of all nets 4.727e+17
  ```

- The following command gives a summary of cell group activity:

  ```
  get_activity -report_cell_group_activity_summary
  ```

  The following is the example output:

  ```
  Group Activity Summary Report:
  Source ->                   activity file          set_switching_activity   propagated             default                Total
  All nets                    2                      1                        2                      5
  10
  Break-up based on net driver:
  Primary Input               2(40%)                 0(0%)                    0(0%)                  3(60%)
  5
  Sequential                  0(0%)                  1(33.33%)                0(0%)                  2(66.67%)
  3
  Combinational               0(0%)                  0(0%)                    2(100%)                0(0%)
  2
  Memory                      0(0%)                  0(0%)                    0(0%)                  0(0%)
  ```

```
0
ICG                          0(0%)                  0(0%)                  0(0%)                  0(0%)
0
Black Box                    0(0%)                  0(0%)                  0(0%)                  0(0%)                  0
```

- The following command gives the list of nets based on primary inputs:
  `get_activity -list_of_nets_based_on_driver primary_inputs`
  The following is the example output:
  `D2 D1 CP2 CP1 dummy0`
- The following command gives the list of nets based on primary inputs or sequential driver:
  `get_activity -list_of_nets_based_on_driver primary_inputs|sequential`
  The following is the example output:
  `Q2 Q1 D2 D1 CP2 CP1 n1 dummy0`
- The following command gives the list of nets based on primary inputs or sequential driver for the instance `ff1b`:
  `get_activity -list_of_nets_based_on_driver primary_inputs|sequential -instance ff1b`
- The following command gives the list of nets based on the propagated source for the instance `i2`:
  `get_activity -list_of_nets_based_on_source_of_activity_info propagated -instance i2`
- The following command gives a summary of cell group activity for the instance `i2`:
  `get_activity -instance i2 -report_cell_group_activity_summary`

get_arcs

```
get_arcs
[-help]
{ [-to to_list] [-from from_list]
 | -of_objects object_list}
[-filter expr]
[-quiet]
```

Creates a collection of timing arcs. Assign these arcs to a variable, or pass them to another command. If no objects match the criteria, an empty collection is returned.

Note: The timing graph must be built before you can use this command. A warning message is issued indicating that the timing graph is not generated.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters a collection based on the specified expression. For any arcs that match the pattern, the software evaluates the expression based on the attribute of the arc. For details on how to create filter expressions, refer to the `filter_collection` command.
`-from from_list`	Adds all timing arcs that fan in directly to the specified pins or ports to the collection. You can specify a list of pins or a list of ports, but you cannot specify a list that contains both pins and ports. You can specify a pattern for the pin or port name, a collection of pins or ports, or a pattern and a collection. **Note:** You cannot specify this parameter with the `-of_objects` parameter.
`-of_objects object_list`	Adds all timing arcs that are associated with the specified cells or nets to the collection. You can specify a list of cells or a list of nets, but you cannot specify a list that contains both cells and nets. You can specify a pattern for the cell or net name, a collection of cells or nets, or a pattern and a collection. **Note:** You cannot specify this parameter with the `-to` or `-from` parameters.
`-quiet`	Suppresses all error and warning messages generated when the `get_arcs` command is run.
`-to to_list`	Adds all timing arcs that fan out directly from the specified list of pins or ports to the collection. You can specify a list of pins or a list of ports, but you cannot specify a list that contains both pins and ports. You can specify a pattern for the pin or port name, a collection of pins or ports, or a pattern and a collection. **Note:** You cannot specify this parameter with the `-of_objects` parameter.

Examples

- The following command returns a collection of the timing arcs fan in directly to the pin `dff1/CP`:
  ```
  get_arcs -from dff1/CP
  ```
- The following command creates a collection of the timing arcs that are associated with the nets in the specified collection:
  ```
  get_arcs -of_objects [get_nets {n1}]
  ```

get_cells

```
get_cells
[-help]
[-filter expr]
[-hierarchical]
[-hsc char]
[-leaf]
[-nocase]
[-quiet]
[-regexp]
[patterns | -of_objects object_list]
```

Creates a collection of instances in the current design whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument in another command.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any cells that match the pattern, the software evaluates the expression based on the attribute of the cell. You can filter cells using any of the cell properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-hierarchical`	Specifies that the patterns should be matched hierarchically. If the flat name of an instance at a particular hierarchical level matches the patterns, it is included in the result. For example. if there is top level hierarchical instance `Top` and it contains another sub-instance called `FF1`, then the following command can be used to get the instance `Top/FF1` (since the given pattern will be matched with instance flat name `FF1`): get_cells -hier FF* However, the following command will display an error: get_cells -hier Top/FF*
`-hsc char`	Specifies the hierarchical delimiter for patterns. For example, if the hierarchical delimiter is `@`, the `sub/I1@I2` pattern matches instance `I2` within the hierarchy limited at `sub/I1`.
`-leaf`	Returns leaf cells only. When the `-leaf` parameter is not specified, the `get_cells` command returns the hierarchical instances present on a net along with the flat instances at the same hierarchical level as the specified object. **Note:** The `-leaf` parameter can only be specified with the `-of_objects` parameter.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.

`-of_objects object_list`	Creates a collection of all cells associated with the specified pins, nets, library cells, or design cells.
	You can specify a list of pins or a list of nets, but you cannot specify a list that contains both pins and nets. You can specify patterns for the pin or net names, a collection of pins or nets, or a combination of patterns and a collection.
	The `-of_objects` parameter supports `get_designs` and `get_lib_cells` as valid collection objects.
	The use model is as follows:
	`get_cells -of_objects [get_designs object_list]`
	`get_cells -of_objects [get_lib_cells object_list]`

`patterns`	Specifies patterns for instance names. Patterns include the `*` and `?` wildcard characters and collections of instances. The `get_cells` command returns a collection of instances that match the patterns. If no such instance is found, an empty collection is returned.
`-quiet`	Suppresses all error and warning messages generated when the `get_cells` command is run.

`-regexp`	Treats the specified patterns as a regular expression patterns.
	When this parameter is specified, the given patterns `start` and `end` are assumed to be matching with the start and end part of the complete object name.
	For example, the following can be used to retrieve the hierarchical instance `Top/block1/block2/FF1`:
	`get_cells Top/*/*/FF1`
	`get_cells -regexp Top/.*/FF1`
	However, the following command will display an error :
	`get_cells Top/*/FF1`
	If this parameter is specified with `-hier` parameter, the given pattern will be matched with the complete hierarchical name of objects instead of only the flat name and all the successful matches are included in the result.
	For example, if there is a top level hierarchical instance `Top` and it contains another sub-instance called `FF1`, then either of these can be used to retrieve `Top/FF1` instance:
	`get_cells -hier FF1`
	`get_cells -hier -regexp Top/FF1`
	However, the following commands will display an error :
	`get_cells -hier Top/FF1`
	`get_cells -hier -regexp FF1`

Examples

- The following command searches for pins within the hierarchy limited at `B`:
  `get_cells -hsc @ B@*`
- The following command searches for cells with set pattern and filter:
  `get_cells "o*" -filter "@ref_lib_cell_name != FD2 && @is_black_box==false"`

Related Commands

- `get_libs`
- `get_lib_cells`
- `get_lib_pins`
- `get_pins`

get_clocks

```
get_clocks
[-help]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
[patterns]
```

Creates a collection of clocks whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to another command.

The `get_clocks` command reports all clocks, including inactive clocks.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any clocks that match the pattern, the software evaluates the expression based on the attribute of the clock. You can filter clocks using any of the clock properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
patterns	Specifies a pattern (or a list of patterns) for clock names. A pattern can contain `*` and `?` wildcard characters, or a collection of instances and clocks. **Note**: If a pattern is not specified, then `get_clocks` returns all the clocks that meet other filters.
`-quiet`	Suppresses all error and warning messages generated when the `get_clocks` command is run.
`-regexp`	Treats the specified patterns as a regular expression patterns. *Default:* Treats the specified pattern as a wild card

Examples

- The following example returns all clocks beginning with the name `CK`:
  ```
  get_clocks CK*
  ```
- The following example returns a collection of all clocks beginning with the name CK and Clk:
  ```
  get_clocks [list CK* Clk* ]
  ```
- The following example returns a collection of all active clocks beginning with the name CK:
  ```
  get_clocks CK* -filter {is_active == true}
  ```
- The following command returns all the clocks that are propagated:
  ```
  get_clocks -filter "is_propagated_clock == true"
  ```

Related Commands

- `all_clocks`
- `create_clock`
- `get_pins`
- `get_ports`
- `report_clocks`

get_designs

```
get_designs
[-help]
[-quiet]
patterns
```

Creates a collection of modules and assigns this collection to a variable or pass it as an argument to an another command. The software uses this information during timing analysis.

Parameters

`-help`	Prints out the command usage.
patterns	Specifies patterns for module names. Patterns include the `*` and `?` wildcard characters and collections of modules. The `get_designs` command returns a collection of modules that match the patterns. If no such module is found, an empty collection is returned.
`-quiet`	Suppresses all error and warning messages generated when the `get_design` command is run.

Examples

- The following command returns a complete list of modules:
  ```
  get_designs *
  ```

get_generated_clocks

```
get_generated_clocks
[-help]
[-filter expr]
[-regexp | -exact]
[-nocase]
[patterns]
```

Creates a collection of generated clocks whose name matches the supplied pattern list. Assign this collection to a variable, or pass it as an argument to another command.

Parameters

`-help`	Prints out the command usage.
`-exact`	Performs exact pattern matching against the object name, instead of wild card or regular expression matching. Use this parameter when the object name includes special characters such as `*`, `?`, and `+`. **Note:** If you specify this parameter, you cannot specify `-regexp`.
`-filter expr`	Filters the collection with the specified expression. For any clocks that match the pattern, the software evaluates the expression based on the attribute of the clock.
`-nocase`	Specifies that pattern matching is not case sensitive.
patterns	Specifies patterns for generated clock names. Patterns include the `*` and `?` wildcard characters, and collections of generated clocks.
`-regexp`	Treats the specified pattern as a regular expression. *Default:* Treats the specified pattern as a wild card **Note:** If you specify this parameter, you cannot specify `-exact`.

Examples

- The following command returns the collection of all the generated clocks present in the design whose name starts with "CK":

```
tempus> get_generated_clocks CK*

CKGEN1 CKGEN2 CKGEN3
```

Related Commands

- create_generated_clock

get_lib_arcs

```
get_lib_arcs
{ [-to to_lib_pins] [-from from_lib_pins]
  | -of_objects {lib_cell_list | timing_arcs} }
[-filter expr]
[-quiet]
```

Creates a collection of library timing arcs. Assign these library timing arcs to a variable, or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

`-filter expr`	Filters the collection with the specified expression. For any library arcs that match the pattern, the software evaluates the expression based on the attribute of the library arc. For details on how to create filter expressions, refer to the `filter_collection` command.
`-from from_lib_pins`	Creates a collection of all library arcs that originate from the specified library pins. You can specify a pattern for the pin name, a collection of pins, or a pattern and a collection. If you specify this parameter with the `-to` parameter, the software creates a collection of all library arcs that exist between the "to" and "from" library pins. **Note:** You cannot specify this parameter with the `-of_objects` parameter.
`-of_objects {lib_cell_list | timing_arcs}`	Creates a collection of all library arcs that are associated with the specified library cells or timing arcs. You can specify a list of library cells or a list of timing arcs, but you cannot specify a list that contains both library cells and timing arcs. You can specify a pattern for the library cell or timing arc name, a collection of library cells or timing arcs, or a pattern and a collection. **Note:** You cannot specify this parameter with the `-to` or `-from` parameters.
`-quiet`	Suppresses all error and warning messages generated when the `get_lib_arcs` command is run.
`-to to_lib_pins`	Creates a collection of all library arcs that terminate at the specified library pins. You can specify a pattern for the pin name, a collection of pins, or a pattern and a collection. If you specify this parameter with the `-from` parameter, the software creates a collection of all library arcs that exist between the "to" and "from" library pins. **Note:** You cannot specify this parameter with the `-of_objects` parameter.

Examples

- The following command reports the timing type off all the timing arcs of the flop 'DFF':

  ```
  tempus> get_property [get_lib_arcs -of_objects DFF] timing_type
  ```

  Sample output:

  ```
  minimum_period minimum_period min_pulse_width min_pulse_width setup_rising hold_rising rising_edge rising_edge
  ```

Related Commands

- `report_lib_arcs`
- `get_lib_cells`
- `get_lib_pg_pins`
- `get_lib_pins`
- `get_libs`

get_lib_cells

```
get_lib_cells
[-help]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
{pattern_list | -of_objects object_list}
```

Creates a collection of library cells from the loaded libraries whose name matches the supplied pattern list. Assign these library cells to a variable or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any library cells that match the pattern, the software evaluates the expression based on the attribute of the library cell. You can filter library cells using any of the library cell properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects object_list`	Creates a collection of all library cells associated with the specified library pins or cells or libs. You can specify a list of library pins or a list of cells or a list of libs, but you cannot specify a list that contains mixed of library pins and cells and libs. You can specify patterns for the library pin or cell or lib names, a collection of library pins or cells or libs, or a combination of patterns and a collection.
`pattern_list`	Returns only those libraries in the collection whose name matches one or more of the patterns specified in the `pattern_list` argument. The characters `?` and `*` are wild cards. The `?` character matches any one character while the `*` character matches zero or more characters. If a library matches multiple patterns, it appears multiple times in the collection. A pattern can have only one hierarchical separator character. The left part of the pattern is the library name pattern and the right part is the cell name pattern. If there is no separator character, the pattern is compared to cell names in all the libraries. A pattern can have only one hierarchical separator character, dividing the pattern into one or two parts. If the hierarchical separator is `/`, the alternatives are: • `cell_name_pattern` Selects all cells whose name matches the `cell_name_pattern` from all libraries. • `lib_name_pattern/cell_name_pattern` Selects all cells whose name matches the `cell_name_pattern` from all libraries whose name matches the `lib_name_pattern`.
`-quiet`	Suppresses all error and warning messages generated when the `get_lib_cells` command is run.
`-regexp`	Treats the specified patterns as regular expression patterns. *Default:* Treats the specified pattern as wild card

Examples

- The following command prints all cells for all libraries:
  ```
  get_lib_cells *
  ```
- The following command prints a collection with all cells in the `lca500kv` lib library:
  ```
  get_lib_cells lca500kv/*
  ```
- The following command prints a collection with all cells whose names starts with `AN` in all libraries starting with `lca`:
  ```
  get_lib_cells lca*/AN*
  ```
- The following command disables delay arcs on the `EF11L050032A` library cell. The `set_disable_cell_timing` command effects all instantiations of the

specified cell, and lets you disable delay arcs from and to internal pins.

```
set_disable_timing -from WACLK -to RBO1 [get_lib_cells {EF11L050032A}]
```

Related Commands

- get_lib_arcs
- get_lib_pg_pins
- get_lib_pins
- get_libs
- get_property

get_lib_pg_pins

```
get_lib_pg_pins
[-help]
[-filter expr]
{-of_objects object_list}
```

Returns a collection of library-level PG pins from the loaded libraries. An empty collection is returned if no PG pin definitions are found in the loaded library.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any library pg pins that match the specified expression, the software evaluates the expression based on the attribute of the library pg pin. You can filter library pg pins along with the `-of_objects` parameter - using the library pg pin properties of `lib_pg_pin` object type (supported through the `get_property` command). For details on how to create filter expressions, refer to the `filter_collection` command.

`-of_objects object_list`	Creates a collection of all library PG pins associated with the specified library cells.
	The `-of_objects` parameter supports `get_lib_cells` as valid objects.
	For example,
	`get_lib_pg_pins -of_objects [get_lib_cells sc9_cmos32lp/DFFQ_X1M] VDD VSS`
	`get_lib_pg_pins *`
	`**ERROR: (TCLCMD-1420): Currently only -of_objects option is supported`

Examples

- The following command gets the collection of all the library-level pg pins of the specified library cell:
  ```
  tempus > get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]
  ```
- The following command filters the collection of all library-level pg pins of the library cell to return only the library pg pins containing the string name defined with the lib_pg_pin name attribute:
  ```
  tempus > get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X] -filter "name =~ *VDD*"
  ```

Related Commands

- `get_lib_cells`
- `get_lib_pins`
- `get_libs`
- `get_property`

get_lib_pins

```
get_lib_pins
[-help]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
{pattern_list | -of_objects object_list}
```

Creates a collection of library cell pins from the loaded libraries whose name matches the supplied pattern list. Assign these library cell pins to a variable or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any library cell pins that match the pattern, the software evaluates the expression based on the attribute of the library cell pin. You can filter library pins using any of the library pin properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects object_list`	Creates a collection of all library pins associated with the specified library cells or pins. You can specify a list of cells or a list of pins, but you cannot specify a list that contains both cells and pins. You can specify patterns for the cell or pin names, a collection of cells or pins, or a combination of patterns and a collection.
pattern_list	Returns only those libraries in the collection whose name matches one or more of the patterns specified in the *pattern_list* argument. The characters `?` and `*` are wild cards; The `?` character matches any one character while the `*` character matches zero or more characters. If a library cell matches multiple patterns, it appears multiple times in the collection. A pattern can have only two hierarchical separator characters, dividing the pattern into one, two, or three parts. If the hierarchical separator is `/`, the alternatives are: • *pin_name_pattern* Selects all pins whose name matches the *pin_name_pattern* from all cells in the libraries. • *cell_name_pattern/pin_name_pattern* Selects all pins whose name matches the *pin_name_pattern* from all cells whose name matches the *cell_name_pattern* in all libraries. • *lib_name_pattern/cell_name_pattern/pin_name_pattern* Select all pins whose name matches the *pin_name_pattern* from all cells whose name matches the *cell_name_pattern* from all libraries whose name matches the *lib_name_pattern*.
`-quiet`	Suppresses all error and warning messages generated when the `get_lib_pins` command is run.
`-regexp`	Treats the specified patterns as a regular expression patterns. *Default:* Treats the specified pattern as a wild card

Examples

- The following command returns all pins in all cells for all libraries:
  ```
  get_lib_pins *
  ```
- The following command returns a collection with all pins for cell `AO1` in any library:
  ```
  get_lib_pins AO1/*
  ```
- The following command returns a collection with all pins whose name starts with `Z` or `Q` for the `AO1` cell in any library:
  ```
  get_lib_pins {AO1/Z* AO1/Q*}
  ```
- The following command returns a collection with all pins for the `AO1` cell in the `lca500kv` lib library:

```
get_lib_pins lca500kv/AO1/*
```

- The following command returns a collection with all pins starting with `Z` in all cells whose names starts with `AN` in all libraries starting with `lca`:

```
get_lib_pins lca*/AN*/Z*
```

Related Commands

- get_lib_cells
- get_lib_pg_pins
- get_libs
- get_property

get_libs

```
get_libs
[-help]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
{-of_objects object_list | pattern_list}
```

Creates a collection of loaded libraries whose name matches those specified in the pattern list. Assign these libraries to a variable or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any libraries that match the pattern, the software evaluates the expression based on the attribute of the library. You can filter libraries using any of the library properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects object_list`	Creates a collection of libraries that belong to the library cells in a given object list.
pattern_list	Returns only those libraries in the collection whose name matches one or more of the patterns specified in the *pattern_list* argument. The characters `?` and `*` are wild cards; The `?` character matches any one character while the `*` character matches zero or more characters. If a library matches multiple patterns, it appears multiple times in the collection.
`-regexp`	Treats the specified patterns as a regular expression patterns. *Default:* Treats the specified pattern as a wild card
`-quiet`	Suppresses all error and warning messages generated when the `get_lib_cells` command is run.

Examples

- The following command displays a list of loaded libraries starting with a or l:
  ```
  get_libs {a* l*}
  ```
- The following command queries all libraries beginning with the letter `l`:
  ```
  get_libs l*
  ```
- The following command creates a collection of libraries that are associated with the library cells in the specified object list:
  ```
  get_libs -of_objects [get_lib_cells *]
  ```

Related Commands

- `get_lib_cells`
- `get_lib_pg_pins`
- `get_lib_pins`
- `get_property`

get_nets

```
get_nets
[-help]
[-hierarchical]
[-hsc char]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
[patterns | -of_objects object_list]
```

Creates a collection of nets in the current design whose name matches the supplied pattern list. You can assign this collection to a variable or pass it as an argument to an another command. If no such net is found, an empty collection is returned.

When you use the `get_nets` command to specify hierarchical nets, the software matches the specified pattern and returns the top level net connected to it.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any nets that match the pattern, the software evaluates the expression based on the attribute of the net. You can filter nets using any of the net properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-hierarchical`	Specifies that the patterns should match hierarchically. If the hierarchical name of a net at any hierarchical level matches the patterns, it is included in the result.
`-hsc char`	Specifies the hierarchical delimiter for patterns.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects object_list`	Creates a collection of all nets associated with the specified cells, pins, or ports. You can specify a list of cells, a list of pins, or a list of ports, but you cannot specify a list that contains cells, pins, and ports together. You can specify patterns for the cell, pin, or port names, a collection of cells, pins, or ports, or a combination of patterns and a collection.
`patterns`	Specifies patterns for net names. Patterns include the `*` and `?` wildcard characters and collections of nets.
`-quiet`	Suppresses all error and warning messages generated when the `get_nets` command is run.
`-regexp`	Treats the specified patterns as regular expression patterns. *Default:* Treats the specified pattern as wildcard

Examples

- The following command returns all nets within the hierarchy limited at `A`:

  ```
  get_nets -hsc @ A@*
  ```

- The following command returns all patterns that match hierarchy and identifies nets to be excluded from optimization:

  ```
  get_nets * -hierarchical -filter "@dont_touch== true"
  ```

Related Commands

- `get_pins`

get_object_name

```
get_object_name
[-help]
collection
```

Returns the name of the object(s) contained in the specified collections.

By default, the command hides escape characters in the output. To enable printing of escape characters in the output, you can set `timing_enable_get_obj_escaped_name_backward_compatible` attribute to `true`.

Parameters

`-help`	Prints out the command usage.
`collection`	Specifies multiple collections that contain objects.

Examples

- The following command returns a tcl list of names, which can match the collection inputs:

  ```
  get_object_name [get_cells and*]
  ```

- The following command defines a variable called `endpoint_list` to store endpoints with the ten worst slacks:

  ```
  set endpoint_list [get_object_name [get_property \
    [report_timing -collection -max_points 10] capturing_point]]
  ```

Related Commands

- `query_objects`

get_path_groups

```
get_path_groups
[-help]
[-regexp]
[-nocase]
patterns
```

Creates a collection of path groups whose name matches those specified in the pattern list. Assign these path groups to a variable, or pass them to another command. If no objects match the criteria, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
`-nocase`	Specifies that pattern matching is not case sensitive.
`patterns`	Creates a collection of path groups whose name matches one or more of the specified patterns. Patterns include the `*` and `?` wildcard characters and collections of path groups. If a path group matches multiple patterns, it appears multiple times in the collection.
`-regexp`	Treats the specified patterns as regular expression patterns. *Default:* Treats the specified pattern as wildcard

Examples

- The following command returns a collection of all the path groups present in the design:

  ```
  tempus> get_path_groups *
  ```

  Sample output:

  ```
  reg2reg reg2gate
  ```

Related Commands

- `reset_path_group`
- `report_path_groups`

get_pg_nets

```
get_pg_nets
[-help]
[-filter expr]
[-of_objects object_list]
```

Creates a collection of netlist-level instance PG nets of leaf cells/pins in the current design. An empty collection is returned if no PG net definitions/connections are found in the design netlist.

Parameters

`-help`	Prints out the command usage.
`-filter` *expr*	Specifies the filters for the collection with the specified expression. For any pg nets that match the pattern, the software evaluates the expression based on the attribute of the net. You can filter pg nets along with the `-of_objects` parameter - using any of the pg net properties of object type `pg_net` (supported through the `get_property` command). For details on how to create filter expressions, refer to the `filter_collection` command.
`-of_objects` *object_list*	Creates a collection of all the PG nets, or PG nets associated with the specified cells or PG pins. You can specify a list of cells or a list of pg pins, but you cannot specify a list that contains cells and pg pins together. The `-of_objects` parameter supports `get_cells` and `get_pg_pins` as valid objects.

Examples

- The following command filters the collection of all pg nets of the cell instance object to return only the pg nets containing the string name defined with the pg_net hierarchical_name attribute:
  ```
  tempus > get_pg_nets -of_objects [get_cells iA/out_reg] -filter "hierarchical_name =~ *VDD*"
  ```
- The following command gets the collection of all pg nets of the cell instance object:
  ```
  tempus > get_pg_nets -of_objects [get_cells iA/out_reg]
  ```
- The following command gets the collection of all pg nets of the pg_pins of the cell instance object:
  ```
  tempus > get_pg_nets -of_objects [get_pg_pins -of_objects [get_cells iA/out_reg]]
  ```

Related Commands

- `get_property`
- `get_nets`

get_pg_pins

```
get_pg_pins
[-help]
[-filter expr]
[-of_objects object_list]
```

Returns a collection of netlist-level instance PG pins in the current design. An empty collection is returned if no PG pin definitions are found in the design netlist.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Specifies the filters for the collection with the given expression. For any pg pins that match the pattern, the software evaluates the expression based on the attribute of the pin. You can filter pg pins along with the `-of_objects` parameter - using the pg pin properties of object type `pg_pin` (supported through the `get_property` command). For details on how to create filter expressions, refer to the `filter_collection` command.

`-of_objects object_list`	Specifies the collection of all the pins associated with the specified cells or nets.
	The `-of_objects` parameter supports `get_cells` as valid objects.
	For example:
	`get_pg_pins -of_objects [get_cells iA/out_reg] iA/out_reg/VSS iA/out_reg/VDD`
	`get_pg_pins iA/out_reg/VDD`
	`**ERROR: (TCLCMD-1420): Currently only -of_objects option is supported.`

Examples

- The following command filters the collection of all the pg_pins of the cell instance object to return only the pg pins containing the string name defined with the pg_pin hierarchical_name attribute:
  ```
  tempus > get_pg_pins -of_objects [get_cells iA/out_reg] -filter "hierarchical_name =~ *VDD*"
  ```
- The following command gets the collection of all pg pins of the cell instance object:
  ```
  tempus > get_pg_pins -of_objects [get_cells iA/out_reg]
  ```

Related Commands

- `get_property`
- `get_pg_nets`

get_pins

```
get_pins
[-help]
[-hierarchical]
[-hsc char]
[-filter expr]
[-leaf]
[-regexp]
[-nocase]
[-quiet]
{patterns | -of_objects object_list}
```

Creates a collection of instance pins whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to another command. If no such pin is found, an empty collection is returned.

Parameters

`-help`	Prints out the command usage.
`-filter expr`	Filters the collection with the specified expression. For any pins that match the pattern, the software evaluates the expression based on the attribute of the pin. If the expression is set to true, the pin is included in the result. You can filter pins using any of the pin properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-hierarchical`	Specifies that patterns should match hierarchically. If the hierarchical name of a pin at any hierarchical level matches the patterns, it is included in the result.
`-hsc char`	Specifies the hierarchical delimiter for patterns.
`-leaf`	Returns leaf pins only. When the `-leaf` parameter is not used, the `get_pins` command returns the hierarchical pins present on a net along with the flat pins at the same hierarchical level as the specified object.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify the `-regexp` parameter in order to use this parameter.
`-of_objects object_list`	Creates a collection of all pins associated with the specified cells or nets. You can specify a list of cells or a list of nets, but you cannot specify a list that contains both cells and nets. You can specify patterns for the cell or net names, a collection of cells or nets, or a combination of patterns and a collection.
patterns	Specifies patterns for pin names. Patterns include the `*` and `?` wildcard characters and collections of pins.
`-quiet`	Suppresses all error and warning messages generated when the `get_pins` command is run.
`-regexp`	Treats the specified patterns as regular expression patterns. *Default:* Treats the specified pattern as wildcard.

Examples

- The following command returns all instance pins within the hierarchy limited at `A`:

  ```
  get_pins -hsc @ A@*
  ```

- The following command filters the collection for `pin_direction` pin attribute:

  ```
  get_pins "in*" -filter "@pin_direction == in"
  ```

Related Commands

- create_clock
- get_cells
- get_ports

get_ports

```
get_ports
[-help]
[-filter expr]
[-regexp]
[-nocase]
[-quiet]
[patterns | -of_objects object_list]
```

Creates a collection of ports whose name matches the supplied pattern list. Assign this collection to a variable or pass it as an argument to another command.

Note: The `get_ports` command always creates a collection of top level ports irrespective of the current instance set using the `current_instance` command.

Parameters

`-help`	Prints out the command usage.
`-filter` *expr*	Filters the collection with the specified expression. For any ports that match the pattern, the software evaluates the expression based on the attribute of the port. If the expression is set to true, the port is included in the result. You can filter ports using any of the port properties supported through the `get_property` command. For details on how to create filter expressions, refer to the `filter_collection` command.
`-nocase`	Specifies that pattern matching is not case sensitive. **Note:** You must specify `-regexp` in order to use this parameter.
`-of_objects` *object_list*	Creates a collection of all ports associated with the specified nets. You can specify patterns for the net names, a collection of nets, or a combination of patterns and a collection.
patterns	Specifies patterns for port names. Patterns include the `*` and `?` wildcard characters and collections of ports.The command returns a collection of ports that match the patterns. If no such port is found, an empty collection is returned.
`-quiet`	Suppresses all error and warning messages generated when the `get_ports` command is run.
`-regexp`	Treats the specified patterns as regular expression patterns. *Default:* Treats the specified pattern as wildcard.

Example

- The following example returns all ports beginning with the name `inb`:

  ```
  get_ports inb*
  ```

- The following command filters the collection for `port_direction` port attribute:

  ```
  get_ports "mode*" -filter {port_direction == in}
  ```

Related Commands

- `all_inputs`
- `all_outputs`
- `get_clocks`
- `get_pins`

get_power

```
get_power
[-help]
[-out_file filename]
[-nets nets_list | -pins pins_list | -insts instances_list]
[-attribute attributes_list]
[-include_unit]
[-tcl_list ]
[-pg_net {pg_net_name_list | all}]
```

Queries various power related properties of design objects like nets, pins, and instances. Using this command, you can retrieve power attributes, such as internal power, switching power, leakage power, total power, and toggle rate. This command is very useful for writing custom scripts.

Note: This command outputs toggle rates as N.A. if the clock domain information for a given net/pin/instance is not found.

Use this command after running the `report_power` or `restore_power_database` command.

Note: The `-nets`, `-pins`, and `-insts` parameters are mutually exclusive. You must specify only one of the parameters.

Parameters

`-attribute` *attributes_list*	Specifies a list of power attributes. If an attribute is not available for the specified design object, the software returns its value as NA. See List of Power Attributes for a mapping of the list of available attributes with the design objects. This is a required parameter.
`-help`	Outputs a brief description that includes the type and default information for each `get_power` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_power`
`-include_unit`	Specifies to include the units for the power attributes.
`-insts` *instances_list*	Specifies a list of instance names. The `-nets`, `-pins`, and `-insts` parameters are mutually exclusive. You must specify one, and only one, of the parameters. **Note:** You can use the `get_cells` command to determine the power attributes for a collection of instances.
`-nets` *nets_list*	Specifies a list of net names. The `-nets`, `-pins`, and `-insts` parameters are mutually exclusive. You must specify one, and only one, of the parameters. **Note:** You can use the `get_nets` command to determine the power attributes for a collection of nets.
`-out_file` *filename*	Specifies the output file into which the power related properties are to be written. The `-tcl_list` and `-out_file` parameters are mutually exclusive; you cannot specify them together.
`-pg_net {`*pg_net_name_list* `| all}`	
	Specifies to query various power-related properties of a specified power net for an instance. This parameter supports the argument all to query properties of all power nets connected to the instance. You must use the `-insts` parameter in conjunction with the `-pg_net` parameter. The `-pg_net` parameter supports the following attributes: `switching_power`, `internal_power`, `leakage_power`, `total_power`, and `dynamic_power`.
`-pins` *pins_list*	Specifies a list of pin names. The `-nets`, `-pins`, and `-insts` parameters are mutually exclusive. You must specify one, and only one, of the parameters. **Note:** You can use the `get_pins` command to determine the power attributes for a collection of pins.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. The `-tcl_list` and `-out_file` parameters are mutually exclusive; you cannot specify them together.

The following power attributes can be queried for the design objects:

List of Power Attributes with Attribute Description

- `switching_power (net, inst)`: switching power of a net or instance
- `internal_power (inst)`: internal power of an instance
- `dynamic_power (inst)`: switching and internal power of an instance
- `leakage_power (inst)`: leakage power of an instance
- `total_power (inst)`: total power of an instance
- `toggle_rate (net, pin, inst)`: Toggle rate - Number of toggles in the specified VCD/TCF in the duration
- `duty_cycle (net, pin)`: Static probability of the signal to stay high
- `ref_clock (net, pin, inst)`: Reference clock name (Fastest clock for multi clock domains)
- `ref_clock_period (net, pin, inst)`: Reference clock period
- `transition_density (net, pin)`: toggle_rate/ref_clock_period or toggle_rate*ref_clock_frequency
- `loading_cap (net, pin, inst)`: Loading capacitance
- `voltage (net, pin, inst)`: Operating voltage for a net or pin. For MSMV instance, this attribute can be used to get breakdown of domain specific power.
- `rise_slew (net, pin)`: Rise slew
- `fall_slew (net, pin)`: Fall slew
- `glitch_rate (net, pin)`: Glitch rate - Number of glitches per duration. Applies only to the vector-driven flow.
- `glitch_power (net, pin, inst)`: Glitch power. Applies only to the vector-driven flow.
- `activity_source (net, pin)`: Source of activity annotation. The options are: propagated, user_annotated, and constant.
- `pin_direction (pin)`: [input | output | inout]
- `cell_type (inst)`:Lists cell type as memory, standard cell, IO, sequential, and so on
- `all_attributes (net, pin, inst)`: Lists all related power attributes for the specified design object.

Examples

- The following command returns the specified power attributes for the net clock1:

  ```
  get_power -nets clock1 -attribute {switching_power internal_power toggle_rate}
  ```

  The following is the sample output:

  ```
  clock1 0.04 NA 2
  ```

- The following command returns the specified power attributes for all nets that match the specified pattern `master*`:

  ```
  get_power -nets [get_nets master*] -attribute {switching_power toggle_rate activity_source duty_cycle} -include_unit
  ```

  The following is the sample output:

  ```
  masterin[1] 0.001mW 0.2 user_annotated 0.5
  masterin[2] 0.002mW 0.2 user_annotated 0.5
  masterin[3] 0.003mW 0.1 propagated 0.5
  masterin[4] 0.005mW 1 constant 1
  ```

- The following command returns power-related properties of all power nets connected to the instance `clk_buf_1`:

  ```
  get_power -pg_net all -insts clk_buf_1 -attribute {switching_power internal_power leakage_power}
  ```

  The following is the sample output:

  ```
  clk_buf_1 VDD 0.002 0.001 0.000001
  clk_buf_1 VDDL 0.0001 0.0001 0.0000001
  ```

get_property

```
get_property
[-help]
var_name
property
[-clock clock_name]
[-view view_name]
[-quiet]
```

Retrieves the attribute value for the specified object(s) property.

You can query the following:

- Cell Properties
- Clock Properties
- Design Properties
- Library Properties
- Library Cell Properties
- Library Pin Properties
- Library PG Pin Properties
- Library Timing Arc Properties
- Net Properties
- PG Net Properties
- Path Group Properties
- Pin Properties
- PG Pin Properties
- Port Properties
- Timing Arc Properties
- Timing Path Properties
- Timing Point Properties
- si_victim Properties
- si_attacker Properties
- User-Defined Properties

The `get_property` command supports DRV constraints (`set_min_fanout`, `set_max_fanout`, `set_min_transition`, `set_max_transition`, `set_min_capacitance`, and `set_max_capacitance`) applied on library pins.

The `get_property` command does not work with partial property names. You are now required to specify the full name of the property.

Parameters

`-help`	Prints out the command usage.
`-clock` *clock_name*	Specifies the clock for which to query certain object properties.
property	Specifies the property for which to retrieve the attribute value.
`-quiet`	Suppresses all error and warning messages generated when the `get_property` command is run.
var_name	Specifies a pointer to a collection containing the object. **Note:** If the collection contains more than one object, the `get_property` command returns the property values for each object in the specified collection.
`-view` *view_name*	Specifies the analysis view in which to query the object attribute. You can query properties only for the following objects in an analysis view: library cells, library pins, ports, cells, pins, timing arcs, and nets. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode. **Note**: The properties (described in the tables below), which are related to slack, delay, arrival, slew, transition, capacitance, uncertainty and latency, depend on the specified view. When the `-view` parameter is not specified, the maximum (or minimum) value for all the views is returned.

Cell Properties

`area`	Returns the area of the cell. If the cell is hierarchical, also includes the net area. *Return type:* float
`clock_gating_integrated_cell`	Returns the clock gating integrated cells present in a library. When specified, the software returns a value that is used to determine the integrated cell functionality to be used by clock gating tools. For example, `get_property [get_cells <cell name>] clock_gating_integrated_cell` *Return Type*: float
`early_cell_check_derate_factor`	Returns the derating factor for early paths specified through the `set_timing_derate` command with the `-early` parameter. *Return type:* float
`early_cell_check_mean_derate_factor`	Returns the derating factor for early paths on the instance – when the `set_timing_derate -mean` parameter is specified. *Return type:* float
`early_clock_check_incremental_derate_factor`	Returns the early clock path instances' incremental derating factor specified through the `set_timing_derate -increment` parameter. *Return type:* float
`early_data_check_incremental_derate_factor`	Returns the early data path instance's incremental derating factor specified through the `set_timing_derate -increment` parameter. *Return type:* float
`early_cell_check_sigma_derate_factor`	Returns the derating factor for early paths on the instance - when the `set_timing_derate -sigma` parameter is specified. *Return type:* float
`early_clk_cell_derate_factor`	Returns the derating factor for early clock paths specified through the `set_timing_derate` command with the `-early` parameter. *Return type:* float

early_clk_cell_mean_derate_factor	Returns the derating factor for early clock paths – when the set_timing_derate -mean parameter is specified. *Return type:* float
early_clk_cell_sigma_derate_factor	Returns the derating factor for early clock paths – when the set_timing_derate -sigma parameter is specified. *Return type:* float
early_data_cell_derate_factor	Returns the derating factor for early paths specified through the set_timing_derate command with the -early parameter. *Return type:* float
early_data_cell_mean_derate_factor	Returns the derating factor for early paths – when the set_timing_derate -mean parameter is specified. *Return type:* float
early_data_cell_sigma_derate_factor	Returns the derating factor for early paths – when the set_timing_derate -sigma parameter is specified. *Return type:* float
early_fall_cell_check_derate_factor	Returns the early cell check derating factor specified through the set_timing_derate command with the -fall parameter. *Return type:* float
early_fall_cell_check_mean_derate_factor	Returns the early cell check derating factor – when the set_timing_derate -cell_check and -mean parameters are specified. *Return type:* float
early_fall_cell_check_sigma_derate_factor	Returns the early cell check derating factor – when the set_timing_derate -cell_check and -sigma parameters are specified. *Return type:* float
early_fall_clk_cell_derate_factor	Returns the early clock path derating factor specified through the set_timing_derate command with the -fall parameter. *Return type:* float
early_fall_clk_cell_mean_derate_factor	Returns the early clock path derating factor – when the set_timing_derate -fall and -mean parameters are specified. *Return type:* float
early_fall_clk_cell_sigma_derate_factor	Returns the early clock path derating factor – when the set_timing_derate -fall and -sigma parameters are specified. *Return type:* float
early_fall_clock_incremental_derate_factor	Returns the early clock path instances' incremental derating factor specified through the set_timing_derate -increment parameter - when the -fall parameter is specified. *Return type*: float
early_fall_data_cell_derate_factor	Returns the early data cell check derating factor – when the set_timing_derate -fall parameter is specified. *Return type:* float
early_fall_data_cell_mean_derate_factor	Returns the early data cell check derating factor – when the set_timing_derate -fall and -mean parameters are specified. *Return type:* float
early_fall_data_cell_sigma_derate_factor	Returns the early data cell check derating factor – when the set_timing_derate -fall and -sigma parameters are specified. *Return type:* float

`early_fall_data_incremental_derate_factor`	Returns the early data path instances' incremental derating factor specified through the `set_timing_derate -increment` parameter - when the `-fall` parameter is specified. *Return type*: float
`early_rise_cell_check_derate_factor`	Returns the early cell check derating factor specified through the `set_timing_derate` command with the `-rise` and `-cell_check` parameter. *Return type:* float
`early_rise_cell_check_mean_derate_factor`	Returns the early cell check derating factor – when the `set_timing_derate -rise` and `-mean` parameters are specified. *Return type:* float
`early_rise_cell_check_sigma_derate_factor`	Returns the early cell check derating factor – when the `set_timing_derate -rise` and `-sigma` parameters are specified. *Return type:* float
`early_rise_clk_cell_derate_factor`	Returns the early clock cell check derating factor specified through the `set_timing_derate` command with the `-rise` parameter. *Return type:* float
`early_rise_clk_cell_mean_derate_factor`	Returns the early clock cell check derating factor – when the `set_timing_derate -rise` and `-mean` parameters are specified. *Return type:* float
`early_rise_clk_cell_sigma_derate_factor`	Returns the early clock cell check derating factor – when the `set_timing_derate -rise` and `-sigma` parameters are specified. *Return type:* float
`early_rise_clock_incremental_derate_factor`	Returns the early clock path instances' incremental derating factor specified through the `set_timing_derate -increment` parameter - when the `-rise` parameter is specified. *Return type*: float
`early_rise_data_cell_derate_factor`	Returns the early data cell check derating factor specified through the `set_timing_derate` command with the `-rise` parameter. *Return type:* float
`early_rise_data_cell_mean_derate_factor`	Returns the early data cell check derating factor – when the `set_timing_derate -rise` and `-mean` parameters are specified. *Return type:* float
`early_rise_data_cell_sigma_derate_factor`	Returns the early data cell check derating factor – when the `set_timing_derate -rise` and `-sigma` parameters are specified. *Return type:* float
`early_rise_data_incremental_derate_factor`	Returns the early data path instances' incremental derating factor specified through the `set_timing_derate -increment` parameter - when the `-rise` parameter is specified. *Return type*: float
`has_sdc_assertions`	Returns a value of `true`, if the specified cell has SDC assertions applied on it. For example, `get_property [get_cells <cell name>] has_sdc_assertions` *Return Type*: Boolean
`hierarchical_level`	Returns the hierarchical level of the specified cell. For example, `get_property [get_cell -hierarchical *] hierarchical_level` *Return Type*: Integer

`hierarchical_name`	Returns the complete hierarchical name of the cell. *Return type:* string
`instance_model`	Reports the model type of an instance. The Instance model type can be ecsmn, ccsn, cdb, udn, or nldm. In case a noise model does not exist, the software returns 'NA'. For example, `get_property [get_cells inst1/buf22 ] instance_model` *Return type:* string
`is_black_box`	Returns a value of `true` if the cell is a black box cell. A cell is considered as a black box if the "function" definition of the cell is missing from the library. *Return type:* boolean
`is_buffer`	Returns a value of `true` if the associated library cell is recognized as a buffer cell. For example, `get_property [get_cells mmu/IB1] is_buffer` *Return type:* boolean
`is_clock_gating_check`	Returns a value of true if the cell is gated. *Return type:* boolean
`is_clock_network_inst`	Returns a value of `true` if a cell is part of the clock network. *Return type*: boolean
`is_combinational`	Returns a value of `true` if the cell is a combinational cell (not a sequential cell). *Return type:* boolean
`is_disable_timing`	Returns a value of `true` if the cell's timing has been disabled using the `set_disable_timing` command. *Return type:* boolean
`is_dont_touch`	Returns a value of `true` if the cell definition includes `dont_touch:true`. *Return type:* boolean
`is_fall_edge_triggered`	Returns a value of `true` if the cell is triggered by the falling edge of the clock. *Return type:* boolean
`is_hierarchical`	Returns a value of `false` if the cell is a leaf or library cell. *Return type:* boolean
`is_integrated_clock_gating_cell`	Returns a value of `true` if the cell is an integrated clock gating cell. *Return type:* boolean
`is_interface_timing`	Returns a value of true if a library cell has interface timing specified for that cell. *Return Type*: boolean
`is_macro_cell`	Returns a value of `true` if the cell is a macro cell. For example, `get_property [get_cells macro1] is_macro_cell` *Return type:* boolean
`is_memory_cell`	Returns a value of `true` if the cell is a memory cell. *Return type:* boolean

`is_msv`	Returns a value of `true` if the cell has multiple supply voltages. For example, `get_property [get_cells msv_cell] is_msv` *Return type:* boolean
`is_mux`	Returns a value of `true` if the cell is mux. For example, `get_property [get_cells TDSP_CORE_INST/PORT_BUS_MACH_INST/p11358A] is_mux` *Return type:* boolean
`is_negative_level_sensitive`	Returns a value of `true` if the cell is used as a negative level-sensitive sequential element, such as a negative level-sensitive latch. *Return type:* boolean
`is_pad_cell`	Returns a value of `true` if the Liberty library `is_pad_cell` attribute is set to `true` for this pad cell. *Return type:* boolean
`is_positive_level_sensitive`	Returns a value of `true` if the cell is used as a positive level-sensitive sequential element, such as a positive level-sensitive latch. *Return type:* boolean
`is_rise_edge_triggered`	Returns a value of `true` if the cell is triggered by the rising edge of the clock. *Return type:* boolean
`is_sequential`	Returns a value of `true` if the cell is a latch or flip-flop, or if the cell has sequential timing checks. *Return type:* boolean
`is_tristate`	Returns a value of `true` if the cell definition includes `three_state:true`. *Return type:* boolean
`is_unmapped`	Returns a value of true if the specified cell name does not have a defined library cell. For example, `get_property [get_cells <cell name>] is_unmapped` *Return Type*: Boolean
`late_cell_check_derate_factor`	Returns the derating factor for late paths specified through the `set_timing_derate` command with the `-late` parameter. *Return type:* float
`late_cell_check_mean_derate_factor`	Returns the derating factor for late paths – when the `set_timing_derate -late` and `-mean` parameters are specified. *Return type:* float
`late_cell_check_sigma_derate_factor`	Returns the derating factor for late paths – when the `set_timing_derate -late` and `-sigma` parameters are specified. *Return type:* float
`late_clk_cell_derate_factor`	Returns the derating factor for late clock paths specified through the `set_timing_derate` command with the `-late` parameter. *Return type:* float
`late_clk_cell_mean_derate_factor`	Returns the derating factor for late clock paths – when the `set_timing_derate -late` and `-mean` parameters are specified. *Return type:* float
`late_clk_cell_sigma_derate_factor`	Returns the derating factor for late clock paths – when the `set_timing_derate -late` and `-sigma` parameters are specified. *Return type:* float

late_clock_check_incremental_derate_factor	Returns the late clock path instance's incremental derating factor specified through the set_timing_derate -increment parameter. *Return type*: float
late_data_cell_derate_factor	Returns the derating factor for late data paths specified through the set_timing_derate command with the -late parameter. *Return type:* float
late_data_cell_mean_derate_factor	Returns the derating factor for late data paths – when the set_timing_derate -late and -mean parameters are specified. For example, get_property [get_cells <cell name>] late_data_cell_mean_derate_factor *Return Type*: Float
late_data_cell_sigma_derate_factor	Returns the derating factor for late data paths – when the set_timing_derate -late and -sigma parameters are specified. *Return type:* float
late_data_check_incremental_derate_factor	Returns the late data path instances' incremental derating factor specified through the set_timing_derate -increment parameter. *Return type*: float
late_fall_cell_check_derate_factor	Returns the late cell check derating factor specified through the set_timing_derate command with the -fall parameter. *Return type:* float
late_fall_cell_check_mean_derate_factor	Returns the late cell check derating factor – when the set_timing_derate -fall and -mean parameters are specified. *Return type:* float
late_fall_cell_check_sigma_derate_factor	Returns the late cell check derating factor – when the set_timing_derate -fall and -sigma parameters are specified. *Return type:* float
late_fall_clk_cell_derate_factor	Returns the late clock cell check derating factor specified through the set_timing_derate command with the -fall parameter. *Return type:* float
late_fall_clk_cell_mean_derate_factor	Returns the late clock cell check derating factor – when the set_timing_derate -fall and -mean parameters are specified. *Return type:* float
late_fall_clk_cell_sigma_derate_factor	Returns the late clock cell check derating factor – when the set_timing_derate -fall and -sigma parameters are specified. *Return type:* float
late_fall_clock_incremental_derate_factor	Returns the late clock path instances' incremental derating factor specified through the set_timing_derate -increment parameter - when the -fall parameter is specified. Return type: float
late_fall_data_cell_derate_factor	Returns the late data cell check derating factor specified through the set_timing_derate command with the -fall parameter. *Return type:* float
late_fall_data_cell_mean_derate_factor	Returns the late data cell check derating factor – when the set_timing_derate -fall and -mean parameters are specified. *Return type:* float

late_fall_data_cell_sigma_derate_factor	Returns the late data cell check derating factor – when the set_timing_derate -fall and -sigma parameters are specified. *Return type:* float
late_fall_data_incremental_derate_factor	Returns the late data path instances' incremental derating factor specified through the set_timing_derate -increment parameter - when the -fall parameter is specified. *Return type*: float
late_rise_cell_check_derate_factor	Returns the late cell check derating factor specified through the set_timing_derate command with the -rise parameter. *Return type:* float
late_rise_cell_check_mean_derate_factor	Returns the late cell check derating factor – when the set_timing_derate -rise and -mean parameters are specified. *Return type:* float
late_rise_cell_check_sigma_derate_factor	Returns the late cell check derating factor – when the set_timing_derate -fall and -sigma parameters are specified. *Return type:* float
late_rise_clk_cell_derate_factor	Returns the late clock cell check derating factor specified through the set_timing_derate command with the -rise parameter. *Return type:* float
late_rise_clk_cell_mean_derate_factor	Returns the late clock cell check derating factor – when the set_timing_derate -rise and -mean parameters are specified. *Return type:* float
late_rise_clk_cell_sigma_derate_factor	Returns the late clock cell check derating factor – when the set_timing_derate -fall and -sigma parameters are specified. *Return type:* float
late_rise_clock_incremental_derate_factor	Returns the late clock path instances' incremental derating factor specified through the set_timing_derate -increment parameter - when the -rise parameter is specified. *Return type*: float
late_rise_data_cell_derate_factor	Returns the late data cell check derating factor specified through the set_timing_derate command with the -rise parameter. *Return type:* float
late_rise_data_cell_mean_derate_factor	Returns the late data cell check derating factor – when the set_timing_derate -rise and -mean parameters are specified. *Return type:* float
late_rise_data_cell_sigma_derate_factor	Returns the late data cell check derating factor – when the set_timing_derate -rise and -sigma parameters are specified. *Return type:* float
late_rise_data_incremental_derate_factor	Returns the late data path instances' incremental derating factor specified through the set_timing_derate -increment parameter - when the -rise parameter is specified. *Return type*: float
lib_cell	Returns a collection of all the library cell objects associated with the specified library pins/cells. For example, `get_property [get_cells AAA] lib_cell -view VIEW1` *Return type*: collection
name	Returns the leaf-level name of the cell. *Return type:* string

`object_type`	Returns the object type `cell`. *Return type:* string
`pin_count`	Returns the total number of signal pins. *Return type:* integer
`power_domain`	Returns the power domains of a cell. For example, `get_property [get_cells FE_OCPC5_psw_en2] power_domain` *Return type:* string
`ref_lib_cell_name`	Returns the name of reference module of the given cell. The reference module of the cell can be a Verilog module or a leaf library cell. *Return type:* string
`retention_cell`	Returns the value of the `retention_cell` attribute derived from Liberty. For example, `get_property [get_cells retention_cell] retention_cell` *Return type:* string
`timing_model_type`	Returns the model type for a given cell or instance. The supported values are `abstracted`, `extracted`, and `qtm`. *Return type:* string
`x_coordinate_max`	Returns the maximum x coordinate of the cell. *Return type:* float
`x_coordinate_min`	Returns the minimum x coordinate of the cell. *Return type:* float
`y_coordinate_max`	Returns the maximum y coordinate of the cell. *Return type:* float
`y_coordinate_min`	Returns the minimum y coordinate of the cell. *Return type:* float
`is_always_on`	Returns a value of `true` if a cell instance is an always-on cell. *Return type*: boolean For example, `get_property [get_cells mmu/U1] is_always_on`
`is_inverter`	Returns a value of `true` if a cell instance is an inverter cell. *Return type*: boolean For example, `get_property [get_cells mmu/INV1] is_inverter`
`is_isolation_cell`	Returns a value of `true` if a cell instance is an isolation cell. *Return type*: boolean For example, `get_property [get_cells mmu/U1] is_isolation_cell`
`is_level_shifter`	Returns a value of `true` if a cell instance is a level-shifter cell. *Return type*: boolean For example, `get_property [get_cells mmu/U1] is_level_shifter`
`is_power_switch`	Returns a value of `true` if a cell instance is a power switch cell. Return type: boolean For example, `get_property [get_cells mmu/U1] is_power_switch`

`is_retention`	Returns a value of `true` if a cell instance is a retention cell. *Return type*: boolean For example, `get_property [get_cells mmu/U1] is_retention`
`level_shifter_type`	Returns the type of level-shifter (e.g., LH, HL, or HL_LH) that a cell instance is. *Return type*: string For example, `get_property [get_cells mmu/U1] level_shifter_type`
`pg_pins`	Returns a collection of the PG pins that are present in a cell instance. *Return type*: collection For example, `get_property [get_cells mmu/U1] pg_pins`
`power_switch_type`	Returns the type of power switch (e.g., coarse-grain, fine-grain) that a cell instance is. *Return type*: string For example, `get_property [get_cells mmu/U1] power_switch_type`
`std_cell_main_rail_name`	Returns the name of the main power rail driving the power pins of a cell instance that is the `pg_pin`, which has the `std_cell_main_rail` library attribute set as `true` for the instance cell. *Return type*: string For example, `get_property [get_cells mmu/U1] std_cell_main_rail_name`

Clock Properties

`child_clocks`	Returns the first level of generated clocks. This property also works for nested generated clocks. *Return type*: collection For example, `get_property [get_clocks CK] child_clocks`
`clock_network_pins`	Returns a collection of all pins on the clock network for the clock. *Return type:* collection
`comment`	Returns the comment set for the specified clock. *Return Type*: String For example, `get_property [get_clocks CK] comment`
`delay_max_fall`	Returns the maximum falling delay value for the clock. *Return type:* float
`delay_max_rise`	Returns the maximum rising delay value for the clock. *Return type:* float
`delay_min_fall`	Returns the minimum falling delay value for the clock. *Return type:* float
`delay_min_rise`	Returns the minimum rising delay value for the clock. *Return type:* float
`divide_by`	Returns dfactor set for the specified clock. For example, `get_property [get_clocks ck] divide_by` *Return Type*: Integer

`driving_network_pins`	Returns a collection of the network pins of the driving clock. For example, `get_property [get_clocks clk] driving_network_pins` *Return Type*: Collection
`duty_cycle`	Returns the duty cycle for the specified clock. For example, `get_property [get_clocks CLK] duty_cycle` *Return Type*: Float
`edges`	Return the edges that are set for a specified clock. For example, `get_property [get_clocks ck] edges` *Return Type*: String
`edge_shift`	Return the edge shift that is set for a specified clock. For example, `get_property [get_clocks ck] edge_shift` Return Type: String
`generated_clocks_extended`	Returns generated clocks derived from the regular clock. *Return type:* collection
`hierarchical_name`	Returns the complete hierarchical name of the clock. *Return type:* string
`hold_uncertainty`	Returns the most conservative uncertainty of all the possible uncertainty assertions associated with a pin, when the `set_clock_uncertainty -hold` parameter is specified. For example, `get_property [get_clocks <clock name>] hold_uncertainty` *Return Type*: Float
`is_context_mapped`	Returns a value of `true` if a clock is mapped to the top in a context. Additionally, this is also reported with the `report_clocks` command. For example, `get_property [get_clocks <clock name>] is_context_mapped` *Return Type*: Boolean
`ideal_transition_max_fall`	Returns the ideal clock transition asserted on clock waveforms specified with the `set_clock_transition` command using the `-fall` and `-max` parameters. *Return type:* float
`ideal_transition_max_rise`	Returns the ideal clock transition asserted on clock waveforms specified with the `set_clock_transition` command using the `-rise` and `-max` parameters. *Return type:* float
`ideal_transition_min_fall`	Returns the ideal clock transition asserted on clock waveforms specified with `set_clock_transition` command using the `-fall` and `-min` parameters. *Return type:* float
`ideal_transition_min_rise`	Returns the ideal clock transition asserted on clock waveforms specified with `set_clock_transition` command using the `-rise` and `-min` parameters. *Return type:* float
`is_active`	Returns a value of `true` if the clock specified is active in the current view. *Return type:* boolean

`is_combinational_source_path`	Returns a value of `true` if `create_generated_clock -combinational` is specified for the creation of the generated clock. For example, `get_property [get_clocks GCLK] is_combinational_source_path` *Return type:* boolean
`is_generated`	Returns a value of `true` if the clock is a generated clock. *Return type:* boolean
`is_inverted`	Returns a value of `true`, if a specified clock is inverted. For example, `get_property [get_clocks ck] is_inverted` Return Type: boolean
`is_library_created`	Returns a value of `true` if `generated_clock` is specified in a Liberty `generated_clock` group and not created by a `create_generated_clock` constraint. For example, `get_property [get_clocks UMC/generated_clock_cell/clkint] is_library_created` *Return type:* boolean
`is_source_inverted`	Returns a value of `true` if a specified clock source is inverted. For example, `get_property [get_clocks clk] is_source_inverted` *Return Type*: boolean
`is_propagated_clock`	Returns a value of `true` if the clock is in propagated mode. *Return type:* boolean
`is_virtual`	Returns a value of `true` if the specified clock is virtual. For example, `get_property [get_clocks v_clk] is_virtual` `Return Type`: boolean
`latency_fall_max`	Returns the maximum fall latency for a clock, when the `set_clock_latency -max -fall` parameters are specified. *Return type:* float
`latency_fall_min`	Returns the minimum fall latency for a clock, when the `set_clock_latency -min -fall` parameters are specified. *Return type:* float
`latency_rise_max`	Returns the maximum rise latency for a clock, when the `set_clock_latency -max -rise` parameters are specified. For example, `get_property [get_clocks <clock name>] latency_rise_max` *Return Type*: float
`latency_rise_min`	Returns the minimum rise latency for a clock, when the `set_clock_latency -min -rise` parameters are specified. For example, `get_property [get_clocks <clock name>] latency_rise_min` *Return Type*: float
`lead_edge_type`	Returns an edge type (rise or fall) corresponding to the leading edge for a specified clock. For example, `get_property [get_clocks <clock name>] lead_edge_type` *Return Type*: string

`master_clock`	Returns a collection of the master clocks for a specified clock. For example, `get_property [get_clocks clk] master_clock` *Return Type*: collection
`master_source`	Returns a collection of the master sources for a specified clock. For example, `get_property [get_clocks ck] master_source` *Return Type*: collection
`max_capacitance_clock_path_fall`	Returns fall max_capacitance value on clock paths. *Return type:* float
`max_capacitance_clock_path_rise`	Returns rise max_capacitance value on clock paths. *Return type:* float
`max_capacitance_data_path_fall`	Returns fall max_capacitance value on data paths. *Return type:* float
`max_capacitance_data_path_rise`	Returns rise max_capacitance value on data paths. *Return type:* float
`max_transition_clock_path_fall`	Returns fall max_transition value on clock paths. *Return type:* float
`max_transition_clock_path_rise`	Returns rise max_transition value on clock paths. *Return type:* float
`max_transition_data_path_fall`	Returns fall max_transition value on data paths. *Return type:* float
`max_transition_data_path_rise`	Returns rise max_transition value on data paths. *Return type:* float
`min_pulse_width_high`	Returns the minimum pulse width for high signals for the specified clock, when the `set_min_pulse_width -high` parameter is specified. For example, `get_property [get_clocks <clock name>] min_pulse_width_high` *Return Type*: float
`min_pulse_width_low`	Returns the minimum pulse width for low signals for the specified clock, when the `set_min_pulse_width -low` parameter is specified. For example, `get_property [get_clocks <clock name>] min_pulse_width_low` *Return Type*: float
`multiply_by`	Returns mfactor set for the specified clock. For example, `get_property [get_clocks clk] multiply_by` *Return Type*: integer
`name`	Returns the name of the clock. *Return type:* string
`object_type`	Returns the object type `clock`. *Return type:* string

`period`	Returns the clock period. *Return type:* float
`setup_uncertainty`	Returns the most conservative uncertainty of all the possible uncertainty assertions associated with pins, when the `set_clock_uncertainty -setup` parameter is specified for a clock. For example, `get_property [get_clocks <clock name>]setup_uncertainty` *Return Type*: float
`source_jitter_early_fall_max`	Returns the amount of the maximum early fall source insertion delay that is due to cycle-to-cycle variation. *Return Type*: float
`source_jitter_early_fall_min`	Returns the amount of the minimum early fall source insertion delay that is due to cycle-to-cycle variation. *Return Type*: float
`source_jitter_early_rise_max`	Returns the amount of the maximum early rise source insertion delay that is due to cycle-to-cycle variation. *Return Type*: float
`source_jitter_early_rise_min`	Returns the amount of the minimum early rise source insertion delay that is due to cycle-to-cycle variation. *Return Type*: float
`source_jitter_full_cycle_hold`	Returns the early full cycle path jitter that is specified on the clock source. *Return Type*: float
`source_jitter_full_cycle_setup`	Returns the late full cycle path jitter that is specified on the clock source. *Return Type*: float
`source_jitter_half_cycle_hold`	Returns the early half cycle path jitter that is specified on the clock source. *Return Type*: float
`source_jitter_half_cycle_setup`	Returns the late half cycle path jitter that si specified on the clock source. *Return Type*: float
`source_jitter_late_fall_max`	Returns the amount of the maximum late fall source insertion delay that is due to cycle-to-cycle variation. *Return Type*: float
`source_jitter_late_fall_min`	Returns the amount of the minimum late fall source insertion delay that is due to cycle-to-cycle variation. *Return Type*: float
`source_jitter_late_rise_max`	Returns the amount of the maximum late rise source insertion delay that is due to cycle-to-cycle variation. *Return Type*: float
`source_jitter_late_rise_min`	Returns the amount of the minimum late rise source insertion delay that is due to cycle-to-cycle variation. *Return Type*: float
`source_latency_early_fall_max`	Returns the early source latency for the falling transition at the maximum corner, when the `set_clock_latency -source -early -fall -max` parameters are specified for the clock. For example, `get_property [get_clocks <clock name>] source_latency_early_fall_max` *Return Type*: float
`source_latency_early_fall_min`	Returns the early source latency for the falling transition at the min corner, when the `set_clock_latency -source -early -fall -min` parameters are specified for the clock. For example, `get_property [get_clocks <clock name>] source_latency_early_fall_min` *Return type:* float

source_latency_early_rise_max	Returns the early source latency for the rising transition at the max corner, when the set_clock_latency -source -early -rise -max parameters are specified for the clock. For example, get_property [get_clocks <clock name>] source_latency_early_rise_max *Return Type*: float
source_latency_early_rise_min	Returns the early source latency for the falling transition at the min corner, when the set_clock_latency -source -early -rise -min parameters are specified for the clock. For example, get_property [get_clocks <clock name>] source_latency_early_rise_min *Return type:* float
source_latency_late_fall_max	Returns the late source latency for the falling transition at the max corner, when the set_clock_latency -source -late -fall -max parameters are specified for the clock. For example, get_property [get_clocks <clock name>] source_latency_late_fall_max *Return type:* float
source_latency_late_fall_min	Returns the late source latency for the falling transition at the min corner, when the set_clock_latency -source -late -fall -min parameters are specified for the clock. For example, get_property [get_clocks <clock name>] source_latency_late_fall_min *Return type:* float
source_latency_late_rise_max	Returns the late source latency for the rising transition at the max corner, when the set_clock_latency -source -late -rise -max parameters are specified for the clock. For example, get_property [get_clocks <clock name>] source_latency_late_rise_max *Return type:* float
source_latency_late_rise_min	Returns the late source latency for the rising transition at the min corner, when the set_clock_latency -source -late -rise -min parameters are specified for the clock. For example, get_property [get_clocks <clock name>] source_latency_late_rise_min *Return type:* float
source_latency_pins	Returns a collection of pins and ports in the source latency network of a generated clock. This property is undefined for master clocks. You can index by clock name with the get_property -clock parameter. *Return type*: collection
source_latency_pins	Returns a list of pin or port object(s), which are present between the target pins of a generated clock and master clock. For example, get_property [get_clocks GCLK] source_latency_pins *Return type:* collection
sources	Returns a collection of the clock's source pins or ports. *Return type:* collection
view_name	In multi-mode multi-corner mode, returns the analysis view name associated with a given clock. *Return type*: string
waveform	Returns the rising and falling edge times of the waveform. *Return type:* string

Design Properties

`analysis_type`	Returns the analysis mode: `single`, `bc_wc`, or `on_chip_variation`. *Return type:* string
`boundary_model_analysis`	Returns a value of `true` if the design is a boundary model. For example, `get_property [get_designs MPTC] boundary_model_analysis` *Return type:* boolean
`capacitance_unit_in_farad`	Returns the capacitance unit specified in the library. *Return type:* float
`chip_size`	Returns the chip size. The chip size can be specified with the `timing_aocv_chip_size` attribute. If this attribute is not set, then the software will calculate the chip size from the design node locations. The attribute setting takes precedence over the software setting. *Return type:* float
`current_unit_in_amp`	Returns the current unit specified in the library. *Return type:* float
`hierarchical_name`	Returns the complete hierarchical name of the design. *Return type:* string
`is_dont_touch`	Returns `true` if the design is specified through the `set_dont_touch` command. *Return type:* boolean
`is_max_transition_asserted`	Returns a value of `true` if a maximum transition value (set using the `set_max_transition` command) is defined in the design. *Return type:* boolean
`is_min_transition_asserted`	Returns a value of `true` if a minimum transition value (set using the `set_min_transition` command) is defined in the design. *Return type:* boolean
`max_capacitance`	Returns the maximum capacitance value for the design. *Return type:* float
`max_fanout`	Returns the maximum fanout value for the design. *Return type:* float
`min_fanout`	Returns the minimum fanout value for the design. *Return type:* float
`max_transition`	Returns the maximum transition design rule limit for the design. *Return type:* float
`min_transition`	Returns the minimum transition design rule limit for the design. *Return type:* float
`min_capacitance`	Returns the minimum capacitance value for the design. *Return type:* float
`object_type`	Returns the object type `design`. *Return type: string*
`operating_condition_max`	Returns the name of the maximum operating condition specified for the design. *Return type:* string

`operating_condition_min`	Returns the name of the minimum operating condition specified for the design. *Return type:* string
`process_max`	Returns the process value of the maximum operating condition specified for the design. *Return type:* float
`process_min`	Returns the process value of the minimum operating condition specified for the design. *Return type:* float
`resistance_unit_in_ohm`	Returns the resistance value from the library. *Return type:* float
`slew_threshold_percent_fall_high`	Returns the upper slew trip point for falling waveforms, specified as a percent. *Return type:* float
`slew_threshold_percent_fall_low`	Returns the lower slew trip point for falling waveforms, specified as a percent. *Return type:* float
`slew_threshold_percent_rise_high`	Returns the upper slew trip point for rising waveforms, specified as a percent. *Return type:* float
`slew_threshold_percent_rise_low`	Returns the lower slew trip point for rising waveforms, specified as a percent. *Return type:* float
`temperature_max`	Returns the temperature value of the maximum operating condition for the design. *Return type:* float
`temperature_min`	Returns the temperature value of the minimum operating condition for the design. *Return type:* float
`timing_context_analysis`	Returns a value of `true` if timing context analysis is enabled on a design. *Return type*: boolean
`time_unit_in_second`	Returns the time units of the design, in seconds. that is, time unit of the design is 1.000000ns, the property returns 1.000000e-09. *Return type:* float
`tree_type_max`	Returns the value of the `tree_type` attribute for the specified maximum operating condition. *Return type:* string
`tree_type_min`	Returns the value of the `tree_type` attribute for the specified minimum operating condition. *Return type:* string
`voltage_max`	Returns the voltage value of the maximum operating condition for the design. *Return type:* float
`voltage_min`	Returns the voltage value of the minimum operating condition for the design. *Return type:* float
`voltage_unit_in_volt`	Returns the voltage units specified in the library for the design. *Return type:* float

Library Properties

`default_max_transition`	Returns the default maximum transition time limit specified for a given library. *Return type:* float

`default_operating_conditions`	Returns the default operating conditions specified for a given library. For example, `get_property [get_libs FreePDK45_lib_v1.0] default_operating_conditions` *Return type*: string
`hierarchical_name`	Returns the hierarchical path name for the library. *Return type:* string
`input_threshold_pct_fall`	Returns the value of threshold point on an input pin signal falling from 1 to 0, which is used in modelling the delay of a signal transmitting from an input pin to an output pin. *Return type:* integer
`input_threshold_pct_rise`	Returns the value of threshold point on an input pin signal rising from 0 to 1, which is used in modelling the delay of a signal transmitting from an input pin to an output pin. *Return type:* integer
`lib_file_path`	Reports full on-disk path of a library. In case of a library database (LDB) file, the property will return the full on-disk path of the original library from which LDB was compiled. *Return type:* string
`lib_version`	Returns the software version in which the library file (LDB) was generated. *Return type:* integer
`nominal_process`	Returns a value of the process either modeled as nominal process or as the process in default operating condition in the library. For example, `get_property [get_libs sc_cmoslp] nominal_process` *Return Type:* float
`nominal_temperature`	Returns a value of the temperature either modeled as nominal temperature or as the temperature in default operating condition in the library. For example, `get_property [get_libs sc_cmoslp] nominal_temperature` *Return Type:* float
`nominal_voltage`	Returns a value of the voltage either modeled as nominal voltage or as the voltage in default operating condition in the library. For example, `get_property [get_libs sc_cmoslp] nominal_voltage` *Return Type:* float
`object_type`	Returns the object type `library`. *Return type:* string
`output_threshold_pct_fall`	Returns the value of threshold point on an output signal falling from 1 to 0, which is used in modelling the delay of a signal transmitting from an input pin to an output pin. *Return type:* integer
`output_threshold_pct_rise`	Returns the value of threshold point on an output signal rising from 0 to 1, which is used in the modelling of a signal transmitting from an input pin to an output pin. *Return type:* integer

`power_rails`	Returns the name of the power rails along with their corresponding voltages as defined in the `voltage_map` attribute in the library.
	For example,
	`get_property [get_libs sc_cmoslp] power_rails`
	`Output: Is a list of lists, each list showing the power rail name along with its associated voltage.`
	`{ {VDDG 1.045} {VDD 1.045} {VSS 0} }`
	Return Type: string

`slew_derate_from_library`	Specifies how the transition times found in the library need to be derated to match the transition times between the characterization trip points. *Return type:* float
`slew_lower_threshold_pct_fall`	Returns the value of lower threshold point used in modeling the delay of a pin falling from 1 to 0. *Return type:* float
`slew_lower_threshold_pct_rise`	Returns the value of lower threshold point used to model the delay of a pin rising from 0 to 1. *Return type:* float
`slew_upper_threshold_pct_fall`	Returns the value of upper threshold point used to model the delay of a pin falling from 1 to 0. *Return type:* float
`slew_upper_threshold_pct_rise`	Returns the value of upper threshold point used to model the delay of a pin rising from 0 to 1. *Return type:* float
`source_file_name`	Returns the full path of all library files that contain the specified library name. The use model is given below: `get_property [get_libs libname] source_file_name [-view viewName]` If the `-view` parameter is specified, the property returns the path of the library files in the given analysis view. If this parameter is not specified, the command returns the path of all library files in all the analysis views. *Return Type:* string
`threshold_voltage_group`	Returns the threshold voltage group of a library. For example, `get_property [get_lib FreePDK45_lib_v1.0] threshold_voltage_group` *Return Type:* string

Library Cell Properties

`area`	Returns the area of the library cell. *Return Type:* float
`cell_footprint`	Returns the cell footprint against a library cell, which is used to assign the same footprint class to all cells th the same layout boundary. For example, `get_property [get_lib_cells 1.08_CTS_HT.db/CTB_F18_HT] cell_footprint` *Return Type*: String
`clock_gating_integrated_cell`	Returns the clock gating integrated cells present in a library cell. For example, `get_property [get_lib_cells CTS_HT.db/ICG_F4_HVT] clock_gating_integrated_cell` *Return Type*: String
`late_clock_check_incremental_derate_factor`	Returns the late clock path library cell check incremental derating factor specified through the `set_timing_ increment` parameter. Return Type: float
`late_data_check_incremental_derate_factor`	Returns the late data path library cell check incremental derating factor specified through the `set_timing_d increment` parameter. Return Type: float

late_fall_clock_incremental_derate_factor	Returns the late clock path library cell check incremental derating factor specified through the set_timing_ increment parameter - when the -fall parameter is specified. Return Type: float
late_fall_data_incremental_derate_factor	Returns the late data path library cell check incremental derating factor specified through the set_timing_d increment parameter - when the -fall parameter is specified. Return Type: float
early_clock_check_incremental_derate_factor	Returns the early clock path library cell check incremental derating factor specified through the set_timing increment parameter. *Return Type*: float
early_data_check_incremental_derate_factor	Returns the early data path library cell check incremental derating factor specified through the set_timing_ increment parameter. *Return Type*: float
early_fall_clock_incremental_derate_factor	Returns the early clock path library cell check incremental derating factor specified through the set_timing increment parameter - when the -fall parameter is specified. *Return Type*: float
early_fall_data_incremental_derate_factor	Returns the early data path library cell check incremental derating factor specified through the set_timing_ increment parameter - when the -fall parameter is specified. *Return Type*: float
early_fall_mean_slew_derate_factor	Returns the early fall mean transition derate factor on the library cell. For example, get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] early_fall_mean_slew_derate_factor *Return Type*: float
early_fall_sigma_slew_derate_factor	Returns the early fall sigma transition derate factor on the library cell. For example, get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] early_fall_sigma_slew_derate_factor *Return Type*: float
early_rise_clock_incremental_derate_factor	Returns the early clock path library cell check incremental derating factor specified through the set_timing increment parameter - when the -rise parameter is specified. *Return Type*: float
early_rise_data_incremental_derate_factor	Returns the early data path library cell check incremental derating factor specified through the set_timing_ increment parameter - when the -rise parameter is specified. *Return Type*: float
early_rise_mean_slew_derate_factor	Returns the early rise mean transition derate factor on the library cell. For example, get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] early_rise_mean_slew_derate_factor *Return Type*: float
early_rise_sigma_slew_derate_factor	Returns the early rise sigma transition derate factor on the library cell. For example, get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] early_rise_sigma_slew_derate_factor *Return Type*: float
early_fall_slew_derate_factor	Returns the derate factor applied on a library cell when the set_timing_derate -transition and -fall par are specified. *Return Type*: float

`early_rise_slew_derate_factor`	Returns the derate factor applied on a library cell when the `set_timing_derate -transition` and `-rise` para are specified. For example, `get_property [get_lib_cells CTS_HT.db/ICG_F4_HT] early_rise_slew_derate_factor` *Return Type*: float
`fall_input_switching_derate_factor`	Returns the derate factor applied on a library cell when the `set_timing_derate -input_switching` paramet specified. For example, `get_property [get_lib_cells PDK45_v1.0/IV_X32] fall_input_switching_derate_factor` *Return Type*: float
`has_ccs_noise`	Returns a value of `true` if a library cell has a CCS noise look-up table. For example, `get_property [get_lib_cells tst/tst_nm6] has_ccs_noise` *Return type:* boolean
`has_socv`	Returns a value of `true` if a library cell has a spatial derate table (SOCV). For example, `get_property [get_lib_cells PK4_lib_v1.0/A4_X2] has_socv` *Return type:* boolean
`has_spatial`	Returns a value of `true` if a spatial derate exists for a library cell. *Return type:* boolean
`hierarchical_name`	Returns the complete hierarchical name of the library cell (the library name followed by the library cell nam For example, `get_property [get_lib_cells -of [get_cell AB_IT/E_0146_tl]] hierarchical_name` *Return Type:* string
`is_black_box`	Returns a value of `true` if the cell is a black box cell. A cell is considered as a black box if the "function" definition of the cell is missing from the library. *Return type:* boolean
`is_buffer`	Returns a value of `true` if the associated library cell is recognized as a buffer cell. For example, `get_property [get_lib_cells -of [get_cell AB_IT/E_0146_tl]] is_buffer` *Return type:* boolean
`is_combinational`	Returns a value of `true` if the library cell is a combinational cell (not a sequential cell). *Return type:* boolean
`is_dont_touch`	Returns a value of `true` if the library cell definition includes `dont_touch:true`. *Return Type:* boolean
`is_dont_use`	Returns a value of `true` if the library cell description includes `dont_use:true`. *Return Type:* boolean
`is_fall_edge_triggered`	Returns a value of `true` if the library cell is triggered by the falling edge of the clock. *Return Type:* boolean
`is_instantiated`	Returns a value of `true` if library cells are instantiated in a design. For example, `get_property [get_lib_cells P4_lib_v1.0/N4_X2] is_instantiated` *Return type*: boolean

`is_interface_timing`	Returns a value of `true` if a library cell has interface timing specified for that cell. *Return type*: boolean
`is_level_shifter`	Returns a value of `true` if a cell instance is a level-shifter cell. *Return type*: boolean
`is_integrated_clock_gating_cell`	Returns a value of `true` if a cell is an integrated clock gating cell. For example, `get_property [get_lib_cells CTS_HT.db/ICG_F7_HT] is_integrated_clock_gating_cell` *Return type*: boolean
`is_inverter`	Returns a value of `true` if a cell instance is an inverter cell. For example, `get_property [get_lib_cells -of [get_cells inv]] is_inverter` *Return type*: boolean
`is_level_shifter`	Returns a value of `true` if a cell instance is a level-shifter cell. For example, `get_property [get_lib_cells -of [get_libs mk_1pv_125c]] is_level_shifter` *Return type*: boolean
`is_macro_cell`	Returns a value of `true` if a macro cell exists in a library. For example, `get_property [get_lib_cells L_mro2/mro2y] is_macro_cell` *Return type*: boolean

`is_memory_cell`	Returns a value of `true` if the cell is a memory cell.
	Return type: boolean
	Note: The library cells that have a memory group defined in the dotlib are considered as memory cells. For
	`cell ( ) {`
	`:`
	`memory() {`
	`type : ram ;`
	`address_width : 10 ;`
	`word_width : 7 ;`
	`}`
	`}`

`is_msv`	Returns a value of `true` if a cell is an MSV cell. For example, `get_property [get_lib_cells mk_1pv_125c/L8MTR] is_msv` *Return type*: boolean
`is_negative_level_sensitive`	Returns a value of `true` if the library cell is used as a negative level-sensitive latch. *Return type:* boolean
`is_pad_cell`	Returns a value of `true` if the Liberty library `is_pad_cell` attribute is set to `true` for this pad cell. *Return type:* boolean
`is_pll_cell`	Returns a value of `true` if the Liberty library `is_pll_cell` attribute is set to `true` for this library cell. *Return type:* boolean
`is_positive_level_sensitive`	Returns a value of `true` if the library cell is used as a positive level-sensitive latch. *Return type:* boolean

`is_rise_edge_triggered`	Returns a value of `true` if the library cell is triggered by the rising edge of the clock. *Return Type:* boolean
`is_sequential`	Returns a value of `true` if the library cell is a latch or flip-flop, or if the cell has sequential timing checks. *Return type:* boolean
`is_tristate`	Returns a value of `true` if the library cell definition includes `three_state:true`. *Return type:* boolean
`late_fall_clock_period_derate_factor`	Returns the derate factor for the clock period when the `set_timing_derate -late` and `-fall` parameters are *Return type*: float
`late_fall_clock_period_mean_derate_factor`	Returns the late/fall derate factor for the clock period when the `set_timing_derate -mean` parameter is spec *Return type*: float
`late_fall_clock_period_sigma_derate_factor`	Returns the late/fall derate factor for the clock period when the `set_timing_derate -sigma` parameter is spe *Return type*: float
`late_fall_mean_slew_derate_factor`	Returns the late/fall mean transition derate factor on the library cell. For example, `get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] late_fall_mean_slew_derate_factor` *Return type:* float
`late_fall_pulse_width_derate_factor`	Returns the late/fall derate factor for the pulse width when the `set_timing_derate -late` and `-fall` parame specified. *Return type*: float
`late_fall_pulse_width_mean_derate_factor`	Returns the late/fall derate factor for the pulse width when the `set_timing_derate -mean` parameter is speci *Return type*: float
`late_fall_pulse_width_sigma_derate_factor`	Returns the late/fall derate factor for the pulse width when the `set_timing_derate -sigma` parameter is spec *Return type*: float
`late_fall_sigma_slew_derate_factor`	Returns the late/fall sigma transition derate factor on the library cell. For example, `get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] late_fall_sigma_slew_derate_factor` *Return type:* float
`late_fall_slew_derate_factor`	Returns the derate factor applied on a library cell, when the `set_timing_derate -transition` and `-fall` par are specified. *Return type:* float
`late_rise_clock_incremental_derate_factor`	Returns the late clock path library cell check incremental derating factor specified through the `set_timing_` `increment` parameter - when the `-rise` parameter is specified. *Return Type*: float
`late_rise_data_incremental_derate_factor`	Returns the late data path library cell check incremental derating factor specified through the `set_timing_d` `increment` parameter - when the `-rise` parameter is specified. *Return Type*: float
`late_rise_clock_period_derate_factor`	Returns the derate factor for the clock period when the `set_timing_derate -late` and `-rise` parameters are *Return type*: float
`late_rise_clock_period_mean_derate_factor`	Returns the late/rise derate factor for the clock period when the `set_timing_derate -mean` parameter is spe *Return type*: float
`late_rise_clock_period_sigma_derate_factor`	Returns the late/rise derate factor for the clock period when the `set_timing_derate -sigma` parameter is sp *Return type*: float

`late_rise_mean_slew_derate_factor`	Returns the late/rise mean transition derate factor on the library cell. For example, `get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] late_fall_mean_slew_derate_factor` *Return type:* float
`late_rise_pulse_width_derate_factor`	Returns the derate factor for the pulse width check when the `set_timing_derate -late` and `-rise` paramete specified. *Return type*: float
`late_rise_pulse_width_mean_derate_factor`	Returns the late/rise derate factor for the pulse width when the `set_timing_derate -mean` parameter is spec *Return type*: float
`late_rise_pulse_width_sigma_derate_factor`	Returns the late/rise derate factor for the pulse width when the `set_timing_derate -sigma` parameter is spe *Return type*: float
`late_rise_sigma_slew_derate_factor`	Returns the late/rise sigma transition derate factor on the library cell. For example, `get_property [get_lib_cells FreePDK45_lib_v1.0/AND2_X1] late_rise_sigma_slew_derate_factor` *Return type:* float
`late_rise_slew_derate_factor`	Returns the derate factor applied on a library cell, when the `set_timing_derate -transition` and `-rise` par are specified. For example, `get_property [get_lib_cells PK4_v1.0/IV_32] late_rise_slew_derate_factor` *Return type:* float
`mis_alignments_factor_min_fall`	Returns the minimum fall tuning factor applied on a library cell, when the `set_multi_input_switching_mode` `switching_alignment_factor` parameter is specified. *Return type:* float
`mis_alignments_factor_min_rise`	Returns the minimum rise tuning factor applied on a library cell, when the `set_multi_input_switching_mod` `switching_alignment_factor` parameter is specified. *Return type:* float
`master_physical_variant_cell`	Returns the master library cell of the specified physical variant cell. For example, `get_property [get_lib_cell cell_w/INV2] master_physical_variant_cell` *Return type:* string
`name`	Returns the leaf-level name of the library cell. *Return Type:* string
`object_type`	Returns the object type `lib_cell`. *Return Type:* string
`physical_variant_cells`	Returns the physical variant cells that are available in a library cell. For example, `get_property [get_lib_cells lca_md/SR1_md] physical_variant_cells` *Return Type:* string
`pin_count`	Returns the total number of signal pins. For example, `get_property [get_lib_cells PK45_v1.0/IN_X32] pin_count` *Return Type:* integer
`receiver_capacitance_type`	Returns the value of input capacitance group being used for delay calculations – C1C2 (2-piece model) or piece model). *Return Type*: string

`retention_cell`	Returns the value of the retention_cell attribute specified in the liberty. *Return type*: string
`rise_input_switching_derate_factor`	Returns the derate factor applied on a library cell, when the `set_timing_derate -input_switching` paramet specified. For example, `get_property [get_lib_cells PK45_v1.0/DL_X2] rise_input_switching_derate_factor` *Return type:* float
`signal_pins`	Returns the signal pins of a cell, along with the library name, library cell name, and pin name. For example, `get_property [get_lib_cells PK45_v1.0/IN_X32] signal_pins` *Return Type:* string
`sync_clear`	Returns the collection of synchronous clear pins. **Note:** For a synchronous clear pin: • The flip-flop's function definition should have the clear signal in the `next_state` attribute's logic expressi • The pin corresponding to the synchronous clear input must specify: `nextstate_type: clear` For example, `get_property [get_lib_cells tcbn05_bwph210l6p51cnod_base_svtffgnp_0p825v_125c_cbest_CCbest_T_em_ccs/SDFKSRPQD1BWP210H6P51C sync_clear` *Return type:* collection
`sync_enable`	Returns the collection of synchronous enable pins. For example, `get_property [get_lib_cells tcbn05_bwph210l6p51cnod_base_ulvtllffgnp_0p825v_125c_cbest_CCbest_T_em_ccs/EDFQD1BWP210H6P51CN sync_enable` *Return type:* collection
`sync_preset`	Returns the collection of synchronous preset pins. For example, `get_property [get_lib_cells tcbn05_bwph210l6p51cnod_base_svtffgnp_0p825v_125c_cbest_CCbest_T_em_ccs/SDFKSRPQD1BWP210H6P51C sync_preset` *Return type:* collection
`test_scan_enable`	Returns the collection of the `test_scan_enable` pins. For example, `get_property [get_lib_cell SR80_W_125_0.855_0.855_CORE_SVT.db/sdffbqs_f4_12t_svt] test_scan_en` *Return type:* collection
`test_scan_enable_inverted`	Returns the collection of the `test_scan_enable_inverted` pins. For example, `get_property [get_lib_cells SR80_W_125_0.855_0.855_CORE_SVT.db/sdffbqs_f4_12t_svt] test_scan_enable_inverted` *Return type:* collection
`test_scan_in`	Returns the collection of the `test_scan_in` pins. For example, `get_property [get_lib_cells SR80_W_125_0.855_0.855_CORE_SVT.db/sdffbqs_f4_12t_svt] test_scan_i` *Return type:* collection

test_scan_in_inverted	Returns the collection of the test_scan_in_inverted pins. For example, get_property [get_lib_cell SR80_W_125_0.855_0.855_CORE_SVT.db/sdffbqs_f4_12t_svt] test_scan_ir *Return type:* collection
test_scan_out	Returns the collection of the test_scan_out pins. For example, get_property [get_lib_cells SR80_W_125_0.855_0.855_CORE_SVT.db/sdffbqs_f4_12t_svt] test_scan_c *Return type:* collection
test_scan_out_inverted	Returns the collection of the test_scan_out_inverted pins. For example, get_property [get_lib_cell SR80_W_125_0.855_0.855_CORE_SVT.db/sdffbqs_f4_12t_svt] test_scan_out_inverted *Return type:* collection
output_signal_level_high	Returns the maximum voltage value of an output pin after a rising transition. For example, get_property [get_lib_arcs -of [get_lib_cells px_ipid_3/px_ipid_3]] output_signal_level_high *Return type:* float
output_signal_level_low	Returns the minimum voltage value of an output pin after a falling transition. For example, get_property [get_lib_arcs -of [get_lib_cells px_ipid_3/px_ipid_3]] output_signal_level_low *Return type:* float
threshold_voltage_group	Returns the voltage group defined at the library level or a library cell specific group definition (specified usi the set_voltage_threshold_group command). *Return type*: string For example, get_property [get_lib_cells L40_HT.db/CG_F8_HT] threshold_voltage_group
timing_model_type	Returns the model type for a given cell or instance. The supported values are abstracted, extracted, and c *Return type*: string
ground_pins	Returns a collection of the ground pins of a library cell. *Return type*: collection For example, get_property [get_lib_cells sc_cmoslp/AO22X] ground_pins
is_always_on	Returns a value of true if the always_on attribute of a library cell is set to true indicating that the library cell always-on cell. *Return type*: boolean For example, get_property [get_lib_cells sc_cmoslp/AON22_X4M] is_always_on
is_isolation_cell	Returns a value of true if the library cell is an isolation cell. *Return type*: boolean For example, get_property [get_lib_cells sc_cmoslp/ISO22_X4M] is_isolation_cell
is_power_switch	Returns a value of true when the switch_cell_type attribute is defined for a library cell indicating that the l is a power-switch cell. *Return type*: boolean For example, get_property [get_lib_cells sc_cmoslp/AO22X] is_power_switch

`is_retention`	Returns a value of `true` when the `retention_cell` attribute is defined for a library cell indicating that the libr a retention cell. *Return type*: boolean For example, `get_property [get_lib_cells sc_cmoslp/RET22_X4M] is_retention`
`level_shifter_type`	Returns the type of level-shifter (e.g., LH, HL, or HL_LH) as defined by the `level_shifter_type` attribute of cell. *Return type*: string For example, `get_property [get_lib_cells sc_cmoslp/A2LVLD_X4M] level_shifter_type`
`lib_pg_pins`	Returns a collection of all PG pins (both power and ground) defined in the library of a library cell. *Return type*: collection For example, `get_property [get_lib_cells sc_cmoslp/AO22X] lib_pg_pins`
`power_pins`	Returns a collection of the power pins of a library cell. *Return type*: collection For example, `get_property [get_lib_cells sc_cmoslp/AO22X] power_pins`
`power_switch_type`	Returns the type of power-switch (e.g., coarse-grain, fine-grain) as defined by the `switch_cell_type` attribu library cells. *Return type*: string For example, `get_property [get_lib_cells sc_cmoslp/AO22X] power_switch_type`
`std_cell_main_rail_name`	Returns the name of a power rail that has the `std_cell_main_rail` library attribute defined as `true` for a libr *Return type*: string For example, `get_property [get_lib_cells sc_cmoslp/AO22X] std_cell_main_rail_name`

Library Pin Properties

`all_q_pin_of_d_pin`	Returns a collection of `all_q_pin_of_d_pin` pins. For example, `get_property [get_lib_pin srpg/SDRFFSRPQ_X1/D] all_q_pin_of_d_pin` *Return type:* collection
`capacitance`	Returns the capacitance for the library pin. *Return type:* float
`capacitance_max_fall`	Returns the maximum value of the falling capacitance range. *Return type:* float
`capacitance_max_rise`	Returns the maximum value of the rising capacitance range. *Return type:* float
`capacitance_min_fall`	Returns the minimum value of the falling capacitance range. *Return type:* float
`capacitance_min_rise`	Returns the minimum value of the rising capacitance range. *Return type:* float
`cell_name`	Returns the library cell name for the corresponding library pins. For example, `get_property [get_lib_pins PDK45_lib_v1.0/AOI211_X2/B] cell_name` *Return type*: string

`direction`	Returns the direction of the library pin: `in`, `out`, or `inout`. *Return type:* string
`drive_resistance_fall`	Specifies the driving resistance of the cell for falling transitions. For example, `get_property [ get_lib_pins FreePDK45_lib_v1.0/BUF_X1/Z ] drive_resistance_fall` *Return type:* float
`drive_resistance_rise`	Specifies the driving resistance of the cell for rising transitions. For example, `get_property [ get_lib_pins FreePDK45_lib_v1.0/BUF_X1/Z ] drive_resistance_rise` *Return type:* float
`fanout_load`	Returns the fanout load value of the library pin. *Return type:* float
`function`	Returns the function for the corresponding library pins. For example, `get_property [get_lib_pins PK4_lib_v1.0/AND2_X1/ZN] function` *Return type*: string
`hierarchical_name`	Returns the hierarchical name of the library pin. *Return type:* string
`input_signal_level`	Returns the power rail name/signal that is defined for a library cell input pin. For example, `get_property [get_lib_pins PK4_lib_v1.0/BUF_X32/A] input_signal_level` *Return type*: string
`input_signal_level_high`	Returns the upper limit of partial voltage swing value that is defined for a library cell input pin. For example, `get_property [get_lib_pins rvt_ss_0p9v/LVLL/A] input_signal_level_high` *Return type*: float
`input_signal_level_low`	Returns the lower limit of partial voltage swing value that is defined for a library cell input pin. For example, `get_property [get_lib_pins rvt_ss_0p9v/LVLL/A] input_signal_level_low` *Return type*: float
`is_analog`	Returns a value of `true` if the library pin is an analog pin. For example, `get_property [get_lib_pins PK4_lib_v1.0/AOI222_X2/ZN] is_analog` *Return type*: boolean
`is_async`	Returns a value of `true` if the library pin is an asynchronous preset pin, or an asynchronous clear pin. *Return type:* boolean
`is_clear`	Returns a value of `true` if the pin is an asynchronous clear pin. *Return type:* boolean
`is_clock`	Returns a value of `true` if the library pin definition includes `clock:true`. *Return type:* boolean
`is_clock_gating_clock`	Returns a value of `true` if the library pin is defined as a clock pin of a clock gating cell. For example, `get_property [get_lib_pins P40_CTS_HVT.db/ICG_HVT/CLKIN] is_clock_gating_clock` *Return type*: boolean

`is_clock_gating_enable`	Returns a value of `true` if the library pin corresponds to the enable pin of a clock gating cell. For example, `get_property [get_lib_pins P40_CTS_HVT.db/ICG_HVT/CLKIN] is_clock_gating_enable` *Return type*: boolean
`is_clock_gating_test`	Returns a value of `true` if the pin corresponds to the test pin of a clock gating cell. For example, `get_property [get_lib_pins */NAND2_X2/A2] is_clock_gating_test` *Return type*: boolean
`is_clock_pin`	Returns a value of `true` if the library pin is a clock pin. For example, `get_property [get_lib_pins srpg/SDRFFSRPQ_X1/CK] is_clock_pin` *Return Type*: boolean
`is_data`	Returns a value of `true` if the library pin with defined sequential arcs is the data pin of a flip-flop. *Return type:* boolean
`is_fall_edge_triggered_clock`	Returns a value of `true` if the library pin is the clock pin of a falling edge triggered device. *Return type:* boolean
`is_fall_edge_triggered_data`	Returns a value of `true` if the library pin is the data pin of a falling edge triggered device. *Return type:* boolean
`is_iq_function`	Returns a value of `true` if the output is an IQ pin. For example, `get_property [get_lib_pins */SDFFRS_X1/Q] is_iq_function` *Return Type*: boolean
`is_iqn_function`	Returns a value of `true` if the output is an IQN pin. For example, `get_property [get_lib_pins */SDFFRS_X1/QN] is_iqn_function` *Return Type*: boolean
`is_isolation_cell_clock`	Returns a value of `true` if the clock pin is a pin of isolation cell clock. For example, `get_property [get_lib_pins cell_w/INV/CLK] is_isolation_cell_clock` *Return Type*: boolean
`is_negative_level_sensitive_clock`	Returns a value of `true` if the library pin is an enable pin of an active low level-sensitive device. *Return type:* boolean
`is_negative_level_sensitive_data`	Returns a value of `true` if the pin is a data pin of an active low level-sensitive device. *Return type:* boolean
`is_pad`	Returns a value of `true` if the library pin is a pad pin. For example, `get_property [get_lib_pins *] is_pad` *Return type*: boolean
`is_pll_feedback_pin`	Returns a value of `true` if the Liberty library `is_pll_feedback_pin` attribute is set to `true` for this library pin. *Return type:* boolean
`is_pll_output_pin`	Returns a value of `true` if the Liberty library `is_pll_output_pin` attribute is set to `true` for this library pin. *Return type:* boolean

`is_pll_reference_pin`	Returns a value of `true` if the Liberty library `is_pll_reference_pin` attribute is set to `true` for this library pin. *Return type:* boolean
`is_positive_level_sensitive_clock`	Returns a value of `true` if the library pin is an enable pin of an active high level-sensitive device. *Return type:* boolean
`is_positive_level_sensitive_data`	Returns a value of `true` if the pin is a data pin of an active high level-sensitive device. *Return type:* boolean
`is_preset`	Returns a value of `true` if the pin is a preset pin. *Return type:* boolean
`is_ref_clock_pin`	Returns a value of `true`: • If the library pin is a reference clock pin. • Valid instance pin object and on an active clock pin where a clock signal reaches, and sequential cell is not disabled by disabled timing arcs or case analysis. Any pin with clock check-arcs or trigger-arcs will also be reported, if an active clock phase is present on that pin. • Clock pin on an inferred clock gating cell. • Valid `lib_pin` object, if there is at least one instance of this `lib_pin` in the design, and if `is_clock_pin` property is set to true. Returns a value of `false`, if clock propagation is disabled with the `set_disable_timing` or `set_sense -stop_propagation` command. For example, `get_property [get_lib_pins srpg/SDRFFSRPQ_X1/CK] is_ref_clock_pin` *Return Type*: boolean
`is_rise_edge_triggered_clock`	Returns a value of `true` if the library pin is the clock pin of a rising edge triggered device. *Return type:* boolean
`is_rise_edge_triggered_data`	Returns a value of `true` if the library pin is the data pin of a rising edge triggered device. *Return type:* boolean
`is_tristate`	Returns a value of `true` if the library pin definition includes `three_state:true`. *Return type:* boolean
`max_capacitance`	Returns the maximum capacitance limit for the library pin. *Return type:* float
`max_fanout`	Returns the maximum fanout value for the library pin. *Return type:* float
`max_transition`	Returns the maximum transition time limit specified for a given library pin. If the limit is not specified for a library pin, then the limit specified at the corresponding library level will be used. *Return type:* float
`min_transition`	Returns the minimum transition time limit specified for a given library pin. If the limit is not specified for a library pin, then the limit specified at the corresponding library level will be used. *Return type:* float
`min_capacitance`	Returns the minimum capacitance limit for the library pin. *Return type:* float
`min_fanout`	Returns the minimum fanout value for the library pin. *Return type:* float
`name`	Returns the leaf-level name of the library cell pin. *Return type:* string

`object_type`	Returns the object type; in this case, `lib_pin`. *Return type:* string
`related_ground_pin_name`	Returns related ground pin names for the corresponding library pins. *Return type:* string For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_ground_pin_name`
`related_power_pin_name`	Returns related power pin names for the corresponding library pins. *Return type:* string
`slew_threshold_percent_fall_high`	Returns the upper slew trip point for falling waveforms, specified as a percent. *Return type:* float
`slew_threshold_percent_fall_low`	Returns the lower slew trip point for falling waveforms, specified as a percent. *Return type:* float
`slew_threshold_percent_rise_high`	Returns the upper slew trip point for rising waveforms, specified as a percent. *Return type:* float
`slew_threshold_percent_rise_low`	Returns the lower slew trip point for rising waveforms, specified as a percent. *Return type:* float
`input_signal_level_voltage`	Returns the voltage value corresponding to the power rail name/signal which is defined for the library-cell input pin. This voltage value is the full rail-to-rail voltage value that is associated with the relative power rail name through the `voltage_map` attribute defined in the library. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] input_signal_level_voltage` *Return type:* float
`is_always_on`	Returns a value of `true` if the `always_on` attribute of a library signal pin is set to `true` indicating that the library cell signal pin is an always-on pin For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_always_on` *Return type:* boolean
`is_isolated`	Returns a value of `true` if the `is_isolated` attribute of a library signal pin is set to `true` indicating that the library cell signal pin is an internally isolated pin. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_isolated` *Return type:* boolean
`is_isolation_cell_data`	Returns a value of `true` if the `isolation_cell_data_pin` attribute of a library signal pin is set to `true` indicating that the library cell signal pin is a data pin of an isolation library cell. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_isolation_cell_data` *Return type:* boolean
`is_isolation_cell_enable`	Returns a value of `true` if the `isolation_cell_enable_pin` attribute of a library signal pin is set to `true` indicating that the library cell signal pin is an enable or control pin of an isolation library cell. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_isolation_cell_enable` *Return type:* boolean

`is_level_shifter_data`	Returns a value of `true` if the `level_shifter_data_pin` attribute of a library signal pin is set to `true`, indicating that the library cell signal pin is a data pin of a library level-shifter cell. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_level_shifter_data` *Return type:* boolean
`is_level_shifter_enable`	Returns a value of `true` if the `level_shifter_enable_pin` attribute of a library signal pin is set to be `true` indicating that the library-cell signal pin is an enable pin of a library level shifter cell. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_isolation_cell_enable` *Return type:* boolean
`is_power_switch_enable`	Returns a value of `true` if the `switch_cell_type` attribute of the corresponding library cell is defined and the `switch_pin` attribute of the given library-cell signal pin is defined to be `true`. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_power_switch_enable` *Return type:* boolean
`is_retention_cell_enable`	Returns a value of `true` if the `retention_pin` attribute of a signal pin is defined. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] is_retention_cell_enable` *Return type:* boolean
`output_signal_level`	Returns the power rail name/signal that is defined for a library cell output and inout pins. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] output_signal_level` *Return type:* string
`output_signal_level_high`	Returns the upper/higher limit of partial voltage swing value that is defined for a library cell output and inout pins. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] output_signal_level_high` *Return type:* float
`output_signal_level_low`	Returns the lower limit of partial voltage swing value that is defined for a library cell output and inout pins. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] output_signal_level_low` *Return type:* float
`pin_capacitance`	Returns the pin capacitance. For example, `get_property [get_lib_pins FreePDK45_lib_v1.0/BUF_X1/A] pin_capacitance` *Return type:* float
`pin_capacitance_max_fall`	Returns the maximum value of the falling capacitance range of the corresponding library pin. For example, `get_property [get_lib_pins FreePDK45_lib_v1.0/BUF_X1/A] pin_capacitance_max_fall` *Return type:* float
`pin_capacitance_max_rise`	Returns the maximum value of the rising capacitance range of the corresponding library pin. For example, `get_property [get_lib_pins FreePDK45_lib_v1.0/BUF_X1/A] pin_capacitance_max_rise` *Return type:* float
`pin_capacitance_min_fall`	Returns the minimum value of the falling capacitance range of the corresponding library pin. For example, `get_property [get_lib_pins FreePDK45_lib_v1.0/BUF_X1/A] pin_capacitance_min_fall` *Return type:* float

`pin_capacitance_min_rise`	Returns the minimum value of the rising capacitance range of the corresponding library pin. For example, `get_property [get_lib_pins FreePDK45_lib_v1.0/BUF_X1/A] pin_capacitance_min_rise` *Return type:* float
`related_ground_pin`	Returns the related ground pins as a collection for the corresponding library cell pins. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_ground_pin` *Return type:* collection
`related_ground_pin_name`	Returns the related ground pin names for the corresponding library cell pins. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_ground_pin_name` *Return type:* string
`related_ground_pin_rail_voltage`	Returns the voltage value corresponding to the ground rail name defined for the `related_ground_pin` attribute of the library cell pin. This voltage value is the full rail-to-rail value that is associated with the relative ground rail name through the `voltage_map` attribute defined in the library. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_ground_pin_rail_voltage` *Return type:* float
`related_power_pin`	Returns the related power pins as a collection for the corresponding library cell pins. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_power_pin` *Return type:* collection
`related_power_pin_name`	Returns the related power pin names for the corresponding library cell pins. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_power_pin_name` *Return type:* string
`related_power_pin_rail_voltage`	Returns the voltage value corresponding to the power rail name defined for the `related_power_pin` attribute of a library cell pin. This voltage value is the full rail-to-rail value that is associated with the relative power rail name through the `voltage_map` attribute defined in the library. For example, `get_property [get_lib_pins sc_cmoslp/AO22X/I1] related_power_pin_rail_voltage` *Return type:* float

Library PG Pin Properties

`is_std_cell_main_rail`	Returns a value of `true` if the library power PG pin is the main rail of the library cell. This is determined by the `std_cell_main_rail` library attribute of the library `pg_pin`. *Return type*: boolean For example, `get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] is_std_cell_main_rail`
`name`	Returns the name of the library PG pin as defined in the library and associated with the `pg_pin` library attribute. *Return type*: string For example, `get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] name`
`object_type`	Returns `lib_pg_pin` as the object type for a library PG pin. *Return type*: string For example, `get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] object_type`

`pg_type`	Returns the type of the library PG pin as associated with the pg_type attribute of the corresponding library PG pin.
	Return type: string
	
	For example,
	`get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] pg_type`
	`get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X] -filter " name =~ *VDD*" ] pg_type`

`rail_name`	Returns the name of the rail to which the PG pin is connected and is associated with the `voltage_name` attribute of the library PG pin. *Return type*: string For example, `get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] rail_name`
`rail_voltage`	Returns the voltage from the voltage_map association of the rail to which the library PG pin is connected. *Return type*: float For example, `get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] rail_voltage`
`tied_to`	Returns the library PG pin which is connected to substrate-bias PG pin. *Return type*: string For example, `get_property [get_lib_pg_pins -of_objects [get_lib_cells sc_cmoslp/AO22X]] tied_to`

Library Timing Arc Properties

from_lib_pin	Returns a collection of the source library pins of the library timing arc. *Return type:* collection
has_ccs_noise	Returns a value of true if a cell has CCS noise data. For example, `get_property [get_lib_arc -from ccs_timing_si/AN2D1BWP20P90/A1 -to ccs_timing_si/AN2D1BWP20P90/Z] has_ccs_noise` *Return type:* boolean
has_mis_fall	Returns a value of true, if MIS (multi-input switching) derate fall value is specified for library timing arcs. For example, `get_property [get_lib_arc -from FreePDK45_lib_v1.0/AND2_X1/A1 -to FreePDK45_lib_v1.0/AND2_X1/ZN] has_mis_fall` *Return type:* boolean
has_mis_rise	Returns a value of true, if MIS (multi-input switching) derate rise value is specified for library timing arcs. For example, `get_property [get_lib_arcs -from FreePDK45_lib_v1.0/OR2_X1/A1 -to FreePDK45_lib_v1.0/OR2_X1/ZN] has_mis_rise` *Return type:* boolean
has_socv	Returns a value of true if a timing library arc in the library has a spatial derate table (SOCV). *Return type:* boolean
is_disabled	Return a value of true if a set_disable_timing constraint exists for a an arc. *Return type:* boolean
mode	Returns the name of the group if the timing arc is defined as part of a Liberty mode group. For example, `get_property [get_lib_arc -from c065lesram76x2x1x96b_PttV1200T025_lib/c065lesram76x2x1x96b/CL -to c065lesram76x2x1x96b_PttV1200T025_lib/c065lesram76x2x1x96b/RY] mode` *Return type:* string
object_type	Returns the object type lib_timing_arc. *Return type:* string
sdf_cond	Returns the sdf_cond expression specified for the timing library arc in the library. *Return type:* string
sense	Returns the timing_sense specified for the timing library arc in the library. *Return type:* string
to_lib_pin	Returns a collection of the sink library pins of the library timing arc. *Return type:* collection
timing_type	Returns the type specified for the timing library arc in the library. *Return type:* string
when	Returns the when condition specified for the library timing arc in the library. *Return type:* string
when_library_string	Returns the exact value of the Liberty 'when' attribute for the specified arc. For example, `get_property [get_lib_arc -from FreePDK45_lib_v1.0/BUF_X1/A -to FreePDK45_lib_v1.0/BUF_X1/Z] when_library_string` *Return type:* string

Net Properties

`annotated_capacitance_max`	Returns the maximum annotated capacitance of the net for late path analysis. For example, `get_property [get_nets w2] annotated_capacitance_max` *Return type:* float
`annotated_capacitance_min`	Returns the minimum annotated capacitance of the net for late path analysis. For example, `get_property [get_nets w2] annotated_capacitance_min` *Return type:* float
`attackers`	Reports the parasitic data on a net - capacitance, resistance, or cross-coupling. *Return type:* collection
`capacitance_max`	Returns the total capacitance of the net for maximum conditions. *Return type:* float
`coupling_capacitance_max`	Returns the maximum value of coupling capacitance of a net over all the views. *Return type:* float
`capacitance_min`	Returns the total capacitance of the net for minimum conditions. *Return type:* float
`coupling_capacitance_min`	Returns the minimum value of coupling capacitance of a net over all the views. *Return type:* float
`delta_delay_max`	Returns the delta delay for the maximum condition. *Return type:* float
`delta_delay_max_fall`	Returns the maximum falling delta delay value. *Return type:* float
`delta_delay_max_rise`	Returns the maximum rising delta delay value. *Return type:* float
`delta_delay_min`	Returns the delta delay for the minimum condition. *Return type:* float
`delta_delay_min_fall`	Returns the minimum falling delta delay value. *Return type:* float
`delta_delay_min_rise`	Returns the minimum rising delta delay value. *Return type:* float
`driver_pins`	Returns a collection of the driver pins of the net. *Return type:* collection
`early_fall_clk_net_derate_factor`	Returns the early fall clock net derating factor specified through `set_timing_derate` command. *Return type:* float
`early_fall_clk_net_delta_derate_factor`	Returns the early fall clock delta delay net derating factor specified through the `set_timing_derate` command. *Return type:* float
`early_fall_data_net_delta_derate_factor`	Returns the early fall data delta net derating factor specified through the `set_timing_derate` command. *Return type:* float
`early_fall_data_net_derate_factor`	Returns the early fall data net derating factor specified through `set_timing_derate` command. *Return type:* float

early_rise_clk_net_delta_derate_factor	Returns the early rise clock delta net derating factor specified through the set_timing_derate command. *Return type:* float
early_rise_clk_net_derate_factor	Returns the early rise clock net derating factor specified through the set_timing_derate command. *Return type:* float
early_rise_data_net_delta_derate_factor	Returns the early rise data delta net derating factor specified through the set_timing_derate command. *Return type:* float
early_rise_data_net_derate_factor	Returns the early rise data net derating factor specified through the set_timing_derate command. *Return type:* float
has_detailed_parasitics	Returns a value of true if the net, or one of its parts, has detailed parasitics associated with it from either RC extraction or SPEF annotation. *Return type:* boolean
hierarchical_name	Returns the complete hierarchical name of the net. *Return type:* string
is_clock_source_latency	Returns a value of true if the net is a part of clock source latency network. The clock source latency network consists of the clock network cells, its pins, and the nets connecting the master clock source pin and the generated clock source pin. For example, `get_property [get_nets A] is_clock_source_latency` *Return type*: boolean
is_clock_source_latency	Returns a value of true if a pin, port or any of the leaf pins of a net is part of a clock source latency network. *Return Type*: Boolean
is_dont_touch	Returns a value of true if a set_dont_touch constraint exists for the net. *Return type:* boolean
is_hierarchical	Returns a value of true if the net is a part of hierarchical module boundary. For example, `get_property [get_nets W1] is_hierarchical` *Return type*: boolean
is_ideal	Returns a value of true if set_ideal_latency, set_ideal_transition, or set_ideal_network constraints exist for the net. *Return type*: Boolean
late_fall_clk_net_delta_derate_factor	Returns the late fall clock delta net derating factor specified through the set_timing_derate command. *Return type:* float
late_fall_clk_net_derate_factor	Returns the late fall clock net derating factor specified through the set_timing_derate command. *Return type:* float
late_fall_data_net_delta_derate_factor	Returns the late fall data delta net derating factor specified through the set_timing_derate command. *Return type:* float
late_fall_data_net_derate_factor	Returns the late fall data net derating factor specified through the set_timing_derate command. *Return type:* float
late_rise_clk_net_delta_derate_factor	Returns the late rise clock delta net derating factor specified through the set_timing_derate command. *Return type:* float
late_rise_clk_net_derate_factor	Returns the late rise clock net derating factor specified through the set_timing_derate command. *Return type:* float

late_rise_data_net_delta_derate_factor	Returns the late rise data delta net derating factor specified through the set_timing_derate command. *Return type:* float
late_rise_data_net_derate_factor	Returns the late rise data net derating factor specified through the set_timing_derate command. *Return type:* float
is_dont_touch	Returns a value of true if a set_dont_touch constraint exists for the net. *Return type:* boolean
load_pins	Returns a collection of the load pins of the net. *Return type:* collection
lumped_capacitance_max	Returns the maximum value of capacitance of a net based on the input library and parasitic model. *Return type:* float
lumped_capacitance_min	Returns the minimum value of capacitance of a net based on the input library and parasitic model. *Return type:* float
lumped_resistance_max	Returns the maximum resistance of the net for late path analysis. You can use -index to return the value for a specific view. For example, get_property [get_nets FE_SIG_N671_FE_OFCN486_ds_data_6_] lumped_resistance_max *Return type:* float
lumped_resistance_min	Returns the maximum resistance of the net for late path analysis. You can use -index to return the value for a specific view. For example, get_property [get_nets FE_SIG_N671_FE_OFCN486_ds_data_6_] lumped_resistance_min *Return type:* float
name	Returns the name of the net. *Return type:* string
num_driver_pins	Returns the number of driver pins on a net. *Return type*: integer
num_load_pins	Returns the number of load pins on the net. *Return type*: collection
object_type	Returns the object type net. *Return type:* string
pin_capacitance_max	Returns the sum of all pin capacitances of the net for maximum conditions. *Return type:* float
pin_capacitance_max_fall	Returns the sum of all pin capacitances of the net for the maximum value of the rising capacitance range. *Return type:* float
pin_capacitance_max_rise	Returns the sum of all pin capacitances of the net for the maximum value of the falling capacitance range. *Return type:* float
pin_capacitance_min	Returns the sum of all pin capacitances of the net for minimum conditions. *Return type:* float
Note: For all the pin_capacitance_* properties (described above), the pin capacitance matches with the capacitance displayed under the pin_load column of the report_timing output.	

pin_capacitance_min_fall	Returns the minimum fall capacitance present at the pin of a net. For example, `get_property [get_nets UMC/w3] pin_capacitance_min_fall` *Return type:* float
pin_capacitance_min_rise	Returns the minimum rise capacitance present at the pin of a net. For example, `get_property [get_nets UMC/w3] pin_capacitance_min_rise` *Return type:* float
resistance_max	Returns the maximum value of resistance of a net. *Return type*: float
resistance_min	Returns the minimum value of resistance of a net. *Return type*: float
timing_constant	Reports a constant value on the net. For example, `get_property [get_nets UMC/constant_net] timing_constant` *Return type*: string
timing_constant_source	Reports the source through which the timing constant is applied. For example, `get_property [get_nets UMC/constant_net] timing_constant_source` *Return type*: string
total_capacitance_max_fall	Returns the total falling capacitance of the net used for late path analysis. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] total_capacitance_max_fall -view func_slow_RCMAX` *Return type*: float
total_capacitance_max_rise	Returns the total rising capacitance of the net used for late path analysis. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] total_capacitance_max_rise -view func_slow_RCMAX` *Return type*: float
total_capacitance_min_fall	Returns the total falling capacitance of the net used for early path analysis. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] total_capacitance_min_fall -view func_fast_RCMIN` *Return type*: float
total_capacitance_min_rise	Returns the total rising capacitance of the net used for early path analysis. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] total_capacitance_min_rise -view func_fast_RCMIN` *Return type*: float
total_lumped_capacitance_max_fall	Returns the maximum value of falling capacitance of a net based on the input library and parasitic model. *Return type:* float
total_lumped_capacitance_max_rise	Returns the maximum value of rising capacitance of a net based on the input library and parasitic model. *Return type:* float
total_lumped_capacitance_min_fall	Returns the minimum value of falling capacitance of a net based on the input library and parasitic model. *Return type:* float
total_lumped_capacitance_min_rise	Returns the minimum value of rising capacitance of a net based on the input library and parasitic model. *Return type:* float

vl_vh_analysis_status	Reports the glitch analysis (VL/VH) status for a net. For example, `get_property [get_nets -of [get_cells FE_SIG_C622_FE_OFCN517_digit_2_]] vl_vh_analysis_status` *Return type:* string
vlu_vho_analysis_status	Reports the glitch analysis status for undershoot (VLU) and overshoot (VHO) glitch for a net. For example, `get_property [get_nets FE_SIG_N642_ds_data_5_ ] vlu_vho_analysis_status` *Return type:* string
wire_capacitance_max	Returns the wire capacitance of the net for maximum conditions. *Return type:* float
wire_capacitance_max_fall	Returns the wire capacitance of the net for the maximum value of the falling capacitance range. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] wire_capacitance_max_fall -view func_slow_RCMAX` *Return type:* float
wire_capacitance_max_rise	Returns the wire capacitance of the net for the maximum value of the rising capacitance range. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] wire_capacitance_max_rise -view func_slow_RCMAX` *Return type:* float
wire_capacitance_min	Returns the wire capacitance of the net for minimum conditions. *Return type:* float
wire_capacitance_min_fall	Returns the wire capacitance of the net for the minimum value of the falling capacitance range. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] wire_capacitance_min_fall -view func_fast_RCMIN` *Return type:* float
wire_capacitance_min_rise	Returns the wire capacitance of the net for the minimum value of the rising capacitance range. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] wire_capacitance_min_rise -view func_fast_RCMIN` *Return type:* float
wire_lumped_capacitance_max	Returns the portion of the net's total lumped capacitance which comes extracted or annotated wire lumped capacitance values. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] wire_lumped_capacitance_max -view func_slow_RCMAX` *Return type:* float
wire_lumped_capacitance_max_fall	Returns the lumped wire capacitance of the net for the maximum value of the falling capacitance range. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] wire_lumped_capacitance_max_fall -view func_slow_RCMAX` *Return type:* float
wire_lumped_capacitance_max_rise	Returns the lumped wire capacitance of the net for the maximum value of the rising capacitance range. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] wire_lumped_capacitance_max_rise -view func_slow_RCMAX` *Return type:* float

wire_lumped_capacitance_min	Returns the portion of the net's total lumped capacitance which comes extracted or annotated wire lumped capacitance values. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] wire_lumped_capacitance_min -view func_fast_RCMIN` *Return type:* float
wire_lumped_capacitance_min_fall	Returns the lumped wire capacitance of the net for the minimum value of the falling capacitance range. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] wire_lumped_capacitance_min_fall -view func_fast_RCMIN` *Return type:* float
wire_lumped_capacitance_min_rise	Returns the lumped wire capacitance of the net for the minimum value of the rising capacitance range. For example, `get_property [get_nets FE_SIG_N643_ds_data_7_] wire_lumped_capacitance_min_rise -view func_fast_RCMIN` *Return type:* float
x_coordinate_max	Returns the maximum value of the `x` coordinate of the net. For example, `get_property [get_nets FE_SIG_N671_FE_OFCN486_ds_data_6_] x_coordinate_max` *Return type:* float
x_coordinate_min	Returns the minimum value of the `x` coordinate of the net. For example, `get_property [get_nets FE_SIG_N671_FE_OFCN486_ds_data_6_] x_coordinate_min` *Return type:* float
y_coordinate_max	Returns the maximum value of the `y` coordinate of the net. For example, `get_property [get_nets FE_SIG_N671_FE_OFCN486_ds_data_6_] y_coordinate_max` *Return type:* float
y_coordinate_min	Returns the minimum value of the `y` coordinate of the net. For example, `get_property [get_nets FE_SIG_N671_FE_OFCN486_ds_data_6_] y_coordinate_min` *Return type:* float

PG Net Properties

asserted_voltage_max	Returns the maximum asserted voltage on the power ground net. *Return type:* string
asserted_voltage_min	Returns the minimum asserted voltage on the power ground net. *Return type:* string

hierarchical_name	Returns the name of the PG net connected to the corresponding PG pin.
	Return type: string
	For example,
	`get_property [get_pg_nets -of_objects [get_cells iA/out_reg] ] hierarchical_name`
	`get_property [get_pg_nets -of_objects [get_cells iA/out_reg] -filter "is_power == true" ] hierarchical_name`

is_ground	Returns a value of `true` if the PG net is a ground net.
	Return type: boolean
	For example,
	`get_property [get_pg_nets -of_objects [get_cells iA/out_reg] ] is_ground`
	`get_property [get_pg_nets -of_objects [get_pg_pins -of_objects [get_cells iA/out_reg] ]] is_ground`

`is_power`	Returns a value of `true` if the PG net is a power net.
	Return type: boolean
	
	For example,
	`get_property [get_pg_nets -of_objects [get_cells iA/out_reg] ] is_power`
	`get_property [get_pg_nets -of_objects [get_cells iA/out_reg] -filter "name =~ *VDD*" ] is_power`

`is_primary`	Returns a value of `true` if the PG net is primary. `get_property [get_pg_nets VDD1] is_primary` *Return type*: boolean
`voltage_max`	Returns the maximum voltage of a given PG net. This could be voltage associated with a driver term of a net, power intent or CPF, `related_pg_pins` attribute in Liberty, or the operating voltage of a power domain or system default voltage. *Return type*: float For example, `get_property [get_pg_nets -of_objects [get_cells iA/out_reg] ] voltage_max`
`voltage_min`	Returns the minimum voltage of a given PG net. This could be voltage associated with a driver term of a net, power intent or CPF, `related_pg_pins` attribute in Liberty or the operating voltage of a power domain or system default voltage. *Return type*: float For example, `get_property [get_pg_nets -of_objects [get_cells iA/out_reg] ] voltage_min`
`object_type`	Returns `pg_net` as the object type for a library PG pin. *Return type*: string For example, `get_property [get_pg_nets -of_objects [get_cells iA/out_reg]] object_type`
`pg_net_group`	Returns the name of the PG net group for the pg_net object. `get_property [get_pg_nets VDDR] pg_net_group` *Return type*: string

`parent_pg_net`	Returns a collection of a parent power net that is the net powering the power switch that drives the PG net. In case of designs that do not have any power switches powering the instances, this property will always return the net directly powering those instances. It therefore be noted that the usage of this property is more relevant for designs in which the leaf cell instances are powered through power switch cells.
	Return type: Collection
	
	For example,
	`get_property [get_pg_nets mmu/U1/vdd] parent_pg_net`
	`get_property [get_pg_nets -of_objects [get_cells iA/pwr_swtch] ] parent_pg_net`

`top_pg_net`	Returns a collection of the top most un-switched power net of the current PG net. Even in the case of a cascading of power switches present in a design, the software traces through all power switches back to the un-switched top-level primary power net. The top-most un-switched power net will be returned - switched output power net will not be returned.
	Return type: Collection
	
	For example,
	`get_property [get_pg_nets -of_objects [get_cells iA/pwr_swtch] ] top_pg_net`
	`get_property [get_pg_nets -of_objects [get_pg_pins -of_objects [get_cells iA/pwr_swtch] ]] top_pg_net`

Path Group Properties

`name`	Returns the name of the path group. *Return type:* string
`hierarchical_name`	Returns the hierarchical name of the path group. *Return type:* string
`group_type`	Returns the type of group, when `group_path` and `-name`, `-from`, `-to` parameters are specified for the path. For example, `get_property [get_path_groups] group_type` *Return Type*: string
`object_type`	Returns the object type `path_group`. *Return type:* string
`view_name`	Returns the view name for the specified internal clock group. For example, `get_property [get_path_groups m_clk -include_internal_groups] view_name` *Return type:* string

Pin Properties

`actual_latency_early_fall_max`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual early fall insertion delay in setup mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual early fall insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`actual_latency_early_fall_min`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual early fall insertion delay in hold mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual early fall insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`actual_latency_early_rise_max`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual early rise source insertion delay in setup mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual early rise source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float

`actual_latency_early_rise_min`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual early rise source insertion delay in hold mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual early rise source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`actual_latency_late_fall_max`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual late fall source insertion delay in setup mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual late fall source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`actual_latency_late_fall_min`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual late fall source insertion delay in hold mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual late fall source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`actual_latency_late_rise_max`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual late rise source insertion delay in setup mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual late rise source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`actual_latency_late_rise_min`	When specified with the `set_clock_latency -clock` parameter, returns the worst actual late rise source insertion delay in hold mode for the specified clock at the given pin. The latency reported is ideal or propagated, depending on the clock phase. When specified without the `-clock` parameter, returns the worst actual late rise source insertion delay on the pin. If multiple clocks reach the pin, the reported latency is the worst among all the clocks. When specified with the `-view` parameter, returns the worst latency for the given view. *Return type:* float
`arrival_clocks`	Returns the data phases, clock source data phases, and latency phases. *Return type:* collection The following attributes support these phases: • `timing_property_arrival_clocks_consider_clock_source_data_phase`: Considers the clock source data phases. • `timing_property_arrival_clocks_consider_data_phase`: Considers the pure data phase. • `timing_property_arrival_clocks_consider_latency_phase`: Considers the latency phase. By default, these attributes are set to `true`.
`arrival_max`	Returns the value of the arrival max at specified pin. You can verify through `report_timing`. For example, `get_property [get_pins <pin name>] arrival_max` *Return Type:* float

`arrival_max_fall`	Returns the arrival time for the falling transition at the max corner. *Return type:* float
`arrival_max_rise`	Returns the arrival time for the rising transition at the max corner. *Return type:* float
`arrival_mean_max_fall`	Returns the mean component of the arrival time for the falling transition at the max corner. *Return type:* float
`arrival_mean_max_rise`	Returns the mean component of the arrival time for the rising transition at the max corner. *Return type:* float
`arrival_mean_min_fall`	Returns the mean component of the arrival time for the falling transition at the min corner. *Return type:* float
`arrival_mean_min_rise`	Returns the mean component of the arrival time for the rising transition at the min corner. *Return type:* float
`arrival_min`	Returns the value of the arrival min at specified pin. You can verify through `report_timing`. For example, `get_property [get_pins <pin name>] arrival_min` *Return Type:* Float
`arrival_min_fall`	Returns the arrival time for the falling transition at the min corner. *Return type:* float
`arrival_min_rise`	Returns the arrival time for the rising transition at the min corner. *Return type:* float
`arrival_sigma_max_fall`	Returns the sigma component of the arrival time for the falling transition at the max corner. *Return type:* float
`arrival_sigma_max_rise`	Returns the mean component of the arrival time for the rising transition at the max corner. *Return type:* float
`arrival_sigma_min_fall`	Returns the sigma component of the arrival time for the falling transition at the min corner. *Return type:* float
`arrival_sigma_min_rise`	Returns the sigma component of the arrival time for the falling transition at the min corner. *Return type:* float
`arrival_skewness_max_fall`	In SOCV analysis mode, this returns the skewness value of the latest falling arrival time to the specified pin across all the concurrent MMMC views. `get_property [get_pins cell1/ZN] arrival_skewness_max_fall` *Return type:* float
`arrival_skewness_max_rise`	In SOCV analysis mode, this returns the skewness value of the latest rising arrival time to the specified pin across all the concurrent MMMC views. `get_property [get_pins cell1/ZN] arrival_skewness_max_rise` *Return type:* float
`arrival_skewness_min_fall`	In SOCV analysis mode, this returns the skewness value of the earliest falling arrival time to the specified pin across all the concurrent MMMC views. `get_property [get_pins cell1/ZN] arrival_skewness_min_fall` *Return type:* float

`arrival_skewness_min_rise`	In SOCV analysis mode, this returns the skewness value of the earliest rising arrival time to the specified pin across all the concurrent MMMC views. `get_property [get_pins cell1/ZN] arrival_skewness_min_rise` *Return type:* float
`arrival_window`	Returns the arrival times for the earliest and latest rising and falling transitions in timing window format. *Return type:* string
`attacker_fall_slew`	Returns the minimum falling slew of an attacker driver pin. For example: `get_property [get_pins inst1/y] attacker_fall_slew` *Return type:* float
`attacker_rise_slew`	Returns the minimum rising slew of an attacker driver pin. For example: `get_property [get_pins inst1/y] attacker_rise_slew` *Return type:* float
`capacitance_max`	Returns the value of the maximum pin capacitance specified for the capacitance attribute in the library for the corresponding pin. For example, `get_property [get_pins <pin_name>] capacitance_max` *Return Type:* float
`capacitance_max_fall`	Returns the maximum value of the falling capacitance range of the corresponding library pin. *Return type:* float
`capacitance_max_rise`	Returns the maximum value of the rising capacitance range of the corresponding library pin. *Return type:* float
`capacitance_min`	Returns the value of the minimum pin capacitance specified for the capacitance attribute in the library for the corresponding pin. For example, `get_property [get_pins <pin_name>] capacitance_min` *Return Type:* float
`capacitance_min_fall`	Returns the minimum value of the falling capacitance range of the corresponding library pin. *Return type:* float
`capacitance_min_rise`	Returns the minimum value of the rising capacitance range of the corresponding library pin. **Note**: For the `capacitance_max_*` and `capacitance_min_*` properties (described above), the capacitance value does not match with the `pin_load` (`report_timing -format`) in the timing report because the pin load is a sum of all pin capacitances of a net. *Return type:* float
`capture_clocks`	Returns the capture clock of the pin. `get_property [get_pins UC/ret_cell/Q] capture_clocks` *Return type*: collection
`cell_name`	Returns the cell name for the corresponding pins. For example, `get_property [get_pins addrs_4_/A] cell_name` *Return type*: string

`clock_pulse_width_high`	Returns the actual high pulse width for the specified pin.
	`create_clock -name CK -period 30 -waveform {0 10} [get_ports {CK}]`
	You can verify through `report_min_pulse_width -clock <clock name>`
	For example,
	`get_property [get_pins <pin_name>] clock_pulse_width_high`
	Return Type: float

`clock_pulse_width_low`	Returns the actual low pulse width for the specified pin.
	`create_clock -name CK -period 30 -waveform {0 10} [get_ports {CK}]`
	You can verify through `report_min_pulse_width -clock <clock name>`
	For example,
	`get_property [get_pins <pin_name>] clock_pulse_width_low`
	Return Type: float

`clock_sources`	Returns a collection of the source pins of all the clocks arriving at the pin. *Return type:* collection
`clocks`	Returns a collection of all of the clocks arriving at the pin. This property will report clocks on hierarchical pins even before a full timing update is performed (with the `update_timing -full` command). *Return type:* collection
`constrained_capture_clocks`	Returns the constrained capture clocks at the specified pin. `get_property [get_pins UC/ret_cell/Q] constrained_capture_clocks` *Return type:* collection
`constant_value`	Returns a value of 0 or 1 if a logic state has been asserted or propagated to the pin. `get_property [get_pins RM_128x16_INST/oae] constant_value` *Return type:* string
`delay_max_fall`	Returns the maximum falling delay - specified using the `set_max_delay -fall` command. *Return type:* float
`delay_max_rise`	Returns the maximum rising delay - specified using the `set_max_delay -rise` command. *Return type:* float
`delay_min_fall`	Returns the minimum falling delay - specified using the `set_min_delay -fall` command. *Return type:* float
`delay_min_rise`	Returns the minimum rising delay - specified using the `set_min_delay -rise` command. *Return type:* float
`direction`	Returns the direction of the pin: `in`, `out`, or `inout`. *Return type:* string
`driver_cell`	Returns the driver cell name for the corresponding pins. For example, `get_property [get_pins ON28_n_17/A] driver_cell` *Return type:* string
`driver_cell_name`	Returns the driving cell instance for the specified pin. For example, `get_property [get_pins <pin name>]driver_cell_name` *Return Type:* string

`driver_power_pin_voltage_max`	Reports the maximum power pin voltage of a driver cell. `get_property [get_pins UC/mcr_cell/a] driver_power_pin_voltage_max` *Return type:* float
`driver_power_pin_voltage_min`	Reports the minimum power pin voltage of a driver cell. `get_property [get_pins UC/mcr_cell/a] driver_power_pin_voltage_min` *Return type:* float
`driver_ref_lib_cell_name`	Returns the library defined driving cell for the specified pin. For example, `get_property [get_pins <pin name>] driver_ref_lib_cell_name` *Return Type:* string
`escaped_name`	Returns the full hierarchical name, including escaped characters (if any). *Return Type:* string
`fanin`	Returns the number of sources connected with net of the specified pin. *Return type:* integer
`fanout`	Returns the number of sinks connected with the net of the specified pin. *Return type:* integer
`fanout_load`	Returns the fanout load for the pin. *Return type:* float

`fastest_clock`	Returns the collection of the clock name defined for the specified pin.
	`create_generated_clock -name ck -source CK -divide_by 2 <pin name>`
	For example,
	`get_property [get_pins <pin_name>] fastest_clock`
	Return Type: collection

`fastest_clock_name`	Returns the clock name defined for the specified pin.
	`create_generated_clock -name clk -source CLK -multiply_by 2 <pin name>`
	For example,
	`get_property [get_pins <pin name>] fastest_clock_name`
	Return Type: string

`fastest_clock_period`	Returns the clock period of the clock defined for the specified pin.
	`create_clock -name CLK -period 10 -waveform {0 5} [get_ports {CK2}]`
	`create_generated_clock -name clk -source CLK`
	`-multiply_by 3 <pin name>`
	You can verify through `report_clocks`.
	For example,
	`get_property [get_pins <pin_name>] fastest_clock_period`
	Return Type: float

`hierarchical_level`	Returns the hierarchical level of the pin. For example, `get_property [get_pins <pin name>] hierarchical_level` *Return Type:* integer
`hierarchical_name`	Returns the complete hierarchical name of the pin. *Return type:* string

`hold_uncertainty`	Returns the most conservative uncertainty of all possible uncertainty assertions associated with the pin. *Return type:* float

`holding_resistance_high`	Returns the highest holding resistance of a pin.
	For example,
	`foreach pin $pinList {`
	`set vicArray [get_property [get_pins $pin] noise_victim]`
	`foreach_in_collection vic $vicArray {`
	`get_property $vic holding_resistance_high`
	`}`
	Return type: float

`holding_resistance_low`	Returns the lowest holding resistance of a pin. *Return type:* float
`interp_info_max`	Returns information for late analysis. When specified, the report output will show the data for interpolation, extrapolation, and/or matched information for late analysis, along with the library details. *Return type:* string
`interp_info_min`	Returns information for early analysis. When specified, the report output will show the data for interpolation, extrapolation, and/or matched information for early analysis, along with the library details. *Return type:* string
`input_signal_level`	Returns the power rail name associated with the input signal pin, only if it has been explicitly specified in the Liberty description of the cell, using the `input_signal_level` attribute. Typically, this attribute can only be defined in overdrive situations where the expected input voltage does not match the `related_power_pin` of the input. In other cases, a value of NA will be returned. *Return type:* string For example, `get_property [get_pins mmu/U1/I] input_signal_level`
`input_signal_level_voltage`	If the `input_signal_level` has an explicit value, then this attribute will return the voltage of that power rail. This is the expected incoming signal voltage for the input pin. A value of NA will be returned if `input_signal_level` is undefined. *Return type:* float For example, `get_property [get_pins mmu/U1/I] input_signal_level_voltage`
`is_always_on`	Returns a value of `true` if an instance pin is an always-on pin. *Return type:* boolean For example, `get_property [get_pins mmu/U1/AON] is_always_on`
`is_isolation_cell_data`	Returns a value of `true` if the instance signal pin is the data pin of an isolation cell instance. *Return type:* boolean For example, `get_property [get_pins mmu/U2/ISO] is_isolation_cell_data`
`is_isolation_cell_enable`	Returns a value of `true` if the instance signal pin is the enable pin of an isolation cell instance. *Return type:* boolean For example, `get_property [get_pins mmu/U2/ISO_EN] is_isolation_cell_enable`

`is_level_shifter_data`	Returns a value of `true` if the instance signal pin is the data pin of a level-shifter cell instance. *Return type:* boolean For example, `get_property [get_pins mmu/LVL/IN] is_level_shifter_data`
`is_level_shifter_enable`	Returns a value of `true` if the instance signal pin is the enable pin of a level-shifter cell instance. *Return type:* boolean For example, `get_property [get_pins mmu/LVL/EN] is_level_shifter_enable`
`is_async`	Returns a value of `true` if the pin is an asynchronous clear or preset pin. *Return type:* boolean
`is_clear`	Returns a value of `true` if the pin is an asynchronous clear pin. *Return type:* boolean
`is_clock`	Returns a value of `true` if the pin has the Liberty pin attribute: `clock`. *Return type:* boolean
`is_clock_gating`	Returns a value of `true` if the pin is defined as a pin of a clock gating cell. *Return type:* boolean
`is_clock_gating_clock`	Returns a value of `true` if the pin is defined as a clock pin of a clock gating cell. *Return type:* boolean
`is_clock_gating_enable`	Returns a value of `true` if the pin corresponds to the enable pin of a clock gating cell. *Return type:* boolean

`is_clock_root`	Returns `true` for a pin or port if real clock or generated clock is defined on the pin/port.
	`create_generated_clock -name clk -source CLK -multiply_by 2 <pin name>`
	For example,
	`get_property [get_pins <pin_name>] is_clock_root`
	Return Type: boolean

`is_clock_source_pin`	Returns a value of `true` for a valid instance pin or port object, if a clock is declared with that clock source at pin/port. *Return Type*: Boolean
`is_clock_source_latency`	Returns a value of `true` if a pin, port or any of the leaf pins of a net is part of a clock source latency network. *Return Type*: Boolean
`is_clock_used_as_clock`	Returns a value of `true` if the pin lies in the clock network and at least one of the clocks arriving at the pin is used as a clock in the downstream network of the pin. When the `-clock` parameter is specified, this property returns `true` if the above condition is satisfied for the specified clock. *Return type*: boolean
`is_clock_used_as_data`	Returns a value of `true` if the pin lies in the clock source path and at least one of the clocks arriving on the pin is used as data in the downstream network of the pin. When the `-clock` parameter is specified, this property returns `true` if the above condition is satisfied for the specified clock. *Return type*: boolean
`is_constrained`	Returns a value of `true` if the specified pin is constrained. `get_property [get_pins UM/A] is_constrained` *Return type*: boolean

is_data	Returns a value of true if the pin is a data pin of a flip-flop (that is, is not a clock pin). *Return type:* boolean
is_disable_timing	Returns a value of true if the pin's timing has been disabled. Return type: boolean **Note**: This property checks if the associated arc for the pin is active for propagation and analysis. For accurate results, re-run the update_timing command.
is_fall_edge_triggered_clock	Returns a value of true if the pin is a clock pin of a flop, and is triggered by the falling edge of a clock. *Return type:* boolean
is_fall_edge_triggered_data	Returns a value of true if the pin corresponds to the data pin of a fall edge triggered device. *Return value:* boolean
is_hierarchical	Returns a value of true if the pin is a pin of a hierarchical module. *Return type:* boolean
is_isolation_cell_data	Returns true if the instance signal pin corresponding to the isolation_cell_data_pin attribute of a library signal pin is set to true, indicating that the library cell signal pin is a data pin of an isolation library cell. **Note** : The is_isolation_cell attribute must be set to true in the library. For example, get_property [get_pins <pin_name>] is_isolation_cell_data *Return Type:* boolean
is_isolation_cell_clock	Returns true if the instance signal pin corresponding to the isolation_cell_clock_pin attribute of a library signal pin is set to true, indicating that the library cell signal pin is a data pin of an isolation library cell. **Note** : The is_isolation_cell attribute must be set to true in the library. *Return Type:* boolean
is_isolation_cell_enable	Returns true if the instance signal pin corresponding to the isolation_cell_enable_pin attribute of a library signal pin is set to true, indicating that the library cell signal pin is an enable or control pin of an isolation library cell. **Note** : The is_isolation_cell attribute must be set to true in the library. For example, get_property [get_pins <pin_name>] is_isolation_cell_enable *Return Type:* boolean
is_latency_network_pin	Returns true if the specified pin is having generated or created clock. create_generated_clock -name ck -source CK -divide_by 2 <pin name> For example, get_property [get_pins <pin name>] is_latency_network_pin *Return Type:* boolean
is_level_shifter_enable	Returns true if the instance signal pin corresponding to the level_shifter_enable_pin attribute of a library signal pin is set to true, indicating that the library cell signal pin is a enable pin of an isolation library cell. **Note:** The is_level_shifter attribute must be set to true in the library. For example, get_property [get_pins <pin_name>] is_isolation_cell_enable *Return Type:* boolean

is_level_shifter_data	Returns true if the instance signal pin corresponding to the level_shifter_data_pin attribute of a library signal pin is set to true, indicating that the library cell signal pin is a data pin of an isolation library cell. **Note:** The is_level_shifter attribute must be set to true in the library. For example, `get_property [get_pins <pin_name>] is_level_shifter_data` *Return Type:* boolean
is_multiple_clock_fanin_point	Returns a value of true if multiple clock phases converge at the pin. *Return type:* boolean
is_mux_select_pin	Returns true if the specified pin is the mux select pin. For example, `get_property [get_pins m1/S] is_mux_select_pin` *Return Type:* boolean
is_negative_level_sensitive_clock	Returns a value of true if the library pin is an enable pin of an active low level-sensitive device. *Return type:* boolean
is_negative_level_sensitive_data	Returns a value of true if the pin is a data pin of an active low level-sensitive device. *Return type:* boolean
is_pin	Returns a value of true. The is_pin and is_port queries are provided for both pin and port objects to provide a quick way of determining the object type of an identifier. is_port always returns false for pins. is_pins always returns false for ports. *Return type:* boolean
is_port	Returns a value of false. The is_pin and is_port queries are provided for both pin and port objects to provide a quick way of determining the object type of an identifier. is_port always returns false for pins. is_pins always returns false for ports. *Return type:* boolean
is_positive_level_sensitive_clock	Returns a value of true if the library pin is an enable pin of an active high level-sensitive device. *Return type:* boolean
is_positive_level_sensitive_data	Returns a value of true if the pin is a data pin of an active high level-sensitive device. *Return type:* boolean
is_power_mode_disabled	Returns a value of true if power mode of the specified pin is disabled. `get_property [get_pins MD_C/C_IS] is_power_mode_disabled -view view_func_8V_OFF` *Return type:* boolean
is_preset	Returns a value of true if the pin is a preset pin. *Return type:* boolean
is_propagated_clock	Returns a value of true if there is an explicit set_propagated_clock assertion at the pin. *Return type:* boolean

`is_ref_clock_pin`	Returns a value of `true` for: • Valid instance pin object and on an active clock pin where a clock signal reaches, and sequential cell is not disabled by disabled timing arcs or case analysis. Any pin with clock check-arcs or trigger-arcs will also be reported, if an active clock phase is present on that pin. • Clock pin on an inferred clock gating cell. Returns a value of `false`, if clock propagation disabled with the `set_disable_timing` or `set_sense -stop_propagation` command. *Return Type*: Boolean
`is_rise_edge_triggered_clock`	Returns a value of `true` if the pin is a clock pin of a flop, and is triggered by the rising edge of a clock. *Return type:* boolean
`is_rise_edge_triggered_data`	Returns a value of `true` if the pin corresponds to the data pin of a rise edge-triggered device. *Return type:* boolean
`is_tristate`	Returns a value of `true` if the pin has the `three_state` attribute in the Liberty timing library. *Return type:* boolean
`is_tristate_enable`	Returns a value of `true` if the pin is the source pin of timing arcs with either the `three_state_enable` or `three_state_disable` attribute in the Liberty timing library. *Return type:* boolean
`is_tristate_output`	Returns a value of `true` if the pin corresponds to a three-state output pin. *Return type:* boolean
`latency_fall_max`	Returns the maximum fall insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`latency_fall_min`	Returns the minimum fall insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`latency_rise_max`	Returns the maximum rise insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`latency_rise_min`	Returns the minimum rise insertion delay specified by an explicit `set_clock_latency` at the pin. *Return type:* float
`lib_pins`	Returns the collection of the library pin for the specified pin. For example, `get_property [get_pins <pin name>] lib_pins` *Return Type:* collection
`max_capacitance`	Returns the maximum capacitance limit for the pin. *Return type:* float
`max_fanout`	Returns the maximum fanout load that the pin can drive. This value is set using `set_max_fanout` or the `default_max_fanout` library attribute. *Return type:* float
`max_time_borrow`	Returns the value of the maximum time borrowed for the specified pin when the `set_max_time_borrow` parameter is specified. For example, `get_property [get_pins <pin name> ] max_time_borrow` *Return Type:* float

max_time_borrow_io	Returns the maximum time that a path arriving at a latch input can borrow from the next stage during latch-based analysis. `get_property [get_pins mro_dc/LP/d] max_time_borrow_io` *Return Type:* float
max_transition	Returns the maximum transition time limit specified for the pin. *Return type:* float
min_capacitance	Returns the value of the minimum capacitance for the specified pin when the `set_min_capacitance` parameter is specified. For example, `get_property [get_pins <pin name>] min_capacitance` *Return Type:* float
min_fanout	Returns the minimum fanout design rule limit of the corresponding library pin. *Return type:* float
min_transition	Returns the value of the minimum transition for the specified pin. For example, `get_property [get_pins <pin name>] min_transition` *Return Type:* float
name	Returns the leaf-level name of the pin. For example, `get_property [get_pins <pin name>] name` *Return Type:* string
netlist_constant_value	Returns a constant logic value of 1'b0 or 1'b1 in the Verilog. `get_property [get_pins UC/flp/pez] netlist_constant_value`
net_name	Returns the name of the net connected to the specified pin. *Return type:* string
noise_victim	Returns a collection of `si_victim` objects that can be used to query glitch attributes. *Return type:* collection
object_type	Returns the object type `pin`. *Return type:* string
output_signal_level	Returns the rail name of the `related_power_pin` attribute of the output/inout signal pin. *Return type:* string For example, `get_property [get_pins mmu/U1/Y] output_signal_level`
pin_capacitance_max	Reports the maximum capacitance of a pin. `get_property [get_pins UMC/buff/A] pin_capacitance_max` *Return type:* float
pin_capacitance_max_fall	Returns the maximum falling capacitance of a pin. *Return type:* float
pin_capacitance_max_rise	Returns the maximum rise capacitance of pin. *Return type:* float
pin_capacitance_min	Returns the minimum capacitance of pin. *Return type:* float

`pin_capacitance_min_fall`	Returns the minimum fall capacitance of Pin. *Return type:* float
`pin_capacitance_min_rise`	Returns the minimum rise capacitance of Pin. *Return type:* float
`pin_direction`	Reports the direction of a pin. `get_property [get_pins UC/buf/B] pin_direction` *Return type:* string
`power_rail_voltage_bidir_input_max`	Reports power rail voltage for input part of bidi pins/ports for maximum operating condition. *Return type:* float
`power_rail_voltage_bidir_input_min`	Reports power rail voltage for input part of bidi pins/ports for minimum operating condition. *Return type:* float
`power_rail_voltage_max`	Reports power rail voltage for maximum operating condition for bidirectional, input and output pins. For bidirectional pins, the power rail voltage for output signal is reported. *Return type:* float
`power_rail_voltage_min`	Reports power rail voltage for minimum operating condition for bidirectional, input and output pins. For bidirectional pins, the power rail voltage for output signal is reported. *Return type:* float
`receiver_ccc_type`	Returns the type of receiver cell model. The return values can be udn, single_ccc, dual_ccc, multi_ccc, or black_box. This attribute applies to receiver pins only. For example, `get_property [get_pins inst1/a] receiver_ccc_type` *Return type:* string
`ref_lib_pin_name`	Returns the name of the library pin associated with the instance pin. *Return type:* string
`related_ground_pin`	Returns the related ground pins as a collection for the corresponding instance pins. *Return type:* collection For example, `get_property [get_pins mmu/U1/I] related_ground_pin`
`related_power_pin`	Returns the related power pins as a collection for the corresponding instance pins. *Return type:* collection For example, `get_property [get_pins mmu/U1/I] related_power_pin`
`setup_uncertainty`	Returns the most conservative uncertainty of all possible uncertainty assertions associated with the pin. *Return type:* float
`si_slew_max`	Returns the maximum slew from all the incoming signals after performing SI analysis. This value is a maximum of `si_slew_max_rise` and `si_slew_max_fall`. *Return type:* float
`si_slew_max_fall`	Reports the maximum falling slew considering the incremental slew from signal integrity analysis. `get_property [get_pins UC/buf/A] si_slew_max_fall` *Return type:* float
`si_slew_max_rise`	Reports the maximum rising slew considering the incremental slew from signal integrity analysis. `get_property [get_pins UC/buf/A] si_slew_max_rise` *Return type:* float

`si_slew_min_fall`	Reports the minimum falling slew considering the incremental slew from signal integrity analysis. `get_property [get_pins UC/buf/A] si_slew_min_fall` *Return type:* float
`si_slew_min_rise`	Reports the minimum rising slew considering the incremental slew from signal integrity analysis. `get_property [get_pins UC/buf/A] si_slew_min_rise` *Return type:* float
`si_slew_mean_max_fall`	Returns the mean component of maximum falling slew from the incoming signals after SI analysis is performed. *Return type:* float
`si_slew_mean_max_rise`	Returns the mean component of maximum rising slew from the incoming signals after SI analysis is performed. *Return type:* float
`si_slew_mean_min_fall`	Returns the mean component of minimum falling slew from the incoming signals after SI analysis is performed. *Return type:* float
`si_slew_mean_min_rise`	Returns the mean component of minimum rising slew from the incoming signals after SI analysis is performed. *Return type:* float
`si_slew_min`	Returns the minimum slew from all the incoming signals after performing SI analysis. This value is a minimum of `si_slew_min_rise` and `si_slew_min_fall`. *Return type:* float
`si_victim`	Returns a collection of the `si_victim` objects that can be used to query graph-based SI delay attributes. *Return type:* collection
`slack_max`	Returns the slack maximum property for a timing path. *Return type:* float
`slack_max_edge`	Returns the edge (rise or fall) of the worst slack-causing path at the specified pin in late mode. *Return type:* string
`slack_max_fall`	Returns the slack time for the falling transition at the max corner. *Return type:* float
`slack_max_rise`	Returns the slack time for the rising transition at the max corner. When the `-view` parameter is not specified, the worst slack for all the views is returned. *Return type:* float
`slack_mean_max`	Returns the maximum mean component of the slack time for a timing path. *Return type:* float
`slack_mean_min`	Returns the minimum mean component of the slack time for a timing path. *Return type:* float
`slack_mean_max_fall`	Returns the mean component of the slack time for the falling transition at the max corner. *Return type:* float
`slack_mean_max_rise`	Returns the mean component of the slack time for the rising transition at the max corner. *Return type:* float
`slack_mean_min_fall`	Returns the mean component of the slack time for the falling transition at the min corner. *Return type:* float
`slack_mean_min_rise`	Returns the mean component of the slack time for the rising transition at the min corner. Return type: float
`slack_min`	Returns the slack time for a timing path at the min corner. *Return type:* float

`slack_min_edge`	Returns the edge (rise or fall) of the worst slack-causing path at the specified pin in early mode. *Return type:* string
`slack_min_fall`	Returns the slack time for the falling transition at the min corner. *Return type:* float
`slack_min_rise`	Returns the slack time for the rising transition at the min corner. *Return type:* float
`slack_sigma_max`	Returns the variation component of the worst slack across all the concurrent MMMC views for Setup-style late data path checks. `get_property [get_pins dff1/Q] slack_sigma_max` *Return type:* float
`slack_sigma_min`	Returns the variation component of the worst slack across all the concurrent MMMC views for Hold-style early data path checks. `get_property [get_pins dff1/Q] slack_sigma_min` *Return type:* float
`slack_skewness_max_fall`	Returns the skewness component of the worst falling slack across all the concurrent MMMC views for Setup-style late data path checks. `get_property [get_pins df1/Q] slack_skewness_max_fall` *Return type:* float
`slack_skewness_max_rise`	Returns the skewness component of the worst rising slack across all the concurrent MMMC views for Setup-style late data path checks. `get_property [get_pins df1/Q] slack_skewness_max_rise` *Return type:* float
`slack_skewness_min_fall`	Returns the skewness component of the worst falling slack across all the concurrent MMMC views for Hold-style early data path checks. `get_property [get_pins df1/Q] slack_skewness_min_fall` *Return type:* float
`slack_skewness_min_rise`	Returns the skewness component of the worst rising slack across all the concurrent MMMC views for Hold-style early data path checks. `get_property [get_pins df1/Q] slack_skewness_min_rise` *Return type:* float
`slack_sigma_max_fall`	Returns the sigma component of the slack time for the falling transition at the max corner. *Return type:* float
`slack_sigma_max_rise`	Returns the sigma component of the slack time for the rising transition at the max corner. *Return type:* float
`slack_sigma_min_fall`	Returns the sigma component of the slack time for the falling transition at the min corner. *Return type:* float
`slack_sigma_min_rise`	Returns the sigma component of the slack time for the rising transition at the min corner. *Return type:* float
`slew_max`	Returns the value of the maximum slew for the specified pin. You can verify through `report_timing`. For example, `get_property [get_pins <pin name>] slew_max` *Return Type:* float
`slew_max_fall`	Returns the slew time for the falling transition at the max corner. *Return type:* float

`slew_max_rise`	Returns the slew time for the rising transition at the max corner. *Return type:* float
`slew_mean_max_fall`	Returns the mean component of the slew time for the falling transition at the max corner. *Return type:* float
`slew_mean_max_rise`	Returns the mean component of the slew time for the rising transition at the max corner. *Return type:* float
`slew_mean_min_fall`	Returns the mean component of the slew time for the falling transition at the min corner. *Return type:* float
`slew_mean_min_rise`	Returns the mean component of the slew time for the rising transition at the min corner. *Return type:* float
`slew_min`	Returns the value of the minimum slew for the specified pin. You can verify through `report_timing`. For example, `get_property [get_pins <pin name>] slew_min` *Return Type:* float
`slew_min_fall`	Returns the slew time for the falling transition at the min corner. *Return type:* float
`slew_min_rise`	Returns the slew time for the rising transition at the min corner. *Return type:* float
`slew_sigma_max_fall`	Returns the sigma component of the slew time for the falling transition at the max corner. *Return type:* float
`slew_sigma_max_rise`	Returns the sigma component of the slew time for the rising transition at the max corner. *Return type:* float
`slew_sigma_min_fall`	Returns the sigma component of the slew time for the falling transition at the min corner. *Return type:* float
`slew_sigma_min_rise`	Returns the sigma component of the slew time for the falling transition at the min corner. *Return type:* float
`slew_skewness_max_fall`	Returns the skewness component of the maximum slew time for falling transitions across all the concurrent MMMC views. `get_property [get_pins UMC/buff/A] slew_skewness_max_fall` *Return type:* float
`slew_skewness_max_rise`	Returns the skewness component of the maximum slew time for rising transitions across all the concurrent MMMC views. `get_property [get_pins UMC/buff/A] slew_skewness_max_rise` *Return type:* float
`slew_skewness_min_fall`	Returns the variation component of the minimum slew time for falling transitions across all the concurrent MMMC views. `get_property [get_pins UMC/buff/A] slew_skewness_min_fall` *Return type:* float
`slew_skewness_min_rise`	Returns the variation component of the minimum slew time for rising transitions across all the concurrent MMMC views. `get_property [get_pins UMC/buff/A] slew_skewness_min_rise` *Return type:* float

source_latency_early_fall_max	Returns the maximum early fall source insertion delay specified by an explicit set_clock_latency at the pin. *Return type:* float
source_latency_early_fall_min	Returns the minimum early fall source insertion delay specified by an explicit set_clock_latency at the pin. *Return type:* float
source_latency_early_rise_max	Returns the maximum early rise source insertion delay specified by an explicit set_clock_latency at the pin. *Return type:* float
source_latency_early_rise_min	Returns the minimum early rise source insertion delay specified by an explicit set_clock_latency at the pin. *Return type:* float
source_latency_late_fall_max	Returns the maximum late fall source insertion delay specified by an explicit set_clock_latency at the pin. *Return type:* float
source_latency_late_fall_min	Returns the minimum late fall source insertion delay specified by an explicit set_clock_latency at the pin. *Return type:* float
source_latency_late_rise_max	Returns the maximum late rise source insertion delay specified by an explicit set_clock_latency at the pin. *Return type:* float
source_latency_late_rise_min	Returns the minimum late rise source insertion delay specified by an explicit set_clock_latency at the pin. *Return type:* float
stolen_slack	Returns the slack stolen at the specified pin. This property can be queried only for pins in late mode. **Note:** The specified pin must be the endpoint of a transparent latch flush arc. *Return type:* float
timing_constant	Reports the constant value specified on a pin. `get_property [get_pins UC/retent_cell/SZ] timing_constant` *Return type:* string
timing_constant_source	Reports the source from which the timing constant was applied. `get_property [get_pins UC/retent_cell/SZ] timing_constant_source` *Return type:* string
user_constant_value	Returns the constant values from constraints for the specified pin. `get_property [get_pins UC/retent_cell/SZ] user_constant_value` *Return type*: integer
waveform_max_fall	Returns the fall waveform for the specified input pins during late analysis. *Return type:* string
waveform_max_rise	Returns the rise waveform for the specified input pins during late analysis. *Return type:* string
waveform_min_fall	Returns the fall waveform for the specified input pins during early analysis. *Return type:* string
waveform_min_rise	Returns the rise waveform for the specified input pins during early analysis. *Return type:* string
x_coordinate	Returns the x coordinates of the placed pin. *Return type:* float
y_coordinate	Returns the y coordinates of the placed pin. *Return type:* float

PG Pin Properties

`hierarchical_name`	Returns the complete hierarchical name of the PG pin.
	Return type: string
	
	For example,
	`get_property [get_pg_pins -of_objects [get_cells iA/out_reg] ] hierarchical_name`
	`get_property [get_pg_pins -of_objects [get_cells iA/out_reg] -filter "pg_type == primary_power" ] hierarchical_name`

`net`	Returns the immediate PG net(s) connected to the corresponding PG pin objects, as a collection. *Return type*: Collection For example, `get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] net`
`object_type`	Returns `pg_pin` as the object type for a PG pin. *Return type*: string

`pg_type`	Returns the type of PG pin.
	Return type: string
	
	For example,
	`get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] pg_type`
	`get_property [get_pg_pins -of_objects [get_cells iA/out_reg] -filter "hierarchical_name =~ *VDD*" ] pg_type`

`ref_lib_pin_name`	Returns the name of the library PG pin associated with an instance PG pin. *Return type*: string For example, `get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] ref_lib_pin_name`
`voltage_max`	Returns the maximum voltage of a given PG pin. This could be the voltage associated with a pin, power intent or CPF, `related_pg_pins` attribute in Liberty, or the operating voltage of a power domain or system default voltage. *Return type:* float For example, `get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] voltage_max`
`voltage_min`	Returns the minimum voltage of a given PG pin. This voltage may be associated with a pin, power intent or CPF, `related_pg_pins` attribute in Liberty, or the operating voltage of a power domain or system default voltage. *Return type*: float For example, `get_property [get_pg_pins -of_objects [get_cells iA/out_reg]] voltage_min`

Port Properties

`actual_latency_early_fall_max`	Returns the maximum fall latency on the specified port caused due to source and network latency asserted using the `set_clock_latency` command for early analysis. `get_property [get_ports CLK] actual_latency_early_fall_max` *Return type:* float
`actual_latency_early_fall_min`	Returns the maximum fall latency on the specified port caused due to source and network latency asserted using the `set_clock_latency` command for early analysis. `get_property [get_ports CLK] actual_latency_early_fall_min` *Return type:* float
`actual_latency_early_rise_max`	Returns the maximum rise latency on the specified port caused due to source and network latency asserted using the `set_clock_latency` command for early analysis. `get_property [get_ports CLK] actual_latency_early_rise_max` *Return type:* float

actual_latency_early_rise_min	Returns the minimum rise latency on the specified port caused due to source and network latency asserted using the set_clock_latency command for early analysis. get_property [get_ports CLK] actual_latency_early_rise_min *Return type:* float
actual_latency_late_fall_max	Returns the maximum fall latency on the specified port caused due to source and network latency asserted using the set_clock_latency command for late analysis. get_property [get_ports CLK] actual_latency_late_fall_max *Return type:* float
actual_latency_late_fall_min	Returns the maximum fall latency on the specified port caused due to source and network latency asserted using the set_clock_latency command for late analysis. get_property [get_ports CLK] actual_latency_late_fall_min *Return type:* float
actual_latency_late_rise_max	Returns the maximum rise latency on the specified port caused due to source and network latency asserted using the set_clock_latency command for late analysis. get_property [get_ports CLK] actual_latency_late_rise_max *Return type:* float
actual_latency_late_rise_min	Returns the minimum rise latency on the specified port caused due to source and network latency asserted using the set_clock_latency command for late analysis. get_property [get_ports CLK] actual_latency_late_rise_min *Return type:* float
aocv_early_input_stage_weight	Returns the input stage weight of the specified port for AOCV mode early analysis. get_property [get_ports int] aocv_early_input_stage_weight *Return type:* float
aocv_early_output_stage_weight	Returns the output stage weight of the specified port for AOCV mode late analysis. get_property [get_ports vcop] aocv_early_output_stage_weight *Return type:* float
aocv_late_input_stage_weight	Returns the input stage weight of the specified for AOCV mode late analysis. get_property [get_ports int] aocv_late_input_stage_weight *Return type:* float
aocv_late_output_stage_weight	Returns the output stage weight of the specified port for AOCV mode late analysis. get_property [get_ports vcop] aocv_late_output_stage_weight *Return type:* float
arrival_clocks	Reports clocks that correspond to the arrival times at the port. get_property [get_ports out] arrival_clocks *Return type:* float
arrival_max	Reports the maximum arrival time at a port. get_property [get_ports out] arrival_max *Return type:* float
arrival_max_fall	Returns the worst arrival on the port for a falling signal. *Return type:* float
arrival_max_rise	Returns the worst arrival on the port for a rising signal. *Return type:* float

`arrival_mean_max_fall`	In SOCV analysis mode, this returns the mean value of the latest falling arrival time to the specified port across all concurrent MMMC views. `get_property [get_ports out ] arrival_mean_max_fall` *Return type:* float
`arrival_mean_max_rise`	In SOCV analysis mode, this returns the mean value of the latest rising arrival time to the specified port across all concurrent MMMC views. `get_property [get_ports out ] arrival_mean_max_rise` *Return type:* float
`arrival_mean_min_fall`	In SOCV analysis mode, this returns the mean value of the earliest falling arrival time to the specified port across all concurrent MMMC views. `get_property [get_ports out ] arrival_mean_min_fall` *Return type:* float
`arrival_mean_min_rise`	In SOCV analysis mode, this returns the mean value of the earliest rising arrival time to the specified port across all concurrent MMMC views. `get_property [get_ports out ] arrival_mean_min_rise` *Return type:* float
`arrival_min`	Reports minimum arrival time at the port. `get_property [get_ports out ] arrival_min` *Return type:* float
`arrival_min_fall`	Returns the best arrival on the port for a falling signal. *Return type:* float
`arrival_min_rise`	Returns the best arrival on the port for a rising signal. *Return type:* float
`arrival_sigma_max_fall`	In SOCV analysis mode, this returns the variation component of the latest falling arrival time to the specified port across all the concurrent MMMC views. `get_property [get_ports out ] arrival_sigma_max_fall` *Return type:* float
`arrival_sigma_max_rise`	In SOCV analysis mode, this returns the variation component of the latest rising arrival time to the specified port across all the concurrent MMMC views. `get_property [get_ports out ] arrival_sigma_max_rise` *Return type:* float
`arrival_sigma_min_fall`	In SOCV analysis mode, this returns the variation component of the earliest falling arrival time to the specified port across all the concurrent MMMC views. `get_property [get_ports out ]  arrival_sigma_min_fall` *Return type:* float
`arrival_sigma_min_rise`	In SOCV analysis mode, this returns the variation component of the earliest rising arrival time to the specified port across all the concurrent MMMC views. `get_property [get_ports out ] arrival_sigma_min_rise` *Return type:* float
`attacker_fall_slew`	Reports fall slew of attackers seen for nets connected to the port. `get_property [get_ports CLK] attacker_fall_slew` *Return type:* float
`attacker_rise_slew`	Reports the rise slew of attackers seen for nets connected to the port. `get_property [get_ports CLK] attacker_rise_slew` *Return type:* float

`arrival_window`	Returns the arrival times for the earliest and latest rising and falling transitions in timing window format. *Return type:* string
`capture_clocks`	Reports capture clocks at the port. *Return type:* collection
`clock_sources`	Returns a collection of the source pins of all the clocks arriving at the port. *Return type:* collection
`clocks`	Returns a collection of all of the clocks arriving at the port. *Return type:* collection
`constant_value`	Returns a constant value of 0 or 1, if a logic state has been asserted or propagated to the port. `get_property [get_ports D] constant_value` *Return type:* integer
`constrained_capture_clocks`	Returns the capture clocks, which are constrained, at the port. `get_property [get_ports C] constrained_capture_clocks` *Return type:* collection
`capacitance_max`	Returns the maximum pin capacitance for the port. *Return type:* float
`capacitance_min`	Returns the minimum pin capacitance for the port. *Return type:* float
`delay_max_fall`	Returns the maximum falling delay value for the port. *Return type:* float
`delay_max_rise`	Returns the maximum rising delay value for the port. *Return type:* float
`delay_min_fall`	Returns the minimum falling delay value for the port. *Return type:* float
`delay_min_rise`	Returns the minimum rising delay value for the port. *Return type:* float
`direction`	Returns the direction of the port: `in`, `out`, or `inout`. *Return type:* string
`driving_cell_no_design_rule`	Returns a value of `true` if there is no design rule transfer from the driving cell set on a port. For example, if a driving cell is defined with the following command, then this property will return `true`: `set_driving_cell -lib_cell <lib_cell_name> -no_design_rule <port>`
`drive_resistance_fall_max`	Returns the falling linear drive resistance at the port for the max condition. *Return type:* float
`drive_resistance_fall_min`	Returns the falling linear drive resistance at the port for the min condition. *Return type:* float
`drive_resistance_rise_max`	Returns the rising linear drive resistance at the port for the max condition. *Return type:* float
`drive_resistance_rise_min`	Returns the rising linear drive resistance at the port for the min condition. *Return type:* float
`driving_cell_fall_max`	Returns the name of the library cell used to compute max falling delays and transitions at the port. *Return type:* string

`driving_cell_fall_min`	Returns the name of the library cell used to compute min falling delays and transitions at the port. *Return type:* string
`driving_cell_from_pin_fall_max`	Returns the input pin of the driving cell used in the max fall delay calculation at the port. *Return type:* string
`driving_cell_from_pin_fall_min`	Returns the input pin of the driving cell used in the min fall delay calculation at the port. *Return type:* string
`driving_cell_from_pin_rise_max`	Returns the input pin of the driving cell used in max rise delay calculation at the port. *Return type:* string
`driving_cell_from_pin_rise_min`	Returns the input pin of the driving cell used in min rise delay calculation at the port. *Return type:* string
`driving_cell_library_fall_max`	Returns the name of the library used to compute max falling delays and transitions at the port. *Return type:* string
`driving_cell_library_fall_min`	Returns the name of the library used to compute min falling delays and transitions at the port. *Return type:* string
`driving_cell_library_rise_max`	Returns the name of the library used to compute max rising delays and transitions at the port. *Return type:* string
`driving_cell_library_rise_min`	Returns the name of the library used to compute min rising delays and transitions at the port. *Return type:* string
`driving_cell_max_fall_itrans_fall`	Returns the falling input transition of the `from_pin` of the driving cell, which is used for the max falling delay calculation at the port. *Return type:* float
`driving_cell_max_fall_itrans_rise`	Returns the falling input transition of the `from_pin` of the driving cell, which is used for the max rising delay calculation at the port. *Return type:* float
`driving_cell_max_rise_itrans_fall`	Returns the rising input transition of the `from_pin` of the driving cell, which is used for the max falling delay calculation at the port. *Return type:* float
`driving_cell_max_rise_itrans_rise`	Returns the rising input transition of the `from_pin` of the driving cell, which is used for the max rising delay calculation at the port. *Return type:* float
`driving_cell_min_fall_itrans_fall`	Returns the falling input transition of the `from_pin` of the driving cell, which is used for the min falling delay calculation at the port. *Return type:* float
`driving_cell_min_fall_itrans_rise`	Returns the falling input transition of the `from_pin` of the driving cell, which is used for the min rising delay calculation at the port. *Return type:* float
`driving_cell_min_rise_itrans_fall`	Returns the rising input transition of the `from_pin` of the driving cell, which is used for the min falling delay calculation at the port. *Return type:* float
`driving_cell_min_rise_itrans_rise`	Returns the rising input transition of the `from_pin` of the driving cell, which is used for the min rising delay calculation at the port. *Return type:* float

`driving_cell_pin_fall_max`	Returns the name of the output pin used for max fall delay calculation at the port. *Return type:* string
`driving_cell_pin_fall_min`	Returns the name of the output pin used for min fall delay calculation at the port. *Return type:* string
`driving_cell_pin_rise_max`	Returns the name of the output pin used for max rise delay calculation at the port. *Return type:* string
`driving_cell_pin_rise_min`	Returns the name of the output pin used for min rise delay calculation at the port. *Return type:* string
`driving_cell_rise_max`	Returns the name of the library cell used to compute max rising delays and transitions at the port. *Return type:* string
`driving_cell_rise_min`	Returns the name of the library cell used to compute min rising delays and transitions at the port. *Return type:* string
`fanin`	Returns the number of sources connected with net of the specified port. *Return type:* integer
`fanout`	Returns the number of sinks connected with the net of the specified port. *Return type:* integer
`fanout_load`	Returns the fanout load for ports. (For input ports, the value is the sum of all fanout loads of all connected pins. For output ports, the value is set using `set_fanout_load`.) *Return type:* float
`fastest_clock`	Reports the clock with maximum frequency associated with the port. `get_property [get_ports CLK] fastest_clock` *Return type:* collection
`fastest_clock_name`	Reports the name of the clock with maximum frequency associated with the port. `get_property [get_ports CLK] fastest_clock_name` *Return type:* string
`fastest_clock_period`	Reports the period of the clock with maximum frequency associated with the port. `get_property [get_ports CLK] fastest_clock_period` *Return type:* float
`hierarchical_level`	Returns the hierarchical level for an instance port. `get_property [get_ports C] hierarchical_level` *Return type:* integer
`hierarchical_name`	Returns the complete hierarchical name of the port. *Return type:* string
`hold_uncertainty`	Returns the most conservative uncertainty of all possible uncertainty assertions associated with the port. *Return type:* float
`input_slew_max_fall`	Returns the max falling slew for the input or inout ports. (This value is set with the `set_input_transition` command.) *Return type:* float
`input_slew_max_rise`	Returns the max rising slew for the input or inout ports. (This value is set with the `set_input_transition` command.) *Return type:* float
`input_slew_min_fall`	Returns the min falling slew for the input or inout ports. (This value is set with the `set_input_transition` command.) *Return type:* float

`input_slew_min_rise`	Returns the min rising slew for the input or inout ports. (This value is set with the `set_input_transition` command.) *Return type:* float
`is_clock_root`	Returns a value of `true` if a real or generated clock is defined on a port. `get_property [get_ports CLK] is_clock_root` *Return type:* boolean
`is_clock_source_latency`	Returns a value of `true` if the object is a part of the latency network. `get_property [get_ports C] is_clock_source_latency` *Return type:*
`is_clock_source_pin`	Returns a value of `true` if the object is a clock source pin. `get_property [get_ports C] is_clock_source_pin` *Return type:* boolean
`is_clock_used_as_clock`	Returns a value of `true` if the port lies in the clock network and at least one of the clocks arriving at the pin is used as a clock in the downstream network of the port. `get_property [get_ports C] is_clock_used_as_clock` *Return type:* boolean
`is_clock_used_as_data`	Returns a value of `true` if the port lies in the clock source path and at least one of the clocks arriving on the port is used as data in the downstream network of port. `get_property [get_ports C] is_clock_used_as_data` *Return type:* boolean
`is_clock_network_port`	Returns a value of `true` if a port is part of the clock network. Return type: boolean
`is_clock_source_latency`	Returns a value of `true` if a pin, port or any of the leaf pins of a net is part of a clock source latency network. *Return Type*: boolean
`is_clock_source_pin`	Returns a value of `true` for a valid instance pin or port object, if a clock is declared with that clock source at a pin/port. *Return Type*: boolean
`is_constrained`	Returns a value of `true` if the specified port is constrained. `get_property [get_ports Ip] is_constrained` *Return type:* boolean
`is_disable_timing`	Returns a value of `true` if the timing of the port has been disabled using the `set_disable_timing` command. *Return type:* boolean
`is_hierarchical`	Returns a value of `true` if the port is a port of a hierarchical module. *Return type:* boolean
`is_latency_network_pin`	Returns a value of `true` if the specified port has a generated clock. `get_property [get_ports CLK] is_latency_network_pin` *Return type:* boolean
`is_pin`	Returns a value of `false`. The `is_pin` and `is_port` queries are provided for both pin and port objects to provide a quick way of determining the object type of an identifier. `is_port` always returns `false` for pins. `is_pins` always returns `false` for ports. *Return type:* boolean

is_port	Returns a value of true. The is_pin and is_port queries are provided for both pin and port objects to provide a quick way of determining the object type of an identifier. is_port always returns false for pins. is_pins always returns false for ports. *Return type:* boolean
is_power_mode_disabled	Returns a value of true if power mode of the specified port is disabled. `get_property [get_ports B_OFF] is_power_mode_disabled -view view_func_18V_OFF` *Return type:* boolean
is_propagated_clock	Returns a value of true if there is an explicit set_propagated_clock assertion at the port. *Return type:* boolean
latency_fall_max	Returns the maximum fall insertion delay specified by an explicit set_clock_latency at the port. *Return type:* float
latency_fall_min	Returns the minimum fall insertion delay specified by an explicit set_clock_latency at the port. *Return type:* float
latency_rise_max	Returns the maximum rise insertion delay specified by an explicit set_clock_latency at the port. *Return type:* float
latency_rise_min	Returns the minimum rise insertion delay specified by an explicit set_clock_latency at the port. *Return type:* float
max_capacitance	Returns the maxium capacitance limit asserted on the specified port. `get_property [get_ports Zf] max_capacitance` *Return type:* float
max_fanout	Returns the maximum fanout load that the port can drive. (This value is set using set_max_fanout, or the default_max_fanout library attribute.) *Return type:* float
max_transition	Returns the maximum transition time limit specified for the port. *Return type:* float
netlist_constant_value	Returns a value if the port has a constant logic value of 1'b0 or 1'b1 in the Verilog. `get_property [get_ports constant] netlist_constant_value` *Return type:* string
net_name	Returns the name of the net connected to the specified port. *Return type:* string
noise_victim	Returns a collection of si_victim objects that can be used to query glitch attributes. *Return type:* collection
object_type	Returns the object type port. *Return type:* string
power_rail_voltage_max	Reports power rail voltage for maximum operating condition for bidirectional, input and output pins. `get_property [get_ports in] power_rail_voltage_max -view Setup_m3C` *Return type:* float
power_rail_voltage_min	Reports power rail voltage for minimum operating condition for bidirectional, input and output pins. `get_property [get_ports in] power_rail_voltage_min -view Hold_m3C` *Return type:* float

`setup_uncertainty`	Returns the most conservative uncertainty of all possible uncertainty assertions associated with the port. *Return type:* float
`si_victim`	Returns a collection of the `si_victim` objects that can be used to query graph-based SI delay attributes. *Return type:* collection
`slack_max`	Returns the worst slack across all the concurrent MMMC views for Setup-style late data path checks. For example, `get_property [get_ports clk_en] slack_max` *Return type:* float
`slack_max_edge`	Returns the edge (rise or fall) of the worst slack-causing path at the specified port in late mode. *Return type:* string
`slack_max_fall`	Returns the worst setup slack for a falling signal at the port endpoint. *Return type:* float
`slack_max_rise`	Returns the worst setup slack for a rising signal at the port endpoint. When the `-view` parameter is not specified, the worst slack for all the views is returned. *Return type:* float
`slack_mean_max`	In SOCV analysis, it will return the mean component of the worst slack across all concurrent MMMC views for setup-style late data path checks. For example, `get_property [get_ports clk_en] slack_mean_max` *Return type:* float
`slack_mean_max_fall`	Returns the mean component of the worst setup slack for a falling signal at the port endpoint. For example, `get_property [get_ports scan_clk] slack_mean_max_fall` *Return type:* float
`slack_mean_max_rise`	Returns the mean component of the worst setup slack for a rising signal at the port endpoint. For example, `get_property [get_ports scan_clk] slack_mean_max_rise` *Return type:* float
`slack_mean_min`	Returns the mean component of the worst hold slack at the port endpoint. For example, `get_property [get_ports clk_en] slack_mean_min` *Return type:* float
`slack_mean_min_fall`	Returns the mean component of the worst hold slack for a falling signal at the port endpoint. For example, `get_property [get_ports clk_en] slack_mean_min_fall` *Return type:* float
`slack_mean_min_rise`	Returns the mean component of the worst hold slack for a rising signal at the port endpoint. For example, `get_property [get_ports clk_en] slack_mean_min_rise` *Return type:* float

slack_min	Returns the worst hold slack at the port endpoint. For example, `get_property [get_ports clk_en] slack_min` *Return type:* float
slack_min_edge	Returns the edge (rise or fall) of the worst slack-causing path at the specified port in early mode. *Return type:* string
slack_min_fall	Returns the worst hold slack for a falling signal at the port endpoint. *Return type:* float
slack_min_rise	Returns the worst hold slack for a rising signal at the port endpoint. *Return type:* float
slew_max	Returns the maximum slew from all of the incoming signals on the port. For example, `get_property [get_ports ahbsi_out[402]] slew_max` *Return type:* float
slew_max_fall	Returns the maximum falling slew from all of the incoming signals on the port. *Return type:* float
slew_max_rise	Returns the maximum rising slew from all of the incoming signals on the port. *Return type:* float
slew_mean_max_fall	Returns the mean component of the maximum falling slew from all of the incoming signals on the port. For example, `get_property [get_ports ahbsi_out[402]] slew_mean_max_fall` *Return type:* float
slew_mean_max_rise	Returns the mean component of the maximum rising slew from all of the incoming signals on the port. For example, `get_property [get_ports ahbsi_out[407]] slew_mean_max_rise` *Return type:* float
slew_mean_min_fall	Returns the mean component of the minimum falling slew from all of the incoming signals on the port. For example, `get_property [get_ports BG_scan_out_3] slew_mean_min_fall` *Return type:* float
slew_mean_min_rise	Returns the mean component of the minimum rising slew from all of the incoming signals on the port. For example, `get_property [get_ports BG_scan_out_3] slew_mean_min_rise` *Return type:* float
slew_min	Returns the minimum slew from all of the incoming signals on the port. For example, `get_property [get_ports BG_scan_out_3] slew_min` *Return type:* float
slew_min_fall	Returns the minimum falling slew from all of the incoming signals on the port. *Return type:* float
slew_min_rise	Returns the minimum rising slew from all of the incoming signals on the port. *Return type:* float

`source_latency_early_fall_max`	Returns the maximum early fall source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_early_fall_min`	Returns the minimum early fall source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_early_rise_max`	Returns the maximum early rise source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_early_rise_min`	Returns the minimum early rise source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_late_fall_max`	Returns the maximum late fall source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_late_fall_min`	Returns the minimum late fall source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_late_rise_max`	Returns the maximum late rise source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`source_latency_late_rise_min`	Returns the minimum late fall source insertion delay specified by an explicit `set_clock_latency` at the port. *Return type:* float
`timing_constant`	Returns the constant value on a port. `get_property [get_ports D] timing_constant` *Return type:* string
`timing_constant_source`	Returns the source by which the constant value is applied on a port. `get_property [get_ports constant] timing_constant_source` *Return type:* string
`total_capacitance_fall`	Returns the total capacitance of ports for falling transition. `get_property [get_ports out ] total_capacitance_fall` *Return type:* float
`total_capacitance_rise`	Returns the total capacitance of ports for rising transition. `get_property [get_ports out ] total_capacitance` *Return type:* float
`transition_max`	Returns the maximum transition limit asserted on the specified port. `get_property [get_ports Ip] transition_max` *Return type:* float
`user_constant_value`	Returns the constant values from constraints for the specified ports. `get_property [get_ports D] user_constant_value` *Return type*: integer
`wire_capacitance_max`	Returns the wire capacitance of the port for the maximum value of the capacitance range. For example, `get_property [get_ports port_pad_data_out[9]] wire_capacitance_max` *Return type:* float

`wire_capacitance_max_fall`	Returns the wire capacitance of a port for the maximum value of the falling capacitance range For example, `get_property [get_ports port_pad_data_out[9]] wire_capacitance_max_fall` *Return type:* float
`wire_capacitance_max_rise`	Returns the wire capacitance of a port for the maximum value of the rising capacitance range. For example, `get_property [get_ports port_pad_data_out[0]] wire_capacitance_max_rise` *Return type:* float
`wire_capacitance_min`	Returns the wire capacitance of a port for the minimum value of the capacitance range. For example, `get_property [get_ports tdigit[2]] wire_capacitance_min` *Return type:* float
`wire_capacitance_min_fall`	Returns the wire capacitance of a port for the minimum value of the falling capacitance range. For example, `get_property [get_ports tdigit[2]] wire_capacitance_min_fall` *Return type:* float
`wire_capacitance_min_rise`	Returns the wire capacitance of a port for the minimum value of the rising capacitance range. For example, `get_property [get_ports tdigit[2]] wire_capacitance_min_rise` *Return type:* float
`wire_lumped_capacitance_max`	Returns the wire lumped capacitance of the net associated to port for maximum conditions. Returns the portion of a port's total lumped capacitance, which may be extracted or annotated wire lumped capacitance values. For example, `get_property [get_ports tdigit[7]] wire_lumped_capacitance_max` *Return type:* float
`wire_lumped_capacitance_max_fall`	Returns the lumped wire capacitance of a port for the maximum value of the falling capacitance range. For example, `get_property [get_ports port_pad_data_out[9]] wire_lumped_capacitance_max_fall` *Return type:* float
`wire_lumped_capacitance_max_rise`	Returns the lumped wire capacitance of a port for the maximum value of the rising capacitance range. For example, `get_property [get_ports port_pad_data_out[0]] wire_lumped_capacitance_max_rise` *Return type:* float
`wire_lumped_capacitance_min`	Returns the portion of a port's total lumped capacitance, which may be extracted or annotated wire lumped capacitance values. For example, `get_property [get_ports tdigit[3]] wire_lumped_capacitance_min` *Return type:* float
`wire_lumped_capacitance_min_fall`	Returns the lumped wire capacitance of a port for the minimum value of the falling capacitance range. For example, `get_property [get_ports tdigit[3]] wire_lumped_capacitance_min_fall` *Return type:* float

`wire_lumped_capacitance_min_rise`	Returns the lumped wire capacitance of a port for the minimum value of the rising capacitance range. For example, `get_property [get_ports tdigit[3]] wire_lumped_capacitance_min_rise` *Return type:* float
`x_coordinate`	Returns the x coordinates of the placed port. *Return type:* float
`y_coordinate`	Returns the y coordinates of the placed port. *Return type:* float

Timing Arc Properties

`aocv_derate_capture_clock_early_fall`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of early capture clock paths with fall sink pin transitions. *Return type:* float
`aocv_derate_capture_clock_early_rise`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of early capture clock paths with rise sink pin transitions. *Return type:* float
`aocv_derate_capture_clock_late_fall`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of late capture clock paths with fall sink pin transitions. *Return type:* float
`aocv_derate_capture_clock_late_rise`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of late capture clock paths with rise sink pin transitions. *Return type:* float
`aocv_derate_data_early_fall`	Returns AOCV derate values for a timing arc on an early data path with fall transitions. *Return type:* float
`aocv_derate_data_early_rise`	Returns AOCV derate values for a timing arc on an early data path with rise transitions. *Return type:* float
`aocv_derate_data_late_fall`	Returns AOCV derate values for a timing arc on a late data path with fall transitions. *Return type:* float
`aocv_derate_data_late_rise`	Returns AOCV derate values for a timing arc on a late data path with rise transitions. *Return type:* float
`aocv_derate_launch_clock_early_fall`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of early launch clock paths with fall sink pin transitions. *Return type:* float
`aocv_derate_launch_clock_early_rise`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of early launch clock paths with rise sink pin transitions. *Return type:* float
`aocv_derate_launch_clock_late_fall`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of late launch clock paths with fall sink pin transitions. *Return type:* float
`aocv_derate_launch_clock_late_rise`	Returns graph based AOCV derate factors in AOCV mode for a given timing arc that is a part of late launch clock paths with rise sink pin transitions. *Return type:* float

`aocv_stage_count_capture_clock_early`	Returns graph based AOCV stage count values in AOCV mode for a given timing arc that is a part of early capture clock paths. *Return type:* float
`aocv_stage_count_capture_clock_late`	Returns graph based AOCV stage count values in AOCV mode for a given timing arc that is a part of late capture clock paths. *Return type:* float
`aocv_stage_count_data_early`	Returns AOCV stage count values for a timing arc on an early data path. *Return type:* float
`aocv_stage_count_data_late`	Returns AOCV stage count values for a timing arc on a late data path. *Return type:* float
`aocv_stage_count_launch_clock_early`	Returns AOCV stage count values in AOCV mode for a given timing arc that is a part of early launch clock paths. *Return type:* float
`aocv_stage_count_launch_clock_late`	Returns AOCV stage count values in AOCV mode for a given timing arc that is a part of late launch clock paths. *Return type:* float
`arc_type`	Returns the Liberty `timing_type` attribute of the arc. *Return type:* string
`delay_max`	Returns the delay for the maximum corner. For example, `get_property [get_arcs -of_objects [get_cells regbuf]] delay_max` *Return type:* float
`delay_max_fall`	Returns the fall delay for the max corner. *Return type:* float
`delay_max_rise`	Returns the rise delay for the max corner. *Return type:* float
`delay_mean_max_fall`	Returns the mean component of the maximum falling delay value for the timing arc. *Return type*: float
`delay_mean_max_rise`	Returns the mean component of the maximum rising delay value for the timing arc. *Return type*: float
`delay_mean_min_fall`	Returns the mean component of the minimum falling delay value for the timing arc. *Return type*: float
`delay_mean_min_rise`	Returns the mean component of the minimum rising delay value for the timing arc. *Return type*: float
`delay_min`	Returns the delay for the minimum corner. For example, `get_property [get_arcs -of_objects [get_cells clkbuf]] delay_min` *Return type*: float
`delay_min_fall`	Returns the fall delay for the minimum corner. *Return type:* float
`delay_min_rise`	Returns the rise delay for the minimum corner. *Return type:* float
`delay_sigma_max`	Returns the 1-sigma variation component of the maximum delay for the specified timing arc. *Return type*: float

`delay_sigma_max_fall`	Returns the 1-sigma variation component of the maximum falling delay for the specified timing arc. *Return type*: float
`delay_sigma_max_rise`	Returns the 1-sigma variation component of the maximum rising delay for the specified timing arc. *Return type*: float
`delay_sigma_min`	Returns the 1-sigma variation component of the minimum delay for the specified timing arc. *Return type*: float
`delay_sigma_min_fall`	Returns the 1-sigma variation component of the minimum falling delay for the specified timing arc. *Return type*: float
`delay_sigma_min_rise`	Returns the 1-sigma variation component of the minimum rising delay for the specified timing arc. Return type: float
`delta_delay_max`	Returns the delta delay for the maximum corner. For example, `get_property [get_arcs -of_objects [get_cells clkbuf]] delta_delay_max` *Return type*: float
`delta_delay_max_fall`	Returns the delta fall delay for the max corner. *Return type:* float
`delta_delay_max_rise`	Returns the delta rise delay for the max corner. *Return type:* float
`delta_delay_min`	Returns the delta delay for the minimum corner. For example, `get_property [get_arcs -of_objects [get_cells clkbuf]] delta_delay_min` *Return type*: float
`delta_delay_min_fall`	Returns the delta fall delay for the min corner. *Return type:* float
`delta_delay_min_rise`	Returns the delta rise delay for the min corner. *Return type:* float
`dynamic_delta_delay_max_fall`	Returns the dynamic delay component of the maximum falling delay through the arc. For example, `get_property -view func_worst_cmax_w_dynv [get_arcs -from i_0/A -to i_0/Y] dynamic_delta_delay_max_fall` *Return type:* float
`dynamic_delta_delay_max_rise`	Returns the dynamic delay component of the maximum rising delay through the arc. For example, `get_property -view func_worst_cmax_w_dynv [get_arcs -from i_0/A -to i_0/Y] dynamic_delta_delay_max_rise`
`dynamic_delta_delay_min_fall`	Returns the dynamic delay component of the minimum falling delay through the arc. For example, `get_property -view func_worst_cmax_w_dynv [get_arcs -from i_0/A -to i_0/Y] dynamic_delta_delay_min_fall`
`dynamic_delta_delay_min_rise`	Returns the dynamic delay component of the minimum rising delay through the arc. For example, `get_property -view func_worst_cmax_w_dynv [get_arcs -from i_0/A -to i_0/Y] dynamic_delta_delay_min_rise`

has_mis_fall	Returns a value of true, if MIS (multi-input switching) derate fall value is specified for timing arcs in the library. *Return type:* boolean
has_mis_rise	Returns a value of true, if MIS (multi-input switching) derate rise value is specified for timing arcs in the library. *Return type:* boolean
is_cellarc	Returns a value of true if the arc belongs to a cell. *Return type:* boolean
is_disabled	Returns a value of true if a set_disable_timing has been applied on the supplied library timing arc. *Return type:* boolean
mode	Returns the mode specified for the timing arc in the library. *Return type:* string
object_type	Returns the object type timing_arc. *Return type:* string
sdf_cond	Returns the sdf_cond expression specified for the timing arc in the library. *Return type:* string
sdf_cond_end	Returns the sdf_cond_end expression specified for the timing arc in the library. *Return type:* string
sdf_cond_start	Returns the sdf_cond_start expression specified for the timing arc in the library. *Return type:* string
sense	Returns the sense of the timing arc. *Return type:* string
sink_pin	Returns the sink pin of the delay arc, or the reference pin of the check arc. *Return type:* collection
source_pin	Returns the source pin of the delay arc, or the signal pin of the check arc. *Return type:* collection
ssi_derate_fall	Returns the SSI (simultaneously switching input) derate falling value specified on library cells. For example, `set_timing_derate 0.5 -fall -data -early -input_switching [get_lib_cells Frlib_v1.0/AND2_X1], get_property [get_arcs -from UC/and1/A1 -to UC/and1/ZN] ssi_derate_fall` *Return type:* float
ssi_derate_rise	Returns the SSI (simultaneously switching input) derate rising value specified on library cells. For example, `set_timing_derate 0.6 -rise -data -early -input_switching [get_lib_cells Frlib_v1.0/AND2_X1] get_property [get_arcs -from UC/and1/A1 -to UC/and1/ZN] ssi_derate_rise` *Return type:* float
when	Returns the when condition specified for the timing arc in the library. *Return type:* string
when_library_string	Returns the exact value of the Liberty 'when' attribute if specified for an arc. `get_property [get_arc -from UC/buff/A -to UC/buff/Z] when_library_string` *Return type:* string

Timing Path Properties

`applied_exception_type`	Returns the type of applied exceptions for a timing path. This property can be used to query dominant timing exceptions affecting a timing path. You can use this property to query on constrained/unconstrained timing paths. The possible return values are - multi_cycle, path_delay and false_path - indicating multicycle path, minimum/maximum delay path, and timing false path, respectively. The software will return one of these values for pin-based timing exceptions. However, the software returns “NA” when false paths are defined for clock objects, or if the paths do not have any related timing exception. *Return Type*: String
`arrival`	Returns the arrival time at the timing path. *Return type*: float
`arrival_mean`	Returns the mean component of the arrival time for the timing path. *Return type*: float
`arrival_sigma`	Returns the sigma component of the arrival time for the timing path. *Return type*: float
`arrival_skewness`	Returns the skewness in the arrival time for a timing path. *Return type*: float
`arrival_stddev`	Returns the standard deviation of the arrival time for a timing path. *Return type*: float
`arrival_wire_stddev`	Returns the standard deviation of arrival time due to wire variation for the specified timing paths. `get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection ] arrival_wire_stddev` *Return type*: float
`capturing_arc`	Returns the capturing arc for the specified timing path. `get_property [report_timing -clock_to clk2 -collection] capturing_arc` *Return type*: collection
`capture_clock_path`	Returns the capturing clock path of the timing path. This property can be used only for path_type full_clock. *Return type:* collection
`capturing_clock`	Returns the capturing clock of the timing path. *Return type:* collection
`capturing_clock_close_edge_time`	Returns the closing edge time of the capturing clock. *Return type:* float
`capturing_clock_close_edge_type`	Returns the capturing clock edge direction: `rise` or `fall`. *Return type:* string
`capturing_clock_is_inverted`	Returns a value of true if the phase of the clock at the end point of the capture clock path is opposite to the phase of the capturing clock at the source. *Return type:* boolean
`capturing_clock_is_propagated`	Returns a value of `true` if the capturing clock is in the propagated mode, at the clock end point. *Return type:* boolean
`capturing_clock_latency`	Returns the ideal or propagated latency of the capturing clock. *Return type:* float

`capturing_clock_latency_mean`	Returns the capturing clock mean latency. `get_property [get_timing_paths ] capturing_clock_latency_mean` *Return type:* float
`capturing_clock_latency_sigma`	Returns the sigma variation for the capturing clock latency. `get_property [report_timing -collection ] capturing_clock_latency_sigma` *Return type:* float
`capturing_clock_latency_skewness`	Returns the skewness variation for the capturing clock latency. `get_property [report_timing -collection ] capturing_clock_latency_skewness` *Return type:* float
`capturing_clock_latency_stddev`	Returns the stddev variation for the capturing clock latency. `get_property [report_timing -collection ] capturing_clock_latency_stddev` *Return type:* float
`capturing_clock_open_edge_type`	Returns the opening edge time of the capturing clock. *Return type:* string
`capturing_clock_pin`	Returns the name of the capturing clock pin. *Return type:* collection
`capturing_point_name`	Returns the name of the capturing point. `get_property [report_timing -collection ] capturing_point_name` *Return type:* string
`capturing_point_name`	Returns the name of the capturing point. `get_property [report_timing -collection ] capturing_point_name` *Return type:* string
`capturing_clock_root_close_edge_time`	Returns the close edge time for the root of capturing clocks for the specified timing paths. `get_property [report_timing -clock_to glck -path_type full_clock -collection] capturing_clock_root_close_edge_time` *Return type:* float
`capturing_clock_root_close_edge_type`	Returns the close edge type for the root of capturing clocks for the specified timing paths. `get_property [report_timing -collection ] capturing_clock_close_edge_type` *Return type:* string
`capturing_clock_source_arrival_time`	Returns the capturing clock arrival time at the clock root pin/port of the capturing clock path. This value is a sum of the clock edge time and the source latency constraints specified at the clock root. **Note:** You must specify `report_timing -path_type full_clock` in order to return capturing clock arrival time information. *Return type:* float
`capturing_clock_source_arrival_time_mean`	Returns the mean component of the capturing clock arrival time at the clock root pin/port of the capturing clock path. *Return type:* float
`capturing_clock_source_arrival_time_sigma`	Returns the sigma component of the capturing clock arrival time at the clock root pin/port of the capturing clock path. *Return type:* float
`capturing_point`	Returns a pointer to the end point of the timing path. *Return type:* collection

`capturing_point_is_level_sensitive`	Returns a value of `true` if the end point of the data path is level sensitive. *Return type:* boolean
`check_delay`	Returns the check delay value for the timing path. *Return type:* float
`check_delay_mean`	Returns the mean component of the check delay value for the timing path. *Return type:* float
`check_delay_sigma`	Returns the sigma component of the check delay value for the timing path. *Return type:* float
`check_delay_sigma_detail`	Returns separate delay sigma variation due to correlated variation of current stage, the correlated variation of previous stage, and uncorrelated variation of the current stage on the timing paths, when the `delaycal_socv_accuracy_mode` attribute is set to `medium`, `high`, or `ultra`. `get_property [report_timing -collection] check_delay_sigma_detail` *Return type:* string
`check_delay_skewness_detail`	Returns separate delay skewness variation due to correlated variation of current stage, the correlated variation of previous stage, and uncorrelated variation of the current stage on the timing path when the `delaycal_socv_accuracy_mode` attribute is set to `high` or `ultra`. `get_property [report_timing -collection] check_delay_skewness_detail` *Return type:* string
`check_delay_skewness`	Returns the skewness component of the check delay value for a timing path. *Return type:* float
`check_delay_stddev`	Returns the check delay standard deviation component of the check delay value for a timing path. *Return type:* float
`check_type`	Returns the check type for the timing path. *Return type:* float
`clock_path_end_point`	Returns the last point in the capture clock path of the timing path. *Return type:* collection
`clock_source_jitter`	Returns the clock source jitter value applied on a specific clock. *Return type:* float
`clock_uncertainty`	Returns the clock uncertainty of the timing path. *Return type:* float
`cppr_adjustment`	Returns the CPPR adjustment of the timing path. *Return type:* float
`cppr_adjustment_mean`	Returns the mean component of CPPR adjustment of the timing path. *Return type:* float
`cppr_adjustment_sigma`	Returns the sigma component of CPPR adjustment of the timing path. *Return type:* float
`cppr_adjustment_skewness`	Returns the skewness component of CPPR adjustment of a timing path. *Return type:* float
`cppr_adjustment_stddev`	Returns the standard deviation component of CPPR adjustment of a timing path. *Return type:* float
`cppr_branch_point`	Returns the CPPR branch point of the given timing path. *Return Type*: collection

`cumulative_manhattan_length`	Returns the cumulative manhattan length for wires of timing paths. `get_property [report_timing -collection ] cumulative_manhattan_length` *Return type:* float
`cumulative_manhattan_length_x`	Returns the cumulative horizontal length for wires of timing paths. `get_property [report_timing -collection ] cumulative_manhattan_length_x` *Return type:* float
`cumulative_manhattan_length_y`	Returns the cumulative vertical length for wires of timing paths. `get_property [report_timing -collection ] cumulative_manhattan_length_y` *Return type:* float
`cycle_adjustment`	Returns the cycle adjustment value for the timing paths. `get_property [report_timing -early -collection] cycle_adjustment` *Return type:* float
`drive_adjustment`	Returns the drive adjustment value for the timing path. Return type: float
`drive_adjustment_mean`	Returns the mean component of the drive adjustment value for the timing path. Return type: float
`drive_adjustment_sigma`	Returns the sigma component of the drive adjustment value for the timing path. Return type: float
`external_delay`	Returns the value of the `set_output_delay` constraint at the end point of the path. *Return type:* float
`gba_slack`	Returns the original slack time for the timing path – before PBA retiming is performed. *Return type:* float
`gba_slack_mean`	Returns the mean component of the slack time for the timing path – before PBA retiming is performed. *Return type:* float
`gba_slack_sigma`	Returns the sigma component of the slack time for the timing path – before PBA retiming is performed. *Return type:* float
`gba_slack_skewness`	Returns the skewness variation of slack for GBA analysis. `get_property [report_timing -skip_io_paths -not_through [get_ports *] -collection -early] gba_slack_skewness` *Return type:* float
`gba_slack_stddev`	Returns the standard deviation of slack for GBA analysis. `get_property [report_timing -collection] gba_slack_stddev` *Return type:* float
`hold`	Returns the hold time at the end point of the path. *Return type:* float
`is_borrow_edge_adjust`	Returns a value of `true` if borrow edge adjustment is set on the timing paths. `get_property [report_timing -through micro_dec/LOCKUP/d -collection] is_borrow_edge_adjust` *Return type*: `boolean`
`is_clock_gating_hold`	Returns a value of `true` if the path is a clock gating hold check. *Return type*: boolean

is_clock_gating_setup	Returns a value of `true` if the path is a clock gating setup check. *Return type*: boolean
is_coverage_violating_path	Returns a value of `true` if a timing path contains violations from the timing check coverage. `get_property [ report_timing -collection] is_coverage_violating_path` *Return type*: boolean
is_path_borrowing	Returns a value of `true` if the specified path is a time borrowing path. `get_property [report_timing -collection -to micro_dec/LOCKUP/d] is_path_borrowing` *Return type*: boolean
is_retimed	Returns a value of `true` if the timing path is recalculated using path-based analysis. `get_property [report_timing -retime path_slew_propagation -collection ] is_retimed` *Return type*: boolean
is_time_given	Returns a value of `true` if for the specified timing path time is applied to the starting point. `get_property [report_timing -to lat/Q -collection] is_time_given` *Return type*: boolean
is_transparent_latch	Returns a value of `true` if the path is a transparent latch path. *Return type:* boolean
launching_clock_latency_mean	Returns mean component of latency of the launching clock. *Return type:* float
launching_clock_latency_sigma	Returns sigma component of latency of the launching clock. *Return type:* float
launch_clock_path	Returns the launching clock path of the timing path. This property can be used only for path_type full_clock. *Return type:* collection
launching_clock	Returns the launching clock of the timing path. *Return type:* collection
launching_clock_is_inverted	Returns a value of true if the phase of the clock at the start point of the launch clock path is opposite to the phase of the launch clock at the source. *Return type:* boolean
launching_clock_is_propagated	Returns a value of `true` if the launching clock is in the propagated mode, at the clock end point. *Return type:* boolean
launching_clock_latency	Returns the ideal or propagated latency of the launching clock. *Return type:* float
launching_clock_latency_skewness	Returns the skewness variation for the launching clock latency. `get_property [report_timing -clock_from clk2 -path_type full_clock -collection ] launching_clock_latency_skewness` *Return type:* float
launching_clock_latency_stddev	Returns the standard deviation for the launching clock latency. `get_property [report_timing -clock_from clk2 -path_type full_clock -collection ] launching_clock_latency_stddev` *Return type:* float

`launching_clock_root_open_edge_type`	Returns the open edge type for the root of the launching clock for the specified timing path. `get_property [report_timing -collection ] launching_clock_open_edge_type` *Return type:* string
`launching_clock_source_arrival_time_skewness`	Returns the skewness variation of the capturing clock arrival time at the source. `get_property [ report_timing -clock_from glck -path_type full_clock -collection ] launching_clock_source_arrival_time_skewness` *Return type:* float
`launching_clock_source_arrival_time_stddev`	Returns the standard deviation of the capturing clock arrival time at the source. `get_property [ report_timing -clock_from glck -path_type full_clock -collection ] launching_clock_source_arrival_time_stddev` *Return type:* float
`launching_clock_open_edge_time`	Returns the opening edge time of the launching clock. *Return type:* float
`launching_clock_open_edge_type`	Returns the launching clock's open edge type at the launching point of the timing path. In case of input delay path with respect to a clock it will return the clock edge type, which is triggering the path. *Return type:* string
`launching_clock_source_arrival_time`	Returns the launching clock arrival time at the clock route pin/port of the launching clock path. This value is the sum of the clock edge time plus any source latency constraint specified at the clock root. **Note:** You must specify `report_timing -path_type full_clock` in order to return launching clock arrival time information. *Return type:* float
`launching_clock_source_arrival_time_mean`	Returns the mean component of the launching clock arrival time at the clock route pin/port of the launching clock path. *Return type:* float
`launching_clock_source_arrival_time_sigma`	Returns the sigma component of the launching clock arrival time at the clock route pin/port of the launching clock path. *Return type:* float
`launching_input_delay`	Returns the input delay if the timing path starts from an input or output port. This value is set using `set_input_delay`. *Return type:* float
`launching_point`	Returns a pointer to the start point of the timing path. *Return type:* collection
`launching_point_is_level_sensitive`	Returns a value of `true` if the begin point of the data path is level sensitive. *Return type:* boolean
`launching_point_name`	Returns the name of the launching point. `get_property [report_timing -collection] launching_point_name` *Return type*: string
`lending_path_d_pin`	Returns the D pin for the specified lending timing path. `get_property [report_timing -to lat/Q -collection -path_type full_clock] lending_path_d_pin` *Return type*: string
`nets`	Returns the nets for the specified timing paths. For example, `get_property [get_timing_paths] nets` *Return type*: string

num_cell_arcs	Returns the number of cell arcs in the given timing path.
	For example,
	`set path [report_timing -from <start_point> -to <end_point> -collection]`
	`get_property $path num_cell_arcs`
	Return Type: float

num_net_arcs	Returns the number of net arcs in the given timing path.
	For example,
	`set path [report_timing -from <start_point> -to <end_point> -collection]`
	`get_property $path num_net_arcs`
	Return Type: float

object_type	Returns the object type `timing_path`. *Return type:* string
other_end_arrival	Returns the capture clock path's begin time, In case of SOCV analysis, mean+N*sigma quantile value is reported. *Return type:* float
other_end_arrival_mean	Returns the mean component of capture clock path's begin time. *Return type:* float
other_end_arrival_sigma	Returns the sigma component of capture clock path's begin time. *Return type:* float
other_end_arrival_sigma_detail	Returns separate other end arrival time variation due to correlated and uncorrelated effects on timing paths, when `set_delay_cal_mode -socv_accuracy_mode` is set to `medium`, `high`, or `ultra`. `get_property [report_timing -collection] other_end_arrival_sigma_detail` *Return type:* string
other_end_arrival_skewness	Returns the skewness of other end arrival time for the timing path. `get_property [report_timing -collection] other_end_arrival_skewness` *Return type:* float
other_end_arrival_skewness_detail	Returns separate other end arrival time skewness due to correlated and uncorrelated effects on timing path, when `set_delay_cal_mode -socv_accuracy_mode` is set to `high` or `ultra`. `get_property [report_timing -collection] other_end_arrival_skewness_detail` *Return type:* string
other_end_arrival_stddev	Returns the standard deviation of other end arrival time for a timing path. `get_property [report_timing -collection] other_end_arrival_stddev` *Return type:* float
path_adjust_value	Returns the path adjust value asserted on timing paths using the `set_path_adjust` command. `get_property [report_timing -through UMC/and1 -collection ] path_adjust_value` *Return type:* float
path_cell_delay	Returns the cell delay for the timing path. `get_property [report_timing -collection] path_cell_delay` *Return type:* float
path_cell_delay_mean	Returns the mean cell delay for the timing path. `get_property [report_timing -collection] path_cell_delay_mean` *Return type:* float

`path_cell_delay_sigma`	Returns the sigma component of cell delay for the timing path. `get_property [report_timing -collection] path_cell_delay_sigma` *Return type:* float
`path_delay_adjustment`	Returns the delay adjustment asserted on paths using the set_max_delay/set_min_delay command. `get_property [report_timing -through UMC/and1 -collection ] path_delay_adjustment` *Return type:* float
`path_delay_wire_stddev`	Returns the standard deviation of path delay due to wire variation for the specified timing path. `get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection ] path_delay_wire_stddev` *Return type:* float
`path_net_delay`	Returns the net delay for the timing paths. `get_property [report_timing -collection] path_net_delay` *Return type:* float
`path_net_delay_mean`	Returns the mean net delay for the timing paths. `get_property [report_timing -collection] path_net_delay_mean` *Return type:* float
`path_net_delay_sigma`	Returns the sigma variation for path net delay for the specified timing path. `get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection ] path_net_delay_sigma` *Return type:* float
`path_delay`	Returns the delay of data path component of a timing path. In case of SOCV analysis, mean+N*sigma quantile delay is reported. *Return type:* float
`path_delay_mean`	Returns the mean delay component of data path of a timing path. *Return type:* float
`path_delay_sigma`	Returns the sigma delay component of data path of a timing path. *Return type:* float
`path_group`	Returns a collection of path groups associated with the timing path. *Return type:* collection
`path_group_name`	Returns the name of the path group for the timing path. *Return type:* string
`path_type`	Returns a value of `max` if the path is for setup, or `min` if the path is for hold. *Return type:* string

`period`	Returns the time duration between the launching clock edge and capturing clock edge of the path. The launching clock edge indicates the launching clock time when the signal at the start point of the path is launched, and the capturing clock edge indicates the capturing clock time when the signal at the path end point is captured. For both of these edges, their ideal clock cycle times are considered for calculating the path period. For those paths, where either the launching or capturing clock is absent or asynchronous, the property prints NA. The property works for both setup (late) paths, and hold (early) paths. Usually the path period is decided by a simple subtraction of effective launching edge from the effective capturing edge of the path. Under certain scenario, these edges are moved back or ahead in time, and accordingly the path period is also affected. The following factors impact period: a) Multicycle paths: The launch or capture edge will move by the cycle factor depending on the type of multi cycle exception, and this cycle factor is taken into account while computing the period. b) Max delay paths: Any max_delay specification overrides the relationship between capturing and launching edges and sets it to the max_delay value. The period of the path is thus returned as equal to the specified max delay value for sequential max delay paths. Note that a combinational max delay path has at least one of the launch or capture clocks either absent or asynchronous, and hence the effective period is not calculated for them. c) Clock uncertainty: The uncertainties applicable to the interaction of launch and capture clocks for this particular path also affect the effective edges, but this factor is not taken into consideration while computing the period. d) Unconstrained paths: The unconstrained paths have no capture clock and hence period can not be calculated for them. The property will return NA. *Return value*: float For more information, see "Examples".
`phase_shift`	Returns the phase shift for the timing paths. `get_property [report_timing -collection] phase_shift` *Return type:* float
`recovery`	Returns the recovery time at the end point of the path. *Return type:* float
`removal`	Returns the removal time at the end point of the path. *Return type:* float
`required_time`	Returns the required time for the timing path. *Return type:* float
`required_time_mean`	Returns the mean component of the required time for the timing path. *Return type:* float
`required_time_sigma`	Returns the sigma component of the required time for the timing path. *Return type:* float
`required_time_sigma_detail`	Returns separate required time variation due to correlated and uncorrelated effects on timing paths, when `set_delay_cal_mode -socv_accuracy_mode` is set to `medium`, `high`, or `ultra`. `get_property [report_timing -collection] required_time_sigma_detail` *Return type:* string
`required_time_skewness`	Returns the skewness in required time of a timing path. `get_property [report_timing -collection] required_time_skewness` *Return type:* float

required_time_skewness_detail	Returns separate required time skewness due to correlated and uncorrelated effects on timing paths, when set_delay_cal_mode -socv_accuracy_mode is set to high or ultra. get_property [report_timing -collection] required_time_skewnesss_detail *Return type:* string
required_time_stddev	Returns the standard deviation of the required time for a timing path. get_property [report_timing -collection] required_time_stddev *Return type:* float
slack_sigma_detail	Returns separate slack sigma variation due to correlated and uncorrelated effects on timing paths, when set_delay_cal_mode -socv_accuracy_mode is set to medium, high, or ultra. get_property [report_timing -collection] slack_sigma_detail *Return type:* string
slack_skewness	Returns slack skewness for the timing paths. get_property [report_timing -collection] slack_skewness *Return type:* float
slack_skewness_detail	Returns separate slack skewness variation due to correlated and uncorrelated effects on timing paths, when set_delay_cal_mode -socv_accuracy_mode is set to high or ultra. get_property [report_timing -collection] slack_skewness_detail *Return type:* string
slack_wire_stddev	Returns the standard deviation of slack due to wire variation for the specified timing paths. get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] slack_wire_stddev *Return type:* float
setup	Returns the setup time at the end point of the path. *Return type:* float
skew	Returns the latency difference between the launching clock latency and capturing clock latency of the path (after considering the effect of CPPR factor). *Return type:* float
slack	Returns the slack time for the timing path. *Return type:* float
slack_mean	Returns the mean component of the slack time for the timing path. *Return type:* float
slack_sigma	Returns the sigma component of the slack time for the timing path. *Return type:* float
slack_sigma_detail	Returns separate slack sigma variation due to correlated and uncorrelated effects on timing paths, when set_delay_cal_mode -socv_accuracy_mode is set to medium, high, or ultra. get_property [report_timing -collection] slack_sigma_detail *Return type:* string
slack_skewness	Returns slack skewness for the timing paths. get_property [report_timing -collection] slack_skewness *Return type:* float
slack_skewness_detail	Returns separate slack skewness variation due to correlated and uncorrelated effects on timing paths, when set_delay_cal_mode -socv_accuracy_mode is set to high or ultra. get_property [report_timing -collection] slack_skewness_detail *Return type:* string

`slack_wire_stddev`	Returns the standard deviation of slack due to wire variation for the specified timing paths. `get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] slack_wire_stddev` *Return type:* float
`slack_stddev`	Returns standard deviation of the slack. *Return type:* float
`spatial_distance`	Reports the bounding box distance for spatial derating applied on a timing path. *Return type:* float
`spatial_distance_cell`	Reports the bounding box distance for cells. *Return type:* float
`spatial_distance_net`	Reports the bounding box distance for nets. *Return type:* float
`time_borrowed`	Returns the amount of time borrowed from the timing end point. *Return type:* float
`time_lent`	Returns the amount of time lent to the timing start point. *Return type:* float
`timing_points`	Returns a collection of the timing points that make up the path. *Return type:* collection
`total_cell_delta_delay`	Returns the total cell incremental delay present in the timing paths. `get_property [report_timing -through clk_I1/ZN -unconstrained -collection] total_cell_delta_delay` *Return type:* float
`total_delta_delay`	Returns the total incremental delay present in the timing paths. `get_property [report_timing -through clk_L1_I1/ZN -unconstrained -collection] total_delta_delay` *Return type:* float
`total_net_delta_delay`	Returns the total net incremental delay present in the timing paths. `get_property [report_timing -through clk_L1_I1/ZN -unconstrained -collection] total_net_delta_delay` *Return type:* float
`underated_slack`	Returns the underated slack time for the timing path. The underated slack value is available with timing path object only if it is generated using the `-derate` parameter. *Return type:* float
`view_name`	Returns the analysis view name associated with the timing path. *Return type*: string
`wire_variation_capacitance_slack_delta`	Returns a list of lists with slack shift due to capacitance of each layer. *Return type*: string
`wire_variation_layer_info`	Returns layer information for wire variation analysis. *Return type*: string
`wire_variation_layer_slack_delta`	Returns a list of lists with total slack shift per layer. *Return type*: string
`wire_variation_resistance_slack_delta`	Returns a list of lists with slack shift due to resistance of each layer. *Return type*: string

`wire_variation_slack_shift`	Returns a list of lists with slack shift. *Return type*: string
`wire_variation_slack_shift_total`	Returns a list of lists with total slack shift. *Return type*: string
`worst_cell_delay`	Returns the worst cell delay of the timing paths. `get_property [report_timing -collection] worst_cell_delay` *Return type:* float
`worst_cell_delay_mean`	Returns the worst mean cell delay of the timing paths. `get_property [report_timing -collection] worst_cell_delay_mean` *Return type:* float
`worst_cell_delay_sigma`	Returns the sigma component for worst cell delay in the timing paths. `get_property [report_timing -collection] worst_cell_delay_sigma` *Return type:* float
`worst_cell_delay_skewness`	Returns the skewness for worst cell delay in the timing paths. `get_property [report_timing -collection] worst_cell_delay_skewness` *Return type:* float
`worst_cell_delay_stddev`	Returns the standard deviation for worst cell delay in the timing paths. `get_property [report_timing -collection] worst_cell_delay_stddev` *Return type:* float
`worst_cell_delta_delay`	Returns the worst cell delta delay of the timing paths. `get_property [report_timing -through clk_L16/ZN -unconstrained -collection] worst_cell_delta_delay` *Return type:* float
`worst_delay`	Returns the worst delay for the timing paths. `get_property [report_timing -collection] worst_delay` *Return type:* float
`worst_delay_mean`	Returns the mean worst delay for the timing paths. `get_property [report_timing -collection] worst_delay_mean` *Return type:* float
`worst_delay_sigma`	Returns the sigma component for worst delay in the timing paths. `get_property [report_timing -collection] worst_delay_sigma` *Return type:* float
`worst_delay_skewness`	Returns the skewness for worst cell delay in the timing paths. `get_property [report_timing -collection] worst_delay_skewness` *Return type:* float
`worst_delay_stddev`	Returns the standard deviation for worst cell delay in the timing paths. `get_property [report_timing -collection] worst_delay_stddev` *Return type:* float
`worst_delta_delay`	Returns the worst delta delay for the timing paths. `get_property [report_timing -through clk_L16/ZN -unconstrained -collection] worst_delta_delay` *Return type:* float

`worst_manhattan_length`	Returns the worst manhattan length of the timing paths. `get_property [report_timing -collection] worst_manhattan_length` *Return type:* float
`worst_manhattan_length_net_name`	Returns the name of the net with worst manhattan length in the timing paths. `get_property [report_timing -collection] worst_manhattan_length_net_name` *Return type:* string
`worst_net_delay`	Returns the worst net delay for the timing paths. `get_property [report_timing -collection] worst_net_delay` *Return type:* float
`worst_net_delay_mean`	Returns the mean worst net delay for the timing paths. `get_property [report_timing -collection] worst_net_delay _mean` *Return type:* float
`worst_net_delay_sigma`	Returns the sigma variation for worst net delay for the specified timing paths. `get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] worst_net_delay_sigma` *Return type:* float
`worst_net_delay_skewness`	Returns the skewness for the worst net delay for the specified timing paths. `get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] worst_net_delay_skewness` *Return type:* float
`worst_net_delay_stddev`	Returns the standard deviation for the worst net delay for the specified timing paths. `get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] worst_net_delay_stddev` *Return type:* float
`worst_net_delta_delay`	Returns the worst net incremental delay for the timing paths. `get_property [report_timing -through clk_L16/ZN -unconstrained -collection] worst_net_delta_delay` *Return type:* float

Timing Point Properties

`adjustment_in_delay`	Returns the adjustment in delay for the specified timing point of a timing path. `get_property [get_property [get_property [report_timing -from dff0/Q -to dff2/D -path_type full_clock -collection] capture_clock_path] timing_points] adjustment_in_delay` *Return type*: float
`arrival`	Returns the arrival time at the timing point. *Return type*: float
`arrival_mean`	Returns the mean component of the arrival time at the timing point. *Return type*: float
`arrival_sigma`	Returns the sigma component of arrival time at the timing point. *Return type*: float

`arrival_skewness`	Returns the skewness in arrival for a timing point. `get_property [get_property [report_timing -collection] timing_points] arrival_skewness` *Return type*: float
`arrival_stddev`	Returns the standard deviation of arrival time for a timing point. `get_property [get_property [report_timing -collection] timing_points] arrival_stddev` *Return type*: float
`arrival_wire_stddev`	Returns the standard deviation of arrival time due to wire variation for the specified timing point. `get_property [get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] timing_points] arrival_wire_stddev` *Return type*: float
`delay`	Returns the delay value for the timing point. *Return type:* float
`delay_mean`	Returns the mean component of the delay value for the timing point. *Return type:* float
`delay_sigma`	Returns the sigma component of the delay value for the timing point. *Return type:* float
`delay_skewness`	Returns the standard deviation of slew due to wire variation for the specified timing points. `get_property [get_property [report_timing -collection] timing_points] delay_skewness` *Return type:* float
`delay_stddev`	Returns the standard deviation of delay for the timing points. `get_property [get_property [report_timing -collection] timing_points] delay_stddev` *Return type:* float
`delay_wire_stddev`	Returns the standard deviation of delay due to wire variation for the specified timing points. `get_property [get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection] timing_points] delay_wire_stddev` *Return type:* float
`delta_delay`	Returns the incremental delay on a timing point after SI analysis is performed. *Return type:* float
`direction`	Returns the direction of the timing points. `get_property [get_property [report_timing -collection] timing_points] direction` *Return type:* string
`fanin_arc`	Returns the delay arc for the corresponding timing point. *Return type*: collection
`hierarchical_name`	Returns the hierarchical name of the timing points. `get_property [get_property [report_timing -collection] timing_points] hierarchical_name` *Return type:* string
`is_fanin_arc_cellarc`	Returns a value of `true` if the fan in arc of the timing points is a cell arc. `get_property [get_property [report_timing -through UMC/and1 -collection] timing_points] is_fanin_arc_cellarc` *Return type:* boolean
`is_hierarchical`	Returns a value of `true` if the timing point is hierarchical. `get_property [get_property [report_timing -through UMC/latch/D -unconstrained -collection] timing_points] is_hierarchical` *Return type:* boolean

`net`	Returns the net for the timing points. `get_property [get_property [report_timing -collection] timing_points] net` *Return type*: collection
`object_type`	Returns the `timing_point` object type. *Return type:* string
`pin`	Returns a collection of pins or ports of the timing point. *Return type:* collection
`retime_mis_delta`	Returns the retime MIS (multi-input switching) delta delay of PBA paths. *Return type:* float
`si_object`	Returns the `si_victim` object type that can be used to query SI delay attributes. **Note**: To enable SI objects, you can set the `timing_report_enable_si_debug` global variable to `true`.
`slew`	Returns the path-specific slew of the timing point. *Return type:* float
`slew_mean`	Returns the mean component of the path-specific slew of the timing point. *Return type:* float
`slew_sigma`	Returns the sigma component of the path-specific slew of the timing point. *Return type:* float
`slew_skewness`	Returns the skewness of slew for the specified timing points. `get_property [get_property [report_timing -collection] timing_points] slew_skewness` *Return type:* float
`slew_stddev`	Returns the standard deviation of slew for the specified timing points. `get_property [get_property [report_timing -collection] timing_points] slew_stddev` *Return type:* float
`slew_wire_stddev`	Returns the standard deviation of slew due to wire variation for the specified timing points. `get_property [get_property [report_timing -retime path_slew_propagation -split_delay -path_type full_clock -collection ] timing_points] slew_wire_stddev` *Return type:* float
`spatial_derate`	Returns the spatial derate on the timing points. `get_property [get_property [report_timing -collection] timing_points] spatial_derate` *Return type:* float
`spatial_derate_sigma`	Returns the sigma component of spatial derate on the timing points. `get_property [get_property [report_timing -collection] timing_points] spatial_derate_sigma` *Return type:* float
`ssi_derate`	Returns the path-specific SSI derate of the timing points. `get_property [index_collection [get_property [report_timing -collection] timing_points] 4] ssi_derate` *Return type:* float
`total_derate`	Returns the total derate on the timing points. `get_property [get_property [report_timing -collection] timing_points] total_derate` *Return type:* float
`total_derate_sigma`	Returns the sigma component of total derate on the timing points. `get_property [get_property [report_timing -collection] timing_points] total_derate_sigma` *Return type:* float

`transition_type`	Returns the transition type of the timing point - rise or fall. *Return type:* string
`user_derate`	Returns the derate set using the `set_timing_derate` command on the timing points. `get_property [get_property [report_timing -collection] timing_points] user_derate` *Return type:* float
`user_derate_sigma`	Returns the sigma variation derate set using the `set_timing_derate` command on the timing points. `get_property [get_property [report_timing -collection] timing_points] user_derate_sigma` *Return type:* float
`voltage`	Returns the voltage at a timing points. `get_property [get_property [report_timing -collection] timing_points] voltage` *Return type:* float
`vtskew_mean_derate`	Reports the mean derating factors of voltage threshold (VT) skew for a timing point. *Return type:* float
`vtskew_sigma_derate`	Reports the sigma derating factors of voltage threshold (VT) skew for a timing point. *Return type:* float

Signal Integrity Attributes

The signal integrity attributes can be queried using the following object types:

- `si_victim`
- `si_attacker`
- `noise_victim`: Returns a collection of `si_victim` objects that can be used to query glitch attributes.
  Return type: collection

These are explained below.

si_victim Properties

The following information is saved under the `si_victim` object type:

		Valid for Properties		
Property Name	**Description**	**si_victim**	**si_object**	**noise_victim**
`annotated_glitch`	Reports the annotated glitch. *Return type:* float	N	N	Y
`attackers`	Returns a collection of `si_attacker` objects that can be used to query attacker attributes. *Return type:* collection	Y	Y	Y
`capacitance`	Returns the total capacitance of victim net. *Return type:* float	Y	Y	Y
`coupling_capacitance`	Returns the total coupling capacitance of a victim net. *Return type:* float	Y	Y	Y

`glitch_area`	Returns the glitch area. *Return type:* float	N	N	Y
`glitch_width`	Returns the glitch width. *Return type:* float	N	N	Y
`hierarchical_name`	Returns the hierarchical name of a victim net. *Return type:* string	Y	Y	Y
`input_peak`	Returns the Receiver Input Peak (RIP) value. *Return type:* float	N	N	Y
`input_peak_slack`	Returns the Receiver Input Peak (RIP) slack value. *Return type:* float	Y	N	N
`input_failure_threshold`	Returns the failure threshold at RIP. This will always be reported irrespective of whether failure happens at RIP or ROP. *Return type:* float	N	N	Y
`level`	Returns the noise level - VL or VH. *Return type:* string	N	N	Y
`num_active_attackers`	Returns the number of active attackers. *Return type:* float	Y	Y	Y
`num_attackers`	Returns the number of total attackers (distinct + accumulated small attacker - as 1). The attackers that are dropped from the accumulated small attackers will not be included with the total attackers. *Return type:* float	Y	Y	Y
`object_type`	Returns the object type `si_victim`. *Return type:* string	Y	Y	Y
`prop_glitch`	Reports the propagated glitch seen at the victim driver output. *Return type:* float	N	N	Y
`receiver_peak`	Returns the Receiver Output Peak (ROP) value. *Return type:* float	N	N	Y
`receiver_peak_slack`	Returns the Receiver Output Peak (ROP) slack value. *Return type:* string	Y	N	N

`receiver_peak_threshold`	Returns the failure threshold at ROP. This will always be reported irrespective of whether failure happens at RIP or ROP. *Return type:* float	N	N	Y
`resistance`	Returns the total resistance of a victim net. *Return type:* string	Y	Y	Y
`transition`	Returns the victim transition direction (rise or fall). *Return type:* string	Y	Y	N
`view_type`	Returns the type of analysis (early or late). *Return type:* string	Y	Y	N

si_attacker Properties

The following Information is saved under the `si_attacker` object type for each attacker of the victim net in the path:

Property Name (for each attacker retrieved from a victim)	Description
`coupling_capacitance`	Returns the coupling of a victim net. *Return type:* float
`hierarchical_name`	Returns the hierarchical name of an attacker. *Return type:* string
`is_active`	Returns whether an attacker is active or not. *Return type:* string
`noise_peak`	Returns the Xtalk delay bump induced by an attacker. *Return type:* float
`object_type`	Returns the `si_attacker` object type. *Return type:* string
`slew`	Returns the slew of attackers. *Return type:* float
`state`	Returns the status of attackers. The status descriptions are given below: • SY: Synchronous • INF: Attacker with infinite timing window • PE: Filtered due to physical exclusion • UF: Filtered by user • CN: Filtered because it is a constant net • SB: Filtered due to small bump • TA: Filtered because of timing window does not overlap • LC: Filtered due to logical correlation • NA: Attacker status is not known *Return type:* string

Examples

The following `get_property` example shows the usage of SI attributes to query noise information:

```
source pinlist.tcl
set chan [open attribute.log w]
foreach pin $pinList {
   set vicArray [get_property [get_pins $pin] noise_victim]
   foreach_in_collection vic $vicArray {
       set input_peak [ get_property $vic input_peak]
       #if { $input_peak > 250 } {}

       set vic_name [ get_property $vic hierarchical_name ]
       set x [ get_property [get_net -of_objects $vic_name ] name ]
       set level [ get_property $vic level]
       set ccap [ get_property $vic capacitance]
       set vdd [get_property [get_pins $vic_name] power_rail_voltage_max]
       set libs [get_libs -of_objects [get_lib_cells -of_objects [get_cells -of_objects [get_pins $vic_name]]]]
       foreach_in_collection libName $libs {set library [get_property $libName hierarchical_name]}
       set prop_glitch [ get_property $vic prop_glitch]
       set cap [format %.2f $ccap]
       set annotated_glitch [ get_property $vic annotated_glitch]
       set receiver_peak [ get_property $vic receiver_peak]
       set input_peak_threshold [ get_property $vic input_failure_threshold ]
       set receiver_peak_threshold [ get_property $vic receiver_peak_threshold]
       puts $chan "--------------------------------------------------------------------------------"
         puts $chan "Peak(mV) Level TotalCap(fF) Vdd(V) Library VictimNet"
         puts $chan "$input_peak $level $cap $vdd $library $vic_name {$x}"

         if { $receiver_peak > 0 } {
           puts $chan "Receiver output peak:"
           puts $chan "Value ReceiverNet"
           puts $chan "$receiver_peak ( $receiver_peak_threshold) $vic_name (CELL)"
         }

         if { $input_peak > 0 } {
           puts $chan "Receiver Input peak:"
           puts $chan "Value ReceiverNet"
           puts $chan "$input_peak ( $input_peak_threshold) $vic_name (CELL)"
         }

         puts $chan "\nConstituents:"
         puts $chan "Source Peak(mV) Offset(ps) Slew(ps) Xcap(fF) Vdd(V) Edge Status Net"
         set aggs [ get_property $vic attackers ]
         foreach_in_collection current_agg $aggs {

           set hierarchical_name [ get_property $current_agg hierarchical_name ]
           set noise_peak [ get_property $current_agg noise_peak]
           set state [ get_property $current_agg state]
           set slew [ get_property $current_agg slew]
           set edge [ get_property $current_agg transition]
           if {[string match ~* $hierarchical_name]} {
           set vdd [get_property [get_pins $vic_name] power_rail_voltage_max]
           } else {
           set vdd [get_property [get_pins -of_objects [get_nets $hierarchical_name]] power_rail_voltage_max]
           }
           set coupling_capacitance [ get_property $current_agg coupling_capacitance]

           puts $chan "Cpl: $noise_peak $slew $edge $vdd $coupling_capacitance $state $hierarchical_name"
           }
         if { $prop_glitch > 0 } {
           puts $chan "Prp: $prop_glitch"
          }


}
close $chan
```

For more information on querying SI attributes, refer to "Signal Integrity Delay and Glitch Analysis" chapter in the *Tempus User Guide*.

User-Defined Properties

The user-defined properties allow you to define certain properties which can be interpreted by the software in a predefined way. When these properties are not specified, the software will revert to the default behavior, as applicable.

Property Name	Type	Description
`capacitive_load_unit`	lib	Provides the load unit mentioned in the library.
`time_unit`	lib	Provides the time unit mentioned in the library.
`aocv_weight`	lib_cell	Defines AOCV weight for a given cell.
`k_input_switching_window_fall*`	lib	Defines the switching window threshold for fall signal during the application of SSI (simultaneous switching input) derating feature. *Default*: 1
`k_input_switching_window_rise*`	lib	Defines the switching window threshold for rise signal during the application of SSI (simultaneous switching input) derating feature. *Default*: 1

* When the above properties are defined in SSI derating environment, the delay of the respective cell multiplied by the above factor will be compared with the difference in (rise/fall) signal arrivals at two inputs to determine if switching derate needs to be applied to cell delay.

Examples

- The following command returns the hierarchical name for the pin contained in the collection referenced by `$pin`:

  ```
  get_property $pin hierarchical_name
  ```

- The following example shows how you can query a user-defined property from `stage_weighting` library at the cell level:

  ```
  set_property [get_lib_arcs -of fast/BUFX2] stage_weighting 3 // sets the value
  get_property [get_lib_arcs -of fast/BUFX2] stage_weighting
  ```

- The following example shows power and ground pins for the corresponding library pins:

  ```
  get_property [get_lib_pins CELLA/A] related_power_pin
  get_property [get_lib_pins CELLA/A] related_ground_pin
  ```

- The following example shows active clocks:

  ```
  get_property [get_clocks Iclk1] is_active true
  ```

- The following commands define the switching window threshold for rise signal:

  ```
  define_property -type float -object_type lib k_input_switching_window_rise
  set_property [get_libs slow] k_input_switching_window_rise 1.1
  ```

- The following example shows that the phase shift returned by the `period` property:

  ```
  report_timing -from [get_clocks refclk] -to [get_clocks m_spi_clk]
  Path 1: VIOLATED (-1.869 ns) Setup Check with Pin SP_IN/sp_reg[0]/CK->D
  Group: m_spi_clk
  Startpoint:(F) spi_data Clock: (R) refclk
  Endpoint: (F) SP_IN/sp_reg[0]/D
  Clock: (R) m_spi_clk

                      Capture      Launch
       Clock Edge:+    4.000        0.000
      Src Latency:+    0.000        0.000
      Net Latency:+    0.780 (P)    0.000 (I)
          Arrival:=    4.780        0.000

            Setup:-    1.292
      Uncertainty:-    0.400
      Cppr Adjust:+    0.000
    Required Time:=    3.088
     Launch Clock:=    0.000
      Input Delay:+    4.000
        Data Path:+    0.958
            Slack:=   -1.869
  #-------------------------------------------------------------------------------
  ```

```
Timing Point        Flags  Arc     Edge  Cell      Fanout  Trans  Delay  Arrival
                                                           (ns)   (ns)   (ns)
#-------------------------------------------------------------------------------
SP_IN/p132A/ZN      -      B2->ZN  R     AOI22_X1  1       0.049  0.506  4.534
SP_IN/p135A/ZN      -      A->ZN   F     INV_X1    1       0.845  0.424  4.958
SP_IN/sp_reg[0]/D   -      D       F     SDFF_X2   1       0.399  0.000  4.958
#-------------------------------------------------------------------------------
……..
tempus > set path [report_timing -from [get_clocks refclk] -to [get_clocks m_spi_clk] -collection]
0x14
tempus > get_property $path period
```

- In this following example, there is a clock reference for launch clock (Null phase- @), so Tempus does not show any clock period for this path:

```
tempus > report_timing -to [get_clocks m_spi_clk] -early
Path 1: MET (0.406 ns) Hold Check with Pin SP_IN/sp_reg[4]/CK->SI
Group: m_spi_clk
Startpoint: (R) SP_IN/sp_reg[3]/CK Clock: (R) m_spi_clk
Endpoint: (R) SP_IN/sp_reg[4]/SI
Clock: (R) m_spi_clk
                    Capture      Launch
     Clock Edge:+    0.000        0.000
    Src Latency:+    0.000        0.000

    Net Latency:+    0.780 (P)    0.767 (P)
        Arrival:=    0.780        0.767

           Hold:+   -0.056
    Uncertainty:+    0.600
    Cppr Adjust:-    0.000
  Required Time:=    1.324
   Launch Clock:=    0.767
      Data Path:+    0.963
          Slack:=    0.406
#-------------------------------------------------------------------------------
Timing Point       Flags  Arc   Edge  Cell     Fanout  Trans  Delay   Arrival
                                                        (ns)   (ns)    (ns)
#-------------------------------------------------------------------------------
SP_IN/sp_reg[3]/CK  -     CK    R     (arrival) 3       0.113  -       0.767
SP_IN/sp_reg[3]/QN  -     CK->QN R    SDFF_X2   1       0.113  0.963   1.730
SP_IN/sp_reg[4]/SI  -     SI    R     SDFF_X2   1       0.255  0.000   1.730
#-------------------------------------------------------------------------------
```

tempus > set path [report_timing -to [get_clocks m_spi_clk] -early]
0x16
tempus > get_property $path period
NA

- The following example shows if a real or generated clock is defined on the pins or ports:

```
tempus > create_clock [get_ports {CLK_t}] -name clk -period 100 -waveform {0 50}
tempus > create_generated_clock -divide_by 2 -source CLK -name d_2 CLK_b
tempus > get_property [get_ports CLK_t] is_clock_root
true

tempus > get_property [get_pins CLK_b] is_clock_root
true
```

Related Commands

- report_property
- set_property
- reset_property
- define_property
- report_object_property_names

index_collection

```
index_collection
[-help]
base_collection index
```

Returns a collection containing the object that exists at the specified index of the specified object collection.

Parameters

`-help`	Prints out the command usage.
`base_collection`	Specifies the base collection.
`index`	Specifies the index number. You can specify a number from `0` to *`collection_size-1`*.

Examples

- The following command returns the object that exists at the index number `2` in the object collection referenced by `$nets`:

  ```
  tempus> index_collection $nets 2
  ```

Related Commands

- `range_collection`

query_objects

```
[-help]
query_objects collection [-limit value]
```

Displays the names of the objects contained inside the specified collection.

> The `query_objects` command displays object names only. To return a tcl list of object names, the `get_object_name` command may be used instead.

Parameters

`-help`	Prints out the command usage.
collection	Specifies the object collection for which to print names.
`-limit` *value*	Specifies the maximum number of object names to print. Specify a value of `0` to query the names of all of the objects in the collection. *Default*: Prints the first 100 object names in the collection.

Examples

- The following commands create collections of pins and nets, add them together to create a new collection, and prints the names of the objects in the collection:

  ```
  set pins [get_pins {i_block1/b1_Sin1 i_block1/b1_CLK1 i_block1/b1_Sout1}]
  set nets [get_nets {Sin1 CLK1 Sout1}]
  add_to_collection newVar $pins
  add_to_collection newVar $cells
  query_objects $newVar
  ```

  The software returns the following information:

  ```
  i_block1/b1_Sin1 i_block1/b1_CLK1 i_block1/b1_Sout1 Sin1 CLK1 Sout1
  ```

Related Commands

- `get_object_name`

range_collection

```
range_collection
[-help]
base_collection
from
to
```

Returns a sub-collection of the base collection. The sub-collection range lies between the ‘from’ and ‘to’ (including ‘from’ and ‘to’) index numbers specification of this command.

Parameters

`-help`	Prints out the command usage.
`base_collection`	Specifies the base collection.
`from`	Specifies the ‘from’ index number.
`to`	Specifies the ‘to’ index number.

The index number can be an integer or an expression, such as, “end OR end-integer” (with no spaces within the expression).

If the specified to/from value is negative (less than 0), then the software will change this value to 0. If the value is greater than the “collection_size – 1”, then it will be changed to “collection_size – 1”.

Examples

- The following example sets `$points` as the base collection:

```
tempus > set points [get_cells *]
0x13
tempus > sizeof_collection $points
22
```

- The following example returns the sub-collection of the base collection `$points` between the index number range from 2 to 4:

```
tempus > range_collection $points 2 4
clk_buf2 clk_buf3 clk_buf4
0x16
tempus > range_collection $points 4 2
clk_buf2 clk_buf3 clk_buf4
0x17
```

- The following example returns a sub-collection starting from index number 0 till the end of the base collection:

```
tempus > range_collection $points 0 end
clk_buf1 pad1 clk_buf2 clk_buf3 clk_buf4 buf1 buf2 buf3 buf4 mem dff1 dff2 clkand2 pu_cell_1 pu_lat_1 u1 cg4 genLoop comboLoop pad2
dff3 latch1
0x18
```

- The following example returns a sub-collection starting from end-integers 4 to 2:

```
tempus > range_collection $points end-4 end-2
genLoop comboLoop pad2
0x19
```

- The following example shows a range starting from a negative value to a value greater than the collection size. The following warning messages are displayed:

```
tempus > range_collection $points -5 100
**WARN: (TCLCMD-1419): 'from' index with value -5 is less than start index, so it is changed to 0 .
**WARN: (TCLCMD-1419): 'to' index with value 100 is greater than end index, so it is changed to 21 .
clk_buf1 pad1 clk_buf2 clk_buf3 clk_buf4 buf1 buf2 buf3 buf4 mem dff1 dff2 clkand2 pu_cell_1 pu_lat_1 u1 cg4 genLoop comboLoop pad2
dff3 latch1
0x1a
```

- The following example shows the warning message displayed for a negative start index number:

```
tempus > range_collection $points end-25 end-20
**WARN: (TCLCMD-1419): 'from' index with value -4 is less than start index, so it is changed to 0 .
clk_buf1 pad1
0x1b
```

- The following command shows an error if the specified index number is not an integer:

```
tempus > range_collection $points s end
**ERROR: (TCLCMD-1418): Bad index 's': must be integer? or end?[-]integer?
```

Related Commands

- index_collection

remove_from_collection

```
remove_from_collection
[-help]
base_collection
[-intersect]
collection_or_obj_list
```

Creates a new collection by removing the objects specified in one collection from another collection.

Parameters

`-help`	Prints out the command usage.
`base_collection`	Specifies the base collection of objects. Objects from the second collection or object list are removed from this collection to create the new collection.
`-intersect`	Provides intersection of base collection elements with second collection of objects or lists.
`collection_or_obj_list`	Specifies the collection or list of objects that should be removed from the base collection to create a new collection. When specified, this parameter returns those elements that intersect and exist in both the *base_collection* and the second collection. Also, all the elements of *base_collection* that do not intersect with second collection will be removed.

Examples

- The following commands create a collection of pins that does not include clock pins:

  ```
  set allP [get_pins *]
  set all_pin_no_clock [remove_from_collection $allP [get_pins -filter "is_clock == true"]]
  ```

  You can use the `sizeof_collection` command to see that the `all_pin_no_clock` collection contains fewer objects (that is, no clock pins) than the original `allP` collection:

  ```
  sizeof_collection $allP
  50

  sizeof_collection $all_pin_no_clock
  47
  ```

Related Commands

- `range_collection`
- `index_collection`

report_object_property_names

```
report_object_property_names [-help]
[-obj_type {cell clock design lib lib_cell lib_pin lib_timing_arc net path_group pin port timing_arc timing_path timing_point}]
```

Lists all of the properties associated with the specified object type.

If you do not specify any object type, this command lists all the supported object types, and all the properties associated with those object types.

Parameters

`-help`	Prints out the command usage.
`-obj_type`	Lists the properties associated with the specified object type. The following object types are supported: cell, clock, design, lib, lib_cell, lib_pg_pin, lib_pin, lib_timing_arc, net, path_group, pg_net, pg_pin, pin, port, si_attacker, si_victim, timing_arc, timing_path, or timing_point Additional SI properties can be included with the `timing_point` object. To enable SI properties, you can set `timing_report_enable_si_debug` attribute to `true`.

Related Commands

- `define_property`
- `get_property`
- `report_property`
- `reset_property`
- `set_property`

report_property

```
report_property
[-help]
{collection | list_of_collections}
[-property_list list of properties]
[{> | >>} filename]
```

Reports the properties associated with each object in the specified collection.

For properties that return a populated collection, the command output displays `{...}`. For any empty collections, `{}` will be displayed.

Parameters

`-help`	Prints out the command usage.
*collection	list_of_collections*	Specifies a collection, or a list of collections to report. The collections can contain any of the following object types: cell, clock, design, lib, lib_cell, lib_pin, lib_pg_pin, pg_net, pg_pin, lib_timing_arc, net, path_group, pin, port, timing_arc, timing_path, or timing_point
`-property_list` *list of properties*	Specifies a list of valid property names to report. **Note**: If the `-property_list` parameter is not specified, then all the properties associated with a collection of objects will be reported.
{`>`	`>>`} *filename*	When specified with >, the software writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, the software writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. You can add the `.gz` extension to the file name to generate a compressed report. **Note:** The file name must be the last argument in the list.

Examples

- The following command shows the values of the specified properties:

```
report_property  [get_pins E_C517_dgt_2_/Z]  -property_list  {name direction clocks}

property           | value
---------------------------
name               | Z
direction          | out
clocks             | {}
```

- The following command shows the values of the specified cell instances:

```
report_property [get_cells DT_MUX_INST/E_C17_23]

property                              | value
-------------------------------------------------
area                                  | 0.532
clock_gating_integrated_cell          |
early_cell_check_derate_factor        | 1.000
early_cell_check_mean_derate_factor   | 1.000
early_cell_check_sigma_derate_factor  | 1.000
early_clk_cell_derate_factor          | 0.900
early_clk_cell_mean_derate_factor     | 0.900
```

Related Commands

- get_property
- report_object_property_names
- set_property
- define_property
- reset_property

sizeof_collection

```
sizeof_collection
[-help]
collection
```

Returns the total number of objects contained in the specified collection. If the collection contains different types of objects, `sizeof_collection` returns the sum of all the objects of different types in the collection.

Parameters

`-help`	Prints out the command usage.
`collection`	Specifies the collection.

sort_collection

```
sort_collection
[-help]
collection
[-dictionary]
{property_or_list_of_properties}
[-descending]
```

Returns a sorted collection of objects based on the property.

Parameters

`-help`	Prints out the command usage.
collection	Specifies the collection to sort. The collection must be a homogenous collection of any one of the following object types: `cell`, `clock`, `design`, `lib`, `lib_cell`, `lib_pin`, `lib_timing_arc`, `net`, `path_group`, `pin`, `port`, `timing_arc`, `timing_path`, or `timing_point`.
`-descending`	Sorts the objects in the collection by descending order. *Default:* Sorts the collection in ascending order.
`-dictionary`	Sorts the objects in a collection by dictionary order.
property | *list_of_properties*	Sorts the collection by the specified property. You can specify any property associated with the object type. You can specify one or more properties. If you specify more than one property, the software sorts the collection in the order by which you specified the properties.

Examples

- The following command returns a collection of all the input ports in the design sorted on the basis of their hierarchical names:

```
tempus > sort_collection [all_inputs] hierarchical_name

CLK1 CLK2 CLK3 CLK4 IN1 IN2 IN3 IN4
```

parse_proc_arguments

```
parse_proc_arguments
-help
-args string
result_array
[forced_proc]
```

Supports argument validation within a Tcl procedure and enables the use of the `-help` option to view the help information associated with the procedure.

Procedures using the `parse_proc_arguments` command will validate the arguments. If the required arguments are missing or have invalid values, the `parse_proc_arguments` command generates a Tcl error and the calling procedure terminates.

The `parse_proc_arguments` command must be the first command called in the procedure. This command cannot be used outside of a procedure.

Parameters

`-help`	Prints out the command usage.
`-args string`	Specifies the list of arguments passed to the Tcl procedure.
result_array	Specifies the result array into which the parsed arguments are to be stored.
forced_proc	Specifies the name of the invoking procedure.

Examples

The following Tcl procedure uses the `parse_proc_arguments` command for argument validation.

```
proc cmdProc {args} {

parse_proc_arguments -args $args results

foreach argname [array names results] {

puts "$argname = $results($argname)"

}

}

define_proc_arguments cmdProc -info "Command Description" \

-define_args \

{{-string "String help" "string_val" string optional}

{-integer "Int help" "int_value" int optional}

{-list "List help" "list_value" list {optional bind_option -Bool}}

{-boolean_a "BoolA help" "" boolean {optional mutual_exclusive_group g0}}

{-boolean_b "BoolB help" "" boolean {optional mutual_exclusive_group g0}}

{-float_num "Float help" "float_value" float}

{-oosa "One of StringA help" "one_of_str_value" one_of_string {optional value_help {values {v1 v2}}}}

{-oosb "One of StringB help" "one_of_str_value" one_of_string {optional {values {v4 v5}}}}

}
```

Now, if the cmdProc procedure is invoked using the following command at the command prompt, argument validation takes place and the following errors are reported.

```
tempus> cmdProc -boolean_a -boolean_b

**ERROR: (Tempus-39): Options '-boolean_a -boolean_b' are mutually exclusive for 'cmdProc'.

Usage: cmdProc [-string string_val] [-integer int_value] [-list list_value] [-boolean_a ]

[-boolean_b ] [-float_num float_value] [-oosa one_of_str_value {v1 v2 }] [-oosb one_of_str_value]]

**ERROR: (Tempus-27): Errors in 'cmdProc'.
```

The following command to invoke the cmdProc procedure will also return the errors shown below.

```
tempus> cmdProc -oosa v5
```

```
**ERROR: (Tempus-42): The arguments specified (v5) in cmdProc -oosa are not valid. Refer to the documentation for valid arguments.
Usage: cmdProc [-string string_val] [-integer int_value] [-list list_value] [-boolean_a ]
[-boolean_b ] [-float_num float_value] [-oosa one_of_str_value {v1 v2 }] [-oosb one_of_str_value]]
**ERROR: (Tempus-27): Errors in 'cmdProc'.
```

define_proc_arguments

```
define_proc_arguments
[-help]
proc_name
[-info info_text]
[-define_args arg_defs]
[-hide]
```

Defines the arguments and attributes of a Tcl procedure. The arguments and attributes are displayed when you view the help associated with the procedure.

The `define_proc_arguments` command, when used with the `parse_proc_arguments` command, enables you to validate command usage. For invalid commands, appropriate error messages are displayed.

Parameters

`-help`	Prints out the command usage.
`proc_name`	Specifies the name of an existing Tcl procedure.
`-info` *`info_text`*	Provides help information for the procedure.
`-define_args` *`arg_defs`*	Defines all possible arguments of the Tcl procedure. It can also be used to define the help text and constraints for individual arguments of the procedure and generates the help for command options. The variable *`arg_defs`* is a list of lists where each list element defines one argument. Each element in the list has the following format: `"arg_name" "option_help" "value_help" "data_type" "attributes"`
	`arg_name`: Specifies the name of the argument.
	`option_help`: Provides a short description of the argument.
	`value_help`: Specifies the help information for the associated option values.
	`data_type:` (Optional) Used for validation of options. Supported data types are `string`, `list`, `boolean`, `int`, `integer`, `float`, or `one_of_string`. *Default:* `string`
	`attributes:` (Optional) Used for validation of options. The attributes can be: • `required`: This argument must be specified . This is mutually exclusive with `optional` and `internal`. • `optional`: Specifying this argument is optional. This is mutually exclusive with `required`. • `internal`: Disables the display of help information for the option in the procedure help. This argument is mutually exclusive with `required`. • `value_help`: Indicates that the valid values for `one_of_string` argument should be listed whenever argument help is shown. • `values {`*`allowable_values`*`}`: This argument must be used with the `one_of_string` data type. • `merge_duplicates` : Ensures that when the option appears more than once in a command, its values are concatenated into a list of values. The default behavior is that the right-most value specified for the option is used. • `bind_option` *`other_arg_name`*: Binds the current option with the option specified as the *`other_arg_name`* variable. • `mutual_exclusive_group` *`group_name`*: Defines a group name and includes the current option in the *`group_name`* group. All options in the same group are treated as mutually exclusive. • *Default:* `optional`
`-hide`	Hides the procedure and its body.

Examples

The following Tcl procedure uses the `define_proc_arguments` command for defining arguments and attributes, and for validating the command usage.

```
proc cmdProc {args} {
parse_proc_arguments -args $args results
foreach argname [array names results] {
puts "$argname = $results($argname)"
}
}
define_proc_arguments cmdProc -info "Command Description" \
-define_args \
{{-string "String help" "string_val" string optional}
{-integer "Int help" "int_value" int optional}
{-list "List help" "list_value" list {optional bind_option -Bool}}
{-boolean_a "BoolA help" "" boolean {optional mutual_exclusive_group
g0}}
{-boolean_b "BoolB help" "" boolean {optional mutual_exclusive_group
g0}}
{-float_num "Float help" "float_value" float}
{-oosa "One of StringA help" "one_of_str_value" one_of_string
{optional value_help {values {v1 v2}}}}
{-oosb "One of StringB help" "one_of_str_value" one_of_string
{optional {values {v4 v5}}}}
}
```

Now, if the cmdProc procedure is invoked with the `-help` option at the command prompt, the following help information is displayed.

```
tempus> cmdProc -help
Usage: cmdProc # Command Description
[-string string_val] (String help)
[-integer int_value] (Int help)
[-list list_value] (List help, Bind_Option=-Bool)
[-boolean_a ] (BoolA help, Mutual_Exclusive_Group=g0)
[-boolean_b ] (BoolB help, Mutual_Exclusive_Group=g0)
[-float_num float_value] (Float help)
[-oosa one_of_str_value] (One of StringA help, Values={v1 v2 })
[-oosb one_of_str_value] (One of StringB help)
```

is_collection_object

```
is_collection_object
[-help]
collection
```

Returns a value of `true` if the specified object is a collection, otherwise `false` is returned.

Parameters

`-help`	Prints out the command usage.
`collection`	Specifies the collection to be checked.

Examples

- The following example shows how the `is_collection_object` command can be used to check for collection objects:

```
set cells [get_cells]
set pins [get_pins]
set nets [get_nets]
set paths [report_timing -max_paths 5 -coll]
set heterogenColl ""
append_to_collection heterogenColl $cells
append_to_collection heterogenColl $pins
append_to_collection heterogenColl $nets
append_to_collection heterogenColl $paths
set name "123"

'1' is returned for all of the following 'is_collection_object' commands
  is_collection_object $cells
  is_collection_object $pins
  is_collection_object $nets
  is_collection_object $paths
  is_collection_object $heterogenColl

'0' is returned for these 'all is_collection_object' commands
  is_collection_object $name
  is_collection_object [get_cells xyz* -quiet]
```

Related Commands

- `sizeof_collection`

2

ART Based Flow Commands

- get_module_view
- set_module_view

get_module_view

```
get_module_view
[-help]
[-top_read_only]
[-hinsts list_of_partition_hinsts]
[-partitions list_of_partitions]
[-tcl_list]
```

Verifies the FlexView mode.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_module_view` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_module_view`
`-hinsts` *`list_of_partition_hinsts`*	Lists partition hinsts.
`-partitions` *`list_of_partitions`*	Lists partitions.
`-top_read_only`	Specifies if top level is readOnly.
`-tcl_list`	Displays the current settings for the specified parameters in Tcl list format only. If you specify `-tcl_list` without any parameters, the software displays the current settings of all `set_module_view` parameters in Tcl list format.

Examples

- The following command verifies the FlexView mode:

```
get_module_view

{TOP all}
{coreinst/ks_core1/amba_dsp1/ram2p_78kx32 readOnly}
{coreinst/ks_core1/leon1 interface}
```

Related Information

- `set_module_view`

set_module_view

```
set_module_view
[-help]
[-top_read_only {true | false}]
[-hinsts {list_of_partition_hinsts}]
[-partitions {list_of_paritions}]
[-type {readOnly|interface|all}]
```

Specifies the FlexView `(readOnly|interface|all)` for the top level and each of the partitions. This way optimization can be limited to the selected sections of the partitions and the top-level logic.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_module_view` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_module_view`.
`-top_read_only {true | false}`	Sets the top level to the read-only mode. By default, optimization can change the logic at the top level to improve timing. By setting this option to `true`, no change will be done to the entire top-level logic. *Default*: `false`
`-hinsts` *`list_of_partition_hinsts`*	Applies the selected FlexView mode to the instance name or list of instance names of partitions.
`-partitions` *`list_of_partitions`*	Applies the selected FlexView mode to a partition or list of partitions. You have a choice to either specify the instance name of the partition or just the partition itself.
`-type {readOnly|interface|all}`	Specifies the FlexView mode for the specified partition(s). • `readOnly` : Specifies that the optimization will not be performed inside the partition. • `interface`: Specifies that optimization will be restricted only to the interface logic of the partition. • `all` : Specifies that optimization will be done for the entire partition.

Example

The following commands sets the top level to the read-only mode:

```
set_module_view -top_read_only true
```

Related Information

- get_module_view

3

Basic Database Access Tcl Commands and Attributes

- add_thru_substrate_insts
- compress_special
- convert_dbu_to_um
- convert_um_to_dbu
- create_cell_obs
- db_hinst_connection
- decompress_special
- define_attribute
- delete_cell_obs
- delete_floating_nets
- delete_obj
- delete_thru_substrate_insts
- get_computed_shapes
- get_db
- get_db Category Attributes
- get_edge
- get_obj_in_area
- get_rect
- get_selectively_expanded_modules
- get_transform_shapes
- get_via_pillars
- init_hier_cell
- is_attribute
- report_floating_nets
- report_metal_stack
- report_net_wires
- report_via_def
- reset_db
- set_db
- set_db Category Attributes
- set_selectively_expanded_modules
- set_via_pillars

- sort_db
- tech Category Attributes
- uniquify
- write_preserves

add_thru_substrate_insts

```
[-help]
[-driver_thru_substrate_cell cell1]
-load_thru_substrate_cell cell2
-net net
[-redraw]
[-selected]
[-shared_load_thru_substrate_cell]
```

Adds thru-substrate instances to a net to specify two-sided routing for a critical net (backside routing has lower RC than frontside routing). All pins or ports of the net must be on the front side, and as an exception, one backside port is allowed. A thru-substrate instance is a device that tunnels through the substrate to connect frontside routing with backside routing.

The driver thru-substrate instance is placed on top of the driver and a load thru-substrate instance is placed on top of its load. If a load thru-substrate instance drives multiple loads, it is placed in the center of the group of loads. It means that all instances with a pin on the net must be placed, and if the net has ports, they must also be placed before when the thru-substrate instances can be added to the net. Hence, this command creates unallowed placement that needs to be resolved with the `place_detail` command.

When a net has a backside port with everything else on the front side, this command creates one thru-substrate instance placed on top of the backside port, so that the net's frontside routing can be brought to the backside, typically to connect to a backside bump. For this usage, specify the same value for the `-load_thru_substrate_cell` and `-driver_thru_substrate_cell` parameters.

Use this command to create one FTV for a net with multiple FS leaf pin, multiple FS port(s), and/or a single BS port.

ⓘ FTV is placed on the top of the BS port.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `add_thru_substrate_insts` parameter. For a detailed description of the command and all its parameters, use the man command: `man add_thru_substrate_insts`
`-driver_thru_substrate_cell` *cell1*	Specifies the thru-substrate macro/cell to instantiate a thru-substrate instance, whose front pin is to be routed to the driver.
`-load_thru_substrate_cell` *cell2*	Specifies the thru-substrate macro/cell to instantiate a thru-substrate instance(s), whose front pin(s) is to be routed to load(s).
`-net` *net*	Specifies the net to add thru-substrate instances on both sides, front and back.
`-redraw`	Redraws the graphics window.
`-selected`	Groups the loads where each group will have its own load thru-substrate instance. • When the `-selected` and `-load_thru_substrate_cell` parameters are specified, the only driver thru-substrate instance is first created and the selected loads either share one load thru-substrate instance or get their own load thru-substrate instances. Additionally, the unselected loads are driven directly by the net driver. • In case the `-selected` parameter is specified and the `-load_thru_substrate_cell` parameter is not specified, each selected load driven by the net driver is reconnected to a new load thru-substrate instance. This may be shared through the `-shared_load_thru_substrate_cell` parameter, which is driven by the driver thru-substrate instance. This process repeats until the loads are grouped as desired.
`-shared_load_thru_substrate_cell`	Loads share one thru-substrate instance (one thru-substrate instance for each load, if not specified).

Related Topics

`delete_thru_substrate_insts`

compress_special

```
compress_special
[-help]
[-area {x1 y1 x2 y2}]
[-incremental]
[-layers layer+]
[-nets net+]
```

Compresses Power-Grid (PG) based on the specified area, layer, or net.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `compress_special` parameter. For a detailed description of the command and all its parameters, use the man command: `man compress_special`
`-area {x1 y1 x2 y2}`	Specifies the coordinates for the area from which PG needs to be compressed. The `compress_special` command compresses only those special wires and special vias, which exist within the area specified by this parameter. *Default*: All PG structures are compressed.
`-incremental`	Recompresses the PG after editing the PG wires or vias. This parameter can only be specified when the current design is in compressed mode.
`-layers layer+`	Specifies the layers on which PG compression is needed. This parameter requires either LEF layer names or a routing layer index value (for example, `1` for the first routing layer, `2` for the second routing layer, and so on).
`-nets net+`	Specifies the nets from which PG compression is needed. If the specified net does not have any special route on it, it is ignored.

Example

The following command compresses `%s special_wires` and `%s special_vias`:

```
compress_special -area {100 100 120 120} -net VDD -incremental
```

Related Topics

`decompress_special`

convert_dbu_to_um

```
convert_dbu_to_um
[-unit number]
{int | {int_list}}
```

Takes an arbitrary list of ints and returns it in the same form, with each database integer int converted to its user unit floating point equivalent.

Parameters

`-unit` *number*	Specifies the convert factor by which to divide the database integer int. *Value:* One of `100`, `200`, `1000`, `2000`, `10000`, or `20000` *Default:* Uses the LEF `UNITS DATABASE MICRONS` int from the Tempus database technology file, or the OpenAccess equivalent.
{*int* | {*int_list*}}	
	Specifies the int or int list to convert. You can specify a single int, or an arbitrary list of ints, including points, rectangles and coordinates. *Type:* Integer

Examples

- The following command converts the database int `100` into its equivalent user unit int, based on a convert factor of `2000`:

  ```
  convert_dbu_to_um -unit 2000 100
  ```

  The software returns the following information:

  ```
  0.050
  ```

- The following command converts the database ints for the list of numbers {1000 1000 2000 2000} into their equivalent user unit ints, based on a convert factor of `1000`:

  ```
  convert_dbu_to_um -unit 1000 {1000 1000 2000 2000}
  ```

  The software returns the following information for the list of numbers. The list is implied, but not shown.

  ```
  1.000 1.000 2.000 2.000
  ```

Related Commands

`convert_um_to_dbu`

convert_um_to_dbu

```
convert_um_to_dbu
[-unit number]
{double | {double_list}}
```

Takes an arbitrary list of doubles and returns it in the same form, with each user unit floating point double converted to its database integer double equivalent.

Parameters

`-unit` *number*	Specifies the convert factor by which to multiply the user unit double. *Value:* One of `100`, `200`, `1000`, `2000`, `10000`, or `20000` Default: Uses the LEF `UNITS DATABASE MICRONS` double from the Tempus database technology file, or the OpenAccess equivalent.
{*double* | {*double_list*}}	
	Specifies the double or double list to convert. You can specify a single double, or an arbitrary list of doubles, including points, rectangles and coordinates. *Type:* Float

Examples

- The following command converts the user unit double `30` into its equivalent database integer double, based on a convert factor of `2000`:

  ```
  convert_um_to_dbu -unit 2000 30
  ```

  The software returns the following information:

  ```
  60000
  ```

- The following command converts the user unit doubles for the list of numbers {`30 40`} into their equivalent database unit doubles, based on a convert factor of `200`:

  ```
  convert_um_to_dbu -unit 200 {30 40}
  ```

  The software returns the following information for the list of numbers. The list is implied, but not shown.

  ```
  6000 8000
  ```

Related Commands

`convert_dbu_to_um`

create_cell_obs

```
create_cell_obs
[-help]
-cell cell_name_or_pointer
[-ignore_pg_net]
-layer layer_name_or_pointer
[-mask int_value]
{{-rects {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2}...} | -polygon {x1 y1 x2 y2 x3 y3 ...}}
[-spacing value | -design_rule_width value]
```

Helps in adjusting the database. The command works in memory and hence, must be used in each session before other commands are run.
It adds an obstruction geometry to a cell on the specified layer and is helpful when you want to add a temporary cell obstruction without changing the cell's LEF and reloading the Tempus.

Parameters

`-help`	Prints a brief description that includes type and default information for each `create_cell_obs` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_cell_obs`
`-cell cell_name_or_pointer`	Specifies the cell name or pointer.
`-design_rule_width value`	Specifies the design rule width.
`-ignore_pg_net`	Ignores the Power/Ground routing while checking for DRC violations (including shorts) involving the added obs. This parameter is equivalent to `LEF MACRO OBS LAYER # EXCEPTPGNET`.
`-layer layer_name_or_pointer`	Specifies the layer name or pointer.
`-mask int_value`	Specifies the integer value of the mask (within the mask range) for the created OBS. *Default:* `0`, which means that the mask is uncolored.
`-polygon {x1 y1 x2 y2 x3 y3 ...}`	Specifies the polygon shape.
`-rects {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2}...}`	Specifies the Obs rect list for adding obstruction geometries.
`-spacing value`	Specifies the spacing value.

Related Commands

`delete_cell_obs`

db_hinst_connection

```
db_hinst_connection
[-help]
hInst hInst
```

Reports connections between two hinsts.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `db_hinst_connection` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man db_hinst_connection`
`hInst`	Specifies the address of the hierarchical instance.

decompress_special

```
decompress_special
[-help]
[-area {x1 y1 x2 y2}]
[-layers layer+]
[-nets net+]
```

Decompresses Power-Grid (PG) objects within the specified area, layer, or net. This command decompresses only the overlapped part in case of partial overlapping. The rest of the part is regrouped.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `decompress_special` parameter. For a detailed description of the command and all its parameters, use the man command: `man decompress_special`
`-area {x1 y1 x2 y2}`	Specifies the coordinates for the area from which PG needs to be decompressed. The `decompress_special` command decompresses only those special wires and special vias, which exist within the area specified by this parameter. *Default*: All PG structures are decompressed.
`-layers layer+`	Specifies the layers on which PG decompression is needed. This parameter requires either LEF layer names or a routing layer index value (for example, `1` for the first routing layer, `2` for the second routing layer, and so on).
`-nets net+`	Specifies the nets from which PG decompression is needed.

Example

The following command decompresses `%s special_wires` and `%s special_vias`:

```
decompress_special -area {100 100 120 120} -layer Via56
```

Related Topics

`compress_special`

define_attribute

```
define_attribute
[-help]
attribute_name
-obj_type object_type
[-obsolete]
{-alias existing_attribute | {-category category [-data_type {bool | int | double | area | coord | string | point | rect
| line | delay | resistance | capacitance | voltage | in_dir | out_dir | in_file | out_file  | object | object*} [-
enum_values values]]
[-check_function check_function]
[-compute_function compute_function]
[-default default]
[-help_string description]
[-hidden]
[-set_function set_function]
[-skip_in_db]}}
```

Creates a new, user-defined attribute with the specified characteristics.

The attribute value can be reset only if a default value is specified. For example, the following command defines a new instance attribute of integer type with a default value equal to 1:

```
define_attribute myattr -default 1 -obj_type inst -data_type int -help_string "my attribute" -category mycat
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `define_attribute` parameter. For a detailed description of the command and all of its parameters, use the man command: `man define_attribute`.
`-alias existing_attribute`	Makes *`attribute_name`* a hidden alias for *`existing_attribute`*, which must have been defined. This parameter is used along with the `-obj_type` parameter. It could also be used with the `-obsolete` parameter to generate an obsolete warning.
`attribute_name`	Specifies the name of the attribute to be defined.
`-category category`	Specifies the category of the attribute. Categories group attributes that perform similar functions, whereas object types describe where in the design an attribute is valid. You can specify any category name, both new and existing category names are allowed.
`-check_function check_function`	Specifies a previously defined Tcl procedure's name to ensure that the newly defined attribute is valid. The Tcl procedure should be of the form `proc {object value}`.
`-compute_function compute_function`	Specifies a previously defined Tcl procedure's name to get the newly defined attribute's value later (with the `get_attribute` command). The Tcl procedure must be of the form `proc {object}`. When you use this option, the attribute becomes a read-only attribute because its value is always computed.
`-data_type {bool int double area coord string point rect line polygon delay resistance capacitance voltage in_dir out_dir in_file out_file}`	

	Defines the data type of the attribute. Possible data types are basic only, no objects.
`-default` *`default`*	Specifies a default value for the attribute. In case this parameter is not specified while creating an attribute, the default value will be: • `no_value` for simple numeric types such as `int`, `double`, `delay`, `capacitance`, `coord`, and `boolean` • "" (empty string) for all other types, including `enum`, `string`, `in_file`, `out_file`, `in_dir`, `out_dir`, `point`, `rect`, `polygon`, `line`, and `obj_type`
`-enum_values` *`values`*	Specifies a list of all the possible values of the enumerated type. For example, `-enum_values {a b c}`.
`-help_string` *`description`*	Specifies the help text for the attribute.
`-hidden`	Specifies whether the defined attribute is a hidden attribute.
`-obj_type` *`object_type`*	Specifies the object type of the attribute. All valid object types can be found by typing `get_db obj_types .name` at the prompt. The attribute is assigned to the specified type.
`-obsolete`	Specifies if the defined attribute is obsolete.
`-set_function` *`set_function`*	Specifies a previously defined Tcl procedure's name. This option allows you to override user-defined values provided it conforms to the parameters in the Tcl procedure you created. The Tcl procedure should be of the form `proc {object new_value current_value}`. Returns new value assigned to the attribute.
`-skip_in_db`	Prevents the `write_db` command to write out the defined attribute.

Examples

- The following command creates a Tcl procedure, `check_fxn`, and then creates an attribute named test_check. The `-check_function` parameter is specified so that the test_check attribute can be tested for validity when it is specified later:

```
proc check_fxn {obj val} {
+ if {$val >0} { return 0}
+ return 1
+ }
define_attribute test_check1 -obj_type inst -data_type int -category test -help_string "test_check_function" -
check_function  check_fxn
```

  Based on the check_function parameter, the checking criteria attribute value can be set only to a negative value. As a result the following command sets the attribute value to -1:

```
set_db insts .test_check1 -1
```

  but the following command displays an error and keeps the previous value.

```
set_db insts .test_check1 2
```

- The following command creates a Tcl procedure, `set_fxn`, and then creates an attribute named `test_set`. The `-set_function` parameter is specified so that you can change the value of the `test_set` attribute (provided it is valid):

```
{proc set_fxn {obj new_val cur_val} {
+ if {$new_val > $cur_val} {
+ return $new_val
```

```
+ }
+ return $cur_val
+ }
+ }
define_attribute test_set -obj_type root -data_type int -category test -help_string "test set function" -
set_function set_fxn
```

Now, the following command sets the attribute value to 1 because the previous value was undefined

```
set_db test_set 1
```

The following command changes the Attribute value to 4 as the new value is greater than the current value

```
set_db test_set 4
```

But with the following command the value is not changed as the new value is less than the current value (4)

```
set_db test_set 2
```

- The following command uses the compute_function to set attribute values. Note that the attribute becomes read-only if `compute_function` is applied.

```
proc comp_func {obj} { return 5}
define_attribute test_compute -obj_type root -data_type int -category test -help_string "test set function" -
compute_function comp_func
get_db test_compute 5
```

Now, the following command will issue an error as attribute value cannot be changed.

```
set_db test_compute 3
```

Related Commands

- delete_obj
- get_db
- reset_db
- set_db

delete_cell_obs

```
delete_cell_obs
[-help]
-cells {base_cell | design | module}+
{-obs_shapes shape+ | [{-rects {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2}...} | -polygon {x1 y1 x2 y2 x3 y3 ...}] -
layers layer+}
```

Helps in deleting an obstruction geometry for a cell on the specified layer. This command deletes only the obstruction geometry added by the `add_cell_obs` command.

Parameters

`-help`	Prints a brief description that includes type and default information for each `delete_cell_obs` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_cell_obs`
`-cells` *cell_name_or_pointer*	Specifies the cell name or pointer.
`-layers` *layer_name_or_pointer*	Specifies the layer name or pointer.
`-rects {{`*llx1 lly1 urx1 ury1*`} {`*llx2 lly2 urx2 ury2*`}...}`	Specifies the Obs rect list for deleting obstruction geometries.
`-obs_shapes` *shape+*	Specifies the cell obstruction shape pointers. **Note:** This parameter must be used with the `-cells` parameter, and cannot be specified with any other parameter except than `-help`.
`-polygon {`*x1 y1 x2 y2 x3 y3 ...*`}`	Specifies the polygon shape.

Examples

Following is an example of using the `-obs_shapes` parameter for specifying the cell obstruction shape pointer:

- The `BUFX1` cell has multiple obs shapes on different layers:

  ```
  get_db base_cell:BUFX1 .obs_layer_shapes.shapes
  ```

  ```
  shape:0x7f7f50db3b00 shape:0x7f7f50db3ae0 shape:0x7f7f50db3ac0 shape:0x7f7f50db3aa0 shape:0x7f7f50db3a80
  ```

- The following command deletes the first two shapes in the list:

  ```
  delete_cell_obs -obs_shapes {shape:0x7f7f50db3b00 shape:0x7f7f50db3ae0} -cells BUFX1
  ```

Related Commands

`create_cell_obs`

delete_floating_nets

```
delete_floating_nets
```

The `delete_floating_nets` command deletes all the nets with zero terms except for assign nets and p/g nets. It supports zero-term net deletion in multiple instantiated cells.
This command takes no parameters and can be used whenever the design is read in.

After the command is executed, it reports the following:

- Total number of dangling nets `nrDnet`
- Total number of deleted dangling nets `nrDdnet`
- Total number of nets `ntNet`
- Ratio of `nrDdnet` over `nrNet`
- Ratio of `nrDnet` over `nrNet`
- `nrDdnet` over `nrDnet`

Related Commands

`report_floating_nets`

delete_obj

```
delete_obj any_object+
[-help]
```

Deletes one or more existing objects from the database. The command can delete `netlist` objects and physical data like `routing`, `floorplan`, and DFT objects, but not the `timing` objects (like `rc_corner`).

The following `obj_types` can be deleted through this command: `special_wire | special_via | place_blockage | route_blockage | pin_blockage | resize_blockage | text | gui_text | gui_line | gui_rect | gui_polygon | marker | row | bump | bus_guide | pin_guide | pin_group | net_group | group | flow | flow_step | wire | via | patch_wire | virtual_wire | port_shape | attribute | route_type | what_if_wire | what_if_via | hinst | hpin | hport | port | module | inst | net | hnet | sdp | formal_verification_constraint | test_signal | test_clock | test_clock_domain | port_bus | hport_bus`

Parameters

`-help`	Provides a brief description of the command that includes the type and default information for parameters. For a detailed description of the command and all its parameters, use the man command: `man delete_obj`
`any_object+`	Specifies the object or the list of objects to be deleted.

Examples

- The following command deletes all the `place_blockages`:
  ```
  delete_obj [get_db place_blockages]
  ```
- The following command deletes the specified `place_blockage` object:
  ```
  delete_obj place_blockage:0x2b77dfbf0048
  ```
- The following command deletes all the special routes from the VDD net:
  ```
  set my_net [get_db pg_nets VDD]
  delete_obj [get_db $my_net .special_wires]
  delete_obj [get_db $my_net .special_vias]
  ```
- The following command deletes a group of SDPs:
  ```
  delete_obj sdp:DTMF_CHIP/wCardInstGrp_1
  ```

Related Commands

- define_attribute
- get_db
- reset_db
- set_db

delete_thru_substrate_insts

```
delete_thru_substrate_insts
[-help]
{-net net_name}
```

Deletes thru-substrate instances for the specified net. A thru-substrate instance is a device that tunnels through a chip to connect frontside routing with backside routing.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `delete_thru_substrate_insts` parameter. For a detailed description of the command and all its parameters, use the man command: `man delete_thru_substrate_insts`
`-net` *net_name*	Specifies the net whose thru-substrate instances are to be deleted.

Related Topics

`add_thru_substrate_insts`

get_computed_shapes

```
get_computed_shapes
[-help]
[-dbu]
[-step step]
[-output polygon|rect|hrect|area]
shape_list [AND shape_list | ANDNOT shape_list | OR shape_list | XOR shape_list
| INSIDE shape_list | OUTSIDE shape_list | ENCLOSE shape_list | STRADDLE shape_list
| BBOX | HOLES | NOHOLES | MOVE {dx  | SIZE value | SIZEX value | SIZEY value] ...
```

Performs geometric (SIZE and BBOX) and logical (AND, ANDNOT, etc) operations on shape (polygon or a rectangle or a list of rectangles and polygons).

Parameters

`-help`	Prints a brief description that includes type and default information for each `get_computed_shapes` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_computed_shapes`
`-dbu`	Specifies values in database units, specified in integers. All `shape_list` values as well as values for MOVE, SIZE, and step should be using units consistent with the `-dbu` setting. The output units are the same as the input units. *Default:* Specifies values in user units, in microns.
`-step`	Specifies step size if output format is rect, required to convert non-orthogonal shapes into series of rectangles. *Default*: minwidth value of first routing layer. The option is coord, and is optional.
`-output` *polygon*|*rect*|*hrect*|*area*	
	Specifies the output format. • `rect`: Vertical rectangles • `hrect`: Horizontal rectangles • `area`: Total area of final shape *Default:* `rect` (string)
`shape_list`	A polygon or rectangle or a list of polygon and rectangle. • polygon: `{{x y} {x y} {x y} ... }`; • rect: `{x1 y1 x2 y2}` This option is a required list.
`AND` *shape_list*	Is a binary operator that is the intersection of `shape_lists`
`ANDNOT` *shape_list*	Is a binary operator which defines the initial `shape_list` minus `shape_list`
`OR` *shapelist*	Is a binary operator of the union of shapelists
`XOR` *shape_list*	Is a binary operator of the `shape_list`: OR minus AND. It is a string and is optional.

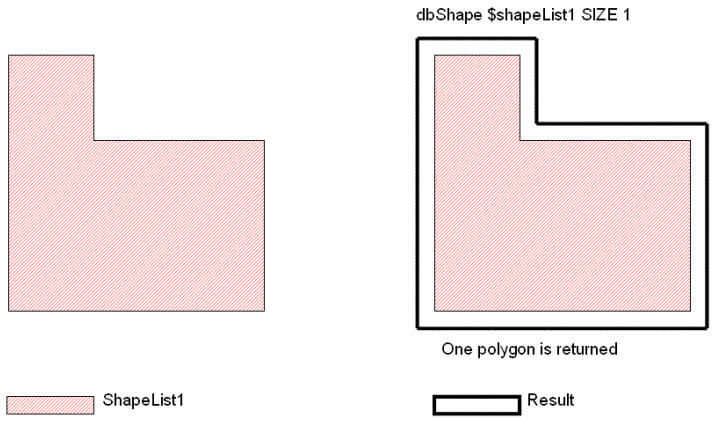
`INSIDE shape_list`	Is a binary operator for initial shapes that are inside (including touching) of one of the shapes from the second list.
`OUTSIDE shape_list`	Is a binary operator for initial shapes that are outside (including touching) of one of the shapes from the second list.
`ENCLOSE shape_list`	Is a binary operator for initial shapes that completely enclose at least one shape from the second list.
`STRADDLE shape_list`	Is a binary operator for initial shapes that are partially inside and partially outside at least one shape from the second list.
`MOVE {dxdy}`	Is a unary operator that moves the by {`dx dy`}
`BBOX`	Is a unary operator that computes the bounding box of `shape_list`. This is a string and optional.
`HOLES`	Is a unary operator that gives list of new shapes that fill holes in the original `shape_list`.
`NOHOLES`	Is a unary operator that gives list of new shapes that include filled holes in the original `shape_list`. This is same as `get_computed_shapes shape_list HOLES` OR `shape_list`
`SIZE value`	Is a unary operator that increases the size of `shape_list` by `value`.
`SIZEX value`	Is a unary operator that increases the size of list of shapes in the X-dbuirection by `value`
`SIZEY value`	Is a unary operator that increases the size of list of shapes in the Y-dbuirection by `value`

Note: All shapes processed by `get_computed_shapes` must be non-zero area. If a *shape_list* contains a list of rectangles and one of them is zero-area, it will be dropped.
Also, during negative sizing or other operations, if a zero-area shape is created, it will be dropped from the output (or intermediate results).

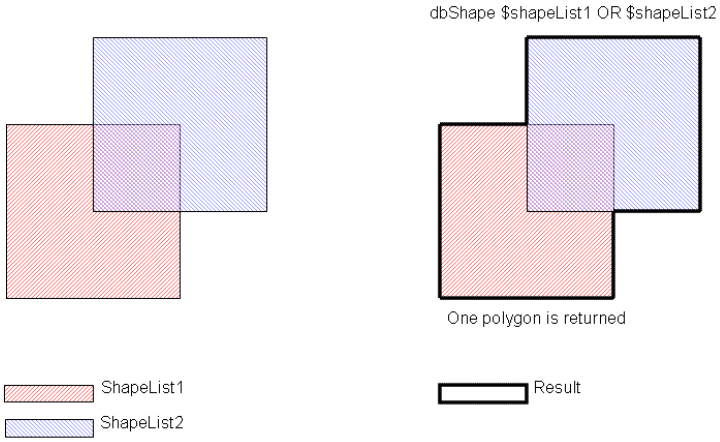
For example, a rectangle that is 4x4 or smaller will disappear when using `get_computed_shapes $rect SIZE -2 SIZE 2` while shapes larger than 4 will be returned to their original state.
The same is true for protrusions with width less than 4.

Examples

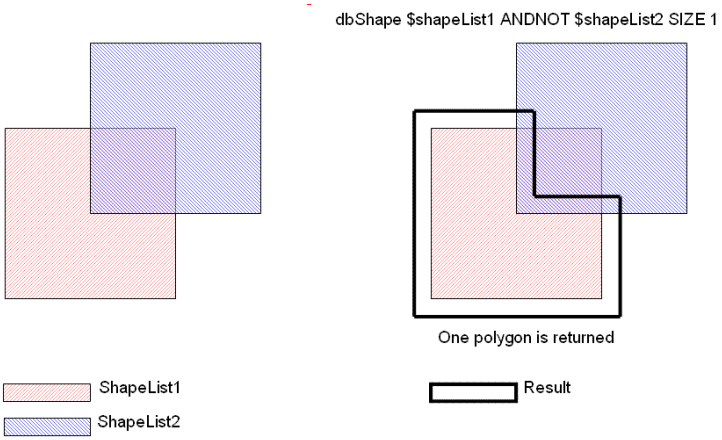
- **Unary**:



- **Binary**:



- **Combination**:



Example: Area calculation

`get_computed_shapes $shapes1 AND $shapes2`
Returns intersection (as a list of rectangles) between the shapes in the `$shapes1` and `$shapes2` lists.

`get_computed_shapes -output area $shapes1 AND $shapes2`
Returns intersection area between the shapes in the `$shapes1` and `$shapes2` lists.

get_db

```
get_db
[-help]
{[start [chain] [pattern1 [pattern2 ..] [[-if expression] | [-expr expression]] [-dbu] [-foreach tcl_body] [-
index {object | obj_type1 name1 [object | obj_type2 name2 …]}] [-invert] [-regexp] [-match_hier] [-unique] [-computed]}
[-category category]
[-depth {[min] max}]
[-quiet | -skip_fail]
```

Returns objects and attributes in the database with different types of filtering. The `get_db` commands takes a dual-ported object, dual-ported object list or a collection as input, then uses a period (.) to traverse to other related objects or to access attributes. You can use additional periods to traverse the data base schema.

Note: A Dual-Ported Object (DPO) is a Tcl_Obj in the C++ programming interface. The pointer is kept as a C++ pointer inside Tcl unless Tcl forces it to be evaluated as a string. In normal usage it never gets converted to a string which is more efficient, but if you do a “puts $var” or return the value to the shell, it will be converted to its “dual” string form.

- The string form is normally a useful name preceded by the `obj_type`, so a `base_cell` object string form might look like `base_cell:and2`.
- A netlist name will have the current design name included, so an instance named `i1/i2` in design top, would look like `inst:top/i1/i2`.
- An object with no name like a wire, will just show the pointer hex-value like `wire:0x22222`.

The `get_db` command accepts collections as an input but it only returns objects in dual-ported object list form.

For descriptions on all objects and attributes, see Common UI Database Object Information.

The root object has the current design data under the `current_design` attribute, while technology and library data are normally directly accessible as root attributes (e.g. layers and base_cells). The `root` object is the default starting point if there is no input list or collection given.

You can query a root attribute in any of the following ways:

- `get_db / .layers`
  where, ‘/’ is the notation for the root object
- `get_db layers`
  This implicitly starts at the root. In this usage, no "." is used before the "layers" attribute name.

For example, you can use the following command to get all the driver-pins of all nets in `current_design`:

```
>get_db current_design .nets.drivers
```

Note: The `current_design` is implicitly created by the `init_design` or the `read_db` commands.

Frequently used current_design attributes like insts or hinsts (hierarchical insts) have root attribute aliases (or shortcuts). These attributes can be accessed by just typing the root attribute alias name.
For example, the following commands gets all instances of the current design with names that start with `i1/i2*`:

- `get_db current_design .insts i1/i2*`
- `get_db insts i1/i2*`
  Where, insts is a root attribute alias for current_design .insts

Some of the more commonly used aliases from the `current_design` attributes include: `nets`, `pins`, `insts`, `hnets`, `hpins`, and `hinsts`.

The `get_db` command supports auto-complete when you press the <`Tab`> key. It will give a list of all matching attributes starting with the letters entered so far, or it will complete the name if it is unique.

For example:

```
get_db insts .par<Tab>
```

```
parent partition
```

If you press the <Shift+Tab> keys together, the matching attributes are displayed along with their help string, like this:

```
>get_db insts .par<Shift+Tab>

parent # The parent of the inst, which is either a hinst for an instance within hierarchy or the design object for an
instance at the toplevel.
partition # If this inst is a physical black_box, this partition will carry the pin constraints and related data for
the blackbox. Otherwise it is empty.
```

There are two database definition objects: `obj_type` and `attribute`. If you want to query all the details for an attribute definition on an `obj_type`, rather than an attribute value, use the following command:

```
>get_db attribute:<obj_type>/<attribute_name> .*
```

You can use `get_db` to query user-defined properties from LEF. Following is the mapping between the LEF `objectType` and database object:

```
LAYER -> layer
LIBRARY -> design
MACRO -> base_cell
NONDEFAULTRULE -> route_rule
PIN -> base_pin
VIA -> via_def
VIARULE -> via_def_rule
```

Note: If `PROPERTYDEFINITION` is defined for `LIBRARY`, the `PROPERTY` will be mapped as a design attribute with a `tech_lef_property_*` prefix.

Note: The Tempus software does not support copy-pasting of text containing pointers. The following should not be copied within the same or different terminal:

- part of a pointer
- pointer from one session to another session
- dynamic object pointer released after a command

Note: For better performance, it is recommended to use the `get_property` and `get_<object>` commands for object search in timing constraints and custom reports.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_db` parameter.

`-category` *category*	Specifies a category of root attributes to return. This is intended to return all the root attributes associated with one application category. You cannot use a *start* argument with the `-category` option, so this option cannot be used for anything but root attributes. The return value is list of root attribute names.
	Example:
	The following command can be used to see all the root attributes associated with the category place:
	`get_db -category place`
	The output is as follows:
	`place_design_floorplan_mode place_design_refine_place place_detail_allow_border_pin_abut ...`
	You can view the complete list of categories with the get_db categories command as follows:
	`get_db categories`
	`active_logic_view add_endcaps add_fillers add_rings ...`

chain	Specifies the *chain* in the form `[.attribute_name1 [.attribute_name2]...]`. A chain always starts with a ".".
	
	The specified attribute names are valid for each level of the traversal in the *chain*.
	
	Example:
	
	The following command returns the names of pins of `$my_insts`
	
	`get_db $my_insts .pins.name`
	
	If the chain has a * or ? in the attribute name, it will be expanded to all matching attribute names, and the display will show the attribute name and current value for every object. This is intended for interactive usage, and no values are returned to Tcl. Values that may require significant time to compute (like timing data that requires the timing-graph) will only be displayed if `-computed` is given.
	
	For example:
	
	`get_db $my_insts .b*`
	
	`Object: inst:top/i1`
	
	`base_cell: base_cell:BUFX2`
	
	`bbox: {34.32 10.08 38.94 15.12}`
	
	`Object: inst:top/i2`
	
	`...`

`-computed`	Forces `get_db` to output the values of attributes that may require significant time to compute (like timing data that requires the timing-graph) when browsing many attributes at the same time. This is intended for interactive usage when using a pattern match with * or ? for an attribute name. If only a single attribute name is given, then the value is always returned.
	
	Example:
	
	`>get_db $my_pins .slack_max*`
	
	`Object: pin:top/inst$odd/Y`
	
	`slack_max: NC`
	
	`slack_max_edge: NC`
	
	`...`
	
	`>get_db $my_pins .slack_max* -computed #if timing-graph not available yet, it will be computed`
	
	`Object: pin:top/inst$odd/Y`
	
	`slack_max: 550.12`
	
	`slack_max_edge: rise`
	
	`...`
	
	`>get_db $my_pins .slack_max #single attribute names always return value and force computation if needed`
	
	`550.12 ...`

`-dbu`	Indicates that attribute values (coord, pt, rect, polygon, area, ...) should be returned as an int in database units.
	
	Default: If not given, the value returned is a double in microns (or microns**2).
	
	Examples:
	• In the following command the return type is a double in microns.
	`get_db $my_inst .location`
	`{1.0 1.0}`
	• In the following command the return type is an int in database units.
	`get_db -dbu $my_inst .location`
	`{1000 1000}`

`-depth {[min] max}`	Specifies the hierarchy depth range. It expands each `design` or `hinst` object in the input list into a list of `design` or `hinst` objects by descending the hinst hierarchy from *`min`* to *`max`* depth. The default value of *`min`* is `0`, if not specified. `0` here signifies the level of the starting object. The `-depth` expansion happens first, and the resulting expanded list is then processed as the input list of objects by `get_db`. For example: `get_db hinst:top/h1 -depth {0 1}` Returns `hinst:top/h1`, and the first level of hinsts below it.

`-expr` *`expression`*	Returns the object if the *`expression`* is true. The *`expression`* argument is a strict Tcl expression that can be passed as is to the Tcl expression command. It has a pre-defined Tcl array named `$obj` that is populated by `get_db` with any attribute or chain value for each object as they are visited, and `$obj(.)` returns the object itself. For example,
	
	`get_db insts -expr {$obj(.base_cell.name) eq "AND2"}  #all insts that are AND2`
	
	Normally the `-if` *`expression`* parameter is faster for simple expressions, but it is limited to a few operators, and simple `$var` substitution. If you need a full Tcl expression semantics including the proc calls inside [], special char escaping, or complex `$var` substitution, you must use `-expr`.
	
	This parameter can be used with a *`chain`* and `pattern` like this:
	
	`get_db $my_insts .pins i1/* -expr {$obj(.base_name) eq "A"}`
	
	Here, the `i1/*` pattern is checked first for pin names that match `i1/*`, then the pin `.base_name` is checked if it is equals "`A`".
	
	`-expr` cannot be used at the same time with `-if`.

`-foreach` *`tcl_body`*	Specifies a list of Tcl commands to process for each object. This parameter avoids the overhead of creating long lists of objects. The Tcl array `$obj()` is populated with the attributes of each object when it is visited, and `$obj(.)` is the object itself. The Tcl commands break and continue also work as expected in a `foreach` loop (break stops the traversal of any more objects and `get_db` returns). Unlike inside `-if`, the `tcl_body` is strict Tcl. The *`.attribute`* names are not substituted, you must use `$obj()` to access the attribute values instead. If you have a *`pattern`* or `-if`, or `-expr` usage, then `-foreach` only processes each `object` that matches the *`pattern`*, `-if`, or `-expr` expression. The following command prints the names of every inst that starts with "abc" and is an AND2 gate: `get_db insts abc* -if {.base_cell.name == AND2} -foreach {puts $obj(.name)}` It returns the total number of `insts` that are printed.

`-if` *expression*	Specifies a boolean expression, with extensions to make pattern matching simpler.
	• == and != do not require a quoted string, and they support pattern matching with * and ?.
	• Other operators like >, <, <=, >=, &&, ||, and the unary negate operator ! are supported.
	• Parentheses "( )" can be used to group complex expressions.
	Note: Unlike the <pattern> argument, inside the -if expression, you cannot use a string pattern to implicitly match the .name attribute of an object. You must use the .name field directly. For example, the following command will not match anything because .base_cell will become something like `base_cell:AND2`.
	`get_db insts -if {.base_cell == buf*} #matches nothing`
	use either of these instead:
	`get_db insts -if {.base_cell.name == buf*} #match the .name attribute`
	`get_db insts -if {.base_cell == base_cell:buf*} #match the DPO name directly`
	Note: If you use an attribute that is an object list (like .pins), every object in the list is tested in an "implicit OR", so if any of them pass, the expression is matched.
	Handling of indexed attributes inside of the `-if` statement:
	• The `-index` option is kept outside of the `-if` statement
	• The `-index` option MUST be valid for ALL attributes inside the `-if` statement otherwise an error occurs
	Examples:
	• The following command finds all instances with `base_cell` names that matches "`buf*`".
	`get_db insts -if {.base_cell.name == buf*}`
	• The following command returns all insts that have one or more pins that match `CLK*`. Only one copy of the matching inst object is returned, even if multiple pin names match CLK*.
	`get_db insts -if {.pins.base_name == CLK*}`
	• The following command gets all pins where `delay_max_rise` is greater than 5 or `slew_max_fall` is smaller than 8. This is a valid command as both delay* and slew* are attributes indexed by `view` (`view` is a legal abbreviation of `analysis_view`).
	`get_db pins -if {.delay_max_rise > 5 || .slew_max_fall < 8 }  -index {view v1}`
	Note: The `-if` parameter does not support the usage of embedded Tcl command as operands. Use the `-expr` for this requirement.

`-index {object | obj_type1 name1 [object | obj_type2 name2 …]}`	Some attributes represent an array of values. For example, a `pin .slack_max_rise` attribute has a different value for each `analysis_view` and each `clock` that can affect that pin. With no index, a default method defined for that attribute is used (for example, the maximum, or minimum of all values in the array). For `slack_max_rise` with no `-index` value, the value returned is the worst-case slack across all `analysis_views` and `clocks`.
	
	If you want to return a specific value, you must include the index value. The index values are object pointers, or a pair of `obj_type` name and object name (for example `clock clk1`). The word `view` is accepted as an abbreviation for `analysis_view`.
	
	For example:
	
	`get_db $my_pins .slack_max_rise -index "view func_wc_cworst"`
	
	Will return the worst case `.slack_max_rise` for the `analysis_view func_wc_cworst` across all clocks. This attribute also accepts a clock index, so you can further refine it like this:
	
	`get_db $my_pins .slack_max_rise -index "view func_wc_cworst clock clk1"`
	
	To get the value for that specific `analysis_view` and `clock`.
	
	In the special case of a clock and `analysis_view` as shown above, a clock object is sufficient for both, because a clock object is based on both the clock name and the view name. So you could also do this:
	
	`set my_clk_obj [get_db clocks -if {.view_name == func_wc_worst && .base_name == clk1}]`
	
	`get_db $my_pins .slack_max_rise -index $my_clk_obj`
	
	The `-index` parameter also accepts a pattern of attribute name to be matched. While specifying an attribute pattern string with `-index`, attributes that do not support `-index`, will return their normal value. But the attributes that support `-index` will return the values specified by the index. For example:
	
	`get_db net:dtmf_recvr_core/scan_out2 .* -index {view func-wc_cworst}`
	
	Will return all the attributes and values for the `net` object. Any attribute that supports a view index will return the value for that view, while the other attributes will return the value as if `-index` was not specified.

`-invert`	Negates the *`pattern`*, `-if` *`expression`*, and `-expr` *`expression`* filters to return the objects/attributes that do not match instead of the ones that do.
`-match_hier`	Specifies that the `*` in the *`pattern`* and *`expression`* expands to the next hierarchical divider character `/` instead of a complete string match. Example: The following command matches i1, i2, but not i1/i2. `get_db insts i* -match_hier`

pattern1 [*pattern2* ..]	Specifies string pattern(s), where:
	* matches 0 or more of any characters
	? matches exactly one character
	More complex patterns require -regexp or -if expression.
	Refer to the following examples:
	• The following command finds all instance names starting with A:
	`get_db insts .name A*`
	`A0 A1/A2 …`
	• The pattern matching can also work directly on a dual-ported object. In that case, it will match the '.name' attribute of the object, but return the object rather than the .name string. This matching works only for objects that have a .name attribute defined. For example, the following command finds all instance objects (not names) with a name starting with A:
	`get_db insts A*`
	`inst:top/A0 inst:top/A1/A2 …`
	• The pattern matching can also work to match multiple lists of the name patterns. Refer to the following example:
	`get_db insts */BC* *TC*`
	`CG/BC1 CG/BC2 TC1 TC2 TC3 TC4`
	Note: For the special case where a pattern string starts with '.' you need to use an extra '.' (an empty attribute name)
	so that '.text' is treated as a pattern and not as an attribute name.
	• The following command finds all nets whose names equal to .odd_name
	`get_db nets . .odd_name`
	• The following command finds all instance names starting with A or B:
	`get_db insts A* B*`

-quiet	Prevents displaying warning and error messages while using the get_db command.
-regexp	Specifies that *pattern* use *regexp* syntax instead of the simple * and ? style. It does not change the -if pattern matching. See the example below to understand the usage of the -regexp parameter: `get_db insts -regexp {u_warmreset_cntr/reset_cntr_reg_(\d+)_}`
-skip_fail	Skips the fail error, when the object does not have such an attribute, or the object/attribute itself cannot be recognized.

start	The starting objects or root attributes. Can be an object list, collection, root (/), or root attribute.
	Example:
	• The following command returns the root attribute `insts` that is an object list of all `insts` in `current_design`.
	`>get_db insts`
	• The following command starts from a dual-ported object list or collection of objects in the Tcl variable `$my_list`, and returns a dual-ported object list.
	`>get_db $my_list`
	• The following command returns a root attribute that is a simple value.
	`>get_db timing_cap_unit`
	• If the name has a * or ?, it will be expanded to all matching root attribute names, and the display will show the attribute names and current value. This is intended for interactive usage, and no values are returned to Tcl. For example,
	`>get_db timing*`
	`Object: root:/`
	`timing_allow_input_delay_on_clock_source: false`
	`timing_analysis_aocv: false`
	`...`

`-unique`	Removes duplicate entries from the return list. If the specified attribute is a list, this parameter returns a unique set of lists., and does not uniquifies the elements within the list. In such a case, use `lsort`.

Built-In Sub-attributes

The types `rect`, `line`, `point`, and `polygon` have built-in sub-attributes. For a full list of all built-in sub-attributes, see Common UI Database Object Information.

Example:

The `rect` type has built-in sub-attributes like:

area	Area of rect
dx	Delta x
dy	Delta y
ll	Lower-left point
ur	Upper-right point
width	Shorter of dx and dy
length	Longer of dx and dy
perimeter	Length of the rect perimeter

A `point` has sub-attributes like:

x	X coordinate
y	Y coordinate

Like other attributes you can chain them together.

For example, an inst object has a .bbox (bounding box) that is type “rect”. Here are some operations allowed on the .bbox attribute of an inst.

```
get_db $my_inst .bbox
{1.0 2.0 3.0 4.0}
get_db $my_inst .bbox.ll
{1.0 2.0}
get_db $my_inst .bbox.area
4.0
get_db $my_inst .bbox.ll.x
1.0
set box [get_db $my_inst .bbox]
{1.0 2.0 3.0 4.0}
get_db $box .ll
{1.0 2.0}
```

Examples

- The following command finds insts with names starting with PTN* that have at least one pin capacitance > 0.1. The pattern is checked first, and those that pass will be tested with the `-if` expression. Inside the `-if` expression, there are many `.pins` for one inst, so each pin is tested. If any of the pins have a capacitance > 0.1, the single inst will be matched.
  ```
  get_db insts PTN2* -if {.pins.capacitance_max_rise  > 0.1}
  ```
- The following command returns insts that are not physical-only cells whose name matches `i1/i2*`.
  ```
  get_db insts i1/i2* -if {!.is_physical}
  ```
- The two commands below generate the same list of pins from `$my_insts` that have a base_name `clk`. The first uses a * pattern to match all pins, while the second has no pattern and defaults to all pins, so both cases check the `-if` expression for all pins of `$my_insts`.

  ```
  get_db $my_insts .pins * -if {.base_name == clk}
  get_db $my_insts .pins  -if {.base_name == clk}
  ```
- The following command finds `buf1` cell instances connected to nets named `clk` and changes their placement status to `fixed`.
  Note: The examples below is inefficient and used to illustrate different methods of traversal done in different styles. A more efficient method would be to start from the nets and looking at their pin connections directly.
  Example 1:

  ```
  set cell_name buf1
  set net_name clk
  foreach inst_obj [get_db insts] {
        set cell_obj [get_db $inst_obj .base_cell]
        if { [get_db $cell_obj .name] == $cell_name } {
                  foreach pin_obj [get_db $inst_obj .pins] {
                        if { [get_db $pin_obj .net.name] == $net_name } {
                            set_db $inst_obj .place_status fixed
                        }
                  }
         }
  }
  ```

  Example 2: Same output using full traversal style with intermediate variables. The second `-if` returns true if any of the .pins.net.name match `$net_name` (implicit OR).

  ```
  set buf1_list [get_db insts -if {.base_cell.name == $cell_name}]
  ```

```
set buf1_clk1_list [get_db $buf1_list -if {.pins.net.name == $net_name}]
set_db $buf1_clk1_list .place_status fixed
```

Example 3: Same output using full traversal style (fully chained)

```
set_db [get_db insts -if {.base_cell.name == $cell_name && .pins.net.name == $net_name}] .place_status fixed
```

Example 4: Or use the `-foreach` method so no list is returned, but the number of buffers processed is returned.

```
get_db insts -if {.base_cell.name == $cell_name && .pins.net.name == $net_name} -foreach {set_db $obj(.)
.place_status fixed}
```

- The following command finds all the insts named `i1/i2` that end with `_buf`:
  ```
  get_db insts i1/i2/*_buf
  ```
  Or your could start inside the `hinst` named `i1/i2`:
  ```
  get_db [get_db hinsts i1/i2] .insts *_buf
  ```
  Or use the hinst dual-ported object name directly. Note that netlist dual-ported object names have the design name included, so if `top` is the design name you could do this:
  ```
  get_db hinst:top/i1/i2 .insts *_buf
  ```
- The following command gets `arrival_max_rise` for the `analysis_view setup_test` for all pins. Here the abbreviation "view" is allowed for analysis_view.
  ```
  get_db pins .arrival_max_rise -index "view setup_test"
  ```
- The following command finds insts that have a cell_port named CK and that pin is connected to a net named clk*:
  ```
  get_db insts -if {.lib_cells.lib_pins.name == */CK && .pins.net.name == clk*}
  ```
- The following command uses -foreach to find the first inst named u1/* with .place_status "fixed" and then break. It uses strict Tcl inside the -foreach.
  ```
  get_db insts u1/* -foreach { if {[string match fixed $obj(.place_status)]} { puts $obj(.name) ; break } }
  ```
- The following `get_db` command uses `-foreach` to count the number of each base_cell used in the netlist inside the cell_count Tcl array. You can then use `array get cell_count` to see a list of all the `base_cell` names and how many times they are used in the netlist.
  ```
  get_db insts .base_cell -foreach {incr cell_count($obj(.name))}
  ```
- The following `get_db` command displays all the details of the `flow_template_type` attribute of the `root obj_type`:
  ```
  get_db attribute:root/flow_template_type .*
  Object: attribute:root/flow_template_type
  base_name: flow_template_type
  category: flow
  check_function: {}
  compute_function: {}
  data_type: string
  default_value: {}
  help: {Type of template being run.}
  indices:
  is_computed: false
  is_obsolete: false
  is_settable: true
  is_user_defined: true
  name: root/flow_template_type
  obj_type: obj_type:root
  set_function: {}
  skip_in_db: false
  ```

- The following example depicts the use of the `-if` parameter to find all HVT insts:

```
get_db insts -if {.base_cell.name == *_HVT}
```

- The following command returns all hinsts 1 or 2 levels inside the design:

```
get_db design:top -depth {1 2}
hinst:top/h1 hinst:top/h1/h2
```

- The following command finds all `local_insts` of each `hinst` at depth 2 that have `.dont_touch == true`:

```
get_db design:top .local_insts -depth {2 2} -if { .dont_touch == true }
inst:top/h1/h2/i1 inst:top/h1/h2/i2
```

- All the following commands return inst with `.name i1/i2`:

```
set a i1/i2
get_db insts i1/i2
get_db insts -expr {$obj(.name) eq $a}
get_db insts -expr {[get_db $obj(.) .name] eq "i1/i2"}
```

- The following command returns all nets with more than 5 loads:

```
get_db nets -expr { [llength $obj(.loads)] > 5 }
```

- The following command returns all insts with `base_cell.name AND2`:

```
get_db insts -expr {$obj(.base_cell.name) eq "AND2"}
```

Related Topics

- `define_attribute`
- `delete_obj`
- `reset_db`
- `set_db`
- For details on LEF property definitions, see the 'Property Definitions' section on the LEF Syntax chapter in the *LEF/DEF Language Reference.*
- For a list of objects attributes that can be queried for information or changed, see Common UI Database Object Information.

get_db Category Attributes

The root attributes in the `get_db` category are:

- get_db_display_limit

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `get_db` category, use the following command:

```
get_db -category get_db
```

get_db_display_limit

`get_db_display_limit {integer}`

Default: `10`

Controls the length of lists and number of objects displayed on the screen using queries that contain wildcards.

Note: This variable does not change the return value of `get_db` command. To control the number of characters in the return value of `get_db` use `tcl_return_display_length_limit`.

Example

The following example sets the number of objects and length of the lists to 1 and then calls `get_db` to display instance attributes. `get_db` command outputs one instance object and every attribute list is truncated to one element.

```
set_db get_db_display_limit 1
get_db insts .*
```

```
Object: inst:TOP/I0
arcs:                   NC
area:                   0.7056
base_cell:              base_cell:BUFFD0BWP
bbox:                   {0.0 0.0 0.0 0.0}
.....
pg_base_pins:           base_pin:BUFFD0BWP/VDD ... (total length 2) #truncated as length of the list is set to 1
pg_pin_nets:            net:TOP/VDD ... (total length 2) #truncated as length of the list is set to 1
pins:                   pin:I0/Z ... (total length 2) #truncated as length of the list is set to 1
...
route_halo_top_layer:   {}
test_reset_inst:        1
```

Note: Number of objects exceeds limit. Use `set_db get_db_display_limit <value>` to control the number of objects evaluated by `get_db <object> .*` query.

Related Topics

- `extract_rc_cap_filter_mode`
- `extract_rc_coupling_cap_threshold`
- `extract_rc_relative_cap_threshold`
- `extract_rc_total_cap_threshold`

get_edge

```
get_edge
[-help]
[-clockwise | -counterclockwise]
[{shape_list} [-direction direction] [-db_units]]
[-index index_number]
```

Returns edges of a polygon of a shape along with the direction list (N, S, E, W and NE, NW, SE, SW). This helps you identify not only all the edges but also the direction in which the edges are pointing. At a time you can observe one polygon with 'n' number of points.

Parameters

`-help`	Prints a brief description that includes type and default information for each `dbedge` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_edge`
`-clockwise`	Sorts the returned list of edges clockwise.
`-counterclockwise`	Sorts the returned list of edges counterclockwise.
`-db_units`	Returns the specified values in database units. Default is `microns`.
`-direction` *direction*	Specifies the types of edges needed. This can be n, s, e, w, ne, nw, se, sw. It can also be a list. For example: `get_edge -direction {n s nw ne} {10 20 30 40}`
`-index` *index_number*	Specifies the index number of the edge to be returned. The index number starts with `0` as the first edge specified in the shape list, and increases gradually along with the shape list. This number may range from `0` to '*number_of_edges* `- 1`'.

`{shape_list}`	Can be a rect like `{llx lly urx ury}` or a polygon like `{{x1 y1}{x2 y2}{x3 y3}.....}`. Only one rect or polygon is accepted.
	Examples
	Using a simple L-shape polygon
	`set shape_list {{0 0} {2 0} {2 1} {1 1} {1 2} {0 2}}`
	• All edges
	`get_edge $shape_list`
	`{{S{0 0}{2 0}} {E{2 0}{2 1}} {N{2 1}{1 1}} {E{1 1}{1 2}} {N{1 2}{0 2}} {W{0 2}{0 0}}}`
	• Only E facing edges
	`get_edge -direction E $shape_list`
	`{{E{2 0} {2 1}} {E{1 1} {1 2}}}`
	The format of the output is
	`{{<direction> <edge_vertex_1> <edge_vertex_2 }....}`

Related Commands

- `get_obj_in_area`
- `get_rect`

get_obj_in_area

```
get_obj_in_area
[-help]
[-areas {{llx1 lly1 urx1 ury1} {llx2 lly2 urx2 ury2} ...}
[-bbox_overlap]
[-dbu]
[-layers {layer1 layer2 ...} [-strict_via_inst_layer_check]]
[-obj_type {bump bus_guide inst inst_all_shapes pin port marker net place_blockage patch_wire pg_pin route_blockage
routes resize_blockage row special special_via special_wire via wire what_if_wire what_if_via port_shape partition}]
[-abut_only] | [-enclosed_only] | [-overlap_only]
[-polygons {x1 y1 x2 y2 x3 y3 ...}]
[-threads]
```

Returns a list of objects based on an area query without needing to select them. This command supports all types of routing blockages such as partial, fill, and slot.

Parameters

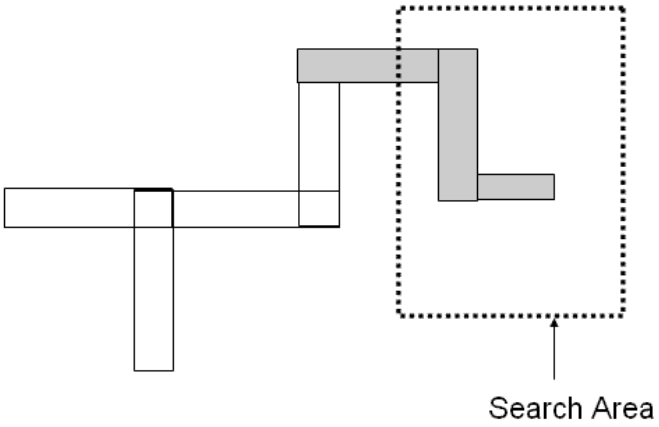
`-help`	Prints a brief description that includes type and default information for each `get_obj_in_area` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_obj_in_area`
`-abut_only`	Returns only those objects that abut the search area. No part of the object polygon overlaps the search area, but at least one point or line of intersection occurs between the object polygon and the search area. A single point of abutment (such as a corner or crossing edges) is also considered as abutted. When multiple areas are specified, the object is returned as long as it abuts one of the search areas.
`-areas {llx lly urx ury}`	
	Specifies the search areas for enclosed and/or intersecting objects. The following are the values in the variable `{llx lly urx ury}` mentioned above: • `ll` = lower left • `ur` = upper right
`-bbox_overlap`	
	Uses the bounding-box of the object rather than the polygon shape of the object for overlap checking.
`-dbu`	Specifies if points are in dbUnit. This is optional. *Default*: um
`-enclosed_only`	
	Returns only those objects, which are completely enclosed inside the search area. The object polygon is fully enclosed by the search area. No part of the object polygon sticks outside the search area, although it may abut the search area edges. When multiple areas are specified, the object will be returned as long as it is enclosed in one of the search areas.
`-layers {layer1 layer2 …}`	
	Returns all the objects with the layer information for the specified layers. This parameter applies only to object types: `port_shape`, `patch_wire`, `route_blockage`, `regular`, `special`, `special_via`, `special_wire`, `via`, `wire`, `what_if_wire`, and `what_if_via`. A `via`, `special_via`, or `what_if_via` object will be returned if any of the via's bottom layer, cut layer, or top layer matches with one of the specified layers. Other object types ignore this parameter and will always be returned.

`-obj_type` *objTypeList*	
	Specifies a list of object types to search for overlaps in the area. Multiple object types can be searched at the same time. The objects currently supported include: `bump bus_guide inst inst_all_shapes pin port port_shape marker net place_blockage patch_wire pg_pin route_blockage routes resize_blockage row special special_via special_wire via wire what_if_wire what_if_via partition` Here: • `routes` is the union of `wire`, `patch_wire`, and `via`. • `special` is the union of `special_via` and `special_wire`. Objects with multiple shapes or non-rectangular shapes use a polygon to check for overlap, as described below. • `bump` has no placement overlap area, so it uses the polygon of all the OBS and PIN shapes. • `inst` uses the polygon of its placement overlap checks (for example the LEF SIZE or OVERLAP shapes), including any placement halo (if there is any) but not any routing halo. OBS shapes or PIN shapes (like standard-cell power-pins) that stick outside the placement overlap area are not included. • `inst_all_shapes` is the same as `inst`, except that it includes all OBS shapes and PIN shapes even if they stick outside the placement overlap area. • `pin` uses the polygon of all the pin-shapes of the `pin`. • `pg_pin` uses the polygon of all the pin-shapes of the `pg_pin`. • `port_shape` uses the polygon of all the shapes of the top-level `port_shape`. • `net` uses the polygon of all the regular routing shapes on the net. Pin-shapes or special-route shapes attached to the net are not included. • `port` uses the polygon of all the pin-shapes of the top-level port. • `port_shape` uses the polygon of all the pin-shapes of the top-level port shape. • `via` uses the polygon of all the shapes of the via. • `special_via` uses the polygon of all the shapes of the via. • `partition` uses the polygon of all the shapes of the partition. *Default*: `inst`
`-overlap_only`	
	Returns only those objects, which are partially overlapped with the search area. Some part of the object polygon overlaps the search area, whereas some of its part is outside the search area. A single point of contact is not considered as an overlap. When multiple areas are specified, the object is returned as long as it overlaps with one of the search areas.
`-polygons {x1 y1 x2 y2 x3 y3 ...}`	
	Specifies the polygon search areas. All the objects overlapping or abutting in the specified area are returned.
`-strict_via_inst_layer_check`	
	Returns only those objects on the specified layer, which match with one of the via's bottom, cut, or top layers. This parameter does not return the objects of the via on the unmatched layers. It works only for `special_via` and `via`.

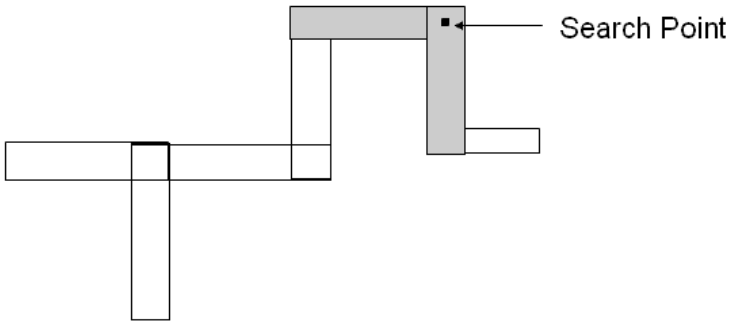
`-threads`	Queries objects in multiple-threads mode. Use this parameter only when multiple areas are specified and too much time is costed. This parameter doesn't work in multiple-threads mode while querying special objects, such as `special_via` and `special_wire`.

Examples

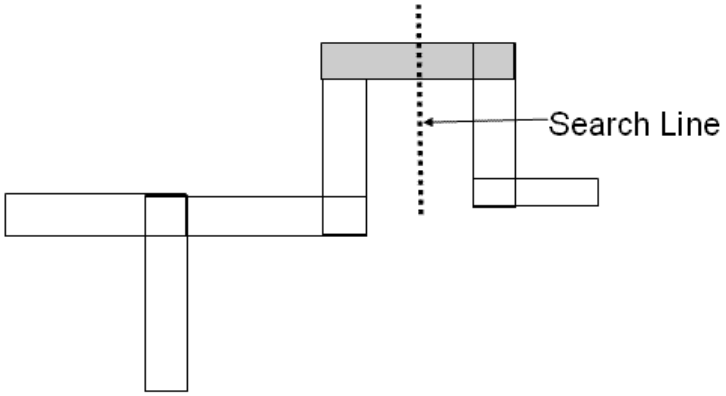
- The option `-areas` can be used as follows:
  - simple: `-areas`
    Normal use with the area defines a box. Objects that are enclosed or intersect the box are returned.



  - point select using: `-areas`
    `-areas {x y x y}`
    Same `x` and `y` for `llx`:
    `lly` and `urx:ury`
    Objects that have bounding boxes that cover the point are returned.



  - Line select using: `-areas`
    `-areas {x1 y x2 y}`
    or
    `-areas {x y1 x y2}`
    Objects that cross the line are returned.



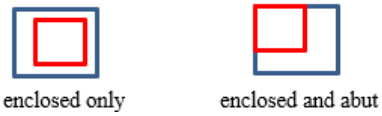
- The following command returns all the instances and routing-blockages on the `metal6` and `metal7` layers that are partially overlapped, abutted, or enclosed in the given triangle area:
  ```
  get_obj_in_area -obj_type {inst route_blockage} -polygons {100 100 110 110 100 110} -layers {Metal6 Metal7}
  ```
- The following command returns all the instances and routing-blockages on the `metal6` and `metal7` layers that are partially overlapped, abutted, or enclosed in the given triangle area using the bounding-box of the objects:
  ```
  get_obj_in_area -obj_type {inst route_blockage} -polygons {100 100 110 110 100 110} -layers {Metal6 Metal7} -bbox_overlap
  ```
- The following command returns all the instances and regular wires on the `metal6` and `metal7`layers that are abutted in the give search areas:
  ```
  get_obj_in_area -obj_type {inst wire} -areas {{0 0 100 100} {150 100 200 200}} -layers {Metal6 Metal7} -abut_only
  ```
- The following command returns all the instances and regular wires on the `metal6` and `metal7`layers that are abutted in the given search areas using the bounding-box of the objects:
  ```
  get_obj_in_area -obj_type {inst wire} -areas {{0 0 100 100} {150 100 200 200}} -layers {Metal6 Metal7} -abut_only -bbox_overlap
  ```
- Following are the figures to illustrate each position relation. The instances presented by the red rectangles are returned with the specified parameter when the search area presented in the blue rectangle are specified.
  - **`-abut_only`**

  - **`-overlap_only`**

  - **`-enclosed_only`**

Note:

1. If none of the above three parameters is specified, all the instances presented by the red rectangles will be returned, by default.
2. For querying object with multi-shape (such as `port_shape`) in multiple search areas, if one of its shapes is enclosed/abutted/overlapped with any one of the specified areas, the object will be returned.

Related Commands

- `get_edge`
- `get_rect`

get_rect

```
get_rect
[-help]
<rect> {-ll | -ur | -llx | -lly | -urx | -ury | -dx | -dy | -width | -length | -center | -area | -size}
```

Returns the coordinates of the points of a rectangle. When you need to find the coordinates of a rectangle, in order to either move the rectangle or reshape the rectangle, the command helps you find the coordinates.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_rect` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_rect`.
`<rect> {-ll | -ur | -llx | -lly | -urx | -ury | -dx | -dy | -width | -length | -center | -area | -size}`	
	Specified rectangle to access values from. • `-area`: Returns the area of the rectangle. • `-center`: Returns the center of the rectangle. • `-length`: Returns the larger of the `-dx` and `-dy` values. • `-ll`: Returns the lower left point of the rectangle. • `-llx`: Returns the lower left x coordinate of the rectangle. • `-lly`: Returns the lower left y coordinate of the rectangle. • `-dx`: Returns the x difference between the larger and smaller x values of the rectangle. • `-dy`: Returns the y difference between the larger and smaller y values of the rectangle. • `-ur`: Returns the upper right point of the rectangle. • `-urx`: Returns the upper right x coordinate of the rectangle. • `-ury`: Returns the upper right y coordinate of the rectangle. • `-size`: Returns the size of the rectangle from lower left to upper right. • `-width`: Returns the smaller of the `-dx` and `-dy` values.

Examples

- `-area {8.1 9.2 10.3 11.4} returns 4.84`
- `-center {8.1 9.2 10.3 11.4} returns 9.2 10.3`
- `-length`: `get_rect -length {8.1 9.2 10.3 11.4}` returns: `2.2`
- `-ll`: `get_rect -ll {8.1 9.2 10.3 11.4}` returns: `8.1 9.2`
- `-llx`: `get_rect -llx {8.1 9.2 10.3 11.4}` returns: `8.1`
- `-lly`: `get_rect -lly {8.1 9.2 10.3 11.4}` returns: `9.2`
- `-size`: `get_rect -size {8.1 9.2 10.3 11.4}` returns: `2.2 2.2`
- `-dx`: `get_rect -dx {8.1 9.2 10.3 11.9}` returns: `2.2`
- `-dy`: `get_rect -dy {8.1 9.2 10.3 11.9}` returns: `2.7`
- `-ur`: `get_rect -ur {8.1 9.2 10.3 11.4}` returns: `10.3 11.4`
- `-urx`: `get_rect -urx {8.1 9.2 10.3 11.4}` returns: `10.3`
- `-ury`: `get_rect -ury {8.1 9.2 10.3 11.4}` returns: `11.4`
- `-width`: `get_rect -width {8.1 9.2 10.3 11.4}` returns: `2.2`

Related Commands

- `get_edge`
- `get_obj_in_area`

get_selectively_expanded_modules

```
  get_selectively_expanded_modules
[-help]
```

Displays the values set by the `set_selectively_expanded_modules` command for abstracting cloned instances of modules with a high number of instantiations, and flattening one or more of them.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `get_selectively_expanded_modules` parameter. For a detailed description of the command and all its parameters, use the man command: `man get_selectively_expanded_modules`

Related Topics

`set_selectively_expanded_modules`

get_transform_shapes

```
get_transform_shapes
[-help]
[-db_units]{-inst instPtr | -hier_inst instPtr | {-cell cellPtr -orient orientEnum -pt {x y}}}
{-local_pt list | -global_pt list}
```

Takes coordinates local to a cell and returns them in context with the global design space. You can specify a specific instance of a cell in the design, or you can specify a library cell, its location in the design, and instruct the software to perform a "what-if" translation of coordinates within that cell.

Parameters

-help	Pints a brief description that includes type and default information for each `get_transform_shapes` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_transform_shapes`
`-cell` *`cellPtr`*	Specifies a pointer to a library cell. You must specify the `-pt` and `-orient` parameters with `-cell`.
`-db_units`	Specifies values in database units, specified in integers. This is an optional parameter. If you use the `-db_units` option, the Boolean value is 1, so the value will be displayed in database units. If you don't specify the `-db_units` option, the Boolean value is 0, the default. *Default*: Specifies values in user units, in microns
`-global_pt` *`globalPts`*	Specifies the global coordinates to convert into local coordinates.
`-hier_inst`	Specifies the hierarchical instance of a database object. This option is mandatory.
`-inst` *`instPtr`*	Specifies a pointer to a specific instance/bump of a cell in the design.The software derives the coordinates and the orientation for the cell instance from the design.
`-local_pt` *`list`*	Specifies the local coordinates to convert into global coordinates
`-orient` *`orientEnum`*	Specifies the orientation of the cell specified with `-cell`. You must specify the `-cell` and `-pt` parameters with `-orient`. Valid orientation values are: `r0`, `r90`, `r180`, `r270`, `mx`, `mx90`, `my`, and `my90`.
`-pt` *`x y`*	Specifies the coordinates for the cell specified with `-cell`. You must specify the `-cell` and `-orient` parameters with `-pt`.

Examples

- Assume that a shape exists on *M1* within a NAND2 cell, from (`1 1`) to (`2 2`). If an instance of the NAND2 cell is placed at (`10 10`), with an `r0` (N) orientation, the following command converts the local points (`1 1 2 2`) of the NAND2 cell into their global coordinates:

  ```
  get_transform_shapes -cell [dbGet -p head.allCells.name NAND2] -pt {10 10} -orient r0 -local_pt {1 1 2 2}
  ```

  The software returns the following information:

  ```
  {11 11 12 12}
  ```

- Assume that the NAND2 cell placed at (`10 10`), with an `r0` (N) orientation is named `i1`. The following command converts the local

points (`1 1 2 2`) of `i1` into their equivalent global coordinates:

```
get_transform_shapes -inst [dbGet -p top.insts.name i1] -local_pt {1 1 2 2}
```

The software returns the following information:

```
{11 11 12 12}
```

get_via_pillars

```
get_via_pillars
[-help]
{-base_pin {base_pin | pg_base_pin} | -pin {pin | pg_pin}}
[-required]
```

The command returns a via pillar list on a cell pin.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `get_via_pillars` parameter. For a detailed description of the command and all its parameters, use the man command: `man get_via_pillars`
`-base_pin {base_pin | pg_base_pin}`	Specifies the base pin name or pointer.
`-pin {pin | pg_pin}`	Specifies the pin name or pointer.
`-required`	Returns the required flag.

Related Information

`set_via_pillars`

init_hier_cell

```
init_hier_cell
[-help]
cell_names
[-dual_view]
```

Initializes one or more hierarchical cells based on the applicable order.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `init_hier_cell` parameter. For a detailed description of the command and all its parameters, use the man command: `man init_hier_cell`
`cell_names`	Specifies the names of the hierarchical cells to be initialized.
`-dual_view`	Binds the top and global views, and initializes the hierarchical cells inside ILMs.

is_attribute

```
is_attribute
[-help]
attribute_name
-obj_type obj_type
```

The `is_attribute` command checks whether attribute exists or not.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `is_attribute` parameter. For a detailed description of the command and all of its parameters, use the man command: `man is_attribute`.
`attribute_name`	Specifies the name of the attribute to be checked.
`-obj_type` *`obj_type`*	Specifies the object type of the attribute. All valid object types can be found by typing `get_db object_types .name` at the prompt. The attribute is assigned to the specified type.

Examples

- The following `is_attribute` command applies check to existing attribute:

```
define_attribute test_delete -obj_type inst -data_type int -category test -help_string "test delete function" -
default 1
is_attribute -obj_type inst test_delete
1
```

- The following `is_attribute` command applies check to non-existent attribute:

```
is_attribute -obj_type pins test_delete
0
```

Related Topics

- For information on how to query DB attributes, see the get_db command.
- For a list of objects attributes that can be queried for information or changed, see the Common UI Database Object Information document.

report_floating_nets

```
 report_floating_nets
[-help]
[-out_file file_name]
```

Reports the number of nets with zero terms, including the PG nets. This command can also be used to report the number of dangling nets, which could or could not be removed by the `delete_floating_nets` command. The command reports the following information, when used:

- Total number of nets
- Number of dangling nets
- Number of removable dangling nets
- Number of non-removable dangling nets

The removable and non-removable nets are reported based on the usage of the `delete_floating_nets` command. Those dangling nets, which could not be removed by the `delete_floating_nets` command, are reported in the output file due to multiple possible reasons, such as the dangling net:

- Is a physical net.
- Is a PG net.
- Is a bus bit net.
- Is an assign net.
- Has multiple module ports and at least one port is associated with some constraints.
- Has some constraints, such as in an ILM module or some timing constraint.

Note: If a dangling net cannot be deleted due to any reason, only the first reason listed above will be reported in the output file.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_floating_nets` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_floating_nets`
`-out_file` *`file_name`*	Reports the list of dangling nets that cannot be deleted by the `delete_floating_nets` command, along with its reasons.

Examples

- The following command reports the information about the nets listed in the `dangling_net.rpt` file:

```
report_floating_nets -out_file dangling_net.rpt
```

- Following is the output of the above command:

```
############################################################
# Generated by: Cadence Tempus 19.10-p002_1
# OS: Linux x86_64(Host ID sjfsb473)
# Generated on: Sat Aug 18 11:14:18 2018
# Design: design_name
# Command: report_floating_nets -out_file dangling_nets.rpt
############################################################

Total number of nets: 123456
The number of dangling nets: 26
Removable dangling nets: 22
Non-removable dangling nets: 4
Is physical net: 2
Is P/G net: 0
Is bus net: 1
Is assign net: 0
Has module port:0
Has constraints: 1


Reasons for the net not being removed:
net:xxxx is physical net ## net:xxxx here is the DPO name of the net
net:xxxx is physical net
net:xxxx is bus net
net:xxxx has constraints
```

Related Commands

`delete_floating_nets`

report_metal_stack

```
report_metal_stack
[-help]
[-non_preferred_direction]
```

Reports the metal stack configuration and routing track information. For each routing layer, this includes the layer name, preferred routing direction, routing offset (offset of routing track from 0), routing pitch (space between each routing track), min-width DRC rule, min-spacing DRC rule, and the routing default-width (wire width of the default routing rule). Note that for a horizontal direction track, the offset and pitch are in the Y direction (for example, the Y offset from 0, and the Y pitch of horizontal tracks). This command only supports uniform track-patterns with a single pitch value, and will not report non-uniform pitch routing track patterns that can be created with `add_tracks` or `create_tracks`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `report_metal_stack` parameters. For a detailed description of the command and all its parameters, use: `man report_metal_stack`
`-non_preferred_direction`	By default, the `report_metal_stack` command displays the metal stack and routing information only related to the preferred routing direction. When the `-non_preferred_direction` parameter is used, the information related to the non-preferred routing direction is also displayed. *Data_type*: bool, optional

Examples

- The following command displays the metal stack and routing information:
  ```
  report_metal_stack
  ```

  The following is an example of the output of the above command:

  ```
  ================================================================================
  Layer   Direction   Offset    Pitch      Min_Spacing  Min_Width   Default_Width
  ================================================================================
  M1      horizontal  0.000000  0.040000   0.020000     0.020000     0.020000
  M2      vertical    0.000000  0.057000   0.020000     0.037000     0.037000
  M3      horizontal  0.000000  0.040000   0.020000     0.020000     0.020000
  M4      vertical    0.000000  0.048000   0.020000     0.028000     0.028000
  ```

- The following command adds in the non-preferred routing direction information:
  ```
  report_metal_stack -non_preferred_direction
  ```

  The following is the output of the above command:

  ```
  ============================================================================================
  Layer  Direction     Preferred Offset    Pitch      Min_Spacing  Min_Width  Default_Width
  ============================================================================================
  M1      horizontal   yes       0.000000  0.040000   0.020000     0.020000     0.020000
  M1      vertical     no        0.000000  0.040000   0.000000     0.000000     0.060000
  M2      vertical     yes       0.000000  0.057000   0.020000     0.037000     0.037000
  M2      horizontal   no        0.000000  0.057000   0.000000     0.000000     0.060000
  M3      horizontal   yes       0.000000  0.040000   0.020000     0.020000     0.020000
  M3      vertical     no        0.000000  0.040000   0.000000     0.000000     0.060000
  M4      vertical     yes       0.000000  0.048000   0.020000     0.028000     0.028000
  M4      horizontal   no        0.000000  0.048000   0.000000     0.000000     0.060000
  ```

report_net_wires

```
report_net_wires
[-help]
[-file fileName]
[-net netName]
```

Displays information about the regular net wires. If the `-net` parameter of this command is not specified, the wires of all the nets are displayed. If the `-file` parameter of this command is not specified,, the wire information on the screen (console) and log file are displayed. The information about the net regular wires is categorized as:

- Node Information
- Edge Information

Node Information

It includes XY location (like `Loc=(2.3, 52.11)`), metal layer (like `Layer=M1(2)`), number of edges on the current node (like `Edges=2`), tie special node name (like `Tie Special`), and global route node name (like `Global Route`). If the node has a terminal attached, the node information also includes the term information, like `Term = a/b/c`, `IO Pin = name`, or `PG Term = x/y/z`.

Edge Information

It includes node number (like `Node3 -> Node5`), edge type (like `Type=HV Wire`), and edge status (like `Status=Routed`). The edge status list includes `None`, `Routed`, `Fixed`, `Cover`, `Shield`, `No Shield`, `Global`, `Assigned`. For different edge types, the edge information reports different edge attributes. Following is the list of various edge types and their attributes:

- `HV Wire:`
  `NDR=name` (non-default rule name)
  `Width=3.3` (wire width)
  `Begin Extension=3.3` (wire beginning extension)
  `End Extension=3.3` (wire ending extension)
  `Mask=1` (color/mask value)
- `Via:`
  `Via Cell=name` (via cell name)
  `NDR=name` (non-default rule name)
  `Top Mask=1` (top metal layer's color/mask value)
  `Cut Mask=1` (cut layer's color/mask value)
  `Bottom Mask=1` (bottom metal layer's color/mask value)
- `Patch:`
  `Box=(100.0,200.0,223.6,250.4)` (patch box coordinates)
  `Mask=1` (patch color/mask value)
  `Attr=304` (patch attributes value)
  If it is a non-routing patch, it will also report trim metal information as shown below:
  `Trim Metal Box=(10.0, 10.0, 11.9, 11.9)` (trim metal box coordinates)
  `Mask=1` (trim metal color/mask value)
  `Layer=name` (trim metal layer name)
- `45 Degree:`
  `NDR=name` (non-default rule name)
  `Width=3.3` (wire width)

`Mask=1` (color/mask value)

- `Zero Length`
- `Term Connect`
- `Virtual`:

`Zero Length`, `Term Connect`, and `Virtual` have no extra attribute information. Following is an example of a single net:

Wires for the `net1` net:

```
Node0: Loc=(44.102, 55.203), Layer=PO(2), Edges=1, Global Route
Term = inst1/b
Node1: Loc=(44.102, 66.304), Layer=Po(2), Edges=1
IO Pin = net1
Edge0: (Node0->Node1), Type=HV Wire, Status=Routed, NDR=ndr1, Width=0.056, Begin Extension=0.056, End
Extension=0.056, Mask=1
```

Parameters

`-help`	Prints a brief description that includes the type and default information for each `report_net_wires` parameter. For a detailed description of the command and all its parameters, use the man command: `man report_net_wires`
`-file` *`fileName`*	Specifies the name of the file.
`-net` *`netName`*	Specifies the name of the net.

report_via_def

```
report_via_def
[-help]
[-detail]
[-on_clock_nets]
[-on_selected_nets]
[-regular]
[-special]
```

Reports single and multi-cut via summary for special and/or regular vias. Such a summary could be used for generating automotive statistics regarding the usage of single and multi-cut used vias.

Parameters

-help	Prints a brief description that includes the type and default information for each `report_via_def` parameter. For a detailed description of the command and all its parameters, use the man command: `man report_via_def`
-detail	Prints the detailed report related to every via def.
-on_clock_nets	If specified, the command reports vias only on the clock nets in the design. If not specified, the command reports vias on all nets in the design by default. This parameter can be used with the `-detail`, `-special`, and `-regular` parameters.
-on_selected_nets	If specified, the command reports vias only on the specified selected nets. If not specified, the command reports vias on all nets in the design by default. This parameter can be used with the `-detail`, `-special`, and `-regular` parameters.
-regular	Prints the report related to regular routing vias.
-special	Prints the report related to special routing vias.

Examples

Use the following command to display a detailed summary of the single and multi-cut vias:

```
report_via_def -detail
```

Following is the sample output of the above command:

```
#-------------------------------------------------------------------------

# V1                1628089 (100.0%)              0 (  0.0%)        1628089

# V2                1685800 ( 99.1%)          15198 (  0.9%)        1700998

# MF                1076133 (100.0%)            131 (  0.0%)        1076264

# Q4                 536537 (100.0%)              0 (  0.0%)         536537

# FD                 250646 ( 71.3%)         101082 ( 28.7%)         351728

# S6                 189504 ( 63.8%)         107464 ( 36.2%)         296968

# S7                 131419 ( 54.8%)         108215 ( 45.2%)         239634
```

```
# S8                  96592 ( 57.5%)          71325 ( 42.5%)         167917

# S9                  64882 ( 50.8%)          62728 ( 49.2%)         127610

# S10                 33187 ( 38.5%)          52936 ( 61.5%)          86123

# S11                 22774 ( 42.7%)          30613 ( 57.3%)          53387

# DI                  12255 (100.0%)              0 (  0.0%)          12255

# II                      0 (  0.0%)           7268 (100.0%)           7268

#-------------------------------------------------------------------------

# Total             5727818 ( 91.1%)         556960 (  8.9%)        6284778
```

reset_db

```
reset_db
[-help]
[start[chain]] |
[-category category_name]]
[-quiet]
[-verbose]
```

Resets attribute value to a pre-defined default value. It returns a single value when resetting a single attribute, a list of values when resetting same attribute on different object instances, or a human readable table for multiple attributes in the following format:

```
Attribute_name   value
```

For example:

```
Place_xxx true
Add_filler  true
```

- The `-category` parameter of the `reset_db` command can be used to reset root attributes only. Both user-defined and system root attributes can be reset using `the -category` parameter.
- The attribute value can be reset if the `default_value` attribute is specified.

When the `-category` parameter is used, the command resets the values of both the hidden and unmapped attributes. In such a case, the command does not return the count of the reset attributes. The output in such cases will be like the following:

```
Some hidden attributes were also reset.
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reset_db` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reset_db`.
start	Can be any object list (or collection) or *shorthand*. For example nets, insts...
chain	Specifies the *chain* in the form `[.attribute_name1 [.attribute_name2]...]`. The specified attribute names are valid for each level of the traversal in the *chain*.
`-category` *category_name*	Specified root attribute category based on predefined list of categories. For example timing. **Note:** Applies only to root level attributes and ignored if applied to lower levels.
`-quiet`	By default `set_db` is quiet. If the `set_db_verbose` root attribute is set to true, then the name of each object that is modified is echoed to the terminal (see the `-verbose` parameter's description for an example). `-quiet` forces `set_db` to be quiet even if`set_db_verbose` is true. This is intended for internal scripts that want to be certain that the verbose output is off.

`-verbose`	Turns on verbose output that echoes the name of each object that is modified. This can be useful during interactive debugging to ensure that the correct objects were affected. For example:
	`set_db -verbose [get_db hinsts D*] .dont_touch false`
	`Setting attribute of hinst 'DATA_BUS_MACH_INST': 'dont_touch' = false`
	`Setting attribute of hinst 'DECODE_INST': 'dont_touch' = false`
	`2 false`
	Note: The `set_db_verbose` root attribute can also be set to true, to force verbose output for all `set_db` and `reset_db` usage without requiring `-verbose`.

Examples

- The following reset_db command resets the value of instance attribute:

```
define_attribute test_reset_inst -obj_type inst -data_type int -category reset -help_string "test reset function"
-default 1
1
set_db insts .test_reset_inst 10
10 10 10 10 10       #Four instances in the design
reset_db insts .test_reset_inst
1 1 1 1 1            #All 4 attributes are reset to default values
```

- The following command resets same set of attributes using the `-category` parameter.

```
reset_db -category reset
```

  The attributes are reset to their default values and a human readable table with new values is displayed.

```
test_reset1 1
test_reset2 1
test_reset3 1
```

Related Topics

- For information on how to query DB attributes, see the `get_db` command.
- For information on how to set DB attributes, see the `set_db` command.
- For a list of objects attributes that can be queried for information or changed, see the Common UI Database Object Information document.
- `set_db Category Attributes`

set_db

```
set_db
[-help]
start
[chain]
value
[-dbu]
[-index {index_type1 name1 [index_type2 name2 …]}]
[-quiet]
[-verbose]
```

Changes the specified attribute value for a database object. The `set_db` command takes a dual-ported object, dual-ported object list, or a collection as input, and then uses a period (.) to traverse to other related objects or to access attributes. You can use additional periods to traverse the data base schema.

This command returns the number of attributes set and the value. See the example below:

```
set_db hinsts .ungroup_ok false
6 false
```

In the above scenario usage, `values == value` as a single command can set only a single value (for multiple objects).

Note: A Dual-Ported Object (DPO) is a `Tcl_Obj` in the C++ programming interface. The pointer is kept as a C++ pointer inside Tcl unless Tcl forces it to be evaluated as a string. In normal usage it never gets converted to a string which is more efficient, but if you do a `puts $var` or return the value to the shell, it will be converted to its dual string form.

- The string form is normally a useful name preceded by the `obj_type`, so a `base_cell` object string form might look like `base_cell:and2`.
- A `netlist` name will have the current design name included, so an instance named `i1/i2` in design top, would look like `inst:top/i1/i2`.
- An object with no name like a wire, will just show the pointer hex-value like `wire:0x22222`.

The `set_db` command does not create or delete objects.

For information on what object attributes are settable, see the Common UI Database Object Information document. It contains a description of all database objects and attributes.

The root object has the current design data under the `current_design` attribute, while technology and library data are normally directly accessible as root attributes (For example, layers and libraries). The root object is the default starting point if there is no input list or collection given.

You can set any root attribute in any of the following ways:

- `set_db / .flow_mail_to name`
  where, '/' is the notation for the root object , or,
- `set_db flow_mail_to name`
  This implicitly starts at the root.

For example, you can use the following command to change the design text font:

```
set_db current_design .texts.font_number  4
```

Note: The `current_design` is implicitly created by the `init_design` or the `read_db` commands.

Frequently used `current_design` attributes like `insts` or `hinsts` (hierarchical insts) have root attribute aliases (or shortcuts). These attributes can be accessed by just typing the root attribute alias name.
For example, the following two commands set all insts .place_status to fixed.

```
set_db current_design .insts.place_status fixed
set_db insts .place_status fixed
```

Some of the more commonly used aliases from the current_design attributes include: insts, hinsts, nets, hnets, ports, hpins, pins.

The `set_db` command supports auto-complete when you press the <Tab> key. It will give a list of all matching attributes starting with the letters entered so far, or it will complete the name if it is unique.

For example:
`set_db plan_design_use<Tab>`
returns `plan_design_use_guide_boundary plan_design_use_sdp_group`

For information on how to access DB objects, see the `get_db` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_db` parameter.
start	The starting objects or attributes. Can be an object list, collection, root (/), or root attribute.
chain	Specifies a *chain* in the form `[.attribute1[.attribute2]...]`. The attribute names must be valid for each level of the traversal in the *chain*, where the first name is valid for the start object. For example, `set_db $my_pins .inst.place_status fixed` where `.inst` is an attribute of an pin object and `.place_status` is an attribute of an inst object.
value	Specifies new value for the specified object attribute elements. The value type depends on which object attribute type is being modified.
`-dbu`	Indicates that attribute values (coord, pt, rect, polygon, area, ...) should be in database units. *Default*: The default is microns (or microns**2).
`-index {`*index_type1 name1* `[`*index_type2 name2* `…]}`	
	Some attributes represent an array of values. For example, a pins .slack_max_rise attribute has a different value for each analysis_view. With no index, all the values defined for that attribute are modified by the `set_db` command. If you want to change a specific value, you must include the index value. The index values are pairs of obj_type names (for example, analysis_view, clock, skew_group) followed by the name. The word “view” is accepted as an abbreviation for “analysis_view”. Example: Some ccopt attributes can take both a view and clock index like this: `-index “view func_wc_cworst clock clk1”`
`-quiet`	By default `set_db` is quiet. If the `set_db_verbose` root attribute is set to true, then the name of each object that is modified is echoed to the terminal (see the `-verbose` parameter's description for an example). `-quiet` forces `set_db` to be quiet even if `set_db_verbose` is true. This is intended for internal scripts that want to be certain that the verbose output is off.

`-verbose`	Turns on verbose output that echoes the name of each object that is modified. This can be useful during interactive debugging to ensure that the correct objects were affected. For example:
	`set_db -verbose [get_db hinsts D*] .dont_touch false`
	`Setting attribute of hinst 'DATA_BUS_MACH_INST': 'dont_touch' = false`
	`Setting attribute of hinst 'DECODE_INST': 'dont_touch' = false`
	`2 false`
	Note: The `set_db_verbose` root attribute can also be set to true, to force verbose output for all `set_db` and `reset_db` usage without requiring `-verbose`.

Examples

- The following command sets a value of root attribute.
  ```
  set_db place_global_solver_effort high
  ```
- The following commands define an instance attribute and change its value to 0.
  ```
  define_attribute myattr -default 1 -obj_type inst -data_type int -help_string "my attribute" -category my_cat
  set_db insts .myattr 0
  ```
- The following command sets an instance attribute value to 5 only if its current value is 0.
  ```
  set_db [get_db insts -if {.myattr == 0}] .myattr 5
  ```

Related Topics

- For information on how to query DB attributes, see the `get_db` command.
- For information on how to reset attribute values, see the `reset_db` command.
- For a list of objects attributes that can be queried for information or changed, see the Common UI Database Object Information document.
- set_db Category Attributes

set_db Category Attributes

The root attribute in the `set_db` category is:

- set_db_verbose

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `get_db` category, use the following command:

```
get_db -category set_db
```

set_db_verbose

```
set_db_verbose {true | false}
```

Default: false

When true, additional messages are displayed describing every object modified by set_db or reset_db commands.

Refer to the following example:

```
set_db set_db_verbose true
set_db [get_db hinsts */D*] .dont_touch true
Setting attribute of hinst 'DATA_BUS_MACH_INST': 'dont_touch' = true
Setting attribute of hinst 'DECODE_INST': 'dont_touch' = true
2 true
set_db set_db_verbose false
Setting attribute of root '/': 'set_db_verbose' = false
1 false
```

```
set_db [get_db hinsts */D*] .dont_touch true
2 true
```

set_selectively_expanded_modules

```
  set_selectively_expanded_modules
[-help]
```

While performing Static Timing Analysis (STA) or Engineering Change Orders (ECO), very large designs often contain modules with a high number of instantiations. Creating RTL-level configurations for smaller portions of such designs is often challenging.

The `set_selectively_expanded_modules` command addresses this issue by specifying modules with instantiations that are to be abstracted, or are not timed, based on the specifications set through the `set_expanded_hinsts` command.

This reduces memory usage and runtime significantly during the STA and ECO processes. It helps in handling large designs on smaller machines and facilitates ECO implementation without the need of manually partition the design into smaller segments.

- This command is recommended exclusively for modules with high instantiation counts and is not recommended to use for single-instance modules.
- All STA views must use consistent flattening specifications. For ECO, the same netlist and flattening specifications must be loaded to ensure alignment.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `set_selectively_expanded_modules` parameter. For a detailed description of the command and all its parameters, use the man command: `man set_selectively_expanded_modules`

Examples

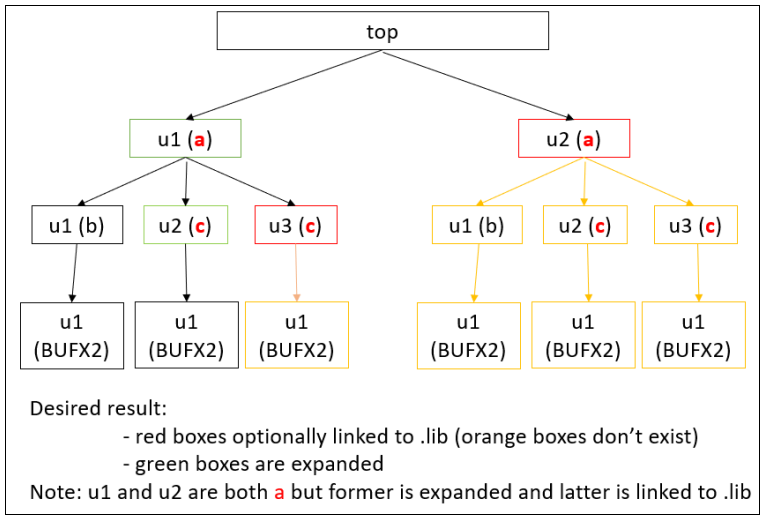
- The following command set presents an example of instantiation of `a` and c modules, which are selectively expanded:

```
read_lib my_lib
read_verilog my_verilog   # instantiations of modules “a” and “c” are to be selectively expanded
set_selectively_expanded_modules “a c”   # expand “u1” and “u1/u2”
set_expanded_hinsts “u1 u1/u2”; # No order requirement: “u1/u2 u1”   is OK, too
set_top_module top
```

Netlist:

```
module top;
  a u1; a u2;
endmodule
module a;
  b u1; c u2; c u3;
endmodule
module b;
  BUFX2 u1;
endmodule
module c;
  BUFX2 u1;
```

```
endmodule
```



Following is the expansion result of the above example:

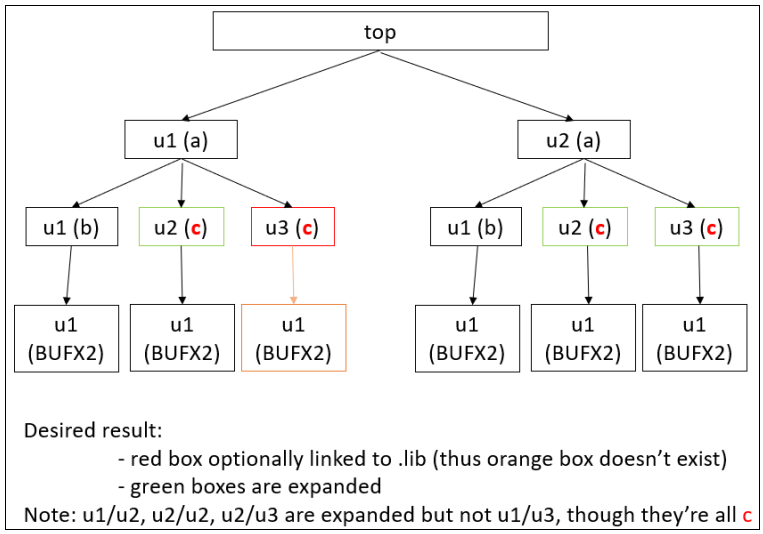
```
COMPONENTS 4 ;
  - u1/u1/u1 BUFX2 ;
  - u1/u2/u1 BUFX2 ;
  - u1/u3 c ;
  - u2 a ;END COMPONENTS
```

- The following command set presents an example of instantiation of c module, which is selectively expanded:

```
read_lib my_lib
read_verilog my_verilog  # instantiations of modules "c" are to be selectively expanded
set_selectively_expanded_modules "c"   # expand "u1/u2", "u2/u2" and "u2/u3"
set_expanded_hinsts "u1/u2 u2/u2 u2/u3"
set_top_module top
```

Netlist:

```
module top;
  a u1; a u2;
endmodule
module a;
  b u1; c u2; c u3;
endmodule
module b;
  BUFX2 u1;
endmodule
module c;
  BUFX2 u1;
endmodule
```



Following is the expansion result of the above example:

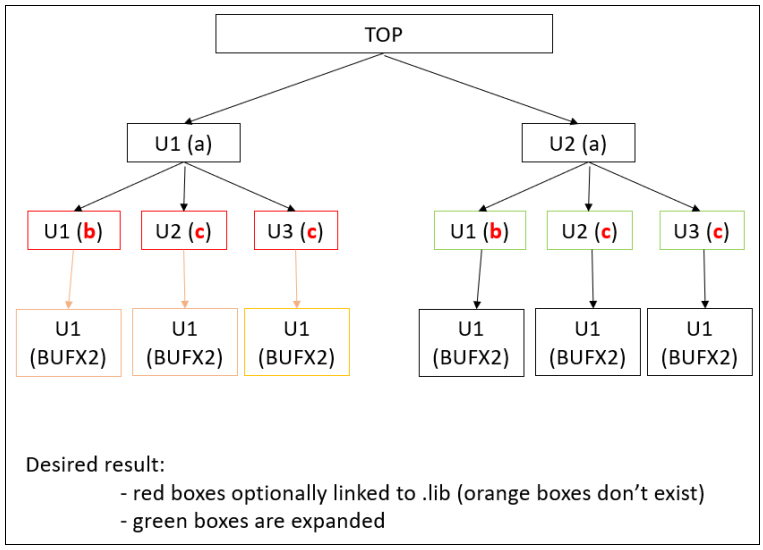
```
COMPONENTS 6 ;
  - u1/u1/u1 BUFX2 ;
  - u1/u2/u1 BUFX2 ;
  - u1/u3 c ;
  - u2/u1/u1 BUFX2 ;
  - u2/u2/u1 BUFX2 ;
  - u2/u3/u1 BUFX2 ;
END COMPONENTS
```

- The following command set presents an example of instantiation of `b` and c modules, which are selectively expanded:

```
read_lib my_lib
read_verilog my_verilog   # instantiations of modules "b" and "c" are to be selectively expanded
set_selectively_expanded_modules "b c"   # expand "u2/u1", "u2/u2" and "u2/u3"
set_expanded_hinsts "u2/u1 u2/u2 u2/u3"
set_top_module top
```

Netlist:

```
module top;
  a u1; a u2;
endmodule
module a;
  b u1; c u2; c u3;
endmodule
module b;
  BUFX2 u1;
endmodule
module c;
  BUFX2 u1;
endmodule
```



Following is the expansion result of the above example:

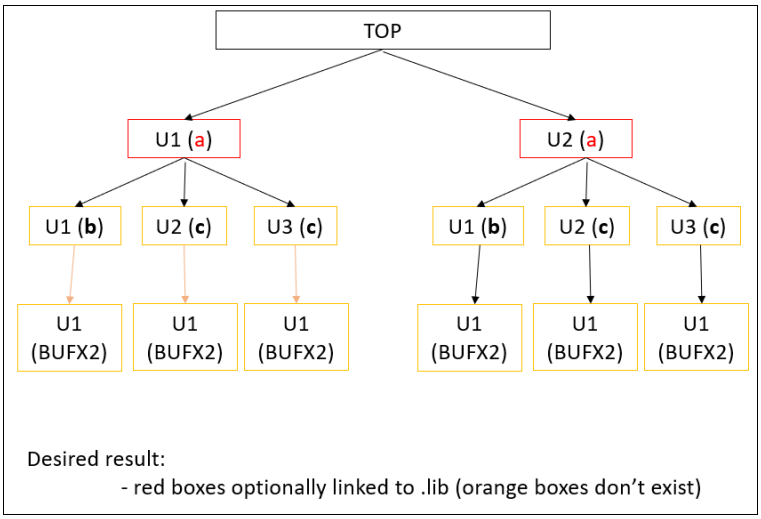
```
COMPONENTS 6 ;
  - u1/u1 b ;
  - u1/u2 c ;
  - u1/u3 c ;
  - u2/u1/u1 BUFX2 ;
  - u2/u2/u1 BUFX2 ;
  - u2/u3/u1 BUFX2 ;
END COMPONENTS
```

- The following command set presents an example of instantiation the a module, which is selectively expanded:

```
read_lib my_lib
read_verilog my_verilog  # instantiations of modules "a" are to be selectively expanded
set_selectively_expanded_modules "a"  # do not expand any instantiation of "a"
set_expanded_hinsts ""; # Note: same result if this line is removed
set_top_module top
```

Netlist:

```
module top;
  a u1; a u2;
endmodule
module a;
  b u1; c u2; c u3;
endmodule
module b;
  BUFX2 u1;
endmodule
module c;
  BUFX2 u1;
endmodule
```



Following is the expansion result of the above example:

```
COMPONENTS 2 ;
  - u1 a ;
  - u2 a ;
END COMPONENTS
```

Related Topics

get_selectively_expanded_modules

set_via_pillars

```
set_via_pillars
[-help]
{-base_pin {base_pin | pg_base_pin} | -pin {pin | pg_pin}}
[-required {0 | 1 | 2 | 3 | 4}]
stack_via_rule*
```

This command sets a via pillar list for a cell `base_pin` or a single via pillar for a `pin`.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `set_via_pillars` parameter. For a detailed description of the command and all its parameters, use the man command: `man set_via_pillars`
`-base_pin {base_pin | pg_base_pin}`	Specifies the base pin name or pointer. A via pillar list is allowed for this parameter.
`-pin {pin | pg_pin}`	Specifies the pin name or pointer. Only one via pillar is allowed for this parameter.
`-required {0 | 1 | 2 | 3 | 4}`	Sets the `.stack_via_rule_Required` attribute for the *base_pin*/*pin* values specified by the `-base_pin` or `-pin` parameters. Setting the `.stack_via_rule_Required` attribute determines whether the stack vias are required for the specified base_pin. • When set to `0` for a `base_pin`, this parameter sets the `.stack_via_rule_Required` attribute to `0`, which ignores this `base_pin` for `stack_via_rule` checking. If the parameter is set to `0` only for a `pin`, other `pin` values instantiated from the same `base_pin` will not be affected. • When set to `1` for a `base_pin`, this parameter sets the `.stack_via_rule_Required` attribute to `1` on the `base_pin` values. When set to `1` for an `pin`, it enforces the via pillar only on the specified `pin`. However, if the via pillar list is empty for an `pin`, no via pillar is used even when the required bit is `1`. If set to `1`, the software performs min via pillar insertion with dirty mode, in which the via pillars are added even if there is DRC. • When set to `2` for a `base_pin`, this parameter sets the `.stack_via_rule_Required` attribute to `2` on the `base_pin` values. If set to `2`, the software performs min via pillar insertion with clean mode, in which the via pillars are not added if there is DRC. • When set to `3` for a `base_pin`, this parameter sets the `.stack_via_rule_Required` attribute to `3` on the `base_pin` values. If set to `3`, the software performs multiple tries from large to min via pillar insertion with dirty mode, in which the via pillars are added even if there is DRC. • When set to `4` for a `base_pin`, this parameter sets the `.stack_via_rule_Required` attribute to `4` on the `base_pin` values. If set to `4`, the software performs multiple tries from large to min via pillar insertion with clean mode, in which the via pillars are not added if there is DRC. The `2`, `3`, and `4` values of this parameter are applicable only for a `base_pin`.

*stack_via_rule**	Provides a list of via pillars for a cell `base_pin`, or a single via pillar for a `pin`. This list further provides the candidates of `pin` via pillar setting. On running:
	`set_via_pillars -base_pin BUFX1/Z -required 1 {vp_rule1 vp_rule2}`
	`set_via_pillars` automatically sets the minimal cost via pillar rule for the corresponding `pin`, which is `vp_rule1`. You can also set the other via pillar rule to the `pin` by running:
	`set_via_pillars -pin i1/i2/Y vp_rule2`
	An empty *`stack_via_rule`* will remove the via pillar from `pin`:
	`set_via_pillars -pin i1/i2/Y -required 1 ""`

Examples

- Where the cell of `inst i1/i2` and `i1/i3` are both `BUFX1`, the following commands indicate that `pin i1/i2/Z` requires a via pillar `M2M4_1X3`:
```
set_via_pillars -base_pin BUFX1/Z -required 1 {M2M4_1X2 M2M4_1X3}
set_via_pillars -pin i1/i2/Z -required 1 M2M4_1X3   #The via pillar for pin must be on the cell base_pin's via pillar list. Otherwise, a proper ERROR should be returned.
```
- The following command removes the via pillar `M2M4_1X3` from `pin i1/i2/Z`:
```
set_via_pillars -pin i1/i2/Z "" -required 1
```
- The following command sets a via pillar list on a cell pin:
```
set_via_pillars -base_pin BUFX1/Z -required 1 {M2M4_1X2 M2M4_1X3}
```
- The following command sets a via pillar with via pillar list of the corresponding cell pin on a `pin`; the cell of inst `i1/i2` is `BUFX1`:
```
set_via_pillars -pin i1/i2/Z M2M4_1X3
```
- The following command sets via pillar to cell pin `AN2_NOM_D16*/Z`:
```
foreach cell [dbGet head.libCells.name -e AN2_NOM_D16*] {set_via_pillars -base_pin $cell/Z -required 0
{VPPERF_M2_M4_1_2_1 VPPERF_M2_M5_1_2_2_1 VPPERF_M2_M9_1_2_2_2_2_2_2_1} }
```

Related Information

`get_via_pillars`

sort_db

```
sort_db
[-help]
obj_list
[attribute_list]
[-descending]
[-dictionary]
```

Returns a sorted DPO (dual-ported object) list based on the contents of one or more attributes. It is based on `lsort` using `get_db` to access the various attributes.

This command sorts objects only based on the attributes that have a "simple" data type of string, float, or integer, or object. If it is an object, it is implicitly sorted by the `.name` field and treated as a string. By default, the `sort_db` command sorts any input list as string. But when the `-dictionary` parameter is specified, the numeric values are sorted numerically, like `lsort`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `sort_db` parameter. For a detailed description of the command and all of its parameters, use the man command: `man sort_db`

`obj_list`	A DPO list to be sorted. If the first element of the specified list is not a DPO, then it is treated as a root attribute name and passed directly to `get_db` to return a DPO list. So, the following two commands have identical results:
	`sort_db insts`
	`sort_db [get_db insts]`

`attribute_list`	Sorts the DPO list by the specified attribute values. The first attribute in the *`attribute_list`* has the highest sort precedence. Each attribute can be a chain identical to what `get_db` supports. A leading "." is optional, so all the following commands presenting the use of `attribute_list` specify legal arguments for an inst list:
	`sort_db insts name`
	`sort_db insts .name`
	`sort_db insts {location.x location.y}`
	`sort_db insts {.location.x .location.y}`

`-descending`	Sorts the list of objects by the descending order.
`-dictionary`	Sorts the objects using string comparison in all cases, except when the strings contain numbers. The numbers in strings are compared as integers/real.

Examples

- The following command sorts all the insts by name:
  `sort_db insts`
- The following command sorts the pins based on their slack. The pins with the same slack are further sorted by the x-location:
  `sort_db [get_db pins] {.slack .location.x}`

Related Topics

- get_db
- reset_db
- set_db

tech Category Attributes

The root attributes in the `tech` category are:

- tech_db_units
- tech_finfet_grid_direction
- tech_finfet_grid_offset
- tech_finfet_grid_pitch
- tech_inst_mask_shift_layers
- tech_mfg_grid

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `tech` category, use the following command:

```
get_db -category tech
```

tech_db_units

`tech_db_units` *int*

Default: `""`

Data_type: Instance

Specifies the database units per micron from LEF UNITS statement or OA techfile, unless `init_min_dbu_per_micron` is larger and overrides the LEF/OA value.

tech_finfet_grid_direction

`tech_finfet_grid_direction` *enum*

Default: `vertical`

Data_type: Enum

Specifies an enum value indicating the direction of the finfet grid from the LEF FINFET statement or OA.

tech_finfet_grid_offset

`tech_finfet_grid_offset` *coord*

Default: `no_value`

Data_type: Coordinate

Specifies the finfet grid offset from `0` in microns from the LEF FINFET statement or OA.

tech_finfet_grid_pitch

`tech_finfet_grid_pitch` *coord*

Default: `no_value`

Data_type: Coordinate

Specifies the finfet grid pitch in microns from the LEF FINFET statement or OA.

tech_inst_mask_shift_layers

```
tech_inst_mask_shift_layers layer*
```

Default: `""`

Data_type: Layer

Specifies the ordered list of layer pointers to the layers that allow instance mask shifting. This is the list of layers that show up in the DEF COMPONENTMASK statement for layers that can have mask values shifted in standard cells.

tech_mfg_grid

```
tech_mfg_grid coord
```

Default: `""`

Data_type: Coordinate

Specifies the manufacturing grid from LEF MANUFACTURINGGRID statement or OA techfile.

Note: This is the default value for each layer, but it can be overridden for specific layers. In some technologies, the `mfg_grid` for higher metal layers is coarser than for the lower layers.

uniquify

```
uniquify
[-help]
{[design | module] | -new_master hinsts} [-exclude modules]
[-verbose]
```

Supports regrouping of master-clone partitions inside the non-unique modules. This command uniquifies or regroups the master/clones (one master module used by two or more `hinst` clones) by creating unique modules for each `hinst` clone, as needed. In all the cases, the new module names are generated by appending the '`_number`' suffix (such as `_1` and `_2`) sequentially to the original module name for creating a unique module name.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `uniquify` parameter. For a detailed description of the command and all its parameters, use the man command: `man uniquify`
design | *module*	Regroups the master/clones within the *design* or *module* and recursively below it. This assures that all the `hinsts` inside those would have their own unique module, except for the modules specified using the `-exclude` parameter. In case multiple `hinsts` are referenced to the specified `module`, those will remain as clones. *Type:* Mandatory
`-new_master` *hinsts*	Splits the `hinst` clones of a master module into the clones of a new master. The new master is also regrouped, except for the modules specified using the `-exclude` parameter. It may also happen that one of such `hinst` clones is forced to have a new master (refer to the **Examples** section). In case a `hinst` is not a clone of the same module, an error message is displayed. *Type:* Mandatory
`-exclude` *modules*	Excludes the specified modules from unification or regrouping. Use this parameter if a specific sub-module is a partition, which will be implemented and shared only once. *Type:* Optional
`-verbose`	Reports the `hinst` clones (if any), which are regrouped. *Type:* Optional

Examples

- The following command uniquifies the whole design, where the `h1`, `h2`, and `h3 hinsts` are clones of the `mod1` module, and the `h4` and `h5 hinsts` (clones of the `mod2` module) are inside `mod1`:

```
uniquify top -verbose
#hinst name module name
h1 mod1 (unchanged)
h2 mod1_1
h3 mod1_2
h1/h4 mod2 (unchanged)
h1/h5 mod2_1
h2/h4 mod2_2
h2/h5 mod2_3
h3/h4 mod2_4
h3/h5 mod2_5
```

  Note that the first clone of each module is not modified. So, `h1` still uses `mod1`, and `h1`/`h4` still use `mod2`. However, these are still reported to see the `hinst` clone, which forces a new module for the other `hinst` clones.

- Using the same scenario as in the previous example, the following command uniquifes the design below `mod1`:

```
uniquify mod1 -verbose
#hinst name module name
h1/h4 mod2 (unchanged)
h1/h5 mod2_1
h2/h4 mod2 (unchanged)
h2/h5 mod2_1
h3/h4 mod2 (unchanged)
h3/h5 mod2_1
```

  Note that only the design inside `mod1` is uniquified. So, `h1`, `h2`, and `h3` are still the clones of `mod1`, and there are still master/clones grouping between `h1`, `h2`, and `h3`. For instance, all the `h4`s use `mod2`, and all the `h5`s use `mod2_1`.

- Using the same scenario as in the first example, the following command splits `h1` and `h2` into a new master. This is normally done, if `h1`, `h2`, and `h3` are partitions and need a different implementation than `h3`. It is recommended not to uniquify `mod2`, if it is a lower-level partition, which will be implemented once and shared by all `h1`, `h2`, and `h3`.

```
uniquify -new_master {h1 h2} -exclude mod2 -verbose
#hinst name module name
h1 mod1_1
h2 mod1_1
```

  Note that `h3` is still using `mod1`, and the `mod2 hinsts` inside `mod1` and `mod1_1` (such as `h4` and `h5`) are still the clones of `mod2`. If there is also a `mod3` inside `mod1`, it will also be uniquified, but nothing inside `mod2` is uniquified.

- Using the same scenario as in the first example, the following command splits `h1` and `h4` into a new master. This implicitly forces `h1` to be uniquified from `h2` and `h3`.

```
uniquify -new_master {h1/h4} -verbose
#hinst name module name
h1 mod1_1 (forced by lower level change)
h1/h4 mod2_1
```

  If there is a `mod4` inside `mod2_1`, it will also be uniquified, but it will not be shared with `mod2`.

write_preserves

```
write_preserves
[-help]
[-dont_touch]
[-dont_use]
[-obj_types {design hinst module inst net hnet pin hpin base_cell}]
[-out_file string]
```

Writes preserve constraints in a consumable TCL file so that the exported file can sourced.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_preserves` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_preserves`
`-dont_touch`	Writes the preserve constraints only for the non-default attributes related to the `dont_touch` object.
`-dont_use`	Writes the preserve constraints only for the non-default attributes related to the `dont_use` object.
`-obj_types {design hinst module inst net hnet pin hpin base_cell}`	
	Writes the preserve constraints only for the specific object types.
`-out_file string`	Displays the TCL file name.

4

Check Commands and Attributes

- check_ac_limit Category Attributes
- check_ac_limits
- check_power_vias
- create_marker
- delete_markers
- gui_show_markers
- read_markers
- report_markers

check_ac_limit Category Attributes

The root attributes in the `check_ac_limit` category are:

- check_ac_limit_additional_individual_violation_report
- check_ac_limit_avg_recovery
- check_ac_limit_check_thermal_aware_em
- check_ac_limit_current_file
- check_ac_limit_current_scale_factor
- check_ac_limit_current_scale_table
- check_ac_limit_default_freq_for_unconstrained_nets
- check_ac_limit_delta_temperature
- check_ac_limit_delta_temperature_layer_list
- check_ac_limit_detailed
- check_ac_limit_effort_level
- check_ac_limit_em_cdf_percentage
- check_ac_limit_em_limit_scale_factor
- check_ac_limit_em_limit_scale_table
- check_ac_limit_em_res_width
- check_ac_limit_em_temperature
- check_ac_limit_em_temperature_layer_list
- check_ac_limit_em_threshold

- check_ac_limit_enable_seb
- check_ac_limit_env_temperature
- check_ac_limit_extraction_tech_file
- check_ac_limit_force_hold_view
- check_ac_limit_handle_pin_obs_via
- check_ac_limit_ict_em_models
- check_ac_limit_ircx_models
- check_ac_limit_lifetime
- check_ac_limit_lifetime_rating_factor_file
- check_ac_limit_max_error
- check_ac_limit_method
- check_ac_limit_min_peak_duty_ratio
- check_ac_limit_min_peak_freq
- check_ac_limit_net_file
- check_ac_limit_nets
- check_ac_limit_out_file
- check_ac_limit_peak_td_method
- check_ac_limit_read_detail_delta_temperature_file
- check_ac_limit_seb_lifetime
- check_ac_limit_seb_table
- check_ac_limit_seb_temperature
- check_ac_limit_report_db
- check_ac_limit_report_unannotated_shapes
- check_ac_limit_selected
- check_ac_limit_simulation_net
- check_ac_limit_simulation_net_file
- check_ac_limit_skip_category_mode
- check_ac_limit_skip_net
- check_ac_limit_skip_net_file
- check_ac_limit_toggle
- check_ac_limit_top_scope_ignore_block_internal_nets_on_boundary_path
- check_ac_limit_use_db_freq
- check_ac_limit_use_qrc_tech
- check_ac_limit_use_rms_delta_t

- check_ac_limit_view

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `check_ac_limit` category, use the following command:

```
get_db -category check_ac_limit
```

check_ac_limit_additional_individual_violation_report

```
check_ac_limit_additional_individual_violation_report {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to generate separate violation reports for each current waveform calculation method (`rms`, `peak`, or `avg`). If this attribute is not specified, a single violation report file is generated using the `check_ac_limit_method {RMS AVG peak}` attribute. This attribute gives you the flexibility of generating either a single collated report or individual reports for each analysis method.

Example:

```
set_db  check_ac_limit_method {rms peak avg} \
check_ac_limit_out_file context1.rpt \
...
check_ac_limit_additional_individual_violation_report true
```

This command will generate 3 different reports, namely `context1.rpt_avg`, `context1.rpt_peak`, and `context1.rpt_rms`. If there are no EM violations for a method, the tool will not generate the corresponding separate report. In this example, if there are violations for `avg` and `rms` but none for `peak`, then it will generate only two reports, `context1.rpt_avg` and `context1.rpt_rms`.

check_ac_limit_avg_recovery

```
check_ac_limit_avg_recovery em_recover
```

Default: `1.0`

Data_type: Double

Specifies the recovery factor used for calculating I_{avg} current density with recovery. By default, the recovery factor (per layer) is read from the QRC techfile. If you specify `em_recover`, it overrides the QRC tech `em_recover` factor for all layers used in I_{avg} limits. The minimum value of this attribute is `-1` and the maximum is `1`.

check_ac_limit_check_thermal_aware_em

```
check_ac_limit_check_thermal_aware_em {true | false}
```

Default: `false`

Specifies to use the thermal map for EM checks. The thermal map file is specified using the `set_rail_analysis_config -read_thermal_map` parameter. The Rail Analysis engine uses the temperature specified in the thermal map file for EM analysis. EM limit has a strong temperature dependency. For a chip that has substantial temperature variation across the chip, the Rail Analysis engine can use the temperature specified in the thermal map file to perform accurate EM analysis.

check_ac_limit_current_file

`check_ac_limit_current_file` *`filename`*

Default: `""`

Data_type: String

Specifies the current of a macro's output pins for EM calculation, in case the macro does not have the respective timing library. When the macro output pin current is set by the current file, the value calculated (if any) from AAE is overwritten.

check_ac_limit_current_scale_factor

`check_ac_limit_current_scale_factor {{avg` *`value`*`} {rms` *`value`*`} {peak` *`value`*`}}`

Default: `""`

Data_type: String

Scales the signal EM current by the respective factor for avg, rms, or peak current specified by `{avg` *`value`*`} {rms` *`value`*`} {peak` *`value`*`}`.

The value specified for this attribute must be > 0 and <= 10.

check_ac_limit_current_scale_table

`check_ac_limit_current_scale_table` *`current_scale_table_file`*

Default: `""`

Data_type: String

Specifies the name of the scale table file used for layer-based scaling of the signal EM current. The scale table contains the scale factor for avg, rms, or peak current for each layer.

check_ac_limit_default_freq_for_unconstrained_nets

`check_ac_limit_default_freq_for_unconstrained_nets` *`freq_in_Hz`*

Default: `1e+06`

Data_type: Double

Specifies the frequency for EM calculation when a net has no defined frequency or has a defined frequency of `0` Hz in the design.

check_ac_limit_delta_temperature

`check_ac_limit_delta_temperature` *`temp`*

Default: `5.0`

Data_type: Double

Specifies the maximum temperature rise permitted in units of Celsius. This value is normally used in the QRC tech file for the RMS current limit equation. For example:

```
em_jmax_ac_rms EQU sqrt(12.66 * delta_T * (w - 0.003)^2 * (w - 0.003 + 0.264) / (w - 0.003 + 0.0443))
```

check_ac_limit_delta_temperature_layer_list

```
check_ac_limit_delta_temperature_layer_list {{<LEF_layer_name> <delta_T_in_C>}+}
```

Default: `""`

Data_type: String

Specifies the delta temperature for specific LEF layers during signal EM RMS current limit analysis.

check_ac_limit_detailed

```
check_ac_limit_detailed
```

Default: `false`

Data_type: Boolean

Generates a detailed report containing information for all signal nets, including those without violations.

check_ac_limit_effort_level

```
check_ac_limit_effort_level {low | medium | high}
```

Default: `low`

Data_type: Enum

Specifies the method used for current calculation of nets in the Signal ElectroMigration analysis flow. The two methods are:

- **non-simulation mode** - use the capacitance ratio to distribute the current down the stream from the driver to the receiver.
- **simulation mode** - simulates resistance and capacitance to get the true current on each segment. This mode is more accurate but requires a longer run time.

This attribute has the following three arguments:

- `low`: enables the non-simulation mode for all nets where the current distribution is based on the capacitance ratio.
- `medium`: enables the simulation mode for the nets with physical loops, and the non-simulation mode for the non-loop nets.
- `high`: enables the simulation mode for all nets.

The `medium` and `high` levels require a VTS-AA license.

Note: For the simulation mode with effort level `medium` or `high`:

- If you use `extract_rc`, run `set_db extract_rc_qrc_cmd_type partial extract_rc_qrc_cmd_file` *`qrc.user.cmd`*, where

qrc.user.cmd must include the following CCL commands:

```
extraction_setup -analysis em -cluster_vias_in_em true
output_db -type spef -subtype "EXTENDED"
```

Refer to the example below:

```
output_db \
 -type spef -busbit_delimiter "[]" -escape_special_character false \
 -hierarchy_delimiter "/" -output_incomplete_nets true \
 -output_unrouted_nets true -pin_delimiter ":" -subtype "EXTENDED" \
 -disable_subnodes false -add_explicit_vias true \
 -include_parasitic_res_width true -include_parasitic_res_width_drawn true
```

- If you run standalone Quantus to generate the required SPEF file in the extended format separately, then instead of `extract_rc`, use the following command for signal EM analysis:
  `read_spef -keep_star_node_locations -extended spef_file`

check_ac_limit_em_cdf_percentage

`check_ac_limit_em_cdf_percentage value`

Default: `1e-09`

Data_type: bool, read/write

Calculates the Cumulative Distribution Factor (CDF) derating factor based on the EM rules specified in the ICT EM file, given by the `check_ac_limit_ict_em_models` attribute. This attribute is used to calculate the fail rate and reliability of a design. The value of the CDF percentage should be between 0 and 100.

When the CDF related rule exits in the ICT EM file, the software uses the default value `1e-09` for `check_ac_limit_em_cdf_percentage` if it is not specified.

check_ac_limit_em_limit_scale_factor

`check_ac_limit_em_limit_scale_factor {{avg value} {rms value} {peak value}}`

Default: `""`

Data_type: String

Scales the calculated signal EM limits by the respective factor for avg, rms or peak current limit specified by `{avg value} {rms value} {peak value}`. The software compares the current through the net against the scaled EM limit to report violations in signal EM.

The value specified for this attribute must be > 0 and <= 10.

check_ac_limit_em_limit_scale_table

`check_ac_limit_em_limit_scale_table em_limit_scale_table_file`

Default: `""`

Data_type: String

Specifies the name of the scale table file used for layer-based scaling of the signal EM limit. The scale table contains the scale factor for avg, rms, or peak current for each layer.

check_ac_limit_em_res_width

```
check_ac_limit_em_res_width {drawn silicon}
```

Default: `drawn`

Data_type: Enum

Specifies width used to get EM limit.

check_ac_limit_em_temperature

```
check_ac_limit_em_temperature value
```

Default: `-1.0`

Data_type: Double

Specifies the temperature expected for the device and is used to look up the temperature scaling factor. By default, no scaling is done. The *value* is a float, in units of degrees Celsius.

check_ac_limit_em_temperature_layer_list

```
check_ac_limit_em_temperature_layer_list {{<LEF_layer_name> <em_temperature_in_C>}+}
```

Default: `""`

Data_type: String

Sets a different temperature for specific LEF layers during signal EM AVG current limit analysis.

check_ac_limit_em_threshold

```
check_ac_limit_em_threshold value
```

Default: `1.0`

Data_type: Double

Specifies to report violations in signal EM above the threshold value. The value is the ratio of signal current (avg/peak/rms) to the respective EM limit.

check_ac_limit_enable_seb

```
check_ac_limit_enable_seb {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to enable the Statistical ElectroMigration Budgeting (SEB) analysis.

check_ac_limit_env_temperature

`check_ac_limit_env_temperature` *value*

Default: `-1.0`

Data_type: Double

Specifies the ambient temperature used to calculate S_{DC} for SEB analysis.

check_ac_limit_extraction_tech_file

`check_ac_limit_extraction_tech_file` *filename*

Default: `""`

Data_type: String

Specifies an extraction technology file for the Signal ElectroMigration analysis flow. If this attribute is not specified, then by default the extraction technology file will be taken from the analysis view (current RC corner).

check_ac_limit_force_hold_view

`check_ac_limit_force_hold_view`

Default: `false`

Data_type: Boolean

Forces `set_signal_em_analysis_mode` to use the hold view for current calculation even during setup fixing.

check_ac_limit_handle_pin_obs_via

`check_ac_limit_handle_pin_obs_via`

Default: `false`

Data_type: Boolean

Specifies to consider the vias defined in the `OBS` (obstructions/blockages) section of LEF in the Signal ElectroMigration flow. By default, the software does not consider via in obstruction.

This attribute does not honor the via obstructions with "`SPACING 0`".

check_ac_limit_ict_em_models

`check_ac_limit_ict_em_models` *file*

Default: `""`

Data_type: String

Specifies the ICT-EM file which only contains the EM rules and excludes the RC extraction data in ICT. The software will use the EM rules in the specified ICT-EM file to calculate EM limits for signal net EM analysis even if there are embedded EM rules in the Quantus QRC tech file.

check_ac_limit_ircx_models

```
check_ac_limit_ircx_models {RC.ircx EMIR.ircx}|{RC.ircx}
```

Default: ""

Specifies an encrypted or unencrypted RC/EM technology (.ircx) file. The foundry-specific IRCX file, which contains the process information for extraction and EM modeling, can be used when the ICT-EM file is not available for electromigration analysis.

You can either specify two IRCX files, one file containing the RC process data (`RC.ircx`) and the other file containing EM rules (`EM.ircx`), or a single IRCX file that includes both the RC and EM sections (`RC.ircx`). A single IRCX file is used for the mature nodes, and separate RC and EM IRCX files are used for the advanced process nodes.

check_ac_limit_lifetime

```
check_ac_limit_lifetime value
```

Default: `87600.0`

Data_type: Double

Specifies the total hours of operation expected for the device, and is used to look up the lifetime scaling factor. *`value`* is used to find the scaling factor by interpolating from a piece-wise-linear table in the LEF or the QRC tech file (`em_jmax_ac model` with `jmax_lifetime` table). If there is no scaling factor table, no scaling is done. *`value`* is a float, in units of hours.

check_ac_limit_lifetime_rating_factor_file

```
check_ac_limit_lifetime_rating_factor_file filename
```

Default: ""

Data_type: String

Specifies a lifetime rating factor file that contains jmax_life values for each layer. These values are defined based on temperature and lifetime pairs. When you provide this file, EM analysis will automatically select the appropriate `jmax_life` factor for each layer according to the specific temperature-lifetime combinations defined within the file. The lifetime and EM temperature can be specified using the `check_ac_limit_lifetime` and `check_ac_limit_em_temperature` attributes.

Following is a snippet of the lifetime rating factor file:

```
*Rating factor of M0/V0/M1/V1/M2/V3/M3...
TEMPERATURE        90        95       100     105     110 ...
LifeTime
-------------------------------------------------------------------------------
0.5                2.578    2.242    2.024   1.774   1.547
1                  2.520    2.191    1.979   1.734   1.512
2                  2.430    2.113    1.908   1.672   1.458
3                  2.382    2.072    1.871   1.639   1.430
...
-------------------------------------------------------------------------------
```

check_ac_limit_max_error

`check_ac_limit_max_error` *value*

Default: `10000`

Data_type: Integer

Specifies the maximum number of errors to report. The command stops when the maximum value is reached.

check_ac_limit_method

`check_ac_limit_method {rms peak avg}`

Default: ""

Data_type: String

Specifies the current waveform calculation method. You can specify one or more of the following options:

- `avg`: Checks I_{avg} limits and reports any average current limit violations in the design.
- `peak`: Checks I_{peak} limits and reports any peak current limit violations in the design.
- `rms`: Checks I_{rms} limits and reports any RMS current limit violations in the design.

check_ac_limit_min_peak_duty_ratio

`check_ac_limit_min_peak_duty_ratio` *duty_ratio*

Default: `-1.0`

Data_type: Double

Sets the minimum duty ratio value for I_{peak} calculation. If the calculated duty ratio of a net is lower than the specified value, the specified value will be used to calculate I_{peak}. The minimum value for *duty_ratio* is `0.0`and the maximum is `1.0`.

check_ac_limit_min_peak_freq

`check_ac_limit_min_peak_freq` *freq_in_Hz*

Default: `1e+06`

Data_type: Double

Ignores I_{peak} current limit calculation for signal nets that have an effective frequency below the specified value (in Hertz).

check_ac_limit_net_file

`check_ac_limit_net_file` *list_file*

Default: `""`

Data_type: String

Specifies the net from file to check the signalEM analysis. The format is one net name per line, with comments using # as the

first character. For example:

```
#list of nets to skip
net_a
net_b
net_c
```

check_ac_limit_nets

```
check_ac_limit_nets {netNames}
```

Default: ""

Data_type: String

Specifies whether to check named or selected nets. If you name only one net, you do not need the braces around the net name. If you specify more than one net, you must separate the net names with a space.

check_ac_limit_out_file

```
check_ac_limit_out_file filename
```

Default: ""

Data_type: String

Specifies the report file for the violation data. The report file includes name of the net, I_{rms}, interpolated ACCURRENTDENSITY limit (I_{limit}), layer, width, X and Y location, rise and fall time, effective frequency for the net (F_{eff}), Cnet and Vdd for each net.

check_ac_limit_peak_td_method

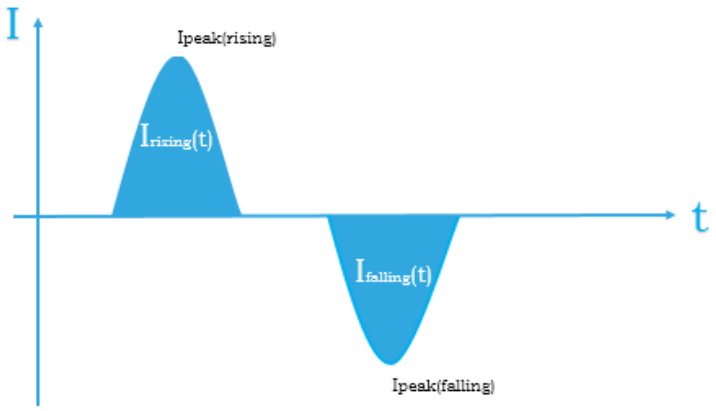
```
check_ac_limit_peak_td_method {effective_width_from_integration effective_half_peak_width
max_equivalent_dc_peak sum_half_peak_width}
```

Default: effective_width_from_integration

Data_type: Enum

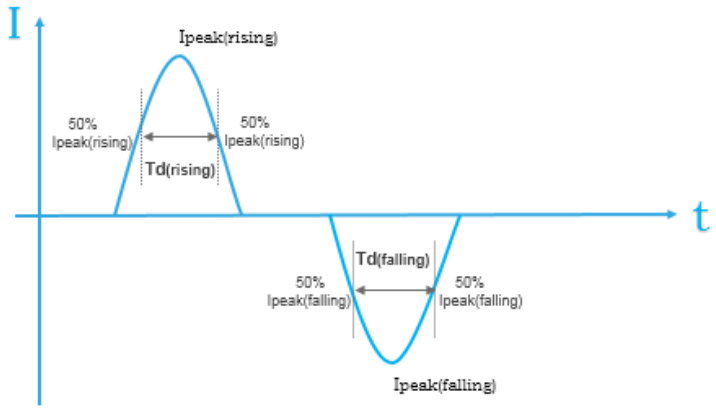
Specifies the Td calculation method for Ipeak check. The legal values are depicted below.

effective_width_from_integration



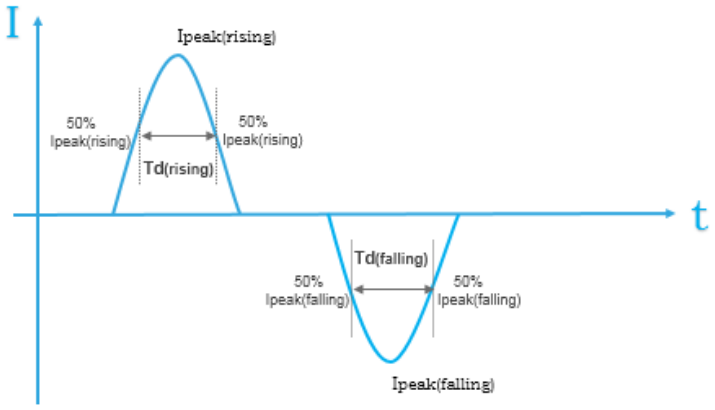
- $\mathrm{Td} = \frac{\int_0^{Tsw} |I(t)| dt}{\mathrm{Ipeak_measured}}$
  - Tsw = Period of one switching cycle (rising + falling)
  - Ipeak_measured = abs.max(Ipeak(rising), Ipeak(ralling))

`effective_half_peak_width`



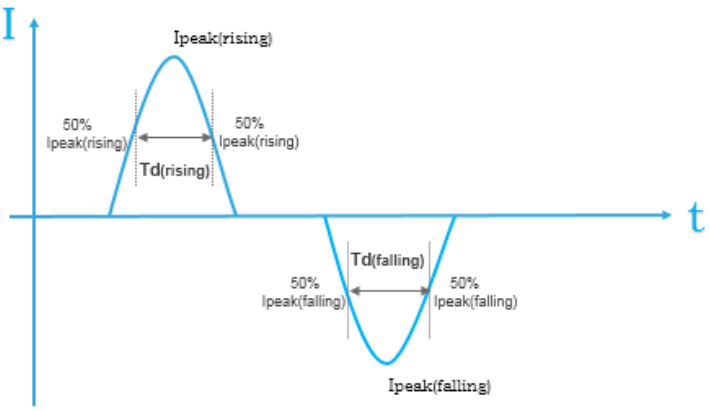
- When Ipeak(rising) >= Ipeak(falling)
  $\mathrm{Td} = \mathrm{Td(rising)} + (\frac{\mathrm{Ipeak(falling)}}{\mathrm{Ipeak(rising)}})^2 * \mathrm{Td(falling)}$
- When Ipeak(rising) < Ipeak(falling)
  $\mathrm{Td} = \mathrm{Td(falling)} + (\frac{\mathrm{Ipeak(rising)}}{\mathrm{Ipeak(falling)}})^2 * \mathrm{Td(rising)}$

max_equivalent_dc_peak



- $R(rising) = Td(rising)/(T/2)$
- $R(falling) = Td(falling)/(T/2)$
- Choose $\max(Ipeak_{(rising)}*sqrt(R_{(rising)}), Ipeak_{(falling)}*sqrt(R_{(falling)}))$ to compare against Ipeak_dc_limit
- $Ipeak_ac_limit = Ipeak_dc_limit/sqrt(R)$
- When $Ipeak_{(rising)}*sqrt(R_{(rising)}) >= Ipeak_{(falling)}*sqrt(R_{(falling)})$
  $Td = 2*Td(rising)$
- When $Ipeak_{(rising)}*sqrt(R_{(rising)}) < Ipeak_{(falling)}*sqrt(R_{(falling)})$
  $Td = 2*Td(falling)$

sum_half_peak_width



- $Td = Td_{(rising)} + Td_{(falling)}$

check_ac_limit_read_detail_delta_temperature_file

`check_ac_limit_read_detail_delta_temperature_file` *filename*

Default: `""`

Data_type: String

Specifies to read the detailed delta temperature file generated by the `report_self_heat` command (self-heat analysis).

check_ac_limit_seb_lifetime

`check_ac_limit_seb_lifetime` *value*

Default: `43800.0`

Data_type: Double

Specifies the life time of product in hours.

check_ac_limit_seb_table

`check_ac_limit_seb_table` *table_filename*

Default: `""`

Data_type: String

Specifies the file name of the Failures in Time (FIT) table. This file is provided by the foundry. The file has the following columns:

- Layer
- MTF
- Sigma
- Activate energy
- Current density exponent

check_ac_limit_seb_temperature

`check_ac_limit_seb_temperature` *temperature*

Default: `-273.0`

Data_type: Double

Specifies the temperature used to update the median time to failure (MTF) with respect to the ambient temperature.

- If this temperature is specified, it would be used globally for all the resistors.
- If this temperature is not specified, the `seb_temperature` may take the following possible values depending on the specific circumstances:
  - If detailed delta temperature file is not used for SEB, the `seb_temperature` always takes the same value as that used for EM analysis.
  - If detailed delta temperature file is used for SEB, the `seb_temperature` is calculated by the following equation:
    `seb_temperature  = env_temperature + delta_T`
    Where `delta_T` is the maximum temperature rise, including joule heat and coupling heat from FEOL, in the resistor's bounding box with zero extension.

check_ac_limit_report_db

`check_ac_limit_report_db`

Default: `false`

Data_type: Boolean

Specifies to write the signal EM data into a database file (*`filename.db`*) after performing signal EM analysis. This file contains the information required for loading and displaying signal EM data in GUI.

check_ac_limit_report_unannotated_shapes

```
check_ac_limit_report_unannotated_shapes {true | false}
```

Default: `false`

Data_type: Boolean

Provides visibility into partial net coverage within SPEF files. When this attribute is set to `true`, the `<report_name>.rpt.unannotated_shapes` is generated. This report identifies and lists nets that contain shapes without resistance annotations, or unanalyzed/ partially analyzed segments. The feature helps to detect incomplete parasitic extraction coverage and improve accuracy during validation and signoff workflows. The reporting of partial net coverage works only when `check_ac_limit_effort_level` is set to `high`.

Use Model:

```
set_db check_ac_limit_effort_level high check_ac_limit_report_unannotated_shapes true
```

Report Format:

```
NET: <net_name>          <- Specifies the name of the analyzed simulation nets with unannotated shapes
LAYER (x1 y1 x2 y2)      <- Layer name and shape coordinates
```

Sample Report:

```
NET: Net_A
LAYER (x1          y1          x2          y2)

M2    (30.2850    12.0800    37.6150    12.2200)
M2    (30.2000    12.0650    30.2850    12.2350)
```

This attribute is applicable only when running in the DP mode; it is not supported in the XP mode.

check_ac_limit_selected

```
check_ac_limit_selected filename
```

Default: `false`

Data_type: Boolean

Analyzes the net specified by selecting a net in the main GUI window.

check_ac_limit_simulation_net

```
check_ac_limit_simulation_net list_of_nets
```

Default: `""`

Forces the specified nets to use the simulation mode.

check_ac_limit_simulation_net_file

`check_ac_limit_simulation_net_file` *`filename`*

Default: `""`

Forces the nets in the specified file to use the simulation mode.

check_ac_limit_skip_category_mode

`check_ac_limit_skip_category_mode {0 | 1 | 2}`

Default: `1`

Data_type: Integer

Specifies to skip nets that belong to categories mapping to the specified mode. This parameter allows you to specify three modes, namely 0, 1 (default), and 2.

The following table lists the common categories for all modes, and the categories specific to each mode:

Common Categories	0 Skip Category Mode	1 Skip Category Mode	2 Skip Category Mode
• Non top scope nets • Internal nets on boundary path • P/G nets • Constant nets • User-defined skipped nets • Nets below effective frequency threshold • Physically broken nets • Nets without loading capacitance annotation • Nets with SPEF back-annotation failures during simulation • Nets failed in analysis	• Slave nets by Verilog Assign • Nets (non-constant) without frequency annotation • Nets without physical shape for output pin • Nets without wire shape connection to output pin • Nets with zero pin current	• Equivalent nets by Verilog Assign • Nets without driver • Nets without receiver	• Nets without pins • Equivalent nets by Verilog Assign • Nets without driver • Nets without receiver

For example, if you set `set_signal_em_analysis_mode -skip_category_mode to 1`, the report file lists all categories listed in the *Common Categories* and *1 Skip Category Mode* columns.

Following is a snippet of the report file for mode **1**:

```
Total Nets in Design:               33451
Total Selected Nets in Design:          4
Total Analyzed Nets:                    0
Total Skipped Nets:                     4
  Non top scope nets:                                                0
  Internal nets on boundary path:                                    0
  Equivalent nets by Verilog Assign:                                 0
  P/G nets:                                                          0
  Constant nets:                                                     2
  User-defined skipped nets:                                         0
  Nets below effective frequency threshold:                          0
  Nets without driver:                                               1
  Nets without receiver:                                             1
  Physically broken nets:                                            0
  Nets without loading capacitance annotation:                       0
  Nets with SPEF back-annotation failures during simulation:         0
  Nets failed in analysis:                                           0
```

check_ac_limit_skip_net

```
check_ac_limit_skip_net net_name_list
```

Default: `""`

Data_type: String

Specifies the net to be skipped in the signalEM analysis.

Refer to the example below:

```
{net_a net_b net_c}
```

check_ac_limit_skip_net_file

```
check_ac_limit_skip_net_file list_file
```

Default: `""`

Data_type: String

Specifies the net file to be skipped in the signalEM analysis.

The format is one net name per line, with comments using # as the first char. For example:

```
#list of nets to skip
net_a
net_b
net_c
```

check_ac_limit_toggle

```
check_ac_limit_toggle value
```

Default: `1.0`

Data_type: Double

Specifies the switching factor for signal nets. A value of `1.0` means that if a flip-flop is clocked by a 20 Mhz clock, it changes state on 100% of the clock cycles. Therefore, the signal has 20 million transitions, which is half the number of transitions of the input clock. As a result, the final switching frequency of the signal net is 10 Mhz.

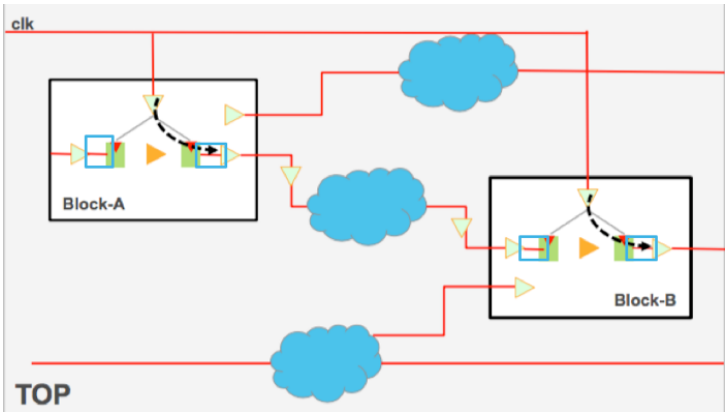
Value Range: `0.0` to `1.0`

check_ac_limit_top_scope_ignore_block_internal_nets_on_boundary_path

```
check_ac_limit_top_scope_ignore_block_internal_nets_on_boundary_path {true | false}
```

Default: `false`

Ignores the internal nets from the first instance to the first flop on the boundary path for the signal EM checking. The following figure represents that when this parameter is used, the internal nets (highlighted in the blue rectangles) are ignored. However, all the nets highlighted through the red lines are reported in the TOP scope (`create_top_scope`) flow:



check_ac_limit_use_db_freq

`check_ac_limit_use_db_freq`

Default: `false`

Data_type: Boolean

Uses the database frequency value as the effective frequency per net.

check_ac_limit_use_qrc_tech

`check_ac_limit_use_qrc_tech`

Default: `true`

Data_type: Boolean

Forces I_{rms} checks to use the QRC tech file instead of the technology LEF file. If either I_{peak} or I_{avg} is also checked, all checks will use the QRC tech file, including I_{rms}. This attribute is enabled by default.

Note: The `-method avg` and `-method peak` analysis will only support AC current density rules from QRC tech file. If either `avg` or `peak` is also specified, then all checks will use the QRC tech file equations, including RMS.

check_ac_limit_use_rms_delta_t

`check_ac_limit_use_rms_delta_t {true | false}`

Default: `false`

Data_type: Boolean

When this parameter is set to `true`, the `delta_T` used to calculate `seb_temperature` would be based on the RMS delta T (joule heat only).

If this parameter is set to `false`, the `delta_T` used to calculate `seb_temperature` would be based on the sum of self-heat delta T and RMS delta T.

check_ac_limit_view

`check_ac_limit_view` *viewName*

Default: `""`

Data_type: String

Specifies the name of the view.

check_ac_limits

```
check_ac_limits
[-help]
```

Checks for the following types of AC current violations on signal nets:

- Root-mean-square (RMS) current limit violations (I_{rms} violations)
- Peak current limit violations (I_{peak} violations)

In addition, `check_ac_limits` can be used for average current limit (I_{avg}) analysis.

AC current limit violations are sometimes also referred as wire self-heat violations. DRC manuals have these current limits to avoid over-heating a signal-nets wire with too much AC current. To prevent wire self-heating or AC signal electromigration, signal interconnects should be analyzed for their AC current carrying capacity and measured against the AC current limits specified by foundry.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `check_ac_limits` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_ac_limits`

Related Commands

- check_ac_limit Category Attributes
- `gui_show_markers`

check_power_vias

```
check_power_vias
[-help]
[-append]
[-area {x1 y1 x2 y2}]
[-check_fill]
[-check_wire_pin_overlap]
[-cut_area_ratio value]
[-edge_to_edge]
[-error integer]
[-exclude_region_file check_file]
[-hookup_pitch pitch_value]
[-ignore_objects {drc_fill | io_wire}]
[-ignore_partial_overlap {true | false | value}]
[-layer_range {bottomLayer topLayer}]
[-nets {netNames} | -selected]
[-non_orthogonal_check [-non_orthogonal_distance {x y}]]
[-pitch pitch_value]
[-rail_layers layer_name]
[-report file]
[-search_range {x y}]
[-shielding]
[-stacked_via]
[-stripe_layers layer_list]
[-stripe_rule value]
[-what_if_report filename]
[-width_range string]
```

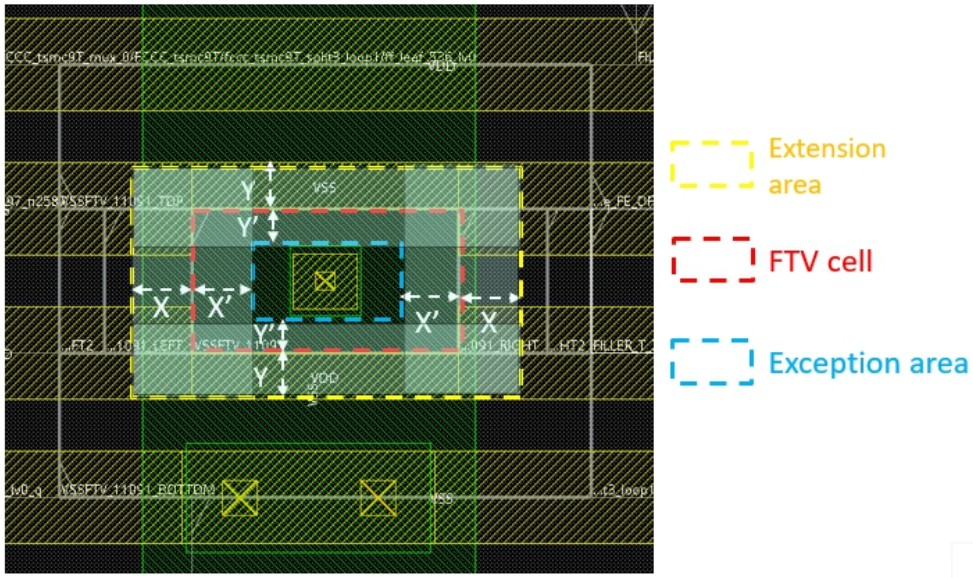
This command has a variety of power-rail overlap checks to look for missing power-grid vias. By default, it checks that orthogonal power-routes on adjacent routing layers have a via between them at every intersection. For example, there should be a via at the point where a Metal3 power stripe overlaps a Metal2 power stripe. By default, horizontal routes get checked only for overlaps with vertical routes. If the power shapes are polygon, the polygon is split into rectangles and the rectangles are used to decide the direction. Support for checking stacked vias between non-adjacent routing layers is optional. Violations are highlighted in the layout window, and a text report is generated with the location of the missing vias.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `check_power_vias` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man check_power_vias`
`-append`	Specifies that all previous `check_power_vias` settings will be kept.
`-area x1 y1 x2 y2`	Specifies the coordinates of the area to be checked.
`-check_fill`	Specifies that tied-off metal fill should also be checked. Floating fill is still ignored. *Default*: Metal fill will be ignored.

`-check_wire_pin_overlap`	Specifies that wire overlaps of pin shapes should also be checked. By default, only wire overlaps with wires are checked.
`-cut_area_ratio` *value*	Checks if the cut area in the metal intersection area is sufficient. `cut_area_ratio = Total cut area / Metal intersect area` If the actual ratio is less than specified ratio, `check_power_vias` marks it as violation. By default, the metal intersection area is the metal overlap area. If `setAddStripeMode -via_using_exact_crossover_size` is set to `false`, the partial overlapped metal will be extended to full width intersection area.
`-edge_to_edge`	Checks the non-orthogonal distance from edge to edge within a layer.
`-error` *integer*	Specifies the maximum number of errors to report.

`-exclude_region_file` *check_file*	Ignores missing VIA checks within the specified `check_file` regions for a cell extension area. The `check_file` format can be a set of the following values:
	• `cell`: Specifies the cell type.
	• `ext_layer_bottom`: Specifies the extension area bottom layer.
	• `ext_layer_top`: Specifies the extension area top layer.
	• `ext_x`: Specifies the extension length in the X direction.
	• `ext_y`: Specifies the extension length in the X direction.
	• `exc_x`: Specifies the exception length in the X direction.
	• `exc_y`: Specifies the exception length in the Y direction.
	Example 1:
	The following set of `check_file` formats ignores missing VIA check within the yellow rectangle (in the output), but checks missing VIA within the blue rectangle:
	`FTV_CELL, B_M0, B_M1, X, Y, X', Y'`
	`INV_D2, M0, M1, X2, Y2, X2', Y2'`
	Following is the output:

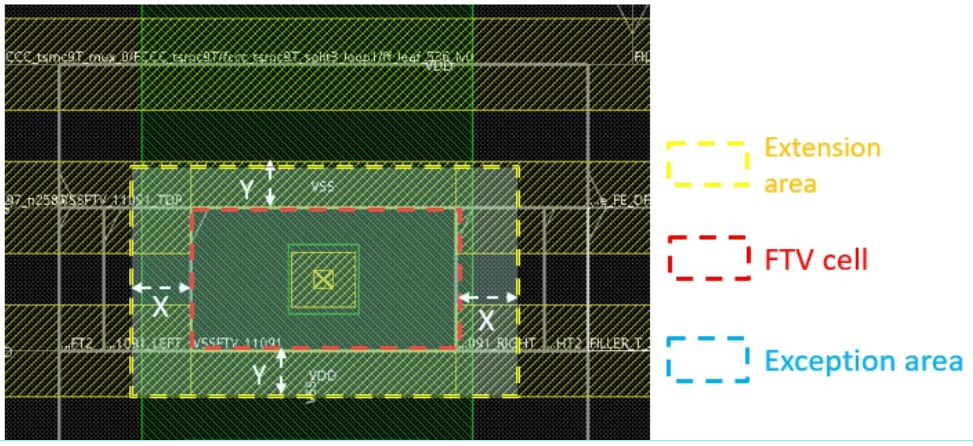


Example 2:

The following set of `check_file` formats ignores missing VIA check within the yellow rectangle (in the output):

```
FTV_CELL, B_M0, B_M1, X, Y
INV_D2, M0, M1, X2, Y2
```

Following is the output:



`-hookup_pitch` *`pitch_value`*	Specifies the distance between two vias, which are on the filler instance and are greater than or equal to the given value.
`-ignore_objects {drc_fill | io_wire}`	Skips checking the power vias of the specified wire type.

`-ignore_partial_overlap {true | false | value}`	Specifies if the partially overlapped metal layers need to be checked for missing vias. Specify one of the following parameter values: • `true`: Ignores checking the vias missing in partially overlapped metal layers. • `false`: Checks the vias missing in partially overlapped metal layers. • *`value`*: Checks the vias missing in partially overlapped metal layers on and above the specified portion of *`value`* range from 0 to 1. For example, specifying the *`value`* as 0.2 will check the vias missing in partially overlapped metal layers on or above 20% of the overlapped wires. The instance pin and physical pin shapes are considered as wires. *Default*: `false`

`-layer_range {`*`bottomLayer topLayer`*`}`	Checks for the missing stacked vias between the bottom and top layers. It also checks for the missing vias between all the adjacent intermediate layers, but does not check for the stacked vias between the intermediate layers. While using this parameter, you can specify full layer names like `METAL1`, or abbreviations like `1`. For example, `check_power_vias -layer_range {2 5}` checks for missing stacked vias between overlapping `metal2` and `metal5`, and also missing vias for overlapping `2/3`, `3/4`, `4/5`.
	This parameter checks the missing adjacent vias if `bottomLayer` is specified, but `topLayer` is not, with considering that `topLayer` is just one layer above. To review the missing stacked via or missing adjacent via DRC marker, use the `gui_show_markers` command for filtering the DRC marker. For example:
	1. To display only the missing stacked via DRC markers, run:
	`gui_show_markers -all -filter {stacked via} -search_description -filter_mode AND`
	2. To display only the missing via DRC markers between the adjacent layers, run:
	`gui_show_markers -all -filter {adjacent via} -search_description -filter_mode AND`
	See `gui_show_markers` for more information on the command usage.

`-nets {`*`netNames`*`}`	Specifies the names of the power nets that will be checked. A single net or a list of nets enclosed in curly braces({}) or quotes (" ") can be specified. Wildcards (* and ?) are supported. For example, when you specify `check_power_vias -nets {v*}`, all PG net names having initial letter as `v` are checked. **Note:** Currently, wildcards are supported for checking PG nets only. *Default*: All power grid nets.

`-non_orthogonal_check`	Checks the overlap area for non-orthogonal crossing wires (e.g. wires that are both in the same direction) in adjacent as well as non-adjacent layers. Checks for missing vias between two or more parallel wires. The via center-to-center distance between parallel wires should be less than or equal to the specified spacing If this parameter is not specified, horizontal wires get checked only against vertical wires. **Note**: You can use `-non_orthogonal_distance {x y}` parameter with `-non_orthogonal_check` to find missing vias within a specified window. When the `x` or `y` distance is provided, `check_power_vias` checks if the wire is parallel with another wire. If yes, it checks the via center-center spacing and via center-wire end distance between parallel wires. If the actual distance is greater than the required distance, it marks a violation.
`-non_orthogonal_distance {x y}`	Specifies distances for finding missing via within a specified window for a non-orthogonal check, with *x* and `y` representing the distance in the horizontal and vertical directions, respectively. When the `x` or `y` distance is provided, `check_power_vias` checks if the wire is parallel with another wire. If yes, it checks: • The via center to via center spacing between parallel wires against the value *x (y)* • The via center to wire end distance against the value *x/2 (y/2)* If the actual distance is greater than the required distance in either case, it marks a violation. When the `x` or `y` distance is not provided or set to `0`, the via to via distances are not checked. **Note**: This parameter can only be specified with the `-non_orthogonal_check` parameter.
`-pitch` *pitch_value*	Specifies the distance between two vias, which are not on the filler instance and are greater than or equal to the given value.
`-rail_layers` *layer_name*	Specifies a single power rail layer for missing via (only the via above power rail) check. The check depends on the `-stripe_rule` option. When `-stripe_rule` is specified, the via to via distance should be less than or equal to the specified value. If a distance value is not specified with the `-stripe_rule` option, the check is ignored. See the examples below. The `-rail_layers` option is not controlled by `-layer_range`. You can specify a power rail layer other than that specified with `-layer_range` for checks against the `stripe_rule` and `search_range` rules.
`-report file`	Specifies the name of the output file for the report.

`-search_range {x y}`	Specifies a rectangle area with `XY`. There should be one valid connection from power rail to power stripe in the specified area. By default, the valid connection would be searched from the uppermost power rail layer to all the power stripe layers. You can use the `-rail_layers` option to specify the power rail layer, and the `-stripe_layers` option to limit the power stripe layers. With the `-rail_layers` and `-stripe_layers` options, only the specified power rail and power stripes layers are used for valid connection check. The x and y values are floats in units of microns. Refer to the **Examples** section for search range examples.
`-selected`	Checks for the missing vias between the selected PG wires and other intersecting selected/unselected PG wires. When this parameter is used with the `-check_wire_pin_overlap` parameter, it checks for the missing vias between the intersection of selected PG wires and pins of the selected instances. This parameter eliminates the requirement of running the command on the entire design, and hence improves the run time.
`-shielding`	Specifies that shielding wires should also be checked. By default, shielding wires are not checked.
`-stacked_via`	Checks for missing vias between all non-adjacent as well as adjacent layers.
`-stripe_layers` *`layer_list`*	Specifies the power stripe layers to be checked for a valid connection path from power rail to power stripe. The check depends on the `-search_range` option. If `-search_range` is specified, at least one valid connection path should be found from the power rail to power stripe in that search range. If a search range is not specified with the `-search_range` option, the check is ignored. You can specify the power rail layer with the `-rail_layers` option. By default, only the uppermost power rail layer is checked. See the examples below. The `-stripe_layers` option is not controlled by `-layer_range`. You can specify power stripe layers other than that specified with `-layer_range` for checks against the `search_range` rule.
`-stripe_rule` *`value`*	Specifies in microns the distance for power via. Only the power via above the power rail is checked. If the via to via center-to-center spacing is greater than the specified value, a violation marker is created. You can specify the power rail layer with `-rail_layers` for checking. By default, only the power via above the uppermost power rail is checked against the rule. Refer to the **Examples** section for stripe rule examples.
`-what_if_report` *`filename`*	
	Writes out `check_power_vias` violations data into a ECO file that can be used directly as input for whatIf rail analysis. When you specify this parameter, the missing via information generated can be used as input for what-if virtual wires.
`-width_range` *`string`*	

	Specifies the metal width to be considered. `check_power_vias` uses the specified width range to filter wires. Only the bottom and top wires that fall in the range are checked. The *`string`* value takes the following format: `-width_range {`*`minWidth`* `maxWidth [layerName …] [`*`minWidth maxWidth [layerName`* `…]] … }` • If *`minWidth`* is 0, all wires with width less than specified *`maxWidth`* will be checked. • If *`maxWidth`* is 0, all wires with width greater than specified *`minWidth`* will be checked. • If *`layerName`* is specified in the string, `widthRange` is enabled only for the specified layer. • If no *`layerName`* is specified in the string, `widthRange` is enabled for all non-specified layers. Refer to the **Examples** section for width range examples. **Note:** Use braces or quotation marks to enclose the string value for `widthRange`.

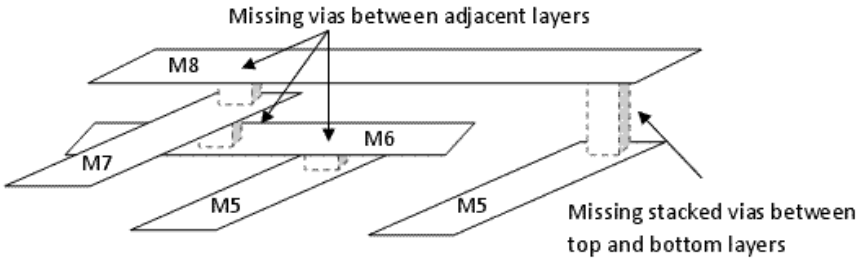
Examples

- Specifies that all vias for all powergrid nets will be checked, except for metal fill and shielding wires, and reports it to a file called `power_via.report`:

  ```
  check_power_vias -report power_via.report
  ```

- The following command checks for missing stacked vias between `METAL5` and `METAL8` layer intersections on the VSS net. In addition, it checks for missing vias between `METAL5-METAL6`, `METAL6-METAL7` and `METAL7-METAL8` layer intersections. The violations are reported in the `powerVia.rpt` file and highlighted in GUI.

  ```
  check_power_vias -layer_range {METAL5 METAL8} -nets VSS -report powerVia.rpt
  ```



- The following command checks that the overlapping power routes on adjacent layers have vias that cover at least 70% of the overlap area..

  ```
  check_power_vias -cut_area_ratio 0.7
  ```

- The following command checks that the overlapping power routes have vias that cover at least 70% of the overlap area for layers between `METAL3` and `METAL4`.

  ```
  check_power_vias -cut_area_ratio 0.7 -layer_range {METAL3 METAL4}
  ```

- The following command checks for wires overlaps for all pin shapes included, enclosed, or touched by the specified area:

  ```
  check_power_vias -area {100 100 1000 1000} -check_wire_pin_overlap
  ```

- The following command checks wires with width greater than `0.032` and less than `0.096` for all layers:

  ```
  check_power_vias -width_range {0.032 0.096}
  ```

- The following command checks wires with width in the `0.032` ~ `0.096` range for M1/M2 and width `0.038~0.114` for M3/M4. For other layers, there is no width control:

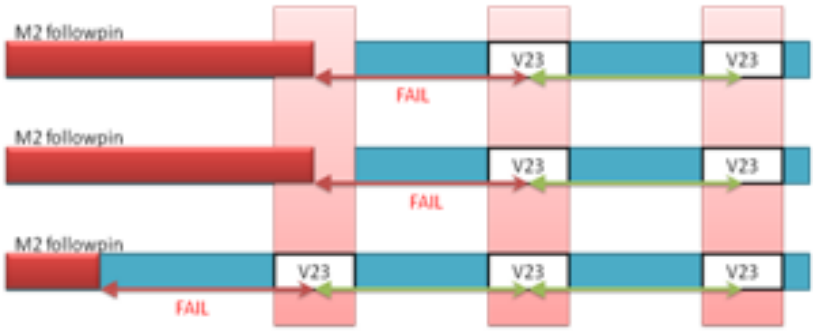
  ```
  check_power_vias -width_range {0.032 0.096 M1 M2 0.038 0.114 M3 M4}
  ```

- The following command checks wires with width in the `0.032` ~ `0.096` range for M1/M2. For other layers, it checks wires with width in the `0.038~0.114` range:

  ```
  check_power_vias -width_range {0.032 0.096 M1 M2 0.038 0.114}
  ```

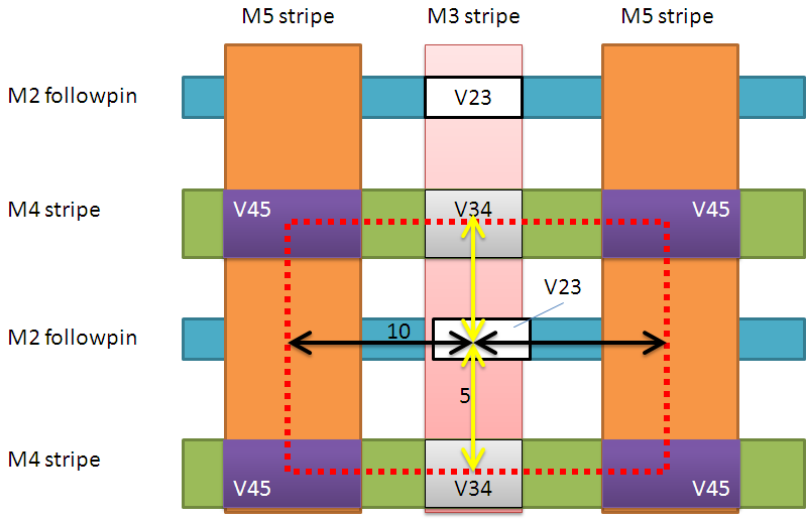
- The following command checks the via between power rail layer `M2` and `M3`. If the via's center-to-center distance is more than `17u`, the tool reports a violation and places a marker on the power rail wires.

  ```
  check_power_vias -stripe_rule 17 -rail_layers M2
  ```



- The following command checks the valid connection for `M2` power rail to stripe layer `M4` and `M5` with the search rectangle area `10*5`. In the diagram below, there is no violation as the connection is built from `M2` to `M4`. If the `V34` vias are removed, then there is a violation as the connection cannot be built from `M2` to `M4`.

  ```
  check_power_vias -search_range {10 5} -rail_layers M2 -stripe_layers {M4 M5}
  ```


M5 stripe
M3 stripe
M5 stripe
M2 followpin
V23
M4 stripe
V45
V34
V45
V23
M2 followpin
10
5
M4 stripe
V45
V34
V45

create_marker

```
create_marker
[-help]
{-bbox {x1 y1 x2 y2} | -poly {x1 y1 x2 y2...}}
[-description description]
[-layer layerName]
[-rule_map]
[-sub_type subtypeName]
[-tool toolName]
[-type typeName]
```

Creates markers for violations in the database and imports DRC markers generated by other software applications. Use this command if you want to import only a specific marker or check a specific marker that has been created by a third party tool.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_marker` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_marker`
`-bbox` *x1 y1 x2 y2*	Specifies the area of the bounding box of the marker. For example, `create_marker -bbox 100 100 100.5 100.5`. *Type:* Real
`-description` *description*	Describes the marker. For example, `create_marker -description "the user mark is non default rule spacing check"`. *Default:* `""` *Type:* String
`-layer` *layerName*	Specifies the layer. For example, `create_marker -layer METAL2`. *Type:* String
`-poly` *{x1 y1 x2 y2...}*	Specifies the area of the bounding polygon of the marker.
`-rule_map`	Specifies if the marker needs to be created through `rule_map`.
`-sub_type` *subtypeName*	Specifies the marker subtype. For example, `create_marker -sub_type user_spacing`. *Default:* `Other` *Type:* String
`-tool` *toolName*	Specifies the source software application. For example, `create_marker -tool Tempus`. *Default:* `Other` *Type:* String
`-type` *typeName*	Specifies the marker type. For example, `create_marker -type user_verify`. *Default:* `Other` *Type:* String

Examples

- The following command generates markers for tool named `Calibre` with type `METAL1` and subtype `S.1`:

  ```
  create_marker -bbox { 1 2 1 2 } -tool Calibre -type METAL1 -sub_type S.1
  ```

  The violation browser displays the marker in the above example as follows:

  ```
  + Calibre
      + METAL1
          + S.1
              ...
  ```

- Suppose the cadence LPA tool has created the following marker on layer `METAL3`:

  ```
  CDNLitho;SPACING; Severity: 3; MARKER_INFO: "CD = 45.39"

  bbox = (468.248, 63.511) (468.310, 63.599)
  ```

  The following command creates a corresponding marker in Tempus design:

  ```
  create_marker -bbox {468.248 63.511 468.310 63.599} -layer M3 /
               -type CDNLitho -sub_type Spacing
  ```

Related Commands

`delete_markers`

delete_markers

```
delete_markers
[-help]
[-all]
[-area {x1 y1 x2 y2}]
```

Deletes markers from a specified area or the entire design area.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_markers` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man delete_markers`
`-all`	Deletes markers from the entire design area.
`-area {x1 y1 x2 y2}`	Specifies a bounding box for the area from which to delete markers. For example, `delete_markers -area {0 0 100 100}`.

Related Commands

`create_marker`

gui_show_markers

```
gui_show_markers
[-help]
[-ac_limit]
[-all]
[-append]
[-area {x1 y1 x2 y2}]
[-connectivity]
[-density]
[-display_by_layer]
[-geometry]
[-max_error_per_type value]
[-no_false_marker]
[-overlap]
[-process_antenna]
[-search_description]
[-short]
[-filter_query LO_formula | {-filter filterString -filter_mode {AND | OR | NOT}}]
```

Displays a list of violations in the. The software generates markers for the violations after you use one or more verification commands.

Geometry or Metal Density Violations

Updates geometry and metal density violation markers incrementally during an Tempus session--that is, it displays the markers generated the first time you run the `check_metal_density` command and adds new markers, or deletes markers, on subsequent runs during the same session. If the software finds violations during a subsequent run that were already found previously, the browser display does not change, as there is no incremental update.

The browser can make incremental changes because `check_metal_density` can check a small area of the design and update the database. As a result of this behavior, Tempus saves the information from the first verification run.

Connectivity, Process Antennas, or AC Limit Violations

Overwrites connectivity, process antenna, and AC limit violation markers if the `check_process_antenna`, or `check_ac_limits` commands are run more than once during an Tempus session. These commands are net-based, not area-based, so the browser does not make incremental updates for them. As a result of this behavior, Tempus does not keep the information from the first verification run.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_show_markers` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_show_markers`
`-ac_limit`	Displays AC limit violations.

`-all`	Displays all the violations that are present in the design.
`-append`	Appends accumulated filtered markers to the existing markers in violation browser.
`-area {x1 y1 x2 y2}`	Specifies the coordinates of the area whose violations to display. For example, `gui_show_markers -area {0 0 100 100}`.
`-connectivity`	Displays connectivity violations.
`-density`	Displays metal density violations.
`-display_by_layer`	Displays the list of violations by the layer instead of route type.
`-filter filterString`	Specifies a text string in the violation message. Violation Browser searches the message of each violation and displays only the violations whose messages match the search conditions. To specify a list of strings, separate each string with a space. To specify a literal string with space in it, enclose the string in double quotation marks. For example, you can specify strings such as the following: • `METAL5` • `(600, 500)` • `METAL3 (700, 400)` • `"Pin of Net"`
`-filter_mode {AND | OR | NOT}`	Specifies the condition for searching multiple strings in the violation browser. You cannot filter complex expressions because the `AND`, `OR`, and `NOT` parameters are mutually exclusive. • The `AND` and `OR` conditions work on single strings or multiple strings. • The `NOT` condition works only on a single string.

`-filter_query` *`LO_formula`*	Specifies a complex logical expression for filtering violations. This option cannot be used along with `-filter` or `-filter_mode`. You can specify the AND, OR, and NOT conditions in the search expression. Use:
	• `&&` between strings in the expression to specify the AND condition.
	• `||` between strings in the expression to specify the OR condition.
	• ! before a string to specify the NOT condition. Note that ! is an one's complement operator. So for example, `!{object(net1) && layer(m2)}` is not allowed.
	The `-filter_query` parameter supports searches in the object, layer, and description areas. The area can be specified using the `object`, `layer`, and `desc` keywords. For example:
	`object(net1) || layer(m2) && desc(xxx)`
	You can also use the wildcards * and ? in the search expression. For example:
	`!object(clk*) && layer(m?)`
	To specify a literal string with space in it, enclose the string in double quotation marks. For example:
	`!object(clk*) && desc("prl spacing")`

`-geometry`	Displays geometry violations.
`-max_error_per_type` *`value`*	Specifies a limit for the number of violations parsed for each violation type. For example, if you set `gui_show_markers -max_error_per_type 500`, the Violation Browser parses and displays only 500 markers per error type. Use this parameter if you are loading a large DRC file with millions of violations to prevent the databse from becoming slow while loading the file and to limit the number of violations you review in the browser. By default, the Violation Browser displays **all** markers.
`-no_false_marker`	Prevents Tempus from displaying false violations.
`-overlap`	Displays overlaps in the design.
`-process_antenna`	Displays process antenna violations.
`-search_description`	Searches for text in the violation message and the violation description.
`-short`	Displays short violations.

Examples

The following command displays all violations whose marker title contains `METAL3`:

```
gui_show_markers -all -filter METAL3
```

The following command displays geometry violations whose marker title contains `METAL3` or `tdsp`:

```
gui_show_markers -geometry -filter {METAL3 tdsp} -filter_mode OR
```

The following command displays geometry violations whose marker title or marker description field contains `Min` and `0.24` and `METAL3`:

```
gui_show_markers -geometry -filter {Min 0.24 METAL3} -filter_mode AND -search_description
```

The following command displays violations occurring on Metal4 or Metal5 layers but excludes short violations:

```
gui_show_markers -filter_query {!Short && (m4 || m5)}
```

Related Commands

- check_ac_limits

read_markers

```
read_markers
[-help]
in_file fileName
[-orient {r0 r90 ...}]
[-rule_map_file mapName]
-type {Assura | Calibre | PVS | Hercules | ICV | cdn_litho | CDNCMP | inShapeLitho | CalibreLitho | DRV |
ClockTran | CLP | VerifyPower}
[-x_offset value]
[-y_offset value]
```

Loads a violation marker file or a hotspot interface format (HIF) file in one of the following formats and creates markers that the software can interpret. Use this command to load third-party markers into the tool:

- Assura™
- Calibre
- PVS
- Hercules
- ICV
- Cadence or Calibre HIF
- Chemical and Mechanical Polishing (CMP) HIF
- inShapeLitho
- Calibre litho hotspot
- NanoRoute litho
- Transition violation report
- Low power violation reports

After you load the file, you can view the markers in the Violation Browser. By cross-probing between Violation Browser and the design area, you can analyze the violations and take steps to resolve them. The Auto Query feature displays the rule names for the markers.

Parameters

`-help`	Prints a brief description that includes type and default information for each `read_markers` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_markers`
`in_file` *fileName*	Specifies the marker file to load. Assura files must have a `.err` extension. Calibre files must have a `.db` extension.
`-orient {`*r0 r90 ...*`}`	Supports advanced orientation transforms for loading rotated DRC violation markers before loading. This helps in overlaying the design to the top of the layout.

`-rule_map_file` *`mapName`*	Specifies the rule map file for a PVS or Calibre Litho report. The rule map file provides the layer/error type information.
	`read_markers` saves this information to the database. When the markers are created, the layer, error type, and object information can be identified on the GUI in the Violation Browser. NR could fix the converted drc markers (width, spacing, enclosure, area, minstep, etc). The rule map file should be a txt file with 3 columns:
	`Rule_name error_type layer_name`
	Below is an example:
	`M1.W_4.5 width metal1`
	`M1.A_0.048 minhole metal1`
	`M1.M_1 minstep metal1`

`-type {Assura | Calibre | PVS | Hercules | ICV | cdn_litho | CDNCMP | inShapeLitho | CalibreLitho | DRV | ClockTran | CLP | VerifyPower}`	
	Specifies the file type. The following file types contain violation markers: • Assura - For Assura violations • Calibre - For Calibre violations • PVS - For PVS violations • Hercules - For Hercules violations • ICV - For ICValidator report violations • CDNCMP - For Cadence CMP Predictor (CCP) violations • DRV - For transition violations (reportTranViolation) • ClockTran, CLP, and VerifyPower - For low power violations The following file types contain hotspot markers, which are used to identify areas susceptible to lithography problems: • cdn_litho (LPA as well as NanoRoute litho HIF) • inShapeLitho • CalibreLitho
`-x_offset` *`value`*	Specifies the X offset. When you specify the X and Y offsets, Violation Browser displays the violations at the correct coordinates after accounting for the offsets.
`-y_offset` *`value`*	Specifies the Y offset, which Violation Browser takes into account to displays violations at the correct coordinates.

Examples

- The following command loads a CDN Litho marker file named `LPA.hif`:

  ```
  read_markers -type cdn_litho -x_offset 0.05 -y_offset 0.05 in_file LPA.hif
  ```

- The following command loads a NanoRoute litho repair file named `NR.hif`:

  ```
  read_markers -type cdn_litho in_file NR.hif
  ```

Related Commands

`report_markers`

report_markers

```
report_markers
[-help]
[-all]
[-ac_limit]
[-connectivity]
[-density]
[-geometry]
[-max_error_per_type integer]
[-no_false_marker]
[-overlap]
[-process_antenna]
[-short]
[-area {x1 y1 x2 y2}]
[-report filename]
[-filter filterString]
[-filter_mode {AND | OR | NOT}]
[-search_description]
```

Reports violations flagged by the verification commands. The report file and violation browser list the violations in the same order.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_markers` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_markers`
`-ac_limit`	Reports AC limit violations.
`-all`	Reports all violations.
`-area {x1 y1 x2 y2}`	Specifies the coordinates of the area for which to report violations. For example, `report_markers -area {0 0 100 100}`.
`-connectivity`	Reports connectivity violations.
`-density`	Reports metal density violations.

`-filter` *`filterString`*	Specifies a text string in the violation message (in the bottom portion of the Violation Browser). When you specify a text string, the Violation Browser searches the message of each violation and displays only the violations whose messages match the search conditions. To specify a list of strings, separate each string with a space. To specify a literal string with space in it, enclose the string in double quotation marks. For example, you can specify strings such as the following: • `METAL5` • `(600, 500)` • `METAL3 (700, 400)` • `"Pin of Net"`
`-filter_mode {AND | OR | NOT}`	
	Specifies the condition for searching multiple strings in the violation browser. You cannot filter complex expressions because the AND, OR, and NOT parameters are mutually exclusive. • The `AND` and `OR` conditions work on single strings or multiple strings. • The `NOT` condition works only on a single string.
`-geometry`	Reports geometry violations.
`-max_error_per_type` *`integer`*	
	Specifies the error marker number limit for each type of error. Specifies a limit for the number of violations parsed for each violation type. For example, if you set `report_markers -max_error_per_type 500`, the Violation Browser parses and displays only 500 markers per error type in the report. Use this parameter if you are loading a large DRC file with millions of violations to prevent the database from becoming slow while loading the file and to limit the number of violations you review in the browser. By default, the Violation Browser displays **all** markers in the report.
`-no_false_marker`	Prohibits Tempus from displaying false violations.
`-overlap`	Reports overlap violations.
`-process_antenna`	Reports process antenna violations.
`-report` *`filename`*	Specifies the report file for the violation information. *Type:* String
`-search_description`	Searches the description field for each marker in addition to the table entries.
`-short`	Reports short violations.

Examples

The following command reports all violations whose marker title contains `METAL3`:

```
report_markers -all -filter METAL3
```

The following command reports geometry violations whose marker title contains `METAL3` or `tdsp`:

```
report_markers -geometry -filter {METAL3 tdsp} -filter_mode OR
```

The following command reports geometry violations whose marker title or marker description field contains `Min` and `0.24` and `METAL3`:

```
report_markers -geometry -filter {Min 0.24 METAL3} -filter_mode AND -search_description
```

Related Commands

`read_markers`

5

Clock Tree Debugger Commands

- gui_ctd_close
- gui_ctd_get_title
- gui_ctd_get_window_id
- gui_ctd_highlight
- gui_ctd_open
- gui_ctd_write_view
- gui_ctd_set_title
- gui_ctd_show_skew_group
- gui_ctd_write_histogram
- gui_ctd_zoom

gui_ctd_close

```
gui_ctd_close
[-help]
[-all | -id WindowIDName]
```

Closes open CTD windows. When no parameter is specified, the command closes the current active CTD window.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_ctd_close` parameter. For a detailed description of the command and all of its parameters, use the man command: `man gui_ctd_close`.
`-all`	Closes all open CTD windows.
`-id` *`WindowIDName`*	Closes the CTD window with the specified window ID.

Examples

- The following command closes all CTD windows in the current session:

  ```
  gui_ctd_close -all
  ```

- The following command closes the CTD window with the ID, `user_id2`:

  ```
  gui_ctd_close -id user_id2
  ```

Related Commands

- gui_ctd_open
- gui_ctd_highlight
- gui_ctd_zoom

gui_ctd_get_title

```
gui_ctd_get_title
[-help]
[-all | -id WindowIDName]
```

Retrieves the titles of the CTD windows. If no parameters are specified, the command retrieves the title of the current active CTD window.

Note: Before using this command, ensure that a CTD window is open. You can open a CTD window using the `gui_open_ctd` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_ctd_get_title` parameter. For a detailed description of the command and all of its parameters, use the man command: `man gui_ctd_get_title`.
`-all`	Retrieves the IDs of all CTD windows.
`-id` *`WindowIDName`*	Retrieves the title of the CTD window with the specified ID.

Examples

- The following command retrieves the titles of all CTD windows in the current session:

  ```
  gui_ctd_get_title -all
  ```

  {user_id1 : Clock Tree Debugger : title1} {user_id2 : Clock Tree Debugger : title2} {user_id3 : Clock Tree Debugger : title3}

- The following command retrieves the title of the CTD window with the ID, `user_id2`:

  ```
  gui_ctd_get_title -id user_id2
  ```

  ```
  user_id2 : Clock Tree Debugger : title2
  ```

  where, `user_id2` is the window ID, Clock Tree Debugger is the window type, and `title2` is the mapping title for this window.

Related Commands

- `gui_ctd_set_title`
- `gui_ctd_get_window_id`

gui_ctd_get_window_id

```
gui_ctd_get_window_id
[-help]
[-all]
[-detail]
```

Retrieves the IDs of the CTD windows. If no parameters are specified, the command retrieves the window ID of the current active CTD window. For example, if you create a window with the ID, `user_id`, and then run this command without specifying any parameters, the software displays the following information:

```
gui_open_ctd -title my_title -id user_id
gui_ctd_get_window_id
user_id
```

Note: Before using this command, ensure that a CTD window is open. You can open a CTD window using the `gui_open_ctd` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_ctd_get_window_id` parameter. For a detailed description of the command and all of its parameters, use the man command: `man gui_ctd_get_window_id`.
`-all`	Retrieves the IDs of all CTD windows.
`-detail`	Retrieves the IDs of CTD windows, with a mapping to their titles.

Examples

- The following command retrieves the ID of all CTD windows in the current session:

  ```
  gui_ctd_get_window_id -all

  user_id1 user_id2 user_id3
  ```

- The following command retrieves the IDs of CTD windows, with a mapping of their titles:

  ```
  gui_ctd_get_window_id -detail

  user_id2 : Clock Tree Debugger : title2
  ```

  where, `user_id2` is the window ID, Clock Tree Debugger is the window type, and `title2` is the mapping title for this window.

Related Commands

- `gui_ctd_get_title`
- `gui_ctd_set_title`

gui_ctd_highlight

```
gui_ctd_highlight
[-help]
[-color colorname]
[-from rootclock]
[-index colorindex]
[-out_file fileName]
[-to sink | -through instancename]
```

Highlights the clock tree path from the clock root to the pin or from one pin to the other if the two pins are on the same path using the specified color or color index. The highlighted clock tree path can be viewed in the Clock Tree Viewer of the CCOpt Clock Tree Debugger (CTD). You can also use the Highlight Clock Path option in the context menu of the CTD to highlight the complete clock tree path of a selected pin to the clock root.

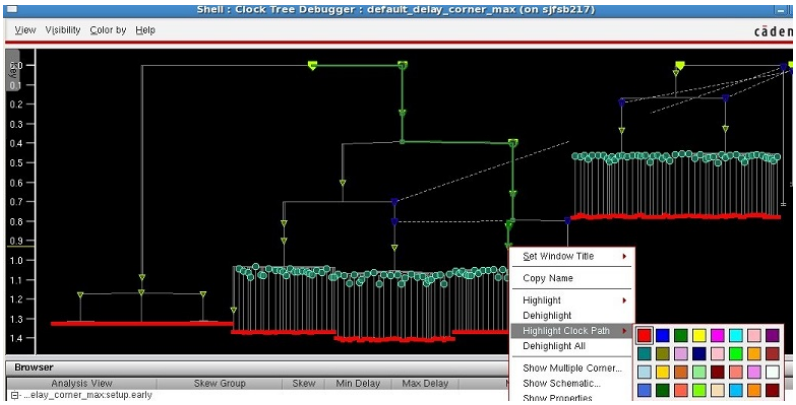
Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_ctd_highlight` parameter. For a detailed description of the command and all of its parameters, use the man command: `man gui_ctd_highlight`.
`-color` *`colorname`*	Specifies the color to be used for highlighting the clock tree path that is being traced. *Default*: red
`-from` *`clockroot`*	Specifies the starting point of the highlight. If this parameter is not specified, the starting point of the highlight is the clock root of the specified sink.
`-index` *`colorindex`*	Specifies the range of colors that can be used to highlight the clock tree path. Minimum is 1 and maximum is 32. The range corresponds to the colors in the color picker provided in the *Highlight Clock Path* option in the context menu of the Clock Tree Viewer. The number 1 means red, 2 means blue, and so on. An example is shown in the image below. *Default:* 1 or red.

`-out_file` *`fileName`*	Specifies the name of the file in which the physical information for each instance and net in the clock path being traced will be dumped.
	The following information is saved in the file:
	`Name Cell Status Location Wire Length`
	`CLK_ROOT PAD Fixed (20,40)`
	`net_1 don't_touch 12.5`
	`Buf_1 BUFX1 placed (100,180)`
	`net_2 35`
	`Buf_2 BUFX2 placed (200, 120)`
	Name is the instance name or the net name
	Cell is the instance cell type. This information is only for instances
	Status is the place status for an instance and constrain status for a net. For dont_touch nets, the status is '`dont_touch`', otherwise this column is left blank
	Location is the physical location of the instance or the coordinate of that instance
	Wire Length is only for nets

`-through` *`instancename`*	Specifies the instance or pin through which the path should be traced.
`-to` *`sink`*	Specifies the end point of the highlighted path.

The image below shows the clock tree path highlighted in green, which corresponds to the color index 3. It also shows the color picker with the range of colors included in the color index.

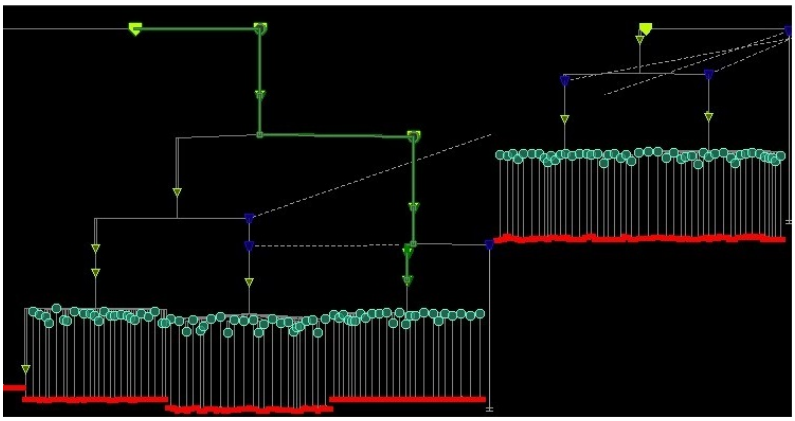


Example

The following command highlights the clock tree path for the specified pin, `ff1`, from the clock root in green color:

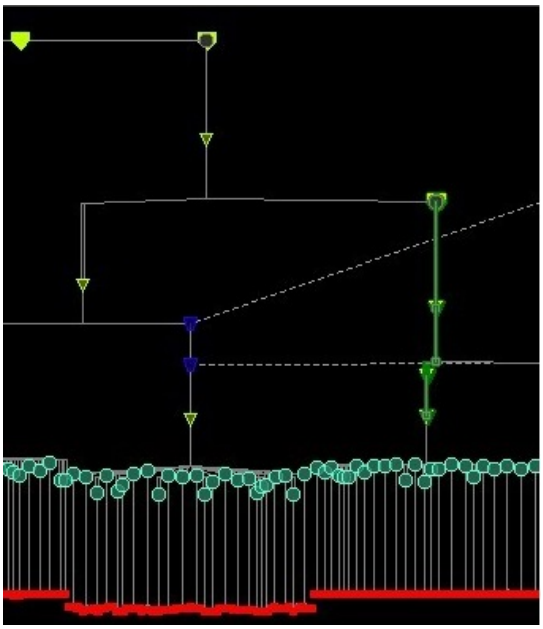
- `gui_ctd_highlight -to ff1 -index 3`

  The CTD shows the following:



- The following command highlights the path between two sinks that are on the same path, in green color:

```
gui_ctd_highlight -to ff1  -from u1  -index 3
```



Related Commands

- gui_ctd_open
- gui_ctd_close

gui_ctd_open

```
gui_ctd_open
[-help]
[-id windowIDName]
[-include_reporting_only_skew_groups]
[-title CTDWindowID]
[-unit_delay]
```

Opens a Clock Tree Debugger (CTD) window with the user-defined window ID and title.

When run without specifying the ID and title, the command opens a CTD window with a system default window title. By default, the window title is the name of the current analysis corner, which shows in the CTD window. For example, "Clock Tree Debugger - func_rcbest".

By default, all CTD windows have their unique window ID. The mechanism of the default window ID is `ctdMainWin_1, ctdMainWin_2, ctdMainWin_3`….and so on. So, by default, the window ID of a new opened Clock Tree Debugger window is `ctdMainWin_1`.

The information in the title comprises the following:

```
Session title: Window type: Window title
```

where,

Window type is always “Clock Tree Debugger”.

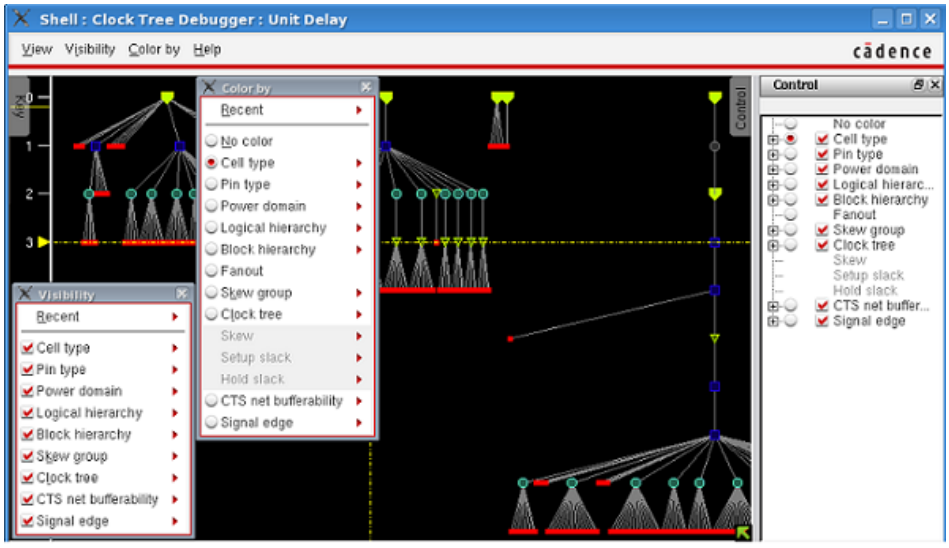
If the user-specified session title is “`my_test.tcl`”, and the default primary analysis corner is “`primary_corner`”, then with gui_ctd_open, the total set of the window title line will be:

```
my_test.tcl : Clock Tree Debugger : primary_corner
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_ctd_open` parameter. For a detailed description of the command and all of its parameters, use the man command: `man gui_ctd_open`.
`-id windowIDName`	Opens a Clock Tree Debugger window with a user-specified window ID. By default, the window IDs will be `ctdMainWin_0`, or `ctdMainWin_1`, and so on.
`-include_reporting_only_skew_groups`	When this parameter is specified, the GUI will include the reporting-only skew groups. By default, this setting if off, which means the reporting-only skew groups are ommitted from the CTD. **Note**: Once the CTD is started, it will not be possible to add or remove the reporting-only skew groups from those that are displayed. You will need to start a new CTD window specifying the new skew group filtering that is required.

`-title` *`CTDWindowID`*	Specifies the user-defined title for the CTD window. Open a Clock Tree Debugger window with a user specified title. For example, if you specify `my_title` as the window title, when session title is `my_test.tcl` and the primary analysis corner is `primary_corner`, then the total set of the window title line will be: `my_test.tcl : Clock Tree Debugger : my_title` **Note**: The window's ID if not specified using the `-id` parameter, will be system default ID. To view the default window ID, use the `gui_ctd_get_window_id` command.

`-unit_delay`	Opens the CTD window in unit delay mode. The unit delay lets you view the depth of clock tree objects easily. This is useful when you have large designs with deep gate-level trees. When debugging clock trees, you can use this view to check how the clock trees are balanced by looking at the depth of the clock trees. Although, the unit delay mode can also be enabled using the *Unit delay* submenu in the *Visibility* menu of the CTD but this is only valid after running clock tree synthesis, and you still need to invoke timing calculation and extraction, which are time consuming. You can use this parameter to open the CTD in unit delay mode even if the design is not placed or without performing CTS. When the CTD is opened in this mode, the following options in CTD menus are disabled: *Visibility* menu: • Timing windows • Delays • Unit delay *Color by* menu and the *Control* panel: • Transition time • Total net cap • Max cap violation • Skew • Setup slack • Hold slack • Timing windows This is shown in below image. 

Examples

- The following command creates a CTD window ID with the user-specified name, `my_test`:

  ```
  gui_ctd_open -id my_test
  ```

  When you use the get_ctd_win_id command to list all window IDs, you get the above window ID in the list:

  ```
  gui_ctd_get_window_id -all
  ```

  ```
  ctdMain_0 my_test
  ```

- The following command creates a CTD window ID, `my_test` and CTD window title of `myTest.tcl`:

  ```
  gui_ctd_open -title myTest.tcl
  ```

  When you use the `-detail` parameter of the `gui_ctd_get_window_id` command, the following information is returned:

  ```
  gui_ctd_get_window_id -detail
  ```

  ```
  ctdMain_1 "Clock Tree Debugger - func_rcbest"
  ```

  where, `ctdMain01` is the default window ID, window type is Clock Tree Debugger, and `func_rcbest` is the name of the current analysis corner.

Related Commands

- `gui_ctd_close`
- `gui_ctd_highlight`
- `gui_ctd_zoom`

gui_ctd_write_view

```
gui_ctd_write_view
[-help]
-file fileName
[-id windowID]
[-legend_on]
[-type {PNG | GIF}
```

Saves the current snapshot of the clock tree viewer into a file.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_ctd_write_view` parameter. For a detailed description of the command and all of its parameters, use the man command: man `gui_ctd_write_view`.
`-file fileName`	Specifies the name of the file where you want to dump the display image. This parameter is required.
`-id windowIDName`	Specifies the window ID whose display is to be saved. The default value is the returned value of command `gui_ctd_get_window_id`.
`-legend_on`	Saves the whole window including the panels that are open.
`-type {PNG | GIF}`	Specifies whether the format of the image should be Glf or PNG.

Example

The following command saves the view snapshot in the `snapshot1.png` file.

```
gui_ctd_write_view -file snapshot1 -type PNG
```

Related Commands

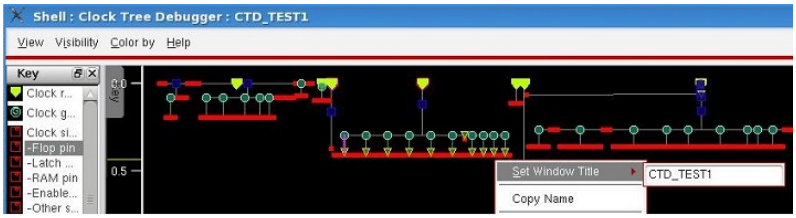
- `gui_ctd_close`
- `gui_ctd_get_window_id`
- `gui_ctd_highlight`
- `gui_ctd_zoom`
- `gui_ctd_open`
- `gui_ctd_write_histogram`

gui_ctd_set_title

```
gui_ctd_set_title
[-help]
title
[-id WindowIDName]
```

Specifies the title of the CTD window. By default, this command is used to set the title of the current active CTD window either if a unique ID has not been specified for this window or if you want to change the existing window ID.

You can also set the window title from the context menu of the Clock Tree Viewer in the CTD. Click Set Window Title, type the title for the CTD session, and press enter. The new title appears in the top bar of the CTD. This is shown in below image.



Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_ctd_set_title` parameter. For a detailed description of the command and all of its parameters, use the man command: `man gui_ctd_set_title`.
`-id` *`WindowIDName`*	
	Specifies the ID of the window whose title is to be set. If there are several CTD windows open in the current session, with window IDs, `user_id1`, `user_id3`, `user_id3`, and so on, with corresponding window titles, `title1`, `title2`, `title3`, and so on, then the following command will only change the `title1` of window with ID, `user_id1`, to `my_new_title`. `gui_ctd_set_title` *`my_new_title`* `-id` *`user_id1`*
`title`	Specifies the title to be given to the specified window. This parameter is required.

Examples

- The following command sets the window title of the current CTD window, `my_title`, to `my_new_title`:

  ```
  gui_ctd_set_title my_new_title
  ```

  The active CTD window title will be changed from:

  ```
  my_test.tcl : Clock Tree Debugger : my_title
  ```

  to:

  ```
  my_test.tcl : Clock Tree Debugger : my_new_title
  ```

- The following command retrieves the window IDs of CTD windows, with a mapping of their window titles:

  ```
  gui_ctd_get_window_id -detail
  ```

  ```
  user_id2 : Clock Tree Debugger : title2
  ```

where, `user_id2` is the window ID, Clock Tree Debugger is the window type, and title2 is the mapping title for this window.

Related Commands

- `gui_ctd_get_title`
- `gui_ctd_get_window_id`

gui_ctd_show_skew_group

```
gui_ctd_show_skew_group
[-help]
[-id window_id_name]
-skew_group skew_group_name
```

Shows the specified skew group in the CTD window.

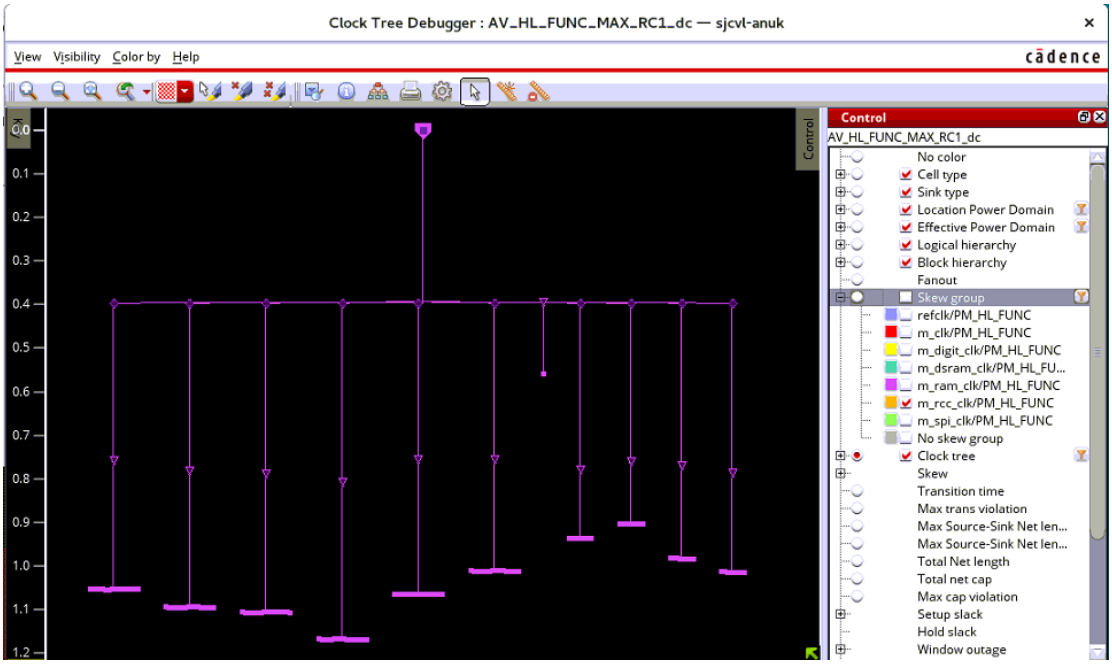
Parameters

`-help`	Outputs a brief description that includes type and default information for each `ctd_show_skew_group` parameter. For a detailed description of the command and all of its parameters, use the man command: man `ctd_show_skew_group`.
`-id window_id_name`	Specifies the CTD window ID. The default value is the ID of the current active CTD window.
`-skew_group skew_group_name`	Specifies the name of the skew group. This is a required parameter.

Example

Use the following command to show the specified skew group in the CTD window:

```
gui_ctd_show_skew_group -skew_group m_rcc_clk/PM_HL_FUNC
```



Related Commands

- gui_ctd_close
- gui_ctd_get_title
- gui_ctd_set_title
- gui_ctd_get_window_id
- gui_ctd_highlight

gui_ctd_write_histogram

```
gui_ctd_write_histogram
[-help]
[-csv]
-file fileName
[-id windowID]
```

Saves the CTD histogram to a file. By default, the file type is PNG. You can also specify that the file be saved in the CSV format.

Before using this command to save the histogram view, first open and configure the histogram in the GUI, including skew group and delay corners.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_ctd_write_histogram` parameter. For a detailed description of the command and all of its parameters, use the man command: man `gui_ctd_write_histogram`.
`-csv`	Saves the histogram in CSV format. This parameter is optional. By default, the histogram is saved as a PNG file.
`-file fileName`	Specifies the name of the file where you want to save the histogram. This parameter is required.
`-id windowIDName`	Specifies the ID of the active CTD window. The default value is the ID of the current active window.

Example

The following command saves the histogram in the `histogram1.csv` file.

```
gui_ctd_write_histogram -file histogram1 -csv
```

Related Commands

- gui_ctd_close
- gui_ctd_highlight
- gui_ctd_zoom
- gui_ctd_open
- gui_ctd_write_view

gui_ctd_zoom

```
gui_ctd_zoom
[-help]
[-id string]
[-in | -out | -selected]
[-vertical_range {ymin ymax}]
```

Zooms into or out of the CTD window.

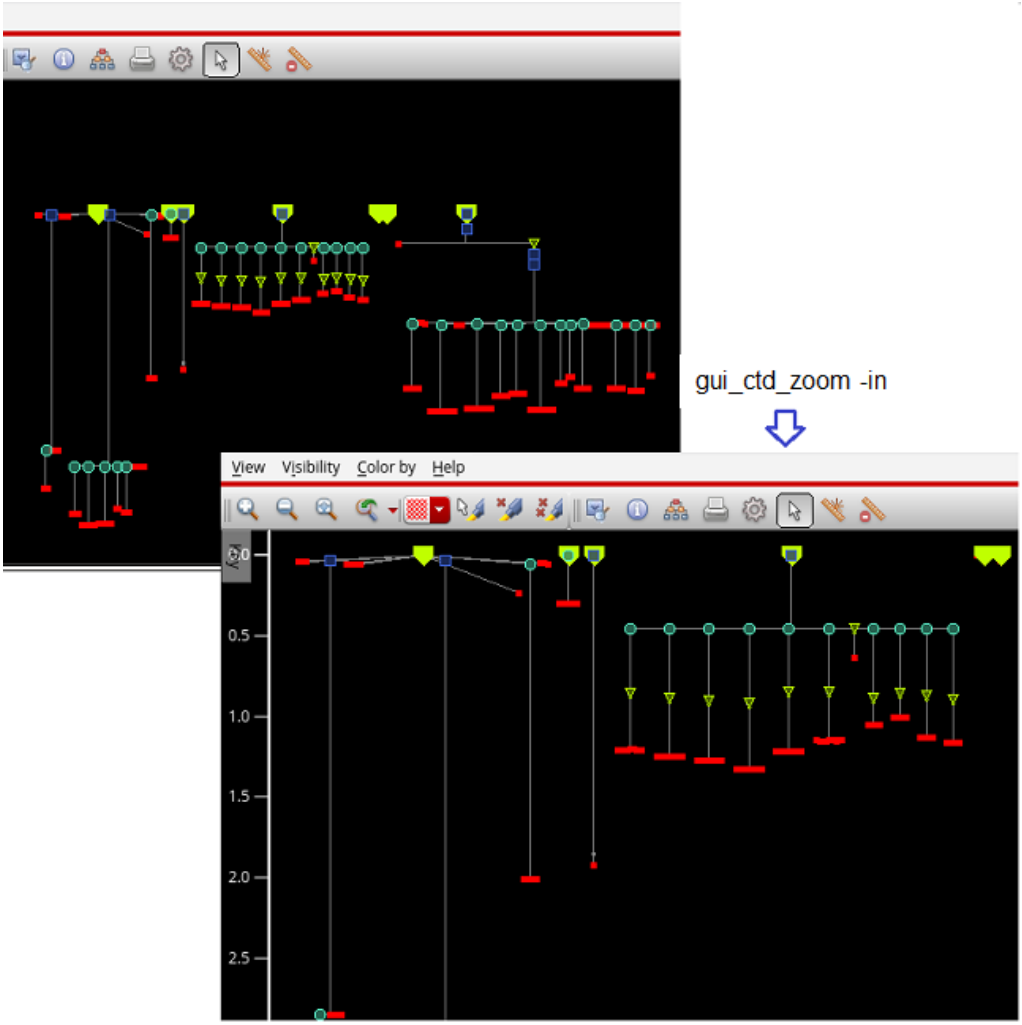
Parameters

`-help`	Prints a brief description of the `gui_ctd_zoom` command. For a detailed description of the command, use the `man` command: `man gui_ctd_zoom`.
`-id string`	Specifies the window ID of the CTD window in which to zoom in or zoom out.
`-in`	Zoom into the CTD window.
`-out`	Zoom out of the CTD window.
`-selected`	Zoom to enclose all selected objects.
`-vertical_range {ymin ymax}`	Zoom to a specific vertical latency range. If CTD is in timing-based mode, the window will zoom to the clock tree parts where latency is between *ymin* and *ymax*, and the unit is the same as global timing unit. If CTD is in unit delay mode, the window will zoom to the clock tree parts where stage depth is between *ymin* and *ymax*. For example, if CTD is in timing-based mode, then the command, `gui_ctd_zoom -vertical_range {0 1.5}` will zoom to the clock tree parts where latency is between 0 and 1.5, the unit is the same as `get_time_unit`. If CTD is in unit delay mode, then the command, `gui_ctd_zoom -vertical_range {0 1.5}` will zoom to the clock tree parts where stage depth is between 0 and 1.5. **Note**: The window only zooms in vertical.

Examples

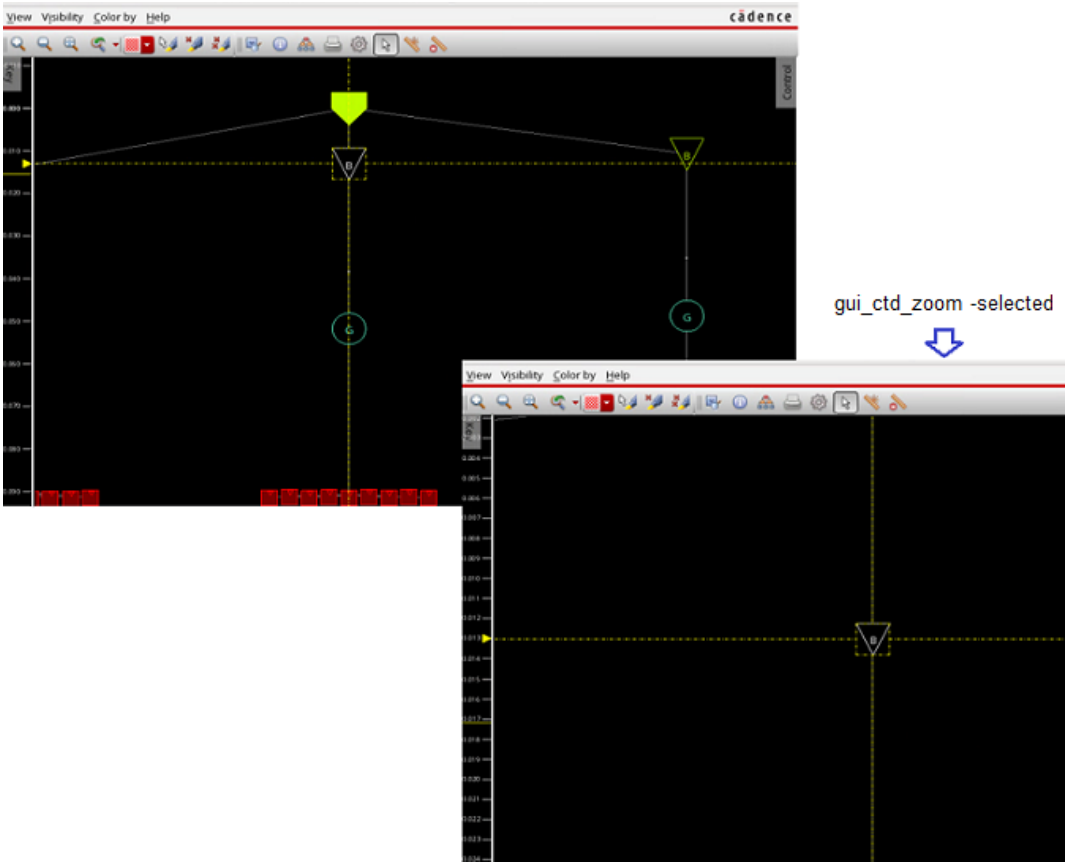
Use the following command to zoom in to the CTD:

```
gui_ctd_zoom -in
```



Select an instance and use the following command to zoom into the selected instance:

```
gui_ctd_zoom -selected
```



Related Commands

- gui_ctd_open
- gui_ctd_close
- gui_ctd_highlight

6

Delay Calculation Commands and Attributes

- create_aging_stress_profile
- delaycal Category Attributes
- get_aging_activity
- get_aging_analysis_mode
- report_aging_activity
- report_aging_stress_profile
- report_delay_calculation
- set_aging_activity
- set_aging_analysis_mode
- write_exclude_nets
- write_set_load

create_aging_stress_profile

```
create_aging_stress_profile
[-help]
[-views {views}]
[-early]
[-late]
[-bti_pmos_scale <scale_val>]
[-bti_nmos_scale <scale_val>]
[-hci_pmos_scale <scale_val>]
[-hci_pmos_scale <scale_val>]
{segment1 segment2 …}
```

Defines the stress profile for aging-aware STA. The profile contains stress duration, stress temperature, and stress voltage per power rail.

You can apply different stress profiles for different views, and different stress profiles for early/late

timing.

Parameters

`-help`	Prints out the command usage.
`-early`	Specifies the early timing path.
`-late`	Specifies the late timing path.
`-bti_pmos_scale <scale_val>`	Specifies the BTI PMOS scale value.
`-bti_nmos_scale <scale_val>`	Specifies the BTI NMOS scale value.
`-hci_pmos_scale <scale_val>`	Specifies the HCI PMOS scale value.
`-hci_nmos_scale <scale_val>`	Specifies the BTI NMOS scale value.
`{segment1 segment2 …}`	Specifies the stress duration, voltages, and temperature information. Here the *segment1*, *segment2* … specifies the stress duration, voltage, and temperature information, for example, `{{duration <float>}[{voltage {<pg_net1> <float> <pg_net2> <float> …}}] [ {temperature <float>}]` }
`-views {views}`	Specifies the views for which stress profile is to be created.

Examples

- The following command specifies a multi-segment stress profile consisting of stress duration, stress temperature, and stress voltage for view “view1”.

  ```
  tempus > create_aging_stress_profile -view view1 \
  {{{ duration 3} {voltage {VDD 0.8 VSS 0.0}}{temperature 125}} \
  {{duration 7} {voltage {VDD 0.45 VSS 0.0}}{temperature 25}}}
  ```

  You can specify initial 3 years of aging with one stress condition and rest of the 7 years with different stress conditions - for a total 10 years of aging:

- The following command specifies the stress profile along with BTI and HCI scales for PMOS and NMOS.

  ```
  tempus > create_aging_stress_profile { { { duration 10 } {voltage {VDD 1.05 VPW
  0.0 VNW \
  1.05 VSS 0.0 }} {temperature 125 } } } -bti_pmos_scale 0.5 -bti_nmos_scale 0.5 \
  -hci_pmos_scale 1 -hci_nmos_scale 1
  ```

Related Commands

- `set_aging_activity`
- `get_aging_activity`
- `report_aging_activity`
- `write_spice_deck`

Related Information

"Aging-Aware STA Analysis" chapter in the *Tempus Stylus User Guide*.

delaycal Category Attributes

The root attributes in the "delaycal" category are:

- delaycal_accuracy_level
- delaycal_accurate_receiver_out_load
- delaycal_advanced_pin_cap_mode
- delaycal_combine_mmmc
- delaycal_disable_randomcell_in_socv
- delaycal_enable_accurate_lle_analysis
- delaycal_degrade_slew_on_early_nets
- delaycal_default_net_delay
- delaycal_default_net_load
- delaycal_disable_positive_slack_shift
- delaycal_enable_corner_based_beol
- delaycal_enable_high_fanout
- delaycal_enable_input_slew_sensitivity_on_constraint
- delaycal_enable_quiet_receivers_for_hold
- delaycal_enable_si
- delaycal_enable_separate_wire_var_sigma_multiplier
- delaycal_enable_wire_load_model
- delaycal_equivalent_waveform_model
- delaycal_equivalent_waveform_type
- delaycal_equivalent_waveform_model_for_timing_check
- delaycal_library_interpolation_mode
- delaycal_honor_slew_propagate_constraint
- delaycal_input_transition_delay
- delaycal_irdrop_data_type

- delaycal_early_irdrop_data_type
- delaycal_late_irdrop_data_type
- delaycal_irdrop_window_based
- delaycal_report_out_bound
- delaycal_separate_params_for_via
- delaycal_slew_out_bound_limit_high
- delaycal_slew_out_bound_limit_low
- delaycal_skip_slew_merge_from_disabled_path
- delaycal_socv_accuracy_mode
- delaycal_socv_lvf_mode
- delaycal_socv_machine_learning_level
- delaycal_socv_use_lvf_tables
- delaycal_support_output_pin_cap
- delaycal_timing_create_clock_use_ideal_slew
- delaycal_use_default_delay_limit
- delaycal_via_corner_sigma_multiplier
- delaycal_via_correlation_type
- delaycal_via_reset
- delaycal_wire_corner_sigma_multiplier
- delaycal_wire_correlation_type
- delaycal_wire_layer_avg_rpt
- delaycal_wire_rc_correlation_factor
- delaycal_wire_reset
- delaycal_wire_use_nsigma_coupling_capacitance
- delaycal_wire_use_nsigma_slew_for_timing_check
- delaycal_wire_var_reporting_sigma_multiplier
- delaycal_wire_variation_correlation_mode

- delaycal_wire_variation_rc_correlation
- delaycal_wire_wire_cell_variation_correlation

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `power` category, use the following command:

```
get_db -category delaycal
```

For delay calculation reports, use the following command:

- `report_delay_calculation`

delaycal_accuracy_level

```
delaycal_accuracy_level level
```

Default: 3

Data_type: Integer

Sets the accuracy level for delay calculations.

You can use this attribute to enable new enhancements for different process nodes, without using the `design_process_node` attribute.

This setting replaces earlier use models where the `set_design_mode -process` parameter was used to enable delay calculation accuracy settings.

The following commands show mapping of old and new options to use for specific nodes:

```
set_design_mode -process 16 == delaycal_accuracy_level 0
set_design_mode -process 10 == delaycal_accuracy_level 1
set_design_mode -process 7 == delaycal_accuracy_level 2
set_design_mode -process 5 == delaycal_accuracy_level 3
```

delaycal_accurate_receiver_out_load

```
delaycal_accurate_receiver_out_load {true | false}
```

Default: `true`

Data_type: Boolean

When set to `true`, Tempus uses distributed interconnect model as receiver output load to compute accurate miller effect.

In this mode, Tempus uses a reduced interconnect model for receivers instead of lumped capacitance as the receiver load. This improves computation of the back miller effect and the delay accuracy.

When set to `false`, the software uses lumped capacitance as the receiver load.

delaycal_advanced_pin_cap_mode

```
delaycal_advanced_pin_cap_mode {0 | 1 | 2}
```

Default: 2

Data_type: Enum

Simulates the current stage and receiver stages together, such that the back miller effect is modeled dynamically.

- `0`: Receiver current sources are computed using ECSM/CCS capacitance tables with the estimated input slew.
- `1`: Receivers are no longer modeled as simple non-linear current source models. This results in increasing the delay calculation accuracy significantly as the modeling back miller effect improves.
- 2: Uses advanced simulation-based approach to model the receiver back miller effect.

delaycal_combine_mmmc

```
delaycal_combine_mmmc {none | early_late | early_late_corner}
```

Default: `early_late`

Data_type: Enum

Specifies whether delay calculation runs are to be combined for delay calculation simulations. This attribute provides user control for improving the runtime for all the analysis types by combining runs for early and late simulations into a single simulation. The views with common delay corners may also be combined into a single delay calculation simulation for added runtime improvement. You can use the following options:

- `none`: Does not combine any delay calculation runs. Individual delay calculation simulations are performed for early and late for example, which will result in significant CPU performance degradation due to the increased number of simulation runs performed.
- `early_late`: Combines early and late simulations of a single corner into one delay calculation simulation and will improve CPU performance by avoiding unnecessary delay calculation simulations.
- `early_late_corner`: Combines early / late simulations of a single corner, like with the `early_late` setting, but also combines views with common delay corners into one single delay calculation simulation. For example, the software will combine views having different `set_case_analysis` statements in one single simulation. This can result in significant improvement in CPU performance by avoiding unnecessary delay calculation simulations.

delaycal_disable_randomcell_in_socv

```
delaycal_disable_randomcell_in_socv {true | false}
```

Default: `false`

Data_type: Boolean

Disables random cell sensitivity in SOCV mode.

delaycal_enable_accurate_lle_analysis

```
delaycal_enable_accurate_lle_analysis {true | false}
```

Default: `false`

Data_type: Boolean

Enables accurate LLE analysis.

delaycal_degrade_slew_on_early_nets

```
delaycal_degrade_slew_on_early_nets {true | false}
```

Default: `true`

Data_type: Boolean

Controls whether slews in early path delay calculation are degraded as they pass across the

interconnect from driver to receiver. You can specify a value of false to turn off interconnect slew degradation for early paths and have the driver slews copied to the receivers. Early path delay calculation affects all Hold paths in best-case/worst-case analysis mode, capture clock paths in on-chip variation Setup analysis, and clock launch and data paths in on-chip variation Hold analysis.

delaycal_default_net_delay

```
delaycal_default_net_delay delay_value
```

Default: 1000ps

Data_type: `String`

Sets the default net delay for delay calculation. The software uses this default value for nets that exceed 1,000 terminals. Delay calculation is performed for nets with fewer than 1,000 terminals.

delaycal_default_net_load

```
delaycal_default_net_load load_value
```

Default: 0.5pF

Data_type: `String`

Sets the default net load for delay calculation. You can use this attribute to specify the default net load value that is used for nets that exceed 1,000 terminals.

delaycal_disable_positive_slack_shift

```
delaycal_disable_positive_slack_shift {true | false}
```

Default: `false`

Data_type: Boolean

Disables positive slack shift caused by back end-of-line (BEOL) variation.

delaycal_enable_corner_based_beol

```
delaycal_enable_corner_based_beol {true | false}
```

Default: `false`

Data_type: Boolean

Enables anchor corner-based BEOL (back end-of-line) analysis.

delaycal_enable_high_fanout

`delaycal_enable_high_fanout {true | false}`

Default: `false`

Data_type: Boolean

When this attribute is set to `true`, the default net delay will be annotated on high fanout nets. A net will be treated as a high fanout net when the fanout is more than the limit specified using the `delaycal_use_default_delay_limit` attribute. The default value of this attribute is 1000 fanouts. The default net delay is also specified using this attribute. The default net delay is 1ns. The cell delay will use the lumped capacitance that is specified using the `delaycal_default_net_load` attribute for table lookup. The default net load is 0.5pF. The driver slew will use the default value of 0ps , which can be changed by using the `delaycal_input_transition_delay` attributes and there will be no slew degradation along the interconnect.

delaycal_enable_input_slew_sensitivity_on_constraint

delaycal_enable_input_slew_sensitivity_on_constraint `{true | false}`

Default: `true`

Data_type: Boolean

Enables the input slew sensitivity on constraints.

When set to `true`, the input slew sensitivities at constrained and related pins will be considered by timing check sigma calculation.

When set to `false`, the timing check arc is derived based on the nominal or mean slews only.

delaycal_enable_quiet_receivers_for_hold

`delaycal_enable_quiet_receivers_for_hold {true | false}`

Default: `false`

Data_type: Boolean

Forces some receivers to be static (“quiet”).

When set to `true`, the software assumes that the receivers are not switching - to compute the worst-

case delays for early analysis. This yields more conservative early delays.

When set to `false`, the software assumes that the receivers may switch.

delaycal_enable_si

```
delaycal_enable_si {true | false}
```

Default: `false`

Data_type: Boolean

Enables SI-Aware delay calculation that also includes cross-talk induced delays.

Note: The SI-Aware delay calculation requires the design to be in Multi-Mode, Multi-Corner (MMMC) Mode and the analysis type to be set to on-chip variations.

delaycal_enable_separate_wire_var_sigma_multiplier

```
delaycal_enable_separate_wire_var_sigma_multiplier {true | false}
```

Default: `false`

Data_type: Boolean

Allows separate reporting sigma multiplier for wire variations.

delaycal_enable_wire_load_model

```
delaycal_enable_wire_load_model {0 | 1}
```

Default: 1

Data_type: Integer

Enables support of Liberty wire-load models and related SDC commands.

delaycal_equivalent_waveform_model

```
delaycal_equivalent_waveform_model {none | no_propagation | propagation}
```

Default: `none`

Data_type: Enum

Specifies the equivalent waveform model to be used.

delaycal_equivalent_waveform_type

```
delaycal_equivalent_waveform_type {moments | simulation}
```

Default: `simulation`

Data_type: Enum

Specifies the equivalent waveform model (EWM) to be used:

The options are described below:

- `moments`: Enables moment-based equivalent waveform model.
- `simulation`: Enables simulation-based equivalent waveform model. The simulation-based EWM is required for predicting delay in noisy waveforms.

It is recommended to use simulation-based EWM as it is more accurate than moment-based EWM.

delaycal_equivalent_waveform_model_for_timing_check

```
delaycal_equivalent_waveform_model_for_timing_check {true | false}
```

Default: `false`

Data_type: Boolean

Models the waveform distortion effect for timing check value look ups at both the data and clock pins.

The model uses simulations to convert the distorted waveforms into equivalent waveforms (pre-driver waveforms in the libraries) by matching the pin CCB output waveforms. This feature requires the CCSN library.

delaycal_library_interpolation_mode

```
delaycal_library_interpolation_mode {linear | non_linear}
```

Default: `linear`

Data_type: Enum

Controls library scaling mode - `linear` or `non_linear` - for the interpolation flow.

delaycal_honor_slew_propagate_constraint

```
delaycal_honor_slew_propagate_constraint {true | false}
```

Default: `true`

Data_type: Boolean

Determines whether to propagate the slew from the disabled timing arcs to the output pin.

When set to `false`, the slew from the disabled timing arc affects the delay of the next stage. Setting this attribute to false results in more accurate slew computation for the disabled timing arcs but can take longer run time.

When set to `true`, the timing constraints defined using commands such as `set_disable_timing` and `set_case_analysis` impact the slew propagation, and the slew from the disabled timing arcs is not propagated.

delaycal_input_transition_delay

```
delaycal_input_transition_delay " "
```

Default: 4ps

Data_type: String

Sets the default input transition delays and the ideal clock transition time for delay calculation. The software uses this default transition time for nets that exceed 1,000 terminals.
It is mandatory to specify time units for this command. Valid time units are ps, ns, etc.
In addition, the default input transition value is applied to those pins to which the slew was not propagated. For example:

- Tiehi or tielo pins
- Macro output pins, which do not have a related input pin

A macro output pin, which has a related input pin will most likely have slew propagation from the related input pin to the output pin.

- Macro output pins for which all the timing arcs are disabled
- Input ports without a user-specified transition value

delaycal_irdrop_data_type

```
delaycal_irdrop_data_type {best_average | worst | best | average | worst_average}
```

Default: `worst`

Data_type: Enum

Allows you to set early/late IR drop EIV values for different EIV voltages.

delaycal_early_irdrop_data_type

```
delaycal_early_irdrop_data_type {best_average | worst | best | average | worst_average}
```

Default: `worst`

Data_type: Enum

Allows you to set early IR drop EIV values for different EIV voltages.

delaycal_late_irdrop_data_type

```
delaycal_late_irdrop_data_type {best_average | worst | best | average | worst_average}
```

Default: `worst`

Data_type: Enum

Allows you to set late IR drop EIV values for different EIV voltages.

delaycal_irdrop_window_based

```
delaycal_irdrop_window_based {none | late | early | both}
```

Default: `both`

Data_type: Enum

Searches window-based EIV values from the EIV database.

delaycal_report_out_bound

```
delaycal_report_out_bound {true | false}
```

Default: `false`

Data_type: Boolean

Generates a report that contains a list of index values (input transition) in the delay tables that are

beyond the index range. The index values detected by this attribute can cause inaccurate delay values to be calculated. The report generated by this attribute can be used for debugging.

When this attribute is set to `true`, the extrapolation information will be dumped into the following two files:

- <design-name>.dc.outbound.slew.early.<view-name>.txt
- <design-name>.dc.outbound.slew.late.<view-name>.txt

The software writes a warning message in the log files noting that there are input transition values that are out of the index range of the timing table. The warning message also provides the file name for viewing the detail report.

delaycal_separate_params_for_via

```
delaycal_separate_params_for_via {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software creates separate parameters for via variation.

delaycal_slew_out_bound_limit_high

```
delaycal_slew_out_bound_limit_high double
```

Default: 3.40282e+38

Data_type: Double

Limits the maximum slew used in the analysis to the specified value times max characterized slew in a library.

delaycal_slew_out_bound_limit_low

```
delaycal_slew_out_bound_limit_low double
```

Default: 0.5

Data_type: Double

Limits the minimum slew used in the analysis to the specified value times min characterized slew in a library.

delaycal_skip_slew_merge_from_disabled_path

```
delaycal_skip_slew_merge_from_disabled_path {true | false}
```

Default: `false`

Data_type: Boolean

Avoids merging slews from unconstrained, segmented paths and black-box output pin.

When set to `true`, the software does not allow merging transitions from disabled paths.

delaycal_socv_accuracy_mode

```
delaycal_socv_accuracy_mode {low | medium | high | ultra}
```

Default: `low`

Data_type: Enum

Allows the ability to control levels of accuracy.

This attribute specifies level-based accuracy effort for delays and slews. You can use one of the following options:

- `low`: Ignores correlation between slews and delays.
- `medium`: Models correlation between immediate slew and delay.
- `high`: Considers skewness of non-gaussian distribution into account for computing quantiles. The skewness is local to one stage only.
- `ultra`: Considers skewness of non-gaussian distribution, and also propagates skewness to the next stage in PBA analysis.

delaycal_socv_lvf_mode

```
delaycal_socv_lvf_mode string
```

Default: `moments`

Data_type: Enum

Controls SOCV LVF analysis modes. You can specify `moments` or `early_late` option.

delaycal_socv_machine_learning_level

```
delaycal_socv_machine_learning_level {0 | 1 | 3}
```

Default: 0

Data_type: Integer

Enables the SOCV machine learning mode.

For accurate SOCV analysis, the Tempus software creates several statistical models, which are not available with standard LVF library, such as correlation between cells delay and output slew, the conversion of moments from late/early sigma tables, and effects of waveform distortion on sigma and mean shift.

You can specify one of the following options:

- `1`: Enables several machine learning algorithms to generate such models internally.
- `3`: Enabling this mode uses improved modeling of slew-delay correlation using advanced machine learning techniques. This option enables accurate quantile modeling, which involves quantile computation from mean, stddev and skewness values.

Note: The default setting of `0` is recommended for nodes 110, 40, 28, and 22nm. For other technology nodes, option 3 can be used. For this setting you must enable the ultra mode using `-socv_accuracy_mode ultra` option.

delaycal_socv_use_lvf_tables

```
delaycal_socv_use_lvf_tables string
```

Default: `all`

Data_type: String

Controls which type of variations – `all`, `delay`, `slew`, or `constraint` - are considered during analysis.

delaycal_support_output_pin_cap

```
delaycal_support_output_pin_cap {true | false}
```

Default: `true`

Data_type: Boolean

Includes the output pin capacitance when calculating delay values. When you specify `false`, the

software excludes only the pin capacitance of the driver for which you are calculating the delay.

delaycal_timing_create_clock_use_ideal_slew

`delaycal_timing_create_clock_use_ideal_slew {true | false}`

Default: `false`

Data_type: Boolean

Enables the use of ideal slew at the inputs of the clock driver cells.

When set to `true`, the software detects created clocks, and uses zero input slew. Other slew values, including those set using `set_annotated_transition` command, will be ignored.

When set to `false`, the software honors `set_annotated_transition` command settings.

delaycal_use_default_delay_limit

`delaycal_use_default_delay_limit` *`value`*

Default: 1000

Data_type: Integer

Sets the pin number threshold for nets for delay calculation. Nets that have more than the specified number of pins are excluded from delay calculation, and use the default delay values instead.
Net default delay values are:

- Net delay: 1 ns
- Transition time: 0 ps
- Net load: 0.5 pF

Note: In post route stage, default delay calculation does not honor this attribute. Slews and delays are computed based on actual parasitics.
To change the default delay values, set the following attributes:

- `delaycal_default_net_delay`
- `delaycal_default_net_load`
- `delaycal_input_transition_delay`

delaycal_via_corner_sigma_multiplier

`delaycal_via_corner_sigma_multiplier` *`String`*

Default: " "

Data_type: string

Provides the scaling multiplier for corner scaling. This attribute allows you to specify per via sigma multiplier and handles the advanced via variation scaling.

delaycal_via_correlation_type

`delaycal_via_correlation_type` *`String`*

Default: `corner`

Data_type: Enum

Specifies the inter-layer correlation values.

delaycal_via_reset

`delaycal_via_reset {true | false}`

Default: `false`

Data_type: Boolean

When set to `true`, the software resets the via variation modes to their default value.

delaycal_wire_corner_sigma_multiplier

`delaycal_wire_corner_sigma_multiplier` *`string`*

Default: " "

Data_type: string

Provides scaling multiplier for corner scaling.

delaycal_wire_correlation_type

`delaycal_wire_correlation_type {statistical | binomial | corner | correlated}`

Default: `statistical`

Data_type: Enum

Provides control to handle inter-layer correlation. The accepted values are - statistical, binomial, corner, and correlated.

delaycal_wire_layer_avg_rpt

`delaycal_wire_layer_avg_rpt {true | false}`

Default: `false`

Data_type: Boolean

Enables layer averaging reporting.

delaycal_wire_rc_correlation_factor

`delaycal_wire_rc_correlation_factor` *`int`*

Default: 1

Data_type: Integer

Allows same layer R and C correlation. The accepted values are 1 and -1.

delaycal_wire_reset

`delaycal_wire_reset {true | false}`

Default: `false`

Data_type: Boolean

Resets wire variation modes to the default value.

delaycal_wire_use_nsigma_coupling_capacitance

`delaycal_wire_use_nsigma_coupling_capacitance {true | false}`

Default: `false`

Data_type: Boolean

Allows to use mean and nSigma coupling capacitance.

delaycal_wire_use_nsigma_slew_for_timing_check

```
delaycal_wire_use_nsigma_slew_for_timing_check {true | false}
```

Default: `false`

Data_type: Boolean

Enables use of wire variation-aware slews for timing check lookup.

delaycal_wire_var_reporting_sigma_multiplier

```
delaycal_wire_var_reporting_sigma_multiplier double
```

Default: `0.0`

Data_type: Double

Specifies deterministic sigma multiplier for reporting BEOL variations.

delaycal_wire_variation_correlation_mode

```
delaycal_wire_variation_correlation_mode {statistical | binomial}
```

Default: `statistical`

Data_type: Enum

Specifies wire variation inter-layer correlation. When set to `statistical`, all the layers are considered independent of each other.

delaycal_wire_variation_rc_correlation

```
delaycal_wire_variation_rc_correlation double
```

Default: `1.0`

Data_type: Double

Specifies same layer R and C correlation for wire variation. When set to 1, the R and C sensitivities are considered as fully correlated.

delaycal_wire_wire_cell_variation_correlation

```
delaycal_wire_wire_cell_variation_correlation {true | false}
```

Default: `false`

Data_type: Boolean

Controls whether to RSS or add random cell variation and wire variation for arrivals and slack.

get_aging_activity

```
get_aging_activity
[-help]
-net <string>
[-views <string>]
```

Returns the current settings of the `set_aging_activity` command.

Parameters

`-help`	Prints out the command usage.
`-net <string>`	Returns the net names.
`-views <string>`	Returns a list of the specified views.

Related Commands

- `create_aging_stress_profile`
- `set_aging_activity`
- `report_aging_activity`
- `write_spice_deck`

Related Information

"Aging-Aware STA Analysis" chapter in the *Tempus Stylus User Guide*.

get_aging_analysis_mode

```
get_aging_analysis_mode
[-help]
[-activity_propagation_mode]
[-constraints]
[-disable_aging]
[-max_duty_cycle]
[-max_toggle_rate]
[-max_toggle_rate_limit <value>]
[-min_duty_cycle]
[-min_toggle_rate]
[-param_priority_sets]
[-recovery]
[-sequential_activity_propagation]
[-sequential_max_duty_cycle]
[-sequential_max_toggle_rate]
[-sequential_min_duty_cycle]
[-sequential_min_toggle_rate]
[-type]
[-use_fixed_duty_cycle]
[-views]
```

Returns the aging analysis related global variable settings and defaults - that have been made using the `set_aging_analysis_mode` command.

Parameters

Displays the following information about the specified `set_aging_analysis_mode` parameters in the Tempus log file and the Tempus console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify any parameters, the software displays information for all of the parameters.

Parameters

`-help`	Prints out the command usage.
parameter_names	Displays information for the specified parameters. You can specify one or more parameters. See `set_aging_analysis_mode` for descriptions of the parameters you can specify.

Related Commands

- `set_aging_analysis_mode`
- `set_db timing_analysis_aging`
- `set_aging_activity`
- `create_aging_stress_profile`
- `write_spice_deck`

Related Information

- "Aging-Aware STA Analysis" chapter in the *Tempus Stylus User Guide*.

report_aging_activity

```
report_aging_activity
[-help]
[-duty_cycle]
-net <string>
[-out_file <string>]
[-toggle_rate]
[-views <string>]
```

Reports the duty cycles and toggle rates.

Parameters

`-help`	Prints out the command usage.
`-duty_cycle`	Specifies the dump duty cycles.
`-net <string>`	Specifies the net names.
`-out_file <string>`	Specifies the output file name.
`-toggle_rate`	Specifies the dump toggle rates.
`-views <string>`	Specifies the list of view names.

Examples

- The following command reports the toggle rate and duty cycle information on net `net_dcu[36]`. The output file “out.txt” will contain the required information:

  ```
  tempus > report_aging_activity -duty_cycly -toggle_rate -net net_dcu[36] -out_file
  out.txt
  ```

Related Commands

- `create_aging_stress_profile`
- `set_aging_activity`
- `get_aging_activity`
- `write_spice_deck`

Related Information

"Aging-Aware STA Analysis" chapter in the *Tempus User Guide*.

report_aging_stress_profile

```
report_aging_stress_profile
[-help]
[-early]
[-late]
[-views <string>]
```

Reports the stress profiles for aging-aware STA defined using the `create_aging_stress_profile` command. The report contains stress duration, stress temperature, and stress voltage per power rail.

Parameters

`-help`	Prints out the command usage.
`-early`	Reports the stress profile associated with early paths.
`-late`	Reports the stress profile associated with late paths.
`-views <string>`	Specifies a list of view names associated with the aging stress profile.

Related Commands

- `create_aging_stress_profile`
- `set_aging_activity`
- `get_aging_activity`
- `report_aging_activity`
- `write_spice_deck`

Related Information

- "Aging-Aware STA Analysis" chapter in the *Tempus Stylus User Guide*.

report_delay_calculation

```
report_delay_calculation
[-help]
[-active_arcs]
-from port_or_pin_name
[-max]
[-min]
[-mis]
[-out_file string]
[-si]
[-show_all_attackers]
[-thresholds]
-to port_or_pin_name
[-view viewname]
[-voltage]
[-waveform]
```

Reports the delay calculation information for a cell or net timing arc. You can use this command to report delay calculation details for the specified timing arc. The delay calculation report may be useful for debugging timing or for verifying timing data.

You can run the `report_timing` command before running `report_delay_calculation` to have the correct timing window and aggressor slew.

Note: The `report_delay_calculation` command does not support AOCV derating.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_delay_calculation` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_delay_calculation`.
`-active_arcs`	Disables reporting of all the non-active parallel delay arcs and reports only the valid check arcs.

`-from` *port_or_pin_name*	Specifies the name of the From port or pin. If the From port or pin is output of a cell and To port or pin is input of a cell, delay is calculated for the net and the delay type in the report is 'net delay.' If the From port or pin is input of a cell and To port or pin is output of a cell, delay is calculated for the cell and the delay type in the report is 'cell delay.'
`-max`	Indicates that the delay calculation report is for the worst-case operating conditions in min-max mode. If neither `min` nor `max` arguments are specified, then the default mode is set to `max`.
`-min`	Indicates that the delay calculation report is for the best-case operating conditions in min-max mode. If neither `min` nor `max` arguments are specified, then the default mode is set to `max`.
`-mis`	Reports advanced multi-input switching (AMIS) details.
`-out_file` *filename*	Specifies the name of the output file in which to save the delay calculation report.
`-si`	Reports signal integrity (SI) related information.
`-show_all_attackers`	Includes all the attackers in the report.
`-thresholds`	Displays the rise delay, rise slew, fall delay, fall slew, and slew delay threshold values for the From and To pins.
`-to` *port_or_pin_name*	Specifies the name of the To port or pin. If the From port or pin is output of a cell and To port or pin is input of a cell, delay is calculated for the net and the delay type in the report is 'net delay.' If the From port or pin is input of a cell and To port or pin is output of a cell, delay is calculated for the cell and the delay type in the report is 'cell delay.'
`-view` *viewname*	Specifies the view name for which to print the delay calculation report when using the multi-mode multi-corner (MMMC) mode.
`-voltage`	Displays the voltage of the cell or net timing arc.
`-waveform`	Reports the waveforms of the pins.

Examples

- The following command shows the cell delay calculation report for the from pin I1/A and to pin I1/Y:

```
tempus> report_delay_calculation -from I1/A -to I1/Y

From pin : I1/A

To Pin : I1/Y

Cell : S2INVV1D1

Library : test

Arc sense : negative unate

Delay type : cell delay

-----------------------------------

RC Summary for net N2

-----------------------------------

Number of capacitance : 7

Net capacitance : 0.026057 pF

Total rise capacitance: 0.027172 pF

Total fall capacitance: 0.027169 pF

Number of resistance : 7

Total resistance : 1094.135127 Ohm

-----------------------------------
```

- The following command shows the net delay calculation report for the from pin ff1/Q and to pin A1/buf1/A with waveforms:

```
tempus> report_delay_calculation -from ff1/Q -to A1/buf1/A -waveform


From pin : ff1/Q
Cell : DFFX2
Library : gpdk045bc
To pin : A1/buf1/A
Cell : BUFX2
```

```
Library : gpdk045bc
Delay type: net

------------------------------------------------------------
From pin waveforms
------------------------------------------------------------
Rise Waveform : { 0.00e+00 0.00000 5.35e-10 0.05400 1.07e-09 0.10800 1.61e-09
0.16200
2.14e-090.21600 2.68e-09 0.27000 3.21e-09 0.32400 3.75e-09 0.37800 4.28e-09
0.43200
4.82e-09 0.486005.35e-09 0.54000 5.89e-09 0.59400 6.42e-09 0.64800 6.96e-09
0.70200
7.50e-09 0.756008.03e-09 0.81000 8.57e-09 0.86400 9.10e-09 0.91800 9.64e-09
0.97200
1.02e-08 1.02600 1.07e-08 1.08000 }

Fall Waveform : { 0.00e+00 1.08000 3.69e-10 1.02600 7.39e-10 0.97200 1.11e-09
0.91800
1.48e-09 0.86400 1.85e-09 0.81000 2.22e-09 0.75600 2.59e-09 0.70200 2.96e-09
0.64800
3.32e-09 0.59400 3.69e-09 0.54000 4.06e-09 0.48600 4.43e-09 0.43200 4.80e-09
0.37800
5.17e-09 0.32400 5.54e-09 0.27000 5.91e-09 0.21600 6.28e-09 0.16200 6.65e-09
0.10800
7.02e-09 0.05400 7.39e-09 -0.00000 }
------------------------------------------------------------
To pin waveforms
------------------------------------------------------------
Rise Waveform : { 2.73e-14 0.00000 5.35e-10 0.05318 1.07e-09 0.10793 1.61e-09
0.16125
2.14e-09 0.21587 2.68e-09 0.26930 3.21e-09 0.32383 3.75e-09 0.37734 4.28e-09
0.43179
4.82e-09 0.48538 5.35e-09 0.53975 5.89e-09 0.59341 6.42e-09 0.64772 6.96e-09
0.70144
```

```
7.50e-09 0.75570 8.03e-09 0.80946 8.57e-09 0.86368 9.10e-09 0.91748 9.64e-09
0.97166
1.02e-08 1.02549 1.07e-08 1.07965 }

Fall Waveform : { 2.38e-14 1.08000 3.69e-10 1.02719 7.39e-10 0.97209 1.11e-09
0.91911
1.48e-09 0.86416 1.85e-09 0.81104 2.22e-09 0.75623 2.59e-09 0.70298 2.96e-09
0.64828
3.32e-09 0.59493 3.69e-09 0.54033 4.06e-09 0.48688 4.43e-09 0.43237 4.80e-09
0.37885
5.17e-09 0.32440 5.54e-09 0.27081 5.91e-09 0.21643 6.28e-09 0.16279 6.65e-09
0.10846
7.02e-09 0.05476 7.39e-09 0.00048 }
-------------------------------------------------------------


-------------------------------------------------------------
RC Summary for net n2
-------------------------------------------------------------
Number of capacitance : 19
Net capacitance : 0.714975 pF
Total rise capacitance: 0.716177 pF
Total fall capacitance: 0.716179 pF
Number of resistance : 24
Total resistance : 101.624695 Ohm
-------------------------------------------------------------
Rise Fall
-------------------------------------------------------------
Net delay : 0.002500 ns 0.002300 ns
From pin transition time: 8.565800 ns 5.910500 ns
To pin transition time : 8.568400 ns 5.913000 ns
-------------------------------------------------------------
```

- The following command shows the attacker status in the SI delay calculation report output (when `si_delay_enable_report` attribute is set to `true`):

```
tempus> report_delay_calculation -si -from seg3/u5/Y -to seg3/u6/A
```

```
From pin : seg3/u5/Y
Cell : INV
Library : cell_w
To pin : seg3/u6/A
Cell : INV
Library : cell_w
Base or SI: SI delay
Delay type: net

-------------------------------------------------------------
RC Summary for net seg3/n5
-------------------------------------------------------------
Number of capacitance : 17
Net capacitance : 0.293534 pF
Total rise capacitance: 0.320849 pF
Total fall capacitance: 0.320780 pF
Number of resistance : 17
Total resistance : 567.671387 Ohm
-------------------------------------------------------------
SI information for rise
-------------------------------------------------------------------------------------
Victim timing window : 1.0743 ns 2.7024 ns CLK_W_3
-------------------------------------------------------------------------------------
Attacker Status Direction Slew Coupling cap Fanout Voltage Bump Timing window
Clock
-------------------------------------------------------------------------------------
seg2/n5 INF Fall 0.303667 ns 0.138759 pF 1 1.080 v 0.198396 v 1.1917 ns 2.8626 ns
CLK_W_2
seg1/n5 INF Fall 0.314333 ns 0.138787 pF 1 1.080 v 0.182412 v 1.3881 ns 3.0723 ns
CLK_W_1
-------------------------------------------------------------------------------------
SI information for fall
-------------------------------------------------------------------------------------
Victim timing window : 1.2016 ns 5.0700 ns CLK_W_3
-------------------------------------------------------------------------------------
Attacker Status Direction Slew Coupling cap Fanout Voltage Bump Timing window
Clock
-------------------------------------------------------------------------------------
seg2/n5 INF Rise 0.550667 ns 0.138759 pF 1 1.080 v 0.138780 v 1.0887 ns 2.0217 ns
CLK_W_2
seg1/n5 INF Rise 0.800167 ns 0.138787 pF 1 1.080 v 0.107892 v 1.2751 ns 2.3031 ns
CLK_W_1
-------------------------------------------------------------------------------------
```

```
Model types
-------------------------------------------------------------------------------
Net Term Cell Model
-------------------------------------------------------------------------------
seg3/n5 seg3/u5/Y INV NLDM
seg3/n5 seg3/u6/A INV NLDM
seg2/n5 Y INV NLDM
seg1/n5 Y INV NLDM
-------------------------------------------------------------------------------
Delays Rise Fall
------------------------------------------------------------
Incr Net Delay : 0.565900 ns 0.093700 ns
Net delay (mean) : 0.696000 ns 0.188900 ns
(3-sigma) : 0.000000 ns 0.000000 ns
Timing Derate (mean) : 1.000000 1.000000
(sigma) : 1.000000 1.000000
Derated Net delay (mean) : 0.696000 ns 0.188900 ns
(3-sigma) : 0.000000 ns 0.000000 ns
------------------------------------------------------------

------------------------------------------------------------
Transitions Rise Fall
------------------------------------------------------------
Input transition (mean) : 0.482600 ns 0.234800 ns
(sigma) : 0.000000 ns 0.000000 ns
Output transition (mean) : 0.622200 ns 0.274100 ns
(sigma) : 0.000000 ns 0.000000 ns
------------------------------------------------------------
Abbreviative attacker status and filtered reason:
------------------------------------------------------------
SY : Synchronous
INF: Attacker with infinite timing window
PE : Filtered due to physical exclusion
UF : Filtered by user
CN : Filtered because it is a constant net
SB : Filtered due to small bump
TA : Filtered because of timing window does not overlap
LC : Filtered due to logical correlation
ACT: Attacker is Active.
------------------------------------------------------------
```

- The following command reports only the valid check arcs:

```
report_delay_calculation  -from  E_IG_C60_ds_dtin_11/A -to E_IG_C60_ds_dtin_11/Z -
```

```
active_arcs

From pin   : E_IG_C60_ds_dtin_11/A
To Pin     : E_IG_C60_ds_dtin_11/Z
Cell       : BUF_X32
Library    : FreePDK45_lib_v1.0
Arc sense  : positive unate
Delay type : cell delay

Base or SI : SI delay
------------------------------------------------------------
RC Summary for net FE_SIG_N660_ds_datain_11_
------------------------------------------------------------
Number of capacitance : 28
Net capacitance       : 0.012320 pF
Total rise capacitance: 0.013380 pF
Total fall capacitance: 0.013362 pF
Number of resistance  : 27
Total resistance      : 494.409882 Ohm
------------------------------------------------------------
               Rise        Fall
------------------------------------------------------------
Input transition time  : 0.182400 ns 0.146100 ns
Cell delay             : 0.333500 ns 0.404900 ns
Timing Derate          : 1.010000    1.010000
Derated Cell delay     : 0.336900 ns 0.409000 ns
Output transition time : 0.275200 ns 0.145200 ns
------------------------------------------------------------

Total number of fetched objects 1
AAE_INFO: Total number of nets for which stage creation was skipped for all views
0
```

- The following command displays the threshold values for specified from and to pins:

```
report_delay_calculation -from E_IG_C60_ds_dtin_11/A -to E_IG_C60_ds_dtin_11/Z -
thresholds

From pin   : E_IG_C60_ds_dtin_11/A
To Pin     : E_IG_C60_ds_dtin_11/Z
Cell       : BUF_X32
Library    : FreePDK45_lib_v1.0
Arc sense  : positive unate
Delay type : cell delay

Base or SI : SI delay
              Rise   Rise      Fall   Fall      Slew
```

```
Thresholds:   Delay  Slew      Delay  Slew      Derate
From pin  :   50     10->90    50     90->10    1.000
To pin    :   50     10->90    50     90->10    1.000
-------------------------------------------------------------
RC Summary for net E_IG_N60_ds_dtin_11
-------------------------------------------------------------
Number of capacitance : 28
Net capacitance       : 0.012320 pF
Total rise capacitance: 0.013380 pF
Total fall capacitance: 0.013362 pF
Number of resistance  : 27
Total resistance      : 494.409882 Ohm
-------------------------------------------------------------
               Rise        Fall
-------------------------------------------------------------
Input transition time  : 0.182400 ns 0.146100 ns
Cell delay             : 0.333500 ns 0.404900 ns
Timing Derate          : 1.010000    1.010000
Derated Cell delay     : 0.336900 ns 0.409000 ns
Output transition time : 0.275200 ns 0.145200 ns
-------------------------------------------------------------

Total number of fetched objects 1
AAE_INFO: Total number of nets for which stage creation was skipped for all views
0
```

- The following command displays the delay calculation report for the worst-case operating conditions in min-max mode:

```
report_delay_calculation -from E_IG_C60_ds_dtin_11/A -to E_IG_C60_ds_dtin_11/Z -
max

From pin   : E_IG_C60_ds_dtin_11/A
To Pin     : E_IG_C60_ds_dtin_11/Z
Cell       : BUF_X32
Library    : FreePDK45_lib_v1.0
Arc sense  : positive unate
Delay type : cell delay

Base or SI : SI delay
-------------------------------------------------------------
RC Summary for net E_IG_N60_ds_dtin_11
-------------------------------------------------------------
Number of capacitance : 28
Net capacitance       : 0.012320 pF
```

```
Total rise capacitance: 0.013380 pF
Total fall capacitance: 0.013362 pF
Number of resistance  : 27
Total resistance      : 494.409882 Ohm
-------------------------------------------------------------
               Rise        Fall
-------------------------------------------------------------
Input transition time  : 0.182400 ns 0.146100 ns
Cell delay             : 0.333500 ns 0.404900 ns
Timing Derate          : 1.010000    1.010000
Derated Cell delay     : 0.336900 ns 0.409000 ns
Output transition time : 0.275200 ns 0.145200 ns
-------------------------------------------------------------

Total number of fetched objects 1
AAE_INFO: Total number of nets for which stage creation was skipped for all views
0
```

- The following command displays the delay calculation report for the best-case operating conditions in min-max mode:

```
report_delay_calculation -from  E_IG_C60_ds_dtin_11/A  -to  E_IG_C60_ds_dtin_11/Z
-min

From pin   : E_IG_C60_ds_dtin_11/A
To Pin     : E_IG_C60_ds_dtin_11/Z
Cell       : BUF_X32
Library    : FreePDK45_lib_v1.0
Arc sense  : positive unate
Delay type : cell delay

Base or SI : SI delay
-------------------------------------------------------------
RC Summary for net E_IG_N60_ds_dtin_11
-------------------------------------------------------------
Number of capacitance : 28
Net capacitance       : 0.012320 pF
Total rise capacitance: 0.013380 pF
Total fall capacitance: 0.013362 pF
Number of resistance  : 27
Total resistance      : 494.409882 Ohm
-------------------------------------------------------------
               Rise        Fall
-------------------------------------------------------------
Input transition time  : 0.160100 ns 0.172500 ns
```

```
Cell delay             : 0.289000 ns 0.457800 ns
Timing Derate          : 0.820000    0.820000
Derated Cell delay     : 0.237000 ns 0.375400 ns
Output transition time : 0.275200 ns 0.145800 ns
-------------------------------------------------------------

Total number of fetched objects 1
```

- The following command displays all the attackers for the specified view:

```
report_delay_calculation -si -from E_IG_C60_ds_dtin_11/A -to
 E_IG_C60_ds_dtin_11/Z -show_all_attackers -view scan_slow_RCMAX

From pin   : E_IG_C60_ds_dtin_11/A
To Pin     : E_IG_C60_ds_dtin_11/Z
Cell       : BUF_X32
Library    : FreePDK45_lib_v1.0
Arc sense  : positive unate
Delay type : cell delay

Base or SI : SI delay
-------------------------------------------------------------
RC Summary for net E_IG_N60_ds_dtin_11
-------------------------------------------------------------
Number of capacitance : 28
Net capacitance       : 0.012320 pF
Total rise capacitance: 0.013380 pF
Total fall capacitance: 0.013362 pF
Number of resistance  : 27
Total resistance      : 494.409882 Ohm
-------------------------------------------------------------
SI information for rise
-------------------------------------------------------------------------------------
Victim timing window  :  7.3136 ns 15.1300 ns  m_clk
-------------------------------------------------------------------------------------
-----------------------------------------
Attacker              Status  Direction  Slew         Coupling cap  Fanout
Voltage  Bump       Timing window           Clock
-------------------------------------------------------------------------------------
-----------------------------------------
E_IG_N636_ds_data_8   INF     Fall       0.187157 ns 0.002732 pF     1     0.800
v  0.110173 v Infinity timing window

-------------------------------------------------------------------------------------
SI information for fall
-------------------------------------------------------------------------------------
```

```
Victim timing window  :  7.6085 ns 13.4483 ns  m_clk
-------------------------------------------------------------------------------
Attacker               Status  Direction  Slew         Coupling cap  Fanout
 Voltage  Bump        Timing window         Clock
-------------------------------------------------------------------------------
-----------------------------------------
E_IG_N636_ds_data_8    INF     Rise       0.685944 ns  0.002732 pF    1      0.800
v  0.037733 v Infinity timing window

-------------------------------------------------------------------------------
Model types
-------------------------------------------------------------------------------
Net                        Term                                  Cell     Model
-------------------------------------------------------------------------------
E_IG_N60_ds_dtin_11        E_IG_C60_ds_dtin_11/Z  BUF_X32     NLDM

                          Rise        Fall
-------------------------------------------------
Input transition time  : 0.182400 ns 0.146100 ns
Cell delay             : 0.371700 ns 0.420300 ns
Timing Derate          : 1.010000    1.010000
Derated Cell delay     : 0.375500 ns 0.424600 ns
Output transition time : 0.275000 ns 0.145000 ns
-------------------------------------------------
```

- The following command displays the voltage of the cell:

```
report_delay_calculation -from E_IG_C486_ds_data_6/A -to E_IG_C486_ds_data_6/Z -
voltage

From pin   : E_IG_C486_ds_data_6/A
Voltage    : 0.8 V
To Pin     : E_IG_C486_ds_data_6/Z
Voltage   : 0.8 V
Cell       : BUF_X32
Library    : FreePDK45_lib_v1.0
Arc sense  : positive unate
Delay type : cell delay

Base or SI : SI delay
-------------------------------------------------------------
RC Summary for net E_IG_N671_FE_C486_ds_data_6
-------------------------------------------------------------
Number of capacitance : 31
Net capacitance       : 0.027240 pF
```

```
Total rise capacitance: 0.027611 pF
Total fall capacitance: 0.027598 pF
Number of resistance  : 30
Total resistance      : 1311.349609 Ohm
                         Rise        Fall
------------------------------------------------------------
Input transition time  : 0.075200 ns 0.058500 ns
Cell delay             : 0.435900 ns 0.407500 ns
Timing Derate          : 1.010000    1.010000
Derated Cell delay     : 0.440300 ns 0.411600 ns
Output transition time : 0.538800 ns 0.194900 ns
------------------------------------------------------------

Total number of fetched objects 1
AAE_INFO: Total number of nets for which stage creation was skipped for all views
0
```

Related Commands

- report_timing

set_aging_activity

```
set_aging_activity
[-help]
[-max_duty_cycle <float>]
[-max_toggle_rate <(number of transitions per second)>]
[-min_duty_cycle <float>]
[-min_toggle_rate <float>]
-net <string>
[-views <string>]
```

Defines the duty cycles on nets.

By default, all the signals in STA are assumed to be 50% of duty cycle at primary pins. The duty cycle for clock waveforms can be specified in the SDC. The same duty cycle is assumed at every clock pin associated with that waveform. You can override the duty cycle defaults for primary ports, as well as force duty cycles on specific nets in the design.

Starting from the initial default values (and user-overrides), Tempus propagates the duty cycle across the logic. Tempus gives higher priority to user-defined duty cycle than duty cycle obtained from the SDC, using either constants or clock waveforms.

Parameters

`-help`	Prints out the command usage.
`-max_duty_cycle <float>`	Specifies the maximum duty cycle. Min: 0, Max: 1
`-max_toggle_rate <(number of transitions per second)>`	Specifies the maximum toggle rate. Min: 0, Max: 3.40282346638529e+38
`-min_duty_cycle <float>`	Specifies the minimum duty cycle. Min: 0, Max: 1
`-min_toggle_rate <float>`	Specifies the minimum toggle rate (number of transitions per second). Min: 0, Max: 3.40282346638529e+38
`-net <string>`	Specifies the net names.
`-views <string>`	Specifies the list of view names.

Examples

- The following command specifies the maximum duty cycle of 0.5 and minimum duty cycle of 0.3 on net `net_ops[0]` for view `view1`:

  ```
  tempus > set_aging_activity -max_duty_cycle 0.5 -min_duty_cycle 0.3 -net
  net_ops[0] -view view1
  ```

- The following command specifies the maximum and minimum toggle rate on net `net_pfs[45]`:

  ```
  tempus > set_aging_activity -min_toggle_rate 1e6 -max_toggle_rate 3e9 -net
  net_pfs[45]
  ```

Related Commands

- `create_aging_stress_profile`
- `get_aging_activity`
- `report_aging_activity`
- `write_spice_deck`

Related Information

- "Aging-Aware STA Analysis" chapter in the *Tempus User Guide*.

set_aging_analysis_mode

```
set_aging_analysis_mode
[-help]
[-activity_propagation_mode {probabilistic | worst_case}]
[-constraints {true | false}]
[-disable_aging]
[-max_duty_cycle <value>]
[-max_toggle_rate <value>]
[-max_toggle_rate_limit <value>]
[-min_duty_cycle <value>]
[-min_toggle_rate <value>]
[-param_priority_sets <string>]
[-recovery {true | false}]
[-sequential_activity_propagation {true | false}]
[-sequential_max_duty_cycle <value>]
[-sequential_max_toggle_rate <value>]
[-sequential_min_duty_cycle <value>]
[-sequential_min_toggle_rate <value>]
[-type {bti | hci | all}]
[-use_fixed_duty_cycle {true | false}]
[-views <string>]
```

Sets the aging analysis related global variable settings and defaults.

By default, all the signals in STA are assumed to be 50% of duty cycle at primary pins. The duty cycle for clock waveforms can be specified in the SDC. The same duty cycle is assumed at every clock pin associated with that waveform. You can override the duty cycle defaults for primary ports, as well as force duty cycles on specific nets in the design.

Starting from the initial default values (and user-overrides), Tempus propagates the duty cycle across the logic.

Parameters

`-help`	Prints out the command usage.
`-activity_propagation_mode {probabilistic | worst_case}`	Specifies the activity (duty cycle and toggle rate) propagation mode - `probabilistic` or `worst_case`. *Default*: `probabilistic`

`-constraints {true | false}`	Enables the aging constraints. *Default*: `false`
`-disable_aging`	Disables aging analysis.
`-max_duty_cycle <value>`	Specifies the default maximum duty cycle. *Default*: 1 *Min*: 0, *Max*: 1
`-max_toggle_rate <value>`	Specifies the default maximum toggle rate (number of transitions per second). *Default*: 100000000 *Min*=0, *Max*=3.40282346638529e+38
`-max_toggle_rate_limit <value>`	Specifies the maximum toggle rate limit. *Default*:=3.40282346638529e+38 *Min*=0, *Max*=3.402
`-min_duty_cycle <value>`	Specifies the default minimum duty cycle. *Default*: 0 *Min*: 0, *Max*: 1)
`-min_toggle_rate <value>`	Specifies the default minimum toggle rate (number of transitions per second). *Default:* 0 *Min:* 0, *Max:* 3.40282346638529e+38)
`-param_priority_sets <string>`	Specifies an ordered list of lists of additional parameters to be used for aging for all the views, e.g., { {p1 p2 p3} {q1 q2} }; parameters from the aging library will always get the highest priority.
`-recovery {true | false}`	Enables the aging recovery analysis. *Default*: `true`

`-sequential_activity_propagation {true | false}`	Enables the activity propagation through sequentials. *Default*: `false`
`-sequential_max_duty_cycle <value>`	Specifies the default sequential maximum duty cycle. *Default*: 1 *Min*: 0, *Max*: 1
`-sequential_max_toggle_rate <value>`	Specifies the default sequential maximum toggle rate (number of transitions per second). *Default*=100000000 *Min:* 0, *Max:* 3.40282346638529e+38
`-sequential_min_duty_cycle <value>`	Specifies the default sequential minimum duty cycle. *Default*: 0 *Min*: 0, *Max*: 1
`-sequential_min_toggle_rate <value>`	Specifies the default sequential minimum toggle rate (number of transitions per second). *Default*: 0 *Min*: 0, *Max*=3.40282346638529e+38)
`-type {bti | hci | all}`	Specifies the type of aging analysis - bti, hci, or all. *Default*: `all`
`-use_fixed_duty_cycle {true | false}`	Enables the fixed duty cycle instead of propagated duty cycle for aging *Default*: `false`
`-views <string>`	Specifies a list of view names for the defined settings.

Examples

- The following command sets the minimum and maximum toggle rate on ports and special nets. This should be specified when HCI analysis is enabled in the flow:

```
tempus > set_aging_analysis_mode -min_toggle_rate 1e+09 -max_toggle_rate 1e+09
```

- The following command sets toggle rate on sequential cells output pins. This should be specified when HCI analysis is enabled in the flow:

```
tempus > set_aging_analysis_mode -sequential_min_toggle_rate 1e+09 -
sequential_max_toggle_rate 1e+09
```

- The following command models recovery for OMI and URI Spice models that are recovery-aware:

```
tempus > set_aging_analysis_mode -recovery true
```

- The following command specifies the minimum and maximum toggle rate and duty cycle for probabilistic propagation mode:

```
tempus > set_aging_analysis_mode -activity_propagation_mode probabilistic \
-min_toggle_rate 1e9 -max_toggle_rate 1e9 -min_duty_cycle 0 -max_duty_cycle 0.2
```

Related Commands

- `set_db timing_analysis_aging`
- `set_aging_activity`
- `create_aging_stress_profile`
- `get_aging_analysis_mode`
- `write_spice_deck`

Related Information

- "Aging-Aware STA Analysis" chapter in the *Tempus Stylus User Guide*.

write_exclude_nets

```
write_exclude_nets
[-help]
out_file
```

Writes out the excluded nets to an output file. Nets that exceed the default value of 1,000 pins are excluded from delay calculation, and default delay values are used. .

You can use `delaycal_excluded_net_file` attribute to use the file containing nets to be excluded during delay calculation. The list of excluded nets can be redirected to an output file.

Parameters

`-help`	Prints out the command usage
`out_file`	Specifies the name of the excluded net file.

write_set_load

```
write_set_load
[-help]
out_file
[-direct]
[-exclude_zero_cap]
[-include_pin_cap]
[-view string]
[-wire_load]
```

Writes the specified capacitance on the specified ports or nets of the top cell to an output file.

Parameters

`-help`	Prints out the command usage.
out_file	Specifies the output file name.
`-direct`	Refers to the ports directly.
`-exclude_zero_cap`	Excludes the net that has zero net capacitance.
`-include_pin_cap`	Includes the pin capacitance.
`-view` *string*	Specifies the view name for the output file.
`-wire_load`	Marks the `set_load -wire_load` option for a port. This interprets the specified capacitance as the external wire capacitance of the port.

7

Distributed MMMC Commands

- distribute_command
- distribute_design_views
- distribute_read_db
- distribute_views
- get_distributed_variables
- reset_servers
- set_distributed_mmmc_mode
- set_distributed_variables

distribute_command

```
distribute_command
[-help]
[ list_of_commands]
[-views list_of_views]
```

Distributes interactive analysis commands. When you use this command, multi-mode multi-corner (MMMC) analysis is not run again and the [*list_of_commands*] is run on existing active views.

You can use `distribute_command` when either of the following commands is run:

- `distribute_read_design -restore_design`
- `distribute_views -views {viewnames}`

Parameters

`-help`	Prints out the command usage.

list_of_commands	Specifies commands to be distributed.
	The commands that are required to be run only at the Tempus client should be enclosed in `{}`. You can specify multiple commands in a single `distribute_command` statement.
	`tempus > distribute_command {`
	`report_timing -through [get_nets net1]`
	`set endPin [get_pins u1/a] ; report_timing -to $endPin`
	`}`
	The commands that are required to take variable value from the master Tempus session should be enclosed in `" "`.
	For example, the following commands create two variables `testVar1` and `testVar2` on Tempus client with variable values being the same as on master Tempus session.
	`set testVar1 val1 ; set testVar2 val2`
	`distribute_command "foreach var {testVar1 testVar2} {set $var $[subst $var]} "`

`-views` *list_of_views*	Specifies the views to be distributed.

Related Commands

- distribute_read_design
- distribute_views
- set_multi_cpu_usage

distribute_design_views

```
distribute_design_views
[-help]
[-client_log_file prefix]
[-create_boundary_model]
[-create_boundary_model_args <string>]
[-load_boundary_model <string>]
[-load_boundary_model_args <string>]
[-write_db {list_of_views | all}]
[-views list_of_views | -read_dir path]
[-views list_of_views | -read_db {views | all}]
[-views list_of_views | -read_cellview {lib | cell}]
[-design_script inputTclCmdFile | -read_dir path]
[-design_script inputTclCmdFile | -read_db {views | all}]
[-design_script inputTclCmdFile | -read_cellview {lib | cell}]
[-read_db {views | all} -read_dir path]
[-views list_of_views [-analysis_script list_of_commands]]
[-analysis_script list_of_commands [-views list_of_views]]
[-views list_of_views [-set_analysis_view_log  | -combine_num_views integer]]
[-out_dir outputDir [-design_script inputTclCmdFile| -read_dir path]]
```

Allows the Tempus software to process a view as soon as a client is available.

When this command is used, the software does not have to wait till time out or for all the clients to be available for processing a view. This reduces the turnaround time in situations where machines are not easily available.

Parameters

`-help`	Prints out the command usage.
`-analysis_script list_of_commands`	Specifies the file that contains commands for reading in the necessary parasitic files and the reporting commands.
`-client_log_file prefix`	Specifies the client log filename prefix with the directory.
`-combine_num_views integer`	Specifies the number of views to combine in a job.
`-create_boundary_model`	Allows the software to generate boundary models.

`-create_boundary_model_args <string>`	Specifies the arguments provided to the `create_boundary_model` command during boundary model generation.
`-design_script inputTclCmdFile`	Specifies the name of the design and MMMC configuration loading script. The `-design_script` is mutually exclusive with restore_* options.
`-load_boundary_model`	Specifies the directory from where boundary model will be loaded.
`-load_boundary_model_args <string>`	Specifies the arguments provided to the `read_boundary_model` command. For example: `distribute_design_views -load_boundary_model {-skip_version_check}` When specified, the software skips version checks while loading the boundary model in DMMMC flow.
`-out_dir outputDir`	Specifies the directory where the run is to be executed.
`-read_cellview {lib | cell}`	Specifies the name of OA library or cell to be restored.
`-read_db {views | all}`	Specifies a list of views.
`-read_dir path`	Specifies the directory path where the design is to be restored.
`-write_db {list_of_views | all}`	Saves the design session information for the specified (or all) the analysis views.
`-set_analysis_view_log`	Redirects the `set_analysis_view` command run log to `set_analysis_view_<viewname>.log` file.

`-views {list_of_views}`	Specifies the views to distribute for both setup and hold analysis • The `-views` parameter is mutually exclusive with `restore_*` options. • The `-views` parameter is mandatory when `-set_analysis_view_log` and `-combine_num_views` parameters are specified.

Related Commands

- `set_multi_cpu_usage`
- `set_distributed_hosts`
- `set_analysis_view`

distribute_read_db

```
distribute_read_db
[-help]
[-client_log_file string]
-out_dir string
[-read_cellview {directory cell_name}]
{-design_script string | [-read_db string [-read_dir string]]}
```

Opens up *N* number of Tempus clients (as set with set_multi_cpu_usage) for a distributed processing session, and loads the design script on Tempus clients.

The distribute_read_db command is used for distributed multi-mode multi-corner (DMMMC) timing analysis only.

Parameters

-help	Prints out the command usage.
-client_log_file *logFilePathName*	Redirects client logs to a new location. By default, Tempus client logs are created in the directory specified with the -out_dir option.
-design_script *string*	Specifies the design loading script. The script contains the common information that needs to be shared by all Tempus client runs. The script must include the design loading commands, as well as all MMMC configuration commands. The script must not include parasitic file loading commands (read_spef), analysis view setting commands (set_analysis_view) or the view specific settings like constraints through set_interactive_constraint_modes. Timing derates settings (set_timing_derate) are corner specific and can be specified in this script.
-out_dir *string*	Specifies the directory where the run is to be executed. All data from the distributed analysis runs will be available in this directory. The software generates view-specific reports in directories with the corresponding view name within this directory.
-read_cellview {directory cell_name}	Specifies the "directory" as a directory name having OpenAccess library name/OA cell and "cell_name" refers to the OA cell name.

`-read_db` *string*	Specifies the design views (previously saved using the `distribute_views -write_design` command) to be restored for both setup and hold analysis. The number of sessions specified for restore can not be more than the machines specified with `-remoteHost` setting.
`-read_dir` *string*	Specifies the directory location which has the saved design views (that were previously saved using the `distribute_views -write_design` command) to be restored.

Examples

Here are some examples.

- The following example command script lists the commands used to perform distributed multi-mode multi-corner timing analysis:

```
tempus > set_multi_cpu_usage -remoteHost 4
tempus > distribute_read_db \
-design_script /stm1/timing/demo_dist_proc/preamble.tcl \
-out_dir $output_directory
#preamble.tcl script includes design loading commands and MMMC configuration
commands

set Views [list lmin-rmax lmax-rmin lmax-rmax]
tempus > distribute_views \
-views $Views \
-script /stm1/timing/demo_dist_proc/commands.tcl
#commands.tcl script contains read_spef and report commands

tempus > reset_servers
```

- The following example command script lists the commands used to save a design in distributed multi-mode multi-corner mode:

```
tempus > set_distributed_hosts -local | -rsh ...
tempus > set_multicpu_usage -remoteHosts N
tempus > distribute_read_db -design_script dmmmc.tcl -out_dir output
```

```
set Views [list lmin-rmax lmax-rmin lmax-rmax]
tempus > distribute_views -views $Views -script dmmmc_report.tcl -write_design all
```

- The following example command script lists the commands used to restore a saved session (with N views) in distributed multi-mode multi-corner mode:

```
tempus > set_distributed_hosts -local
tempus > set_multicpu_usage -remoteHosts N
set Views [list lmin-rmax lmax-rmin lmax-rmax]

tempus > distribute_read_db -read_db $Views -read_dir output -out_dir new_output
```

  # output is the same directory used in above example where views were saved.

```
tempus > distribute_command {source dmmmc_reports2.tcl}
```

  # dmmmc_reports2.tcl can have analysis commands but do not need read_spef etc.

- The following example restores the `dtmf_chip` OpenAccess design, which is saved at the path, `/home/cdns/oa_dir`:

```
tempus > set outDir "/home/cdns/" distribute_read_db -read_db {view1 view2} -
read_dir $outDir -read_cellview {oa_dir dtmf_chip} -out_dir output
```

Related Commands

- set_multi_cpu_usage
- set_analysis_view
- set_timing_derate
- distribute_views
- set_interactive_constraint_modes

Related Topics

- Distributed Multi-Mode Multi-Corner Timing Analysis chapter in the *Tempus User Guide*

distribute_views

```
distribute_views
[help]
[-combine_num_views]
[-force_distribute]
-views list_of_views
-script filename
[-write_db {list_of_views | all}]
[-set_analysis_view_log]
```

Distributes the specified analysis views to the Tempus client(s), then sources the specified command script to perform timing and signal integrity analysis.

The `distribute_views` command is used for distributed multi-mode multi-corner (DMMMC) timing analysis only. Distributed multi-mode multi-corner timing analysis is executed in batch mode. You can use the `distribute_command` for running a single command interactively.

By default, this command distributes a single view per Tempus client. The command can also perform concurrent multi-mode multi-corner timing analysis (CMMMC), within the DMMMC framework. You can combine the analysis views for CMMMC analysis on Tempus client(s). This can be done in the following two ways:

1. Delay corner based auto grouping :
   You can specify a simple list of view names in the `-view` option and use the `-combine_num_views` option to specify the maximun number of views to be combined per Tempus client. For example

   ```
   tempus > distribute_views -combine_num_views 4 -views {setup1 setup2 setup3 setup4 setup5 setup6 setup7 setup8} -script ...
   ```

   Assuming setup1 to setup4 have delay corner dc1 and setup5 to setup8 have delay corner dc2 then above example will group setup1 to setup4 to Tempus client1 and setup5 to setup8 will be grouped to Tempus client2

2. User controlled view grouping:
   You can specify a list of list format for combining multiple concurrent views on Tempus client(s). For example,

   ```
   distribute_views -views { {setup1 setup2 setup3 setup4 } { setup5 setup6 setup7 setup8 } } -script ...
   ```

The above command will group setup1 to setup4 views to Tempus client1 will group setup5 to setup8 views to Tempus client2 irrespective of the delay corner. To get optimum runtime while using view grouping, it is recommended to use delay corner based auto grouping as mentioned in example 1 above.

In this release, Tempus supports one step method for generating merged MMMC view report from DMMMC master shell. After running the `distribute_views` command, you can issue a single `report_timing/report_constraint` command, which generates reports collectively from all active distributed views (that is, views available for interactive DMMMC analysis). The merged output report is arranged as per slack/criticality order across active views.
All the options of report_constraint command, except `-connection_class`, and all the options of `report_timing` command are supported in this mode.

Parameters

`-help`	Prints out the command usage.
`-combine_num_views`	Combines upto N views (having same delay corner) to run at the same time on the same Tempus client.
`-force_distribute`	Forces the full timing update for given set of views (specified in subsequent distribute_views command) even if those views are already active on Tempus client(s). By default The existing active view(s) will not be re-analyzed and the remaining views from the distribution list will be dispatched for full timing update.

`-write_db {`*`list_of_views`*` | all}`	Saves the design session information for the specified analysis views.
	
	You can use the `-write_db all` option to save all the distributed multi-mode multi-corner (DMMMC) sessions with views that are specified using the `-views` parameter.
	
	The design information is saved in the following location:
	
	`${outdir}/<viewname>/<topcell.enc.dat>`
	
	where `${outdir}` is the location specified using the `distribute_read_design -outdir` command.
	
	Note: When you specify this parameter, any previously saved sessions in `outdir` will be overwritten.
	
	When the `-write_db` parameter is specified, a distributed OpenAccess (OA) database will be created in the following format during a distributed MMMC run:
	
	`LibName = $outDir`
	`CellName = same as OA design Cell Name`
	`Views = List of MMMC Views`

`-script` *filename*	Specifies the file that contains list of commands for reading in the necessary parasitic files and the reporting commands required for timing and signal integrity analysis.
	
	Commands for reading parasitic information must be specified in the script before the reporting commands. You must specify parasitic files specific for all RC corners. For example:
	
	`read_spef -rc_corner corner1 rc1.spef`
	
	`read_spef -rc_corner corner2 rc2.spef`
	
	`read_spef -rc_corner corner3 rc3.spef`
	
	`read_spef -rc_corner corner4 rc4.spef`
	
	If the file includes report commands that redirect their output to a specified file the software generates the file in the active view directory, in the output directory specified with the `distribute_read_design` command.
	
	(for example: for below command specified in the script
	
	report_timing >rt.rpt
	
	software will generate rt.rpt file in ${outdir}/<viewname> directory for each active view where ${outdir} is the location specified using the distribute_read_design -outdir command.
	
	For report commands with no redirection specified, the software generates the report in the corresponding log file.

`-set_analysis_view_log`	Redirects `set_analysis_view` run log to set_analysis_view_<viewname>.log file.
`-views` *list_of_views*	Specifies views to distribute for analysis. **Note**: The `-views` parameter is mandatory when the `-set_analysis_view_log` and `-combine_num_views` parameters are specified.

Examples

Here are some examples.

- The following example command script `run_dmmmc.tcl` lists the commands used to perform

distributed multi-mode multi-corner timing analysis. The script performs distributed analysis for four views on four Tempus clients and then saves the design:

```
tempus > set_multi_cpu_usage -remote_host 4
tempus > distribute_read_db \
-design_script /stm1/timing/demo_dist_proc/preamble.tcl \
-out_dir $output_directory

set views [list lmin-rmax lmin-rmin lmax-rmin lmax-rmax]
tempus > distribute_views \
 -views $views \
  -script /stm1/timing/demo_dist_proc/commands.tcl \
   -write_db {all}

tempus > reset_servers
```

- The following command script `commands.tcl` generates `setup.rpt` and `hold.rpt` reports for all the four views:

```
tempus > read_spef -rc_corner rmin /stm1/timing/demo_dist_proc/min.spef.gz
tempus > read_spef -rc_corner rmax /stm1/timing/demo_dist_procmax.spef.gz
tempus > report_timing -late > setup.rpt
tempus > report_timing -early > hold.rpt
```

- The following command script `report_common.tcl` generates `setup.rpt` and `hold.rpt` reports for two views each:

```
set mySetupViews "lmax-rmin lmax-rmax"
set myHoldViews "lmin-rmax lmin-rmin"

###### REPORTS FOR SETUP VIEWS ####
[get_db [get_db analysis_views -if {.is_setup||.is_hold} ] .name -u]
set reportSetup 0
foreach vName $currentView { if { [lsearch $mySetupViews $vName ] != -1 } { set
reportSetup 1 ; break; } }
if { $reportSetup } {
## ALL Setup View reporting Commands
report_timing -late > setup.rpt
}
```

```
###### REPORTS FOR HOLD VIEWS ####
set currentView [analysis_views] ;
set reportHold 0
foreach vName $currentView { if { [lsearch $myHoldViews $vName ] != -1 } { set
reportHold 1 ; break; } }
if { $reportHold } {

## ALL Hold View reporting Commands
report_timing -early > hold.rpt
}
```

Related Topics

- Distributed Multi-Mode Multi-Corner Timing Analysis chapter in the *Tempus User Guide*

Related Commands

- set_multi_cpu_usage
- read_spef
- report_timing

get_distributed_variables

```
get_distributed_variables
[-help]
```

Retrieves variables from Tempus clients. The results of each variable are stored in the master session as a TCL array with the view name as the key or search index.

When the result is created in the form of TCL variables at the Tempus client, the resulting variables can be retrieved back to the master session anytime after one of the following commands is run:

- `distribute_views -views`
- `distribute_read_design -restore_design`

Parameters

`-help`	Prints out the command usage.
`list_of_slave_variables`	List of slave variables to be returned to master.
`-result_array_2d`	Generates tcl array at master for each child array variable with viewname as additional index. *Default*: Array variables retrieved as tcl list indexed only by view name

Examples

Here are some examples.

- Assuming setupView and holdView views are active on two Tempus clients. The following example command script retrieves the value of variables slackTest and arrivalTest from each Tempus client to master session:

```
# define/set some value for variables at the Tempus client:

1) tempus > distribute_command {
 array set arrivalTest [list a 1 b 2 c 5 d 7] ;
}

get_distributed_variables arrivalTest
```

```
Puts "ARRIVAL = [array get arrivalTest]"

Output of this example will be:
    ARRIVAL = holdView {d 7 a 1 b 2 c 5} setupView {d 7 a 1 b 2 c 5}
```

2)
```
tempus > distribute_command {
 array set slackTest [list a -1 b -2 c -5 d -7]
}
get_distributed_variables -array_as_multi_dim_array slackTest

Puts "SLACK = [array get slackTest]"
```

Output of this example will be:

```
SLACK = setupView,a -1 holdView,d -7 setupView,b -2 holdView,a -1 setupView,c -5
holdView,b -2 setupView,d -7 holdView,c -5
```

Related Commands

- distribute_read_design
- distribute_views

reset_servers

```
reset_servers
-help
[-view viewName]
```

Releases the Tempus client(s) and ends the distributed processing session. Specify the `reset_servers` command after analysis is finished.

The `reset_servers` command is used for distributed multi-mode multi-corner timing analysis only.

Parameters

`-help`	Prints out the command usage.
`-view` *viewName*	Removes user-specified view(s) and releases the corresponding Tempus client. This parameter can be used after the `distribute_views` command is issued. After removing some view(s), you can resume performing interactive analysis for remaining views, however, you cannot run the `distribute_views` command again in the same session.

Examples

Here are some examples.

- The following example command script will release all Tempus clients after performing distributed multi-mode multi-corner timing analysis for all views:

```
tempus > set_multi_cpu_usage -remoteHost 4

tempus > distribute_read_design \
-design_script /stm1/timing/demo_dist_proc/preamble.tcl \
-outidr $output_directory
set views [list lmin-rmax lmax-rmin lmax-rmax lmin-rmin]
#preamble.tcl script includes design loading commands and MMMC configuration
commands

tempus > distribute_views \
```

```
-views $views \
-script /stm1/timing/demo_dist_proc/commands.tcl
#commands.tcl script contains read_spef and report commands

reset_servers
```

Related Topics

- Distributed Multi-Mode Multi-Corner Timing Analysis chapter in the *Tempus User Guide*

set_distributed_mmmc_mode

```
set_distributed_mmmc_mode
[-help]
[-client_init_execute_script filename]
[-eco {true | false}]
[-enable_high_capacity_eco]
[-eco_merge_mmmc]
[-enable_smart_mmmc]
[-lic_startup_extra {list_of_options}]
[-load_def_only]
[-min_hosts_required integer]
[-set_filler_mode {list_of_options}]
[-interactive {true | false} [-views {list_of_views | all}]]
[-use_one_host_per_batch_job {true | false}]
```

Controls distributed MMMC analysis settings.

Parameters

`-help`	Prints out the command usage.
`-client_init_execute_script` *filename*	Specifies a file containing the TCL code that will be executed on a Tempus client(s) immediately after it is invoked. This allows you to load settings on a client prior to loading a database or design initialization.
`-eco {true | false}`	Enables/disables the timing signoff flow integrated with distributed MMMC.
`-enable_high_capacity_eco`	Enables database size reduction in the Tempus Paradime flow or Certus for high capacity ECO timing/power closure based on timing violations, list of partition names, or user-defined path group. **Note**: It is mandatory to apply `set_distributed_mmmc_mode -eco true` command before `set_distributed_mmmc_mode -enable_high_capacity_eco` in Tempus Paradime flow. In case of Certus, it should be set before `distribute_read_db`.

`-eco_merge_mmmc`	Creates ECO analysis view(s) from view definition files in restore sessions. The `-eco_merge_mmmc` parameter must be used only after `set_distributed_mmmc_mode -eco true` command is set.
`-enable_smart_mmmc`	Enables reduced memory or runtime solution when many timing views are enabled in Tempus Paradime flow or Certus. **Note**: It is mandatory to apply `set_distributed_mmmc_mode -eco true` command before `set_distributed_mmmc_mode -enable_smart_mmmc` in Tempus Paradime flow. In case of Certus, it should be set before `distribute_read_db`.
`-lic_startup_extra {`*`list_of_options`*`}`	Specifies a list of licenses to be checked out by Tempus clients. When specified, the software shows the extra Innovus startup licenses that are checked out by Tempus clients.
`-load_def_only`	Specifies if a client database being restored in a flow contains LEF information and needs to load DEF (specified using the `eco_def_file_list` variable), to make it a fully physical flow.
`-interactive {true | false}`	Enables or disables the interactive mode.
`-min_hosts_required` *`integer`*	Specifies minimum number of machines required to start a session in case all the machines cannot be acquired after time out.
`-set_filler_mode {list_of_options}`	Specifies a list of `set_db` command options. This parameter allows to pass a list of options that the `set_db` command applies during physical data loading in Tempus Paradime.
`-use_one_host_per_batch_job {true | false}`	Enables/disables one batch mode job per host. A new client is started for each view or each combined view job. This is used when running flows, such as manual ECO flows, where netlist updates during analysis makes scheduling of subsequent views difficult.
`-views {`*`list_of_views`* `| all}`	Specifies a list of views for interactive mode. The `all` option selects all the view.

Examples

- The following commands enable the distributed MMMC analysis in interactive mode for views view1 and view2:

```
tempus > set viewnames {view1 view2}
tempus > set_distributed_mmmc_mode -views $viewnames -interactive true
```

- The following command disables the interactive mode once all the reporting/ECO on interactive views is done:

```
tempus > set_distributed_mmmc_mode -interactive false
```

- The following command lists all the filler cell names to be applied on `set_filler_mode` command:

```
set_distribute_mmmc_mode -set_filler_mode {-core "FILLX1 FILLX2 FILLX4"}
```

Related Commands

- `set_multi_cpu_usage`
- `distribute_read_db`
- `distribute_views`
- `set_db`

set_distributed_variables

```
set_distributed_variables
[-help]
list_of_variables
[value]
```

Allows you to set variables on the Tempus master and clients.

The variables can be set after issuing the `distribute_read_db` command on the master.

Parameters

`-help`	Prints out the command usage.
`list_of_variables`	Specifies a list of variables to be set on master and client(s).
`value`	Specifies the value which should be set on the master and client.

Examples

- The following command sets a variable `var` having value `1` on the master and client(s):

  ```
  tempus > set_distributed_variables var 1
  ```

- The following command sets multiple variables with value `1` on master and client(s):

  ```
  tempus > set_distributed_variables "var1 var2 var3" 1
  ```

- The following command sets multiple variables each with different values on master and client(s):

  ```
  tempus > set var1 1
  1
  tempus > set var2 2
  2
  tempus > set var3 3
  3
  tempus > set_distributed_variables "var1 var2 var3"
  ```

Related Commands

- get_distributed_variables
- distribute_read_db
- distribute_command

8

Flowkit Commands and Attributes

- check_flow
- create_flow
- create_flow_step
- define_feature
- delete_flow_config
- edit_flow
- end_flow
- flow Category Attributes
- flowtool Category Attributes
- get_feature
- get_flow_config
- init_flow
- is_feature
- is_flow
- read_flow
- report_flow
- reset_flow
- run_flow
- schedule_flow
- set_feature
- set_flow_config
- set_flowkit_read_db_args

- set_flowkit_write_db_args
- write_flow
- write_flow_template

check_flow

```
check_flow
[-help]
[-fail string]
[-flow string]
[-from string]
[-proc string]
[-step string]
[-to string]
```

Creates a single flow_step object and fills-in the required, and some optional attributes. Further modifications of the step can be performed using the `set_db` command. If you try to define a step with the same name as an existing step, it results in error. To redefine a flow step, you must use the `set_db` command to modify the attributes, or delete and recreate it.

Note: Unlike Tcl procedures, flow steps do not take arguments and refer to attributes instead. The use of attributes requires that the data on which a step relies, is stored in the database and may be examined later.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `check_flow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man check_flow`.
`-fail` *string*	Reports a problem. this is not used for analysis. It can only be used from Tcl called as part of static checking. Calling from outside static checking results in error.
`-flow` *string*	Runs the specified flow. the `from`/`to` values are based on the flow steps. *Default:* `flow_step_order`
`-from` *string*	Specifies the name of the step to start the flow from. By default, the flow starts from the first step that has not been run, or the first step if all steps have been run previously. **Note:** This must be a step in the flow.
`-proc` *string*	Checks the specified procedure instead of analysing a flow step.
`-step` *string*	Runs just the specified step. This takes priority over `from`/`to` options.
`-to` *string*	Specifies the name of the step to run to (inclusive; in other words, the named step is executed). By default, runs to the end of the last step in the flow. Must be a step in the flow, later than the `-from` step (if any).

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

create_flow

```
create_flow
[-help]
[steps]
-name name
[-owner string]
[-skip_metric]
[-tool tool]
[-tool_options options]
```

Creates a new, named flow object. `run_flow` then allows the execution of a named flow (in addition to its existing functionality).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_flow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_flow`.
`[steps]`	Specifies a list of flow step names in the order they must be executed. **Note:** This is an optional parameter. You can either provide `[steps]` or use the `-tool` parameter.
`-name` *name*	The name of the flow object to create.
`-owner` *string*	Specifies the owner of the flow.
`-skip_metric`	Restricts the tool from capturing the flow's metric snapshot.
`-tool` *tool*	Specifies the name of the tool with which to run this flow. This option allows you to create flows for other tools than the one currently running. If not specified, default is the current tool.
`-tool_options` *options*	Specifies options for use when launching this flow.

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

create_flow_step

```
create_flow_step
[-help]
body
[-categories string]
[-check checkTcl]
[-description string]
[-exclude_time_metric]
-name name
[-owner string]
[-write_db]
[-skip_metric]
```

Creates a single `flow_step` object and fills-in the required, and some optional attributes. Further modifications of the step can be performed using the `set_db` command. If you try to define a step with the same name as an existing step, it results in error. To redefine a flow step, you must use the `set_db` command to modify the attributes, or delete and recreate it.

Note: Unlike Tcl procedures, flow steps do not take arguments and refer to attributes instead. The use of attributes requires that the data on which a step relies, is stored in the database and may be examined later.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_flow_step` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_flow_step`.
`body`	Specifies a block of Tcl that is run as the main action of the step.
`-categories` *`string`*	Defines the metric categories to calculate the flow step.
`-check` *`checkTcl`*	Specifies a block of Tcl to run step-specific static checks.
`-description` *`string`*	Describes the purpose for the current flow step.
`-exclude_time_metric`	Ensures that the CPU and wall time of the corresponding step is not included in any parent steps.
`-name` *`name`*	Specifies the name of the Tcl of the step. This can consist of alphanumeric characters and underscore only, and may not start with a digit. The following names have special meanings and may not be used for steps: • next • end
`-owner` *`string`*	Specifies the owner of the flow.
`-write_db`	Automatically saves a database when the flow step is complete.
`-skip_metric`	Skips generating metrics when the step is complete. By default, every step saves the design and runs metrics when completed.

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

define_feature

```
define_feature
[-help]
feature
[-default string]
[-description string]
[-group string]
[-obj string]
[-possible_values string]
[-required]
[-type {bool string enum}]
```

Defines the feature for a flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `define_feature` parameter. For a detailed description of the command and all of its parameters, use the man command: `man define_feature`
`feature`	Specifies the name of the feature.
`-default` *`string`*	Specifies the default value of the feature.
`-description` *`string`*	Specifies the detailed description of the flow feature. If it matches multiple features, then these features are listed along with a short description for each. If it matches exactly one feature, a full description of this feature is printed. Matching no features results in an error.
`-group` *`string`*	Outputs the exclusive group that contains features.
`-obj` *`string`*	Specifies the flow or flow_step.
`-possible_values` *`string`*	Specifies the list of values that are valid for the features.
`-required`	Specifies that the mentioned feature is mandatory.
`-type {bool string enum}`	Specifies the type of feature value.

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

delete_flow_config

```
delete_flow_config
[-help]
key ...
```

Deletes the configuration value at the specified key, allowing the user to change the type of a `setup.yaml` key by first deleting it.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_flow_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_flow_config`
`key ...`	Specifies the key to delete the configuration value.

See also:

`set_flow_config`

edit_flow

```
edit_flow
[-help]
[-after string]
[-append string]
[-before string]
[-enabled_feature string]
[-owner string]
[-prepend string]
[-reset]
[-unique]
```

Allows you to insert flow steps into an existing flow without redefining the flow itself. This command is very useful for flow customization.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `edit_flow` parameter.
`-after` *string*	Adds the specified `flow` and/or `flow_step` objects after the flow supporting a full canonical path to a flow step object.
`-append` *string*	Appends the specified `flow` and/or `flow_step` objects before or after the flow.
`-before` *string*	Adds the specified `flow` and/or `flow_step` objects before the flow supporting a full canonical path to a flow step object.
`-enabled_feature` *string*	Executes the `edit_flow` command only if the specified user-defined features are enabled.
`-owner` *string*	Specifies the owner of the flow being edited.
`-prepend` *string*	Adds the specified `flow` and/or `flow_step` objects before or after the flow.
`-reset`	Resets the customizations for the specified `flow` and/or `flow_step` objects before or after the flow.
`-unique`	Prevents the addition of the flow and/or `flow_step` objects, if those are already provided. This avoids duplication of the flow defined in the YAML file and the `edit_flow` statements scripted in the `tech_flow_steps` file. **Note:** This parameter prevents the specified `flow_step` from getting added to the same flow twice, irrespective of the values provided to the `-after` or `-before` parameters. For example, if `edit_flow -append bob -unique` is called twice to insert different `flow_steps` in the floorplan flow, it will add `bob` only once.

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

Examples

- Following is an example of customizing the reference flow process using the `flow_config.template` file:

```
create_flow_step -name write_sdf {
write_sdf [get_db flow_report_directory]/[get_db local_flow].sdf
}
edit_flow -after flow_step:report_late_timing -append flow_step:write_sdf
```

- Following is an example of inserting an additional `step_2` step after the `step_1` step:

```
set file_dir [file dirname [ dict get [ info frame [ info frame ] ] file ] ]
source ${file_dir}/scripts/run_flow.tcl
create_flow_step -name step_2 {
}
edit_flow -append step_2 -after step_1
```

- Following is an example of adding flow customization to the `flow_config.tcl` file:

```
create_flow_step -name output_files {
set design_name [get_db current_design .name]
write_netlist [get_db flow_report_directory]/${design_name}.v
write_def [get_db flow_report_directory]/${design_name}.def
}
edit_flow -append flow_step:output_files -after flow_step:block_finish
```

end_flow

```
end_flow
[-help]
flow_or_step
```

Ends the specified flow or step. This command accepts a single string argument, which is a flow or step name. The specified string must match a flow or step in the path for the current step. Else, a Tcl error occurs. Flowkit/flowtool then skips all the contiguous steps to match the given flow or step name in the hierarchy.

Note: Calling `end_flow` outside of a `flow_step` inside `run_flow` (for example, `init_flow plugin`, `flow_header_tcl` or as a plain command) will produce an error.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `end_flow` parameter. For a detailed description of the command and all its parameters, use the man command: `man end_flow`
`flow_or_step`	Specifies the flow or step to be ended and skipped.

See also:

Flow chapter of the *Innovus Stylus Common UI Migration Guide* document.

flow Category Attributes

The root attributes in the `flow` category are:

- flow_branch
- flow_caller_data
- flow_db_directory
- flow_error_errorinfo

- flow_error_message
- flow_error_write_db
- flow_exclude_time_for_init_flow
- flow_exit_when_done
- flow_features
- flow_feature_values
- flow_footer_tcl
- flow_header_tcl
- flow_hier_path
- flow_history
- flow_log_directory
- flow_log_prefix_generator
- flow_mail_on_error
- flow_mail_to
- flow_metrics_file
- flow_metrics_snapshot_parent_uuid
- flow_metrics_snapshot_uuid
- flow_overwrite_db
- flow_plugin_canonical_current
- flow_plugin_names
- flow_plugin_steps
- flow_post_db_overwrite
- flow_remark
- flow_report_directory
- flow_run_tag
- flow_schedule
- flow_starting_db

- flow_startup_directory
- flow_status_file
- flow_step_canonical_current
- flow_step_check_tcl
- flow_step_current
- flow_step_last
- flow_step_last_msg
- flow_step_last_status
- flow_step_next
- flow_summary_tcl
- flow_template_feature_definition
- flow_template_tools
- flow_template_type
- flow_template_version
- flow_user_templates
- flow_top
- flow_verbose
- flow_working_directory
- flow_write_db_snapshot
- flow_write_db_snapshot_exclude_time
- flow_yamllint_exec

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `flow` category, use the following command:

```
get_db -category flow
```

flow_branch

`flow_branch` *`string`*

Default: ""

Specifies the branch being run for a hierarchical flow.

flow_caller_data

`flow_caller_data` *`string`*

Default: *""*

Specifies the data used by the caller of the tool for identifying the flow.

flow_db_directory

`flow_db_directory` *`string`*

Default: `dbs`

Specifies the flow directory where results databases are stored.

flow_error_errorinfo

`flow_error_errorinfo` *`string`*

Default: ""

Specifies the TCL error stack in the error database.

flow_error_message

`flow_error_message` *`string`*

Default: ""

Specifies the TCL error message in the error database.

flow_error_write_db

`flow_error_write_db {true | false}`

Default: `true`

Runs the database when an error occurs in a flow step.

flow_exclude_time_for_init_flow

```
flow_exclude_time_for_init_flow {true | false}
```

Default: `false`

Marks all those snapshots as `exclude_time_metric`, which were created after running the `init_flow` command. This marking adds user steps to the `init_flow` internal plugin locations.

flow_exit_when_done

```
flow_exit_when_done {true | false}
```

Default: `false`

Exits the flow after running the final step.

flow_features

```
flow_features string
```

Default: ""

Specifies the global feature definitions.

flow_feature_values

```
flow_feature_values string
```

Default: ""

Specifies the global feature settings.

flow_footer_tcl

```
flow_footer_tcl string
```

Default: ""

Specifies the TCL script to run when a flow ends.

flow_header_tcl

`flow_header_tcl` *`string`*

Default: ""

Specifies the TCL script to run when a flow starts.

flow_hier_path

`flow_hier_path` *`string`*

Default: ""

Displays the path of the current flow within the flow hierarchy.

flow_history

`flow_history` *`string`*

Default: ""

Displays the complete flow run history.

flow_log_directory

`flow_log_directory` *`string`*

Default: `logs`

Specifies the flow directory where log files are stored.

flow_log_prefix_generator

`flow_log_prefix_generator` *`string`*

Default: ""

Specifies the TCL script to create log filenames in flowtool.

flow_mail_on_error

```
flow_mail_on_error {true | false}
```

Default: `false`

Sends an email to 'flow_mail_to', if an error is detected.

flow_mail_to

```
flow_mail_to string
```

Default: ""

Specifies the email address where results are sent.

flow_metrics_file

```
flow_metrics_file string
```

Default: ""

Specifies the flow metrics file for reporting results.

flow_metrics_snapshot_parent_uuid

```
flow_metrics_snapshot_parent_uuid string
```

Default: ""

Specifies the snapshot uuid to which the results from this flow will be appended.

flow_metrics_snapshot_uuid

```
flow_metrics_snapshot_uuid string
```

Default: ""

Specifies the snapshot uuid of the most recent flow step executed.

flow_overwrite_db

```
flow_overwrite_db {true | false}
```

Default: `false`

Enables overwriting databases while saving.

flow_plugin_canonical_current

`flow_plugin_canonical_current` *`string`*

Default: ""

Identifies the canonical plugin step currently being run, enabling accurate detection of plugin within the Flowtool-based flows. When a plugin is not run, this attribute returns an empty value. Following is an example value of this attribute:

```
tool flowtool flow Cadence.plugin.flowkit.read_db.pre canonical_path {.end_steps
flow_step:test} step flow_step:test features {} str test
```

flow_plugin_names

`flow_plugin_names` *`string`*

Default: *`""`*

Displays the names of all the internal plugin points.

flow_plugin_steps

`flow_plugin_steps` *`string`*

Default: *`""`*

Displays the list of plugin steps that have been added through `edit_flow`.

flow_post_db_overwrite

`flow_post_db_overwrite` *`string`*

Default: *`""`*

When this attribute is set within `skip_db flow_step`, you can identify the name and type of database from the current to future flows as `flow_starting_db`.

flow_remark

`flow_remark` *`string`*

Default: *`""`*

Displays the remarks from the last loaded `flow.yaml` file.

flow_report_directory

`flow_report_directory` *`string`*

Default: `reports`

Specifies the flow directory where reports are written.

flow_run_tag

`flow_run_tag` *`string`*

Default: ""

Specifies the tags for the particular flow run.

flow_schedule

`flow_schedule` *`string`*

Default: ""

Specifies the set of flows to execute after the current flow completes.

flow_starting_db

`flow_starting_db` *`string`*

Default: ""

Specifies the starting database for the flow run.

flow_startup_directory

`flow_startup_directory` *`string`*

Default: `/home/user_id`

Specifies the directory from where the software was run.

flow_status_file

`flow_status_file` *string*

Default: ""

Specifies the file for flow status.

flow_step_canonical_current

`flow_step_canonical_current` *string*

Default: ""

Specifies the currently running canonical flow step.

flow_step_check_tcl

`flow_step_check_tcl` *string*

Default: ""

Specifies the procedure to check the steps.

flow_step_current

`flow_step_current` *string*

Default: ""

Specifies the currently running flow step.

flow_step_last

`flow_step_last` *string*

Default: ""

Specifies the last flow step to be completed.

flow_step_last_msg

`flow_step_last_msg` *`string`*

Default: ""

Specifies the message provided for the last step that was run.

flow_step_last_status

`flow_step_last_status` *`string`*

Default: `not_run`

Specifies the status for the last flow step run.

flow_step_next

`flow_step_next` *`string`*

Default: ""

Specifies the next step to run in the flow.

flow_summary_tcl

`flow_summary_tcl` *`string`*

Default: ""

Specifies the TCL script to run during run_flow summary.

flow_template_feature_definition

`flow_template_feature_definition` *`string`*

Default: ""

Features list and status for the current template.

flow_template_tools

`flow_template_tools` *`string`*

Default: ""

Specifies the list of tools included in the flow templates being run.

flow_template_type

`flow_template_type` *`string`*

Default: ""

Specifies the type of template being run.

flow_template_version

`flow_template_version` *`string`*

Default: ""

Specifies the version of the template being run.

flow_user_templates

`flow_user_templates` *`string`*

Default: ""

Specifies the flow user template definitions.

flow_top

`flow_top` *`string`*

Default: ""

Specifies the flow to be executed by default.

flow_verbose

`flow_verbose {true | false}`

Default: `true`

Enables printing the run information in the log file.

flow_working_directory

`flow_working_directory` *`string`*

Default: "."

Specifies the flow directory where the flow is being run.

flow_write_db_snapshot

`flow_write_db_snapshot {true | false}`

Default: `true`

Data_type: bool, read/write

Creates a snapshot for `write_db` on flow steps, which saves a database.

flow_write_db_snapshot_exclude_time

`flow_write_db_snapshot_exclude_time {true | false}`

Default: `true`

Data_type: bool, read/write

Marks all snapshots created for the `write_db` command on the flow steps, which save a database as `exclude_time_metric`.

flow_yamllint_exec

`flow_yamllint_exec` *`string`*

Default: `yamllint`

Specifies where `yamllint` can be found to check the `yaml` syntax.

flowtool Category Attributes

The root attributes in the `flowtool` category are:

- flowtool_exit_timeout
- flowtool_extra_arguments

- flowtool_metrics_qor_html
- flowtool_metrics_qor_excel
- flowtool_metrics_qor_text
- flowtool_metrics_qor_vivid
- flowtool_predict_full_names
- flowtool_schedule_flow_immediate
- flowtool_summary_tcl

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `flowtool` category, use the following command:

```
get_db -category flowtool
```

flowtool_exit_timeout

`flowtool_exit_timeout` *int*

Default: `30`

Data_type: Integer

Specifies the maximum time (in seconds) for which flowtool will wait to complete post exiting from the tool.

flowtool_extra_arguments

`flowtool_extra_arguments` *string*

Default: `""`

Data_type: String

Specifies the extra arguments passed to flowtool when running in the tool mode..

flowtool_metrics_qor_html

`flowtool_metrics_qor_html` *`string`*

Default: ""

Data_type: String

Specifies the metrics file in the HTML format to write after tool's exit.

flowtool_metrics_qor_excel

`flowtool_metrics_qor_excel` *`string`*

Default: ""

Data_type: String

Specifies the metrics file in the Excel format to write after tool's exit.

flowtool_metrics_qor_text

`flowtool_metrics_qor_text` *`string`*

Default: ""

Data_type: String

Specifies the metrics file in the txt format to write after tool's exit.

flowtool_metrics_qor_vivid

`flowtool_metrics_qor_vivid` *`string`*

Default: ""

Data_type: String

Specifies the metrics file to write after the tool's exit.

flowtool_predict_full_names

`flowtool_predict_full_names {true | false}`

Default: `false`

Data_type: Boolean

Shows the `skip_metric` flow name in prediction.

flowtool_schedule_flow_immediate

```
flowtool_schedule_flow_immediate {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, flowtool will start the scheduled flows immediately. When set to `false`, it waits for the software to exit before starting the scheduled flows.

flowtool_summary_tcl

```
flowtool_summary_tcl string
```

Default: `puts [report_metric -format text]`

Data_type: String

Specifies the TCL script to run during run_flow summary.

get_feature

```
get_feature
[-help]
[feature]
[-obj string]
```

Returns the feature for a flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_feature` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_feature`
`feature`	Specifies the name of the feature.
`-obj` *`string`*	Specifies the `flow` or `flow_step` to be returned.

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

get_flow_config

```
get_flow_config
[-help]
key ...
[-quiet]
```

Returns the feature configuration for a flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_flow_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_flow_config`
`key ...`	Specifies the key to get configuration value.
`-quiet`	Suppresses errors in case the key does not exist.

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

init_flow

```
init_flow
[-help]
```

The command processes flow information provided by flowtool to setup and run a provided flow sequence. It outputs the affected flow root attributes, such as:

- init_flow summary
- Flow script: /grid/tfo/vol102/tiqa/flow/demo_dtmf/scripts/run_flow.tcl
- Flow: flow:block
- From: prects.block_start
- To: prects.block_finish
- Working directory: /grid/tfo/vol102/tiqa/flow/demo_dtmf
- Starting database: /grid/tfo/vol102/tiqa/flow/demo_dtmf/dbs/block.fplan.block_finish.enc
- Run tag:
- Branch name:
- Metrics file: /grid/tfo/vol102/tiqa/flow/demo_dtmf/flow.metrics
- Status file: /grid/tfo/vol102/tiqa/flow/demo_dtmf/flow.status

Parameters

`-help`	Outputs a brief description that includes type and default information for each `init_flow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man init_flow`

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

is_feature

```
is_feature
[-help]
[feature_name]
[-obj string]
```

Tests if the specified flow feature is defined. The format of this command is similar to that of the `get_feature` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `is_feature` parameter. For a detailed description of the command and all of its parameters, use the man command: `man is_feature`
`feature_name`	Specifies the name of the feature to be tested.
`-obj` *`string`*	Specifies if the `flow` or `flow_step` will be returned.

See also:

- `get_feature`
- Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

is_flow

```
is_flow
[-help]
[-after string]
[-before string]
[-hierarchical]
[-inside string]
[-quiet]
```

Allows you to query during the execution of a flow for further action.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `is_flow` parameter.
`-after` *string*	Specifies the current execution after the `flow` or `flow_step` object.
`-before` *string*	Specifies the current execution before the `flow` or `flow_step` object.
`-hierarchical`	Displays all parent flows.
`-inside` *string*	Specifies the current execution within the `flow` object.
`-quiet`	Ensures that errors are not reported for missing `flow` or `flow_step` objects.

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

read_flow

```
read_flow
[-help]
filename
[-no_check]
```

Loads the content for the current flow and debugs the flow in the tool. The command accepts single argument as the name of the files that describe the `flow` and `flow_step` objects, as shown below:

```
read_flow scripts/run_flow.tcl
```

```
read_flow scripts/flow.yaml
```

This command should not be used for YAML or TCL files, in general. For example, the following command should not be used, in which the `setup.yaml` file is used by the flow, but does not contain flow information:

```
read_flow scripts/setup.yaml
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_flow` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_flow`.
`filename`	Specifies the flow script to read.
`-no_check`	Skips running the `yamllint` and `check_tcl` flow checks.

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

report_flow

```
report_flow
[-help]
```

Generates a report with the following information:

- The flow type, section list and generation date of the templates
- The flow step order
- One line for every item in flow_history , indicating the success of that item and the configuration of optional flow features, and any error message

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_flow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_flow`.

Example

```
Flow Features: low_power
   (optional): prefer_area ccopt
Flow Generated: 2013-05-14 14:40 GMT
Flow Step Order:
  init place prects cts postcts postcts_hold signoff

Step       Status     prefer_area    ccopt    Message
-------------------------------------------------------------------------
init       success    on             off
place      success    on             off
prects     success    on             off
cts        error      on             off      "Prefer_area" doesn't allow CTS.
cts        success    off            off
```

This report indicates:

- The flow was generated with the `low_power` feature enabled, and the `prefer_area` and `ccopt` features optional.
- The init, place and prects steps were run with `prefer_area` enabled, ccopt disabled, and

succeeded.

- The cts step was then run with `prefer_area` enabled, `ccopt` disabled, and it failed.
- The cts step was then run with both `prefer_area` and `ccopt` disabled, and it succeeded.

There is some overlap between `report_flow` and the summary printed at the end of `run_flow` commands. The difference is that `report_flow` reports information about the entire flow history (since the last `run_flow -reset`, if any), whereas the summary at the end of `run_flow` only includes data about the steps run by that command.

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

reset_flow

```
reset_flow
[-help]
[-all]
[-edits]
[-features]
[-flows]
[-history]
[-steps]
```

Resets the content for the current flow in the software by clearing any existing flows, steps, or features. The functionality of the `-history` parameter of this command is equivalent to `run_flow -reset`.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `reset_flow` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man reset_flow`
`-all`	Resets all the flows, steps, features, edits, and history set in the software. *Data_type*: bool, optional
`-edits`	Resets all the flow edits appended through the `edit_flow` command. *Data_type*: bool, optional
`-features`	Resets all the flow features and associated values. *Data_type*: bool, optional
`-flows`	Resets all the flow objects. *Data_type*: bool, optional
`-history`	Resets flow history and state of all the attributes (such as `flow_step_canonical_current`, `flow_history`, and `flow_top`). *Data_type*: bool, optional
`-steps`	Resets all the `flow_step` objects. *Data_type*: bool, optional

See also:

Flow chapter of the *Innovus Stylus Common UI Migration Guide* document.

run_flow

```
run_flow
[-help]
[-directory string]
[-flow string]
[-from string]
[-no_check]
[-no_summary]
[-predict {summary | tcl}]
[-reset]
[-step string]
[-to string]
[-yaml string]
```

The command runs one or more flow steps by default, in the order specified in a single flow.

If more than one flow is defined, the `run_flow` command must be run with `-flow`. However, if `-step` is used, that step does not have to be part of a flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `run_flow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man run_flow`.
`-directory` *string*	Specifies the working directory in which to run the flow. If a directory is not specified, the current directory is used. **Note:** Any relative paths in attributes such as *flow_report_directory* are relative to this directory.
`-flow` *string*	Runs the specified flow (created with the `create_flow` command). `from`/`to` options are based on the flow steps. *Default:* `flow_step_order`

`-from` *string*	Specifies the name of the step to start from. By default, flow starts from the step immediately after the last successfully run step (determined by examining the value of *flow_history*), or the first step if that attribute is not set.
`-no_check`	Skips the error checks prior to running. By default, the checks are run.
`-no_summary`	Skips the full summary from printing. By default, summary is printed and emailed if `flow_mail_to` is set.
`-predict {summary | tcl}`	Displays the projected future steps to run honoring the `-step`, `-flow`, `-from`, and `-to` parameters.
`-reset`	Before running, clears the flow history. After this, the flow starts from the first step if no explicit `-from` step is specified.
`-step` *string*	Runs just the specified step. This takes priority over `from`/`to` options.
`-to` *string*	Specifies the name of the step to run to (inclusive; in other words, the named step is executed). By default, runs to the end of the last step in the flow. Must be a step in the flow, later than the `-from` step (if any).
`-yaml` *string*	Specifies the yaml file to source before running the flow.

Examples

- The following command prints the hierarchical list of steps to be run post the specified run:

  ```
  run_flow -from cts.block_start -to cts.schedule_report_cts -predict
  ```

  Following is the output of the above command:

  ```
  Flow steps which would run
  --------------------------
  cts.block_start
  cts.init_innovus
  cts.init_innovus.extra_settings
  cts.add_clock_spec
  cts.add_clock_tree
  cts.add_tieoffs
  cts.block_finish
  cts.schedule_report_cts
  ```

- The following command prints the list of steps as DPO names to be run post the specified run:

  ```
  run_flow -from cts.block_start -to cts.schedule_report_cts -predict -tcl
  ```

  Following is the output of the above command:

  ```
  flow_step:block_start flow_step:init_innovus flow_step:extra_settings
  flow_step:add_clock_spec flow_step:add_clock_tree flow_step:add_tieoffs
  flow_step:block_finish flow_step:schedule_report_cts
  ```

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

schedule_flow

```
schedule_flow
[-help]
[-branch string]
[-db string]
[-defer]
[-directory string]
[-flow string]
[-include_in_metrics]
[-no_db]
[-sync | -no_sync]
```

Schedules a flow to be run after the current flow. The details of scheduled flows are stored in the `flow_schedule` attribute.e

Parameters

`-help`	Outputs a brief description that includes type and default information for each `schedule_flow` parameter. For a detailed description of the command and all of its parameters, use the man command: `man schedule_flow`.
`-branch` *`string`*	Specifies the branch for the new flow.
`-db` *`string`*	Specifies the starting database of the flow. If not supplied, the final database of the preceding flow (the one currently running) is used.
`-defer`	Prevents the scheduled flow from running, until requested.
`-directory` *`string`*	Specifies the working directory to use for the flow. By default, the current directory is used.
`-flow` *`string`*	Specifies the flow to be scheduled.
`-include_in_metrics`	Includes the flow in the current metric snapshot.
`-no_db`	When specified, flowtool does not use the database saved at the end of the step as the starting database for this flow.
`-no_sync`	When specified, flowtool does not wait for this flow to complete before continuing.
`-sync`	When specified, flowtool waits for all flows in the current branch to complete before starting.

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

set_feature

```
set_feature
[-help]
[feature]
[-obj string]
-value string
```

Saves the feature for a flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_feature` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_feature`
feature	Specifies the name of the feature.
`-obj` *string*	Specifies the `flow` or `flow_step` to be saved.
`-value` *string*	Specifies the value to set the feature.

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

set_flow_config

```
set_flow_config
[-help]
key
value ...
```

Saves the feature configuration for a flow.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_flow_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_flow_config`
`key`	Specifies the key to set configuration value.
`value ...`	Specifies the value to set configuration value.

See also:

Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

set_flowkit_read_db_args

```
set_flowkit_read_db_args
[-help]
[-dynamic_view analysis_view]
[-hold_views hold_views]
[-leakage_view analysis_view]
[-lef_files list_of_files]
[-setup_views setup_views]
[{-oa_lib_cell_view {libname cellname viewname}}] [{-no_timing}] [{-no_timing_graph}]
[{-mmmc_file fileName}]
```

Controls the run-time options used by `read_db` when a flow script is loading `flow_starting_db` specified by `flowtool` or used by `schedule_db`.

Note: This command supports all the parameters supported by `read_db`.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `set_flowkit_read_db_args` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_flowkit_read_db_args`
`-dynamic_view analysis_view`	Specifies the analysis view for the dynamic power analysis.
`-hold_views hold_views`	Specifies the hold views for loading rather than the active hold views.
`-leakage_view analysis_view`	Specifies the analysis view for the leakage power analysis.

`-lef_files` *list_of_files*	Specifies the list of LEF files for loading. Use this option if you want to override the existing LEF files in the design being restored and load different LEF files instead. • This option is intended to allow upgrading to new versions of LEF files that are compatible with the old versions. The list of all LEF files used by this design must be provided. • If any pin locations, cell boundaries, or OBS shapes are changed for used cells, new DRC violations may occur. • If pins are added or removed from used cells, or used cells are removed, fatal errors may occur while loading the netlist. • If the removed pins and cells are unused in the design, warnings or errors may still occur, if any library-specific data was stored for those cells or pins during the design flow (such as `dont_touch` data).
`-mmmc_file` *fileName*	Specifies the path name to a Multi-Mode/Multi-Corner (MMMC) configuration file (for example, the same file format used by `read_mmmc`) to replace the MMMC file saved by `write_db` (and all the corresponding .lib files, .sdc files, etc. referenced by the MMMC file).

`-no_timing`	Skips reading of the MMMC file, timing libraries and timing constraints. **Note:** • Most designs that use LEF rely on the .lib information to define the bus order for MACROs that have bus bit terminals. Therefore, using the `-no_timing` parameter can result in the connectivity to bus ports of leaf level instances being reversed. When `.lib` information is not available, the default is for bus ports to be treated as descending (for example, [7:0]). While using Verilog modules or OpenAccess abstract views to define the bus order, the connectivity will still be correct even when `-no_timing` is specified. • Timing libraries cannot be loaded in the same session after a design is read using the `-no_timing` option. You must use the `set_flowkit_read_db_args` command without the `-no_timing` parameter in a new session to load the design with timing libraries and constraints. • If you save a design after `set_flowkit_read_db_args -no_timing`, it will not save timing information unless you use `set_flowkit_write_db_args -add_ignored_timing`.
`-no_timing_graph`	Skips reading the timing graph even if it was saved to the disk by `set_flowkit_write_db_args`.
`-oa_lib_cell_view {`*`libname cellname viewname`*`}`	Specifies the library name, cell name, and view name for reading an OpenAccess DB design.
`-setup_views` *`setup_views`*	Specifies the setup views for loading rather than the setup views active during `set_flowkit_write_db_args`.

Related Topics

- `read_db`
- `set_flowkit_write_db_args`
- Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

set_flowkit_write_db_args

```
set_flowkit_write_db_args
[-help]
[-add_ignored_timing]
[-def]
[-lib]
[-no_constraint]
[-no_fill]
[-no_pvs_fill]
[-no_wait string]
[-sdc]
[-user_path]
[-verilog]
[{-rc_extract}] [{-timing_graph}] [{-oa_lib_cell_view string}] [{-oa_view string}] [{-lib_to_ldb}]
```

Allows you to save databases through flowkit in the Open Access format and specify the library/cell/view information. The `set_flowkit_write_db_args` command can be used to extend the options used to save a design on flow-steps, which are marked with `skip_db false`. This command needs to be placed inside the `flowstep body_tcl` of interest since it only affects that `flowstep` and resets back to default state when that step completes. The format of this command is similar to what would be used for `write_design`:

```
create_flow_step -name save {set_flowkit_write_db_args -rc -lib2ldb}
```

This saves a database named `save`, which uses the `-rc` and `-lib2ldb` options for `write_db`. Previous and subsequent flowsteps use default `write_design` options unless overridden using the `set_flowkit_write_db_args` command.

Note: This command supports all the parameters supported by `write_db`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_flowkit_write_db_args` parameter.

`-add_ignored_timing`	Copies the old timing data when loaded in physical-only mode. If you use `set_flowkit_read_db_args -no_timing` to read the DB, the timing data is not loaded. This option allows you to copy the ignored timing data from the original `read_db` location and add it back during `set_flowkit_write_db_args` like this:
	`set_flowkit_read_db_args -no_timing test1`
	`<physical editing commands ...>`
	`set_flowkit_write_db_args -add_ignored_timing test2`
	This usage model is intended to allow faster physical-only editing, like DRC fixing, where the netlist is not modified. If you change the netlist, unpredictable results may occur when reading the `test2` DB using timing data copied from `test1`.

`-def`	Saves a DEF file, in addition to the normal DB files in the DB directory. This parameter is normally not used for flat or block flows and is not used by `set_flowkit_read_db_args`. It can be useful in a hierarchical flow, where you want to use `assemble_design` to read only the netlist and the DEF files without opening up each design DB and writing out the Verilog and DEF files separately (see `-verilog`).
`-lib`	Resolves symbolic links of the timing lib files and side files. Normally external file references, such as to the LEF, .lib, SDC, files are saved using full path-name, symbolic links to the original files inside a libs sub-directory inside the DB directory. This parameter forces copies of the files rather than symbolic links in the libs sub-directory so there are no external file references. That makes the DB directory much larger, and makes it suitable for sending to a different network where the library and external data files are not available. An OA DB will still have OA library references using the normal cds.lib methodology, but any other external file references will be copied inside the cellview directory.

`-lib_to_ldb`	Converts the `.lib` timing libraries referenced in the MMMC file to the `.ldb` format and writes them in a libs sub-directory inside the output design directory. References to the original .lib files in the MMMC file are changed to the corresponding `.ldb` files inside the libs sub-directory. This makes the design directory much larger because it has all the timing library data inside it and is not recommended for normal flows. Normally, you should setup `.ldb` files in an external library directory and have the MMMC files use them directly to avoid this overhead. Using `.ldb` files will make `set_flowkit_read_db_args` faster when it reads the library data, so this option is useful for doing "throw-away" debugging experiments that require repeated sessions at the cost of a one-time conversion, and carrying the library data with the design.
`-no_constraint`	Saves the timing graph data without the SDC constraints data. This parameter requires the `-timing_graph` parameter and is useful in reducing the disk size, and speeding up `set_flowkit_read_db_args` and `set_flowkit_write_db_args`. However, the timing-graph data cannot be loaded by any other release version. Additionally, the timing constraints would be lost without the SDC files, if loaded by a different release version. **Note:** You can still load the timing-graph with the current release version, and then use `set_flowkit_write_db_args` without the `-no_constraint` parameter to regenerate the SDC files from the timing-graph and then use `set_flowkit_read_db_args` with a later release.
`-no_fill`	Prevents writing of any metal or via fill shapes inside the database. This is useful during the ECO routing, since the fill data is regenerated anyway. This way, it is not worth saving the current fill data. All the fill data that would go to DEF FILLS section (floating fill data), or the DEF SPECIALNETS section with + SHAPE FILLWIRE or FILLWIREOPC will be ignored and not written.

`-no_pvs_fill`	Prevents saving the PVS fill data file. In some PVS flows, a stream (GDS) file from PVS is saved inside the software DB directory. During `set_flowkit_read_db_args`, this file is not read, but can be optionally loaded or used later in the flow. By default, if the PVS fill file exists, `set_flowkit_write_db_args` will copy it from the `set_flowkit_read_db_args` location to the new DB directory. If ECO changes to the current design have made the PVS fill file invalid, use this parameter to stop it from being copied.
`-no_wait` *string*	Writes the design in a separate process. You can continue executing new commands simultaneously while the design is being saved. This is useful when you want to save intermediate results without waiting for the save to complete.
`-oa_lib_cell_view` *string*	Saves the database to an Open Access library cell view.
`-oa_view`	Specifies the Open Access view name.
`-rc_extract`	Writes RC extraction data, including data for incremental extraction. By default, the software does not automatically save RC extraction data to save disk space and run-time. The `-rc_extract` parameter does not save RC extraction data if the design is not extracted or if extract_rc_engine = pre_route.
`-sdc`	Saves the timing constraint file even if it is not modified.
`-timing_graph`	Saves the timing graph data.
`-user_path`	Saves external file references exactly same as the user entered it.
`-verilog`	Saves a Verilog file inside the DB directory, in addition to the normal DB files. This parameter is not normally used for flat or block flows. It can be used in a hierarchical flow, where you want to use `assemble_design` to read only the Verilog and the DEF files without opening up each design DB and writing out the Verilog and DEF files separately.

References

- `write_db`
- `set_flowkit_read_db_args`
- Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

write_flow

```
write_flow
[-help]
[-directory string]
[-out_file string]
[-flow_tcl]
[-flow_yaml]
[-include_state]
[-setup_yaml]
```

Generates a Tcl script that may be sourced to reproduce the current flow. The script includes commands to:

- Set the value of all top-level attributes with non-default values to their current values
- Recreate all flow step objects with their current attributes, aside from those relating to run history (specifically, `run_count` and `status`).

Sourcing this script reproduces (but not runs) the current flow in a fresh session, and forms a user readable and editable record of the flow (compared to the flow data saved in the DB, which should not be read or edited directly).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_flow` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_flow`.
`-directory` *string*	Specifies the directory is which all the files need to be written.
`-out_file` *string*	Specifies the name of the output file to generate.
`-flow_tcl`	Writes only the `flow.tcl` file in the directory.
`-flow_yaml`	Writes only the `flow.yaml` file in the directory.
`-include_state`	Writes out the current flow state. This can be used to restore the flow.
`-setup_yaml`	Writes only the `setup.yaml` file in the directory.

Related Topics

- `write_flow_template`
- Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

write_flow_template

```
write_flow_template
[-help]
[-describe string]
[-directory string]
[-enable_feature string]
[-ip_steps string]
[-list]
[-optional_feature string]
[-tech_steps string]
[-template_file string]
[-tools]
[-type string]
[-user_steps string]
[-version string]
```

Creates a set of template TCL files to be used as the basis for a flow. This command is used at the beginning after which, you can manage the generated and modified files. When invoking the command, you will select the type of flow required (implying a subset of template scripts to include) and the arrangement of sections across generated files.

On completion, a message is given to indicate creation of the flow template in the specified directory.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_flow_template` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_flow_template`.
`-describe string`	Specifies detailed description of the available flow features. If it matches multiple features, then these features are listed along with a short description for each. If it matches exactly one feature, a full description of this feature is printed. Matching no features results in an error.

`-directory` *string*	Specifies the name of the directory for saving the generated files. If no directory is specified, the command uses the name *flow*. If required, the tool appends "_" and an integer to the directory name to avoid overwriting an existing directory. For example, *flow_1*, *flow_2* and so on).
`-enable_feature` *string*	Specifies a list of flow features that are unconditionally incorporated into the generated scripts.
`-ip_steps` *string*	Overlays the contents of the generated `flow.yaml` file and inserts the specified scripts under the `ip_flow_steps` flow.
`-list`	Lists the available flow templates.
`-optional_feature` string	Specifies a list of flow features that are conditionally incorporated into the generated scripts. Both the enabled and disabled code are included, with conditionals based on a new user attribute.
`-tech_steps` *string*	Overlays the contents of the file(s) generated by running the `write_flow_template` command. The file(s) provided with this parameter are copied into the `scripts` directory inside a `tech` sub-folder and appear in `tech_flow_steps` for the stylus flows.
`-template_file` *string*	Specifies the flow template file.
`-tools`	Returns the list of tools whose scripts should be included in the flow.
`-type` *string*	Specifies the type of flow to be used.
`-user_steps` *string*	Overlays the contents of the generated `flow.yaml` file and inserts the specified scripts under the `user_flow_steps` flow.
`-version` *string*	Specifies the version of the flow to be used.

Related Topics

- `write_flow`
- Flow chapter of the *Tempus Stylus Common UI Migration Guide* document.

9

General Commands and Attributes

- alias
- check_ccr
- decrypt
- define_proc
- define_variables
- delete_dangling_ports
- delete_floating_constants
- deselect_obj
- encrypt
- eval_legacy
- get_common_ui_map
- get_legacy
- is_common_ui_mode
- Message Commands
- pop_snapshot_stack
- print_message
- program Category Attributes
- push_snapshot_stack
- rename_obj
- rename_snapshot
- report_area
- report_dangling_ports
- report_hidden_usage
- report_obj Category Attributes
- resume
- select_obj
- set_layer_preference
- set_log_post_cmd

- set_proc_verbose
- suspend
- system Category Attributes
- time_info
- uniquify_filename
- Utility Commands

alias

```
alias
[-help]
new_cmd
old_cmd
[-args {{new old} ...}]
```

Creates command name aliases for Tcl commands, except built-in Tcl commands. This command supports argument name aliases for application commands and procs that use `define_proc` and `define_variables` commands for parsing the arguments.

The argument aliases are private, so these do not display with `-help`, and also do not support partial option completion or matching. However, the command aliases are public, so <tab> complete works.

You can use this command to migrate an existing Tcl script when some of the commands or argument names have changed.

Parameters

`-help`	Prints a brief description that includes type and default information for each `alias` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man alias`
`new_cmd`	Specifies the new command name that is an alias for the existing *`old_cmd`* name. It can be the same name as the *`old_cmd`* if you only want to create new `-arg` aliases, and do not need a new command alias.
`old_cmd`	Specifies the existing old command name.
`-args {{`*`new old`*`} …}`	Specifies a list of new and old argument names to create aliases. You can give multiple *`new`* names for the same *`old`* name. The new aliases are private, and do not show up with `-help`. You can create more than one alias for the same *`old`* name.

Example

The following command adds a new command name alias `new_read_sdf` and some argument name aliases for `read_sdf`:

```
>alias new_read_sdf read_sdf -args {
  {-ignore_error -continue_on_error}
  {-summary -report_summary}
  {-sum -report_summary}
}
```

Then, the following commands

```
>new_read_sdf in.sdf -ignore_error -summary
>new_read_sdf in.sdf -ignore_error -sum
>new_read_s in.sdf -ignore_error -sum
```

would behave the same as

```
>read_sdf in.sdf -continue_on_error -report_summary
```

Note: Tcl partial command matching works if the partial command is unique as shown in the above example, but the alias argument names are private, and will not support partial option matching, or appear in the `-help` output.

Related Commands

- define_proc
- define_variables

check_ccr

```
check_ccr
[-help]
ccr_number
```

Returns 1 if the specified CCR is integrated into this build and 0 if it is not.

- It includes all integrated CCRs that have the `Checked_In` or `Validated` status at the time of the build.
- It only includes master CCRs. Duplicate CCRs are not included.
- It currently lists all CCRs fixed in the current release stream or previous release. For example, checking in release version 16.22, will find all CCRs integrated into 16.10 through 16.22 that are in the current 16.22 release.

Parameters

`-help`	Prints a brief description that includes type and default information for each `check_ccr` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_ccr`
`ccr_number`	Specifies the CCR number to check.

Examples

- In the example below, the return value "1" indicates that the CCR 1234567 is integrated into this build.

```
tempus> check_ccr 1234567

1
```

decrypt

```
decrypt
[-help]
file_name
```

Decrypts and evaluates a Tcl file that was encrypted with the encrypt command.

Parameters

`-help`	Prints out the command usage.
file_name	Specifies the name of the Tcl file to be decrypted and evaluated. *Data_type*: string, required

Related Topics

- encrypt

define_proc

```
define_proc
[-help]
proc_name
arg_list
proc_body
[-description proc_help]
[-extended_help description]
[-hidden]
```

This command defines a Tcl proc that uses an extended argument syntax to check for legal argument names and legal values. It is a super-set of the functionality supported by define_proc_arguments, and parse_proc_arguments and easier to use.

The arguments are parsed, and if any errors occur, an error message is written, and a TCL_ERROR is returned, so the *proc_body* is not executed. If the arguments are parsed successfully the Tcl vars are set accordingly, and the *proc_body* is executed.

It supports simple types like int, string, etc., built-in types like point, rect, and objects like inst, net, pin.

Commands defined with define_proc also have the same support as other Tempus commands, including:

- help support (e.g. "cmd -help" to see the options, and "help c*" would find the command named cmd, etc.)
- tab-completion of partial command or option names (e.g. "cmd<tab>" or "cmd -opt<tab>")
- partial argument name matching is allowed (e.g. "cmd -opt" will work if -opt only matches one option name prefix like -opt_test)
- redirect output to files or Tcl vars using > or >> (e.g. "cmd > my_file", or "cmd > &my_tcl_var")

Parameters

arg_list	Defines the argument list of the Tcl procedure in the following format for each argument:
	`{var default arg_name data_type flags help}`
	For example
	`{report_file report.out {-out_file file_name} string optional {The output file for the report} }`
	In this format:
	• *var* is the local Tcl variable to assign, such as report_file in the example above. The value for the argument is assigned to this variable before the body of `define_proc` is invoked.
	• *default* is the default value, such as report.out in the example above. If the argument is not given, then *var* will be assigned the *default* value. Note, unlike an argument value passed to the command, the type of the default value is not checked.
	For example, the following command would set int_var to "no_value" if -my_int is not given. Normally the default value should be a legal int value, but you can use this to check if the argument was given for an optional argument when there is no good default value.

```
{int_var no_value {-my_int int} int optional {int value help description}}
```

The default value is ignored if the argument is a required argument (see *flags* below).

- *arg_name* is the argument name with optional value help. Both named {-opt value} and unnamed {value} arguments are supported.

  If the first word starts with a dash, it is a named argument with optional help string. For example, `{-out_file file_name}` will appear in the help output as: -out_file <file_name>.

  If no help string is given like this:
  `{-out_file}`

  The help output will include the data_type like: -out_file <string>.

  If the first word does not start with a dash, it is an unnamed argument and the first word is used as the help string. For example, `{out_file}` will appear in the help output as: `<out_file>`.

- *data_type* is the argument data type. Many types are supported including simple types, bool type, enum type, built-in types, and object types.
  - These simple types are supported: **int, float, double, string.** They all require an argument value that matches the type (e.g. -my_int 23, or -my_string "my test string"). **float** and **double** are both allowed, and mean the same thing. For example

    ```
    {int_var 0 {-my_int int} int optional {int value help description}}
    ```

    Note that a integer passed to a float or double type will get a .0 added to the end so it can be passed to expr and forced to use floating point arthimetic.

  - The **bool** type supports an argument with or without a value. If the *arg_name* definition has a value help string, then a bool value is required, otherwise a value is not allowed. So, a switch that takes no value would look like:

    ```
    {switch_var false -my_switch bool optional "-my_switch help description"}
    ```

    and would be used like this with no value

    ```
    -my_switch
    ```

    The switch_var will be set to true if -my_switch is given, and set to the default value of false if not given.
    A bool with a value required, would look like this

    ```
    {bool_var false {-my_bool bool} bool optional "bool with value help description"}
    ```

    and would be used like this

    ```
    -my_bool true
    ```

    Any Tcl allowed bool value can be used for the input value (e.g. true, false, 1, 0, on, off, etc.) but the bool_var will always have either true, false or the *default* value if the argument is not given.

  - These built-in types are supported: **point, rect.**
    - **point** is exactly two doubles. The input can be either a single list or list of lists style, so both {1.0 2.0} or {{1.0 2.0}} are allowed as input. The point_var will always be assigned as a simple list of two values for either type of input. An integer value gets a .0 added, so it can be used in a Tcl expr and forced to be treated as a double. So an input like {1 2} will be

assigned to the point_var as {1.0 2.0}.

- **rect** is exactly four doubles. It can be in a single list or list of lists style, so {0 1 2 3}, {{0 1} {2 3}} or {{0 1 2 3}} are allowed. The rect_var will always be assigned as a simple list of four values.

- An **enum** type must include the list of legal enum values like this {**enum** val1 val2 ...}. It supports an optional list_type of ***** (a list of 0 or more) or **+** (a list of 1 or more) as the last enum value. It allows an optional {**hidden** val1 val2 ...} list for hidden enum values. The enum value inputs are case-insensitive, but the return value assigned to *var* is always forced to be all lower-case characters.
  For example,
  `{sort_var name {-sort} {enum name age {hidden city state} +} optional {help description}}`
  would allow `-sort age`, or `-sort "name CITY"`, but would give errors for `-sort ""`, or `-sort abc`.
  The help string will show the enum values but not the hidden enum values like this:

  ```
  -sort {name age} # help description (enum+, optional)
  ```

- An **object** type allows object names and pointers to be passed in as values like this {**object** *obj_type1 [obj_type2 ...]* [*list_type*]}. It supports an optional *list_type* of ***** (a list of 0 or more) or **+** (a list of 1 or more) as the last value. If *list_type* is not given, exactly one value is required for input. For example,

  ```
  {driver_var {} {-drivers} {object pin port *} optional {help description}}
  ```

  would allow a list of 0 or more pin or port objects for input. The input could be a list of pin or port pointers, or a name-pattern like "i1/p*" that allows * and ? pattern matching. *driver_var* will be assigned a list of pointers of the correct type (or types). So both of these uses would have the same result:

  ```
  -drivers [concat [get_db pins i1/p*] [get_db ports i1/p*]]

  -drivers i1/p*
  ```

  In both cases, all the pins and ports that match i1/p* would be returned as a list of objects and assigned to *driver_var*.

  If any objects are passed in that are not pin or port objects, an error will occur during parsing.

  If at least one pointer is required (e.g. list_type **+** or no list_type), and none are matched by the name pattern, an error will occur during parsing.

  Design objects like **hinst, inst, hnet, net, hpin, pin**, **port**, etc and library objects like **base_cell, base_pin, lib_cell, lib_pin**, etc, and technology objects like **layer**, **route_rule**, etc. are supported, but only objects that have a .name attribute can use a name-pattern in addition to pointers (e.g. a wire object has no .name so it cannot be looked up with a name-pattern). See the Stylus Common UI Database Reference for a list of all the obj_types.

- *flags* is list of flags that may be an empty list. The supported flags are

  - **optional** means the argument is optional. If the argument is not given, *var* will be set to the *default* value. Arguments are **optional** by default, so this is mostly to make it easy to see it in

	the definition. It cannot be combined with **required**.
	◦ **required** means the argument is required. It is an error to not have the argument.
	◦ **multiple** means the argument can appear multiple times and *var* will be set to a list of all the values.
	◦ **hidden** means the argument is not shown in the help output, partial argument name matching will not match it, and <tab> complete will not show it. **hidden** must be optional, so it cannot be combined with **required**, **required_group,** or **exclusive_required_group** flags.
	◦ **obsolete** means the argument still works but is considered obsolete. It will give a warning message if it is used that the user should stop using this argument to be compatible with future releases. An **obsolete** argument will automatically be **hidden** and cannot be a required argument. See **alias** if you just want to change the *arg_name* to a new *arg_name*.
	◦ {**exclusive_group** *group_name*} means only one of the arguments can be given from the same *group_name*. For example, only one of -my_int or -my_int2 are allowed:
	`{int_var 0 {-my_int int} int {{`**`exclusive_group`** `group1}} {help for -my_int}}`
	`{int_var2 0 {-my_int2 int} int {{`**`exclusive_group`** `group1}} {help for -my_int2}}`
	◦ {**exclusive_required_group** *group_name*} means exactly one of the arguments must be given from the same *group_name.*
	◦ {**required_group** *group_name*} means one or more of the arguments must be given from the same *group_name.*
	◦ {**requires** *arg_name1* [*arg_name2* ...]} means this argument requires all the other arguments in the list. For example both -arg1 and -arg2 are required if -my_int is given.
	`{int_var 0 {-my_int int} int {optional {`**`requires`** `-arg1 -arg2}} {help for -my_int}}`
	◦ {**alias** *obsolete_name* ***[warn]***} means an alias of *obsolete_name* to the *arg_name* will be added. If **warn** is given, a warning message will be given if the *obsolete_name* is used telling the user to change to *arg_name,* otherwise *obsolete_name* will behave just like *arg_name.* The *obsolete_name* is not shown in help output. This can be used to migrate users to new argument names, but keep the old name for backward compatibility. For example, to migrate from `-old_my_int` to `-new_my_int` you would use this:
	`{int_var 0 {-new_my_int int} int {{`**`alias`** `-old_my_int}} {help for -new_my_int}}`
	• *`help`* is a help string for the argument, such as *"a file name to read"*.

`-description` *`proc_help`*	Specifies the help description for the defined procedure, shown by "help cmd" or "cmd -help".
`-extended_help` *`description`*	Specifies the extended help description to the procedure. It is printed after `-description` and *`arg_list`*. Shown by "help cmd" or "cmd -help".
`-help`	Outputs the command usage and a brief description about the command parameters.

`-hidden`	Makes the procedure hidden and not public. That means <tab> complete will not return this proc name, and "info commands *" will not return this proc name. But if you type the full name like "info commands full_cmd_name" it will return the command, or "full_cmd_name -help" will return help for it.
`proc_body`	Defines the Tcl procedure body that uses var names from *`arg_list`*.
`proc_name`	Specifies the name of the Tcl procedure.

Example

Here is an example of some common usage:

```
define_proc test -description "An example of the extended argument syntax" {
    {out_file "default.out" out_file string optional "The output file name"}
    {my_count 0 {-count num_inputs} int required "The number of inputs"}
    {verbose false -verbose bool optional "Verbose output"}
    {check_err not_set {-check_err bool} bool optional "Override global check setting"}
    {effort low -effort {enum low medium high} optional "How much effort to use"}
    {my_insts {} {-insts} {object inst +} optional "One or more insts"}
} {
# Proc body goes here
    ...
}
```

The above procedure generates the following help output:

```
> test -help
```

An example of the extended argument syntax

```
Usage: test [-help] [<out_file>] [-check_err <bool>] -count <num_inputs> [-effort {low medium high}] [-
verbose]
-help                       # Prints out the command usage
<out_file>                  # The output file name (string, optional)
-check_err <bool>           # Override global check setting (bool, optional)
-count <num_inputs>         # The number of inputs (int, required)
-effort {low medium high}   # How much effort to use (enum, optional)
-insts <inst>+              # One or more insts (Iinst)+, optional)
-verbose                    # Verbose output (bool, optional)
```

define_variables

```
define_variables
[-help]
name_list
[-transient  | -delete ]
```

Creates Tcl variables in the global namespace if they do not already exist, and adds them to a save list to be saved by `write_db` and restored by `read_db`. The variables will be saved even if the root attribute `write_db_auto_save_user_globals` is false. It can also remove Tcl globals that would be automatically saved if `write_db_auto_save_user_globals` is true. It is intended for Tcl global variables, but it also supports namespace variables.

During read_db, the Tcl variables are created if they do not already exist and the value is set. If a namespace variable is given, it will also create the namespace if it does not already exist.

Parameters

`-delete`	Deletes the Tcl variables and removes them from the save and transient lists. It returns an error if the Tcl variables do not exist.
`-help`	Outputs a brief description that includes type and default information for each `define_variables` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man define_variables`
`name_list`	Specifies a list of Tcl variable names. If the specified Tcl variable does not already exist, it is created and added to a save list of variables to be saved by `write_db`.
`-transient`	Adds the Tcl variables to a transient list used by `write_db` (and removes them from the save list if they are there). The transient list will override the automatic save list created if the root attribute `write_db_auto_save_user_globals` is `true`, so the variables will not be saved.

Examples

The following command saves the variables `global1` and `namespace1::var1` with `write_db`:

```
>define_variables {global1 namespace1::var1}
```

Related Topics

- `write_db_auto_save_user_globals`
- `read_db`
- `write_db`

delete_dangling_ports

```
delete_dangling_ports
-help
-partition
```

Deletes ports that are disconnected from both the inside and outside of a module, and ports that are connected within the module but disconnected outside the module.

Parameters

`-help`	Prints a brief description that includes type and default information for each `delete_dangling_ports` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_dangling_ports`
`-partition`	Specifies the partition only flag of the type boolean. This is an optional.

delete_floating_constants

```
delete_floating_constants
-help
```

Deletes dangling 1'b1s or 1'b0s from the Verilog. They are either dangling nets assigned 1'b1 or 1'b0 or 1'b1/1'b0 connections that are not hierarchically connected to a leaf instance.

Parameters

None

Example

In the code below only BUFX2 is a leaf.

```
//
// Original Verilog
module hier4 (A,B);
input A;
input B;

BUFX2 u0 (.A(A));
BUFX2 u1 (.A(1'b1));
endmodule

module hier3 ();

hier4 u0 (.A(1'b1),.B(1'b0));
hier4 u1 (.A(1'b0),.B(1'b1));
endmodule

module hier2 ();

hier3 u0 ();
endmodule

module top ();

// Internal wires
wire a;
wire b;
wire c;
wire d;

assign a = 1'b1 ;
assign b = 1'b1 ;
assign c = 1'b0 ;
assign d = 1'b0 ;

hier2 u0 ();
BUFX2 u1 (.A(a));
BUFX2 u2 (.A(c));
endmodule
```

```
//
// Verilog after delete_floating_constants is issued

module hier4 ( A, B);
input A;
input B;

BUFX2 u0 (.A(A));
BUFX2 u1 (.A(1'b1));
endmodule

module hier3 ();

hier4 u0 (.A(1'b1));
hier4 u1 (.A(1'b0));
endmodule

module hier2 ();

hier3 u0 ();
endmodule

module top ();

// Internal wires
wire a;
wire c;

assign a = 1'b1 ;
assign c = 1'b0 ;

hier2 u0 ();
BUFX2 u1 (.A(a));
BUFX2 u2 (.A(c));
endmodule
```

deselect_obj

```
deselect_obj
[-help]
{[-all ] | [obj_list]}
```

Deselects the object or the list of objects from a set. The object type can be any object type drawn on GUI such as instance wire, via instance.

Parameters

`-help`	Prints a brief description that includes type and default information for each `deselect_obj` parameter. For a detailed description of the command and all of its parameters, use the man command: `man deselect_obj`
`-all`	Deselects all selected objects.
`obj_list`	Specifies the list of objects to be deselected.

Example

```
deselect_obj [dbGet -p top.insts.name i1]
```

encrypt

```
encrypt
[-help]
input_file_name
[-pragma]
[> file]
[-tcl  | -verilog ]
```

Encrypts a file in an nc-protect compatible format.

Parameters

`-help`	Prints out the command usage.
input_file_name	Specifies the file to be encrypted. *Data_type*: string, required
> file	Specifies the name of the encrypted file. By default, the encrypted file is printed to standard out (stdout). *Data_type*: redirect_operator, optional
`-pragma`	Specifies to only encrypt the text between the protect begin and protect end NC Protect pragmas. Encryption block must start with pragma protect comment. *Data_type*: bool, optional
`-tcl`	Specifies that the file to be encrypted is a Tcl file. This is the default option. *Data_type*: bool, optional
`-verilog`	Uses Verilog style comments for NC Protect pragmas. *Data_type*: bool, optional

Related Topics

- decrypt

eval_legacy

```
eval_legacy
[-help]
script
```

Runs a legacy command in the legacy Tcl interpreter. This command is not required in normal usage. However, it may be needed in cases where some legacy usage is not yet available in the Common UI. This command is intended as a temporary workaround as users transition to the Common UI and will be removed in a future release.

You should first check if there is a Common UI equivalent command before using this command by using the `get_common_ui_map` command:

```
>get_common_ui_map <legacyCommand>
```

Most legacy commands are available, but legacy commands that have behavior changes in the Common UI will not work properly (`saveDesign`, `restoreDesign`, `init_design`, `report_timing` etc.).

The legacy commands run in a separate Tcl interpreter, so Tcl variables in the current interpreter are not available. That means the *script* should normally be inside " " rather than { } so Tcl variable substitution occurs in the current interpreter before passing the script to the legacy Tcl interpreter.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `eval_legacy` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man eval_legacy`
`script`	Specifies the name of the legacy script.

Example

The following command runs `setFillerMode` and `addFiller` commands in the legacy Tcl interpreter.

```
set my_prefix fill_

eval_legacy "setFillerMode -corePrefix $my_prefix"

eval_legacy "addFiller ..."
```

get_common_ui_map

```
get_common_ui_map
-help
legacy_name
```

Provides the mapping of a legacy command and its options in Stylus Common UI.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_common_ui_map` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_common_ui_map`
`legacy_name`	Specifies the legacy command name or the UDM name.

Examples

- The following command gets the mapping of the `placeDesign` command in Stylus Common UI:

  ```
  get_common_ui_map placeDesign
  ```

  Output:

  ```
  placeDesign             place_design
  -clkGateRecloning       -clock_gate_recloning
  -ilm                    -ilm
  -incremental            -incremental
  -noPrePlaceOpt          -no_pre_place_opt
  -outDir                 -out_dir
  ```

get_legacy

```
get_legacy
-help
common_ui_cmd_attr_name
```

Returns the legacy command, legacy Tcl global, or legacy `setXXXMode -option` that corresponds to the specified Stylus Common UI command or root attribute.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_legacy_command` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_legacy`
`common_ui_cmd_attr_name`	Specifies the Stylus Common UI command name or attribute name. *Data_type*: string, required

Examples

- The following command returns the `set_ilm_mode` option that corresponds to the Stylus Common UI attribute `ilm_keep_high_fanout_ports`:

  ```
  @tempus 1> get_legacy ilm_keep_high_fanout_ports
  Mode option: set_ilm_mode -keepHighFanoutPorts
  ```

- The following command returns the legacy Tcl global that corresponds to the Stylus Common UI attribute `init_lef_files`:

  ```
  @tempus 2> get_legacy init_lef_files
  Tcl global: init_lef_file
  ```

is_common_ui_mode

```
is_common_ui_mode
[-help]
```

Checks if the Stylus Common UI mode is enabled.

Parameters

None

Examples

The following example checks it on the Tempus prompt:

```
tempus> is_common_ui_mode

1
```

Return value `1` indicates that the Stylus Common UI mode is enabled.

Message Commands

- get_message
- report_messages
- set_message

get_message

```
get_message
-help
-id msgID
[-severity | -limit | -suppress | -count | -short | -long | -on_error  | -stop_script_limit | -detail ]
```

Returns the limit, severity, or suppression information about a message.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_message` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_message`
`-count`	Returns the number of times this message has been issued this session.
`-detail`	Returns the message of specified id.
`-id msgID`	Specifies a single message ID, in the form "ENCFOO-123".
`-limit`	Returns the limit for the specified message as an integer. -1 is unlimited. 0 is suppressed.
`-long`	Returns the long message description, as used for the Description section in "`man msgID`"
`-on_error`	Returns the error action for the specified message.
`-severity`	Returns the severity, `error`, `warn`, `info`, or `default`, of the specified message as a string.
`-short`	Returns the short message description. Equivalent to "`help msgID`" .
`-stop_script_limit`	Returns the message limit allowed for every public command. Message limit set as -1 means unlimited.
`-suppress`	Returns `1` if the message is suppressed, `0` if not suppressed.

Examples

The following commands return the suppression, severity, and limit levels for the message ID `IMPDB-1241`:

- `tempus> get_message -id IMPDB-1241 -suppress`
  `0`

- `tempus> get_message -id IMPDB-1241 -severity default`
- `tempus> get_message -id IMPDB-1241 -limit 20`

Related Information

- set_message

report_messages

```
report_messages
[-help]
[[{-errors | -warnings | -all } [-count ]] | -suppressed] [[-prefix <string> [-detail] [-summary ]]]
[-detail | -count | -suppressed ] [-summary | -count | -suppressed]
```

Reports the messages that have been issued since the software was launched.

When you run the `report_messages` command without any parameters, it prints a summary table and does not return any Tcl value. However, when you specify a parameter(s), it returns a list a of message IDs.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_messages` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_messages`
`-all`	Returns a list of all issued message IDs as a Tcl result. The result does not include suppressed message IDs.
`-count`	Returns a count of every message when the `-errors`, `-warnings`, or `-all` parameter is specified.
`-detail`	Returns a list of detailed message of the specified prefix.
`-errors`	Returns a list of issued error message IDs as a Tcl result.
`-prefix`	Returns a list of issued error message IDs matched with the specified prefix as a Tcl result.
`-summary`	Returns a message summary of the specified prefixes.
`-suppressed`	Returns a list of suppressed message IDs as a Tcl result.
`-warnings`	Returns a list of issued warning message IDs as a Tcl result.

Examples

- The following command reports the count of all issued message IDs:

  ```
  report_messages -all -count
  ```

  ```
  {IMPMSMV-1810 855} {IMPEXT-3570 6} {IMPESI-3140 1} {IMPMF-5054 1} {CHKTIM-4 16} {IMPTCM-6 1} {IMPTCM-46 2} {IMPTCM-48 1} {SPEF-1152 4} {TCLCMD-981 2} {TCLCMD-1110 1} {TCLCMD-1247 1} {TCLCMD-1470 1}
  ```

- The following command reports the count of all issued warning message IDs:

  ```
  report_messages -count -warnings
  ```

  ```
  {IMPMSMV-1810 858} {IMPEXT-3570 6} {IMPESI-3140 1} {IMPMF-5054 1} {CHKTIM-4 16} {SPEF-1152 4}
  ```

- The following command reports a list of suppressed message IDs:

```
report_messages -suppressed

IMPESI-3086 IMPESI-3188 IMPESI-3194 IMPESI-3199 IMPESI-3198 IMPESI-3197 IMPESI-3196
```

set_message

```
set_message
-help
-id list_of_msgIDs
[-reset_count  | -severity {warn error info} | -limit number | -no_limit | -suppress | -unsuppress | -
on_error {msg_only exit stop_script}| -stop_script_limit limitNumber]
```

Specifies to change the message severity levels, `INFO, WARNING` or `ERROR`, at the beginning of the message. This command allows you to control the messages that are displayed for a particular design. For example, if there are some warnings that are critical for the design, you can change the message severity to `ERROR`. Similarly, if there are messages that you want to ignore, you can change the message severity to `INFO`.

Parameters

-help	Outputs a brief description that includes type and default information for each `set_message` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_message`.
`-id` *list_of_msgIDs*	Specifies the list of message IDs that require a change in the message severity level. If you do not specify a message ID, the change in severity level is applied to all message IDs.
`-limit` *number*	Sets the message limit to *number* per every public command. The default limit is 20, so any specific message will be written out 20 times for one command and then further output of that message will be disabled (along with an additional message saying the message limit has been exceeded). If the number is set to 0, the message will be completely suppressed for the next public command that triggers the message. You can unsuppress the message by setting the number greater than 0.
`-no_limit`	Removes any existing limit on the selected messages. Suppressed messages are not unsuppressed.
`-on_error {msg_only | exit | stop_script}`	Specifies an extra action for error messages (not for warn or info messages). If no message is specified, the action applies to all error messages. • `msg_only`: Prints error message only, and the command proceeds as it normally would. This is the default. • `exit`: Exits immediately and returns to Linux. • `stop_script`: Waits for the current Tcl command to finish, then stops executing any Tcl script, and returns to the interactive Tcl prompt. If the message severity is changed from `error` to `warn` or `info`, the software will no longer trigger extra action.

`-reset_count`	Resets the count of the specified message. You can use this option when you want to clear previously generated warnings or errors before beginning the next major step in the design flow.
`-severity {warn | error | info | reset}`	Specifies the new severity level for the message ID being modified. The `reset` argument sets the message severity back to its original state.
`-stop_script_limit` *limitNumber*	Sets the message limit for a public command. If you set `-stop_script_limit` for a command as `10` and message for the command is triggered for the 10th time, the tool will wait for the current Tcl command to finish, then stop executing any Tcl script, and return to the interactive Tcl prompt. The default value of `-stop_script_limit` is `-1`, which is unlimited.
`-suppress`	Suppresses the message.
`-unsuppress`	Unsuppresses the message.

Examples

- The following command changes the message IMPLF-81 from `ERROR` to `WARN`:
  ```
  set_message -id IMPLF-81 -severity warn
  ```
- The following command returns all messages to their original severity:
  ```
  set_message -severity reset
  ```
- The following command resets the specified message to its original severity:
  ```
  set_message -id ENCLF-18  -severity reset
  ```

Related Information

- `get_message`

pop_snapshot_stack

`pop_snapshot_stack`

Pops the metrics saved since the last push.

The `pop_snapshot_stack` command works in conjunction with `push_snapshot_stack` to create metric hierarchies. Any metrics captured inside a push/pop must be saved via `create_snapshot` (implicit or explicit) or they will be lost.

Parameters

None

Related Information

- `create_snapshot`
- `push_snapshot_stack`

print_message

```
print_message
-help
[ -id msgID | formatString]
[args]
[-error]
[-warning]
```

Generates a message that is included in the `report_messages` output if the message is designated as an error or a warning.

Parameters

args	Specifies the format argument value to print.
-error	Specifies the message is an error. A prefix will be added.
formatString	Specifies the format specifier like sprintf or Tcl's format command.
-help	Prints a brief description that includes the type and default information for each `print_message` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man print_message`.
-id *msgID*	Specifies the message id.
-warning	Specifies that the message is a warning. A prefix will be added.

Examples

The following commands generate an error and a message, which are included in the `report_messages` output:

```
> print_message -error "A test %s." "from the user"
ERROR: (user): A test from the user.


> print_message -warning "A test %s." "from the user"
WARN: (user): A test from the user.


> print_message "A test %s." "from the user"
A test from the user.


> report_messages
*** Summary of all messages that are not suppressed in this session:
```

```
Severity      ID       Count         Summary
ERROR       no_id       1           A test %s.
ERROR       user        1           A test %s.
WARN        user        1           A test %s.
*** Message Summary: 1 warning(s), 2 error(s)
```

Related Command

- report_messages

program Category Attributes

The root attributes in the program category are:

- cmd_file
- log_file
- logv_file
- program_major_version
- program_name
- program_short_name
- program_version
- selected

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `program` category, use the following command:

```
get_db -category program
```

cmd_file

```
cmd_file fileName
```

Default: ""

Specifies the cmd log file name of the program.

log_file

```
log_file fileName
```

Default: ""

Specifies the log file name of the program.

logv_file

```
logv_file fileName
```

Default: ""

Specifies the verbose log file name of the program.

program_major_version

```
program_major_version
```

Default:

Returns the major version and one digit after the decimal point.

For example, the following command gives the major version of the software:

```
@innovus 1> get_db program_major_version
20.1
```

program_name

```
program_name
```

Default:

Specifies the name of the program.

program_short_name

```
program_short_name
```

Default:

Specifies the short name of the program.

program_version

```
program_version
```

Default:

Specifies the version of the program.

selected

```
selected list
```

Default: ""

Specifies the list of currently selected objects. It can be set to a list of objects as long as they are selectable.

push_snapshot_stack

```
push_snapshot_stack
```

Creates a hierarchy for metric capture. All metrics captured inside the hierarchy is saved at this level during snapshot creation (implicit or explicit). Snapshot hierarchies can be nested so any number of hierarchies are supported.

Note: Each `push_snapshot_stack` must have a corresponding `pop_snapshot_stack`.

Parameters

None

Related Command

- pop_snapshot_stack

rename_obj

```
  rename_obj
[-help]
[-escape_ok]
[-flexible]
object
new_name
[-quiet]
```

Renames the `base_name` of a single design object, which could only be a `inst`, `hinst`, `design`, `module`, `port`, `hport`, or `hnet`. This command changes only the `base_name` of the specified objects, and cannot move those across the netlist hierarchy. This command also returns the new name of the renamed object.

The names used for SDC timing constraints can only be modified using this command, if they have been loaded into the memory. So, ensure that all the timing views are loaded using the `set_analysis_view` command before running the `rename_obj` command so that the SDC constraints are updated properly. The modified SDC files can be retrieved after renaming the objects.

The `rename_obj` command cannot update the names used inside a CPF `power_intent` file. This is because the CPF usage assumes that the CPF file is golden and should not be modified. To change the names used inside a CPF file, first update the CPF file manually, and then use the updated CPF file to run the `rename_obj` command.

An IEEE1801 `power_intent` file is allowed to be changed during the design flow. So, changing names using the `rename_obj` command will automatically update the IEEE1801 object names. You can use the `rename_obj` command to write out the new IEEE1801 file with the updated names.

Parameters

`-help`	Provides a brief description of the command that includes the type and default information for its parameters. For a detailed description of the command and all its parameters, use the man command: `man rename_obj`
`-escape_ok`	Allows `new_name` to have either the hierarchy chars '`/`' or the bus-bit chars '`[]`' in it, which will automatically be escaped. If this parameter is not specified, any set of '`/`' or '`[]`' chars in `new_name` will cause an error and the object will not be renamed. *Data_type*: bool, optional
`-flexible`	If `new_name` coincides with an existing name, this parameter appends a suffix (such as `_1` and `_2`) to differentiate it. If this parameter is not specified, a name collision in such a scenario causes an error, and the specified object is not renamed. *Data_type*: bool, optional

object	Specifies the object to be renamed. The object here must be an `inst`, `hinst`, design, `module`, `port`, `hport`, or `hnet`. If *object* is a `port` or `hport`, any `hnet` connected to it with the same `base_name` is also renamed to keep it matched (for example, renaming `hport:top/i1/clkin` also renames `hnet:top/i1/clkin`). If *object* is an `hnet` that is connected to a `port` or `hport` with the same `base_name`, an error occurs. In such a case, you must change the `port` or `hport` name to proceed. If an `hnet` name matches with its associated `net` name (for example, `get_db $my_hnet .net`), then the net name is also renamed on using the `rename_obj` command. For example, renaming `hnet:top/net1` also renames `net:top/net1`. Therefore, to rename a `net`, you need to rename the `hnet` with the same name. This command does not allow renaming a bus-bit object. So any `port`, `hport`, or `hnet` that has a `.base_name` like `busA[0]` (a bus-bit object), cannot be renamed, While a `.base_name` with a scalar name (like `busA\[0\]`) can be changed. *Data_type*: (`inst | hInst | vCell | topCell | term | hTerm | hNet`), required
new_name	Specifies the new `base_name` for the specified object. *Data_type*: string, optional
`-quiet`	Suppress messages about renamed objects. *Data_type*: string, optional

Examples

- The following command renames an `inst base_name` to `i2_new`:

```
rename_obj inst:top/i1/i2 i2_new
inst:top/i1/i2_new              #returned object with new name
```

- The following command renames an `inst base_name i2\/i3` with escaped hierarchy:

```
rename_obj {inst:top/i1/i2\/i3} i2/i3_new -escape_ok
inst:top/i1/i2\/i3_new          #returned inst object, no hierarchy change
```

- The following command renames a scalar `hport` name to remove escaped bus-bit chars:

```
rename_obj {hport:top/i1/busA\[0\]} busA_0
hport:top/i1/busA_0             #returned hport object
```

  Note: `hnet:top/i1/busA\[0\]` is also renamed to `hnet:top/i1/busA_0`.

- The following command renames a `port` name:

```
rename_obj port:top/clkin clkin_new
port:top/i1/clkin_new           #returned port object
```

  Note: Both `hnet:top/i1/clkin` and `net:top/i1/clkin` are renamed.

rename_snapshot

```
rename_snapshot
[-help]
[-branch newBranch]
[-id metricID]
[-name newName]
-uuid snapshotuuid
```

Allows you to rename existing snapshots.

`rename_snapshot` does not support canonical paths and cannot change the location of a snapshot within the hierarchy.

Parameters

`-branch newBranch`	Specifies the new branch for the snapshot. *Data_type*: string, optional
`-help`	Outputs a brief description that includes type and default information for each `rename_snapshot` parameter. For a detailed description of the command and all of its parameters, use the man command: `man rename_snapshot`
`-id metricID`	Specifies the metric database ID to change. If you do not specify any id, the tool will take current database as default. *Data_type*: string, optional **Note**: You can find the uuid of a snapshot by using `get_metric` for the uuid metric.
`-name newName`	Specifies the new name for the snapshot. The tool will error out if you set the name to empty string. *Data_type*: string, optional
`-uuid snapshotuuid`	Specifies the snapshot UUID. The tool will error out if there is no snapshot with the provided uuid. *Data_type*: string, required

Related Information

- `get_metric`

report_area

```
  report_area
[-help]
[-detail]
[-hierarchical_instance hinst_name]
[-include_physical]
[-min_area area_per_module]
[-min_count instance_count]
[-out_file filename]
[-show_leaf_cells]
[-show_msv_cells]
[-sort_by {name count area}]
[-table_style {horizontal vertical}]
[-summary | -depth depth_of_hierarchy]
```

Reports the combined standard cell area in each of the hierarchy modules and the top-level design. Also reports the cell area based on different cell types if the `-detail` option is specified.

Note: The units used in the report depend on the units used in the library. The area for each module is calculated from all the standard cells within each module using the LEF information.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_area` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_area`
`-depth` *depth_of_hierarchy*	Specifies the depth of hierarchy, all levels if not specified; the top of the hierarchy (design) is 1.
`-detail`	Reports the area in different cell type (buffer, inverter, combinational, flop, latch, clock_gate, macro, physical).
`-hierarchical_instance` *hinst_name*	Reports the area of the leaf instances in the specified hierarchical instance. You can use the `-depth` parameter to control the analysis depth of the specified hierarchical instance.
`-include_physical`	Specifies to include physical cells in the area calculation and report. By default, it reports only logical cells.
`-min_count` *instance_count*	Specifies the minimum instance count per module. The generated output reports the area of hinsts with number of instances greater than the specified `min_count value`. *Default*: `0`

- min_area *area_per_module*	Specifies the minimum area per module. The generated output reports the area of hinsts with area greater than the specified min_area lib_units. *Default*: 0
-out_file *filename*	Specifies the name of the output file.
-show_leaf_cells	Specifies to include the leaf cells at the design level and each hinst level. These are the instances whose parent is the hinst display. *Default*: 0 (Shows the hinst children in the report)
-show_msv_cells	Includes multiple supply voltage cells in the report. **Note:** This parameter is used with -table_style vertical.
-sort_by {name count area}	Specifies the order of hinst names. *Default*: name
-summary	Prints the area summary for the design. It is equivalent "-depth 0".
-table_style {horizontal vertical}	Specifies whether the base report should be expanded in the vertical or horizontal direction. *Default*: horizontal

Examples

- The following command reports the top-level of the hierarchy (design):

```
report_area -depth 1

Hinst Name                    Module Name            Inst Count           Total Area
-------------------------------------------------------------------------------------------------------
----------
dtmf_recvr_core                                      8234                 207975.789
ARB_INST                      arb                    90                   221.578
DATA_SAMPLE_MUX_INST          data_sample_mux        69                   83.790
DIGIT_REG_INST                digit_reg              11                   59.052
```

- The following command reports the cell area based on different cell types along with cell area of the top-level design:

```
report_area -depth 1 -detail

Hinst Name             Module Name      Inst Count   Total Area   Buffer     Inverter
Combinational   Flop        Latch    Clock Gate   Macro      Physical

-------------------------------------------------------------------------------------------------------
-------------------------------------------------------------

dtmf_recvr_core                         8234         07975.789    1151.514   1604.512    81148.322
   4389.532    0.000    0.000        77866.281  0.000

ARB_INST               arb              90           221.578      6.118      12.236      51.870
```

```
    151.354     0.000    0.000        0.000      0.000

DATA_SAMPLE_MUX_INST   data_sample_mux  69           83.790       23.674     18.354      41.762
    0.000       0.000    0.000        0.000      0.000
```

- The following command reports the area of the leaf instances in the specified hierarchical instance, DATA_SAMPLE_MUX_INST:

```
report_area -hierarchical_instance  DATA_SAMPLE_MUX_INST

Hinst Name                    Module Name            Inst Count           Total Area

-------------------------------------------------------------------------------------

DATA_SAMPLE_MUX_INST                                 69                   83.790
```

- The following command reports the physical cells in the area calculation:

```
report_area -include_physical

Hinst Name                   Module Name             Inst Count           Total Area
-------------------------------------------------------------------------------------------
dtmf_recvr_core                                      8234                 207975.789
ARB_INST                     arb                     90                   221.578
DATA_SAMPLE_MUX_INST         data_sample_mux         69                   83.790
DIGIT_REG_INST               digit_reg               11                   59.052
DMA_INST                     dma                     126                  336.756
```

- The following command shows the leaf cells at each hinst level:

```
report_area -show_leaf_cells

Hinst Name                Module Name               Inst Count        Total Area
-----------------------------------------------------------------------------------------
dtmf_recvr_core                                     8234              207975.789
(Leaf Cells)                                        408               117724.285
ARB_INST                  arb                       90                221.578
(Leaf Cells)                                        90                221.578
DATA_SAMPLE_MUX_INST      data_sample_mux           69                83.790
(Leaf Cells)                                        69                83.790
```

- The following command shows the multiple supply voltage cells and prints the area summary for the design.

```
report_area -show_msv_cells -table_style {vertical} -summary

Hinst Name               Module Name            Inst Count           Total Area
-------------------------------------------------------------------------------------------
dtmf_recvr_core                                 8234                 207975.789
```

- The following command shows the minimum area per module along with the multiple supply voltage cells:

```
report_area -min_area 81 -show_leaf_cells
```

Hinst Name	Module Name	Inst Count	Total Area
dtmf_recvr_core		8234	207975.789
(Leaf Cells)		408	117724.285
ARB_INST	arb	90	221.578
(Leaf Cells)		90	221.578
DATA_SAMPLE_MUX_INST	data_sample_mux	69	83.790
(Leaf Cells)		69	83.790
DMA_INST	dma	126	336.756
(Leaf Cells)		126	336.756

report_dangling_ports

```
report_dangling_ports
-help
-partition
-out_file fileName
```

Reports ports that are disconnected from both the inside and outside of a module, and ports that are connected within the module but disconnected outside the module.

Note: If the design has some dangling or unconnected ports, the tool will add `FE_UNCONNECTED_*` nets for these dangling or unconnected ports in the design during optimization. You can report these dangling ports using the `report_dangling_ports` command.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_dangling_ports` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_dangling_ports`
`-out_file`	Specifies the name of the report generated by this command.
`-partition`	The option a partition only flag of the type boolean. This is an optional.

Examples

- The following command writes information about the dangling ports to the `test.rpt` file:

  ```
  report_dangling_ports -out_file test.rpt

  Total number of dangling ports: 125
  ```

- The following command writes information about the dangling ports to the `sample.rpt` file:

  ```
  report_dangling_ports -partition -out_file sample.rpt

  Total number of dangling ports: 0
  ```

report_hidden_usage

```
report_hidden_usage
in_files
```

Reports a summary of all the hidden application Tcl globals and mode options in the specified files. Hidden globals and mode options are normally used for temporary fixes or early access, and often disappear or change in a new release. This command will help to find out the hidden settings used in existing scripts that might not be needed in the new release.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_hidden_usage` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_hidden_usage`
`in_files`	Specifies input files to search for hidden usage.

Examples

- The following command reports the hidden settings in the floorplan.tcl and place.tcl files.

```
>report_hidden_usage {floorplan.tcl cts.tcl} > hidden.rpt

##########################################################################
#  Generated by:    Cadence Tempus 18.10-a072_1
#  OS:              Linux x86_64(Host ID sjfib146)
#  Generated on:    Tue Apr 17 22:47:31 2018
#  Design:          top
#  Command:         report_hidden_usage floorplan.tcl cts.tcl > hidden.rpt
##########################################################################


### floorplan.tcl ###
-------------------------------------------------------------------------------------
Line                      Hidden name                    Command
-------------------------------------------------------------------------------------
```

```
12                          floorplan_xxx                   set_db floorplan_xxx true

32                          init_xxx                        set_db init_xxx 100


### cts.tcl ###

-------------------------------------------------------------------------------------

Line                         Hidden name                    Command

-------------------------------------------------------------------------------------

3                            cts_xxx                        set_db cts_xxx true
```

report_obj Category Attributes

The root attributes in the report_obj category are:

- report_obj_display_limit

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `program` category, use the following command:

```
get_db -category program
```

report_obj_display_limit

`report_obj_display_limit` *`displayLimit`*

Default: `1000`

Data_type: int, read/write

Controls the number of objects that will be displayed when using `report_obj`. If the value is `10`, 'report_obj insts' will only display 10 instance objects with all their non-default attributes.

resume

`resume [-help]`

Resumes a suspended script from the point where it had stopped.

Use the `suspend` and `resume` commands to debug your scripts interactively. The `suspend` command stops a script and restores access to the Tempus prompt. You can then type any command required for debugging. Whenever you want to resume the suspended script, type `resume` at the Tempus prompt.

Parameters

`-help`	Prints the usage of the `resume` command. For a detailed description of the command, use the `man` command: `man resume`.

Related Command

- `suspend`

select_obj

```
select_obj
[-help]
<any_object>+
```

Selects the object or the list of objects for putting into a set. The object type can be any object type selectable on the GUI such as inst and net.

Parameters

`-help`	Prints a brief description that includes type and default information for each `select_obj` parameter. For a detailed description of the command and all of its parameters, use the man command: `man select_obj`
`<any_object>+`	Specifies the list of objects.

Example

```
select_obj [get_db insts i1/*]
```

set_layer_preference

```
set_layer_preference
[-help]
layer_name
[-is_visible {1 | 0}]
[-is_selectable {1 | 0}]
[-color {color_name | color_value}]
[-line_width {lw}]
[-stipple stipple_name | -stipple_data width height "stipple_data"]
```

Sets a new preference for the specified object or layer.

Parameters

`-help`	Prints a brief description that includes type and default information for each `set_layer_preference` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_layer_preference`
`-color`	Specifies a color for the object. Specify either a color name or a numeric RGB value.
`-is_selectable`	Specifies whether the object is selectable. Specify `1` to make the object selectable. Specify `0` to make the object non-selectable.

`-is_visible`	Specifies whether the object is visible. Specify `1` to make the object visible. Specify `0` to make the object invisible.
	Examples
	• You can turn off the visibility of all custom layers by using the following command:
	`set_layer_preference customLayers -is_visible 0`
	• You can turn off display of flightlines by using the following command:
	`set_layer_preference flightLine -is_visible 0`

`layer_name`	Specifies the name of the layer or the object type. The valid layer names are `M#`, where `#` is `0` through `1`, and `M#AVio`, `M#Cont`, `M#ContPin`, `M#ContRB`, `M#Pin`, `M#RB`, `M#RTrk`, and `M#Vio`. The valid object types are `blackBox`, `block`, `blockHalo`, `bump`, `bump1`, `bump2`, `bump3`, `cellBlkgObj`, `channel`, `clkTree`, `congest`, `congestH`, `congestObj`, `congestV`, `datapath`, `fence`, `gcellOvflow`, `guide`, `hinst`, `inst`, `ioSlot`, `ioRow`, `macroSitePattern`, `metalFill`, `mGrid`, `net`, `netRect`, `nonPrefTrackObj`, `obstruct`, `pinObj`, `pinText`, `power`, `powerNet`, `ptnFeed`, `ptnPinBlk`, `pwrdm`, `pwrdm1`, `pwrdm2`, `pwrdm3`, `layerBlk`, `region`, `routeGuide`, `ruler`, `screen`, `select`, `sitePattern`, `stdRow`, `term`, `text`, `trackObj`, `userGrid`, and `violation`.
`-line_width`	Specifies the number of pixels in the border of the specified object. Specify this value as integer.
`-stipple`	Specifies information about the stipple pattern of the object. The valid stipple names are `None`, `Horizontal`, `Vertical`, `Grid`, `Slash`, `Backslash`, `Cross`, and `Brick`.
`-stipple_data`	Specifies the stipple information for the object. Specify this information in x-window bitmap format.

Example

The following command sets a layer preference for the module object in green with dotted stipple pattern (8 by 4 bitmap with dot at every 4 pixels):

```
set_layer_preference hinst -color green -stipple_data 8 4 "0x00 0x11 0x00 0x00"
```

set_log_post_cmd

```
set_log_post_cmd
[-help]
[-disable command_list]
[-enable command_list]
[-report]
```

Turns on/off specific commands post-expansion. It prevents showing lots of `get_db` or set commands and cluttering of the log file.

Note: This command does not affect pre-expansion logging.

Parameters

`-help`	Prints a brief description that includes type and default information for each `set_log_post_cmd` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_log_post_cmd`
`-disable command_list`	Specifies a a list of command names to not log
`-enable command_list`	Specifies a a list of command names to be logged.
`-report`	Writes out each disabled commands to stdout. One command is written per line.

set_proc_verbose

```
set_proc_verbose
[-help]
{[procedure_name [-quiet ]] | [-report ]}
```

Makes a procedure verbose.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_proc_verbose` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_proc_verbose`.
`procedure_name`	Specifies the name of the procedure to be made verbose.
`-quiet`	Changes the procedure back to not being verbose.
`-report`	Reports procedures that are verbose.

Examples

- The following commands sets `myprocedure` as verbose:

  ```
  > set_proc_verbose myprocedure
  myprocedure
  ```

- The following commands reports `myprocedure` as the verbose procedure:

  ```
  > set_proc_verbose -report
  myprocedure
  ```

suspend

```
suspend [-help]
```

Suspends your script and returns to the Tempus prompt.

The `suspend` command gives you more flexibility in managing and debugging your scripts. You can insert multiple `suspend` commands in a script. The `suspend` command can also be used in nested scripts. When the script reaches a point where you have inserted the `suspend` command, it stops and restores access to the Tempus prompt. You can then type any command required for debugging.

Whenever you want to resume your script, just type `resume` at the Tempus prompt.

Parameters

`-help`	Prints the usage of the `suspend` command. For a detailed description of the command, use the `man` command: `man suspend`.

Related Command

- `resume`

system Category Attributes

The root attributes in the system category are:

- auto_file_dir
- auto_file_prefix
- distributed_mmmc_disable_reports_auto_redirection
- gui_verbose
- jedai_catalog_token
- jedai_catalog_url
- log_verbose_type
- read_db_directory
- read_db_file_check
- read_db_stop_at_design_in_memory
- read_db_tool_name
- read_db_version
- script_search_path
- soft_stack_size_limit
- source_echo_filename
- source_continue_on_error
- set_db_verbose
- source_verbose
- source_verbose_line_length_limit
- tcl_error_info
- tcl_report_header_with_value
- ui_precision
- ui_precision_capacitance
- ui_precision_derating
- ui_precision_power
- ui_precision_sensitivities
- ui_precision_timing
- write_db_auto_save_user_globals
- write_db_restrict_under_batch_mode

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `system` category, use the following command:e

```
get_db -category system
```

auto_file_dir

```
auto_file_dir directoryName
```

Default: ./

Specifies the directory to store files/sub-directories 'auto-generated' by the tool. This includes report files or other output files with default names that are automatically created by some commands in the current run directory. Default names that have been overridden with a name chosen by the user, are not affected. This allows multiple simultaneous runs from the same run directory to direct output files to different result directories to avoid collisions.

auto_file_prefix

```
auto_file_dir fileName
```

Default: ""

Specifies a prefix to be applied to all files 'auto-generated' by the tool. This includes report files or other output files with default names that are automatically created by some commands in the current run directory. So a default name like "cmd.report" will become "<prefix>cmd.report". Default names that have been overridden with a name chosen by the user, are not affected. This allows multiple simultaneous runs from the same run directory to direct default output files to different locations to avoid collisions.

distributed_mmmc_disable_reports_auto_redirection

```
distributed_mmmc_disable_reports_auto_redirection {true| false}
```

Default: `false`

Allows reports to be generated in the specified location.

When set to `true`, the software does not allow the reports for different views to get overwritten. When set to `false`, the reports are generated automatically in the DMMMC view directory structure.

gui_verbose

```
gui_verbose {all db none}
```

Default: `db`

Controls which GUI actions are logged. You can select:

- `all`: Logs all GUI actions.
- `db`: Logs the GUI actions that affect the DB, such as `gui_select`, but does not capture display-only actions such

as `gui_dim_foreground`.

- `none`: Does not log any GUI actions. So, the .log and .cmd files are incomplete.

jedai_catalog_token

```
jedai_catalog_token token
```

Default: `""`

Data_type: string

Authenticates and provides access to the OpenAI services.

jedai_catalog_url

```
jedai_catalog_url <jedai_server_url>
```

Default: `""`

Data_type: string

Enables the AI chat box in the Tempus GUI. You need to set the following setting in `tempus_startup.tcl`.

```
#set server url
set_db jedai_catalog_url <jedai_server_url>
```

log_verbose_type

```
log_verbose_type {pre post both}
```

Default: `pre`

When verbose logging is enabled, the default logging of each commands is pre-Tcl expansion. In some cases it is helpful to see post-Tcl expansion, or even both pre-Tcl and post-Tcl expansion when doing detailed debugging. You can switch between settings in the middle of a script.

Related topics

- `set_proc_verbose`

read_db_directory

```
read_db_directory directory_name
```

Default: ""

Specifies the absolute path of the database, that is used to restore the session. In case of a non-restored session, this variable returns a blank value.

The value of this global variable is set internally by the software.

read_db_file_check

```
read_db_file_check {true | false}
```

Default: `true`

By default, `read_db` will check all the files inside the saved DB directory. If any file is edited, renamed, or deleted, the tool will give an error. However, read_db will not perform this check if `read_db_file_check` is set to `false`.

read_db_stop_at_design_in_memory

```
read_db_stop_at_design_in_memory {true | false}
```

Default: `true`

By default, tool gives an error and stops if you call `read_db` more than once in the same session unless `read_db_stop_at_design_in_memory` is set to `false`. Using `read_db` twice in one session is not recommended as settings from the first database may not be cleaned up properly and can cause problems in the second database.

read_db_tool_name

```
read_db_tool_name
```

Default: ""

Indicates the tool name of the tool that created the restored db on the disk database.

read_db_version

```
read_db_version
```

Default: ""

Indicates the version of the tool that generates an on disk database.

script_search_path

```
script_search_path search_directory_path
```

Default: "."

Provides a set of directories for the software to search for files that are sourced using the Tcl `source` command. The software will search in each search path directory for the specified file.

soft_stack_size_limit

```
soft_stack_size_limit size_limit
```

Default: `8`

Specifies the soft stack size limit in mbytes. When the tool is launched, the soft stack size limit is auto-enlarged to 0.2% RAM. If 0.2%RAM is larger than the hard limit, the tool sets it to the hard limit.

source_echo_filename

```
source_echo_filename {true| false}
```

Default: `false`

Displays the file name being sourced before the actual sourcing.

If this is true, and verbose 'source' command logging is off (e.g. source_verbose is false), any 'source' command will be logged with 'Sourcing <filename>'. This can be useful to find which source files are calling other source files, or to log all the startup Tcl files that are loaded even though verbose logging is off.

source_continue_on_error

```
source_continue_on_error  {true| false}
```

Default: `false`

Enables or disables `source` to continue the script on TCL_ERROR. If `true`, the `source` command will ignore Tcl errors, and continue processing the script files. If `false`, a Tcl error will halt the script.

set_db_verbose

```
set_db_verbose {true| false}
```

Default: `false`

Specifies additional messages are issued describing what attribute is being set on what object.

source_verbose

```
source_verbose {true| false}
```

Default: `true`

Data_type: bool

Controls whether scripts are sourced in quiet or verbose mode:

- By default (`true`), files are sourced in verbose mode. This is equivalent to `source` *`filename`* `-verbose`.
- If you set this variable to `false`, files are sourced in quiet mode.

source_verbose_line_length_limit

```
source_verbose_line_length_limit value
```

Default: `-1`

Data_type: `int`

Limits the command printed to stdout and the .log file to the specified number of characters for verbose script source. This is normally only needed if you log commands "post Tcl expansion" when long Tcl lists may occur in the command arguments.

tcl_error_info

```
tcl_error_info {true| false}
```

Default: `false`

Data_type: bool, read/write

When set to `true`, the `tcl_error_info` prints the error information if there is an issue in the Tcl script.

tcl_report_header_with_value

```
tcl_report_header_with_value {true| false}
```

Default: `false`

Data_type: boolean, read/write

When set to `true`, prints the Tcl variable value in the report header.

Example

In the following example, `tcl_report_header_with_value` is set to `true`, the report shows the value, `myct`, set for the variable in the report header.

```
[DEV]tempus 8> report_analysis_coverage -check_type   \
+      $myct

#################################################################
#  Generated by:      Cadence Innovus 24.10-dev.jinchuan
#  OS:                Linux x86_64(Host ID sjfib902)
#  Generated on:      Wed Jan 17 23:22:11 2024
#  Design:            DTMF_CHIP
#  Command:           report_analysis_coverage -check_type   \
#    $myct
#################################################################
    +-----------------------------------------------------+
    |           TIMING CHECK COVERAGE SUMMARY             |
    |-----------------------------------------------------|
    | Check | No. of |    Met     | Violated |  Untested  |
    |  Type | Checks |            |          |            |
    |-------+--------+------------+----------+------------|
    | Setup |  1821  | 219 (12%)  |  0 (0%)  | 1602 (87%) |
    +-----------------------------------------------------+
```

If the `tcl_report_header_with_value` is set to `false`, the report will show the name of the variable in the report and not its value.

```
[DEV]tempus 9> report_analysis_coverage -check_type   \
+      $myct
#################################################################
#  Generated by:      Cadence Innovus 24.10-dev.jinchuan
#  OS:                Linux x86_64(Host ID sjfib902)
#  Generated on:      Wed Jan 17 23:22:20 2024
#  Design:            DTMF_CHIP
```

```
#  Command:           report_analysis_coverage -check_type   \
#     setup
#############################################################
```

ui_precision

`ui_precision` *number*

Default: 3

Specifies the number of significant digits to be displayed in the absence of an explicit precision for that type.

ui_precision_capacitance

`ui_precision_capacitance` *number*

Default: 3

Specifies the number of significant digits to be displayed for quantities of type capacitance.

ui_precision_derating

`ui_precision_derating` *number*

Default: 3

Specifies the number of significant digits to be displayed for derating factors such as those specified via set_timing_derate constraints or AOCV derating libraries.

ui_precision_power

`ui_precision_power` *number*

Default: 3

Specifies the number of significant digits to be displayed for quantities of type power.

ui_precision_sensitivities

`ui_precision_sensitivities` *number*

Default: 3

Specifies the number of significant digits to be displayed for statistical sensitivity values and SOCV sigma values.

ui_precision_timing

`ui_precision_timing` *number*

Default: 3

Specifies the number of significant digits to be displayed for delay type values including cell and net delays, transitions,

arrival and required times, and slacks.

write_db_auto_save_user_globals

```
write_db_auto_save_user_globals {true| false}
```

Default: `true`

If `true`, `write_db` automatically saves the Tcl global variables created by the user. User globals are defined as any “info globals” names that do not exist at startup (such as created by the user executed code).

You can use the `define_variables` command to add or delete specific Tcl variables from the automatic save list.

write_db_restrict_under_batch_mode

```
write_db_restrict_under_batch_mode {0| 1}
```

Default: `0`

Data_type: bool, read/write

Controls whether to save a database under the batch mode. By default, a new database is saved under the batch mode. However, if `write_db_restrict_under_batch_mode` is set to `true`, the tool gives an error.

time_info

```
time_info
[-help]
[stage_name]
[-current]
[-report]
[-stamp]
[-table metric_table_id]
```

Reports and tracks runtime and memory performance.

When the `time_info` command is used without any name, an entry for the stage is created in the table named as "undefined". When the command is used multiple times without any stage name, the stage name is incremented, for example undefined1, undefined2, undefined3. This will ensure that each stage name in the table is unique.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `time_info` parameter. For a detailed description of the command and all of its parameters, use the man command: `man time_info`.
`stage_name`	Specifies the name of the stage.
`-current`	Reports the current stage.
`-report`	Reports all stages up to this point
`-stamp`	Creates a time stamp.
`-table` *`metric_table_id`*	Specifies metrics table ID to support multiple tables. *Default*: 'default'

Examples

- The following command shows the usage of the `-current` parameter:

```
tempus > time_info -current
=======================================================
CPU Runtime: 0:00:52.5
Wall Runtime: 0:03:41
Memory Usage: 1387.4 MBytes
Peak Memory Usage: 1505.3 MBytes
Current Time: 12:17:13 (Jul07)
=======================================================
```

- The following command shows the suage of -report:

```
tempus > time_info -report
Total Wall | Stage Wall | Wall % | Date - Time     | Memory   | Stage
----------------+----------------+------------+-------------------+------------+----------------------
-
0:00:00    | 0:00:00    | 0.0%   | 8:27:08 (Jul07) | 824.3 MB | init
----------------+----------------+------------+-------------------+------------+----------------------
-
Number of threads: 1 * 1 (id: default)
```

uniquify_filename

```
uniquify_filename
[-help]
filename
```

Uniquifies a file.

Parameters

filename	Specifies the name of the file to uniquify.
-help	Prints a brief description that includes type and default information for each `uniquify_filename` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man uniquify_filename`

Utility Commands

- check_netlist
- convert_legacy_to_common_ui
- create_metric_page
- create_metric_table
- create_metric_table_heading
- create_snapshot
- define_metric
- delete_metric
- delete_metric_page
- delete_metric_run
- delete_metric_table
- delete_metric_table_heading
- get_all_layers
- get_metric
- get_metric_alias
- get_metric_config
- get_metric_definition
- gui_bind_key
- gui_move_cursor
- metric Category Attributes
- puts
- read_metric
- read_metric_config
- redirect
- report_disk
- report_metric
- report_netlist_statistics
- report_obj
- reset_metric_config
- reset_metric_run_id
- run_metric_category

- set_compress_level
- set_license_check
- set_limited_access_feature
- set_metric
- set_metric_alias
- set_metric_header
- set_preference
- source
- update_inst_name
- update_metric
- update_metric_definition
- view_log
- write_metric
- write_metric_config
- read_metric_html_package
- get_metric_header
- get_preference
- delete_metric_header

check_netlist

```
check_netlist
-out_file filename
[-sub_module_ports]
```

Performs several checks on the design netlist and creates a report, named `checknetlist.rpt`. This report shows the following information:

- Design Summary
- Design Statistics
- I/O Pad Summary
- Design Rule Checking
- Submodule Port Definition Checking (optional)

The relevant information can be found in the related checklist directory.

You can use this command after the design netlist is imported.

Parameters

`-out_file filename`	Specifies the name of the file where the report will be written. If a name is not specified, the default filename, `checknetlist.rpt`, is used.
`-sub_module_ports`	If this option is included, `check_netlist` checks the port definitions for any submodules.

Example

- The error conditions are indicated by an asterisk (*) preceding the reported statistic, such as:

```
Number of Ports Connect to Core Cells *: 57
Number of Ports Connect to multiple Pads *: 0
Number of Floating Ports *: 0
```

- The warnings, or possible errors are indicated by a question mark in front of the statistic, such as:

```
Number of undefined Floating Pins ?: 0
```

- The following example shows the information contained in the generated by `check_netlist` report:

```
Design: TOPModule
------ Design Summary:
Total Standard Cell Number (cells) : 5908
Total Block Cell Number (cells) : 4
Total I/O Pad Cell Number (cells) : 0
Total Standard Cell Area ( um^2) : 133864.32
```

```
Total Block Cell Area ( um^2) : 366346.56
Total I/O Pad Cell Area ( um^2) : 0.00

------ Design Statistics:
Number of Instatnces : 5912
Number of Nets : 6217
Average number of Pins per Net : 3.69
Maximum number of Pins in Net : 541

------ I/O Pad summary
Number of Primary I/O Ports : 57
Number of Input Ports : 28
Number of Output Ports : 29
Number of Bidirectional Ports : 0
Number of Power/Ground Ports : 0
Number of Ports Connect to Core Cells *: 57
Number of Ports Connect to multiple Pads *: 0
Number of Floating Ports *: 0

------ Design Rule Checking:
Number of Output Pins connect to Power/Ground *: 0
Number of Input Floating Pins *: 1
Number of Output Floating Pins : 245
Number of InOut Floating Pins : 0
Number of UniPort Floating Pins : 0
Number of undefined Floating Pins ?: 0
Number of Multiple Driving Nets *: 1
Number of Parallel Driving Nets : 1
Number of Tristate Driving Nets : 0
Number of Input Floating Nets(No FanIn) *: 0
Number of Output Floating Nets(No FanOut) : 1
Number of Hign Fanout Nets (>50) : 3

------ Sub Module Port Definition Checking
```

```
* module arb has 3 port define error

* module accum_stat has 1 port define error

* module tdsp_core_glue has 128 port define error
```

convert_legacy_to_common_ui

```
convert_legacy_to_common_ui
[-help]
in_dir_or_file
out_dir_or_file
```

Allows usage of legacy scripts to be used in Stylus Common UI.

Parameters

`-help`	Prints out the command usage.
in_dir_or_file	Specifies the name of the legacy script or script directory name.
out_dir_or_file	Specifies the Stylus Common UI script or the script directory name.

create_metric_page

```
create_metric_page
[-help]
[-after page_name]
[-before page_name]
[-format {html excel text}]
-name page_name
```

Creates a new metric page.

Parameters

`-after` *page_name*	Specifies to insert after the page with this name.
`-before` *page_name*	Specifies to insert before the page with this name.
`-format {html excel text}`	Specifies configuration for the `report_metric` format.
`-help`	Prints out the command usage. For a detailed description of the command, use the man command: `man create_metric_page`
`-name` *page_name*	Specifies the name of the new page. The page name must be unique.

Related Command

- `report_metric`

create_metric_table

```
create_metric_table
[-help]
[-auto_hide]
[-collapsible_key]
[-format {html excel text}]
-heading table_heading
[-hide_footers]
[-hide_graph_footers]
[-hierarchy_separator separator]
[-id table_id]
[-key regexp_key]
[-per_snapshot]
[-show_label_every frequency]
-type {horizontal vertical cpu dict time_statement histogram imageplot}
```

Creates a metric table.

Parameters

-auto_hide	Specifies whether to auto hide the table.
-collapsible_key	Specifies to include collapse key columns in the table with the option to expand them.
-format {html excel text}	Specifies configuration for the report_metric format.
-heading *heading_id*	Specifies the id of the heading to append.
-hide_footers	Hides the minimum, average, and maximum rows from the bottom of the table. Graph buttons are still shown.
-hide_graph_footers	Hides the graph buttons from the bottom of the table.
-hierarchy_separator *separator*	Shows the metric names with hierarchy using the provided separator.
-id *table_id*	Specifies the id for the new table.
-key *regexp_key*	Specifies the key for any regexp.
-per_snapshot	Specifies to expand the table per snapshot.
-show_label_every *frequency*	Specifies the frequency of X-axis labels in the histogram.
-type {horizontal vertical cpu dict time_statement histogram imageplot}	Specifies the type (horizontal vertical cpu dict time_statement histogram imageplot) of table

Related Topics

- report_metric command

create_metric_table_heading

```
create_metric_table_heading
[-help]
[-format {html excel text}]
[-id heading_id]
-page page_name
-title heading_title
[-type {fixed collapse}]
```

Create a new metric heading.

Parameters

-help	Prints out the command usage. For a detailed description of the command, use the man command: `man` create_metric_table_heading
`-format {html excel text}`	Specifies configuration for the `report_metric` format.
`-id heading_id`	Specifies the id for the new heading.
`-page page_name`	Specifies the page name where the heading will be added.
`-title heading_title`	Specifies the title of the heading.
`-type {fixed collapse}`	Specifies the type of heading element, fixed or collapse.

Related Topics

- `report_metric` command

create_snapshot

```
create_snapshot
-help
[pattern]
[-auto {min default}]
[-categories [* flow design setup power check clock hold route]]
[-exclude_time_metric]
-name metric_name
```

Creates snapshots of the current metric list at various points in the flow.

All super commands that save metrics call `create_snapshot` to save all metrics in a consistent manner. When `create_snapshot` is called, it creates an internal record designated by a label. The label for super commands are the command names.

Metrics that do not require extraction and/or timing/power analysis are calculated when the metric snapshot is saved. Timing/clock/power metrics are reported only if they are up-to-date(i.e. if `time_design`/`report_ccopt*`/`report_power` have been run). Metrics are saved as part of db and can be accessed using `get_db`.

Note: Metric values should be saved such that the unit is easily known/accessible.

Parameters

`-auto {min default}`	Specifies the set of auto metrics to run.
`-categories [* flow design setup power check clock hold route]`	Specifies the metric categories to compute.
`-exclude_time_metric`	Excludes CPU and wall time from parent snapshot calculations.
`-name` *metric_name*	Specifies the name of the metric.
`-help`	Outputs a brief description that includes type and default information for each `set_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_metric`
pattern	Specifies specific metrics to save.

Example

The following code shows the manual usage of `create_snapshot`:

```
# ------------------------------------------------------------------
# Manual Usage:
# ------------------------------------------------------------------

opt_design -preCTS
report_power ...
create_snapshot -label prects
ccopt_design
```

```
report_power
create_snapshot -label cts
add_filler
route_design
create_snapshot -label route ; # Timing and power are not up-to-date
opt_design -postRoute
report_power
create_snapshot -label postroute
```

The following code shows the automatic/mixed usage of `create_snapshot`:

```
# ------------------------------------------------------------------
# Automatic / Mixed Usage:
# ------------------------------------------------------------------

setDesignMode -enableMetrics true
opt_design -preCTS ; # Saves snapshot "opt_design_<count>"
report_power
create_snapshot -name "prects"
ccopt_design ; # Saves snapshot "ccopt_design"
report_power
create_snapshot -name "postcts" ; # Includes up-to-date power metrics
add_filler
route_design ; # Saves snapshot "route_design"
opt_design -postRoute ; # Saves snapshot "opt_design_<count>"
report_power
create_snapshot -name "postroute" ; # Includes up-to-date power metrics
```

http://rdwiki.cadence.com/display/ssvpubsmgmt/Parameters+not+Documented+-+16.20

define_metric

```
define_metric
[-help]
[-compare compareFunction]
[-description metricDescription]
[{-id databaseID}]
[-inheritance_tcl TclName]
-name metric_name
[-no_inheritance]
[-precision number_of_decimal_places]
[-target metric_name]
[-tcl TCL_command_list [-tools listOfTools]]
[-threshold reporting_unit]
[-type metricType]
[-units reporting_unit]
[-warning threshold]
```

Defines new metrics.

You can use this command to create new metrics that you need to track. To create new metrics, you need to provide the TCL code required to generate the metric. The TCL can be a sequence of commands that return a value or a procedure that returns a value.

The following command provides the TCL commands `dbFindInstsByCell HDR*` to define the design.switch_cells metric.

```
define_metric -name design.switch_cells -tcl { return [dbFindInstsByCell HDR*] }
```

Similarly, a proc can be used to define a metric. The following command provides the `my_get_switch_cells{}` proc to define the `design.switch_cells` metric.

```
define_metric -name design.switch_cells -tcl { return [my_get_switch_cells{}] }
```

The metric is calculated and stored with the other metrics when `create_snapshot` is executed. All `define_metric` commands are saved and restored in the design database.

Parameters

`-compare compareFunction`	Specifies the compare function, `[moreBetter|lessBetter|moreBetterBelowTarget|lessBetterAboveTarget]`. The comparison options show the good or bad color when the value is below or above the target value as appropriate. *Unit*: `string`
`-description metricDescription`	Specifies the description of the metric. *Unit*: `string`
`-id databaseID`	Specifies the database ID.

`-inheritance_tcl` *`TclName`*	Specifies Tcl for combining children values. The metric is not executed in the global namespace. The equivalent proc args is {children}. *Unit*: `string`
`-name` *`metric_name`*	Specifies the name of the metric. *Unit*: `string`
`-no_inheritance`	Disables the inheritance of the metric. *Unit*: `boolean`
`-help`	Outputs a brief description that includes type and default information for each `define_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man define_metric`
`-precision` *`number_of_decimal_places`*	Specifies the number of decimal places. *Unit*: `integer`
`-target` *`metric_name`*	Specifies the target or the name of metric containing the target value.
`-tcl {`*`TCL_command_list`*` }`	Specifies the list of TCL commands or proc to generate (return) the metric which is of the format `{<value> <unit>}`. *Unit*: `string`
`-threshold` *`reporting_unit`*	Specifies the threshold for comparing numbers in HTML. If the delta between the two numbers being compared is less than the threshold then the metric will be colored “yellow”. If it is outside the threshold, then it is colored red or green depending on the compare function i.e. whether more is better or less is better.
`-tools` *`listOfTools`*	Specifies the list of tools that the tcl code will run.

`-type` *`metricType`*	Specifies the type of metric.
	Unit: `string`
	The various metrics types are:
	• imageplot: A metric type that the renders to an image in the HTML report.
	• Format:
	`<name1> { \`
	` fill_color "<color>" \`
	`  stroke_color "<color>"\`
	`  data <list of data sets>`
	`} \`
	`<name2> { \`

```
<sub-name1> { \
    fill_color <color> \
    data <list of data sets> \
  } \
  <sub-name2> {...} \
}
```

Where a data set can be one or more of the following:

The data on each layer is a string of coordinates (separated by sequences of whitespace and comma) with types. These types are:

- R - rectangle (x, y, width, height)
- B - bounding box (x_min, y_min, x_max, y_max)
- G - closed polygon (x, y, x, y, ...)
- L - open polyline (x, y, x, y, ...)
- C - circle (x, y, r)
- P - point (x, y)

For rectangles, boxes, circles and points, multiple shapes can be defined without having to repeat the type letter. e.g. R1,2,3,4,5,6,7,8 is two rectangles.

A value can be defined for shapes by adding a V suffix to the type; this value can then be used to color the shapes according to the value for heat maps. The value is always the first number after the type. This means that two points with values 20 and 40 may look like: PV20,1,2 40,6,5 or: PV20,1,2PV40,6,5.

Example imageplot TCL dict data:

```
macros { \
fill_color "#FF0000" \
  stroke_color "#0000FF88" \
  data "R5,8,1,2 6,9,2,1 G1,2,3,4,5,6,7,8" \
} \
drcs { \
placement { \
    fill_color "#FF0000" \
    data "P3,4" \
  } \
  routing {...} \
}
```

Layers in the image are named (macros, congestion, normal, other in the above example) and support hierarchy. Layers should not be nested beyond one level deep. The hierarchy is used in the renderer (see below) to toggle the visibility of the layers. It is an error for any layer to be called data (reserved to keep hierarchy clear).

Colors can be defined either as hex or RGB values with optional opacity. Hex values are prefixed with #. RGB values are separated with whitespace.

- #FF0000: Red
- #00FF0040: Green (25% opaque)
- 0 0 255: Blue
- 255 0 255 192: Magenta (75% opaque)

- histogram: The data will be structured as a set of series with shared X-axis labels that are rendered as a stacked histogram.

  Format (TCL dict):

```
labels <list of labels> \
axis_x <name> \
axis_y <name> \
series { \
<series1> { \
   values <list of buckets \
   color "#FF0000" \
} \
<series2> { \
   values {2 1 0 0} \
   color "#0000FF" \
} \
}
```

  Example:

```
get_metric timing.setup.histogram
timing.setup.histogram {labels {-0.320 -0.240 -0.160 -0.080} \
                        series { \
                          reg2reg {color #FF0000 values {43 3 2
3 0}} \
                          reg2cgate {color #0000FF values {22 4
5 0 0}} \
                        } \
```

	axis_x WNS(ns) \ axis_y {Failing End Points} \ } The labels (a list of strings, shown in the order provided) define the X-axis labels for the histogram, the values in each series define the Y-axis values. Colors can be defined either as hex or RGB values with optional opacity. Hex values are prefixed with #. RGB values are separated with whitespace. #FF0000 - Red 0 0 255 - Blue • html: Supports the inclusion of raw HTML in a metric. You can place simple html into tables allowing simple links to reports. For example, the following command defines test_html as an html type of metric: define_metric -name test_html -type html You can the html metric, test_html, using the set_metric command: set_metric -name test_html -value {test}
-units *reporting_unit*	Specifies the unit for reporting. *Unit*: string
-warning *threshold*	Specifies the warning threshold or name of metric containing the threshold value.

Examples

The following command defines the design.vtmix metric :

```
define_metric -name design.vtmix -visibility default -tcl {return [TCL commands]}
```

Related Topics

- create_snapshot command

delete_metric

```
delete_metric
-help
[{-id databaseID}]
[-name metric_name]
```

Deletes a previously defined (user) metric from all labels.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_metric`
`-id databaseID`	Specifies the database ID.
`-name metric_name`	Name of the metric to delete.

Examples

- The following command deletes the `design.vtmix` metric:

```
delete_metric -name design.vtmix
```

delete_metric_page

```
delete_metric_page
[-help]
[-format {html excel text}]
-name page_name
```

Deletes a metric page.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the man command: `man delete_metric_page`
`-format {html excel text}`	Specifies configuration for the `report_metric` format.
`-name page_name`	Specifies the name of the metric page to be deleted.

Related Topics

- `report_metric` command

delete_metric_run

```
delete_metric_run
-help
[-name metric_name]
```

Deletes the specified metric run.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_metric_run` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_metric_run`
`-name` *`metric_name`*	Name of the metric run to delete.

delete_metric_table

```
delete_metric_table
[-help]
[-format {html excel text}]
[-id table_id]
```

Deletes a metric table.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the man command: `man delete_metric_table`
`-format {html excel text}`	Specifies configuration for the `report_metric` format.
`-id table_id`	Specifies the id for the table to be deleted.

Related Topics

- `report_metric` command

delete_metric_table_heading

```
delete_metric_table_heading
[-help]
[-format {html excel text}]
```
[-id *heading_id*]

Deletes a metric heading.

Parameters

-help	Prints out the command usage. For a detailed description of the command, use the man command: `man delete_metric_table_heading`
`-format {html excel text}`	Specifies configuration for the specified `report_metric` format.
`-id` *`heading_id`*	Specifies the id of the heading to be deleted.

Related Topics

- `report_metric` command

get_all_layers

```
get_all_layers
[-help]
[type]
```

Returns a complete list of all layers and floorplan object settings. If you specify type, the software returns all the layers of the specified type. This command can be used at any stage in the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `get_all_layers` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_all_layers`
type	Specifies the type of the layer. Tempus supports six types of layers, which can be specified as follows: • `object`: If you specify type as `object`, the software returns all object layers, which represent db objects, such as instances, modules, pins, and so on. • `display`: If you specify type as `display`, the software returns display-only or view-only layers, including flightlines, rulers, and congestion. • `multi`: If you specify type as `multi`, the software returns multiple color layers, such as congestion, maps, and yield map. • `metal`: If you specify type as `metal`, the software returns wire/via layers, including metal/via, pin, and blockage. • `custom`: If you specify type as `custom`, the software returns custom layers, which are used to represent custom objects and GDSII data. • `internal`: If you specify type as `internal`, the software returns all internal layers.

Example

Returns all metal layer names:

```
get_all_layers metal
```

get_metric

```
get_metric
-help
[-branch list_of_patterns]
[-exclude_inherited]
[-id id_name]
[-names label_name]
pattern_name
[-pending]
[-pending_top]
[-raw]
```

Gets the current state of metric(s).

You can use this command to query the current state of any metric or metrics from the current specified list of metrics as well as a previous state.

Parameters

`-branch list_of_patterns`	Specifies list of patterns to match snapshot branches. This will limit all output to metrics stored in snapshots whose branch matches the specified branch. This requires any inheritance to be re-run to exclude inherited values from non-matching snapshots.
`-help`	Outputs a brief description that includes type and default information for each `get_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_metric`
`-exclude_inherited`	Specifies not to return any inherited value.
`-id id_name`	Name of id (if read_metric was used). Default is current database.
`-names label_name`	Name of label to get metrics from (wildcards supported); no label reports current metrics
`pattern_name`	Metric name (wildcards supported); * gets all metrics
`-pending`	Returns metrics from the pending (not yet created) snapshot.
`-pending_top`	Returns metrics from the entire pending (not yet created) snapshot stack.
`-raw`	Returns metrics as raw value (without normalization to defined units).

Examples

- The following command gets all metrics (current behavior):

```
get_metric *
```

- The following command gets all timing metrics associated with the postcts snapshot:

```
get_metric -label postcts timing.*
```

Related Commands

- report_metric
- set_metric

get_metric_alias

```
get_metric_alias
[-help]
-from old_name
[-no_overwrite]
[-order order_of_priority]
-to new_name
```

Returns a tcl dictionary of aliases as dictionaries along with the from, to, and order properties.

Parameters

None

Example

- The following command queries the set of aliases:

  ```
  > get_metric_alias
  ```

  Output

  ```
  >path_group:reg2reg {to reg2reg order 1} path_group:reg2cgate {to reg2cgate order 2}
  ```

Related Commands

- set_metric_alias

get_metric_config

```
get_metric_config
-help
-format {html excel text}
```

Gets the current metric configuration setting.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_metric_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_metric_config`
`-format {html excel text}`	Specifies configuration for the specified `report_metric` format.

Related Topics

- `report_metric`
- `write_metric_config`

get_metric_definition

```
get_metric_definition
-help
-name metric_name
```

Returns the metric definition of the specified metric.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_metric_definition` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_metric_definition`
`-id metricid`	Specifies the database ID to output.
`-name label_name`	Specifies the metric name.

gui_bind_key

```
gui_bind_key
[-help]
key
-cmd cmd
[-is_public]
```

Enables you to create keyboard shortcuts.

You can use this command at any time after starting Tempus.

Parameters

`-help`	Prints a brief description that includes type and default information for each gui_bind_key parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man` gui_bind_key
key	Specifies the key to be used as the shortcut. You can specify a: • Single shortcut key-Specify any one of the following keys: ◦ `A-Z` ◦ Function keys (`F1-F12`) ◦ Arrow keys ◦ Number keys ◦ Special character keys, such as `@` and `#` ◦ `Space`, `Delete`, and `Esc` keys • Key combination-If you want to use a key combination, specify `Shift`, `Ctrl`, or `Alt` as the first key and any one of the keys allowed as a single shortcut key as the second key in the combination. Examples: ◦ `Alt+G` ◦ `Ctrl+B`
`-cmd` *cmd*	Specifies the command to be executed when key is pressed.
-is_public	Specifies the bindkey as user defined.

Example

Runs the `gui_fit` command when you press `f` on your keyboard:

```
gui_bind_key f "gui_fit"
```

gui_bind_key

gui_move_cursor

```
gui_move_cursor
[-help]
x y
```

Places the cursor at the specified location in the main window.

You can specify this command at any time after starting the Tempus software.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_move_cursor` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_move_cursor`
x	Specifies the x coordinate of the cursor location.
y	Specifies the y coordinate of the cursor location.

Example

The following command places the cursor at the coordinates (`133`, `277`).

```
gui_move_cursor 133 277
```

metric Category Attributes

The root attributes in the metric category are:

- metric_advanced_url_endpoint
- metric_capture_3d_hotspots
- metric_capture_depth
- metric_capture_design_image
- metric_capture_design_image_blockages
- metric_capture_design_image_blockages_exclude_layers
- metric_capture_design_image_blockages_threshold
- metric_capture_design_image_power_intent
- metric_capture_design_image_route_drc
- metric_capture_max_drc_markers
- metric_capture_min_count
- metric_capture_overwrite
- metric_capture_pba_tns_histogram
- metric_capture_reg2reg_metrics
- metric_capture_timing_analysis_mode
- metric_capture_tns_histogram
- metric_capture_tns_histogram_buckets
- metric_capture_tns_histogram_max_slack
- metric_capture_tns_histogram_paths
- metric_capture_timing_paths
- metric_capture_timing_path_groups
- metric_capture_per_view
- metric_capture_vth_metrics
- metric_capture_vth_per_power_domain
- metric_category_default
- metric_capture_wns_average_paths
- metric_current_run_id
- metric_enable
- metric_summary_metrics

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `metric` category, use the following command:

```
get_db -category metric
```

metric_advanced_url_endpoint

```
metric_advanced_url_endpoint base_url
```

Default: `""`

Specifies the base URL for the advanced metric server.

metric_capture_3d_hotspots

```
metric_capture_3d_hotspots {true | false}
```

Default: `true`

Data_type: boolean, read/write

Specifies the depth for capturing hinst design and power metrics.

metric_capture_depth

```
metric_capture_depth integer
```

Default: 1

Specifies the depth for capturing hinst design and power metrics.

metric_capture_design_image

```
metric_capture_design_image {true | false}
```

Default: `true`

Captures design image.

metric_capture_design_image_blockages

```
metric_capture_design_image_blockages {true | false}
```

Default: `true`

Data_type: string, read/write

Captures blockages in the design image.

metric_capture_design_image_blockages_exclude_layers

```
metric_capture_design_image_blockages_exclude_layers layer_names
```

Default: `""`

Data_type: string, read/write

Specifies a list of layers to exclude when capturing the blockage image metric (design.blockage.image). For example, it can be useful for skipping blockages related to metal fill.

metric_capture_design_image_blockages_threshold

```
metric_capture_design_image_blockages_threshold threshholdValue
```

Default: `100`

Data_type: integer, read/write

Threshold for capturing blockages in metric image.

metric_capture_design_image_power_intent

```
metric_capture_design_image_power_intent {true | false}
```

Default: `true`

Data_type: string, read/write

Captures power intent objects in the design image.

metric_capture_design_image_route_drc

```
metric_capture_design_image_route_drc {true | false}
```

Default: `true`

Data_type: string, read/write

Captures route DRC markers in the design image.

metric_capture_max_drc_markers

```
metric_capture_max_drc_markers max_number
```

Default: `1000`

Specifies the maximum number of DRC markers to include in metric image.

metric_capture_min_count

```
metric_capture_min_count integer
```

Default: `1000`

Specifies the minimum instance count for capturing hinst design and power metrics.

metric_capture_overwrite

```
metric_capture_overwrite {true | false}
```

Default: `false`

Overwrite pending metrics during `create_snapshot` category capture.

metric_capture_pba_tns_histogram

```
metric_capture_pba_tns_histogram {true | false}
```

Default: `false`

Captures the PBA histogram data for TNS.

metric_capture_reg2reg_metrics

```
metric_capture_reg2reg_metrics [true | false}
```

Default: `false`

Data_type: bool, read/write

Forces the capture of reg2reg metrics regardless of path groups.

metric_capture_timing_analysis_mode

```
metric_capture_timing_analysis_mode {gba | ipba}
```

Default: `gba`

Data_type: enum, read only

Controls the timing analysis mode for metric capture during create_snapshot categories setup and hold; gba, ipba.

- `gba`: graph based analysis
- `ipba`: path based analysis, retime_method == path_slew_propagation, retime_mode == exhaustive, timing_path_based_exhaustive_pba_bounded_mode == true

metric_capture_tns_histogram

```
metric_capture_tns_histogram {true | false}
```

Default: `true`

Captures the histogram data for TNS.

metric_capture_tns_histogram_buckets

```
metric_capture_tns_histogram_buckets integer
```

Default: `50`

Specifies the number of buckets for the TNS histogram.

metric_capture_tns_histogram_max_slack

`metric_capture_tns_histogram_max_slack` *`maximum_slack_value`*

Default: `0`

Specifies the maximum slack value for the TNS histogram. You can change its value if you need to shift the y axis.

metric_capture_tns_histogram_paths

`metric_capture_tns_histogram_paths` *`integer`*

Default: `10000`

Specifies the number of paths to capture for the TNS histogram.

metric_capture_timing_paths

`metric_capture_timing_paths` *`integer`*

Default: `10`

Specifies the number of paths to capture for detailed display.

metric_capture_timing_path_groups

`metric_capture_timing_path_groups` *`path_group`*

Default: `""`

Data_type: string

Controls the path groups for which detailed timing path information (full timing path report and display) is to be captured . By default, it is empty and the tool captures all and reg2reg. If the global is set to a list of groups, the tool will use the listed groups. The related metric name is `timing.setup.paths.path_group:<group>`.

metric_capture_per_view

`metric_capture_per_view {true | false}`

Default: `true`

Captures timing metrics per analysis_view.

metric_capture_vth_metrics

`metric_capture_vth_metrics {true | false}`

Default: `true`

Data_type: bool, read/write

Captures vth metrics.

metric_capture_vth_per_power_domain

```
metric_capture_vth_per_power_domain {true | false}
```

Default: `false`

Captures vth metrics per power domain.

metric_category_default

```
metric_category_default category
```

Default: `design`

Specifies to capture the default metric categories if none are provided.

metric_capture_wns_average_paths

```
metric_capture_wns_average_paths paths
```

Default: `""`

Data_type: string, read/write

Specifies a list of integers defining the number of paths to use for average wns metric capture.

Example

The following command:

```
set_db metric_capture_wns_average_paths "100 300 500"
```

Captures the following metrics (in part):

```
timing.setup.wns.avgpath:100.path_group:reg2reg
timing.setup.wns.avgpath:300.path_group:reg2reg
timing.setup.wns.avgpath:300.path_group:reg2reg
```

metric_current_run_id

```
metric_current_run_id run_id
```

Default: ""

Specifies the current unique run ID returned by the advanced metric server.

metric_enable

```
metric_enable {true | false}
```

Default: `false`

Specifies the current unique run ID returned by the advanced metric server.

metric_summary_metrics

```
metric_summary_metrics {flow.cputime flow.realtime timing.setup.tns timing.setup.wns}
```

Default: `flow.cputime flow.realtime timing.setup.tns timing.setup.wns`

Specifies summary metrics to be inherited when viewing snapshots and runs.

puts

```
puts
[-help]
string
```

Outputs information to the screen (stdout) and Tempus log files (.log/.logv).

read_metric

```
read_metric
[-help]
-id snapshotName
metricsFile
[-merge]
[-previous UniqueID]
```

Loads a previously saved metric file for querying, reporting, and comparing.

The `read_metric` command allows you to manually load previous saved metrics files into a Tempus session. You can load any number of these files. However, these files should have a unique id.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_metric` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_metric`
`-id snapshotName`	Specifies the name of snapshot to create or overwrite.
`metricsFile`	Specifies the name of the metrics file.
`-merge`	Merges the metrics into the named snapshot, instead of replacing.
`-previous` *`UniqueID`*	Specifies the uuid of previous snapshots for the new pending snapshot.

Example

The following code shows the usage of the `read_metric` command along with the `write_metric` command:

```
# ------------------------------------------------------------------

# Generate a XML file for all timing, power, and flow metrics from

# from prects to postcts for a previous run (id v101)

# ------------------------------------------------------------------

read_metric -id v101 run101/dbs/postcts.enc.dat/design.metrics


write_metric -id v101 -format xml -labels prects-postcts v101.xml
```

Related Topics

- report_metric
- write_metric

read_metric_config

```
read_metric_config
[-help]
 fileName
[-format {html excel text}]
```

Reads the YAML file written by the `write_metric_config` command.

Parameters

fileName	Specifies the name to read metric configuration from.
`-format {html excel text}`	Specifies the configuration for specified `report_metric` format.
`-help`	Outputs a brief description that includes type and default information for each `read_metric_config` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_metric_config`

Example

The following example writes the YAML structure in the `text` format in `small.text`, reads the YAML file, and generates the report in the text format.

```
> write_metric -format text small.text ;
> read_metric_config -format text small.yaml
> report_metric  -format text
```

Related Topics

- report_metric
- reset_metric_config
- write_metric_config

redirect

```
redirect
[-help]
[file_or_var_name]
[command]
[-bg |-tee]
[-stdin | -append]
[-stdin | -variable]
[-variable]
[-stderr | -stdin]
```

Redirects the output to a file or variable. You can also directly use redirect characters without explicit use of the redirect command for Tempus commands.

Any Tcl return value from *command* is returned by redirect. If redirect is used at the prompt, any echoing of the Tcl return at the prompt is not redirected.

You can also use the `redirect` command to write the contents directly to a gzip compressed file by ending the file name with .gz.

Parameters

`-help`	Prints a brief description that includes type and default information for each `redirect` parameter.
file_or_var_name	File or variable name for output to go.
`-bg`	Runs the commands in a forked process in background. **Note**: Forking processes have significant impact on memory. Ensure that you have enough memory and do not use this option in a loop. *Data_type*: bool, optional
command	Command to be executed.
`-append`	Appends stdout/stderr to file or variable.
`-stderr`	Redirects stderr.
`-stdin`	Redirects stdin.
`-tee`	Redirects output to both screen and file or variable.
`-variable`	Redirects to a Tcl variable.

It is also possible to use redirect characters similar to Unix usage, for Tempus commands, but not for built-in Tcl commands. This usage works for any Tempus command that does not explicitly register > or >> as options (if > or >> are visible in the man page or -help output for the command, it is explicitly registered). Note that the "source" command is overlaid by Tempus to add various verbose options, so it also supports these characters.

The table below shows the usage of redirect characters, and how to use & to indicate a Tcl var name.

Redirect Characters	Meaning

> <name>	redirect stdout to the file <name>
>> <name>	redirect stdout and append to the file <name>
> &<name>	redirect stdout to the Tcl var <name>
>> &<name>	redirect stdout and append to the Tcl var <name>

Examples

- `redirect timing.txt "time_design -postRoute"`
  Redirects the output to a new file `timing.txt`.
- `redirect test.out.gz "source test.tcl" -append`
  Appends the output to the file `test.out.gz in compressed gzip format.`
- `redirect timing.txt "time_design -postRoute" -append -tee`
  Redirects the output to both screen and a file.
- `source test.tcl > a.log.gz` #redirect to the file a.log.gz in compressed gzip format
- `report_area >> a.log` #redirect and append to the file a.log
- `report_area > &a` `#redirect to the Tcl var a`
- `report_area >> &a` `#redirect and append to the Tcl var a`

report_disk

```
report_disk
[-help]
[-dir directory_name]
```

Reports the disk metrics of the current working directory and the /tmp directory. The output of this command allows you to investigate Mb/sec throughput of the disk and disk I/O requirements.

This command also reports the used and free disk space for the directory specified by the `-dir` parameter (default is /tmp). When clients are started, it reports disk throughput and space on each client.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_disk` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_disk`.
`-dir` *`directory_name`*	Specifies the name of the directory in which disk metric information is written. The default directory is `/tmp`.

Examples

- The following is a sample output for `report_disk` before and after starting clients:

```
report_disk
DISK_INFO : 206.66 Mb/S   381G total   1.1G used   359G free   /tmp
{206.66 381G 1.1G 359G}

distribute_start_clients -count 2
INFO: Starting 2 client processes.
Connected to sjfsb439 39262 1 ( PID=31660 )
Connected to sjfsb439 35186 0 ( PID=31658 )
INFO: Received application variables on master.
0

report_disk
DISK_INFO (Master)   : 182.53 Mb/S   381G total   1.1G used   359G free   /tmp
DISK_INFO (Client 0) : 94.90 Mb/S    381G total   1.1G used   359G free   /tmp
DISK_INFO (Client 1) : 104.28 Mb/S   381G total   1.1G used   359G free   /tmp
{182.53 381G 1.1G 359G} {94.90 381G 1.1G 359G} {104.28 381G 1.1G 359G}
```

- The following is an example that shows the usage of the -dir parameter:

```
report_disk

DISK_INFO : 173.85 Mb/S   118G total   4.9G used   107G free   /tmp

{173.85 118G 4.9G 107G}
```

```
report_disk -dir .
DISK_INFO : 98.51 Mb/S   9.6T total   9.3T used   395G free   .
{98.51 9.6T 9.3T 395G}
set_distributed_hosts -lsf -queue ssv
INFO (PRL-36): The timeout for a remote job to respond is 3600 seconds.
Detected a queue named ssv in the local LSF cluster.
Submit command for task runs will be: /grid/sfi/farm/bin/bsub -q ssv

set_multi_cpu_usage -remoteHost 2 -cpuPerRemoteHost 1 -localCpu 1
distribute_start_clients
Starting 2 client processes.
Job <5810482> is submitted to queue <ssv>. The associated lsf task ID is '0'
Job <5810559> is submitted to queue <ssv>. The associated lsf task ID is '1'
Connected to sjfsb080 43051 0 ( PID=18912 )
Connected to sjfsb080 36921 1 ( PID=19194 )
Received application variables on master.

report_disk
DISK_INFO (Master)   : 163.19 Mb/S   118G total   4.9G used   107G free   /tmp
DISK_INFO (Client 0) : 93.94 Mb/S   190G total   96G used   84G free   /tmp
DISK_INFO (Client 1) : 82.12 Mb/S   190G total   96G used   84G free   /tmp
{163.19 118G 4.9G 107G} {93.94 190G 96G 84G} {82.12 190G 96G 84G}

report_disk -dir ./
DISK_INFO (Master)   : 85.83 Mb/S   9.6T total   9.2T used   404G free   ./
DISK_INFO (Client 0) : 33.77 Mb/S   9.6T total   9.2T used   404G free   ./
DISK_INFO (Client 1) : 32.47 Mb/S   9.6T total   9.2T used   404G free   ./
{85.83 9.6T 9.2T 404G} {33.77 9.6T 9.2T 404G} {32.47 9.6T 9.2T 404G}
```

report_metric

```
report_metric
[-help]
pattern
-out_file fileName
-format {html json}
[-id metricID]
[-names listOfNames]
```

Allows the user to generate reports in various file formats.

Parameters

`-out_file` *fileName*	Specifies the name of the target file.
`-format {html json}`	Specifies the output type.
`-help`	Outputs a brief description that includes type and default information for each `report_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_metric`
`-id` *metricID*	Specifies the ID of metrics to report.
`-names` *listOfNames*	Specifies the list of names. **Note**: Wildcards are supported.
pattern	Specifies the pattern to match.

Related Topics

- `read_metric`
- `update_metric`
- `write_metric`

report_netlist_statistics

```
report_netlist_statistics
```

Reports statistical data of the design netlist such as number of nets, number of IOs, number of pins in the design and number of terms per net(s).

Example

```
*** Statistics for net list DTMF_CHIP ***
Number of cells      = 4263
Number of nets       = 4551
Number of tri-nets   = 0
Number of degen nets = 21
Number of pins       = 16490
Number of i/os       = 57

Number of nets with    1 terms = 21 (0.5%)
Number of nets with    2 terms = 2510 (55.2%)
Number of nets with    3 terms = 771 (16.9%)
Number of nets with    4 terms = 510 (11.2%)
Number of nets with    5 terms = 223 (4.9%)
Number of nets with    6 terms = 108 (2.4%)
Number of nets with    7 terms = 76 (1.7%)
Number of nets with    8 terms = 46 (1.0%)
Number of nets with    9 terms = 30 (0.7%)
Number of nets with >=10 terms = 256 (5.6%)

*** 72 Primitives used:
Primitive ADDFHX1 (195 insts)
Primitive ADDHX1 (15 insts)
Primitive AND2X1 (279 insts)
Primitive AND3X1 (1 insts)
Primitive AND4X1 (8 insts)
Primitive AOI211X1 (65 insts)
Primitive AOI21X1 (282 insts)
```

report_obj

```
report_obj
[<start> [<pattern>]]
[-all | [-computed] | [-selected]]
[-help]
[-categories list_of_categories]
[-file fileName]
[-index index_name index_value ...]
[-set_by_user | -read_only ]
[-tcl | -verbose]
```

Creates a report about the attributes for a given pattern/category/object. This command is mainly intended for root attributes but can be used for any object.

The default output reports those attributes that are set to non-default values and are not computed. This makes it easy to see any changes that were made in the run scripts.

You can also add more attributes or filter out some attributes using various parameters below.

Parameters

`-help`	Prints a brief description that includes the type and default information for each system root attribute.
`-all`	Reports all non-computed attributes and their current value, even if the current value matches the default value.
`-categories` *list_of_categories*	Reports only those attributes that are in the given categories. Wildcards * and ? are allowed. Attributes that do not have a category are assigned the reserved category name `NULL`. The report puts all the attributes in a category together.
`-computed`	Includes attributes that are computed (for example `is_computed == true`). By default, the computed attributes are not included. If the data is not computed, this parameter triggers building the timing graph, and so on, that may take significant run time. This parameter can be used with `-all`.
`-file` *fileName*	Specifies the output file name.
`-index` *index_name index_value ...*	Specifies the list of indices in a form of object pointers or pairs of *obj_type object_name*, for example. '-index {view view1}' or '-index $obj'.
`<pattern>`	Returns all the attribute names that match the specified `<pattern>` using the * and ? wildcards. The `<start>` value is required before the `<pattern>` value. *Default:* *
`-read_only`	Reports attributes with `.is_settable = false`.
`-set_by_user`	Reports the options set by the user.
`<start>`	Returns attributes for the objects in the <start> list. *Default:* `root`
`-selected`	Reports attributes for only selected objects. This parameter cannot be used with `-all`.
`-tcl`	Returns the objects and their values in a TCL style list, which is a list of keyword-value pairs.
`-verbose`	Returns the object's data type and default value. This parameter cannot be used with the `-tcl` parameter.

Examples

- The following command displays the root attributes for the `timing` category that are not computed, and do not match the default value with their current values:

```
report_obj -categories timing -all

********************************************************************************* Object:
root:*********************************************************************************

========================================================= Category:
```

```
timing=====================================================

Attribute Name                                          Current Value

-------------------------------------------------------------------------------

budget_timing_compare_with_new_opt_data                0budget_timing_push_down_virtual_clock
0budget_timing_write_clock_to_virtual_clock_map         0timing_all_registers_include_icg_cells
truetiming_allow_input_delay_on_clock_source              falsetiming_analysis_aocv
false

...
```

- The following command displays all the root attributes that start with place*.
- `report_obj / place* -all`

```
**************************************************************************************
Object: /**************************************************************************************

Attribute Name                                                Current Value ----------------------------
-------------------------------------------------------------place_blockages
{}place_cell_edge_spacing                                   {}place_design_floorplan_mode
falseplace_design_refine_place                                 true...
```

- The following command displays the attributes that do not match the default value for the net objects in the list:

```
report_obj [get_db nets *clk]
************************************************************************************** Object:
net:top/tclk***************************************************************************************
*

Attribute Name                                                Current Value ----------------------------
-------------------------------------------------------------arcs
arc:0x2b1c79348088 arc:0x2b1c793485b0base_name
tclkbbox                                                        {1073741.8235 1073741.8235 -
1073741.824 -1073741.824}...

************************************************************************************** Object:
net:top/tclk2**************************************************************************************
**

Attribute Name                                                Current Value ----------------------------
-------------------------------------------------------------arcs
arc:0x2b1c79348088base_name                                                    tclk2bbox
{1073741.8235 1073741.8235 -1073741.824 -1073741.824}...
```

reset_metric_config

```
reset_metric_config
```

Removes all metric configuration.

Parameters

None

Related Topics

- get_metric_config
- read_metric_config
- write_metric_config

reset_metric_run_id

```
reset_metric_run_id
```

Clears the run ID and the related data from the current state so that other commands do not add snapshots to the original reference run in AUM.

Parameters

None

run_metric_category

```
run_metric_category
[-help]
-category category_name
```

Runs the specified category.

Parameters

`-help`	Describes the command usage.
`-category category_name`	Specifies the name of the category, such as design, setup, hold, clock, power, route, check, flow, to be run. Each category updates the metric associated with that category.

set_compress_level

`set_compress_level` *`compression_level`*

Specifies the compression speed to use when compressing large data files. A faster compression speed creates a less compressed file. A slower compression speed creates a more compressed file. By default, the software uses the `gzip` default speed compression_level.

Parameters

`-help`	Prints a brief description that includes type and default information for each `set_compress_level` parameter. For a detailed description of the command and all of its parameters, use the man command: man `set_compress_level`
`compression_level`	Specifies the speed level to use when compressing files. You can specify a compression_level between 1 and 9, where: `1` Performs the quickest compression, and least compressed files. `9` Performs the slowest compression, and most compressed files. **Note:** To return to the `gzip` default compression_level (`Z_DEFAULT_COMPRESSION`), specify `-1`.

set_license_check

```
set_license_check
[-help]
[-checkout lic]
[-default]
[-option_list "lic1 lic2 ..."]
[-server_license_report]
[-status_report]
[-wait_time t]
```

Manages licenses for product options. Specifies licenses to check out, license wait time, and the status of all product option licenses.

Note: This command does not manage multi-CPU licenses. For information on the commands that manage multi-CPU licenses, see the "*Multiple-CPU Processing Commands*" chapter.

Parameters

`-help`	Prints out the command usage.
`-checkout` *`lic`*	Checks out the licenses for the specified product options when this command runs.
`-default`	Restores the default. values for this command.
`-option_list "`*`lic1 lic2 ...`*`"`	Specifies licenses to check out dynamically. Dynamic licenses are not checked out until they are needed. If you specify a license that is not allowed with your base license, or a license that is not available, the software does not check out that license and instead issues a warning message. If you specify more than one license, begin and end the option list with double quotation marks or braces.
`-server_license_report`	Prints out all the licenses that exist on the license server.
`-status_report`	Reports the current status of all licenses and the specified wait time. Outputs the following information: • The name of the base license that is checked out • The current wait time • A list of the allowed license options • A list of the available license options • A list of the currently checked-out license options.
`-wait_time` *`t`*	Allows the software to wait for up to `t` minutes when checking-out an optional license. If the license is available in less than the specified wait time, the system checks out the next needed license without waiting. *Default:* `0` (no wait time) *Value range:* `0` to `10,000`

Examples

- The following command specifies that licenses for Tempus-L and Tempus-XL may be checked out. Tempus-L licenses are checked out before Tempus-XL licenses.
  ```
  set_license_check -option_list {tpsl tpsxl}
  ```
- The following command sets the wait time to 5 minutes:
  ```
  set_license_check -wait 5
  ```

set_limited_access_feature

```
set_limited_access_feature
[-help]
featureName
{1 | 0}
```

Enables or disables a limited access feature.

The Tempus software provides certain limited access features that showcase advanced technology integrated into the software. These features have been internally qualified at Cadence but have had only limited customer testing.

Note: Limited access features are enabled by a variable specified through the `set_limited_access_feature` command. To use the limited access features, contact your Cadence representative to qualify your usage, and to make sure that your requirements are met before deploying a feature widely.

Parameters

`-help`	Prints a brief description that includes type and default information for each `set_limited_access_feature` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_limited_access_feature`
featureName	Specifies the name of the variable that enables the limited access feature.
`{1 | 0}`	Enables/disables a feature - `1` enables a feature, `0` disables a feature.

set_metric

```
set_metric
-name metric_name
-help
-value metric_value
```

Sets any defined metric to a value.

Parameters

`-name metric_name`	Specifies the name of the metric.
`-help`	Outputs a brief description that includes type and default information for each `set_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_metric`
`-value metric_value`	Specifies the value of the metric.

Examples

The following command sets the value `dbFindInstsByCell HDR*` to the metric `design.switch_cells`.

```
set_metric -name design.switch_cells [dbFindInstsByCell HDR*]
```

set_metric_alias

```
set_metric_alias
[-help]
-from old_name
[-no_overwrite]
[-order order_of_priority]
-to new_name
```

Renames a metric name, prefixed with the name of a layer, analysis view, path group etc, to another name. For example, you can rename `path_group:reg2reg` to `reg2reg` using the `set_metric_alias` command.

Parameters

`-from` *`old_name`*	Specifies the metric object name to be changed.
`-help`	Outputs a brief description that includes type and default information for each `set_metric_alias` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_metric_alias`.
`-no_overwrite`	Specifies to not overwrite an already set alias.
`-order` *`order_of_priority`*	Specifies the order of priority of metric object in tables.
`-to` *`new_name`*	Specifies the new name for the metric object.

Example

- The following command renames `path_group:reg2reg` to `reg2reg` with priority order `1`.

  ```
  > set_metric_alias -from path_group:reg2reg -to reg2reg -order 1
  ```

- The following command renames `path_group:reg2cgate` to `reg2cgate` with priority order `2`.

  ```
  > set_metric_alias -from path_group:reg2cgate -to reg2cgate -order 2
  ```

- The following command queries the set of aliases:

  ```
  > get_metric_alias
  ```

  Output

  ```
  >path_group:reg2reg {to reg2reg order 1} path_group:reg2cgate {to reg2cgate order 2}
  ```

Related Topics

- `get_metric_alias`

set_metric_header

```
set_metric_header
[-id db_id]
-name header_name
-help
-value header_value
```

Sets the name of a metric header.

Parameters

`-id db_id`	Specifies the database ID.
`-name header_name`	Specifies the name of the header.
`-help`	Outputs a brief description that includes type and default information for each `set_metric_header` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_metric_header`
`-value header_value`	Specifies the value of the header.

set_preference

```
set_preference
[-help]
preference_name
value
```

Defines preferences, such as, commands issued during a session can be logged to a log or the screen.

Parameters

`-help`	Outputs a brief description that includes type and default information for each set_preference parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_preference`.
preference_name	Specifies the name of the preference variables that you want to set. To view preferences in the Tempus GUI, select ***View - Set Preferences*** from the main menu. For more details, refer to the "Set Preferences" section of the *Tempus Menu Reference* guide.
value	The value for the preference. For example, the `CmdLogMode` preference has the following values. 0--write to command only 1--write to command and log (default) 2--write to command, log, and screen When writing to a log and screen, the commands are preceded by <CMD>.

Preference Variables

For a complete list of preference variables that you can set using the `set_preference` command and their GUI equivalent in the Preferences form.

Examples

The following command writes log to command, log, and screen:

```
tempus > set_preference CmdLogMode 2
```

Related Topics

For more information on setting preferences from the GUI, see:

- The Preferences form in the "Tools Menu" chapter in the *Tempus Menu Reference*.

Related Commands

- `get_preference`

source

```
source
[-help]
filename
[-quiet | verbose]
[-quiet | -echo ]
```

Reads a file and executes the commands in the file (scripts).

Parameters

`-help`	Describes the command usage.
`-echo`	Enables verbose source. If -echo is specified, the tool will ignore it and continue with the source instead of giving an error. **Note:** This is optional.
`filename`	Specifies the name of the file to source.
`-quiet`	Forces source to be normal Tcl source command. **Note:** This is optional.
`-verbose`	Forces source to be in verbose mode. **Note:** This is optional.

update_inst_name

```
update_inst_name
-inst instName
-new_base_name baseName
```

Changes the base name of the specified instance to the given base name.

Note: The `update_inst_name` command does not change the path of hierarchy.

Parameters

`-inst` *instName*	Specifies the name of the instance.
`-new_base_name` *baseName*	Specifies the new name for the instance.

Example

The following example changes the instance name from `h3/inv3` to `h3/h2/inv1`:

```
update_inst_name -inst h3/inv3 -new_base_name h2/inv1
```

update_metric

```
update_metric
[-help]
[-format {html excel text}]
[-graph_type {treemap line none}]
[-group group_title]
[-header header_name]
[-link_file_metric metric_name]
[-link_table table]
[-metric metric_name]
[-no_shared_max]
[-renderer html_format]
[-show_graph]
-table table_id
[-title html_table_title]
```

Updates a metric.

Parameters

`-format {html excel text}`	Specifies configuration for the specified `report_metric` format.
`-graph_type {treemap line none}`	Specifies the type of graph to use in the table (treemap |line |none). *Default*: `line`
`-group group_title`	Specifies the group title to be used in the html table.
`-header header_name`	Specifies the header name to be added.
`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man update_metric`
`-metric metric_name`	Specifies the metric name to be added.
`-no_shared_max`	When specified for a `-renderer` histogram metric, the Y axis of all the histograms in a single column stop sharing the same max Y value.
`-link_file_metric metric_name`	Specifies the name of the metric containing filename to open when the metric value is clicked.
`-link_table table`	Specifies the table to scroll to when the metric value is clicked.
`-renderer html_format`	Forces a specific html format renderer for the histogram metric.
`-show_graph`	Specifies whether to show this metric in a comparison graph.
`-table table_id`	Specifies the id of table to which the metric will be added.
`-title html_table_title`	Specifies the title to be used in the html table.

Related Topics

- read_metric
- report_metric
- write_metric

update_metric_definition

```
update_metric_definition
[-help]
[-compare compareFunction]
[-description metricDescription]
[{-id databaseID}
[-inheritance_tcl TclName]
-name metric_name
[-no_inheritance]
[-precision number_of_decimal_places]
[-target metric_name]
[-tcl {TCL_command_list}[-tools listOfTools]]
[-threshold reporting_unit]
[-type metricType]
[-units reporting_unit]
[-warning threshold]
```

Updates the specified properties of an already-existing metric definition with the same name.

Note: If a metric does not exist with the name, the command returns an error.

Parameters

Specifies the list of tools in which the tcl code will run.*Data_type*: string, optional

`-compare` *compareFunction*	Specifies the compare function, `[moreBetter|lessBetter|moreBetterBelowTarget|lessBetterAboveTarget]`. The comparison options show the good or bad color when the value is below or above the target value as appropriate. *Unit*: `string`
`-description` *metricDescription*	Specifies the description of the metric. *Unit*: `string`
`-id` *databaseID*	Specifies the database ID. *Unit*: `string`
`-inheritance_tcl` *TclName*	Specifies Tcl for combining children values. The metric is not executed in the global namespace. The equivalent proc args is {children}. *Unit*: `string`
`-name` *metric_name*	Specifies the name of the metric. *Unit*: `string`
`-no_inheritance`	Disables the inheritance of the metric. *Unit*: `boolean`

`-help`	Outputs a brief description that includes type and default information for each `update_metric_definition` parameter. For a detailed description of the command and all of its parameters, use the man command: `man update_metric_definition`
`-precision` *`number_of_decimal_places`*	Specifies the number of decimal places. *Unit*: `integer`
`-target` *`metric_name`*	Specifies the target or the name of metric containing the target value.
`-tcl { TCL_command_list }`	Specifies the list of TCL commands or proc to generate (return) the metric which is of the format `{<value> <unit>}`. *Unit*: `string`
`-threshold` *`reporting_unit`*	Specifies the threshold for comparing numbers in HTML. If the delta between the two numbers being compared is less than the threshold then the metric will be colored “yellow”. If it is outside the threshold, then it is colored red or green depending on the compare function i.e. whether more is better or less is better.
`-tools` *`listOfTools`*	Specifies the list of tools in which the tcl code will run. *Unit*: `string`

`-type` *`metricType`*	Specifies the type of metric.
	Unit: `string`
	The various metrics types are:
	• imageplot: A metric type that the renders to an image in the HTML report.
	• Format:
	`<name1> { \`
	` fill_color "<color>" \`
	`  stroke_color "<color>"\`
	`  data <list of data sets>`
	`} \`
	`<name2> { \`
	`<sub-name1> { \`
	`  fill_color <color> \`
	`  data <list of data sets> \`
	`  } \`
	`  <sub-name2> {...} \`
	`}`

Where a data set can be one or more of the following:

The data on each layer is a string of coordinates (separated by sequences of whitespace and comma) with types. These types are:

- R - rectangle (x, y, width, height)
- B - bounding box (x_min, y_min, x_max, y_max)
- G - closed polygon (x, y, x, y, ...)
- L - open polyline (x, y, x, y, ...)
- C - circle (x, y, r)
- P - point (x, y)

For rectangles, boxes, circles and points, multiple shapes can be defined without having to repeat the type letter. e.g. R1,2,3,4,5,6,7,8 is two rectangles.

A value can be defined for shapes by adding a V suffix to the type; this value can then be used to color the shapes according to the value for heat maps. The value is always the first number after the type. This means that two points with values 20 and 40 may look like: PV20,1,2 40,6,5 or: PV20,1,2PV40,6,5.

Example imageplot TCL dict data:

```
macros { \
fill_color "#FF0000" \
  stroke_color "#0000FF88" \
  data "R5,8,1,2 6,9,2,1 G1,2,3,4,5,6,7,8" \
} \
drcs { \
placement { \
    fill_color "#FF0000" \
    data "P3,4" \
  } \
  routing {...} \
}
```

Layers in the image are named (macros, congestion, normal, other in the above example) and support hierarchy. Layers should not be nested beyond one level deep. The hierarchy is used in the renderer (see below) to toggle the visibility of the layers. It is an error for any layer to be called data (reserved to keep hierarchy clear).

Colors can be defined either as hex or RGB values with optional opacity. Hex values are prefixed with #. RGB values are separated with whitespace.

- #FF0000: Red
- #00FF0040: Green (25% opaque)
- 0 0 255: Blue
- 255 0 255 192: Magenta (75% opaque)

- histogram: The data will be structured as a set of series with shared X-axis labels that are rendered as a stacked histogram.

  Format (TCL dict):

```
labels <list of labels> \
axis_x <name> \
axis_y <name> \
series { \
<series1> { \
   values <list of buckets \
   color "#FF0000" \
} \
<series2> { \
   values {2 1 0 0} \
   color "#0000FF" \
} \
}
```

  Example:

```
get_metric timing.setup.histogram
timing.setup.histogram {labels {-0.320 -0.240 -0.160 -0.080} \
                        series { \
                          reg2reg {color #FF0000 values {43 3 2
3 0}} \
                          reg2cgate {color #0000FF values {22 4
5 0 0}} \
                        } \
                        axis_x WNS(ns) \
                        axis_y {Failing End Points} \
                        }
```

  The labels (a list of strings, shown in the order provided) define the X-axis labels for the histogram, the values in each series define the Y-axis values.

	Colors can be defined either as hex or RGB values with optional opacity. Hex values are prefixed with #. RGB values are separated with whitespace.
	`#FF0000 - Red`
	`0 0 255 - Blue`
	• html: Supports the inclusion of raw HTML in a metric. You can place simple html into tables allowing simple links to reports. For example, the following command defines `test_html` as an html type of metric:
	`define_metric -name test_html -type html`
	You can the html metric, test_html, using the `set_metric` command:

`-units` *reporting_unit*	Specifies the unit for reporting. *Unit*: `string`
`-warning` *threshold*	Specifies the warning threshold or name of metric containing the threshold value.

Examples

The following command defines the `design.vtmix` metric :

```
update_metric_definition -name design.vtmix -tcl {return [TCL commands]}
```

Related Topics

- `get_metric_definition`

view_log

```
view_log
[-help]
[file_name logFileName]
```

Opens up a log file in a separate console window for viewing within the Tempus.

The log file viewer within the Tempus opens the current log file by default, and updates it in real time. If you specify a log file name, the software opens up the specified log file.

The log file displayed contains `[+]` and `[-]` markers that expand and collapse command information, so that you can control how much detail you want to read. Log messages are color coded for easier identification: blue messages are warnings, red messages are errors.

You can access documentation for commands in the log file that are underlined. Click on the command in the file, and the software opens an HTML window displaying the *Tempus Text Command Reference* documentation for that command.

You also can access the log file viewer from the *Tools Menu - Log Viewer*.

Note: You can specify `view_log` more than once to view multiple log files in separate console windows simultaneously. However, you cannot open multiple versions of the current log file.

Parameters

`file_name` *logFileName*	Specifies the name of the log file to open.
`-help`	Outputs a brief description that includes type and default information for each `view_log` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man view_log`.

write_metric

```
write_metric
[pattern]
{-out_file fileName}
[-format output_format]
[-id metric_id]
[-merge]
[-names list_of_patterns]
```

Writes the current set of metrics to a file or stdout.

The write_metric command is used by `write_db` to write the metrics in the JSON format in the database directory. Super commands use `write_metric` to write metrics into the log file (stdout). This can also be used to write metrics in various formats by the user.

Parameters

`-out_file fileName`	Specifies the name of the file in which metrics is written.
`-format output_format`	Specifies the output format, json, xml, csv, tcl. *Default*: JSON
`-help`	Outputs a brief description that includes type and default information for each `write_metric` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_metric`
`-id metric_id`	Specifies a list of labels from read_metrics. *Default*: Current metric
`-merge`	Merges the results into an existing file.
`-names list_of_patterns`	Specifies the list of patterns to match snapshot names.
`pattern`	List of regular expressions to chose metrics to write. Default is all metrics (*)

Example

The following code shows the usage of the `write_metric` command:

```
# ------------------------------------------------------------------

# Generate a XML file for all timing, power, and flow metrics from

# from prects to postcts for a previous run (id v101)

# ------------------------------------------------------------------

read_metric -id v101 run101/dbs/postcts.enc.dat/design.metrics
```

```
write_metric -id v101 -format xml
```

Related Topics

- read_metric
- report_metric
- write_db

write_metric_config

```
write_metric_config
[-help]
 fileName
[-format {html excel text}]
```

Writes the YAML structure for the requested format (text, html, or excel) to a file. As the configuration is written in YAML format, you can customize the page layout and table content you need in the report such as adding pages and tables, or adding new metrics to existing tables, removing metrics from existing tables, and rearranging the metric ordering.

Parameters

fileName	Specifies the name of the file in which metric config is written.
-format {html excel text}	Specifies the config for specified report_metric format.
-help	Outputs a brief description that includes type and default information for each write_metric_config parameter. For a detailed description of the command and all of its parameters, use the man command: man write_metric_config

Example

generates the report in the text format.

> write_metric_config -format text small.text ; *# Edit the summary table*

> read_metric_config -format text small.yaml

> report_metric -format text

Related Topics

- read_metric_config
- report_metric

read_metric_html_package

```
read_metric_html_package
[-help]
filename
```

Overwrites the HTML for `report_metric -format vivid` and the vivid page configuration (accessible via `read_metric_config`/`write_metric_config`) with the contents from the specified *`filename`*.

Parameters

`filename`	Specifies the name of the file from where the tool will read the metric package. If the specified *`filename`* is not a supported format, the tool will give an error message that the file format is invalid.
`-help`	Outputs a brief description that includes the type and default information for each `read_metric_html_package` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_metric_html_package`

Related Topics

- read_metric_config
- `write_metric_config`

get_metric_header

```
get_metric_header
-help
[-id db_id]
-name header_name
```

Returns the metric header information set by the `set_metric_header` command.

Parameters

`-id db_id`	Specifies the database ID.
`-name header_name`	Specifies the name of the header.
`-help`	Outputs a brief description that includes type and default information for each `get_metric_header` parameter. For a detailed description of the command and all of its parameters, use the man command: `man get_metric_header`

Example

The following command sets the value `bar` of the metric header `foo`:

```
> set_metric_header -name timing -value high
```

The following command returns the header value of the metric header `foo`

```
> get_metric_header -name timing

high
```

Related Topics

- set_metric_header

get_preference

```
get_preference
[-help]
preference_name
```

Returns the current preference settings.

Parameters

`-help`	Prints out the command usage
`preference_name`	Specifies the name of a preference.

Related Commands

- `set_preference`

delete_metric_header

```
delete_metric_header
[-help]
[-id database_id]
-name header_name
```

Deletes the metric header.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the man command: `man delete_metric_header`
`-id` *`database_id`*	Specifies the database ID *Data_type*: string, optional
`-name` *`header_name`*	Specifies the header name to be deleted. *Data_type*: string, required

Example

- The following command deletes the metric header, `User`.

  ```
  delete_metric_header -name User
  ```

10

GUI Commands

- clear_all_rulers
- create_gui_shape
- create_ruler
- display_temperature_result
- gui_add_marker
- gui_add_ui
- gui_attach_to_cursor
- gui_clear_highlight
- gui_create_user_bundle_net
- gui_delete_objs
- gui_delete_ui
- gui_delete_user_bundle_net
- gui_deselect
- gui_dim_foreground
- gui_find_ui
- gui_fit
- gui_get_edge_number
- gui_get_objects_from_schematic
- gui_get_schematic
- gui_get_ui
- gui_get_visible_bumps
- gui_get_visible_net_groups

- gui_get_visible_nets
- gui_highlight
- gui_highlight_pin
- gui_highlight_pin_connection
- gui_list_marker
- gui_open_schematic
- gui_pan
- gui_pan_center
- gui_pan_page
- gui_redraw
- gui_remove_marker
- gui_report_ruler
- gui_schematic_add_selected
- gui_schematic_load_view
- gui_schematic_write_view
- gui_select
- gui_select_highlighted
- gui_set_obj_color
- gui_set_power_rail_display
- gui_set_power_rail_layers_nets
- gui_set_tool
- gui_set_ui
- gui_set_visible_bumps
- gui_set_visible_net_groups
- gui_set_visible_nets
- gui_show_edge_number
- gui_show_user_bundle_nets

- gui_view_box
- gui_write_picture
- gui_zoom
- read_power_rail_results
- report_power_rail_results
- view_last
- view_next
- write_to_gif

clear_all_rulers

```
clear_all_rulers -help
```

Removes all ruler lines from the main window. You can also clear the ruler lines with the *Clear All Rulers* icon on the toolbar in the main window.

Parameters

`-help`	Prints a brief description that includes type and default information for each `clear_all_rulers` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man clear_all_rulers`

Related Information

- `create_ruler`

create_gui_shape

```
create_gui_shape
[-help]
[-layer layerName]
[-name guiShapeName]
[-width width]
{-rect {x1 y1 x2 y2} | -line {x1 y1 x2 y2 ...} | -polygon {x1 y1 x2 y2 ...}}
[ -line {x1 y1 x2 y2 ...} [-arrow ] ]
```

Adds a shape to the specified custom layer. You can use the `create_gui_shape` command at any point in the design flow. The shapes added by the command annotate comment shapes and do not show up in a DEF output.

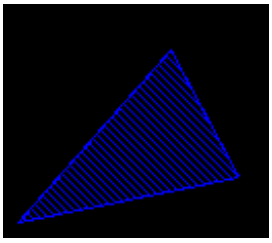
Parameters

`-help`	Prints a brief description that includes type and default information for each `create_gui_shape` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_gui_shape`
`-arrow`	Draws an arrow in the middle of the line segment to indicate the start to end direction. This parameter can only be used with `-line`. *Data_type*: bool, required
`-layer` *layerName*	Specifies the GUI layer name to use or create for the shape. If the specified layer does not exist, it is created automatically. If a layer is not specified, the tool automatically creates a layer with the name `defaultLayer` and adds the shape to it. *Data_type*: string, optional
`-line {`*x1 y1 x2 y2 ...*`}`	Creates a line, with *x1 y1 x2 y2 ...* specifying the coordinates for the line segments. The values are specified in microns. *Data_type*: (point)+, optional
`-name` *guiShapeName*	Specifies the name of the GUI shape. *Data_type*: string, optional
`-polygon {`*x1 y1 x2 y2 ...*`}`	Creates a polygon shape, with *x1 y1 x2 y2 ...* specifying the coordinates for the polygon segments. The values are specified in microns. *Data_type*: (point)+, optional
`-rect {`*x1 y1 x2 y2*`}`	Specifies the coordinates of the lower-left and upper-right points of the rectangle. The values are specified in microns. *Data_type*: rect, optional
`-width` *width*	Specifies the width of the shape border in pixels. The minimum (default) value is `1` and the maximum is `7`. *Data_type*: int, optional

Examples

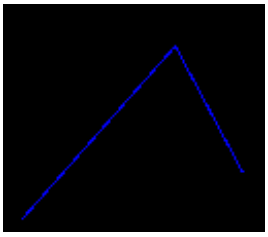
- The following command adds a triangular shape in layer `1`:

```
create_gui_shape -layer 1 -polygon {95.841 -645 96.592 -644.147 96.926 -644.777}
```



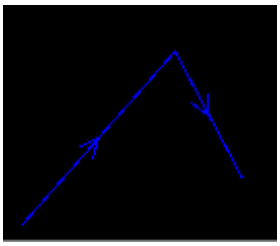
- The following command adds a line shape with two segments:

```
create_gui_shape -layer 1 -line {95.841 -645 96.592 -644.147 96.926 -644.777}
```



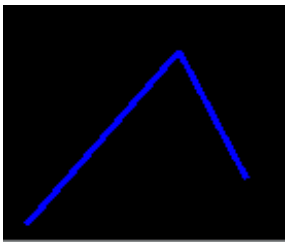
- The following command adds a line shape with two segments and direction arrows:

```
create_gui_shape -layer 1 -line {95.841 -645 96.592 -644.147 96.926 -644.777} -
arrow
```



- The following command adds a line shape of 3-pixel width:

```
create_gui_shape -layer 1 -line {95.841 -645 96.592 -644.147 96.926 -644.777} -
width 3
```



create_ruler

```
create_ruler
[-help] {{-center {x y} -radius value} | -coords {x1 y1 x2 y2 ...}}
```

Adds a ruler. For an orthogonal ruler, use `-coordinates` to specify ruler segments. For a circular ruler, use `-center` and `-radius`.

Parameters

`-help`	Prints a brief description that includes type and default information for each `create_ruler` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_ruler`
`-center {x y}`	Specifies the coordinates for the center of a circular ruler. *Data_type*: point, optional
`-coords {x1 y1 x2 y2 ...}`	Specifies the beginning and ending coordinates for the ruler line segments. The values are in microns. *Data_type*: (point)+, optional
`-radius value`	Specifies the radius of a circular ruler. *Data_type*: double, optional

Example

- The following command creates a circular ruler of radius 500 with the center at `{0 0}`:

  ```
  create_ruler -center {0 0} -radius 500
  ```

display_temperature_result

```
 display_temperature_result
[-help]
-layer <string>
-reset
```

Displays the tile-based temperature map file generated by Sigrity Celsius.

Parameters

`-help`	Prints a brief description that includes type and default information for each `display_temperature_result` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man display_temperature_result`
`-layer <string>`	Displays the temperature map for the specified layer. **Example**: `display_temperature_result -layer Metal6`
`-reset`	Resets the temperature map display.

gui_add_marker

```
gui_add_marker
[-help]
-color <string>
-name <string>
-point {x y}
-type {X TICK STAR}
```

Places a marker to indicate location on the design layout.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-color <string>`	Specifies the color of the marker. You can specify 16 colors for the marker. These are: `red blue green yellow magenta cyan pink orange brown purple violet teal olive gold maroon wheat`
`-name <string>`	Specifies the name of the marker.
`-point {x y}`	Specifies the location `{x y}` at which the marker will be placed on the design layout.
`-type {X TICK STAR}`	Specifies the shape of the marker. There are three shapes for the markers: X, TICK, and STAR.

Example

- The following command shows that a X type marker of color RED has been placed at the location {100 100}.
  ```
  gui_add_marker -name aa -color red -point {100 100} -type X
  ```

gui_add_ui

```
gui_add_ui
[-help]
name
[-before name]
[-checked {true | false}]
[-command string]
[-disabled {true | false}]
[-enable_variable string]
[-foreground string]
[-icon filename]
[-in name]
[-label string]
[-menu_separate {true | false}]
[-movable {true | false}]
[-newline {true | false}]
[-shortcut string]
[-tooltip string]
-type {menu | toolbar | submenu | command | check | radio | separator | toolbutton}
[-underline integer]
[-value string ]
[-variable string]
[-visible_variable string]
```

Adds a new interface element with the specified name of the specified type. Use this command to customize the graphical interface by adding menus, toolbars, status bar, and other interface elements, as required.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_add_ui` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_add_ui`

name	Specifies as a string the internal name of the interface element you want to add. **Note:** Choose the internal name of the element carefully as this is what is used internally by the software to identify the element for any further manipulation.
-before *name*	When specified, adds the new element before the element identified by *name*.
-checked {true | false}	Specifies whether or not the check box or radio button you are adding should be checked. **Note:** You can use the -checked parameter only if you are adding elements of type check or radio. *Default*: false
-command *string*	Specifies the Tcl command to be executed when the element you are adding is invoked. **Note:** You can use the -command parameter only if you are adding elements of type command, check, radio or toolbutton.
-disabled {true | false}	Specifies the state of the interface element you are adding. **Note:** You can use the -disabled parameter only if you are adding elements of type menu, submenu, command, check, radio or toolbutton. The parameter is NOT applicable for elements of type toolbar and separator. *Default*: false
-enable_variable *string*	Represents the Tcl global variable for Enable status.
-foreground *string*	Specifies foreground color of the icon. **Note:** You can use the -foreground parameter only if you are adding an element of type toolbutton.

`-icon` *`filename`*	When adding a toolbutton, specifies the file name of the toolbutton icon. *`filename`* can be the full path of the icon file or an icon name supported by the tool. The icon names used in the tool can be queried using the `gui_get_ui` command. **Note:** The `-icon` parameter is applicable only for elements of type `toolbutton`.
`-in` *`name`*	Specifies the name of the parent element. Use this parameter along with `-before` to specify the location of the element you are adding. By default, `-in` adds the element to the end of the parent element. Use `-before` to add the element at a specific location within the parent element. **Note:** If `-in` is not specified, the new element is created but is not added to any location.
`-label` *`string`*	Specifies the label to be displayed on the new element. **Note:** This parameter is NOT applicable for elements of type `separator`.
`-menu_separate {true | false}`	
	Specifies whether or not a menu or submenu being added can be torn off or detached. **Note:** This parameter is applicable for elements of type `menu` and `submenu`. *Default*: `true` *Data_type*: bool, optional
`-movable {true | false}`	When adding a toolbar, specifies whether or not it will be movable. *Data_type*: bool, optional
`-newline {true | false}`	When adding a toolbar, specifies whether or not the toolbar should be added as a new row. **Note:** The `-newline` parameter is applicable only for elements of type `toolbar`. *Default*: `false`

`-shortcut` *string*	Specifies the shortcut key for a command, check box, radio button, or toolbutton. **Note:** You can use the `-shortcut` parameter only if you are adding elements of type `command`, `check`, `radio`, or `toolbutton`.
`-tooltip` *string*	Specifies the tooltip text for the toolbar or toolbutton you are adding. **Note:** The `-tooltip` parameter is applicable only for elements of type `toolbar` and `toolbutton`.
`-type {menu | toolbar | submenu | command | check | radio | separator | toolbutton}`	
	Specifies the type of the interface element you want to add. Currently, elements of type `main` and `statusbar` are not supported.
`-underline` *integer*	Specifies the integer index of the character to be underlined in the element label. The index begins with `0` and should have a value less than or equal to the length of the element label. For instance, if you want the second letter of the element label to appear underlined, specify *integer* as `1`. **Note:** The `-underline` parameter is NOT applicable for elements of type `toolbar, separator,` and `toolbutton`. **Note:** This parameter can be used only if the `-label` parameter is also specified.
`-value` *stringValue*	When adding a radio button, specifies the value to be stored in the associated variable when the radio button is selected. **Note:** The `-value` parameter is applicable only for elements of type `radio`.
`-variable` *string*	When adding a check box or radio button, specifies the Tcl global variable for the item.
`-visible_variable` *string*	Represents the Tcl global variable for Visible status

Examples

- The following command will add a new, non-detachable menu labeled *ExampleMenu* in the main window:

```
gui_add_ui exmpMenu -type menu -label ExampleMenu -menu_separate false -in main
```

- The following command will add a new menu item labeled *ExampleCommand* in *ExampleMenu*:

```
gui_add_ui exmpCommand -type command -label ExampleCommand -in exmpMenu -command
[list puts Example Command]
```

- The following command adds a new, unmovable toolbar to the layout window:

```
gui_add_ui newTB -type toolbar -in main.layout -movable false
```

Related Information

- gui_delete_ui
- gui_find_ui
- gui_get_ui
- gui_set_ui

gui_attach_to_cursor

```
gui_attach_to_cursor
[-help]
[ -selected  | objects ]
```

Attaches selected or specified objects to the cursor. The use model is as follows:

1. Select the object(s) to be moved by using the `gui_attach_to_cursor` command.
2. Zoom into the location where you want to place the object(s).
3. Click at the required location to place the selected object(s) there.

This command is useful during floorplanning when you may need to move multiple objects into the same zoomed in area.

Parameters

`-help`	Prints a brief description of the `gui_attach_to_cursor` command. For a detailed description of the command, use the `man` command: `man gui_attach_to_cursor`.
`objects`	Specifies the object pointer list.
`-selected`	Attaches the selected objects to the cursor.

Example

The following command attaches objects `A` and `B` to the cursor:

```
gui_attach_to_cursor {A B}
```

Now, move the cursor to the layout and click at the desired location to place objects `A` and `B` there.

Related Information

- `gui_select`
- `gui_zoom`

gui_clear_highlight

```
gui_clear_highlight
[-help]
[obj_list]
[-all | -index indexValue | -original | -select]
```

Removes highlights from all the highlighted objects at any stage of the design flow. You can choose to clear highlight from all objects, selected objects, objects highlighted with original color, or objects with specified index values.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_clear_highlight` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_clear_highlight`
`-all`	Specifies that highlight will be cleared from all objects.
`-index` *`indexValue`*	Specifies that highlight will be cleared from the objects with the specified index value. The index value is the value of the Highlight Set as specified through the *View - Highlight Selected* menu or through the gui_highlight command. The *View - Highlight Selected* menu provides eight highlight patterns to choose from. The value of index should, therefore, be a number be from 1 to 8. As an example, if you specify a value of 3 for index, all highlight will be cleared from from all objects that are highlighted as Highlighted Set 3.
`obj_list`	Specifies the object pointer list for clearing highlight.
`-original`	Specifies that highlight should be cleared from the objects highlighted with original color.
`-select`	Specifies that highlight should be cleared from all selected objects.

Examples

- The following command clears highlight from all highlighted objects.

  ```
  gui_clear_highlight
  ```

- The following command clears highlight from all objects that have been highlighted as Highlight Set 4 through the *View - Highlight Selected* menu or through the `gui_highlight` command.

  ```
  gui_clear_highlight -index 4
  ```

- The following command clears highlight from all highlighted objects that are currently selected in the display window.

  ```
  gui_clear_highlight-select
  ```

Related Topics

- `gui_highlight`
- `gui_highlight_pin`

gui_create_user_bundle_net

```
gui_create_user_bundle_net
[-help]
name
-nets net_list
-start object_name
-end object_name
```

Creates a bundle of specified nets that can be displayed as flightlines. Bundled nets help you with data flow analysis before and after layout. Net bundles can be created and viewed in floorplan and amoeba views. Each custom net bundle displays the number of nets in it and works for macros, blobs, and giudes.

Using `gui_create_user_bundle_net`, you can create custom bundled nets that allow you to define your own net bundles as per your requirements. This is useful while debugging physical properties of interconnects within a complicated hierarchical design because you can define start and end points of a net bundle. As a result, you can create and display only relevant flightlines, instead of looking at all the flightlines of specified nets.

Note: One net can be part of multiple net bundles simultaneously.

You can display flightline net bundles using the gui_show_user_bundle_nets command and delete them using the gui_delete_user_bundle_net command.

Note: You can choose to display flightlines for only bundled nets by using the *Only Show Net Bundle Flight Line* option in the Preferences form. If this option is not selected, the net bundle flightlines are displayed along with other selection-based flightlines by default.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_create_user_bundle_net` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_create_user_bundle_net`
`name`	Specifies the name of the custom net bundle.
`-nets` *`net_list`*	Specifies the list of nets to be included in the net bundle. You can use patterns with wildcards to specify net names. All the nets that match the specified pattern are put in the same bundle.
`-start` *`object_name`*	Specifies the name of object which is the start point of the nets. The object type includes inst, hinst, port, base_pin, and group.
`-end` *`object_name`*	Specifies the name of object which is the end point of the nets. The object type includes inst, hinst, port, base_pin, and group.

Examples

- The following command creates a bundle called `bundle_1`, which contains flightlines of nets `net_1` and `net_2`:

  ```
  gui_create_user_bundle_net bundle_1 -nets {net_1 net_2} -start Module1 -end
  Module2
  ```

  You can then use the gui_show_user_bundle_nets command to display `bundle_1` flightlines as follows:

  ```
  gui_show_user_bundle_nets -net_bundle bundle_1
  ```

Related Information

- `gui_delete_user_bundle_net`
- `gui_show_user_bundle_nets`

gui_delete_objs

```
gui_delete_objs
[-help]
[-shape | -text | -marker | -all | -selected | -list string]
```

Deletes specified or selected GUI objects.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man gui_delete_objs`
`-all`	Deletes all GUI objects. *Data_type*: bool, optional
`-list string`	Deletes the specified list of GUI objects. Here, *string* represents a list of object pointers. You can also specify input in the Tcl Dual Ported Object (DPO) format. *Data_type*: string, optional
`-marker`	Deletes GUI markers. *Data_type*: bool, optional
`-selected`	Deletes selected GUI objects. *Data_type*: bool, optional
`-shape`	Deletes GUI shapes. *Data_type*: bool, optional
`-text`	Deletes GUI texts. *Data_type*: bool, optional

Related Information

- `gui_add_marker`
- `create_gui_shape`

gui_delete_ui

`gui_delete_ui [-help] name`

Deletes an existing interface element with the specified name. If multiple interface elements are found with the same name, the first matched element is deleted. Use this command to customize the graphical interface by removing menus, menu items, and other interface elements that you do not require any longer.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_delete_ui` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_delete_ui`
`name`	Specifies the internal name of the interface element as a string. An error is returned if an interface element with the specified name does not exist. **Note:** Here, *`name`* represents the internal name assigned to the element when it was added to the interface and not the label that is displayed on the interface. If you specify the label instead of the name, the software will return an error.

Examples

- The following command will delete an interface element with the internal name *`Menu1`*:

  ```
  gui_delete_ui Menu1
  ```

Related Information

- gui_add_ui
- gui_find_ui
- gui_get_ui
- gui_set_ui

gui_delete_user_bundle_net

```
gui_delete_user_bundle_net
[-help]
name
```

Deletes the specified flightline net bundle. If a net is part of multiple net bundles and you delete one of the net bundles, other net bundles containing the net are not affected.

You can create flightline net bundles using the `gui_create_user_bundle_net` command and display them using the `gui_show_user_bundle_nets` command.

Note: `gui_delete_user_bundle_net` removes the specified net bundle. If you want to retain the net bundle but just want to clear its flightlines from the display area, use the `-remove` option of `gui_show_user_bundle_nets` instead.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_delete_user_bundle_net` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_delete_user_bundle_net`
name	Specifies the name of the flightline net bundle to be removed.

Examples

- The following command deletes the flightline net bundle `bundle_1`:

  ```
  gui_delete_user_bundle_net bundle_1
  ```

Related Information

- `gui_create_user_bundle_net`
- `gui_show_user_bundle_nets`

gui_deselect

```
gui_deselect
[-help]
{-all  |
-rect {x1 y1 x2 y2} |
-point {x y} |
-line {x1 y1 x2 y2}}
```

Deselects objects as per the parameters you specify. If `-all` is specified, deselects all selected objects.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_deselect` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_deselect`
`-all`	Deselects all objects.
`-line {x1 y1 x2 y2}`	Deselects objects through which the specified line passes.
`-point {x y}`	Deselects objects at the specified point.
`-rect {x1 y1 x2 y2}`	Deselects objects within the specified box coordinates.

Related Information

- deselect_obj
- gui_select
- select_obj

gui_dim_foreground

```
gui_dim_foreground
[-help]
-light_level {default medium dark}
```

Dims the display area to the specified level. Use this command to view selected and highlighted objects clearly against the dimmed foreground. This command is especially useful when you are debugging.

The equivalent bindkey is F12.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_dim_foreground` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_dim_foreground`.
`-light_level {default medium dark}`	
	Specifies the brightness level of the foreground in the display area.

Related Information

- `gui_highlight`
- `gui_select`

gui_find_ui

```
gui_find_ui
[-help]
[name]
[-before name]
[-checked true | false]
[-command string]
[-disabled true | false]
[-enable_variable string]
[-foreground string]
[-icon filename]
[-in parent_name]
[-label string]
[-menu_separate true | false]
[-movable true | false]
[-newline true | false]
[-shortcut string]
[-tooltip string]
[-type {menu toolbar submenu command check radio separator toolbutton}]
[-underline index]
[-value string]
[-variable string]
[-visible_variable string]
```

Enables you to retrieve an element's name by specifying a property-value combination.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_find_ui` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_find_ui`

name	Specifies the name of the parent element to define the search scope. If multiple elements are found with the same *name*, the first matched element is searched. If *name* is not specified, a global search is performed. *Data_type*: string, optional
-before *name*	Returns the element that is positioned just before the element identified by *name*. *Data_type*: string, optional
-checked {true | false}	Returns the names of check boxes and radio buttons with the specified checked state. *Data_type*: bool, optional
-command *string*	Returns the name of the interface element that executes the specified command string. **Note:** You can use the -command parameter only for elements of type command, check, radio or toolbutton. *Data_type*: string, optional
-disabled {true | false}	Returns the name of the interface elements with the specified state. **Note:** You can use the -disabled parameter only for elements of type menu, submenu, command, check, radio or toolbutton. *Data_type*: bool, optional
-enable_variable *string*	Returns the name of the interface elements with the specified Enable status. *Data_type*: string, optional
-foreground *string*	Returns the name of the tool button with the specified foreground color of the icon. **Note:** You can use the -foreground parameter only for elements of type toolbutton. *Data_type*: string, optional

`-icon` *`filename`*	Returns the name of the tool button with the specified image file in its icon. *`filename`* can be the full path of the icon file or an icon name supported by the tool. The icon names used in the tool can be queried using the `gui_get_ui` command. **Note:** The `-icon` parameter is applicable only for elements of type `toolbutton`. *Data_type*: string, optional
`-in` *`parent_name`*	Returns the name of the interface elements with the specified parent name. *Data_type*: string, optional
`-label` *`string`*	Returns the name of the interface element with the specified label. **Note:** This parameter is NOT applicable for elements of type `separator`. *Data_type*: string, optional
`-menu_separate true | false`	
	Returns the menus or submenus with the specified tear-off status. **Note:** This parameter is applicable for elements of type `menu` and `submenu`. *Data_type*: bool, optional
`-movable true | false`	Returns whether the object is movable or not. *Data_type*: bool, optional
`-newline {true | false}`	Returns the name of the toolbars with the specified newline status. **Note:** The `-newline` parameter is applicable only for elements of type `toolbar`. *Data_type*: bool, optional

`-shortcut` *`string`*	Returns the name of the interface elements with the specified shortcut key. **Note:** You can use the `-shortcut` parameter only for elements of type `command`, `check`, `radio`, or `toolbutton`. *Data_type*: string, optional
`-tooltip` *`string`*	Returns the name of the toolbar or toolbutton with the specified tooltip text. **Note:** The `-tooltip` parameter is applicable only for elements of type `toolbar` and `toolbutton`. *Data_type*: string, optional
`-type {menu | toolbar | submenu | command | check | radio | separator | toolbutton}`	
	Returns the name of the interface elements of the specified type. *Data_type*: enum, optional
`-underline` *`index`*	Returns the name of the interface elements that has character with the specified integer index underlined in the element label. The index begins with `0`, so for example, to find elements that have the third character in their label underlined, specify an index value of 2. *Data_type*: int, optional
`-value` *`stringValue`*	Returns the name of the radio button with the specified value. **Note:** The `-value` parameter is applicable only for elements of type `radio`. *Data_type*: string, optional
`-variable` *`string`*	Returns the name of the check boxes or radio buttons with the specified the Tcl global variable. *Data_type*: string, optional

`-visible_variable` *`string`*	Returns the name of the interface elements with the specified Visible status. *Data_type*: string, optional

Examples

- The following command returns the names of all the menus in the main window:
  ```
  gui_find_ui main -type menu
  ```
- The following command returns the names of detachable submenus (that is, with `menu_separate` setting as `true`) in the *View* menu that are detachable main window:
  ```
  gui_find_ui view -menu_separate true
  ```
- The following command returns the name of the element before the Help menu in the main window:
  ```
  gui_find_ui -before help
  ```
- The following command returns the name of the element with shortcut key F:
  ```
  gui_find_ui -shortcut F
  ```
  The software returns the following:
  ```
  tbView.ac.view.fit
  ```

Related Information

- `gui_add_ui`
- `gui_delete_ui`
- `gui_get_ui`
- `gui_set_ui`

gui_fit

```
gui_fit [-help]
```

Fits the entire design in the viewable window. You can use this command at any stage of the design flow to view the design in entirety.

Parameters

`-help`	Prints a brief description of the `gui_fit` command. For a detailed description of the command, use the `man` command: `man gui_fit`.

Related Information

- gui_zoom

gui_get_edge_number

```
gui_get_edge_number
[-help]
[-coordinate]
[-edge index_num]
-objects name|inst_ptr hinst_ptr
```

Returns the total number of edges and/or edge coordinates for the specified block.

You can use the related `gui_show_edge_number` command to display edge numbers on the sides of the selected or specified objects in the GUI.

Parameters

`-help`	Prints a brief description of the `gui_get_edge_number` command. For a detailed description of the command, use the `man` command: `man gui_get_edge_number`.
`-coordinate`	Returns the coordinates of all the edges. *Data_type*: bool, optional
`-edge` *`index_num`*	Returns the coordinates of the specified edge for the specified object. The *`index_num`* value should be greater than or equal to 0. *Data_type*: int, optional
`-objects` *`name|inst_ptr hinst_ptr`*	
	Specifies the object(s) for which edge numbers and/or coordinates are to be returned. You can specify the object(s) either by name or as inst/hinst pointers. *Data_type*: (inst|hinst)+, required

Examples

- The following command returns the total number of edges of the specified instance:

  ```
  gui_get_edge_number -objects
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U202
  ```

  Returns:

  ```
  4
  ```

- The following command returns the coordinates of all the edges of the specified instance:

  ```
  gui_get_edge_number -objects
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U202 -coordinate
  ```

  Returns:

  ```
  {edge0:{ 97.152000 -645.084000 97.152000 -643.896000 } edge1:{ 97.152000 -
  643.896000 98.736000 -643.896000 } edge2:{ 98.736000 -643.896000 98.736000 -
  645.084000 } edge3:{ 98.736000 -645.084000 97.152000 -645.084000 } }
  ```

- The following command returns the coordinates of edge with index number `0` for the specified instance:

  ```
  gui_get_edge_number -objects
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U202 -edge 0
  ```

  Returns:

  ```
  {97.152000 -645.084000 97.152000 -643.896000}
  ```

Related Information

- `gui_show_edge_number`

gui_get_objects_from_schematic

```
gui_get_objects_from_schematic
[-name string]
-type {all insts modules nets ports}
```

Gets the names of all or the specified type of objects from the current (active) or specified schematic view.

Parameters

`-help`	Prints a brief description of the `gui_get_objects_from_schematic` command. For a detailed description of the command, use the `man` command: `man gui_get_objects_from_schematic`.
`-name` *string*	Specifies the name of the schematic view from which objects need to be returned. If `-name` is not specified, returns objects from the current schematic view. You can use the `gui_get_schematic` command to get the name of the current or all open schematics.
`-type {all insts modules nets ports}`	
	Specifies the type of the objects to be returned. If `all` is specified, returns the name of all objects from the current or specified schematic view.

Examples

- The following command returns the instances from the current schematic:

  ```
  > gui_get_objects_from_schematic -type insts
  ```

  Sample output:

  ```
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/FE_OFC1329_q02_8_
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/FE_OFC1326_q02_10_
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/TESTACT_WSRAM/assignbuf_
  709853 ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U170
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U182
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U184
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U187
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U189
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/U202
  ```

- The following command returns the nets from the schematic view named `W0`:

  ```
  > gui_get_objects_from_schematic -name W0 -type nets
  ```

  Sample output:

  ```
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/n7
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/tpo_a_9_ VSS VDD
  ```

Related Information

- gui_get_schematic
- gui_schematic_add_selected
- gui_schematic_load_view
- gui_schematic_write_view

gui_get_schematic

```
gui_get_schematic
[-help]
[-current  | -all ]
```

Gets the name of the current (active) schematic view. If `-all` is specified, returns the name of all open schematic views.

Parameters

`-help`	Prints a brief description of the `gui_get_schematic` command. For a detailed description of the command, use the `man` command: `man gui_get_schematic`.
`-all`	Gets the names of all open schematic views. *Data_type*: bool, optional
`-current`	Gets the name of the current active schematic view. *Data_type*: bool, optional

Related Information

- `gui_schematic_add_selected`

gui_get_ui

```
gui_get_ui
[-help]
[name]
[-quiet]
[ -before
 | -checked
 | -children
 | -command
 | -disabled
 | -dock_widget
 | -enable_variable
 | -foreground
 | -geometry
 | -icon
 | -in
 | -label
 | -menu
 | -menu_separate
 | -message
 | -movable
 | -newline
 | -shortcut
 | -statusbar
 | -title
 | -titlebar_color
 | -toolbar
 | -tooltip
 | -type
 | -underline
 | -value
 | -variable
 | -visible
 | -visible_variable
 | -window_status ]
```

Retrieves the current value of the specified property of the element that is being queried. Using the `gui_get_ui` command, you can query all the properties in the `gui_add_ui` command for each type. In addition, depending on the `type`, some special properties are supported too as described below:

- For elements of type `menu/submenu/toolbar`:

    - `gui_get_ui` *`name`* `-children`
      Returns a list of all the child elements of the specified interface element.

- For `main` (main window):
    - `gui_get_ui main -menu`
      Returns a list of all the menus in the main window.
    - `gui_get_ui main -toolbar`
      Returns the list of all the toolbars in the main window.
    - `gui_get_ui main -statusbar`
      Returns the status bar in the main window.
    - `gui_get_ui main -geometry`
      Returns the geometry of the main window in the format *`width`*x*`height`*+*`x`*+*`y`*.
    - `gui_get_ui main -title`
      Returns the title of the main window.
    - `gui_get_ui main -titlebar_color`
      Returns the color of the title bar of the main window.
    - `gui_get_ui main -closecmd`
      Returns the close confirmation command of the main window.
    - `gui_get_ui main -visible`
      Returns the visibility status of the main window.
- For `statusbar`:
    - `gui_get_ui name -message`
      Returns the current message of the status bar.
    - `gui_get_ui name -visible`
      Returns the visibility status of the status bar.
    - `gui_get_ui name -in`
      Returns the parent element of the status bar.

`statusbar` also supports the `-enable_variable` and `-visible_variable` parameters.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_get_ui` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_get_ui`
`name`	Specifies as a string the name of the interface element you want to query. If multiple widgets are found with the same *`name`*, the first matched widget is queried. Returns an error if there is no element with the specified name. *Data_type*: string, optional
`-before`	When specified, returns the element that occurs before the element identified by *`name`*. *Data_type*: string, optional
`-checked`	Returns the on/off state of the specified check box or radio button. If the check box or radio button is selected, returns `true`. If it is deselected, returns `false`. *Data_type*: bool, optional
`-children`	Returns child elements of specified interface element. **Note:** You can use this parameter with elements of type `menu`, `submenu`, and `toolbar`. *Data_type*: string, optional
`-command`	Returns the Tcl command to be executed when the specified element you are adding is invoked. **Note:** You can use the `-command` parameter only with elements of type `command, check, radio` or `toolbutton`. *Data_type*: string, optional

`-disabled`	When specified, returns whether the specified interface element is enabled (`false`) or disabled (`true`). **Note:** You can use the `-disabled` parameter only with elements of type `menu, submenu, command, check, radio` or `toolbutton`. The parameter is NOT applicable for elements of type `toolbar` and `separator`. *Data_type*: bool, optional
`-dock_widget`	Returns the dock widget in the window. *Data_type*: string, optional
`-enable_variable`	Represents the Tcl global variable for Enable status. *Data_type*: string, optional
`-foreground`	Returns the foreground color of the icon for the specified toolbutton. **Note:** You can use the `-foreground` parameter only with elements of type `toolbutton`. *Data_type*: string, optional
`-geometry`	Returns the width, height, and coordinates of the main window. **Note:** The `-geometry` parameter is applicable only for the main window, which is type `main`. *Data_type*: string, optional
`-icon`	Returns the file name of the specified toolbutton. **Note:** The `-icon` parameter is applicable only for elements of type `toolbutton`. *Data_type*: string, optional
`-in`	Returns the name of the parent element. *Data_type*: string, optional
`-label`	Returns the label of the specified element. **Note:** This parameter is NOT applicable for elements of type `separator`. *Data_type*: string, optional

`-menu`	Returns the names of the menus in the main window. **Note:** This parameter is applicable only for the main window, which is type `main`. *Data_type*: string, optional
`-menu_separate`	Returns the tear-off status of the specified menu or submenu. **Note:** This parameter is applicable for elements of type `menu` and `submenu`. *Data_type*: bool, optional
`-message`	Returns the message on the status bar. **Note:** This parameter is applicable only for the status bar. *Data_type*: string, optional
`-movable`	Returns whether object is movable or not. *Data_type*: bool, optional
`-newline`	Returns whether or not the specified toolbar starts in a new row. **Note:** The `-newline` parameter is applicable only for elements of type `toolbar`. *Data_type*: bool, optional
`-quiet`	Displays the current settings for the specified parameters in the Tcl list format. *Data_type*: bool, optional
`-shortcut`	Returns the shortcut key for a command, check box, radio button, or toolbutton. **Note:** You can use the `-shortuct` parameter only with elements of type `command`, `check`, `radio`, or `toolbutton`. *Data_type*: string, optional
`-statusbar`	Returns the name of the main window's status bar. **Note:** This parameter is applicable only for the main window (`main`). *Data_type*: string, optional

`-title`	Returns the text on the main window's title bar. **Note:** This parameter is applicable only for the main window (`main`). *Data_type*: string, optional
`-titlebar_color`	Gets the color of the title bar of the window. **Note:** This parameter is applicable only for the main window (`main`). *Data_type*: string, optional
`-toolbar`	Returns the toolbars in the window. *Data_type*: string, optional
`-tooltip`	Returns the tooltip text for the specified toolbar or toolbutton. **Note:** The `-tooltip` parameter is applicable only for elements of type `toolbar` and `toolbutton`. *Data_type*: string, optional
`-type`	Returns the type of the specified element. *Data_type*: enum, optional
`-underline`	Returns the integer index of the underlined character in the element label. **Note:** The `-underline` parameter is NOT applicable for elements of type `toolbar, separator,` and `toolbutton`. *Data_type*: int, optional
`-value`	Returns the value stored in the associated variable of the specified radio button. **Note:** The `-value` parameter is applicable only for elements of type `radio`. *Data_type*: string, optional
`-variable`	Returns the Tcl global variable for the specified check box or radio button. *Data_type*: string, optional
`-visible`	Returns the visibility status of the element. *Data_type*: string, optional

`-visible_variable`	Represents the Tcl global variable for Visible status. *Data_type*: string, optional

`-window_status`	Returns the widget window status. This option returns one of the following values, depending on the window status, assuming that tool was launched in the GUI mode:
	• `Minimized`
	• `Maximized`
	• `Normal`
	For example, if the main window is in the default window mode, and you run the following command:
	`gui_get_ui main -window_status`
	the software returns:
	`Normal`
	If the window is in the background, you can use the following command to bring the window to the forefront:
	`gui_set_ui main -visible 1`
	Data_type: string, optional

Examples

- The following command returns the names of all the menus in the main window:
  ```
  gui_get_ui main -menu
  ```
- The following command returns the window status of the main window:
  ```
  gui_get_ui main -window_status
  ```
  The software returns the following (assuming the window is in the maximized mode at the time):
  ```
  Maximized
  ```
- The following command returns the tear-off status of the *View* menu:
  ```
  gui_get_ui view -menu_separate
  ```
  The software returns the following:
  ```
  true
  ```

Related Information

- gui_add_ui
- gui_delete_ui
- gui_find_ui
- gui_set_ui

gui_get_visible_bumps

```
gui_get_visible_bumps
[-help]
```

Returns the names of all visible bump cells in the design if some bumps have been hidden in the design.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_get_visible_bumps` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_get_visible_bumps`

Examples

- The following command returns the name of all visible bumps in the designs:
  ```
  gui_get_visible_bumps
  ```

Related Information

- `gui_set_visible_bumps`

gui_get_visible_net_groups

```
gui_get_visible_net_groups
[-help]
```

Returns the names of the visible net groups if some net groups have been hidden in the design. If all net groups are visible, this command has no effect.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_get_visible_net_groups` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_get_visible_net_groups`

Examples

- The following command returns the name of all visible net groups in the designs:

  ```
  gui_get_visible_net_groups
  test_ng
  ```

Related Information

- `gui_set_visible_net_groups`

gui_get_visible_nets

```
gui_get_visible_nets
[-help]
```

Returns the names of the visible nets if some nets have been hidden in the design. If all nets are visible, this command has no effect.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_get_visible_nets` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_get_visible_nets`

Examples

- The following command hides all nets except the specified net:

  ```
  > gui_get_visible_nets
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/tpo_a_9_
  ```

Related Commands

- `gui_set_visible_nets`

gui_highlight

```
gui_highlight
[-help]
[<any_object>+]
[-index indexValue | -auto_color  | -original]
[-color colorValue | -auto_color  | -original]
[-pattern patternValue | -original]
```

Highlights selected or specified objects with the Highlight Set available through the *View - Highlight Selected* menu. The value for the index corresponds to the Highlight Set that will be applied. For example, a value of 4 for the index specifies that all selected objects will be highlighted with the same color and pattern as available for the *Highlight Set #4* in the *View - Highlight Selected* menu. You can use this command at any stage in the design flow.

You can also provide a specific color and/or pattern to be used. All selected objects will be highlighted with the color and/or pattern that you specify.

If you specify a color and or pattern, the Highlight Set corresponding to the index value will be updated to use the color and pattern specified. For example, if you specify an index value of 5, color `green`, and pattern `backslash`, the *Highlight Set #5* will get updated to have the color `green` and the pattern `backslash`.

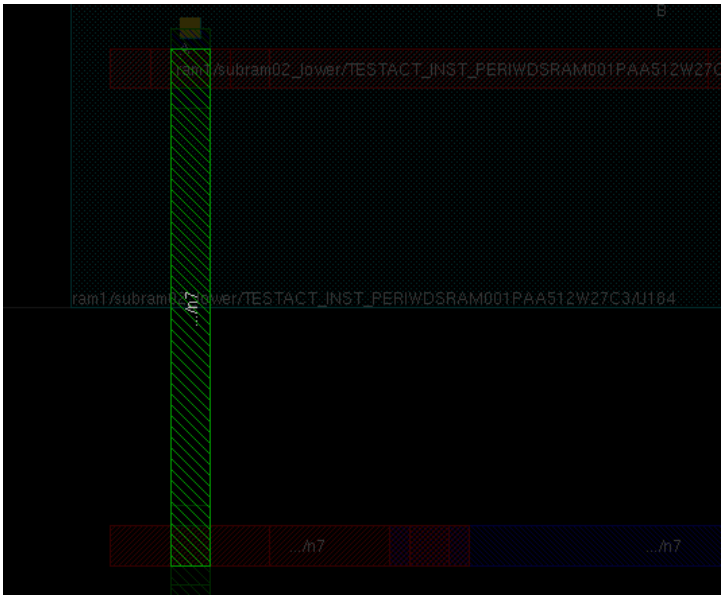
You can remove highlight from the highlighted objects through the `gui_clear_highlight` command or through the *View - Clear Highlight* menu.

Note: If an object is selectable, you can highlight it using the `gui_highlight` command.

Use the `gui_highlight_pin` command to highlight shapes for specified SIGNAL/POWER pins.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_highlight` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_highlight`
any_object+	Specifies the object pointer list for highlighting.
`-auto_color`	Specifies the object color automatically. **Note**: This parameter cannot be used with `-index` or `-color`.

`-index` *`indexValue`*	Specifies the index value for the specified object. All objects that are selected will be highlighted with the same color and pattern as defined in corresponding Highlight Set in the *View - Highlight Selected* menu As an example, if you specify a value of 3 for index, all objects that are selected will be highlighted with the same color and pattern as available for the *Highlight Set #3* in the *View - Highlight Selected* menu. The value of index should be between 1 to 64.
`-color` *`colorValue`*	Specifies the color value for the specified object. The color value is the same as that available through the Edit Highlight Color form (*View - Edit Highlight Color*). You can also use the Red Green Blue (RGB) value in the following format: #*`V1V2V3`* where V1, V2, and V3 are the hexadecimal values for the red, green, and blue color components respectively. For example, you can specify the value `#CC66FF`. *Default:* The color defined for the Highlight Set corresponding to the index value is used.
`-original`	Highlights the object in original color. This option is useful if you want to check a specific object more closely with the background dimmed. An object highlighted in original color will stand out in the dimmed background:  **Note**: This parameter cannot be used with `-index`, `-color`, or `-pattern`.

`-pattern` *`patternValue`*	Specifies the pattern value for the specified object. The pattern value is the same as the Stipple Value available through the Edit Highlight Color form. *Default:* The pattern (Stipple Value) for the Highlight Set corresponding to the index value is used.

Examples

- The following command highlights all selected objects with the same color and pattern as defined for the *Highlight Set #4* in the *View - Highlighted Selected* menu.

  ```
  gui_highlight -index 4
  ```

- The following command highlights all selected objects with the color `green` and the pattern `dot4`. The index values is set to 5 and, therefore, the command will also change the definition of the *Highlight Set #5* (*View - Highlight Selected* menu) to have color `green` and pattern `dot4`.

  ```
  gui_highlight -index 5 -color green -pattern dot4
  ```

- The following command highlights all selected objects with the color corresponding to the RGB hexadecimal value `#33FF66` and the pattern `horizontal`. The index values is set to 3 and, therefore, the command will also change the definition of the *Highlight Set #5* (*View - Highlight Selected* menu) to have color `#33FF66` and pattern `horizontal`.

  ```
  gui_highlight -index 3 -color #33FF66 -pattern horizontal
  ```

Related Information

- `gui_clear_highlight`
- `gui_highlight_pin`

gui_highlight_pin

```
gui_highlight_pin
[-help]
-inst string
-pin string
[-original | -index indexValue ]
[-original | -color colorValue ]
[-original | -pattern patternValue ]
```

Highlights the shapes of all specified SIGNAL/POWER instance pins with the Highlight Set available through the *View - Highlight Selected* menu. The value for the index corresponds to the Highlight Set that will be applied. For example, a value of 4 for the index specifies that all selected objects will be highlighted with the same color and pattern as available for the *Highlight Set #4* in the *View - Highlight Selected* menu.

You can also provide a specific color and/or pattern to be used. All specified pins will be highlighted with the color and/or pattern that you specify.

Note: If you specify a color and or pattern, the Highlight Set corresponding to the index value will be updated to use the color and pattern specified. For example, if you specify an index value of `5`, color `green`, and pattern `backslash`, the *Highlight Set #5* will get updated to have the color `green` and the pattern `backslash`.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `gui_highlight_pin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man gui_highlight_pin`

`-color` *colorValue*	Specifies the color value for the specified object. The color value is the same as that available through the Edit Highlight Color form, which can be accessed by selecting *View - Edit Highlight Color*. You can also use the Red Green Blue (RGB) value in the following format: `#`*V1V2V3* where *V1*, *V2*, and *V3* are the hexadecimal values for the Red, Green, and Blue color components, respectively. For example, you can specify the value `#CC66FF`. *Default:* The color defined for the Highlight Set corresponding to the specified index value is used.
`-index` *indexValue*	Specifies the index value for the specified object. By default, all specified pins are highlighted with the same color and pattern as defined in the *Highlight Set* corresponding to the specified index value in the *View - Highlight Selected* submenu. For example, if you specify a value of 3 for index, all specified pins are highlighted with the same color and pattern as available for *Highlight Set #3* in the *View - Highlight Selected* submenu. **Note**: The *View - Highlight Selected* submenu provides eight highlight patterns. Therefore, the value of the index should be a number from 1 to 8.
`-inst` *string*	Specifies the name of instance.
`-original`	Highlights the instance pin in original color. This option is useful if you want to check a specific instance pin more closely with the background dimmed. An object highlighted in original color will stand out in the dimmed background. **Note**: This parameter cannot be used with `-index`, `-color`, or `-pattern`.
`-pattern` *patternValue*	Specifies the pattern value for the specified object. The pattern value is the same as the *Stipple* value available through the Edit Highlight Color form. *Default:* The pattern (*Stipple* value) defined for the Highlight Set corresponding to the specified index value is used.
`-pin` *string*	Specifies the name of SIGNAL/POWER pin to be highlighted. You can specify more than one pin name.

Examples

- The following command highlights the SIGNAL pins `E` and `D` of instance `U1` using the color yellow and the solid pattern:

```
gui_highlight_pin -inst U1 -pin {E D} -color yellow -pattern solid -index 1
```

- The following command highlights the SIGNAL pin `E` and the POWER pin `gnd` of instance `U1` using the color yellow and the solid pattern:

```
gui_highlight_pin -inst U1 -pin {E gnd} -color yellow -pattern solid -index 1
```

Related Information

- `gui_clear_highlight`
- `gui_highlight`
- `gui_highlight_pin_connection`

gui_highlight_pin_connection

```
gui_highlight_pin_connection
[-help]
-from_pin pin
[-mode {wire_segment | whole_net | flight_line}]
-to_pin pin
[-with_arrow | -select_only]
[-net_color_index index_value | -select_only]
[-inst_color_index index_value | -select_only]
```

Highlights the full timing path or its segment in the text format while generating the sign-off timing report. Use this command to quickly debug the timing path (timing point information) used in the tool.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `gui_highlight_pin_connection` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man gui_highlight_pin_connection`
`-from_pin` *pin*	Specifies the name of the starting pin to be highlighted.
`-inst_color_index` *index_value*	Specifies the instance color index value (an integer value from 1 to 64) for the specified timing path.
`-mode {wire_segment | whole_net | flight_line}`	Specifies the draw mode between the starting and ending pins.
`-net_color_index` *index_value*	Specifies the net color index value (an integer value from 1 to 64) for the specified timing path.
`-select_only`	Selects only the specified path for highlighting.
`-to_pin` *pin*	Specifies the name of the ending pin to be highlighted.
`-with_arrow`	Specifies whether or not the connection is defined with an arrow.

Example

The following command highlights a line drawn from the specified input pin to output pin while generating the sign-off timing report:

```
gui_highlight_pin_connection -from_pin ffa/Q -U7/A -mode flight_line
```

The timing report generated is shown below:

:::::::Timing Report::::::::::::

```
  Startpoint: ffa (rising edge-triggered flip-flop clocked by CLK)
```

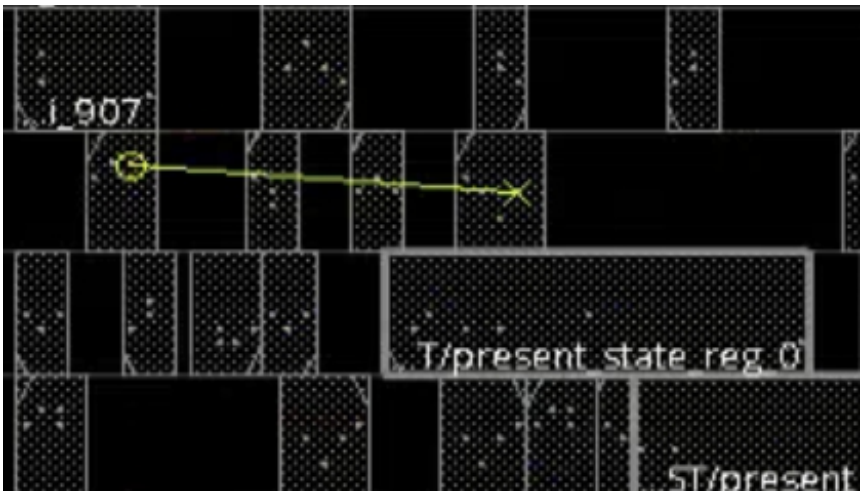
```
Endpoint: ffd (rising edge-triggered flip-flop clocked by CLK)
Path Group: CLK
Path Type: max
Point                          Fanout    Cap     Trans       Incr       Path
-------------------------------------------------------------------------------
clock CLK (rise edge)                                         0.00       0.00
clock network delay (propagated)                              4.90       4.90
ffa/CLK (DTC10)                                 0.30          0.10       5.00 r
ffa/Q (DTC10)                                   0.57          1.70       6.70 f
b (net)                          2     3.85
U7/A (IV110)                                    0.57          0.00       6.70 f
U7/Y (IV110)                                    1.32          0.84       7.55 r
QA (net)                        50    32.59
U12/A (NA310)                                   3.02          4.00      11.55 r
U12/Y (NA310)                                   2.47          4.04      15.58 f
n9 (net)                         3     8.87
U17/A (NA211)                                   2.47          0.00      15.58 f
U17/Y (NA211)                                   1.01          1.35      16.94 f
n14 (net)                        2     4.87
U23/A (IV120)                                   1.01          0.00      16.94 f
U23/Y (IV120)                                   0.51          0.37      17.30 r
n13 (net)                        1     2.59
U15/A2 (BF003)                                  0.51          0.00      17.31 r
U15/Y (BF003)                                   0.88          0.81      18.12 f
n12 (net)                        1    15.61
U16/B2 (BF003)                                  3.88          3.00      21.12 f
U16/Y (BF003)                                   2.46          0.99      22.11 r
n7 (net)                         1     2.61
```

```
  U10/A (AN220)                                  2.46       0.00       22.11 r
  U10/Y (AN220)                                  0.46       1.04       23.15 r
  n15 (net)                       1      2.63
  ffd/D (DTN10)                                  0.46       0.00       23.15 r
  data arrival time                                                    23.15

 clock CLK (rise edge)                                      10.00       10.00
  clock network delay (propagated)                           3.50       13.50
  ffd/CLK (DTN10)                                                       13.50 r
  library setup time                                        -1.33       12.17
  data required time                                                    12.17
  ---------------------------------------------------------------------------
  data required time                                                    12.17
  data arrival time                                                    -23.15
  ---------------------------------------------------------------------------
  slack (VIOLATED)                                                     -10.98

:::::::END:::::::::::
```

The drawn layout along with the highlighted line is shown in the below picture:



gui_list_marker

```
gui_list_marker
```

Gives a list of GUI markers in the design layout.

Example

- The following command shows a list of GUI markers:
  ```
  gui_list_marker
  ```
  The following is displayed:
  ```
  {name:aa  coordinate:(100.000 100.000)}
  ```

gui_open_schematic

```
gui_open_schematic
[-help]
[-name string]
[-type {module cone}]
[-objs <inst | net | pin | port | hinst>+ | -selected]
```

Opens the Schematic Viewer.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man gui_open_schematic`
`-name` *`string`*	Specifies the unique window name for the schematic view to be opened.
`-objs <inst | net | pin | port | hinst>+`	
	Specifies the object(s) to be added in the schematic view. If multiple objects are specified, the Schematic Viewer can be opened only in the flattened mode (`-type cone`). As the default schematic mode is hierarchical, this means you must always specify `-type cone` when specifying multiple objects. **Note**: This option cannot be used with `-type module`. *Data_type*: (*inst*|*net*|*pin*|*port*|*hinst*)+, optional
`-selected`	Specifies that the selected object(s) are to be added in the schematic view. If multiple objects are selected, the Schematic Viewer can be opened only in the flattened mode (`-type cone`). As the default schematic mode is hierarchical, this means you must always specify `-type cone` when selecting multiple objects. **Note**: This option cannot be used with `-objs` or `-type module`. *Data_type*: bool, optional
`-type {module cone}`	Specifies the type of schematic to be opened. If you specify `cone`, the tool opens the Schematic Viewer to display schematic in the flattened mode. If you specify `module`, the tool opens the Schematic Viewer to display schematic in the hierarchical mode. *Default*: `module` *Data_type*: enum, optional

Examples

- The following command opens the schematic for the specified instance in the hierarchical mode:

  ```
  gui_open_schematic -obj DTMF_INST/PLLCLK_INST
  ```

- The following command opens the schematic for the specified instance in the flattened mode:

  ```
  gui_open_schematic -obj DTMF_INST/PLLCLK_INST -type cone
  ```

- The following opens the schematic for the selected object in the flattened mode:

  ```
  gui_open_schematic -selected -type cone
  ```

- The following command opens the schematic for multiple selected objects in the flattened mode:

  ```
  gui_open_schematic -selected -type cone
  ```

gui_pan

```
gui_pan
[-help]
dx
dy
```

Specifies that the new center of the viewing window should shift from the current center of the viewing window by (*x*, *y*) microns. Use this command at any stage in the design flow to pan or move the design view in a specific direction.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_pan` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_pan`.
dx	Specifies that the design center should shift *dx* microns on the x-axis from the center of the viewing window. You can specify positive or negative values for *dx*. A positive value specifies shifting to the right on the x axis.
dy	Specifies that the design center should shift `dy` microns on the y axis values from the center of the viewing window. You can specify positive or negative values for *dy*. A positive value specifies shifting up on the y axis.

Example

The following example shifts the design center to the right by 200 microns and down by 300 microns from the center of the viewing window.

```
gui_pan 200 -300
```

Related Information

- gui_pan_center
- gui_pan_page

gui_pan_center

`gui_pan_center [-help] x y`

Pans in the viewable window to the center point defined by the specified coordinates. Use this command at any stage of the design flow to view the design area around a specific point.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_pan_center` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_pan_center`.
x y	Specifies the x and y coordinates of the center point.

Example

The following command pans the viewable window to the center point (x=100, y=200)

```
gui_pan_center 100 200
```

Related Information

- `gui_pan`
- `gui_pan_page`

gui_pan_page

`gui_pan_page [-help] x y`

Pans the viewable window in pages defined by the offsets. The size of each page is equal to the size of the current viewable window. Use this command at any stage of the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_pan_page` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_pan_page`.
x y	Specifies `x` and `y` offsets to indicate number of pages to pan. A positive value of *x* specifies a pan of *x* pages to the right. A positive number of *y* specifies a pan of *y* pages toward the top. You can specify integer values for *x* and *y*.

Examples

- The following command pans the display to the right by one page
  `gui_pan_page 1 0`
- The following command pans the display to the left by one page
  `gui_pan_page -1 0`

Related Information

- `gui_pan`
- `gui_pan_center`

gui_redraw

```
gui_redraw [-help]
```

Refreshes the display in the viewable window. This command performs the same function as the *Redraw* icon (CTRL+R keyboard shortcut). You can use this command at any stage in the design flow.

Parameters

`-help`	Prints a brief description of the `gui_redraw` command. For a detailed description of the command, use the `man` command: `man gui_redraw`.

gui_remove_marker

```
gui_remove_marker
[-help]
{-name <string> | -all }
```

Removes a marker from the design layout.

Parameters

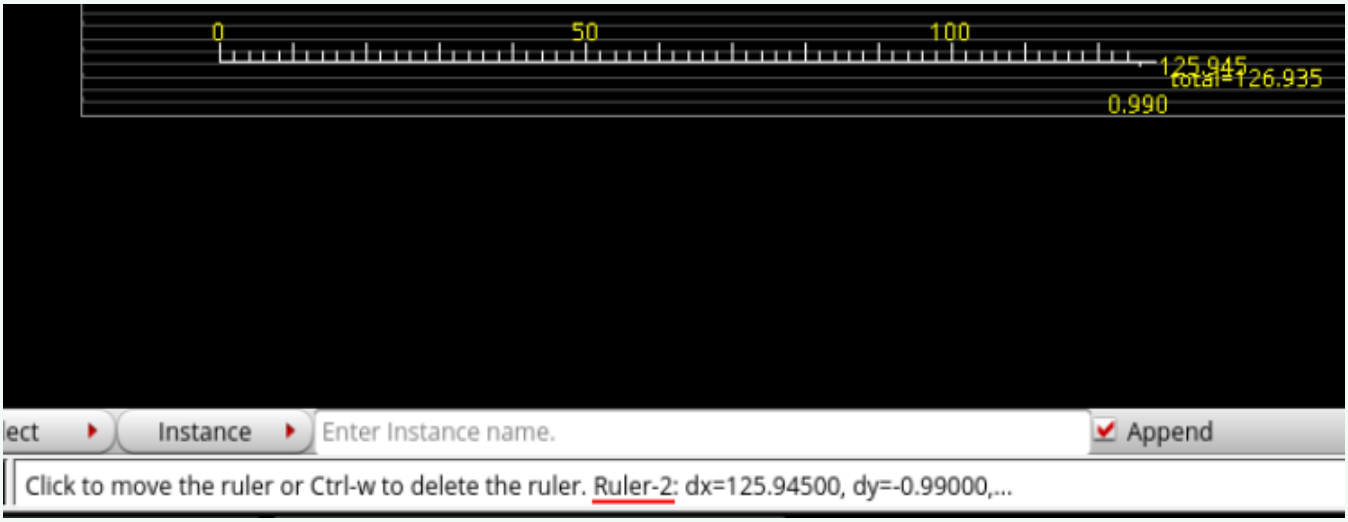
`-help`	Outputs the command usage and a brief description about the command parameters.
`-name <string>`	Specifies the name of the marker to be removed from the design layout.
`-all`	Removes all markers from the design layout.

gui_report_ruler

```
gui_report_ruler -help
```

Reports the coordinates for all rulers in the main window.

To check the ID of a ruler, select the *Create Ruler* widget and hover the cursor over the ruler – the ruler ID is displayed on the status bar, as shown below.



Parameters

`-help`	Prints a brief description of the `gui_report_ruler` command. For a detailed description of the command, use the `man` command: `man gui_report_ruler`

Example

- The following command reports the coordinates of all rulers:

  ```
  > gui_report_ruler
  ```

  The command returns the ruler coordinates, as shown below:

  ```
  {Ruler-1 coords: {248.4450 146.9800} {247.4550 12.6400}} {Ruler-0 coords: {13.8450 226.4950} {87.9300 225.0150}}
  ```

Related Information

- clear_all_rulers
- create_ruler

gui_schematic_add_selected

```
gui_schematic_add_selected
[-help]
[-name window_name]
```

Adds the selected object to the current or specified schematic view.

Parameters

`-help`	Prints a brief description of the `gui_schematic_add_selected` command. For a detailed description of the command, use the `man` command: `man gui_schematic_add_selected`.
`-name` *`window_name`*	Specifies the unique window name for an open schematic view to which the selected object needs to be added. *Data_type*: string, optional

Related Information

- `gui_get_schematic`

gui_schematic_load_view

```
gui_schematic_load_view
[-help]
-file file_name
[-name name]
```

Loads the specified schematic file.

> This commands work only in the cone view (flattened mode).

Parameters

`-help`	Prints a brief description of the `gui_schematic_load_view` command. For a detailed description of the command, use the `man` command: `man gui_schematic_load_view`.
`-file` *file_name*	Specifies the name of the schematic file to be loaded. *Data_type*: string, required
`-name` *name*	Specifies a unique window name for the opened schematic view. *Data_type*: string, optional

Related Information

- `gui_open_schematic`
- `gui_schematic_write_view`

gui_schematic_write_view

```
gui_schematic_write_view
[-help]
-file file_name
[-name string]
```

Saves the current schematic view into the specified file.

> This commands work only in the cone view (flattened mode).

Parameters

`-help`	Prints a brief description of the `gui_schematic_write_view` command. For a detailed description of the command, use the `man` command: `man gui_schematic_write_view`.
`-file file_name`	Specifies the name of the file in which to save the schematic view. *Data_type*: string, required
`-name string`	Specifies the name of the schematic view. *Data_type*: string, optional

Related Information

- `gui_open_schematic`
- `gui_schematic_load_view`

gui_select

```
gui_select
[-help]
{-point {x y} |
-line {x1 y1 x2 y2} |
-rect {x1 y1 x2 y2}}
[-toggle  | -append  | -next]
[-point {x y} [-next ]]
```

Selects objects as per the parameters you specify. Use `-point` to select the object at the specified coordinate, `-line` to select the objects through which the specified line passes, or `-rect` to select the objects falling within the specified box coordinates.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_select` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_select`
`-append`	Appends the selection. This is equivalent to pressing the `Shift` key while selecting an object using the GUI.
`-line {x1 y1 x2 y2}`	Selects by line. Use this parameter to select the objects through which the specified line passes.
`-next`	Steps to next object under point. This is same as using space bar while selecting an object using the GUI.
`-point {x y}`	Selects the object at the specified point.
`-rect {x1 y1 x2 y2}`	Selects by box. Use this parameter to select the objects falling within the specified rectangular area.
`-toggle`	Toggles the selection. This is equivalent to pressing the `Ctrl` key while selecting an object using the GUI.

Examples

- The following command selects the objects that are totally enclosed within the box defined by the coordinates (`x = 10, y = 200`) and (`x = 100, y = 50`):
  ```
  gui_select -rect {10 200 100 50}
  ```
- The following command selects all the objects falling across the specified line coordinates:
  ```
  gui_select -line {168.654 994.049 688.129 436.224}
  ```
- The following command selects the objects crossed by the specified line and appends them to the current selection:
  ```
  gui_select -line {169.622 996.049 698.290 446.240} -append
  ```
- The following command toggles the selection on the objects in the specified bounding box. If the objects are previously selected, the command deselects them and vice versa:
  ```
  gui_select -rect {10 200 100 50} -toggle
  ```

Related Information

- `deselect_obj`
- `gui_deselect`
- `select_obj`

gui_select_highlighted

```
gui_select_highlighted
[-help]
[-type string]
```

Selects highlighted objects. If `-type` is specified, selects highlighted objects of the specified type.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_select_highlighted` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_select_highlighted`
`-type` *`string`*	Specifies the type of the highlighted objects to be selected. Options are `instance`, `wire`, and `via`.

Examples

- The following command selects all highlighted instances.
  ```
  gui_select_highlighted -type instance
  ```

Related Commands

- `gui_highlight`
- `gui_select`

gui_set_obj_color

```
gui_set_obj_color
[-help]
{[-obj_name InstOrHInstName | -obj_type typeList]
[-multicolor | -color_id colorID]
[-include_children {off | color | multicolor}]
[-reset]}
```

Sets the color for objects of the specified name or type. You can choose to include children when specifying object color for a hierarchical instance (HInst).

The `gui_set_obj_color` command is typically used during floorplanning to color objects that are currently displayed in the design display area. You can use this command to distinguish between objects of different type or between objects belonging to different instances, hierarchical instances, instance groups, or power domains.

Before using the `gui_set_obj_color` command, you should have:

- Loaded the floorplan
- Ungrouped objects so that only required objects are displayed in the design display area in the Floorplan or Physical view.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_set_obj_color` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_set_obj_color`.
`-color_id` *`colorID`*	Specifies the color ID from 0 to 29. The color value for the color ID is the same as specified in the Module Color Preference form (accessed by clicking *Floorplan - Edit Floorplan - Color Module*). *Default*: Color ID `0`
`-include_children {off | color | multicolor}`	Specifies whether the children of the hierarchical instance will be colored simultaneously when you specify a color for an HInst. You can set one of the following options: • `off`: The child instances will not be colored. • `color`: The child instances are colored the same color as their parent instance. • `multicolor`: The child instances are colored in multiple colors.
`-multicolor`	Uses multiple colors to color objects of the specified type, instance, or hierarchical instance.
`-obj_name` *`InstOrHInstName`*	Specifies the name of the instance, hierarchical instance, power domain, or instance group whose objects are to be colored. You can specify an object list, such as `objectName {A,B}`.
`-obj_type` *`typeList`*	Specifies the list of type of objects that are to be colored, such as `SDP`, `BusGuide`, `Module`, `PowerDomain`, and `InstGroup`.
`-reset`	Resets the object color.

Example

- The following command colors all SDP objects in multiple colors:
  `gui_set_obj_color -obj_type SDP -multicolor`
- The following command colors the `DTMF_INST/TDSP_CORE_INST` instance with color `3`, as

specified in the Module Color Preference form:

```
gui_set_obj_color -obj_name DTMF_INST/TDSP_CORE_INST -color_id 3
```

- The following command colors the `DTMF_INST/TDSP_CORE_INST` instance and its children with color `3`, as specified in the Module Color Preference form:

  ```
  gui_set_obj_color -obj_name DTMF_INST/TDSP_CORE_INST -color_id 3 -include_children color
  ```

- The following command colors the children of the `DTMF_INST/TDSP_CORE_INST` instance in multiple colors:

  ```
  gui_set_obj_color -obj_name DTMF_INST/TDSP_CORE_INST -include_children multicolor
  ```

gui_set_power_rail_display

```
gui_set_power_rail_display
[-help]
[-enable_result_browser {true | false}]
[-enable_rlrp {true | false}]
[-enable_voltage_sources {true | false}]
[-filter_max max_value]
[-filter_min min_value]
[-legend {off ne nw se sw}]
[-plot plot_type]
[-range_max max_value]
[-range_min min_value]
[-reset_color_scale {true | false}]
[-filter_color {on | off | color_name}]
[-enable_percentage_range {true | false}]
[-thermal_layer_name thermal_layer]
[-tile_count_x value]
[-tile_count_y value]
[-tile_size_x value]
[-tile_size_y value]
[-customized_range {value1 value2 ... value9}]
[-grid {on | off}]
[-enable_auto_apply {true | false}]
```

Controls the display of the main window widgets, such as legend, result browser, voltage source, filter range, and RLRP path.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_set_power_rail_display` parameter. For a detailed description of the command and all of its parameters, use the man command: `man gui_set_power_rail_display`.
`-customized_range {value1 value2 ... value9}`	

	Specifies the color scale range values for the 8 filters between the minimum and maximum values of the color gradient based (*Basic*) filter. The value list must contain 9 values. The filter ranges for the 8 buckets are evenly divided by default. This parameter can be used to customize the filter ranges. You can either modify the color range using the GUI (Basic button) or the `-customized_range` parameter. **Example**: `gui_set_power_rail_display -plot ir -customized_range { 1 10 12 13 14 15 16 17 49 }`
`-enable_auto_apply {true | false}`	
	Specifies to automatically redraw the layout after modifying the color scale range values. This parameter allows you to enable/disable the *Auto Apply for Color Scale* checkbox of the Power Rail Plots form using the Tcl script instead of the GUI option. The default value of this parameter is `false`.
`-enable_percentage_range {true | false}`	
	Specifies to display the color gradients for the following plots by percentage in the GUI: • IR Drop • IVDD • IVDN The percentage value is based on the nominal voltage value of each rail/instance. It displays high percentage voltages in red and low percentage voltages in blue. *Default* : `false`
`-enable_result_browser {true | false}`	
	When set to `true` it enables the result browser. *Default* : `false`
`-enable_rlrp {true | false}`	

	When set to `true` it enables the display of the RLRP path. The rail analysis must include RLRP analysis (`set_rail_analysis_config -enable_rlrp_analysis true`) in order to have the required data for this functionality. *Default* : `false`
`-enable_voltage_sources {true | false}`	
	Displays voltage sources (power pads) using white pixels in the layout canvas. *Default* : `false`
`-filter_color {on | off | color_name}`	
	Allows to set the color of the filtered out resistors specified using the scissor filter range. By default, the filtered resistors are not displayed. If you want to view these resistors, you can use this parameter to enable the visibility of resistors and set desired colors for them. The valid values of this parameter are: • `on` – enables the visibility of the filtered out resistors. • `off` - disables the visibility of the filtered out resistors. • *`color_name`* - enables the visibility of filtered out resistors and set the color of resistors per the specified color name.
`-filter_max` *`max_value`*	
	Specifies the maximum value of the scissor filter range that will be plotted. You can use the scissor filter range to filter out the resistors and nodes that you do not want to view, and obtain statistics only for the instances in the specified filter range. Optional, but if specified, the `-plot` option is required, and requires both `-filter_min` and `-filter_max` to be specified.
`-filter_min` *`max_value`*	
	Specifies the minimum value of the scissor filter range that will be plotted. Optional, but if specified, the `-plot` option is required, and requires both `-filter_min` and `-filter_max` to be specified.
`-grid {on | off}`	

	Allows you to control the grid display when viewing the Voltage Source Current (VC) and Voltage across Package Model (VU) plots. The default value of this parameter is `on`. You can set this parameter to `off` to hide the grid view for better display and easier analysis of the plot data.
`-legend {off ne nw se sw}`	
	Displays the filter color and range details. When you set `set_power_rail_display -legend` to one of the location (`ne nw se sw`) , this opens the legend at the specified location on the design layout. Use `set_power_rail_display -legend off` to close the legend. *Default*: `off` .
`-plot` *plot_type*	
	Plots the specified plot type. See Plot Types for a description of each of the plot types.
`-range_max` *max_value*	
	Specifies the maximum value of the color scale. Optional, but if specified, the `-plot` option is required, and requires both `-range_min` and `-range_max` to be specified.
`-range_min` *min_value*	
	Specifies the minimum value of the color scale. Optional, but if specified, the `-plot` option is required, and requires both `-range_min` and `-range_max` to be specified.
`-reset_color_scale {true | false}`	
	Specifies to reset the maximum (`-range_max`) and minimum (`-range_min`) values of the color scale to the default plot data values. For example, when you specify `-plot ir`, the default value for `-range_min` is `0` and `-range_max` is `100`. You can modify the color scale, such as set `-range_min` to `0` and `-range_max` to `20`. To reset the color scale of the IR drop plot to default, you can specify the `-reset_color_scale` parameter.
`-thermal_layer_name` *thermal_layer*	

	Displays the specified layer of the temperature map. **Example:** `set_power_rail_display -thermal_layer_name VIA_7`
`-tile_count_x` *value*	

	Specifies the number of tiles in the x direction. This parameter is used to resize the tile power density plot.
	You can either specify the tiles by count (`tile_count_x` and `tile_count_y`) or specify the tiles by size (`tile_size_x` and `tile_size_y`). When you resize the tile power density (`tpd`) plot in the batch or GUI mode, the software automatically invokes the `report_power` command and resizes the plot. This feature allows you to resize the `tpd` plot without having to restore the power database (`power.db`) again.
	The following is an example of specifying tiles by size in the batch mode:
	`report_power -out_file tiles_count.rpt -power_density_tiles_row_col {100 100}`
	`set_power_rail_display -plot tpd -tile_size_x 100 -tile_size_y 100`

`-tile_count_y` *value*	
	Specifies the number of tiles in the y direction. This parameter must be specified with the `tile_count_x` parameter.
`-tile_size_x` *value*	

	Specifies the horizontal dimension of each tile polygon in um. This parameter is used to resize the tile power density plot.
	You can either specify the tiles by count (`tile_count_x` and `tile_count_y`) or specify the tiles by size (`tile_size_x` and `tile_size_y`). When you resize the tile power density (`tpd`) plot in the batch or GUI mode, the software automatically invokes the `report_power` command and resizes the plot. This feature allows you to resize the `tpd` plot without having to restore the power database (`power.db`) again.
	The following is an example of specifying tiles by size in the batch mode:
	`report_power -out_file tiles_count.rpt -power_density_tiles_row_col {100 100}`
	`set_power_rail_display -plot tpd -tile_size_x 100 -tile_size_y 100`

`-tile_size_y` *value*	

	Specifies the vertical dimension of each tile polygon in um. This parameter must be specified with the `tile_size_x` parameter.

Plot Types

Plot Type	Description
`ac`	Available/feasible capacitance (Farad) Displays the feasible decoupling capacitance that can be added in the area. The feasible capacitance in the area is calculated based on the filler cells placement. The program matches the placed filler cells with the decap cells available in the power-grid library and computes the feasible capacitance. The capacitance of the decap cell is stored in the power-grid library.
`cap`	Ggrid capacitance (Farad) Plots the capacitance of the extracted power-grid network.
`cvi1`	Customized File 1 Plots the user-specified instance voltage file.
`dd`	Decoupling capacitance (decap) density (Farad/um ^{2}) Plots decap density of the design. The decap density is the cell internal decoupling capacitance in the unit area (µm ^{2}).
`dr`	Decoupling capacitance required Reports the decoupling capacitance required to fix dynamic IR-drop violations. This option suggests how much decap needs to be added in a given area to fix the IR-drop violations.
`effr`	Effective resistance (ohm) - instance based Display the effective resistance results for all instances.
`node_reff`	Effective resistance (ohm) - node based Display the effective resistance results for all nodes.

`ipc`	Instance Peak Current Display the peak current of each instance.
`ivdn`	Instance Voltage Drop (Volts) – net-based Plots the net-based instance voltage drop.
`ivdd`	Instance Voltage Drop (Volts) – domain-based Plots the domain-based instance voltage drop. For domain based, at least one power and one ground net must be selected.
`fd`	Filler cell density ($1/\mu m^2$) Displays the filler cell density per unit area (μm^2). The plot can be used to analyze the feasibility of adding decap cells by replacing the filler cells and not changing the placement of the logic cells.
`freq`	Frequency domain (Hz) Displays the associated clock frequency of all instances in the design. The instances associated with multiple clock domains are displayed using the fastest clock frequency. This plot can be used to analyze and debug the power distribution in the design.
`ip`	Instance total power (mW) The total power consumed by each instance is displayed. The total power is the sum of the internal, switching and leakage power of the instance. The power data is read from the power database and can be specified using the `-power_db` option or it is queried directly from memory, if power calculation is run during the session.
`ip_i`	Instance internal power (mW) The power consumed by charging and discharging of the internal interconnect and device capacitances of the instance.
`ip_l`	Instance leakage power (mW) The power consumed by devices when not switching.
`ip_s`	Instance switching power (mW) The power consumed by charging and discharging of the interconnect or output load.

`ipd_i`	Specifies internal power density in mW/um ^{2} .
`ipd_l`	Specifies leakage power density in mW/um ^{2} .
`ipd_s`	Specifies switching power density in mW/um ^{2} .
`ipd_t`	Specifies total power density in in mW/um ^{2} .
`ir`	IRdrop or voltage drop (Volts) Displays the IRdrop across each extracted power-grid segment in the design.
`jrms`	Specifies current density as j/jmax based on the Irms current. This plot is only supported for dynamic analysis. A jrms gif file will be saved in the state directory that shows where the Irms based EM violation occurred. This plot type allows you to verify the analysis results of "jrms" using the GUI.
`load`	Loading capacitance (Farad) Displays the output loading capacitance driven by the instance. This plot can be used to analyze and debug the regions with high switching power.
`pi`	Power-switch (gate) current violation: I/Idsat Displays the ratio of current through the power-switch and the saturation current (Idsat) of the power-switch. This plot can be used to analyze and debug whether the current through power-switches exceeds the saturation current characterized for the power-switch devices. The saturation current and on-resistance of the power-switches are characterized and stored inside the power-grid library of the power-switch cell. A ratio of greater than 1 means that the current requirement of the power-gated block can not be met with placed power-switch instance. Either the power-switch needs to be made larger or power-switch instances need to be added.
`pv`	Voltage drop across power-switch (Volt) Displays the IRdrop across power-switch instances. This plot can be used to analyze and debug the IRdrop inside the power-gated block, e.g. if the IRdrop across power-switches is already high, the IRdrop inside the power-gated block will be much higher. In this case, the power-switch placement will have to be refined in order to resolve the IRdrop problem.

`rc`	Resistor current (Amp) Displays the current across power-grid resistor segments. This plot can be used to analyze and debug how the current flows from the voltage sources into the rest of the design, e.g. high current should flow from pads to the top-level routing level layers and then to the lower level metal layers. This plot can also be used to debug current density violations (`rj`).
`rlrp`	The total resistance of the instance along the least resistive path. Displays the total resistance of the instance along the least resistive path, meaning the resistance of the instance along the path from where it gets the most amount of current. This plot can be used to analyze and debug the power-grid integrity early in the design stage. An instance with high resistance is likely to have high static or dynamic IRdrop depending on the vector that is simulated later in the design cycle. This plot can be also used during decap optimization, since regions with high dynamic IRdrop and high resistance may not be improved upon by adding decaps, because the effectiveness of decoupling capacitance depends on the resistance of the power-grid network. The plot displays the RLRP paths for power (purple color) and ground (white color) nets in different colors for the identification of paths and ease of analysis. You can also highlight the least resistive path of the instance using the GUI and selecting the instance with left mouse button. The *RLRP Path* checkbox should be enabled.
`res`	Grid resistance (ohm) Displays the total resistance of the least resistive path from each node to voltage source. This plot can be used to view high resistance regions of the power-grid. It is useful during IRdrop and decap analysis to identify weak power-grid connectivity. The GUI also allows you to view the resistor-based RLRP path. To view the resistor-based RLRP path: 1. Select the *`res`* Rail plot, and the *RLRP Path* checkbox. 2. Select a resistor. The GUI highlights the least effective resistance path to the nearest voltage source, and opens the *Resistance Path* form.

`rj`	Resistor current density, J/Jmax. Displays the current density (current per unit area) violation on each power-grid segment using the ratio of calculated current density and maximum allowed current density. A ratio of greater than 1 means that the current density limit of the segment is violated. The current density limits are supplied by the foundry and can be read during analysis using the `set_rail_analysis_config -em_models` command.
rs	Specifies the plot type for resistor sensitivity.
`sem_avg`	Average Current Density Displays average current limit violations in the design. This plot is enabled when you specify `check_ac_limits -method avg`.
`sem_peak`	Peak Current Density Displays peak current limit violations in the design. This plot is enabled when you specify `check_ac_limits -method peak`.
`sem_rms`	RMS Current Density Displays RMS current limit violations in the design. This plot is enabled when you specify `check_ac_limits -method rms`.
`slack`	Instance slack (Seconds) Displays worst instance slack. The slack data is queried from either the timing database or if the power database is specified using `-power_db` option, then the worst slack data is read from the power database. When using an external TWF, the slack data is read from the TWF.
`tc`	Tap current (Amp) Displays the current consumed by the instance devices or current taps. During rail analysis, it is based on the power consumption of the instance and is distributed inside the cell. The current distribution inside the cell is based on the extracted device netlist stored inside the power-grid library of the cell. The current distribution is generally based on the device width, length, and spice models. This plot can be used to analyze and debug IRdrop inside the macro.

`td`	Transition density (Hz) Displays the worst transition density (activity * frequency) of the instance. This plot can be used to analyze and debug activity distribution and high power density regions.
`thermal`	Instance thermal Displays the instance temperature distribution for the chip. The plot shows tile-based temperature for instances. When you click on an instance, the *Attribute Viewer* window opens and displays the instance temperature in the *Thermal* field.
`unc`	Unconnected segments Displays power-grid segments that are disconnected from the voltage source (power pads). This plot can be used to debug unusually high IRdrop in the region.
`vc`	Voltage source current (Amp) Displays current through the voltage sources (power pads). This plot can be used to analyze the current carrying capacity of the power-pad.
`vu`	Voltage drop across package (Volt) Displays voltage drop across package. If analysis is run with package model attached to the power pads, this plot displays the IRdrop due to package RLCK elements.

Examples

- The following command specifies that `div` data will be plotted without loading design DEF:

```
gui_set_power_rail_display\
-plot div
```

- The following command enables the visibility of the filtered out resistors without changing the color:

```
gui_set_power_rail_display -filter_color on
```

- The following command enables the visibility of the filtered out resistors and also changes the color to red:

```
gui_set_power_rail_display -filter_color red
```

Related Topics

- "Static Power, IRdrop and EM Analysis" in the *Voltus User Guide*

gui_set_power_rail_layers_nets

```
set_power_rail_layers_nets
[-help]
[-design_opacity value]
[-rail_analysis_opacity value]
[-nets {{all_power 0/1}{all_ground 0/1}} |
 -enable_nets {names} |
 -disable_nets {names} ]
[-enable_switch_nets {all|net_names} |
 -disable_switch_nets {all|net_names} ]
[-enable_report_layers {all|layer_names} |
 -disable_report_layers {all|layer_names} |
 -layers {{name report0/1 visible0/1}{...}} ]
[-enable_visible_layers {all|layer_names} |
 -disable_visible_layers {all|layer_names} |
 -layers {{name report0/1 visible0/1}{...}} ]
[-enable_visible_signal_em_layers {all|layer_names} |
 -disable_visible_signal_em_layers {all|layer_names} ]
[-show_layer_net_setting_list {true | false}]
[-via_size {1 | 2 | 3 | 4 | 5 | 6}]
```

Controls the visibility of the layers, the opacity of the rail analysis or design database, and the visibility of power/ground nets.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `gui_set_power_rail_layers_nets` parameter. For a detailed description of the command and all of its parameters, use the man command: `man gui_set_power_rail_layers_nets`.
`-design_opacity value`	

	Specifies the design opacity value for the design database. The opacity range is from 0-100%, the default value is `60`.
`-disable_nets {`*`names`*`}`	
	Hides the specified nets. This parameter is mutually exclusive to the `-enable_nets` and `-nets` parameters.
`-disable_report_layers {all|`*`layer_names`*`}`	
	Hides the report (user-defined layers to be used for processing) layers. You can use `all` to hide all layers, or list the specific layers that are not to be displayed. This parameter is mutually exclusive to the `-enable_report_layers` and `-layers` parameters.
`-disable_switch_nets {all|`*`net_names`*`}`	
	Hides the specified switch nets. You can use `all` to hide all the switch nets, or list the specific switch nets that are not to be displayed. This parameter is mutually exclusive to the `-enable_switch_nets` parameter.
`-disable_visible_layers {all|`*`layer_names`*`}`	
	Specifies which layers should not be made visible when plotting. You can use `all` to hide all layers, or list the specific layers that are not to be displayed. This parameter is mutually exclusive to the `-enable_visible_layers` and `-layers` parameters.
`-disable_visible_signal_em_layers {all|`*`layer_names`*`}`	
	Specifies which signal EM layers should not be made visible when plotting. You can use `all` to hide all layers, or list the specific layers that are not to be displayed. This parameter is mutually exclusive to the `-enable_visible_signal_em_layers` parameter.
`-enable_nets {`*`names`*`}`	
	Displays only the specified nets. This parameter is mutually exclusive to the `-disable_nets` and `-nets` parameters.

`-enable_report_layers {all|`*`layer_names`*`}`	
	Specifies to select the user-defined layers to be used for processing. You can use `all` to select all layers, or list the specific layers that are to be selected. This parameter is mutually exclusive to the `-disable_report_layers` and `-layers` parameters.
`-enable_switch_nets {all|`*`net_names`*`}`	
	Displays the specified switch nets. You can use `all` to display all the switch nets, or list the specific switch nets that are to be displayed. This parameter is mutually exclusive to the `-disable_switch_nets` parameter.
`-enable_visible_layers {all|`*`layer_names`*`}`	
	Specifies which layers should be made visible when plotting. You can use `all` to display all layers, or list the specific layers that are to be made visible. This parameter is mutually exclusive to the `-disable_visible_layers` and `-layers` parameters.
`-enable_visible_signal_em_layers {all|`*`layer_names`*`}`	
	Specifies which signal EM layers should be made visible when plotting. You can use `all` to select all layers, or list the specific layers that are to be selected. This parameter is mutually exclusive to the `-disable_visible_sem_layers` parameter.
`-instance_voltage_method {Worst | Best | Avg | WorstAvg}`	
	Allows different methods of computing effective instance voltage during rail analysis. Using this option, you can write out the Best, Worst, or Average effective instance voltage between power and ground nets. The WorstAvg method computes the average instance voltage during each evaluation window defined by the IV Window field, and reports the worst average instance voltage out of all the windows. If IV Method is WorstAvg and IV Window is whole, then this method will report average voltage across all time steps in simulation. Default : `Worst`
`-instance_voltage_window {timing/switching | whole}`	

	Controls the window where effective instance voltage (EIV) is evaluated. EIV is obtained by processing the voltage waveforms and finding the worse-case effective voltage between the power and ground pins during a specific window. There are two possible options to evaluate EIV: • `timing/switching` – both switching activity (current is non-leakage) and timing window • `whole` – entire rail simulation not filtered to a specific window
`-layers {{name report0/1 visible0/1}{...}}`	
	Controls the visibility of layers. You can enable or disable the reporting and visibility of specific layer. For example, `-layer {{M1 1 1} {M2 0 0}` means enable reporting and visibility of layer M1, and disable reporting and visibility of layer M2 This parameter is mutually exclusive to the `-disable_report_layers, -disable_visible_layers, -enable_visible_layers,` and `-enable_report_layers` parameters.
`-nets {{all_power 0/1}{all_ground 0/1}}`	
	Controls the display of all the power and ground nets. When `all_power` is set to `0`, hides all power nets, and when set to `1`, displays all power nets. When `all_ground` is set to `0`, hides all ground nets, and when set to `1`, displays all ground nets. This parameter is mutually exclusive to the `-enable_nets` and `-disable_nets` parameters.
`-rail_analysis_opacity` *value*	
	Specifies the opacity value for the rail analysis output. The opacity range is from 0-100%, the default value is `100`.
`-show_layer_net_setting_list {true | false}`	

	Displays the current status (OFF/ON) of all layers and nets in the console. The following is an example of the output:
	`#------------------------------#`
	`# Layers  Report  Visible #`
	`# RD  ON  ON #`
	`# M0  ON  ON #`
	`.....`
	`#------------------------------#`
	`# PowerNets  Status #`
	`# VDD  ON #`
	`# VDDm  ON #`
	`#------------------------------#`
	`# GroundNets Status #`
	`# VSS  ON #`
	`#------------------------------#`

`-via_size {1 | 2 | 3 | 4 | 5 | 6}`	
	Specifies to modify and control the size of vias in the GUI. The default via size is `1`. The possible via size values are `1`, `2`, `3`, `4`, `5`, and `6`.

Examples

- The following command specifies the layers that will be visible:

```
gui_set_power_rail_layers_nets \
 -enable_visible_layers{RD M1 V12 M2 V23 M3 V34 M4 V45 M5 V56 M6} \
 -enable_nets {vdd vdd_1}
```

- The following commands describe the use model for enabling and disabling switch nets. Here, consider a net `VDD` with three switch nets `VDD_sw0`, `VDD_sw1`, and `VDD_sw2`:
  - The following command hides the switch net `VDD_sw0`:

    ```
    set_power_rail_layers_net -disable_switch_nets {VDD {VDD_sw0}}
    ```

    Displays `VDD_sw1` and `VDD_sw2`.
  - The following command hides all the switch nets:

    ```
    set_power_rail_layers_net -disable_switch_nets {VDD {all}}
    ```
  - The following command displays the switch net `VDD_sw0`:

    ```
    set_power_rail_layers_net -enable_switch_nets {VDD {VDD_sw0}}
    ```
  - The following command displays all the switch nets:

    ```
    set_power_rail_layers_net -enable_switch_nets {VDD {all}}
    ```

Related Topics

- "Static Power, IRdrop and EM Analysis" in the *Voltus User Guide*

- The following commands describe the use model for enabling and disabling switch nets. Here, consider a net `VDD` with three switch nets `VDD_sw0`, `VDD_sw1`, and `VDD_sw2`:
  - The following command hides the switch net `VDD_sw0`:

    ```
    set_power_rail_layers_net -disable_switch_nets {VDD {VDD_sw0}}
    ```

    Displays `VDD_sw1` and `VDD_sw2`.
  - The following command hides all the switch nets:

    ```
    set_power_rail_layers_net -disable_switch_nets {VDD {all}}
    ```
  - The following command displays the switch net `VDD_sw0`:

    ```
    set_power_rail_layers_net -enable_switch_nets {VDD {VDD_sw0}}
    ```
  - The following command displays all the switch nets:

    ```
    set_power_rail_layers_net -enable_switch_nets {VDD {all}}
    ```

gui_set_tool

```
gui_set_tool [-help] mode
```

Selects specified tool widget in the main window and enters interactive mode.

Parameters

`-help`	Prints a brief description that includes type and default information for each gui_set_tool parameter. For a detailed description of the command and all of its parameters, use the man command: man gui_set_tool.
mode {select | move | ruler | util | cut | obstruct | layerBlk | ptnFeed | ptnPinBlk | ptnPinGuide | addWire | addVia | moveWire | cutWire | stretchWire | addPoly | addBusGuide | p2pRoute}	

	Specifies the interactive mode to enter. The following modes are supported: • `select` - Chooses the Select tool widget in the main window. • `move` - Selects the Move/Resize/Reshape tool widget in the main window. • `ruler` - Selects the Create Ruler tool widget in the main window. • `util` - Selects the Query Area Density tool widget in the main window. • `cut` - Selects the Cut Rectilinear tool widget in the main window. • `obstruct` - Selects the Create Placement Blockage widget in the main window. • `layerBlk` - Selects the Create Routing Blockage tool widget in the main window. • `ptnFeed` - Selects the Create Physical Feedthrough tool widget in the main window. • `ptnPinBlk` - Selects the Create Pin Blockage tool widget in the main window. • `ptnPinGuide` - Selects the Create Pin Guide tool widget in the main window. • `addWire` - Selects the Edit Wire tool widget in the main window. • `addVia` - Selects the Add Via tool widget in the main window. • `moveWire` - Selects the Move Wire tool widget in the main window. • `cutWire` - Selects the Cut Wire tool widget in the main window. • `stretchWire` - Selects the Stretch Wire tool widget in the main window. • `addPoly` - Selects the Add Polygon tool widget in the main window. • `addBusGuide` - Selects the Edit Bus Guide tool widget in the main window. • `p2pRoute` - Selects the Point to Point Route tool widget in the main window.

Example

The following command is used to enter the Create Ruler mode in the design window.

```
gui_set_tool ruler
```

gui_set_ui

```
gui_set_ui
[-help]
name
[-before name]
[-checked {true | false}]
[-command string]
[-disabled {true | false}]
[-enable_variable string]
[-foreground string]
[-geometry string | -full_screen {true | false}]
[-icon filename]
[-in name]
[-label string]
[-message string]
[-movable {true | false}]
[-menu_separate {true | false}]
[-newline {true | false}]
[-shortcut string]
[-statusbar string]
[-title string]
[-titlebar_color string]
[-tooltip string]
[-underline integer]
[-value string ]
[-variable string]
[-visible {true | false}]
[-visible_variable string]
```

Sets the specified property of an element. Use this command to customize the tool's graphical interface by configuring various interface elements, such as submenus and commands. The properties that can be set for a specific element depends on the `type` of that element. To see the list of properties that can be set for each type, see the "Accepted Properties for Each Type" section below.

For elements of type `menu`, `submenu`, `command`, `check`, `radio`, `separator`, `toolbar`, and `toolbutton`, the `gui_set_ui` command works like the tk command *`widget`*`configure`. For the main window (`main`), it works like the tk command `wm`.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_set_ui` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_set_ui`
`name`	Specifies the name of the element whose property is to be set. An error is returned if specified *`name`* does not exist. *Data_type*: string, required
`-before` *`name`*	Sets the specified element before the element identified by *`name`*. *Data_type*: string, optional
`-checked {true | false}`	Checks or unchecks the specified element. **Note:** You can use the `-checked` parameter only for elements of type `check` or `radio`. *Data_type*: bool, optional
`-command` *`string`*	Specifies the Tcl command to be executed when the specified element is invoked. **Note:** You can use the `-command` parameter only if you are adding elements of type `command, check, radio` or `toolbutton`. *Data_type*: string, optional
`-disabled {true | false}`	Disables or enables the specified interface element. **Note:** You can use the `-disabled` parameter only for elements of type `menu, submenu, command, check, radio` or `toolbutton`. The parameter is NOT applicable for elements of type `toolbar` and `separator`. *Data_type*: bool, optional
`-enable_variable` *`string`*	Sets the Tcl global variable for Enable status. *Data_type*: string, optional

`-foreground` *`string`*	Specifies the foreground color of the icon of the specified interface element. **Note:** You can use the `-foreground` parameter only if you are adding an element of type `toolbutton`. *Data_type*: string, optional
`-full_screen {true | false}`	
	Maximizes the main window. This property can be set only for the element `main`. **Note:** This option cannot be used with `-geometry`. *Data_type*: bool, optional
`-geometry` *`string`*	Modifies the geometry of the specified window element. *Data_type*: string, optional
`-icon` *`filename`*	Specifies the file name of the toolbutton icon. *`filename`* can be the full path of the icon file or an icon name supported by the tool. The icon names used in the tool can be queried using the `gui_get_ui` command. For list of supported image formats for the icon file, see the "Supported Image Formats" section of the Customizing the User Interface chapter in the *User Guide*. **Note:** The `-icon` parameter is applicable only for elements of type `toolbutton`. *Data_type*: string, optional
`-in` *`name`*	Specifies the name of the parent element. *Data_type*: string, optional
`-label` *`string`*	Specifies the label to be displayed on the element being modified. **Note:** This parameter is NOT applicable for elements of type `separator`. *Data_type*: string, optional

`-message` *string*	Specifies the message to be displayed on the element being modified. **Note:** This parameter is applicable for elements of type `statusbar`. *Data_type*: string, optional
`-menu_separate {true | false}`	
	Specifies whether or not a menu or submenu can be torn off or detached. • When set to `true` (default), a dashed line appears at the top of the menu or submenu to indicate that it can be detached. • When set to `false`, the dashed line no longer appears at the top of the menu or submenu and it cannot be detached. **Note:** This parameter is applicable for elements of type `menu` and `submenu`. *Data_type*: bool, optional
`-movable {true | false}`	Specifies whether or not the element being modified should be movable. **Note:** The `-movable` parameter is applicable only for elements of type `toolbar`. *Data_type*: bool, optional
`-newline {true | false}`	Specifies whether or not the element being modified should be on a new row. **Note:** The `-newline` parameter is applicable only for elements of type `toolbar`. *Data_type*: bool, optional
`-shortcut` *string*	Specifies the shortcut key for a command, check box, radio button, or toolbutton. **Note:** You can use the `-shortcut` parameter only if you are modifying elements of type `command`, `check`, `radio`, or `toolbutton`. *Data_type*: string, optional

`-statusbar` *string*	Sets the statusbar of the specified window element. *Data_type*: string, optional
`-title` *string*	Specifies the window title. *Data_type*: string, optional
`-titlebar_color` *string*	Sets the color of the title bar of the window. **Note:** This parameter is applicable only for the main window (`main`). *Data_type*: string, optional
`-tooltip` *string*	Specifies the tooltip text for the toolbar or toolbutton you are adding. **Note:** The `-tooltip` parameter is applicable only for elements of type `toolbar` and `toolbutton`. *Data_type*: string, optional
`-underline` *integer*	Specifies the integer index of the character to be underlined in the element label. The index begins with `0` and should have a value less than or equal to the length of the element label. For instance, if you want the second letter of the element label to appear underlined, specify *integer* as `1`. **Note:** The `-underline` parameter is NOT applicable for elements of type `toolbar, separator,` and `toolbutton`. *Data_type*: int, optional
`-value` *stringValue*	Specifies the value to be stored in the associated variable when the specified radio button is selected. **Note:** The `-value` parameter is applicable only for elements of type `radio`. *Data_type*: string, optional
`-variable` *string*	When modifying a check box or radio button, specifies the Tcl global variable for the item. *Data_type*: string, optional
`-visible {true | false}`	Specifies the visibility status of the element being modified. *Data_type*: bool, optional

`-visible_variable` *string*	Sets the Tcl global variable for Visible status. *Data_type*: string, optional

Accepted Properties for Each Type

The accepted properties for each type are listed below.

- For the main window (`main`), the following properties can be set:

  `-geometry {`*widthxheight* | *widthxheight+x+y*`}`
  `-title` *title*
  `-close_command` *string*
  `-visible true | false`

- For a status bar (`statusbar`), the following property can be set:

  `-enable_variable` *string*
  `-in` *parentname*
  `-message` *msg*
  `-visible true | false`
  `-visible_variable` *string*

- For elements of type `menu` or `submenu`, the following properties can be set:

  `-enable_variable` *string*
  `-before` *nextSiblingName*`]`
  `-disabled true | false`
  `-in` *parentName*
  `-label` *label*
  `-menu_separate true | false`
  `-underline` *index*
  `-visible true | false`
  `-visible_variable` *string*

- For elements of type `command`, `radio`, and `check`, the following properties can be set:

  `-disabled true | false`
  `-label` *label*
  `-underline` *index*
  `-command` *commandName*

`-in` *parentName*
`-before` *nextSiblingName*
`-enable_variable` *string*
`-visible true | false`
`-visible_variable` *string*
`-shortcut` *string*

For `radio` and `check`, the `-checked` and `-variable` properties can also be set.

- For elements of type `toolbutton`, the following properties can be set:

`-state normal | disabled`
`-label` *label*
`-tooltip` *toolTipText*
`-underline` *index*
`-foreground` *colorName*
`-icon` *iconName*
`-command` *commandName*
`-in` *parentName*
`-before` *nextSiblingName*
`-enable_variable` *string*
`-visible true | false`
`-visible_variable` *string*
`-shortcut` *string*

Examples

- The following command makes the *View* menu in the main window non-detachable:

```
gui_set_ui view -menu_separate false
```

- The following command sets the color of the main window's title bar to blue:

```
gui_set_ui main -titlebar_color blue
```

Related Information

- gui_add_ui
- gui_delete_ui
- gui_get_ui
- gui_find_ui

gui_set_visible_bumps

```
gui_set_visible_bumps
[-help]
{-bump_cells bump_cell_name| -all}
```

Displays only the bumps with the specified bump cell names and hides all other bumps.

To redisplay all bumps, specify the `-all` parameter.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_set_visible_bumps` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_set_visible_bumps`
`-all`	Displays all bumps. By default, all bumps are displayed. When you specify `gui_set_visible_bumps bump_cell_name`, the tool displays only the bumps with the specified bump cell names and hides all other bumps. You can then use `gui_set_visible_bumps -all` to redisplay all bumps.
`-bump_cells names`	Displays only the bumps with the specified bump cell names and hides all other bumps. You can specify multiple bump cell names.

Examples

- The following command displays only the bumps with bump cell as BUMPCELL:
  ```
  gui_set_visible_bumps -bump_cells BUMPCELL
  ```
- The following command redisplays all bumps:
  ```
  gui_set_visible_bumps -all
  ```

Related Information

- gui_get_visible_bumps

gui_set_visible_net_groups

```
gui_set_visible_net_groups
[-help]
{net_group_names | -all}
```

Displays only the specified net groups and hides all other net groups.

To display all net groups, specify the `-all` parameter.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_set_visible_net_groups` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_set_visible_net_groups`
`-all`	Displays all the net groups.
net_group_names	Displays only the specified net groups and hides all other net groups. You can specify multiple net group names.

Examples

- The following command hides all net groups except the specified one:
  ```
  gui_set_visible_net_groups test_ng
  ```

Related Information

- `gui_get_visible_net_groups`

gui_set_visible_nets

```
gui_set_visible_nets
[-help]
{nets | -all}
```

Displays only the specified nets and hides all other nets. To display all nets, specify the `-all` parameter.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_set_visible_nets` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_set_visible_nets`
`-all`	Displays all nets.
nets	Displays only the specified nets and hides all other nets. You can specify multiple net names.

Examples

- The following command hides all nets except the specified net:

  ```
  gui_set_visible_nets
  ram1/subram02_lower/TESTACT_INST_PERIWDSRAM001PAA512W27C3/tpo_a_9_
  ```

Related Commands

- `gui_get_visible_nets`

gui_show_edge_number

```
gui_show_edge_number [-help]
[-objects string]
[-reset]
```

Displays edge numbers on the sides of the selected or specified objects in the GUI. Currently, you can display edge numbers on partitions and blocks. This feature is useful if you need to indicate the partition or block edge number in your script.

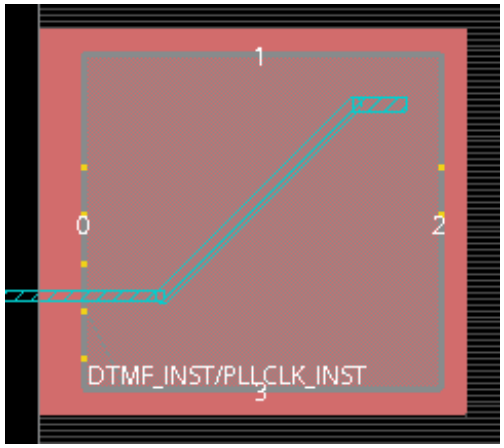
Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_show_edge_number` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_show_edge_number`
`-objects` *`string`*	Specifies the name of the object for which the edge numbers should be displayed. If you want to specify multiple object names, enclose them in braces or double quotation marks. Currently, you can display the edge numbers for partitions and blocks.
`-reset`	Clears the edge numbers from the specified or selected objects on the GUI. If you want to clear the edge numbers for all objects, use the `-reset` parameter with `-objects *`.

Examples

- The following command displays the edge numbers on the `DTMF_INST/PLLCLK_INST` block in the GUI:

  ```
  gui_show_edge_number -objects DTMF_INST/PLLCLK_INST
  ```



- The following command clears the edge numbers from all blocks:

  ```
  gui_show_edge_number -objects * -reset
  ```

Related Information

- `gui_select`

gui_show_user_bundle_nets

```
gui_show_user_bundle_nets
[-help]
[-keep_existing]
-net_bundle bundleName
[-remove]
```

Displays flightlines for the specified net bundle. Bundled nets help you with data flow analysis before and after layout. Net bundles can be created and viewed in floorplan and amoeba views. When you run the gui_show_user_bundle_nets command, the software switches automatically to the net bundle mode and clears flightlines based on selection.

You can create flight line net bundles using the `gui_create_user_bundle_net` command and delete them using the `gui_delete_user_bundle_net` command.

Note: When selecting objects, you can choose to display flightlines for only bundled nets by using the *Only Show Net Bundle Flight Line* option in the Preferences form. If this option is not selected, the net bundle flightlines are displayed along with other flightlines.

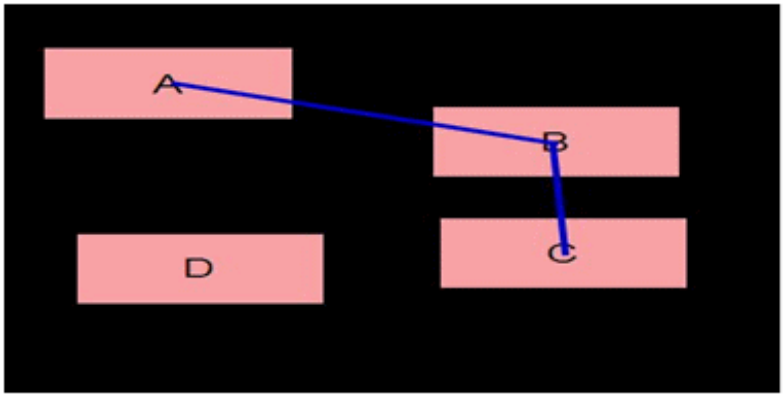
Parameters

`-help`	Prints a brief description that includes type and default information for each gui_show_user_bundle_nets parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_show_user_bundle_nets`
`-keep_existing`	Retains existing net bundle flightlines while displaying the flightline of specified net bundles. If this option is not specified, the command clears existing flightlines before displaying specified net bundle flightlines.
`-net_bundle` *bundleName*	Specifies the name of the net bundle whose flightlines you want to display. You can specify more than one net bundle.
`-remove`	Clears flightlines of specified net bundle from the display area. If no net bundle name is specified, all flightlines based on net bundles are removed. **Note:** This option only clears the flightlines of the net bundle from the display area. If you want to remove the net bundle itself, use the `gui_delete_user_bundle_net` command.

Examples

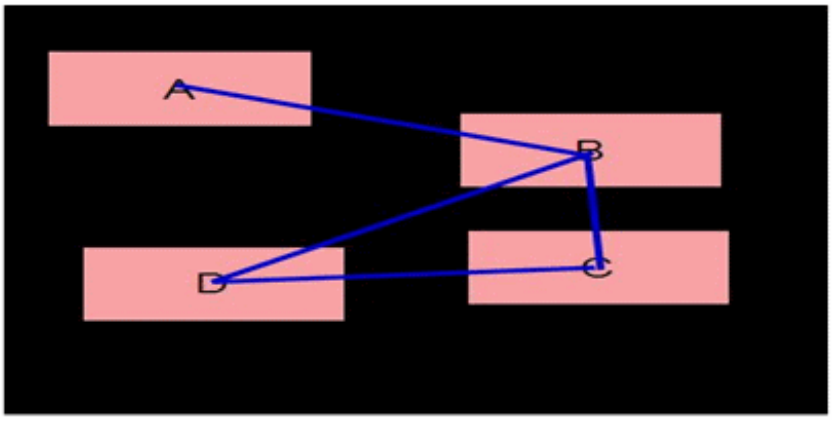
- The following command displays flightlines of the net bundle `bundle_1`. Here, `bundle_1` contains `net_1`, which is just between A and B, and `net_2`, which is just between B and C:

```
gui_show_user_bundle_nets -net_bundle bundle_1
```



- The following command displays flightlines of the net bundle `bundle_2` while retaining flightlines of `bundle_1`. Here, `bundle_2` contains `net_3`, which is among B, C, and D:

```
gui_show_user_bundle_nets -net_bundle bundle_2 -keep_existing
```



Note: The same flightlines could also be displayed in a single step by using the following command:

```
gui_show_user_bundle_nets -net_bundle {bundle_1 bundle_2}
```

gui_view_box

```
gui_view_box [-help]
```

Reports the coordinates of the current area being viewed in the GUI.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man gui_view_box`

Example

The following command reports the coordinates of the current area being viewed in the GUI:

```
tempus 9> gui_view_box
93.367 -646.465 103.294 -638.724
```

Related Information

- `gui_zoom`

gui_write_picture

```
gui_write_picture
[-help]
filename
[-format {bmp gif jpg jpeg png pbm pgm ppm xbm xpm}]
[-honor_umask]
[-width integer_value -height integer_value]
[-window string]
```

Saves a snapshot of the layout or specified window with the specified name in the current directory. You can specify the size as well as the format in which you want to save the snapshot. The command also prints out overlay maps with legends.

You can use this command at any stage of the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_write_picture` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_write_picture`

`filename`	Specifies the name of the picture file as a string. You can specify the file extension as well.
	
	Note: The file extension you specify does not control the format of the picture. The picture file is dumped in the PNG format by default or the format you explicitly specify with `-format`. For example, the following command saves the `testShot` picture file in the PNG format:
	`> gui_write_picture testShot.gif`
	
	`current_directory``@file testShot.gif`
	`testShot.gif: PNG image data, 540 x 688, 8-bit/color RGB, non-interlaced`
	
	Data_type: string, required

`-format {bmp gif jpg jpeg png pbm pgm ppm xbm xpm}`	

	Specifies the format of the picture file. *Default*: `png` *Data_type*: enum, optional
`-height` *integer_value*	
	Specifies the height of the picture as an integral value. **Note**: The `-width` parameter is required when `-height` is specified. *Min*: 100 *Max*: 20000 *Data_type*: int, optional
`-honor_umask`	Honors the umask settings while saving the picture. If you specify this option, the permission of the saved picture file will be set by subtracting the umask value from the full permission value. For example, if the full permission setting of a file is `0666` and the umask setting is `0022`, the tool saves the picture file with the permission setting of `0666` - `0022`, that is `0644`. *Data_type*: bool, optional
`-width` *integer_value*	
	Specifies the width of the picture as an integral value. **Note**: The `-height` parameter is required when `-width` is specified. *Min*: 100 *Max*: 20000 *Data_type*: int, optional
`-window` *string*	Specifies the ID of the window that you want to dump. You can use the following IDs: • For Layout Viewer: `designview.win` • For Cell Viewer: `.uiCellView.win` If `-window` is not specified, the main layout window is dumped out by default. *Data_type*: string, optional

Examples

- The following command captures the layout window and saves it in the default PNG format with the name *test2* and the dimensions 1000 x 1000 in the current directory:

  ```
  gui_write_picture test2 -width 1000 height 1000
  ```

- The following command captures the layout window and saves it in the GIF format with the name *test1.gif* in the current directory:

  ```
  gui_write_picture -format gif test1.gif
  ```

- The following command captures the layout viewer window and saves it in PNG format with the name testShot.png in the current directory:

  ```
  gui_write_picture testShot.png -window designview.win
  ```

  Note: You must have the layout viewer open.

- The following command captures the view in the cell viewer window and saves it in PNG format with the name testCell.png in the current directory:

  ```
  gui_write_picture testCell.png -window .uiCellView.win
  ```

Related Information

- write_to_gif

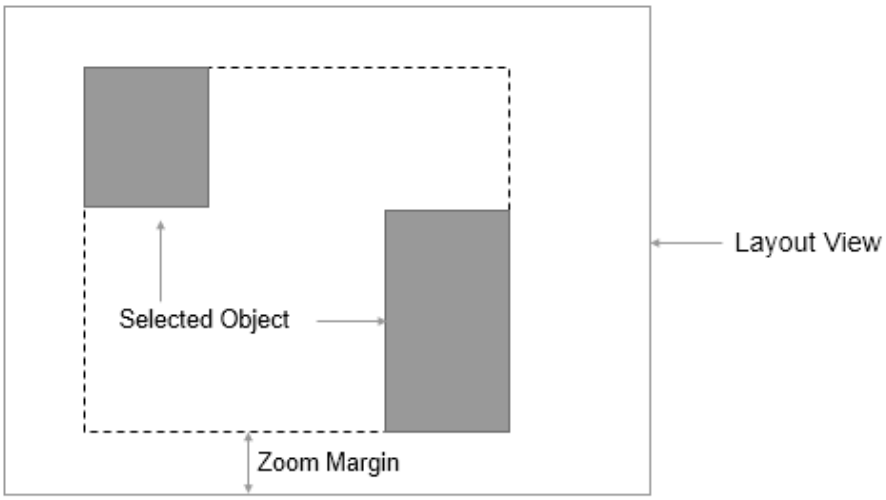
gui_zoom

```
gui_zoom
[-help]
[-to {x y}[-to_radius value]]
[-selected [-margin integer_value]]
{-in  | -out  | -selected  | -to {x y} | -rect {x1 y1 x2 y2}}
```

Zooms into or out of the viewable window as per the specified parameters.

Parameters

`-help`	Prints a brief description that includes type and default information for each `gui_zoom` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_zoom`.
`-in`	Zooms into the viewable window by two times.

`-margin integer_value`	Specifies the zoom margin ratio for the viewable window. The zoom margin is the minimum spacing between the bounding box of the selected object(s) and the layout view.  The minimum value is `0` and the maximum value is `99`. The `gui_zoom -selected` parameter must be specified if you are specifying `-margin`. *Default*: 50
`-out`	Zooms out from the viewable window by two times.
`-rect {x1 y1 x2 y2}`	Zooms in the viewable window to show all of the rectangular area that is defined by the specified coordinates.
`-selected`	Zooms in the viewable window to the area that encloses the selected objects.
`-to {x y}`	Zooms in the viewable window to the point defined by the coordinates, `x` and `y`.
`-to_radius value`	Zooms in the viewable window to a radius of the specified value in microns, with center at the point specified by `-to {x y}`. *value* is of type float. *Default*: 150 microns **Note**: The `-to {x y}` parameter must be specified with this parameter.

Examples

- The following command zooms in the viewable window to the area enclosed by the lower-left coordinates (`450 531`) and the upper-right coordinates (`550 631`):

  ```
  gui_zoom -rect {450 531 550 631}
  ```

- When the following command is run, the display zooms in to point with x=100 and y=200. The display area is centered at this point and has the default radius of 150 microns:

  ```
  gui_zoom -to {100 200}
  ```

- When the following command is run, the display zooms in to point with x=100 and y=200. The display area is centered at this point and has a radius of 50 microns:

  ```
  gui_zoom -to {100 200} -to_radius 50
  ```

read_power_rail_results

```
read_power_rail_results
[-help]
[-detail_delta_temperature_file file]
[-effective_resistance_file file]
[-force]
[-instance_delta_temperature_file file]
[-power_db file ]
[-rail_directory dir]
[-tile_delta_temperature_file file]
[-cell_library { library1.cl}]
[-reset]
[-custom_value_inst_file_1 file]
[-nets net_names]
[-instance_voltage_method { best | worst | avg | worstavg }]
[-instance_voltage_window { timing | whole }]
[-instances instancenames]
[-enable_plots plot_names]
[-signal_em_db file]
[-short_file file]
[-esd_directory dir]
[-thermal_file filename.txt]
[-esd_em dir]
[-die_instance_name dieinstname]
[-data_type {rail | all}]
[-profiling_power_file file]
[-tile_thermal_config_file filename]
[-physical_db dir_name]
[-top_cell_name topcell_name]
[-power_directory power_output_directory]
```

Loading power/rail analysis results into GUI.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_power_rail_results` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_power_rail_results`.
`-cell_library { library1.cl}`	
	Specifies the single cell library that is required to load the state directory. The specified cell library must contain the technology layer information that is required to load the state directory. Therefore, you do not need to load all cell libraries used during analysis.
`-custom_value_inst_file_1 file`	
	Specifies the name of the custom value instance file (CVIF). This file can be used to display a user-specified instance voltage file in GUI. The format of the CVIF file is: *value1* *instancename1* *value2* *instancename2* .... where, *value1* and *value2* are voltage values mapping to the specified instance. You can use the `cvil` argument of the `gui_set_power_rail_display -plot` command parameter to view the EIV map in GUI.
`-data_type {rail | all}`	
	Controls the type of data to be loaded in the GUI. The possible arguments of this parameter are: • `rail` - Loads only the rail analysis results. • `all` - Loads both the power and rail analysis results. This is the default value.
`-detail_delta_temperature_file file`	

	Specifies the name of the detailed wire-based delta temperature file from self-heating analysis.
`-die_instance_name` *`dieinstname`*	
	Specifies the name of the die whose results you need to view. This parameter is only applicable to the multi-die analysis flow (`set_rail_analysis_config -die_mode multi-die`). The `-die_instance_name` parameter must be specified with the `-rail_directory` parameter (multi-die rail directory).
`-effective_resistance_file` *`file`*	
	Specifies the result file from effective resistance analysis (only instance-based result is supported). When the rail analysis plot type `effr` is selected and a result file is specified, the layout shows effective resistance analysis result for each instance. **Note**: Currently, this feature only supports loading one net at a time.
`-enable_plots` *`plot_names`*	
	Specifies the list of power or rail plots names that are to be loaded.
`-esd_directory` *`dir`*	
	Specifies the ESD analysis directory.
`-esd_em` *`dir`*	
	Specifies the name of the directory where the current density/EM results generated by a previous ESD analysis run reside. This parameter allows you to selectively load the required EM result directory if there are multiple ESD EM result directories under the `ESD` directory. You must either load the rail directory (`-rail_directory`) before running this command, or specify the rail directory with the `-esd_em` parameter.
`-force`	

	Specifies to block the pop-up window that appears when the `read_power_rail_results -reset` parameter is specified. The pop-up window displays a confirmation message for resetting the power and rail directories. Example: `read_power_rail_results -reset -force`
`-instances` *`instancenames`*	
	Specifies a list of instances to load in the GUI. This parameter loads only those partitions that include the specified instances. The `-instances` parameter allows you to analyze parts of the designs, thereby reducing the loading time and memory footprint. The `-instances` parameter is applicable to the Voltus-XP flow only.
`-instance_delta_temperature_file` *`file`*	
	Specifies the name of the instance-based delta temperature file from self-heating analysis. This is the device instance temperature.
`-instance_voltage_method { best | worst | avg | worstavg }`	
	Loads the instance voltage data for the specified EIV computing method. If this parameter is not specified (default), the instance voltage results for all the four methods are loaded.
`-instance_voltage_window { timing | whole }`	
	Loads the instance voltage data for the selected EIV window. If this parameter is not specified (default), the instance voltage results for both the timing and switching windows, and the entire rail simulation are loaded.
`-nets` *`net_names`*	
	Loads only the specified nets in the domain-based state directory to the GUI. If this parameter is not specified with the `-rail_directory` parameter, all the nets will be loaded.

`-power_db` *file*	
	Specifies the power database file. This file was created by using the `power_write_db` attribute.
`-power_directory` *power_output_directory*	
	Specifies to load the power analysis results and display the power plots directly without loading the rail analysis results.
`-profiling_power_file` *file*	
	Specifies to display the tile-based power density results generated from the vector profiling flow.
`-physical_db` *dir_name*	Specifies the name of the hierarchical design directory to be loaded to the GUI along with the rail analysis results.
`-reset`	
	Frees the loaded analysis state directory or power database from the memory.
`-rail_directory` *dir*	
	Specifies the rail analysis directory. You can load all the domain results by selecting the automatically generated analysis directory under the output directory (`<domain>_<temp>_<dynamic/avg>_#`), or load individual nets by going one level further and selecting the net directory.
`-short_file` *file*	
	Specifies the path to the short violations report (`voltus_short.report`) . This file is generated using the `set_rail_analysis_config -report_shorts true` command parameter, and is located in the work directory.
`-signal_em_db` *file*	
	Specifies to read the signal electromigration data from the specified database file generated by the `check_ac_limits -report_db` command parameter.

`-thermal_file` *`filename`*`.txt`	

	Specifies to load the tile-based temperature map file generated by Sigrity PowerDC to be used for both power calculation and IR drop analysis.
	The following is an example of reading the thermal map file:
	`read_power_rail_results -rail_directory output/static_rail/core_25C_avg_1/ -thermal_file output/power_map_by_layer.txt`
	`gui_set_power_rail_display -plot thermal`

`-tile_delta_temperature_file` *`file`*	
	Specifies the name of the tile-based delta temperature file of self-heating analysis. This is the worst instance temperature among all wire segments in a tile. This is also layer-based. The FEOL or cell layer is used to show the device instance temperature.
`-tile_thermal_config_file` *`filename`*	

	Specifies to load the configuration file that contains the mapping between the hierarchical levels with corresponding layer-based temperature map files generated by Celsius (system-level thermal analysis tool). When specified, you can view tile-based temperature distribution for each layer of the chip.
	
	The following is the format and example of the hierarchical configuration file:
	<hierarchy> <block name> <X_tile_no,Y_tile_no> <LEF/DEF Flip/Rotation> <temperature map>
	`TOP DIE 0,0 N /thermal_data/DIE_TemperatureMap`
	`TOP/IP1 IP1 25,50 N /thermal_data/IP1_TemperatureMap`
	`TOP/IP1/B1 B1 50,125 N /thermal_data/B1_TemperatureMap`
	`TOP/IP1/B2 B2 200,125 S /thermal_data/B2_TemperatureMap`
	
	This is a mandatory parameter for the `tile_thermal` plot when loading hierarchical temperature maps.

`-top_cell_name` *`topcell_name`*	
	Specifies the name of the top module or top cell name to be loaded. This parameter must be specified with the `-physical_db` parameter.

Examples

- The following command specifies the appropriate files needed for viewing the instance voltage and power data:

```
read_power_rail_results\
   -rail_directory analysis/domain_25C_avg_1 \
   -power_db powermeter.db \
```

- The following command specifies to load only the rail directory:

```
read_power_rail_results -rail_directory dynamic_rail/core_25C_dynamic_1 -data_type rail
```

- The following command specifies to display the tile-based power density results in the GUI:

```
read_power_rail_results -profiling_power_file ./vectorProfileResults/profile.rpt.density
```

- The following command specifies to load the power analysis results and display the power plots directly by running the power analysis without loading the rail analysis results:

```
read_power_rail_results -power_directory ./dynamic_power
```

Related Topics

- "Static Power, IRdrop and EM Analysis" in the *Voltus User Guide*

report_power_rail_results

```
report_power_rail_results
[-help]
[-append]
[-enable_filter_summary]
[-file_name file ]
[-filter_max max_value]
[-filter_min min_value]
[-ignore_limit_bound]
[-layers {All, layer_names } ]
[-limit N ]
[-nets {all | all_pwr | all_gnd | net_names}]
[-plot plot_type]
[-range_max max_value]
[-range_min min_value]
[-region x1 y1 x2 y2]
[-rlrp_inst {instance_name}]
```

Generates RLRP, region-based, and rail/power/capacitance reports.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_power_rail_results` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_power_rail_results`.
`-append`	Appends instance reports to the current report file.
`-enable_filter_summary`	
	Displays a summary of the filtered data in the histogram given in the *Summary* tab of the Result Browser form.
`-file_name file`	

	Specifies the name of the text based violation report file to be created. The default file name is: • for region-based report: `region_x1_y1_x2_y2.rpt` • for RLRP report: `rlrp_instname.rpt` • for plot-based report: `power_rail_`*`plot`*`.rpt`
`-filter_max` *`max_value`*	
	Specifies the maximum value to include in the report. Use this option to further filter out the data within the range specified. This is used to obtain statistic of filter range. Optional, but if specified, the `-plot` option is required, and requires both `-filter_min` and `-filter_max` to be specified.
`-filter_min` *`max_value`*	
	Specifies the minimum value to include in the report. Use this option to further filter out the data within the range specified. This is used to obtain statistic of filter range. Optional, but if specified, the `-plot` option is required, and requires both `-filter_min` and `-filter_max` to be specified.
`-ignore_limit_bound`	
	Specifies to ignore the maximum upper limit of `10000`, and instead use the higher value given with the `-limit` parameter for specifying the number of data points to be printed in the power/rail report.
`-layers {all,` *`layer_names`* `}`	
	Specifies the layers to be reported. You can use `all` for reporting all layers, or list the specific layers that are to be reported. If this parameter is specified, you must specify the `-plot` parameter.
`-limit` *`N`*	

	Specifies the number of data points printed in the report. If this parameter is specified, you must specify the `-plot` parameter. *Default* : first `1000` data points
`-nets {all | all_pwr | all_gnd | net_names}`	

	Specifies the nets to be used for analysis reporting. You can use `all` for reporting all nets, `all_pwr` for reporting all power nets, `all_gnd` for reporting all ground nets, or list the specific nets that are to be reported. If this parameter is specified, you must specify the `-plot` parameter.
	
	This parameter also allows you to specify the switched and always-on nets to be used for analysis reporting.
	
	The following is the use model to generate a report for the specified switched net:
	`report_power_rail_results -plot ir -nets {switchedNetName}`
	
	The following is the use model to generate a report for the specified always-on net, including its switched nets:
	`report_power_rail_results -plot ir -nets {alwaysOnNetName} -all`
	
	You must specify `-all` to include all switched nets associated with the specified always-on net.

`-plot plot_type`	
	Plots the specified plot type. See Plot Types for a description of each of the plot types.
`-range_max max_value`	
	Specifies the maximum value to report. Optional, but if specified, the `-plot` option is required, and requires both `-range_min` and `-range_max` to be specified.
`-range_min min_value`	

	Specifies the minimum value to report. Optional, but if specified, the `-plot` option is required, and requires both `-range_min` and `-range_max` to be specified.
`-region` *x1 y1 x2 y2*	
	Measures tap current and capacitance information in the specified region defined by x1, y1, and x2 y2 coordinates, and prints results in the log file.
`-rlrp_inst` *instance_name*	
	Reports the RLRP path for the specified instance. The default file name of the RLRP path report is `<`*rlrp_instname*`.rpt>`. You can modify the file name using the `-file_name` parameter.

Plot Types

Plot Type	Description
`ac`	Available/feasible capacitance (Farad) Displays the feasible decoupling capacitance that can be added in the area. The feasible capacitance in the area is calculated based on the filler cells placement. The program matches the placed filler cells with the decap cells available in the power-grid library and computes the feasible capacitance. The capacitance of the decap cell is stored in the power-grid library.
`cap`	Ggrid capacitance (Farad) Plots the capacitance of the extracted power-grid network.
`dd`	Decoupling capacitance (decap) density (Farad/um ^{2}) Plots decap density of the design. The decap density is the cell internal decoupling capacitance in the unit area (µm ^{2}).

`dr`	Decoupling capacitance required Reports the decoupling capacitance required to fix dynamic IR-drop violations. This option suggests how much decap needs to be added in a given area to fix the IR-drop violations.
`ivdn`	Instance Voltage Drop (Volts) – net-based Plots the net-based instance voltage drop.
`ivdd`	Instance Voltage Drop (Volts) – domain-based Plots the domain-based instance voltage drop. For domain based, at least one power and one ground net must be selected.
`fd`	Filler cell density (1/µm ^{2}) Displays the filler cell density per unit area (µm ^{2}). The plot can be used to analyze the feasibility of adding decap cells by replacing the filler cells and not changing the placement of the logic cells.
`freq`	Frequency domain (Hz) Displays the associated clock frequency of all instances in the design. The instances associated with multiple clock domains are displayed using the fastest clock frequency. This plot can be used to analyze and debug the power distribution in the design.
`ip`	Instance total power (mW) The total power consumed by each instance is displayed. The total power is the sum of the internal, switching and leakage power of the instance. The power data is read from the power database and can be specified using the `-power_db` option or it is queried directly from memory, if power calculation is run during the session.
`ip_i`	Instance internal power (mW) The power consumed by charging and discharging of the internal interconnect and device capacitances of the instance.
`ip_l`	Instance leakage power (mW) The power consumed by devices when not switching.

`ip_s`	Instance switching power (mW) The power consumed by charging and discharging of the interconnect or output load.
`ipd_i`	Specifies internal power density in mW/um ^{2} .
`ipd_l`	Specifies leakage power density in mW/um ^{2} .
`ipd_s`	Specifies switching power density in mW/um ^{2} .
`ipd_t`	Specifies total power density in in mW/um ^{2} .
`ir`	IRdrop or voltage drop (Volts) Displays the IRdrop across each extracted power-grid segment in the design.
`jrms`	Specifies current density as j/jmax based on the Irms current. This plot is only supported for dynamic analysis. A jrms gif file will be saved in the state directory that shows where the Irms based EM violation occurred. This plot type allows you to verify the analysis results of "jrms" using the GUI.
`load`	Loading capacitance (Farad) Displays the output loading capacitance driven by the instance. This plot can be used to analyze and debug the regions with high switching power.
`pi`	Power-switch (gate) current violation: I/Idsat Displays the ratio of current through the power-switch and the saturation current (Idsat) of the power-switch. This plot can be used to analyze and debug whether the current through power-switches exceeds the saturation current characterized for the power-switch devices. The saturation current and on-resistance of the power-switches are characterized and stored inside the power-grid library of the power-switch cell. A ratio of greater than 1 means that the current requirement of the power-gated block can not be met with placed power-switch instance. Either the power-switch needs to be made larger or power-switch instances need to be added.

`pv`	Voltage drop across power-switch (Volt) Displays the IRdrop across power-switch instances. This plot can be used to analyze and debug the IRdrop inside the power-gated block, e.g. if the IRdrop across power-switches is already high, the IRdrop inside the power-gated block will be much higher. In this case, the power-switch placement will have to be refined in order to resolve the IRdrop problem.
`rc`	Resistor current (Amp) Displays the current across power-grid resistor segments. This plot can be used to analyze and debug how the current flows from the voltage sources into the rest of the design, e.g. high current should flow from pads to the top-level routing level layers and then to the lower level metal layers. This plot can also be used to debug current density violations (`rj`).
`rlrp`	The total resistance of the instance along the least resistive path. Displays the total resistance of the instance along the least resistive path, meaning the resistance of the instance along the path from where it gets the most amount of current. This plot can be used to analyze and debug the power-grid integrity early in the design stage. An instance with high resistance is likely to have high static or dynamic IRdrop depending on the vector that is simulated later in the design cycle. This plot can be also used during decap optimization, since regions with high dynamic IRdrop and high resistance may not be improved upon by adding decaps, because the effectiveness of decoupling capacitance depends on the resistance of the power-grid network. You can also highlight the least resistive path of the instance using the GUI and selecting the instance with left mouse button. The *RLRP Path* checkbox should be enabled.
`res`	Displays the total resistance of the least resistive path from each node to voltage source. This plot can be used to view high resistance regions of the power-grid. It is useful during IRdrop and decap analysis to identify weak power-grid connectivity.

`rj`	Resistor current density, J/Jmax. Displays the current density (current per unit area) violation on each power-grid segment using the ratio of calculated current density and maximum allowed current density. A ratio of greater than 1 means that the current density limit of the segment is violated. The current density limits are supplied by the foundry and can be read during analysis using the `set_rail_analysis_config -em_models` command.
rs	Specifies the plot type for resistor sensitivity.
`slack`	Instance slack (Seconds) Displays worst instance slack. The slack data is queried from either the timing database or if the power database is specified using `-power_db` option, then the worst slack data is read from the power database. When using an external TWF, the slack data is read from the TWF.
`tc`	Tap current (Amp) Displays the current consumed by the instance devices or current taps. During rail analysis, it is based on the power consumption of the instance and is distributed inside the cell. The current distribution inside the cell is based on the extracted device netlist stored inside the power-grid library of the cell. The current distribution is generally based on the device width, length, and spice models. This plot can be used to analyze and debug IRdrop inside the macro.
`td`	Transition density (Hz) Displays the worst transition density (activity * frequency) of the instance. This plot can be used to analyze and debug activity distribution and high power density regions.
`unc`	Unconnected segments Displays power-grid segments that are disconnected from the voltage source (power pads). This plot can be used to debug unusually high IRdrop in the region.
`vc`	Voltage source current (Amp) Displays current through the voltage sources (power pads). This plot can be used to analyze the current carrying capacity of the power-pad.

`vu`	Voltage drop across package (Volt) Displays voltage drop across package. If analysis is run with package model attached to the power pads, this plot displays the IRdrop due to package RLCK elements.

Examples

- The following command will report the worst 100 IR segments on Metal 1 of the VDD net:
  ```
  report_power_rail_results -plot ir -layers M1 -limit 100 -nets VDD
  ```
- The following command will report the IR drop of the VDD_ring switched net:
  ```
  report_power_rail_results -plot ir -net {VDD_ring}
  ```
- The following command will report the IR drop of the VDD_AO always-on net, including its switched nets:
  ```
  report_power_rail_results -plot ir -net {VDD_AO} -all
  ```

Related Topics

- "Static Power, IRdrop and EM Analysis" in the *Voltus User Guide*

view_last

```
view_last [-help]
```

Displays the last view window or the previous view. This command provides the same function as the Zoom Previous icon (bindkey *w*).

Parameters

`-help`	Prints a brief description of the `view_last` command. For a detailed description of the command, use the `man` command: `man view_last`.

Related Information

- `gui_zoom`
- `view_next`

view_next

```
view_next [-help]
```

Displays the next view window. This command provides the same function as the *Next* toolbar widget. You can use the `view_next` command or the *Next* toolbar widget to switch from the current display to the next location in the *Go To Saved Area* list or the next zoom level in the zoom sequence saved in the design memory. The equivalent bindkey is `Y`. When you press the `Y` bindkey or execute the `view_next` command repeatedly, the tool cycles through all saved views (saved areas and zoom levels).

Parameters

`-help`	Prints a brief description of the `view_next` command. For a detailed description of the command, use the `man` command: `man view_next`.

Related Topics

- `gui_zoom`
- `view_last`

write_to_gif

```
write_to_gif
[-help]
out_file
[-honor_umask]
```

Saves a snapshot of the current screen to a GIF file with the specified name in the current directory. You can use this command at any stage of the design flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `write_to_gif` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_to_gif`.
`out_file`	Specifies the name of the output file as a string.
`-honor_umask`	Honors the umask settings while saving the GIF file. If you specify this option, the permission of the saved GIF file will be set by subtracting the umask value from the full permission value. For example, if the full permission of file is `0666` and the umask setting is `0022`, the tool saves the GIF file with the permission setting of `0666 - 0022`, that is `0644`. *Data_type*: bool, optional

Example

The following example captures the current screen and saves it to *`test1.gif`* in the current directory.

```
write_to_gif test1.gif
```

Related Information

- `gui_write_picture`

11

ILM and FlexILM Commands and Attributes

- clear_active_logic_view
- commit_module_model
- create_active_logic_view
- create_ilm_from_files
- flatten_ilm
- get_ilm_type
- ilm Category Attributes
- read_ilm
- report_ilm_status
- reset_ilm
- set_ilm
- set_ilm_type
- unflatten_ilm
- update_partition
- write_ilm

clear_active_logic_view

```
clear_active_logic_view
[-help]
```

Resets the reduced timing graph created by `clear_active_logic_view` back to the original timing graph.

Parameters

`-help`	Outputs a brief description of the `clear_active_logic_view` command. For a detailed description of the command, use the `man` command: `man clear_active_logic_view`

commit_module_model

```
commit_module_model
[-help]
[-copy_mmmc_latencies]
[-merged_mmmc_map_file mapFiles]
[-merged_mmmc_sdc_files mergedSdcFiles]
[-mmmc_file fullChipSdc]
[-mmmc_module_model_tag modelTag]
[-propagate_clocks]
[-cpf_file filename | -1801 filename]Specifies and commits flexILM.
```

Commits module models.

Note: The ILM is not supported for Signoff in Tempus.

Parameters

`-1801 filename`	Specifies IEEE 1801 file.
`-copy_mmmc_latencies`	Copies clock latencies of all the analysis views at the top-level design and applies them after assembling the design: When this option is specified, the command will do following after assembling the design: `update_analysis_view -name $view -latency_file $latency_map($view)` Where `latency_map` has clock latency files for all the views.
`-cpf_file filename`	Specifies the name of the full chip CPF file to be used.
`-help`	Outputs a brief description that includes the type and default information for each `commit_module_model` parameter. For a detailed description of the command and all of its parameters, use the man command: `man commit_module_model`
`-merged_mmmc_map_file mapFiles`	Specifies merge hierarchical mmmc map files.
`-merged_mmmc_sdc_files mergedSdcFiles`	Specifies SDC files for different dies.
`-mmmc_file fullChipSdc`	Specifies the name of the full chip MMMC view file.
`-mmmc_module_model_tag modelTag`	Specifies MMMC module model tag.

`-propagate_clocks`	Changes clock ideal to propagated clock. When this option is specified, you can run the following commands:
	`set_interactive_constraint_modes [all_constraint_modes -active]`
	`set_propagated_clock [all_clocks]`

create_active_logic_view

```
create_active_logic_view
[help]
[-type {flat_top timing_budget}[-set_dont_touch | -unset_dont_touch ]]
```

Trims the timing graph to ignore logic inside the first level registers in partitions to improve runtime for timing-related commands.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_active_logic_view` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_active_logic_view`.
`-set_dont_touch`	Sets the dontTouch value as `true`. This freezes I/O interfaces (Interface logics and interface nets) and prevents the opt_design `-post_cts -hold` from optimizing the block interface logics. The ILM ECO data can be loaded back.
`-type {flat_top timing_budget}`	• `flat_top`: Used to mark the top-level timing graph to mask all logic inside the interface logic of each partition. • `timing_budget`: Instructs the command to use the same settings as `create_timing_budget` when marking the top-level timing graph. This option should be specified if you plan to run `time_design` or `opt_virtual` with ART based and later on run `create_timing_budget` based on the resulted timing graph.
`-unset_dont_touch`	Sets dontTouch value as `false`. This resets the dontTouch on I/O logics before reading the ECO data.

create_ilm_from_files

```
create_ilm_from_files
[-help]
[-cell block_name]
[-incremental]
-model_type {timing | cts | si}
[-overwrite]
[-si_timing_window_file xtwf_file.spef]
{-out_dir directory_name | -oa_cellview lib cell view}
[-verilog_file cell_name.v [-cell_prefix cell_prefix -uniquify_netlist]]
[-sdc_file file_name -timing_view view_name]
[-spef_file spef_file.spef -rc_corner RC_corner_name]
[-opt_stage stage]
```

Creates an ILM directory from existing data. It can only create non-merged models. This command is useful in situations where an ILM is to be created through external inputs or using data from third party models into a format that could be recognized by Tempus.

If the data files already exist in the specified directory, the tool stops and generates an error message. You can overwrite existing files by specifying -overwrite. You can add files to the directory by specifying `-incremental`.

The command also creates a tcl script (cell_name_ILM.tcl) that you can source in your command file. The script contains the following command:

```
read_ilm -cell cell_name -out_dir dir_name
```

Note: The ILM is not supported for Signoff in Tempus.

Parameters

`-cell` *`block_name`*	Specifies the name of the block.
`-cell_prefix` *`cell_prefix`*	Specifies the cell prefix.
`-oa_cellview` *`lib cell view`*	Specifies the library name, cell name, and view name in the OpenAccess database format.
-opt_stage *stage*	Specifies the optimization stage, preCTS or postCTS.

`-out_dir` *`dir_name`*	Stores the ILM data in the specified directory: the absolute directory path or the path relative to the current directory.
`-incremental`	Lets you add files in addition to the existing files to the directory.
`-model_type {timing | cts | si}`	Specifies the type of ILM model, i.e timing, cts or si,.
`-help`	Outputs a brief description that includes the type and default information for each `create_ilm_from_files` parameter. For a detailed description of the command and all of its parameters, use the man command: man `create_ilm_from_files`.
`-overwrite`	Overwrites existing files in the directory.
`-rc_corner` *`RC_corner_name`*	Specifies the RC corner name for MMMC analysis. It identifies the specified corner to be used with -spef_file.
`-sdc_file` *`file_name`*	Specifies an existing .sdc file that you want to include in the directory. It identifies the sdc file to be used with the specified view.
`-spef_file` *`spef_name`*`.spef`	Specifies an existing .spef file that you want to include in the directory. It identifies the spef file to be used with the specified corner.
`-timing_view` *`view_name`*	Specifies the view name for the ILM module. It identifies the specified view to be used with `-sdc`.

`-uniquify_netlist`	Uniquifies the input Verilog netlist. This ensures that there are no errors when ILMs are being loaded. Using this parameter, each sub-module for an ILM has a unique name, which is composed of the time stamp and its original name. For example, in top_sm WedJun31138212009$data_path, top_sm specifies the top-level design, WedJun31138212009 specifies the time stamp, and $data_path specifies the original module name. *Default*: `Off`
`-verilog_file` *`netlist_name`*`.v`	Specifies the verilog file you want to include in the ILM data directory.
`-si_timing_window_file` *`xtwf_file.spef`*	Specifies the .xtwf file that you want to include in the directory.

Examples

- The following commands create two module directories:

```
create_ilm_from_files -cell blk1 -out_dir A -model_type timing -verilog_file V1.v
-spef_file 1.spef -rc_corner corner1 -sdc_file sdcfile1 -timing_view view1


create_ilm_from_files -cell blk2 -out_dir A -model_type si -verilog_file V2.v -
spef_file 2.spef -rc_corner corner2 -sdc_file sdcfile2 -timing_view view2
```

- The command creates two tcl files in the current directory. The `blk1_ILM.tcl` file contains the following command:

```
read_ilm -out_dir A -cell blk1
```

  The `blk2_ILM.tcl` file contains the following command:

```
read_ilm -out_dir A -cell blk2
```

flatten_ilm

```
flatten_ilm
[-help]
```

Flattens the interface logic models (ILMs) at the top level so that the entire design can be analyzed at the top. When you use this command, the software switches from blackbox view to ILM view.

If you do not run `flatten_ilm`, the ILM netlist will not be visible to the Tempus software. You can switch between the ILM view and the blackbox view using the `flatten_ilm` and `unflatten_ilm` commands.

All super commands flatten the ILMs internally and hence no explicit `flatten_ilm` is required. However, you should use this command for other timing related commands. For example, for the `report_timing` command, you need to run `flatten_ilm` before reporting timing. It would not work in the unflattened mode.

Note: The ILM is not supported for Signoff in Tempus.

Parameters

`-help`	Outputs a brief description that includes the usage of the `flatten_ilm` command.

Example

```
flatten_ilm

report_timing
```

Related Topic

- `report_timing`

get_ilm_type

```
get_ilm_type
[-help]
[-model]
```

Displays the ILM model information.

Note: The ILM is not supported for Signoff in Tempus.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `get_ilm_type` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_ilm_type`
`-model`	Specifies the model type.

Example

- The following example displays the current setting for the `-model` parameter:

  ```
  get_ilm_type -model
  ```

  The software displays the following information:

  ```
  timing
  ```

Related Topic

- `set_ilm_type`

ilm Category Attributes

The root attributes in the ilm category are:

- ilm_filter_internal_path
- ilm_keep_async
- ilm_keep_flatten
- ilm_keep_high_fanout_ports
- ilm_keep_inst_in_sdc
- ilm_keep_loopback
- ilm_max_num_insts
- ilm_max_num_registers
- ilm_slack_driven

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `ilm` category, use the following command:

```
get_db -category ilm
```

ilm_filter_internal_path

```
ilm_filter_internal_path {true|false}
```

Default: `false`

Data_type: `bool`

When set to true, group paths are created for internal paths with slack adjustment of 9999. Therefore, all internal paths are filtered out from the timing report.

Related Command

- `write_ilm`

ilm_keep_async

```
ilm_keep_async {true|false}
```

Default: `false`

Data_type: `bool`

Keeps paths from asynchronous input ports. For example, paths from an input port to the reset pin of interface sequential instances, but not to the internal sequential instances.

ilm_keep_flatten

```
ilm_keep_flatten {true|false}
```

Default: `true`

Data_type: `bool`

Makes flow work under the flattenIlm mode.

ilm_keep_high_fanout_ports

```
ilm_keep_high_fanout_ports {true|false}
```

Default: `true`

Data_type: `bool`

Keeps high-fanout input port paths.

ilm_keep_inst_in_sdc

```
ilm_keep_inst_in_sdc {true|false}
```

Default: `false`

Data_type: `bool`

Keeps instances which have constraints on it.

ilm_keep_loopback

```
ilm_keep_loopback {true|false}
```

Default: `false`

Data_type: `bool`

Keeps loop back paths.

When set to `true`, then any path touching a net used for the ILM model is also included. This is done for better model accuracy, so that the actual capacitance on an output port could affect the delay on a net which feeds an internal path that is normally not included in the ILM model. This also degrades the reduction ratio (`#ILM_insts`/`#all_inst`) for the block.

ilm_max_num_insts

`ilm_max_num_insts` *maxInstances*

Default: `1000`

Data_type: int, read/write

Specifies the maximum number of instances.

ilm_max_num_registers

`ilm_max_num_registers` *maxRegisters*

Default: `40`

Data_type: int, read/write

Specifies the maximum number of registers.

ilm_slack_driven

`ilm_slack_driven {true|false}`

Default: `false`

Data_type: bool, read/write

Specifies slack driven ILM generation.

read_ilm

```
read_ilm
[-help]
[-no_si]
[-no_spef]
[-oa_cellview cell view]
```

Specifies ILM data directory for the specified block. You can run `read_ilm` multiple times in the same session. Each time you specify `read_ilm`, the software overwrites the previous setting for the block. If master/clones exist in the design, the cell name will have the name of the master partition.

Note: You can use this command only if the ILMs are unflattened (`unflattenIlm`) or the design is in black box view. You cannot change ILM settings in flattened or ILM view.

The syntax is order-dependent. You must specify `-cell` before you specify `-directory`.

Note: The ILM is not supported for Signoff in Tempus.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_ilm` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_ilm`
`-no_si`	Specifies to read the non-SI data from the ilm directory.
`-no_spef`	Specifies to read the non-Spef data from the ilm directory.
`-oa_cellview` *`cell view`*	Specifies the library name, cell name, and view name in the OpenAccess database format.

Related Topic

- `write_ilm`

report_ilm_status

```
report_ilm_status
[-help]
```

Reports ILM status information including ILM block names, ILM data locations, and if the design is in the flattened or unflattened state.

You can use this command at any time in the flow, but it only reports ILM information if you have already specified ILMs in the design.

Note: The ILM is not supported for Signoff in Tempus.

Parameters

`-help`	Outputs a brief description of the `report_ilm_status` command. For a detailed description of the command, use the man command: `man report_ilm_status`

Examples

The following command generates the ILM status report for `dtmf_chip`:

```
report_ilm_status

*** ILM status report ***

Current design: dtmf_chip

The following blocks are specified as an ILM block:

  Hierarchical instance(HInst) DTMF_INST/RAM_128x16_TEST_INST (cell: ram_128x16_test)

  Hierarchical instance(HInst) DTMF_INST/RAM_256x16_TEST_INST (cell: ram_256x16_test)

  Hierarchical instance(HInst) DTMF_INST/TDSP_CORE_INST (cell: tdsp_core)

  Hierarchical instance(HInst) DTMF_INST/RESULTS_CONV_INST (cell: results_conv)

Current view is ILM (after flattenIlm)

Current ILM data is for cts-mode.

ILM blocks data location:

ram_128x16_test  :  ../../PTN/ram_128x16_test/ilm
```

```
ram_256x16_test  :  ../../PTN/ram_256x16_test/ilm

tdsp_core  :  ../../PTN/tdsp_core/ilm

results_conv  :  ../../PTN/results_conv/ilm

*** end of ILM status report ***
```

reset_ilm

```
reset_ilm
[-help]
-cell cellName
```

Unspecifies the ILM cell.

This is used to unspecify a previously specified ILM. This would be useful in situations where ILMs are to be removed from the design, to put them in non-ILM or black box mode.

Note: The ILM is not supported for Signoff in Tempus.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reset_ilm` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reset_ilm`
`-cell` *`cellName`*	The name of the block to unspecify.

Related Topic

- `set_ilm`

set_ilm

```
set_ilm
-cell cellName
-in_dir dirName
```

Specifies ILM data directory for the specified block. You can run `set_ilm` multiple times in the same session. Each time you specify `set_ilm`, the software overwrites the previous setting for the block. If master/clones exist in the design, the cell name will have the name of the master partition.

Note: The ILM is not supported for Signoff in Tempus.

Parameters

`-cell cellName`	Specifies the block name.
`-in_dir dirName`	Specifies the directory for the files you want to read.

Example

- The following commands specifies directory `dir_dsp_core` for block `tdsp_core`, and directory `dir_onemore` for block `onemore`:

  ```
  set_ilm -cell tdsp_core -in_dir dir_dsp_core
  set_ilm -cell onemore -in_dir dir_onemore
  ```

Related Topic

- `reset_ilm`

set_ilm_type

```
set_ilm_type
[-help]
[-model {timing | si}]
```

Controls the behavior of commands in the presence of ILMs.

ILM models can be an SI model or a non-SI model. In the flow, sometimes an explicit `flatten_ilm` needs to be performed to perform the underlying operation. Based on the nature of operation, an appropriate model could be invoked/switched with the above command.

Use the `get_ilm_type` command to display the current settings for the `set_ilm_type` command.

Note: The ILM is not supported for Signoff in Tempus.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_ilm_type` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_ilm_type`.
`-model {timing | si}`	Specifies the ILM model that is to be used by the `flatten_ilm` command. All the timing-related commands require data to be in the flattened state (physical command require unflattened state). Design commands such as `optDesign`, `clockDesign`, `timeDesign`, and so on automatically set this parameter before flattening ILMs. However, the CTE commands and some clock commands, which can be run in the flattened mode require manual settings. • Use the `si` model for flattening the SI ILM model. • Use the `timing` model for flattening the non-SI related timing commands. **Note:** In case of non-merged models, use the `cts` model before running `flatten_ilm` followed by the command which directly deals with the clocks. *Default*: `timing`

Examples

- The following command can be used to get timing reports containing the SI push-out delays on nets using the `-set_ilm_type -model` command:

```
set_ilm_type -model si

# Flattens to SI model

flatten_ilm

# Turns on SI aware delay calculation

set_db delaycal_enable_si true

# Adds incremental delay column (for SI push-out delays)
#  in timing output:

set_global_report_timing_format {instance arc cell fanout load slew delay
incr_delay arrival}

# Minimizes the width of the report such that it easily
# fits into the screen without wrapping

set_table_style -name report_timing -no_frame -indent 0

report_timing
```

Note: You can also invoke the Global Timing Debugger (*Timing - Debug Timing - Generate*)

Related Topic

- `get_ilm_type`

unflatten_ilm

```
unflatten_ilm
[-help]
```

Reverts the ILM back to its original hard macro view.

You can use this command after the `flatten_ilm` command.

Note: `unflatten_ilm` will take no effect if the design is already in the unflattened mode.

Many physical commands would not work in the flattened mode. They would need to be unflattened before performing the operation.

For example:

```
flatten_ilm

report_timing

unflatten_iLM
```

Note: The ILM is not supported for Signoff in Tempus.

Parameters

`-help`	Outputs a brief description that includes the usage of the `unflatten_ilm` command.

Example

The following command flattens all partitions and their ILM designs to the top-level. After reporting timing, the design is reverted back to the unflattened state with the `unflatten_ilm` command.

```
flatten_ilm

report_timing

unflatten_ilm
```

Related Topic

- `flatten_ilm`

update_partition

```
update_partition
[-pin_location]
[-post_eco_suffix postECOSuffix]
{-flex_ilm_eco}
{-golden_block_dir directory}
-flex_ilm_dir directory
[-eco_dir directory]
[-full_netlist_with_eco_dir directory -flex_ilm_eco_dir directory]
```

Creates a new partitioned output that contains the optimised blocks and top. It also performs ECO on the original block. The ECO output is saved as new Tempus database in the partitioned block. The ECO would happen only on `flexILM` partitions and not the regular blocks.

Note: The ILM is not supported for Signoff in Tempus.

Parameters

`-eco_dir` *directory*	Specifies the scratch directory for partitioning and running distributed ECO job after optimization.
`-flex_ilm_dir` *directory*	Specifies the new partitioned output directory containing optimized blocks/top and ECO results.
`-flex_ilm_eco`	Performs ECO on FlexILM modules. **Note:** This is default.
`-flex_ilm_eco_dir` *directory*	Specifies the directory to store the ECO job.
`-full_netlist_with_eco_dir` *directory*	Specifies the directory to store the ECO result of the full netllist.
`-golden_block_dir` *directory*	Specifies the golden database directory for individual blocks.
`-pin_location`	Updates new partition pin locations into the golden block DB. This option does not allow new added logic ports.
`-post_eco_suffix` *postECOSuffix*	Specifies the suffix name that will be part of generated postECO DB name.

write_ilm

```
write_ilm
[-help]
-dir dirname
[-model_type {timing|si|all}]
[-no_core_sdc]
[-no_spef]
[-no_xtwf]
[-ignore_ports ListOfPorts]
[-include_objects ListOfInstancesorNets]
[-validation_dir]
[-write_sdf]
```

Creates the specified directory containing the Interface Logic Module (ILM) data for timing analysis and the Extended Interface Logic Module (XILM) data for SI analysis.

The information is saved in the following directories:

For Timing:

- `$dir/ilm_data`
    - `*.v`
    - `*.sdc`
    - `*.spef`

For SI:

- `$dir/si_ilm_data`
    - `*.v`
    - `*.sdc`
    - `*.spef`
    - `*.xtwf`

Note: The ILM is not supported for Signoff in Tempus.

Parameters

`-help`	Prints out the command usage.
`-dir` *dirname*	Specifies the location of the ILM and XILM files, which are generated in the `ilm_data` and the `si_ilm_data` subdirectories, respectively.
`-include_objects` *ListOfInstancesorNets*	Accepts a collection of instances and/or nets. All instances and nets specified are preserved in the generated ILM. When instances are preserved, their driving and load nets are also preserved. When nets are preserved, only the driving instances are preserved.
`-ignore_ports` *ListOfPorts*	The given ports and the associated interface path(s) which are starting or ending at these ports will not be included in the ILM model.
`-model_type {timing|si|all}`	Generates ILM and XILM models. • The `-model_type timing` parameter creates only ILM models. • The `-model_type si` parameter creates only XILM models. • The `-model_type all` parameter creates both the ILM and XILM models.
`-no_core_sdc`	Does not create the core SDC data for ILM/XILM models.
`-no_spef`	Does not generate the SPEF data for ILM and XILM models.
`-no_xtwf`	Does not create the xtwf data for ILM/XILM models. **Note:** When the `-no_core_sdc` and `-no_xtwf` parameters are specified, `write_ilm` does not call any delay or timing calculation and skips creating both the SDC and XTWF files for ILM/XILM models.
`-validation_dir`	Creates a script and necessary data used to validate ILM/XILM.
`-write_sdf`	Creates an SDF file that is based only on the logic kept in the ILM model after netlist is trimmed. In the MMMC mode, one SDF file is written for one view.

Examples

- The following command generates the ILM ignoring certain ports and including certain objects in the `BlockB` directory. It also writes out the ILM SDF file under the `BlockB` directory.

  ```
  ilmview > write_ilm -write_sdf -ignore_ports $ignorePorts include_objects
  $includeObjects -dir BlockB
  ```

- In SMSC designs, the following command generates ILM files in the three subdirectories (`ilm_setup`, `ilm_hold` and `si_ilm_setupHold`) under directory BlockB:

  ```
  ilmview > write_ilm -dir BlockB
  ```

- In MMMC designs, the following command generates ILM files in the one subdirectories (MMMC) under directory BlockB:

  ```
  ilmview > write_ilm -dir BlockB
  ```

Related Topic

- `read_ilm`

12

Import and Export Commands and Attributes

- add_route_vias Category Attributes
- check_design
- design Category Attributes
- init_design
- init Category Attributes
- read_db
- read_def
- read_instance_voltage
- read_libs
- read_mmmc
- read_netlist
- read_physical
- read_physical Category Attributes
- read_rcdb
- read_sdc
- relink_db_files
- set_cell_power_domain
- set_library_unit
- set_power_def_files
- set_power_lib_files
- write_db
- write_db Category Attributes
- write_def

- write_def Category Attributes
- write_mmmc
- write_netlist Category Attributes
- write_via_defs
- check_design Category Attributes
- reset_sdc
- switch_top_cell

add_route_vias Category Attributes

The root attributes in the `add_route_vias` category are:

- add_route_vias_auto
- add_route_vias_ndr_only

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `add_route_vias` category, use the following command:

```
get_db -category add_route_vias
```

add_route_vias_auto

```
add_route_vias_auto {true | false}
```

Default: `false`

Enables auto via generation when design is loaded. The option is of type bool.

add_route_vias_ndr_only

```
add_route_vias_ndr_only {true | false}
```

Default: `false`

Generates vias for non default rules only.

check_design

```
check_design
[-help]
[-id violation_id]
[-out_file fileName]
{[-type {power_intent timing hierarchical pin_assign budget assign_statements place opt
cts route signoff all}] | -type_3d {tech_lib netlist power_intent timing place floorplan
route report_only all}} [-format {table detail}][-no_check ]}
```

Checks that all prerequisites to run all or part of the hierarchical or block implementation flow are met.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `check_design` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_design`.
`-format {table detail}`	Specifies the format for reporting violations. • `table`: Prints the results in the table format. • `detail`: Prints details of violations.
`-id violation_id`	Specifies the violation IDs.

`-type`	Specifies the category of the flow to be checked: • `power_intent`: Performs checks related to MSV setup including power domain/fence checks. • `timing`: Performs checks related to timing setup integrity. • `hierarchical`: Performs check if the partition has a fence and if the fence is inside the core boundary. • `assign_pin`: Performs checks related to pin assignment for partitioning flows. • `budget`: Performs checks related to time budgeting for partitioning flows. • `assign_statements`: Performs checks for the netlist for assign statements. • `place`: Performs checks related to placement such as PG rail alignment and pin access issues. • `opt`: Performs checks related to optimization such as checks for available cells. • `cts`: Performs checks related to clock tree definition such as constraint targets and route types. • `route`: Performs checks related to routing such as congestion, placement, and pin access issues. • `signoff`: Checks related to the signoff activities including extraction and ECOs. • `all`: Runs all categories.

`-type_3d {tech_lib netlist power_intent timing place floorplan route report_only all}}`	Specifies the 3DIC design categories to check. You can specify single or multiple categories • `tech_lib` ◦ Checks the library integrity and corrects binding ◦ Checks all timing lib wh • `netlist` ◦ Checks the library integrity and corrects binding • `power_intent` ◦ Checks related to MSV setup including power domain/fence checks. ◦ Checks related power domain bind to correct design • `timing` ◦ Performs checks related to timing setup integrity. • `place` ◦ Checks related to placement such as pin alignment • `floorplan` ◦ Checks related to floorplanning • `route` ◦ Checks related to routing • `all` ◦ Checks all categories

`-no_check`	Just echo out all the messages that would be checked, don't do any checks Do not run the checks but echo the checks that would run per the categories provided by the user
`-out_file` *fileName*	Specifies the output report file. Contents dumped to this file in the Tcl format.

design Category Attributes

The root attributes in the design category are:

- design_backside_bottom_routing_layer
- design_backside_top_routing_layer
- design_bottom_routing_layer
- design_compressed_pg_db
- design_cong_effort
- design_early_clock_flow
- design_ideal_hold_fixing
- design_merge_trim_shapes
- design_optimization_density_screen_margin
- design_pessimistic_mode
- design_tech_node
- design_top_routing_layer
- design_trim_grid_group

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `design` category, use the following command:

```
get_db -category design
```

design_backside_bottom_routing_layer

`design_backside_bottom_routing_layer` *layerName*

Default: `""`

Data_type: string, read/write

Specifies the highest backside LEF layer name for global and detail routing.

design_backside_top_routing_layer

```
design_backside_top_routing_layer layerName
```

Default: ""

Data_type: string, read/write

Specifies the lowest backside LEF layer name for global and detail routing.

design_bottom_routing_layer

```
design_bottom_routing_layer layerName
```

Default: ""

Data_type: bool

Specifies the lowest LEF layer name for global and detail routing.

design_compressed_pg_db

```
design_compressed_pg_db {true | false}
```

Default: `true`

Data_type: bool

Specifies whether the compressed PG feature is enabled. When it is set to `true`, `read_db` will automatically compress the PG data while loading the design. You can use `get_db current_design .is_pg_compressed` to check if PG in the design is compressed.

design_cong_effort

```
design_cong_effort {low medium high auto}
```

Specifies the effort level for relieving congestion.

Default: `auto`

Data_type: `enum {low medium high auto}`

Specify one of the following values:

- `auto`: Automatically determines whether the design is congested and performs extra congestion driven effort for highly congested designs.

- `high`: Runs more iterations of placement in an effort to achieve better congestion results. This parameter increases the run time.
- `low`: Runs fewer iterations of placement to arrive quickly at a legal placement. This parameter might decrease placement quality.
- `medium`: Runs placement on designs at a normal effort level.

design_early_clock_flow

```
design_early_clock_flow {true | false}
```

Default: `false`

Data_type: `bool`

Enables early clock flow.

design_ideal_hold_fixing

```
design_ideal_hold_fixing {true | false}
```

Default: `false`

Data_type: bool, read/write

Enables the ideal hold fixing flow.

design_merge_trim_shapes

```
design_merge_trim_shapes {{layers layer_name_list gap gap_value max_merge_num number_of_ma
x_merge masks {mask_num ...}} ...}
```

Default: `""`

Data_type: string, read/write

Specifies the setting for trim shapes merge.

- `layers`: You use this t specify the trim layer list which needs to merge trim shapes.
- `Gap`: You use this setting when the end to end gap is less the value.
- `max_merge_num`: You use this to specify the max trim shapes number that are merged.
- `masks`: You use this to specify the trim shapes color that needs to be merged.

design_optimization_density_screen_margin

```
design_optimization_density_screen_margin {{layers layer_name_list gap gap_value max_merge
_num number_of_max_merge masks {mask_num ...}} ...}
```

Default: 0.1

Min: 0
Max: 1

Data_type: double, read/write

Increases margin on the top of original density screen for optimization.

design_pessimistic_mode

```
design_pessimistic_mode {true false}
```

Default: false

Data_type: enum {true false}

Specifies that the delay calculation compatibility mode should be used for correlating to external signoff analysis tools. This mode adjusts delay calculation settings to ensure the best match to the third party engine while maintaining a degree of pessimism to help limit optimistic outliers in the external signoff analysis tool.

design_tech_node

```
design_tech_node {N12 N10 N7 N7Plus N6 N5 N3 S11 S10 S8 S7 S5 S4 S3 G7 G5 ICF I7
unspecified}
```

Default: unspecified

Data_type: enum {N22 N12 N10 N7 N7Plus N6 N5 N5PlusPlus N4 N3 N3E N2 A16 A14 A12 S11 S10 SF8 SF7 SF5 SF4 SF3 SF2 SF1.4 G7 G5 ICF I7 I5 P1222 P1276 P1277 P1278 P1278x P1280 20A 18A C12 C7 unspecified}

Sets the design technology node.

The node value is normally automatically set by read_physical while loading the LEF technology and the tool prints a message mentioning the auto-detected node. However, if the auto-detection does not work, you will see a message like this:

```
## Check design process and node:
## Both design process and tech node are not set.
```

In such cases, you will need to set the value directly

using `design_process_node` and `design_tech_node`.

design_top_routing_layer

`design_top_routing_layer` *`layerName`*

Default: `""`

Data_type: `string`

Specifies the highest LEF layer name for global and detail routing.

design_trim_grid_group

`design_trim_grid_group` *`groupName`*

Default: `""`

Data_type: `string`

Specifies the GROUP name of the set from which the trim metal grid will be used for placement and routing.

If the specified group name does not exist in LEF, the tool will return an ERROR.

init_design

`init_design [-help]`

Initializes the database and ensures that the tool is ready for full execution.

During `init_design`, the tool steps through the defined MMMC objects, the design object, and the power requirements and builds up the full representation of the design. After initialization is complete, the design is in a consistent state and is ready for manipulation (floorplanning, synthesis, and so on) or timing.

The `init_design` command corresponds to the *Init* step in the Initialization flow. In this step, the full database is prepared for execution. The constraints are loaded into the correct mode, the power data is loaded and applied to the appropriate instances, and the operating conditions are applied to the power domains. Extensive error checking is performed, including checking of objects, and checking for completeness of data. After the *Init* step is complete, the database is completely consistent and ready for execution.



The `init_design` command can be broken into the following sub-steps, which are executed by the tool without requiring any user intervention:

1. Power Domain Creation

First, the power domains are populated by the design data. The design instances associated with a particular domain are defined in the Common Power Format (CPF) or UPF file. This instance data is used to populate the power domain attributes. At the end of this sub-step, the different power domains are resolved, and every instance in the system is associated with its power domain.

2. Power Domain Binding to Domain Corners

After the power domains are fully resolved, they can be bound to the library sets in the various `create_domain_corner` commands. In this sub-step, the active domain corners are processed with each instance getting assigned to the appropriate library set.

3. Power Domain Object Processing

After the power domains are associated with library sets, the power domain objects (state retention, iso cells, and level shifters) can be processed. The power object rules are processed against the defined library sets for each domain to identify the power cells to be used.

4. Constraint Loading

During constraint loading, the SDC files required by the active view are loaded and error checking is done. In addition, the RC corner defined by the required domain corner is processed and loaded. In both cases, syntax and error checking is performed.

5. Active View Setting

The final step is to set the active view based on the `set_active_view` command, and to perform final checks to ensure the view is fully and completely defined. At the conclusion of this command, the system is ready for timing.

After a design has been initialized with `init_design`, the design is ready for execution - which can include incremental updates to timing.

Note: The annotation data, such as SPEF or SDF, cannot be loaded if the design has not been initialized properly with the `init_design` command.

Timing Incremental Updates

After `init_design`, you can enter MMMC commands at the command line. These commands populate the mmmc database objects, and perform error checking to report missing objects. For example, if you create a timing condition involving libsets that do not exist, an error will be issued. No data load takes place during command entry. Instead, the loading of new libraries, constraints, and so on, is deferred until an additional `init_design` is entered.

After you have completed entering the MMMC commands, issue the `init_design` command to load the new timing intent. After `init_design` is complete, the database is ready for timing again.

The exception to requiring `init_design` is `set_analysis_view`. When this command is issued and datasets are not present in memory, the tool will effectively issue an `init_design` to load in the required data. If the data is already in memory, the view information is changed without any additional load.

Note: Design data (netlist) and physical data (LEF/OA) incremental update is not allowed. After the first `init_design` is completed, these steps are effectively disabled. If you issue a `read_netlist` or `read_physical` command after the first `init_design`, the tool returns an error.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command, use the `man` command: `man init_design`

Related Topics

- `init Category Attributes`
- `read_netlist`
- `read_physical`

init Category Attributes

The root attributes in the `init` category are:

- init_check_netlist
- init_delete_floating_hnets
- init_keep_empty_modules
- init_read_netlist_allow_port_mismatch
- init_read_netlist_allow_undefined_cells
- init_oa_tech_lib
- init_power_intent_files
- init_sync_relative_path
- init_timing_enabled

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `init` category, use the following command:

```
get_db -category init
```

init_check_netlist

```
init_check_netlist {true | false}
```

Default: `false`

Data_type: `bool`

After reading in the netlist, it performs the same checks as `check_design -netlist`.

init_delete_floating_hnets

```
init_delete_floating_hnets {true | false}
```

Default: `true`

Data_type: `bool`

If true, an hnet (Verilog wire statement) that has nothing connected to it, is deleted from the netlist. The default value is true in Tempus to delete non-timed nets and reduce memory.

init_keep_empty_modules

```
init_keep_empty_modules {true | false}
```

Default: `true`

Data_type: `bool`

Keeps empty modules. If set to `false`, the software converts the empty modules into special leaf cells that do not have a physical library associated with them.

If the module is a hard macro, using a value of `true` or `false` produces the identical behavior: the software keeps the module.

Related Commands

- `read_db`

init_read_netlist_allow_port_mismatch

```
init_read_netlist_allow_port_mismatch {true | false}
```

Default: `false`

Data_type: `bool`

Specifies whether new Instance ports can be created during Verilog netlist parsing.

init_read_netlist_allow_undefined_cells

```
init_read_netlist_allow_undefined_cells {true | false}
```

Default: `false`

Data_type: bool

Creates a dummy .lib file and a user-defined noise model (UDN) for cells that are not present in the loaded .lib or cdB.

init_oa_tech_lib

```
init_oa_tech_lib lib_name
```

Default: ""

Data_type: string, read/write

Specifies the OpenAccess library to use as a source of technology information. If not specified, the first library in the `init_oa_ref_lib` list will be used. Applies only when `read_netlist` is used to process a Verilog netlist. It is set by the `read_physical -oa_tech_lib` command.

init_power_intent_files

```
init_power_intent_files string
```

Default: ""

Data_type: `string`

Specifies the power intent files read by the previous run of the `read_power_intent` command.

init_sync_relative_path

```
init_sync_relative_path {true | false}
```

Default: `false` (You must change your working directory to the directory saved in the previous design session before you restore the design.)

Data_type: `bool`

Synchronizes all relative paths in the top.globals file to the current working directory. Use this parameter if you want to restore the design without first changing your current working directory to the directory saved in the previous design session.

Example

- The following command synchronizes all relative paths to the current working directory:
  `set_db init_sync_relative_path true`

Related Commands

- `read_db`

init_timing_enabled

`init_timing_enabled {true | false}`

Default: `true`

Data_type: `bool`

If `read_physical` is issued before `read_mmmc` or `read_libs` in Genus and Tempus, an error will be issued. In Innovus, doing this will result in `init_timing_enabled` being set to `false` by the tool. Innovus is now in physical only mode, and any attempt to set timing data will result in an error.

Related Commands

- `read_physical`
- `read_mmmc`
- `read_libs`

read_db

```
read_db
[-help]
[-debug]
[-module_dirs list_of_dirs]
[-no_timing_graph]
[-read_globals tcl_file_name]
[-physical_data]
{[<in_dir> [-mmmc_file <string>]] | [-oa_lib_cell_view {<libname> <cellname> <viewname>}
[-mmmc_file <string>][-lef_files list_of_files]]}
[-setup_views <string> -hold_views <string>]
[-leakage_view <string> -dynamic_view <string>]
```

Loads the saved design files of a previous design session. The files included depend on the work completed in the design session when it was saved. The directory can include floorplan files, special route files, placement files, early global route files, and power switch state files.

To restore designs saved in Tempus format, usage is as follows:
`read_db {sessionName.dat}`

Note: You must rebuild the timing graph after you restore the design. Run extraction, then timing analysis.

You must specify all pin types in the LEF file. Without these definitions, the Tempus software attempts

to determine the pin type from the pin name when restoring a design, and does not always do this correctly. If this is the case, Tempus generates the following error message:

```
**ERROR: Current net list does not match wires file.
```

> Modifying the contents of the `sessionName.dat` directory created by `saveDesign` can cause major unexpected results. Never modify the `design.dat` directory contents.

To restore OpenAccess designs, usage is as follows:

```
read_db
{-oa_lib_cell_view string}
```

You can use the `read_db` command after starting the Tempus software.

Note: Tempus supports only the following begin/end extension values for DEF `NETS` style wiring segments (0, width/2 or LEF `LAYER WIREEXTENSION`). If any of the wire extensions has unsupported extension value, then Tempus converts it to a special wire (DEF `SPECIALNETS`). While writing back OpenAccess, these special wires will again be converted back to DEF `NETS` style wiring.

From the 14.1 release, the `read_db` command accepts a tar file, when a tar ball is given. For instance, if you specify the following command:

```
read_db myDesign.inn.dat.tar.gz
```

`read_db` will untar the tar ball under the current working directory, restore the design, and then remove the temporary data directory. This enables a single DB file operation all the time.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `read_db` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_db`
`-dynamic_view string`	Specifies the analysis view for the dynamic power analysis.
`-debug`	Enables debug data.

`-hold_views` *string*	Specifies the hold views for loading, rather than the hold views active during `write_db`.
in_dir	Specifies the `.dat` directory from which to load the files. If the session name of the saved design includes the extension `.enc`, make sure that you include this extension in the *in_dir* name: for example, `session1.enc.dat`.
`-leakage_view` *string*	Specifies the analysis view for leakage power analysis.
`-lef_files` *list_of_files*	Specifies the LEF files name for loading. Use this option if you want to override the existing LEF files in the design being restored and load different LEF files instead. • This option is intended to allow upgrading to new versions of LEF files that are compatible with the old versions. The list of all LEF files used by this design must be given. • If any pin locations or cell boundaries or OBS shapes are changed for used cells, new DRC violations may occur. • If pins are added or removed from used cells, or used cells are removed, fatal errors may occur when loading the netlist. • If the removed pins and cells are unused in the design, warnings or errors may still occur if any library-specific data was stored for those cells or pins during the design flow (such as dont_touch data) **Examples** Existing LEFs used by `write_db`: `tech.lef`, `A.lef` You can change `A.lef` to `A_v2.lef` as given below: `read_db place.dat -lef_files {tech.lef A_v2.lef}`

`-mmmc_file`	Specifies the path name to a Multi-Mode/Multi-Corner (MMMC) configuration file (`viewDefinition.tcl`). Use this option if you want to override the settings in the existing MMMC configuration file in the design being restored and instead load a different MMMC file.
`-module_dirs` *list_of_dirs*	Specifies module-level sessions which should override the corresponding modules in the top-level hierarchical session.
`-no_timing_graph`	Restores the design data without a timing graph.
`-oa_lib_cell_view` *string*	Specifies the library name, cell name, and view name for restoring design information from the previous design session in the OpenAccess database format.
`-physical_data`	Enables physical data to be read.
`-read_globals` *tcl_file_name*	Specifies the UDM variables to be honored when restoring the design. You need to provide a tcl file stating the pre-defined UDM variable names to be honored.
`-setup_views` *string*	Specifies the setup views for loading rather than the setup views active during `write_db`.

Examples

- The following command loads the saved design files from the `test.inn.dat` directory:

  ```
  read_db test.inn.dat
  ```

- The following command restores the `my.enc.dat` design without loading the timing libraries and MMMC configuration information:

  ```
  read_db -no_timing_graph my.inn.dat
  ```

- The following command restores the `placed.enc.dat` design but overrides the existing MMMC settings in the design with the settings in the `new_mmmc.tcl` file:

  ```
  read_db placed.inn.dat -mmmc_file new_mmmc.tcl
  ```

- The following command restores the `"designLib top placed"` OpenAccess cellview but overrides the existing MMMC settings in the design with the settings in the `new_mmmc.tcl` file:

  ```
  read_db -oa_lib_cell_view "designLib top placed" -mmmc_file new_mmmc.tcl
  ```

Related Topics

- `write_db`

read_def

```
read_def
[-help]
[def_files]
[-design]
[-design_offset (x y)]
[-design_orient orientation]
[-force_create_phys_inst]
[-keep_pin_geometry]
[-preserve_shape]
[-rdl_def def_file]
[-rdl_offset (x y)]
[-rdl_orient orientation]
[-read_flat_dir directoryName]
[-skip_pg]
[-skip_signal_nets]
[-top_scope | -top_scope_ignore_block_internal_nets_on_boundary_path ]]
[-write_flat_dir directoryName]
```

Reads a single DEF file. This command can be used to supply single DEF to generate a physical database.

This command can be used at any step after importing a design. The typical usage is:

- Load a DEF file containing the floorplan saved by Tempus or other tool.
- Load a DEF file containing the scan chain information so that placement can do scan chain reorder.
- Load a DEF file containing the cell's placement and routing saved by Tempus or other tool.

The `read_def` command cannot read multiple or hierarchical DEF files. To read multiple or hierarchical DEF files and create a flattened database, use the `read_hierarchical_def` command.

ⓘ When you load a DEF file, you must be at the same level in the hierarchy where the DEF file was saved.

The Tempus software loads the DEF file information into the database in several ways, depending on the object:

- Tracks, gcells, rows, die area

The software deletes all of the existing objects in the database and reads in the new information.

Note: The `read_def` command can read a rectilinear die area, but converts it to a rectangular die area with blockages at the cut off area.

- Blockages, fills, special nets
  The software compares the physical data (such as shape and layer) to the existing data in the database. If it finds the same data in the database, it ignores the data in the DEF file. If the data does not already exist, the software adds it to the database. For special nets, the software merges wire segments in the DEF file with any existing ones in the database that overlap and have the same attributes, such as layer, width, shape, and direction. The software does not delete any existing data.
- Nondefault rules, vias
  The software adds the objects to the database if they do not already exist there. If a nondefault rules definition with the same name already exists, the software ignores the one in the DEF file. If a via definition with the same name already exists, and the via is a fixed via, the software ignores the one in the DEF file. If a generated via definition with the same name already exists, the geomoetries are read, but the name might be changed. The software does not delete any existing objects of these types.
- Nets
  For each net in the DEF file, the software removes the existing regular routing (not special routing), then adds the routing from the DEF file for this net.
- Pins
  For each pin in the DEF file, the software removes the existing physical information for the pin, then adds the physical information from the DEF file for this pin. If a pin does not already exist, and it is a power or ground pin, the software adds it to the database. If the pin is not a power or ground pin, the software generates an error message.
- Groups, regions
  If a group name in the DEF file matches an existing hierarchical instance name in the database, the region's boundary constraint is attached to the hierarchical instance, overriding any existing boundary constraint. The software checks whether the group members belong to the hierarchical instance, and generates a warning message if any do not belong. If a group name does not match any hierarchical instance name, the software creates the instance group with an attached region boundary constraint, if one exists. Regions that are not used by groups in the same DEF file are ignored. The software does not remove existing groups.

  Note: You can import DEF files containing wildcards (`*`) in the `GROUPS` statement. The `read_def` command does not support multiple groups per region.

- Components
  You cannot add new logical cells. You can add new physical cells, such as:
  - Cells with `CLASS PAD`
  - Cells with `CLASS COVER` and no signal pin
  - Instance with `+ SOURCE DIST`

You can use the `read_def` command after importing a design.

After loading the DEF netlist using the read_def -verilog_from_def_netlist_flow command, you can perform floorplanning, non-timing driven placement and routing, wire Innovusting, and verification. You cannot use the DEF netlist flow for parasitic extraction, delay calculation, and timing-driven placement and routing effectively because the DEF names do not properly match the Verilog names used in timing constraints and timing analysis.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `read_def` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_def`
`def_files`	Specifies the names of the DEF files to load.
`-design`	Specifies to read def files with the same top cell name
`-design_offset (x y)`	Specifies the placement for the top cell.
`-design_orient orientation`	Specifies the orientation for the top cell.
`-force_create_phys_inst`	Specifies to force create physical cell instances.
`-keep_pin_geometry`	Keeps pins shapes equivalent to the block pin shapes, in the form of special route.

`-preserve_shape`	Prevents creation of symbolic center-line connected routing data and preserves the routing data as-is.
`-rdl_def def_file`	Specifies the RDL def file.
`-rdl_offset (x y)`	Specifies the offset of RDL def.
`-rdl_orient orientation`	Specifies the orientation for RDL def.
`-read_flat_dir directoryName`	Reads flat data from the specified directory.
`-skip_pg`	Skips merging routing on pg nets.
`-skip_signal_nets`	Specifies to skip reading signal nets.
`-top_scope`	Specifies to merge routing on top scope nets only.
`-top_scope_ignore_block_internal_nets_on_boundary_path`	Specifies merging routing on top scope nets which are not block internal nets on boundary path.
`-write_flat_dir` *`directoryName`*	Writes flat data to the specified directory.

Examples

- The following command reads the compressed DEF file `dma_mac_filled.def.gz`:

  ```
  tempus > read_def dma_mac_filled.def.gz
  ```

Related Commands

- read_hierarchical_def
- `write_def`

read_instance_voltage

```
read_instance_voltage
[-help]
[-early]
[-ir_drop <<fileName>>]
[-late]
[-list]
[-max]
[-min]
[-reset]
```

Loads the instance-based IR-drop information that is output by power-rail analysis tools, such as Voltus, to account for the effects of IR drop when calculating the delays and slews. You can read in four different files for early-max, early-min, late-max, and late-min IR-drop analysis.

Parameters

`-help`	Prints out the command usage.
`-early | -late`	Applies the effects of IR-drop to the early path (data hold/clock setup) or late path (data setup/clock hold) in your design. Use the `-early` or `-late` parameter in conjunction with the `-min` or `-max` parameter to account for the effects of early-max, early-min, late-max, or late-min IR-drop analysis.

`-ir_drop <<fileName>>`	Specifies the names of the IR-drop files to load. The format of an IR drop file is shown below:
	`VERSION "2.0"`
	`CREATION "Tue Jul 21 02:10:15 2015"`
	`CREATOR "user1"`
	`PROGRAM "Vendor"`
	`DIVIDERCHAR "/"`
	`DESIGN "design1"`
	`UNITS VOLTAGE VOLT 1`
	`INSTANCE_COUNT 1379138`
	`NOMINAL_VOLTAGE 1.1`
	`POWER_NET vdd`
	`GROUND_NET gnd`
	`WINDOW SWITCHING_AND_TIMING`
	`RP_VALUE IV/EIV`
	`RP_FORMAT BRIEF`
	`RP_INST_LIMIT 2000000000`
	`RP_THRESHOLD -1e+06`
	`RP_PIN_NAME FALSE`
	`INSTANCESUPPLY 1379138 vdd 1.1 EFFECTIVE`
	`BEGIN`
	`- ICC_PRECTS_33 0.95948 C12T28SOI_LLTHP_XOR2X33_P16`
	`- ICC_PRECTS_5 0.96383 C12T28SOI_LL_CNNAND2X15_P16`
	`- u_stc2/ICC_RTO_TIMING_6 0.96524 C12T28SOI_LLBR1P2NHP_BFX34_P16`
	`- u_ca7_scu_l1d_tagrams/U207 0.96705 C12T28SOI_LLBR1P8NHP_BFX48_P16`
	`- u_ca7_scu_l1d_tagrams/U244 0.96759 C12T28SOI_LLBR1P2NHP_BFX43_P16`

`-list`	Provides a list of all specified IR-drop files.
`-min | -max`	Applies the effects of IR-drop to the minimum operating condition or maximum operating condition. Use the `-min` or `-max` parameter in conjunction with the `-early` or `-late` parameter to account for the effects of early-max, early-min, late-max, or late-min IR-drop analysis. **Note:** If the `-min` or `-max` parameter is used without the `-early` or `-late` parameter, the IR drop is applied to both early and late analysis. To apply the IR drop for late analysis only, use the `-min` or `-max` parameter with the `-late` parameter. To apply the IR drop for early analysis only, use the `-min` or `-max` parameter with the `-early` parameter.
`-reset`	Resets all IR-drop files.

Examples

- The following commands apply the IR drop for early-min, late-min, early-max, and late-max IR-drop analysis:

```
tempus > read_instance_voltage -early -min -ir_drop { earlymin.ir }
tempus > read_instance_voltage -late -min -ir_drop { latemin.ir }
tempus > read_instance_voltage -early -max -ir_drop { earlymax.ir }
tempus > read_instance_voltage -late -max -ir_drop { latemax.ir }
```

- The following commands apply the IR drop for late-min and late-max IR-drop analysis:

```
tempus > read_instance_voltage -late -min -ir_drop { latemin.ir }
tempus > read_instance_voltage -late -max -ir_drop { latemax.ir }
```

- The following commands apply the IR drop to early and late IR-drop analysis:

```
tempus > read_instance_voltage -early -ir_drop { early.ir }
tempus > read_instance_voltage -late -ir_drop { late.ir }
```

read_libs

```
read_libs
[-help]
[<in_file <file1[file2] ...>>]
[-aocv_libs <string>]
[-lef_files <string>]
[-library_side_file <string>]
[-aocv_libs aocvLibs]
[-max_aocv <string>]
[-max_libs maxLibs]
[-max_noise_libs maxNoiseLibs]
[-min_aocv <string>]
[-min_libs minLibs]
[-min_noise_libs minNoiseLibs]
[-noise_libs noiseLibs]
[-oa_ref_libs oaLibs]
[-power_ground_files powerGridLibFiles]
[-socv_libs soceLibs]
```

Schedules reading of one or more technology library files. These files either contain timing-related or noise-related information for the primitive cells used in the design.

Timing libraries can be in text format or Cadence binary library format (LDB). The LDB (library database) files generated from other tools, like Innovus, Voltus, or Genus are read in to Tempus with an error message (TECHLIB-1249). The LDB files are not forward compatible, that is, a LDB generated from 15.2 version cannot be used with 15.1 or prior versions of the software.

Note: The `read_libs` command determines the priority of ECSM vs CCS data (if both are present). The software first checks for ECSM construct, if it is missing then CCS construct is loaded.

Note: You can use wildcards with the `read_libs` command as follows:

```
read_libs [glob *.lib]
```

Parameters

`-help`	Prints a brief description that includes the type and default information for each `read_libs` parameter.
`in_file <file1[file2] ...>`	Specifies to read the library files.
`-aocv_libs` *`aocvLibs`*	Specifies to read the default AOCV libraries.
`-lef_files <string>`	Specifies the name of the LEF files to be loaded.
`-library_side_file <string>`	Specifies to read the default incremental files.
`-max_aocv` *`string`*	Specifies to read the max AOCV libraries.
`-max_libs` *`maxLibs`*	Specifies to read the max libraries.
`-max_noise_libs` *`maxNoiseLibs`*	Specifies to read the CDBs for max noise analysis.
`-min_aocv` *`string`*	Specifies to read the minimum AOCV libraries.
`-min_libs` *`minLibs`*	Specifies to read the min LIBs.
`-min_noise_libs` *`minNoiseLibs`*	Specifies to read the CDBs for min noise analysis.
`-noise_libs` *`noiseLibs`*	Specifies to read the default CDB files for noise analysis.
`-oa_ref_libs` *`oaLibs`*	Specifies to read the OA reference library.
`-power_ground_files` *`powerGridLibFiles`*	Specifies to read the power grid library files.
`-socv_libs` *`soceLibs`*	Specifies to read the default SOCV libraries.

read_mmmc

```
read_mmmc
[-help]
file_name
```

Reads in the Multi-Mode, Multi-Corner (MMMC) file. MMMC refers to the full set of timing libraries, constraints, and the associated modes.

When the `read_mmmc` command is run, the MMMC file is read and processed. The majority of elements in the MMMC control file is saved in the MMMC caching attributes. Except for the library sets defined by the first `set_analysis_view` command, no data is loaded into the database. This command does basic syntax checking on the MMMC commands, rejecting commands that have incorrect options and commands that are not MMMC or tcl commands. However, no consistency checking is performed. For example, if a constraint mode is used in an analysis view, but is not defined, it will not be flagged.

The MMMC file contents are cached in attributes in the database. These attributes allow you to write out a full and complete MMMC file later in the flow. In addition, the names of the MMMC files are cached for querying.

The exceptions to data set load are the libraries pointed to by the first `set_analysis_view` command in the MMMC file. These library sets are loaded into the database. This load uses the standard library parser, and includes standard library syntax checking.

The `read_mmmc` command corresponds to the *Timing* step of the Initialization flow:



In the *Timing* step, the MMMC objects are set up by populating the basic MMMC information attributes. As Synthesis needs library information before the *Design* step, the *Timing* step also loads some library information. The library data to be loaded is pointed to by the first `set_analysis_view` command in the MMMC file.

See **Example** section for more details.

At a minimum, the MMMC file must contain one each of:

```
create_library_set
create_rc_corner
create_timing_condition
create_delay_corner
create_constraint_mode
create_analysis_view
set_analysis_view
```

If the MMMC file does not contain this minimum set, an error will be issued.

Note: The library sets can be specified with absolute paths in the MMMC, or relative to the local directory or to library search path attributes. The tool first attempts to load based on the name in the MMMC. If the library is not found, the tool sweeps through the search directory list in the specified order. Only the first hit is loaded.

Parameters

`-help`	Prints a brief description that includes type and default information for each `read_mmmc` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_mmmc`
`file_name`	Specifies the name of the MMMC file.

Examples

When reading an MMMC file with `read_mmmc`, the tool sets the MMMC attributes with the data specified in the file. When the first `set_analysis_view` command is processed, the tool loads in the libraries specified by this analysis view. No other data is loaded.

For example, consider an MMMC file with the following commands:

```
create_library_set -name LS1 -timing [list lib1.lib lib2.lib lib3.lib]
create_library_set -name LS2 -timing [list lib4.lib lib5.lib lib6.lib]
create_library_set -name LS3 -timing [list lib7.lib lib8.lib lib9.lib]
create_rc_corner -name QX -qrc_tech file.qrc
create_timing_condition -name TC1 -library_set LS1
create_timing_condition -name TC2 -library_set LS2
```

```
create_timing_condition -name TC3 -library_set LS3
create_delay_corner -name DC1 -timing_condition {TC1 PD2@TC2} -rc_corner QX
create_delay_corner -name DC2 -timing_condition {TC1 PD2@TC3} -rc_corner QX
create_constraint_mode -name CM1 -sdc_files cond1.sdc
create_constraint_mode -name CM2 -sdc_files cond2.sdc
create_analysis_view -name AV1 -constraint_mode CM1 -delay_corner DC1
create_analysis_view -name AV2 -constraint_mode CM2 -delay_corner DC2
set_analysis_view -setup AV1 -hold AV2
```

When the `set_analysis_view` command is processed, the tool traces backwards from that command to determine the library set. In this case, the analysis view specified by `set_analysis_view` is AV1. The AV1 analysis view points to domain corner DC1, which in turn points to two library sets, LS1 and LS2. As a result, lib1.lib and lib2.lib are loaded into the database.

In this case, lib3.lib is not loaded into the database during `read_mmmc` because it is not in the library sets required by the analysis view.

Related Commands

- `write_mmmc`
- `init_design`
- `read_physical`
- `read_netlist`

read_netlist

```
read_netlist
[-help]
{{in_file | -oa_cell_view {lib cell view}} [-top top_module_name]}
```

Reads in a Verilog structural netlist. A structural Verilog file contains only structural Verilog-1995 constructs, such as module and gate instances, concurrent assignment statements, references to nets, bit-selects, part-selects, concatenations, and the unary ~ operator.

`read_netlist` also supports OpenAccess through the `-oa_cell_view` option.

The `read_netlist` command corresponds to the *Design* step of the Initialization flow:



During the *Design* step, the design is loaded into the database with `read_netlist`. After issuing this command, the database objects are populated and can be queried and manipulated (ungrouped, renamed, and so on). Full error checking is performed on the design details. This includes, but is not limited to, unresolved references, port mismatches, and empty modules.

Parameters

`-help`	Prints a brief description that includes the type and default information for each read_netlist parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_netlist`.
`in_file`	Specifies the list of netlist files.
`-oa_cell_view {`*`lib cell view`*`}`	Specifies the name of the design object to be loaded in {*`lib cell view`*} format. No other options can be used if this option is specified.
`-top` *`top_module_name`*	Specified the top module name. By default, `read_netlist` identifies the highest module and links to that. If there are multiple top modules, the command returns an error.

Related Topics

- `init_design`
- `read_physical`

read_physical

```
read_physical
[-help]
{[-oa_ref_libs lib1 [lib2] ... [-oa_search_libs lib1 [lib2] ...][-oa_tech_lib lib1 [lib2]
...]] | -lefs lib1 [lib2] ... /[-add_oa_ref_libs lib1 [lib2]... [-oa_lib_path libPath]]| -
add_lefs lib1 [lib2] ...}
```

Reads the physical library information. When this command is run, the physical library information is loaded into the database. Both LEF and OA are supported, but not simultaneously.

The `read_physical` command corresponds to the *Physical* step of the Initialization flow:



During the *Physical* step, the physical libraries (OA or LEF) are loaded into the tool's database using `read_phsyical`. When the command is issued, the database is loaded with the specified objects, and is available for querying of attributes associated with the physical data, such as size, layer details, and so on.

The tool uses the standard OA parser, and performs full error checking on the physical data. When the command is complete, a full timing library set and a physical library set would have been loaded into the database.

If `-lef` is used, the technology LEF must be first specified.

This command is optional for Synthesis and STA, but is required for Implementation.

The tool can support a physical only mode when the attribute `init_physical_only` is set to `true`. This attribute must be set prior to `read_physical`, otherwise an error will be issued.

Parameter

`-help`	Prints a brief description that includes type and default information for each `read_physical` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_physical`
`-add_lefs` *`lib1`* [*`lib2`*]	Specifies a list of LEF files to read and to add to the original list of LEF files that were added before `init_design`. This option can only be used after `init_design`, and cannot be combined with any other option. You can add new macros, non-default rules, sites etc. using this option, but you cannot use it to modify any existing LEF data that is already loaded. The LEF files added using this option will be used when the saved design is loaded again using `read_db`.
`-add_oa_ref_libs` *`lib1`* [*`lib2`*]	Specifies the list of OpenAccess library names to be loaded incrementally. These library names are added in the `cds.lib` file, if not present. Use this parameter to specify standard cell libraries or I/O driver libraries that should all be loaded after the design is read.

`-lefs` *lib1* [*lib2*] ...	Specifies the list of LEF files to be loaded. The first LEF file must have the technology section present. The file names are saved to the `init_lef_files` root attribute. This option cannot be combined with `-oa_ref_libs`. **Note**: This option can only be used before `init_design` or `read_db`. If you specify this option after `init_design`, the tool will give an error.
`-oa_ref_libs` *lib1* [*lib2*] ...	Specifies the list of OpenAccess library names to be loaded. These names must also be listed in the `cds.lib` file. All the abstract views in the specified libraries are loaded. Use this parameter to specify standard cell libraries or I/O driver libraries that should all be loaded, even if some of the cells are not currently used in the netlist. The first OpenAccess library that is specified is also used to load the OpenAccess technology data. The specified library names are saved to the `init_oa_ref_libs` root attribute.
`-oa_lib_path` *libPath*	Specifies the path of oa reference libraries. This option can only be used with the `-add_oa_ref_libs` option.
`-oa_search_libs` *lib1* [*lib2*] ...	Specifies the list of OpenAccess libraries to be searched. This parameter can be specified only with `-oa_ref_libs`. As the netlist is read, these libraries are searched for cell abstracts to be load on demand for any Verilog module that does not have a lib_cell already defined. This parameter is typically used for libraries of block IP, such as RAMs, where only the abstracts that are actually used in the netlist need to be loaded.
`-oa_tech_lib` *lib1* [*lib2*] ...	Specifies the OA tech file to use if the netlist is coming from Verilog rather than an OA cellview.

Related Topics

- init_design
- read_db

read_physical Category Attributes

The root attribute in the `read_physical` category is:

- read_physical_extend_polygon_cell_shapes
- read_physical_check_antenna
- read_physical_check_colored_shape
- read_physical_merge_multi_port_to_single_port
- read_physical_must_join_all_ports

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `read_physical` category, use the following command:

```
get_db -category read_physical
```

read_physical_extend_polygon_cell_shapes

```
read_physical_extend_polygon_cell_shapes { {layer extension_amount}… }
```

Default: `""`

Data_type: `string`

Specifies a list of layer and a value for creating obs shape for 2D pin/obs shapes. Cells could have 2D objects, which require larger line-end spacing. You can use `read_physical_extend_polygon_cell_shapes` to automatically add SPACING 0 OBS for routing during LEF in.

For any 2D shapes, you can extend edges that are perpendicular to the preferred routing direction of the rectilinear shape by specifying the extension_amount in `read_physical_extend_polygon_cell_shapes`.

read_physical_check_antenna

```
read_physical_check_antenna {true | false}
```

Default: `true`

Data_type: `bool`

If you set `read_physical_check_antenna` to `true`, LEF or OA cells will have process antenna consistency checks done for LEF ANTENNA* data (or OA equivalent data) attached to their pins. Input pins with 0 gate-area, output pins with 0 diff-area, and inout pins with 0 gate-area or 0 diff-area values will generate warnings. If `false`, the tool will not perform consistency checking.

read_physical_check_colored_shape

```
read_physical_check_colored_shape {true | false}
```

Default: `false`

Data_type: `bool`

Specifies to check and ignore colored shapes on colorless layers while reading the LEF or DEF data. When set to `true`, the tool errors out if a colored shape is on a colorless layer. When set to `false`, the tool gives error message and ignores the color.

read_physical_merge_multi_port_to_single_port

```
read_physical_merge_multi_port_to_single_port {true | false}
```

Default: `false`

Data_type: `bool`

When set to `true`, multiple ports in a LEF PIN are merged into a single port. However, the pin should not have the `MUSTJOINALLPORTS` attribute as it specifies that all of the ports must be connected individually.

read_physical_must_join_all_ports

```
read_physical_must_join_all_ports {true | false}
```

Default: `true`

Data_type: `bool`

If set to `false`, the LEF_MUSTJOINALLPORTS property on a LEF PIN auto-creates MUSTJOIN pins for each PORT. If set to `true`, the MUSTJOIN pins are not created, but the router must connect to every port with a signal wire or signal via that physically overlaps all the ports.

Related Command

read_physical

read_rcdb

```
read_rcdb
[-help]
dirName
```

Restores the parasitic data that was previously saved with the `write_rcdb` command.

When you use the `read_rcdb` command, the design data must be the same as when you use the `write_rcdb` command. You can restore the parasitic data on systems that are same as those used for saving the data. For example, if the data was saved on Linux in 32bit mode, you can restore it on Linux 32 bit mode only.

> ⓘ Restoring RCDB that you generated using previous major version(s) of the software is not supported and might result in an error. Use `write_rcdb` to generate a new RCDB.

Parameters

`-help`	Prints out the command usage.
dirName	Specifies the name of the parasitic file that you saved using the `write_rcdb` command. **Note:** The parasitic file must have the .rcdb.d extension.

Examples

- The following command reads an Encounter RC extracted database:
  ```
  tempus > read_rcdb test.rcdb.d
  ```

Related Commands

- write_rcdb

read_sdc

```
read_sdc
[-help]
[-reset]
sdc_files
```

Loads a timing constraints file. By default, the constraints files are loaded incrementally.

The constraints can be in the design constraints (SDC) format, or they can be more complicated Tcl constraints embedded in a Tcl script using commands such as `get_property` and `filter_collection`.

A list of constraints files can also be specified in the configuration file.

Parameter

`-help`	Prints out the command usage.
`-reset`	Replaces all the previously loaded constraints information with the details specified in the specified SDC file.
`sdc_files`	Specifies the timing constraints file.

Examples

- The following command reads a SDC file called dma_mac_m.sdc:

  ```
  tempus > read_sdc dma_mac_m.sdc
  ```

Command Order

Use this command after importing the design and loading the timing library.

Related Commands

- `write_sdc`
- `read_sdc`

relink_db_files

```
relink_db_files
[-help]
-db_dir db_directory
-new_lib_dirs {new_lib_directory1 new_lib_directory2 ...}
```

Relinks symbolic links to library files inside a DB directory.

By default, `write_db` saves the location of library files as symbolic links under `<db_dir>/libs/…`. When a DB directory is copied to a new network where all the libraries are in a different location, or a new version of the library files are needed, you can use `relink_db_files` to change the symbolic links to new locations .

The `relink_db_files` command will search each `-new_lib_dirs` directory and the sub-directory tree below it for each library file name inside `<db_dir>`, and create a symbolic link to the new location. Only the base file name is used for the search (except for `qrcTechFile`). Therefore, two path names inside the new directory, such as `/timing/fast/std.lib` and `/timing/slow/std.lib`, will have duplicate names of "`std.lib`". The duplicate names are not supported and will cause an error. You can use the `read_db` command to update all the LEF full-path names or use a new MMMC files with all the new .lib full-path names if duplicate file names need to be updated. If there are missing library file names, the tool will show an error.

The `relink_db_files` command supports `.lib`, `.lef`, and `qrcTechFile` library file types.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `relink_db_files` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man relink_db_files`
`-db_dir` *db_directory*	The current db directory which contains incorrect library symbolic links.
`-new_lib_dirs` *{new_lib_directory1 new_lib_directory2 ...}*	The new directories to search for library files.

Example

If the current LEF files are all under`/libs/lefs_v1.0/` (or sub-directories) and the current `.lib` and `qrcTechFiles` are all under`/libs/timing_v1.0/` (or sub-directories), you use the following command to update the LEF files to use `/libs/lefs_v2.0/`:

```
>relink_db_files -db_dir my_db -new_lib_dirs {/libs/lefs_v2.0 /libs/timing_v1.0}
```

Note: The command must find every library file. Therefore, you need to list the top of the old timing directory tree along with the top of the new LEF files directory tree.

Related Information

- `write_db`
- `read_db`

set_cell_power_domain

```
set_cell_power_domain
[-help]
[-reset]
[-file file_name]
```

Defines domains for multi-VDD and multi-VSS cells. When reading an instance ASCII power file to perform power-rail analysis, if there are missing power values associated with ground pins, the software uses the information specified with the `set_cell_power_domain` command to derive domain association and distributes power to each ground net accordingly.

This domain information is stored in a side file that is used by power analysis to calculate power using the correct voltage for each domain. If you are specifying an external instance ASCII power file with pin information, this domain information is used to correctly distribute power to each pin.

Use this command before performing power and rail analysis.

Parameters

`-file` *file_name*	

	Specifies the name of the file containing the cell names and corresponding power-ground pairs.
	The format of the file is:
	`CELL: <cell name1>`
	`<power1> <gnd1>`
	`<power2> <gnd2>`
	`CELL: <cell name2>`
	`<power1> <gnd1>`
	`...`

`-help`	Outputs a brief description for the `set_cell_power_domain` parameter. For a detailed description of the command, use the `man` command: `man set_cell_power_domain`.

`-reset`	Resets all specified options back to default values.

Examples

- The following command define two domains, `VDD:VSS` at `1v` and `VDDm:VSSm` at `0.84v`, of the cell `mycell` in the file `domain_file`:

  ```
  set_cell_power_domain -file domain_file
  ```

set_library_unit

```
set_library_unit
[-help]
[-cap capUnit]
[-time timeUnit]
```

Sets the time and capacitance unit values for constraints and report generation.

The timing constraints (SDC) files do not contain any unit specification for capacitance and time values. Hence, the units for these values are considered to be same as those specified in the timing libraries. The commands, e.g., `report_timing`, require consistent units to generate reports. If an inconsistency occurs in the timing libraries, the default unit of time (as ns) and capacitance (as pf) is used.

You can use this command when you need to override the library units, or force the setting of units when libraries have different units.

Note: You must set the `set_library_unit` command before the design and constraints are read.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `set_library_unit` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_library_unit`.
`-cap` *capUnit*	Specifies the capacitance load unit in the timing library. *Default:* `1.0` pf
`-time` *timeUnit*	Specifies the time unit to be used for timing constraints. The valid values are 1ns, 1ps, 10ps, 100ps. *Default:* `1.0` ns

Example

- The following command sets the time unit for the design to `10`ps:

```
set_library_unit -time 10ps
```

set_power_def_files

```
set_power_def_files fileNameList
[-die_instance dieinstname]
```

Specify the name of the DEF file(s). This command can be used in lieu of `read_def` command for analysis purpose. When you use the `set_power_def_files` command, the software passes specified DEF files to the power and rail analysis programs for internal processing without loading their physical data in memory. As a result, you will not be able to display and query layout of the design. These programs will only process the necessary data from DEF file(s). The usage of this command is recommended in dynamic power analysis, and static and dynamic rail analysis for large (>5M components) designs with hierarchical DEFs or in general to obtain better performance during analysis. You will be able to save memory and runtime that takes to load design during analysis. Once the analysis is complete, you can subsequently load DEFs in the same or new session using the `read_def` command to view design layout and analysis results.

Note that for MSMV design, in order to use `set_power_def_files`, static power analysis mode will require CPF (Common Power Fomat) that defines logical connectivity of the instances to power nets. If CPF is not available, you can optionally run dynamic power analysis which calculates dynamic as well as static power.

`-die_instance` *dieinstname*	
	Specifies the die instance name for multi-die analysis. This option is only available in multi-die analysis.
fileNameList	List of DEF files.

set_power_lib_files

```
set_power_lib_files
[-help]
[<in_file <file1[file2] ...>>] [-reset]
```

Specify the name of the timing library file(s) for dynamic power analysis. This command allows you to run the dynamic power analysis flow without reading a dotlib into design database to reduce background memory consumption. You must have an external TWF file to run this flow as the timing engine will not be invoked from within the software. This flow is recommended for large designs where front-end processing of timing libraries is high. Some of the database query commands will not work if `set_power_lib_files` is used.

This command works in conjunction with the `specify_def` and `specify_spef` commands.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_power_lib_files` parameter.
in_file <file1[file2] ...	
	List of timing library files.
`-reset`	Resets all specified options back to default values.

Examples

- The following Tcl shows the use model of the **set_power_lib_files** command that works in conjunction with the specify_def/specify_spef flow to perform dynamic power analysis:

```
read_lib -lef tech1.lef tech2.lef
set_power_lib_files /../library1.lib /../library2.lib
specify_def adder_routed.def
specify_spef {test.spef }
read_twf test.twf2
read_verilog test2.v
set_top_module test
set_default_switching_activity -comb_clockgate_ratio 1 -period 2.01ns -input_activity
0.2 -icg_ratio 1
report_power
```

write_db

```
write_db
[-help]
[-keep_partial]
[-no_timing_graph]
[-overwrite]
{{out_dir [[-def ] [-common ]]} | {-oa_lib_cell_view {libname cellname viewname} }}
[-no_rc]
[-lib]
[-user_path]]
```

Saves the files for design import, floorplan, placement, routing, and power switch state information in the specified directory. You can save a design to the same location from which you restored the design. The new data, including the netlist and constraints, overwrite the previous data.

You can use the `write_db` command at any time after importing a design.

> Modifying the contents of the *filename.dat* directory created by `write_db` can cause major unexpected results. Never modify the directory contents.

To save a design in OpenAccess database format, use `write_db -oa_lib_cell_view {libname cellname viewname}`.

The `write_db` command saves a tool design in the portable mode by default. In this mode, the design and library data as well as Tempus control files do not have any path dependency. As a result, the saved database can be restored from any directory within the same network. The saved database can also be moved to any directory within the same network without any change. You can set the `write_db_portable` variable to `false` if you do not wish to save the design in portable mode.

The portable `write_db` feature is currently not supported for OpenAccess databases.

Note: The `write_db` command will issue an error message if you try to save a mulitple-suppy voltage (MSV) design in native format. Instead, create a Commmon Power Format (CPF) file containing CPF-based MSV commands and load the file, or use the following commands in this specific order to save your native MSV design in a CPF database:

1. `saveCPF`
2. `loadCPF`
3. `commitCPF`
4. `write_db`

For backward compatibility, if you must save the MSV design in a native database, set the following variable before saving the design:

```
setVar dbgSaveOldMsvDBOnly
```

Cadence does not recommend that you do this: use the `saveCPF`, `commitCPF`, `loadCPF`, `write_db` flow instead.

Note: The write_db command does not save the state of the timing_enable_simultaneous_setup_hold_mode timing attribute. When the attribute is set to true, various physical design functions in the system are disabled. When the design is restored (using the read_db command), the physical design commands are required to be active so that the system can be properly initialized.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_db` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_db`.
`-common`	Saves the common-db design data to the specified directory.
`-def`	Saves a DEF file, in addition to saving place and route files when you save the design.
`-keep_partial`	Keeps the partially saved database for debugging purposes only, because restoring this database is not recommended. If the `write_db` command does not complete successfully, the session data will be removed by default.

`-oa_lib_cell_view {`*`libname cellname viewname`*`}`	Saves the design in OpenAccess database format using the specified library, cell and view names. The design is saved in the *`libname/cellname/viewname`* directory. **Note**: Usually, the cell name is the same as the design top cell name. If the cell name specified is different from the top cell name, it means that the top module created in OpenAccess would have the new cell name instead of the original name.
`out_dir`	Represents the user-defined name of the current Tempus session. The recommended file extension is `.enc`. The associated data directory is *dir_name*`.enc.dat`.
`-lib`	Resolves symlinks of timing library files and side files. When you specify `-lib` with write_db while saving a design in OpenAccess format, the links under *`libname`*`/libs` directory are replaced with real files. When you specify `-lib` with write_db while saving a design in Tempus format, the links under *`filename.enc.dat`*`/libs` directory are replaced with real files.
`-no_rc`	Does not write RC extraction data.
`-no_timing_graph`	Does not save timing graph information.
`-overwrite`	Overwrites the information.
`-user_path`	By default all external file references (such as. library files) are stored as absolute paths without any symbolic-links. So, the DB directory is portable anywhere on the same network. This option stores external file references exactly as you entered it For example, full paths like /x/y or relative paths like ../x/y, ./x/y), and keeps any symlinks used inside the path names. *Data_type*: bool, optional **Example**:

Consider that there are two library files a.lib and b.lib under:

```
/global/project1/libs/mmmc
```

And two lef files c.lef and d.lef under:

```
/global/project1/libs/lef
```

There are two symlinks to the libs:

```
/usa/db1/libs1.0/mmmc/a.lib
→ /global/project1/libs/mmmc/a.lib

/usa/libs/mmmc/b.lib           → /global/proj
ect1/libs/mmmc/b.lib

And two symlinks to the lefs:

/usa/db1/libs1.0/lef/c.lef
    → /global/project1/libs/lef/c.lef

/usa/libs/lef/d.lef            → /global/proj
ect1/libs/lef/d.lef
```

Your current work dir is /usa/db1.1/. You have an init script like:

```
read_mmmc ./mmmc.tcl

read_physical -lefs
{/usa/db1/libs1.0/lef/c.lef
../libs/lef/d.lef}

read_netlist design.v

init_design
```

Where mmmc.tcl has:

```
create_library_set -name test -timing
{/usa/db1/libs1.0/mmmc/a.lib
../libs/mmmc/b.lib }
```

Then after `write_db`, without `-user_path`, will save the libs and lefs to link to the real files.

```
> write_db db1.2
```

The ./mmmc.tcl file is always stored inside the DB dir, so no external reference is used.

By default, external file references are resolved to their fullpath name without any symlinks.

The .lib files inside the mmmc.tcl file will be referenced as: `{/global/project1/libs/mmmc/a.lib /global/project1/libs/mmmc/b.lib}`

The .lef files will be referenced as: `{/global/project1/libs/lef/c.lef /global/project1/libs/lef/d.lef}`

This is portable and can be loaded by read_db anywhere on the same network.

```
/usa/db1.2/libs1.2/mmmc/a.lib    →
/global/project1/libs/mmmc/a.lib

/usa/db1.2/libs1.2/mmmc/b.lib    →
/global/project1/libs/mmmc/b.lib

/usa/db1.2/libs1.2/lef/c.lef    →
/global/project1/libs/lef/c.lef

/usa/db1.2/libs1.2/lef/d.lef    → /global/pro
ject1/libs/lef/d.lef
```

With `-user_path` will save the libs and lefs to keep the symlinks of the current db.

```
> write_db db1.2 -user_path
```

The .lib files will be referenced as entered by the user: `{/usa/db1/libs1.0/mmmc/a.lib ../libs/mmmc/b.lib}`

The .lef files will referenced as entered by the user: `{/usa/db1/libs1.0/lef/c.lef ../libs/lef/d.lef}`

So `read_db` must be run in the same directory where `write_db` was run, or in a directory with the

	same ./y and /x directory structure.
	`/usa/db1.2/libs1.2/mmmc/a.lib → /usa/db1/libs1.0/mmmc/a.lib`
	`/usa/db1.2/libs1.2/mmmc/b.lib → ../../../libs/mmmc/b.lib`
	`/usa/db1.2/libs1.2/lef/c.lef → /usa/db1/libs1.0/lef/c.lef`
	`/usa/db1.2/libs1.2/lef/d.lef → ../../../libs/lef/d.lef`

Example

- The following command copies the current design import configuration, floorplan, placement, and route files into directory `test.enc`:
  ```
  write_db test.enc
  ```

Related Topics

- `read_db`
- `write_db Category Attributes`

write_db Category Attributes

The root attributes in the `write_db` category are:

- write_db_create_read_file
- write_db_include_metal_fill_rules

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `write_db` category, use the following command:

```
get_db -category write_db
```

write_db_create_read_file

```
write_db_create_read_file {true | false}
```

Default: `false`

Data_type: `bool`

When set to `true`, '`write_db my_db`' will also create a '`my_db.read`' file with a '`read_db my_db`' command inside it. This is a Tcl file that can be passed to the Linux executable to make it easy to load the DB at startup like this:

```
'tempus -files my_db.read'.
```

For OA usage, it would have the appropriate '`read_db -oa_lib_cell_view {<lib><cell> <view>}`' command inside it.

write_db_include_metal_fill_rules

```
write_db_include_metal_fill_rules {0 | 1}
```

Default: `0`

Data_type: `int`

Specifies whether metal fill settings are to be saved in the .init file when saving the design. Metal fill

settings include `set_metal_fill`, `set_metal_fill_spacing_table`, and `set_via_fill` settings. By default, these metal fill settings are lost when you exit Tempus. To enable Tempus to record metal fill settings in the .init file, you must set the `write_db_include_metal_fill_rules` to `1` before saving the design. You can easily reload your settings by restoring the saved design:

```
set write_db_include_metal_fill_rules 1

set_metal_fill -iterationName iter1 ...

write_db

read_db

#The previous metal fill settings are reloaded.
```

Related Commands

- `write_db`
- `read_db`

write_def

```
write_def
[-help]
def_file
[-add_half_wire_extension_on_pin]
[-all_layers]
[-bump_as_pin]
[-cut_row]
[-flatten_via_pillar]
[-floorplan]
[-io_row]
[-netlist]
[-no_core_cells]
[-no_special_net]
[-no_std_cells]
[-no_tracks]
[-output_mask_layers layerNames]
[-scan_chain]
[-selected]
[-skip_trim_metal_layers]
[-trim_metal_gap_fill]
[-unit unit_per_micron]
[-unplaced]
[-used_via]
[-verilog_from_def_netlist_flow]
[-with_shield]
[-routing  [-routing_to_special_net  | -wrong_way_routing_to_special_net ]]
[-early_global_route]
```

Writes the specified information to a DEF file. By default, the `write_def` command writes floorplan, and all placed and unplaced standard cell information to the DEF file.

The `write_def` command does not support multiple groups per region. If you run Early Global Route before you run the `write_def` command, the software does not write information about `FIXED` status wires to the DEF file. The `write_def` command supports rectilinear die area.

By default, the `write_def` command does not write out any `NONDEFAULTRULE` or `VIA` defined in the LEF file. To write out a LEF via, set the `write_def_include_lef_vias` attribute to `true`. To write out a nondefault rule, set the `write_def_include_lef_ndr` attribute to `true`.

You can use the `write_def` command after importing a design.

Note: You can select the DEF version for your output file by setting the `write_def_lef_out_version`

attribute to the version number. For example, to set the version number to 5.6, type the following command:

```
set_db write_def_lef_out_version 5.6
```

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_def` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_def`.
`-add_half_wire_extension_on_pin`	Typically, signal routing in the `NETS` section uses the default half-width wire extensions everywhere except for zero extensions on pins. This parameter forces the zero-extension on pins to the default half-width extension, and makes the DEF compatible for older tools that do not support DEF zero wire extensions. **Note:** Using this parameter can cause DRC violations.
`-all_layers`	Writes out any wires and shapes on non-routing layers to SPECIALNETS section.
`-bump_as_pin`	Writes bump cells as top-level pins in the `PINS` section. If a PORT in a block has a PASSIVATION layer, the tool treats it as an embedded bump. `write_def -bump_as_pin` outputs all the layer information for embedded bumps in the PIN section as for normal bumps. Use the `-bump_as_pin` parameter to generate a DEF file for use with Quantus gate-level extraction.
`-cut_row`	Cuts rows against placement obstructions, block halos, or power domain fence minimum gaps. *Default:* Placement blockages are written to the DEF file.

def_file	Specifies the DEF output file. If you add `.gz` as a suffix, the `write_def` command will automatically compress the generated file.
`-flatten_via_pillar`	Some advanced nodes that use SADP (self-aligned double patterning) use via-pillars. A via-pillar is a single via definition with two or more disjoint metal-shapes on the bottom (or top) routing layer of the via. This option will flatten the via pillar into several separate "normal" vias (e.g. a via with only a single metal shape on the top and bottom via layers). The multiple normal vias will be geometrically equivalent to the single via pillar. This can be useful for external tools that do not support a via pillar.
`-floorplan`	Writes the floorplan data to the DEF file. This information includes rows, chip size, `CLASS PAD` instances, `CLASS BLOCK` instances, and all placed standard cells.
`-io_row`	Generates I/O rows based on I/O pad placement, and writes the I/O row information to the DEF file.
`-netlist`	Writes the netlist (that is, the routing connectivity information) to the DEF file. This option is typically used after the design has been placed. **Note:** Use `-netlist` with `-unplaced` if the design has not been placed.
`-no_core_cells`	Excludes core cell information from the DEF file to reduce the data size for import into APD.
`-no_special_net`	Excludes special vias from the DEF file.
`-no_std_cells`	Excludes placed and unplaced standard cell information from the DEF file. **Note:** If you specify this parameter with the `-floorplan` or `-unplaced` parameters, it overrides their setting.

`-no_tracks`	Excludes output track and gcell statements to the DEF file.
`-output_mask_layers` *layerNames*	Outputs MASK data on specified layers only.
`-routing`	Writes the routing information to the `NETS` section of the DEF file. If the `-routing` option is not specified, both power and signal vias are not written to the DEF file. **Note:** The `-routing` parameter implies the `-netlist` parameter; therefore, if you specify this parameter, you do not need to specify `-netlist`.
`-routing_to_special_net`	Writes all regular NETS routing to the SPECIALNETS section.
`-scan_chain`	Writes scan chain information to the DEF file.

`-selected`	Writes out information for selected as follows: • If an instance or net is selected, writes only the DEF header and COMPONENT section for instances you have selected. If you specify the `-routing` parameter along with this parameter, the `write_def` command writes selected nets with their special routing and regular routing to the NETS section of the DEF file. • If a net is not selected but individual segments or vias are selected, writes only the selected vias to the DEF file. All LEF vias are excluded by default. If the `write_def_include_lef_vias` attribute is set to `true`, `write_def -selected` includes selected LEF vias when writing out selected vias. If the `-no_special_net` option is specified, special vias on the selected net will not be written out. • If a bump is selected, writes bump pin connection to the `SPECIALNETS` section of the DEF file. If a net is selected but bump pins are not individually selected, bump pin connection is not written. • If a blockage is selected in the main window, writes blockage information to the DEF file.
`-skip_trim_metal_layers`	Specifies to not write trimmetal shapes in the FILLS section.
`-early_global_route`	Writes information about the wires generated by Early Global Router to the `NETS` section of the DEF file. This option implies the `-routing` parameter; therefore, if you specify this parameter, you do not need to specify `-routing`.

`-trim_metal_gap_fill`	Writes trimmetal gap fills to a dummy net _TRIMMETAL_FILLS_RESERVERD in SPECIALNETS section.
`-unit` *`unit_per_micron`*	Defines the units for the DEF file.
`-unplaced`	Writes information about unplaced standard cells to the DEF file.

`-used_via`	Specifies that only used vias should be written to the DEF file. Unused vias are ignored. All LEF vias are also excluded by default. To include used LEF vias in the DEF file, set the `write_def_include_lef_vias` attribute is set to `true` before specifying `write_def -used_via`. • If the `-routing` option is specified along with `-used_via`, used signal vias are also written out. • If the `-no_special_net` option is specified along with `-used_via`, used special vias are not written out. • If you specify both `-used_via` and `-selected`, the `-used_via` option is ignored and only selected vias are written to the DEF file. The following warning message is issued: `**WARN: (ENCDF-1039): Option -used_via will be ignored because option -selected is specified. Only selected vias will be output.` • If neither `-selected` nor `-used_via` is specified and the `write_def_include_lef_vias` attribute is set to `false` (default), all DEF vias are written out. • If neither `-selected` nor `-used_via` is specified and the `write_def_include_lef_vias` attribute is set to `true`, all DEF and LEF vias are written out.
`-verilog_from_def_netlist_flow`	

	Reconciles the names in the DEF and Verilog files generated by the `loadDefFile` command. The `loadDefFile` command generates a flat Verilog file in which all of the special characters are escaped. Use this parameter to convert the escaped names back to the original names in the DEF file.
`-with_shield`	By default, `write_def` does not write out the shielding routing on the power/ground net when outputting regular routing of selected nets in NETS section. When `-with_shield` is specified, `write_def` writes out those shielding routing (if any) on the power/ground net which is in the SPECIALNETS section. The`-with_shield` option must be used with the `-routing` and `-selected` options. **Note:** Multiple nets can be selected. If none of the selected nets have shielding routing, the `-with_shield` option has no effect.
`-wrong_way_routing_to_special_net`	Writes wrong way routing in the SPECIALNETS section.

Example

- The following command writes floorplanning data, and information on placed and unplaced standard cells to `TOPCHIP_SP.def`:

  ```
  write_def -floorplan -unplaced TOPCHIP_SP.def
  ```

- The following command also writes floorplanning data, and information on placed and unplaced standard cells:

  ```
  write_def test.def
  ```

- The following command writes floorplan and standard cell information to the DEF file. It does not write out unplaced standard cell information.

  ```
  write_def -floorplan
  ```

- The following command excludes all information on placed and unplaced standard cells from the DEF file:

  ```
  write_def -floorplan -no_std_cells
  ```

The resulting DEF file contains floorplan data only.

- The following command writes the shield wires that are shielding the selected nets to the DEF file:

```
write_def -selected -routing -with_shield test.def
```

- The following command writes bump cells as top-level pins in the PINS section of the `test.def` file:

```
write_def -bump_as_pin test.def
```

The PINS section in the test.def file is as follows:

```
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
PINS 102 ;
- iot_schm_2m16m_3v + NET iot_schm_2m16m_3v + DIRECTION OUTPUT + USE SIGNAL
+ SPECIAL + POLYGON ap ( 93300 65970 ) ( 65970 65970 )
( 65970 93300 ) ( 27330 93300 ) ( 27330 65970 ) ( 0 65970 )
( 0 27330 ) ( 27330 27330 ) ( 27330 0 ) ( 65970 0 )
( 65970 27330 ) ( 93300 27330 ) + FIXED ( 386690 2658905 ) N ;
- tck + NET tck + DIRECTION INPUT + USE SIGNAL
+ SPECIAL + POLYGON ap ( 93300 65970 ) ( 65970 65970 )
( 65970 93300 ) ( 27330 93300 ) ( 27330 65970 ) ( 0 65970 )
( 0 27330 ) ( 27330 27330 ) ( 27330 0 ) ( 65970 0 )
( 65970 27330 ) ( 93300 27330 ) + FIXED ( 53350 2658905 ) N ;
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
```

Related Commands

- write_def Category Attributes

write_def Category Attributes

The root attribute in the write_def category is:

- write_def_lef_out_version

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `write_def` category, use the following command:

```
get_db -category write_def
```

write_def_lef_out_version

`write_def_lef_out_version` *version_num*

Default: `5.8`

Data_type: `enum {5.5 5.6 5.7 5.8 6.0}`

Specifies LEF/DEF output version. Possible string values are `5.3`, `5.4`, `5.5`, `5.6`, `5.7`, and `5.8`. The default is `5.8`.

write_mmmc

```
write_mmmc
[-help]
[-library]
```

Writes out the MMMC file.

Parameters

`-help`	Prints out the command usage.
`-library`	Writes out only the library part of the MMMC file.

Related Topic

- read_mmmc

write_netlist Category Attributes

The root attributes in the write_netlist category are:

- write_netlist_full_pin_out
- write_netlist_port_association_style

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `write_netlist` category, use the following command:

```
get_db -category write_netlist
```

write_netlist_full_pin_out

```
write_netlist_full_pin_out {true | false}
```

Default: `false`

Data_type: `bool`

Specifies whether to write out every instance port connection, including unconnected ports unused in the input netlist.

write_netlist_port_association_style

```
write_netlist_port_association_style {true | false}
```

Default: `false`

Data_type: `bool`

Writes out the netlist with implicit port mapping (that is, ordered port connections or position-based port connections) for hierarchical instances.

write_via_defs

```
write_via_defs
[-help]
[-generate]
[out_file filename]
[-all_vias | -signal | -all_rules | -default | -ndr ]
```

Helps in debugging and testing. All the boolean options are default to false. If none is set, the vias are dumped out in LEF format.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_via_defs` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_via_defs`.
`-all_rules`	Dumps out vias from all routing rules. this option is of type boolean.
`-all_vias`	Dumps out all vias. This option is of type boolean.
`-generate`	Dumps out auto-generated vias. This option is of type boolean.
`-default`	Dumps out vias of default routing rule. This option is of type boolean.
`out_file` *filename*	Specifies the output file name. This option is of type string. *Default*: `dump_via.out`
`-ndr`	Dumps out vias of nondefault routing rules. This option is of type boolean.
`-signal`	Dumps out signal vias. This option is of type boolean.

Example

The following command dumps out all signal vias to the default output file, `dump_via.out`:

```
write_via_defs -signal
```

Here's the sample output:

```
#### Begin signal vias ####
VIA V01  # LEF
  LAYER POLY ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M1L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C01L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V01
VIA V01N  # LEF
  LAYER NDIFF ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M1L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C01L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V01N
VIA V01P  # LEF
  LAYER PDIFF ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M1L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C01L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V01P
VIA V1L2L  # LEF
  LAYER M1L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M2L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C1L2L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V1L2L
VIA V2L3L  # LEF
  LAYER M2L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M3L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C2L3L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END V2L3L
VIA V3L4L  # LEF
  LAYER M3L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER M4L ; RECT -0.033 -0.033 0.033 0.033 ;
  LAYER C3L4L ;
    RECT -0.033 -0.033 0.033 0.033 ;
.......................
.......................
.......................
END SP4V4L5LHVR1S5
VIA SP4V4L5LHVR3S5  # LEF
  LAYER M4L ; RECT -0.066 -0.033 0.264 0.033 ;
  LAYER M5L ; RECT -0.033 -0.066 0.033 0.066 ;
  LAYER C4L5L ;
    RECT -0.033 -0.033 0.033 0.033 ;
END SP4V4L5LHVR3S5
#### End signal vias ####
```

check_design Category Attributes

The root attributes in the `check_design` category are:

- check_design_across_hierarchy
- check_design_report_async_pins
- check_design_report_scan_pins
- check_design_threshold_fanout

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `read_physical` category, use the following command:

```
get_db -category check_design
```

check_design_across_hierarchy

```
check_design_across_hierarchy {true | false}
```

Default: `true`

Data_type: `boolean, read/write`

Checks for undriven/multi-driven/unloaded/constant pins/ports by checking across hierarchical boundaries. By default checks are across hierarchy.

check_design_report_async_pins

```
check_design_report_async_pins {true | false}
```

Default: `true`

Data_type: `boolean, read/write`

Checks async pins in a netlist design.

check_design_report_scan_pins

`check_design_report_scan_pins {true | false}`

Default: `true`

Data_type: `boolean, read/write`

Checks scan pins in a netlist design.

check_design_threshold_fanout

`check_design_threshold_fanout` *`threshhold`*

Default: `10000`

Data_type: `int, read/write`

Checks for upper threshold in a netlist design.

reset_sdc

`reset_sdc`

Resets the timing constraints information. Alternatively, you can use the `-reset` parameter of the `read_sdc` command.

Parameters

None

Examples

- The following command resets the timing constraint information:
  ```
  tempus> reset_sdc
  ```

Related Commands

- read_sdc
- write_sdc

switch_top_cell

```
switch_top_cell
[-help]
[cellName]
```

Switches the top cell to specify die name, and switches current tech lib to the relative die tech lib

Parameters

`-help`	Outputs a brief description that includes type and default information for each `switch_top_cell` parameter.
`cellName`	Specifies the top cell name.

13

Interactive ECO Commands and Attributes

- connect
- connect_hpin
- connect_pin
- create_hport
- create_module
- create_port
- delete_inst
- disconnect
- disconnect_hpin
- disconnect_pin
- gui_show_buffer_tree
- update_inst

connect

```
connect
[-help]
port|pin|hpin|hport|pg_pin+
[-net net_object | -net_name net_name | -constant 0|1 | -hnet hnet_object ]
```

Connects objects together. If these object exist in different levels of hierarchies, hpins/hport will be created to make these connections.

If more than one object has a driver (i.e. has a net connected), it creates a multi driver situation. If only one object has a net, all other objects will be connected to that net. If no object has a net, a net will be created and they all will be connected to that net and the `-net_name` option provides a mechanism to optionally name the new net. Otherwise, a new net will be created using n_<i> convention. If the `-net` option is used (along with a net object), then all objects will be disconnected

from their existing connection and connected to the new net. For local connections, it returns then hnet object pointer used to make the connection. For hierarchical connection, it returns the parent net object pointer.

> The `connect` command is a limited-access feature in this release. It is enabled by a variable specified using the `set_limited_access_feature` command. To use this feature, contact your Cadence representative to explain your usage requirements, and make sure this feature meets your needs before deploying it widely.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `connect` parameter. For a detailed description of the command and all of its parameters, use the man command: `man connect`
port|*pin*|*hpin*|*hport*|*pg_pin*+	List of objects to connect. Multiple pin/port objects are supported. Note that only object pointers like hport:h1/hport1 are allowed, and simple names like h1/hport1 are not allowed. All objects in the list will be connected together so if objects have existing drivers, then a multidriver situation can be created. *Data_type:* port|pin|hpin|hport|pg_pin)+, required

`-constant` *`0|1`*	Connect the object to a constant (0 or 1). This option will only work for hpin/pin/hport/port objects. When specified, any existing net connections to the *objects* specified will be disconnected prior to connecting to the constant. It does not connect the objects together, but instead adds a constant to each separate object at the level of hierarchy where the object exists. If an hpin or a pin is connected to an hnet (e.g. is not dangling), the constant is added to the hnet, otherwise it is added directly to the hpin/pin. If a constant is added to a port/hport, it is always added to the hnet of the port/hport. If there are existing drivers or constants attached to any net of the hnets mentioned above, an error message is given, but the connection is done anyway. *Data_type*: bool, optional
`-hnet` *`hnet_object`*	Specifies the hnet object to be used for the connections. It is mutually exclusive with `-net_name`, `-net`, and `-constant`. Additionally, the `-hnet` option can only be used to make a local connection (i.e. at the same level of hierarchy).
`-net` *`net_object`*	Net object to be used for the connections. The object will be connected to the hnet of the specified net at the same level of hierarchy as the object. If there is no hnet at the same level of hierarchy, hports/hpins will be created using the base_name of the net object. Also, when `-net` is specified, any existing net connections to the *objects* specified will be disconnected prior to connecting to this net (for example, equivalent to running disconnect on each object first). *Data_type:* (net), optional

`-net_name` *net_name*	Net name to create when only pins specified. Net name to create when only pins specified (i.e. when -net is not specified). This is the base_name of the net and will replace the name of all nets connected to the pins in the object list. If a hierachical connection is being made the base_name will also be used to name any hports/hpins that are created. If *net_name* has any illegal chars, it will be automatically escaped. Thus -name "i1/n2" will result in "i1\/n2" for the .escaped_name, since it uses DEF escaping semantics. This is equivalent to checking if *net_name* is a legal Verilog identifier, and if not, prepend \ and add a trailing space to the internal Verilog name. *Data_type:* string, optional

Examples

The following command connects two pins with the given net name:

```
connect {pin:H1/I1/Y pin:H1/I3/A} -net_name H1/N1
```

Related Information

- disconnect

connect_hpin

```
connect_hpin
[-help]
-hinst moduleName
-net netName
[-no_new_hpins]
-pin_name hpinName
```

Attaches an hpin in the specified hinst (or top level design) to a net.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `connect_hpin` parameter. For a detailed description of the command and all of its parameters, use the man command: `man connect_hpin`
`-hinst` *hinstName*	Specifies the hinst to that has the hpin. To specify the top design, enter `` `- ``.
`-net` *netName*	Specifies the name of the net.
`-no_new_hpins`	Prevents the creation of new hpins. If the hpin cannot connect to the net in a different hierarchy through existing hpins, the software displays an error message and the command stops running. Default: If you do not specify this parameter, the software creates new hpins as required to reach the net.
`-pin_name` *pinName*	Specifies the base_name of the hpin on the hinst.

Examples

- The `connect_hpin` command connects the hpin with base_name `p1` on hinst named h1/h2 to the net `h1/h2/net3`:

  ```
  connect_hpin -hinst h1/h2 -pin_name p1 -net h1/h2/net3
  ```

Related Topic

- disconnect_hpin

connect_pin

```
connect_pin
[-hport portName]
-inst instName
-net netName
[-no_new_port]
-pin termName
[-reference_pin refInstName refTermName]
```

Attaches a terminal to a net. If the terminal already connects to a different net, the software first detaches the terminal from the current net, then attaches it to the new net.

Note: Detaching a terminal that drives an output terminal of a module produces a Verilog violation at the output terminal if you use `disconnect_pin`. Instead, use `connect_pin` to attach the terminal to a new net. The `connect_pin` command automatically detaches the terminal from the net connecting to the output terminal, then attaches the terminal to the net you specify. This command also enables you to specify a hierarchical net name, in addition to a flat net name.

Parameters

`-inst` *instName*	Specifies the instance containing the terminal. If the instance name does not exist, the software displays an error message and the command stops.
`-net` *netName*	Specifies the name of the net to attach to the terminal. If the net name does not exist, Tempus displays an error message and the command stops.
`-no_new_port`	Prohibits Tempus from creating hierarchical ports when it attaches a terminal. If the terminal cannot connect to the net through existing ports, Tempus displays an error message and the command stops. *Default:* If you do not specify this parameter, Tempus creates hierarchical ports as needed.
`-reference_pin` *refInstName refTermName*	Specifies the pin *refPinName* on instance *refTermName* connected to the net that Tempus connects to the terminal. You cannot specify the `-port` parameter if you use this parameter.
`-hport` *portName*	Specifies the hierarchical port used to connect the terminal with the net. If you specify this parameter, the hierarchical port must exist in the module that contains the instance. The hierarchical port must connect to the net. This parameter lets you use a specific port to maintain the same netlist topology, which simplifies equivalence checking later in the design flow. You cannot specify the `-pin` parameter if you use this parameter. *Default:* If you do not specify this parameter, Tempus uses existing ports or creates new hierarchical ports as necessary to connect the terminal to the net.
`-pin` *termName*	Specifies the name of the terminal that Tempus connects to the specified net. If the terminal name does not exist, Tempus displays an error message and the command stops.

Examples

- The following command attaches terminal `in1` of instance `i1/i2/i3` to net `i1/i2/net26`:

  ```
  connect_pin i1/i2/i3 in1 i1/i2/net26
  ```

  If `i1/i2/i3` does not exist, or if `in1` is not a terminal of `i1/i2/i3`, or if `i1/i2/net26` does not exist, the software displays an error message and the command stops.

- The following command attaches terminal `in1` of instance `i1/i2/i3` to net `net27`, using

hierarchical port `myPort`:

```
connect_pin i1/i2/i3 in2 net27 -port myPort
```

The `myPort` port must exist in the module definition for `i1/i2`, and `myPort` must already connect to `net27`. If this is not the case, the software displays an error message and the command stops.

- The following command attaches terminal `in3` on instance `i1/i2/i3` through existing ports to `net28`:

  ```
  connect_pin -no_new_port i1/i2/i3 in3 net28
  ```

  If ports do not exist, the software displays an error message and the command stops.

Related Topic

- `disconnect_pin`

create_hport

```
create_hport
[-help]
[-bus msb:lsb]
-direction {in out inout}
-name hport_name
[-parent module|hinst|design]
```

Create new hports and corresponding hpins for the parent object. It also implicitly creates hnet and net objects inside the parent object with the same name as the new hport. If the new hport is a bus, then the bus_bits objects are also created. The hnet object has no corresponding hnet or net object but is left dangling. It returns the newly created hport object pointer.

Parameters

`-bus msb:lsb`	Specifies the bit range for bus hports. You must separate the start and end IDs with a colon (:). If there is an existing bus (for example, 3:0) then additional bits can be added provided they are adjacent to the existing bits (7:4 for example). By default, the tool creates a scalar hport. *Data_type*: string, optional
`-direction {in out inout}`	Specifies the direction of the new port, in, out, inout. *Data_type*: enum, required
`-help`	Outputs a brief description that includes the type and default information for each `create_hport` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_hport`

`-name` *`hport_name`*	Specifies the name of the new hport. If this is the full hierarchical path, then the parent will be derived from the name (i.e. the parent name is the string before the last "/". In this case, the base_name of the new hport (i.e. the string after the last "/") should not have any special characters that require escaping. When the `-parent` option is used, then the name is the base_name of the new hport. If the name has any illegal chars, it will be automatically escaped. Therefore, `-name "hp1/2"` will result in `"hp1\/2"` for the .escaped_name, since it uses DEF escaping semantics. This is equivalent to checking if *`hport_name`* is a legal Verilog identifier, and if not, prepend \ and add a trailing space to the internal Verilog name. *Data_type*: string, required
`-parent` *`module|hinst|design`*	Specifies the parent for the new hport. *Data_type*: (hinst|design|module), optional

Example

```
CPU
 |
 +-----------------+
 |                 |
 H1 (alu)          H2 (fsm)
 |
 +-----------------+
 |                 |
+ H1 (add)         H2 (mult)
```

- The following command creates new scalar objects `hpin:CPU/H1/H1/P1`, `hport:CPU/H1/H1/P1`, and `hnet/net H1/H1/P1`.

  ```
  create_hport -name P1 -parent add -direction in
  ```

- The following command creates a new 8-bit objects `hpin:H1/H2/BP1`, `hport:H1/H2/BP1`, and `hnet/net H1/H2/BP1`.

  ```
  create_hport -name BP1 -parent H1/H2 -bus 7:0
  ```

- The following command creates new scalar objects `hpin:CPU/H1/H2/P2`, `hport:CPU/H1/H2/P2`, and `hnet/net H1/H1/P2`.

  ```
  create_hport -name H1/H2/P2 -direction out
  ```

- The following commands add a single bit to an hport:

  ```
  get_db hports *data_in*
  {hport:cpu/H1/data_in[3]} {hport:cpu/H1/data_in[2]} {hport:cpu/H1/data_in[1]}
  {hport:cpu/H1/data_in[0]}
  create_hport -parent H1 -name data_in -bus 4:4 -direction in
  ```

create_module

```
create_module
[-help]
-name module_name
```

Create new hports and corresponding hpins for the parent object. It also implicitly creates hnet and net objects inside the parent object with the same name as the new hport. If the new hport is a bus, then the bus_bits objects are also created. The hnet object has no corresponding hnet or net object but is left dangling. It returns the newly created hport object pointer.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_module` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_module`
`-name` *`module_name`*	Specifies the name of the new module. If the name has any illegal chars, it will be automatically escaped. Therefore, `-name "m1/m2"` will result in `"m1\/m2"` for the .escaped_name, since it uses DEF escaping semantics. This is equivalent to checking if *`hinst_name`* is a legal Verilog identifier, and if not, prepend \ and add a trailing space to the internal Verilog name. *Data_type*: string, required

Example

- The following command creates `module:<design>/mod1`.

  ```
  create_module -name mod1
  ```

- The following command creates `hinst:<design>/H1`.

  ```
  create_hinst -name H1 -module mod1
  ```

- The following command creates `hpin:<design>/H1/in`.

  ```
  create_hport -name in -module mod1 -input
  ```

- The following command creates `hpin:<design>/H1/out`.

  ```
  create_hport -name out -module mod1 -output
  ```

create_port

```
create_port
[-help]
[-bus msb:lsb]
-direction {in out inout}
-name port_name
```

Create new ports for a design object and returns the newly created port object pointer. It also implicitly creates hnet and net objects with the same name connected to the port.

Parameters

`-bus msb:lsb`	Specifies the bit range for bus ports. You must separate the start and end IDs with a colon (:). If there is an existing bus (for example, 3:0) then additional bits can be added provided they are adjacent to the existing bits (7:4 for example). Default is to create a scalar port. *Data_type*: string, optional
`-direction {in out inout}`	Specifies the direction of the new port, in, out, inout. *Data_type*: enum, required
`-help`	Outputs a brief description that includes the type and default information for each `create_module` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_module`. *Data_type*:
`-name module_name`	Specifies the name of the new port. If the name has any illegal chars, it will be automatically escaped. Therefore, `-name "p1/2"` will result in `"p1\/2"` for the .escaped_name, since it uses DEF escaping semantics. This is equivalent to checking if *port_name* is a legal Verilog identifier, and if not, prepend \ and add a trailing space to the internal Verilog name. *Data_type*: string, required

Example

- The following command creates a new input scalar port called P1:

```
create_port -name P1 -direction in
```

- The following command creates a new 8-bit output port called BP1

```
create_port -name BP1 -bus 7:0 -direction out
```

delete_inst

```
delete_inst
-inst list_of_instances
[-module moduleName ]
```

Deletes an instance from a design. Tempus checks whether the instance is physical or logical and deletes the instance accordingly.

You can use wildcards (`*?`) to specify the instances you want the tool to delete.

Parameters

`-inst list_of_instances`	Specifies the name of the instance to delete.
`-module moduleName`	Specifies the name of the Verilog module.

Example

- The following command deletes instance `i1/i2`:

  ```
  delete_inst i1/i2
  ```

- The following command deletes all instances whose names begin with `i`:

  ```
  delete_inst i1/i*
  ```

Related Topic

- `update_inst`

disconnect

```
disconnect
[-help]
pin|hpin|port|hport|pg_pin+
[-keep_dangling_net]
```

Disconnects pins, ports, hports, or hpins from any hnet or net. For example, if A, B, and C are connected to net N , and you disconnect A, then B and C remain connected to N, but A is now left unconnected. For hpin/hport objects, they are disconnected only from the local hnet they are connected to. So, for an hpin, it will be disconnected from the hnet above the hpin (outside the hinst of the hpin). For hports, it will be disconnected from the hnet below the hport (the hnet inside the hinst of the hport) but the associated hpin/hport to hnet connection is not affected. You need to disconnect them both if you want both connections removed. Also, if an hpin/pin is directly connected to a constant (e.g. has .constant 0/1 and no hnet), the .constant attribute on the hpin/pin will be cleared, and return “no_constant”.

By default, dangling hnets that have no port, hport, hpin, or pin attached to it are deleted. Any constants like 1’b0/1’b1 are ignored, so a constant hnet is considered dangling if there are no port, hport, hpin or pin objects attached to it.

> The `disconnect` command is a limited-access feature in this release. It is enabled by a variable specified using the `set_limited_access_feature` command. To use this feature, contact your Cadence representative to explain your usage requirements, and make sure this feature meets your needs before deploying it widely.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `disconnect` parameter. For a detailed description of the command and all of its parameters, use the man command: `man disconnect`
`-keep_dangling_net`	Keeps the net object even if it has no loads or drivers. Default is to delete dangling nets. *Data_type*: bool, optional
pin|hpin|port|hport|pg_pin+	List of objects to disconnect (port/pin/hpin/hport/pg_pin). Only objects are supported, no simple names. *Data_type*: (pin|hpin|port|hport|pg_pin)+, required

Example

Consider the following example:

```
@tempus1>get_db hport:H1/B1 .hnet

hnet:H1/B1

@tempus2>get_db hpin:H1/B1.hnet

hnet:P1
```

If the following command is run, the tool will disconnect hport:H1/B1 from hnet:H1/B1, but leave hpin:H1/B1 connected to hnet:P1

```
@tempus2>disconnect hport:H1/B1
```

disconnect_hpin

```
disconnect_hpin
[-help]
-hinst hinstName
-pin_name portName
```

Detaches the net connected to the specified port on the specified instance.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each disconnect_hpin parameter. For a detailed description of the command and all of its parameters, use the man command: `man disconnect_hpin`
`-hinst hinstName`	Specifies the name of the hierarchical instance.
`-pin_name portName`	Specifies the port on the module.

Example

- The following command detaches port `p1` from `moduleA`:

  ```
  disconnect_hpin moduleA p1
  ```

Related Topic

- connect_hpin

disconnect_pin

```
disconnect_pin
-inst instName
[-net netName]
-pin terminalName
```

Disconnects a terminal from a net.

Note: Detaching a terminal that drives an output terminal of a module produces a Verilog violation at the output terminal if you use `disconnect_pin`. Instead, use `connect_pin` to attach the terminal to a new net. The `connect_pin` command automatically detaches the terminal from the net connecting to the output terminal, then attaches the terminal to the net you specify.

Parameters

`-inst` *instName*	Specifies the instance that contains the terminal you want to detach.
`-net` *netName*	Specifies the net that is already connected to the terminal. If the terminal does not connect to the specified net, or if the net does not exist, the software displays an error message and the command stops. The software uses this parameter for error checking only.
`-pin` *terminalName*	Specifies the terminal to disconnect. If the terminal does not exist on the specified instance, the software displays an error message and the command stops.

Example

- The following command disconnects terminal `in1` of instance `i1/i2/i3`:

  ```
  disconnect_pin i1/i2/i3 in1 i1/i2/net26
  ```

  If instance `i1/i2/i3` does not exist, or terminal `in1` is not a terminal of `i1/i2/i3`, the software displays an error message and the command stops. The net `i1/i2/net26` is specified, so if net `i1/i2/net26` does not exist, or if the terminal is not already connected to net `i1/i2/net26`, the software displays an error message and the command stops.

Related Topic

- connect_pin

gui_show_buffer_tree

```
gui_show_buffer_tree
-net netName
[-out_file fileName]
```

Displays a buffer tree associated with the specified net.

Parameters

`-net netName`	Specifies the name of the net for which you want to display the buffer tree.
`-out_file fileName`	Specifies the text file containing buffer locations.

Example

- The following command highlights instances and wires in the buffer tree associated with the net `reset` in the design display area:

  ```
  gui_show_buffer_tree -net reset
  ```

update_inst

```
update_inst
[-help]
-base_cell base_cell
[-dont_snap_to_placement_grid]
[-location {x y}] -name inst_name
[-orient {r0 r90 r180 r270 mx mx90 my my90}]
[-place_status {fixed soft_fixed placed unplaced cover}]
```

Updates the base cell of an existing instance object and returns the updated inst object pointer. The new base cell and existing base cell must have the identical pins.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `update_inst` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man update_inst`
`-base_cell` *`base_cell`*	Specifies the base cell to use for the new instance. *Data_type*: (base_cell), required)
`-dont_snap_to_placement_grid`	If `-dont_snap_to_placement_grid` is specified, the tool places the instance at the specified location even if the location is off the placement grid. This parameter is used with `-location`. *Default*: false *Data_type*: bool, optional
`-name` *`instName`*	Specifies the name of the instance to update. *Data_type*: (inst), required
`-location {`*`x y`*`}`	Specifies the location of the changed instance. *Default:* Default is the same location. *Data_type*: point, optional
`-orient {r0 r90 r180 r270 mx mx90 my my90}`	Specifies orientation of the changed instance. *Default:* Default is the same orientation. *Data_type*: enum, optional
`-place_status {fixed soft_fixed placed unplaced cover}`	Specifies the placement status of the changed instance. *Default*: Default is the same place_status. *Data_type*: enum, optional

Example

- The following command updates the base_cell of `inst i1/i2` from `BUFX2` to `BUFX4`, and then marks it as fixed:

```
update_inst -name i1/i2 -base_cell BUFX4 -place_status fixed
```

Related Information

- `create_inst`
- `delete_inst`

14

Low Power Commands and Attributes

- add_pg_netlist_power_intent
- check_power_domains
- clone_msv_gate
- commit_power_intent
- cut_power_domain_by_overlaps
- delete_power_intent
- get_cpf_user_attributes
- power_intent Category Attributes
- read_power_intent
- report_isolation
- report_partition_boundary_ports
- report_power_domains
- report_shifters
- report_voltage
- set_lib_binding_by_voltage
- update_power_domain
- write_power_intent

add_pg_netlist_power_intent

```
add_pg_netlist_power_intent
[-help]
[-default_net string]
```

Initiates power intent inferencing after loading the design with the template view definition file. This command creates an individual power domain for each power net, similar to the SMSC PGV flow. It

also updates the view definition file and `delay_corners` with the power domain information. This command creates library sets for each power domain based on PVT. Power domain PVT is inferred based on the voltage from the associated power net of the domain, and the process and temperature of design-level PVT for the corner.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `add_pg_netlist_power_intent` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man add_pg_netlist_power_intent`
`-default_net` *string*	Specifies the default power net.

check_power_domains

```
check_power_domains
[-help]
[-all_inst_in_power_domain]
[-always_on_bias]
[-always_on_buffer_type]
[-antenna_diode {fileName ...}]
[-cell_bind {cell ...}]
[-drc_file string]
[-lib_binding_voltage]
[-lib_cell_binding]
[-global_connection [module_name_list]]
[-nets_missing_iso [fileName]
[-ignore_net_to_port_iso]
[-ignore_net_to_port_shifter]
[-place [-place_report report_name]]
[-power_switch]
[-power_within_min_gap]
[-retention {fileName ...}]
[-nets_missing_shifter [fileName] [-ignore_net_to_port_shifter]]
```

Verifies whether the power domains are set up correctly in terms of timing library binding and PG connection. This command also checks whether any power domains overlap. The `check_power_domains` command checks that libraries specified for each power domain of each `dc_corner` are complete for each power domain member list.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `check_power_domains` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man check_power_domains.`
`-all_inst_in_power_domain`	Verifies all instances are in the power domain.

`-always_on_bias`	Checks the `always_on_bias` net connection.
`-always_on_buffer_type`	Checks if the AO buffer in each domain has correct type in terms of AO Nwell design and IEEE1801 Nwell specification. *Data_type*: bool, optional
`-antenna_diode {`*`fileName ...`*`}`	
	Verifies antenna diode-related low power violations for the specified file.
`-cell_bind {`*`cell ...`*`}`	Checks against all views and power domains if binding exists in the list of specified cells.
`-drc_file` *`file Name`*	Creates the Design Rule Checking (DRC) violation markers and save those in the specified file.
`-lib_binding_voltage`	Checks voltage against the binding library.
`-lib_cell_binding`	Verifies that all cells are bound to correct timing library cells.
`-global_connection` *`module_name_list`*	
	Verifies that power and ground pins of instances within the power domain are connected to the power and ground nets of that power domain. In addition, this parameter verifies that the tie-high and tie-low connections of an instance are connected to the same power and ground nets of the power domain to which the cell belongs. If you do not specify a list of module names, all modules are verified.
`-nets_missing_iso`	Verifies if the nets are missing isolation instances according to the power intent file. It also checks if the isolation uses right type of cell specified in the IEEE1801 map_isolation_cell command (if any).
`-ignore_net_to_port_iso`	Ignores the nets connecting to I/O when verifying isolation using `-nets_missing_iso`.

`-ignore_net_to_port_shifter`	Ignores the nets connecting to I/O when verifying shifter using `-nets_missing_shifter`.
`-place`	Verifies if the instances are placed in the boundary of their power domain.
`-place_report` *report_name*	
	Specifies a name and location for the report generated when you use this command with the `-place` parameter.
`-power_switch`	Verifies whether each row in the power domains contain at least one power switch. The command issues a warning message detailing which rows do not contain at least one switch. It also checks if the power switch uses right type of cell specified in the IEEE1801 `map_power_switch_cell` command (if any).
`-power_within_min_gap`	
	Verifies that the power domain ring is within the distance specified by the `update_power_domain -gap_sides` command parameter.
`-retention {`*fileName ...*`}`	
	Specifies a name for storing the verify retention information. It checks if the retention uses the right type of cell specified in the IEEE1801 `map_retention_cell` command (if any).
`-nets_missing_shifter`	Verifies if the nets are missing level shifter according to the power intent file. It also checks if the level shifter uses right type of cell specified in the IEEE1801 `map_level_shifter_cell` command (if any).

Examples

The following command issues warning messages if instances that are not members of a power domain are placed within that power domain boundary, or if any power domains overlap:

```
check_power_domains -place
```

Related Commands

- `report_power_domains`
- `update_power_domain`

clone_msv_gate

```
clone_msv_gate
[-iso]
[-shifter]
[-instances inst1 inst2 .... ]
[-max_fanout integer]
[-max_transition num]
[-max_cap num]
[-honor_pre_placed]
[-honor_dont_touch]
```

Ensures that the cloned isolation or level shifter can drive a group of the receivers and be placed close to this group of receivers. By doing this, the timing QoR can be improved.

Note: The command will not automatically update your CPF file to match the new netlist. It is advised that you run Conformal Low Power to check for any issues. When the enable signal for isolation cells is cloned (creating new ports on the boundary of the power domain) the users may need to update the `create_isolation_rule` in CPF by defining the isolation control differently.

Parameters

`-honor_dont_touch`	Does not clone don't touch instances or cells. The default is to clone don't touch instances.
`-honor_pre_placed`	Does not clone pre-placed (fixed) instances. The default is to clone fixed instances
`-instances` *inst1 inst2 ...*	Clones specified instances.
`-max_fanout` *integer*	Specifies the maximum fanout count.
`-iso`	Clones isolation cells.
`-max_cap` *num*	An optional check to clone instances with maximum capacitance violation. The default is not to use `-max_cap` violation.
`-max_transition` *num*	Cloning will be only on instances with maximum transition time violation. The default is `2ns`.
`-shifter`	Clones level shifters.

commit_power_intent

```
commit_power_intent
[-help]
[-verbose]
```

Commits power intent specifications loaded in the session using the `read_power_intent` command(s) to be used in the verification and implementation of the structure and behavior of the design in context of a given power management architecture. The method supports incremental refinement of power intent specifications required for IP-based design flows.

Note: The `commit_power_intent` command needs to be run after all the `read_power_intent` command(s) have been issued in the flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `commit_power_intent` parameter. For a detailed description of the command and all of its parameters, use the man command: `man commit_power_intent`
`-verbose`	Prints the detailed information related to the `commit_power_intent` command.

Related Commands

- `delete_power_intent`
- `read_power_intent`

cut_power_domain_by_overlaps

```
cut_power_domain_by_overlaps
power_domain_name
[-help]
```

Stylus Common UI does not support the overlapped power domains. When you want to floorplan one power domain (inner domain) completely inside the other power domain (outer domain), use this command to cut the outer domain as a hollow or donut shape.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `cut_power_domain_by_overlaps` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man cut_power_domain_by_overlaps`
`power_domain_name`	Specifies the outer power domain where the nested power domain will reside.

delete_power_intent

```
delete_power_intent
[-help]
[-keep_fence]
```

The command deletes all the power intent data stored by the `commit_power_intent` command in current session and cleans up the design.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `delete_power_intent` parameter. For a detailed description of the command and all its parameters, use the man command: `man delete_power_intent`
`-keep_fence`	Retains domain groups and their fences at the time of removing the LP database for completing non-LP status of applications. After completing the non-LP status of the required applications, use `read_power_intent` and `commit_power_intent -keep_rows` to restore the retained domain groups and domain fences of the removed LP database.

Related Commands

- `commit_power_intent`
- `read_power_intent`
- `write_power_intent`

get_cpf_user_attributes

```
get_cpf_user_attributes
[-help]
-domain domainName | -net netName
```

Reports the `user_attributes` of the power domain or net specified in the CPF file. If the domain or net has no `user_attributes` in the CPF file, then this command reports an empty string.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_cpf_user_attributes` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_cpf_user_attributes`
`-domain`	Specifies the name of the power domain containing the user_attributes.
`-net`	Specifies the name of the net having the `user_attributes`.

power_intent Category Attributes

The root attributes in the `power_intent` category are:

- handle_old_lib_with_signal_level
- power_intent_allow_back_to_back_isolation
- power_intent_allow_nested_default_domain
- power_intent_allow_power_domain_min_gap_zero
- power_intent_allow_shifter_voltage_mismatch
- power_intent_assume_iso_enable_pin_is_always_on
- power_intent_check_all_nets_for_domain_crossing
- power_intent_do_not_use_top_domain_for_port_voltage
- power_intent_honor_power_domain_for_domain_crossing_route
- power_intent_honor_power_domain_for_intra_domain_route
- power_intent_include_dot_lib_related_pg_pin
- power_intent_use_cpf_global_connect_for_always_on_buffer
- power_intent_use_cpf_global_connect_for_shifter
- power_intent_use_effective_domain_for_iso_shifter_insertion

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `power_intent` category, use the following command:

```
get_db -category power_intent
```

handle_old_lib_with_signal_level

```
handle_old_lib_with_signal_level {true | false}
```

Default: `false`

Data_type: Boolean

Allows using the old-style `.lib` with `signal_level` and `power_rail` in the MSV cell identification and term voltage.

power_intent_allow_back_to_back_isolation

```
power_intent_allow_back_to_back_isolation {true | false}
```

Default: `false`

Data_type: Boolean

Supports back-to-back isolation insertion along one cross-domain net based on the CPF ISO rule. Set true to enable.

power_intent_allow_nested_default_domain

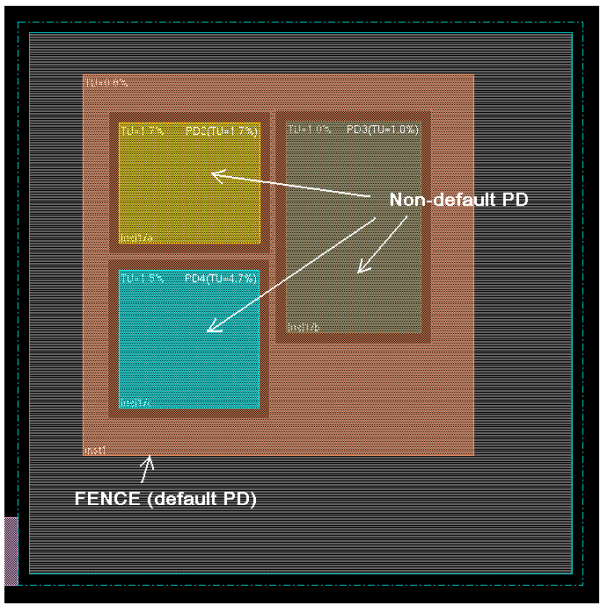
```
power_intent_allow_nested_default_domain {true | false}
```

Default: `true`

Data_type: Boolean

Allows a non-default power domain member to be logically nested in a default power domain. This default power domain member has fence constraint.

By default, due to grouping, the fence constraint is lost for the non-default power domain member. Set true to enable.



In the netlist: modules `HInst11` and `HInst12` are under `HInst1`; and `HInst1` is under the top.

You have the following:

```
create_power_domain -instances HInst1

create_power_domain -default -instances {HInst11 HInst12}...
```

That means `HInst11` and `HInst12` are cut from `HInst1` to default power domain. If there is a fence/region constraint for `HInst11` and `HInst12` in default power domain, then please turn this global on. Otherwise, the fence/region for `HInst11` and `HInst12` will be lost.

power_intent_allow_power_domain_min_gap_zero

```
power_intent_allow_power_domain_min_gap_zero {true | false}
```

Default: `true`

Data_type: Boolean

Allows users to set power domain zero `minGap`.

power_intent_allow_shifter_voltage_mismatch

```
power_intent_allow_shifter_voltage_mismatch {true | false}
```

Default: `true`

Data_type: Boolean

When set to `false`, it prevents insertion of level shifters by allowing voltage tolerance.

power_intent_assume_iso_enable_pin_is_always_on

```
power_intent_assume_iso_enable_pin_is_always_on {true | false}
```

Default: `true`

Data_type: Boolean

Insert ISO/LS for all the inter-domain nets using effective domain based on the CPF rules. Set `true` to enable.

power_intent_check_all_nets_for_domain_crossing

```
power_intent_check_all_nets_for_domain_crossing {true | false}
```

Default: `true`

Data_type: Boolean

Checks all the nets for domain crossings. Set `true` to enable.

power_intent_do_not_use_top_domain_for_port_voltage

```
power_intent_do_not_use_top_domain_for_port_voltage {true | false}
```

Default: `false`

Data_type: Boolean

Controls voltage using top fterm domain for IO pin voltage.

power_intent_honor_power_domain_for_domain_crossing_route

```
power_intent_honor_power_domain_for_domain_crossing_route {true | false}
```

Default: `true`

Data_type: Boolean

Routes nets belonging to more than one power domain so that the routing does not violate power domain boundaries, in addition to routing intra-power domain and global nets using the specifications of the power domains.

For example, when you specify `power_intent_honor_power_domain_for_domain_crossing_route true`, if a net belongs to power domains PD1 and PD3, but not PD2, Early Global Route routes the net without going over the PD2 power domain boundary.

power_intent_honor_power_domain_for_intra_domain_route

```
power_intent_honor_power_domain_for_intra_domain_route {true | false}
```

Default: `true`

Data_type: Boolean

Routes intra-power domain and global nets using the specifications of the power domains. If you specify `power_intent_honor_power_domain_for_intra_domain_route true`, Early Global Route routes intra-power domain nets only within their power domains. Global nets that do not connect with a power domain do not get routed through that power domain.

power_intent_include_dot_lib_related_pg_pin

```
power_intent_include_dot_lib_related_pg_pin {true | false}
```

Default: `true`

Data_type: Boolean

Honors library related PG pins. Set true to enable.

power_intent_use_cpf_global_connect_for_always_on_buffer

```
power_intent_use_cpf_global_connect_for_always_on_buffer {true | false}
```

Default: `false`

Data_type: Boolean

Honors connection pecs for AO buffer based on the CPF rules. Set `true` to enable.

power_intent_use_cpf_global_connect_for_shifter

```
power_intent_use_cpf_global_connect_for_shifter {true | false}
```

Default: `false`

Data_type: Boolean

Honors connection specs for shifters based on the CPF rules. Set `true` to enable.

power_intent_use_effective_domain_for_iso_shifter_insertion

```
power_intent_use_effective_domain_for_iso_shifter_insertion {true | false}
```

Default: `true`

Data_type: Boolean

Honors effective domains for ISO/LS insertions. Set `true` to enable.

read_power_intent

```
read_power_intent
[-help]
power_intent_file
{-cpf | -1801 | -msv_db }
```

Reads-in power intent file and does syntax check. Multiple `read_power_intent` commands can be issued in the flow. This method supports incremental refinement of power intent specifications required for IP-based design flows.

Note: The `commit_power_intent` command needs to be run after all the `read_power_intent` command(s) have been issued in the flow.

Parameters

`-help`	Prints a brief description that includes type and default information for each `read_power_intent` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_power_intent`
`power_intent_file`	Specifies the name of the power intent file to be read.
`-cpf`	Specifies that the CPF format power intent file will be used.
`-1801`	Specifies that the IEEE1801 format power intent file will be used.
`-msv_db`	Specifies that the IEEE1801 data format `(.cpfdb)` power intent file will be used. **Note:** `-msv_db` is for partition flow and must be read for power intent in partition implementation.

Related Commands

- commit_power_intent
- delete_power_intent
- write_power_intent

report_isolation

```
report_isolation
[-help]
[-highlight]
[-out_file fileName]
[-from powerDomain]
[-to powerDomain]
```

Reports or shows the added isolation cells and instances.

Note: Use this command after you add isolation cells.

Parameters

-help	Outputs a brief description that includes type and default information for each report_isolation parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man` report_isolation.
`-from`	Reports all the isolation instances connected to the output pins of the power domain specified in `-from` option. *Default:* Reports all isolation cells.
`-highlight`	Highlights isolation instances in the main window.
`-out_file` *fileName*	Writes isolation cell and instance names to a report. *Default:* The software writes the report to a default file: *designName*`.isolation.rpt`
`-to`	Reports all the isolation instances connected to the input pins of the power domain specified in `-to` option. *Default:* Reports all isolation cells.

Examples

The following command highlights isolation cells and writes the cell names to `test.isolation.rpt`:

```
report_isolation -highlight -out_file test.isolation.rpt

#DomainName CellName            InstanceName

PD1         lvl_in_pp_f1_pm_dh  sub_inst2/LIN

PD1         lvl_in_pp_f1_pm_dh  sub_inst2/LCLK
```

report_partition_boundary_ports

```
report_partition_boundary_ports
-pins
```

The `report_partition_boundary_ports` command prints out the boundary pins' debug information in a log which shows how to assign the power domain for the partition pins.

This is useful when debugging CPF partitions as you get the information about how Tempus assigns the power domain for the partition boundary pins during debugging.

Parameters

-pins	Specifies a list of the partition pins. If wildcard * is specified, it means all partition pins.

Example

Specify the partition pins(Hterm) you want to debug:

```
report_partition_boundary_ports "I2/d I2/clk"
```

After partition, the command looks for the following DBG info in the log file
DBG: deriving hterm I2/d power domain
DBG: ….

report_power_domains

```
report_power_domains
[-help]
[-bind_lib]
[-insts inst_name]
[-iso_inst]
[-modules {list_of_modules}
[-nets net_name]
[-out_file outFile]
[-pg_nets]
[-pins]
[-power_domain powerDomainName [-check_pg_scope]]
[-power_domain_boundary_port string]
[-power_domain_coverage pd1 pd2]
[-shifter]
[-supply_crossing supp1 supp2]
[-verbose]
[-voltage]
```

Reports information about power domains. The report includes the following attributes:

- Power domain name
- Power domain bounding box
- Timing library
- Mingap
- RS extension
- Row type
- Row spacing

The command also reports whether the power domain is always on.

Parameters

`-help`	Outputs a brief description that includes type and default information for each report_power_domains parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man` report_power_domains.
`-bind_lib`	Reports the timing libraries available for instance binding including voltage scaling. However, the actual libraries that are used by the AAE delay calculation might be a subset of the timing libraries reported by this command.
`-check_pg_scope`	Checks if there are any PG connections that cross UPF-defined scope. All PG connections for UPF scope should be at the boundary PG port. This parameter is used with the `-power_domain` parameter.
`-insts` *inst_name*	Reports the power domain-related information for the specified instance: the input, output, and power/ground connections.
`-iso_inst`	Reports all isolation cells in the power domain.
`-modules {list of modules}`	Checks whether the specified instances or hierarchical instances are in the power domain. *Default:* The software does not perform this check.
`-nets` *net_name*	Reports power domain-related information for the specified net: • The driver instance of the net and its power domain. • The receiver instances of the net and their power domains.
`-out_file` *outFile*	Specifies the name of the output file. *Default:* *designName*`-powerdomain.rpt`
`-pg_nets`	Reports all the power/ground nets in the power domain.
`-pins` *pin1, pin2 ...*	Lists the pins and the option is of type string. This option is optional.
`-power_domain` *powerDomainName*	Specifies the power domain whose attributes you want to report. *Default:* If you do not specify the power domain, this command report data about all power domains.

`-power_domain_boundary_port` *string*	Reports if the specified port name of the power domain boundary has an IEEE1801 constraint.
`-power_domain_coverage` *pd1 pd2*	Reports the coverage between two specified domains.
`-shifter`	Reports all shifters in the power domain.
`-supply_crossing` *supp1 supp2*	Reports if the two specified supplies have the on/off and low/high crossing.
`-verbose`	Prints out the detail information and is optional.
`-voltage`	Reports domain voltage.

Examples

- The following command reports the power domain-related information for the specified instance:

```
report_power_domains -insts i4 -verbose -bind_lib

Analyzing i4 (Cell: BUFX1) power domain: AO

------- from side --------

  Pin A => Net n4

    sub_inst1/i4/Q in power domain AO

------- to side --------

  Pin Y => Net n5

    sub_inst2/LIN/a in power domain PD1 connected to domain AO

=============PG pins =================

VSS connected to Net: VSS

VDD12 connected to Net: rail1


VSS is connected to net:VSS by CPF GNC rule: -net VSS -pins  VSS -powerDomain AO

VDD12 is connected to net:rail1 by CPF GNC rule: -net rail1 -pins  VDD12 -powerDomain
AO
```

- The following command reports the attributes of the specified power domain in the `out.lib` file:

```
report_power_domains -power_domain pd_VDDalive -out_file out.lib

 PowerDomain  'pd_VDDalive':

 ========================================

   Voltage = 0.0000

   Timing Lib:

   Setup View: Setup_SSG_m40C

   Hold View: Hold_SSG_m40C

   minGaps = T:0.0000 B:0.0000 L:0.0000 R:0.0000

   rsExts  = T:0.0000 B:0.0000 L:0.0000 R:0.0000

   core2Side = T:0.0000 B:0.0000 L:0.0000 R:0.0000

   rowSpaceType = 2

   rowSpacing = 0.0000

   rowFlip = noFlip

   AlwaysOn = 0
```

- The following command reports the timing libraries available for instance binding for the specified power domain:

```
report_power_domains -power_domain pd_VDDalive -bind_lib -out_file out.lib

Timing library binding for instance(s):

==================================

INST = pm_inst/buf5 (tcbn20socbw22p90ulvt_ffg_0p99v_m40c_cbest)

INST = pm_inst/buf4 (tcbn20socbw22p90ulvt_ffg_0p99v_m40c_cbest)
```

- The following command reports all isolation cells for the specified power domain:

```
report_power_domains -power_domain pd_VDDalive -iso_inst -out_file out.lib

ISOLATION instances for powerDomain (pd_VDDalive)

==========================================

  ====> Total '0' isolation instance(s) for powerDomain 'pd_VDDalive'
```

- The following command reports all the power/ground nets for the specified power domain:

```
report_power_domains -power_domain pd_VDD_top -pg_nets -out_file out.lib

GlobalNet Connection (pd_VDD_top) =
```

```
====================================

* (VDD_top:VDD) (VDDalive:VPP)  (VSS:VSS) (VSS:VBB)
```

- The following command reports all shifters and voltage for the specified power domain:

```
report_power_domains -power_domain pd_VDD_top -shifter -voltage -out_file out.lib

SHIFTER(s) for powerDomain (pd_VDD_top)

==================================

====> Total '0' shifter(s) for powerDomain 'pd_VDD_top'


Power Domain Voltages

========================================

Analysis View: Setup_SSG_m40C

Power Domain: pd_VDD_top, Primary Power Net: VDD_top, 0.99v

Power Domain: pd_VDD1, Primary Power Net: VDD1, 0.99v

Power Domain: pd_VDDalive, Primary Power Net: VDDalive, 0.99v
```

- The following command reports the attributes of the specified power domain:

```
report_power_domains -power_domain pd_VDD_top -verbose -out_file out.lib

SHIFTER(s) for powerDomain (pd_VDD_top)

==================================

====> Total '0' shifter(s) for powerDomain 'pd_VDD_top'

ISOLATION instances for powerDomain (pd_VDD_top)

========================================

====> Total '0' isolation instance(s) for powerDomain 'pd_VDD_top'

GlobalNet Connection (pd_VDD_top) =

====================================

  *     (VDD_top:VDD) (VDDalive:VPP)  (VSS:VSS) (VSS:VBB)
```

- The following command reports power domain-related information for the specified net:

```
report_power_domains -nets CTS_2

---- Analyzing CTS_2 ---

IN  : ff2/CP pd_VDD_top
```

```
IN  : ff1/CP pd_VDD_top

OUT : CTS_ccl_a_buf_00020/Z pd_VDD_top
```

- The following command reports the power domain-related information for the specified instance:

```
report_power_domains -insts CPF_PS_psoI_PD_top_1_HDRSID2BWP22P90ULVT_39_132_106

Analyzing CPF_PS_psoI_PD_top_1_HDRSID2BWP22P90ULVT_39_132_106 (Cell:
HDRSID2BWP22P90ULVT) power domain: pd_VDD_top

------- from side --------

  Pin NSLEEPIN (related power pin TVDD) => Net psoPSI_PD_top_EnNet__1_106_39_131_0

    CPF_PS_psoI_PD_top_1_HDRSID2BWP22P90ULVT_39_131_105/NSLEEPOUT in power domain
pd_VDD_top connected to domain pd_VDDalive

------- to side --------

  Pin NSLEEPOUT (related power pin TVDD) => Net psoPSI_PD_top_EnNet__1_107_39_132_0

    CPF_PS_psoI_PD_top_1_HDRSID2BWP22P90ULVT_39_133_107/NSLEEPIN in power domain
pd_VDD_top connected to domain pd_VDDalive

=============PG pins =================

VSS connected to Net: VSS

VPP connected to Net: VDDalive

VDD connected to Net: VDD_top

VBB connected to Net: VSS

TVDD connected to Net: VDDalive
```

- The following command checks whether the specified instances or hierarchical instances are in the power domain:

```
report_power_domains -modules { CPF_PS_psoI_PD_top_1_HDRSID2BWP22P90ULVT_39_132_106
CPF_PS_psoI_PD_top_1_HDRSID2BWP22P90ULVT_39_100_78} -out_file output

INST = CPF_PS_psoI_PD_top_1_HDRSID2BWP22P90ULVT_39_132_106 (pd_VDD_top)

INST = CPF_PS_psoI_PD_top_1_HDRSID2BWP22P90ULVT_39_100_78 (pd_VDD_top)
```

Related Commands

- check_power_domains
- update_power_domain

report_shifters

```
report_shifters
[-help]
[-cell cellName]
[-from powerDomainName]
[-to powerDomainName]
[-out_file fileName]
[-highlight]
```

Reports the level shifters that have been read into or defined for the Tempus software.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_shifters` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_shifters`.
`-cell`	Reports shifters with the specified cell. *Default:* Reports all level shifters.
`-from` *powerDomainName*	Reports all the shifter instances connected to the output pins of the power domain specified in the `-from` option. *Default:* Reports all level shifters.
`-highlight`	Highlights placed level shifters.
`-out_file` *fileName*	Reports all instances in an output file. *Default:* The instances will be reported to a default output file *designName*`.shifter.rpt`. A warning message is printed to indicate a default output file is created.
`-to` *powerDomainName*	Reports shifter instances connected to the input pins of the power domain specified in the `-to` option. *Default:* Reports all level shifters.

Examples

- The following command reports all level shifters in design `chip`:

```
loadShifter -infile chip.vsf
addShifter
report_shifters
```

- The following command reports all level shifters in the `shifter` output file in highlighted mode:

```
report_shifters -highlight -out_file shifter.rpt
#DomainName     CellName        InstanceName
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LIN
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LCLK
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LOUT
```

- The following command reports all level shifters in the `shifter_power` output file from the `AO` pin to the `PD1` pin:

```
report_shifters -from AO -to PD1 -out_file shifter_power.rpt
#DomainName     CellName        InstanceName
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LIN
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LCLK
```

- The following command reports all level shifters in the `shifter_cell` output file for the `A2LVLUO_X1M_A12TH` cell:

```
report_shifters -cell A2LVLUO_X1M_A12TH -out_file shifter_cell.rpt
#DomainName     CellName        InstanceName
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LIN
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LCLK
PD1     A2LVLUO_X1M_A12TH       sub_inst2/LOUT
```

report_voltage

```
report_voltage
[-help]
[-early]
[-inst string]
[-late]
[-net string]
[-pin string]
[-quiet]
[-view string]
```

This command reports the voltage for the net/pin (term)/inst in the specific view under a specific corner.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `report_voltage` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_voltage`
`-early`	Specifies that the voltage is reported in the early mode.
`-inst string`	Specifies the instance power domain voltage (PVT).
`-late`	Specifies that the voltage is reported in the late mode.
`-net string`	Specifies the net voltage (driver's pin voltage).
`-pin string`	Specifies the term voltage.
`-quiet`	Displays only the pin voltage-related information without other additional unnecessary information to avoid huge log size.
`-view string`	Specifies the analysis view.

Examples

- The following command specifies to display the derived voltage for the net, `n1`:

```
tempus> report_voltage -net n1

-> driving term: i1/Y, mode: late
=======================
derived voltage = 0.9
0.9
```

- The following command specifies to display the net (driver's pin) voltage and its related information:

```
tempus> report_voltage -net n1 -quiet

0.9
```

- The following command specifies to display the term voltage reported in the late mode:

```
tempus> report_voltage -pin sub_inst2/in -late

-> hterm: sub_inst2/in, connected to net n5
-> net: n5, volt 0.9
=======================
derived voltage = 0.9
0.9
```

- The following command specifies to display the term voltage reported in the early mode:

```
tempus> report_voltage -pin sub_inst2/in -early

-> hterm: sub_inst2/in, connected to net n5
-> net: n5, volt 1.1
=======================
derived voltage = 1.1
1.1
```

set_lib_binding_by_voltage

```
set_lib_binding_by_voltage
[-help]
[-default_power_net string]
[-process float]
[-temperature float] -power_net_voltages {pwr_net1@volt1 pwr_net2@volt2 ...} | {-
early_power_net_voltages {pwr_net1@volt1 pwr_net2@volt2 ...} -late_power_net_voltages
{pwr_net1@volt1 pwr_net2@volt2 ...}} | {-data_early_power_net_voltages {pwr_net1@volt1
pwr_net2@volt2 ...} -data_late_power_net_voltages {pwr_net1@volt1 pwr_net2@volt2 ...} -
clock_early_power_net_voltages {pwr_net1@volt1 pwr_net2@volt2 ...} -
clock_late_power_net_voltages {pwr_net1@volt1 pwr_net2@volt2 ...}}
```

Initiates auto-inferencing of power intent, library binding, and view definition. It also provides voltages for each power net in the design. Use this command to assert voltages for power rails. This command creates the view definition file for single corner analysis and assigns the operating voltage for each power domain accordingly. It also creates library sets for each power domain based on the assigned PVT and associates the corresponding library set for the power domains.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_lib_binding_by_voltage` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man set_lib_binding_by_voltage`
`-clock_early_power_net_voltages {`*`pwr_net1@volt1 pwr_net2@volt2 ...`*`}`	Assigns power net voltage for early corner of the clock path.
`-clock_late_power_net_voltages {`*`pwr_net1@volt1 pwr_net2@volt2 ...`*`}`	Assigns power net voltage for late corner of the clock path.
`-data_early_power_net_voltages {`*`pwr_net1@volt1 pwr_net2@volt2 ...`*`}`	Assigns power net voltage for early corner of the data.
`-data_late_power_net_voltages {`*`pwr_net1@volt1 pwr_net2@volt2 ...`*`}`	Assigns power net voltage for late corner of the data.
`-default_power_net` *`string`*	Specifies the default power net.
`-early_power_net_voltages {`*`pwr_net1@volt1 pwr_net2@volt2 ...`*`}`	Assigns power net voltage for early corner.
`-late_power_net_voltages {`*`pwr_net1@volt1 pwr_net2@volt2 ...`*`}`	Assigns power net voltage for late corner.
`-power_net_voltages {`*`pwr_net1@volt1 pwr_net2@volt2 ...`*`}`	Assigns power net voltage.
`-process` *`float`*	Defines filtering condition for the timing library characterized process.
`-temperature` *`float`*	Defines filtering condition for timing library characterized temperature.

update_power_domain

```
update_power_domain
[-help]
power_domain_name
[-add_block_box string]
[-core_to_bottom valueInMicron]
[-core_to_left valueInMicron]
[-core_to_right valueInMicron]
[-core_to_top valueInMicron]
[-default_tech_site string]
[-disjoint_hinst_box_list string]
[-extra_row_pattern {row_pattern_site_name ...}]
[-first_row_site_index integer]
[-gap_sides {values}|-gap_edges {values}]
[-last_row_site_index integer]
[-power_extend_sides {values}|-power_extend_edges {values}]
[-row_flip {first second noflip auto}]
[-row_pattern_site site_name]
[-row_space_type {0 1 2}]
[-row_spacing float]
```

Renames a power domain or changes its structure.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `update_power_domain` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man update_power_domain`.
`-add_block_box`	Adds a hard block to the disjoint power domain. `-add_block_box` supports both rectangular and rectilinear macros.

`-core_to_left` *valueInMicron*	Specifies the row margin from left boundary.
`-core_to_right` *valueInMicron*	Specifies the row margin from right boundary.
`-core_to_top` *valueInMicron*	Specifies the row margin from top boundary.
`-core_to_bottom` *valueInMicron*	Specifies the row margin from bottom boundary.
`-default_tech_site` *string*	Specifies the default tech site for power domain.
`-disjoint_hinst_box_list`	Specifies a region for an Hinst. It is defined as an Hinst and Region pair.
`-extra_row_pattern {`*row_pattern_site_name* `...}`	Specifies the ROWPATTERN site name for the specified power domain. You can also specify a list of ROWPATTERN sites with this parameter. • The rows are created for the *specified* ROWPATTERN sites within the domain fence, if specified, from the domain fence bottom edge, as per the defined order and orientation of all sites. • The ROWPATTERN site needs to be defined in the LEF file. *Default*: ""
`-first_row_site_index` *integer*	Specifies the first row site index value.
`-last_row_site_index` *integer*	Specifies the last row site index value.

`-power_extend_edges`	Specifies the maximum search distance for power extension connections by domain edge in microns. You can specify the value per edge in clockwise order, starting with the vertical edge at the lower-left corner (smallest Y, and then smallest X).
`-gap_edges`	Specifies the minimum space value to other domain by power domain edge in microns. You can specify the value per edge in clockwise order, starting with the vertical edge at the lower-left corner (smallest Y, and then smallest X).
`-gap_sides` *`T B L R`*	Defines the distance, in microns, that must be reserved from the power domain boundary edges for power routing. You can specify four values, one for each edge.
`power_domain_name`	Specifies the current name of the power domain to be modified. This is a positional parameter, and you must specify this value immediately after the command name.

`-row_flip {first | second | noFlip | auto}`	Flips the orientation of either the first or second row, then follows the flipped and abutted row pattern.
	`first`
	(from bottom to top). MX -> R0 -> MX -> R0 - > …
	`second`
	(from bottom to top). R0 -> MX -> R0 -> MX -> …
	`noFlip`
	(from bottom to top). R0 -> R0 -> R0 -> R0 -> …
	The noFlip option preserves the R0 orientation for all rows (above).
	`auto`
	Detects orientation of row outside of power domain and aligns with power domain bottom. Orientation is applied to the first row at the bottom of power domain.
	So if row on left is R0, the pattern will be (from bottom to top). R0 -> MX -> R0 - > MX -> …
	If row on left is MX, the pattern will be (from bottom to top). MX -> R0 -> MX -> R0 - > …
	The `auto` option resets the software for this automatic behavior if you have previously specified `first`, `second`, or `noFlip`.
	Default: `auto`
	Note: *saveRowImage* resets the option `-row_flip` to `auto`. In this case, try moving the power domain and check the row changes.

`-row_pattern_site` *site_name*	Specifies the site name for the row pattern.
`-row_spacing` *value*	Specifies the row spacing between each row (1) or each pair of rows (2) as specified in `-row_space_type`.
`-row_space_type {0|1|2}`	Determines whether the row spacing value applies between no row (0), each row (1), or each pair of rows (2).
`-power_extend_sides` *T B L R*	Specifies the boundary for legal targets to be used by the power planning and routing commands, in conjunction with the power domain boundary. You can specify four values, one for each edge.

Example

The following command changes the route search extension values of `PowerDomain1` to 15 µm on each side:

```
update_power_domain PowerDomain1 -power_extend_sides 15 15 15 15
```

Related Commands

- `check_power_domains`
- `report_power_domains`

write_power_intent

```
write_power_intent
[-help]
file_name
incremental_1801
{-cpf [[-hinsts -top_hier] | [-hinst_only]] | -1801}
[-no_extra_connects]
```

Writes the CPF/IEEE1801 file. The command options help to write the file for CPF as well as for the IEEE1801 standard.

Parameters

`-help`	Prints a brief description that includes type and default information for each `write_power_intent` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_power_intent`
`file_name`	Specifies name of the IEEE1801 input file.
For CPF	
`-cpf`	When specified, Tempus writes out the CPF file.
`-hinst_only`	Specifies the hierarchical instance name. Writes (partitions) the CPF for the hierarchical instance specified in this option. **Note:** Must be used along with `-cpf` option.
`-hinsts`	Writes top-level CPF where the hierarchical instances specified in `-hinsts` option are hard blocks when specified together with `-top_hier -cpf`.
`-top_hier`	Writes top-level CPF when specified together with `-hinsts` and `-cpf`.
For IEEE1801	

`-1801`	Writes the IEEE1801 file from the low-power database. It filters out the non-existing instances from the original IEEE1801 file and appends to it the following: • PG connection for the newly-inserted always-on buffers • Nwell/Pwell connections for the low-power cells • `-no_isolation`/`shifter` for the isolation/level shifter strategies for the newly-created ports along the power domain boundary
`-incremental_1801`	Specifies the incremental IEEE1801 power intent.
`-no_extra_connects`	If this parameter is specified, `write_power_intent -1801` does not write out any extra `connect_supply_net` (PG connection). For example: `write_power_intent -1801 -no_extra_connects out.upf` **Note:** Extra connects means that the extra `connect_supply_net` commands appended by the tool, which are not included in the original UPF.

Related Commands

- `commit_power_intent`
- `delete_power_intent`

15

Manual ECO Commands and Attributes

- connect_net
- create_inst
- create_hinst
- create_hnet
- create_net
- delete_nets
- delete_routes
- disconnect_net
- eco_add_repeater
- eco_delete_repeater
- eco_update_cell
- eco Category Attributes
- read_eco
- write_eco

connect_net

```
connect_net
[-help]
net_name
list_of_pins
```

Attaches nets to pins. The syntax is order-dependent.

After any ECO command, the tool performs either an incremental or full timing update when you run the next command that reports timing, for example, `report_timing`.

Note: The tool supports this command for flattened designs only.

Parameters

list_of_pins	Specifies a list of pins/ports to attach to the specified net or nets.
net_name	Specifies the net to attach to the pins.

Examples

The following example attaches `net26` to pins `i1/A`, `i2/B`, and `i3/Z`.

```
tempus> connect_net net26 "i1/A i2/B i3/Z"
```

The following command is equivalent to the above example:

```
tempus> connect_net [get_nets net26] [get_cells "i1/A i2/B i3/Z"]
```

Related Commands

- create_inst
- create_net
- connect_pin
- update_inst_name
- delete_inst
- delete_nets
- disconnect_net
- disconnect_pin

create_inst

```
create_inst
[-help]
[-dont_snap_to_placement_grid]
[-location {x y}]
[-orient {r0 r90 r180 r270 mx mx90 my my90}]
[-physical] { -base_cell <base_cell> } { -name <inst_name> } [ -parent
<module|hinst|design> ] [ -place_status <place_status> ]
```

Adds an instance and places it in the design. The syntax is not order dependent.

After any ECO command, the tool performs either an incremental or full timing update when you run the next command that reports timing, for example, `report_timing`.

Note: The software supports this command for flattened designs only.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_inst` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_inst`.
`-base_cell <base_cell>`	Specifies the `base_cell` to be used for the new instance.
`-dont_snap_to_placement_grid`	Places the instance at the location specified using the `-location` parameter even if the specified location is on or off the placement grid. By default, `create_inst` snaps the placement location to the placement grid. *Default:* `false`
`-location {x y}`	Specifies the location of the placed instance. *Default:* If you do not specify a location, the software places the instance at the design origin.

`-name <inst_name>`	Specifies the name of the new instance. If this is the full hierarchical path, then the parent will be derived from the name (that is the parent name is the string before the last "/". In this case, the `base_name` of the new instance (that is the string after the last "/" should not have any special characters that require escaping. When the `-parent` parameter is used, then the name is the `base_name` of the new instance. If the name has any unknown characters, it will be automatically escaped. Thus `-name "i1/i2"` will result in `"i1\/i2"` for the `.escaped_name`, since it uses DEF escaping semantics. This is equivalent to checking if `<inst_name>` is a legal Verilog identifier, and if not, prepend \ and add a trailing space to the internal Verilog name.
`-orient {r0 r90 r180 r270 mx mx90 my my90}`	Specifies orientation of the placed instance. *Default:* `r0`
`-parent <module|hinst|design>`	Specifies the parent for the new instance. This supports both the object or the name for the specified object types. **Note**: Only honored for logical instances and will be ignored when used with `-physical`.
`-physical`	Creates a physical instance without updating the netlist. This can only be used with a `base_cell` that does not have any `lib_cells`. Also, this results in the instance .is `-physical` attribute being set to `true`.

`-place_status <unplaced|place|soft_fixed|fixed|cover>`	Specifies the placement status of the added instance during placement and optimization. You can specify the following as the placement status: • `fixed`: The status of the added instance is fixed. It has a location and cannot be moved by automatic tools, but can be moved using interactive commands. • `placed`: The status of the added instance is placed. It has a location, and can be moved using automatic tools and interactive commands. • `soft_fixed`: The status of the added instance is `soft_fixed`. It cannot be moved by global placement and can only be moved by the legalization step of detail placement • `unplaced`: The status of the added instance is unplaced. It does not have a location. Default: `unplaced` **Note:** This parameter is used with the `-location` option. It should not be used along with the `-parent` parameter.

Examples

The following example creates two instances, `inst1` and `inst2`, of library cell `LIB/BUFX2`:

```
tempus> create_inst -insts "inst1 inst2" -cell [get_lib_cells LIB/BUFX2]
```

Related Commands

- `connect_net`
- `delete_inst`

create_hinst

```
create_hinst
[-help]
[-parent <module|hinst|design>] { -name <hinst_name> } { -module <module|design> }
```

Creates a new `hinst` object and returns the newly created object pointer.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_hinst` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_hinst`
`-module <module>`	Specifies the module to be used for the new `hinst`. The new `hinst` will have `hports`/`hpins` created based on the module ports as well as `hnets` of the same name.
`-name <hinst_name>`	Specifies the name of the module instance to add.
`-parent <hinst|module|design>`	Specifies the parent for the new `hinst`. It supports both the object or the name for the specified object types. Default is the current design.

Examples

- The following command adds a hierarchical instance `A/B/spare` of module type `spareModule` to the design, here `A/B` should be an existing hierarchical instance in the design:
  ```
  create_hinst -cell spareModule -hinst A/B/spare
  ```
- The following commands create a new module `modA` with the instance `inst_modA` and its clone `inst_modA_clone`:
  ```
  create_hinst -cell modA -hinst inst_modA
  create_hinst -cell modA -hinst inst_modA_clone
  ```

The resulting Verilog netlist would have:

```
module modA();
endmodule
modA inst_modA();
modA inst_modA_clone();
```

Related Commands

- create_inst
- create_hnet
- create_net
- connect_pin
- update_inst_name
- delete_inst
- delete_nets
- disconnect_net
- disconnect_pin

create_hnet

```
create_hnet
[-help]
[-bus <msb:lsb>] { -name <hnet_name> } [ -parent <module|hinst|design> ] [ -power | -ground ]
```

Creates a new `hnet` (and parent net) object and returns the newly created `hnet` object pointer.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_hnet` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_hnet`
`-bus <msb:lsb>`	Specifies the bit range for bus nets. By default, the software creates a scalar hnet.
`-name <hnet_name>`	Specifies the name of the new instance. If this is the full hierarchical path, then the parent will be derived from the name (that is the parent name is the string before the last "/". In this case, the `base_name` of the new instance (that is the string after the last "/" should not have any special characters that require escaping. When the `-parent` parameter is used, then the name is the `base_name` of the new instance. If the name has any unknown characters, it will be automatically escaped. Thus `-name "i1/i2"` will result in `"i1\/i2"` for the `.escaped_name`, since it uses DEF escaping semantics. This is equivalent to checking if `<inst_name>` is a legal Verilog identifier, and if not, prepend \ and add a trailing space to the internal Verilog name.
`-ground`	Creates a ground net.
`-parent <module|hinst|design>`	Specifies the parent for the new `hinst`. It supports both the object or the name for the specified object types. Default is the current design.
`-power`	Creates a power net.

Examples

```
CPU
 |
 +-----------------+
 |                 |
 H1 (alu)          H2 (fsm)
```

```
 |
 +----------------+
 |                |
+ H1 (add)        H2 (mult)
```

- The following command creates a scalar top level net/hnet named `N1`

  ```
  create_hnet -name N1
  ```

- The following command creates a 8-bit net/hnet named `H1/H1/B1`

  ```
  create_hnet -name B1 -bus 7:0 -parent module:add
  ```

- The following command creates an 8-bit net/hnet named `H1/H1/B1`

  ```
  create_hnet -name B1 -bus 7:0 -parent H1/H1
  ```

Related Commands

- `create_inst`
- `create_hinst`
- `create_net`
- `connect_pin`
- `update_inst_name`
- `delete_inst`
- `delete_nets`
- `disconnect_net`
- `disconnect_pin`

create_net

```
create_net
[-help]
-module <string>
-names <list_of_nets>
```

Adds a net to the design.

After any ECO command, the tool performs either an incremental or full timing update when you run the next command that reports timing, for example, `report_timing`.

Note: The tool supports this command for flattened designs only.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_net` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_net`
`-module <string>`	Adds the new net to the specified verilog module. If the module name contains special characters, you need to start the module name with a backlash '\' and end it with a space.
`-names <list_of_nets>`	Specifies the list of nets to be added. If the net name contains special characters, you need to start the part of the net name, which contains special characters, with a backlash '\' and end it with a space. **Note:** You must specify *`-names`* before you specify other parameters. The software issues an error message if you do not specify the *`-names`* first.

Examples

- The following command adds nets `n1`, `n2`, and `n3`:

  ```
  tempus> create_net -names "n1 n2 n3"
  ```

- The following command adds net with base name `ant_net[1]` to `module H[1]`:

```
tempus> create_net -names {\ant_net[1] } -module {\H[1] }
```

Related Commands

- `connect_net`
- `delete_inst`
- `delete_nets`

delete_nets

```
delete_nets
[-help]
[-all <string>]
-module <string>
net
```

Logically deletes nets from a design. If a net is routed, wire segments associated with the net are also deleted.

You can use wildcards (`*?`) to specify the nets you want Tempus to delete.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `delete_nets` parameter. For a detailed description of the command and all of its parameters, use the man command: `man delete_nets`
`-all <string>`	Deletes all nets from a design.
`-module <string>`	Deletes the net from the specified verilog module. If the module name contains special characters, you need to start the module name with a backlash '\' and end it with a space.
net	Specifies the list of nets to be deleted.

Example

- The following command deletes net1:

  ```
  tempus> delete_nets net1
  ```

- The following command deletes all nets whose name begin with net1:

  ```
  tempus> delete_nets net1*
  ```

- The following command deletes all nets from the design:

  ```
  tempus> delete_nets -all
  ```

- The following command deletes the net with base name `ant_net[1]` from `module H[1]`:

```
tempus> delete_nets {\ant_net[1] } -module {\H[1] }
```

Related Commands

- `connect_net`
- `delete_inst`

delete_routes

```
delete_routes
[-help]
-net net*
```

Deletes physical wires, any vias connected to the wires, and violation markers associated with the wires.

Note: The `delete_routes` command deletes only those vias that are connected to wires.

Parameters

`-help`	Prints a brief description that includes type and default information for each `delete_routes` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man delete_routes`
`-net net*`	Specifies the net or list of nets from which to delete wires. You can use wildcard values in this field. For example, if you specify `V*`, deletes wires from nets `VDD` and `VSS`. **Note**: The wires and vias that are shielding the nets specified with `delete_routes -net` are not deleted. *Data_type*: (net)*, mandatory

Examples

- The following command deletes the wires and vias belonging to the net `carry4`:

  ```
  delete_routes -net carry4
  ```

Related Command

- `delete_nets`

disconnect_net

```
disconnect_net
[-help]
-net <net_name>
{-pins <list_of_pins> | -all}
```

Disconnects a net and connects it to new pins or ports. The syntax is not order-dependent.

After any ECO command, the tool can performs either an incremental or full timing update when you run the next command that reports timing, for example, `report_timing`.

Note: The tool supports this command for flattened designs only.

Parameters

`-all`	Detaches all terminals connected to the net. You cannot specify a pin/port list if you specify this parameter.
`-net`	Specifies the net to connect.
`-pins`	Specifies the pins/ports from which the specified net is to be detached.

Examples

- The following command detaches `net26` from pins `i1/A`, `i2/B`, and `i2/Z`:

  ```
  tempus> disconnect_net -net net26 -pins "i1/A i2/B i3/Z"
  ```

- The following command is equivalent of above example.

  ```
  tempus> disconnect_net -net [get_nets net26] -pins [get_cells "i1/A i2/B i3/Z"]
  ```

Related Commands

- `connect_net`
- `delete_inst`

eco_add_repeater

```
eco_add_repeater
[-help]
[-name <string>]
[-new_net_name <string>]
[-relative_distance_to_sink <float>]
[-spread_count <integer>]
[-spread_distance <float>]
[-spread_prefix <string>]
{-net <string> | -pins <string>} [-location {{x1 y1} {x2 y2}...} [-buffer_orient
<string>]] [-sort_by {slack trans area} [-max_cells <integer>]] [-cells <string> [-
evaluate_only ]] [-evaluate_only | -evaluate_all ] [{[-evaluate_all ] [-evaluate_only
]} [{[-view <string>] [-early ] [-late ] [-output_as_tcl ] [-sort_by {slack trans
area}]}]] [ -early | -late ][ -fall | -rise ] [-first_spread_distance <double>]
```

Adds either a single buffer or two inverters to a net. In multi-mode multi-corner (MMMC) mode, this command adds a repeater and updates timing in all views. Optionally, you can specify the location of the buffer or inverters relative to the sink pin.

Note:

Evaluation is not yet supported for the `eco_add_repeater` command.

You do not need to provide physical information (LEF/DEF) in order to perform ECO with this command.

Parameters

`-buffer_orient string`	Specifies the legalized orientation for the repeater instance. **Note**: You can specify this parameter with the `-location` parameter only.
`-cells string`	Specifies the library cell to be used as repeater. If the cell is an inverter, two inverters in series will be added.

`-early`	Specifies that timing will be evaluated for early analysis. In such a case, only the active hold analysis views will be considered. If this parameter is not specified, timing will be evaluated for late analysis. **Note**: This parameter must be specified with `-evaluate_only` or `-evaluate_all` only.
`-evaluate_all`	Specifies to only evaluate timing for all buffer/inverter/delay cells with matching footprint (or all footprints) without any changes in the netlist.
`-evaluate_only`	Specifies to only evaluate timing for the given buffer/inverter/delay cells without any changes in the netlist.
`-fall`	Specifies that the timing will be evaluated for fall transition. If this parameter is not specified, timing will be evaluated for both rise and fall transitions and the worst result is reported. **Note**: This parameter must be specified with `-evaluate_only` or `-evaluate_all` only.
`-first_spread_distance` *double*	Specifies the distance from driver after which the first repeater is to be added. **Note**: This parameter must be specified with `-spread_distance`.
`-late`	Specifies that timing will be evaluated for late analysis. In such a case, only the active setup analysis views will be considered. If this parameter is not specified, timing will still be evaluated for late analysis. **Note**: This parameter must be specified with `-evaluate_only or -evaluate_all` only.
`-location {`*x y* `|` *x1 y1 x2 y2*`}`	Specifies the x and y coordinates for the location of the buffer or inverter pair. If you do not specify this parameter, the tool determines the location. If the cell is a buffer and the location is specified, the newly created buffer is placed at the specified location. If the cell is an inverter and only one location is specified, both inverters will be placed at the same location.

`-max_cells <integer>`	Specifies the maximum number of improving and degrading cells (each) to be reported for evaluation.
`-name` *`inst_name`*	Specifies the base names of the new instances to be added.
`-net` *`net_name`*	Specifies the name of net on which the buffer will be added. All sinks of the net are buffered when this parameter is specified.
`-new_net_name netname`	Specifies the base name(s) for the new net(s) to be added.
`-output_as_tcl`	Displays the output as a TCL list instead of displaying as a text.
`-pins` *`pin_or_ports`*	Specifies a list of terminals with one common net.
`-relative_distance_to_sink` *`value`*	Specifies the location of the buffer instance based on its (relative) distance from the sink pin. The value is a number between 0 and 1 (both inclusive). A low value (such as 0.1) places the buffer near the sink. A high value (such as 0.9) places the buffer near the driver. You can use the value 0 to do sink term buffering and value 1 to do driver term buffering. The fraction is based on the length of the wire. Fractional values can only be specified for two-pin nets.
`-rise`	Specifies that the timing will be evaluated for rise transition. If this parameter is not specified, timing will be evaluated for both rise and fall transitions and the worst result is reported. **Note**: This parameter must be specified with `-evaluate_only` or `-evaluate_all` only.
`-spread_count` *`integer`*	Specifies the number of repeaters to be added for buffering. *Minimum*: 1 *Maximum*: INT_MAX
`-spread_distance` *`float`*	Specifies the distance (starting from driver) after which repeaters are to be added. *Minimum*: 2.2250738585072e-308 *Maximum*: 1073741.823
`-spread_prefix string`	Specifies the prefix used for new instances and nets added during spread buffering.

`-sort_by {slack trans area}`	Specifies the sorting order for evaluation.
`-view <string>`	Specifies the active analysis views for which timing must be evaluated. If this parameter is not specified, timing will be evaluated for all active analysis views and results reported for the worst view. **Note**: This parameter must be specified with `-evaluate_only or -evaluate_all only`.

Examples

- The following examples are the two methods to add two back-to-back inverters:
  1. The following example adds two instances of INV0 inverter cell:
     ```
     set eco_check_logical_equivalence true ; # default.
     eco_add_repeater -cells {INV0} -name {{inst_0 inst_1}} ...
     ```
  2. The following example adds two instances, one each of INV0 and INV1 inverter cells: -
     ```
     set eco_check_logical_equivalence false ; # default: true.
     eco_add_repeater -cells {INV0 INV1} -name {inst_0 inst_1} ...
     ```
     After the first (1) method, you can run the following command to change one of the two inverter instances to a different inverter cell: -
     ```
     eco_update_cell -cells {INV1} -insts {<full hierarchical name of inst_1>}
     ```
- The following command inserts a repeater with cell `BUFX4` on the net connected to terminal `DTMF_INST/RESULT_CONV_INST/RE_PHC369_r1336_0_/Y`:
  ```
  tempus> eco_add_repeater -term DTMF_INST/RESULT_CONV_INST/RE_PHC369_r1336_0_/Y -cell BUFX4
  ```
- The following command inserts a repeater with cell `BUFX4` and attaches it to (driver) terminal `DTMF_INST/RESULT_CONV_INST/RE_PHC369_r1336_0_/Y`:
  ```
  tempus> eco_add_repeater -term DTMF_INST/RESULT_CONV_INST/RE_PHC369_r1336_0_/Y -cell BUFX4 -relativeDistToSink 1
  ```
- The following command inserts a repeater with cell `BUFX4` and attaches it to (sink) terminal `DTMF_INST/RESULT_CONV_INST/RE_PHC369_r1336_0_/A`:
  ```
  tempus> eco_add_repeater -term DTMF_INST/RESULT_CONV_INST/RE_PHC369_r1336_0_/A -
  ```

```
cell BUFX4 -relativeDistToSink 0
```

- The following command inserts a repeater with cell `BUFX4` on net n123, places it at the given location, and names the new repeater instance `ECO_inst`:

```
tempus> eco_add_repeater -net n123 -cell BUFX4 -loc {23.45 98.12} -name {ECO_inst}
```

- The following command inserts a repeater with cell `BUFX4` at a relative distance of 0.3 from the sink terminal of the two-pin net `n123`:

```
tempus> eco_add_repeater -net n123 -cell BUFX4 -relativeDistToSink 0.3
```

- The following command reports the evaluation results (worst setup timing across all views) of buffering pin, `MPY_32_INST/M16X16_INST/i_910/B0`, by adding each buffer/inverter/delay cell in the library that matches the footprint of cell, `BUFX4`:

```
tempus> eco_add_repeater -pins MPY_32_INST/M16X16_INST/i_910/B0 -cells BUFX4 -
evaluate_all

------------------------------------------------------------------------------
-------------------------------------------------------------

Library-cell       Area        Timing-slack    Worst-view         Transition
 Trans-slack     Worst-Trans-Sink

------------------------------------------------------------------------------
-------------------------------------------------------------

Reference          0.0000      -0.1242         core+typ-rcMax     rise
 2.8485          MPY_32_INST/M16X16_INST/i_910/B0

CLKBUFXL(buf)      13.3056     -0.1785         core+typ-rcMax     rise
 2.9125          MPY_32_INST/M16X16_INST/i_910/B0

CLKBUFX8(buf)      23.2848     -0.2197         core+typ-rcMax     rise
 2.9580          MPY_32_INST/M16X16_INST/i_910/B0

CLKBUFX4(buf)      16.6320     -0.1928         core+typ-rcMax     rise
 2.9497          MPY_32_INST/M16X16_INST/i_910/B0

CLKBUFX3(buf)      13.3056     -0.1892         core+typ-rcMax     rise
 2.9453          MPY_32_INST/M16X16_INST/i_910/B0

CLKBUFX2(buf)      13.3056     -0.1854         core+typ-rcMax     rise
 2.9311          MPY_32_INST/M16X16_INST/i_910/B0

CLKBUFX20(buf)     79.8336     -0.4212         core+typ-rcMax     rise
 2.9477          MPY_32_INST/M16X16_INST/i_910/B0

CLKBUFX1(buf)      13.3056     -0.2002         core+typ-rcMax     rise
```

```
 2.9081          MPY_32_INST/M16X16_INST/i_910/B0

CLKBUFX16(buf)     63.2016     -0.3583         core+typ-rcMax     rise
 2.9509          MPY_32_INST/M16X16_INST/i_910/B0

CLKBUFX12(buf)     53.2224     -0.3147         core+typ-rcMax     rise
 2.9542          MPY_32_INST/M16X16_INST/i_910/B0

BUFXL(buf)         13.3056     -0.2028         core+typ-rcMax     rise
 2.8799          MPY_32_INST/M16X16_INST/i_910/B0

BUFX8(buf)         29.9376     -0.2222         core+typ-rcMax     rise
 2.9595          MPY_32_INST/M16X16_INST/i_910/B0

BUFX4(buf)         16.6320     -0.1836         core+typ-rcMax     rise
 2.9545          MPY_32_INST/M16X16_INST/i_910/B0

BUFX3(buf)         13.3056     -0.1805         core+typ-rcMax     rise
 2.9502          MPY_32_INST/M16X16_INST/i_910/B0

BUFX2(buf)         13.3056     -0.1818         core+typ-rcMax     rise
 2.9358          MPY_32_INST/M16X16_INST/i_910/B0

--------------------------------------------------------------------------------
--------------------------------------------------------------
```

- The following command reports the evaluation results (worst setup timing across all views) of buffering pin, `MPY_32_INST/M16X16_INST/i_910/B0`, by adding cell, `BUFX4`:

```
tempus> eco_add_repeater -pins MPY_32_INST/M16X16_INST/i_669/A -cells BUFX4 -
evaluate_only

--------------------------------------------------------------------------------
--------------------------------------------------------------

Library-cell       Area      Timing-slack     Worst-view          Transition
Trans-slack    Worst-Trans-Sink

--------------------------------------------------------------------------------
--------------------------------------------------------------

Reference         0.0000     -0.1242          core+typ-rcMax      rise
2.6766         MPY_32_INST/M16X16_INST/i_639/A1

BUFX4(buf)        16.6320    -0.1816          core+typ-rcMax      rise
2.6555         MPY_32_INST/M16X16_INST/i_639/A1
```

- The following command reports the evaluation results (worst setup timing across all views) of buffering net, `MPY_32_INST/M16X16_INST/n_647`, by adding cell, `CLKBUFX20`:

```
tempus> eco_add_repeater -net MPY_32_INST/M16X16_INST/n_647 -cells CLKBUFX20 -
```

```
evaluate_only

-------------------------------------------------------------------------------
-------------------------------------------------------------

Library-cell         Area         Timing-slack     Worst-view
 Transition    Trans-slack    Worst-Trans-Sink

-------------------------------------------------------------------------------
-------------------------------------------------------------

Reference            0.0000       -0.0745          core+typ-rcMax       rise
    2.8849         MPY_32_INST/M16X16_INST/i_388/A

CLKBUFX20(buf)       79.8336      -0.2270          core+typ-rcMax       rise
    2.9586         MPY_32_INST/M16X16_INST/i_388/A
```

- The following command reports the evaluation results (worst setup timing across all views) of buffering pin, `MPY_32_INST/M16X16_INST/i_669/A`, by adding cell, `BUFX4`:

```
tempus> eco_add_repeater -pins MPY_32_INST/M16X16_INST/i_669/A -cells BUFX4 -
evaluate_only -output_as_tcl

{MPY_32_INST/M16X16_INST/n_917 {Reference ref 0.0000 -0.1242 core+typ-rcMax rise
2.6766 MPY_32_INST/M16X16_INST/i_639/A1} {BUFX4 buf 16.6320 -0.1816 core+typ-rcMax
rise 2.6555 MPY_32_INST/M16X16_INST/i_639/A1}}
```

- The following command reports the evaluation results (hold timing analysis for only `core+typ-rcMin` view) of buffering pin, `MPY_32_INST/M16X16_INST/i_699/A1`, by adding cell, `BUFX16`:

```
tempus> eco_add_repeater -pins MPY_32_INST/M16X16_INST/i_699/A1 -cells BUFX16 -
evaluate_only -view core+typ-rcMin -early

-------------------------------------------------------------------------------
-------------------------------------------------------------

Library-cell      Area        Timing-slack    Worst-view           Transition
 Trans-slack      Worst-Trans-Sink

-------------------------------------------------------------------------------
-------------------------------------------------------------

Reference         0.0000      2.1814          core+typ-rcMin       rise
 2.6919           MPY_32_INST/M16X16_INST/i_699/A1

BUFX16(buf)       43.2432     2.3008          core+typ-rcMin       fall
 2.4436           MPY_32_INST/M16X16_INST/i_731/B
```

- The following commands add one instance of inverter `INVX1` after every 50 um distance,

starting from the net's driver (polarity of some or all sinks may get reversed when odd number of instances are added):

```
set_db eco_check_logical_equivalence false
eco_add_repeater -net n123 -cells INVX1 -spread_distance 50
```

- The following commands add one instance of inverter `INVX1` after every 50 um distance, starting from the net's driver, and if required, one instance of this inverter at a sink to preserve its polarity:

  ```
  set_db eco_check_logical_equivalence true
  eco_add_repeater -net n123 -cells INVX1 -spread_distance 50
  ```

- The following command adds 10 instances of buffer `BUFX1`, equi-distant from each other, such that only the specified sinks are buffered:

  ```
  eco_add_repeater -pins {sink1 sink3} -cells BUFX1 -spread_count 10
  ```

- The following command adds one instance each of buffers `BUFX1`, `BUFX2`, `BUFX3`, and `BUFX4`, equi-distant from each other, such that all the sinks of this net are buffered [total 4 instances added]:

  ```
  eco_add_repeater -net n123 -cells {BUFX1 BUFX2 BUFX3 BUFX4} -spread_count 4
  ```

- The following commands add two instances each of inverters `INVX1` and `INVX3`, and one instance each of buffers `BUFX2` and `BUFX4`, each buffer or inverter equi-distant from each other, such that only the specified sinks are buffered [total 6 instances added]:

  ```
  set_db eco_check_logical_equivalence true
  eco_add_repeater -pins {sink1 sink3} -cells {INVX1 BUFX2 INVX3 BUFX4} -
  spread_count 4
  ```

- The following command adds one instance of buffer BUFX1 after every 50 um distance starting from the net's driver:

  ```
  eco_add_repeater -net n123 -cells BUFX1 -spread_distance 50 -spread_prefix {ab}
  ```

  The added instances are named as `ab_inst_0`, `ab_inst_1`, … and the added nets are named as `ab_net_0`, `ab_net_1`, …

- The following command adds four instances of buffer BUFX1, equidistant from each other so that all sinks of the net are buffered:

  ```
  eco_add_repeater -net n123 -cells BUFX1 -spread_count 4 -spread_prefix {mn}
  ```

  The added instances are named: `mn_inst_0`, `mn_inst_1`, … and the added nets are named as `mn_net_0`, `mn_net_1`, …

  Note: The suffix number automatically starts after the last buffer or inverter added earlier.

- The following command adds the first buffer (`BUFFD4BWP`) after 20 micron distance from the driver term, and subsequent buffers after 50 micron distance from the previous buffer.

  ```
  eco_add_repeater -net iso -cell BUFFD4BWP -spread_distance 50 -
  ```

```
first_spread_distance 20
```

Related Commands

- eco_update_cell
- eco_delete_repeater
- set_db

eco_delete_repeater

```
eco_delete_repeater
[-help] {-insts <string> | -inverter_pair <string>} [-early  | -late ] [[-evaluate_only
] [{[-view <string>] [-early ] [-late ] [-fall | -rise ] [-output_as_tcl ]}]]
```

Deletes a specified buffer or pair of inverters connected back-to-back. In multi-mode multi-corner (MMMC) mode, this command deletes a repeater and updates timing in all views. The command searches backward, then forward, for another inverter pair to delete. This command merges parasitics by default. You might see an unconnected wire after committing the change; however, there is no impact on timing results due to such unconnected wires.

Note: Evaluation is not yet supported for the `eco_delete_repeater` command. You do not need to provide physical information (LEF/DEF) in order to perform ECO with this command.

Parameters

`-early`	Specifies that timing will be evaluated for early analysis. In such a case, only the active hold analysis views will be considered. If this parameter is not specified, timing will be evaluated for late analysis. **Note**: This parameter must be specified with `-evaluate_only` or `-evaluate_all only`.
`-evaluate_only`	Specifies to only evaluate timing without any changes in netlist.
`-fall`	Specifies that the timing will be evaluated for fall transition. If this parameter is not specified, timing will be evaluated for both rise and fall transitions and the worst result is reported. **Note**: This parameter must be specified with `-evaluate_only` only.
`-insts <string>`	Specifies a buffer or inverter (pair) for deletion. Two inverters are deleted only if they are tied together and if both of them can be deleted. To delete an inverter pair, specify only one of the two inverters.
`-inverter_pair <string>`	Specifies a pair of inverters to be deleted.

`-late`	Specifies that timing will be evaluated for late analysis. In such a case, only the active setup analysis views will be considered. If this parameter is not specified, timing will still be evaluated for late analysis. **Note**: This parameter must be specified with `-evaluate_only` or `-evaluate_all only`.
`-output_as_tcl`	Displays the output as a TCL list instead of displaying as text.
`-rise`	Specifies that the timing will be evaluated for rise transition. If this parameter is not specified, timing will be evaluated for both rise and fall transitions and the worst result is reported. **Note**: This parameter must be specified with `-evaluate_only` only.
`-view <string>`	Specifies the active analysis views for which timing must be evaluated. If this parameter is not specified, timing will be evaluated for all active analysis views and results reported for the worst view. **Note**: This parameter must be specified with `-evaluate_only` or `-evaluate_all only`.

Examples

- Consider three inverters connected in series: `instA` -> `inv1` -> `inv2` -> `inv3` -> `instB` (where instances `instA` and `instB` are not inverters).
  - The following command deletes inverters `inv1` and `inv2`: -
    ```
    tempus> eco_delete_repeater -inst inv1
    ```
  - The following command deletes inverters `inv2` and `inv3`: -
    ```
    tempus> eco_delete_repeater -inst inv3
    ```
  - If `inv1` can be deleted, the following command deletes inverters `inv1` and `inv2` (even if `inv3` can also be deleted): -
    ```
    tempus> eco_delete_repeater -inst inv2
    ```
  - If `inv1` cannot be deleted but `inv3` can be deleted, the following command deletes inverters `inv2` and `inv3`: -

```
tempus> eco_delete_repeater -inst inv2
```

- The following command deletes instances `DTMF_INST/RESULTS_CONV_INST/FE_PHC845_r1644_2_` and `DTMF_INST/RESULTS_CONV_INST/FE_PHC845_r1643_3_`:

```
tempus> eco_delete_repeater -insts {DTMF_INST/RESULTS_CONV_INST/FE_PHC845_r1644_2_
DTMF_INST/RESULTS_CONV_INST/FE_PHC845_r1643_3_}
```

- The following command reports the evaluation results (worst setup timing across all views) of deleting buffer, `TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/FE_PSC514_n_1670`:

```
tempus> eco_delete_repeater -insts
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/FE_PSC514_n_1670 -evaluate_only
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
--

Type        Timing-slack   Worst-Slack-View             Worst-Slack-Dir  Trans
Trans-slack   Worst-Trans-View             Worst-Trans-Sink

--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
---

Reference    -30.8714      default_emulate_view           rise          0.3807
4.6193     default_emulate_view
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/Fp1697A/A

Target       -31.7678      default_emulate_view           rise          1.4085
3.5915     default_emulate_view
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/Fp1697A/A
```

- The following command reports the evaluation results (worst setup timing across all views) of deleting buffer, `TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/FE_PSC514_n_1670`:

```
tempus> eco_delete_repeater -insts
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/FE_PSC514_n_1670 -evaluate_only -
output_as_tcl

{TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/FE_PSC514_n_1670 buf {Reference -
30.8714 default_emulate_view rise 0.3807 4.6193 default_emulate_view
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/Fp1697A/A} {Target -31.7678
default_emulate_view rise 1.4085 3.5915 default_emulate_view
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/Fp1697A/A}}
```

- The following command reports the evaluation results (worst setup timing across all views) of deleting these two buffers separately (i.e. mutually exclusive):

```
tempus> eco_delete_repeater -insts
{TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/FE_PSC514_n_1670
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/FE_PSC518_n_1178} -evaluate_only -
output_as_tcl

{TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/FE_PSC514_n_1670 buf {Reference -
30.8714 default_emulate_view rise 0.3807 4.6193 default_emulate_view
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/Fp1697A/A} {Target -31.7678
default_emulate_view rise 1.4085 3.5915 default_emulate_view
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/Fp1697A/A}}
{TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/FE_PSC518_n_1178 buf {Reference -
30.8714 default_emulate_view rise 0.1655 4.8345 default_emulate_view
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/Fp0671A/A} {Target -30.8888
default_emulate_view rise 1.3715 3.6285 default_emulate_view
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/Fp0671A/A}}
```

- The following command reports the evaluation results (hold timing analysis for only `default_emulate_view` view) of deleting buffer, `TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/FE_PSC514_n_1670`:

```
tempus> eco_delete_repeater -insts
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/FE_PSC514_n_1670 -evaluate_only -
view default_emulate_view -early

--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
---
Type        Timing-slack   Worst-Slack-View             Worst-Slack-Dir  Trans
 Trans-slack   Worst-Trans-View             Worst-Trans-Sink
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
---
Reference      7.5499      default_emulate_view           fall          0.3807
 4.6193     default_emulate_view
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/Fp1697A/A
Target         7.5579      default_emulate_view           fall          1.4081
 3.5919     default_emulate_view
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/Fp1697A/A
```

- The following command reports the evaluation results (worst setup timing across all views) of deleting buffer, `TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/FE_PSC514_n_1670`:

```
tempus> eco_delete_repeater -insts
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/FE_PSC514_n_1670 -evaluate_only -
view default_emulate_view -late
```

```
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
---
Type        Timing-slack   Worst-Slack-View            Worst-Slack-Dir  Trans
 Trans-slack   Worst-Trans-View            Worst-Trans-Sink
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
---
Reference    -30.8714      default_emulate_view           rise          0.3807
 4.6193     default_emulate_view
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/Fp1697A/A
Target       -31.7678      default_emulate_view           rise          1.4085
 3.5915     default_emulate_view
TDSP_CORE_INST/MPY_32_INST/M16X16_INST/mul_8_14/Fp1697A/A
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
---
```

Related Commands

- eco_update_cell
- eco_add_repeater
- set_db

eco_update_cell

```
eco_update_cell
[-help]
[-pin_maps <string>]
{-cells <string> | -up_size  | -down_size  | -evaluate_all } [-evaluate_only  | -
evaluate_all ] [-early  | -late ] [-location {x y} [-orient <string>]] [-sort_by {slack
delay trans area} [-max_cells <integer>]] {-insts <string> | -from <string> | -
from_rise <string> | -from_fall <string>} {-insts <string> | -to <string> | -to_rise
<string> | -to_fall <string>} [{[-from <string>] [-from_rise <string>] [-from_fall
<string>] [-to <string>] [-to_rise <string>] [-to_fall <string>]} [{[-timing_type
{combinational combinational_fall combinational_rise falling_edge hold_falling
hold_rising recovery_falling recovery_rising removal_falling removal_rising rising_edge
setup_falling setup_rising}] [-view <string>] [-early ] [-late ] [-output_as_tcl ] [-
sort_by {slack delay trans area}]}]] [{[-evaluate_all ] [-evaluate_only ]} [{[-from
<string>] [-from_rise <string>] [-from_fall <string>] [-to <string>] [-to_rise
<string>] [-to_fall <string>] [-timing_type {combinational combinational_fall
combinational_rise falling_edge hold_falling hold_rising recovery_falling
recovery_rising removal_falling removal_rising rising_edge setup_falling setup_rising}]
[-view <string>] [-early ] [-late ] [-output_as_tcl ]} [-sort_by {slack delay trans
area}]]]
```

Changes the cell of an instance. In multi-mode multi-corner (MMMC) mode, this command changes the cell and updates timing in all views. This command upsizes or downsizes the specified instance based on the available footprint. Cell mapping is also possible as long as the output pins are functionally equivalent. This command also checks for and observes power domains.

This command has the following features:

- Changes the cell
- Performs incremental timing updates of the paths passing through the cell
- Supports spef, LEF, and DEF-based flows

Note:

You do not need to provide physical information (LEF/DEF) in order to perform ECO with this command.

When resizing instances inside multiple-instantiated blocks, the resize would be applied to all instantiations of the block.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `eco_update_cell` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man eco_update_cell`
`-cells` *string*	Specifies name of the new cell of the given instance. The function of the original cell has to match the function of the new cell for all the outputs used by the old cell. The number of input pins have to be the same for both the original cell and the new cell.
`-down_size`	Downsizes the specified instance with the next available cell in the same footprint unless the specified cell is already of the lowest driving strength. Downsizing an instance on the noncritical path can reduce the loading of its driver on the critical path.
`-early`	Specifies that timing will be evaluated for early analysis. In such a case, only the active hold analysis views will be considered. If this parameter is not specified, timing will be evaluated for late analysis. **Note**: This parameter must be specified with `-evaluate_only`/`-evaluate_all` only.
`-evaluate_all`	Allows to evaluate new timing(s) through the instance by using all functionally equivalent library cell(s) for the instance.
`-evaluate_only`	Allows to evaluate new timing through the instance by using the specified library cell for the instance.
`-from` *string*	Specifies the input pin for the instance to be evaluated for resize. Both the rise and fall transitions are considered in timing report. **Note**: This parameter must be specified with `-evaluate_only`/`-evaluate_all` only.
`-from_fall` *string*	Specifies the input pin for the instance to be evaluated for resize. Only the fall transition is considered in timing report. **Note**: This parameter must be specified with `-evaluate_only`/`-evaluate_all` only.

`-from_rise` *string*	Specifies the input pin for the instance to be evaluated for resize. Only the rise transition is considered in timing report. **Note**: This parameter must be specified with `-evaluate_only`/`-evaluate_all` only.
`-insts` *string*	Specifies the name of the instance to be resized.
`-late`	Specifies that timing will be evaluated for late analysis. In such a case, only the active setup analysis views will be considered. If this parameter is not specified, timing will still be evaluated for late analysis. **Note**: This parameter must be specified with `-evaluate_only`/`-evaluate_all` only.
`-location {x y}`	Specifies the legalized location where the instance will be placed.
`-max_cells <integer>`	Specifies the maximum number of improving and degrading cells (each) to be reported for evaluation.
`-orient` *string*	Specifies the legalized orientation for the instance. You can specify this parameter with the `-location` parameter only.
`-output_as_tcl`	Returns the evaluation result as a Tcl list. By default, the command returns text-based format. **Note**: This parameter must be specified with `-evaluate_only`/`-evaluate_all` only.
`-pin_maps` *string*	Specifies the pin mapping for the new cell based on the old cell. This option is required if the new master cell has different pin names than the original cell. The net connected to *oldpin1* of the original cell should be connected to *newpin1* of the new cell, and so forth.
`-sort_by {slack delay trans area}`	Specifies the sorting order for evaluation output.

`-timing_type <timing_type>`	Specifies the type of the timing arc to be used for timing evaluation between the specified `-from/-from_rise/-from_fall` and `-to/-to_rise/_to_fall pins`. The software will check for valid timing_types: • If only one arc is available, it will proceed. • If multiple arcs available, you need to specify which timing_type to be considered via this parameter. **Note**: This parameter must be specified with `-evaluate_only/-evaluate_all` only. For valid timing types, refer to section, **List of Valid Timing-types.**
`-to` *string*	Specify one of the following: • The output pin of the instance to be evaluated for resize, or • The input pin for the instance driven by the instance to be evaluated for resize. **Note**: This parameter must be specified with `-evaluate_only`/`-evaluate_all` only.
`-to_fall` *string*	Specifies name of the target pin of the cell to be resized, or the input pin of the next-stage cell. Only the fall transition is considered in the timing report.
`-to_rise` *string*	Specifies name of the target pin of the cell to be resized, or the input pin of the next-stage cell. Only the rise transition is considered in the timing report.
`-up_size`	Upsizes the specified instance with the next available cell in the same footprint unless the specified cell is already of the highest driving strength. Upsizing an instance that drives a large load can improve the driver delay and the transition time at the receivers.
`-view` *string*	Specifies the active analysis views for which timing must be evaluated. If this parameter is not specified, timing will be evaluated for all active analysis views and results reported for the worst view. **Note**: This parameter must be specified with `-evaluate_only or -evaluate_all only`.

Examples

- The following command substitutes the cell for instance `DTMF_INST/RESULTS_CONV_INST/FE_PHC268_r1477_1_` with cell `DLY3X2`. Pin A maps to pin I and pin Y maps to pin O of the new cell.

```
tempus> eco_update_cell -insts DTMF_INST/RESULTS_CONV_INST/FE_PHC268_r1477_1_ -cells DLY3X2 -pin_maps {A I Y O}
```

- The following command substitutes the cell for instance `DTMF_INST/RESULTS_CONV_INST/FE_PHC268_r1477_1_` with cell `BUFX2`.

```
tempus> eco_update_cell -insts DTMF_INST/RESULTS_CONV_INST/FE_PHC268_r1477_1_ -cells BUFX2
```

- The following command upsizes the cell for instance `DTMF_INST/RESULTS_CONV_INST/FE_PHC268_r1477_1_` and places the instance at the given location.

```
tempus> eco_update_cell -insts DTMF_INST/RESULTS_CONV_INST/FE_PHC268_r1477_1_ -up_size -location {23.45 98.12}
```

- The following command evaluates timing for instance, `TDSP_CORE_INST/TDSP_CORE_GLUE_INST/lsh_120_47/FE_PSC502_n_661`, using library cell `BUF_X32` (prints delay for A to Y arc, slack at A pin, and slew at Y pin, corresponding to the worst timing path) :

```
tempus > eco_update_cell -cells BUF_X32 -from TDSP_CORE_INST/TDSP_CORE_GLUE_INST/lsh_120_47/FE_PSC502_n_661/A -to TDSP_CORE_INST/TDSP_CORE_GLUE_INST/lsh_120_47/FE_PSC502_n_661/Z -evaluate_only

From Pin: TDSP_CORE_INST/TDSP_CORE_GLUE_INST/lsh_120_47/FE_PSC502_n_661/A

To Pin:   TDSP_CORE_INST/TDSP_CORE_GLUE_INST/lsh_120_47/FE_PSC502_n_661/Z

------------------------------------------------------------------------------
------------------------------------------------------------------------------
---------------------------

Library-cell          From   To     Arc-type       Delay     Timing-slack  Trans
   Trans-slack     Area      Worst-Slack-View       Worst-Trans-View

------------------------------------------------------------------------------
------------------------------------------------------------------------------
---------------------------

*BUF_X32              fall   fall  combinational   0.3410    -30.8714      0.0997
  4.9003          2.6600    default_emulate_view   default_emulate_view
```

- The following command evaluates timing for instance,

`TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_`, using library cell `BUF_X16` for combinational timing arc and prints the delay, estimated slack and slew (for the worst timing path across all views)

```
tempus > eco_update_cell -cells BUF_X16 -from
TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/A -to
TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/Z -evaluate_only -timing_type
combinational

From Pin: TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/A
To Pin:   TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/Z
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
---------------------------
Library-cell                          From  To   Arc-type                    Delay
  Timing-slack  Trans  Trans-slack      Area       Worst-Slack-View       Worst-
Trans-View
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
---------------------------
*BUF_X16                              rise rise  combinational               1.4778
   -30.8714    2.5867   2.4133          1.5960    default_emulate_view
default_emulate_view
```

- The following command evaluates timing for instance, `TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_`, using library cell `BUF_X16` for `default_emulate_view` and prints the delay, estimated slack and slew (for the worst slack timing path corresponding to the specified fall/rise transitions) `-from_fall` `TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/A -to_rise` `TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/Z:`

```
tempus > eco_update_cell -cells BUF_X16 -from
TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/A -to
TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/Z -evaluate_only -view
default_emulate_view

From Pin: TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/A

To Pin:   TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/Z

--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
---------------------------

Library-cell                          From  To   Arc-type
```

```
Delay   Timing-slack  Trans    Trans-slack      Area     Worst-Slack-View
Worst-Trans-View

-------------------------------------------------------------------------------
-------------------------------------------------------------------------------
---------------------------

*BUF_X16                              rise rise  combinational
1.4778    -30.8714    2.5867   2.4133          1.5960    default_emulate_view
 default_emulate_view
```

- The following command evaluates timing for instance, `TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_`, using library cell, `BUF_X16`:

```
tempus > eco_update_cell -cells BUF_X16 -from
TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/A -to
TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/Z -evaluate_only -
output_as_tcl

TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/A
TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/Z {{*BUF_X16 rise rise
combinational 1.4778 -30.8714 2.5867 2.4133 1.5960 default_emulate_view
default_emulate_view}}
```

- The following command evaluates timing(s) using all functionally equivalent library cell(s) for instance, `TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_`, and prints the delay, estimated slack and slew (for worst timing path for each individual cell):

```
tempus > eco_update_cell -from
TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/A -to
TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_addre s_2_/Z -evaluate_all

From Pin: TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/A

To Pin:   TDSP_CORE_INST/DECODE_INST/FE_OFC193_port_address_2_/Z

-------------------------------------------------------------------------------
-------------------------------------------------------------------------------
---------------------------

Library-cell              From  To   Arc-type                    Delay     Timing-
slack  Trans     Trans-slack     Area       Worst-Slack-View         Worst-Trans-
View

-------------------------------------------------------------------------------
-------------------------------------------------------------------------------
```

```
---------------------------

*BUF_X16                  rise rise  combinational              1.4778    -
30.8714      2.5867    2.4133          1.5960     default_emulate_view
 default_emulate_view

BUF_X8                    rise rise  combinational              2.8729    -
33.9254      5.0978    -0.0978         1.0640     default_emulate_view
 default_emulate_view

BUF_X4                    rise rise  combinational              5.5002    -
40.0555      10.1001   -5.1001         0.7980     default_emulate_view
 default_emulate_view

BUF_X32                   fall fall  combinational              0.5854    -
30.4140      0.3648    4.6352          2.6600     default_emulate_view
 default_emulate_view

BUF_X2                    rise rise  combinational              5.5351    -
40.0973      10.0987   -5.0987         0.7980     default_emulate_view
 default_emulate_view

BUF_X1                    rise rise  combinational              5.5560    -
40.1181      10.0877   -5.0877         0.7980     default_emulate_view
 default_emulate_view

-------------------------------------------------------------------------------
-------------------------------------------------------------------------------
---------------------------
```

List of Valid Timing-types

- `combinational`
- `combinational_fall`
- `combinational_rise`
- `falling_edge`
- `hold_falling`
- `hold_rising`
- `recovery_falling`
- `recovery_rising`
- `removal_falling`
- `removal_rising`
- `rising_edge`
- `setup_falling`
- `setup_rising`

Related Commands

- `eco_add_repeater`
- `eco_delete_repeater`
- `set_db`

eco Category Attributes

The root attributes in the eco category are:

- eco_add_ports
- eco_batch_mode
- eco_check_logical_equivalence
- eco_client_tempus_executable
- eco_def_file_list
- eco_delay_calculation_effort
- eco_dynamic_view
- eco_drv_view_list
- eco_hold_view_list
- eco_honor_dont_touch
- eco_honor_dont_use
- eco_honor_fixed_status
- eco_honor_fixed_wires
- eco_honor_power_intent
- eco_leakage_view
- eco_lef_file_list
- eco_prefix
- eco_preserve_hpin_function
- eco_refine_place
- eco_select_partition_list
- eco_setup_view_list
- eco_si_effort
- eco_update_only_layers
- eco_update_timing

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `eco` category, use the following command:

```
get_db -category eco
```

eco_add_ports

`eco_add_ports {true|false}`

Default: `true`

Data_type: Boolean

Determines whether to add hierarchical module ports during buffering.

Note: This attribute works only for the Manual ECO commands.

eco_batch_mode

`eco_batch_mode {true|false}`

Default: `false`

Data_type: Boolean

Speeds up the runtime in case the user performs lots of interactive ECO operations. When the user specifies `eco_batch_mode true`, various ECO operations are performed in a batch and initialization steps are not performed for every ECO call. In addition, updating timing is performed only once for all the ECO operations. Tool will only perform timing initialization at the first ECO command in the batch mode and timing update at the time of exiting the batchMode. User needs to specify`batchMode false` to exit the `batchMode`.

The batch mode is activated by `eco_batch_mode true` and exited by `eco_batch_mode false`, and only `eco_add_repeater`, `eco_delete_repeater` and `eco_update_cell` could be given in between eco_batch_mode true and `eco_batch_mode false`:

```
eco_batch_mode true
eco commands (eco_add_repeater, eco_delete_repeater, eco_update_cell)

eco_batch_mode false
```

If a user activates the batch mode, then the user needs to remember to exit it. If the user does not exit the batch mode, the user might see some unexpected results. For example, if the tool automatically added cell padding for blocks when `place_detail` is called by ECO commands for some reason, the cell padding will not be removed and the block size will change unless the user exits the batch mode.

All other settings should be made prior to activating the batch mode. If you would like to change settings, you need to exit the batch mode first.

Note: This option supports all commit ECO commands. Evaluation commands are not supported.

Related Commands

- `place_detail`

eco_check_logical_equivalence

```
eco_check_logical_equivalence {true|false}
```

Default: `true`

Data_type: Boolean

Enables functionality checking when swapping cells. The software can swap cells that are logically equivalent.

When `-eco_check_logical_equivalence` is `true`:

- `eco_add_repeater` adds inverter pair if the specified cell is inverter.
- `eco_delete_repeater` deletes inverter pair if the specified instance is inverter.
- `eco_update_cell` changes cell to another cell type with the same function as long as they have same number of pins.

When `-eco_check_logical_equivalence` is set to `false`:

- `eco_add_repeater` can add only one single inverter if the specified cell is inverter.
- `eco_delete_repeater` can delete only one single inverter if the specified instance is inverter.
- `eco_update_cell` can perform nonequivalent cell change as long as they have same number of pins.

Related Commands

- `eco_add_repeater`
- `eco_delete_repeater`
- `eco_update_cell`

eco_client_tempus_executable

`eco_client_tempus_executable` *`string`*

Default: " "

Data_type: String

Sets the executable name for Tempus clients in ECO mode.

eco_def_file_list

`eco_def_file_list` *`string`*

Default: " "

Data_type: String

Sets the DEF files for using with the Tempus logical flow.

eco_delay_calculation_effort

`eco_delay_calculation_effort {low | medium | high}`

Default: `high`

Data_type: String

Determines the delay calculation effort level as `low`, `medium`, or `high`.

eco_dynamic_view

`eco_dynamic_view` *`string`*

Default: " "

Data_type: String

Sets the dynamic power view name for the ECO master session.

Note: This global is used only in the Paradime ECO flow.

eco_drv_view_list

`eco_drv_view_list` *`string`*

Default: *" "*

Data_type: String

Specifies the list of DRV views for Tempus ECO run.

Note: This global is used only in the Paradime ECO flow. The views specified in `eco_drv_view_list` along with `eco_setup_view_list` will be considered as DRV views.

eco_hold_view_list

`eco_hold_view_list` *`string`*

Default: " "

Data_type: String

Sets the view names for Tempus ECO master.

eco_honor_dont_touch

`eco_honor_dont_touch` *`cellName`*

Default: `true`

Data_type: Boolean

Checks for the don't touch attribute on nets and instances. If you specify a terminal list, this parameter checks each terminal for the don't touch attribute. The `ecoAddRepeater` command honors the don't touch nets. The `eco_delete_repeater` and `eco_update_cell` commands honor don't touch instances.

Related Commands

- `eco_delete_repeater`
- `eco_update_cell`

eco_honor_dont_use

```
eco_honor_dont_use {true|false}
```

Default: `true`

Data_type: Boolean

Checks for the don't use attribute on cells. The `eco_add_repeater`, `eco_delete_repeater`, and `eco_update_cell` commands honor don't use cells.

Related Commands

- `eco_add_repeater`
- `eco_delete_repeater`
- `eco_update_cell`

eco_honor_fixed_status

```
eco_honor_fixed_status {true|false}
```

Default: `true`

Data_type: Boolean

Checks for fixed net wires.

It restricts the addition or deletion on fixed net so that the fixed status of the wires is not altered.

eco_honor_fixed_wires

```
eco_honor_fixed_wires {true|false}
```

Default: `true`

Data_type: Boolean

Checks for fixed net wires.

It restricts the addition or deletion on fixed net so that the fixed status of the wires is not altered.

When `-eco_honor_fixed_wires` is set to `false`, `eco_add_repeater` can insert a cell on the wires with the FIXED/COVER status and change the status of these wires from FIXED/COVER to ROUTED.

eco_honor_power_intent

```
eco_honor_power_intent {true|false}
```

Default: `true`

Data_type: Boolean

Performs Multi Supply Voltage Design (MSV) checks during ECO.

When this parameter is set to `true`:

- `eco_update_cell` does not allow:
  - Resizing a regular cell with an always-on cell (and vice-versa).
  - Resizing with cells belonging to different power domains.
- `eco_add_repeater` does not allow:
  - Adding buffer to a power domain it doesn't belong to.
  - Buffering a cross power domain net if it can cause MSV violations.
  - Adding a regular buffer to a net that requires an always-on buffer.
  - Adding an always-on buffer to a net that does not require an always-on buffer.
- `eco_delete_repeater` does not allow deletion of a buffer or an inverter-pair that will cause:
  - A redundant isolation cell.
  - A CPF-constrained net to be deleted.

eco_leakage_view

```
eco_leakage_view string
```

Default: " "

Data_type: String

Sets the leakage power view name for the ECO master session.

Note: This global variable is used only in the Paradime ECO flow.

eco_lef_file_list

`eco_lef_file_list` *`string`*

Default: " "

Data_type: String

Sets the LEF files for using with Tempus logical flow.

eco_prefix

`eco_prefix` *`prefix`*

Default: `ECO`

Data_type: String

Adds a prefix to the inserted cells.

eco_preserve_hpin_function

`eco_preserve_hpin_function` *`prefix`*

Default: `false`

Data_type: Boolean

Determines whether to preserve logical functions at the hierarchical module ports.

Note: This option works only for manual ECO commands.

eco_refine_place

`eco_refine_place {true|false}`

Default: `true`

Data_type: Boolean

Legalizes placement whenever `eco_add_repeater` or `eco_update_cell` are used. If `false`, you must

run `place_detail` after ECO if you want to legalize placement.

Note: Specifying `eco_refine_place true` can affect performance when large numbers of cells are added or modified.

eco_select_partition_list

`eco_select_partition_list` *`string`*

Default: " "

Data_type: String

Lists the subset of partitions for which the physical data has to be loaded while merging the DEF files although user might point to all available DEF files using the `eco_def_file_list` global variable.

Note: This global variable is used only in the Paradime ECO flow.

eco_setup_view_list

`eco_setup_view_list` *`string`*

Default: " "

Data_type: String

Sets the view names for Tempus ECO master.

eco_si_effort

`eco_si_effort {low|medium}`

Default: `low`

Data_type: String

Sets the SI effort level as `low` or `medium` used in the timing update.

eco_update_only_layers

`eco_update_only_layers` *`string`*

Default: " "

Data_type: String

Specifies the layers on which the wires can be cut during buffering.

eco_update_timing

```
eco_update_timing {true|false}
```

Default: `true`

Data_type: Boolean

Controls whether to `eco*` commands recompute timing. If `true`, the eco* commands recompute timing. If `false`, the commands do not recompute timing, and us can later use `report_timing` to recompute timing.

Related Commands

- eco_add_repeater
- eco_delete_repeater
- eco_update_cell

read_eco

```
read_eco
[-help]
eco_files
```

Reads a file that was generated in db format using the `write_eco -format db` command.

Parameters

`eco_files`	Specifies the name of the file that contains the ECO directives. You can specify more than one file. File names must be separated by one space.

Examples

The following example reads `myfile` generated by the `write_eco` command.

```
tempus> read_eco myfile
```

Related Commands

- write_eco

write_eco

```
write_eco
[-help]
-format {innovus tempus}
-hierarchical
out_file
-prefix <prefix>
[-skip_non_eco_commands]
```

Logs ECO commands into an output file per file format. You can later load the files in the following ways:

- Source the innovus format file in the Innovus tool
- Source the `tempus` format file into the Tempus tool

The following commands can be appended to the report file:

- `create_net`
- `delete_nets`
- `create_inst`
- `delete_inst`
- `connect_net`
- `disconnect_net`
- `eco_update_cell`
- `eco_add_repeater`
- `eco_delete_repeater`

Parameters

`-format {innovus tempus}`	Generates an ECO file in the specified format.
`-hierarchical`	Generate an ECO file for optimized partitions.
out_file	Specifies the name of the file generated using this command.
`-prefix <prefix>`	Specifies the prefix to be used in naming the output ECO file.
`-skip_non_eco_commands`	Removes all non-ECO commands, which are not specified by the user from the automatically generated ECO files.

Examples

- The following command logs ECO commands into file `Tempus.eco`, in the `tempus` format:
  ```
  write_eco -format tempus Tempus.eco
  ```
- The following command logs ECO command into file `innovus.eco`, in the `innovus` format:
  ```
  write_eco -format innovus innovus.eco
  ```
- The following command removes all non-ECO commands from the automatically generated ECO file, `opt_eco_tempus_cui.tcl`:
  ```
  write_eco opt_eco_tempus_cui.tcl -format tempus -skip_non_eco_commands
  ```

Related Commands

- read_eco

16

Multiple-CPU Processing Commands

- check_multi_cpu_usage
- get_distributed_hosts
- get_multi_cpu_usage
- monitor_distributed_hosts
- report_resource
- set_distributed_hosts
- set_multi_cpu_usage

check_multi_cpu_usage

```
check_multi_cpu_usage
[-help]
```

Checks if distributed processing can work in the software environment. This command also ensures that the working directory is accessible from the master machine as well as the client machines. The `check_multi_cpu_usage` command is specified after configuring the multi-CPU settings (`set_distributed_hosts` and `set_multi_cpu_usage`) to check if all the specified CPUs can be accessed. This command should be invoked before running any commands that use multiple CPUs.

Parameters

`-help`	Prints out the command usage.

Examples

Here are some examples.

- The following example command script checks the distributed computing environment when `set_distributed_hosts -local` is specified:

```
tempus > check_multi_cpu_usage


/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413
Connected to sjfnl624 35779 0 ( PID=13350 )
Connected to sjfnl624 54715 1 ( PID=13356 )
Connected to sjfnl624 40469 3 ( PID=13397 )
Connected to sjfnl624 50442 2 ( PID=13378 )
Task 0 successfully accessed the current working directory.
Task 3 successfully accessed the current working directory.
Task 1 successfully accessed the current working directory.
```

```
Task 2 successfully accessed the current working directory.
Task 3: CPU(workload/CPU): 7.84/12 mem(free/total): 94118680k/98813572k.
Task 1: CPU(workload/CPU): 7.84/12 mem(free/total): 94118432k/98813572k.
Task 2: CPU(workload/CPU): 7.84/12 mem(free/total): 94117068k/98813572k.
Task 0: CPU(workload/CPU): 7.84/12 mem(free/total): 94119084k/98813572k.
Test heartbeat for task 0...
The distributed processing environment has been set up correctly.
```

- The following example command script checks the distributed computing environment when when `set_distributed_hosts -lsf -queue lnx64` is specified:

```
tempus > check_multi_cpu_usage

Job <5685326> is submitted to queue <lnx64>. The associated lsf task ID is '0'
Executing
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413/.clre_sub_22662_0
Job <5685328> is submitted to queue <lnx64>. The associated lsf task ID is '1'
Executing
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413/.clre_sub_22662_1
Job <5685329> is submitted to queue <lnx64>. The associated lsf task ID is '2'
Executing
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413/.clre_sub_22662_2
Job <5685330> is submitted to queue <lnx64>. The associated lsf task ID is '3'
Executing
/it/sjclapa06p6_scr02/clre/exp_2017_05_30/0223/dy/2014/2017/0413/.clre_sub_22662_3
Connected to sjfnl625 43039 1 ( PID=30833 )
Connected to sjfnl625 34830 2 ( PID=30832 )
Connected to sjfnl625 33887 3 ( PID=30831 )
Connected to sjfib046 60125 0 ( PID=32727 )
Task 1 successfully accessed the current working directory.
Task 3 successfully accessed the current working directory.
Task 2 successfully accessed the current working directory.
Task 0 successfully accessed the current working directory.
Task 3: CPU(workload/CPU): 1.43/12 mem(free/total): 74177208k/82277684k.
Task 2: CPU(workload/CPU): 1.43/12 mem(free/total): 74177208k/82277684k.
Task 1: CPU(workload/CPU): 1.43/12 mem(free/total): 74177208k/82277684k.
Task 0: CPU(workload/CPU): 19.79/20 mem(free/total): 352917068k/396858936k.
Test heartbeat for task 0...
The distributed processing environment has been set up correctly.
```

Related Commands

- set_multi_cpu_usage
- get_multi_cpu_usage

get_distributed_hosts

```
get_distributed_hosts
[-help]
[-args]
[-custom_script]
[-env_script]
[-hosts]
[-mode]
[-queue]
[-report_lsf_info <file>]
[-resource]
[-shell_timeout]
[-single_cpu_lsf_args]
[-timeout]
[-wait_for_lsf_info <integer>]
```

Displays settings for the `set_distributed_hosts` command.

Parameters

`-args`	Displays the additional arguments when the software is running in the Sun Grid Engine (SGE) or Load Sharing Facility (LSF) mode.
`-custom_script`	Displays the name of the custom script when the software is running in `custom` mode.
`-env_script`	Displays the value of the environment script for slave start up.
`-hosts`	Displays the list of host names when the software is running in `rsh` mode.
`-help`	Outputs the command usage and a brief description about the command parameters.
`-mode`	Displays the distributed processing mode. The possible values are `local`, `sge`, `rsh` , `ssh` , `lsf` and `custom`. *Default*: `local`

`-queue`	Displays the queue name when the software is running in `sge` or `lsf` mode.
`-report_lsf_info` *file*	Specifies to restore the LSF information to the specified file. This parameter can only be used when `setDistributeHost -report_lsf_info` is specified in the current session. You must specify the `get_distributed_hosts -report_lsf_info` only after the LSF tasks exit to obtain the task CPU time. If you specify this command before the task exits, the command gives a warning message.
`-resource`	Displays the resource string when the software is running in `sge` or `lsf` mode.
`-shell_timeout`	Displays the number of seconds the host machine waits for `rsh/ssh` machines to become available for multiple-CPU processing.
`-single_cpu_lsf_args`	Displays the lsf arguments when the software is running in `lsf` mode for a single CPU application.
`-timeout`	Displays the default timeout for a submitted job that will never start.
`-wait_for_lsf_info` *value*	Specifies the timeout period required to wait for task exit and generate the LSF information.

Examples

- The following command displays the current setting for the `-mode` parameter:

  ```
  get_distributed_hosts -mode
  local
  ```

- The following command saves the LSF information to the `lsf.log` file:

  ```
  get_distributed_hosts -report_lsf_info lsf.log
  ```

Related Commands

- set_distributed_hosts

get_multi_cpu_usage

```
get_multi_cpu_usage
[-help]
[-auto_page_fault_monitor]
[-cpu_per_remote_host]
[-keep_license]
[-local_cpu]
[-remote_host]
[-thread_info]
[-verbose]
```

Displays the requested number of threads or hosts.

To request threads or hosts, use `set_multi_cpu_usage`.

Parameters

`-help`	Prints out the command usage.
`-auto_page_fault_monitor`	Displays the warning rank set for performance issues caused by major page faults
`-cpu_per_remote_host`	Displays the number of CPUs available on every remote host for Superthreading.
`-help`	Outputs the command usage and a brief description about the command parameters.
`-keep_license`	Displays the current setting for the `set_multi_cpu_usage -keep_license` parameter.
`-local_cpu`	Displays the number of available local CPUs for multi-threading.
`-remote_host`	Displays the number of available remote hosts for distributed processing or Superthreading.
`-thread_info`	Displays the level of informational messages for each thread.
`-verbose`	Displays a message with the current multiple-CPU setting.

Examples

- The following command displays the requested number of threads for multi-threading:

  ```
  get_multi_cpu_usage -local_cpu
  ```

- The following command displays the requested number of remote hosts for Superthreading or distributed processing:

  ```
  get_multi_cpu_usage -remote_host
  ```

- The following command displays the requested number of CPUs per remote hosts for Superthreading:

  ```
  get_multi_cpu_usage -cpu_per_remote_host
  ```

Related Commands

- set_multi_cpu_usage

monitor_distributed_hosts

```
monitor_distributed_hosts
[-help]
[-log string]
[-overwrite]
[-period integer]
```

Monitors hosts and periodically dumps memory or CPU usage information for each host to a file. This command automatically determines the hosts to be monitored. By default, after every 60 seconds hosts are monitored and the disk information is written to a log file. This command is not required for DMMMC because it also performs client monitoring.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man monitor_distributed_hosts`.
`-log` *string*	Specifies the log file name. *Default*: host-monitor.log
`-overwrite`	Overwrites the log file, if it already exists.
`-period` *integer*	Specifies the sampling period after which hosts will be monitored. *Default*: 60

Examples

- A sample monitor log file is shown below:

```
Status keys: E excellent, G good, B bad, T terrible.
Memory values:
total = Real memory of host
progs = Real memory used by processes, including swappable cache
used = Real memory used by processes, excluding swappable cache
Started at: 17/04/25 04:24:22
lgi15(0) TMPDIR is : /tmp/tempus_temp_20836_lgi15_ark_lEAcvw

============ ======== ======================== ========================
Host         CPU      Memory (GB)              TMPDIR (GB)       (Mb/s)
name(id)     util %   total progs used %used   total   avail     rate
============ ======== ===== ===== ===== ====== ========================
04:24:52
lgi15(0)        6 E     32   24    15   46 E    100.74  94.67 E  262 E
```

Related Commands

- set_multi_cpu_usage

report_resource

```
report_resource
[-help]
[-peak]
[-verbose]
[-start string | [-end string [-session_cpu] [-diff_memory]]]
[-check_MemCpu {true | false} [-period integer]
[-memory_threshold float]]
```

Displays the peak memory used during a software session. This command is used to measure the peak memory at any given point in the flow and and make the script better for automation flows.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-check_memory_cpu {true | false}`	Checks the machine's physical memory and CPU resources. When specified, the software enables memory/CPU checking and reports a warning if the free memory is below 5 percent of the total memory or if the average CPU load is large.
`-diff_memory`	Outputs different peak memory and current memory between the step start and step end.
`-end string`	Specifies the end point to measure the peak memory attained during a specific command.
`-memory_threshold float`	Specifies the threshold as ratio of total machine/LSF memory. A warning is issued if the available memory goes below this threshold. *Min*: 0 *Max*: 1
`-peak`	Reports current peak memory till a given point in a flow. This parameter also works with the `-start` and `-end` parameters of the `report_resource` command so that peak memory observed by an individual command can be reported.

`-period` *integer*	Specifies the period for checking the physical memory and CPU usage. *Default*: 1800s *Min*: 1 *Max*: INT_MAX
`-session_cpu`	Reports the current CPU and real-time after the reports' last step status.
`-start` *string*	Specifies the start point to measure the peak memory attained during a specific command. **Note:** The `-start` and `-end` parameters support any user-specified string, such as fullFlow, report_timing, and readDesign. You must specify the same string for both the `-start` and `-end` parameters to measure the peak memory attained during a specific command.
`-verbose`	Gives detailed memory usage information. It displays how much memory is being used at any time and of what form (physical vs. virtual). When specified, the following information is displayed: • `Cpu(s)` is the number of available processors in the machine. • `Load` average is the system load averages for the last 1 minute, 5 minutes, and 15 minutes. • `Mem` and `Swap` are the current memory information of the machine. • `Data Resident Set (DRS)` is the amount of physical memory devoted to other than executable code. "`current mem`" shows this value (Total current DRS) . • `Private Dirty (DRT)` is the memory which must be written to disk before the corresponding physical memory location can be used for some other virtual page. "`peak res`" shows this value (Total peak DRT). This is the minimum number that you must reserve to run the program. • `Virtual Size (VIRT)` is the total amount of virtual memory used by the task. • `Resident Size (RES)` is the non-swapped physical memory a task has used. The number of "Total Peak RES" is the recommended physical memory to reserve.

Examples

- The following command specifies to display the memory used during a software session:

  ```
  tempus> report_resource
  ```

  The following message is displayed:

  ```
  Current (total cpu=0:04:19, real=18:28:46, peak res=3540.1M, current mem=3403.7M)
  ```

- The following command specifies to display the peak memory up to a specified point in a flow:

  ```
  tempus> report_resource -peak
  ```

  The following message is displayed:

  ```
  Current peak res=3540.1M
  ```

- The following command specifies to display the detailed memory information:

  ```
  tempus> report_resource -verbose
  ```

  The following output is displayed:

  ```
  Current (total cpu=0:04:19, real=18:30:07, peak res=3540.1M, current mem=3403.7M)


  Cpu(s) 40, load average (past 1 minute): 4.25, load average (past 5 minutes):
  5.66, load average (past 15 minutes): 10.70
  Mem:   790948060k total,  741280860k used,   49667200k free,       7512k buffers
  Swap:  134217724k total,   18325308k used,  115892416k free,  244555880k cached

  Memory Detailed Usage :
                  Data Resident Set(DRS)    Private Dirty(DRT)    Virtual
  Size(VIRT)    Resident Size(RES)
  Total  current :      5575.0M            2338.3M            9711.3M
   3403.7M
        peak    :      5761.5M            2364.6M            9897.9M
   3540.1M
  ```

- The following command specifies to display the end point to measure the peak memory attained during the `report_timing` command.

  ```
  tempus> report_resource -end report_timing
  ```

  The following message is displayed:

  ```
  Ending "report_timing" (total cpu=0:00:00.7, real=0:03:57, peak res=3430.0M,
  current mem=3403.7M)
  ```

- The following command script specifies to display the peak memory attained during the `report_timing` command:

  ```
  tempus > report_resource -start report_timing
  ```

```
tempus > report_timing -machine_readable -max_paths 100
tempus > report_resource -end report_timing
```

The following message is displayed:

```
Ending "report_timing" (total cpu=0:00:55.9, real=0:00:56.0, peak res=1032.5M,
current mem=1213.2M)
```

- The following command script specifies to display the peak memory attained till the `report_timing` command:

```
tempus > report_timing -machine_readable -max_paths 100
tempus > report_resource -peak
```

The following message is displayed:

```
Current peak res=1032.4M
```

- The following command script specifies to display CPU time/memory information during `Test`:

```
tempus > report_resource -start Test
tempus > report_resource -start read_design
tempus > report_resource -end read_design
tempus > report_resource -start report_timing
tempus > report_timing -machine_readable -max_paths 100
tempus > report_resource -end report_timing
tempus > report_resource -end Test
```

The following message is displayed:

```
Ending "report_timing" (total cpu=0:00:55.8, real=0:00:56.0, peak res=1032.4M,
current mem=1213.1M)
Ending "Test" (total cpu=0:01:37, real=0:01:38, peak res=1032.4M, current
mem=1213.1M)
```

Related Commands

- `report_timing`

set_distributed_hosts

```
set_distributed_hosts
[-help]
[out_file]
[-add string]
[-args string]
[-custom]
[-custom_script string]
[-custom_script_list string]
[-env_script string]
[-local]
[-lsf]
[-nc]
[-no_wait_task_come_up]
[-queue string]
[-remove string]
[-report_lsf_info]
[-resource string]
[-shell_timeout integer]
[-single_cpu_lsf_args string]
[-ssh]
[-time_out integer]
```

Specifies the multiple-CPU processing configuration for distributed processing or Superthreading. The software finds hosts from the last `set_distributed_hosts` command specified before a distributed processing or Superthreading command starts.

Use `get_distributed_hosts` to display the current settings for the `set_distributed_hosts` command.

ⓘ This command is required for distributed processing and Superthreading.

Parameters

`-help`	Prints out the command usage

out_file	Specifies to report LSF information in a single report. This parameter allows you to collect the system statistics on the status of the distributed jobs. It has the following information: • Host name for master and LSF tasks • Number of CPUs on each machine that are being used by a process • Job ID/PID for each task process • Total memory used on each machine • Total virtual memory used on each machine • Total CPU time used on each machine You must specify this parameter with the `-lsf` parameter. To get the LSF information, specify the `get_distributed_hosts out_file` command parameter. You must specify the `get_distributed_hosts out_file` only after the LSF tasks exit to obtain the task CPU time. If you specify this command before the task exits, the command gives a warning message.
`-add` *string*	Adds the specified hosts to the `rsh/ssh` configuration. If you specify more than one host, separate each host name with a space. Each time you specify a host, the software uses it to run one client process. To run more than one client process on a host, specify it more than once. For example, if you have two machines that each has two CPUs and you want to use both CPUs in both machines, type a command like the following: `set_distributed_hosts -add {`*machine1 machine2 machine1 machine2*`}`
`-args` *string*	Specifies any additional arguments you need to provide to the Sun Grid Engine (SGE) or Load Sharing Facility (LSF) job submission command.
`-custom`	Specifies the use of custom job submission script for distributed processing.

`-custom_script` *`string`*	Specifies a custom script to run distributed processing. This includes custom job submission script and arguments for "custom" distributed processing mode. You can use this parameter with `-custom` only.
`-custom_script_list` *`string`*	Specifies custom job submission script list and arguments for "custom" distributed processing mode. for example, `{{{stringCommand1} count1} {{stringCommand2} count2} ... {{stringCommandN} countN}}`. Specifies to customize the launch of multiple jobs, based on resources required for these jobs. This parameter allows you to specify multiple launch strings in order of priority with a maximum allowed job count for each string. If you request for N number of jobs, the command starts submitting jobs starting from *`string1`*, until the corresponding count is reached, at which point it starts submitting the next string in the list. Therefore, the command submits the first "*`count1`*" jobs with *`string1`*, the next "*`count2`*" jobs with *`string2`* and so on. It stops when the total submitted jobs has reached the requested number (N). **Note**: This parameter can only be specified with the `-custom` parameter.
`-env_script` *`string`*	Specifies a user-defined shell environment script that is run for each of the client hosts. The parameter allows you to pass these environment settings to the client hosts by including them in the shell environment script. The license variables used by the software are also automatically passed to the client hosts to avoid license errors.
`-local`	Run distributed processing job on the local machine.
`-lsf`	Specifies a Load Sharing Facility configuration. If you do not specify any parameters for `-lsf`, the software uses default settings. **Note:** To use this parameter, LSF must already be set up (typically by specifying the LSF_ENVDIR and LSF_SERVERDIR environment variables) and bsub must be in your search path. Contact your LSF administrator for details.
`-nc`	Allows the software to use NC commands (Network Computer from RTDA) for distributed processing.

`-no_wait_task_come_up`	Does not allow the software to wait for a long time if a task does not come up.
`-queue` *string*	Specifies the queue for the LSF or SGE configuration.
`-remove` *string*	Removes the specified hosts or all hosts from the `rsh/ssh` configuration. If you specify more than one host, separate each host name with a space.
`-report_lsf_info`	Collects the LSF slave information after EDP finish.
`-resource` *string*	Specifies a resource string for the SGE or LSF queue. **Note:** The correct resource string is specific to your installation. Contact your LSF administrator for the appropriate parameters and values.
`-shell_timeout` *integer*	Specifies the number of seconds the host machine waits for `rsh/ssh` machines to become available for multiple-CPU processing. *Default:* `5`
`-single_cpu_lsf_args` *string*	Specifies the `bsub` options. For information, see the LSF documentation. **Note:** You must use this parameter to adjust the LSF argument for a single CPU application. Alternatively, use the `-args` parameter to adjust the LSF argument for an application that runs in the Superthreading mode.
`-ssh`	Specifies a secure shell configuration.

`-time_out` *integer*	Specifies the default timeout for a submitted job that will never start. This functionality prevents the master machine from hanging when the clients do not come up for some reason. After timeout, a warning message appears informing you that the distributed CPUs did not come online within the expected time. The master will continue waiting for the clients and it is up to you to continue waiting or kill the run if there are problems with the Distributed Resource Management (DRM) system. A sample warning message is given below: Task id 2 did not come up within the specified timeout period (300 seconds). Use `set_distributed_hosts -timeout` to change the value. This may be a real error due to incorrect or incomplete platform/machine specification provided via set_distributed_hosts to your distributed resource management system (LSF/Sun Grid Engine etc.). If running locally (`set_distributed_hosts -local`), the current machine may be overloaded with too many jobs and swapping (check free memory and CPUs with top). For local runs, consider reducing the number of CPUs you are using via the `set_multi_cpu_usage` command or free up some memory by killing other memory intensive processes. *Default:* 3600 for all the modes (LSF, NC, SGE, RSH, SSH, local and custom **Note**: When the `set_distributed_hosts` command is not specified, the software sets the default mode as local and the EDP timeout is `3600` seconds.

Examples

LSF

The following command uses an existing LSF environment, and relies on the general LSF set up by the system administrator. In this example, the name of the queue implies that all the machines have 4 Gb of memory.

```
set_distributed_hosts -lsf -queue mem4G
```

The following command schedules a LSF job named `bigQueue` to run on a Linux machine for a

maximum of 30 minutes. No other jobs can run on the machine while `bigQueue` is running.

```
set_distributed_hosts -lsf -queue bigQueue -args {-x -W 30} \
   -resource {OSNAME==Linux}
```

The following command schedules a LSF job named `smallQueue` to run on Linux machines with minimum speed of 2000 MHz.

```
set_distributed_hosts -lsf -queue smallQueue -args {-W 20} \
    -resource {OSNAME==Linux && SPEED>2000}
```

Note: Check with your LSF administrator for the correct values for the resource string, as the parameters and their values differ in each installation. For example, the parameter name for the speed of the machine might be `MHZ` instead of `SPEED` (as in the preceding example), and the units might be specified in gigahertz instead of megahertz.

The following command specifies to report LSF information:

```
set_distributed_hosts -lsf -args "-q lnx64" out_file
getDistributeHost out_file lsf.log
```

Note: Use the above two commands together to save the LSF information in the specified file.

Local

The following command runs all distributed processing jobs on the machine on which you are running the master version of the software:

```
set_distributed_hosts -local
```

Custom

The following command runs distributed processing with a custom script:

```
set_distributed_hosts -custom -custom_script {bsub -q bigQueue -x -R "OS==Linux &&
memory>20000"}
```

The following command runs distributed processing with a custom script list:

```
set_distributed_hosts -custom -custom_script_list {{{bsub -P ICD -W 2 -q admin -R
"OSNAME==Linux && OSREL==EE50"}1} {{rsh hpc1 -e} 1}}
```

Examples

- The following command allows the Tempus client to source the `my_env_file` before start up on the same machine:

  ```
  set_distributed_hosts -local -env_script my_env_file
  ```

  This command allows the master to pass on the user-defined environment settings.

- The following command allows the Tempus client to run using the LSF:

  ```
  set_distributed_hosts -lsf -noWaitTaskComeUp
  ```

  When the `-noWaitTaskComeUp` parameter is specified, the Tempus software will not wait for all the clients to come up when parts of the clients are launching timeout.

Related Commands

- `get_distributed_hosts`

set_multi_cpu_usage

```
set_multi_cpu_usage
[-help]
[-acquire_license integer]
[-auto_page_fault_monitor {0 | 1 | 2 | 3}]
[-cpu_per_remote_host integer]
[-keep_license {true | false}]
[-license_list string]
[-local_cpu string]
[-release_license]
[-remote_host integer]
[-reset]
[-thread_info {0 | 1 | 2}]
[-verbose]
```

Specifies the number of threads to use for multi-threading, or the maximum number of computers to use for distributed processing, or the maximum number of computers and the number of threads to use for Superthreading. Optionally, reports usage information.

ⓘ This command is required for multi-threading, distributed processing, and Superthreading.

The `set_multi_cpu_usage` command enables multi-threaded timing reporting under the following conditions:

- `timing_report_group_based_mode` attribute is set to `true`. If there are multiple paths with the same slack then the selection can change on multiple runs. This may result in some changes in the `report_timing` output.
- `report_analysis_coverage` command is in non-verbose mode.
- `check_timing -include clock_crossing` parameter is specified.

Parameters

`-help`	Prints out the command usage.

`-acquire_license` *integer*	Acquires licenses to enable the specified number of CPUs. For example, if you specify `-acquire_license 20`, the software checks out enough licenses to enable required CPUs:
	`tempus 12> set_multi_cpu_usage -acquire_license 20`
	`Total CPU(s) requested: 20`
	`Total CPU(s) enabled with current License(s): 16`
	`Current free CPU(s): 16`
	`Additional license(s) checked out: 1`
	`Tempus_Timing_Signoff_MP license(s) for 16 CPU(s)`
	`Periodic Lic check successful`
	`Feature usage summary:`
	`Tempus_Timing_Signoff_XL`
	`Tempus_Timing_Signoff_MP`
	`Total CPU(s) now enabled: 32`
	After the licenses are checked out, they can be used for multi-threading, distributed processing, or super-threading.

`-auto_page_fault_monitor {0 | 1 | 2 | 3}`	Specifies to set a warning rank for performance issues caused by major page faults. The parameter supports the following values: • `0` - specifies to disable the reporting of major page faults. • `1` - specifies to set the warning rank as 1. This means `PF/total PF` > 0.8, `b` > 10, and `wa` > 85% • `2` - specifies to set the warning rank as 2. This means `PF/total PF` > 0.5, `b` > 5, and `wa` > 65% • `3` - specifies to set the warning rank as 3. This means `PF/total PF` > 0.2, `b` > 1, and `wa` > 40% where `PF` is the number of major page faults from the software, `total PF` is the total number of major page faults in the machine, `b` is the number of blocked processors, and `wa` is CPU waiting time.

`-cpu_per_remote_host` *integer*	Specifies the number of CPUs on each of the remote machines. This parameter is required for Superthreading. ⓘ For Superthreading, you must use this parameter in conjunction with the -remote_host parameter. *Default:* `1`
`-keep_license {true | false}`	Specifies whether to keep the acquired multiple CPU-licenses until the current session ends. Specify this parameter before running any commands that require multiple-CPU applications. To release all multiple-CPU licenses immediately, use the releaseLicense option. *Default:* `true`
`-license_list {enccpu | nru | edsl | edsxl}`	Specifies the list or order of licenses the software should use for checking out licenses during multiple-cpu processing. The parameter does not support an empty license list {}. You must specify at least one license from the following list: enccpu, nru, edsl, and edsxl.

`-local_cpu {integer | max}`	Specifies the number of CPUs on a local machine. When the `-localcpu` parameter is set to `1` (default), the multi-threaded timing propagation mode is enabled. Also, both CPU multi-threading and distributed process are enabled.
	
	The `set_multi_cpu_usage -localCpu` command automatically controls the total number of threads used up to the specified number. The number of specified threads cannot be more than the number of CPUs available in the local computer. Consider a computer having 8 CPUs, then the following command will issue a warning that the number of specified local CPUs is greater than the actual number of CPUs available:
	`set_multi_cpu_usage -localCpu 10`
	
	The local CPU count can be the maximum value that is shown as “processors” on the system under “/proc/cpuinfo”. This value can be obtained by running the “checkSysConf” utility in the "tools/bin" folder.
	
	`$> tools/bin/checkSysConf`
	`========== checkSysConf: Version 3.53`
	`==========`
	`OS Version .....: Red Hat Enterprise Linux Workstation release 7.6 (Maipo)`
	`Kernel Version .....: 3.10.0-957.5.1.el7.x86_64`
	`Hardware Type .....: x86_64`
	`Memory .....: 2063415 Megabyte`
	`CPU model .....: Intel(R) Xeon(R) Gold 6254 CPU @ 3.10GHz`
	`No. of CPUs .....: 72`
	`Total Swap Space.....: 131072 Megabyte`
	`==========`
	
	`$> cat /proc/cpuinfo | grep "processor" | tail -1 | awk -F ":" '{print $2 + 1}'`
	`72`
	
	Default: 1

`-release_license`	Releases all multiple-CPU license(s) immediately. By default, the software holds multiple-CPU licenses until the end of the current session. To specify that the software release multiple-CPU licenses after every multiple-CPU command runs, use -keep_license.
`-remote_host integer`	Specifies the number of remote machines. This parameter is required for distributed processing and Superthreading. *Default:* `0`
`-reset`	Resets the specified parameters back to default values.

`-thread_info {0 | 1 | 2}`	Reports usage information.
	This parameter works for multi-threading only.
	Default: `0`
	Specify one of the following values:
	0: Does not write messages to the log file.
	1: Writes the final message to the log file.
	For example,
	All threaded jobs finished (10 elapsed sec: 0 processor sec (parent), 0 system sec (parent), 0 processor sec (threads), 0 system sec (threads)).
	2: Writes additional starting/ending information for each thread.
	For example,
	`Starting threaded job 1...`
	`Starting threaded job 2...`
	`Starting threaded job 3...`
	`Ending threaded job 2 (1 elapsed sec, 0 processor sec, 0 system sec, 1.160M).`
	`Ending threaded job 3 (2 elapsed sec, 0 processor sec, 0 system sec, 0.895M).`
	`Ending threaded job 1 (10 elapsed sec, 0 processor sec, 0 system sec, 1.172M).`
	`All threaded jobs finished (10 elapsed sec: 0 processor sec (parent), 0 system sec (parent), 0 processor sec (threads), 0 system sec (threads)).`
	Note: This parameter works in multi-threading mode only.

`-verbose`	Displays messages when changing the multiple-CPU settings.

Examples

Here are some examples.

- The following command specifies that 8 CPUs can be used on a local machine for multi-threading. This is based on the assumption that 8 CPUs are available on the local machine; otherwise the software will utilize the available CPUs:

```
tempus > set_multi-cpu_usage -localCpu 8
```

- The following command specifies that the number of remote machines to be used is 4 for distributed processing, and also for each of these hosts, 4 CPUs will be used for super threading:

  ```
  tempus > set_multi_cpu_usage -remoteHost 4 -cpuPerRemoteHost 4
  ```

  Here, during DMMMC analysis, in total 16 analysis sessions can be run in parallel.
  In addition, multi-CPU configuration settings for distributed processing must be done using the `set_distributed_hosts` command.

- The following command releases all the multi-CPU licenses immediately:

  ```
  tempus > set_multi_cpu_usage -releaseLicense
  ```

Related Commands

- `get_multi_cpu_usage`
- `check_multi_cpu_usage`

17

OpenAccess Commands and Attributes

- compare_oa_cellview
- create_oa_lib
- create_oa_lib_property
- get_oa_default_rule_lib
- get_oa_design_data
- oa_read_write Category Attributes
- read_oa
- register_trigger
- unlock_oa_design
- update_oa_lib
- write_lef_library
- write_oa_techfile

compare_oa_cellview

```
compare_oa_cellview
[-help]
[-ignore {pin_purpose}]
[-out_file report_file_name]
{ {{-source {from_lib from_cell from_view} -compare {to_lib to_cell to_view}} | {-
lib lib -cell cell -view {sourceView compareView}}} }
```

Compares specified cell views. Use this command to compare two OpenAccess databases. For instance, you can use this command to compare the lib/cell/view specified using `-source` to the lib/cell/view specified with `-compare`. In an abstract to layout comparison, boundary and terminals (name, direction, pin location and size) are compared. This command can be used without design import.

Parameters

`-help`	Prints a brief description that includes type and default information for each `compare_oa_cellview` parameter. For a detailed description of the command and all of its parameters, use the man command: `man compare_oa_cellview`.
`-cell cell`	Specifies the cell for comparing two views. *Data_type*: string, optional
`-compare {to_lib to_cell to_view}`	Specifies the library/cell/view that is to be compared. *Data_type*: string, optional
`-ignore {pin_purpose}`	Specifies the list of attributes to be excluded from the cell comparison. As of now, the only attribute that can be ignored is the purpose, which is specified using the value `pin_purpose` with `-ignore`. *Data_type*: (enum)+, optional
`-lib lib`	Specifies the library for comparing two views. *Data_type*: string, optional
`-out_file report_file_name`	Specifies the report file in which all the comparison detail is to be written. *Data_type*: string, optional
`-source {from_lib from_cell from_view}`	Specifies the source library, cell, and view. *Data_type*: string, optional
`-view {sourceView compareView}`	Specifies different views of the same library/cell. *Data_type*: string, optional

Examples

- The following command compares two views of the same cell that are present in the same library `zambezi45` and writes the report to the `test1a` file:

```
compare_cellview -source {zambezi45 pll_vcodiv layout} -compare {zambezi45
pll_vcodiv abstract} -out_file test1a
```

A sample output is displayed below:

```
#################################################
# #
# Compare_cellview comparison #
# #
#################################################

Source Design : 'zambezi45/pll_vcodiv/layout' is identified as layout design
Compare Design : 'zambezi45/pll_vcodiv/abstract' is identified as abstract design

Source Design seems to have been modified after the Compare Design has been created

Compare Celltype

Source Design has celltype 'none' & Compare Design has Celltype 'block'

Compare Cell Boundary

Source Design has different PRBoundary than Compare Design

Compare Terminals

Source Design has Pin 'en' which has Port with shape (36.215 0 36.355 0.14) on layer
'Metal5' and purpose 'pin' is not present in Compare Design
Compare Design has Pin 'en' which has a Port with shape (36.215 0 36.36 0.14) on
layer 'Metal5' and purpose 'net' is not present in Source Design
...

Antenna Information

Antenna Information is defined for Pin 'en' in Compare but not defined in Source
Design
Antenna Information is defined for Pin 'vcocalen' in Compare but not defined in
Source Design
...

Source Design is not equivalent to Compare Design
```

- The following command compares the views `fplan1` and `fplan2` of the library `DesignLib` and the cell `myAlu`:

```
compare_oa_cellview -source {DesignLib myAlu fplan1} -compare {DesignLib
myAlu fplan2}
```

create_oa_lib

```
create_oa_lib
[-help]
oa_lib
[-out_dir path ]
[-no_compress]
{-oa_attach_tech_lib techlib |
-oa_copy_tech_lib techlib |
-oa_reference_tech_libs techliblist}
```

Copies or attaches a technology database to a library. If the library exists, this command does not update the library.

Note: The parameters must appear in the order specified by the syntax.

The `create_oa_lib` command is part of the OpenAccess design flow. Use this command to create an OpenAccess library at the specified path. `create_oa_lib` needs to be called only once before the design is saved. Subsequent `saveDesign` calls to the same library will automatically find out the library path that gets stored in `cds.lib`.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_oa_lib` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_oa_lib`.
`oa_lib`	Specifies the name of a library to be created. The software creates the library and adds its name to the `cds.lib` file.
`-out_dir` *`path`*	Specifies the name of a directory in which to save the library. The path you specify must include the directory that will become the library itself. *Default:* `./` *`oa_lib`* (Saves the library to the current working directory)
`-no_compress`	Creates a new library without any compression irrespective of the compression settings provided by the `setOaxMode` command.
`-oa_attach_tech_lib` *`techlib`*	Attaches the technology database specified by the technology library to the library that is to be created.
`-oa_copy_tech_lib` *`techlib`*	Copies the technology database specified by *`techLib`* to the library specified by *`oa_lib`* , and saves the library to the current working directory.
`-oa_reference_tech_libs` *`techliblist`*	Creates a local technology database in the design library that can be modified incrementally.

Examples

- The following command creates the library `myLib` in the current directory, and attaches the technology database in `myRefLib` to `myLib`:

  ```
  create_oa_lib myLib -oa_attach_tech_lib myRefLib
  ```

  Note: Note: If `myRefLib` is read-only, the `saveDesign -cellview` command generates an error if it needs to update the technology database.

- The following command creates the library `TEST_LIB` at the specified path, and attaches the technology database in `TEST_REF_LIB` to TEST_LIB:

  ```
  create_oa_lib TEST_LIB -out_dir ./outputs/TEST_LIB -oa_attach_tech_lib TEST_REF_LIB
  ```

- The following command generates an error because only one reference library can be specified:
  ```
  create_oa_lib myLib -oa_attach_tech_lib {myRefLib1 myRefLib2}
  ```
- The following command creates the library `myLib`, and copies the technology database from `myRefLib` to `myLib`. If `myRefLib` is modified, the two technology databases will become out of sync. Additionally, any future write operations from the `saveDesign -cellview` command will modify the technology database copy in `myLib` and also cause the databases to be out of sync.
  ```
  create_oa_lib myLib -oa_copy_tech_lib myRefLib
  ```
- The following command creates a new library which references existing technology databases (`refLib1` and `refLib2`), but allows local technology updates if required (typically to add new custom vias to the local techology database).
  ```
  create_oa_lib myLib -oa_reference_tech_libs {refLib1 refLib2}
  ```

Related Information

- `create_oa_lib_property`
- `oa_read_write` Category Attributes
  - `oa_new_lib_compress_level`
- `write_db`

create_oa_lib_property

```
create_oa_lib_property
[-help]
-oa_lib libName
-name prop_name
-type {boolean integer float string}
-value propValue
```

Adds a new property to the OpenAccess library, `oalib`. This command cannot be used to change the value of an existing property.

Note: You cannot use the name `techLibName` for the property you are adding because this name is used by the OpenAccess infrastructure to store the property attached to the library.

Parameters

`-help`	Prints a brief description that includes type and default information for each `create_oa_lib_property` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_oa_lib_property`.
`-oa_lib` *`libName`*	Specifies the name of the OpenAccess library to which the property has to be added.
`-name` *`prop_name`*	Specifies the name of the property being created.
`-type {boolean integer float string}`	
	Specifies the type of the property being created.
`-value` *`propValue`*	Specifies the value to be set for property being created. The value is interpreted from the input string and converted to the specified type.

Example

In Tempus, the following set of commands creates a library named *`myLib`*, adds the *`prop1`* property to it, and saves the design before exiting:

```
createLib myLib -attachTech adiTech

add_oa_lib_property -oa_lib myLib -name prop1 -type string -value value1
```

```
saveDesign -cellview {myLib block1 layout}

exit
```

You can now add more properties to the library in Virtuoso. When you restore the design in Tempus, all properties previously added to the library, whether in Tempus or Virtuoso, reflect correctly.

Related Information

- create_oa_lib

get_oa_default_rule_lib

```
get_oa_default_rule_lib
[-help]
[-rule rule_name]
[-libs lib_list]
[-verbose]
```

Searches through a specified library (or libraries) to check whether a specified LDRS exists or not. You can run this check before reading a library.

Parameters

`-help`	Prints a brief description that includes type and default information for each `get_oa_default_rule_lib` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man get_oa_default_rule_lib`
`-libs` *`lib_list`*	Specifies the list of library names to search through. If not specified, all libraries are searched.

`-rule` *`rule_name`*	Specifies the name of the rule to find.
	
	If not specified, the `init_oa_default_rule` attribute is used. If the attribute has an empty string value, the standard lookup (`LEFDefaultRouteSpec` by type, then by name) is used.
	
	If the specified *`rule_name`* does not exist, the command displays an error such as:
	`Rule '<rule_name>' was not found in specified libraries.`
	
	If the specified *`rule_name`* is found, the command returns the name of the library where the specified *`rule_name`* was found. This could be from the `-libs` *`lib_list`* or any ITDB that one of those libraries refers to.
	
	If the specified *`rule_name`* is found but is not a legal LDRS type as determined by the OAX reader code, the command displays an error such as:
	`Constraint group '<rule_name>' was found in library '<lib_name>', but is not a legal LEFDefaultRouteSpec`

`-verbose`	Echoes details of library name where the rule was found (including descending through technology references from libraries specified in the *`lib_list`*.

Examples

- The following example shows how `get_oa_default_rule_lib` can be used to search for a specific library and the graph of techs that it references:

```
set_db init_oa_ref_libs "tech_lib macro_lib"
set_db init_oa_default_rule LDRS_65nm_6lm
set tech_lib [lindex $init_oa_ref_libs 0]
set lib_found [get_oa_default_rule_lib -libs $tech_lib] # use the
init_oa_default_rule to look up the library
if {$lib_found != ""} {
Puts "$init_oa_default_rule was found in $lib_found or in the graph of libraries that
$tech_lib references"
} else {
Puts "$init_oa_default_rule was not found in $tech_lib nor in the graph of libraries
that $tech_lib references"
}
```

- The following command shows how the `-verbose` option can be used:

```
get_oa_default_rule_lib -verbose
INFO: Default rule 'LEFDefaultRouteSpec' was found in library 'FEOAreflib8'.
FEOAreflib8
```

Related Information

- `init` Category Attributes

get_oa_design_data

```
get_oa_design_data
[-help]
{{-all_oa_libs} | {{-design_data dd_objHandle | -oa_lib libName | -oa_cell {lib cell} | -
oa_cellview {lib cell view}}
[-children | -file | -path | -type |-name]}}
```

Returns OpenAccess library information.

Parameters

`-all_oa_libs`	Returns a list of design data (`dd`) object handles for the list of libraries present in the library definition file (`cds.lib`).
`-children`	Returns the `dd` object handles of the children of the specified `dd` object handle. Children of libraries are cells. Children of cells are views.
`-design_data` *dd_objHandle*	Specifies the `dd` object to be processed.
`-file`	Returns the `dd` object handles of all the files in the specified `dd` object handle.
`-name`	Returns the name of the specified `dd` object handle.
`-oa_cell {`*lib cell*`}`	Specifies the names of the library and cell to be processed.
`-oa_cellview {`*lib cell view*`}`	Specifies the names of the library, cell, and view to be processed.
`-oa_lib` *libName*	Specifies the name of the library to be processed.
`-path`	Returns the absolute path name (`dd` path) of the specified `dd` object handle.
`-type`	Returns the type of the specified `dd` object handle.

Examples

- The following command returns the path of the specified library/cell/view as shown below:

  ```
  get_oa_design_data -cellview {pdkLib cell1 abstract} -path
  ```

  returns:

  ```
  /libs/pkdLib/cell1/abstract
  ```

  This command is equivalent to the following SKILL function:

  ```
  ddGetObjReadPath(ddGetObj("pdkLib" "cell1" "abstract"))
  ```

- The following proc traverses the cds.lib structure (expanding #INCLUDE as required) to get the complete list of OpenAccess libraries and their full paths.

  ```
  # expand #INCLUDE as needed and print all libraries and their full paths
  proc print_all_lib_paths {} {
     Puts [format "%16s -> %s" "Library Name" "Library Path"]
     Puts "-------------------------------------------------------------------"
     foreach oa_lib [get_oa_design_data -all_oa_libs] {
         Puts [format "%16s -> %s" [get_oa_design_data -design_data $oa_lib -name]
  [get_oa_design_data -design_data $oa_lib -path]]
     }
  }
  ```

oa_read_write Category Attributes

The root attributes in the `oa_read_write` category are:

- oa_allow_analysis_only
- oa_allow_bit_connection
- oa_allow_tech_update
- oa_bindkey_file
- oa_cell_view_dir
- oa_convert_diagonal_path_to
- oa_cut_rows
- oa_display_resource_file
- oa_display_resource_file_in_library
- oa_drc_fill_purpose
- oa_enclose_quick_abstract_pins
- oa_full_layer_list
- oa_full_path
- oa_hybrid_rows
- oa_inst_placed_if_none
- oa_lib_create_mode
- oa_lock
- oa_logic_only_import
- oa_merge_trim
- oa_new_lib_compress_level
- oa_non_rectilinear_shapes
- oa_pin_purpose
- oa_push_pin_constraint
- oa_quick_abstract_for_custom_cells
- oa_read_system_reserved_layers

- oa_silently_ignore_unsupported_vias
- oa_text_purpose
- oa_tie_net
- oa_update_mode
- oa_use_virtuoso_bindkey
- oa_use_virtuoso_color
- oa_view_sub_type
- oa_write_cds_fixed_vias
- oa_write_mask_data_locked
- oa_write_net_voltage
- oa_write_relative_path

To set the value of any root attribute, use the set_db command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the get_db command:

```
get_db attribute_name
```

To get the current value of all attributes in the `oa_read_write` category, use the following command:

```
get_db -category oa_read_write
```

oa_allow_analysis_only

`oa_allow_analysis_only {true | false}`

Default : `false`

Data_type: Boolean

Allows analysis only flow.

`assemble_design` is not allowed when `oa_allow_analysis_only` is set to `false` and `oa_update_mode` is set to `true`, thereby locking the OpenAccess databse. To allow `assemble_design` to proceed, you must change `oa_allow_analysis_only` to `true`. If you set `oa_allow_analysis_only` to `true` and the Tempus session is in `oa_update_mode` = `true` mode (whether set explicitly or internally on the fly), the `oa_update_mode` setting is ignored.

Note that there may be loss of some semantic information, such as MPPs, while saving the design

when `oa_allow_analysis_only` is set to `true`. However, the saved design would be geometrically correct.

oa_allow_bit_connection

```
oa_allow_bit_connection {true | false}
```

Default: `false`

Data_type: Boolean

Creates databases for instances that only have bit-based terminals, without requiring the corresponding bus terminals.

When set to `true`, it saves the connectivity information by bits if the bus connections are not available.

When set to `false`, an error will be issued if the abstract view is missing expected bus terminal information.

oa_allow_tech_update

```
oa_allow_tech_update {true | false}
```

Default: `true`

Data_type: Boolean

Allows the technology database to be updated. The common reason that a technology database would need to be updated is that new vias have been added to the design that are not in a parameterized form. Via masters that are a simple list of rectangles require that a "custom via" be created in the technology database, If the design library had been created with an attached technology, the value should be set to `false` allowing a reference library technology that is shared by multiple designs to have design specific vias to be added to it. For design libraries that have been created in reference mode, allowing the technology database to be updated is typically fine but normally it is best to always use paramaterized vias for OpenAccess-based flows.

Using `set_db generate_special_via_symmetrical_via_only true` before any power planning or routing will typically prevent the creation of non-parameterized vias in the flow.

When `oa_allow_tech_update` is set to `false`, the creation of the OpenAccess database will fail if any custom vias would need to be created within the design library.

Related Commands and Attributes

`generate_special_via_symmetrical_via_only`

oa_bindkey_file

`oa_bindkey_file` *`fileName`*

Default: `""`

Data_type: String

Maps into Tempus equivalent bindkeys. If the bindkey file is not specified, the Virtuoso default bindkey information is used. This attribute must be used with the `oa_use_virtuoso_bindkey` attribute.

Related Commands and Attributes

`oa_use_virtuoso_bindkey`

oa_cell_view_dir

`oa_cell_view_dir` *`string`*

Default: ""

Data_type: String

Specifies the path name to the physical location of the lib/cell/view that was restored using `read_db -oa_lib_cell_view`. `oa_cell_view_dir` is a system-defined, transient (non-persistent) attribute used in the OpenAccess flow. This should be treated as a *read-only* attribute.

Related Commands

`read_db`

oa_convert_diagonal_path_to

`oa_convert_diagonal_path_to {polygon | default_style}`

Default: `polygon`

Data_type: Enum

Enables interoperability of the end style of geometries of type path segment between Virtuoso and Tempus.

This attribute applies to the diagonal path segments only.

If the end-style of a diagonal path segment is `custom`, then the end style is retained as is.

If the end style of a diagonal path segment is not `custom`, then if you set the value to `polygon`, Tempus converts the path segment into a geometrically equivalent polygon.

If the value specified is `default_style`, then the end style of the path segment is set to `default_style`. In this case, Tempus retains the semantics of the object to be a path segment.

Important: To ensure that there is no loss of semantic or geometric information, the path segments in Virtuoso -XL and Virtuoso-GXL should use the `custom` end-style on diagonal path segments which can be created using *Create-Shape-Geometric Wire*.

oa_cut_rows

```
oa_cut_rows {true | false}
```

Default: `false`

Data_type: Boolean

Cuts rows against placement obstructions, block halos, or power domain fence minimum gaps.

The following command cuts rows against placement obstructions, block halos, or power domain fence minimum gaps:

```
set_db oa_cut_rows true
```

oa_display_resource_file

```
oa_display_resource_file pathName
```

Default: `""`

Data_type: String

Allows you to specify a specific `display.drf` file that is to be used. It overrides all the searching options and the specified file is used.

The following command searches for the `mydisplay.drf` file irrespective of the value of the `oa_display_resource_file_in_library` attribute:

```
set_db oa_display_resource_file /servers/oastore/mydisplay.drf
```

oa_display_resource_file_in_library

```
oa_display_resource_file_in_library {true | false}
```

Default: `false`

Data_type: Boolean

Allows the tool to search for the `display.drf` file in the technology library (or complete technology graph in case of ITDB) when set to `true`. If not found, it looks for the file in the current working directory followed by the home directory.

When set to `false`, the tool searches for the `display.drf` file first in the current working directory followed by the home directory.

oa_drc_fill_purpose

```
oa_drc_fill_purpose {gap_fill | drawing}
```

Default: `gap_fill`

Data_type: Enum

Specifies whether to save DRCFILL shapes to the `gap_fill` or `drawing` purpose. By default, DEF routes with + SHAPE DRCFILL are mapped to the `gapFill` purpose in OpenAccess. However, some OpenAccess tech files do not have this purpose defined so any shapes mapped to the `gapFill` purpose would not be visible in Virtuoso and would not be written out in GDS or OASIS. In such cases, set the `oa_drc_fill_purpose` attribute to `drawing` to save these shapes to the `drawing` purpose. When you do so, these shapes get an additional internal attribute so they can round-trip back to DRCFILL shapes properly.

oa_enclose_quick_abstract_pins

```
oa_enclose_quick_abstract_pins  {true | false}
```

Default: `true`

Data_type: Boolean

Includes shapes that enclose layout pin shapes.

oa_full_layer_list

```
oa_full_layer_list {true | false}
```

Default: `true`

Data_type: Boolean

Imports the complete list of OpenAccess layers, when importing an OpenAccess design.

If you specify `false`, only OpenAccess layers defined with `oacMaterial` and type are imported.

oa_full_path

```
oa_full_path {true | false}
```

Default: `false`

Data_type: Boolean

Creates the `DEFINE` statements in the `lib.def` files using full paths. If you specify `-oa_full_path false`, the software creates the statements using relative paths.

The following command instructs the Tempus software to create the `DEFINE` statements using full paths:

```
set_db oa_full_path true
```

oa_hybrid_rows

```
oa_hybrid_rows {true | false}
```

Default: `false`

Data_type: Boolean

Reads hybrid rows from the Process Design Kit (PDK). When this attribute is set to `true`, the tool checks whether the sites are abutted horizontally or vertically and calculates the complex site width and height accordingly.

oa_inst_placed_if_none

```
oa_inst_placed_if_none {true | false}
```

Default: `true`

Data_type: Boolean

Maps the placement status of instances in the OpenAccess database to the placement status in the Tempus software. When set to `true`, the instances in the OpenAccess database that have an unknown placement status are treated as "placed" in the Tempus software. When set to `false`, instances with an unknown placement status are treated as "unplaced."

oa_lib_create_mode

```
oa_lib_create_mode {none | attach | copy | reference}
```

Default: `reference`

Data_type: Enum

Specifies the mode in which `write_db -oa_lib_cell_view` creates a library.

- `none`: Turns off the ability to have `write_db` create a library if it does not already exist.
- `attach`: Attaches the technology database specified by the technology library to the library that is to be created.
- `copy`: Copies the technology database specified by `techLib` to the library specified by `libName`, and saves the library to the current working directory.
- `reference`: Creates a local technology database in the design library that can be modified incrementally. This is the recommended setting to handle the case where the design may have "custom vias" added at some point by power planning/routing.

Related Commands

- `write_db`

oa_lock

```
oa_lock {true | false}
```

Default: `false`

Data_type: Boolean

Locks the OpenAccess database when restoring the design. When set to `true`, other tools can still access the OpenAccess database in read-only mode, but cannot edit the database as long as it is locked for the Tempus session. To remove the lock, specify the `reset_design` command, or exit the Tempus software.

Notes

- If you set the `oa_update_mode` attribute to `true`, the software automatically locks the database, even if the `oa_lock` attribute is set to `false`.
- The value of `oa_lock` cannot be changed while a design is in memory.

Related Commands

- `unlock_oa_design`

oa_logic_only_import

```
oa_logic_only_import {true  | false}
```

Default: `false`

Data_type: Boolean

Controls what information the software reads when importing an OpenAccess maskLayout database.

When set to `true`, the software reads only the logical (Verilog netlist equivalent) connectivity during the `init_design` process. When set to `false` (the default value), the software automatically reads physical information, such as floorplan, placement, and routing, from the database, and updates physical net connectivity for power and ground connections.

Note: The `oa_logic_only_import` attribute applies only to OpenAccess maskLayout databases. For other types of OpenAccess databases, the software always reads only the logical connectivity.

Related Commands

- `init_design`

oa_merge_trim

```
oa_merge_trim {true  | false}
```

Default: `true`

Data_type: Boolean

Merges the trim metal shapes during save when the trim metal gap is less than or equal to the max_gap in microns.

oa_new_lib_compress_level

```
oa_new_lib_compress_level value
```

Default: `1`

Data_type: Integer

Sets the compression level of a library during the library creation process. Cellviews written to a library will follow the compression level of the library and not the current `oa_new_lib_compress_level` setting.

Valid values range between 0 to 9. A value of 0 indicates that compression is turned off.

Related Commands

- `create_oa_lib`

oa_non_rectilinear_shapes

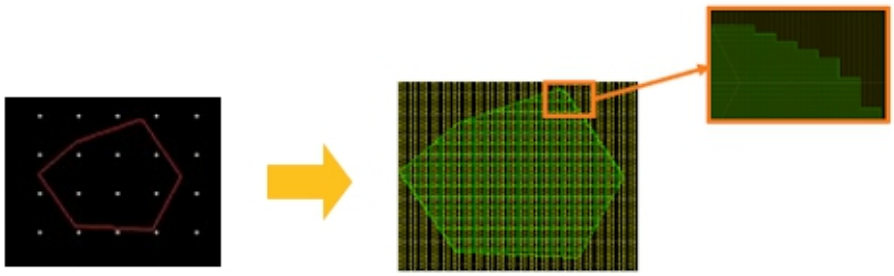
`oa_non_rectilinear_shapes {bbox | approximate}`

Default: bbox

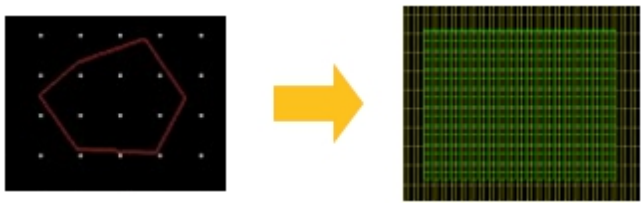
Data_type: Enum

Translates non-rectilinear shapes to rectilinear shapes while reading in an OpenAccess design:

- `approximate` - Converts non-rectilinear shapes into approximate stepped rectilinear shapes.



- `bbox` - Converts non-rectilinear shapes into bounding box shapes.



Related Commands

- `read_oa`

oa_pin_purpose

`oa_pin_purpose {true | false}`

Default: `false`

Data_type: Boolean

Writes out pin shapes with the purpose `pin`. When set to `false`, writes out pin shapes with the

purpose `drawing`.

oa_push_pin_constraint

```
oa_push_pin_constraint {true | false}
```

Default: `false`

Data_type: Boolean

Specifies whether or not `write_db` should push an interoperable pin constraint into its corresponding blackbox abstract cellview.

Note: Interoperable pin constraints refer to pin constraints that are understood by Tempus, Virtuoso, and PVS.

Related Commands

`write_db`

oa_quick_abstract_for_custom_cells

```
oa_quick_abstract_for_custom_cells {pcells stdcells all off}
```

Default: `all`

Data_type: Enum

Trims design shapes of custom cells. By default, this capability is turned on for all types of custom cells. The following settings are allowed for this attribute:

- `pcells` - Trims design shapes of only custom parameterized cells (PCells).
- `stdcells` - Trims design shapes of only custom standard cells.
- `all` - Trims design shapes of all custom cells.
- `off` - Does not trim design shapes of any custom cell.

> This feature needs the INVS30 (Innovus Mixed Signal Option) license. If this license is not available, the abstract that gets generated for custom cells will not work properly in Innovus.

oa_read_system_reserved_layers

```
oa_read_system_reserved_layers  {all | text | off}
```

Default: `all`

Data_type: Enum

Reads the system-reserved layers from the PDK. The following settings are supported:

- `all` - All system-reserved layers are read.
- `text` - Only text layer is read.
- `off` - No system-reserved layer is read.

oa_silently_ignore_unsupported_vias

```
oa_silently_ignore_unsupported_vias  {true | false}
```

Default: `false`

Data_type: Boolean

Specifies whether `read_db` should issue messages about vias that are not supported in Tempus when a Virtuoso design is read in.

oa_text_purpose

```
oa_text_purpose purpose_name
```

Default: `drawing`

Data_type: String

Specifies the purpose name that will be used when text labels are written.

By default, any text label created in Tempus (typically via the `create_text` command) is automatically mapped to the `drawing` purpose. Use the `oa_text_purpose` attribute to map the text labels to a user-defined purpose.

Note: `oa_text_purpose purpose_name` overrides the default `drawing` purpose on output.

oa_tie_net

```
oa_tie_net {tieHighNetName tieLowNetName}
```

Default: ""

Data_type: String

If these names exist in the design locally in any module, then the global names should be changed.

Specifies the global tie high and tie low net names to be created in the OpenAccess database. The nets are used to represent the Verilog `1'b1/1'b0` connections to the instance ports.

oa_update_mode

```
oa_update_mode {true | false | auto}
```

Default: `false`

Data_type: Enum

Ensures that `read_db -oa_lib_cell_view` processes parameterized cells (PCells) and other custom objects correctly in the Tempus database. For information about generating/regenerating the PCell cache, refer to the "Preparing the IP Library" section in the "Technology Data Preparation" chapter of the *Mixed Signal (MS) Interoperability Guide*.

The `oa_update_mode` attribute also ensures that `write_db -oa_lib_cell_view` updates the Tempus-modified portion of an OpenAccess database without making changes to PCells or other custom objects.

> In the `oa_update_mode true` mode, the tool locks the source cellview for the duration of the session.

This attribute provides the ability to import OpenAccess database created from Virtuoso that contains objects which are not understood by Tempus. Complete the digital part in Tempus and update only the objects modified by Tempus in the original OpenAccess database. If your design has any of the following objects, they would only be visible (not editable) in Tempus:

- Pcells
- Modgen cells
- Fig-groups
- Pcell vias
- RODs (Example, Multi-Part Paths, and so on)
- User defined properties on design objects such as nets, shapes (`lxStickyNet`), instances, abstract views, intermediate hierarchical instances, and so on.

You won't be allowed to edit or modify these objects.

On using the `oa_update_mode` attribute, the property on shapes (for example, `lxStickyNet`) and nets created in VLS-XL / GXL would be kept as-is.

In case of designs where parameterized vias are used, Tempus understands these attributes and evaluates them on-the-fly to create an exact shape and orientation of the cuts.

If the `oa_update_mode` attribute is set to `auto` and the cellview contains objects that are not supported in Tempus, the software internally enables update mode with a message such as follows:

```
INFO: Objects have been found in the cellview requiring updateMode to be enabled so that there is no loss of semantic information when the cellview is saved.
```

Note that enabling update mode can affect `write_db` performance.

Notes

- The value of the `oa_update_mode` attribute cannot be changed while a design is in memory. After this attribute has been set to `true` before importing an OpenAccess design into Tempus, you cannot convert the `oa_update_mode` attribute to `false` in any new Tempus session.
- The on-disk OpenAccess cellview is locked between the restore and save steps when the `oa_update_mode` attribute is set to `true`.
- In `oa_update_mode auto`, the lock is applied at the time that `read_oa` or `read_db -oa_lib_cell_view` starts and released at the end of the command if no objects were encountered that required updateMode to be enabled.

oa_use_virtuoso_bindkey

```
oa_use_virtuoso_bindkey {true | false}
```

Default: `false`

Data_type: Boolean

Allows you to use Virtuoso bindkeys.

oa_use_virtuoso_color

```
oa_use_virtuoso_color  {true | false}
```

Default: `false`

Data_type: Boolean

Allows you to display colors and stipples for the design layers (including full custom layers). Using this attribute, Tempus utilizes the same colors and stipples that you would see if you open the design in

Virtuoso.

oa_view_sub_type

```
oa_view_sub_type {vce | vxl | none}
```

Default: `vxl`

Data_type: Enum

Indicates which Virtuoso application should be used for viewing the layout cellviews in the saved OpenAccess database.

When set to `vce`, cellviews are opened in Virtuoso Chip Editor.

When set to `vxl`, cellviews are opened in Virtuoso XL.

When set to `none`, it indicates that the OpenAccess database can be edited by any editor within Virtuoso platform based on its license availability.

oa_write_cds_fixed_vias

```
oa_write_cds_fixed_vias {new_vias_only | off}
```

Default: `new_vias_only`

Data_type: enum, read/write

Controls how fixed vias are created when saving a design with custom vias from Innovus to OpenAccess:

- `new_vias_only` - Any newly created custom via is saved as a fixed via while any pre-existing custom via in the Incremental Technology Database (ITDB) is reused when saving the design.
- `off` - No cdsFixedVias are created. All fixed/custom vias in the design are saved as oaCustomVia.

> This feature needs the Mixed Signal Option license. If this license is not available, the `off` value will be used and no fixed vias will be created.

oa_write_mask_data_locked

```
oa_write_mask_data_locked {true | false}
```

Default: `false`

Data_type: Boolean

Sets the shapes with mask coloring (mask values other than 0) to locked in the saved cellview, when set to `true`.

By default, `write_db` write out shapes with their mask coloring information in an unlocked state. Some tools treat the unlocked color information as the recommended color. However, for some foundry processes, shapes with unlocked mask coloring information are written to GDSII/Stream as gray (uncolored). In such cases, set the `oa_write_mask_data_locked` parameter to `true` to lock the mask coloring of shapes explicitly.

Note: This option affects only the saving of shapes with non-zero mask coloring values. Review the map file that will be used to create GDSII/Stream output from the OpenAccess cellview to determine if the map file requires shapes to be locked for them to be written out as colored.

Related Commands

- `write_db`

oa_write_net_voltage

```
oa_write_net_voltage {true | false}
```

Default: `false`

Data_type: Boolean

Saves the minimum and maximum net voltage across all loaded timing views to every net. This is useful for Virtuoso, and other tools that may read the cellview, to understand the minimum and maximum net voltage without needing to interpret the CPF or .lib file to get that information. The round-trip of the information into Tempus is not dependent on this setting.

oa_write_relative_path

```
oa_write_relative_path {true | false}
```

Default: `true`

Data_type: Boolean

Specifies whether or not to save the relative path in the configuration file. Defines the current working directory (`$cwd`) in the configuration file as ".". When set to `true`, a design can be restored, even if the directory path of the design file is changed.

read_oa

```
read_oa
[-help]
oa_lib
oa_cell
oa_view
[-filter {block_insts blockages boundary fixed_core_insts floorplan pad_insts pin_shapes
regions regular_routing special_routing}]
[-nets { netnames }]
```

Restores floorplan, placement, and routing data using the OpenAccess format. This command only reads physical information from the OpenAccess database. For information on restoring an OpenAccess database, see read_db.

You can use the `read_oa` command after importing the design.

Note: Tempus supports only the following begin/end extension values for DEF `NETS` style wiring segments: 0, width/2 or LEF `LAYER WIREEXTENSION`. If any of the wire extensions has unsupported extension value, then Tempus converts it to a special wire (DEF `SPECIALNETS`). While writing back OpenAccess (`write_oa` or `write_db -oa_lib_cell_view`), these special wires will again be converted back to DEF `NETS` style wiring.

Parameters

`-help`	Outputs a brief description that includes type and default information for each parameter of the `read_oa` command. For a detailed description of the command and all of its parameters, use the `man` command: `man read_oa`.
`oa_lib`	Specifies the OpenAccess library name.
`oa_cell`	Specifies the name of the top cell to import.
`oa_view`	Specifies cellview name to import.
`-filter {block_insts blockages boundary fixed_core_insts floorplan pad_insts pin_shapes regions regular_routing special_routing}`	
	Lists the OpenAccess cellview information data to be read. When not specified, all information is read from the OpenAccess cellview. When `-filter` is specified, only the object types that are chosen will be read.

Additionally, when reading pin_shapes or instances, information in the cellview that refers to terminals or instances that do not exist in memory will be ignored.

The legal `-filter` options are:

- `block_insts` - Indicates that any CLASS BLOCK instances should be updated. Physical-only block instances may be added to the database if the placement status is `fixed` or `cover`.
- `blockages` - Indicates that blockages should be processed. If a blockage is attached to an instance that does not exist in the in-memory database, then it is ignored.
- `boundary` - Indicates that the design boundary information, including rows and tracks, should be updated.
- `fixed_core_insts` - Indicates that any CLASS CORE instances that have placement status `fixed` or `cover` should be updated. Physical-only core instances may be added to the database if the placement status is `fixed` or `cover`.
- `floorplan` - Is equivalent to specifying "`block_insts blockages boundary fixed_core_insts pad_insts pin_shapes regions special_routing`"
- `pad_insts` - Indicates that any CLASS PAD instances should be updated. Physical-only pad instances may be added to the database if the placement status is `fixed` or `cover`.
- `pin_shapes` - Indicates that pin shapes should be read.
- `regions` - Indicates regions should be updated.
- `regular_routing` - Indicates that nets should be processed and regular routing (and associated shield net wiring) and net constraints updated.
- `special_routing` - Indicates that nets should be processed and special routing (except shield nets wiring) should be read.

Use `read_oa -filter` to read floorplanning information from a cellview whose connectivity does not match the current in-memory connectivity of the tool. The `-filter` option allows you to load some data selectively, while not loading others. When `-filter` is used, error checking for database consistency (nets, instances, terminals, etc.) is disabled. While similar to Virtuoso Load Physical View (LPV), the `-filter` option is only a subset and not intended to be a complete

	replacement.
`-nets {`*`netnames`*`}`	
	Specifies the list of nets whose routing (routing and special) and constraints have to be read with the `-filter` option. Shielding wiring/vias are also read in if they are used to shield any of the specified nets. If this option is used, typically `regular_routing` is not specified as part of the `-filter` objects.

Example

- The following command restores data using OpenAccess:

```
read_oa design amba_usb layout
```

Related Information

- `read_db`
- `oa_read_write` Category Attributes
- `write_db`
- *Mixed Signal (MS) Interoperability Guide*

register_trigger

```
register_trigger
[-help]
[{-pre | -post}
command_name
proc_name]
```

Registers the `-pre` or `-post` callback procedures with the specified commands.

The procedure registered with `-pre` parameter runs before the *`command_name`* parameter is used while the procedure registered with `-post` parameter runs after the *`command_name`* parameter is used.

Note: The arguments passed to *`command_name`* are passed to *`proc_name`*. If the procedure registered with the `-pre` parameter returns a `TCL_ERROR`, then *`command_name`* is not run.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `register_trigger` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man register_trigger`.
`-pre`	Specifies the procedure to be called before the command.
`-post`	Specifies the procedure to be called after the command.
`command_name`	Specifies the command name. The supported command names are `read_db`, `write_db`, `read_oa`, and `write_oa`.
`proc_name`	Specifies the procedure name that is to be registered as callback.

Related Information

- `read_db`
- `read_oa`
- `write_db`

unlock_oa_design

```
unlock_oa_design
[-help]
oa_lib
oa_cell
oa_view
```

Unlocks the specified OpenAccess design.

A design cannot be opened if `setOaxMode -locking` is set to `true`, and the design is currently locked. You can use the `unlock_oa_design` command to free the lock so that other tools can access and edit the database. You also can use the command to override the lock, if the design is incorrectly locked.

You can use the `unlock_oa_design` command at any time.

Parameters

oa_cell	Specifies the name of the top oa_cell to unlock.
-help	Outputs a brief description that includes type and default information for each `unlock_oa_design` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man unlock_oa_design`.
oa_lib	Specifies the name of the OpenAccess oa_library in which the design was saved.
oa_view	Specifies the name of the oa_cell oa_view to unlock.

Example

- The following command unlocks the oa_view `layout` in the oa_cell `dtmf_chip` in the oa_library `designLib`:

  ```
  unlock_oa_design designLib dtmf_chip layout
  ```

Related Information

- `oa_read_write` Category Attributes
  - `oa_lock`

update_oa_lib

```
update_oa_lib
[-help]
lib_name
[-compress_level value]
[-create_cdsinfo_tag]
[-tech_attach_to_reference]
```

Updates the specified OpenAccess library from within Innovus. You can use this command to:

- Converts the specified library to use a tech reference from the attached tech so that the non-parameterized or custom vias can be written to it. If an OpenAccess design has non-parameterized vias, you should run `update_oa_lib` before invoking `write_db`. The tool is then able to save the design and does not throw an error because it can now write non-parameterized vias to the converted library.
- Add a cdsinfo.tag to legacy libraries created by earlier versions of Innovus.
- Modify the compression level for the library.

If you run the `update_oa_lib` command on a library that is not in attached mode, a warning is displayed. However, the script continues to run. This is useful if you want to make sure that libraries are writable for new vias and the `write_db` command does not fail.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `update_oa_lib` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man update_oa_lib`.
`-compress_level` *value*	Sets the compression level of the specified library. Cellviews written to the library follow the specified compression level. However, existing cellviews in the library are not modified. Use the oazip utility to modify the existing cellviews in the library. Valid compression level values range from 0 to 9. A value of 0 indicates that compression is turned off.
`-create_cdsinfo_tag`	Adds a `cdsinfo.tag` file to the specified library.
lib_name	Specifies the name of the library to be updated.
`-tech_attach_to_reference`	Converts the specified library to use a tech reference from the attached tech so that new custom via definitions can be added to the library.

Examples

- The following command sets the compression level of the `lib1` library to 9. However, it does not change the compression of the existing cellviews in the library:

  ```
  update_oa_lib lib1 -compress_level 9
  ```

  The tool displays warning such as the following:

  ```
  Changed library compressLevel from '0 (uncompressed)' to '9'. Cellviews within the library are not affected by changing the library's compressLevel. To update the cellviews, use the oazip utility.
  ```

- The following command decompresses the `my_lib` library, which previously is at compression level `1`:

  ```
  update_oa_lib my_lib -compress_level 0
  ```

Related Information

- `oa_read_write` Category Attributes

- *Mixed Signal (MS) Interoperability Guide*

write_lef_library

```
write_lef_library
[-help]
[lef_file]
[-lef_layer_order]
[-all_auto_generated_via | -macro_only]
[-macro_only [-no_property]]
[-tech_only | -macro_only]
[-tech_only | -macro_list string]
[-use_spacing_table_default]
```

Writes the LEF library data (composed of technology data and macros) that is present in the tool's database.

The `write_lef_library` command facilitates comparisons between LEF-based input and OpenAccess-based input. LEF allows cases where `VIARULE GENERATE` has `WIDTH` specified for either the top or bottom layer. However, such cases are not supported by the OpenAccess tech. To make the comparison between LEF-based and OpenAccess-based input easier, `write_lef_library` dumps `"WIDTH 0 TO 1000 ;"` for any `VIARULE GENERATE` that does not have WIDTH specified in the original LEF.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_lef_library` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_lef_library`.
`lef_file`	Specifies the output LEF file name. *Data_type*: string, optional
`-all_auto_generated_via`	
	Includes all auto-generated vias. *Data_type*: bool, optional
`-lef_layer_order`	Writes out only the layers specified in the lefLayerOrder constraint. *Data_type*: bool, optional

`-macro_list` *string*	
	Writes the macros specified in the macro list. *Data_type*: string, optional
`-macro_only`	Writes only the macro data to the LEF file. *Data_type*: bool, optional
`-no_property`	Writes data in the LEF file without any properties. This option can be used only with the `-macro_only` option. *Data_type*: bool, optional
`-tech_only`	Writes only the technology data to the LEF file. *Data_type*: bool, optional
`-use_spacing_table_default`	
	Replaces any spacing value that matches the default spacing value with the "`-`" character in the LEF file. When reading spacing tables from a LEF file, `lef2oa` replaces any "`-`" table entry with the default spacing value. The spacing table in the OpenAccess version therefore contains the default values for all such entries. By default, `write_lef_library` writes out the actual values in the spacing table in the LEF file. When `-use_spacing_table_default` is specified, `write_lef_library` replaces any spacing value that matches the default value with the "`-`" character in the LEF file. This makes it easier for you to compare the LEF file generated by `write_lef_library` with the original LEF file. *Data_type*: bool, optional

Example

- The following command writes the technology data into the `tech.lef` file:
  ```
  write_lef_library tech.lef -tech_only
  ```

  You can set the LEF version to 5.7 or 5.8 using `write_def_lef_out_version`:
  ```
  set_db write_def_lef_out_version 5.8
  ```

Related Information

- write_oa_techfile
- *Mixed Signal (MS) Interoperability Guide*

write_oa_techfile

```
write_oa_techfile
[-help]
[out_file]
[-include_rules]
[-process_node coord]
```

Writes out a DFII style ASCII technology file that can be given to the `techLoadDump` executable to create an equivalent OpenAccess technology database.

This command is used to create and verify OpenAccess technology files that match LEF technology files. By default, `write_oa_techfile` writes out only DRC information and no LDRS or VIA information.

The expected flow for new processes is as follows:

1. Read a LEF file into the tool with `init_design`.
2. Use `write_oa_techfile` to write out the tech (`.tf`) file.
3. Run `techLoadDump` using the `.tf` file.
4. Use `lef2oa -pnrLibDataOnly` to create the LDRS and vias.

For more information, refer to the chapter "Technology File Layer Attributes" of the *Virtuoso Technology Data ASCII Files Reference*.

Parameter

`-help`	Outputs a brief description that includes type and default information for each `write_oa_techfile` parameter. For a detailed description of the command and all of its parameters, use the man command: man `write_oa_techfile`.
`out_file`	Specifies the output ASCII file name.
`-include_rules`	Includes LDRS and VIA information in the output tech file. This option should be used only if the LEF default and NDRs do not utilize fixed vias.
`-process_node` *`coord`*	Specifies a process node value in nm. When specified, `write_oa_techfile` creates a processNode entry in the techfile for processes using the MASK construct. For example, if you specify `-process_node 20` to indicate a 20nm process, the following entry is included in the control() section of the techfile: `processNode(0.020)` If `-process_node` is not specified and the `write_oa_techfile` command detects MASK constructs on any layer, it returns an error.

Examples

- The following set of commands updates an OpenAccess technology file with any missing information from a LEF technology file:

```
#Initialize a design using LEF files
write_lef_library -tech_only lef.lef  #A LEF file in canonical
                                      #sorted order for easy
                                      #comparison with diff
write_oa_techfile lef.tf              #A DFII ASCII tech file
                                      #that can be given to
                                      #techLoadDump executable
#Edit existing OA tech file to add any missing information in
#the lef.tf file
#Initialize a design using the new OA tech. Then:
write_lef_library -tech_only oa.lef #A LEF file in canonical
                                    #order derived from OA tech
#Compare the two LEF files. Any differences need to be fixed
#or explained.
diff lef.lef oa.lef
```

Related Information

- write_lef_library
- *Mixed Signal (MS) Interoperability Guide*

18

Machine Learning Commands

- init_ml
- write_ml_design
- write_ml_library
- write_ml_timing_graph
- write_ml_timing_path

init_ml

```
init_ml
[-help]
[-comment string]
[-flow_phase string]
[-name string]
-prefix string
[-scenario string]
```

Initializes the environment to start writing data for JedAI. Run this command before using any of the following commands to ensure that content is properly labeled for big data ingestion and registration:

- `write_ml_design`
- `write_ml_library`
- `write_ml_timing_graph`
- `write_ml_timing_path`

Parameters

`-help`	Prints a brief description that includes the type and default information for each `init_ml` parameter. For a detailed description of the command and all of its parameters, use the man command `man init_ml`.
`-comment` *string*	Specifies a comment. *Data_type*: string, optional
`-flow_phase` *string*	Specifies the flow phase. *Data_type*: string, optional
`-name` *string*	Specifies the design name. *Data_type*: string, optional
`-prefix` *string*	Specifies the prefix for the output files. *Data_type*: string, required
`-scenario` *string*	Specifies the scenario. *Data_type*: string, optional

Example

- The following command initializes the environment to start writing data for JedAI for the `cts` flow phase with the `default` scenario for the `dtmf` design.

```
init_ml -prefix out/ -name dtmf -flow_phase cts -scenario default -comment "This is a demo."
```

Related Information

- write_ml_design
- write_ml_library
- write_ml_timing_graph
- write_ml_timing_path

write_ml_design

```
write_ml_design
[-help]
[-no_power]
[-objects object_list]
-prefix string
[-verbose]
[-views view_list]
```

Writes design-related data for JedAI to csv files with specified prefix. It includes data that is design specific and relates to logical, physical, and timing type objects.

Before running `write_ml_design`, the following two commands should be run:

- `report_timing`
- `write_global_slack_report -include_ports`

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_ml_design` parameter. For a detailed description of the command and all of its parameters, use the man command `man write_ml_design`.
`-no_power`	Specifies that power attributes should not be dumped. *Data_type*: bool, optional
`-objects` *object_list*	

	Specifies the list of db object(s) to be dumped, which could include one or more of the following values:
	`analysis_view arc base_cell base_pin blockage clock congestion constraint_mode`
	`delay_corner design gcell_grid hinst hnet hpin hport inst layer marker net netlist`
	`opcond option p2p partition patch_wire pg_base_pin pg_pin pin pin_shape port`
	`rc_corner route_rule row site skew_group special_net timing_condition track_pattern unit_parasitic via_def`
	Use curly braces to enclose multiple values.
	Data_type: tcl_obj, optional

`-prefix` *string*	Specifies the prefix for the output files. *Data_type*: string, required
`-verbose`	Displays a more detailed message. *Data_type*: bool, optional
`-views` *view_list*	
	Dumps the data for the specified analysis view(s). *Data_type*: tcl_obj, optional

Example

- The following command writes the attributes of all design objects to csv files with the specified prefix:

```
write_ml_design -prefix out/
```

Related Information

- `init_ml`
- `report_timing`
- `write_global_slack_report`
- `write_ml_library`
- `write_ml_timing_graph`
- `write_ml_timing_path`

write_ml_library

```
write_ml_library
[-help]
[-objects object_list]
-prefix string
```

Writes library-related data for JedAI to csv files with the specified prefix. It includes data that is library specific and relates to lib_cells, timing_tables, and power_tables.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_ml_library` parameter. For a detailed description of the command and all of its parameters, use the man command `man write_ml_library`.
`-objects object_list`	
	Specifies the list of db object(s) to be dumped, which could include one or more of the following values: `lib_arc lib_cell lib_feature lib_pin library library_set power_table timing_table` Use curly braces to enclose multiple values. *Data_type*: tcl_obj, optional
`-prefix string`	Specifies the prefix for the output files. *Data_type*: string, required

Example

- The following command writes library-related data for JedAI to csv files with the specified prefix:

  ```
  write_ml_library -prefix out/
  ```

Related Information

- init_ml
- write_ml_design
- write_ml_timing_graph
- write_ml_timing_path

write_ml_timing_graph

```
write_ml_timing_graph
[-help]
[-noedge]
-prefix string
[-register_only]
[-views view_list]
```

Writes timing-related data for JedAI to csv files. It includes data representing the complete timing graph for a given set of analysis views.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_ml_timing_graph` parameter. For a detailed description of the command and all of its parameters, use the man command `man write_ml_timing_graph`.
`-noedge`	Specifies that edge data should not be dumped. This option can be used to save time. *Data_type*: int, optional
`-prefix` *string*	Specifies the prefix for the output files. *Data_type*: string, required
`-register_only`	Specifies that only nodes for register should be dumped. *Data_type*: bool, optional
`-views` *view_list*	Dumps the data for the specified analysis view(s). *Data_type*: tcl_obj, optional

Example

- The following command writes the timing-related data for JedAI to csv files with the specified prefix:

```
write_ml_timing_graph -prefix out/
```

Related Information

- init_ml
- write_ml_design
- write_ml_library
- write_ml_timing_path

write_ml_timing_path

```
write_ml_timing_path
[-help]
[-anchor_file string]
[-begin_end_pair]
[-chain_levels int]
[-chain_path]
[-each_endpoint]
[-from pin_list]
[-gen_anchor]
[-hpin]
[-io_only]
[-max_paths int]
[-max_slack float]
[-nworst int]
[-partitions partition_list]
[-path_group string]
[-per_partition]
-prefix string
[-retime {aocv ssta path_slew_propagation aocv_path_slew_propagation}]
[-retime_mode {path exhaustive}]
[-start_pathid int]
[-to pin_list]
[-verbose]
[-views view_list]
```

Writes timing-related data for JedAI to csv files. It includes data for individual timing paths. Timing path data can be computed in either graph-based analysis (GBA) or path-based analysis (PBA) fashion using the `-retime` option.

Parameters

`-help`	Prints a brief description that includes the type and default information for each `write_ml_timing_path` parameter. For a detailed description of the command and all of its parameters, use the man command `man write_ml_timing_path`.

`-anchor_file` *string*	Specifies the name of the anchor file to be used as input. *Data_type*: string, optional
`-begin_end_pair`	Reports all violations between unique source and target register pairs. *Data_type*: bool, optional
`-chain_levels` *int*	Specifies the chain level if `-chain_path` is enabled. *Data_type*: int, optional
`-chain_path`	Gets chained paths. *Data_type*: bool, optional
`-each_endpoint`	Specifies whether to dump at least one path for each end point. *Data_type*: bool, optional
`-from` *pin_list*	Dumps data from the specified pin(s). *Data_type*: tcl_obj, optional
`-gen_anchor`	Generates the anchor file. *Data_type*: bool, optional
`-hpin`	Dumps hpin in timing points. *Data_type*: bool, optional
`-io_only`	Specifies that only the io path should be dumped. *Data_type*: bool, optional
`-max_paths` *int*	Specifies the max_paths for `report_timing`. *Data_type*: int, optional
`-max_slack` *float*	Specifies the max_slack for `report_timing`. *Data_type*: float, optional
`-nworst` *int*	Specifies the nworst for `report_timing`. *Default*: `100` *Data_type*: int, optional

`-partitions` *partition_list*	
	Specifies a partition list if `-per_partition` is set. *Data_type*: tcl_obj, optional
`-path_group` *string*	Specifies the path_group for `report_timing`. *Data_type*: string, optional
`-per_partition`	Specifies that only paths through top level partitions should be dumped. *Data_type*: bool, optional
`-prefix` *string*	Specifies the prefix for the output files. *Data_type*: string, required
`-retime {aocv ssta path_slew_propagation aocv_path_slew_propagation}`	
	Specifies the method to be used for reanalysis of specified set of paths with `report_timing`. *Data_type*: enum, optional
`-retime_mode {path exhaustive}`	
	Passes the specified retime mode to `report_timing`. To understand the difference between the two retime modes, `path` and `exhaustive`, see `report_timing`. *Data_type*: enum, optional
`-start_pathid` *int*	Specifies the starting path id. *Default*: `1` *Data_type*: int, optional
`-to` *pin_list*	Dumps data to the specified pin(s). *Data_type*: tcl_obj, optional
`-verbose`	Displays a more detailed message. *Data_type*: bool, optional

`-views` *view_list*	Dumps the timing path for the specified analysis view(s). *Data_type*: tcl_obj, optional

Example

- The following command writes out the attributes of timing paths to csv files with the specified prefix:

```
write_ml_timing_path -prefix out/ -max_paths 10000 -nworst 1
```

Related Information

- `init_ml`
- `report_timing`
- `write_ml_design`
- `write_ml_library`
- `write_ml_timing_graph`

19

Placement Commands and Attributes

- add_fillers Category Attributes
- gui_clear_scan_chains
- gui_clear_spare_cells
- gui_show_scan_chains
- gui_show_spare_cells
- place Category Attributes
- report_congestion
- set_cell_edge_spacing
- set_cell_edge_type
- report_cell_edge_type
- report_density_map
- report_pin_density_map
- write_physical_context_data
- read_physical_context_data
- reset_physical_context_data

add_fillers Category Attributes

The root attributes in the add_fillers category are:

- add_fillers_cells
- add_fillers_check_trim_rule
- add_fillers_double_height_filler_insertion

- add_fillers_enable_restore_filler_flow
- add_fillers_prefix
- vt_swap_out_dir
- vt_swap_out_file
- vt_swap_to_fix_filler_violations

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `add_fillers` category, use the following command:

```
get_db -category add_fillers
```

add_fillers_cells

```
add_fillers_cells {{list-of-cells1} {list-of-cells2} ...}
```

Default: `""`

Data_type: `string`

Specifies the filler cells to add with add_fillers. Enclose the filler cell names in quotation marks (") or curly braces ({ }). You can specify multiple cell lists at one time, using curly braces ({ }) to separate each one.

add_fillers_check_trim_rule

```
add_fillers_check_trim_rule {true | false}
```

Default: `false`

Data_type: bool, read/write

Controls trim rule checking during the filler insertion. When set to true, the trim rule is checked, and runtime increases.

add_fillers_double_height_filler_insertion

```
add_fillers_double_height_filler_insertion {true | false}
```

Default: `false`

Data_type: bool, read/write

Enables double height filler during filler insertion.

add_fillers_enable_restore_filler_flow

```
add_fillers_enable_restore_filler_flow {true | false}
```

Default: `true`

Data_type: bool, read/write

When this option is set to `false`, fillers are not saved in the memory (which is default i.e the save and restore filler insertion flow is default ON).

add_fillers_prefix

```
add_fillers_prefix prefix
```

Default: `""`

Data_type: `string`

Specifies the prefix for the added filler cells.

vt_swap_out_dir

```
vt_swap_out_dir vtSwapReportFileDirectory
```

Default: Current directroy ./

Data_type: string, read/write

Specifies the directory of the vt swap report file,

vt_swap_out_file

```
vt_swap_out_file fileName
```

Default: `""`

Data_type: string, read/write

Specifies the file name of the vt swap report.

vt_swap_to_fix_filler_violations

```
vt_swap_to_fix_filler_violations {true | false}
```

Default: `false`

Data_type: string, read/write

Invokes the swap cell to fix the filler violation.

gui_clear_scan_chains

```
gui_clear_scan_chains
[-help]
[scanPartitionName]
```

Removes scan chains from the display.

Parameters

scanPartitionName	Specifies the scan partition name to remove from the display.

Related Topic

- gui_show_scan_chains

gui_clear_spare_cells

`gui_clear_spare_cells`

Removes spare cell instances from the display.

Parameters

None

Related Topic

- `gui_show_spare_cells`

gui_show_scan_chains

```
gui_show_scan_chains
[-help]
[scan_chain_name]
```

Displays a scan chain. You must load the scan information, place the design, and view the design in the Placement view. Use this command after scan chains are traced.

Parameters

scan_chain_name	Specifies the scan chain. *Default:* If you do not specify this parameter, the software displays all scan chains.

Examples

- The following command displays all scan chains in the design:
  ```
  gui_show_scan_chains
  ```
- The following command displays the scan chain whose name is `chain0`.
  ```
  gui_show_scan_chains chain0
  ```

gui_show_spare_cells

gui_show_spare_cells

Displays the results of placing the spare instances. You must specify the spare cell types or spare instances before placing the design.

Parameters

None

Examples

The following set of commands displays all spare instances defined by the set_spare_insts command:

```
specifySpareGate -cell ADDFXLT1T10
placeDesign
gui_show_spare_cells
```

Related Topic

- gui_clear_spare_cells

place Category Attributes

The following types of place category attributes are covered in this chapter:

- place_3d_use_existing_bump_tsv
- place_cell_edge_spacing
- place_check_report_fixed_inst_on_blockage
- place_design_enable_3d
- place_design_integrity_ir_fix_effort
- place_design_refine_macro
- place_detail_allow_border_pin_abut
- place_detail_allow_single_height_row_symmetry_x
- place_detail_check_route
- place_detail_color_aware_legal
- place_detail_context_aware_legal
- place_detail_honor_inst_pad
- place_detail_ir_aware_max_drive_strength
- place_detail_pad_fixed_insts
- place_detail_swap_eeq_cells
- place_detail_m3_stripe_push_down
- place_detail_m3_stripe_shrink
- place_detail_use_diffusion_transition_fill
- place_detail_use_gate_array_filler_groups
- place_detail_pgftv_insertion_cell_list
- place_global_adaptive_subd1

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `place` category, use the following command:

```
get_db -category place
```

place_3d_use_existing_bump_tsv

```
place_3d_use_existing_bump_tsv {true|false}
```

Default: `false`

Data_type: `string, read-only`

Enables 3D placement to use preplaced but unassigned bump/TSV in the design for signal net assignment. The preplaced bump/TSV must be the specified cells for 3D concurrent bump/TSV insertion and legalization.

Note: When you set this option to `true`, you need to ensure that enough bump/TSV is preplaced and the corresponding bump grid is created in each die before invoking the 3D placement.

place_cell_edge_spacing

```
place_cell_edge_spacing {cell_edge_type1 cell_edge_type2 spacing}
```

Default: `""`

Data_type: `string`

Specifies the list of cell edge-type spacing values from `LEF CELLEDGESPACINGTABLE` or OA techfile that affects the placer. It specifies the min-spacing between different edge-types of std-cells, in a list format: `{cell_edge_type1 cell_edge_type2 spacing}`. The edge-type names match the base_cell's right_edge_type and left_edge_type attribute values.

place_check_report_fixed_inst_on_blockage

```
place_check_report_fixed_inst_on_blockage {true | false}
```

Default: `false`

Data_type: bool, read/write

When `true, check_place` reports placement blockage violations for fixed instances under placement blockages.

place_design_enable_3d

`place_design_enable_3d {true | false}`

Default: `false`

Data_type: `bool, read/write`

Enables 3D placement in Integrity 3D-IC platform. You need to set this to true before invoking concurrent macro placement (`place_design -concurrent_macros`), standard cell placement (`place_opt_design -place`), and bump/TSV insertion (`place_bump_tsv_detail`) in a 3D design.

place_design_integrity_ir_fix_effort

`place_design_integrity_ir_fix_effort {low|medium|high}`

Default: `medium`

Data_type: `enum, read/write`

Sets the power integrity placement IR fixing effort.

place_design_refine_macro

`place_design_refine_macro {true | false}`

Default: `false`

Data_type: bool, read/write

Specifies whether to run `place_macro_detail` after `place_design -concurrent_macros`. When `place_design -concurrent_macros` is run, it sets the value of `place_opt_run_global_place` attribute to `seed`.

place_detail_allow_border_pin_abut

`place_detail_allow_border_pin_abut {true | false}`

Default: `false`

Data_type: `bool`

Some libraries have cells with pin geometry that is not set back by half-routing pitch from the normal PR-boundary of that cell. In cases where the pin geometries touch the cell PR_boundary,

placement of an instance of this cell can cause a DRV violation with a normal pin geometry of the other cells. To avoid these DRVs, placer will add one Site padding to the cell-side during pre-processing, thus satisfying the min-spacing rules.
The `place_detail_allow_border_pin_abut` attribute disables this pre-emptive DRV avoidance behavior, and no auto-padding is done when the option is set to `true`.

place_detail_allow_single_height_row_symmetry_x

`place_detail_allow_single_height_row_symmetry_x` *siteName*

Default: `""`

Data_type: bool, read/write

Specifies a single height row tech-site, which allows symmetry X during detail place.

place_detail_check_route

`place_detail_check_route {true | false}`

Default: `false`

Data_type: `bool`

Considers pre-routed FIXED signal wires and avoids creating DRC violations between FIXED signal wires and instance pins. When this attribute is not specified, `place_detail` ignores FIXED signal wires.

Note: Using this attribute might result in longer run time.

If you specify `place_detail -io_pin_blockage`, you do not need to specify `-check_route` for signal pins because `place_detail` will treat the signal pins similar to supply pins for blockages. However, signal wires still need this attribute.

place_detail_color_aware_legal

`place_detail_color_aware_legal {true | false}`

Default: `false`

Data_type: `bool`

Enables placer to honor and fix double pattern constraints between adjacent cells.

Note: This attribute does not affect cell to pre-route color DRC checks.

place_detail_context_aware_legal

`place_detail_context_aware_legal {none|all|optional|required|user|ignore_soft}`

Default: `all`

Data_type: `enum* {none all optional required user ignore_soft}`

Instructs placer to support cell edge to cell edge spacing constraints as defined in the LEF files.

The `optional`, `required`, `user` argument values can be used separately or together, but they are exclusive to `all` and `none`.

place_detail_honor_inst_pad

`place_detail_honor_inst_pad {true | false}`

Default: `false`

Data_type: `bool`

Honors padding from `set_inst_padding` in detail placement.

place_detail_ir_aware_max_drive_strength

`place_detail_ir_aware_max_drive_strength` *`maxDriveStrength`*

Default: `0.0`

Data_type: `double, read/write`

Specifies the current density value for the IR critical instances above which the instances gets padded during `place_detail`. This means that the IR critical instances with the current density value more than the specified value gets padded.

Note: There may be larger cell movement with this option, which can degrade the timing. So, it is preferred to be used at the pre-CTS stage in the flow. The current density values for the IR-critical instances are dumped in the Voltus Reports.

place_detail_pad_fixed_insts

`place_detail_pad_fixed_insts {true | false}`

Default: `false`

Data_type: `bool`

Specifies whether the `place_detail` command honors cell padding on `FIXED` instances.

place_detail_swap_eeq_cells

```
place_detail_swap_eeq_cells {true | false}
```

Default: `false`

Data_type: `bool`

Determines whether master cells can be replaced by EEQ cells during detailed placement to improve routability. After detailed placement, the software reports the number of replacements. EEQ cells are defined in the LEF file.

place_detail_m3_stripe_push_down

```
place_detail_m3_stripe_shrink site_units
```

Default: `-1`

Data_type: `int`

Specifies the width threshold in site units for virtual push down of M3 stripe to M2 for M1 pin-access.

If the default value is set to `-1`, it indicates that no M3 stripe will be pushed down. If the default value is set to `0`, it indicates that all M3 stripes will be pushed down. If the default value is larger than `0`, then all M3 stripes having width greater than the specified value will be pushed down.

place_detail_m3_stripe_shrink

```
place_detail_m3_stripe_shrink site_units
```

Default: `0`

Data_type: `int`

Specifies the shrink effort of the virtual M2 stripe by given total site units. This global should work with place_detail_m3_stripe_push_down.

If the default value is set to `0`, it indicates that the entire pushed down M3 stripe is used to evaluate M1 pin access. If the default value is set to more than `0`, it indicates that M3 stripe will shrink by the specified value before it is pushed down to M2. The left side will shrink first. For example, if the

specified value is set to `1`, the M3 stripe will shrink by 1 site on left, and if the specified value is set to `2`, 1 site on left will shrink, along with 1 site on right.

place_detail_use_diffusion_transition_fill

```
place_detail_use_diffusion_transition_fill {true | false}
```

Default: `false`

Data_type: `bool`

Specifies that the design has fillers with jogged diffusion obs. Use this attribute for designs where fillers with jogged diffusion obs are to be inserted between cells with varying diffusion widths.

place_detail_use_gate_array_filler_groups

```
place_detail_use_gate_array_filler_groups {true | false}
```

Default: `false`

Data_type: `bool`

Specifies that the filler cell instances will only be replaced by logic instances that belong to same GA groups.

place_detail_pgftv_insertion_cell_list

```
place_detail_pgftv_insertion_cell_list listOfCells
```

Default: `""`

Data_type: string, read/write

Specifies the list of cells as PG FTV cells.

place_global_adaptive_subd1

```
place_global_adaptive_subd1 {true|false}
```

Default: `false`

Data_type: bool, read/write

When set to true, placement will call Opt API to get a list of cells not usable for subD1 and disable

the usage of cells in the placement engine.

`place_global_adaptive_subd1` can be used independently to get the list of subD1 cells.

report_congestion

```
report_congestion
[-help]
[-3d]
[-hotspot]
[-include_blockage]
[-num_hotspot number]
[-overflow]
```

Reports the early global route congestion and the local hotspot score.

Route needs be called before using this command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_congestion` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_congestion`.
`-3d`	Honors 3d congestion map. When specified, it enables the layer-based congestion reporting.
`-hotspot`	Reports the local hotspot score.
`-include_blockage`	Includes blockage area in hotspot.
`-num_hotspot number`	Specifies the number of hotspots to be reported.
`-overflow`	Reports horizontal and vertical overflow.

- The following command reports the local hotspot score:

  ```
  report_congestion -hotspot
  ```

  Output:

```
Local HotSpot Analysis: normalized max congestion hotspot area = 0.00, normalized
total congestion hotspot area = 0.00 (area is in unit of 4 std-cell row bins)
```

set_cell_edge_spacing

```
set_cell_edge_spacing
[-help]
{<edgeType1> <edgeType2>
-spacing spacing
[-except_exact_spacing distance]
[-except_abutted | -forbidden_spacing ][-except_non_filler_in_between] } | [-reset]
```

Creates the cell edge spacing rule as in `CELLEDGESPACINGTABLE` in LEF.

> The tool by default supports the latest Tcl settings. If a user tries to change the edge spacing, the tool will generate a warning message that resetting edge spacing would overwrite the old value.

Note: This command is not supported for save and restore design.

Parameters

`-except_abutted`	Specifies that two abutted macros of the given edge types are also allowed.
`-except_non_filler_in_between`	Specifies that the edge spacing is only for adjacent instances. If two MACROs (cells) have a edge-spacing rule, say 0.8 micron, then normally the rule applies regardless of presence of another macro (cell) in the gap between. When the `-except_non_filler_in_between` option is used, the rule is exempted when a non-filler cell is present between the two constrained cell edges.
`-except_exact_spacing` *`distance`*	Specifies an exception distance to the minimum spacing. *Data_type*: double, optional
`edgeType1`	Specifies edge `edgeType1` for edge spacing.
`edgeType2`	Specifies edge `edgeType2` for edge spacing.
`-forbidden_spacing`	Forbids `edgeType1` and `edgeType2` without the space.
`-help`	Outputs a brief description that includes type and default information for each `set_cell_edge_spacing` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_cell_edge_spacing`
`-reset`	Resets the cell edge spacing to LEF.
`-spacing` *`spacing`*	Specifies edge spacing between *`edgeType1`* and *`edgeType2`*. If there is no spacing defined between two edge types, it indicates that there is no spacing constraint (or 0.0 µm) between them.

Example

The following command creates `0.18 µm` spacing between `grp1` and `grp2`:

```
set_cell_edge_spacing grp1 grp2 0.18
```

set_cell_edge_type

```
set_cell_edge_type
[-help]
-cell cellName
{-left edgeType | -right edgeType | -bottom edgeType | -reset}
[{-left edgeType | -right edgeType} [-cell_row rowNumber]]
[{-left edgeType | -right edgeType} [-half {top | bottom}]]
[{-top edgeType | -bottom edgeType} [-range {x y}]] [{-cell cellName} [-reset ]]
```

Specifies edge type for cell edges as `EDGETYPE` in macro LEF. It allows to specify one edge at a time for a particular cell.

To ensure the correct edge type specification, edge types asserted by `set_cell_edge_type` command should be pre-defined by the `set_cell_edge_spacing` command.

For example:

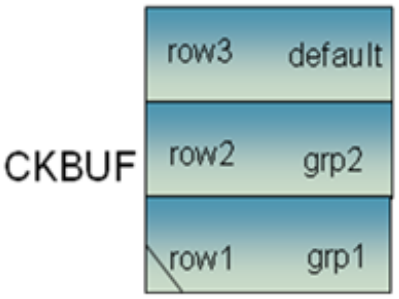
```
set_cell_edge_spacing grp1 grp2 0.18
set_cell_edge_type -cell BUF1X -left grp1
```

If the edge type is specified without corresponding spacing rule, tool gives a warning and ignores the edge type specification.

Note: This command is not supported for save and restore design.

Parameters

`-bottom edgeType`	Specifies an edge type on the bottom side.
`-cell cellName`	Specifies cell to which to add edge constraint.

`-cell_row` *rowNumber*	Specifies the row number for horizontal edge type.
	You can define different edge types for each row of multiple height cells using `-cell_row`.
	For example, there is triple height cell CKBUF.
	 
	The following commands define different edge types for different rows of the CKBUF cell.
	`set_cell_edge_type -cell CKBUF -right grp1 -cell_row 1`
	`set_cell_edge_type -cell CKBUF -right grp2 -cell_row 2`
	In the above example, grp1 is applied to the first row (bottom row) of the right edge, grp2 is applied to the second row of right edge, and no specific edge type is defined for the third row of right edge and is of DEFAULT type.
	The legal value of `-cell_row` is from 1 to n, where n is the number of cell height of the cell.

`-help`	Outputs a brief description that includes type and default information for each `set_cell_edge_type` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_cell_edge_type`
`-half {top bottom}`	Defines the edge types corresponding to the top or bottom half of the cell. The following example associates the edge type 'grp1' with the bottom row, top half of the multi-height cell SDFF8: `set_cell_edge_type -cell SDFF8 -right grp1 -cell_row 1 -half top`
`-left` *edgeType*	Specifies an edge type on the left side.
`-range {x y}`	Specifies the range for the vertical edge type.

`-reset`	Resets the specified cell constraint to lef status.
`-right` *edgeType*	Specifies an edge type on the right side.
`-top` *edgeType*	Specifies an edge type on the top side.

Example

The following commands specify one edge at a time for the `BUF1X` cell

- `set_cell_edge_type -cell BUF1X -left grp1`
- `set_cell_edge_type -cell BUF1X -right grp1 -cell_row 1`
- `set_cell_edge_type -cell BUF1X -right grp1 -cell_row 2`

Related Topic

- `set_cell_edge_spacing`

report_cell_edge_type

```
report_cell_edge_type
[-help]
[-out_file cellName]
[-file fileName]
```

Reports the edge type of the specified cell.

Parameters

`-cell` *cellName*	Specifies the cell name for which edge type is to be reported. If the cell name is not specified, all the cell edge types are reported in the edge_type.rpt file report file.
`-out_file` *fileName*	Specifies the file name to dump cell edge type(s) . If no file name is specified, the edge_type.rpt file is used to generate the report.
`-help`	Outputs a brief description that includes type and default information for each `report_cell_edge_type` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_cell_edge_type`.

Related Topic

- `set_cell_edge_type`

report_density_map

```
report_density_map
[-help]
[-grid_length rowHeight]
[-grid_unit microns]
[-ignore_block {true|false}]
[-ignore_filler {true|false}]
[-threshold density]
```

Generates a map that uses colors to represent placement density and generates a placement density report. If you run this command without parameters, it generates a map with a threshold of 0.75 and measures the grid in row height.

Use this command any time after placement.

Use the Display Density Map form to select colors and step-size for the density levels in the map.

Parameters

`-grid_unit` *microns*	Specifies the value (in micron) of side of square grid for density calculations.
`-grid_length` *rowHeight*	Specifies the value (in row-height) of side of square grid for pin density calculations.
`-help`	Outputs a brief description that includes type and default information for each `report_density_map` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man report_density_map`.
`-ignore_block {true|false}`	Skips block cells instances from both supply and demand calculation.
`-ignore_filler { true | false}`	Ignores the density of fillers in the post-filler density heat map to show density of standard cells. *Default*: `false`
`-threshold` *density*	Specifies a threshold density value for the grid. Any grid under the threshold value is not reported. The value must be positive.

Example

The following are output from command `report_density_map`. It generates a map with default threshold of 0.75 and measured in row height:

```
powerDomain TOP: bins with density > 0.75 = 0.519 % ( 6 / 1156 )

bin size: 59 sites by 10 row(s). total 1156 bins ( 34 by 34 )

density range #bins %

0.75 - 0.8 5 0.4325

0.8 - 0.85 1 0.08651

total 6 0.519
```

report_pin_density_map

```
report_pin_density_map
[-help]
[-step step_value]
[-grid_unit micron]
[-grid_length numberRow]
[-threshold density]
```

Generates a map that uses colors to represent pin density and generates a pin density report. Use this command any time after placement.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_pin_density_map` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_pin_density_map`
`-step` *step_value*	Specifies the range step for displaying the pin density map. *Default*: `0.05`
`-grid_unit` *micron*	Specifies the value of side of square grid for pin density calculations. *Default:* `50` *Minimum*: `0.000000` *Maximum:* `1000.000000`
`-grid_length` *numberRow*	Specifies the value (in row-height) of side of square grid for pin density calculations. *Default:* `10` *Minimum:* `1` *Maximum:* `100`
`-threshold` *density*	Specifies a threshold density value over which the densities must be reported. Any grid under the threshold value is not reported. The value must be positive. *Default*: `0.5` *Minimum*: `0.000000` *Maximum*: `2.000000`

Examples

The following command reports the pin density map:

```
report_pin_density_map
*** Starting report pin density ***
default core: bins with pin density > 0.5 = 0 % (0 / 1)
bin size: 90 sites by 10 row(s). total 1 bins ( 1 by 1 )
pin density range   #bins     %
```

```
total                   0       0
```

write_physical_context_data

```
write_physical_context_data
[-help]
fileName
[-designation name]
[-type technologyName]
```

Writes the physical context data needed for timing analysis. The data that is being written is associated with each instance in the design, or module.

The command can be used, both in Innovus and Tempus, if Tempus is in physical-mode and has the physical data in DB to use for writing out the LDE data.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_physical_context_data` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_physical_context_data`.
`-designation` *`name`*	Specifies the Rx layer width map from micron to enum -um:nsw{:um:nsw}.
`fileName`	Specifies the name of the output file.
`-type` *`technologyName`*	Specifies the technology name.

Related Information

- `read_physical_context_data`

read_physical_context_data

```
read_physical_context_data
[-help]
fileName
[-clear]
[-type technologyName]
```

Reads the physical context data needed for timing analysis. The data that is being read is associated with each instance in the design, or module.

The command can be used in both Innovus and Tempus to restore the state of LDE as long as there is no netlist change. To read the data, there need not be physical information in the DB, as the data can be annotated directly to instances - to be used by STA.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_physical_context_data` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_physical_context_data`.
`-clear`	Clears physical context data before reading.
`fileName`	Specifies the names of the input files.
`-type` *`technologyName`*	Specifies the technology name.

Related Information

- `write_physical_context_data`

reset_physical_context_data

```
reset_physical_context_data
[-help]
```

Clears the physical context data in the DB.

Parameters

None

Related Information

- `read_physical_context_data`
- `write_physical_context_data`

20

Power Calculation Commands and Attributes

- encrypt_thermal_file
- get_default_switching_activity
- get_dynamic_power_simulation
- merge_vtm
- power Category Attributes
- propagate_activity
- query_power_data
- read_activity_file
- read_activity_map_file
- read_power_db
- read_temperature
- report_inst_power
- report_power
- report_unannotated_nets
- reset_power_activity
- set_default_switching_activity
- set_dynamic_power_simulation
- set_instance_temperature_file
- set_power
- set_power_analysis_temperature
- set_power_include_file
- set_power_output_dir
- set_switching_activity
- set_twf_attribute
- set_virtual_clock_network_parameters
- write_power_constraints
- write_tcf
- analyze_ir_paths

encrypt_thermal_file

```
encrypt_thermal_file
-input ASCII.txt
-output $encrypted_conductivity.txt
```

Specifies to encrypt the file containing the thermal conductivity value for layers, and to protect foundry's thermal conductivity parameters used for power map generation. This encrypted file is then specified with the `report_power -thermal_conductivity_inputs` parameter for generating a tile-based power map file.

Parameters

`-input` *`ASCII`*`.txt`	

	Specifies the name of the ASCII thermal file that needs to be encrypted. This file contains the thermal conductivity parameters used for power map generation.
	The `-input` parameter supports both layer type and layer name based thermal conductivity file.
	The following is a snippet of the input thermal conductivity file content (layer type based file):
	`Metal 430`
	`Silicon 250`
	`Dielectric 0.6`

`-output` *`$encrypted_conductivity.txt`*	

	Specifies the name of the encrypted output thermal file.
	The following is a snippet of the encrypted thermal conductivity file content:
	`Metal $encryption_value`
	`Silicon $encryption_value`
	`Dielectric $ecncryption_value`

Examples

- The following command encrypts the specified input file (`format_1.txt`) to generate the encrypted output file (`format_1_encr.txt`):
  ```
  encrypt_thermal_file -input encrypt/format_1.txt -output encrypt/format_1_encr.txt
  ```
  The encrypted file (`format_1_encr.txt`) is then specified for generating a tile-based power map file, as shown below:
  ```
  report_power \
      -out_file                    static.rpt \
      -thermal_power_map_tiles     {5 5}\
      -thermal_power_map_format    stack \
      -thermal_power_map_file      powermap_format1_encr_in.txt \
      -thermal_conductivity_inputs ./encrypt/format_1_encr.txt
  ```

get_default_switching_activity

```
get_default_switching_activity
[-help]
[-clip_activity_to_domain_freq]
[-clock_gates_enable]
[-clock_gates_output]
[-clock_gates_output_ratio]
[-combinational_clock_gate_ratio]
[-duty]
[-global_activity]
[-input_activity]
[-integrated_clock_gate_ratio]
[-macro_activity]
[-period]
[-sequential_activity]
```

Displays the following information about a `set_default_switching_activity` parameter in the system log file and in the system console:

- Parameter name
- Current value

If you do not specify a parameter, the software displays information for all of the `set_default_switching_activity` parameters.
The `get_default_switching_activity` command prints only the user-specified value for the parameters. If a parameter has not been set using the `set_default_switching_activity` command, the `get_default_switching_activity` command does not output any information in the console.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_default_switching_activity` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_default_switching_activity`.
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `set_default_switching_activity` for description of the parameters you can specify.

Examples

The following command displays the current setting of the `-period` parameter:

```
get_default_switching_activity -period
```

The software displays the following information:

```
-period {20}
```

get_dynamic_power_simulation

```
get_dynamic_power_simulation
[-help]
[-activity_pattern]
[-period]
[-resolution]
```

Displays the following information about a `set_dynamic_power_simulation` parameter in the system log file and in the system console:

- Parameter name
- Current value

If you do not specify a parameter, the software displays information for all of the `set_dynamic_power_simulation` parameters.
The `get_dynamic_power_simulation` command prints only the user-specified value for the parameters. If a parameter has not been set using the `set_dynamic_power_simulation` command, the `get_dynamic_power_simulation` command does not output any information in the console.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `get_dynamic_power_simulation` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_dynamic_power_simulation`.
`parameter_names`	Displays information for the specified parameters. You can specify one or more parameters. See `set_dynamic_power_simulation` for description of the parameters you can specify.

Examples

The following command displays the current setting of the `-period` parameter:

```
get_dynamic_power_simulation -period
```

The software displays the following information:

```
-period {20}
```

merge_vtm

```
merge_vtm
[-help]
-config filename
[-rundir directory]
[-merged_file filename]
[-format {simple | metal | stack}]
```

Specifies to import the top-level and other block-level Voltus thermal models (VTMs) having the specified design configuration to generate a merged uniform single VTM for Celsius usage.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`-config` *filename*	

	Specifies the name of the design configuration file.
	The format of the file is:
	`$`*instance_name* `$`*Design_name* `$X(um),$Y(um) $Cell_orientation $`*VTM_file format*
	The file includes:
	• instance name
	• design name
	• X and Y locations of the instance
	• LEF/DEF style orientation for the instance
	• path to the thermal map file
	• format of the thermal map file
	Example:
	`octa octa 0,0 N vtm_output_top/top.vtm.txt both`
	`inst_8 super_filter 62.97,47.035 N vtm_output_block/block.vtm.txt both`
	`inst_1 super_filter 1157.09,765.605 FE vtm_output_block/block.vtm.txt both`
	`inst_2 super_filter 777.085,765.605 E vtm_output_block/block.vtm.txt both`
	`inst_3 super_filter 397.085,765.605 W vtm_output_block/block.vtm.txt both`
	`inst_4 super_filter 17.085,765.605 FW vtm_output_block/block.vtm.txt both`
	`inst_5 super_filter 1127.97,47.035 S vtm_output_block/block.vtm.txt both`
	`inst_6 super_filter 772.97,47.035 FS vtm_output_block/block.vtm.txt both`
	`inst_7 super_filter 417.97,47.035 FN vtm_output_block/block.vtm.txt both`

`-format {simple | metal | stack}`	
	Specifies the format of the thermal map file. The possible arguments are: • `simple`: contains only power tables. • `metal`: contains only metal density tables • `stack`: contains both power and metal density tables.
`-merged_file` *filename*	
	Specifies the name of the merged VTM.
`-rundir` *directory*	
	Specifies the output folder for the merged thermal model.

Examples

- The following command specifies to generate a merged VTM:
  ```
  merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_out_both -merged_file merge_vtm.txt
  ```
- The following command specifies to generate a merged VTM containing only power tables:
  ```
  merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_out_simple -merged_file merge_vtm.txt -format simple
  ```
- The following command specifies to generate a merged VTM containing only metal density tables:
  ```
  merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_out_metal -merged_file merge_vtm.txt -format metal
  ```
- The following command specifies to generate a merged VTM containing both power and metal density tables:
  ```
  merge_vtm -config /Run_Merge_VTM/design_both.cfg -rundir merged_vtm_both_both -merged_file merge_vtm.txt -format stack
  ```

power Category Attributes

The root attributes in the `power` category are:

- power_add_simulation
- power_adjust_input_activity_in_iterations
- power_adjust_macro_activity_in_iterations
- power_keep_clock_gate_ratio_in_iterations
- power_analysis_temperature
- power_annotation_detail_report
- power_auto_twf_delay_annotation
- power_average_rise_fall_cap
- power_block_independent_peakpower
- power_bulk_pins
- power_capacity
- power_case_insensitive_mapping
- power_clock_source_as_clock
- power_comprehensive_automapping
- power_compress_reports
- power_constant_override
- power_corner
- power_create_driver_db
- power_current_generation_method
- power_db_name
- power_decap_cell_list
- power_default_frequency
- power_default_slew
- power_default_supply_voltage
- power_detailed_view_current_only
- power_disable_clock_gate_clipping
- power_disable_leakage_scaling
- power_disable_static
- power_distribute_switching_power
- power_distributed_combine_report_format
- power_distributed_setup
- power_domain_based_clipping
- power_dynamic_glitch_filter
- power_enable_auto_queue
- power_enable_auto_mapping
- power_enable_disk_mapping

- power_enable_duty_propagation_with_global_activity
- power_enable_dynamic_current_slew_load_interpolation
- power_dynamic_scale_clock_by_frequency
- power_dynamic_scale_clock_by_name
- power_enable_interactive_reports
- power_enable_mt_state_propagation
- power_equivalent_annotation
- power_dynamic_vectorless_ranking_methods
- power_enable_dynamic_scaling
- power_enable_flop_state_propagation
- power_enable_force_propagation
- power_enable_generated_clock
- power_enable_input_net_power
- power_enable_multithread_reports
- power_enable_power_target_flow
- power_enable_pba_for_tempus_pi
- power_enable_rtl_dynamic_vector_based
- power_enable_scan_report
- power_enable_single_cycle_scheduling
- power_enable_slew_based_ccs_pin_cap
- power_enable_state_propagation
- power_enable_stable_clock_gating
- power_enable_stable_flop_scheduling
- power_enable_tempus_pi
- power_enable_xp
- power_enhanced_blackbox_avg
- power_enhanced_blackbox_max
- power_event_based_leakage_power
- power_external_load_config_file
- power_extraction_tech_file
- power_extractor_include
- power_fanout_limit
- power_force_library_merging
- power_from_x_transition_factor
- power_from_z_transition_factor
- power_generate_flop_ranking_data
- power_generate_activity_mapping_report
- power_generate_current_for_rail

- power_generate_static_report_from_state_propagation
- power_generate_leakage_power_map_based_on_calculated_leakage
- power_grid_libraries
- power_handle_glitch
- power_handle_transport_glitch
- power_handle_tristate
- power_hier_delimiter
- power_honor_combinational_logic_on_clock_net
- power_honor_negative_energy
- power_honor_net_activity
- power_honor_non_mission_leakage
- power_hybrid_analysis
- power_ignore_control_signals
- power_ignore_data_phase_for_clock
- power_ignore_end_toggles_in_profile
- power_ignore_glitches_at_same_time_stamp
- power_ignore_inout_pin_cap
- power_ignore_macro_leakage_scale_for_temperature
- power_ignore_unconstraint_net_with_timing_data
- power_include_file
- power_include_initial_x_transitions
- power_include_sequential_clock_pin_power
- power_include_timing_in_current_file
- power_ir_derated_timing_view
- power_leakage_scale_factor_for_temperature
- power_lib_files
- power_library_preference
- power_macro_pgv_leakage
- power_macro_toggle_pin_percentage
- power_match_state_for_logic_x
- power_memory_current_scaling_method
- power_multibit_flop_toggle_behavior
- power_multibit_flop_toggle_pin_percentage
- power_multi_scenario_simulation
- power_off_pg_bulk_leakage
- power_merge_switched_net_currents
- power_method
- power_min_leaf_count

- power_off_pg_nets
- power_output_current_data_prefix
- power_output_dir
- power_partition_count
- power_partition_twf
- power_pin_based_twf
- power_precision
- power_pre_simulation_empty_period
- power_pre_simulation_period
- power_pre_simulation_power_exclude_period
- power_quit_on_activity_coverage_threshold
- power_read_rcdb
- power_relax_arc_match
- power_report_library_usage
- power_report_missing_bulk_connectivity
- power_report_missing_input
- power_report_black_boxes
- power_report_clock_instance_switching_info
- power_report_idle_instances
- power_report_instance_switching_info
- power_report_instance_switching_list
- power_report_missing_nets
- power_report_scan_chain_statistics
- power_report_statistics
- power_report_time_display_fraction_digits
- power_report_twf_attributes
- power_reuse_flop_ranking_data
- power_reuse_flop_ranking_data_hier
- power_scale_to_sdc_clock_frequency
- power_scan_chain_activity
- power_scan_chain_name_pattern
- power_scan_control_file
- power_set_power_event_based
- power_settling_buffer
- power_show_cell_usage_stats
- power_smart_window
- power_scan_multi_bit_flop_chain_type
- power_split_bus_power

- power_start_time_alignment
- power_state_dependent_leakage
- power_stateprop_ignore_unannot_pins
- power_static_multi_mode_scenario_file
- power_static_netlist
- power_switching_power_on_rise_only
- power_thermal_input_file
- power_thermal_leakage_temperature_scale_table_file
- power_to_x_transition_factor
- power_to_z_transition_factor
- power_transition_factor_based_duty
- power_transition_time_method
- power_twf_delay_annotation
- power_twf_load_cap
- power_unified_power_switch_flow
- power_use_cell_leakage_power_density
- power_use_fastest_clock_for_dynamic_scheduling
- power_use_lef_for_missing_cells
- power_use_only_ddv_current
- power_worst_case_vector_activity
- power_worst_step_size
- power_worst_window_count
- power_worst_window_size
- power_worst_window_type
- power_worst_window_coverage
- power_worst_window_reports
- power_write_dynamic_currents
- power_use_physical_partition
- power_use_twf_stable_randomized_arrival_delay
- power_use_zero_delay_vector_file
- power_vector_based_multithread
- power_vector_profile_mode
- power_write_gui_db
- power_write_bbox
- power_write_db
- power_write_default_pti_files
- power_write_profiling_db
- power_write_simulation_db

- power_write_static_currents
- power_x_transition_factor
- power_x_count_transition_using_3_states
- power_z_transition_factor
- power_zero_delay_vector_toggle_shift

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `power` category, use the following command:

```
get_db -category power
```

power_add_simulation

```
power_add_simulation {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to add simulation if there are no simulation vectors or if all primary sources are not annotated by vectors. This parameter can be used to ensure complete design coverage. Event simulation is done for primary sources like primary input, flops, and macros.

This attribute is only applicable to the event-based power analysis flow.

Related Commands

- `set_power`

power_adjust_input_activity_in_iterations

```
power_adjust_input_activity_in_iterations {true | false}
```

Default: `true`

Data_type: Boolean

Specifies whether to keep the input activity fixed or automatically adjust the activity value for each iteration of state propagation. This attribute is only applicable to the power target flow (`power_enable_power_target_flow true`).

Related Commands

- `set_power`

power_adjust_macro_activity_in_iterations

```
power_adjust_macro_activity_in_iterations {true | false}
```

Default: `true`

Data_type: Boolean

Specifies whether to keep the macro activity fixed or automatically adjust the activity value for each iteration of state propagation. This attribute is only applicable to the power target flow (`power_enable_power_target_flow true`).

Related Commands

- set_power

power_keep_clock_gate_ratio_in_iterations

```
power_keep_clock_gate_ratio_in_iterations {true | false}
```

Default: false

Data_type: Boolean

Specifies whether to keep the clock gate output (CGO) fixed or automatically adjust the value for each iteration of state propagation. This attribute is only applicable to the power target flow (power_enable_power_target_flow true).

Related Commands

- set_power

power_analysis_temperature

```
power_analysis_temperature temperature
```

Default: -274.000000

Data_type: Double

Specifies the temperature for static power calculation.

Note: Static power calculation gets the temperature value of each instance from the following sources (in order of precedence):

- Instance temperature file
- power_analysis_temperature attribute
- The current operating condition temperature

Related Commands

- report_power

power_annotation_detail_report

```
power_annotation_detail_report {true | false}
```

Default: false

Data_type: Boolean

Specifies to generate a detailed annotation report per instance in the Vector Profiling flow (power_method vector_profile). When specified, it creates the AnnotationDetail.rpt file in the user-specified output directory.

Related Commands

- report_power

power_auto_twf_delay_annotation

```
power_auto_twf_delay_annotation {true | false}
```

Default: false

Data_type: Boolean

Specifies to automatically detect the zero-delay vectors and perform delay annotation. As a result, you do not need to specify additional parameters (`set_db power_use_zero_delay_vector_file` and `read_activity_file -zero_delay`) to indicate the vectors that have delay annotation. This attribute is applicable only to the event-based power analysis flow.

Related Commands

- `report_power`

power_average_rise_fall_cap

```
power_average_rise_fall_cap {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to use the average of rise and fall capacitance from the liberty file.

Related Commands

- `report_power`

power_block_independent_peakpower

```
power_block_independent_peakpower {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to identify the worst power interval of each block, and run power analysis of the full design using the worst interval of each block. This attribute allows you to perform power and current analysis for design blocks that have independent peak power intervals.

When set to `false`, power analysis is performed using the common worst interval found using simulation of all vectors together.

Related Commands

- `report_power`

power_bulk_pins

```
power_bulk_pins {bulk_pin_list}
```

Default: `""`

Data_type: String

Defines power and ground bulk LEF pins for the design.

Use this attribute when library cells have bulk pins defined in LEF but the liberty file does not contain power associated with these pins. If a liberty file does not have body bias definition, power analysis does not distribute power to the body bias domain.

Related Commands

- `report_power`

power_capacity

Default: `low`

Data_type: Enum

Enables segmented power processing. When this attribute is specified, the software performs concurrent processing of vector simulation annotation, power calculation, and report construction.

This attribute enables performance optimization with some trade-off on memory consumption and runtime. The arguments of `power_capacity` are:

- `low` - reduces memory for a flat run.
- `medium` - further reduces memory with moderate increase in runtime (1.2X).
- `high` - maximum memory reduction at about 2X runtime.

The `power_capacity` attribute is used for reducing memory when processing long simulation vectors. In case of the FSDB and SHM format activity file, the runtime can increase up to 2X.

Related Commands

- `report_power`

power_case_insensitive_mapping

```
power_case_insensitive_mapping {true | false}
```

Default: `false`

Data_type: Boolean

Specifies whether the instance name mapping between the RTL netlist and GATE level netlist in the mapping file is case sensitive or not. When set to `true`, the name mappings are not case sensitive.

power_clock_source_as_clock

```
power_clock_source_as_clock {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to use the correct clock frequency for activity calculation. If you stop the arrival of the clock signal by setting `set_case_analysis` to `false`, the transition density at the output net of the flop is calculated using the correct clock frequency instead of the default frequency.

Related Commands

- `report_power`

power_comprehensive_automapping

```
power_comprehensive_automapping {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to perform all-inclusive instance name mapping between the RTL netlist and GATE level netlist. When set to `true`, the software honors all possible naming rules (includes handling of characters like '[' , ']', '.', '/', '\\', '_', ':' and handling of strings like "MBFF", "_reg", "_MB_" etc.) for better annotation coverage.

This feature is applicable to both file-based mapping (`read_activity_map_file -rtl_to_gate | -two_column_mapping_file`) and automatic mapping (`power_enable_auto_mapping`). When `power_comprehensive_automapping` is specified with either `-rtl_to_gate` or `-two_column_mapping_file`, the RTL mapping entries are given priority over the automatic rule-based mapping entries and are used to annotate the design objects.

Related Commands

- `report_power`

power_compress_reports

```
power_compress_reports {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to compress the power output files in the GNU Zip format. The compressed files will have the suffix .gz.

The possible arguments are:

- `true`: Sets the compression option with compression level 6
- `false`: default, no compression
- 1-9: the compression levels vary from 1 to 9. 1 provides the fastest compression speed but at a lower ratio, so the file size will be larger. 9 provides the highest compression ratio but at a lower speed.

Related Commands

- `report_power`

power_constant_override

```
power_constant_override {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to give higher precedence to propagated constants defined using the `set_case_analysis` command. It allows propagated constants to override global activity.

Related Commands

- `report_power`

power_corner

```
power_corner {min|max}
```

Default: `{}`

Data_type: String

Defines the library corner (for non-MMMC setup).

- `min` corner means power calculator will use min timing libraries for the power calculation.
- `max` corner means power calculator will use max timing libraries for the power calculation.

Related Commands

- report_power

power_create_driver_db

```
power_create_driver_db {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to determine all the drivers/receivers on the ground net.

Related Commands

- report_power

power_current_generation_method

```
power_current_generation_method {avg | peak}
```

Default: `avg`

Data_type: String

Specifies the current waveform generation method during dynamic power analysis. The two possible arguments of this attribute are:

- `avg` - Specifies to use the average current value of each time step. The output current filename is *`filename`*`.ptiavg`. The default value is `avg`.
- `peak` - Specifies to use the peak current value of each time step. The output current filename is *`filename`*`.ptipeak`. It is recommended to use the `peak` method when using the Composite Current Source Power (CCSP) Liberty files.

Related Commands

- report_power

power_db_name

```
power_db_name filename
```

Default: `""`

Data_type: String

Specifies the name of the binary power database file (`filename.db`).

You can specify this attribute in scenarios where more than one db is generated and you need to track which one was generated in which mode when the restoring the db, or if your design has several blocks and analysis is done on one block all the time and all the databases are named `power.db`.

Related Commands

- report_power

power_decap_cell_list

```
power_decap_cell_list cell_list
```

Default: `""`

Data_type: String

Specifies the physical only cells, such as decap cells, to include in the power report.

Note: This is only supported for Static Power Analysis.

Related Commands

- `report_power`

power_default_frequency

```
power_default_frequency value
```

Default: `-1.0`

Data_type: Double

Specifies to set the default frequency in the MHz unit for net not annotated by TWF. You can specify a numerical value without unit.

Related Commands

- `report_power`

power_default_slew

```
power_default_slew value
```

Default: " "

Data_type: String

Specifies to set the default slew in the ps unit for net not annotated by TWF. You can specify a numerical value without unit.

Related Commands

- `report_power`

power_default_supply_voltage

```
power_default_supply_voltage value
```

Default: " "

Data_type: String

Specifies to set a default voltage for power nets in a scenario where the power engine cannot determine the voltage of these nets. The default value of this attribute is `1.0`.

Related Commands

- `report_power`

power_detailed_view_current_only

```
power_detailed_view_current_only {true | false}
```

Default: `true`

Data_type: Boolean

Controls instance power reporting for cells that have power information available in PGV. When set to `true`, specifies to skip writing of the switching and leakage current and use only the internal current from DDV PGV. When set to `false`, specifies to report the switching and leakage current values in addition to the internal current from DDV PGV. If a cell is defined in both PGV and dotlib, leakage is computed from dotlib. It is recommended to set this attribute to `true` because that would prevent over-estimation of the instance current.

Related Commands

- `report_power`

power_disable_clock_gate_clipping

```
power_disable_clock_gate_clipping {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to enable clock gate output clipping so that the output transition density of an ICG cell does not exceed the input clock pin transition density.

Related Commands

- `report_power`

power_disable_leakage_scaling

```
power_disable_leakage_scaling {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to exclude leakage power during scaling factor computation when target power is specified using the `set_power` command. When this option is set to `true`, the leakage power does not get scaled, and only the internal and switching components are scaled to meet the user-specified total power.

This attribute is relevant only for the `set_power` commands for which total power is specified, and has no effect for the `set_power` command specified with the `-internal`, `-switching`, or `-leakage` parameters.

Related Commands

- `report_power`

power_disable_static

```
power_disable_static {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true` in the dynamic vector-based or dynamic vectorless flows, the command performs only dynamic analysis and turns-off static power calculation.

Related Commands

- `report_power`

power_distribute_switching_power

```
power_distribute_switching_power {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to equally distribute the switching power among all drivers if a net has more than one driver. When set to `false`, stores all switching power on one of the net drivers.

Related Commands

- `report_power`

power_distributed_combine_report_format

```
power_distributed_combine_report_format {none | detailed | reduced}
```

Default: `reduced`

Data_type: String

Specifies to generate consolidated XP power reports. This parameter allows you to combine power reports that are stored across multiple partition directories to give a single unified view of power data. The possible arguments of this parameter are:

- `none` - generates distributed reports.
- `reduced` - generates consolidated reports for the following:
  - power.missingdata.* (static and dynamic flows)
  - power.stateprop.* (dynamic flow)
  - power.stats.setpower (dynamic flow)
- `detailed` - generates consolidated reports for the following:
  - power.rpt (static and dynamic flows)
  - power.instpwr.rpt (static flow)
  - power.lib.rpt (static flow)
  - power.pgpinpwr.rpt (static flow)
  - power.pgnet.detailed.rpt
  - power.missingdata.* (static and dynamic flows)
  - power.pgnet.detailed.rpt (static and dynamic flows)
  - power.stateprop.* (dynamic flow)
  - power.stats.setpower (dynamic flow)
  - pg_trigger.txt (dynamic flow)
  - trigger.txt (dynamic flow)
  - *.totalcurrent (dynamic flow)
  - voltus_power.cellstat (static and dynamic flows)
  - voltus_power.ptifiles (static and dynamic flows)

Related Commands

- `report_power`

power_distributed_setup

```
power_distributed_setup file
```

Default: `""`

Data_type: String

Specifies a file containing the customized setup details for running power analysis in the distributed mode.

The specified file has 4 columns:

`INST/CELL <name> hier/block <power.inc>`

where,

- `INST/CELL`: Keyword indicating cell or instance to be run in the distributed mode. Cell can be useful for repeated DEF blocks.
- *`name`* : Name of the hierarchical block or instance.
- `hier/block`: Keyword indicating hierarchy or block. `hier` implies that the full hierarchy will be preserved by including the DEFs from top to the block, and `block` implies that only the lower level DEF will be included
- *`power.inc`* : name of the power include file. This column is optional.

The unspecified instance/cell will be run with the top level as a single group. Each distributed block/instance should have corresponding TWF and SPEF to improve runtime.

Note: Each hierarchy or block will be run independently from other hierarchies/blocks, thus the external load or cell seen at the pin of the hierarchy/block can be different from flat analysis. If the partition boundary analysis is important, distributed processing is not recommended.

Related Commands

- report_power

power_domain_based_clipping

```
power_domain_based_clipping {true | false}
```

Default: `false`

Data_type: Boolean

Determines the frequency to be used to clip propagated activity that is too high. When set to `false`, the frequency of the fastest clock in the design is used. If you set this attribute to `true`, the fastest clock domain on the net is used to clip propagated activity.

Related Commands

- report_power

power_dynamic_glitch_filter

```
power_dynamic_glitch_filter value
```

Default: `-1.0`

Data_type: Double

Sets the absolute threshold time (in ns) for filtering glitches in net toggles. When you specify a time value for this parameter, glitches in net toggles that are equal to or smaller in width than the specified time value are removed.

Related Commands

- report_power

power_enable_auto_queue

```
power_enable_auto_queue {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to automatically determine the queue for power jobs. This attribute is used in the Voltus-XP distributed power flow with hierarchical DEFs. In the XP flow, partitions are divided into multiple queues depending on the number of CPUs (For example, Default queue: 1 CPU, Small queue: 2 CPUs, Medium queue: 4 CPUs, Large queue: 8 CPUs, and so on). By default, all power partitions run on the same queue. When the attribute is set to `true`, the blocks can run on different queues depending on the size (number of instances) of the blocks.
The `power_enable_auto_queue` attribute will be beneficial when there are uneven power block partitions. For example, if there are some blocks with 20-30M instances and there are some with 3-5M instances. Using this attribute, bigger blocks will be automatically assigned to a bigger queue, resulting in better runtime.

Related Commands

- `report_power`

power_enable_auto_mapping

```
power_enable_auto_mapping {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to automatically perform instance name mapping between the RTL netlist and GATE level netlist. This attribute allows you to map the RTL vector to the Gate level netlist without a mapping file.

You can use the `read_activity_file -name_mapping_rule <filename>` command parameter to specify a file containing customized or design-specific rules.

Related Commands

- `report_power`

power_enable_disk_mapping

```
power_enable_disk_mapping {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to enable disk caching of the toggles. Therefore, the toggles are stored in the disk instead of keeping them in memory. This has made large vectors or long simulation duration runs possible even with lesser available memory. This attribute is only applicable to the event-based power analysis flow (`power_method event_based`) in the XP mode.

Example:

The following command specifies to enable disk caching of toggles:

```
set_db power_method event_based power_enable_disk_mapping true power_enable_xp true
```

Related Commands

- `report_power`

power_enable_duty_propagation_with_global_activity

```
power_enable_duty_propagation_with_global_activity {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to propagate duty cycle with the global switching activity setting for all data nets in the dynamic vectorless analysis flow.

Related Commands

- report_power

power_enable_dynamic_current_slew_load_interpolation

```
power_enable_dynamic_current_slew_load_interpolation {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to use the charge-based slew load interpolation. When this attribute is set to `true`, Power Analysis takes dynamic currents from the 4 nearest slew/load values and performs either a 4-way (lower slew, lower load, upper slew, upper load) or a 2-way (lower slew, upper slew OR lower load, upper load) interpolation to calculate the dynamic current waveform. The attribute allows you to derive a more realistic dynamic current waveform based on the exact slew load values from the .lib dynamic current table.

Related Commands

- report_power

power_dynamic_scale_clock_by_frequency

```
power_dynamic_scale_clock_by_frequency { freq_Afactor_Afreq_Bfactor_Bfreq_Cfactor_C... }
```

Default: `""`

Data_type: String

Specifies the scale factors for the clock frequency values. This parameter allows you to perform frequency scaling for clocks running at relatively low frequencies, thereby ensuring higher toggle coverage for limited simulation period. The parameter is applicable only to the dynamic vectorless flow. The following units are supported for frequency: Hz, KHz, MHz, and GHz.

Related Commands

- report_power

power_dynamic_scale_clock_by_name

```
power_dynamic_scale_clock_by_name { clk_A factor_A clk_B factor_B clk_C factor_C ... }
```

Default: `""`

Data_type: String

Specifies the scale factors for the low frequency clocks. This parameter allows you to perform frequency scaling for clocks running at relatively low frequencies, thereby ensuring higher toggle coverage for limited simulation period. The parameter is applicable only to the dynamic vectorless flow. The following units are supported for frequency: Hz, KHz, MHz, and GHz.

Related Commands

- report_power

power_enable_interactive_reports

```
power_enable_interactive_reports {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to generate multiple dynamic reports based on the power analysis run done for a netlist. When this attribute is set to `true`, you can generate various types of power reports interactively using the `report_power` command without the need to perform power calculation on the same netlist again. The `power_enable_interactive_reports` attribute is useful when you need to generate reports for the unaltered netlist within the same power analysis session.

This attribute is applicable only to the event-based power analysis flow with area-based partitioning (`power_use_physical_partition`) in the XP mode.

Example:

The following command specifies to enable interactive generation of power reports:

```
set_db power_method event_based power_use_physical_partition true power_partition_count 2 power_enable_interactive_reports
true power_enable_xp true
```

Related Commands

- report_power

power_enable_mt_state_propagation

```
power_enable_mt_state_propagation {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to enable multi-threading in state propagation to help improve the runtime for large simulations. For example, when the attribute was specified for a design with 300 million instances and simulation duration of 120 microseconds, the runtime has improved from 21 hours to 1.5 hours.

This attribute is only applicable to the event-based power analysis flow (`power_method event_based`).

Related Commands

- report_power

power_equivalent_annotation

```
power_equivalent_annotation {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to enable equivalent annotation for Q or Qbar pins when only one of them is annotated.

Related Commands

- set_power

power_dynamic_vectorless_ranking_methods

```
power_dynamic_vectorless_ranking_methods {load | clock | vector_activity}
```

Default: `""`

Data_type: String

Specifies to change the ranking method from the default to the following:

- `load` - Prioritizes scheduling cells with higher fanout
- `clock -` Prioritizes scheduling cells associated with faster clocks
- `vector_activity` - Prioritizes scheduling cells with higher activity

Related Commands

- `report_power`

power_enable_dynamic_scaling

```
power_enable_dynamic_scaling {true | false}
```

Default: `false`

Data_type: String

Enables `set_power` scaling for dynamic power analysis. The power and scaling specifications are used to scale dynamic current on both the power and ground currents of the specified instance/cell.

This attribute honors the user-defined average (static) power for leaf-level instances and hierarchies (instances/cells/modules) along with power net specification in the dynamic current scaling flows such that the average of the dynamic current is equal to the assigned average power. When you specify this attribute, the average of the dynamic current of these leaf-level instances will match the scaled static power numbers.

> This attribute is not supported in the state-propagation-based vectorless flow (`power_enable_state_propagation true`). It is applicable only to the probability-based vectorless flow.

Related Commands

- `set_power`
- `report_power`

power_enable_flop_state_propagation

```
power_enable_flop_state_propagation {true | false}
```

Default: `false`

Data_type: Boolean

Enables accurate clock state propagation for sequential cells. When this attribute is set to `true`, flops propagate in a manner that toggles on the output pins are in sync with the clock. This attribute helps to align the output pins' toggles with the clock toggles.

Related Commands

- `set_power`
- `report_power`

power_enable_force_propagation

```
power_enable_force_propagation {true | false}
```

Default: `false`

Data_type: bool

Enables force propagation to ensure that the output toggles are according to the clock signal rather than the input pin states. This attribute assigns clock activity to outputs when input propagation is not feasible, thereby enhancing overall activity.

Related Commands

- set_power

power_enable_generated_clock

```
power_enable_generated_clock {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to get generated clock frequency during activity propagation for static power analysis.

Related Commands

- set_power

power_enable_input_net_power

```
power_enable_input_net_power {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to calculate the switching power of input nets.

Related Commands

- set_power

power_enable_multithread_reports

```
power_enable_multithread_reports {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to enable multi-threading for generation of power calculation reports. When set to `true`, it improves the memory and runtime for power reporting in large designs with more than 100 million instances.

Related Commands

- set_power

power_enable_power_target_flow

```
power_enable_power_target_flow {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to enable the power target flow. When set to `true`, the tool allows you to specify power targets for full-chip power analysis. You can use the `set_power` command to specify the power targets for full-chip, physical block, logical block, and power/ground net. When these targets are specified, Voltus will automatically scale the toggle rates of clock gate ratio, input, macro, and sequential activities to meet the specified power targets.

Related Commands

- `set_power`

power_enable_pba_for_tempus_pi

```
power_enable_pba_for_tempus_pi {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to enable the Path-Based Analysis (PBA) for slack calculation in the Tempus Power Integrity (Tempus PI) flow. By default, the Graph-Based Analysis (GBA) is used. This attribute is only applicable to the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`).

Related Commands

- `set_power`

power_enable_rtl_dynamic_vector_based

```
power_enable_rtl_dynamic_vector_based {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to take an RTL or partial VCD/FSDB file as input and use that for dynamic vector-based flow. This attribute allows you to estimate current even for instances which are missing from the activity file.

You must use this attribute in conjunction with the `power_method dynamic_vectorbased` attribute.

Related Commands

- `set_power`

power_enable_scan_report

```
power_enable_scan_report {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to generate all the scan mode analysis reports. The following reports will be generated:

- `voltus_power.scanff.duplicated.rpt` - flip-flops that are specified in multiple chains

- `voltus_power.scanff.missing.rpt` - flip-flops that are missing in the scan control file but exist in the design
- `voltus_power.scanff.nonassigned.rpt` - flip-flops that do not exist in the design and are dropped from the scan control file
- `voltus_power.scanff.nonscan`.rpt - assigned flip-flops that have no scan-out pin
- `voltus_power.scanff.nonswitched.rpt` - non-switching flip-flops in the scan control file
- `voltus_power.scanff.rpt` - detailed scan chain report containing global statistics for all scan chains and statistics for each scan chain in the design
- `voltus_power.scanff.stateconflict.rpt` - flip-flops that have conflicting state assignment

Related Commands

- `set_power`

power_enable_single_cycle_scheduling

```
power_enable_single_cycle_scheduling {true | false}
```

Default: `false`

Data_type: Boolean

Schedules macro instances in every dominant clock cycle in the dynamic vectorless flow. This attribute can be used achieve faster toggle coverage. After the scheduling of every instance and attaining the maximum toggle coverage, the scheduler cycles back to the same switch source element and then schedules it again with the opposite polarity. This attribute helps in achieving the same toggle coverage within half the simulation period compared to dual cycle scheduling.

When the attribute is set to `false`, the macros are scheduled every two cycles wherein the adjacent cycle is used for reversing the transition polarity.

This is a root attribute.

- To set the value of any root attribute, use the `set_db` command:
  `set_db attribute_name value`
- To get the current value of any root attribute, use the `get_db` command:
  `get_db attribute_name`

Related Commands

- `set_power`

power_enable_slew_based_ccs_pin_cap

```
power_enable_slew_based_ccs_pin_cap {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to use the Composite Current Source (CCS) capacitance value based on the slew given in the input pin section of the library cell in the Liberty file. If it is set to `true`, the tool honors the CCS pin capacitance; if it is set to `false`, it takes `max_capacitance` from .lib. This attribute can be used when timing windows and/or a Tempus timing session is not available.

Related Commands

- `set_power`

power_enable_state_propagation

```
power_enable_state_propagation {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to enable the vectorless state-propagation-based dynamic analysis flow.

For more information, refer to the *State-Propagation-Based Vectorless Methodology* section in the " *Dynamic Power and IRDrop Analysis* " chapter of the *Voltus User Guide*.

Related Commands

- `set_power`

power_enable_stable_clock_gating

```
power_enable_stable_clock_gating {true | false}
```

Default: `false`

Data_type: Boolean

Perform balanced scheduling of ICGs independent of the predecessor ICG state.

This attribute is used for the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`) only.

Related Commands

- `set_power`

power_enable_stable_flop_scheduling

```
power_enable_stable_flop_scheduling {true | false}
```

Default: `false`

Data_type: Boolean

Perform balanced scheduling of flops independent of the ECOs done on the design.

This attribute is used for the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`) only.

Related Commands

- `set_power`

power_enable_tempus_pi

```
power_enable_tempus_pi {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to enable the Tempus PI analysis flow. When this attribute is set to `true`, data path scheduling is timing-aware, that is, scheduling is based on timing slacks. This attribute is only applicable to the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`).

For more information, refer to the **Tempus Power Integrity Analysis** chapter in the *Tempus User Guide*.

Related Commands

- set_power

power_enable_xp

```
power_enable_xp {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to run standalone report generation in the XP mode.

Related Commands

- set_power

power_enhanced_blackbox_avg

```
power_enhanced_blackbox_avg {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to use the average toggle rate of related inputs for enhanced blackbox propagation.

Related Commands

- set_power

power_enhanced_blackbox_max

```
power_enhanced_blackbox_max {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to use the maximum toggle rate of related inputs for enhanced blackbox propagation.

Related Commands

- set_power

power_event_based_leakage_power

```
power_event_based_leakage_power {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to calculate state-dependent leakage power. When this attribute is set to `true`, it calculates leakage power and current based on the state of an instance at each time point.

For example, at time 0ns, instance instA is in state !I ZN (0 on input, 1 on output), and at time 71ns, the instance is in state I !ZN (1 on input, 0

on output). The leakage power is calculated based on the different states of instA at different time points.

Related Commands

- set_power

power_external_load_config_file

```
power_external_load_config_file filename
```

Default: " "

Data_type: String

Specifies to honor the user-specified load capacitance values for power calculation. This attribute allows you to specify an external load capacitance file when you have a timing window file without the EXTERNAL_LOAD constructs.

File Format:

The format of the user-specified input file is:

```
INST|CELL    blockName    external_load_flle
```

Here, the format of *external_load_flle* is:

```
NET|PORT       name       load_value
```

Examples:

Configuration file:

```
INST          TOP           top_wcard.load
```

External load file (top_wcard.load):

```
UNIT pF
NET   result     0.02
NET   overflow   0.02
```

Related Commands

- set_power

power_extraction_tech_file

```
power_extraction_tech_file filename
```

Default: " "

Data_type: String

Specifies the extraction technology file to be used for top-level power-grid extraction.

Related Commands

- set_power

power_extractor_include

```
power_extractor_include filename
```

Default: " "

Data_type: String

Specifies the extractor (ZX) commands and variables in the form of an include file. This parameter is applicable only for the standalone report

generation flow in the XP mode (`power_enable_xp true`).

Related Commands

- `set_power`

power_fanout_limit

```
power_fanout_limit value
```

Default: `-1`

Data_type: Integer

Specifies the fanout limit for nets. If an instance has nets more than or equal to the fanout limit, its switching/internal/leakage power is taken to be zero.

Related Commands

- `set_power`

power_force_library_merging

```
power_force_library_merging {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to force merge PGVs even if there are some minor conflicts in the merging libraries. This parameter is applicable only for the standalone report generation flow in the XP mode (`power_enable_xp true`).

Related Commands

- `set_power`

power_from_x_transition_factor

```
power_from_x_transition_factor value
```

Default: `0.5`

Data_type: Double

Specifies how transitions from X to 0/1 are counted.

Related Commands

- `set_power`

power_from_z_transition_factor

```
power_from_z_transition_factor value
```

Default: `0.25`

Data_type: Double

Specifies how transitions from Z to 0/1 are counted.

Related Commands

- set_power

power_generate_flop_ranking_data

```
power_generate_flop_ranking_data directoryname
```

Default: ""

Data_type: String

Specifies the name of the directory where the flop ranking data is saved.

Related Commands

- set_power

power_generate_activity_mapping_report

```
power_generate_activity_mapping_report  {true | false}
```

Default: false

Data_type: Boolean

Specifies to generate the activity_mapping.rpt report in the vector-based dynamic power flow. This report gives the list of signal name mappings between the Golden (RTL netlist - input name in the vector file) and Revised (Gate netlist - annotated design object name) netlists that were used in the flow. The report displays the list of annotated and unannotated objects and helps to debug the mapping file for unannotated design objects.

activity_mapping.rpt supports activity mapping done through the following methods:

- User-specified mapping file (read_activity_map_file -two_column_mapping_file)
- Conformal-generated RTL mapping file (read_activity_map_file -rtl_to_gate)
- Voltus auto-generated mapping file (power_enable_auto_mapping).

Related Commands

- set_power

power_generate_current_for_rail

```
power_generate_current_for_rail railnames
```

Default: ""

Data_type: String

Generates current files for the specified rail.

By default, the command generates current files for all power rails.

Related Commands

- set_power

power_generate_static_report_from_state_propagation

```
power_generate_static_report_from_state_propagation  {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to directly generate the static power analysis result based on the RTL-VCD input and state-propagation. When this attribute is specified, the RTL vector power analysis flow does not require the generation of a TCF/FSDB file. This attribute is applicable only to the RTL vector-based dynamic power analysis flow.

Related Commands

- set_power

power_generate_leakage_power_map_based_on_calculated_leakage

```
power_generate_leakage_power_map_based_on_calculated_leakage  {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to scale the instance leakage power by directly multiplying the user-specified scaling factor with the final calculated leakage power value, instead of scaling the leakage power value from the input dotlib.

If this parameter is not specified, the user-specified scaling factor will be applied on the leakage power values from the input dotlib.

Related Commands

- set_power

power_grid_libraries

```
power_grid_libraries library_list
```

Default: `""`

Data_type: String

Specifies the name of the Cadence power cell libraries (`.cl`). You can also specify a list of defined power-grid libraries having decap cells tagged in them.

Note: When specifying a list of PGVs, you must specify the technology library as the first PGV library in the list, followed by the standard and macro PGV libraries. The selection between the standard and macro PGV library is order based. Therefore, if multiple power-grid libraries are specified for a cell, the power engine maps the cell with the first library after the technology library.

Related Commands

- set_power

power_handle_glitch

```
power_handle_glitch {true | false}
```

Default: `false`

Data_type: Boolean

A transition is defined as a glitch when the time difference between two sequential toggles are less than half of the rise transition time + fall transition time in the vector-based static power calculation.

If set to `true`, glitch power will be identified and calculated according to this definition and it will be reported in a separate column. Switching or internal power calculation does not include power due to glitches.

If set to `false`, glitch power will not be identified, and the respective power will be calculated as normal signal switches. Hence, the total power will be a little higher.

Related Commands

- `set_power`

power_handle_transport_glitch

```
power_handle_transport_glitch {true | false}
```

Default: `false`

Data_type: Boolean

Transport glitch is referred to as functional toggles for a net in a single clock period before it reaches a stable value. You can use this attribute to report the internal and switching power values for transport glitch.

The transport glitch power is reported in the following reports:

- Average Power Summary Report (`power.rpt.avgpwr`) - reported in the Total, Group, Clock, and Rail power sections.
- Instance Power Report (`power.rpt.instpwr`) - reported for all instances.
- Hierarchy Power Report (`power.rpt.hierpwr`) - reported for all hierarchies.

This attribute is automatically set to `true` with `power_handle_glitch true`. That is, reports both inertial and transport glitch power. When set to `false`, only the inertial glitch power is reported.

Related Commands

- `set_power`

power_handle_tristate

```
power_handle_tristate {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, all tristate device enable pin values will be taken into account when determining the propagation of the activity through tristate gates.

Related Commands

- `set_power`

power_hier_delimiter

```
power_hier_delimiter character
```

Default: " "

Data_type: String

Specifies the hierarchical delimiter in the DEF file. You can use this attribute to override the existing hierarchical delimiter.

Related Commands

- set_power

power_honor_combinational_logic_on_clock_net

```
power_honor_combinational_logic_on_clock_net {true | false}
```

Default: `true`

Data_type: Boolean

Specifies the behavior of combination logic on clock net.

- `true` (default) - specifies to honor activity propagation through combinational logic.
- `false` - specifies that the combination logic will be treated like a combinational clockgate.

Related Commands

- set_power

power_honor_negative_energy

```
power_honor_negative_energy {true | false}
```

Default: `true`

Data_type: Boolean

A value of `true` specifies that the software will keep negative internal energy numbers from the `.lib` internal power table. When set to `false`, negative power is treated as 0.

Related Commands

- set_power

power_honor_net_activity

```
power_honor_net_activity {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to use activity from the `Net` section of the Switching Activity Interchange Format (SAIF) file for power calculation and optimization, and to ignore the pin-based state-dependent path delay (SDPD) activity. The default value of this attribute is `true`.

When this attribute is set to `false`, the power calculation engine can use both the net-based and pin-based activity. In this case, precedence will be given to activity from net or pin that appears later in the SAIF file.

Related Commands

- set_power

power_honor_non_mission_leakage

```
power_honor_non_mission_leakage {true | false}
```

Default: `false`

Data_type: Boolean

Supports modeling of well-to sub-leakage information when the circuit power supply is off for the level shifter cells. This is referred to as the non-mission mode leakage power calculation. The leakage power for a level shifter cell depends on the state of each PG pin. The Liberty file contains sections for PG pin conditions of each mode/state and non-mission mode leakage power for each PG pin of that mode.

When this attribute is set to `true`, the software identifies the pg_pin mode and assigns non-mission mode leakage power associated with the mode.

The use model of the non-mission mode leakage power calculation feature is:

```
power_honor_non_mission_leakage true
power_off_pg_nets {VDD VDDO}
```

Example:
Let us consider a level shifter cell with the PG mode "`retained_VDDO_off`" and the following `pg_pin_condition`:

```
pg_setting_value (retained_VDDO_off) {
pg_pin_condition : "!VBB*VDD*VDDI ";
pg_pin_active_state (VBB, V1);
pg_pin_active_state (VDD, V2);
pg_pin_active_state (VDDI, V2);
```

When you specify a list of PG nets to be turned off along with the `power_honor_non_mission_leakage` attribute, all cells/instances that have the PG state "`retained_VDDO_off`" associated with the specified PG pin condition, will use the corresponding non-mission leakage power.

Related Commands

- set_power

power_hybrid_analysis

```
power_hybrid_analysis {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to perform both vector-based and vectorless power analysis in a single session.

Related Commands

- set_power

power_ignore_control_signals

```
power_ignore_control_signals {true | false}
```

Default: `true`

Data_type: Boolean

A value of `true` specifies that control signals will be ignored when propagating activity.

Related Commands

- set_power

power_ignore_data_phase_for_clock

```
power_ignore_data_phase_for_clock {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to ignore data frequency in the clock path for the clock instances in the design. When this attribute is set to `true`, the software propagates only the fastest clock pin frequency instead of the data pin frequency for activity propagation through clock network. In the default mode, the software picks the fastest frequency between the data phase and clock phase on that pin.

Related Commands

- `set_power`

power_ignore_end_toggles_in_profile

```
power_ignore_end_toggles_in_profile {true | false}
```

Default: `false`

Data_type: Boolean

By default, the software profiles the toggles in the user-selected vector time window by steps including the toggles at the start/end boundary of the window. In each step, the toggles on the start boundary edge of the step is counted in the current step; the toggles on the end boundary edge is counted in the next step.

When this attribute is set to `false`, the toggles on the end boundary edge is also counted in the last step, as there is no next step to it. This may lead to double count of the power profile for the last step, especially for zero-delay vector profiling.

When this attribute is set to `true`, the software ignores the toggle on the end boundary of the last step.

Related Commands

- `set_power`

power_ignore_glitches_at_same_time_stamp

```
power_ignore_glitches_at_same_time_stamp {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to ignore glitches at the same transient time in the vector-based static and dynamic power analysis flow. When this attribute is set to `true`, only the last signal value change is honored, and all the preceding value changes at the same time stamp are ignored. The last signal value will overwrite the previous value for the same signal at the same time stamp, so the last value will represent the state of the signal at that time stamp.

Related Commands

- `set_power`

power_ignore_inout_pin_cap

```
power_ignore_inout_pin_cap {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, ignores the bidirectional pin capacitances (`direction : inout`) defined for I/O cells in the .lib file, when calculating switching power and internal power; this also includes the default capacitance value for `inout` pins.

Related Commands

- set_power

power_ignore_macro_leakage_scale_for_temperature

```
power_ignore_macro_leakage_scale_for_temperature {true | false}
```

Default: `false`

Data_type: Boolean

The `power_leakage_scale_factor_for_temperature` attribute is used to perform linear scaling of the leakage power for temperature in both the standard cell and macro libraries. The `power_ignore_macro_leakage_scale_for_temperature` attribute allows you to selectively apply the scale factor only for standard cells, and not use the scale factor for macro cells.

Related Commands

- set_power

power_ignore_unconstraint_net_with_timing_data

```
power_ignore_unconstraint_net_with_timing_data {true | false}
```

Default: `false`

Specifies to ignore unconstrained nets/pins from toggle generation. These nets/pins do not have an associated clock but contain valid timing data with arrival windows in the TWF file. If set to `false`, the tool applies default frequency toggles to these unconstrained nets, which could cause false IR violation warnings due to high voltage drop.

Related Commands

- set_power

power_include_file

```
power_include_file file
```

Default: " "

Data_type: String

Specifies a power analysis include file. You can use this file to specify certain commands of the dynamic engine which do not have a Voltus equivalent. This attribute is used to pass the legacy power analysis commands and `report_power` command options only in the dynamic mode (vectorbased and vectorless). If the `power_method` attribute is set to `static`, the `power_include_file` attribute is ignored.
The `power_include_file` attribute is required only if the `power_static_netlist` attribute is set to `def`.

Related Commands

- set_power

power_include_initial_x_transitions

```
power_include_initial_x_transitions {true | false}
```

Default: `true`

Data_type: Boolean

Controls how the initial X state toggle should be counted. The initial X state means the state at time `0`.

When set to `true`, this means that the software counts the power caused by the `X->0` or `X->1` toggle. When set to `false`, the software does not count the power caused by the initial X state toggle.

Related Commands

- `set_power`

power_include_sequential_clock_pin_power

```
power_include_sequential_clock_pin_power {true | false}
```

Default: `false`

Data_type: Boolean

Reports the clock pin power of flip-flops as part of the clock network power.

Related Commands

- `set_power`

power_include_timing_in_current_file

```
power_include_timing_in_current_file {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to write timing bits for non-switching instances in the current files during the state-propagation-based flow. When specified, rail analysis will report effective instance voltage (EIV) for both switching and non-switching instances. If the parameter is set to `false`, rail analysis will report EIV for the switching instances only.

Related Commands

- `set_power`

power_ir_derated_timing_view

```
power_ir_derated_timing_view view_name
```

Default: `" "`

Data_type: String

Specifies the name of the timing view used to calculate timing slack and to rank the flops in the Tempus PI flow. This attribute must be specified if there are multiple views.

The `power_ir_derated_timing_view` attribute defines the timing view used to detect the IR-sensitive paths using a lower voltage than the actual analysis. This lower voltage simulates the IR-drop for each instance in the design. The software uses this view to prioritize the end-points that need to be prioritized for switching.

This attribute is only applicable to the state-propagation-based vectorless flow (`set_db power_method dynamic_vectorless power_enable_state_propagation true`).

Related Commands

- `set_power`

power_leakage_scale_factor_for_temperature

```
power_leakage_scale_factor_for_temperature view_name
```

Default: `1.0`

Data_type: Double

Performs a linear scaling of the leakage power for temperature in all libraries. This replaces the k-factor for temperature in the .lib file. For process and voltage, the scaling will continue to be based on the corresponding k-factors in the .lib file. A value of 0.8 scales the leakage power in the library to 80 percent. A value of 1.2 scales the leakage power in the library to 120 percent.

Related Commands

- set_power

power_lib_files

```
power_lib_files in_file <file1[file2] ...
```

Default: `""`

Data_type: String

Specifies the name of the timing library file(s) for dynamic power analysis. This attribute allows you to run the dynamic power analysis flow without reading a dotlib into design database to reduce background memory consumption. You must have an external TWF file to run this flow as the timing engine will not be invoked from within Voltus. This flow is recommended for large designs where front-end processing of timing libraries is high. Some of the database query commands will not work if this attribute is used.

Related Commands

- set_power

power_library_preference

```
power_library_preference {voltage | ecsm_ccsp}
```

Default: `voltage`

Data_type: String

Instances of the same cell can be attached to different power domains in a design. If you have specified different PVT libraries, then the software will by default bind each instance to the closest library definition.

You can use this attribute to select either the closest Effective Current Source Model (ECSM)/Composite Current Source-power (CCSP), or the closest voltage library for the instance:

- `voltage`: when specified, enables library binding to the closest voltage library.
- `ecsm_ccsp`: when specified, enables library binding to the closest ECSM/CCSP library.

Related Commands

- set_power

power_macro_pgv_leakage

```
power_macro_pgv_leakage {true | false}
```

Default: `true`

Data_type: Boolean

Controls leakage power reporting for cells that have power information available in both dotlib and PGV. When the attribute is set to `false`, the leakage current from Liberty is used and when the attribute is set to `true`, the leakage current from PGV is used.

Related Commands

- `set_power`

power_macro_toggle_pin_percentage

```
power_macro_toggle_pin_percentage percentage
```

Default: `0.0`

Data_type: Double

Specifies the percentage of output data pins (ignoring the control pins) to be scheduled for a macro cell. This attribute is applicable to the state-propagation based vectorless flow. The default macro pin toggle percentage is 50%.

Related Commands

- `set_power`

power_match_state_for_logic_x

```
power_match_state_for_logic_x value
```

Default: `x`

Data_type: String

Controls how the logic X will be evaluated in the boolean function of the '`when`' state of a power table.

The attribute uses the following arguments:

- `0` - regards X as 0
- `1` - regards X as 1
- `x/X` - regards X as neither `0` nor `1`, that is, whenever there is any X logic in it, the boolean function will be evaluated `0` (false).

Related Commands

- `set_power`

power_memory_current_scaling_method

```
power_memory_current_scaling_method {1 | 2 | 3}
```

Default: `0`

Data_type: Integer

Specifies to enable scaling of the NLPM-based currents by a user-specified scale factor for macro cells that are switching. The applicable arguments are `1`, `2`, and `3`, which means enable a scaling factor of `0.5`, `0.25`, and `0.1` respectively.

Related Commands

- `set_power`

power_multibit_flop_toggle_behavior

```
power_multibit_flop_toggle_behavior { simultaneous | independent | sbff }
```

Default: `independent`

Data_type: Enum

Specifies the toggle behavior of multi-bit flip-flops (MBFF). The possible arguments are:

- `simultaneous`: The multiple bits of the MBFF will toggle or not toggle at the same time. The MBFF will be considered as one flip-flop.
- `independent`: The multiple bits of the MBFF will toggle independently. The MBFF will be considered as one flip-flop.
- `sbff`: The MBFF will be considered as multiple Single-Bit Flip-Flops (SBFFs). Each bit will be considered as one flip-flop, and will toggle independently.

Related Commands

- `set_power`

power_multibit_flop_toggle_pin_percentage

```
power_multibit_flop_toggle_pin_percentage percentage
```

Default: `0.0`

Data_type: Double

Specifies the percentage of multi-bit flip-flops (MBFF) pins to be scheduled. This attribute is applicable to the state-propagation based vectorless flow. The attribute will only work when `power_multibit_flop_toggle_behavior` is set to `independent`.

Related Commands

- `set_power`

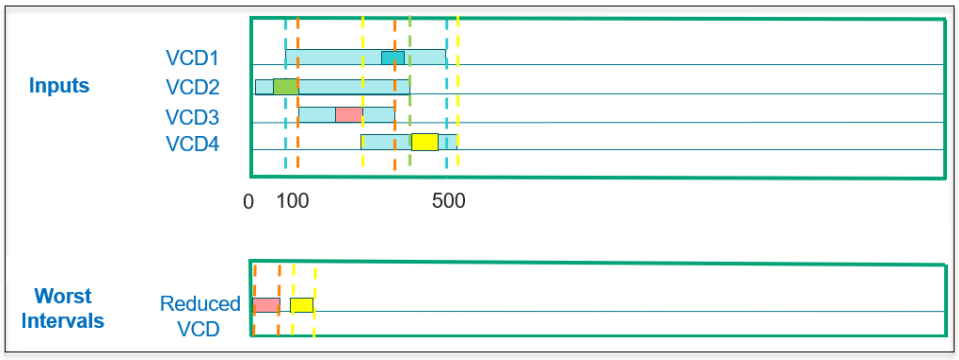
power_multi_scenario_simulation

```
power_multi_scenario_simulation {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to create a long vector by combining multiple vectors and to identify the worst power intervals across the combined vector. The long vector includes all scenarios that are used sequentially and are non-overlapping.



This attribute allows you to perform power and current analysis using worst peak power interval across multiple simulation vectors.

Related Commands

- set_power

power_off_pg_bulk_leakage

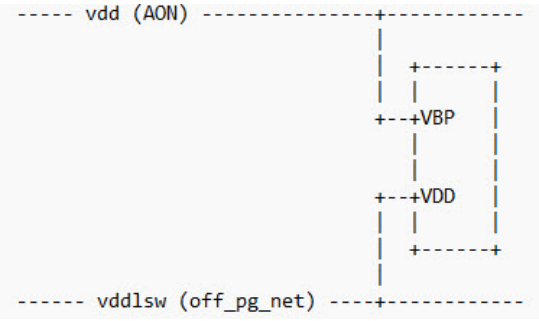
```
power_off_pg_bulk_leakage { true | false }
```

Default: `false`

Data_type: bool

Specifies to account for the leakage power through an instance's bulk pin when it is connected to a power-ground (PG) rail in the ON state.

The following diagram illustrated that the instance contributes bulk pin leakage to the ON state rail (**vdd**) through the *VBP* pin and zero leakage to the OFF state rail (`vddlsw`) through the VDD pin.



Related Commands

- set_power

power_merge_switched_net_currents

```
power_merge_switched_net_currents {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to merge switched net currents into always-on currents. For designs with a large number of switched nets, this attribute helps improve performance by merging switched net instance currents with always-on instance currents. However, when this attribute is set to `true`, you will not be able to use the generated current files to run power-up analysis.

Related Commands

- set_power

power_method

```
power_method { static | dynamic_vectorless | dynamic_vectorbased | dynamic_mixed_mode }
```

Default: `static`

Data_type: Enum

Specifies the type of analysis to be performed. You can perform two types of power analysis, namely static and dynamic.

- `static` - specifies to perform static power analysis to calculate average power.

Dynamic power analysis can be a vector-based or vectorless approach.

- `dynamic_vectorbased` - uses the full VCD output of a logic simulator to identify the instances that are switching and when they switch.
- `dynamic_vectorless` - uses the timing arrival window information from a static timing analysis tool to determine when instances switch. A timing arrival window describes when a signal can change within a clock cycle. An additional algorithm then determines the instances that switch.
- `dynamic_mixed_mode` - specifies to combine the vector-based and vectorless (mixed) dynamic analysis. This flow can be used in the following cases:
  - **case 1**: vectors are not available for every block of a full-chip design.
    In this case, the software uses the vectorless methodology for blocks with incomplete vectors
  - **case 2**: blocks with vectors having unannotated combinational/ sequential logic.
    In this case, the software performs state propagation for the missing combinational/ sequential logic. It uses the vectorless methodology for a logic cone that does not have the primary inputs or flops annotated from the vector, and then performs state propagation

    To enable the dynamic mixed mode analysis flow, you must specify the following commands:

    ```
    set_default_switching_activity -clock_gates_output 2.0 -global_activity <activity>
    set_db power_method dynamic_mixed_mode
    read_activity_file -scope <> -block <>
    ```
    -> (for available blocks)

Related Commands

- `set_power`

power_min_leaf_count

```
power_min_leaf_count value
```

Default: `0`

Data_type: Integer

Specifies the minimum number of leaf instances required for writing the hierarchy data in the profiling database. It allows you to restrict the number of hierarchies for which waveforms are reported in the profiling database when using both the `power_write_profiling_db` attribute and `report_power -hierarchy` parameters.

Any hierarchy whose leaf instance count in its complete sub-tree is less than the `power_min_leaf_count` value will be skipped from getting reported in the profiling database. This attribute can be used to skip very small hierarchies and as a result save on the database writing time and disk space.

Related Commands

- `set_power`

power_off_pg_nets

```
power_off_pg_nets net_list
```

Default: `""`

Data_type: String

The name of the power rails that will be turned off. The net name can contain instance hierarchy if appropriate. The software will take this data into account when figuring out the list of power gates which are switched off.

Note: A net is turned off if all connected power gates are turned off by control signals. When a net is off, all of the instances connected to the net will only contribute leakage current into the connecting nets.

Related Commands

- set_power

power_output_current_data_prefix

```
power_output_current_data_prefix prefix
```

Default: ""

Data_type: String

Specifies a prefix to the static or dynamic current files generated by the software. When power analysis is done in the multi-CPU mode, the output current data directory name and the underlying current files will also use the same prefix by default.

Related Commands

- set_power

power_output_dir

```
power_output_dir directory
```

Default: ""

Data_type: String

Specifies the name of the directory that power information will be output to.

Related Commands

- set_power

power_partition_count

```
power_partition_count value
```

Default: 0

Data_type: Integer

Specifies the number of physical partitions or jobs for a distributed run in the area-based partitioning flow. This attribute is only applicable to the event-based power analysis flow (power_method event_based) in the XP mode. The power_partition_count attribute must be specified with the power_use_physical_partition attribute.

Related Commands

- set_power

power_partition_twf

```
power_partition_twf {true | false}
```

Default: false

Data_type: Boolean

Specifies to read the input TWF data (flat or hierarchical TWF files), and generate block-level TWFs. This attribute must be specified when the block/partition level TWF files are not available.

Related Commands

- set_power

power_pin_based_twf

```
power_pin_based_twf  {true | false}
```

Default: `false`

Data_type: Boolean

Specifies whether to generate pin-based TWF or net-based TWF.

- When set to `false`, a net-based TWF file is generated. Net-based TWF file is smaller in size and therefore results in faster processing.
- When set to `true`, a pin-based TWF file is generated. Pin-based TWF file is bigger in size and requires longer processing time, however, is more accurate because the slews numbers are pin-based.

Note: This attribute must be used with the `read_sdc` command.

Related Commands

- set_power

power_precision

```
power_precision value
```

Default: `8`

Data_type: Integer

Specifies precision of decimal range [1-8] to ensure consistent decimal places are displayed for each power component. This attribute is used for precision handling in `report_instance_power` and `report_power`.

Related Commands

- report_power

power_pre_simulation_empty_period

```
power_pre_simulation_empty_period value
```

Default: `""`

Data_type: String

Specifies to add an empty pre-simulation period before simulation during IR drop analysis. Power analysis is not done for the pre-simulation period and flat dynamic currents are created.

Related Commands

- report_power

power_pre_simulation_period

```
power_pre_simulation_period value
```

Default: `""`

Data_type: String

Specifies to set a pre-simulation period for IR drop analysis. Pre-simulation is used for generating power analysis reports and dynamic currents.

Related Commands

- report_power

power_pre_simulation_power_exclude_period

```
power_pre_simulation_power_exclude_period value
```

Default: ""

Data_type: String

Specifies to exclude power analysis during the pre-simulation period used for IR drop analysis. Pre-simulation is used only to create dynamic currents and not used for power reports.

Related Commands

- report_power

power_quit_on_activity_coverage_threshold

```
power_quit_on_activity_coverage_threshold threshold_value
```

Default: 0.0

Data_type: Double

Specifies the activity (VCD/FSDB/TCF) coverage threshold value. This attribute allows you to control the acceptable activity coverage that you are expecting and enforce the software to quit when it does not meet the criteria. When you specify the threshold value, the software exits if the real activity coverage is below the threshold.

Related Commands

- report_power

power_read_rcdb

```
power_read_rcdb {true | false}
```

Default: false

Data_type: Boolean

Specifies to write a SPEF file containing only the total C for signal nets and pass it to the Dynamic Power engine. Dynamic power analysis requires only the total C for signal nets. Therefore, instead of writing a large SPEF file (with distributed RC) from the RCDB, this attribute specifies to create and pass a SPEF file with only total C for all the signal nets. This significantly reduces the parasitic annotation time in the Dynamic Power engine.

Related Commands

- report_power

power_relax_arc_match

```
power_relax_arc_match  {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to relax arc match requirements. This attribute is used only when the exact arc is not found in a .lib. When set to `true`, the internal energy is calculated as an average of multiple arcs with relaxed arc matching. If this attribute is set to `false`, relaxed arc search is not done and 0 internal energy is returned.

Related Commands

- report_power

power_report_library_usage

```
power_report_library_usage {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to report library usage statistics in a text report. The report contains information such as, cell name, library format used (CCSP/ECSM/NLPM) for each cell, library name, and so on. This attribute allows you to perform library integrity checking for debugging purpose.

The format of the text report with sample content is given below:

Example:

#cell_name	net_name	op_voltage	lib_voltage	lib_format	missing_tables	library_name
INV	VDD,VDDA	0.9	1.0	NLDM	leakage,timing	A
INV	VDDC	0.75	0.75	CCSP,NLDM	leakage	B

Related Commands

- report_power

power_report_missing_bulk_connectivity

```
power_report_missing_bulk_connectivity {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to report missing bulk pins in the missing input report (`power_report_missing_input`).

Related Commands

- report_power

power_report_missing_input

```
power_report_missing_input {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to report missing netlist information in input files, such as library, LEF/DEF, PGV, SPEF, TWF, and missing logical connectivity for a

cell instance (PGNET).

The name of the report file is: `<prefix>_missingdata.<spef|lefdef|lib|pgv|twf|pgnet>`

Related Commands

- report_power

power_report_black_boxes

```
power_report_black_boxes {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to report cells that are used as black boxes.

A black box is a cell which is either missing from the liberty file or has no functionality defined in the liberty file.

Related Commands

- report_power

power_report_clock_instance_switching_info

```
power_report_clock_instance_switching_info
```

Default: None

Data_type: String

Specifies to generate a report for clock instances that switched during the state-propagation based vectorless flow. The generated report contains the toggle time, instance name, input pin event, and output pin event for each switching event of the clock instance.

Related Commands

- report_power

power_report_idle_instances

```
power_report_idle_instances {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to generate a report (`idleinstance.rpt`) that lists all the idle or non-switching instances.

The following is a snippet of the `idleinstance.rpt` report:

```
DFF22/DFF    F
DFF42/DFF    F
...
```

Here, the first column is the instance name and the second column is the type of instance (`F` indicates flop and `C` indicates combinational).

Related Commands

- report_power

power_report_instance_switching_info

```
power_report_instance_switching_info {all | output_logic | none}
```

Default: `none`

Data_type: Enum

Specifies to generate a report for combinational instance switched during state-propagation. In the output file, it contains the toggle time, instance name, input pin event, and output pin event for each switching event of the combination gate. The possible arguments of this attribute are:

- `all` - Specifies to report instance toggle information for both input and output pins. When this attribute is set to `all`, the switching of both the input and output pins of all the instances are reported in the `voltus_power.stateprop.switchinst` file and the switching of flops are reported in the `voltus_power.stateprop.switchsrc` file.
- `output_logic` - Specifies to report instance toggle information for the output pins only. When this attribute is set to `output_logic`, the switching of the output pins of all the instances are reported in the `voltus_power.stateprop.switchinst` file and the switching of flops are reported in the `voltus_power.stateprop.switchsrc` file.
- `none` - Disables generation of instance toggle report. This is the default option. When this attribute is set to `none`, only switching of flops are reported in the `voltus_power.stateprop.switchsrc` file. The file does not report switching of instances.

This attribute is used for generating switching details of state-propagation and should be specified only for debugging purpose.

The following is a snippet of the `voltus_power.stateprop.switchinst` report:

```
*-------------------------------------------------
*Switch Time   Instance Name   Pin Name   Logic
*-------------------------------------------------
*
   0.097        B11/BUFF         I        rise
   0.097        DFF11/DFF        Q        rise
...
```

Here,

- `Switch Time`: switching time of instances that can be either combinational or sequential
- `Instance Name`: instance name
- `Pin Name`: input/output pin of the instance
- `Logic`: present logic of the pin that can be either "rise" or "fall"

Related Commands

- `report_power`

power_report_instance_switching_list

```
power_report_instance_switching_list filename
```

Default: " "

Data_type: String

Specifies to generate a report (`voltus_power.stateprop.switchlist`) containing all the switching times for the specified instances when the `power_enable_state_propagation` attribute is set to `true`.

The format of the specified file is:

```
instname1
instname2
...
```

The following command is an example of the state-propagation-based dynamic vectorless flow, and also specifies to generate a report (`voltus_power.stateprop.switchlist`) containing all the switching times for the instances specified in the `inst_list` file:

```
set_db power_method dynamic_vectorless power_enable_state_propagation true power_report_instance_switching_list inst_list
```

The following is the format of the `voltus_power.stateprop.switchlist` report:

```
*-----------------------------------------------------------------------------------------
*Instance Name   Pin Name   Pin Direction   Frequency   Switch Time
*-----------------------------------------------------------------------------------------
*
```

Here,

- `Instance Name`: instance name that can be either combinational or sequential
- `Pin Name`: name of the input/output pin of the instance
- `Pin Direction`: indicates whether the pin is an INPUT (I) or OUTPUT (O) pin
- `Frequency`: clock frequency
- `Switch Time`: gives the switching pattern for the instance, that is, <*time1*> <*logic*> <*time2*> <*logic*>. The logic of the pin can be "`r`" or "`f`". `r` indicates rise and `f` indicates fall.

The following is a snippet of the `voltus_power.stateprop.switchlist` report:

```
*-----------------------------------------------------------------------------------------
*Instance Name   Pin Name   Pin Direction   Frequency   Switch Time
*-----------------------------------------------------------------------------------------
*
   DFF11/DFF        QN        O              100       0.106 f 10.107 r
   DFF11/DFF        Q         O              100       0.097 r 10.099 f
   DFF11/DFF        CP        I              100       0.059 r 5.056 f 10.059 r 15.056 ...
...
```

Related Commands

- `report_power`

power_report_missing_nets

```
power_report_missing_nets {true | false}
```

Default: `false`

Data_type: Boolean

Controls reporting of missing nets in both static and dynamic power analysis.

This attribute controls reporting for TWF and activity files in static analysis, and controls reporting for TWF/SDC/SPEF and activity files in dynamic analysis.

Related Commands

- `report_power`

power_report_scan_chain_statistics

```
power_report_scan_chain_statistics {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to generate a report file called `voltus_power.scanchain.stat` during scan mode analysis that contains scan chain statistics.

The report has 2 parts:

- **Global Statistics**: this section has the cumulative statistics of all scan chains in the design
- **Chain Statistics**: this section has the statistics for each scan chain in the design

The Global and Chain Statistics sections have the following information:

- `Total`: total instances specified in the control file or in a scan chain
- `AssignedFF`: flip-flops defined in the scan control file which exist in the design
- `SwitchedFF`: flip-flops that have switching event at the output
- `Non-ScanFF`: flip-flops that have no scan-out pin
- `DuplicatedFF`: flip-flops that are specified in multiple chains
- `StateConflictFF`: flip-flops that have conflicting state assignment

Related Commands

- `report_power`

power_report_statistics

```
power_report_statistics {true | false}
```

Default: `false`

Data_type: Boolean

Reports a set of statistics on the instance power, instance power density, clock power, or transition density. This attribute is useful in viewing and debugging user defined activity factors.

Related Commands

- `report_power`

power_report_time_display_fraction_digits

```
power_report_time_display_fraction_digits value
```

Default: `-1`

Data_type: Integer

Sets the number of fractional digits or decimal places displayed for the instance switching time values in the `voltus_power.stateprop.switchinst` file. This attribute displays fractional digits that show the exact time difference if there are two events (rise/fall) of an instance switching at the same time.

Related Commands

- `report_power`

power_report_twf_attributes

```
power_report_twf_attributes {summary| detailed}
```

Default: ""

Data_type: String

Controls the generation of the TWF attribute report based on the TWF attributes that are specified using the `set_twf_attribute` command.

The possible arguments are:

- `summary` - generates a summarized static timing analysis report "`voltus_power.twf_attribute.summary`" that displays the number of user-defined TWF attributes (delay, slew, and slack values for both the rise and fall transitions) and the number of annotated TWF attributes for dynamic power analysis. The following is a snippet of the report:

```
STATISTICS:
                  User Defined            Annotated               Annotated/Total(%)
shift_arrival_time       5                       4                       80
rise_slew                2                       2                       100
fall_slew                2                       2                       100
rise_delay               5                       4                       80
fall_delay               5                       4                       80
rise_slack               2                       2                       100
fall_slack               2                       2                       100
```

- `detailed` - generates a detailed static timing analysis report " `voltus_power.twf_attribute.detailed`" that includes the summary report, and a detailed report of the rise and fall transitions for the TWF attributes of each annotated instance/pin. It also includes the shift arrival time report.

  This argument also allows you to generate a report `voltus_power.twf.rpt` containing the TWF statistics to highlight clock-frequency and delay used for the static and dynamic power calculation.

  The TWF statistics report is generated only if:

  - the `read_twf` command is specified before specifying the `power` category attributes.
  - the `power_method static power_static_netlist def` or `power_method dynamic_vectorless | dynamic_vectorbased` command is specified.

  The format of the `voltus_power.twf.rpt` report is:

```
# Voltus TWF Statistics
# Units: Freq (Hz), Delay (second), Slew (second)
# TWF File List:
#      TWF_file1
#      TWF_fileN

USER DEFINED TWF ATTRIBUTE STATISTICS
<Clock Freq> <Min Rise_Delay> <Rise Delay> <Max Rise Delay> <Min Fall Delay> <Fall Delay> <Max Fall Delay> <Net>
```

  The `USER DEFINED TWF ATTRIBUTE STATISTICS` section is included in the report only if the TWF attributes are defined using the `set_twf_attribute` command.

Example :

#check all instances of inst_C1 get shifted by 3ns

```
set_twf_attribute -pin inst_C1 -shift_arrival_time 3
```

#check all instances of inst_D1 get shifted by 3ns

```
set_twf_attribute -net inst_D1 -shift_arrival_time 3

set_db power_method dynamic_vectorless ; set_db power_write_db false ; set_db power_disable_static false ; set_db
power_enable_state_propagation true; set_db power_current_generation_method peak ; set_db power_grid_libraries
/../library_pv.cl ; set_db power_report_twf_attributes summary
```

Related Commands

- `report_power`

power_reuse_flop_ranking_data

```
power_reuse_flop_ranking_data directoryname
```

Default: ""

Data_type: String

Specifies the name of the directory where the flop ranking data to be reused is placed.

Related Commands

- report_power

power_reuse_flop_ranking_data_hier

```
power_reuse_flop_ranking_data_hier blockname
```

Default: ""

Data_type: String

Allows you to choose the block/hierarchy whose ranking data is to be used for the block-level analysis. This attribute is used when there are multiple instantiations of the same DEF block.

Related Commands

- report_power

power_scale_to_sdc_clock_frequency

```
power_scale_to_sdc_clock_frequency {true | false}
```

Default: false

Data_type: Boolean

Specifies to enable the VCD clock frequency to be scaled up to the SDC clock frequency. This allows you to match the VCD clock to the SDC clock to run analysis. By default, the clock frequency is scaled based on the activity file. When you set the power_scale_to_sdc_clock_frequency attribute to true, the power report and current waveforms apply the after-scaled clock frequency.

The default behavior for clock frequency scaling in the RTL and Gate level VCD flows is different, as described below:

- **RTL VCD** - By default, the clock frequency is scaled to the SDC clock frequency (power_scale_to_sdc_clock_frequency true) when the RTL VCD flow is enabled (power_enable_rtl_dynamic_vector_based true).
  You can disable clock frequency scaling by setting the power_scale_to_sdc_clock_frequency attribute to false.
- **Gate VCD** - By default, scaling of the clock frequency is disabled (power_scale_to_sdc_clock_frequency false) .
  You can enable the VCD clock frequency to be scaled up to the SDC clock frequency by setting the power_scale_to_sdc_clock_frequency attribute to true. This works only when power_use_zero_delay_vector_file is set to true, which means that the gate VCD is a zero-delay gate VCD.

Related Commands

- report_power

power_scan_chain_activity

```
power_scan_chain_activity {design_name  chain_name  activity_number} | {vectorless  activity_number}
```

Default: ""

Data_type: String

Specifies to either provide vectorless activity for all scan chains or provide a specific activity for a specific chain. This attribute allows you to specify vectorless or user-defined activity to scan chains.

The following are the two use models:

- Generate the scan control file for a specific chain:
  ```
  set_db power_scan_chain_activity {design_name  chain_name  activity_number}
  ```
- Generate the scan control file for all scan chains:
  ```
  set_db power_scan_chain_activity {vectorless  activity_number}
  ```

The `power_scan_chain_activity` attribute is supported only in the XP mode.

Related Commands

- `report_power`

power_scan_chain_name_pattern

```
power_scan_chain_name_pattern {design_name  chain_name  pattern}
```

Default: `""`

Data_type: String

Specifies to automatically generate a scan-chain control file from a scan-chain based DEF file for the shift mode scan-chain analysis. This attribute allows you to create a control file for a large-sized DEF containing long chains or for multiple-DEF files. The generated scan-chain control file is saved in the power output directory.

The use model of the attribute is:

- `power_scan_chain_name_pattern {all pattern}` - generate scan control file for all scan chains.
  Example: `power_scan_chain_name_pattern {all 11010}`
- `power_scan_chain_name_pattern {design_name  chain_name  pattern}` - generate scan control file for a specific chain.
  Example: `power_scan_chain_name_pattern {counter AutoChain_1_seg1_clk_rising 10010}`

Using this attribute, you can generate the control file for both flat and hierarchical DEFs. For more information, refer to Scan Control File Format.

Related Commands

- `report_power`

power_scan_control_file

```
power_scan_control_file  filename
```

Default: `""`

Data_type: String

Specifies the scan control file name required to run the Scan Mode Analysis flow.

The scan control file contains details about the scan analysis mode used for testing. For more information, refer to Scan Control File Format.

Related Commands

- `report_power`

power_set_power_event_based

```
power_set_power_event_based {true | false}
```

Default: `false`

Data_type: Boolean

Specifies the instance-based power constraints that are to be taken into consideration for the power analysis flow. The attribute controls whether the power constraints are applied to the dynamic or static flow. The power constraints are specified using the `set_power` command.

The default value of `false` specifies to apply the power constraints in the static flow. When set to `true`, the constraints are applied to the dynamic flow.

Example:

The following command specifies to honor the instance-based power constraints in the dynamic power analysis flow:

```
set_db power_method event_based power_set_power_event_based true
set_power -ascii_power_file instPower.txt
report_power -out_file power.rpt
```

Related Commands

- `report_power`

power_settling_buffer

```
power_settling_buffer value
```

Default: " "

Data_type: String

Specifies the settling buffer time (in ps unit) between multiple windows of a VCD file. This attribute is used when you specify multiple pairs of the start time and end time for the non-overlapping multiple windows specified in an activity file (using the `read_activity_file` command).

Multiple current waveforms representing multiple VCD windows are merged into a single continuous current waveform. To capture the waveform for signals at the edge of user windows, a buffer time is added between the windows being stitched. This buffer time ensures that the stitched waveform is contiguous. The generated waveform hence has the duration of user windows and the intervening settling buffers. The default settling buffer period is 400ps.

You must specify the buffer value without unit.

Related Commands

- `report_power`

power_show_cell_usage_stats

```
power_show_cell_usage_stats {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to show the cell usage statistics in the power report. It provides cell usage statistics alongside power values, indicating the number of cells used from a specific library and the corresponding power percentage. This helps identify libraries with the greatest and least impact on power consumption.

You must ensure that this attribute is specified before using the `report_power` command, as shown below:

```
set_db power_show_cell_usage_stats true
report_power -out_file p1.rpt
```

Following is a snippet from the report:

```
Total leakage power = 0.00118471 mW
```

```
Cell usage statistics:
Library j1os_h , 88 cells ( 7.351713%) , 6.56434e-06 mW ( 0.554089%  )
Library j1os_l , 123 cells ( 10.275689%) , 0.000799809 mW ( 67.511081%  )
Library j1os , 986 cells ( 82.372597%) , 0.000378334 mW ( 31.934830%  )
```

Related Commands

- report_power

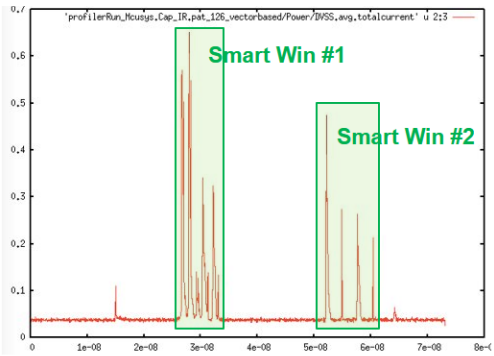
power_smart_window

```
power_smart_window {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to enable the smart window feature to extract peak power scenarios from a vector-based activity file. A smart window is a variable size window to capture maximum regions of high power dissipation. An activity file can have multiple variable-sized regions that are ranked in the descending order of power metric.



The use model of the `power_smart_window` attribute is:

```
set_db power_smart_window true
set_db power_worst_step_size value
set_db power_worst_window_type [ power | activity | delta_power | delta_activity | ir | critical_inst_power ]
```

Related Commands

- report_power

power_scan_multi_bit_flop_chain_type

```
power_scan_multi_bit_flop_chain_type type
```

Default: `liberty`

Data_type: String

Specifies the type of multi-bit scan flip-flop.

- `liberty` - specifies to use the syntax in Liberty to decide the multi-bit flip-flop type.
- `serial` - specifies to pick the first flop as the starting flop, and treat the scan chain as serial, regardless of .lib.
- `parallel` - specifies to treat the scan chain as parallel, regardless of .lib.

Related Commands

- report_power

power_split_bus_power

```
power_split_bus_power {true | false}
```

Default: false

Data_type: Boolean

A value of false applies the internal power number to each individual bit.

A value of true divides the internal power number by bus width, before applying it to the bus bit.

Related Commands

- report_power

power_start_time_alignment

```
power_start_time_alignment {true | false}
```

Default: true

Data_type: Boolean

Specifies to align all the signals of the activity file at time zero. When set to true, the simulation period is max(endtime - starttime). When it is false, the simulation period is max(endtime)- min(startime).

Note: This attribute is not supported in the vector profiling flow. In the vector profiling flow, the start time will always be the min(starttime) and the end time will always be max(endtime)

Related Commands

- report_power

power_state_dependent_leakage

```
power_state_dependent_leakage {true | false}
```

Default: true

Data_type: Boolean

When set to false, power analysis performs state independent leakage power calculation.

Related Commands

- report_power

power_stateprop_ignore_unannot_pins

```
power_stateprop_ignore_unannot_pins {true | false}
```

Default: false

Data_type: Boolean

Specifies to ignore unannotated pins in the functional logic of output pins for the RTL propagation flow. This attribute is set to true if there are

unannotated input pins for instances and such pins are used in the functional logic of output pins that are propagated. For example output pin Q is propagated with functional logic A & B where A is unannotated and might cause wrong propagation for the Q pin.

Related Commands

- report_power

power_static_multi_mode_scenario_file

```
power_static_multi_mode_scenario_file  filename
```

Default: ""

Data_type: String

Specifies a file with information on instance scaling scenario for the static multi-mode IR analysis flow. When this parameter is specified, Power Analysis creates the multi-mode static current file that is used during rail analysis. The format of the input file is:

```
INST_NAME       PIN_NAME        SCALE_FACTOR_1       …..   SCALE_FACTOR_N
```

Here, the first column indicates the macro instance name, the second column indicates the pin name, followed by multiple current scaling factors. Each column of the scaling factor is referred to as a scenario. You must specify positive values only for scale factors.

The following is a snippet from the specified input file:

```
top/macroA     VDD     1     0.5     0.5     1
top/macroA     VDDM    0.5   0.5     0.5     1
top/macroB     VDD     0.5   1       0.5     0.5
```

If scale factors are not specified for certain scenarios, the tool will automatically use the instance's last stored scale factor value to complete current file generation.

In the following example, scale factor value `0.45` will be used in the 3rd and 4th scenario for instanceC, and scale factor value `0.35` will be used in the 4th scenario for instanceD:

```
instanceA VDD 0.45 0.55 0.65 0.31
instanceB VDD 0.31 0.5 0.67 0.21
instanceC VDD 0.25 0.45
instanceD VDD 0.31 0.25 0.35
```

Related Commands

- report_power

power_static_netlist

```
power_static_netlist  {verilog | def}
```

Default: `verilog`

Data_type: String

Specifies to perform static power analysis using a Verilog or DEF only neltist.

By default, the static power analysis is performed using the Verilog netlist as the primary netlist. Using this attribute, you can specify DEF when a DEF netlist with logical connectivity is available, and use it as primary and not consider the Verilog netlist.

This attribute is also enabled when `set_rail_analysis_config -report_power_in_parallel` is `true` for static power computation.

Related Commands

- report_power

power_switching_power_on_rise_only

```
power_switching_power_on_rise_only {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to perform switching power analysis only for rise toggles.

Related Commands

- report_power

power_thermal_input_file

```
power_thermal_input_file  file
```

Default: `""`

Data_type: String

Specifies to read a power map file to perform thermal analysis. Thermal input file contains die area divided into tiles, and temperature value for each tile. This temperature of each tile is used to calculate power of all the instances that fall in that tile's bounding box.

The format of the thermal input file is:

```
DIE_TEMPERATURE_MAP
DIE_AREA   xmin(um) ymin(um) xmax(um) ymax(um) nx(int) ny(int)

NUMBER_OF_LAYERS    nLayers(int)
LAYER    layer_name(string)
TEMPERATURE

XCOOR    x0(0)    x0(1)    x0(2)... ...x0(nx-1)
YCOOR    y0(0)    y0(1)    y0(2)... ...y0(ny-1)

T0(0,0)    T0(1,0)    ...    T0(nx-1,0)
T0(0,1)    T0(1,1)    ...    T0(nx-1,1)

:
T0(0,ny-1)    T0(1,ny-1)         T0(nx-1,ny-1)
END_TEMPERATURE
END_LAYER
END_DIE_TEMPERATURE_MAP
```

where,

- `DIE_AREA` specifies the coordinates of the four points of die outline as well as the tile number in X and Y direction.
- `NUMBER_OF_LAYERS` is the number of layers in each die instance
- `LAYER` specifies the LEF layer name
- XCOOR and YCOOR are the coordinates of each tile
- `T0(0,0)` …. is the temperature value assigned to a tile at a specific coordinate

Related Commands

- report_power

power_thermal_leakage_temperature_scale_table_file

```
power_thermal_leakage_temperature_scale_table_file file
```

Default: ""

Data_type: String

Specifies a file containing the user-defined temperature values and temperature scale factors for each instance. This attribute allows you to use the current file (.ptiavg) with re-adjusted leakage power values to perform thermal-aware IR drop analysis.
The `power_thermal_leakage_temperature_scale_table_file` attribute must be specified with the `power_thermal_input_file` attribute to import the temperature map file generated by Celsius.

When the new attribute is specified, the generated power will include the scaled leakage power values.

The following is the format of the temperature-aware leakage scaling table:

```
#Type    Instance_Name        temp1             temp2              temp3              temp4
(Type: Lib/Cell/inst)

inst     inst_name1     scalefactor@temp1   scalefactor@temp2  scalefactor@temp3   scalefactor@temp4
inst     inst_name2     scalefactor@temp1   scalefactor@temp2  scalefactor@temp3   scalefactor@temp4
```

File Format Example:

```
#Type     Name        25    50    125    150

inst     ring/b213    1     2     5      10
inst     ring/b212    1     2     5      10
inst     ring/b211    1     2     5      10
inst     ring/b210    1     2     5      10
```

Related Commands

- `report_power`

power_to_x_transition_factor

```
power_to_x_transition_factor value
```

Default: `0.5`

Data_type: Double

Specifies how 0/1 to X transitions are counted.

Related Commands

- `set_power`

power_to_z_transition_factor

```
power_to_z_transition_factor value
```

Default: `0.25`

Data_type: Double

Specifies how 0/1 to Z transitions are counted.

Related Commands

- `set_power`

power_transition_factor_based_duty

```
power_transition_factor_based_duty {true | false}
```

Default: `false`

Data_type: Double

Specifies that duty for X and Z toggles is calculated using the X and Z transition factor of toggle specified using the following attributes:

- `power_from_x_transition_factor`
- `power_from_z_transition_factor`
- `power_to_x_transition_factor`
- `power_to_z_transition_factor`

The default value of `false` specifies to use the duty value given with the `set_default_switching_activity`/`set_switching_activity -duty` parameter.

Related Commands

- `report_power`

power_transition_time_method

```
power_transition_time_method  {min | avg | max}
```

Default: `max`

Data_type: String

Specifies the minimum, maximum, or average transition time method that will be used with the integrated timer or the external TWF.

Related Commands

- `report_power`

power_twf_delay_annotation

```
power_twf_delay_annotation {min | avg | max}
```

Default: `avg`

Data_type: String

Allows you to choose between min, max, or avg arrival times from the timing window file (TWF). The arrival time is annotated to the vectors processed from the RTL VCD/FSDB Zero-Delay flows.

Related Commands

- `report_power`

power_twf_load_cap

```
power_twf_load_cap {min | avg | max}
```

Default: `max`

Data_type: String

Allows you to select minimum, maximum, or average value of TWF external load or capacitance for power calculation.

Related Commands

- report_power

power_unified_power_switch_flow

```
power_unified_power_switch_flow {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to enable the unified analysis of the always-on and switch nets. When set to `true`, the power-up and always-on analyses are performed in a single flow, wherein the switch nets and always-on nets are simulated together resulting in improved performance and accuracy.

Related Commands

- report_power

power_use_cell_leakage_power_density

```
power_use_cell_leakage_power_density {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to use library leakage density times area (`default_leakage_power_density`), if defined, instead of the library's default cell leakage power (`default_cell_leakage_power`). This is used when leakage power for cells are missing or not defined.

Related Commands

- report_power

power_use_fastest_clock_for_dynamic_scheduling

```
power_use_fastest_clock_for_dynamic_scheduling {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to use only the respective fastest clock associated with each net or pin to schedule events for a given simulation period.

Related Commands

- report_power

power_use_lef_for_missing_cells

```
power_use_lef_for_missing_cells {true | false}
```

Default: `false`

Data_type: Boolean

Allows a combination of LEF and PGV in the dynamic analysis flow. If there are cells that are common to LEF and PGV, the PGV cell will get the precedence.

Related Commands

- report_power

power_use_only_ddv_current

```
power_use_only_ddv_current {true | false}
```

Default: true

Data_type: Boolean

Controls the use of static current of a detailed dynamic power-grid view (DDV PGV) cell for static power calculation. The parameter is set to true by default, that is, the DDV PGV static current is not used in the static power calculation flow. If you want to use the DDV PGV static current for static power calculation, set this parameter to false.

Related Commands

- report_power

power_worst_case_vector_activity

```
power_worst_case_vector_activity {true | false}
```

Default: false

Data_type: Boolean

Specifies to use the worst activity value when multiple vectors are specified in the static power calculation flow.

Related Commands

- report_power

power_worst_step_size

```
power_worst_step_size value
```

Default: ""

Data_type: String

Specifies the step size or resolution for vector profiling. Activity/ Power profiling is carried out for every step size that you specify.

When this attribute is used with power_worst_window_size, the specified step size is the resolution at which window sliding is performed. The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns will be applied.

Related Commands

- report_power

power_worst_window_count

```
power_worst_window_count value
```

Default: 1

Data_type: String

Specifies the number of worst windows from the last vector profile run to be used for dynamic power analysis. By default, the software implements a single worst window. This attribute provides n top windows with maximum activity/power/IR. This attribute is used when you

need to combine dynamic power analysis and vector profiling for multiple worst-case windows in a single run.

Related Commands

- report_power

power_worst_window_size

```
power_worst_window_size value
```

Default: ""

Data_type: String

Specifies the duration of dynamic analysis. This attribute is used for sliding window vector profiling that involves sliding the profiling windows by using a very small resolution value to get the window with the worst current profile. power_worst_window_size must be specified with power_worst_step_size.

The supported time units are s, ms, us, ns, and ps. If no explicit time unit is specified, the default ns will be applied.

Related Commands

- report_power

power_worst_window_type

```
power_worst_window_type  {power | activity | delta_power | delta_activity | ir | critical_inst_power}
```

Default: ""

Data_type: String

Specifies the window type to be used for multi-window analysis. The possible arguments are:

- power - worst instance power
- activity - worst instance activity
- delta_power - worst dp/dt (rate of change of power)
- delta_activity - worst da/dt (rate of change of activity)
- ir - worst IR
- critical_inst_power - worst power for critical/IR-sensitive instance (worst timing and IR)

Related Commands

- report_power

power_worst_window_coverage

```
power_worst_window_coverage { top | tile }
```

Default: ""

Data_type: String

Specifies how the worst window type is applied for multi-window analysis in the vector profiling flow. The following are the possible options of this attribute:

- top: worst window selection for the full design

- `tile`: interval selection as per the worst tile power

Use Model:
```
set_db power_method event_based power_worst_window_coverage tile
report_power -out_file power.rpt -power_density_tiles 100
```

This flow should be run with `power_method event_based`.

When the `power_worst_window_coverage tile` attribute is specified, you should include any one of the following parameters to specify the total number of tiles:

- `report_power -power_density_tiles_size " 15 15 "`
- `report_power -power_density_tiles_row_column " 2 2 "`
- `report_power -power_density_tiles 100`

The software generates the `power.rpt.density` report that has the power density tiles profile report with an additional section "Intervals covering unique tiles having worst power density".

Related Commands

- `report_power`

power_worst_window_reports

```
power_worst_window_reports {full | worst | both}
```

Default: `full`

Data_type: String

Specifies to generate static power analysis reports for the specified worst window type. The possible arguments are:

- `full`: creates the static power report for the full simulation window specified with the `-start`/`-end` parameters of the `read_activity_file` command.
- `worst`: creates the static power report for the worst window only.
- `both`: creates the static power report for both the full simulation and worst windows.

The worst window report is created with '**.1**' added in the file name, as shown below:

- Static reports for full simulation: `power.rpt.avgpwr`, `power.rpt.instpwr`
- Static reports for worst window: `power.rpt.1.avgpwr`, `power.rpt.1.instpwr`

Related Commands

- `report_power`

power_write_dynamic_currents

```
power_write_dynamic_currents {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to create dynamic current files either for full simulation duration or for user-specified duration. This attribute is only applicable to the event-based power analysis flow (`power_method event_based`).

Example:
```
set_db power_method event_based power_write_dynamic_currents true power_enable_state_propagation true
read_activity_file sim.vcd -format vcd
```

```
report_power -out_file power.rpt
```

Related Commands

- report_power

power_use_physical_partition

```
power_use_physical_partition {true | false}
```

Default: `false`

Data_type: Boolean

Enables tile or area-based partitioning of a design for distributed processing. When this attribute is specified, the top-level design is divided into physical tiles, which are then grouped to ensure that each group or partition has the same instance count. Each partition is considered and processed as a separate job.

This attribute is only applicable to the event-based power analysis flow (`power_method event_based`) in the XP mode. The `power_use_physical_partition` attribute must be specified with the `power_partition_count` attribute.

Example:

The following command specifies to enable area-based partitioning, and distribute the event-based analysis into four jobs:

```
set_db power_method event_based power_enable_xp true power_use_physical_partition true power_partition_count 4
report_power -distribute
```

Related Commands

- report_power

power_use_twf_stable_randomized_arrival_delay

```
power_use_twf_stable_randomized_arrival_delay  {none | data_only | all}
```

Default: `none`

Data_type: String

Specifies to annotate randomized arrival time based on net names. The parameter ensures that when there is a change in design, the net delay is not impacted. This parameter is supported in both the static and dynamic flows.

The possible arguments are:

- `none` - disables randomized timing window annotation.
- `all` - performs randomized annotation for both the clock and data nets.
- `data_only` - performs randomized annotation only for the data nets.

Related Commands

- report_power

power_use_zero_delay_vector_file

```
power_use_zero_delay_vector_file {true | false}
```

Default: `false`

Data_type: Boolean

Enables zero delay mode in vector-based dynamic analysis to avoid pessimism in current estimation. When set to `true`, the software reads

the cycle accurate (zero delay) VCD/FSDB file and adds the timing/delay information from the Timing Analyzer or external TWF file to the VCD/FSDB file.

Related Commands

- `report_power`

power_vector_based_multithread

```
power_vector_based_multithread {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to enable multi-threading for the VCD/FSDB based dynamic vector-based flows.

To enable multi-threading in the vector-based flows, set this attribute to `true`.

Related Commands

- `report_power`

power_vector_profile_mode

```
power_vector_profile_mode {activity | event_based | power_density | transient}
```

Default: `event_based`

Data_type: Enum

Specifies the method used to perform vector profiling. The possible arguments are:

- `activity` - performs activity profiling. This parameter generates only the worst activity report, and does not generate the average power report. The name of the file is as specified using the `report_power -out_file` parameter.
- `event_based` - generates the average power report. This is the average power of all events in the user-specified window. This report includes a summary based on the power components, cell type, power rail, and clock network. The worst window report is captured in a file specified with the `report_power -out_file` parameter, and the report is suffixed by '`.avgpower`'. When `-out_file` is not specified, the `vectorprofile.report.avgpower` report file is created.
- `power_density` - performs area or power density profiling. For this method, power is reported for each tile of the design, where design can be considered to be physically divided into equal area tiles. By default, a design is divided in 10x10 tiles.
- `transient` - generates the event-based average power report and current files for the worst power window.

Related Commands

- `report_power`

power_write_gui_db

```
power_write_gui_db {true | false}
```

Default: `true`

Data_type: Boolean

Specifies to create a binary database of power results in the XP mode. When set to `true`, the tool creates sliced power database file for each rail partition and saves it to the `gui.db` sub-directory under the power output directory

(example: `power_dir/part_*/gui.db/rail_part_*/power.x.db` (x = 1, 2, ..., 14). The generated power database allows you to perform GUI interactive debugging using the *Power & Rail Plots* and *Power Debug* forms.

Related Commands

- `report_power`

power_write_bbox

```
power_write_bbox  {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to save the boundary box of each instance, its placement, and rotation at the top level. When set to `true`, generates the power database with each instance boundary box, placement, and rotation information at the top level.

Related Commands

- `report_power`

power_write_db

```
power_write_db {true | false}
```

Default: `false`

Data_type: Boolean

A value of `true` creates a binary database of power results.

Related Commands

- `report_power`

power_write_default_pti_files

```
power_write_default_pti_files {true | false}
```

Default: `true`

Data_type: Boolean

For instances that are not hooked to any power/ground rail, the current for these instances are reflected in the default static current files:

- `static_default_ground_rail.ptiavg`
- `static_default_power_rail.ptiavg`

Related Commands

- `report_power`

power_write_profiling_db

```
power_write_profiling_db {true | false}
```

Default: `false`

Data_type: Boolean

Writes out profiling databases which can later be viewed as histograms using the SimVision interface. You can specify this parameter during both the activity and power vector profiling. The profiling databases support instance level and power/ground net histograms. It creates an output file name with the `*.trn` extension.

Related Commands

- `report_power`

power_write_simulation_db

```
power_write_simulation_db  {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to save functional waveforms for each instance when parsing an activity file. When set to `true`, creates SimVision waveform database (SHM database) for toggle simulation used during power analysis, which can later be viewed as histograms using the SimVision interface.

The simulation database (`shmdb.trn`) is generated in the power analysis output directory.

Related Commands

- `report_power`

power_write_static_currents

```
power_write_static_currents {true | false}
```

Default: `false`

Data_type: Boolean

A value of `true` generates the current data files per net.

Note: When you perform static power calculation in the dynamic power analysis flow, you can use either the `power_disable_static` or `power_write_static_currents` attribute to enable static power calculation and write static current file. If only `power_disable_static` is set to false and `power_write_static_currents` is not set, the tool automatically sets `power_write_static_currents` to true. Similarly, if `power_write_static_currents` is set to true, and `power_disable_static` is not set, the tool automatically sets `power_disable_static` to false.

Related Commands

- `report_power`

power_x_transition_factor

```
power_x_transition_factor value
```

Default: `0.5`

Data_type: Double

When using VCD/FSDB, one can specify how transitions to and from X are counted. These transitions are defined as shown in Table 1 below. Transitions to and from the X state will be treated as full transitions multiplied by the specified factor.

Note: 0 -> X -> 1 or 1 -> X -> 0 count as full transitions and they are not controlled by the attribute.

Related Commands

- report_power

power_x_count_transition_using_3_states

```
power_x_count_transition_using_3_states {true | false}
```

Default: false

Data_type: Boolean

Specifies to calculate the toggle rate using 3 states, pre-previous, previous and current states.

Related Commands

- report_power

power_z_transition_factor

```
power_z_transition_factor value
```

Default: 0.25

Data_type: Double

When using VCD/FSDB, one can specify how transitions to and from Z are counted. These transitions are defined as shown in Table 1 below. Transitions to and from the Z state will be treated as full transitions multiplied by the specified factor.

Note: 0 -> XZ-> 1 or 1 -> Z-> 0 count as full transitions and they are not controlled by the attribute.

Related Commands

- report_power

Table **1: Transitions from/to X and Z values and**

Transition	Definition	Default
to/from X	[0 or 1 or Z] <-> X	0.5
to/from Z	[0 or 1] <-> Z	0.25

power_zero_delay_vector_toggle_shift

```
power_zero_delay_vector_toggle_shift value
```

Default: ""

Data_type: String

Specifies to shift the current waveform in the zero-delay VCD flow. This attribute is useful in removing the idle period for the current waveform result with a long idle period (small flat current) before the first high current peak because of delay annotation. The attribute allows to offset the idle period from the delay values.

The default value is 0ns. The supported time units are s, ms, us, ns, and ps. The default unit is ns. You can specify a negative or a positive value. A positive value means right shift in the timeline or add more delay to the original arrival time, and a negative value means left shift in the timeline or trim the unwanted idle period.

The power_zero_delay_vector_toggle_shift attribute enables you to make a more realistic current waveform that can be easily aligned with the other current waveforms.

Note: This attribute is similar to the set_twf_attribute -shift_arrival_time parameter, with the only difference

that power_zero_delay_vector_toggle_shift supports both positive and negative values while -shift_arrival_time supports only a positive value.

Related Commands

- report_power

Scan Control File Format

The use model for the Scan Mode Analysis flow is:

```
set_db power_method dynamic_vectorbased power_scan_control_file filename
```

The format of the scan control file is:

```
MODE = [ SHIFT | VECTORLESS]
VECTORLESS_GLOBAL_ACTIVITY = <factor> [Between 0 - 1]
<chain_name> <flop_inst> <initial_state> <master_cell_name> <scale> <mode> <scan_in_pin> <scan_out_pin> <scan_en_pin>
```

Note that <chain_name> is optional, <initial_state> is optional in vector-less mode, and <master_cell_name>, <scale>, <mode>, <scan_in_pin>, <scan_out_pin>, <scan_en_pin> are optional and only needed for the first internal scan flop inside one macro.

Where,

- Mode specifies the mode used for scan design analysis. The two modes are: SHIFT and VECTORLESS.
- VECTORLESS_GLOBAL_ACTIVITY specifies the activity specification used in the VECTORLESS mode.
- chain_name is the name of the scan chain.
- initial_state is the initial state of all scan flops.
- master_cell_name is the name of the macro.
- scale is the scaling factor applied to the current.
- mode is mode name of the scan mode in the PGV view.
- scan_in_pin is the macro's scan_in pin name.
- scan_out_pin is the macro's scan_out pin name.
- scan_en_pin is the macro's scan_enable pin name.

propagate_activity

```
propagate_activity
[-set_net_frequency {true | false}]
```

Propagates the activity file in the database after it is read in using the read_activity_file command. This command propagates the activity for all primary inputs, nets, and other devices in the design whose activity has not been previously defined through user attributes.

propagate_activity can be used to propagate activities in the design and check for activity annotation before running power calculation. This is not a mandatory step as execution of report_power will also do the same and then do power calculation based on the propagated activities.

Parameters

`-set_net_frequency {true | false}`	
	Specifies that activity from the generated TCF file needs to be used to capture net frequency for signal EM analysis. When this parameter is set, you do not have to read the TCF file to set net frequency.

Example

- The following commands read and propagate the activity file in the database:

  ```
  read_activity_file
  propagate_activity
  ```

Related Topics

- "Signal ElectroMigration" in the *Voltus User Guide*

query_power_data

```
query_power_data
[-help]
[<pti_files <file1[file2] ...>>]
[-average_current]
[-clock_average]
[-write_pwl]
[-frame <string>]
[-insts <inst_list>]
[-inst_list_file filename]
[-list]
[-out_dir <string>]
[-peak_current]
[-peak_time]
[-pin]
[-time_steps]
[-verbose]
[-total_current <input_current_file> ]
[-output_tap_name <tap_name>]
```

The `query_power_data` command allows you to query the binary current files (.ptiavg/.ptipeak) that Power Calculation/Rail Analysis creates. It reads the tap current data in the current files and reports various information for debugging.

Parameters

`-help`	Outputs the command usage and a brief description about the command parameters.
`<pti_files <file1[file2] ...>>`	
	Specifies the names of the VDD and VSS instance voltage pti files.
`-average_current`	
	Specifies to write the average current of all instances.
`-clock_average`	
	Specifies to write the clock based current report.
`-write_pwl`	
	Specifies to write the tap current in the PWL format. You can use this parameter to view the contents of the binary data file.
`-frame <string>`	
	Specifies intervals for the clock_average report.
`-insts <inst_list>`	

	Specify the list of instances to be dumped in the ASCII format, one instance per line. If not specified, it prints all instances. The `-insts` parameter is specified with the `-write_pwl`, `-peak_current`, `-peak_time`, and`-average_current` parameters.
`-inst_list_file` *filename*	
	Specifies a file with the list of instances to be dumped in the ASCII format.
`-list`	
	Specifies to write the tap names from the input file.
`-out_dir` <*string*>	
	Specifies the name of the output file.
`-output_tap_name` <*tap_name*>	
	Specifies the name of the output tap in the new current file, which is created using the `-total_current` parameter.
`-peak_current`	
	Specifies to write the peak current of all instances.
`-peak_time`	
	Specifies to write the peak current time of all instances.
`-pin`	
	Specifies to print the pin names with the instance names. The `-pin` parameter is specified with the `-write_pwl` and `-list` parameters.
`-verbose`	
	Specifies to print the report to standard output.
`-time_steps`	
	Specifies to summarize the minimum current, maximum current, average current, and total current for each interval in the data file. You can use this data to estimate which interval in a transient response might contain the worst-case data.
`-total_current` <*input_current_file*>	

	Specifies to create a new current (.ptiavg) file containing the total current for all taps of a net. This parameter can be used if you have a very large current file generated by the Power Calculation engine and want to see only a total current waveform in the SimVision waveform viewer. *input_current_file* is the current file that will be read in (output from the Power Calculation engine). Use the following syntax to create a total current file: `query_power_data -total_current <input_current_file> -out_dir <output_total_current_file>` `-out_dir` is used to specify a user-defined output current filename, and is not a mandatory parameter. If not specified, the output file will be generated automatically with the name of the input file suffixed with `.ptiavg`.

Example

- The following command creates a total current waveform file `VDD` from the input file `dynamic_VDD.ptiavg`:

```
query_power_data -output_tap_name vdd_tap -total_current dynamic_VDD.ptiavg -out_dir VDD
```

read_activity_file

read_activity_file *activity_file*
[-end { *time1 time2 ...timen* }]
[-format {vcd | tcf | saif | fsdb | phy | shm}]
[-hierarchy_separator *separator*]
[-reset]
[-start { *time1 time2 ...timen* }]
[-write_net_freq {true | false}]
[-block *block_name*]
[-scope *scope_name*]
[-scale_time *scalefactor*]
[-start_time_shift *value*]
[-rtl {true|false}]
[-name_mapping_rule *file*]
[-weight *value*]
[-zero_delay {true|false}]
[-cell *cell_name*]

Specifies the name and type of activity file to be used as input. This command must be used when specifying activity files for static or dynamic power calculation.

Parameters

-block *block_name*	
	Specifies the name of the FSDB/SAIF/TCF/VCD block to map a sub-block activity file with a top level Verilog file. Provides the ability to support multiple activity files based upon block instance names. The block argument is added to the FSDB/SAIF/TCF/VCD name before comparing it to the Verilog file.
-cell *cell_name*	

	Specifies the master cell name for the activity file. This parameter allows you to support master-cell-based activity specification. That is, you need to specify only the master cell name for the activity file, instead of specifying all the instances that were instantiated from a master cell.
	Example
	Let us consider a design that has a cell named "super_filter" with four block instances "super_filter1", "super_filter2", "super_filter3",and "super_filter4":
	Instance-Based Use Model:
	`read_activity_file -format VCD -block super_filter1`
	`read_activity_file -format VCD -block super_filter2`
	`read_activity_file -format VCD -block super_filter3`
	`read_activity_file -format VCD -block super_filter4`
	Master-Cell-Based Use Model:
	`read_activity_file -format VCD -cell super_filter`

-end { *time1 time2 ...timen* }	

	When you use the vector-driven method of performing dynamic instance-based power-consumption calculation, you can specify the part of the activity file from which you want power analysis to use for the power calculation. This option specifies the end time of this range and is used in conjunction with the `-start` option. Specifies the end time as time-value pairs to report the average power across non-overlapping multiple windows specified in the activity file for a block, level of hierarchy, or other part of the design that you want to analyze. Units are in seconds (`s`), milliseconds (`ms`), microseconds (`us`), nanoseconds (`ns`), or picoseconds (`ps`). *Default*: The time unit is defined in the first `.lib` file read during design import. You can specify the `$worst_power_window_end` variable with the `-end` parameter to take the worst case end time from the last vector profiler run. Not required for TCF and SAIF. **Note**: You can read multiple windows in a single VCD file by specifying multiple pairs of the start time and end time for the non-overlapping multiple windows specified in the activity file. The use model for reading multiple windows is: `read_activity_file -format VCD design.vcd -start {start1 start2 start3} -end {end1 end2 end3}` The *start1/2/3* and *end1/2/3* values should be in the ascending order. Multiple current waveforms representing multiple VCD windows are merged into a single continuous current waveform. To capture the waveform for signals at the edge of user windows, a buffer time is added between the windows being stitched. This buffer time ensures that the stitched waveform is contiguous. The generated waveform hence has the duration of user windows and the intervening settling buffers. The default settling buffer period is 400ps.
`-format {vcd | tcf | saif | fsdb | phy | shm}`	

	Specifies the net activity of the design using an activity file in the VCD, TCF, FSDB, SAIF, PHY, or SHM format. The arguments specified with the `-format` parameter are case-insensitive.
	
	• The VCD file format uses the toggle count information as the basis for the power consumption calculation. These files can be in compressed (zipped) format using the .gz extension.
	
	• The TCF file format contains data on the switching activity of the nets in the design. The switching activity includes the toggle count information and the probability of the net or pin being in the logic 1 state.
	These files can be in compressed (zipped) format using the .gz extension.
	
	Supports extended TCF which contains transition density for X and Z as shown in the following extended TCF example:
	
	"out" :"0.0 0" "0.005000 2" "0.0 0"
	
	The fields are defined as follows:
	
	Field 1: net or pin name
	Field 2: duty and transition density for transition of 0-1
	Field 3: duty and transition density for transition of X(0-X-1)
	Field 4: duty and transition density for transition of Z(0-Z-1)
	
	• The SAIF file format contains aggregate switching activity information on the nets and/or ports in the design. It does not contain time-based information. This file format is used for quick average power analysis of the design.
	
	• The FSDB file format specifies the net activity of the design using an activity file in FSDB format. The software supports the FSDB version up to 5.8.
	Use Model
	`read_activity_file -format fsdb fsdbfile`
	`report_power`
	(or)
	`report_vector_profile`

Example

```
read_activity_file -format fsdb test.fsdb -scope top
```

- The PHY (Palladium database format generated in the Palladium environment) format contains the simulation activity data generated during emulator-based simulation. The PHY format can be used in all vector-based power analysis flows (vector profiler, static power analysis and dynamic vector-based power analysis).

  Use Model

  `read_activity_file-format phy <path to PHY database directory>`

  Example

  ```
  read_activity_file -format phy ./phy2/trace.phy -scope test
  ```

  Notes:

  - The xeDebug software must be present in the `PATH` environment. Add xeDebug in the path using the following command:
    ```
    set path = (<install_dir>/bin $path)
    ```
    The xeDebug software version should be the same as the one using which the PHY database is generated.
  - The PHY database generated from the following UXE/VXE version is supported:

    ```
    UXE18.6.0 onwards
    VXE18.6.0 onwards
    ```
  - When reading a PHY format activity file, you require a DPA (DPA_ToggleCountFormat) license.
  - For information on error messages displayed while reading the PHY database, refer to the `xe.msg` file in the PHY database directory.
  - Make sure that the PHY database is not locked before invoking the software.
  - Make sure that the xeDebug software is not already running on the machine.
  - Ensure that the following directories are present in the top-level PHY database directory: `*.phy`, `PDB`, `cellList`, `QTDB`, `dbFiles`, and `.design`.
  - It is recommended that the write permission is assigned on the entire PHY database. In case the write permission cannot be granted, you must set the `SWFV_FREEZE_PHY` environment variable to `1`.
  - If you want to run parallel executables on the same top-level PHY directory with multiple `*.phy` databases, it is suggested to create local copies of the multiple top-level directories, with links to the original PHY database. This will allow local copies of the PHY database to acquire separate locks.
- The Simulation History Manager (SHM) database format is a record of the data signal changes that occur during design simulation. The SHM database consists of a directory, typically with the "`.shm`" suffix. The directory contains two disk files, namely `*.trn` and `*.dsn`. For more information about the SHM format, refer to the *SimVision Analysis Environment* manual available on the Cadence Online Support (COS) website.

  Example

  ```
  read_activity_file -format shm  /../shm/ADDER.trn -scope adder_e/u1 -start 0ns -end 60ns
  ```

`-hierarchy_separator` *separator*

Specifies the separator character in the hierarchical net names, bus names, and pin names for the block or other part of the design that you want to analyze. It must be the same as the separator character in the design netlist.

Default: slash (/)

`-name_mapping_rule` *file*

	Specifies to enable rule-based mapping to map RTL vectors to gate-level netlist. This parameter helps in resolving name mismatch between RTL and gate level netlists.
	The name mapping rule file format is:
	`Name_Mapping_Rule_file`
	`# RTL  # Gate`
	`<rtlstring> <gatestring>`
	where, each line is a rule. The string in the first column(RTL) is what we want to replace with the string in the second column(Gate). The rules in every line are added onto the previous rules.
	For example, to replace all the square `[]` brackets with under score _ for RTL-to-Gate matching, the rule file will be:
	`#RTL  #GATE`
	`[  _`
	`]  _`

`-reset`	If `-reset` is specified, all previous activity files specified using the `read_activity_file` command will be reset and ignored. *Default*: All the previous activity files specified using the `read_activity_file` command are loaded and used (They are additive).
`-rtl {true|false}`	
	Specifies to process the vector as an RTL vector and perform state propagation. The propagation behavior is global and the full design is propagated. RTL vectors are zero-delay vectors so delay annotation is always done for signals from this vector file. *Default*: `false`
`-scale_time` *scalefactor*	
	Scales the FSDB/SAIF/TCF/VCD duration value by the specified scale factor. For example, an FSDB scale of 2 will double the FSDB duration value. **Note:** The `-start` and `-end` parameters of the `read_activity_file` command are post scaled.
`-scope` *scope_name*	
	Specifies the FSDB/SAIF/TCF/VCD scope, that is, the name of the module within the activity file associated with the block or other part of the design that you want to analyze. The scope argument is removed from the FSDB/SAIF/TCF/VCD name before comparing it to the Verilog file. If the scope value has backslash '\' or square brackets '[]', you must specify the scope within curly brackets {{…..}} to ensure that '\' or '[]' are correctly processed by the Tcl Interpreter. In the following example, the scope value is specified within curly brackets as it contains '\' and '[]': `read_activity_file -scope {{reg_based_buffer.reg_inst\/sv_xpol_trans_q_reg[0][14][7][sv_re][4]}}` If '\' and '[]' are not there, curly brackets are not required.
`-write_net_freq {true | false}`	

	This option should be set when an activity from a TCF or VCD file needs to be used to capture net frequency for signal EM analysis (`check_ac_limits`). In addition, you must use the `check_ac_limits -use_db_freq` option to read the toggle rates from the activity file.
	Default: `false`
	Example
	The following example reads a TCF file which is used to capture net frequency for signal EM analysis:
	`read_activity_file -format tcf -write_net_freq true design.tcf`
	`check_ac_limits -use_db_freq -report signal_em.rpt`

`-start { time1 time2 ... timen }`	
	When you use the vector-driven method of performing dynamic instance-based power-consumption calculation, you can specify the part of the activity file from which you want power analysis to use for the power calculation. This option specifies the start time of this range and is used in conjunction with the `-end` option. Specifies the start time as time-value pairs to report the average power across non-overlapping multiple windows specified in the VCD file for a block, level of hierarchy, or other part of the design that you want to analyze. Units are in seconds (`s`), milliseconds (`ms`), microseconds (`us`), nanoseconds (`ns`), or picoseconds (`ps`). *Default*: The time unit is defined in the first `.lib` file read during design import. You can specify the `$worst_power_window_start` variable with the `-start` parameter to take the worst case start time from the last vector profiler run. Not required for TCF and SAIF. **Note**: You can read multiple windows in a single VCD file by specifying multiple pairs of the start time and end time for the non-overlapping multiple windows specified in the activity file. The use model for reading multiple windows is: `read_activity_file -format VCD design.vcd -start {start1 start2 start3} -end {end1 end2 end3}` The *start1/2/3* and *end1/2/3* values should be in the ascending order. Multiple current waveforms representing multiple VCD windows are merged into a single continuous current waveform. To capture the waveform for signals at the edge of user windows, a buffer time is added between the windows being stitched. This buffer time ensures that the stitched waveform is contiguous. The generated waveform hence has the duration of user windows and the intervening settling buffers. The default settling buffer period is 400ps.
`-start_time_shift value`	
	Specifies to shift the start time of the specified activity files. When you specify the `-scale_time` parameter with the `-start_time_shift` parameter, it scales the start time shift along with the start/end time.
activity_file	The name of the activity file.
`-weight value`	

	Specifies the weight number of the TCF file. `1.0` is the default value. The value should be a floating number that is smaller than `1.0`. This parameter allows you to merge multiple TCF files at different function modes for static power estimation of a design.
	In the following example, TCF *file1*, *file2*, and *file3* has the weight numbers 0.3, 0.2, and 0.4, respectively.
	`read_activity_file -format TCF file1 -weight 0.3`
	`read_activity_file -format TCF file2 -weight 0.2`
	`read_activity_file -format TCF file3 -weight 0.4`
	Here, *file1*, *file2*, and *file3* will be merged together to represent the toggle and duty for the nets/pins in the design. The relative weight of 0.3/0.9, 0.2/0.9, and 0.4/0.9 would be used for static power calculation. In this example, 0.9 is the sum of weights.

`-zero_delay {true|false}`	

	Specifies to process the vector as a zero-delay vector and perform delay annotation. *Default* : `false`

Activity Precedence

The following commands can define activity:

```
read_activity_file file
     [-format { VCD | TCF | SAIF | FSDB ]}
     [-hierarchy_separator separator ]
     [-start time ]
     [-end time ]
     [-reset]
     [-scope scope_name ]
     [-block block_name ]

set_default_switching_activity
    [-input_activity factor ]
    [-seq_activity factor ]
    [-period value ]
    [-duty value ]
    [-global_activity factor ]
    [-hier hierarchy _name]
    [-clock_gates_output activity_factor ]

set_switching_activity
    [-reset]
    [-clock clock_name]
    [-period value ]
    [-unclocked]
    [-activity factor | -density transition_density ]
    [-net net_name | -port port_name | -pin pin_name ]
    [-duty value ]
    [-inst]
```

The precedence of activity is as follows:

1. User-Defined: Activities applied to specific nets/pins/ports
2. Activity File: Activity specification through a VCD/FSDB/SAIF/TCF format
3. Clock Gates Output Activity: Activity specification to the output of all clock gates
4. SDC/TWF constants: Set constants to a list of pins or ports for use by the timing engine (timing analysis)
5. Hierarchical Global Activity: Activity specification for a specific hierarchy
6. Global Activity: Activity specification for all instances that are part of the data network.
7. Sequential Element Activity: Activity specification applied to the output of all sequential elements.
8. Primary Input Activity: Activity specification for all primary inputs

If you specify multiple activity data for the same items, the power engine will use the one with the higher precedence.

Examples

- The following command reads the compressed ap_wait_test_pll1_mod .vcd.gz VCD file that contains the toggle count information for the power consumption calculations, specifies the crm_ap module within the VCD file as the module associated with the part of the design being analyzed, specifies the slash character (/) as the hierarchical separator for hierarchical net names, bus names, and pin names for the part of the design being analyzed, and specifies the start (180002ns) and end (189802ns) times from the VCD file for the part of the design being analyzed:

```
read_activity_file -format VCD \
-scope testbench/top/ap/mcu_platform/crm_ap \
-hier_separator \
-start 180002ns \
-end 189802ns \
ap_wait_test_pll1_mod.vcd.gz
```

- The following command reads the compressed A.vcd.gz VCD file that contains the toggle count information for the power consumption calculations, specifies A1 as the block to map a sub-block VCD file with a block instance at the top-level, and specifies the start (180002ns) and end (189802ns) times from the VCD file for the part of the design being analyzed:

```
read_activity_file -format VCD \
-block ap/A1 \
-start 180002ns \
-end 189802ns \
A.vcd.gz
```

- The next example specifies a VCD activity file called dmac_mac.vcd for the scope top/dma_dut.

```
read_activity_file -format VCD -start 10ns -end 20ns \
-scope top/dma_dut dmac_mac.vcd
```

- The following command specifies multiple pairs of the start time and end time for the multiple windows in the VCD file：

```
read_activity_file \
-format  VCD \
-scope  adder/u1 \
-start  {0ps 2000ps} \
-end    {1100ps 3100ps} \
-block  ../vcd/dut_5buf_full_chip.vcd
```

Related Topics

- "Dynamic Power and IRDrop Analysis" in the *Voltus User Guide*

Related Commands

- report_power

read_activity_map_file

```
read_activity_map_file
[-help]
[-golden {rtl | gate}]
[-gate_block block_name]
[-reset]
[-rtl_block block_name]
-rtl_to_gate mapping_file
[-two_column_mapping_file mapping_file]
```

Specifies instance name mapping between RTL netlist and GATE level netlist. Power analysis uses this instance name mapping to annotate the output nets of this instance from the pin based RTL VCD or TCF.

You must specify this command before `report_power`.

Parameters

`-gate_block` *`block_name`*	
	Specifies the GATE activity block name to map the block-level RTL mapping files to the top-level RTL activity files. The GATE block name is appended before the GATE level entries in the mapping file to match the entries in the top-level RTL activity file.
`-golden {rtl | gate}`	
	Specifies whether RTL or Gate is Golden. The one not specified as golden will be considered as Revised. Currently the Conformal MatchPoint file does not specify this. *Default*: `rtl`
`-help`	Outputs a brief description that includes the type and default information for each `read_activity_map_file` parameter. For a detailed description of the command and all of its parameters, use the man command `man read_activity_map_file`.
`-reset`	Specifies to reset and ignore all the previous mapping files specified using the `read_activity_map_file` command. *Default* : All the previous mapping files specified using the `read_activity_map_file` command are loaded and used.
`-rtl_to_gate` *`mapping_file`*	
	Specifies instance name mapping between RTL netlist and GATE level netlist. RTL level VCD or TCF are obtained from simulation on RTL Verilog netlist. Power analysis uses this instance name mapping to annotate the output nets of this instance from the pin based RTL VCD or TCF.
`-rtl_block` *`block_name`*	
	Specifies the RTL activity block name to map the block-level RTL mapping files to the top-level RTL activity files. The RTL block name is appended before the RTL level entries in the mapping file to match the entries in the top-level RTL activity file.
`-two_column_mapping_file` *`mapping_file`*	

	Specifies a file that contains the instance name mapping between the RTL netlist and GATE level netlist.
	This parameter can be used to specify a mapping file instead of the LEC generated mapping file (`-rtl_to_gate` *`mapping_file`*) to map points from RTL to Gate netlist. The format of the two column mapping file is:
	`#Gate name =>  # RTL name`
	`gate_inst_name1  => rtl_inst_name1`
	`...`
	Here, the first column corresponds to the gate name and the second column corresponds to the RTL name.

Example

- The following command specifies a mapping file rtl_map:
  ```
  read_activity_map_file -rtl_to_gate rtl_map
  ```
- The following command performs instance name mapping between RTL netlist and GATE level netlist, wherein the GATE level netlist is Golden and the RTL netlist is Revised; it specifies the mapping file MapFile.alt
  ```
  read_activity_map_file -rtl_to_gate MapFile.alt -golden gate
  ```
- The following commands appends the RTL block name and GATE block name before the RTL and GATE level entries in the mapping file to match the entries in the top-level RTL activity file:
  ```
  read_activity_map_file -rtl_to_gate LEC.top \
  -rtl_block  "core/block_top" \
  -gate_block "core/block_top"
  ```

An example of the mapping file output from Conformal is described as follows:

```
Mapped points: SYSTEM class
1-th mapped points:
 (G) + 1 PI /x_ick
 (R) + 595 PI /x_ick
2-th mapped points:
 (G) + 2 PI /x_jreset_cp_p
 (R) + 594 PI /x_jreset_cp_p
3-th mapped points:
 (G) + 3 PI /x_mreset_cp_p
 (R) + 593 PI /x_mreset_cp_p
........
60-th mapped points:
 (G) + 4582 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg[35]
 (R) + 3856 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg_35_

61-th mapped points:
 (G) + 4583 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg[34]
 (R) + 3855 DFF /cpexec0/cpddecls0/gi_opls/q_reg_reg_34_

62-th mapped points:
 (G) + 4633 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[21]
 (R) + 3805 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_21_

63-th mapped points:
 (G) + 4634 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[20]
 (R) + 3804 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_20_

64-th mapped points:
 (G) + 4635 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[19]
 (R) + 3803 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_19_

65-th mapped points:
 (G) + 4636 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg[18]
 (R) + 3802 DFF /cpexec0/cpddecex0/gi_opex/q_reg_reg_18_
```

where, `G` refers to the Golden netlist name and `R` refers to the Revised netlist name.

By default, the tool assumes that the Golden netlist name refers to the RTL net name and Revised netlist refers to the GATE level net name.

Related Topics

- "RTL Activity File Flow in Static Power Calculation" in the *Voltus User Guide*

read_power_db

```
read_power_db
[-help]
-in_file {file1 file2 file3 ...}
[-hierarchy {prefix1 prefix2 prefix3 ...}]
[-hierarchy_separator "/"]
```

Restores any `power.db` file in the Voltus/Tempus session. This command can be used after restoration of a design to view the static power results or incremental report generation without having to do power analysis again.

Note: You will be able to restore the power database that you created previously, only if you created the power database using the `power_write_db` attribute. Also, the command loads power databases created in the 9.1 release only, and not in the older releases.

- The command can map a hierarchical instance to a flat netlist.
- It can restore multiple power dbs for blocks and top level, that are generated separately.
- When the information for a net or an instance is present in multiple power dbs, the last one will overwrite.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `read_power_db` parameter. For a detailed description of the command and all of its parameters, use the man command `man read_power_db`.
`-in_file {file1 file2 file3 ...}`	
	Specifies the name of the power database file.
`-hierarchy {prefix1 prefix2 prefix3 ...}`	
	Specifies to map each instance in the file with a prefixN/instance (for example, prefixA/inst) in the flat netlist, where '/' is the hierarchy seperator. This is an optional parameter. *Default*: `all`
`-hierarchy_separator "/"`	
	Specifies the separator character in the hierarchical net names, bus names, and pin names for the block or other part of the design that you want to analyze. It must be the same as the separator character in the design netlist. This is an optional parameter. *Default*: slash (`/`)

Use Model

```
read_db / read_design
read_power_db -in_file power.db
report_power # for text based power reports
```

Example

For each instance in file 'file2' a prefix "A" is added to instance name such as instC, A/instC. No prefix is added to `file1' since prefix1 is none for file1.

```
read_power_db -in_file {file1 file2} -hierarchy {none A}
```

read_temperature

```
read_temperature
[-help]
[-default_temperature value]
[in_file filename]
[-reset]
```

Loads the instance-based temperature information to account for the effects of temperature when calculating the delays and slews.

Parameters

`-help`	Prints out the command usage.
`-default_temperature` *value*	Specifies the default temperature value (in celsius) to use. **Note:** The temperature settings in the temperature file take precedence over this parameter.
`-in_file` *filename*	Specifies the name of the temperature file that contains the instance-based temperature values (in celsius). When used with the `-default_temperature` option, the instances that do not appear in these files will use the specified default temperature. The format of the temperature file is as follows: `VERSION "1.0" ;` `CREATION "Date" ;` `PROGRAM "Vendor" ` ` `DIVIDECHAR "/";` `TEMPERATURE_UNIT "C" ;` `- instance1 temperature ;` `- instance2 temperature ;` `- instance3 temperature ;` `- instance4 temperature ;` `- instance5 temperature ;` `- END;`
`-reset`	Resets all the temperature settings.

Examples

- The following command loads the temperature file named `sampletemp`:

  ```
  tempus > read_temperature -in_file sampletemp
  ```

Related Commands

- `set_instance_temperature_file`

report_inst_power

```
report_inst_power
[-help] instance [-out_file filename][-inst_file filename]
```

Generates a text based report for the specified instance. This command enables you to determine how the different components of static power are calculated. You can use the report to debug static power numbers after performing power analysis (`report_power`).

This command is available through both the static and dynamic power engines.

Note: For dynamic power engine, the `report_inst_power` command does not generate text reports incrementally. If the dynamic run has been performed and you need the debug information for a static power number, you should re-run power analysis.

Parameters

`-out_file filename`	Writes out the static power analysis report into the file that you specify.
`-help`	Outputs the command usage and a brief description about the command parameters.
instance	Specifies the instances to include in the instance power report.

`-inst_file filename`	Specifies a file that includes a list of instances. The parameter allows you to report and debug power numbers for multiple instances.
	A sample file is given below:
	`INST1`
	`INST2`
	`INST3`

Example

- The following command specifies an instance power report file `debug.rep` for the instance `top/i_1`:
  ```
  report_inst_power -out_file debug.rep top/i_1
  ```

Related Topics

- "Static Power, IRdrop and EM Analysis" in the *Voltus User Guide*
  - "Debugging Instance Power"

report_power

```
report_power
[-block block_name]
[-cap]
[-cap_unit unit]
[-cell {cell_list}]
[-cell_fit_file filename]
[-cell_list_file filename]
[-cell_type {all | {macro io combinational sequential clock_combinational clock_sequential} }]
[-clock_domain names]
[-clock_gating_efficiency]
[-clock_network {all | {clock_list}}]
[-cluster_gating_efficiency]
[-combinational_sequential_power_distribution]
[-compress compression_ratio]
[-distribute]
[-exclude_cells {cell_list}]
[-exclude_cells_file filename ]
[-exclude_instances_file filename ]
[-exclude_insts filename ]
[-float_precision value]
[-format { simple | detailed }]
[-group_type user_defined_group_name]
[-hierarchy {all | hierarchy_level}]
[-hierarchical_instances {inst1 inst2 inst3 …}]
[-include_group_instance_count]
[-include_sequential_in_clock]
[-insts {instance_list}]
[-inst_list_file filename]
[-leakage]
[-net [-nworst number_of_nets]]
[-no_wrap]
[-out_dir directory]
[-output_cell_fit_file filename]
[-out_file filename]
[-power_db_directory directory]
[-power_unit unit ]
[-power_density_tiles value]
[-power_density_tiles_row_column {value1 value2}]
[-power_density_tiles_size {value1 value2}]
[-pg_net {all | pg_net_name_list}]
[-power_domain {all | {power_domain_list}}]
[-print_memory_power]
[-pg_pin]
[-report_prefix prefix]
[-register_gating_efficiency]
[-sort {internal | switching | leakage | total}]
[-stat]
[-threshold_voltage_group { all | group_name}]
[-thermal_leakage_temperature {temp1 temp2 temp3 …}]
[-thermal_power_map_file file_name]
[-thermal_power_map_tiles {xint yint}]
[-thermal_power_map_format {simple | stack }]
[-thermal_conductivity_inputs file_name]
[-toggle_rate]
[-time_based_report]
[-thermal_leakage_temperature_scale_table_file filename]
[-thermal_leakage_temperature_scale_table {$temp1 $scalefactor1 $temp2 $scalefactor2 $temp3 $scalefactor3 .....}]
```

```
[-thermal_material_file filename]
[-thermal_power_map_si_unit {true | false}]
[-thermal_power_map_header_include_file filename]
[-thermal_power_map_power_table_tiles {Xint Yint}]
[-thermal_power_map_density_table_tiles {Xint Yint}]
[-thermal_power_map_format_version {new|default}]
[-thermal_power_map_bottom_layer layer_name]
[-thermal_power_map_top_layer layer_name]
[-threshold value]
[-time_unit unit]
[-use_geometry_cell_size {true | false}]
[-view view_name]
```

Reports global as well as specific instance power, clock network power, clock domain power, and net switching power. It also reports the power for power domains and specific power nets. Units are reported in milliwatts.

This command starts the execution of power analysis (both static and dynamic).

> The parameters of the `report_power` command, except `-cap`, are not supported in the dynamic only mode (specified using the `power_disable_static` attribute). These parameters are only supported in the static mode. However, `-clock_gating_efficiency`, `-register_gating_efficiency`, and `-power_domain` parameters are not supported even if the static power mode is enabled.

Parameters

`-block` *block_name*	

	Specifies to generate the different power analysis reports for a block. This parameter is useful in viewing and debugging power consumption issues for a specific block after the power run is complete.
	Example:
	The following command will create the summary and pg_net reports for the design block {A/B}:
	`report_power -block {A/B} -out_file blockB.rpt -pg_net -pg_pin {all} -insts {*}`
	Following are the sample block-specific reports:
	`blockB.rpt.avppower`
	`blockB.rpt.pgnetpwr`
	`blockB.rpt.pgpinpwr`
	`blockB.rpt.instpwr`

`-cap`	Dumps out the list of net, pin, and total capacitances in three different columns.
`-cap_unit` *unit*	
	Sets the capacitance unit to be used for power report generation. The default unit is pF (picoFarads).
`-cell` **{***cell_list***}**	
	Specifies the cells to include in the power report. Accepts wildcards (*)
`-cell_fit_file` *filename*	

	Specifies a file that accepts a cell-based FIT input table with two columns: cell name and FIT value. This parameter is used to generate an instance-level FIT report based on the cell-level data.
	Instance-based FIT report is supported only in the XP mode.
	File Format Example for the FIT Input File:
	`##Cell Name FIT`
	`----------`
	`CellA .25`
	`CellB .5`

`-cell_list_file` *filename*	
	Specifies a file containing a list of cells that are to be included in the power reports.
`-cell_type {all | {macro io combinational sequential clock_combinational clock_sequential}}`	
	Specifies the cell type to include in the power report. *Default*: `all` When you specify the cell types, `-cell_type combinational` or `sequential`, the power report now does not show the clock combinational or clock sequential cells.
`-clock_domain` *names*	
	Specify clock domains in power reports. **Usage** report_power -clock_domain *name* -out_file *filename*
`-clock_gating_efficiency`	

	Generates a report that includes the Clock Gating Efficiency (CGE) for all clock gating instances as well as different hierarchies in the design. Average CGE for different hierarchies is also reported.
	Clock Gating Efficiency is calculated as:
	`CGE = (toggles at clock gate output/toggles at clock gate input)`
	`Average CGE = (avg of all CGEs above)`
	When you generate a CGE report using the command `report_power -clock_gating_efficiency -out_file` *cge.rpt*, the following information will be displayed:
	• **Clock-gating Efficiency Report** - for each clock domain, it includes the toggle rate, number of registers, number of clock gates, average clock toggle at registers, average toggle savings at registers, and average toggle savings histogram.
	• **Hierarchical View of Average Toggle Savings** - number of clock gates and average toggle savings for each hierarchical module in the design

`-clock_network` {`all` | {*clock_list*}}	
	Reports the power consumption of the clock network, including the power for generated clocks. Use `clock_list` to report the power consumed by specific clocks. *Default*: `all`
`-cluster_gating_efficiency`	
	Specifies to generate the cluster efficiency report in the RGE report. This report gives the CGE/RGE metrics for the registers at the fanout of each clock gate instance in the design.

`-combinational_sequential_power_distribution`	
	Specifies to generate a report that gives the ratio of combinational versus sequential power for all modules of a given hierarchy. You must specify this parameter with the `-insts` parameter.
`-compress` *`compression_ratio`*	
	Specifies to compress the power analysis report files in the GNU Zip format. The compressed files will have the suffix `.gz`. *`compression_ratio`* is from 0-9. `0` indicates that the report files are not to be compressed. The compression levels vary from `1` to `9`. `1` provides the fastest compression speed but at a lower ratio, so the file size will be larger. `9` provides the highest compression ratio but at a lower speed. *Default*: 0
`-distribute`	Enables standalone distributed power analysis. This parameter allows you to perform dynamic power analysis in the multi-CPU mode without running rail analysis.
`-exclude_cells {`*`cell_list`*`}`	
	Specifies a list of cells that should be excluded from power analysis.
`-exclude_cells_file` *`filename`*	
	Specifies a file containing a list of cells (one per line) which should be excluded from power analysis.
`-exclude_instances_file` *`filename`*	
	Specifies a file containing a list of instances (one per line) which should be excluded from power analysis.
`-exclude_insts {`*`instance_list`*`}`	
	Specifies a list of instances that should be excluded from power analysis.
`-float_precision` *`value`*	
	Sets the precision value to be used for power report generation. The default value is 4.
`-format { simple | detailed }`	
	Specifies the type of report summary required for static power reporting. The possible arguments of this parameter are: • `simple` - reports the amount of power consumed by all power nets in a condensed format, breaking down power based on the switching/internal/leakage components. • `detailed` - reports the amount of power consumed by all power nets in a detailed format. The detailed format includes additional information, such as output pin capacitance (`Cpin Sum`), output net capacitance (`Cnet Sum`), Total Load Capacitance (`Cpin Sum` + `Cnet Sum = Total Load_Cap`), maximum frequency at the input and output pins (`Max_Clk_Freq`), maximum activity factor at the input and output pins (`Max_Activity`), cell name of the instance (`Cell Name`), clock or data cell (`C/D`), and library used in power analysis for the specific cell (`Lib`). The libraries are numbered in the header section. The name of the generated report file is `report.*.detailed.rpt`. Currently, the `-format` detailed report is only available for the `-pg_net` report.
`-include_sequential_in_clock`	

	Includes the leaf flip-flop power in the clock network power report. *Default*: The leaf flip-flop power will not be included in the clock network power report.
`-hierarchical_instances {`*`inst1 inst2 inst3 …`*`}`	
	Specifies to report all the power content for the specified hierarchical instances/ blocks. It reports the `Total Power` (a breakup of power components) and `Group` (a breakup of power by cell type) of the specified block at each hierarchy. The name of the report can be specified with the `-out_file` parameter (`<`*`outfile`*`>.hierinstpwr.rpt`), or a prefix can be specified with the `-report_prefix` parameter. The report is written in the specified output directory.
`-hierarchy {all |` *`hierarchy_level`*`}`	
	Specifies the hierarchy level that is included in the power report. You can specify the *`hierarchy_level`* as a number that corresponds to a specific hierarchy level in the design. A *`hierarchy_level`* of 0 specifies the top level. *Default*: `all`
`-include_group_instance_count`	
	Specifies to report the instance count for each cell type. When specified, you can view the number of instances for the sequential, macro, IO, combinational, clock sequential, and clock combinational cell types in the power report (power.rpt). The parameter is supported in both the static and dynamic power analysis flows. This parameter allows you to verify if there are any missing instances.
`-insts {`*`instance_list`*`}`	
	Specifies the instances to include in the power report. The power report provides the amount of power consumed by a specified list of hierarchical or leaf-level instances. *`instance_list`* accepts wildcards (*) for leaf-level instances only. An instance with 0 power will not be listed in the power report.
`-inst_list_file` *`filename`*	
	Specifies a file containing a list of instances that are to be included in the power reports.
`-leakage`	Reports only leakage power of the design.
`-net`	Reports the net switching power, as well as the load capacitance, toggle rate, and switching values for each net in the design.

`-no_wrap`	Displays the `report_power -cell_type all` output on a single line in the report output file.
	Example
	`Instance Toggle Internal Switching Leakage Total Percentage Cell`
	`Power Power Power Power (%) Name`
	`----------`
	`dma_dut/dmamaster_dut/internal_memory/RAM4096x32/ram0 7.102e+08 5.58 0.002307 0.02424 5.607 18.19 TSDN65ethernet_mac_1/rx_fifo_module_to_dma/data/dual_port_ram_4_tx_fifo/ram2P1024x32/ram`

`-nworst` *`number_of_nets`*	
	Reports the number of nets with the highest net switching power. **Note:** You must use the `-nworst` parameter in conjunction with the `-net` parameter.
`-out_file` *`filename`*	

	Specifies the name of the report file in which the top-level summary of the power consumption information is written. You can include the directory name. You can convert a power report file to the gzip file format by specifying `-out_file filename.gz`. **Note:** If you do not specify this parameter, the software directs the power report to the command line, as well as the log file, and no power report is written to a separate file.
`-out_dir directory`	
	Specifies the name of the power analysis output directory that saves the power reports. The default is the current working directory.
`-output_cell_fit_file filename`	

	Specifies the name of the output file for the generated FIT report, which includes three columns: instance name, corresponding cell name, and computed FIT value. This parameter is specified with the `-cell_fit_file` parameter. The output file is generated in the power output directory.
	File Format Example for the FIT Output File:
	`#Instance Name Cell Name FIT`
	`############################################################`
	`inst_A/al_inst/g5 INV_X1A_A9TL 0.25`
	`inst_A/al_inst/g8 INV_X1B_A9TL 0.25`
	`....`
	`Total FIT: 115`

`-pg_net {all | pg_net_name_list}`	
	When set to `all`, reports the amount of power consumed by all power nets. If you specify a list, reports the power consumed by each instance that is connected to the specified list of power nets. **Note:** When using this parameter, the static power calculation engine always performs propagation and power calculation. *Default*: `all`
`-pg_pin`	
	Reports power consumed by all PG pins in a design.
`-power_db_directory directory`	
	Specifies the location of the power database that the Power Calculation engine uses for generating standalone power reports.
`-power_domain {all | {power_domain_list}}`	
	Specifies all power domains or lists power domains to be included in the report. It prints a summary of power break down for each domain before listing power for each instance in the domain.
`-power_density_tiles value`	
	Specifies the number of tiles used for power density calculation. *Default*: `100`
`-power_density_tiles_row_column {value1 value2}`	

Defines the number of tiles by rows * columns. *value1* is the number of tile rows and *value2* is the number of tile columns.

The following example specifies to perform power density aware vector profiling using 2 tile rows and 2 column rows:

```
set_db power_method vector_profile power_vector_profile_mode power_density power_worst_step_size 10ns
report_power  -power_density_tiles_row_column  {2 2} -time_based_report -out_file tiling1.rpt
```

The following is the vector profile report for this command:

```
Tile 0_0_0 5.06 5.33 14.95 14.555
Tile 1_0_1 14.95 5.33 24.84 14.555
Tile 2_1_0 5.06 14.555 14.95 23.78
Tile 3_1_1 14.95 14.555 24.84 23.78
...
...
```

`-power_density_tiles_size {`*value1 value2*`}`

Defines the absolute tile size. *value1* is the horizontal width of a tile (in micron) and *value2* is the vertical height of a tile (in micron).

`-power_unit` *unit*

Sets the power unit to be used for power report generation. The default unit is mW (miliAmps).

`-print_memory_power`

Specifies to report power values for the memory and macro cells under separate groups (`Memory` and `Macro`).

The following is a snapshot of the power report showing separate groups for memory and macro cells:

Group	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
Sequential	0.4031	0.1283	0.006583	0.538	18.38
Memory	0.1624	0.004345	0.011	0.1778	6.074
IO	0	0	0	0	0
Physical-Only	0	0	2.91e-06	2.91e-06	9.945e-05
Combinational	0.8711	1.05	0.01579	1.937	66.2
Clock (Combinational)	0.002302	0.01018	7.654e-05	0.01255	0.429
Clock (Sequential)	0.01177	0.008387	0.000347	0.0205	0.7005
Macro (non-mem)	0.2317	0.007259	0.001234	0.2402	8.208
Total	1.682	1.209	0.03503	2.926	100

`-group_type` *user_defined_group_name*

Specifies to report power (internal/switching/leakage/total) for a user-defined collection of cells that are stated under a power group in the power summary report. You can use the following Tempus commands to create power groups:

- `define_property` - define a property.
- `set_property` - assign the defined property (group type) and its value (power group) to a list of cells.

The use model of the `-group_type` parameter is:

```
report_power -group_type user_defined_group_type
```

Here, *`user_defined_group_type`* is the user-defined power group type applied to a list of cells in a design. When the `-group_type` parameter is specified, the command reports power for the power groups associated with the specified group type in the power summary report.

Example:

The following command defines the `poly_bias` property (group type) on library cells:

```
define_property poly_bias -obj_type lib_cell -type string
```

The following commands apply the property `poly_bias` with the group name `P0` and `P10` to the collection of cells:

```
set_property [get_lib_cells -of_objects [get_cells -hierarchical * -filter {ref_name=~*PB0*}]]
poly_bias  P0
set_property [get_lib_cells -of_objects [get_cells -hierarchical * -filter {ref_name=~*PB10*}]]
poly_bias  P10
```

The following command reports power for the user-defined power groups that are associated with the property `poly_bias`:

```
report_power -out_file basic_group_type.rpt -group_type poly_bias
```

The following is a snippet of the power report that shows the user-defined power groups `P0` and `P10` that are associated with the property `poly_bias`, in addition to the existing groups (sequential, macro, IO, and so on):

```
User Group                      Internal   Switching     Leakage       Total  Percentage
                                Power      Power         Power         Power  (%)
-------------------------------------------------------------------------------------
P0                               0.01498   0.0002017     0.01765     0.03283       15.63%
P10                               0.1268     0.04026     0.01011      0.1772       84.37%
-------------------------------------------------------------------------------------
Total                             0.1418     0.04047     0.02776        0.21         100%
-------------------------------------------------------------------------------------
```

```
Group                           Internal   Switching     Leakage       Total  Percentage
                                Power      Power         Power         Power  (%)
-------------------------------------------------------------------------------------
Sequential                       0.02958   0.0003768     0.01458     0.04453       16.62
Macro                                  0           0           0           0           0
IO                                     0           0           0           0           0
Combinational                   0.008217    0.002633     0.01025      0.0211       7.877
Clock (Combinational)             0.1239     0.04421    0.007419      0.1755       65.51
Clock (Sequential)               0.01773    0.008783   0.0002459     0.02676       9.988
-------------------------------------------------------------------------------------
Total                             0.1794       0.056     0.03249      0.2679         100
-------------------------------------------------------------------------------------
```

`-register_gating_efficiency`

	Generates a report that includes the Register Gating Efficiency (RGE) and Data Aware Gating Efficiency (DAGE) for all sequential cell instances as well as different hierarchies in the design. Average RGE and DAGE for different hierarchies is reported. It also displays a section on ICG cluster information, that is, information about registers at the fanout of each ICG instance along with their RGE metrics.
	Register Gating Efficiency is calculated as:
	`RGE = 1 - (toggles at register clock pin/root clock toggles)`
	When you generate an RGE report using the command `report_power -register_gating_efficiency -out_file rge.rpt`, the following information will be displayed:
	• **Register-gating Efficiency Report** - for each clock domain, it includes the toggle rate, number of registers, number of clock gates, average clock toggle at registers, average toggle savings at registers, and average toggle savings histogram.
	• **Register Gating Opportunities Report** - this report is sorted based on the Q/CLK toggle ratio. When a register's Q/CLK ratio is greater than .25 (25%), Q toggles every other clock cycle. However, when it is less than 25%, there is a gating opportunity to reduce power.
	Data Aware Gating Efficiency is calculated as:
	`DAGE = 1 - ((toggles at clock pin - 2*toggles at Q pin)/(root clock toggles))`

`-report_prefix` *prefix*	
	Specifies a prefix for the report in which the top-level summary of the power consumption information is written. All the new report names will be *prefix*`.*.rpt`, and the detailed summary will be *prefix*`.rpt`.
`-sort` {`internal	switching	leakage	total`}	
	Sorts the instance-based power consumption data by `internal`, `switching`, `leakage`, or `total` power. *Default*: `total`
`-stat`	

	Reports a set of statistics on the net status after propagation, for example, duty, transition density, or activity. This parameter is useful in viewing and debugging user-defined activity factors after the power run is complete.
	Example:
	`report_power -time_based_report -distribute -out_file power.rpt -stat`
	`----------`
	`-- Net status after propagation --`
	`----------`
	`Net  Transition Density  Duty  Activity  Ref. Freq`
	`----------`
	`VDDA1  0  1  0  1.25e+08`
	`VSSA1  0  0  0  1.25e+08`
	`...`

`-thermal_conductivity_inputs` *file_name*	

	Specifies a file containing the thermal conductivity value for layers. You can use this parameter to include the thermal conductivity value of the following layers in the power map file:
	• Silicon
	• Metal
	• Dielectric
	• Micro Bump (optional for 3DIC design)
	• Fill material in micro bump layer (optional for 3DIC design)
	The thermal conductivity file format is:
	Section 1 (layer type based thermal conductivity value)
	`#default`
	`$layer_type1 $thermal_conductivity_parameter1`
	`$layer_type2 $thermal_conductivity_parameter2`
	`...`
	Section 2 (layer name based thermal conductivity value)
	`#perlayer`
	`$layer1_name $thermal_conductivity_parameter1 $dielectric_thermal_conductivity_1`
	`$layer2_name $thermal_conductivity_parameter2 $dielectric_thermal_conductivity_2`
	`...`
	Here,
	• You can either specify both Section 1 (layer type based) and Section 2 (layer name based), or specify one of the two sections. If both sections are specified, then layer name based section takes precedence over layer type based section.
	• The layer name based section has thermal conductivity value and its dielectric layer's thermal conductivity value for each metal/via layer.
	• If section 2 has no parameters defined, the software will use the default parameters defined in the section 1.
	Example:
	###default
	`Metal 430`
	`Silicon 250`
	`Dielectric 0.6`
	###Perlayer
	`Metal1 430 0.4`
	`Via1 410 0.5`
	`Metal2 390 0.5`
	`Via2 410 0.5`
	...
	For information on the thermal conductivity input file and power map file, refer to the "Static Power, IRdrop and EM Analysis" chapter in *Voltus User Guide*.

`-thermal_leakage_temperature {temp1 temp2 temp3 …}`	
	Defines the temperature to calculate leakage for the power map file.
`-thermal_leakage_temperature_scale_table {$temp1 $scalefactor1 $temp2 $scalefactor2 $temp3 $scalefactor3 .....}`	

Specifies the global scale factor for a temperature value to enable scaling of the leakage power values coming from the input dotlib file. This parameter allows you to scale the power values only at certain specific temperatures.

This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. `-thermal_leakage_temperature_scale_table` and `-thermal_leakage_temperature_scale_table_file` are mutually exclusive.

`-thermal_leakage_temperature_scale_table_file` *`filename`*

Specifies a file containing the user-defined temperature values and the corresponding scale factors for each library, cell, or instance. The format of the file is:

```
#Type Name temp1 temp2 temp3 temp4
Lib    library_name   scalefactor@temp1   scalefactor@temp2  scalefactor@temp3
scalefactor@temp4
Cell   cell_name      scalefactor@temp1   scalefactor@temp2  scalefactor@temp3
scalefactor@temp4
inst   instance_name  scalefactor@temp1   scalefactor@temp2  scalefactor@temp3
scalefactor@temp4
```

This parameter allows you to generate a multi-temperature leakage power map table from a single input dotlib file in the power map generation flow. `-thermal_leakage_temperature_scale_table` and `-thermal_leakage_temperature_scale_table_file` are mutually exclusive.

File Format Example for the Type Lib:

```
#Type Name         25 50 125 150
Lib   fast         1 2 10 100
Lib   bufao        1 2 10 100
Lib   header       1 2 10 100
Lib   pso_ring_wc  1 2 10 100
```

`-thermal_power_map_density_table_tiles {`*`Xint Yint`*`}`

Specifies the number of tiles in the X and Y directions for the metal density table.

`-thermal_power_map_file` *`file_name`*

Generates the power map file with the specified name in the output directory.

`-thermal_power_map_format {simple | stack }`

Generates a tile-based power map file in the simple or stack format to perform accurate thermal analysis. You must use the stack format to support multiple dies in a package for thermal analysis.

`-thermal_power_map_format_version {new|default}`

Determines whether to generate the default or the new power map file format. When set to `new`, the power map file format with non-uniform tile numbers for the power (`-thermal_power_map_power_table_tiles`) and metal density tables (`-thermal_power_map_density_table_tiles`) can be generated.

The following are the features of the new report format:

- version number added to the header "Version : 2020.0"
- nx/ny under the "DIE_AREA" section has been removed
- XCOR/YCOR for each layer has been removed
- nx/ny has been added for the POWER and METAL_DENSITY sections

`-thermal_power_map_header_include_file` *`filename`*

	Specifies a file that contains comments or additional information to be included in the header section of the power map file.
`-thermal_power_map_power_table_tiles {`*`Xint Yint`*`}`	
	Specifies the number of tiles in the X and Y directions for the power table.
`-thermal_power_map_si_unit {true | false}`	
	Specifies to change chip thermal power map units to SI in the thermal power map file. This parameter was introduced to support the thermal power map file format used by foundry. When this parameter is set to `true`, the values for all the power map parameters (length, power, and thermal conductivity) will be reported in the SI units. • **Length Unit:** default is `um` and SI is `m` • **Power Unit**: default is `mW` and SI is `W` • **Metal/Silicon Thermal Conductivity Unit**: default is `mW/(um * C)` and SI is `W/(m * C)` *Default* : `true`
`-thermal_material_file` *`filename`*	
	Specifies the name of a file containing the user-specified substrate layer thickness. The layer thickness value for the power map generation flow can either be a user-specified value or derived from the technology file. If both the values are available, the user-specified value takes precedence.
`-thermal_power_map_bottom_layer` *`layer_name`*	
	Writes zero metal density tables for layers lower than the user-specified layer in the power map file. This parameter prevents duplication of metal density tables when merging the top and block thermal power map files.
`-thermal_power_map_tiles {`*`xint yint`*`}`	
	Defines the tile number in the X and Y direction. By default 10*10 will be used.
`-thermal_power_map_top_layer` *`layer_name`*	
	Writes zero metal density tables for layers higher than the user-specified layer in the power map file. This parameter prevents duplication of metal density tables when merging the top and block thermal power map files.
`-threshold` *`value`*	
	Filters out the instances whose total power dissipation is more than the specified value. Units are in milliwatts. *Default*: `0`
`-threshold_voltage_group { all |` *`group_name`*`}`	
	Reports leakage by voltage threshold group in the power report. This parameter supports the argument `all` (default value) or any other string, such as hvt or lvt. *Default*: `all`
`-time_based_report`	
	Writes out a detailed VCD profiling report into the file that you specify.
`-time_unit` *`unit`*	
	Sets the time unit to be used for power report generation. The default unit is ns (nanoseconds).
`-toggle_rate`	

	Specifies to dump out instance-based activity to the outfile. The toggle rate for each instance is based on the clock frequency to which it belongs. If multiple clocks are reaching the instance, then the fastest clock is used as a reference.
`-use_geometry_cell_size {true | false}`	
	Specifies whether to use the actual geometry or placement bounding box for macro PGV cell size during generation of tile-based power map file. This parameter is used when a macro's power is distributed across multiple tiles. When this parameter is set to `true`, the geometry-based bounding box will be used for power map generation and power will be distributed evenly across tiles. *Default* : `false`
`-view` *`view_name`*	
	Specifies the active timing view that was created using the multi-mode multi-corner (MMMC) `set_analysis_view -setup|-hold` command. Power analysis is performed for the specified view. The command prints the `MMMC` view name in the output file even if you do not specify the `-view` parameter. If `report_power -view` *`view_name`* is set, it will override the view set in the `power_view` attribute. However, if you specify the `set_analysis_view -dynamic` and `-leakage` parameters, then the view set using these `set_analysis_view` command parameters has higher priority over the `report_power -view` command. *Default*: Uses current view in Tempus

Examples

- The following command reports the amount of power consumed by the clock nets clk and test_clk (including leaf flip-flop power) and writes the output information to a power report file named clk_pwr.rep:

```
report_power -clock_network{clk test_clk} \
-include_sequential_in_clock -out_file clk_pwr.rep
```

- The following command reports the amount of power consumed down to hierarchy level 3 from the top and writes the output information to a power report file named hier.rep:

```
report_power -hierarchy 3 -out_file hier.rep
```

- The following command compresses the power report file pg_net.rep with gzip:

```
report_power -out_file pg_net.rep.gz
```

- The following command reports the amount of power consumed by leaf-level instances that match ECO* and writes the output information to a power report file named inst.rep:

```
report_power -insts ECO* -out_file inst.rep
```

- The following command reports the amount of power consumed by all cells that match buf*6 and writes the output information to a power report file named cell.rep:

```
report_power -cell buf*6 -out_file cell.rep
```

- The following command reports the amount of power consumed by all I/O and sequential cells and writes the output information to a power report file named cell_type.rep:

```
report_power -cell_type {io sequential} -out_file cell_type.rep
```

- The following command sorts the instance-based power consumption data by leakage power and writes the output information to a power report file named sort.rep:

```
report_power -sort leakage -out_file sort.rep
```

- The following command reports the total power of all instances whose total power dissipation is less than 1 milliwatt and writes the output information to a power report file named threshold.rep:

```
report_power -threshold 1 -out_file threshold.rep
```

- The following command reports the net switching power, load capacitance, toggle rate, and switching values for each net in the design,

reports the top 100 nets with the highest net switching power, and writes the output information to a power report file named net.rep:

```
report_power -net -nworst 100 -out_file net.rep
```

- The following command reports the amount of power consumed by each power domain and writes the output information to a power report file named pd_all.rep:

  ```
  report_power -power_domain all -out_file pd_all.rep
  ```

- The following command reports the amount of power consumed by each power domain PD2 and writes the output information to a power report file named pd.rep:

  ```
  report_power -power_domain PD2 -out_file pd.rep
  ```

- The following command reports the amount of power consumed by view view_max-hp-max-lp1 and writes the output information to a power report file named view.rep:

  ```
  report_power -view view_max-hp-max-lp1 -out_file view.rep
  ```

- The following command reports the amount of power consumed by each instance connected to power nets vdd and vdd1 and writes the output information to a power report file named pg_net.rep:

  ```
  report_power -pg_net {vdd vddl} -out_file pg_net.rep
  ```

- The following command reports the leakage for the threshold voltage group HVT:

  ```
  report_power -leakage -threshold_voltage_group { HVT }
  ```

- The following command writes out a detailed vector profiling report for the combinational and macro cell types in the `vectorprofile.rpt` file:

  ```
  report_power -time_based_report -out_file vectorprofile.rpt -cell_type {combinational macro}
  ```

- The following command is an example of generating multiple reports at the same time:

  ```
  report_power -out_dir static_power -report_prefix design -net {name1} -insts {inst_name}
  ```

  The following reports will be generated:

  ```
  static_power/design.rpt
  static_power/design.netpwr.rpt
  static_power/design.instpwr.rpt
  ```

- The following command specifies to report power for combinational cells on the clock network.

  ```
  report_power -cell_type clock_combinational -out_file static_power/power_clk.rpt
  ```

- The following command specifies to report the power content for the `inst1` hierarchical instance in the `report.hierinstpwr.rpt` report:

  ```
  report_power -output static -report_prefix report -clock_network {all} -instances * -no_wrap -hierarchy {all} -
  hierarchical_instances inst1 -cell_type {all}
  ```

- The following command generates a power map file in the simple format for the specified tile:

  ```
  report_power -out_file powermeter.txt -thermal_power_map_file powermap.txt -thermal_power_map_tiles { 4 6 } -
  thermal_power_map_format simple
  ```

- Example of the output of the power report.

  ```
  *------------------------------------------------------------------------------
  *        -  - Version  64-bit
  *       Date & Time:    2008-Jan-27 15:49:51 (2008-Jan-27 23:49:51 GMT)
  *------------------------------------------------------------------------------

  *       Design: dtmf_chip
  *
  *       Liberty Libraries used:
  *               /sev//DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_rvt_ss_1p08v_125c.lib
  *               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_rvt_ss_0p8v_1p08v_125c.lib
  *               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_rvt_ss_0p8v_125c.lib
  *               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_lvt_ss_1p08v_125c.lib
  ```

```
*               /sevDTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_lvt_ss_0p8v_1p08v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_lvt_ss_0p8v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_hvt_ss_1p08v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_hvt_ss_0p8v_1p08v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/scmetropmk_cmos10lp_hvt_ss_0p8v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_rvt_ss_1p08v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_rvt_ss_0p8v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_lvt_ss_1p08v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_lvt_ss_0p8v_125c.lib
*    /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_hvt_ss_1p08v_125c.lib
*     /sev/DTMF_CHIP/TIMING/max/scmetro_cmos10lp_hvt_ss_0p8v_125c.lib
*     /sev/TIMING/max/rom_512x16A_ss_1v08_125c_syn.lib
*     /sev/DTMF_CHIP/TIMING/max/pllclk_slow.lib
*               /sev/DTMF_CHIP/TIMING/max/iometroil_cmos10lp_hvt_ss_1p08v_2p3v_2p5v_125c.lib
*               /sev/DTMF_CHIP/TIMING/max/iometroil_cmos10lp_hvt_ss_0p8v_2p3v_2p5v_125c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_rvt_ff_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_rvt_ff_1p1v_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_lvt_ff_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_lvt_ff_1p1v_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_hvt_ff_1p32v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/scmetropmk_cmos10lp_hvt_ff_1p1v_1p32v_m40c.lib
*

/sev/DTMF_CHIP/TIMING/min/scmetro_cmos10lp_rvt_ff_1p32v_m40c.lib
*
/sev/DTMF_CHIP/TIMING/min/scmetro_cmos10lp_lvt_ff_1p32v_m40c.lib
*
/sev/DTMF_CHIP/TIMING/min/scmetro_cmos10lp_hvt_ff_1p32v_m40c.lib
*
/sev/DTMF_CHIP/TIMING/min/rom_512x16A_ff_1v32_m40c_syn.lib
*
/sev/DTMF_CHIP/TIMING/min/pllclk_fast.lib
*               /sev/DTMF_CHIP/TIMING/min/iometroil_cmos10lp_hvt_ff_1p32v_2p7v_2p5v_m40c.lib
*               /sev/DTMF_CHIP/TIMING/min/iometroil_cmos10lp_hvt_ff_1p1v_2p7v_2p5v_m40c.lib
*
*       Power Domain used:
*               Rail:     VDD_AO        Voltage:       1.32
*               Rail: VDD_TDSPCore      Voltage:       1.32
*               Rail: VDD_TDSPCore_R    Voltage:       1.32
*
*       Parasitic Files used:
* /sev/DTMF_CHIP/SPEF/dtmf_chip.decoup.spef
*
*       DEF Files used:
* /sev/DTMF_CHIP/DEF/dtmf_chip.def
*
*       Power Units = 1mW
*       Time Units = 1e-09 secs
```

```
*       report_power
--------------------------------------------------------------------------------
  Total Power
--------------------------------------------------------------------------------
Total Internal Power:      0.01943(62.6774%)
Total Switching Power:     0.01057(34.0968%)
Total Leakage Power:       0.001(3.2258%)
Total Power:               0.031
--------------------------------------------------------------------------------
Group      Internal      Switching     Leakage       Total  Percentage
Power      Power         Power         Power  (%)
--------------------------------------------------------------------------------
Sequential    0.0006738   0.0002215   4.045e-05   0.0009358     3.019
Macro         0.0004916   1.515e-05   0.0002609   0.0007677     2.477
IO            0.01743     0.009085    0.0003172   0.02683      86.56
Combinational 0.0008324   0.001247    0.0003815   0.002461      7.941
--------------------------------------------------------------------------------
Total 0.01943     0.01057       0.001       0.031         100
--------------------------------------------------------------------------------
Rail                   Voltage   Internal   Switching   Leakage Total  Percentage                        Power     Power
     Power        Power  (%)
--------------------------------------------------------------------------------
VDD_AO                  1.32  0.0008238   0.0003404   0.0002694    0.001434       4.625
VDD_TDSPCore           1.32   0.001126    0.001022   0.0003931    0.002541       8.197
VDD_TDSPCore_R         1.32  5.833e-06   3.253e-07   8.371e-07   6.996e-06     0.02257


Clock                    Internal   Switching     Leakage       Total  Percentage
                             Power      Power         Power         Power  (%)
--------------------------------------------------------------------------------

m_digit_clk               0           0           0           0           0
m_ram_clk                 0           0           0           0           0
m_dsram_clk               0           0           0           0           0
m_spi_clk           1.018e-06    1.09e-06   2.572e-08   2.134e-06    0.006884
m_rcc_clk           1.282e-05   1.513e-05   2.672e-07   2.821e-05     0.09103
m_clk               5.039e-05   7.278e-05   7.658e-07   0.0001239      0.3998
refclk              1.11e-05   3.017e-05   2.488e-07   4.152e-05       0.134
--------------------------------------------------------------------------------
Total               7.533e-05   0.0001192   1.307e-06   0.0001958      0.6317
--------------------------------------------------------------------------------

*       Power Distribution Summary:
*  Highest Average Power:     IOPADS_INST/Prefclkip (PBCSCT2B):     0.003727
*  Highest Leakage Power: DTMF_INST/ROM_512x16_0_INST (rom_512x16A): 8.899e-05
*               Total Cap:      2.46221e-10 F
*               Total instances in design:  8525
*               Total instances in design with no power:    24
```

```
*                 Total instances in design with no activty:    24
*               Total Fillers and Decap:     8
--------------------------------------------------------------------------------
```

- Next command creates a leakage report (see example report below):

```
report_power -leakage
*--------------------------------------------------------------------------------
*    Tempus Timing Signoff Solution 08.10-b050_1 (64bit) 08/21/2008 19:23 (Linux 2.6)
*
*
*       Date & Time:    2008-Aug-24 21:34:17 (2008-Aug-25 04:34:17 GMT)
*
*--------------------------------------------------------------------------------
*
*       Design: dma_mac
*
*       Liberty Libraries used:
*               DATA/timing_libs//tcbn65lp_LVL_c060217wc0d90d9.lib
*               DATA/timing_libs//tcbn65lpcgwc.lib
*               DATA/timing_libs//tcbn65lplvtwc.lib
*               DATA/timing_libs//tcbn65lpwc.lib
*               DATA/timing_libs//tsdn65lpa1024x32m8f_100a_wc.lib
*               DATA/timing_libs//tsdn65lpa128x16m8f_100a_wc.lib
*               DATA/timing_libs//tcbn65lpcgwc0d72.lib
*               DATA/timing_libs//tcbn65lplvtwc0d72.lib
*               DATA/timing_libs//tcbn65lpwc0d72.lib
*               DATA/timing_libs//tsdn65lpa1024x32m8f_0p84v_wc.lib
*               DATA/timing_libs//tsdn65lpa128x16m8f_0p84v_wc.lib
*
*       Power Domain used:
*               Rail:      VDDlu        Voltage:       0.84
*               Rail:       VDDm        Voltage:       1.08
*               Rail:        VDD        Voltage:       1.08
*
*       Power Units = 1mW
*
*       Time Units = 1e-09 secs
*
*       report_power -out_file Leakage.rep -leakage
*
--------------------------------------------------------------------------------
Total Power
--------------------------------------------------------------------------------
Total Leakage Power:       0.1879
--------------------------------------------------------------------------------
Group                           Leakage       Percentage
                                Power         (%)
--------------------------------------------------------------------------------
Sequential                        0.0127       6.756
```

```
Macro                                0.1556        82.8
IO                                      0           0
Combinational                     0.01962       10.44
------------------------------------------------------------------------------
Total                              0.1879         100
------------------------------------------------------------------------------
Rail                   Voltage   Leakage       Percentage
                                 Power         (%)
------------------------------------------------------------------------------
VDDau                        0         0           0
VDDlu                     0.84   0.06477       34.47
VDDm                      1.08  0.003587       1.909
VDD                       1.08    0.1195       63.62

Clock                            Leakage       Percentage
                                 Power         (%)
------------------------------------------------------------------------------
phy_rxclk_2                             0           0
phy_txclk_2                     0.0004786      0.2547
phy_rxclk_1                             0           0
phy_txclk_1                      3.39e-05     0.01804
clk                              0.001723      0.9171
------------------------------------------------------------------------------
Total                            0.002236        1.19
------------------------------------------------------------------------------



------------------------------------------------------------------------------

*       Power Distribution Summary:
*               Highest Average Power: ethernet_mac_2/tx_fifo_module_from_dma/fifo1/dual_port_
   ram_4_tx_fifo/ram2P1024x32/ram (TSDN65LPA1024X32M8F):      0.02424
*               Highest Leakage Power: ethernet_mac_2/tx_fifo_module_from_dma/fifo1/dual_port_
   ram_4_tx_fifo/ram2P1024x32/ram (TSDN65LPA1024X32M8F):      0.02424
*               Total Cap:      1.01625e-10 F
*               Total instances in design: 29235
*               Total instances in design with no power:  3596
*          Total instances in design with no activty:   115
*               Total Fillers and Decap:     0
------------------------------------------------------------------------------

Total leakage power = 0.187903 mW
  Cell usage statistics:
  Library tsdn65lpa128x16m8f_1v08, 1 cells ( 0.003900%) , 0.01016 mW (    5.407063%  )
  Library tsdn65lpa1024x32m8f_1v08, 6 cells ( 0.023402%), 0.145427 mW (    77.394568%  )
  Library tcbn65lp_LVL_c060217wc0d90d9 , 120 cells ( 0.468037%), 0.000175605 mW    ( 0.093455%  )

  Library unknown , 3881 cells ( 15.137096%) , 0.00630072 mW ( 3.353178%  )
```

```
Library tcbn65lpcgwc , 118 cells ( 0.460236%) , 0.000625092 mW ( 0.332667%  )
Library tcbn65lplvtwc , 3762 cells ( 14.672959%) , 0.0110529 mW ( 5.882233%  )
Library tcbn65lpwc , 17751 cells ( 69.234370%) , 0.0141619 mW ( 7.536835%  )
```

report_unannotated_nets

```
report_unannotated_nets
[-help]
[-file filename]
-type annotationType
```

Shows the percentage of nets annotated by a specific type and also prints the names of all nets that are not annotated by that type. Annotation information is stored on a per-file basis for activity, VCD, TCF, and user commands to provide detailed annotation reports when requested.

`report_unannotated_nets` can be invoked multiple times with different report types (`-type`). You must specify this command before `report_power`.

Parameters

`-file` *filename*	Specifies the text file name that includes the annotation report information. If no file is specified, the report will be written to the default output file `annotation_`*type*`.log`, where *type* is taken from the `-type` argument specified.
`-help`	Outputs the command usage and a brief description about the command parameters.
`-type` *annotationType*	The type can be any of the following: • `all` Shows annotation reports for all types. • `activity` Shows annotation data for all types of activity specifications including TCF, VCD, SDC, TWF, and user commands that set activity. • `cap` Shows annotations from the SPEF files. • `fsdb` Shows annotations from the FSDB files. • `saif` Shows annotations from the SAIF files. • `tcf` Shows annotations from the TCF files. • `timing` Shows annotations from the TWF files. This type is applicable only to the dynamic power analysis flow. • `vcd` Shows annotation from the VCD files.

Example

- The following command dumps the nets in the netlist that are missing in the vcd file:

  ```
  report_unannotated_nets -type vcd -file vcd.log
  ```

  The net names are written in the file `vcd`.log.

- The following command dumps the nets in the netlist which do not have an activity annotated from any source (VCD/TCF/SDC/TWF/user defined) and writes the net names in the file `unannotatednets.log`:

  ```
  report_unannotated_nets -type all -file unannotatednets.log
  ```

reset_power_activity

```
reset_power_activity
```

Resets all activity in the database. You can specify this command to reset all activity and power information in a given session.

> You can reset activity from an activity file using `read_activity_file -reset`, reset activity for nets, pins, and ports using `set_switching_activity -reset`, and reset the default switching activities using `set_default_switching_activity -reset`.

Parameters

None

set_default_switching_activity

```
set_default_switching_activity
[-black_box_density value]
[-black_box_duty value]
[-block string]
[-duty value]
[-global_activity factor]
[-hierarchy hierarchy_name]
[-input_activity factor]
[-period value]
[-sequential_activity factor]
[-reset]
[-reset_type { global_activity | seq_activity ... }]
[-clock_gates_enable {activity_factor}]
[-integrated_clock_gate_ratio num]
[-combinational_clock_gate_ratio num]
[-clock_gates_output_ratio num]
[-clip_activity_to_domain_freq {true | false}]
[-clock_gates_output {activity_factor}]
[-name activity_name]
[-macro_activity factor]
```

Specifies the switching activity for all primary inputs, nets, and other devices in the design whose activity has not been previously defined through user attributes, the toggle count format (TCF) file, the simulation vectors file (VCD, SHM, and so on), or the tracing of the clock network.

You can use this command to specify user-defined activities at the design level.

Parameters

`-black_box_density value`	
	Specifies to set the default transition density on all the output pins of all blackboxes in the design. The `-black_box_density` and `-black_box_duty` parameters must be used together. If one or both of these parameters are not specified, then the software will use the average transition density from all the input pins of the blackboxes.
`-black_box_duty value`	
	Specifies to set the default duty on all the output pins of all blackboxes in the design. The `-black_box_density` and `-black_box_duty` parameters must be used together. If one or both of these parameters are not specified, then the software will use the average transition density from all the input pins of the blackboxes.
`-block string`	
	Specifies the name of the master cell for activity setting. The parameter allows you to define the user-defined switching activity for a block partition. This parameter is useful for assigning block-based activity. The `-block` parameter is applicable only to the block-based distributed power flow.
`-clip_activity_to_domain_freq {true | false}`	
	Specifies to perform domain-based activity clipping. This parameter clips data paths to 1x frequency whereas the clock networks are still at 2x frequency. This parameter allows you to have more control to determine the rate of clipping with respect to frequency.
`-clock_gates_enable {activity_factor}`	

	Specifies the activity at the enable pin(s) of the clock gating cells.
`-clock_gates_output {`*`activity_factor`*`}`	
	Specifies the activity at the output pin(s) of the clock gating cells. This works for both ICGs and combinational clock gating cells.
`-clock_gates_output_ratio` *num*	
	Specifies to control transition density at the output pin(s) of the integrated and combinational clock gating cells. This parameter is equivalent to specifying the parameters `-icg_ratio` and `-comb_clockgate_ratio` together with the same value. For example, `set_default_switching_activity -clock_gates_output_ratio 0.6` is equivalent to `set_default_switching_activity -icg_ratio 0.6 -comb_clockgate_ratio 0.6`. **Note**: This parameter has a cascading effect when there are multiple levels of ICG cells on the clock path. **For example,** when `-clock_gates_output_ratio` is set to `0.5`, the first level ICG cell will have 0.5 ratio of the root clock's toggle density, the second level ICG cell will have 0.5*0.5=`0.25` ratio of the root clock's toggle density, and the third level ICG cell will have 0.5*0.5*0.5=`0.125` ratio of the root clock's toggle density, and so on.
`-combinational_clock_gate_ratio` *num*	
	Specifies to set the propagation ratio for combinational clock gate cell outputs. When specified, this parameter sets the output activity of any instance that is identified as a combinational clock gating cell to *num* times the fastest transition density arriving on the instance's clock inputs. **Note:** This parameter does not apply to combinational clockgates with a constant data input, that is, a select pin tied low.
`-duty` *value*	Specifies the duty cycle, which is the probability that the signal is a logical 1. **Note:** You must specify the `-period` parameter when assigning a duty. *Default*: `0.5` (equivalent to a 50 percent duty cycle)
`-global_activity` *factor*	
	Specifies the average number of times that all unset nodes switch in a clock cycle. If not specified the fastest domain frequency will be used unless one turns off clock domains.
`-hierarchy` *hierarchy_name*	
	A hierarchical name that the global activity is being assigned to. If not provided, the whole design is assumed.
`-integrated_clock_gate_ratio` *num*	
	Specifies to set the propagation ratio for integrated clock gate (ICG) cell outputs. When specified, this parameter sets the output pin of all identified ICG instances to *num* times the fastest transition density arriving on clock inputs of the instance.
`-input_activity` *factor*	
	Specifies the switching activity of primary inputs in a clock cycle. It can be any positive number. *Default*: `0.2` (Switches once every five clock cycles.)
`-macro_activity` *factor*	
	Specifies the activity factor for macros in a design. The allowed values for this parameter are between `0` to `1`. If you set the macro activity to `.30`, 30% of macros in the design would be selected for switching in a given cycle during dynamic simulation. When `-macro_activity` is specified for the static power analysis, all output pins of the macros are assigned the specified activity. *Default*: `0.2`
`-name` *activity_name*	

	Allows you to set the activity level for the specified jitter activity name. You can define a low/high activity level for jitter analysis.
	Examples:
	`set_default_switching_activity -name low -clock_gates_output 0 -global_activity 0.01`
	`set_default_switching_activity -name high -clock_gates_output 2 -global_activity 0.2`

`-period` *value*	Specifies the default operating period (1/frequency). Specifies the period that the activity is referenced to. If not specified, the software uses the *dominant* clock in the design. If no clock is specified, the software then generates an error and prompts you to enter one. Units in seconds (`s`), milliseconds (`ms`), microseconds (`us`), nanoseconds (`ns`), or picoseconds (`ps`). **Note:** If you enter the period as a negative number, the software reverts back to using the default frequency. *Default*: The time unit is defined in the first `.lib` file read during design import.
`-sequential_activity` *factor*	
	Specifies the activity of outputs of the sequential logic. This value will be used at the output ports of flops for propagating activity. If the sequential activity is not specified, propagation will be done for the full design starting from the input ports.
`-reset`	Resets all specified options back to default values.
`-reset_type { global_activity | seq_activity ... }`	
	Specifies to selectively reset the activity specifications. **Note:** You must use the `-reset_type` parameter in conjunction with the `-reset` parameter.

Activity Precedence

See "Activity Precedence".

Examples

- The following command specifies that the primary inputs in the design will switch once every ten clock cycles, with a clock period of 7520 ns and a duty cycle of 50 percent:
  ```
  set_default_switching_activity -input_activity 0.10 -period 7520ns -duty .5
  ```
- The following command specifies that all unswitched nodes in the design will switch once every ten clock cycles, with a clock period of 7520 ns and a duty cycle of 50 percent:
  ```
  set_default_switching_activity -global_activity 0.10 -period 7520ns -duty .5
  ```
- The next example defines the switching activity, sequential activity, and the period:
  ```
  set_default_switching_activity -input_activity 0.2 -period 10 \
      -sequential_activity 0.1
  ```
- The next example sets the default global activity to 0.5 for the hierarchy dma_dut:
  ```
  set_default_switching_activity -global_activity 0.5 -hierarchy dma_dut
  ```
- The following command resets the sequential and global activity specification:
  ```
  set_default_switching_activity -reset -reset_type
  {sequential_activity,global_activity}
  ```

Related Commands

- `read_activity_file`
- `get_activity`

- propagate_activity
- reset_power_activity
- set_switching_activity

set_dynamic_power_simulation

```
set_dynamic_power_simulation
[-period value]
[-resolution value]
[-reset]
[-activity_pattern {time1 activity_name1 time2 activity_name2 .... }]
```

Specifies the period and resolution for a dynamic power simulation. This command is used to specify the simulation period in the dynamic vectorbased/vectorless and power-up flows.

Parameters

-activity_pattern {*time1 activity_name1 time2 activity_name2 .... }*	
	Divides the total simulation time based on the windows given in the `activity_pattern` parameter. For a particular window, it uses the defined activity specified with the `set_default_switching_activity` command. When the user-defined low cycle and high cycle time is specified for the input/sequential activity, all the clocks will be in the low or high cycle until completely past the cycle transition time. Following is the use model of this parameter: • `set_default_switching_activity` **`-name low -input_activity 0.05 -sequential_activity 0.1`** `-clock_gates_output 0.2` • `set_default_switching_activity` **`-name high -input_activity 0.1 -sequential_activity 0.2`** `-clock_gates_output 2.0` • `set_dynamic_power_simulation -resolution 50ps` **`-activity_pattern {0 high 5ns low 10ns high}`** Here, high activity is used for 0ns to 5ns, low activity is used for 5ns to 10ns, and high activity is used for 10ns to 15ns.
`-period` *value*	Specifies the simulation period that dynamic power analysis will use. The supported time units are `s`, `ms`, `us`, `ns`, and `ps`. If no explicit time unit is specified, the default `ns` (nanosecond) will be applied.
`-reset`	Resets all specified options back to default values.
`-resolution` *value*	Specifies the transient time step size or resolution. The supported time units are `s`, `ms`, `us`, `ns`, and `ps`. If no explicit time unit is specified, the default `ps` (picosecond) will be applied.

Examples

- The following command sets up the resolution for a dynamic simulation:
  ```
  set_dynamic_power_simulation -resolution 100ps
  ```

set_instance_temperature_file

```
set_instance_temperature_file
file
[-reset]
```

Specifies the instance temperature file to support thermal-aware leakage. You can use this command to do thermal-aware power calculation. Based on the temperature specified in the file, the software will calculate temperature and leakage power.

In the presence of a trilib set, the internal and leakage power will be interpolated as the per the given temperature specification.

Parameters

file	Specifies the name of the instance temperature file..
-reset	Resets the temperature file.

set_power

```
set_power
[value]
[-cap value]
[-fanout_limit value]
[-reset]
[-leakage value]
[-pg_pin {pg_pin}]
[-pg_net railname]
[-pwl_waveform]
[-sticky]
[-dynamic_async_mode { mode_name }]
[-dynamic_switch_pattern pattern]
[-dynamic_switch_event { switching_events }]
[-scale_factor value}]
[-include_ddv_modes {pattern_list}]
[-internal]
[-switching value]
[-clocks {all|clock_name}]
[-exclude_clocks]
[-exclude_ddv_modes {pattern_list}]
[-base_cells cell_name value]
[-insts inst_list value]
[-pin pin_name pin_power]
[-force]
[-custom_macro_pwl_waveform filename]
[-region]
[-repeat]
[-ascii_power_file filename]
[-trigger_time_adjustment time]
[-no_propagation]
[-static_mode { custom_label_name | mode_name }]
[-signal_net netname]
[-slew value]
[-default_dynamic_pgv_mode { mode_name }]
```

Specifies the amount of power for all or part of the design. This command is used to define user-specified power on cells and instances. You must specify this command before `report_power`.

The `set_power` command can accept a cell/instance list using the `get_cells`/`get_lib_cells` commands. You can use the `get*` commands to create a collection of cells/instances whose name matches the supplied pattern list. Therefore, providing you with the flexibility in providing lists through use of regular filter patterns. This feature is supported in both static and dynamic analysis engines.

Parameters

```
-ascii_power_file filename
```

	Specifies the name of an instance list file. This file contains the list of instances in the following format: *instance_name power_value <pin_name>* Here, • *instance_name* is the instance name to which power is to be distributed. • *power_value* is the power value for the instance in watts. • *pin_name* is an optional argument to specify the pin name. When specified, allows you to assign power to the specific pin of the instance. • The three fields should be in one line, and each instance should be in a new line.
-base_cells *cell_name value*	
	Specifies the cell name. If the cell or instance name is not specified, the power will be distributed to all rails. If you want to specify power for a group of cells/instances, use the get_cells/get_lib_cells commands. The -cells parameter can accept these commands to create a collection of cells/instances whose name matches the supplied pattern list. For example, -cells[get_lib_cells NAND*] 2mw
-cap *value*	
	Specifies the capacitance value to all instances with nets more than or equal to the fanout limit (-fanout_limit).
-clocks {all|*clock_name*}	
	Specifies the power value for clock networks. You can specify the power value for a specific clock network or the clock network power value for the whole design. **Note:** This parameter cannot be used with the -cells, -insts and -pg_net parameters.
-custom_macro_pwl_waveform *filename*	
	Specifies to use a custom trigger file to apply instance or cell PWL waveforms. Given one master cell PWL waveform, the software can automatically generate scaled PWL waveforms of the other companion cells defined in the trigger file proportional to the cell internal power. **Note**: This parameter must be used with set_power -dynamic_switch_pattern in the vectorless flow. For more information on the trigger file format, see Trigger File Format.
-default_dynamic_pgv_mode { *mode_name* }	
	Specifies the current characterization mode to be used during dynamic power and jitter analyses. This parameter can be specified when using a PGV with multiple functional modes. When this parameter is specified, the Power Calculation engine uses this preferred mode in scheduling the switching events for the macro blocks, and saves these events in the trigger file. The use model of this parameter is: set_power -base_cells {} | -insts {} -default_dynamic_pgv_mode { *mode_name* } This parameter is mutually exclusive to the -dynamic_switch_pattern parameter. If a cell or instance has both the -dynamic_switch_pattern and -default_dynamic_pgv_mode parameters defined in the same Tcl script, -dynamic_switch_pattern takes higher priority.
-dynamic_async_mode { *mode_name* }	

	Specifies to trigger asynchronous macros based on the length of the PWL waveform to create a single continuous waveform. This parameter allows you to specify the asynchronous mode for an instance or a cell. The start and end of the PWL saved in the PGV must have similar current value in order to avoid sudden jump in the current.
	Use model:
	`set_power -dynamic_async_mode {mode} [-insts {name} | -base_cells {name}]`
	Example:
	`set_power -base_cells pll -dynamic_asynchronous_mode {READ_C}`
	Here, power analysis will trigger the `READ_C` mode and automatically splice the waveform based on its length.

`-dynamic_switch_pattern` *pattern*	

	Specifies stimulus pattern on the pins of an instance and cell. Specify this parameter only in the dynamic mode for the type `instance` and `cell`.
	The *pattern* is applied to all pins (input and output) on the instance/cell. It is a string.
	pattern specifies the switching pattern. If there is more than one clock domain with timing windows on a pin, only the windows on the fastest clock will be used, and the pattern will be applied to that clock period. It is a string of `0`, `1`, `r`, and `f` values. `0` for cycles not to switch and `1` for cycles to switch. `r` is for rise only and `f` for fall only.
	The pattern string should include enough digits to cover all of the clock cycles in the simulation period. If there are not enough digits, the pattern will be padded with 0 values at the end. If there are too many, the pattern will be truncated to the number of applicable cycles. Transitions will be scheduled in the center of each applicable timing window.
	Notes
	• `0` and `1` is supported only in the probability-based vectorless flow. It is not supported in the state-propagation-based vectorless flow and the vector-based flow.
	• In case of macros, you can specify the mode names, that is, which mode switches in which clock cycle `-dynamic_switch_pattern { mode1 - mode2 - }`.
	You can specify "`-`" to indicate that instances will not switch in a particular clock cycle. If "`-`" is the first value in the string of values specified within `{}`, you must insert a space before "`-`", as shown below:
	• no switching: `-dynamic_switch_pattern { - - }`
	• no switching in the 1st clock period: `-dynamic_switch_pattern { - r }`
	Examples:
	In the following example, instance will not switch in the 4th clock cycle:
	`set_power -dynamic_switch_pattern { f r f - r }`
	In the following example, the mode `write` will switch in the 1st clock cycle, the mode `read` will switch in the 2nd and 4th clock cycle, and none of the modes will switch in the 3rd clock cycle:
	`set_power -dynamic_switch_pattern { write read - read }`
	Note: You can use the wildcard (*) character for the `-insts` and `-cell` parameters to specify a pattern of instance/cell names to which the specified switching pattern is to be applied.

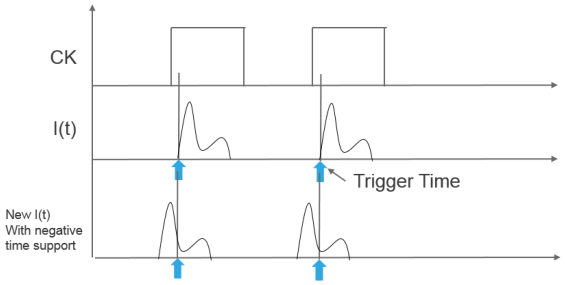
`-dynamic_switch_event { switching_`*events*` }`	
	Specifies the switching events for cells, instances, and pins along with their rise and fall time. `switching_`*events* is a string of toggle time followed by "r" or "f", where "r" is for rise toggle and "f" is for fall toggle. The `-dynamic_switch_event` parameter should not be specified with the `-repeat` parameter.
`-exclude_clocks`	Excludes clock networks when specifying a scale factor or power specification for the whole design. This parameter cannot be used with the `-cells`, `-insts` and `-pg_net` parameters.
`-exclude_ddv_modes {`*pattern_list*`}`	

	Excludes the current characterization (or Dynamic Detailed View) modes that match with the specified patterns. This parameter allows you to control the modes to be switched, instead of switching all the current characterization modes defined in the power-grid view. **Use Model** `set_power -insts|-base_cells name -exclude_ddv_modes {pattern_list}` Here: • *pattern_list* is a list of mode names, separated by space or comma • the instance, cell, and pattern names can contain wildcard characters
`-fanout_limit value`	
	Specifies the fanout limit for nets. If an instance has nets more than or equal to the fanout limit, its switching/internal/leakage power is taken to be zero. When this parameter is specified, the user-defined power value is applied to all the driver instances that have nets more than or equal to the fanout limit.
`-force`	
	Specifies to apply the `set_power` command even if there are Verilog mismatches. For dynamic or DEF-based static analysis, this parameter allows you to apply `set_power` to the DEF instances that do not exist in Verilog.
`-help`	Outputs the command usage.
`-include_ddv_modes {pattern_list}`	
	Includes only the current characterization (or DDV) modes that match with the specified patterns. This parameter allows you to control the modes to be switched, instead of switching all the current characterization modes defined in the power-grid view. **Use Model** `set_power -insts|-base_cells name -include_ddv_modes {pattern_list}` Here: • *pattern_list* is a list of mode names, separated by space or comma • the instance, cell, and pattern names can contain wildcard characters
`-internal`	Specifies the target internal power of the design, cell, or instance. To specify internal power for cells and instances, you can use the `-internal` parameter in conjunction with the `-cells` or `-insts` parameter.
`-insts inst_listvalue`	
	Specifies the instance name. If the cell or instance name is not specified, the power will be distributed to all rails.
`-leakage`	Specifies the target leakage power of the design, cell, instance, or power net. To specify leakage power for cells and instances you can use `-leakage` option in conjunction with `-cells` or `-insts` option. To specify leakage power of a certain power net in the design you can use `-pg_net` option along with `-leakage` option.
`-no_propagation`	
	Specifies that the user-defined switch pattern is applied only after state-propagation is complete. The use model of the command parameter is: `set_power -dynamic_switch_pattern {pattern} -no_propagation` The `-no_propagation` parameter must be specified with the `-dynamic_switch_pattern` parameter. When the `-no_propagation` parameter is specified, the dynamic power engine performs state-propagation first, and then overrides the single instance switching event by the user-defined dynamic switch pattern without any further propagation.
`-pg_net railname`	

	Optional. The specified power or pwl for the cell, instance or design is applied to the specified rail. When not specified, the power or pwl is applied to all the rails associated with the instance. This option is generally used for MSMV cells. When this option is not specified the behavior is different for pwl and power options. When not specified and power option is used, the power is applied for the whole design, cell, or instance. When not specified and pwl option is used, the pwl is applied to every rail of the cell or instance.
`-pg_pin {pg_pin}`	
	Specifies the switch pattern for a specific PG pin of an instance. This parameter must be specified with the `-dynamic_switch_pattern` parameter.
`-pin pin_name pin_power`	

	Specifies pin-wise power numbers for a specific instance. This parameter allows you to control the power assigned to power ground pins of an instance. Following is the use model of this parameter:
	`set_power -insts <inst_name> -pin <VDDA> <pin_power>`
	`set_power -insts <inst_name> -pin <VDDB> <pin_power>`

`-pwl_waveform`	Specifies that power argument (`power | pwl`) is a waveform rather than a power number.
`-region`	Specifies the region for power target.
`-repeat`	Repeats dynamic switch pattern of all the clock cycles in the simulation period.
`-reset`	This parameter causes all previous `set_power` commands to be ignored. Note: You can selectively reset the instance/cell specifications. In addition, you can also selectively reset the cell and instance specifications of a list of objects through the `get_cells` and `get_lib_cells` commands.
`-scale_factor value`	
	Scales the power value by the specified scale factor.
`-signal_net netname`	
	Specifies the signal net name for power assignment. When this parameter is specified, the driver instances connected to the signal net will be assigned the user-defined power value.
`-slew value`	
	Specifies the slew value to all instances with nets more than or equal to the fanout limit (`-fanout_limit`).
`-static_mode { custom_label_name | mode_name }`	
	Specifies the mode name or the custom label name associated with a cell or instance for computing instance current from PGV with static multi-voltage multi-frequency (MvMf) modes. This current is used to compute the instance's internal power, and its switching and leakage power components would be set to zero. The use model of this parameter is given below: `set_power -insts -static_mode READ` If you specify custom label of a static MvMf mode, currents would be taken from it directly. Alternatively, if only mode name is specified for a cell/instance, the software snaps to the nearest custom label defined within this mode, based on the instance's voltage and frequency to get the currents.
`-sticky`	Applies the simulation based PWL waveform as is, to the specified cell/instance for dynamic power analysis. Use this option only if you have specified `-pwl_waveform`.
`-switching value`	
	Specifies the target switching power of the design, cell, or instance. To specify switching power for cells and instances, you can use the `-switching` parameter in conjunction with the `-cells` or `-insts` parameter.
`-trigger_time_adjustment time`	

	Specifies to adjust the trigger time of the current waveforms for the various functional modes (read/write/idle) of the custom macros. This parameter allows you to start the waveform before the trigger time (negative time support) to account for the setup time for the operation, as shown below:  The specified time will be applied to the computed trigger time for each mode in the trigger files generated by power analysis. For example, if the trigger time (from timing window/vector) is computed as 1ns, 2ns, and 3ns, and if you specify `set_power -trigger_time_adjustment -100ps`, the software will adjust the trigger time to .9ns, 1.9ns, and 2.9ns.

value	Specifies power in milliwatts during static power calculation or piecewise linear (`-pwl`) current waveform, during dynamic power calculation for cell or instance. The command accepts a unit specification (e.g. 10w).
	The format of the PWL waveform is:
	`{time1 current1 time2 current2 ...time_n current_n}`
	Time is in ns and current in mA.
	Example:
	`set_power -pwl_waveform -pg_net {} -cells BUFX2MTL {0 0 10 0 10.1 5 10.5 0}`
	Here, at `0ns` its value is `0mA`, at `10ns` its value is `0mA`, at `10.1ns` its value is `5mA`, and so on.

Examples

- The following command sets power for vdd rail to 10 milliwatts:
  ```
  set_power -pg_net vdd 10mw
  ```
- The following command sets a power value of .005 milliwatts on power net vdd for instance clkin_neg_L14_I1:
  ```
  set_power -pg_net vdd -insts Top/A/clkin_neg_L14_I1 .005
  ```
- The following command sets a power value of .005 milliwatts for any cell named INVX2 in the design:
  ```
  set_power -cells INVX2 .005mw
  ```
- The following command defines a piecewise linear waveform of {0 0 10 0 10.1 5 10.5 0} for the cell of type BUFX2MTL for all power rails.
  ```
  set_power -pwl_waveform -pg_net {} -cells BUFX2MTL {0 0 10 0 10.1 5 10.5 0}
  ```
- The following command specifies the leakage power for the cell DFFRX1.
  ```
  set_power -leakage -cells DFFRX1 10mw
  ```
- The following commands specify the target leakage power for power nets VDD1 and VDD2.
  ```
  set_power -leakage -pg_net VDD1 100mw
  set_power -leakage -pg_net VDD2 200mw
  ```
- The following command specifies the PWL description of current for the instance SEHD130_1024X32X1CM8. The -sticky option tells the software to use the specified PWL simulation based waveforms as is.
  ```
  set_power -pwl_waveform -insts SEHD130_1024X32X1CM8 -sticky {0 0 1 1 2 2 7 0}
  ```

- The following command scales the total power of the instance i_1 by a scale factor of 2:
  ```
  set_power -scale_factor 2 -insts i_1
  ```
- The following command sets the total power of the design to 20mw while keeping the clock network power the same:
  ```
  set_power 20mw -exclude_clocks {all}
  ```
- The following command assigns a switching power of 3mw to all instances of the cell AND1:
  ```
  set_power -cells AND1 -switching 3mw
  ```
- The following command assigns 2mw power to all cells that match the specified pattern:
  ```
  set_power -cells[get_lib_cells NAND*] 2mw
  ```
- The following command specifies that mode1 switches in the first cycle, mode2 switches in the third cycle and no switching in others:
  ```
  set_power -dynamic_switch_pattern { mode1 0 mode2 0 }
  ```
- The following command resets power of all cells that match the specified pattern:
  ```
  set_power -reset -cells[get_cells A/*]
  ```
- The following command does not schedule DDV mode `scan*`:
  ```
  set_power -base_cells { * } -exclude_ddv_modes { scan* }
  ```
- The following command does not schedule DDV mode `scan*` for instance "`A1`":
  ```
  set_power -insts { A1 } -exclude_ddv_modes { scan* }
  ```
- The following command only schedules DDV mode names with the `*read*` and `*write*` pattern:
  ```
  set_power -base_cells { * } -include_ddv_modes { *read*, *write* }
  ```
- The following commands specify the toggle time for the pin names Q7 and Q6:
  ```
  set_power -dynamic_switch_event { 1.1ns r 5.5ns f } -insts clk_b_i2 -pin Q7
  set_power -dynamic_switch_event { 2.2ns r 6.6ns f } -insts clk_b_i2 -pin Q6
  ```

Trigger File Format

For specifying a custom trigger file, you must use the `-custom_macro_pwl_waveform` parameter. The format of the trigger file is:

Trigger File Format	Description
#Cell names	
`MASTER CELL <cellname1>`	Name of the reference cell.
`COMPANION_CELLS <cellname1 cellname2 … >`	Name of the cells that will use scaled PWL from the master cell.
#Mode and conditional details for trigger	
`MODE_NAME <mode1>` `CONDITIONAL_INPUT <conditional input statement for cellname1>`	Mode name and conditional input statement for the cell. Conditional input of current for the given mode.
`CONDITIONAL_PIN <pin_name> (rise|fall|both)`	Conditional pin statement for the cell. Signal pin for which the tool should rise or fall.
`CONDITIONAL_STIMULUS_FILE <usim1.txt>`	Simulation vector detail in the given mode.
`USER_PWL_FILE <pwl1.txt>`	User-specified PWL file that contains information about the PWL waveforms (power/ground currents). For an example of a PWL file, see PWL Format Example.
`END_MODE`	End mode section

#End cell section	
END	End cell section

Example of a Custom Macro Power Trigger File

```
MASTER_CELL        SRAM_A1
COMPANION_CELLS    { SRAM_A2, SRAM_A3, SRAM_A4, SRAM_A5 }
      MODE_NAME WRITE
               CONDITIONAL_INPUT = ( !SD & !SLP & !CEB & !WEB )
               CONDITIONAL_PIN = CLK { RISE }
               USER_PWL_FILE write_rise.txt
       END_MODE
      MODE_NAME READ
               CONDITIONAL_INPUT = ( !SD & !SLP & !CEB & WEB )
               CONDITIONAL_PIN = CLK { RISE }
               USER_PWL_FILE read_rise.txt
       END_MODE
      ...
END
```

Example of “USER_PWL_FILE” (Multiple regions)

```
UNIT CURRENT mA
UNIT TIME ns
       NET VSS
               REGION { 0 0 0.1 -1 0.2 -2 0.3 -3 0.4 -10 0.5 -50 0.6 -40 0.7 -30 0.8 -20 1 0 2 0 3 0 3.9 0 4 -0.1 4.5 -0.2 5
-4 5.5 -3 6 -2 7 0 }
       NET VDD
               REGION { 0 0 0.1 1 0.2 2 0.3 3 0.4 10 0.5 50 0.6 40 0.7 30 0.8 20 1 0 2 0 3 0 3.9 0 4 0.1 4.5 0.2 5 4 5.5 3 6
2 7 0 }
       NET VDDM
               REGION { 0 0 0.1 1 0.2 2 0.3 3 0.4 10 0.5 50 0.6 40 0.7 30 0.8 20 1 0 2 0 3 0 3.9 0 4 0.1 4.5 0.2 5 4 5.5 3 6
2 7 0 }
```

set_power_analysis_temperature

```
set_power_analysis_temperature value
[-help]
```

Sets the temperature for static power calculation.

In the presence of a trilib set, the internal and leakage power will be interpolated as the per the given temperature specification.

Parameters

`-help`	Prints out the command usage.
value	Specifies the temperature for static power calculation. **Note:** The static power calculation software gets the temperature value of each instance from the following sources (in order of precedence): • Instance temperature file • `set_power_analysis_temperature` command • The current operating condition temperature

set_power_include_file

```
set_power_include_file
file
[-block string]
[-reset]
```

Specifies a power analysis include file. You can use the `set_power_include_file` command to specify certain commands of the dynamic engine which do not have a Voltus equivalent. For more information on these commands, refer to the Power Analysis Options chapter.

The `set_power_include_file` command is required only if the `power_static_netlist` attribute is set to `def`.

Note: The `set_power_include_file` command is used to pass the legacy power analysis commands and `report_power` command options only in the dynamic mode (vectorbased and vectorless). If the `power_method` attribute is set to `static`, the `set_power_include_file` command is ignored.

Parameters

`file`	Name of the include file. This file will include all commands that can not be translated to Voltus commands. In addition, you can also include the `report_power` command options to generate different power reports.
`-block string`	Specifies the master cell name to which the powermeter options are to be applied. The parameter allows you to apply certain commands of the dynamic engine only to a specific block partition. This parameter allows you to apply the include options to a specific block instead of applying them globally to all blocks. The `-block` parameter is applicable only to the block-based distributed power flow.
`-reset`	Resets all options back to default values.

Examples

- The following command specifies a power include file of `power.inc`:

  ```
  set_power_include_file power.inc
  ```

- The following example shows the contents of the `power.inc` file.
  # power.inc file must use the pre-defined design and power objects as "design" # and "ChipPwr" respectively. The power output object is not supported.

  ```
  design calCellsToIgnore BUFX1
  ChipPwr calRailVoltage VDD 1.2

  report_power -net -out_file Net.rep
  report_power -cell {INV*} -out_file Invpower.rep
  ```

set_power_output_dir

```
set_power_output_dir dir
[-reset]
```

Specifies the name of the directory that power information will be output to.

Parameters

dir	Specifies the name of the directory in which power analysis output information is written.
-reset	Resets all options back to default values.

Examples

- The following command specifies the output directory called `static_power_AvallOn` in which the power information is written.

  ```
  set_power_output_dir static_power_AVallOn
  ```

set_switching_activity

```
set_switching_activity
[-reset]
[-activity factor | -transition_density transition_density ]
[-bidir_port]
[-clock clock_name]
[-duty value]
[-inst instance_name]
[-net net_name | -port port_name | -pin pin_name | -input_port port_name |-output_port port_name ]
[-period value]
[-integrated_clock_gate_ratio num]
[-combinational_clock_gate_ratio num]
[-hierarchy hierarchy_name]
[-scale value]
[-cell cell_name]
[-force]
[-unclocked]
[-quiet]
```

Specifies activity for nets, pins, and ports. This command is used to specify user-defined activities at specific pins/nets/ports. You must specify this command before `report_power`.

The `set_switching_activity` command can accept a net/pin/port list using the `get_pins`/`get_ports`/`get_nets` commands. You can use the `get*` commands to create a collection of nets/pins/ports whose name matches the supplied pattern list. Therefore, providing you with the flexibility in providing lists through use of regular filter patterns. This feature is supported in both static and dynamic analysis engines.

Parameters

`-activity` *factor*	
	Specifies the activity for a net, pin, or top-level port. The *activity_factor* specifies the number of times the net, pin, or port switches in a clock cycle. Enter any positive number. If if the activity is set to 0.5, the activity happens once for every two clock cycles. If the activity is set to 2.0, the activity happens two times per clock cycle. There is no default value.
`-bidir_port`	
	Specifies the name of the bidirectional port.
`-cell` *cell_name*	
	Specifies the activity on all output pins for all instances of the specified cell.
`-clock` *clock_name*	
	Assigns global activity factor to all nets in domain of *clock_name* if and only if *clock_name* is the fastest clock in the domain set. To use this option, `-clock` and `-activity` must be used without any other options.
`-combinational_clock_gate_ratio` *num*	
	Specifies to set the propagation ratio for combinational clock gate cell outputs. When specified, this parameter sets the output activity of any instance that is identified as a combinational clock gating cell to *num* times the fastest transition density arriving on the instance's clock inputs. You can specify this ratio for a net, pin, port, or clock.
`-transition_density` *transition_density*	

	Specifies the transition density for a specific net, pin, or top-level port. Transition density specifies the number of transitions per second. it can be any positive number. There is no default value. **Note:** If you specify the `-transition_density` parameter in conjunction with the `-period` parameter, the software issues a warning and the `-period` parameter is ignored.
`-duty` *value*	
	Specifies the duty cycle, which is the probability that the signal is a logical 1. The `-period` parameter must be specified to add a duty. If not specified, the `set_switcing_activity` command picks this value from the external TWF or SDC through the Common Timing Engine (CTE). *Default*: Uses the duty specified by the `set_default_switching_activity` command if the `-duty` parameter is not specified.
`-force`	
	Specifies to apply user-defined activities for specific nets, pins, and ports, without having to load a verilog netlist. When a verilog netlist is not specified, the `set_switching_activity` command does not support the `-cell` parameter, and the net/pin/port list using the `get_pins/get_ports/get_nets` commands.
`-hierarchy` *hierarchy_name*	
	A hierarchical name that an activity is being assigned to.
`-integrated_clock_gate_ratio` *num*	
	Specifies to set the propagation ratio for integrated clock gate (ICG) cell outputs. When specified, this parameter sets the output pin of all identified ICG instances to *num* times the fastest transition density arriving on clock inputs of the instance. You can specify this ratio for a net, pin, port, or clock.
`-input_port` *port_name*	
	Specifies the name of the input port. The port names are in the following format: *input-port*. This parameter supports wildcards (`*`)
`-inst` *instance_name*	
	Specifies the activity on a specific instance.
`-net` *net_name*	
	Specifies the name of the net. This parameter supports wildcards (`*`).
`-output_port` *port_name*	
	Specifies the name of the output port. The port names are in the following format: *output-port*. This parameter supports wildcards (`*`)
`-period` *value*	
	Specifies the period that the activity is referenced to. If not specified, a default frequency is used. If a negative number is specified, the static power calculation engine reverts back to the default frequency. Units in seconds (`s`), milliseconds (`ms`), microseconds (`us`), nanoseconds (`ns`), or picoseconds (`ps`). **Note:** If you specify the `-period` parameter in conjunction with the `-transition_density` parameter, the software issues a warning and the `-period` parameter is ignored. *Default*: The time unit is defined in the first `.lib` file read during design import. If the time unit is not defined, the period specified in the `set_default_switching_activity` command is used.
`-pin` *pin_name*	

	Specifies the name of a pin. The pin names are in the following format: *instance-name + default-hierarchyarchy-separator + pin*. This parameter supports wildcards (*)
-port *port_name*	
	Specifies the name of a top-level port. The port names are in the following format: *top-level-port*. This parameter supports wildcards (*)
-unclocked	Applies activity (with reference period) to all nets that doesn't have clock domain.
-quiet	Suppresses all error and warning messages generated when the set_switching_activity command is run.
-reset	Resets all specified options back to default values. The reset option should be the first option specified, otherwise all options set prior to this will be reset. **Note:** You can selectively reset the pin/port/net specifications. In addition, you can also selectively reset the pin/port/net specifications of a list of objects through the get_pins/get_ports/get_nets commands.
-scale *value*	
	Specifies to scale frequencies of the specified clock domains. This parameter allows you to scale transition densities of all instances associated with the clock domain by the specified scale factor to compute internal and switching power. This parameter must be specified with the -clock parameter. The -scale parameter is only applicable to static power and dynamic vectorless power analysis.

Activity Precedence

See "Activity Precedence".

Examples

- The following command specifies that net n189 will switch once every ten clock cycles, with a clock period of 7520ns and a duty cycle of 50 percent:
  ```
  set_switching_activity -net n189 -activity 0.10 -period 7520ns -duty .5
  ```
- The following command sets the transition density of port out_cpu_top to 20000 transitions per second:
  ```
  set_switching_activity -port out_cpu_top -transition_density 20000
  ```
- The following command applies a transition density of 1e+06 to all input ports that match the specified pattern:
  ```
  set_switching_activity -port[get_ports "mode*" -filter {port_direction==in}] -transition_density 1e+06
  ```
- The following command resets the previously defined activity on all nets that match the specified pattern:
  ```
  set_switching_activity -reset -net[get_nets n23*]
  ```
- The following command specifies the clock gate ratio for the clock domain sys_clk:
  ```
  set_switching_activity -clock sys_clk -integrated_clock_gate_ratio 0.8 -combinational_clock_gate_ratio 0.5
  ```
- The following command specifies to set the default switching activity for all other clock domains of the design:
  ```
  set_default_switching_activity -integrated_clock_gate_ratio 1.0 -combinational_clock_gate_ratio 1.0
  ```

set_twf_attribute

```
set_twf_attribute
[-help]
[-clock clock_name]
[-clock_edge {rise | fall}]
[-clock_frequency value]
[-fall_delay {min:max or single_value}]
[-fall_slack {min:max or single_value}]
[-fall_slew {min:max or single_value}]
[-net net_name]
[-pin pin_name]
[-reset]
[-rise_delay {min:max or single_value}]
[-rise_slack {min:max or single_value}]
[-rise_slew {min:max or single_value}]
[-type {data | clock}]
[-shift_arrival_time time]
[-clear_twf_attribute]
```

Specifies static timing analysis information. The command allows you to override timing parameters, such as slew and clock, on desired pins/nets from an existing timing windows file (TWF).

Parameters

`-clear_twf_attribute`	
	Specifies to reset the TWF file values for a specific pin or net. The use model of this parameter is: `set_twf_attribute -clear_twf_attribute -pin/-net <name>`
`-clock clock_name`	
	Specifies the clock name. This indicates that the nets and pins that are timing constrained have arrival times (timing windows) caused by this clock.
`-clock_edge {rise | fall}`	
	Specifies the capturing clock edge direction: rise or fall.
`-clock_frequency value`	
	Specifies the clock frequency. You must specify the `-clock` parameter along with the `-clock_frequency` parameter to assign the clock for which the frequency needs to be updated.
`-fall_delay {min:max` or `single_value}`	
	Specifies the fall delay for the pin. This parameter is not supported in the static power analysis flow.
`-fall_slack {min:max or single_value}`	
	Specifies the fall slack for the pin.
`-fall_slew {min:max or single_value}`	

	Specifies the fall slew for the pin.
`-net net_name`	
	Specifies the net name. The `-net` and `-pin` parameters are mutually exclusive.
`-pin pin_name`	
	Specifies the pin name. The `-net` and `-pin` parameters are mutually exclusive.
`-reset`	Resets all specified options back to default values.
`-rise_delay {min:max` or `single_value}`	
	Specifies the rise delay for the pin. This parameter is not supported in the static power analysis flow.
`-rise_slack {min:max` or `single_value}`	
	Specifies the rise slack for the pin.
`-rise_slew {min:max` or `single_value}`	
	Specifies the rise slew for the pin.
`-shift_arrival_time time`	

	Specifies to filter the idle period of the current waveform in the dynamic power analysis flow. This parameter is useful in removing the idle period of the current waveform result with long idle period (small current) before the 1st high current peak because of TWF delay annotation. The parameter allows to offset idle period from the TWF delay values.
	The use model is given below:
	`set_twf_attribute -shift_arrival_time 3.0ns`
	
	The default value is 0ns. The supported time units are s, ms, us, ns, and ps. The default unit is ns. You must specify a positive value only. A positive value means shift right in the time line or add more delay to the original arrival time.
	
	You can specify the `-clock` parameter to limit the changes to a clock domain. In addition, you can further add the `-pin/-net` parameter to limit the changes to a block. The following are examples of shifting different delays to different clock domains because different clocks may have different clock insertion delays at the top level:
	
	`set_twf_attribute -shift_arrival_time 1.2ns -clock clk_a`
	`set_twf_attribute -shift_arrival_time 2.3ns -clock clk_b`
	
	If a clock domain is not specified, the delay will be applied to all the specified nets/pins.
	
	Note: The final rise/fall delays for the annotated nets/pins after the shift has been applied are reported under the "Shift Arrival Time Report" section of the detailed TWF Attributes report. This report is generated using the `power_report_twf_attributes detailed` attribute.
	
	The following example illustrates a sample command and report snippet:
	
	Command:
	
	`set_twf_attribute -shift_arrival_time 3.0ns`
	
	Report:
	
	`Clock domain annotated`
	`  - CK1_1`
	`  - CK1_2`
	`  - CK2_1`
	`  - CK2_2`
	
	`Rise Delay  Fall Delay  Net Name`
	`1e-09  1e-09  CK1_1`
	`1e-09  1e-09  CK1_2`
	`1e-09  1e-09  CK2_1`
	`...`

`-type {data | clock}`	
	Specifies whether the net or pin is part of the clock or data network.

Examples

- The following command specifies to use the following TWF attributes for the net `netB1`:

```
set_twf_attribute -net netB1 -rise_delay 2.1978:2.1978 -rise_slew 0.05:0.05 -rise_slack 0.2741:3.6961
-fall_delay 2.1819:2.1819 -fall_slew 0.05:0.05 -fall_slack 0.237:3.6948 -clock CK1_1 -clock_frequency 100 -type data
```

- The following commands reset the TWF values for the "`DFF11/DFF/Q`" pin to default, and then sets the rise/fall delay:

```
set_twf_attribute -clear_twf_attribute -pin "DFF11/DFF/Q"
set_twf_attribute -pin "DFF11/DFF/Q" -rise_delay 0.35 -fall_delay 0.45 -clock CLK1
```

`analyze_resistance -threshold 50` **-net_threshold**

set_virtual_clock_network_parameters

```
set_virtual_clock_network_parameters
[-help]
[-cell cell_name]
[-clock list_of_clocks]
[-library library_name]
[-max_fanout value]
[-wire_load_model wireload_model]
[-reset]
```

Specifies to estimate power for a virtual clock tree. When the command is specified, the software reports the power of the clock network based on the buffers inserted in the clock tree. It also reports the number of buffers used for implementing the clock tree and the depth of the clock tree.

You can use this command for fast power estimation of clock network power of a pre-CTS netlist.

This command should be invoked before running the `report_power` command.

Parameters

`-help`	Outputs a brief description that includes the type and default information for `each` set_virtual_clock_network_parameters parameter. For a detailed description of the command and all of its parameters, use the man command man `set_virtual_clock_network_parameters`.
`-cell cell_name`	
	Specifies the name of the buffer or inverter cell that has to be used to build the clock tree.
`-clock list_of_clocks`	
	Specifies the list of clocks for which the clock tree needs to be implemented. The default is for all the clocks.
`-library library_name`	
	Specifies the name of the library which contains the cell.
`-max_fanout value`	
	Specifies the maximum fanout that the buffer cell can drive.
`-wire_load_model wireload_model`	
	Specifies the wire load model used to compute the wire load model for the nets in the clock network.
`-reset`	Resets all specified parameters back to default values.

Example

The following command specifies to use the cell `BUFX12`, with a maximum fanout value of `2`, to build a virtual clock tree:

```
set_virtual_clock_network_parameters -cell { BUFX12 } -library typical -max_fanout 2 -wire_load_model B0X0
```

write_power_constraints

```
write_power_constraints
[-help]
[-out_file filename]
```

Writes out all the power constraints such as the power analysis mode, activity file specifications, simulation period specifications, and so on, into a file. You can use this command at any time during the session to dump out the power commands/ options specified till that point.

You can source this file into other Voltus/Tempus/Innovus session in order to use the same settings.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `write_power_constraints` parameter. For a detailed description of the command and all of its parameters, use the man command `man write_power_constraints`.
`-out_file` *`filename`*	Specifies the output file into which the power constraints are to be written.

Example

```
write_power_constraints -out_file test1.tcl
```

write_tcf

```
write_tcf file
[-block block_name]
[-exclude_hier_insts list_of_instances]
[-primary_input]
[-sequential_output]
[-pin]
[-macro_output]
```

Writes out the propagated switching activity information to a toggle count format (TCF) file. The TCF file contains switching activity information, toggle count information, and the probability of the net or pin being in the logic1 state.

You must specify this command before `report_power`.

Parameters

`-block block_name`	Specifies to generate the block-level TCF from the top-level design. When specified, it writes out the block-level duty and activity in the generated TCF file. The use model of this parameter is: `write_tcf -pin -block blockname TCF_filename`
`-exclude_hier_insts list_of_instances`	Excludes the specified hierarchical modules or instances when writing out a pin-based TCF. This parameter allows you to ignore the activity generated for the excluded instances. The use model of this parameter is: `write_tcf -pin -exclude_hier_insts {list_of_instances}`
`-macro_output`	Specifies to generate a TCF file that includes all the activities for the macro outputs in the design.
`-pin`	Specifies to write out pin-based TCF.
`-primary_input`	Generates a TCF file that includes only the primary inputs in the design.
`-sequential_output`	
	Generates a TCF file that includes all the activities for the sequential outputs in the design.
file	Specifies the name of the output TCF file.

Example

- The following command writes the propagated switching activity information to TCF file `mmm.tcf`:
  ```
  write_tcf mmm.tcf
  ```
- The following command writes a pin-based TCF for the whole design:
  ```
  write_tcf -pin pin_design.tcf
  ```
- The following command writes out a TCF which only includes activity information for the primary inputs and sequential outputs in the design:
  ```
  write_tcf -primary_input -sequential_output Power.tcf
  ```

analyze_ir_paths

```
analyze_ir_paths
[-help]
-clock_period <float>
[-hierarchical_activity_file <string>]
[-max_slack <float>]
[-pre_simulation_period <float>]
-seq_activity <float>
-simulation_period <float>
```

Generates a physical-aware multi-cycle switching activity file containing toggle information of voltage sensitive paths as well as timing critical paths in the design for Tempus Power Integrity analysis. This command generates 'swsrc.txt' file in the run directory, which then is fed back to the software using the `read_activity_file` command.

Parameters

`-help`	Prints out the command usage.
`-clock_period <float>`	Specifies the period of dominant clock of the design in library time units. Tempus PI refers to this specified period for multi-cycle activity generation.

`-hierarchical_activity_file <string>`	Provides a mechanism for different sequential activity for several partitions in the design. The following is the required format:
	`<hierarchical instance 1> <activity>`
	`<hierarchical instance 2> <activity>`
	…

`-max_slack <float>`	Specifies the maximum slack limit in library time units, above which the timing paths are not considered for activity generation.
`-pre_simulation_period <float>`	Specifies the pre-simulation duration that will be used by Tempus PI for activity generation. This parameter is useful when package model information is read-in using `set_package` command, as it allows the package oscillations to settle down.
`-seq_activity <float>`	Specifies the activity of sequential endpoints in the design. This value will be used to control the number of IR sensitive paths, along with their fanout cone, to toggle in each cycle.
`-simulation_period <float>`	Specifies the simulation period in library time units that Tempus PI dynamic power analysis will use. The larger the simulation period, the more will be the number of paths covered in the analysis. The following formula can be used to set the simulation period: *sim_cycles = (2/seq_act)*N* Where, • *seq_act* is the input sequential activity (e.g., 0.05, 0.1 etc.) • *N* is the number of nworst paths to be covered per endpoint • *sim_period* is equal to the sim_cycles*<time period of dominant clock> It is recommended to have N>=1, that is, at least one path per endpoint should be scheduled in the analysis.

Examples

- The following command generates the activity file for design with dominant clock of 2GHz, with input sequential activity being 0.1 and simulation duration being 20ns:

```
tempus > analyze_ir_paths -seq_activity 0.1 -simulation_period 20 -clock_period 0.5
```

- The following example specifies different sequential activity for different hierarchical partitions, with 0.2 sequential activity for unspecified partitions and top:

```
tempus > analyze_ir_paths -seq_activity 0.2 -simulation_period 30 -clock_period 0.5 -hierarchical_activity_file
hact.txt
Contents of 'hact.txt':
hierA/inst1 0.25
hierB/inst2 0.15
```

- The following command specifies the pre-simulation duration, particularly useful for package oscillations to settle when package models have been read-in:

```
tempus > analyze_ir_paths -seq_activity 0.1 -simulation_period 30 -clock_period 0.5 -pre_simulation_period 10
```

21

Power Route Category Attributes

- route_special Category Attributes

route_special Category Attributes

The root attributes in the `route_special` category are:

- route_special_allow_non_preferred_direction_route
- route_special_avoid_over_core_row_layer
- route_special_block_pin_connect_ring_pin_corners
- route_special_block_pin_route_with_pin_width
- route_special_connect_broken_core_pin
- route_special_core_pin_ignore_obs
- route_special_core_pin_merge_limit
- route_special_core_pin_length
- route_special_core_pin_length_as_inst
- route_special_core_pin_max_via_scale
- route_special_core_pin_refer_to_follow_pin
- route_special_core_pin_reference_macro
- route_special_core_pin_site_rail_width
- route_special_core_pin_snap_to
- route_special_core_pin_stop_route
- route_special_extend_nearest_target
- route_special_jog_threshold_ratio
- route_special_layer_preferred_direction_cost

- route_special_layer_non_preferred_direction_cost
- route_special_pad_pin_min_via_size
- route_special_pad_pin_split
- route_special_pad_pin_split_from_center
- route_special_pad_ring_use_lef
- route_special_secondary_pin_max_gap
- route_special_secondary_pin_rail_width
- route_special_signal_pin_as_pg
- route_special_split_long_via
- route_special_pg_pin_as_signal
- route_special_target_number
- route_special_target_search_distance
- route_special_time_limit
- route_special_endcap_as_core
- route_special_via_through_to_closest_ring
- route_special_via_connect_to_shape
- route_special_welltap_as_endcap

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `route_special` category, use the following command:

```
get_db -category route_special
```

route_special_allow_non_preferred_direction_route

`route_special_allow_non_preferred_direction_route {true | false}`

Default: `false`

Data_type: Boolean

Allows wrong way routing.

Note: If the metal1 preferred routing direction is vertical and standard cells have their VDD and VSS pins on layer metal1, sroute will route metal1 wires on horizontal direction to continue extending the followpin connections to core rings by this option

route_special_avoid_over_core_row_layer

```
route_special_avoid_over_core_row_layer layer_name
```

Default: `max layer of standard cell pin`

Data_type: String

Specifies the number of layers to avoid routing of block pin and pad pin over the core row area.

route_special_block_pin_connect_ring_pin_corners

```
route_special_block_pin_connect_ring_pin_corners {true | false}
```

Default: `false`

Data_type: Boolean

Specifies that the block wire used for a connection is to have the same width as the pin from which it connects.

route_special_block_pin_route_with_pin_width

```
route_special_block_pin_route_with_pin_width {true | false}
```

Default: `false`

Data_type: Boolean

Specifies that the block wire used for a connection is to have the same width as the pin from which it connects.

route_special_connect_broken_core_pin

```
route_special_connect_broken_core_pin {true | false}
```

Default: `false`

Data_type: Boolean

When set to `1`, `sroute` checks the signal pin, second power pin, and obs, and then breaks the followpin to avoid violations. When set to `0`, `sroute` does not check internal geometries and does not break the followpin.

route_special_core_pin_ignore_obs

```
route_special_core_pin_ignore_obs {none | placement_blockage | overlap_obs | block_halo
}
```

Default: `none`

Data_type: Enum

Ignores the specified blockage while connecting the follow pin.

route_special_core_pin_merge_limit

```
route_special_core_pin_merge_limit value
```

Default: `0.0`

Data_type: Double

Joins follow pin rail gaps smaller than this variable, bigger gaps will remain open. The option is of type *float*.

route_special_core_pin_length

```
route_special_core_pin_length value
```

Default: `0.0`

Data_type: Double

Connects core pins of specified length. The option is of type *float*.

route_special_core_pin_length_as_inst

```
route_special_core_pin_length_as_inst {true | false}
```

Default: `false`

Data_type: Boolean

Connects core pins whose length equals or is greater than the length of instance.

route_special_core_pin_max_via_scale

```
route_special_core_pin_max_via_scale widthPct heightPct
```

Default: ""

Data_type: String

Controls the length and width of vias at the crossover of core pin rails and stripes. Specify this value as percentages of the width of the via at the top of the stack. Smaller vias leave room for more routing tracks on layers between the vias.

route_special_core_pin_refer_to_follow_pin

```
route_special_core_pin_refer_to_follow_pin {true | false}
```

Default: `false`

Data_type: Boolean

If set to `true`, parallel metal followpins are generated based on the existing standard cell pins, when there is no other metal followpin on the standard cell pins.

route_special_core_pin_reference_macro

```
route_special_core_pin_reference_macro macro
```

Default: `""`

Data_type: String

Uses specified macros as references for followpin creation. The option is of type *string*.

route_special_core_pin_site_rail_width

```
route_special_core_pin_site_rail_width {{site1 layer11 Width11 layer12 Width12 …}
{site2 layer21 Width21 layer22 Width22 …} …}
```

Default: `""`

Data_type: String

Defines the followpin layer width for the specified site rail for followpin generation. The followpin width defined by this attribute has higher priority than the width of the same layer defined in the macro LEF, but lower than that defined by the `route_special -core_pin_width` parameter.

Following are the values used with this attribute:

- `site`: Specifies the site name defined in the LEF file.
- `layer`: Specifies the name of the layer defined in the LEF file.
- `width`: Specifies the followpin width on the top and bottom sides of the defined site.

route_special_core_pin_snap_to

```
route_special_core_pin_snap_to {M1_pin | opt_routing_track | grid | half_grid}
```

Default: `M1_pin`

Data_type: Enum

Snaps M2 followpins in the location to save routing tracks in a preventive manner. When set to:

- `M1_pin`: Generated metal2 followpins are snapped to metal1 stdcell pin center.
- `opt_routing_track`: Generated metal2 followpins are snapped to track or half track to save routing tracks.
- `grid`: Generated M2 followpins are snapped to the nearest routing track.
- `half_grid`: Generated M2 followpins are snapped to the nearest half routing track.

route_special_core_pin_stop_route

```
route_special_core_pin_stop_route {atRowEnd atCellEnd}
```

Default: `RowEnd`

Data_type: Enum

Extends the follow pin wire either to the row-end or cell-end.

route_special_extend_nearest_target

```
route_special_extend_nearest_target {true | false}
```

Default: `false`

Data_type: Boolean

Extends the nearest target so that it can be connected. The option is of type *bool*.

route_special_jog_threshold_ratio

```
route_special_jog_threshold_ratio value
```

Default: `10.0`

Data_type: Double

Defines the distance ratio between a jogging target and a straight target. The option is of type *float*.

route_special_layer_preferred_direction_cost

```
route_special_layer_preferred_direction_cost layer1 cost1 layer2 cost2...
```

Default: `""`

Data_type: String

Specifies layer cost of preferred direction. The option is of type `string`.

route_special_layer_non_preferred_direction_cost

```
route_special_layer_non_preferred_direction_cost layer1 cost1 layer2 cost2...
```

Default: `""`

Data_type: String

Specifies layer cost of non-preferred direction.

route_special_pad_pin_min_via_size

```
route_special_pad_pin_min_via_size viaSizePercent
```

Default: `20.0`

Data_type: Double

Permits the software to reduce the size of vias at pad pins when changing layers, if there is not enough room for a standard size via.

route_special_pad_pin_split

```
route_special_pad_pin_split width spacing
```

Default: `""`

Data_type: String

Splits the width and spacing for wide pad pins.

route_special_pad_pin_split_from_center

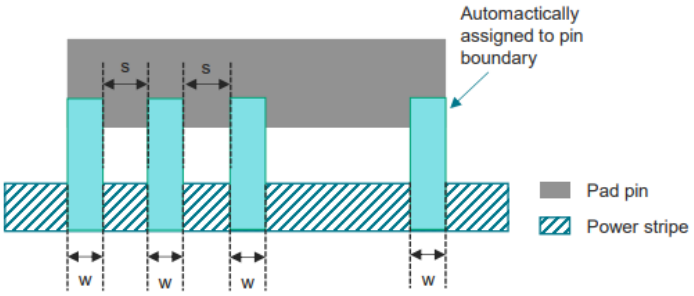
```
route_special_pad_pin_split_from_center {true | false}
```

Default: `false`

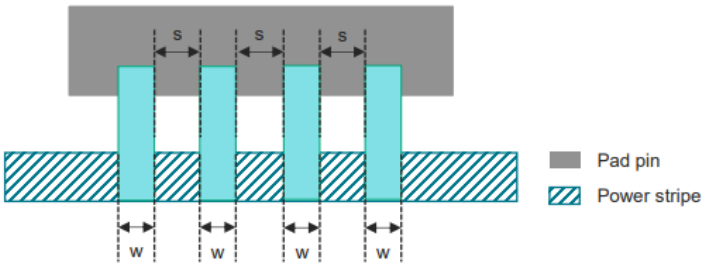
Data_type: Boolean

Splits the width and spacing specified by the `route_special_pad_pin_split` attribute to enable connections to the center of wide pad pins. This parameter must be used with the `route_special_pad_pin_split` attribute. See the examples below:

- ```
  set_db route_special_pad_pin_split {w s}
  set_db route_special_pad_pin_split_from_center false
  ```




- ```
  set_db route_special_pad_pin_split {w s}
  set_db route_special_pad_pin_split_from_center true
  ```



route_special_pad_ring_use_lef

```
route_special_pad_ring_use_lef {true | false}
```

Default: `true`

Data_type: Boolean

Splits the width and spacing for wide pad pins.

route_special_secondary_pin_max_gap

```
route_special_secondary_pin_max_gap value
```

Default: `0.0`

Data_type: Double

Specifies variable value for gaps bigger than this value cause level shifter rail to break. The option type is *float*.

route_special_secondary_pin_rail_width

```
route_special_secondary_pin_rail_width value
```

Default: `0.0`

Data_type: Double

Specifies width of followpin rail. The option type is *float*.

route_special_signal_pin_as_pg

```
route_special_signal_pin_as_pg {true | false}
```

Default: `false`

Data_type: Boolean

Specifies signal pins as PG pins in pad ring creation. The option type is *bool*.

route_special_split_long_via

`route_special_split_long_via {`*`threshold_value step_value offset_value height_value`*`}`

Default: `0 0 -1 -1`

Data_type: String

Splits vias longer than the specified length (`threshold_value`, in micrometers) into smaller vias with specified center-to-center spacing (`step_value`, in micrometers), left/bottom end offset (`offset_value`, in micrometers) and vertical/horizontal height (`height_value`, in micrometers) values. This is independent of the option `-split_via`.

Note: If the offset_value is given as negative, the vias will be placed symmetrically on both ends. The default value of `offset_value` is `-1`.

route_special_pg_pin_as_signal

`route_special_pg_pin_as_signal {`*`cellName: pinName`*`}`

Default: `""`

Data_type: String

Lists the cell/pin information NOT extracted from CPF.

route_special_target_number

`route_special_target_number` *`value`*

Default: `0`

Data_type: Integer

Specify number of target to connect. The option type is *int*.

route_special_target_search_distance

```
route_special_target_search_distance dist
```

Default: `0.0`

Data_type: Double

Specifies target search distance. The option type is *float*.

route_special_time_limit

```
route_special_time_limit value
```

Default: `0.0`

Data_type: Double

Specify time limit (in seconds) that the router is allowed in routing each port. The option type is *float*.

route_special_endcap_as_core

```
route_special_endcap_as_core {true | false}
```

Default: `false`

Data_type: Boolean

Specifies to treat Endcap macro as core macro. The option type is *bool*.

route_special_via_through_to_closest_ring

```
route_special_via_through_to_closest_ring {true | false}
```

Default: `false`

Data_type: Boolean

Specifies that a via should connect only to the closest ring layer when multiple rings overlap.

route_special_via_connect_to_shape

```
route_special_via_connect_to_shape {padring|ring|stripe|blockring|blockpin|coverpin|nos
hape|blockwire|corewire|followpin|iowire}
```

Default: padring ring stripe blockring blockpin coverpin noshape blockwire corewire followpin iowire

Data_type: Enum

Specifies which shapes vias can connect to.

route_special_welltap_as_endcap

`route_special_welltap_as_endcap` *`string`*

Default: ""

Data_type: String

When used, `route_special` treats the `Welltap` cells as the `Endcap` cells. Additionally, the `ENDCAP`class cells are ignored by default, when connecting the `followpin` wires.

22

RC Extraction Commands and Attributes

- extract_parasitics
- extract_rc Category Attributes
- read_parasitics
- report_net_parasitics
- report_rcdb
- report_unit_parasitics
- write_parasitics
- write_rcdb

extract_parasitics

```
extract_parasitics
[-help]
```

Extracts resistance and capacitance for the interconnects and stores the results in an RC database. Optionally, an ASCII report file can be generated. Use this command after performing trial routing or final routing, and after setting the extract_rc Category Attributes.

Parameter

`-help`	Outputs a brief description of the `extract_parasitics` command. For a detailed description of the command, use the man command: `man extract_parasitics`.

Example

The following command extracts interconnect resistance and capacitance:

```
extract_parasitics
```

RC database is created.

The `extract_parasitics` command can be followed by the `write_parasitics` command to extract the parasitics from the RC database, and to generate an ASCII report containing the database information. You can find all the extraction reports in the `extLogDir`.

Related Commands

- `read_parasitics`
- `write_parasitics`
- `extract_rc` Category Attributes

extract_rc Category Attributes

The root attributes in the `extract_rc` category are:

- extract_rc_cap_filter_mode
- extract_rc_cerebrus_license_only
- extract_rc_compress_rcdb
- extract_rc_coupled
- extract_rc_coupling_cap_threshold
- extract_rc_lef_tech_file_map
- extract_rc_local_cpu
- extract_rc_pvs_fill
- extract_rc_qrc_cmd_file
- extract_rc_qrc_cmd_type
- extract_rc_qrc_stream_map_file
- extract_rc_qrc_run_mode
- extract_rc_relative_cap_threshold
- extract_rc_shrink_factor
- extract_rc_signoff_stream_layer_map
- extract_rc_total_cap_threshold
- extract_rc_tsv_subckt_file
- extract_rc_turbo_reduce
- extract_rc_write_def_options_for_signoff_extract

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `extract_rc` category, use the following command:

```
get_db -category extract_rc
```

extract_rc_cap_filter_mode

```
extract_rc_cap_filter_mode { relative_only | relative_and_coupling |
relative_or_coupling }
```

Default: For standalone Quantus extraction, if the process node is 130nm or below, the default value is `relative_and_coupling`. If the process node is above 130nm, the default value is `relative_only`.

Data_type: Enum

Specifies the capacitance filtering mode. The filtering modes are conditions which define how to use the `extract_rc_coupled` and `extract_rc_coupling_cap_threshold` attribute values, or their combination. Based on the filtering mode specified, the software determines the nets in the design, which will have their coupling capacitance lumped to the ground.

Note: The behavior of the extract_rc_total_cap_threshold attribute is independent of the `extract_rc_cap_filter_mode` attribute setting. Nets that have a total capacitance value lower than the value specified with the `extract_rc_total_cap_threshold` attribute will have their coupling capacitances grounded regardless of the `extract_rc_cap_filter_mode` attribute setting.

You can use one of the following modes:

- `relative_only`: Grounds the coupling capacitance of nets based on the threshold value set by the `extract_rc_relative_cap_threshold` attribute.
- `relative_and_coupling`: Grounds the coupling capacitance of nets only when it is lower than the threshold value specified with the `extract_rc_relative_cap_threshold` attribute as well as the `extract_rc_coupling_cap_threshold` attribute. In this mode, you enforce maximum restriction on grounding the coupling capacitances.
- `relative_or_coupling`: Grounds the coupling capacitance of nets if it is lower than the threshold value specified with one of the `extract_rc_relative_cap_threshold` or `extract_rc_coupling_cap_threshold` attributes.

Related Command

- `extract_parasitics`

extract_rc_cerebrus_license_only

```
extract_rc_cerebrus_license_only {true | false}
```

Default: `false`

Data_type: Boolean

This attribute is applicable to the Cerebrus-Quantus flow.

When set to `true`, only the Cerebrus license will be considered and Innovus will not fall back to use Quantus (QTS) license when Cerebrus license(s) is/are not available. By default, the attribute is set to `false`, which means the regular QTS license will be considered when Cerebrus license(s) is/are not available.

extract_rc_compress_rcdb

```
extract_rc_compress_rcdb {true | false}
```

Default: `false`

Data_type: Boolean

Generates compressed RCDB data.

Note: This functionality reduces the disk usage but leads to marginal increase in run time.

Related Command

- `extract_parasitics`

extract_rc_coupled

```
extract_rc_coupled {true | false}
```

Default: `true`

Data_type: Boolean

Separately outputs grounded and coupling capacitance. The 0 value of this attribute is typically used for static timing analysis. Signal Integrity analysis requires the coupled mode.

extract_rc_coupling_cap_threshold

```
extract_rc_coupling_cap_threshold file_name
```

Default: 0.1fF

Data_type: Double

Specifies the threshold value that determines when the extractor lumps a net's coupling capacitance to ground. This attribute is applicable only when the `extract_rc_coupled` attribute is set to 1. The software decouples the coupling capacitance of nets when the total coupling capacitance between the pair of nets is lower than the threshold specified with this attribute. The actual decoupling also depends on the setting of the `extract_rc_cap_filter_mode` attribute.

Related Command

- `extract_parasitics`

extract_rc_lef_tech_file_map

```
extract_rc_lef_tech_file_map filename
```

Default: "" (empty string)

Data_type: String

Specifies the location of the optional layer map file for the Standalone Quantus QRC engine.

Note: If used, the layer map file must be in the Common Command Language (CCL) format. The following is an example of the layer map file format:

```
extraction_setup \
-technology_layer_map \
# <LEF/DEF Name> #<Tech File Name>
M1 METAL_1 \
VIA1 VIA_1 \
M2 METAL_2 \
```

Note: For cell-level Quantus layermap, the DEF/LEF layer names are case sensitive, but ICT layer names are case-insensitive.

Related Command

- `extract_parasitics`

extract_rc_local_cpu

```
extract_rc_local_cpu number_of_cpus
```

Default: As specified in the `set_multi_cpu_usage -local_cpu` setting.

Data_type: Integer

Specifies the number of CPUs to be used for IQuantus extraction. This attribute is only applicable for the IQuantus extraction engine. When specified, it overrides the `set_multi_cpu_usage -local_cpu` option for IQuantus.

Related Command

- `extract_parasitics`

extract_rc_pvs_fill

```
extract_rc_pvs_fill {true | false}
```

Default: `false`

Data_type: Boolean

When this attribute is set to true, the PVS fill data attached to the Innovus database is sent to Quantus so that the effects of the fill data are extracted. This attribute only affects signoff Quantus extraction. It has no effect on tQuantus or iQuantus extraction done inside Stylus. This atttribute requires that either the `extract_rc_qrc_stream_map_file` attribute or the `extract_rc_signoff_stream_layer_map` attribute is specified.

extract_rc_qrc_cmd_file

```
extract_rc_qrc_cmd_file filename
```

Default: "" (empty string)

Data_type: String

Specifies the user-created Quantus command file to be used for running Standalone Quantus QRC. This command file should be in the new CCL syntax.

Note: This attribute is only valid if `extract_rc_qrc_cmd_type partial` or `custom` is selected.

Related Command

- extract_parasitics

extract_rc_qrc_cmd_type

```
extract_rc_qrc_cmd_type {auto | partial | custom}
```

Default: auto

Data_type: Enum

Specifies whether to derive the settings for running Quantus from the software settings, or from user-created command file using the extract_rc_qrc_cmd_file attribute.

This attribute supports the following values:

- auto: Derives the Quantus settings automatically from the software parameter settings.
  Note: In this mode, the extract_rc_lef_tech_file_map attribute is required to be specified.
- partial: Appends user commands from the qrcCmdFile to the settings derived from the software parameter settings. You must provide the partial mode command file in CCL syntax. Use the partial command file only when you add extraction directives for those options that cannot be set in software. Following are some important restrictions:
  - Do not specify the following options in the partial mode command file:
    - Threshold setting
    - Coupled/decoupled mode
    - Quantus Techfile (s)
    - Operating Temperature
  - Do not specify "-directory_name" and "-temporary_directory_name" and "-file_name" in output_setup section of the command file
  - Do not specify output file using output_setup -file_name command in partial command file

    There is no checking for syntax or content of this file, and it will be appended as is to the automatically created Quantus command file.
- custom: Gets the complete Quantus settings from the user-created command file. Except for

input and output, no settings from software parameters are used. You must provide the complete Quantus command file in CCL syntax.
Following are some important restrictions:

- Do not specify an input DEF/OA DB in the custom mode command file
- Do not specify "`-directory_name`" and "`-temporary_directory_name`" in `output_setup` section of the command file.
- Specify output SPEF with the following convention for the name:`<design.spef>`.
  In MMMC, the output SPEF files will be named: `<design>_<corner>_<Temperature>.spef`. Here "design" refers to the top cell name.

  Note: Do not prefix or postfix the output file name.

- Specify the Quantus Techfile(s) to be used. In MMMC, the corner names should match the names used in the `create_rc_corner` command
- Provide Quantus Techfile layer mapping information in custom command file
- If GDS is provided for cell contents, provide a Quantus Techfile to GDS layer number matching

  Note: There is no checking for syntax or content of the command file prior to running the extraction. The command file will be used as is. If the output file is not specified correctly, the resulting parasitic file will not be read into the Tempus RC database.

Related Command

- `extract_parasitics`

extract_rc_qrc_stream_map_file

`extract_rc_qrc_stream_map_file` *`filename`*

Default: `""` (empty string)

Data_type: String

This attribute is specified only when `-pvs_fill` parameter is set to true. When the `extract_rc_pvs_fill` attribute is set to `true`, a Quantus stream/oasis map Tcl command is

required to map the PVS layer/dtype data to the corresponding Quantus layers.

If the `pvs_fill.data` is generated based on (GDS), the map file should be in the following format:

```
#layer gds_layer gds_dtype
<layer_name> <layer_number> <data_type>
…
```

If the `pvs_fill.data` is generated based on OASIS, the map file should be in the following format:

```
#layer oasis_layer oasis_dtype
<layer_name> <layer_number> <data_type>
…
```

Where,

- `layer_name` is the layername in the design
- `layer_number` is the layer ID in the GDS/OASIS file
- `data_type` is the datatype specified in the GDS/OASIS file

For example,

```
M1  31  0
M1  31  1
M2  32  0
M2  32  1
```

Related Command

- `extract_parasitics`

extract_rc_qrc_run_mode

```
extract_rc_qrc_run_mode {concurrent | sequential}
```

Default: `concurrent`

Data_type: Enum

Specifies the Quantus run mode in MMMC designs.

Note: For MMMC designs, in Quantus partial command type, do not specify the output SPEF file name and RC corner name in the command file to run Quantus in concurrent or sequential mode. For MMMC designs, in Quantus custom command type, do not specify the output SPEF file name and RC corner name together in the command file. Quantus sequential mode is not supported in custom command type.

This attribute supports the following values:

- `concurrent` : Runs Quantus for all active corners concurrently.
- `sequential` : Runs Quantus for all active corners sequentially.

Related Command

- `extract_parasitics`

extract_rc_relative_cap_threshold

```
extract_rc_relative_cap_threshold value
```

Default: `1.0`

Data_type: Double

Sets a ratio threshold value that determines when the extractor lumps a net's coupling capacitance to ground. This attribute is applicable only when the `extract_rc_coupled` attribute is set to 1. If the total coupling capacitance between a pair of nets is less than the percentage (specified with this attribute) of the total capacitance of the net with the smaller total capacitance in the pair, the coupling capacitance between these two nets will be considered for grounding. The actual decoupling also depends on the setting of the `extract_rc_cap_filter_mode` attribute.

The range of this value is 0 to 1.

Related Command

- `extract_parasitics`

extract_rc_shrink_factor

```
extract_rc_shrink_factor shrink_factor_value
```

Default: `1.0`

Data_type: Double

Sets the value by which the software shrinks the design before extraction. The shrink factor is used only for extraction calculations. It does not change the actual design. This attribute must only be used when restoring design.

Note: This attribute is not recommended for newer technologies where shrink factor is defined in the Quantus QRC techfiles or capacitance tables.

Related Command

- `extract_parasitics`

extract_rc_signoff_stream_layer_map

`extract_rc_signoff_stream_layer_map` *`file_name`*

Default: "" (empty string)

Data_type: String

This attribute is specified only when the `extract_rc_pvs_fill` attribute is set to `true`. When the `extract_rc_pvs_fill` attribute is set to `true`, a Quantus stream/oasis map Tcl command is required to map the PVS layer/dtype data to the corresponding Quantus layers.

If the `pvs_fill.data` is generated based on stream (GDS/OASIS), the map file should be in the following format:

```
#layer gds_layer gds_dtype use-type color-ID

<layer_name> <layer_number> <data_type> <use-type> <color-ID>
```

…

Where,

- `layer_name` is the LEF layername in the design
- `layer_number` is the layer ID in the GDS/OASIS file
- `data_type` is the datatype of the layer in the GDS/OASIS file
- `use_type` is the keyword for identifying the use of the data. The valid values are:
  - `gray`: library data in the GDS/OASIS file
  - `fill`: regular metal fill data in the GDS/OASIS file

  - `active`: active metal fill data in the GDS/OASIS file

- `color-ID` is the color ID. Use "`1`" or "`2`" if the layer is colored, or use hyphen "`-`" if the layer is not colored

For example,

```
M1   31   0 gray -
M1   31   1 fill -
M2   32   0 gray -
M2   32   1 fill -
```

Related Command

- `extract_parasitics`

extract_rc_total_cap_threshold

```
extract_rc_total_cap_threshold value_in_fF
```

Default: `0.0fF`

Data_type: Double

Specifies the threshold value (femtoFarads) that determines when the extractor lumps a net's coupling capacitance to ground. This attribute is applicable only when the `extract_rc_coupled` attribute is set to 1. The software grounds the coupling capacitances for the nets which have a total capacitance value less than the value specified with this attribute.

Related Command

- `extract_parasitics`

extract_rc_tsv_subckt_file

```
extract_rc_tsv_subckt_file filename
```

Default: "" (empty string)

Data_type: String

Specifies the name of the Through Silicon Via (TSV) subckt file to be used. The file contains the

.subckt model information for TSV. The file has a Spice format. A sample file is shown below.

```
.subckt TSV top bottom
R1 top n2 0.03075
R2 n2 bottom 0.03075
C1 n2 0 28f
.ends
```

Where,

`top` and `bottom` refer to the terminal nodes on the top and bottom layers of the TSV

`n2` is the internal node between the top and bottom layers

`R1 top n2 0.03075` means that a resistor with a resistance of `0.03075 Ohms` is added between the `top` and `n2` nodes

`R2 n2 bottom 0.03075` means that a resistor with a resistance of `0.03075 Ohms` is added between the `bottom` and `n2` nodes

`C1 n2 0 28f` means that a ground capacitance of `28 fF` is added on the `n2` node

Note: The IQuantus engine might not keep the subckt connectivity as it is. The optimization of RES network will remove node n2 (if there is no other connections) and keep a single resistor between the top and bottom nodes with an accumulated resistance. For example, in the above example, the accumulated resistance will be `0.0615 Ohms`. Capacitance will also be moved to a “higher layered node”. In the above example, this layer is `top`.

Related Command

- `extract_parasitics`

extract_rc_turbo_reduce

```
extract_rc_turbo_reduce {0 | 1 | auto}
```

Default: `auto`

Data_type: Enum

Controls the turbo reduction feature for Standalone Quantus QRC extraction. You can set the following options:

- `auto`: When set to `auto` , the software automatically turns `on` turbo reduction and proceeds when the required Quantus QRC license is available. However, if the required Quantus QRC

license is not installed in the license server, the software automatically turns off Turbo reduction and proceeds with a warning message. If the license is installed in the license server but temporarily not available, then two scenarios are possible. First, if the license wait mode is not turned on, then Quantus will look for the license right away and error out because the license is not available. Second, if the license wait mode is turned on, then Quantus will wait and check for the license periodically until timeout. If the license is available before the timeout, turbo reduction will be performed but if the license is not available before timeout, QRC will error out.

- 1: When set to `true`, the turbo reduction engine is `on`. However, if the required Quantus QRC license is not installed in the license server, then standalone Quantus will error out with a message and the run will stop. It is because setting the option to true means the user really wants to perform turbo reduction.
- `0`: The turbo reduction engine is switched `off`.

For details, see the table below.

Behavior of Turbo Reduction

	Availability of Quantus QRC AA License		
Option Selected – extract_rc_turbo_reduce {1 | 0 | auto}	Quantus QRC AA Installed and Available (IQRC gets the license right away even in the wait mode because sufficient licenses are available in the license server)	Quantus QRC AA Installed but not Available (If license wait mode is not turned on, then Quantus looks for the license . If the wait mode is turned on and if the license is temporarily not available, then IQRC waits and checks for the license periodically until timeout.)	Quantus QRC AA Not Installed on License Server
auto	perform turbo reduction	1. error out if wait mode is not on or if the specified wait time is exceeded 2. perform turbo reduction if wait mode is on and the license is available within the specified time.	print warning and do not perform turbo reduction
1	perform turbo reduction	error out	error out because the user explicitly wants to use turbo reduction
0	do not perform turbo reduction	do not perform turbo reduction	do not perform turbo reduction

extract_rc_write_def_options_for_signoff_extract

`extract_rc_write_def_options_for_signoff_extract` *`list_of_def_options`*

Default: `""` (empty string)

Data_type: String

Specifies the list of `write_def` options for signoff extraction.

For signoff extraction, `set_db extract_rc_engine post_route set_db extract_rc_effort_level signoff`, the software internally calls the `write_def` command. However, in some specific scenarios, you need to provide additional options for the `write_def` command. Use this attribute to provide the list of `write_def` options to be included.

When this attribute is set, `extract_parasitics` will include these options while calling the `write_def` command.

Related Command

- `extract_parasitics`

read_parasitics

```
read_parasitics
[-help]
[-force]
[-keep_star_node_locations]
{[{{[-rc_corner string]}}]}
[-via_variation_file viaVariationFile]
```

Reads SPEF and RCDB-based RC parasitics information and already created RCDBs in the software. For hierarchical designs, use this command to read top-level as well as block-level parasitic information.

Note: The following are important aspects to be considered for using this command:

- The RCDB parasitic file must have the `.rcdb.d` extension.
- The RCDB-based parasitic data that is provided as input to this command must be the one that was saved in the same major release of the software. The command will not read the RCDB data saved in a previous release. For example, if you provide RCDB saved in the 10.1.x release of the software as input to the `read_parasitic`s command in the 11 software release, then it cannot be read.
- Only the RCDB parasitic data on systems that are same as those used for saving the data can be read. For example, if the data is saved on Linux in 64-bit mode, you can only restore it on Linux 64-bit mode.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_parasitics` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_parasitics`.
`-force`	Allows reading of parasitic data even when block level RCDB/ SPEF is missing for a corner. **Note:** This parameter is only applicable for hierarchical designs. It is required that the top cell should have parasitic data specified for all the active RC corners.

`-keep_star_node_locations`	Specifies whether or not to read the *N statements in the SPEF file. The *N statements define the location of the RC nodes. When this parameter is specified, the *N statements are read and when it is not specified, the *N statements are ignored.
`-rc_corner` *list of RCcorner specific RCDBs/SPEfs for hierarchical design | RCcorner specific RCDB/SPEF*	

	Specifies the RC corner name along with a corner-specific RCDB or SPEF for both top and block levels. For hierarchical designs, this parameter lists the corner-specific SPEFs/RCDBs for top and block levels.
	To read multi-value SPEFs, specify the spef_list.
	The `spef_list` provides the list of SPEFs
	and `spef_field` provides the corresponding SPEF fields to read for the specified RC corner.
	In the example below, for RC corner `RC1`, `spef_field 2` from `<spef1>` and `spef_field 3` from `<spef2>` will be read.
	`read_parasitics \`
	`-rc_corner RC1 {spef_list {<spef1> <spef2>} spef_field {2 3}}`
	Similarly, you can specify this for multiple RC corners.
	`-rc_corner RC2 {spef_list {<spef3> <spef4>} spef_field {1 3}} \`
	`-rc_corner RC3 …. \`
	Note: A corner-specific RCDB parasitic file can only be provided as input to the RC corner for which it was generated. It cannot be provided as input to other RC corners.

`-via_variation_file` *viaVariationFile*	

	Specifies the name of the file for modeling via variation. The contents of the file include all the via layer names with the corresponding MEAN_SHIFT and STDDEV so that all the VIA resistance in SPEF can be mapped to their own variation multiplier. This option is used to specify the via_variation file only when the design has a single corner.
	For example,
	`read_parasitics`
	`-rc_corner rcCornerName \`
	`-via_variation_file` *`viaVariationFileName`*

Examples

- The example below is for hierarchical design with Top (T), Block 1(B1) and Block 2(B2), and two active RC corners, C1 and C2.
  - The following command specifies the RC corner names along with corner-specific RCDB and SPEF for both top and block levels:

    ```
    tempus> read_parasitics \
    -rc_corner C1 Top_C1.spef Block1_C1.spef Block2_C1.spef.gz \
    -rc_corner C2 Top_C2.rcdb.d Block1_C2.spef Block2_C2.rcdb.d
    ```

- The example below is for hierarchical design with Top (T), Block 1(B1) and Block 2(B2), and three active RC corners, C1, C2, and C3.
  - The following command specifies the multi-corner RCDB database to be read for both top and block levels and a single RCDB will be generated for the complete design:

    ```
    tempus> read_parasitics \
    -rc_corner C1 Top_C1.spef.gz \
    -rc_corner C2 Top_C2.compressed.rcdb.d \
    -rc_corner C3 Top_C3.rcdb.d
    ```

Related Commands

- extract_parasitics
- report_net_parasitics
- report_unit_parasitics
- write_parasitics

report_net_parasitics

```
report_net_parasitics
[-help]
net_name
-rc_corner rc_corner_name
[-out_file output_file_name]
```

Reports the RC parasitics of the specified net.

The `report_net_parasitics` command reports a list of attackers in the order of worst coupling capacitance by default. The software determines and reports the probable top N worst attackers for a victim. The command output will show worst coupling capacitance, in addition to RC parasitics.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_net_parasitics` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_net_parasitics`.
`-out_file` *output_file_name*	Specifies the name of the output file.
`-rc_corner` *rc_corner_name*	Specifies the RC corner name for which the RC parasitics are to be reported.
net_name	Specifies the name of the net for which the RC parasitics are to be reported.

Example

- The following command reports the RC parasitics for net, NET1 for RC corner, `rc_best`:

```
report_net_parasitics -rc_corner rc_best NET1

Net: NET1
RC Corner: rc_best
Total Cap = 0.473082 ff
Total Res = 2.104041 ohms
Total XCap = 0.334035 ff
Cap Distribution:

Total Ground Cap = 0.139046 ff
XCap With Nets:

NET3 = 0.059964 ff
NET4 = 0.040277 ff
NET5 = 0.233794 ff
```

- The following command reports the RC parasitics for the specified RC corner and net named `E_CN541_ds_dtin_10`:

```
report_net_parasitics  -rc_corner  corner_worst_RCMAX  E_CN541_ds_dtin_10
Net: E_CN541_ds_dtin_10
RC Corner: corner_worst_RCMAX
Total Cap  = 10.881560 ff
Total Res  = 294.239014 ohms
Total XCap = 4.972800 ff
Cap Distribution:
  Total Ground Cap = 5.908761 ff
  XCap With Nets:
    DATA_SAMPLE_MUX_INST/E_N27_n_39 = 2.341080 ff
    scan_in2 = 2.208500 ff
    E_IG_N67E_N231_digit_5_ = 0.423220 ff
```

Related Commands

- extract_parasitics
- report_unit_parasitics
- read_parasitics
- write_parasitics

report_rcdb

```
report_rcdb
[-help]
dir_name
```

Displays the contents of the RCDB being read.

Note:

1. This command is supported only for reading flat single RCDB. Multiple RCDB's cannot be read simultaneously using this command.
2. The RCDB being read must be generated on the same platform and architecture on which it is being read. For example, if you have generated the RCDB on Linux platform, then the `report_rcdb` command cannot report its content on the IBM AIX platform. Similarly, an RCDB generated on a 64-bit system cannot be reported on a 32-bit system.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_rcdb` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_rcdb`.
`dir_name`	Specifies the location of the RCDB directory.

Example

- The following command reports the contents of the RCDB being read in the `report.rcdb.d` directory:

```
write_rcdb report.rcdb.d

report_rcdb report.rcdb.d/

RCDB VERSION            : 11.15
OS-bit                  : 64
OS-endianness           : little-endian
RCDB Compression        : off
RCDB Source             : Innovus
DESIGN                  : dtmf_recvr_core
RC-Corners:             : corner_worst_RCMAX, corner_worst_RCMIN
Coupled                 : yes
Node-loc                : no
Statistical             : no
```

Related Commands

- extract_parasitics
- read_parasitics
- write_parasitics
- write_rcdb

report_unit_parasitics

```
report_unit_parasitics
[-help]
-layer metal_layer_name
-rc_corner rc_corner_name
[-space wire_space_in_um]
[-via_res via_resistance]
[-width wire_width_in_um]
```

Reports unit parasitics for individual metal layers and vias. The report prints the per unit resistance and capacitance for the specified layer. The resistance values are reported in ohms per micron for min width and min spacing of the layer. The capacitance values are reported in femtofarad (fF) per micron for min width and min spacing of the layer.

Via parasitics are printed considering the layer mentioned in the command as the lower layer of the via. The default via specified in the LEF file will be considered in this case.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_unit_parasitics` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_unit_parasitics`.
`-layer` *metal_layer_name*	Specifies the name of the metal layer for which the unit parasitics are to be reported. This parameter is required.
`-rc_corner` *rc_corner_name*	Specifies the name of the RC corner for which the unit parasitics are to be reported. This parameter is required.
`-space` *wire_space_in_um*	Specifies the wiring space in microns. *Default*: minimum spacing
`-via_res` *via_resistance*	When this parameter is specified, the resistances of all vias, whose upper layer is specified using the `-layer` parameter, are reported.
`-width` *wire_width_in_um*	Specifies the wiring width in microns. *Default*: minimum width

Examples

- The following command reports the unit parasitics for metal layer, `Metal5`, for the specified RC corner, `rc_worst`:

  ```
  report_unit_parasitics -layer Metal5 -rc_corner rc_worst
  ```

  The software gives the following information:

  ```
  Unit parasitics for layer Metal5 (in fF,ohm)
      RC-Corner = rc_worst
        Cap = 0.213204
        Res = 0.900873
  Parasitics for default Via "VIA5_0_VH"
        Via Cap = 0.017901
        Via Res = 8.859576
  ```

- The following commands reports the unit parasitics for `Metal5` with spacing two times the minimum spacing:

  ```
  report_unit_parasitics -layer Metal5 -rc_corner rc_worst -space [expr 2 * [dbGet
  [dbGet head.layers.extName Metal5 -p].minSpacing]]
  ```

  The software gives the following information:

  ```
  Unit parasitics for layer Metal5 (in fF,ohm)

      RC-Corner = rc_worst

          Cap = 0.188657
          Res = 0.900873

  Parasitics for default Via "VIA5_0_VH"

          Via Cap = 0.017901
          Via Res = 8.859576
  ```

- The following commands reports the unit parasitics for Metal5 with width two times the minimum width:

  ```
  report_unit_parasitics -layer Metal5 -rc_corner rc_worst -width [expr 2 * [dbGet
  [dbGet head.layers.extName Metal5 -p].minWidth]]
  ```

  The software gives the following information:

  ```
  Unit parasitics for layer Metal5 (in fF,ohm)

      RC-Corner = rc_worst
      Cap = 0.198657
      Res = 0.555333
  ```

```
Parasitics for default Via "VIA5_0_VH"

    Via Cap = 0.017901
    Via Res = 8.859576
```

- The following commands reports the unit parasitics for vias with top layer of Metal5:

  ```
  report_unit_parasitics -layer Metal5 -rc_corner rc_worst -via_res
  ```

  The software gives the following information:

```
Unit parasitics for layer Metal5 (in fF,ohm)

    RC-Corner = rc_worst
    Cap = 0.213204
    Res = 0.900873

Parasitics for default Via "VIA5_0_VH"

    Via Cap = 0.017901
    Via Res = 8.859576
    Other Via res
    VIA45_1C 8.859576
    VIA45_1C_H 8.859576
    VIA45_1C_V 8.859576
    VIA45_1ST_N 8.859576
    VIA45_1ST_S 8.859576
    VIA45_2C_E 4.429788
    VIA45_2C_W 4.429788
    VIA45_2C_N 4.429788
    VIA45_2C_S 4.429788
    VIA45_2ST_N 4.429788
    VIA45_2ST_S 4.429788
    VIA45_4C 2.214894
```

Related Commands

- extract_parasitics
- report_net_parasitics
- read_parasitics
- write_parasitics

write_parasitics

```
write_parasitics
[-help]
[-exclude_ilm]
[-no_resistances]
[-use_eeq_cell_with_liberty_info]
{{{-spef_file file_name} [-exclude_nets_file file_name | -include_nets_file file_name |
-net_name list_of_nets]} } [-view view_name | -rc_corner rc_corner_name] [-cap_unit {pf
| ff}]
```

Reads the parasitic database and outputs the parasitics in the Standard Parasitic Exchange Format (SPEF).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_parasitics` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_parasitics`.
`-cap_unit {pf | ff}`	Specifies the unit of capacitance as `fF` or `pF` in the output SPEF file. *Default:* `pf`
`-exclude_ilm`	Writes out only the top-level SPEF while excluding the ILM parasitics. For example, `write_parasitics -exclude_ilm -spef abc.spef -rc_corner corner1`
`-exclude_nets_file` *file_name*	Specifies the name of the file that contains nets to be excluded from the output file.
`-include_nets_file` *file_name*	Specifies the name of the input file that contains the net names to be extracted. *Default:* Extracts all net names

`-net_name` *`list_of_nets`*	Specifies the names of the nets to be written in the SPEF file. *Default:* Writes all nets
`-no_resistances`	Generates a SPEF file without the resistance information. The SPEF file generated by this parameter can only be used to improve the performance of third-party simulator tools in early stages of the design. **Caution**: Most tools expect the resistance information to be present in the SPEF file for performing extraction. Therefore, SPEF files with no resistance information should not be used with ostrich for RC correlation.
`-rc_corner` *`rc_corner_name`*	Specifies the RC corner that is used to generate the parasitic output file in the multi-corner flow.
`-spef_file` *`file_name`*	Specifies the name of the SPEF output file.
`-use_eeq_cell_with_liberty_info`	Writes EEQ master cell in case the EEQ cells are present.
`-view` *`view_name`*	Specifies that the RC corner associated with the specified view is used to generate the output SPEF file in the multi-corner flow.

Examples

- Reads the parasitic database created during RC extraction and writes the interconnect capacitance in the SPEF format to the file `TOPCHIP_SP.spef` for RC corner, `corner1`:

  ```
  write_parasitics -rc_corner corner1 -spef_file TOPCHIP_SP.spef
  ```

- Reads the parasitic database created during RC extraction for the view, `view1`:

  ```
  write_parasitics -view view1
  ```

Related Commands

- extract_parasitics
- report_net_parasitics
- report_unit_parasitics
- read_parasitics

write_rcdb

```
write_rcdb
[-help]
out_dir
```

Saves the parasitic data that you specify in a Tempus session. Use the `read_rcdb` command to restore the parasitic data.

Parameters

`-help`	Outputs a brief description that includes type and default information for the `write_rcdb` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_rcdb`.
`out_dir`	Specifies the location of the RCDB directory. The directory name should follow the `*.rcdb.d` format.

Example

- The following example writes the parasitic data to a directory called `myparasitics.rcdb`:
  `write_rcdb myparasitics.rcdb.d`

Related Commands

- `extract_parasitics`
- `read_parasitics`
- `write_parasitics`
- `report_rcdb`

23

Signal Integrity Commands and Attributes

- analyze_jitter
- check_noise
- define_bundled_bus
- define_glitch_holding_resistance
- define_parameter
- get_glitch_threshold
- get_multi_input_switching_mode
- get_via_variation_mode
- merge_noise_reports
- report_bundled_bus
- report_double_clocking
- report_glitch_thresholds
- report_instance_cdb
- report_jitter
- report_mutex_condition
- report_noise
- report_noise_lib_pin
- report_noise_propagation
- report_parameter
- report_voltage_scaling
- report_voltage_swing
- reset_glitch_derate
- reset_noise_lib_pin
- set_advanced_variation_mode
- set_annotated_glitch
- set_cdb_binding

- set_glitch_derate
- set_glitch_threshold
- set_model_priority
- set_mutex_condition
- set_noise_lib_pin
- set_outbound_report
- set_quiet_aggressor
- set_via_variation_mode
- si Category Attributes
- update_glitch
- write_noise_eco_nets
- write_spice_deck

analyze_jitter

```
analyze_jitter
[-help]
[-eiv_cycle_interval <integer>]
[-eiv_end_time float]
[-eiv_file string]
[-eiv_num_cycle <integer>]
[-eiv_start_time float]
[-eiv_power_ground string]
[-eiv_start_cycle <integer>]
[-enable_spectre_x {true | false}]
[-force_delete_output_dir]
[-from string]
[-from_fall string]
[-from_rise string]
[-level {true | false}]
[-levels_combine integer]
[-lsf_job_string string]
[-mode {period | cycle_to_cycle | time_interval_error}]
[-model_file string]
[-level {true | false}]
[-out_dir string]
[-run_simulation]
[-skip_stage_fanout string]
[-spectre string]
[-spectre_cmd_options string]
[-subckt_file string]
[-through string]
[-to string]
[[ > | >> ]]
```

Allows the software to perform Spice jitter analysis on a clock network using user-specified EIV database. The EIV database stores the effective instance voltages (EIV) per cycle for different instances in the design.

The software reads in the EIV files and identifies the worst two consecutive cycles for each instance where the voltage variation is maximum. The voltage levels from these identified cycles are used as the corresponding supplies for different instances in the generated Spice deck - which is simulated using the Spectre simulator to compute the cycle-to-cycle IR induced jitter.

Parameters

`-help`	Prints out the command usage.
`-eiv_cycle_interval <integer>`	Allows the software to run the `analyze_jitter` command natively for EIV cycles with the specified cycle interval.

`-eiv_end_time` *float*	Specifies the end time for EIV reading. The voltages specified in the EIV database for time-stamps beyond this value are ignored. *Default*: The last time-stamp from EIV file is considered as the end time.
`-eiv_file` *string*	Specifies the effective instance voltage files for accounting IR Drop in jitter computation.
`-eiv_num_cycle <integer>`	Runs the `analyze_jitter` command natively for the specified number of EIV cycles.
`-eiv_power_ground` *string*	Specifies the power and ground net names for each EIV file specified using the `-eiv_file` parameter. For example, "vdd1 vss1 vdd2 vss2", vdd1 and vss1 are for first and vdd2 and vss2 for the second EIV file defined using the `-eiv_file` parameter.
`-eiv_start_cycle <integer>`	Runs the `analyze_jitter` command natively from the starting EIV cycle.
`-eiv_start_time` *float*	Specifies the start time for EIV reading. The voltages specified in the EIV database for time-stamps before this value are ignored. *Default*: The first time-stamp from the EIV file is considered as the start time.
`-enable_spectre_x`	Enables spectreX license checkout when launching Spectre.
`-force_delete_output_dir`	Completely removes the directory specified by the user using the `-out_dir` parameter.
`-from` *string*	Creates a Spice deck of the clock network starting from the specified pin.
`-from_fall` *string*	Creates a Spice deck of the clock network starting from the falling transition of the specified pin.
`-from_rise` *string*	Creates a Spice deck of the clock network starting from the rising transition of the specified pin.
`-level {true | false}`	When set to `true`, the software traverses down all the way till it hits the clock pin of a flipflop, or any stop point specified with the `-to` parameter, or when it passes depth specified with the `-depth` parameter. *Default*: `false`

`-levels_combine integer`	Specifies the number of levels, going up from the leaf nodes, at which the sub-trees can be combined. By default, the whole clock network is simulated as a single Spice netlist for jitter analysis. But for large networks, running single simulation can result in significant increase in the run time. When you use this parameter, the software breaks down the clock network into multiple sub-trees, and runs Spice simulations in parallel. For the following values, the behavior is as follows: • 0: Divides the network such that the path traversed from a clock source to each leaf node is part of a separate sub-tree. • 1: Each sub-tree contains the path traversed from a clock source to one-level up of a leaf node. And all the leaf nodes in the fanout of the identified level are part of this sub-tree, and so on. **Note**: The `-max_spectre_lic` parameter must have a value greater than 1 to achieve this parallelization.
`-lsf_job_string` *string*	Specifies the handle to the farm job for launching simulation.
`-max_spectre_lic` *integer*	Specifies the number available Spectre licenses that can be utilized for simulation parallelization. *Default*: 1 license is assumed to be available, thereby disabling parallelization.
`-mode {period | cycle_to_cycle | } time_interval_error}`	Runs the `analyze_jitter` command natively for the specified mode type. *Default*: `period`
`-model_file` *string*	Specifies the path of Spice model files that contain the Spice model information.
`-level {true | false}`	When set to `false`, the Spice deck is generated only for the traced paths starting from the `-from` pin and ending at the `-to` pin. It does not traverse through the fanout branches that do not lead to the `-to` specified pin. When set to `true`, traversal occurs through the whole fanout cone of the `-from` pin till it hits a clock pin of a flop, or any stop point specified with the `-to` parameter *Default*: `false`
`-out_dir` *string*	Specifies the output directory for the created Spice decks and the simulation output files. *Default*: `tempus_pathsim`

`-run_simulation`	Enables jitter simulation, in addition to generating the Spice deck.
`-skip_stage_fanout` *string*	Accepts two positive integers in the format "N1 N2", and scans up to N1 stages in the fanin cone of each endpoint. The software skips the segments where a stage is found to have a fanout greater than N2. For example, a jitter path Clk_root1 -> … -> B3 -> B2 -> B1 -> Reg2, B2 and B1 are the last 2 levels. 1） Assume B1's fanout is 100 and B2's fanout is 80, then • When N1 = 1, N2 = 80, B1 will be skipped. • When N1 = 2, N2 = 100, B1 will be skipped • When N1 = 2, N2 = 80, B2 & B1 will be skipped **Note**: B2 will always cause B1 to be skipped even when B1's fanout is less than 80. So the reduction continues to increase when the level changes from 1 to 2. 2） Assume B1's fanout is 80, and B2 fanout is 100, then, • When N1 = 2, N2 = 90, both B2 and B1 will be skipped.
`-spectre` *string*	Specifies the location of the Spectre path.
`-spectre_cmd_options` *string*	Specifies the command line options for Spectre.
`-subckt_file` *string*	Specifies the file(s) containing subckt definitions of all the instances for which the Spice simulation needs to be performed.
`-through` *string*	Creates the Spice deck of a clock network that starts from the `-to` pin and traverses through the specified pin(s).
`-to` *string*	Creates the Spice deck of a clock network terminating at the specified pin(s).
`>` *filename*	Redirects the `-run_simulation` output to a file.
`>>` *filename*	Appends the `-run_simulation` output to a file.

Examples

- The following command generates the Spice deck for jitter analysis of the entire clock network starting from the `CLK` pin in the `JITTER_SPICE` directory and runs the simulations:

  ```
  tempus> analyze_jitter -from CLK -level true -out_dir JITTER_SPICE -eiv_file ./eiv.txt -
  eiv_power_ground {vdd vss} -run_simulation
  ```

  When the Spice simulations are completed, a summary of the jitter analysis will be saved in the output directory that contains the jitter values of all the endpoints, as shown below:

  ```
  ---------------------------------------------------------------------------------------------
  -------
  Start-point-> [through-point-> through-point] -> End-point Rise/Fall Cycle-to-Cycle Jitter
  Value (ps)
  ---------------------------------------------------------------------------------------------
  -------
  CLK -> DFF_2/DFF/D (F) 10.782
  CLK -> DFF_2/DFF/D (R) 11.072
  CLK -> DFF_2/DFF/CP (R) 9.054
  CLK -> DFF_2/DFF/CP (F) 9.438
  ```

- The following command runs jitter analysis for the clock network that originates from the `CLK` node, and traverses through an intermediate node INV/ZN:

  ```
  tempus> analyze_jitter -from CLK -though INV/ZN -level true -out_dir JITTER_SPICE -eiv_file
  ./eiv.txt -eiv_power_ground {vdd vss} -run_simulation -levels_combine 1 -max_spectre_lic 8
  ```

  The specified clock network is not simulated as a single netlist, but is broken down into multiple sub-trees. All the leaf nodes driven by a single driver are part of one sub-tree. The simulations on these smaller netlists are scheduled in such a way that a maximum of 8 simulations are run in parallel at any given time.

Related Commands

- `report_jitter`

check_noise

```
check_noise
[-help]
[-all]
[-check_only {receiver_no_propagation_model | receiver_no_dc_tolerence |
driver_no_current_source_model | driver_no_model}]
[-instance_list string]
[-instance_only {true | false}]
[-model_type {receiver | driver}]
[-pins string]
[-unique {true | false}]
[-verbose]
[-view string]
[ [ > | >> ] filename ]
```

Performs several consistency and completeness checks on noise models specified for a design. These checks help to ensure that the right set of models are available for the SI analysis engine to perform cross-talk noise analysis.

The settings from both the `set_glitch_threshold` and `set_annotated_glitch` commands are taken into account when the `check_noise` command is issued.

Parameters

`-help`	Prints out the command usage.
`-all`	Displays all the debug information.
`-check_only {receiver_no_propagation_model | receiver_no_dc_tolerence | driver_no_current_source_model | driver_no_model}`	
	Checks the warning name. The following warnings are checked: • `receiver_no_propagation_model`: Shows how many design pins are missing the receiver model for propagation. • `receiver_no_dc_tolerence`: Shows how many pins have no DC VH/VL tolerance specified. • `driver_no_current_source_model`: Shows how many design pins are missing the current source model. • `driver_no_model`: Specifies how many design pins are missing the driver model.
`-instance_list` *string*	Specifies the names of the instances.
`-instance_only {true | false}`	Prints each instance only once.
`-model_type {receiver | driver}`	Checks the presence of a valid model type - `receiver` or `driver` model. If the specified model is missing, a warning will be issued. The valid receiver models are the models having vivo tables, such as, CDB, CCSN, ECSMN. The valid driver models are CDB, CCS, ECSM, NLDM, and UDN.
`-pins` *string*	Specifies the instance pin names.
`-unique {true | false}`	Reports unique cells or pins for each power domain.
`-verbose`	Generates verbose details.
`-view` *string*	Specifies the view name.
filename	Redirects output to a file.
`>>`*filename*	Appends to an output file.

Examples

Here are some examples.

- The following command generates the output as shown below:

```
tempus> check_noise -type receiver_model

**WARN: (IMPESI-3125): AAE_WARN: The analysis view specified to check_noise is not found or
specified. Checking view default_emulate_view_setup.
------------------------------------------------------------------
CHECK NOISE SUMMARY
------------------------------------------------------------------
Warning Number of Warning
------------------------------------------------------------------
receiver_no_propagation_model 17931
receiver_no_dc_tolerence 17931

------------------------------------------------------------------
# CHECK NOISE WARNING DESCRIPTIONS:

# receiver_no_propagation_model: The receiver model for glitch
# propagation or ROP is not available, only RIP analysis will be performed.

# receiver_no_dc_tolerence: The input pin do not have vh/vl dc
# tolerence specified in the noise file. Default tolerences will be used.

------------------------------------------------------------------
```

Related Commands

- check_timing
- set_glitch_threshold
- set_annotated_glitch

define_bundled_bus

```
define_bundled_bus
[-help]
[-add_members string]
[-delete_members string]
[-name string]
```

Allows you to control user-defined bus groups.

The Tempus software automatically identifies bus nets. However, in some cases the software may not be able to mark bus bits properly (or treat marked bits as bus). For improved bus identification, the `define_bundled_bus` command allows you to control the nets that you want to add (or remove) from the list of recognized bus bits.

Parameters

`-help`	Prints out the command usage.
`-add_members string`	Specifies the name of nets to be added to the group.
`-delete_members string`	Specifies the name of nets to be deleted from the group.
`-name string`	Specifies the name of the group.

Examples

- The following example command will create a bus named bus1 with 3 net members x[1] and x[2]:

```
define_bundled_bus -name bus1 -add_members {x[1] x[2]}
report_bundled_bus -type user_defined
################################################################

Summary:
Total number of user defined bus groups reported = 1
Total number of tool inferred bus groups reported = 0

----------------------------------------------------------------

Group Name (user_defined) : bus1

Constituents:
- x[1]
- x[2]
```

- The following example command will add one more member x[3] and delete member x[2] to the bus 'bus1':

```
define_bundled_bus -name bus1 -add_members x[3] -delete_members x[2]

Added [1] net(s) to group 'bus1'.

Deleted [1] net(s) from user-defined group 'bus1'.
```

Related Commands

- report_bundled_bus

define_glitch_holding_resistance

```
define_glitch_holding_resistance
[-help]
[-glitch_type {both | vl | vh | vlu | vho | vl_vh | vlu_vho}]
[-instance_pin string]
[-lib_pin string]
-value double
[-views string]
```

Allows you to specify the holding resistance.

You can use this command to define a net-specific holding resistance instead of using the default holding resistance of 1000 ohms.

This command can be specified more than once per net. If there is a duplicate specification, then the last setting is used.

Parameters

`-help`	Prints out the command usage.
`-glitch_type {both | vl | vh | vlu | vho | vl_vh | vlu_vho}`	Specifies the glitch type for resistance Hold application. The option `both` applies the value to VL and VH.
`-instance_pin` *string*	Specifies a list of instance pin names.
`-lib_pin` *string*	Specifies the instance/pin name of the driver that the holding resistance should be applied to.
`-value` *double*	Specifies the value in ohms of the holding resistance to be used by glitch analysis.
`-views` *string*	Specifies a list of view names. If this parameter is not specified, then by default all the views are applied.

Examples

- The following command sets the holding resistance on the Y pin of inst1 to 233ohms for VL:
  ```
  tempus > define_glitch_holding_resitance -glitch_type vl -pins {inst1/Y} 233
  ```
- The following command sets the holding resistance on the Y pin of inst2 to 380ohms for both VL & VH:
  ```
  tempus > define_glitch_holding_resitance -glitch_type both -pins {inst1/Y} 233
  ```

define_parameter

```
define_parameter
[-help]
[<setup/hold>]
[-correlation <string>]
[-distribution <string>]
[-moments <string>]
[-name <string>]
[-range <string>]
[-related <string>]
-scope <string>
[-view <string>]
```

Provides variation information to Tempus. You can directly specify the range information and Tempus will internally calculate moments and use it for wire variation calculation.

Parameters

`-help`	Prints out the command usage.
`<setup/hold>`	Defines parameter that is valid for setup or hold.
`-correlation <string>`	Specifies correlation between different parameters of the same scope - parameters need to be pre-defined.
`-distribution <string>`	Specifies the parameter distribution. You can also use user-defined distribution.
`-moments <string>`	Specifies the parameter moments - mean or std-dev.
`-name <string>`	Specifies the name of the parameter.
`-range <string>`	Specifies the parameter minimum or maximum range - min or max.
`-related <string>`	Specifies the related information on sub-scope of parameter within the specified `-scope`. For back end-of-line (BEOL), the allowed arguments are: • <layer name>: capacitance • <layer name>:resistance
`-scope <string>`	Specifies the parameter scope - vtskew, wire_variation, or via_variation.
`-view <string>`	Specifies the view name.

Examples

- The following command defines R/C range for normal distribution for M1 layer for setup for view view1:

```
define_parameter setup -view view1 -name param_M1_R -scope wire_variation -distribution gaussian -range {1 1.15}
define_parameter setup -view view1 -name -param_M1_C -scope wire_variation -distribution gaussian -range {0.95 1}
```

- The following command defines R/C moments for normal distribution for M1 layer for setup for all views:

```
define_parameter setup -name param_M1_R -scope wire_variation -distribution gaussian -moments {0.2 0.01}
define_parameter setup -name param_M1_C -scope wire_variation -distribution gaussian -moments {0.95 1}
```

- The following command defines correlation between metal layers - if M1, M2 and M3 are fully correlating:

```
define_parameter -scope wire_variation  -correlation {{param_M1_C param_M2_C param_M3_C} {1.0}}
```

Related Commands

- report_parameter

get_glitch_threshold

```
get_glitch_threshold
[-help]
[-cells string]
[-failure_dependency {none |  input | last}]
[-failure_point {none | input | internal | last}]
[-glitch_type {both | vl | vh | vlu | vho}]
[-library_pins string]
[-nets string]
[-pin_type {all | clock | data | latch}]
[-pins string]
[-threshold_type {reporting | failure}]
[-views string]
```

Displays information for the specified `set_glitch_threshold` parameters for the current Tempus session. If you do not specify a parameter, the software displays information for all `set_glitch_threshold` parameters.

Parameters

parameter_names	Displays information for the specified parameter. You can specify one or more parameters. See `set_glitch_threshold` for descriptions of the timing analysis mode parameters you can specify.

Examples

- The following command shows the failure threshold for ROP for single stage cells:
  ```
  get_glitch_threshold -failure_point last
  ```
  Here's the command output:
  ```
  value=0.4 -threshold_type reporting -glitch_type both -pin_type all -failure_point last -
  failure_dependency none
  ```
- The following command shows the failure threshold for the ROP for the internal net of multi-stage cells:
  ```
  get_glitch_threshold -failure_point internal
  ```
  Here's the command output:
  ```
  value=0.2 -threshold_type reporting -glitch_type both -pin_type all -failure_point internal -
  failure_dependency none
  ```
- The following command shows the failure threshold for the RIP for the input net of cells without CCSN/ECSMN/cdB
  ```
  get_glitch_threshold -failure_point input
  ```
  Here's the command output:

```
value=0.5 -threshold_type reporting -glitch_type both -pin_type all -failure_point input -
failure_dependency none
```

Related Commands

- set_glitch_threshold

get_multi_input_switching_mode

```
get_multi_input_switching_mode
[-help]
{[-disable_lib_cells ] [-switching_alignment_factor ] [-mis_alignment_mode]}
```

Reports the MIS (multi-input switching) analysis settings configured using `set_multi_input_switching_mode` command.

Parameters

`-help`	Prints out the command usage.
`-disable_lib_cells <string>`	Reports all the user-specified library cells for which multi-input switching (MIS) mode support is disabled.
`-mis_alignment_mode {0 | 1}`	Returns the current value set (0 or 1) of the MIS alignment mode.
`-switching_alignment_factor <float>`	Returns the switching alignment factor specified globally.

Related Commands

- set_multi_input_switching_mode
- report_delay_calculation
- report_timing
- write_spice_deck

Related Information

- "Advanced Multiple-Input Switching Analysis" section of Analysis and Reporting chapter in the *Tempus Stylus User Guide*.

get_via_variation_mode

```
get_via_variation_mode
[-help]
[-corner_sigma_multiplier]
[-correlation_type]
[-reset]
[-separate_params_for_via]
[-quiet]
[-nonDefault]
```

Displays the following information about the specified `set_via_variation_mode` command parameters in the Tempus log file and the Tempus console:

- Parameter name
- Current value
- Type (Boolean, string, and so on)
- Whether the current value was set by user

If you do not specify any parameters, the software displays information for all of the parameters.

Parameters

`-help`	Prints out the command usage.
parameter_names	Displays information for the specified parameters. You can specify one or more parameters. See `set_via_variation_mode` for descriptions of the parameters you can specify.

Related Commands

- `set_via_variation_mode`

merge_noise_reports

```
merge_noise_reports
[-help]
-base_dir <string>
[-delay {max | min}]
[-double_switching]
[-failure]
[-reverse_voltage_change_threshold <float>]
[-sort_by {noise | receiver_peak}]
[-threshold <float>]
-out_file <string>
[-views <string>]
```

Generates merged noise reports for the specified views.

Parameters

`-help`	Prints out the command usage.
`-base_dir <string>`	Specifies the name of the directory provided as input to the `report_noise -merge_dir` parameter.
`-delay {max | min}`	Specifies the uncertainty type (maximum or minimum delay).
`-double_switching`	Enables double switching reporting.
`-failure`	Shows output glitch report for failure nets only.
`-reverse_voltage_change_threshold <float>`	Reports the minimum reverse swing threshold values.
`-sort_by {noise | receiver_peak}`	Specifies the sorting of the report based on noise or receiver_peak.
`-threshold <float>`	Reports the glitch (in mV) or uncertainty (in ps) filter threshold values.
`-out_file <string>`	Specifies the name of the output file.
`-views <string>`	Specifies the name of views to include in the merged output.

Related Commands

- `report_noise`

report_bundled_bus

```
report_bundled_bus
[-help]
[-name string]
[-nets string]
[-type {user_defined | tool_inferred}]
[[ > | >> ]]
```

Generates a report for bundled buses.

Parameters

`-help`	Prints out the command usage.
`-name string`	Specifies the name of the user-defined bus group(s) to be reported.
`-nets string`	Specifies the name of the net(s) to be reported.
`-type {user_defined | tool_inferred}`	Reports the type of bundled bus(es) - user-defined or tool-inferred - to include in the report. By default, the `report_bundled_bus` command generates reports for both user-defined and software-inferred type of buses.
`> filename`	Redirects the output to the specified file.
`>> filename`	Appends to the specified output file.

Examples

- The following command reports details of user-defined bus group `bus1`:

```
tempus > report_bundled_bus -name bus1

Summary:
Total number of user defined bus groups reported  = 1
Total number of tool inferred bus groups reported = 0

-------------------------------------------------------

Group Name (user_defined) : bus1

Constituents:
- FE_OF_13
```

- The following command reports details of user-defined bus group `bus2` for the specified nets:

```
tempus > report_bundled_bus -nets FE_OF_2
```

```
Summary:
Total number of user defined bus groups reported  = 1
Total number of tool inferred bus groups reported = 0

-----------------------------------------------------

Group Name (user_defined) : bus2

Constituents:
- FE_OF_2
```

- The following command reports the `user_defined` bundled buses:

```
tempus > report_bundled_bus -type user_defined

Summary:
Total number of user defined bus groups reported  = 2
Total number of tool inferred bus groups reported = 0

-----------------------------------------------------

Group Name (user_defined) : bus1

Constituents:
- FE_OF_13
-----------------------------------------------------

Group Name (user_defined) : bus2

Constituents:
- FE_OF_2
- FE_OF3_data_out_1
-----------------------------------------------------
```

- The following command shows the user-defined bus groups:

```
report_bundled_bus

Summary:
Total number of user defined bus groups reported = 3
Total number of tool inferred bus groups reported = 41

-----------------------------------------------------
Group Name (user_defined) : grp1

Constituents:
- EJ_PartNumber[15]
- EJ_PartNumber[14]
- EJ_PartNumber[13]
- EJ_PartNumber[12]
- EJ_PartNumber[11]
- EJ_PartNumber[10]
- EJ_PartNumber[9]
- EJ_PartNumber[8]
- EJ_PartNumber[7]
- EJ_PartNumber[6]
- EJ_PartNumber[5]
```

```
- EJ_PartNumber[4]
- EJ_PartNumber[3]
- EJ_PartNumber[2]
- EJ_PartNumber[1]
- EJ_PartNumber[0]
-------------------------------------------------------
Group Name (user_defined) : grp2

Constituents:
- EJ_MufID[10]
- EJ_MufID[9]
- EJ_MufID[8]
```

Related Commands

- define_bundled_bus

report_double_clocking

```
report_double_clocking
[-help]
[-max_delta_delay float]
[-max_slew float]
[-nworst integer]
[-out_file string]
[-reverse_slope_limit float]
[-reverse_voltage_change_threshold float]
[-view string]
```

Reports noise impact on clock nets by generating information on the double clocking, transition type, and reverse slope.

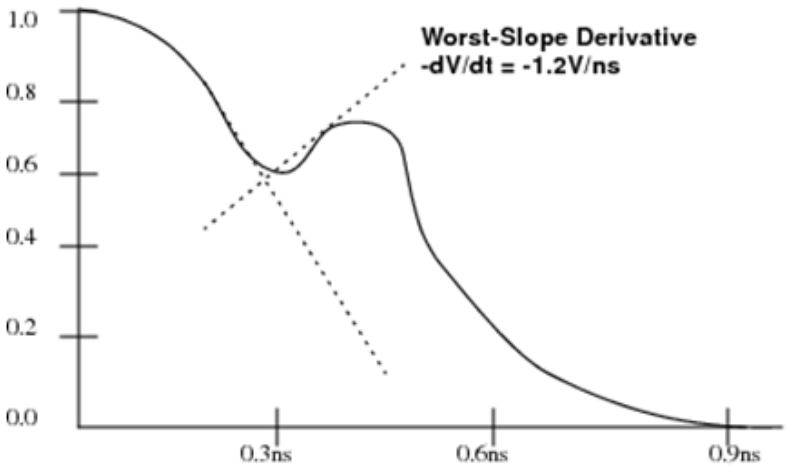
Note: The `report_double_clocking` command requires that clocks have been defined and applied prior to timing analysis with SI (`update_timing` or `report_timing`). The `report_double_clocking` command does not work with the glitch-only flow (`update_glitch`).

Double clocking occurs when the noise glitch attacks a transitioning signal. The multiple transition or double clocking check supplements the transition time check performed in static timing analysis. Transition time is defined as the duration of time that a signal takes to make the transition between two threshold points. However, transition time does not account for the waveform shape between the voltage thresholds.

For special signals such as clocks it is important to analyze the waveform shape between the voltage threshold in case the signal transitions slowly or in the opposite direction (even for a short duration of time) causing multiple clock edges. This command enables you to report the worst-case slope (dV/dtfor rising signal and -dV/dt for falling signal) of a signal transitioning between the voltage thresholds. The slope derivative is measured in units of volts per nanosecond (V/ns). This value can be positive if the signal does not change direction, 0 if the signal is flat, and negative if the signal changes direction even for a short duration of time.

The following figure shows two falling signal transitions:

Figure: Negative Worst-Slope Derivative



In the above figure, the falling signal transitions have identical transition times. However, the worst-case slope derivatives for the two falling waveforms are different. If a clock signal exhibits a negative slope derivative as in the case of Figure 13-2, there is a possibility of a double clocking functional failure. A higher reverse slope results in a higher possibility of a double clocking functional failure.

The double clocking report contains the following sections and their corresponding fields:

- **Double Clocking**
- Rise Slope (V/ns): Specifies the worst reverse slope of the rising clock signal as a measure of volt per nano-second.
- Fall Slope (V/ns): Specifies the worst reverse slope for the falling clock signal as a measure of volt per nano-second.
- VictimNet: Specifies the name of the victim net.

- **Transition Type**

- **Reverse Slope**

Parameters

`-help`	Prints out the command usage.
`-out_file <`*`string`*`>`	Specifies the output filename. *Default*: {$parm_filename_prefix}_clock.text
`-max_delta_delay` *`float`*	Reports clock nets, for which the delay change caused due to noise, exceeds the specified number. *Default*: 0
`-max_slew` *`float`*	Reports clock nets for which the transition time, including SI, exceeds the specified number. *Default*: 0
`-nworst` *`integer`*	Specifies the number of clock nets to report in each category. *Default*: 1
`-reverse_slope_limit` *`float`*	Reports the clock nets for which the slope is below the specified number. *Default*: 0

`-reverse_voltage_change_threshold` *float*	Controls the reverse slope failure criteria by allowing you to supply a minimum voltage change as a percentage of VDD. The reverse slope must be sustained in order for it to be considered as failing. *Default*: 0.1
`-view` *string*	Specifies the view name for which the report to be generated.

Examples

- The following command reports clock nets that have a reverse slope less than -1:
  ```
  report_double_clocking -reverse_slope_limit -1
  ```
- The following command reports 100 clock net violations of each category:
  ```
  report_double_clocking -nworst 100
  ```

Related Commands

- `update_timing`
- `report_timing`

report_glitch_thresholds

```
report_glitch_thresholds
[-help]
[-failure_point {input | output}]
[-glitch_type <string>]
[-instance_pins <string>]
[-library_pins <string>]
[-list_unannotated_instance_pins]
[-list_unannotated_library_pins]
[-view <string>]
```

Reports user-defined glitch thresholds applied on library or instance pins.

Parameters

`-help`	Prints out the command usage.
`-failure_point {input | output}`	Specifies if threshold failure point is input or output. *Default*: `input`
`-glitch_type <string>`	You can specify one or more glitch types (space-separated) for reporting. The valid options are: `vl`, `vh`, `vlu`, `vho`, and `all`. When the `-glitch_type` parameter is specified, only the user-specified glitch types will be checked. *Default*: `all`
`-instance_pins <string>`	Specifies a list of instance pin names.
`-library_pins <string>`	Specifies a list of library pin names to be reported.
`-list_unannotated_instance_pins`	Reports list un-annotated instance pins. Reports all the instance input pins which do not have glitch threshold applied.
`-list_unannotated_library_pins`	Shows a list of all the library input pins which are not covered by library pin-based/corresponding library cell-based glitch threshold.
`-view <string>`	Specifies the view name.

Examples

- The following report shows instance-pin specific glitch threshold summary:

```
+-----------------------------------------------------------------+
```

```
                Instance-Pin Glitch Thresholds Summary
+-------------------------------------------------------------------+
| Glitch Type | Inst Pin Count   | Annotated (%) | Not Annotated (%)|
+-----------------+--------------+---------------------+-------------+
| VL          | 153001           | 21075 13.77%  | 131926 86.23 %   |
+-------------------------------------------------------------------+
| VH          | 153001           | 21075 13.77%  | 131926 86.23 %   |
+-------------------------------------------------------------------+
| VLU         | 153001           | 0 0.00 %      | 153001 100.00%   |
+-------------------------------------------------------------------+
| VHO         | 153001           |  0 0.00 %     | 153001 100.00%   |
+-------------------------------------------------------------------+
+-------------------------------------------------------------------+
```

The following section of the above report shows library-cell/library-pin specific glitch threshold summary:

```
               Library-Pin Glitch Thresholds Summary
+-------------------------------------------------------------------------+
| Glitch Type | Lib Pin Count   | Annotated (%)    | Not Annotated (%)|
+-----------------+--------------+--------------------+------------------+
VL            |     742         |0 0.00%           |742 100.00%
+-------------------------------------------------------------------------+
VH            |     742         |0 0.00%           | 742 100.00%
+-------------------------------------------------------------------------+
VLU           |     742         |653 88.01%        |89 11.99%
+-------------------------------------------------------------------------+
VHO           |     742         |653 88.01%        |89 11.99%
+-------------------------------------------------------------------------+

...

...
```

- The following command generates a verbose report:

```
report_glitch_thresholds \
-list_unannotated_library_pins -list_unannotated_instance_pins

+-----------------------------------------------------+
              Unannotated Instance Pins
+-----------------------------------------------------+
Instance Pin              Unannotated Glitch Type
+-----------------------------------------------------+
buf inst1/I               {VL, VH, VLU, VHO}
buf 1nst2/I               {VL, VHO}
...
...
+-----------------------------------------------------+
+-----------------------------------------------------+
```

```
             Unannotated Library Pins
+-------------------------------------------------------+
Library Pin               Unannotated Glitch Type
+-------------------------------------------------------+
Lib_lV/BUF/I              {VL, VH, VLU, VHO}
Lib_OpV/BUF/I             {VL, VH}
...
...
+-------------------------------------------------------
```

Related Commands

- set_glitch_threshold
- get_glitch_threshold

Related Information

- "Signal Integrity Delay and Glitch Analysis" chapter in the *Tempus Stylus User Guide*.

report_instance_cdb

```
report_instance_cdb
[-help]
[-analysis_type {early | late}]
[{instance_list or collection}]
[-out_file string]
[-quiet]
[-view viewname]
```

Allows you to report the cdB associated with all the instances or a subset of instances in a design.

Note: This command must be issued after `report_timing`, `update_timing`, or `update_glitch` commands.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_instance_cdb` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_instance_cdb`
`-analysis_type {early | late}`	Specify the analysis type for the query. *Default*: `late`
instance_list or *collection*	Specify the instance for which the cdB association is desired. By default, the cdB association for all the instances is reported.
`-out_file` *string*	Specify the output filename for the instance report to go to. By default, the report output is displayed on the screen.
`-quiet`	Suppresses warning messages for cells missing in CDB files.
`-view {`*viewnames*`}`	Specifies the view names. By default, values for all the views are reported.

Examples

- The following command reports the cdB instance connection for all the instances in the design and write it to the inst_cdb.rpt file:

```
tempus > report_instance_cdb -output_file inst_cdb.rpt
```

- The following command reports the cdB instance connection for the u5 instance to the screen:

```
tempus > report_instance_cdb u5


VIEW: default_emulate_view -- ANALYSIS TYPE: late
u5 : inv_t1 : test1.cdb.txt
```

- The following command reports the cdB instance connection for the early analysis type for the u5 and u6 instances

```
tempus > report_instance_cdb -analysis_type early {u5 u6}

VIEW: default_emulate_view -- ANALYSIS TYPE: early
u5 : inv_t1 : test1.cdb.txt
u6 : inv_t2 : test2.cdb.txt
```

Related Commands

- read_libs
- read_mmmc
- report_timing
- update_timing
- update_glitch

report_jitter

```
report_jitter
[-help]
[-eiv_end_time float]
[-eiv_file string]
[-eiv_power_ground string]
[-eiv_start_time float]
[-from string]
[-from_fall string]
[-from_rise string]
[-mode {period | cycle_to_cycle | time_interval_error}]
[-out_dir string]
[-through <string>]
[-to string]
[ [ > | >> ] ]
```

Provides a native jitter interface to report jitter through the native Tempus engine.

The existing `analyze_jitter` command will continue to be used for Spice-based jitter analysis.

Parameters

`-help`	Prints out the command usage.
`-eiv_end_time` *float*	Specifies the end time in EIV.
`-eiv_file` *string*	Specifies the effective instance voltage files for accounting IR Drop in Jitter computation.
`-eiv_power_ground` *string*	Specifies the power and ground net name for each EIV file. It is mandatory to provide power and ground net name for each eiv file provided using the `-eiv_file` parameter. For example, "vdd1 vss1 vdd2 vss2", vdd1 and vss1 are for first and vdd2 and vss2 for second EIV file - specified using the `-eiv_file` parameter.
`-eiv_start_time` *float*	Specifies the start time in EIV.
`-from` *string*	Creates the Spice deck with mesh net starting from pin.
`-from_fall` *string*	Creates the Spice deck with mesh net starting from falling transition of specified from pin.
`-from_rise` *string*	Creates the Spice deck with mesh net starting from rising transition of specified from pin.
`-mode {period | cycle_to_cycle | time_interval_error}`	Runs the `analyze_jitter` analysis natively - period is default jitter type.
`-out_dir` *string*	Specifies the output directory for the created Spice.
`-through <string>`	Creates the Spice deck with mesh net going by through points.
`-to` *string*	Creates the Spice deck with mesh net ending at to pins.
`>` *filename*	Redirects the `-run_path_simulation`/`-run_simulation` output to a file.
`>>` *filename*	Appends the `-run_path_simulation`/`-run_simulation` output to a file.

Examples

- The following command reports analysis jitter details (for `period` mode) in the specified output directory:
  ```
  report_jitter -eiv_file eiv_file1 -out_dir jitter_dir -mode period
  ```
- The following command reports analysis jitter details (for `cycle_to_cycle` mode) in the specified output directory:
  ```
  report_jitter -eiv_file eiv_file1 -out_dir jitter_dir -mode cycle_to_cycle
  ```

Related Commands

- analyze_jitter

report_mutex_condition

```
report_mutex_condition
[-help]
[-filename <string>]
[-net <string>]
```

Reports mutex nets of a victim net.

When this command is used, all the coupled nets that are participating in mutex for a victim will be reported. The coupled nets will be reported as a group of mutex nets (user-defined). By default, all the mutex conditions are reported.

Parameters

`-help`	Prints out the command usage.
`-filename <string>`	Specifies the name of the file that saves the command output results.
`-net <string>`	Reports the coupled nets mutex to the specified net name. You can specify multiple net names with multiple `report_mutex_condition` commands.

Examples

- The following example shows mutex nets of a victim net. This report does not indicate the attackers not listed due to mutex command:

```
Vic: Net1
Attackers: Att1, Att2, Att3
set_mutex_condition {Att1, Att2 }
set_mutex_condition {Att2, Att3 }
set_mutex_condition {Att1, Att3 }
```

`report_mutex_condition -net Net1` will report:

```
- Victim Net: Net1
Mutex groups:
{Att1, Att2 },
{Att2, Att3 },
{Att1, Att3 }
```

- The following example shows that if a coupled net is having LC chain then the entire chain will be reported along with other mutex group nets:

```
Vic: Net1
Attackers: Att1, Att2, Att3
Att1 chain: Agg1 Agg11 Agg111
Att2 chain: Agg2 Agg22 Agg222
Att3 chain: Agg3 Agg33 Agg333
set_mutex_condition {Att1, Att2 }
set_mutex_condition {Att2, Att3 }
set_mutex_condition {Att1, Att3 }
```

`report_mutex_condition -net Net1` will report:

```
- Victim Net: Net1

Mutex groups:
{{ Agg1 Agg11 Agg111 }, { Agg2 Agg22 Agg222}}
{{ Agg2 Agg22 Agg222}, { Agg3 Agg33 Agg333}}
{{ Agg1 Agg11 Agg111 }, { Agg3 Agg33 Agg333}}
```

Related Commands

- set_mutex_condition

report_noise

```
report_noise
[-help]
[-cell_type string]
[-clock]
[-delay {max | min}]
[-double_clocking]
[-double_switching]
[-exclude_failures]
[-expand_virtual_attacker_bus]
[-failure]
[-format string]
[-gui_victim_nets_file]
[-histogram]
[-level string]
[-max_threshold float]
[-merge]
[-merge_dir string]
[-nets string]
[-noisy_waveform]
[-out_file string]
[-output_format string]
[-overshoot_undershoot]
[-pins string]
[-precision integer]
[-quiet_nets]
[-remove_status_row <string>]
[-report_virtual_attacker_constituents]
[-reverse_voltage_change_threshold <float>]
[-slack float]
[-skip_nets_or_pins_in_header]
[-sort_by string]
[-style {default | detailed}]
[-threshold string]
[-type string]
[-view string]
```

Generates text-based noise reports.

Note: You need to ensure that the `si_glitch_enable_report` (to generate glitch report) or `si_delay_enable_report` (to generate detailed delay report) attributes are set to `true`, before the `report_timing`, `update_timing`, or `update_glitch` commands are run.

For more information on attributes, see si Category Attributes.

Parameters

`-help`	Prints out the command usage.
`-clock`	Prints out the signal integrity report for the clock network.
`-cell_type` *string*	Generates a glitch report for the user-specified cell-type. You can specify one of the following options: • `bbox`: Generates a report for only the black-box receivers • `clock`: Generates a report for only the clock pin receivers • `data`: Generates a report for only the data pin receivers • `latch`: Generates a report for only the latch pin receivers
`-delay {max | min}`	Reports delay uncertainty instead of noise values. The valid options are `min`, `max` - for minimum or maximum path delay analysis. The `report_noise -clock -delay min | max` command outputs minimum/maximum delay report for clock nets.
`-double_clocking`	Enables double clocking reporting.
`-double_switching`	Reports double switching analysis data in noise reports.
`-exclude_failures`	Excludes all failing nets from the glitch report.
`-expand_virtual_attacker_bus`	Reports virtual attacker components and expands bus bits in the noise reports. This parameter works only when the bus virtual attacker feature is enabled using the `si_enable_virtual_attacker_constituent_report` true setting.

`-failure`	Reports receiver peak failures and DC tolerance failure nets. Use this option only for glitch noise failure reports.
	Option: Directory Name:
	`-sort_by noise glitch`
	`-sort_by rcvr_peak glitch`

`-format` *string*	Specifies the output report format fields. The valid fields are: arc, attacker_edge, attacker_logical_correlation_phase, attacker_net, attacker_offset, attacker_peak, attacker_slew, attacker_source, attacker_status, attacker_vdd, attacker_wire_length, attacker_xcap, cell_type, delay, glitch_slack, incr_delay, level, peak, receiver_cell, receiver_input_failure_threshold, receiver_input_pin, receiver_output_failure_threshold, receiver_output_peak, slack, slack_status, stage_info, thresh_source,

verbose_logical_correlation_report, victim_edge, victim_library, victim_logical_correlation_phase, victim_net, victim_slew, victim_total_area, victim_total_cap, victim_total_cross_cap, victim_vdd, victim_width, victim_wire_length

Note: The `victim_total_cross_cap` option is supported only for SI delay reporting.

Each field corresponds to the following columns of the glitch report:

- arc: Arc
- attacker_edge: Edge
- attacker_net: Net
- attacker_offset: Offset (ps)
- attacker_peak: Peak (mV)
- attacker_slew: Slew (ps)
- attacker_source: Constituents: Source
- attacker_status: Statue
- attacker_vdd Vdd (V)
- attacker_wire_length
- attacker_xcap: Xcap (fF)
- glitch_slack: Slack
- level: Level
- peak: Peak (mV)
- receiver_cell: Receiver cell
- receiver_input_failure_threshold: Receiver Input Threshold
- receiver_input_pin: ReceiverNet
- receiver_output_failure_threshold: Failure threshold
- receiver_output_peak: Receiver output peak Value
- stage_info
- thresh_source: ReceiverThresholdType (Global | User | Library)
- victim_library: Library
- victim_net: VictimNet
- victim_total_area: TotalArea

	• victim_total_cap: TotalCap (fF) • victim_vdd: Vdd (V) • victim_width: Width (ps) • victim_wire_length The following options are not available with the single line format: • delay • incr_delay • slack • thresh_source • victim_edge • victim_slew • victim_total_cross_cap To choose whether the report format should be on a single line or multiple lines, you can use the `-output_format` parameter.
`-gui_victim_nets_file`	Generates the GUI format glitch reports (`aae or aae_<viewname>` files under the reporting directory) and incremental delay reports (`aae_<max/min>_incDelay or aae_<viewname>_<max/min>_incDelay` files under the reporting directory) to be used in the Display Noise Net form.
`-histogram`	Generates the noise histogram.
`-level {vl | vh | vlu | vho}`	Specifies the level for which a report needs to be generated when single-level report is desired. You can specify one of these options - vl, vh, vlu, and vho.
`-max_threshold` *float*	Specifies the maximum peak noise or receiver peak thresholds to report on.
`-merge`	Generates a merged report that shows only the worst glitch slack for each pin level combination across views.
`-merge_dir` *string*	Specifies the name of the merge database.
`-nets` *string*	Reports the specified nets in the noise report.
`-noisy_waveform`	Flags nets with bumpy waveforms at the receiver output.

`-out_file` *string*	Specifies the name of the output file name prefix. The name of the output file is generated by combining the prefix and the specified view name. If you do not specify this option: • The software automatically names the text report as 'glitch' for noise glitch reports and 'delay' for noise delay reports. • The software dumps the generated report to the console window for better usability. **Note**: If the text report file with the `.gz` extension is used, the output file is gzipped to generate a compressed report.
`-output_format` *string*	Specifies if the output format should be on a single line or on multiple lines. You can specify one of the following options: • `single_line` • `multiple_line` If the `single_line` option is specified and the attacker info is chosen as a field, then all the attacker related format options will be printed together. The `-output_format` parameter only works with the `-format` option and does not affect the default reporting output format. *Default*: `multiple_line`
`-overshoot_undershoot`	Reports the overshoot (VHO) and undershoot (VLU) glitches. *Default*: If this option is not used during glitch reporting, then VL and VH type of glitches will be reported.
`-pins` *string*	Specifies the name of pins to be reported.
`-precision integer`	Specifies the precision value up to which the glitch data is to be printed. The allowed values are 0, 1, 2 and 3. *Default*: `2`
`-quiet_nets`	Generates the quiet attackers report.
`-remove_status_row <string>`	Shows only the active attackers and filters out Sb, UF, and LC from the glitch report.
`-report_virtual_attacker_constituents`	Generates reports for virtual attacker constituents.

`-reverse_voltage_change_threshold <float>`	Reports the clock nets with reverse swing greater than the user-specified threshold value (ratio of VDD). By default, the threshold value is set to 0.1.
`-slack` *float*	Generates a report for bumpy nets based on slack. *Units*: Seconds
`-skip_nets_or_pins_in_header`	Generates the glitch report without a list of nets or pins in the report header.
`-sort_by` *string*	Requests the sorting of entries. You can sort by: • `noise` • `receiver_peak` • `slack` • `cell_type` Tempus determines the default HTML output directories based on the options you choose. Depending on the sorting option you select, Tempus sorts the report fields in the following order: **-sort_by Option Level 1 Level 2 Level 3 Level 4** rcvr_peak receiver peak noise netname level noise noise - netname level
	For example, if you use the `-sort_by noise` option, the first level of sorting will be based on noise, followed by receiver peak, netname, and level. **Note**: The report displays the `vl` noise level before the `vh` noise level. *Default*: `receiver_peak`
`-style {default | detailed}`	Generates reports in one of the two styles - `extended` and `default`. • `detailed`: Generates detailed reports with victim and aggressor timing windows, dropped aggressor slews, and reasons for dropped aggressors. Additionally, the report also shows driver pins and driver library pins. • `default`: Generates the default report, which is similar to the text report generated using the `-text` parameter.

`-threshold` *string*	Specifies the minimum peak noise, sensitivity, absolute delay uncertainty, relative delay uncertainty, and receiver peak thresholds to report on. *Default*: • 0.10 of vdd for nets - when the glitch report is generated using the `report_noise -sort_by noise` option. • 10 (ps) for nets - when a detailed delay report is generated using the `report_noise -delay` parameter). **Note**: • The `-threshold` parameter accepts percentage values. • All `-sort_by` values that represent voltages will be in mV.
`-type` *string*	Specifies the type of noise to be reported - glitch, delay or slew.
`-view` *string*	Specifies the view name.

File and Directory Naming Convention for Reports

The software uses a set of naming conventions for the report files and directories that are generated by the `report_noise` command. Each type of report file or directory is identified by a combination of strings that are attached to its name. The software adds the appropriate string to the report file or directory name.

Noise Reports

For noise reports, the `report_noise` command uses the following naming convention for report files or directories:

```
glitch
```

Delay Reports

For delay reports, the `report_noise` command uses the following naming convention for report files or directories:

```
delay
```

Report File or Directory Name Examples

This section contains examples of using the `report_noise` command with different options to generate different types of reports and depicts the corresponding impact on the name of the report file or directory.

Examples of Generating Noise Report Files or Directories

The following table shows the combination of different options used with the `report_noise` command for noise reports and the corresponding names of the generated report files or directories:

Command(s)	Report File/Directory Name
`report_noise -sort_by noise -out_file example_glitch.txt`	`example_glitch.txt`
`report_noise -sort_by rcvr_peak -out_file example_recevier_peak.txt`	`example_receiver_peak.txt`
`report_noise -failure -out_file failure.txt`	`failure.txt`

Examples of Generating Delay Report Files or Directories

The following table shows the combination of different options used with the `report_noise` command for delay reports and the corresponding names of the generated report files or directories:

Command(s)	Report File/Directory Names
`report_noise -delay min -out_file min_delay.txt`	`min_delay.txt`
`report_noise -delay max -out_file max_delay.txt`	`max_delay.txt`

Sample Delay Report

```
Type  IncrementalDelay(ps)  Delay(NomMax/Max)  VictimNet
R     1725.800              128.300/1854.100   inst1/I {victim}

Constituents:
       Offset(ps)           Slew(ps)   Xcap(fF)   Edge   Net
Cpl:   3707.803             3592.375   48.692     F      att1
Cpl:   1028.237             967.125    6.592      F      att2
Cpl:   1055.632             963.625    4.348      F      att3
Cpl:   885.057              744.750    4.330      F      att4
```

Examples

- The following command prints out a signal integrity report for the clock network:

```
tempus > report_noise -clock -out_file noise.txt
Report Options:
------------------------------------------------------------
 -type glitch
 -clock
```

```
 -threshold 10 % (of vdd)
 [-style default]
 [-sortBy noise]
Level     : VH and VL
-------------------------------------------------------------
-------------------------------------------------------------
            Input              Internal          Last
-------------------------------------------------------------
         VL        VH       VL       VH      VL      VH
-------------------------------------------------------------
Clock    0.400     0.400    0.050    0.050   0.050   0.050
-------------------------------------------------------------
Data     0.400     0.400    0.300    0.300   0.150   0.150
-------------------------------------------------------------
Latch    0.400     0.400    0.100    0.100   0.100   0.100
-------------------------------------------------------------
```

- The following command generates a glitch report for black-box receivers:

```
tempus > report_noise -cell_type bbox -out_file clock1.txt
Report Options:
-------------------------------------------------------------
 -type glitch
 -cell_type { bbox}
 -threshold 10 % (of vdd)
 [-style default]
 [-sortBy noise]
Level     : VH and VL
-------------------------------------------------------------
-------------------------------------------------------------
            Input              Internal          Last
-------------------------------------------------------------
         VL        VH       VL       VH      VL      VH
-------------------------------------------------------------
Clock    0.400     0.400     0.050    0.050   0.050   0.050
-------------------------------------------------------------
Data     0.400     0.400     0.300    0.300   0.150   0.150
-------------------------------------------------------------
Latch    0.400     0.400     0.100    0.100   0.100   0.100
-------------------------------------------------------------
```

```
Glitch Violations Summary :
--------------------------
Number of DC tolerance violations (VH + VL) = 50
Number of Receiver Output Peak violations (VH + VL) = 0
Number of total problem noise nets = 4
```

- The following command reports maximum path delay analysis delay details at the specified threshold:

```
tempus > report_noise -delay max -threshold 1600 -out_file delay_max_format.rpt \
     -format {attacker_slew  attacker_xcap  attacker_status attacker_edge \
      attacker_net stage_info slack  victim_slew  victim_total_cross_cap victim_net }
Report Options:
-------------------------------------------------------------------------------
 -type delay
 -threshold 1600 (ps)
 -delay max
 [-style default]
 [-sortBy IncrDelay]
-------------------------------------------------------------------------------
-------------------------------------------------------------------------------
Slack(ps)   Slew(ps)  Total Xcap(fF) VictimNet
-14559.980 2179.800          4.891 TSP_CRE/TSP_CRE_GLE/n_269
Type  IncrDelay(ps)  Delay(NomMax/Max)   Vdd(mV)  VictimInst
R     2751.200       1397.800/4149.000    800   TSP_p8A/ZN {A2_X1:B2}
 Stage information:
                Cell            Pin
    Driver   : AOI22_X1         TSP_CRE/p85A/ZN
Constituents:
Slew(ps)  Xcap(fF)  Status     Edge  Net
104.26    2.23      SY         F     rom_data[12]
Slack(ps)   Slew(ps)  Total Xcap(fF) VictimNet
-11604.494 2351.300         13.576 DATA_MUX/FE_39
```

- The following command reports double switching analysis data:

```
tempus > report_noise -double_switching -out_file switching.rpt
Report Options:
-------------------------------------------------------------------------------
 -type delay
 -threshold 10 (ps)
 -delay max
 [-style default]
```

```
 [-sortBy IncrDelay]
-------------------------------------------------------------------------------
-------------------------------------------------------------------------------
Type  IncrDelay(ps)  Delay(NomMax/Max)  ReverseSwing(mV)  Vdd(mV) VictimNet
R      -63.500       1423.800/1360.300  123.750           1980    PLLCLK/refclk {refclk_L2_N}

Type  IncrDelay(ps)  Delay(NomMax/Max)  Vdd(mV) VictimInst
R     201.200        479.200/680.400    1980    refclk_L2_I/ZN {INV_X32:A}

Constituents:
     Offset(ps) Slew(ps)  Xcap(fF)  Edge   Status  Vdd(mV)  Net
Cpl: 655.000    5.000     11.390    F      SY      800      refclk
```

- The following command reports nets with the specified level VL:

```
tempus > report_noise -nets FE_O14_tsp_data_out_1 -level vl -out_file noise.txt
Report Options:
-------------------------------------------------------------------------------
 -type glitch
 -threshold 10 % (of vdd)
 [-style default]
 [-sortBy noise]
Level     : VL
-----------------------------------------------------------
-----------------------------------------------------------
       Input                Internal           Last
-----------------------------------------------------------
         VL       VH       VL      VH       VL     VH
-----------------------------------------------------------
Clock    0.400    0.400    0.050   0.050   0.050    0.050
-----------------------------------------------------------
Data     0.400    0.400    0.300   0.300   0.150    0.150
-----------------------------------------------------------
Latch    0.400    0.400    0.100   0.100   0.100    0.100
-----------------------------------------------------------
Glitch Violations Summary :
--------------------------
Number of DC tolerance violations (VH + VL) =  0
Number of Receiver Output Peak violations (VH + VL) =  0
```

```
Number of total problem noise nets =  0
```

- The following command generates a maximum path analysis noise report for the specified reporting fields:

```
tempus > report_noise -delay max -format {delay incr_delay victim_slew victim_total_cap \
            victim_total_cross_cap victim_vdd victim_net attacker_offset attacker_slew \
            attacker_xcap attacker_vdd attacker_edge attacker_status attacker_net} -out_file
custom.rpt
Report Options:
--------------------------------------------------------------------------------
 -type delay
 -threshold 10 (ps)
 -delay max
 [-style default]
 [-sortBy IncrDelay]
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
Delay(NomMax/Max)  IncrDelay(ps) Slew(ps)  TotalCap(fF) Total Xcap(fF) Vdd(mV) VictimNet
   0.500/   0.500        0.000 2179.800        7.066          4.891      800
TSP_CRE/TSP_CRE_GLE/n_269
Type  IncrDelay(ps)  Delay(NomMax/Max)   Vdd(mV)  VictimInst
R     2751.200       1397.800/4149.000    800   TSP_p8A/ZN {A2_X1:B2}
R     2714.800       1297.200/4012.000    800   TSP_p8A/ZN {A2_X1:B1}
R     2733.000       1313.200/4046.200    800   TSP_p8A/ZN {A2_X1:A2}
R     2720.700       1180.700/3901.400    800   TSP_p8A/ZN {A2_X1:A1}
Constituents:
Offset(ps)  Slew(ps)  Xcap(fF)  Vdd(mV)  Edge  Status     Net
3614.00     104.26    2.23      1200     F     SY         rom_data[12]
3422.00     273.94    1.28      800      F     SY         TSP_CRE_/PRG_BS/n_21
3632.00     91.54     0.49      1200     F     SY         rom_data[15]
3665.00     51.77     0.33      800      F     SY         m_clk__L18_N0
3078.00     679.49    0.35      800      F     SY         TSP_CRE/FE_OFN21
3066.00     560.35    0.21      800      F     SY         TSP_CRE/data_out[11]
```

- The following command generates a threshold report:

```
tempus > report_noise -threshold 0.20 -out_file threshold.rpt
Report Options:
--------------------------------------------------------------------------------
 -type glitch
 -threshold 20 % (of vdd)
 [-style default]
```

```
 [-sortBy noise]
Level     : VH and VL
```

- The following command generates a report sorted by noise:

```
tempus > report_noise -out_file sort_by_glitch_input.txt -sort_by noise
Report Options:
-------------------------------------------
 -type glitch
 -threshold 10 % (of vdd)
 [-style default]
 [-sortBy noise]
Level     : VH and VL
-----------------------------------------------------
-----------------------------------------------------
   Input          Internal        Last
-----------------------------------------------------
         VL      VH     VL      VH      VL     VH
-----------------------------------------------------
Clock  0.400   0.400   0.050  0.050   0.050  0.050
-----------------------------------------------------
Glitch Violations Summary :
--------------------------
Number of DC tolerance violations (VH + VL) = 48
Number of Receiver Output Peak violations (VH + VL) = 0
Number of total problem noise nets = 38
```

- The following command reports noise for the view named `default_emulate_view`:

```
tempus > report_noise -delay max -threshold 1600 -txtfile delay_max_format.rpt -format
{attacker_slew  attacker_xcap  attacker_status attacker_edge attacker_net stage_info slack
 victim_slew  victim_total_cross_cap victim_net }
*****************************************************
Noise Delay Report (for view default_emulate_view)
*****************************************************
Report Options:
-----------------------------------------------------
-type delay
-threshold 1600 (ps)
-delay max
[-style default]
[-sortBy IncrDelay]
```

```
-------------------------------------------------------
-------------------------------------------------------
Slack(ps)   Slew(ps)   Total    Xcap(fF)    VictimNet
-2.973      0.635      277.574  seg3/u6/A   {seg3/n5}

Stage information:
            Cell       Pin
Driver:     INV        seg3/u5/Y
Receiver:   INV        seg3/u6/A

Constituents:
Slew(ps)   Xcap(fF)   Status   Edge   Net
331.500    138.787    INF      F      seg1/n5
358.667    138.759    SY       F      seg2/n5
-------------------------------------------------------
Slack(ps)  Slew(ps)   Total Xcap(fF)  VictimNet
14.906     0.597      278.115         seg1/u6/A {seg1/n5}

Stage information:
           Cell       Pin
Driver :   INV        seg1/u5/Y
Receiver : INV        seg1/u6/A

Constituents:
Slew(ps)   Xcap(fF)   Status   Edge   Net
279.500    139.280    INF      F      w3
359.333    138.807    INF      F      seg3/n5
-------------------------------------------------------
Slack(ps) Slew(ps)    Total Xcap(fF)  VictimNet
15.737    0.587       270.604         seg1/u12/A {seg1/n11}

Stage information:
           Cell    Pin
Driver :   INV     seg1/u11/Y
Receiver : INV     seg1/u12/A

Constituents:
Slew(ps)   Xcap(fF)   Status   Edge   Net
276.500    135.654    INF      F      w5
337.167    134.517    INF      F      seg3/n11
```

- The following command reports noise for the specified reporting fields:

```
tempus > report_noise -delay max -format {delay incr_delay victim_slew victim_total_cap
victim_total_cross_cap victim_vdd victim_net attacker_offset attacker_slew attacker_xcap
attacker_vdd attacker_edge attacker_status attacker_net} -txtfile custom.rpt
-------------------------------
Delay(NomMax/Max) IncrDelay(ps) Slew(ps) TotalCap(fF) Total Xcap(fF) Vdd(V) VictimNet

-43.400/2418.200  2461.600      12.372   639.088      599.796        1.62   i1/A {n1}
```

```
Type  IncrDelay(ps) Delay(NomMax/Max)  VictimInst

R     3253.300      5457.100/8710.400  drv/Y {INV:A}

Constituents:
Offset(ps)  Slew(ps)  Xcap(fF)  Vdd(V)  Edge  Status  Net
-245.000    30.000    52.145    1.62    F     att1
-199.000    120.000   26.751    0.00    F     INF     ~n1
------------------------------
```

- The following command shows glitch values greater than 0.20 VDD as:

```
tempus > report_noise -threshold 0.20
Output report file name prefix is (null)
Info: Processing report for view default_emulate_view ...
############################################################
# Command: report_noise -threshold 0.20
# View: default_emulate_view
############################################################

Report Options:
-------------------------------------------------------------------------------
-type glitch
-threshold 20% (of vdd)
```

- To generate a glitch report, you can use the following command:

```
tempus > report_noise -txtfile glitch.txt
```

- To generate a glitch report sorted by input noise, you can use the following command:

```
tempus > report_noise -txtfile sort_by_glitch_input.txt -sort_by noise
```

- To generate a glitch failure report, you can use the following command:

```
tempus > report_noise -txtfile glitch_failure.txt -failure
```

- To generate a glitch report for VL noises sorted by input noise, you can use the following command:

```
tempus > report_noise -sort_by noise -level vl
```

- To generate a glitch report for only latch receiver pin cell types, you can use the following command:

```
tempus > report_noise -txtfile latch_glitch.txt -cell_type latch
```

For more information on glitch reporting, refer to "SI Glitch Reporting" in the *Tempus User Guide*.

Related Commands

- report_timing
- update_timing
- report_delay_calculation

report_noise_lib_pin

```
report_noise_lib_pin
[-help]
[> filename]
```

Generates a report providing information on all the user-specified noise mappings in the design.

Parameters

`-help`	Prints out the command usage.
`> filename`	Writes the report to the specified file name. If the file already exists, the software will overwrite it.

Examples

- The following command specifies the library pin name whose definition will be mapped from the given pin name to the given pin name:

  ```
  tempus > set_noise_lib_pin -from PK45_lib_v1.0/A2_X1/B1 -to PK45_lib_v1.0/A2_X1/B2
  ```

  The following command generates a report as follows as per the above definition:

  ```
  tempus > report_noise_lib_pin
  ------------------------------------------------
  from_pin                          to_pin
  ------------------------------------------------
  PK45_lib_v1.0/A2_X1/B1    PK45_lib_v1.0/A2_X1/B2
  ```

- The following command generates the output report ‘noise_mapping.rpt’ which contains all the noise mappings present in the design:

  ```
  tempus> report_noise_lib_pin > noise_mapping.rpt

  noise_mapping.rpt
  ------------------------------------------------
  from_pin                    to_pin
  ------------------------------------------------
  library1/inverter/I         library2/macro/A
  library1/buffer/I           library3/memory/ADD[0]
  ------------------------------------------------
  ```

Related Commands

- set_noise_lib_pin
- reset_noise_lib_pin

report_noise_propagation

```
report_noise_propagation
[-help]
[-level {vl | vh | worst | all}]
[-max_paths integer]
[-max_slack float]
[-nets string]
[-nworst integer]
[-pins string]
[-source_depth integer]
[-views string]
[[ > | >> ]]
```

Generates text-based noise propagation reports.

Note: You need to set `si_glitch_enable_report` attribute to `true` to enable glitch reporting.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man report_noise_propagation`
`-level {vl | vh | worst | all}`	Reports the paths for specified glitch level. The allowed values are: VL, VH, worst, or all.
`-max_paths` *integer*	Reports the specified number of paths.
`-max_slack` *float*	Reports the paths with glitch slack equal to or lesser than the specified value.
`-nets` *string*	Reports the paths with end point being all the input pins connected to the net.
`-nworst` *integer*	Specifies the maximum number of sources per end point for which paths need to be reported.
`-pins` *string*	Reports the paths for the specified end point pins.
`-source_depth` *integer*	Specifies the maximum number of sources to be traced in any given branch in the fanin cone of violation.
`-views` *string*	Reports the paths for the specified views.
`>` *filename*	Redirects output to the specified file.
`>>` *filename*	Appends output to the specified output file.

Examples

- The following command reports Reports the maximum number of paths for glitch level VH for the specified views:

```
tempus > report_noise_propagation -level VH -max_paths 2 -views func_fast_RCMAX

Path 1: VIOLATED RIP failure criteria
Endpoint: PLLCLK_INST/refclk (VH)
Source: PLLCLK_INST/refclk (VH)
Analysis View: func_fast_RCMAX
RIP Threshold                792.00
- Receiver input Peak        1319.70
= Glitch Slack               -527.70
```

```
-----------------------------------------------------------
RIP       Prp       Level   Cell       Pin
-----------------------------------------------------------
1319.70   1965.96   VH      pllclk     PLLCLK_INST/refclk
-----------------------------------------------------------
Path 2: VIOLATED RIP failure criteria
Endpoint: RM_128x16_TST_INST/D[4] (VH)
Source: RM_128x16_TST_INST/D[4] (VH)
Analysis View: func_fast_RCMAX
RIP Threshold                480.00
- Receiver input Peak        873.27
= Glitch Slack               -393.27
-------------------------------------------------------------
RIP     Prp     Level   Cell           Pin
-------------------------------------------------------------
873.27  0.00    VH      ram_256x16A    RM_128x16_TST_INST/D[4]
-------------------------------------------------------------
```

- The following command reports Reports the paths with end point being all the input pins connected to the net, having glitch level VH for the specified views:

```
tempus > report_noise_propagation -nets FE_OF540_data_out_12 -level VH -views func_fast_RCMAX
Path 1: VIOLATED RIP failure criteria
Endpoint: RM_128x16_TST_INST/D[12] (VH)
Source: RM_128x16_TST_INST/D[12] (VH)
Analysis View: func_fast_RCMAX
RIP Threshold                480.00
- Receiver input Peak        579.31
= Glitch Slack               -99.31
-----------------------------------------------------------------
RIP      Prp       Level    Cell            Pin
-----------------------------------------------------------------
579.31   1191.49   VH       ram_256x16A     RM_128x16_INST/D[12]
-----------------------------------------------------------------
```

Related Commands

- report_noise

report_parameter

```
report_parameter
[-help]
[-correlation]
[-name <string>]
```

Generates a report showing `define_parameter` command settings.

Parameters

`-help`	Prints out the command usage.
`-correlation`	Reports the parameter correlation.
`-name <string>`	Reports the parameter name.
`-scope {vtskew | wire_variation | beol | lle layout_effect}`	Reports the parameter scope. Allowed options are: • vtskew • wire_variation • beol • lle layout_effect
`-view <string>`	Shows the name of the specified view name.

Examples

- The following command reports moments information of M2 layer resistance:

```
tempus > define_parameter -name param_M2_R -scope wire_variation -distribution gaussian -range
{1, 1.15}
tempus > report_parameter -scope wire_variation -view func_spg_m4c_cmax
-------------------------------------------------------------------------
                    PARAMETERS ( SCOPE: wire_variation )
-------------------------------------------------------------------------
Name          Mean-shift    Std-dev     Distribution     Analysis Type
-------------------------------------------------------------------------
param_M2_R    0.075         0.025       Normal           Setup
param_M2_R    0.075         0.025       Normal           Hold
-------------------------------------------------------------------------
```

Related Commands

- `define_parameter`

report_voltage_scaling

```
report_voltage_scaling
[-help]
objectListOrCollection
[-analysis_type {early | late}]
[-view string]
[-out_file string]
```

Reports instances whose timing data is interpolated due to voltage mismatch. When a cell is operating in the design at a voltage different from voltages at which it is characterized in the timing libraries, delay calculation (DC) interpolates/derives the timing data at the operating voltage.

Parameters

`-help`	Prints out the command usage.
`-analysis_type {early | late}`	Specifies whether early or late views have to be checked for voltage scaling. If unspecified, only 'late' views are checked.
`-out_file` *`string`*	Specifies the name of the output file to which the report will be written. When the specified file name contains a directory path, the directory hierarchy is automatically created, if not present. If this option is not specified, the report is written to standard output (screen).
`objectListOrCollection`	Specifies the name(s) or a collection of leaf-level instance(s) for which voltage scaling needs to be checked and reported. If not specified, all instances of the design are checked.
`-view` *`string`*	Specify the name(s) of view(s) to be checked for voltage scaling. If unspecified, all active analysis views are checked.

Examples

- The following example shows a sample `report_voltage_scaling` report:

```
tempus 2 > report_voltage_scaling i_6 -analysis_type late -view func_worst_cmax

VIEW: func_worst_cmax -- ANALYSIS TYPE: late
<pin-name> : <cell-name> : <actual-voltage> : <low-lib-name> : <high-lib-name>
i_6/Y      : BUFX5MTH    : 0.820            : elc_08v        : elc_09v
```

  The report is generated with the `func_worst_cmax` view. You can use the `-view` parameter to report for other views. You can run the `report_analysis_views` command to list views.

Related Commands

- report_analysis_views

report_voltage_swing

```
report_voltage_swing
[-help]
-high_threshold float
-low_threshold float
[-out_file string]
[-required_width {all | clock}]
[-view string]
```

Reports the voltage swings that are shorter than the user-specified thresholds for both clock and data signals.

Parameters

`-help`	Outputs a brief description of the `report_voltage_swing` command. For a detailed description of the command, use the man command: `man report_voltage_swing`
`-high_threshold float`	Specifies the high threshold percentage of the pin supply voltage.
`-low_threshold float`	Specifies the low threshold percentage of the pin supply voltage.
`-out_file string`	Specifies the name of the output file.
`-required_width {all | clock}`	Reports the required high and low pulse widths corresponding to the user-specified high and low thresholds. You can specify one of the following options: • `clock`: Reports the required widths for all clock pins in the design. • `all`: Reports the required pulse widths for all input pins in the design. *Default*: `all`
`-view string`	Specifies the name of the view being defined. **Note**: If this option is not specified, it will dump report for all views.

Examples

- The following command shows the total number of voltage swing violations at the specified high and low thresholds:

```
report_voltage_swing -high_threshold 0.6 -low_threshold 0.1


Total number of voltage swing (Rail to Rail) violations : 505
Total number of voltage swing (Rail to Rail) violations : 453
Total number of voltage swing (Rail to Rail) violations : 142
Total number of voltage swing (Rail to Rail) violations : 142
```

- The following command shows the total number of voltage swing violations for the view, `func_slow_RCMAX`, at the specified high and low thresholds:

```
report_voltage_swing -high_threshold 0.6 -low_threshold 0.1 -view func_slow_RCMAX

Total number of voltage swing (Rail to Rail) violations : 505
```

- The following command reports the required pulse widths for all input pins in the design corresponding to the specified high and low thresholds in the `rep` output file:

```
report_voltage_swing -high_threshold  1 -low_threshold 0.9 -required_width all -out_file rep

pinName                                                 isCLK    HighThreshold
LowThreshold   requiredHighWidth(s)   requiredLowWidth(s)    Vdd       Vss

Dumping Voltage Swing report for View : func_slow_RCMAX

TDSP_DS_CS_INST/p214748365A414/A                        No       0.800           0.720
      5.062e-12              -4.050e-12             0.800     0.000

TDSP_CORE_INST/TDSP_CORE_MACH_INST/p214748365A/A        No       0.800           0.720
      5.062e-12              -4.050e-12             0.800     0.000

TDSP_CORE_INST/PROG_BUS_MACH_INST/p214748365A/A         No       0.800           0.720
      5.062e-12              -4.050e-12             0.800     0.00
```

reset_glitch_derate

```
reset_glitch_derate
[-help]
[-derate_height]
[-derate_width]
[-glitch_type string]
[-instance_pin string]
[-offset]
[-pin string]
[-view string]
```

Removes the derating factors for glitch waveforms that were previously set using the `set_glitch_derate` command.

Parameters

`-help`	Prints out the command usage.
`-derate_height`	Resets the derate height.
`-derate_width`	Resets the derate width.
`-glitch_type string`	Specifies the glitch type vl, vh, or both.
`-instance_pin string`	Specifies the instance pin names on which derate factors are to be reset.
`-offset`	Resets the derate offset.
`-pin string`	Specifies the library pin names on which derate factors are to be reset.
`-view string`	Specifies the view name.

Related Commands

- `set_glitch_derate`

reset_noise_lib_pin

```
reset_noise_lib_pin
[-help]
[-all]
[-to string]
```

Allows you to reset any noise property mapping, specified previously using the `set_noise_lib_pin` command.

Parameters

`-help`	Prints out the command usage.
`-all`	Resets all the noise mappings in the design.
`-to` *string*	Specifies the library pin name for which the user-specified noise mapping (if any) needs to be reset. **Note**: The `-to` parameter accepts library cell pin names (that can be obtained using the `get_lib_pins` command). This parameters does not honour collections or instance pin names.

Examples

- The following command resets any noise mapping previously applied to the library pin library1/macro_cell/A :
  ```
  tempus> reset_noise_lib_pin -to library1/macro_cell/A
  ```
- The following command resets all the existing noise mappings in the design:
  ```
  tempus> reset_noise_lib_pin -all
  ```

Related Commands

- `set_noise_lib_pin`
- `report_noise_lib_pin`

set_advanced_variation_mode

```
set_advanced_variation_mode
[-help]
[-disable_arcs <string>]
-scope {vtskew | aging | wire_variation | beol | lle | layout_effect | lde}
```

Sets the advanced variation scope to configure generic effect specific settings.

Parameters

`-help`	Prints out the command usage.
`-disable_arcs <string>`	Disables the generic effect for any arc types, such as, constraint:all, constraint:setup, constraint:hold.
`-scope {vtskew | aging | wire_variation | beol | lle | layout_effect | lde}`	Provides variation specific settings.

Related Commands

- set_vt_skew_derate

set_annotated_glitch

```
set_annotated_glitch
[-help]
[-overshoot string]
[-ports string]
[-undershoot string]
[-view string]
[-voltage_high string]
[-voltage_low string]
```

Allows the ability to annotate glitches at any specified pin/port. This command setting will override the VH/VL glitch with the specified value on the given point and use that glitch value to propagate downstream.

If the defined annotated glitch waveforms do not start and end at 0, then the software will not consider those waveforms and a warning will be issued. In the following example both VL and VH waves should start and end with 0.0 values. Since the VH wave is not ending at 0, this waveform will be ignored.

```
set_annotated_glitch -voltage_low {0 0 100e-12 0.01 200e-12 0.04 300e-12 0.03 600e-12 0} -ports
dff1/CK -voltage_high {0 0 100e-12 0.1 200e-12 0.4 300e-12 0.3 600e-12 0.1} .
```

The software does not save (and report) waveforms below 5% of VDD. Consider for example, the annotated glitch defined on a pin as:

```
set_annotated_glitch -ports dff1/CK -voltage_high {0 0 100e-12 0.01 200e-12 0.04 300e-12 0.03 600e-
12 0}
```

In this case, the glitch peak is 0.04 (40mV), which is below 5% of VDD (1080mV) - equal to 54mV. Since this annotated glitch waveform peak is very low, the `report_noise` command will not report it. However, if the failure criteria is lower than 5%, then the software will save and report small glitch violations.

Parameters

`-help`	Prints out the command usage.
`-ports` *string*	Specifies port or pin names on which you need to apply glitches. This parameter works only when `si_enable_glitch_propagation` attribute is set to `true` in the `report_timing`/`update_timing` flow. In the `update_glitch` flow, the `si_enable_glitch_propagation` attribute is already set to `true`.
`-overshoot` *string*	Specifies a list of VHO glitch wave points.
`-undershoot` *string*	Specifies a list of VLU glitch wave points.
`-view` *string*	Specifies a list of view names.
`-voltage_high` *string*	Specifies a list of VH glitch wave points. You must specify at least 3 time value pairs.
`-voltage_low` *string*	Specifies a list of VL glitch wave points. You must specify at least 3 time value pairs.

Examples

- The following example annotates the specified VL glitch wave points:
```
tempus > set_annotated_glitch -voltage_low {0 0.0  0.2e-9 0.540 0.405e-9 0} -voltage_high
{0.1e-9  0  0.205e-9 0.50 0.405e-9 0.200  0.6e-9 0.0} -ports {A}
```
- The following command overrides the glitch value at RIP with the annotated glitch, because glitch is annotated at the receiver input:
```
tempus > set_annotated_glitch -voltage_low {0 0 100e-12 0.1 200e-12 0.4 300e-12 0.3 600e-12 0}
-ports seg2/u9/D
tempus > update_glitch
tempus > report_noise -nets seg2/n8

------------------------------------------------------------------------
Peak(mV) Level TotalCap(fF) TotalArea Width(ps) Vdd(mV) Library VictimNet
399.924  VL    144.11       110.00    550.104   1080    cell_w  seg2/u9/D {seg2/n8}

Receiver Input Threshold:
Value    (Threshold)  ReceiverNet Receiver failure_type
399.924  (431.244)    seg2/u9/D   (DFF)    [bbox]

Constituents:
Source Peak(mV) Offset(ps) Slew(ps) Xcap(fF) Vdd(mV) Edge Status Net
```

```
Prp:            399.924    --       --       --      --   --     seg2/u8/Y
-------------------------------------------------------------------------
```

- In the following example, the impact of aggressors coupled with the victim stage (along with the annotated glitch) is also considered because annotation is done at the driver output:

```
tempus > set_annotated_glitch -voltage_low {0 0 100e-12 0.1 200e-12 0.4 300e-12 0.3 600e-12 0}
-ports seg2/u8/Y
tempus > update_glitch
tempus > report_noise -nets seg2/n8

-------------------------------------------------------------------------------
Peak(mV) Level TotalCap(fF) TotalArea Width(ps) Vdd(mV) Library  VictimNet
548.208  VL    144.11       159.30    581.172   1080    cell_w   seg2/u9/D {seg2/n8}

Receiver Input Threshold:
Value (Threshold) ReceiverNet Receiver failure_type
548.208 (432.648) seg2/u9/D   (DFF)    [bbox]

Constituents:
Source Peak(mV) Offset(ps) Slew(ps) Xcap(fF) Vdd(mV) Edge  Status  Net
Cpl:           80.148     371.000  340.000  68.138  1080  R ACT   seg3/n8
Cpl:           75.159     381.000  348.500  62.007  1080  R ACT   seg1/n8
Prp:           392.904    --       --       --      --    --      seg2/u8/Y
-------------------------------------------------------------------------
```

Related Commands

- report_noise
- update_glitch
- report_timing
- update_timing

set_cdb_binding

```
set_cdb_binding
[-help]
-from string
-to string
```

Provides an overriding mechanism that allows overriding a UDN to Liberty mapping at the Tempus command file level.

This command helps to correct any mismapped cases by enabling the software to select a user-specified cdB to Liberty mapping.

Parameters

`-help`	Prints out the command usage.
`-from string`	Specifies the name of the CDB file and the cell name.
`-to string`	Specifies the name of the library file.

Examples

- When loading multiple cdB's with the same PVT, but different secondary voltages, you can use the `set_cdb_binding` command to ensure that they get bound correctly, as shown below:

```
set_cdb_binding -from pvt1_1.cdb -to pvt1_1.lib
set_cdb_binding -from pvt1_2.cdb -to pvt1_2.lib
```

set_glitch_derate

```
set_glitch_derate
[-help]
[-derate_height float]
[-derate_width float]
[-glitch_type string]
[-instance_pin string]
[-offset float]
[-pin string]
[-view string]
```

Sets the derating factors for glitch waveforms and determines the effect of on-chip variation on noise.

Parameters

`-help`	Prints out the command usage.
`-derate_height float`	Specifies the derate height value. This parameter scales all the glitch waveforms on the specified pins. A value of 1.0 indicates that no glitch waveform derating will take place. A value greater than 1.0, such as 1.1, indicates a 10 percent increase in the magnitude of the complete glitch waveform, which results in an increase in propagated noise and noise failure. A value less than 1.0 reduces the magnitude of the complete glitch waveform.
`-derate_width float`	Specifies the derate width value.
`-glitch_type string`	Specifies the glitch type - vl, vh, or all.
`-instance_pin string`	Specifies the instance pin names on which derate is to be applied.
`-offset float`	Specifies the derate offset value.
`-pin string`	Specifies the library pin names on which derate value is to be applied.
`-view string`	Specifies the name of the view.

Related Commands

- reset_glitch_derate
- update_glitch

set_glitch_threshold

```
set_glitch_threshold
[-help]
[-cells string]
[-failure_dependency {none | input | last}]
[-failure_point {none | input | internal | last}]
[-glitch_type {both | vl | vh | vlu | vho | vl_vh | vlu_vho }]
[-library_pins <string>]
[-nets string]
[-pins string]
[-pin_type {all | clock | data | latch}]
[-threshold_type {reporting | failure | propagation}]
-value float
[-views string]
```

Allows for additional control of failure criteria for SI glitch analysis.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man set_glitch_threshold`.
`-cells string`	Specifies list of library cell names.
`-failure_dependency {none | input | last}`	Specifies the failure dependency. *Default*: `none`
`-failure_point {none | input | internal | last}`	Specifies where the failure or reporting criteria applies. ◦ `none`: Criteria does not apply. ◦ `input`: Threshold applies when the failure point is at the input pin of the receiver cell. ◦ `internal`: Threshold is applied when the failure point is internal to the cell. This occurs for multi-stage cells. The failure point will be the output of the first stage inside the cell. ◦ `last`: Threshold is applied when the failure point is the output of the cell. This occurs for single-stage cells, such as, inverters. *Default*: `internal` and `last`

`-glitch_type {both | vl | vh | vlu | vho | vl_vh | vlu_vho}`	Specifies the glitch type the command applies to. • `both`: Refers to `vl` and `vh` type of glitches. • `vl_vh`: Includes VL and VH noise. • `vlu_vho`: Includes VLU and VHO noise. *Default*: `both`
`-library_pins` *string*	Specifies a list of library pin names.
`-nets` *string*	Specifies the list of nets the command applies to. Default is not net-specific.
`-pins` *string*	Specifies the list of pins the command applies to. The default is not pin-specific.
`-pin_type {clock |data | latch |all}`	Specifies the pin type the command applies to. *Default*: all
`-threshold_type {failure | reporting | propagation}`	Specifies whether the command options apply to failure, reporting, or propagation criteria. In case of VL/VH glitch analysis, it is mandatory to specify the `-threshold_type` parameter. This is because when the computed glitch (ROP) at any given stage exceeds the failure threshold (when glitch propagation is enabled), then it is not propagated and is flagged as failure in the noise report. With the `-threshold_type failure` setting, the propagation will stop as per the new threshold, and the failure flagging will change accordingly. However, with the `-threshold_type reporting` setting, the propagation threshold remains unchanged, but the failure reporting is updated. When `-threshold_type propagation` is used, the software allows for failing nets with ROP values less than the propagation threshold to propagate through single stage cells. In case of VLU/VHO glitch analysis, it is optional to use the `-threshold_type` parameter because the above flow is not applicable. *Default*: `failure`
`-value` *float*	Specifies the value for the threshold as a ratio of the local VDD. *Default*: 0.1
`-views` *string*	Specifies a list of view names.

Examples

- The following command moves the failure point from ROP to RIP for `inst1/a1` and uses a RIP failure of *0.2*ReceiverVDD*:

  ```
  set_glitch_threshold -failure_point input -pin inst1/a1 -threshold_type failure -value 0.2
  ```

- The following command sets clock failure criteria to 0.1*VDD, latch failure criteria to 0.2*VDD and all other nets to 0.3*VDD (set before update_glitch):

  ```
  tempus> set_glitch_threshold -threshold_type failure -pin_type all -failure_point last -value
  0.3
  tempus> set_glitch_threshold -threshold_type failure -pin_type clock -failure_point last -
  value 0.2
  tempus> set_glitch_threshold -threshold_type failure -pin_type latch -failure_point last -
  value 0.1
  ```

- The following command sets the failure threshold at the output of the 1st channel connect inside all the instances of macro1 to be 0.2 (set before update_glitch):

  ```
  tempus> set_glitch_threshold -threshold_type failure -cell {macro1} -failure_point internal -
  value 0.2
  ```

- The following command changes the reporting criteria for latches to be 0.15*VDD (set after `update_glitch`):

  ```
  tempus> set_glitch_threshold -threshold_type reporting -pin_type latch -value 0.15
  ```

  If you need to set all the data failure criteria to be 0.3, then you need to make the following settings:

  ```
  tempus> set_glitch_threshold -failure_point internal \
  -threshold_type failure -value 0.3 \
  -pin_type data

  tempus> set_glitch_threshold -failure_point last \
  -threshold_type failure -value 0.3 \
  -pin_type data
  ```

Related Commands

- `update_glitch`

set_model_priority

```
set_model_priority
[-help]
[-objects string]
[-priority_list string]
```

Allows you to define the priority order of models.

Parameters

`-help`	Prints out the command usage
`-objects string`	Specifies a list of library cells or library pins for which priority is specified.
`-priority_list string`	Specifies a list of pin model priority. Default: `{ecsm ccs nldm cdb udn}`

Examples

- To force CCSN to be used, you can set `timing_library_read_ccs_noise_data` attribute to `true` before reading the timing libraries and then use the following command before using the `update_glitch` command:

  ```
  set_model_priority -priority_list ccs
  ```

Related Commands

- `update_glitch`

set_mutex_condition

```
set_mutex_condition
[-help]
[netNames]
```

Allows the software to use one of the specified attackers - each attacker is evaluated and the worst one (for each victim net) is selected.

This command applies a mutex group to a set of attackers (for a victim) to reduce pessimism. A mutex group is a group of nets of which only one can transition in a given cycle.

Parameters

`-help`	Prints out the command usage.
netNames	Specifies the net names of the mutex group.

Examples

- The following example shows that only one of the specified nets, between abc[0-3], can switch at any given point of time:

  ```
  tempus> set_mutex_condition {abc[0] abc[1] abc[2] abc[3]}
  ```

  Each of the specified attackers will be evaluated and the worst one will be used.

Related Commands

- report_mutex_condition

set_noise_lib_pin

```
set_noise_lib_pin
[-help]
-from string
-to string
```

Allows you to map noise properties of an instance pin to that of different library cell pin.

Parameters

`-help`	Prints out the command usage.
`-from` *string*	Specifies the library pin name whose definition will be mapped from the given pin name.
`-to` *string*	Specifies the library pin name whose definition will be mapped to the given pin name. When multiple `set_noise_lib_pin` commands are run on the same `-to` pin, the software will honor the first command only. **Note**: The `-from` and `-to` parameters also accept library cell pin names (that can be retrieved using the `get_lib_pins` command). These parameters do not honor collections or instance pin names.

Examples

- The input pin A of the `macro_cell` cell inherits the noise characteristics of the input pin I of the library cell, `ccs_noise_lib/inv`. This information is used to calculate the noise immunity of the pin, `library1/macro_cell/A`.
  ```
  tempus> set_noise_lib_pin -to library1/macro_cell/A -from ccs_noise_lib/inv/I
  ```
- The following syntax specifies the library pin name whose definition will be mapped from the given pin name to the given pin name:
  ```
  tempus> set_noise_lib_pin -from [from_lib/from_cell/from_pin] -to [to_lib/to_cell/to_pin]
  ```
- The following syntax specifies that a bidi pin is mapped to two from pins; that is, – 1 input pin and 1 output pin.
  ```
  tempus> set_noise_lib_pin -to library1/macro_cell/PAD -from "ccs_noise_lib/inv/I
  ccs_noise_lib/inv/Z"
  ```

Related Commands

- `report_noise`

set_outbound_report

```
set_outbound_report
[-help]
-delay {early | late | both}
[-out_file_prefix string]
[-report_constant_nets {true | false}]
[-type {lower | upper | both}]
```

Controls what extrapolation information is to be saved in the log file(s). The slew extrapolation information can be stored into separate files.

Parameters

`-help`	Prints out the command usage
`-delay {early | late | both}`	Control whether extrapolation information is to be shown for min or max (or both) analyses. *Default*: `both`
`-out_file_prefix string`	Specify the prefix to be used in the name(s) of the report file(s). *Default*: `design-name`
`-report_constant_nets {true | false}`	Controls whether extrapolation information is to be shown for constant nets or not.
`-type {lower | upper | both}`	Controls whether extrapolation information is to be shown for lower or upper (or both) outbound slews.

Examples

The following command will will dump slew extrapolation information for maximum analysis into the myreport.dc.outbound.slew.late.<view-name>.txt file:

```
> set_outbound_report -delay late -out_file_prefix myreport
```

set_quiet_aggressor

```
 set_quiet_aggressor
[-help]
[-analysis_type {early | late}]
[-aggressor string]
[-transition {rise | fall}]
[-victim string]
[-view string]
```

Specifies one or more coupled nets as non-attackers (silent) for the delay and glitch analysis of a given victim net.

You can run the `update_glitch` or `update_timing` command and view the results in the `report_noise` or `report_timing` command output.

Parameters

`-help`	Prints out the command usage.
`-analysis_type {early | late}`	Specifies the analysis type for which the attackers will be assumed quiet. If no value is specified, attackers are skipped for both `early/late` analysis.
`-aggressor string`	Specifies the attackers which should be made quiet for given victim net(s). If no attacker is specified, all the aggressors of given victim(s) will be ignored. **Note**: If no victim is specified, the specified attackers are made quiet for all victim nets in the design.
`-transition {rise | fall}`	Specifies the victim transition for which attackers will be assumed quiet. If no value is specified, attackers are skipped for both `rise` and `fall` transitions of the victim net.
`-victim string`	Specifies a list of victim nets for which attackers should be made quiet. Specific attackers can be skipped using the `-aggressor` option of this command. If the `-aggressor` option is not specified, all the attackers of the given nets are made silent and SI analysis will not be performed on these nets. **Note**: If this option is not specified, the attackers specified in the `-aggressor` option are made quiet for all the nets in the design.
`-view string`	Specifies the view for which attackers should be made quiet. If no view is specified, attackers are assumed quiet in all views

Examples

Consider a design having an attacker `agg` to victim nets `vic1` and `vic2`. The following command sets the attacker `agg` as quiet for both the victim nets `vic1` and `vic2`.

```
tempus > set_quiet_aggressor -aggressor {agg}
```

- The following command sets the attackers A1 and A2 as quiet when V1 is rising during late analysis.

  ```
  tempus > set_quiet_aggressor -victim V1 -aggressor A1 A2 -transition rise -analysis_type late
  ```

- The following command sets the attacker A1 quiet for victim V1 in the analysis view VIEW1. In this case, A1 may be a valid attacker during SI analysis for victim V1 in views other than VIEW1.

  ```
  tempus > set_quiet_aggressor -victim V1 -aggressor A1 -view {VIEW1}
  ```

Consider a design having victim `vic` with attackers `agg1` and `agg2`:

- The following command sets both the attackers `agg1` and `agg2` as quiet for the victim net `vic`.

  ```
  tempus > set_quiet_aggressor -victim {vic}
  ```

- The following command sets the attacker `agg1` as quiet for the given victim net `vic`

  ```
  tempus > set_quiet_aggressor -aggressor {agg1} -victim {vic}
  ```

Related Commands

- `report_noise`
- `report_timing`

set_via_variation_mode

```
set_via_variation_mode
[-help]
[-reset]
[-corner_sigma_multiplier <std::string>]
[-correlation_type {corner}]
[-reset {true | false}]
[-separate_params_for_via {true | false}]
```

Allows you to scale via resistance from SPEF based on user-specified scaling factor.

This command allows you to specify per via sigma multiplier and handles the advanced via variation scaling.

Parameters

`-help`	Prints out the command usage
`-corner_sigma_multiplier <std::string>`	Provides the scaling multiplier for corner scaling. *Default:* " "
`-correlation_type {corner}`	Specifies inter-layer correlation values. *Default:* `corner`
`-reset`	Resets all or specified options to their default value.
`-reset {true | false}`	Resets via variation modes to their default value. *Default*: `false`
`-separate_params_for_via {true | false}`	Allows to create separate parameters for via variation. *Default:* `false`

Examples

- The following command uses correlation corner for scaling purpose:
  ```
  set_via_variation_mode -view view1 -correlation_type corner -corner_sigma_multiplier
  {{{param_V1 -3} {param_V2 -3}..}
  ```

  where V1, V2 are via name and “-3” is used as scaling factor.

- The following command will scale the via resistance as shown below:

```
set_via_variation_mode -rc_variation false -corner_sigma_multiplier {{{param_V1  2} {param_V2
  2}..}
```

  Via Calculation below:

```
Via1_Rnew = Via1_Rspef *(1 + 0.021 + 2*0.110)
Via2_Rnew = Via2_Rspef *(1 + 0.010 + 2*0.120)


                  Via1:
table_meanshift: 0.021
table_stdev :    0.110
                  Via2:
table_meanshift: 0.010
table_stdev :    0.120
```

Related Commands

- get_via_variation_mode

si Category Attributes

The root attributes in the **si** category are:

- si_accumulated_small_aggressor_factor
- si_accumulated_small_aggressor_mode
- si_accumulated_small_aggressor_threshold
- si_aggressor_alignment
- si_clock_synchronicity
- si_delay_clock_delta_threshold
- si_delay_delta_threshold
- si_delay_delta_annotation_mode
- si_delay_enable_double_clocking_check
- si_delay_enable_logical_correlation
- si_delay_enable_report
- si_delay_only
- si_delay_separate_on_data
- si_delay_double_switching
- si_enable_bus_attacker_correlation
- si_enable_drv_with_delta_slew
- si_enable_glitch_overshoot_undershoot
- si_enable_glitch_propagation
- si_enable_glitch_propagation_spice_deck
- si_enable_lle_glitch
- si_enable_two_stage_driver_weakening
- si_max_virtual_attacker_constituents
- si_enable_user_defined_bus_merging
- si_enable_virtual_attacker_constituent_report
- si_glitch_accuracy_level
- si_glitch_analysis_type
- si_glitch_constrained_input_threshold_failure_point
- si_glitch_enable_dynamic_receiver_peak_limits

- si_glitch_enable_report
- si_glitch_input_full_propagation_threshold
- si_glitch_input_full_propagation_voltage_high_threshold
- si_glitch_input_full_propagation_voltage_low_threshold
- si_glitch_input_threshold
- si_glitch_input_voltage_high_threshold
- si_glitch_input_voltage_low_threshold
- si_glitch_receiver_clock_peak_limit
- si_glitch_receiver_latch_peak_limit
- si_glitch_receiver_peak_limit
- si_individual_aggressor_simulation_filter
- si_individual_aggressor_threshold
- si_individual_aggressor_clock_threshold
- si_nonlinear_aggressor_slew
- si_num_iteration
- si_reselection
- si_reselection_delay_threshold
- si_reselection_hold_slack
- si_reselection_setup_slack
- si_secondary_attacker_decoupling_factor
- si_skip_noise_model_check
- si_skip_timing_window
- si_switch_probability
- si_unconstrained_net_use_infinite_timing_window
- si_use_infinite_timing_window
- si_virtual_inter_die_attacker_slew

To set the value of any root attribute, use the `set_db` command:

- `set_db` *`attribute_name value`*

To get the current value of any root attribute, use the `get_db` command:

- `get_db` *`attribute_name`*

To get the current value of all attributes in the `assign_pins` category, use the following command:

- `get_db -category si`

The SI attributes can be reported by using the following command:

- `get_db si_*`

si_accumulated_small_aggressor_factor

`si_accumulated_small_aggressor_factor` *`double`*

Default: `1.0`

Data_type: Double

Specifies the multiplication factor that controls the effect of the cap mode accumulated small attacker. An accumulated small attacker is an imaginary attacker that represents the sum of all the weak attackers on a specified net. To reduce the pessimism of using the accumulated small attacker, adjust this value to anything between 0 and 1. A value of 0 means that the cap mode accumulated small attacker will not be used.

Note: This option does not work with the current mode accumulated small attacker, which is default for process sizes 45nm and below.

si_accumulated_small_aggressor_mode

`si_accumulated_small_aggressor_mode {cap | current | zero_mean | numerical}`

Default: `{}`

Data_type: Enum

Specifies whether to use the current matching based method for accumulated small attackers. You can specify the following options:

- `cap`: Uses the accumulated small attacker method, which matches the total coupling capacitance of an attacker with the small attackers that are being virtualized. This is the default mode for all process nodes above 45nm.
- `current`: Uses the current matching based method for accumulated small attackers. With this method, the transitions on virtual attackers are modeled using Piece Wise Linear (PWL) waveforms. The PWL waveforms are generated in a way that the current induced by the transition matches the total current induced by all accumulated small attacker components.
- `zero_mean`: Uses the accumulation method where series of small attackers (that is, attackers with less than the individual attacker threshold value), are assumed to transition in opposing directions and cancel one another in aggregate.

Note: For 45nm and below process nodes, the default value is `current`.

si_accumulated_small_aggressor_threshold

```
si_accumulated_small_aggressor_threshold double
```

Default: 10.01

Data_type: Double

Sets the accumulated small attacker glitch threshold. If the virtual attacker produces a glitch whose peak is below this threshold, the component coupling capacitors of the accumulated small attacker are split into two ground capacitors and there is no accumulated small attacker for the victim. The value is specified as a ratio of VDD. To disable the accumulated small attacker, you must set a value greater than 1.

si_aggressor_alignment

```
si_aggressor_alignment {path | path_overlap | timing_aware_edge | input_arrival}
```

Default: `input_arrival`

Data_type: Enum

Specifies the attacker alignment. You can specify the following options:

- `path`: The victim worst arrival edge is used while determining the attackers. Crosstalk impact is computed according to the latest/earliest arrival edge of victim for max/min analysis.
- `path_overlap`: The full timing window of victim is considered while determining the attackers. Crosstalk impact is calculated according the victim edge, lying anywhere in timing window, which may experience maximum impact.
- `timing_aware_edge`: Realistic analysis similar to path-based analysis is performed. Each attacker's impact is annotated on victim transition where attacker timing window edge aligns with the victim. However, to ensure GBA bounds PBA analysis, the timing-aware edge mode handles some nets based on their timing properties in GBA analysis.
- `input_arrival`: The attackers use timing windows formed by 2-sigma arrivals regardless of the analysis sigma multiplier. The victim driver input arrivals will be used for SI computation. Also, the mean victim timing window and edge will be used for SI computation in GBA and PBA modes, respectively. This behavior allows pessimism reduction in GBA SI analysis with narrower timing windows. And provides a more accurate PBA SI analysis, with no dependency of driver cell GBA delays.
  For more information, refer to the Attacker Selection Based on Alignment section in the *Tempus Stylus User Guide*.

si_clock_synchronicity

```
si_clock_synchronicity {asynchronous | synchronous}
```

Default: `synchronous`

Data_type: Enum

Treats the clocks either synchronous or asynchronous to each other.

si_delay_clock_delta_threshold

`si_delay_clock_delta_threshold` *double*

Default: `-1.0`

Data_type: Double

Specifies the clock delta delay analysis threshold value.

si_delay_delta_threshold

`si_delay_delta_threshold` *double*

Default: `-1.0`

Data_type: Double

Specifies the delta delay analysis threshold value.

si_delay_delta_annotation_mode

`si_delay_delta_annotation_mode {lumpedOnNet | arc}`

Default: `arc`

Data_type: Enum

Calculates the delay for each arc (cell arc and net arc) in a path. By default, AAE calculates the delay, including SI for each arc and if the user chooses to view the SI delay separately, it would show annotated to each arc. Alternatively, you can choose to annotate the SI delay solely to the net. In this case, the worst-case arc is used to calculated the SI delay and it is annotated solely to the net.

- `lumpedOnNet`: Lumps the delta delay on a the net, if it is broken out.
- `arc`: Separates the delta delay on each cell arc and net arc, as calculated.

si_delay_enable_double_clocking_check

`si_delay_enable_double_clocking_check {true | false}`

Default: `false`

Data_type: Boolean

Enables double clocking check in AAE.
This option enables double clocking analysis and `report_double_clocking` command can be run to print a

report.

si_delay_enable_logical_correlation

```
si_delay_enable_logical_correlation {true | false}
```

Default: `false`

Data_type: Boolean

Enables the AAE-SI logical correlation. Logical correlation refers to correlation of logic between (aggressor) nets which are connected to each other. A simple case would be input and output nets of an invertor never switch in the same direction, so cannot be attackers to the same net simultaneously.

si_delay_enable_report

```
si_delay_enable_report {true | false}
```

Default: `false`

Data_type: Boolean

Enables SI delay reports.

si_delay_only

```
si_delay_only {true | false}
```

Default: `false`

Data_type: Boolean

Performs noise-on-delay analysis exclusively. This means that when you use this parameter, the software does not perform glitch analysis and only performs noise-on-delay analysis. In addition, this parameter performs SI analysis.

si_delay_separate_on_data

```
si_delay_separate_on_data {true | false}
```

Default: `false`

Data_type: Boolean

Separates incremental delays on signal nets.

si_delay_double_switching

```
si_delay_double_switching {none | clock | data_sequential | all}
```

Default: `none`

Data_type: Enum

Specifies the type of data (clock, data sequential, or all) for which double switching analysis will be performed.

si_enable_bus_attacker_correlation

```
si_enable_bus_attacker_correlation {true | false}
```

Default: `false`

Data_type: Boolean

Enables the software to treat bits of bus as correlated attackers.

When set to `true`, the bits of bus type nets are grouped together based on their cumulative weight.

The bus bits that act as attackers to a specific victim are treated as a group. All the attackers are grouped into different bundles where normal (non-bus) attackers form one group and bits of the same bus form multiple other groups. In case of a victim that has a number of small couplings to bits of a bus, the combined effect of a full bus can be used to decide whether an attacker is significant.

si_enable_drv_with_delta_slew

```
si_enable_drv_with_delta_slew {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the `max_transition` violations are reported based on the SI slews. Also, this allows SI slew reporting/fixing by the `opt_signoff` command and is applied prior running the `write_eco_opt_db` command.

si_enable_glitch_overshoot_undershoot

```
si_enable_glitch_overshoot_undershoot {true | false}
```

Default: `false`

Data_type: Boolean

Enables glitch overshoot/undershoot.

si_enable_glitch_propagation

```
si_enable_glitch_propagation {true | false}
```

Default: `false`

Data_type: Boolean

Enables glitch propagation through single stage cells and driver weakening through single stage and dual stage cells. When this attribute is set to `true`, the software does not delete the glitch waveform saved on instance input pins. The waveform is later used by spice deck to propagate glitch at the next stage receiver pin.

si_enable_glitch_propagation_spice_deck

```
si_enable_glitch_propagation_spice_deck {true | false}
```

Default: `false`

Data_type: Boolean

Enables the driver input glitch waveform to be printed in the SPICE deck such that SPICE can also perform single stage glitch propagation. When this option is set to `true`, the software does not delete the glitch waveform saved on instance input pins. The waveform is later used by spice deck to propagate glitch at the next stage receiver pin.

si_enable_lle_glitch

```
si_enable_lle_glitch {true | false}
```

Default: `false`

Data_type: Boolean

Enables SI glitch analysis for LLE.

si_enable_two_stage_driver_weakening

```
si_enable_two_stage_driver_weakening {true | false}
```

Default: `true`

Data_type: Boolean

Enables the two-stage driver weakening mode.

A non-failing significant glitch at the input of a two-stage driver can propagate through the first stage and weaken the second stage. This might cause the crosstalk-induced glitch to amplify at the next stage receiver (though it may not be significant enough to propagate). When two-stage driver weakening is enabled, the software considers weakened holding resistance for the two-stage driver for glitch analysis.

The dual stage driver weakening honors the glitch propagation threshold criteria.

Note: This feature is supported with CCS-N models only.

si_max_virtual_attacker_constituents

```
si_max_virtual_attacker_constituents integer
```

Default: `5`

Data_type: Integer

Specifies the maximum number of virtual attacker constituents to report.

si_enable_user_defined_bus_merging

```
si_enable_user_defined_bus_merging {true | false}
```

Default: `false`

Data_type: Boolean

Enables the bus merging feature. If a net is in two groups, then the groups will be merged.

si_enable_virtual_attacker_constituent_report

```
si_enable_virtual_attacker_constituent_report {true | false}
```

Default: `false`

Data_type: Boolean

Enables virtual attacker constituent reporting.

The software provides support for virtual attacker constituent reporting (when the bus attacker correlation feature is enabled using the `si_enable_bus_attacker_correlation` attribute.

The virtual attacker constituent reporting feature is useful for identifying significant buses and their bits - this information can be used for glitch fixing.

si_glitch_accuracy_level

```
si_glitch_accuracy_level int
```

Default: `0`

Data_type: Integer

Specifies the accuracy levels for glitch analysis.

si_glitch_analysis_type

```
si_glitch_analysis_type {source | full_propagation}
```

Default: `source`

Data_type: Enum

Specifies the glitch analysis type - `source` or `full_propagation`.

- `full_propogation`: Propagates glitch noise through single and 2-stage cells with CCSN (for example, inv or buf), and stops glitch propagation at any sequential cell, or 3 or more stage cell input.

- `source`: Propagates glitch through 1-stage cells only, and performs driver weakening through 2-stage cells with CCSN.

si_glitch_constrained_input_threshold_failure_point

```
si_glitch_constrained_input_threshold_failure_point {input | both}
```

Default: `input`

Data_type: Enum

Allows you to control the glitch failure criteria when a user specifies an object-specific RIP threshold.

When set to `input`, the software skips ROP analysis and performs RIP-based failure check on the receiver pins for cases where object-specific RIP threshold is set using the `set_glitch_threshold -failure_point input -net` (or `-pin`) command.

When set to `both`, the software performs both RIP and ROP failure checks, wherever possible. If user-specified RIP failure criteria is violated, then it will be flagged as a RIP violation. If RIP criteria passes and receiver noise models exist, then the ROP is computed and checked against the ROP failure threshold for failure determination.

For the following cases where the object-specific RIP thresholds are not specified, the attribute behavior remains unchanged:

- For receiver pins with noise models present, ROP-based failure check is performed. The RIP-based failure criteria is not checked at all.
- For receiver pins with noise models missing, the RIP-based failure criteria is checked for violations.

si_glitch_enable_dynamic_receiver_peak_limits

```
si_glitch_enable_dynamic_receiver_peak_limits {true | false}
```

Default: `false`

Data_type: Boolean

Enables automatic computation of ROP threshold value of two-stage cells, using the CCS-noise data on timing arcs.

When set to `true`, the software computes the noise threshold of the last stage from the table of DC currents.

si_glitch_enable_report

```
si_glitch_enable_report {true | false}
```

Default: `false`

Data_type: Boolean

Enables the storage of glitch information for further debug using the `report_noise` command.

When set to `false`, the `time_design` command will still report glitches in the timingReports/*.SI_Glitches.rpt file but the report_noise command will not be available for use.

When set to `true`, the `report_noise` command may be used and glitch information will be more aligned to Tempus signoff values.

si_glitch_input_full_propagation_threshold

`si_glitch_input_full_propagation_threshold` *`double`*

Default: 0.75

Data_type: Double

Specifies the receiver input glitch threshold beyond which Tempus will not propagate the glitch in full-propagation mode, and an RIP violation will be flagged.

si_glitch_input_full_propagation_voltage_high_threshold

`si_glitch_input_full_propagation_voltage_high_threshold` *`double`*

Default: 0.75

Data_type: Double

Specifies the receiver VH input glitch threshold beyond which Tempus will not propagate the glitch in full-propagation mode, and an RIP violation will be flagged.
This attribute takes precedence over `si_glitch_input_full_propagation_threshold`.

si_glitch_input_full_propagation_voltage_low_threshold

`si_glitch_input_full_propagation_voltage_low_threshold` *`double`*

Default: 0.75

Data_type: Double

Specifies the receiver VL input glitch threshold beyond which Tempus will not propagate the glitch in full-propagation mode, and an RIP violation will be flagged.
This option takes precedence over `si_glitch_input_full_propagation_threshold`.

These attributes are applicable only when the full propagation mode is enabled (using the `si_glitch_analysis_type full_propagation` attribute).

si_glitch_input_threshold

`si_glitch_input_threshold` *`double`*

Default: 0.4

Data_type: Double

Specifies the input glitch failure threshold value during AAE analysis.

si_glitch_input_voltage_high_threshold

`si_glitch_input_voltage_high_threshold` *`double`*

Default: `0.4`

Data_type: Double

Specifies the input glitch vh failure threshold value during AAE analysis. The value is the fraction of local VDD – if VDD is 0.9V and the threshold is 0.4, then the criteria for failure is 0.4 * 0.9=0.36V.

During timing window re-convergence, only certain nets are reselected for glitch analysis in each iteration (if any of the above input glitch threshold parameters is specified). The criteria for glitch reselection is based on the specified input glitch threshold value, if this value is smaller than 30% of VDD. If not, then all the glitches larger than 30% of VDD are selected for re-computation in each iteration.

si_glitch_input_voltage_low_threshold

`si_glitch_input_voltage_low_threshold` *`double`*

Default: 0.4

Data_type: Double

Specifies the input glitch vl lower threshold value during AAE analysis. The value is a fraction of the local VDD - if VDD is 0.9V and the threshold value is 0.4, then the criteria for failure is 0.4 * 0.9=0.36V.

During timing window re-convergence, only certain nets are re-selected for glitch analysis in each iteration (if any of the above input glitch threshold parameters is specified). The criteria for glitch re-selection is based on the specified input glitch threshold value, if this value is smaller than 30% of VDD. If not, then all the glitches larger than 30% of VDD are selected for re-computation in each iteration.

si_glitch_receiver_clock_peak_limit

`si_glitch_receiver_clock_peak_limit` *`double`*

Default: `0.05`

Data_type: Double

Defines the glitch check limit for latch inputs in receiver peak mode. This parameter is specified as a ratio of VDD.

si_glitch_receiver_latch_peak_limit

`si_glitch_receiver_latch_peak_limit` *`double`*

Default: `0.1`

Data_type: Double

Defines the receiver peak/glitch peak limit at all nodes. The value specified for this parameter is a ratio of VDD.

si_glitch_receiver_peak_limit

```
si_glitch_receiver_peak_limit double
```

Default: `0.15`

Data_type: Double

Defines the receiver peak/glitch peak limit.

si_individual_aggressor_simulation_filter

```
si_individual_aggressor_simulation_filter {true | false}
```

Default: `true`

Data_type: Boolean

Uses electrical filtering to reduce pessimism due to small attackers being preserved when their simulated value is under the individual attacker threshold value.

Note: This option can only be used when the accumulated small attacker is turned off (`-accumulated_small_attacker_threshold` is `1.00` or larger).

si_individual_aggressor_threshold

```
si_individual_aggressor_threshold double
```

Default: 1.5% or 0.015

Data_type: Double

Sets the glitch tolerance for individual attacker nets on the victim net. Tempus estimates the noise contributed by an individual attacker on the victim. If the peak of this noise exceeds the value of `individual_attacker_threshold`, the attacker is significant and is individually reported in the constituents section of the report. Otherwise, this aggressor becomes eligible for the virtual attacker. If the virtual attacker peak does not exceed the value of `accumulated_small_attacker_threshold`, the aggressor nets in the virtual attacker are grounded and there is no virtual attacker.

Note: Minimum allowed individual attacker to be kept is `0.1%` of VDD, which is equivalent to an `individual_attacker_threshold` of `0.001`.

si_individual_aggressor_clock_threshold

```
si_individual_aggressor_clock_threshold double
```

Default: 0.015

Data_type: Double

Defines the AAE virtual attacker clock threshold.

si_nonlinear_aggressor_slew

```
si_nonlinear_aggressor_slew {true | false}
```

Default: `true`

Data_type: Boolean

Uses a non-linear attacker. If set to false, the software uses a linear attacker.

si_num_iteration

```
si_num_iteration int
```

Default: `2`

Data_type: Integer

Specifies the maximum number of timing window iterations that should be performed during SI delay analysis. Performing delay analysis results in changes to the timing windows, which can potentially change the victim/attacker net combinations. Timing window iterations enable the software to reach to a point where the delay changes are negligible.

Note: The minimum allowed timing window iterations is 2 for performing SI analysis.

si_reselection

```
si_reselection {delta_delay | slack | xcap_ratio}
```

Default: `slack`

Data_type: Enum

Specifies the SI reselection criteria during timing window iterations.

- `delta_delay`: Reselects a net for calculation if the delta delay on the net is greater than the delay value.
- `slack`: Reselects nets that have failing slacks during timing window iterations.
- `xcap_ratio`: Specifies the xcap ratio.

si_reselection_delay_threshold

```
si_reselection_delay_threshold double
```

*Default: 1e-11 (*seconds)

Data_type: Double

Allows you to change the delay reselection criteria.

si_reselection_hold_slack

`si_reselection_hold_slack` *`double`*

*Default: -1e-12 (*seconds)

Data_type: Double

Specifies the slack reselection threshold for hold.

si_reselection_setup_slack

`si_reselection_setup_slack` *`double`*

Default: -1e-12 (seconds)

Data_type: Double

Specifies the slack reselection threshold for setup.

si_secondary_attacker_decoupling_factor

`si_secondary_attacker_decoupling_factor` *`double`*

Default: `1.0`

Data_type: Double

Specifies decoupling factor for secondary aggressors.

si_skip_noise_model_check

`si_skip_noise_model_check` *`string`*

Default: " "

Data_type: String

Specifies the list of pins for which noise model check will not take place.

si_skip_timing_window

`si_skip_timing_window` *`string`*

Default: " "

Data_type: String

Specifies the list of nets for which timing window is to be skipped. The timing window becomes infinite for the net when it is both a victim and aggressor.

When specified, only the last setting is retained.

si_switch_probability

```
si_switch_probability double
```

Default: 0.3

Data_type: Double

Sets the attacker switching probability. If you specify a value less than 0.0 or more than 1.0 , the switching probability is automatically set to the default value of 0.3.

si_unconstrained_net_use_infinite_timing_window

```
si_unconstrained_net_use_infinite_timing_window {true | false}
```

Default: `true`

Data_type: Boolean

Specifies that unconstrained nets will have an infinite timing window. To turn this off and use a constrained timing window, set this to `false`.

si_use_infinite_timing_window

```
si_use_infinite_timing_window {true | false}
```

Default: `false`

Data_type: Boolean

Uses infinite timing windows in a single SI run.

si_virtual_inter_die_attacker_slew

```
si_virtual_inter_die_attacker_slew value
```

Default: `2.5e-11`

Data_type: Float

Specifies the virtual inter-die attacker slew value.

update_glitch

```
update_glitch
[-help]
[-effort {medium | high}]
```

Allows you to run glitch analysis without running SI delay analysis. This command provides the ability to run glitch analysis on all the nets in a design or on a subset of nets.

Using the `update_glitch` command without `report_timing` or `update_timing` will only run glitch analysis.

Note: It is not recommended to run the `update_glitch` command after the `report_timing` or `update_timing` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `update_glitch` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man update_glitch`.
`-effort {medium | high}`	Specifies the effort level for update glitch. When set to `high`, glitch analysis uses more accurate modeling of glitch for small attackers, thereby reducing pessimism. **Note**: The run time in `high` mode may be slightly higher, and is largely dependent on the distribution of coupling. *Default*: `medium`

Examples

- The following command will run glitch analysis without SI delay:
  ```
  > update_glitch
  ```

Related Commands

- `report_noise`
- `report_timing`
- `update_timing`

write_noise_eco_nets

```
write_noise_eco_nets
[-help]
out_file
[-problem_nets <string>]
```

Tempus performs ECO analysis for noise failures. The default ECO file is named `glitch`.

Note: The `write_noise_eco_nets` command should always be called after glitch analysis has been performed.

Parameter

`-help`	Prints out the command usage.
out_file	Specifies the name for the ECO file. By default, the file name is `glitch`.
`-problem_nets <string>`	Reports on the specified list of net names.

Examples

The following command generates an ECO file for all the glitch failures.

```
tempus> write_noise_eco_nets glitch_eco.txt
```

Related Commands

- `report_noise`

write_spice_deck

```
write_spice_deck
[-help]
[-add_driver_for_constraint_check {true | false}]
[-add_side_load_measurement {true | false}]
[-add_voltage_for_missing_sensitization]
[-aging_correlation_type {full | timingonly}]
[-aging_stress_num_cycles <integer greater than 0>]
[-aging_stress_use_timing_slew {false | true}]
[-aging_level_only <string>]
[-bbox_cells string]
[-constraint_check {true | false}]
[-clock_mesh_depth integer]
[-clock_mesh_from_fall_pin string]
[-clock_mesh_from_pin string]
[-clock_mesh_from_rise_pin string]
[-clock_mesh_no_hard_stop_at_to_pins {true | false}]
[-clock_mesh_to_pin string]
[-enable_fast_simulation {true | false}]
[-enable_mis {tied_input | independent_input | none}]
[-enable_spectre_x {true | false}]
[-exclude_side_load_cells string]
[-exclude_side_load_cells_for_missing_subckt {true | false}]
[-generate_output_only {true | false}]
[-glitch_simulation_mode {input_output | output}]
[-force_delete_output_dir]
[-glitch_receiver_pin string]
[-ground_nets string]
[-include_cell_variation {true | false}]
[-lsf_job_string string]
[-measure_point {all | path | stage}]
[-merge_spice_deck {true | false}]
[-monte_carlo_sim_init_args string]
[-monte_carlo_sim_transient_args string]
[-omipath <path>]
[-out_dir string]
[-out_file_prefix string]
[-power_nets string]
[-reliability_model_type {URI | OMI | TMI}]
[-reliability_commands string]
[-report_fields string]
[-report_noise_cmd string]
[-report_timing_cmd string]
[-run_simulation]
[-segment {data_path | launch_path | launch_clock_path | capture_path all}]
[-segment_end string]
```

```
[-segment_start string]
[-supply_voltage_file string]
[-supply_voltage_file_stress <file>]
[-si_sweep_range float]
[-si_sweep_steps integer]
[-si_sweep_victim {true | false}]
[-side_path_levels integer]
[-spectre_analysis_options <string>]
[-spectre_cmd_options string]
[-spectre_exe_path string]
[-spice_include_file string]
[-spice_model_files string]
[-spice_stimuli_file string]
[-spice_subckt_files string]
[-temperature float]
[-through string]
[-uri_lib_file string]
[-uri_mode <string>]
[-use_control_param {true | false}]
[-use_distributed_load_for_side_receivers {true | false}]
[-user_defined_measure_statements {append | overwrite}]
[-user_defined_measure_statements_file string]
[-use_ndw_for_constraint_check {true | false}]
[-use_noise_model_when_cond {true | false}]
[-view string]
[-waveform_aware_pulse_width_checks {true | false}]
[-xtalk {delay | voltage_low_glitch | voltage_high_glitch}]
[ [ > | >> ] ]
```

Generates the SPICE trace for a path.

While generating a Spice Deck, the `write_spice_deck` command looks for library cells required to run Spice and adds all cells as ".include" in the generated Spice netlist.

The output SPICE deck includes:

- Active and passive devices such as field-effect transistors (FETs), capacitors, and resistors.
- Initial conditions and voltage sources
- Actual names of the gates, nets, and cell instances, which helps the simulator interpret this information effectively. For example, consider the following excerpt, which shows an entry for a gate named `U0` in the SPICE deck:

  ```
  *Gate U0
  *---------------------
  *Cell ports in order : x a VSS VDD
  *---------------------
  XU0 U0/x U0/a 0 U0/VDD inv_ahvt_64
  ```

```
*--------------------
```

Note: By default, the software uses the default list of power node names (`VDD` and `VCC`) and ground node names (`GND` and `VSS`).

Note: In order to output a Spice deck that generates the same report output as the `report_timing` command, you must run the `write_spice_deck` command with the `-report_timing`, `-segment_start`, and `-segment_end` parameters.

Parameters

`-help`	Prints out the command usage.
`-add_driver_for_constraint_check {true | false}`	Enables constraint check deck generation with pre-driver.
`-add_side_load_measurement {true | false}`	Adds the side load measurement in clock mesh Spice deck. By default, Tempus only measures the latencies, delays, and slews of arcs that are part of clock mesh specified using the `-from` parameter. When set to true, the software additionally measures the latencies and slews of the side recievers at each stage. *Default*: `false`
`-add_voltage_for_missing_sensitization`	Adds the voltage for missing sensitization.
`-aging_correlation_type {full | timingonly}`	Creates the aging-aware Spice deck. • `timingonly`: Writes out the Spice deck for timing-only correlation mode. The Spice deck contains device degradation parameters computed by Tempus. • `full`: Writes out the Spice decks for full correlation. This involves two Spice simulations. The first Spice simulation computes device degradation using stress simulation. The second Spice simulation uses this data for transient simulation, where delays and slew are measured for timing correlation.

`-aging_stress_num_cycles <integer greater than 0>`	Specifies the number of cycles in stress simulation in full Spice correlation mode. When this parameter specified, the periodic stress waveform propagates through the entire path on which correlation is required. It is recommended to use the value 6, which is based on the path depth.
`-aging_stress_use_timing_slew {false | true}`	Uses the slew from STA in stress Spice deck for full Spice correlation, instead of using the default slew of 1ps.
`-aging_level_only <`*`string`*`>`	Indicates the type of aging, such as BTI, HCI, or both. This parameter is required for URI models only. **Note**: The exact value of aging level is foundry specific.
`-bbox_cells` *`string`*	Specifies a list of cells to be considered as black boxes.
`-clock_mesh_depth` *`integer`*	Creates clock mesh spice deck of depth.
`-clock_mesh_from_fall_pin` *`string`*	Creates the spice deck with mesh net starting from falling transition of specified from pin.
`-clock_mesh_from_pin` *`string`*	Createsthe spice deck with mesh net starting from pin.
`-clock_mesh_from_rise_pin` *`string`*	Creates the spice deck with mesh net starting from rising transition of specified from pin.
`-clock_mesh_no_hard_stop_at_to_pins {true | false}`	When it is `true`, it will traverse down all the way till it hits the clock pin of a flip-flop or or if it passes the depth specified with the `-depth` parameter.
`-clock_mesh_to_pin` *`string`*	Creates the spice deck with mesh net ending at to pins.
`-constraint_check {true | false}`	Enables constraint check deck generation.

`-enable_fast_simulation {true | false}`	When set to `true`, the software enables the fast simulation mode. In this mode, additional clock cycles are not required to initialize sequential cells in the Spice deck - this speeds up Spice simulations. All the sequential cells are initialized using the initial conditions and only a single clock edge is applied as input. By default, this parameter is set to `false`. In this mode, two clock cycles are applied as input to the clock network, where the first triggering edge is used for initialization of sequential cells, and the second edge is used for delay measurement. **Note**: For better performance with sequential cell simulations, you can set the `-enable_fast_simulation` parameter to `true` during constraint check generation.
`-enable_spectre_x {true | false}`	Enables the spectreX license checkout when launching Spectre.
`-enable_mis {tied_input |independent_input | none}`	Enables advanced multi-input switching (AMIS) during Spice deck creation. You can specify one of the following options: • `tied_input`: Multiple inputs of an instance will be tied together in a netlist. You can set the `timing_analysis_multi_input_switching_mode` attribute to `1` to enable this behavior. • `independent_input`: Allows the Spice deck to write out input stimulus for side inputs for multi-input gates. The switching times of each input depends on the timing windows. This option can be used when the `timing_analysis_multi_input_switching_mode` attribute is set to `2`. • `none`: Disables AMIS during the Spice deck creation. *Default*: `none`
`-exclude_side_load_cells` *string*	Specifies a list of cells for which the pin capacitance will be used for side loads instead of instantiating them.
`-exclude_side_load_cells_for_missing_subckt {true | false}`	When set to `true`, the software will add side load pin capacitance for missing Spice subckt cell instead of instantiating it.

`-force_delete_output_dir`	Completely removes the directory specified by the user using the `-out_dir` parameter.
`-generate_output_only {true | false}`	Generates the report output only, without triggering any simulations. *Default*: `false`
`-glitch_simulation_mode {input_output | output}`	Specifies the glitch simulation mode. When set to input_output, the software uses both the RIP and ROP for Spice calculations. When output is specified, the Spice ROP will be computed using the RIP.
`-glitch_receiver_pin` *string*	Specifies receiver pin for glitch spice deck.
`-ground_nets` *string*	Specifies a list of ground nodes.
`-include_cell_variation {true | false}`	Writes out the Spice deck required to run simulations. When set to `true`, the software runs Monte-Carlo simulations from within the Tempus software. *Default*: `false`
`-lsf_job_string` *string*	Specifies a handle to the farm job for launching the simulation.
`-monte_carlo_sim_init_args` *string*	Passes necessary Monte Carlo parameters that are required to run MC simulations. The Monte Carlo analysis refers to “statistics blocks”, where statistical distributions and correlations of netlist parameters are specified. This parameter initializes these parameters under the statistics block. **Note**: This parameter can be used when `-include_cell_variation` is set to `true`.
`-monte_carlo_sim_transient_args` *string*	Initializes full transient time of simulation and transient time step.
`-measure_point {all | path | stage}`	Specifies whether to add all, path, or stage measurements. *Default*: `all`
`-merge_spice_deck {true | false}`	Allows you to enable merged Spice deck.
`-omipath <string>`	Specifies the OMI (Open Model Interface) directory - required only for OMI.

`-out_dir` *string*	Specifies the name of the output directory that will contain the generated delay and/or glitch spice decks. If simulation is also performed using the `-run_simulation` parameter, the Spice simulations output will also be saved in this directory. *Default*: `tempus_pathsim`
`-out_file_prefix` *string*	Adds the specified prefix to the generated Spice deck files. If this parameter is not used, one of the following default names of the generated Spice decks will be used: • `path_<index>_setup.sp: for setup paths` • `path_<index>_hold.sp: for hold paths` • `path_<index>_vl_glitch.sp: for VL glitch` • `path_<index>_vh_glitch.sp: for VH glitch` The `<index>` in the above file names is controlled based on the number of decks that are generated using the `write_spice_deck` command. A user-specified string (specified using this parameter) is used as a prefix to the above mentioned file names.
`-power_nets` *string*	Specifies list of power nodes.

`-reliability_commands` *string*	Allows you to specify the analysis type.
	Spice deck creation while using OMI models requires you to provide the following strings using the `-reliability_commands` parameter, to direct the Spice simulator for the appropriate analysis.
	The following three Spice parameters need to be defined as per the required analysis type:
	Analysis bti_on hci_on btirec
	BTI-Only 1 0 0
	BTI-Only with Recovery 1 0 1
	HCI-Only 0 1 0
	BTI+HCI without Recovery 1 1 0
	BTI+HCI with Recovery 1 1 1
	A few examples are given below:
	BTI-only without recovery:
	`-reliability_commands ".param hci_on=0 bti_on=1 she_on=0 btirec_on=0"`
	HCI-only:
	`-reliability_commands ".param hci_on=1 bti_on=0 she_on=0 btirec_on=0"`
	BTI+HCI without recovery:
	`-reliability_commands ".param hci_on=1 bti_on=1 she_on=0 btirec_on=0"`
	BTI-only with recovery:
	`-reliability_commands ".param hci_on=0 bti_on=1 she_on=0 btirec_on=1"`
	BTI+HCI with recovery:
	`-reliability_commands ".param hci_on=1 bti_on=1 she_on=0 btirec_on=1"`
	Note: HCI support for OMI is not available.

`-reliability_model_type {URI | OMI | TMI}`	Specifies the aging-aware deck type - URI, OMI, or TMI.
`-report_fields` *string*	Changes run path simulation internal reporting format. The format columns should be compatible with those in the `report_timing_format` attribute. These settings will be used for the subsequent `write_spice_deck` command statement.

`-report_noise_cmd` *string*	Specifies the `report_noise` command parameters as a list of arguments within brackets. When specified, the Spice deck will be generated for all the receiver pins that are reported by the `report_noise` command for the specified options. When specified with the `-run_simulation` parameter, the software writes out a comparison report comparing various glitch parameters between Tempus and Spice, such as, RIP, input glitch area, and ROP (only if ROP node name is available in the noise models).
`-report_timing_cmd` *string*	Specifies arguments to `report_timing` command.
`-run_simulation`	Enables path simulation in addition to generating Spice Deck.
`-segment {data_path | launch_path launch_clock_path | capture_path all}`	Creates the Spice deck of a segment out of the path spice deck. The following command/global variable settings have the following impact on the `-segment` parameter options: • `-path_type full_clock` is defined under the write_spice_deck-report_timing parameter, then the -segment default value is `launch_path`. • `-path_type full_clock` is not specified, then default value is `data_path`. • `timing_disable_retime_clock_path_slew_propagation` attribute is set to `false`, the default value is `launch_path` (irrespective of `-path_type` settings). • `timing_disable_retime_clock_path_slew_propagation` attribute is set to `true` (default), then `launch_clock_path` and `capture_path` options are supported only when the `-path_type full_clock` option of the `write_spice_deck -report_timing` parameter is used.
`-segment_end` *string*	Specifies the name of the segment end term.
`-segment_start` *string*	Specifies the name of the segment start term.
`-si_sweep_range` *float*	Range to sweep the victim.
`-si_sweep_steps` *integer*	Number of steps by which the victim is swept.

`-si_sweep_victim {true | false}`	Enables victim sweep.
`-side_path_levels` *integer*	Specifies side path level.
`-spectre_analysis_options <string>`	Adds Spectre analysis options to the run. When specified, the software provides instructions to Spectre, e.g., run a session with extra accuracy.
`-spectre_cmd_options` *string*	Specifies command line options for Spectre.
`-spectre_exe_path` *string*	Specifies location of Spectre path.
`-spice_include_file` *string*	Specifies the string to be included in the generated spice deck.
`-spice_model_files` *string*	Specifies the name of the model files to be included in the generated spice.

`-spice_stimuli_file` *string*	Allows you to specify the file path containing stimuli to be used for the specified library cells. The cells' sensitization coming from the user-specified stimuli file takes precedence over Tempus inferred sensitization.
	
	Syntax for delay Spice decks is given below:
	`lib_cell` *string* `<from | from_rise | from_fall>` *string* `<to | to_rise | to_fall>` *string* `input_vector` *string* `num_clk_pulse` *integer*
	
	Syntax for glitch Spice decks is given below:
	`lib_cell` *string* `glitch_type <VL | VH> output_pin` *string* `input_vector` *string* `num_clk_pulse` *integer* `# For victim driver sensitization`
	`lib_cell` *cell_name* `glitch_type <VL | VH> input_pin` *string* `input_vector` *string* `num_clk_pulse` *integer* `# For victim receiver sensitization`
	
	The `num_clock_pulse` *integer* holds significance in case of sequential arcs, and specifies where to align the data pin transition with respect to the clock transition.
	
	For example, a value of ‘1’ (default setting) specifies that the data pin transition needs to be aligned before the last clock transition where delay measurement is intended. This value can be used in case of regular flip-flops.
	The value ‘2’ specifies that data pin transition needs to be aligned with respect to the second-last triggering clock edge. This value can be used for cascaded 2-bit flip flops.
	
	Additionally for sequential arcs, the first pin in the user-

	specified input vector needs to be the data pin, the signal of which will be latched in the sequential cell.
	Here are some examples of stimuli file:
	• The following example shows the stimuli file containing sensitization for CP(rise)->Q(fall) and CP(rise)->Q(rise) timing arcs of library cell FF1:
	`lib_cell FF1 from_rise CP to_fall Q input_vector !D&CDN&!SE&!SI num_clk_pulse 2`
	`lib_cell FF1 from_rise CP to_rise Q input_vector D&CDN&!SE&!SI num_clk_pulse 2`
	• The following example shows the stimuli file containing sensitization for the victim driver cell AOI for both VL and VH glitches:
	`lib_cell AOI glitch_type VL output_pin Z input_vector !A&!B&!C`
	`lib_cell AOI glitch_type VH output_pin Z input_vector A&!B&C`

`-spice_subckt_files` *string*	Specifies the subckt file, to be included in generated spice.

`-supply_voltage_file` *string*	Specifies the file containing user-specified power/ground voltages to be used for SPICE simulations. By default, the design operating voltages will be used. The format of the supply voltage file is as follows: `GLOBAL POWER` #Specifies the global supply for all power pins `GLOBAL GROUND` #Specifies the global supply for all ground pins `GLOBAL` #Specifies the voltage to be used for given pg pin for all instances `POWER` #Specifies the VDD to be used for specified I/O port PWLs `GROUND` #Specifies the VSS to be used for specified I/O port PWLs `POWER` / #Specifies the VDD used for specified instanceAC/AEURAtms signal pin PWL or sensitization `GROUND` / #Specifies the VSS used for specified instanceAC/AEURAtms signal pin PWL or sensitization `/` #Specifies the voltage used for specified instanceAC/AEURAtms pg pin
`-supply_voltage_file_stress <`*file*`>`	Specifies an optional extra file containing the stress voltages for supply or ground pins that are in cell sub-circuit definition.
`-through` *string*	Creates the Spice deck with mesh net going by `-through` points.
`-temperature` *float*	Specifies a user-defined temperature to be used in SPICE simulations. *Unit*: Celsius
`-uri_lib_file` *string*	Specifies the URI model file (required only for URI). *Default*: `none`
`-uri_mode` *string*	Specifies the URI appendage option. *Default*: `appendage`

`-use_control_param {true | false}`	Allows you to control the power/ground voltage levels in the created Spice deck through user-specified parameters. When set to `true`, an additional file with the same name as the deck, and extension .control is created that contains the parameterized design voltage levels of the PG pins of all the cells instantiated in the deck. Your can modify the *.control file as per the requirement without modifying the original deck.
`-use_distributed_load_for_side_receivers {true | false}`	Allows you to control whether to use the receiver output load as distributed or lumped load. *Default*: `false`
`-user_defined_measure_statements {append | overwrite}`	Specifies whether to overwrite the default measurement statements in the Spice deck by user-defined measurements specified using the `-user_defined_measure_statements_file` parameter, or append it with default measure statements. *Default*: `append`
`-user_defined_measure_statements_file` *string*	Allows you to add the delay/slew measurement statements for any given segment of the path for which Spice deck is generated. The format of the file is given below: `<MEASURE_TAG_NAME TRIGGER_PIN_NAME TARGET_PIN_NAME>` Multiple such lines with unique tags can be specified in the file. The measure statement is written in the Spice deck for delay measurement between `TRIGGER_PIN` and `TARGET_PIN`, and slew measurement at `TARGET_PIN`.
`-use_ndw_for_constraint_check {true | false}`	Enables constraint check deck generation with NDW waveforms.
`-use_noise_model_when_cond {true | false}`	Allows to use the when condition from a pin noise model.
`-view` *string*	Specifies the view name, if glitch spice deck is create for other than default view.
`-waveform_aware_pulse_width_checks {true | false}`	Generates the Spice deck for waveform-aware pulse width checks.

`-xtalk {delay | voltage_low_glitch | voltage_high_glitch}`	Generates the SI SPICE Deck for the stage. A stage can start with a victim driver input to victim receiver input or receiver output. You can specify one of the following options: • `delay:` Used to dump the SI delay SPICE deck. • `voltage_low_glitch` and `voltage_high_glitch:` Specifies the type of glitch (VL or VH) the SPICE deck will be used for. This option must be used for glitch SPICE deck generation.
`> filename`	Redirects the output to a file.
`>> filename`	Appends to an output file.

Examples

- The following section in SPICE deck specifies global supplies and transient analysis window time.

```
*---------------------
*Command and Options
*---------------------
.GLOBAL vdd vbbnw
V0Xvdd  vdd 0 1.0
V0Xvbbnw vbbnw 0 1.0
.GLOBAL vss vbbpw
V0Xvss vss 0 0
V0Xvbbpw vbbpw 0 0
.TEMP -40.0000
.TRAN 1p 10256p
*---------------------
```

- The following section shows an example of a net and its instance connections.

```
*-------------------------
*Net Instances
*-------------------------
*Net module1/inst_1/net
*-------------------------
Xmodule1/inst_1/net module1/inst_1/q
+ module1/inst_2/a wc:module1/inst_1/q
*-------------------------
```

- The following section shows an example of a gate instance and its port connections.

```
*---------------------------------------
*Gate Instances
```

```
*----------------------------------------
*Gate module1/inst_2
*----------------------------------------
*Cell ports in order : a y vss vdd vbbpw vbbnw
*----------------------------------------
Xmodule1/inst_2/a module1/inst_2/y
+ 0 module1/inst_2/vdd 0 module1/inst_2/vbbnw
+ inv1
*----------------------------------------
```

- The following command run simulations and then generates a report:

  ```
  write_spice_deck -report_timing "-net -from clk -to ca53/as_12/ff/D -path_type full_clock" -
  run_simulation -include_cell_variation true > spice.socv.rpt
  ```

- The following command generates a report (if simulations have already been run):

  ```
  write_spice_deck -generate_output_only true -report_timing "-net -from clk -to ca53/as_12/ff/D
  -path_type full_clock" -run_simulation -include_cell_variation_true -format {arrival
  spice_arrival spice_arrival_detail arrival_error_detail hpin} > spice.socv.rpt
  ```

- The following section shows the measurement statements for delay and slew.

  ```
  *---------------------
  *Measurement statements
  *---------------------
  .MEASURE TRAN Stage_0001_module1/inst_1/clk_DELAY_
  module1/inst_1/q                                    ---------clk to q delay
  + TRIG V(module1/inst_1/clk) VAL=0.5 TD=167.9p RISE=1       of module1/inst_1
  + TARG V(module1/inst_1/q) VAL=0.5 TD=167.9p RISE=1
  .MEASURE TRAN Stage_0001_module1/inst_1/q_SLEW
  + TRIG V(module1/inst_1/q) VAL=0.30 TD=167.9p RISE=1
  + TARG V(module1/inst_1/q) VAL=0.70 TD=167.9p RISE=1  ---------Slew at
  .MEASURE TRAN Stage_0002_module1/inst_1/q_DELAY_             module1/inst_1/q pin
  module1/inst_2/a
  + TRIG V(module1/inst_1/q) VAL=0.5 TD=167.9p RISE=1   ---------Interconnect Delay
  + TARG V(module1/inst_2/a) VAL=0.5 TD=167.9p RISE=1         and Input Slew
  .MEASURE TRAN Stage_0002_module1/inst_2/a_SLEW
  + TRIG V(module1/inst_2/a) VAL=0.30 TD=167.9p RISE=1  ---------Interconnect Delay
  + TARG V(module1/inst_2/a) VAL=0.70 TD=167.9p RISE=1        and Input Slew
  ```

  Note: The `DELAY` and `SLEW` keywords in the `MEASURE` statements indicate that these are the measurement statements for delay and slew respectively.

- The following command sets the simulation time stop criteria to be 13.40 ns and step time to be 1ps:

```
tempus > write_spice_deck -monte_carlo_sim_init_args "stop=1.34e-08 step=1p"
```

- The following command writes out statistics block in a deck to run Monte Carlo simulations:

```
tempus > write_spice_deck -monte_carlo_sim_init_args "numruns=3000 seed=1234 distribute=fork
numprocesses=8 variations=mismatch scalarfile=MCDataSetLoad.txt dut=[%s]"
simulator lang = spectre

mc1 montecarlo numruns=3000 seed=1234 distribute=fork numprocesses=8 variations=mismatch
scalarfile=MCDataSetLoad.txt dut=[ XStage2.XCTS_ccl_a_inv_01730 XStage3.XCTS_ccl_inv_01726
XStage4.XCTS_ccl_inv_01718 XStage5.XCTS_ccl_a_inv_01712 \
XStage6.XCTS_cpc_sk_inv_03405 XStage7.XCTS_cpc_sk_inv_03406 XStage8.XCTS_cdb_inv_03303
XStage9.XCTS_cdb_inv_02837 \
XStage10.XCTS_ccl_inv_01669 XStage11.XCTS_ccl_a_inv_01540 XStage12.XCTS_cpc_sk_inv_03860
XStage13.XCTS_cpc_sk_inv_03859 \
XStage14.Xugenq_uaesflushu1_aes_flush_u1_reg XStage15.XPLACEOPTFE_RC_149_0
XStage16.XPLACEOPTFE_RC_150_0 \
XStage17.XU22704 XStage18.XU22707 XStage19.XU22857 XStage20.XPLACEOPTFE_OFC2135_n30081
XStage21.XU23049 \
XStage22.XU140015 XStage23.XU132065 XStage24.XU132066 XStage25.XU132710 XStage26.XU132716
XStage27.XU133277 \
XStage28.XU133278 XStage29.XU133279 XStage30.XU133285 ]

{
tran1 tran stop=2.88312e-08 step=1p
}
simulator lang = spice
```

- The following commands shows constraint checks applied during setup and hold mode:

```
Setup: create_spice_deck -spice_model_files $model_file -subckt_file $subckt_file -out_dir
output -spectre $spectre_path -spectre_cmd_options $spectre_opt -run_simulation -
enable_fast_simulation true -constraint_check true -report_timing {-late -retime
path_slew_propagation -to EP1}

Hold: create_spice_deck -spice_model_files $model_file -subckt_file $subckt_file -out_dir
output -spectre $spectre_path -spectre_cmd_options $spectre_opt -run_simulation -
enable_fast_simulation true -constraint_check true -report_timing {-early -retime
path_slew_propagation -to EP1}
```

- A sample constraint check report is given below:

```
Path 1: SPICE based Setup Check
Clock:       dff2/CK (^)
Data:        dff2/D (^)
Deck:        path_1_setup_iter11.sp -----> Final Deck
Slew:        1e-09(Data), 1e-09(Clock)  ----------> Data and Clock Pin Transitions
Setup:       1.59911e-10 (Spice), 1.57721e-10 (Tempus) --------> Setup Time
-------------------------------------------------------
Iter   Offset(D-CK) Pass/Fail  Deck
       (ps)
-------------------------------------------------------
0      -2500.000    Pass      path_1_setup_iter0.sp  |
1      1249.999     Fail      path_1_setup_iter1.sp  |
2      -625.000     Pass      path_1_setup_iter2.sp  |
3      312.500      Fail      path_1_setup_iter3.sp  |
4      -156.250     Fail      path_1_setup_iter4.sp  |
5      -390.624     Pass      path_1_setup_iter5.sp  |------> PASS/FAIL Iterations
6      -273.438     Pass      path_1_setup_iter6.sp  |
7      -214.843     Pass      path_1_setup_iter7.sp  |
8      -185.548     Pass      path_1_setup_iter8.sp  |
9      -170.900     Pass      path_1_setup_iter9.sp  |
10     -163.574     Pass      path_1_setup_iter10.sp |
11     -159.911     Pass      path_1_setup_iter11.sp -----> Final Deck
-------------------------------------------------------
```

Related Commands

- report_timing
- set_aging_activity
- report_aging_activity
- create_aging_stress_profile

24

System Commands - Basic Tool-Tcl Commands

- gui_hide
- gui_show
- help
- man
- tempus

gui_hide

`gui_hide`

Closes the graphical user interface (GUI) for Tempus.

Parameters

`-help`	Prints out the command usage.

Related Commands

- `gui_show`

gui_show

```
gui_show
[-help]
[on | off]
```

Opens the Tempus main window. This is usually done after running the command, `Tempus -init` *`tclCommandFile`*.

If you run the `gui_show` command when the Tempus main window is already displayed, the form is refreshed and the display is fitted to the design area.

Note: If you started the Tempus software with the `tempus -gui_show off` command, you cannot then use the `gui_show` command to display the Tempus main window.

Parameters

`-help`	Prints a brief description that includes type and default information for each `win` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_show`
`on | off`	Specifies to on or off the window.

Related Commands

- `gui_hide`

help

```
help
[-help]
[<pattern1>]
[<pattern2>]
[-category category]
[-detail  | -summary ]
[-command  | -attribute  | -obj  | -message ]
```

Displays help information related to a command, object, attribute or message based on the specified arguments.

By default, only commands, objects and attributes are searched for help.

Parameters

`-attribute`	Searches attributes only.
`-category` *`category`*	Returns only those attributes that are in the given category.
`-command`	Searches commands only.
`-detail`	Provides detailed help, even when multiple help strings are returned. By default, a short help description is returned if more than one items are matched.
`-message`	Searches messages only. Note, By default messages are not searched. This option is required to see the message help.
`-obj`	Searches objects only.
`pattern1`	Searches for any command, object, or attribute that matches pattern1. Pattern matching with * and ? is allowed. If the pattern ends in : (e.g. inst:) only objects will be matched. If the pattern starts with . (e.g. .place*) then only attributes will be matched.
`pattern2`	Returns help only for those attributes that match pattern2 for the objects matching pattern1. For example, the following command will return all attributes that match `place*` on the `inst` object. `help inst place*`
`-summary`	Provides a short description even for a single match. By default, if only a single item is matched, the full help description is returned.

Examples

- The following command prints the help for any command, object, or attribute that matches "port*". Note that for attributes, the object it is on is shown in ().

```
tempus> help port*
Objects:
port: External logical ports of the design. See pg_ports for power/ground ports

Attributes:
port(bump)    Port attached to the bump
ports(design) Ports of design. Does not include pg_ports.
ports(root)   Short-cut for [get_db current_design .ports]
```

- The following command shows the help output for the attribute `.base_cell` on the object `inst:`. The : and . chars in `inst:` and `.base_cell` are optional as they restrict the search to objects and attributes, respectively.

```
tempus> help inst: .base_cell
Attribute: base_cell (on obj_type: inst)
Description: The base_cell of the inst.
category:
default:
is_computed: false
is_settable: true
is_user_defined: false
skip_in_db: true
type: base_cell
```

- The following output shows the help for a single command:

```
tempus 14> help write_metric
Command: write_metric:

Usage: write_metric [-help] [<pattern>] [-format <string>] [-id <string>] [-merge]
[-names <string>] {-file <string> | -inline }

-help      # Prints out the command usage
<pattern>   # List of patterns to match metrics (string, optional)
-file <string> # Filename to write (string, optional)
-format <string> # Data format for the return value (string, optional)
-id <string> # Database ID to output (string, optional)
-inline # Return the result instead of writing it to a file (bool, optional)
-merge # Merge the results into an existing file (bool, optional)
-names <string> # List of patterns to match snapshot names (string, optional)
```

- The following command shows the help output for a message.

```
tempus 14> help -message IMPTCM-48
Error/Warning message: IMPTCM-48: "%s" is not a legal option for command "%s".
Either the current option or an option prior to it is not specified correctly.
```

man

```
man
command_name | msg_id
```

Displays information for a command or variable from the *Tempus Text Command Reference*, or prints detailed information for a specified message ID. If no manual page is located, man prints an error message.

The man pages for Tempus commands and variables are located in the following directory:

<install_dir>/<platform>/share/tempus/man

Parameter

command_name	Indicates the name of the text command (or variable) for which the manual information is required.
msg_id	Specifies the unique message ID for which you want to display detailed information. The output is categorized in three sections: • NAME: Displays the message ID and its severity level. • SUMMARY: Displays the summary of the message text. • DESCRIPTION: Displays detailed information about the message. The detailed information can include the background of the message, likely causes, and alternative steps to troubleshoot the problem.

Example

- The following command returns the manual description of the command report_timing:

  ```
  man report_timing
  ```

- The following command returns the manual description of the ENCSYC-2 message ID:

  ```
  man ENCSYC-2
  ```

tempus

```
tempus
[-abort_on_error]
[-batch]
[-cds_lib_file in_file]
[-stylus]
[-cpus integer]
[-disable_user_startup]
[-distributed]
[-execute Tcl_commands]
[-files in_file_list]
[-help]
[-lic_multi_cpu lic_list]
[-lic_options lic_list]
[-lic_startup lic_list]
[-lic_startup_options lic_list]
[-log prefix]
[-no_gui]
[-no_logv]
[-overwrite]
[-eco]
[-version]
[-wait minutes]
```

Starts the Tempus software. If you do not specify any parameters, the following actions occur:

- A log file and a command file are created in the form `tempus.log#` and `tempus.cmd#`.
- The first available license is checked out.
- The Tempus main window opens.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man tempus`
`-abort_on_error`	Exits if an error is detected at any point. If any Tcl command returns `TCL_ERROR` to the top level, or any `Error (XXX): …` message occurs, the process will exit.

`-batch`	Runs without interactive prompt. After all the options, including `-execute` and `-file`, are processed, the job exits.
`-cds_lib_file` *`in_file`*	Loads the specified file instead of the default cds.lib file in the current directory to obtain the names and locations of OpenAccess libraries. The cds.lib file is used to list the OpenAccess library names, and their physical locations.
`-stylus`	Launches Tempus in Stylus Common UI mode.
`-cpus` *`integer`*	Specifies the number of CPUs to be used for multi-threading on current host. This calls the appropriate Tcl command to set the local CPU count. This happens before any `-execute` or `-file` commands are processed. This may require extra multi-CPU licenses to be checked out (see `-lic_multi_cpu`).
`-disable_user_startup`	Reads only the init file in the installation hierarchy. Does not read init files in the home or current directory. This option suppresses the default loading from the home and current directory, which can be used to guarantee there are no local over-rides to various project settings. For more information, see "Initialization Files" in the Installation and Startup chapter of the *Tempus User Guide* for init files that are loaded by default.
`-distributed`	Starts Tempus in the Distributed Static Timing Analysis (DSTA) mode. DSTA supports all the command-line parameters that are supported by non-distributed Tempus, except for the following parameters: • `-cds_lib_file` • `-no_logv` • `-no_gui` This is because DSTA does not support OA, GUI, and verbose log. When you specify these parameters with `tempus -distributed`, an error message will be displayed.
`-execute` *`Tcl_commands`*	Specifies the Tcl commands to be executed as a quoted string at startup.

`-files` *`in_file_list`*	Specifies one or more Tcl files to be sourced at startup, after the commands specified using the `-execute` parameter are processed. You can set the shell environment variable `CDS_STYLUS_SOURCE_VERBOSE` to `0` if you do not want the source command output to default to verbose.
`-lic_multi_cpu` *`lic_list`*	Controls which licenses are to be used for higher multi-CPU usage in priority order from left to right. The allowed license names are: `tpsmp`: Tempus Timing Signoff Solution MP `tpsl`: Tempus Timing Signoff Solution L `tpsxl`: Tempus Timing Signoff Solution XL **Note:** For information on licenses, see "Product and Licensing Information".
`-lic_options` *`lic_list`*	Specifies the order that extra option licenses should be checked-out if needed. In some cases more than one option license can be used to enable a given feature. You can control which options are allowed to be checked-out, and in what priority (from left to right) when required during the run. The allowed license names are: `tpstso`: Tempus Timing Signoff Solution TSO `tpspi`: Tempus Timing Signoff Solution - PI (Power Integrity) Option `tpsng`: Tempus Timing Signoff Solution Next Generation `vtsxl`: Voltus IC Power Integrity Solution `vtsl`: Voltus IC Power Integrity Solution `vtsaa`: Voltus IC Power Integrity Solution Advanced Analysis GXL Option Option licenses can be checked-out dynamically when a specific Tcl command requires it. This may cause the Tcl script to stall and wait for a license, or to exit if the wait period expires. For example, `-lic_options "tpstso tpsxl"` will only allow these two options to be checked-out dynamically, and in the given priority order. **Note:** For information, see "Product and Licensing Information".

`-lic_startup` *`lic_list`*	Requests one of these base licenses at startup in priority order from left to right. The supported licenses are: `tpsl`: Tempus Timing Signoff Solution L `tpsxl`: Tempus Timing Signoff Solution XL `vdsp`: Virtuoso Digital Signoff Power Solution For example, `-lic_startup "tpsxl tpsl"` will try to check-out `tpsxl` first and then `tpsl`. The check-outs will be attempted repeatedly until it succeeds or the specified wait (`-wait`) period expires.
`-lic_startup_options` *`lic_list`*	Specifies additional option licenses needed at startup. If not provided, they will be checked out when needed during the run. The allowed license names are: `tpstso`: Tempus Timing Signoff Solution TSO `tpspi`: Tempus Timing Signoff Solution - PI (Power Integrity) Option `tpsng`: Tempus Timing Signoff Solution Next Generation `vtsxl`: Voltus IC Power Integrity Solution `vtsl`: Voltus IC Power Integrity Solution `vtsaa`: Voltus IC Power Integrity Solution Advanced Analysis GXL Option For example, `-lic_startup_options "tpsl, tpsxl"` will try to check-out both `tpsl` and `tpsxl` when Tempus starts up. The check-outs will be attempted repeatedly until it succeeds or the `-wait` period expires.

`-log` *`prefix`*	Specifies the prefix for the `.log`, `.cmd`, and `.logv` files. If three arguments are given, such as `-log "a b c"`, then the three names are used exactly without any extension added. So, `-log "mylog mycmd mylogv"` will result in `mylog mycmd mylogv` files. The default for *`prefix`* is `tempus`. So if `-log` is not specified, `tempus.log`, `tempus.cmd`, and `tempus.logv` files will be created. If you specify `-log test`, the tool creates `test.log`, `test.cmd` and `test.logv` files. The `.log` file contains the normal logging output, the `.cmd` file contains the Tcl commands that were executed, and the `.logv` file contains a super-set of the .log output plus "verbose" debugging output that is needed only for detailed debugging. If the `.log` file already exists, an integer is added to the end of all the files to make them unique. This means if `-log test` is specified, the tool will create `test.log test.cmd test.logv` for the first run, `test.log1 test.cmd1 test.logv1` for the next run, and so on. Only the existence of .log is checked, the existence of the .cmd or .logv files is not checked. The `-overwrite` option can be used to disable this behavior and overwrite an existing .log file. If the prefix has a period in it, the last extension is stripped off for the .cmd and .logv usage. For example, `-log out.log` will result in `out.log out.cmd out.logv` files, and `-log out.a.log` will result in `out.a.log out.a.cmd out.a.logv`files. You can disable any given file by using `/dev/null`. For example, `-log "my.log /dev/null /dev/null"` will only create `my.log`. *Default*: `tempus.log` **Note**: If you provide the log file name under a directory path and the directory path does not exist, Tempus will create the directory path and save the log file under the directory.
`-no_gui`	The GUI and Tk are not initialized at startup so only the user type-in window (e.g., xterm) is visible. You can start the GUI later in the session with the `gui_show` command.
`-no_logv`	Specifies that verbose log file will not be created.

`-overwrite`	Overwrites an existing .log file (and associated .cmd and .logv files) instead of adding an integer to make the files unique.
`-eco`	Needed for running the Timing Signoff Optimization flow `eco_opt_design` to load the power data from the .lib files at startup. By default, Tempus will not load the power data in .lib files to reduce memory usage, but the Timing Signoff ECO command `eco_opt_design` needs the power data to run properly.
`-version`	Returns the program version information.
`-wait` *minutes*	Specifies the number of minutes the system waits for a license to become available before exiting. *Default*: 0 (no wait time) *Value Range*: 0 to 10,000

Initialization Files

By default, various initialization files are loaded at startup, if they exist. They can be used to configure the GUI, load utility Tcl files, or configure Tempus settings. The initialization files are read in the following order:

1. `~/{.enc.pref.tcl, .enc.color.tcl}` in the home directory if the GUI is enabled
2. `./{enc.pref.tcl, enc.color.tcl}` in the current directory if the GUI is enabled
3. `~/.enc` in the home directory
4. `<install_dir>/etc/tempus/enc.tcl` in the installation directory
5. `~/enc.tcl` in the home directory
6. `./enc.tcl` in the current directory

If the `-stylus` parameter is specified, the initalization files are loaded in the following order instead:

1. `~/.cadence/tempus/{gui.pref.tcl, gui.color.tcl, workspaces}` in the home directory if the GUI is enabled
2. `.cadence/tempus/{gui.pref.tcl, gui.color.tcl, workspaces/}` in the current directory if the GUI is enabled

3. `<install_dir>/etc/tempus/tempus.tcl` in the installation directory
4. `~/.cadence/tempus/tempus.tcl` in the home directory
5. `./.cadence/tempus/tempus.tcl` in the current directory
6. `./tempus_startup.tcl` in the current directory

Setting Up the Run Time Environment

If *install_dir* is the location of the Tempus installation, then you should setup the run time environment as follows:

- Add the *install_dir*`/bin` directory to the path. The `bin` directory has links to all the public executable files in the install hierarchy.
- If you want the Stylus Common UI Tempus man pages to be available with the Unix `man` command, then you can add the *install_dir*`/share/tempus/stylus/man` to your MANPATH envar.
- If you want the Tcl man pages to be available with the Unix `man` command, then you can add the *install_dir*`/share/tcltools/man` to the MANPATH envar.

For example, you can add the following to the startup shell script:

```
set install_dir = /tools/tempus16.2/lnx86

set path = ($install_dir/bin $path)

setenv
MANPATH $install_dir/share/tempus/stylus/man:$install_dir/share/tcltools/man:$MANPATH
```

Note: When Tempus launches, it automatically adds the legacy or the Stylus man pages (when `-stylus` option is specified), and the Tcl man pages at the beginning of the current MANPATH inside Tempus. Therefore, from within Tempus the `man` command will show both the sets of man pages before any other man pages.

Temporary File Locations

Each Tempus session creates its own temporary directory to store temporary files at the beginning of the run.

By default, the `tmp_dir` is created in the `/tmp` directory. If the Unix envar TMPDIR is set, then the `tmp_dir` is created inside `$TMPDIR`.

The name of the `tmp_dir` will appear as given below:

```
tempus_temp_[pid]_[hostname]_[user]_xxxxxx
```

Where `_xxxxxx` is a string that is added to make the directory unique. For example:

```
tempus_temp_10233_farm254_bob_nfp9ez
```

The temporary directory is automatically removed on exit or if the run is terminated with a catchable signal (e.g., SIGSEGV).

Examples

- The following command starts the Tempus software with log file named myLog.log and command log file named myCmdLog.log:

  ```
  tempus -log myLog.log -cmd myCmdLog.log
  ```

- The following command starts Tempus, and enables the Tempus Signoff Optimization (TSO) feature of the software:

  ```
  tempus -eco
  ```

25

Tempus ECO Commands and Attributes

- opt_signoff
- opt_signoff Category Attributes
- read_hierarchical_def
- read_partition
- write_eco_opt_db

opt_signoff

```
opt_signoff
[-help]
[-area]
[-drv]
[-dynamic]
[-hold]
[-leakage]
[-power]
[-setup]
[-update_timing]
```

Triggers ECO. Depending on the `opt_signoff` category attributes settings, it will either auto generate the ECO Timing DB or load existing one, then proceed to the timing fixing phase and finally output an ECO file and detailed diagnostics in log file.

Note: This command requires a 64-bit executable.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `opt_signoff` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man opt_signoff`.
`-area`	Reclaims area on the netlist without causing any DRV/Setup/Hold degradation.
`-drv`	Optimizes DRV violations. DRV optimization fixes violations in the following order: 1. glitch 2. drv (max_transition/max_capacitance violations) 3. xtalk delta delay **Note**: • DRV optimization supports buffer addition and resizing. • CellEM violations are fixed only when `set_db opt_signoff_fix_cell_em` is set to `true`.
`-dynamic`	Prints initial and final power numbers. It optimizes the dynamic power to reduce the total power consumption.
`-hold`	Optimizes hold violations.
`-leakage`	Optimizes leakage power. Leakage optimization supports VT swapping.
`-power`	Optimizes total power in the design. During total power optimization, the software performs swapping and sizing on both the combinational and sequential cells, and also do buffer removal when enabled. It reduces leakage, internal, and switching power in the design.
`-setup`	Optimizes setup timing violations. Setup optimization supports buffer addition, VT swapping and resizing.
`-update_timing`	Generates the updated timing data to be used during optimization. Note: This parameter is applicable only for the Tempus Paradime flow or Certus. It is only valid if `opt_signoff_allow_multiple_incremental` is `true`.

Examples

- The following commands are used to annotate the parasitics of three active RC corners in the design using SPEF files before running timing analysis or opt_signoff:

```
tempus> read_spef -rc_corner rc_max max.spef.gz
tempus> read_spef -rc_corner rc_min1 min1.spef.gz
tempus> read_spef -rc_corner rc_min2 min2.spef.gz
```

- The following command performs hold fixing while not degrading setup timing:

```
tempus> opt_signoff -hold
```

- The following command recovers area by performing cell downsizing and buffer deletion while ensuring not to create any new Setup/Hold/DRV violations.

```
tempus> opt_signoff -area
```

- The following command runs multiple types of optimization in incremental mode and generates new ECO DB (assuming `opt_signoff_read_eco_opt_db` is not used). It also generates the new timing data to be used during power and setup optimizations.

```
set_db opt_signoff_allow_multiple_incremental true
opt_signoff -drv
opt_signoff -setup
opt_signoff -hold
opt_signoff -power -update_timing
opt_signoff -setup -update_timing
```

Related Commands

- read_spef

opt_signoff Category Attributes

The root attributes in the `opt_signoff` category are:

- opt_signoff_add_inst
- opt_signoff_add_load
- opt_signoff_allow_multiple_incremental
- opt_signoff_allow_skewing
- opt_signoff_along_route_buffering
- opt_signoff_buffer_cell_list
- opt_signoff_check_type
- opt_signoff_clock_cell_list
- opt_signoff_clock_max_level
- opt_signoff_create_smart_mmmc_db
- opt_signoff_delete_inst
- opt_signoff_disable_geometry_checks
- opt_signoff_dont_touch_partition_list
- opt_signoff_drv_margin
- opt_signoff_eco_file_prefix
- opt_signoff_em_type
- opt_signoff_enable_high_capacity_eco
- opt_signoff_fix_cell_em
- opt_signoff_fix_clock_drv
- opt_signoff_fix_data_drv
- opt_signoff_fix_glitch
- opt_signoff_fix_hold_allow_setup_degrade
- opt_signoff_fix_hold_allow_setup_optimization

- opt_signoff_fix_hold_allow_setup_tns_degrade
- opt_signoff_fix_hold_with_margin
- opt_signoff_fix_hold_variation_aware
- opt_signoff_fix_ir_drop
- opt_signoff_fix_max_cap
- opt_signoff_fix_max_transition
- opt_signoff_fix_osus_glitch
- opt_signoff_fix_si_slew
- opt_signoff_fix_slack_in_range_lower_limit
- opt_signoff_fix_slack_in_range_upper_limit
- opt_signoff_fix_xtalk
- opt_signoff_fix_xtalk_preserve_hold
- opt_signoff_fix_xtalk_preserve_setup
- opt_signoff_high_effort_cell_list
- opt_signoff_hold_target_slack
- opt_signoff_hold_target_slack_per_view
- opt_signoff_hold_xtalk_delta_threshold
- opt_signoff_hold_xtalk_slack_threshold
- opt_signoff_keep_tmp_files
- opt_signoff_ignore_drv_checks
- opt_signoff_include_hold_path_groups
- opt_signoff_include_setup_path_groups
- opt_signoff_inter_partition
- opt_signoff_inter_partition_exclusive
- opt_signoff_intra_partition
- opt_signoff_legal_only
- opt_signoff_load_cell_list

- opt_signoff_max_cap_margin
- opt_signoff_max_local_density
- opt_signoff_max_opt_runtime
- opt_signoff_max_paths
- opt_signoff_max_runtime
- opt_signoff_max_slack
- opt_signoff_max_transition_margin
- opt_signoff_nworst
- opt_signoff_optimize_core_only
- opt_signoff_optimize_replicated_modules
- opt_signoff_optimize_sequential_cells
- opt_signoff_partition_list_file
- opt_signoff_path_group_file_name
- opt_signoff_pba_effort
- opt_signoff_post_mask
- opt_signoff_power_opt_focus
- opt_signoff_post_sta_tcl
- opt_signoff_power_aware
- opt_signoff_pre_sta_tcl
- opt_signoff_prefix
- opt_signoff_read_eco_opt_db
- opt_signoff_read_irdrop_db
- opt_signoff_read_smart_mmmc_db
- opt_signoff_resize_inst
- opt_signoff_retime
- opt_signoff_retime_depth
- opt_signoff_retime_depth_early

- opt_signoff_retime_mode
- opt_signoff_route_congestion_threshold
- opt_signoff_routing_congestion_aware
- opt_signoff_select_drv_net_file
- opt_signoff_select_drv_pin_file
- opt_signoff_select_hold_endpoints
- opt_signoff_select_setup_endpoints
- opt_signoff_setup_recovery
- opt_signoff_setup_target_slack
- opt_signoff_setup_target_slack_per_view
- opt_signoff_setup_xtalk_delta_threshold
- opt_signoff_setup_xtalk_slack_threshold
- opt_signoff_set_hold_endpoints_margin
- opt_signoff_set_setup_endpoints_margin
- opt_signoff_skew_core_only
- opt_signoff_skip_drv_net_file
- opt_signoff_smart_mmmc_effort
- opt_signoff_smart_mmmc_force_keep_hold_view_list
- opt_signoff_smart_mmmc_force_keep_setup_view_list
- opt_signoff_smart_mmmc_max_view_count
- opt_signoff_smart_mmmc_reduce_clients
- opt_signoff_swap_inst
- opt_signoff_post_mask_gate_array_filler_cells
- opt_signoff_post_mask_decap_cells
- opt_signoff_post_mask_filler_cells
- opt_signoff_post_mask_logic_cells
- opt_signoff_touch_partition_list

- opt_signoff_upsize_in_power_opt
- opt_signoff_verbose
- opt_signoff_write_eco_opt_db

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `opt_signoff` category, use the following command:

```
get_db -category opt_signoff
```

opt_signoff_add_inst

```
opt_signoff_add_inst {true | false}
```

Default: `true`

Data_type: Boolean

Specifies whether or not Setup/Hold/DRV optimizer is allowed to add buffer/delay/swap cell instances.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_add_load

```
opt_signoff_add_load {true | false}
```

Default: `false`

Data_type: Boolean

Specifies whether Hold optimizer is allowed to add dummy cell instances.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- opt_signoff

opt_signoff_allow_multiple_incremental

```
opt_signoff_allow_multiple_incremental {true | false}
```

Default: false

Data_type: Boolean

Allows to run multiple types of optimization in incremental mode. Note that there should not be any report_* commands between successive opt_signoff commands. The recommended order to fix violations is DRV, Hold, and Setup. Hold and Setup can be interchanged but DRV optimization must be performed first. You must specify a unique ECO file prefix before each optimization step. You can then source all the ECO files in the same order as these were generated to run the What-If analysis later.

Note:

If set, this attribute must be applied before opt_signoff.

Related Commands

- opt_signoff

opt_signoff_allow_skewing

```
opt_signoff_allow_skewing {true | false}
```

Default: false

Data_type: Boolean

Allows clock skewing to be done during Setup timing optimization to further improve Setup slack. When set to true, opt_signoff -setup works on any level of the clock tree to modify the clock skew in order to improve Setup timing closure. Modification on the clock tree can be either resizing of the last level of the clock tree or buffering to pad endpoint clock pin.

Note:

- If set, this parameter must be applied before opt_signoff.
- When opt_signoff_allow_skewing is active, opt_signoff will automatically unfix all clock tree

instances/nets. This is also performed when implementing the ECOs.

- As for data path optimization, skewing is not allowed to create new Hold violated paths.

Related Commands

- `opt_signoff`

opt_signoff_along_route_buffering

```
opt_signoff_along_route_buffering {true | false}
```

Default: `true`

Data_type: Boolean

When set to 1, the software uses node location provided through the parasitic data in order to insert buffers along the routing topology not only at the start or end of the net. This helps to better balance the slew or the load for long nets, which is typically needed for setup timing optimization or DRV fixing.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_buffer_cell_list

```
opt_signoff_buffer_cell_list cell_list
```

Default: ""

Data_type: String

Specifies a list of buffer/delay cells that can be used for the timing optimizer, regardless of whether those cells have a dontUse attribute. When the list is empty, the software will automatically identify the list of usable buffer/delay cells.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- opt_signoff

opt_signoff_check_type

```
opt_signoff_check_type {early | late | both}
```

Default: both

Data_type: Enum

Specifies the check type for path based analysis.

Note: If set, this attribute must be applied before write_eco_opt_db.

Related Commands

- opt_signoff

opt_signoff_clock_cell_list

```
opt_signoff_clock_cell_list cell_list
```

Default: " "

Data_type: String

Restricts the list of buffers/inverters/gating or any other cells that are allowed to be used by ECO on the clock tree.

Note: If set, this attribute must be applied before opt_signoff.

Related Commands

- opt_signoff

opt_signoff_clock_max_level

```
opt_signoff_clock_max_level integer
```

Default: 0

Data_type: Integer

Specifies the maximum levels in clock skewing thus limiting the amount of clock tree levels and avoiding the creation of an unbalanced clock tree. This attribute sets an upper boundary when adding a buffer or pair of inverters to the clock nets. The software counts the number of levels in the clock tree branch that is being modified to ensure that the maximum level will be equal to or less than the value set by this attribute.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_create_smart_mmmc_db

```
opt_signoff_create_smart_mmmc_db {true | false}
```

Default: `false`

Data_type: Boolean

Enables saving the additional compact timing information in the ECO DB directory during the STA session. When set to `true`, the attribute must be applied before the `write_eco_opt_db` command and has no effect on the `opt_signoff` command.

Related Commands

- `opt_signoff`

opt_signoff_delete_inst

```
opt_signoff_delete_inst {true | false}
```

Default: `true`

Data_type: Boolean

Enables buffer deletion when doing timing, power, or area reduction. This option is not honored by the `-leakage` parameter of `opt_signoff` since it performs only swapping.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- opt_signoff

opt_signoff_disable_geometry_checks

```
opt_signoff_disable_geometry_checks {true | false}
```

Default: false

Data_type: Boolean

Performs Vth swapping without checking the cell pins and geometry. Skipping the geometry checks during Vth swapping allows the software to only check the cell size while performing Vth swapping. This will lead to more Vth swaps. However, if you use this attribute, ecoRouting will need to be done once the ECOs are applied since the pin geometries between old and new cells might not match.

Note: If set, this attribute must be applied before opt_signoff.

Related Commands

- opt_signoff

opt_signoff_dont_touch_partition_list

```
opt_signoff_dont_touch_partition_list string
```

Default: " "

Data_type: String

Selects a list of partition module names not to be touched during optimization(s). For paths that traverse across the specified hierarchies, optimization would only be applied to a portion of the paths that lie on the non-dont_touch side.

Notes:

- This parameter is exclusive to opt_signoff_touch_partition_list.
- This parameter must be applied before opt_signoff and can be changed between each call of opt_signoff.

Related Commands

- opt_signoff

opt_signoff_drv_margin

```
opt_signoff_drv_margin value
```

Default: 0.0

Data_type: Integer

Specifies a margin value by which ratio the DRV check will be relaxed or tightened.

Note:

- The software only accepts values from -0.7 to +0.7. A positive value indicates tighter constraints.
- If set, this attribute must be applied before opt_signoff.

Related Commands

- opt_signoff

opt_signoff_eco_file_prefix

```
opt_signoff_eco_file_prefix prefix
```

Default: " "

Data_type: String

Specifies the prefix name for the ECO files that are generated.

Note: If set, this attribute must be applied before opt_signoff.

Related Commands

- opt_signoff

opt_signoff_em_type

```
opt_signoff_em_type {avg | peak | rms | all}
```

Default: `all`

Data_type: Enum

Fixes the electromigration (EM) type during DRV optimization in standalone Tempus ECO. You can specify any one of the following options:

- `avg`: Specifies the average current limit violations.
- `peak`: Specifies the peak current limit violations.
- `rms`: Specifies the Root-Mean-Square (RMS) current limit violations.
- `all`: Specifies the worst of the `avg/peak/rms` violations.

Related Commands

- `opt_signoff`

opt_signoff_enable_high_capacity_eco

```
opt_signoff_enable_high_capacity_eco {true | false}
```

Default: `false`

Data_type: Boolean

Enables database size reduction on the manager session for high capacity ECO timing/power closure, based on timing violations, list of partition name, or user-defined path_group. You must apply this parameter before the `write_eco_opt_db` command, whereas it is not required during the ECO fixing session. Note that in the Tempus Paradime flow or Certus, this parameter is replaced by `set_distribute_mmmc_mode -enable_high_capacity_eco`, and is set before `distribute_read_db`.

Related Commands

- `opt_signoff`

opt_signoff_fix_cell_em

```
opt_signoff_fix_cell_em {true | false}
```

Default: `false`

Data_type: Boolean

Fixes the cell level signal electromigration violations during DRV fixing using resizing and additional transforming techniques.

Note:

- If set, this parameter must be applied before `opt_signoff -drv`.
- If set during DRV fixing, the tool will fix only the cell EM violations. This implies that fixing of max_transition/max_capacitance/SI glitch/SI Xtalk/IR drop will be disabled.

Related Commands

- `opt_signoff`

opt_signoff_fix_clock_drv

```
opt_signoff_fix_clock_drv
```

Default: `false`

Data_type: Boolean

Fixes max_cap/max_tran violations on clock nets.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_data_drv

```
opt_signoff_fix_data_drv
```

Default: `true`

Data_type: Boolean

Fixes max_cap/max_tran violations on the data networks. This is performed only when the `opt_signoff -drv` command is run.

Related Commands

- `opt_signoff`

opt_signoff_fix_glitch

```
opt_signoff_fix_glitch  {true | false}
```

Default: `false`

Data_type: Boolean

Performs SI glitch fixing during DRV fixing using resizing and buffering techniques.

Notes:

- This attribute must be set during ECO DB generation.
- The glitch fixing is done before max_tran/max_cap fixing.
- If set, this attribute must be applied before `write_eco_opt_db`.

Related Commands

- `opt_signoff`

opt_signoff_fix_hold_allow_setup_degrade

```
opt_signoff_fix_hold_allow_setup_degrade {none | low | medium}
```

Default: `none`

Data_type: Enum

Guides the Hold fixing engine to determine the effort it has to put to resolve timing violations, and how much flexibility it has to impact the Setup timing.

- When set to `low`, the Hold optimization engine will buffer sink pins that have large Setup positive slack even though the net Setup slack is hardly meeting timing.

- When set to `medium`, the Hold optimization engine will behave the same as in the `low` mode but will also allow Setup TNS to degrade, if required.

Note:

- This attribute is honored by the Hold optimization engine.
- If set, this attribute must be applied before `opt_signoff`.
- When `opt_signoff_fix_hold_allow_setup_degrade` is set to `low` or `medium`, the `opt_signoff_fix_hold_allow_setup_tns_degrade` attribute is ignored.

Related Commands

- `opt_signoff`

opt_signoff_fix_hold_allow_setup_optimization

```
opt_signoff_fix_hold_allow_setup_optimization {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, incremental Setup fixing is performed during Hold fixing in order to create more Setup margins on Hold violated nodes that could not be fixed earlier due to Setup timing conflict.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_hold_allow_setup_tns_degrade

```
opt_signoff_fix_hold_allow_setup_tns_degrade {true | false}
```

Default: `false`

Data_type: Boolean

When specified, the Setup Total Negative Slack can be degraded during Hold time violations fixing while still maintaining the Setup Worst Negative Slack in every Setup active views.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_hold_with_margin

```
opt_signoff_fix_hold_with_margin value
```

Default: `0.0`

Data_type: Double

Specifies a slack margin value in nanoseconds for Hold time violations fixing. The purpose is to overfix any Hold violations by the amount of the margin value. This margin is not applied on nets that are already meeting the Hold target slack.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_hold_variation_aware

```
opt_signoff_fix_ir_drop  {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, more sizing and Vth swapping is performed during Hold fixing in order to reduce the cost of buffer insertion. This is particularly effective when the Hold timing paths have incremental delay due to SI or large SOCV impacted instances.

Note:

- This attribute is only honored by `opt_signoff -hold`.
- If set, this attribute must be applied before `opt_signoff`.

opt_signoff_fix_ir_drop

```
opt_signoff_fix_ir_drop  {true | false}
```

Default: `false`

Data_type: Boolean

Fixes the IR drop voltages for all instances in the design.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_max_cap

```
opt_signoff_fix_max_cap  {true | false}
```

Default: `true`

Data_type: Boolean

Specifies that nets with max_cap violations should be fixed.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_max_transition

```
opt_signoff_fix_max_transition {true | false}
```

Default: `true`

Data_type: Boolean

Specifies that nets with max_tran violations should be fixed during DRV optimization.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- opt_signoff

opt_signoff_fix_osus_glitch

```
opt_signoff_fix_osus_glitch {true | false}
```

Default: `false`

Data_type: Boolean

Fixes nets with overshoot and undershoot glitches during DRV optimization.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- opt_signoff

opt_signoff_fix_si_slew

```
opt_signoff_fix_si_slew  {true | false}
```

Default: `false`

Data_type: Boolean

Reports signal integrity (SI) slews violations and fixed during DRV optimization. This is run as the first step of optimization before proceeding for any other incremental optimization.

This attribute must be set during ECO DB generation.

Note: If set, this attribute must be applied before `write_eco_opt_db` and `opt_signoff`.

Related Commands

- opt_signoff

opt_signoff_fix_slack_in_range_lower_limit

```
opt_signoff_fix_slack_in_range_lower_limit
```

Default: `-1e+30`

Data_type: Double

Sets the lower slack limit for a violated path to be considered during timing closure.

Related Commands

- `opt_signoff`

opt_signoff_fix_slack_in_range_upper_limit

```
opt_signoff_fix_slack_in_range_upper_limit
```

Default: `1e+30`

Data_type: Double

Sets the upper slack limit for a violated path to be considered during timing closure.

Related Commands

- `opt_signoff`

opt_signoff_fix_xtalk

```
opt_signoff_fix_talk  {true | false}
```

Default: `false`

Data_type: Boolean

Reduces the crosstalk on the net that violates the thresholds set by you.

Note: If set, this attribute must be applied before `write_eco_opt_db` and `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_xtalk_preserve_hold

```
opt_signoff_fix_talk_preserve_hold  {true | false}
```

Default: `false`

Data_type: Boolean

When this attribute is set to `true`, the Xtalk fixing engine is not allowed to degrade Hold timing.

Note:

- This attribute is only honored by `opt_signoff -drv` when Xtalk fixing is enabled.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_fix_xtalk_preserve_setup

```
opt_signoff_fix_talk_preserve_setup  {true | false}
```

Default: `false`

Data_type: Boolean

When this attribute is set to `true`, the Xtalk fixing engine is not allowed to degrade Setup timing.

Note:

- This attribute is only honored by `opt_signoff -drv` when Xtalk fixing is enabled.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_high_effort_cell_list

```
opt_signoff_high_effort_cell_list list of cells
```

Default: " "

Data_type: String

Enables the controlled utilization of high-leakage or high-power cells during signoff optimization. These cells are typically classified as dont_use or dont_touch due to their power characteristics; however, they may be beneficial in specific scenarios. When this attribute is specified, ensure that the don't_use constraint is set to `false` for cells included in `opt_signoff_high_effort_cell_list`.

- In setup timing optimization, the cells are considered only if they are essential to meet timing requirements without causing a significant increase in power consumption.
- In power optimization, the cells may be inserted if their usage leads to an overall decrease in power usage, such as enhancements in fanin/fanout cone.

This attribute permits considering the specified cells as high-effort resources, using them only when necessary to enhance timing or power efficiency, while maintaining the broader design objective of reducing power impact.

Related Commands

- `opt_signoff`

opt_signoff_hold_target_slack

```
opt_signoff_hold_target_slack value
```

Default: `0.0`

Data_type: Double

Specifies a target slack value in nanoseconds for hold timing optimize.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_hold_target_slack_per_view

```
opt_signoff_hold_target_slack_per_view {viewName 1 slack1 viewName2 slack2 … }
```

Default: " "

Data_type: String

When enabled, this attribute allows to force a specific hold target slack per view. If a given Hold active view is not listed in this attribute, the software will use the global Hold target slack set by the `opt_signoff_hold_target_slack` attribute for that view.

Note:

- This attribute is honored by Setup, Hold, Area, Power, and Leakage optimization engines.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_hold_xtalk_delta_threshold

```
opt_signoff_hold_xtalk_delta_threshold value_in_ns
```

Default: `0.3 ns`

Data_type: Double

Fixes the nets with the crosstalk delta delay value equal to or greater than a specified threshold value for hold views.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_hold_xtalk_slack_threshold

```
opt_signoff_hold_xtalk_slack_threshold value_in_ns
```

Default: `1000.0 ns`

Data_type: Double

Fixes nets with the slack threshold value equal to or less than a specified threshold value for hold views.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- opt_signoff

opt_signoff_keep_tmp_files

```
opt_signoff_keep_tmp_files {true | false}
```

Default: `false`

Data_type: Boolean

Keeps all the temporary files generated by DMMMC and Tempus clients for debugging purposes.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- opt_signoff

opt_signoff_ignore_drv_checks

```
opt_signoff_ignore_drv_checks  {true | false}
```

Default: `false`

Data_type: Boolean

Specifies that DRV checks should be ignored in setup/hold/area/leakage/dynamic optimization.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- opt_signoff

opt_signoff_include_hold_path_groups

```
opt_signoff_include_hold_path_groups
```

Default: " "

Data_type: String

Allows you to select which path group should be considered for Hold timing during `opt_signoff`. It is set before `write_eco_opt_db`, and also specified before `opt_signoff` in case of a different session.

Related Commands

- `opt_signoff`

opt_signoff_include_setup_path_groups

`opt_signoff_include_setup_path_groups`

Default: " "

Data_type: String

Allows you to select which path group should be considered for Setup timing during `opt_signoff`. It is set before `write_eco_opt_db`, and also specified before `opt_signoff` in case of a different session.

Related Commands

- `opt_signoff`

opt_signoff_inter_partition

`opt_signoff_inter_partition`

Default: " "

Data_type: String

Selects a list of partition module names to be optimized for the interface logic of the specified partition. During the High Capacity ECO flow, it will reduce the database having the interface logic of the listed partitions only. Each listed partition is treated separately, thus all from/to paths for each partition are included. This attribute must be applied before `write_eco_opt_db` and `opt_signoff`. In case of the Tempus Paradime flow or Certus, it must be applied before `distribute_read_db`.

Note:

It is mandatory to list the partition name inside the file specified with `opt_signoff_partition_list_file`.

- For High Capacity ECO flow, `opt_signoff_partition_list_file` should be specified before `distribute_read_db`.
- For non-High Capacity ECO flow, `opt_signoff_partition_list_file` should be specified before first optimization.

However, any partition name, which is a part of `opt_signoff_inter_partition` and missing in `opt_signoff_partition_list_file` will result in an error, as shown below:

```
**ERROR: (IMPESO-616):   Partition 'ptn_wrapper' is missing in
opt_signoff_partition_list_file file. Partition specified for inter/intra partition
fixing should be present in opt_signoff_partition_list_file list as well.
```

In case of Certus, since `opt_signoff_partition_list_file` is not required, the software will use `opt_signoff_distributed_optimization_block_restore_file` to identify the partition names.

Related Commands

- `opt_signoff`

opt_signoff_inter_partition_exclusive

```
opt_signoff_inter_partition_exclusive
```

Default: " "

Data_type: String

Selects a list of partition module pair names to be optimized for the interface logic between the specified partitions. During the High Capacity ECO flow, the tool will reduce the database having the interface logic of the listed partitions pairs only. Each listed partition pair is treated separately, all from/to paths between partition pairs are included.

Notes:

- This parameter must be applied before `write_eco_opt_db` and `opt_signoff`. However, in case of the Tempus Paradime flow or Certus, it must be applied before `distribute_read_db`.
- It is mandatory to list the partition name(s) inside the file specified using the `opt_signoff_partition_list_file` parameter:
    - For High Capacity ECO flow, the `opt_signoff_partition_list_file` parameter must be specified before `distribute_read_db`.

- For non-High Capacity ECO flow, the `opt_signoff_partition_list_file` parameter must be specified before the first optimization.

However, an error is issued when any partition name, which is a part of `opt_signoff_inter_partition_exclusive` is missing in `opt_signoff_partition_list_file`, as shown below:

```
**ERROR: (IMPESO-616): Partition 'tdsp_core' is missing in
opt_signoff_partition_list_file file. Partition specified for
inter/intra/inter_partition_exclusive partition fixing should be present in
opt_signoff_partition_list_file list as well.
```

In case of Certus, since `opt_signoff_partition_list_file` is not required, the software will use `opt_signoff_distributed_optimization_block_restore_file` to identify the partition names.

opt_signoff_intra_partition

```
opt_signoff_intra_partition
```

Default: " "

Data_type: String

Selects a list of partition module names to be optimized for the core logic of the specified partition. During the High Capacity ECO flow, it will reduce the database having the core logic of the listed partitions only. This attribute must be applied before `write_eco_opt_db` and `opt_signoff`. In case of the Tempus Paradime flow or Certus, it must be applied before `distribute_read_db`.

Note:

It is mandatory to list the partition name inside the file specified with `opt_signoff_partition_list_file`.

- For High Capacity ECO flow, `opt_signoff_partition_list_file` should be specified before `distribute_read_db`.
- For non- High Capacity ECO flow, `opt_signoff_partition_list_file` should be specified before first optimization.

However, any partition name, which is a part of `opt_signoff_intra_partition` and missing in `opt_signoff_partition_list_file` will result in an error :

```
**ERROR: (IMPESO-616):   Partition 'ptn_wrapper' is missing in
opt_signoff_partition_list_file file. Partition specified for inter/intra partition
fixing should be present in opt_signoff_partition_list_file list as well.
```

In case of Certus, since `opt_signoff_partition_list_file` is not required, the software will use `opt_signoff_distributed_optimization_block_restore_file` to identify the partition names.

Related Commands

- `opt_signoff`

opt_signoff_legal_only

```
opt_signoff_legal_only {true | false}
```

Default: `true`

Data_type: Boolean

Enables the tool to use the physical data to search for legal location while doing any buffer insertion or instance movement. This ensures that there is no instance overlap at the end of the ECO fixing stage, and after sourcing the ECO file in the implementation tool, you can directly run ECO routing.

When this parameter is set to `false`, `opt_signoff` will not check for legal location although physical data is provided. In this mode, opt_signoff behaves in a logical mode flow, which means that physical data will be ignored and this might create an instance overlap and will not check local density while inserting buffers. Also, the routing congestion checks are skipped when this parameter is set to `false`. So it is recommended that you first run ECO fixing in the legal-only mode (which is the default mode) and then, depending on the requirement, you may perform an incremental run by setting `opt_signoff_legal_only` to `false`.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_load_cell_list

```
opt_signoff_load_cell_list cell_list
```

Default: `""`

Data_type: String

Specifies the list of load cells to be inserted during Hold fixing, regardless of whether those cells have a dontUse attribute.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_max_cap_margin

```
opt_signoff_max_cap_margin cap_margin_value
```

Default: 0.0

Data_type: Double

Minimum value: -10.000000

Maximum value: 0.990000

Specifies the ratio by which the max_cap check should be relaxed or tightened.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_max_local_density

```
opt_signoff_max_local_density threshold_value
```

Default: `-1.0`

Data_type: Double

Applies a local placement density threshold that the software will honor during timing optimization. When the local density for a given area has overpassed the threshold, the software stops adding or moving more instances in that local area.

Using the default value, the software will not perform any local placement density checks because by default it is running with `opt_signoff_legal_only true`.

This attribute accepts values greater than 0. For example, to allow a maximum local placement density of 98%, set the value to 0.98.

Note:

- This attribute is honored by DRV, Setup, and Hold optimization engines.
- If set, this attribute must be applied before `opt_signoff`.
- The `opt_signoff_max_local_density` (greater than 1) and `opt_signoff_legal_only false` settings can cause some placement in later stages while doing eco_route.

Related Commands

- `opt_signoff`

opt_signoff_max_opt_runtime

```
opt_signoff_max_opt_runtime integer
```

Default: 0

Data_type: Integer

Specifies the timeout period for the timing or power optimization phase after which `opt_signoff` will exit the session. When the value of this attribute is set to a positive value, the software restricts the optimization runtime by that value, specified in minutes. The default value of this attribute is `0`. This means that there is no limit for the timeout period. When the timeout period is reached, the software will exit the optimization phase, and displays the Tempus prompt with a stable DB.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_max_paths

```
opt_signoff_max_paths integer
```

Default: `5000000`

Data_type: Integer

Specifies the max paths or points control for path-based analysis.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- opt_signoff

opt_signoff_max_runtime

```
opt_signoff_max_runtime integer
```

Default: 0

Data_type: Integer

Specifies the time out period (in minutes) after which opt_signoff will exit the session.

Note: If set, this attribute must be applied before opt_signoff.

Related Commands

- opt_signoff

opt_signoff_max_slack

```
opt_signoff_max_slack value
```

Default: 0.0

Data_type: Double

Specifies the maximum slack value to be used during ECO Timing DB generation when using PBA. Setting a positive opt_signoff_max_slack value forces the tool to retime positive GBA slack paths so that ECO optimization engine can benefit from an extra PBA Setup slack credit when doing Hold fixing or Power recovery. A positive opt_signoff_max_slack value implies larger runtime during ECO Timing DB generation, but tool will retime only as many paths as what is specified through the opt_signoff_max_paths attribute.

Note: If set, this attribute must be applied before write_eco_opt_db.

Related Commands

- opt_signoff

opt_signoff_max_transition_margin

```
opt_signoff_max_transition_margin tran_margin_value
```

Default: 0.0

Data_type: Double

Minimum value: -10.000000

Maximum value: 0.990000

Specifies the ratio by which the max_tran check should be relaxed or tightened.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_nworst

```
opt_signoff_nworst integer
```

Default: `50`

Data_type: Integer

Specifies the nworst control for path-based analysis.

Note: If set, this attribute must be applied before `write_eco_opt_db`.

Related Commands

- `opt_signoff`

opt_signoff_optimize_core_only

```
opt_signoff_optimize_core_only {true | false}
```

Default: `false`

Data_type: Boolean

Performs only reg2reg optimization, when set to `true`.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_optimize_replicated_modules

```
opt_signoff_optimize_replicated_modules {true | false}
```

Default: `false`

Data_type: Boolean

Allows the software to perform timing optimization inside replicated hierarchies while generating only one ECO file per sets of clones. The cell names of the replicated hierarchies have to be listed in a file pointed by the `opt_signoff_partition_list_file` attribute.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_optimize_sequential_cells

```
opt_signoff_optimize_sequential_cells {true | false}
```

Default: `true`

Data_type: Boolean

Allows the software to perform timing optimization for sequential cells.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_partition_list_file

```
opt_signoff_partition_list_file filename
```

Default: ""

Data_type: String

Enables the software to select the hierarchical instances that should be treated as partitions. The file specified with this parameter must contain a list of module names of those hierarchical instances. The `opt_signoff` command will generate an ECO file for the top module and the partitions (with relative placement locations for cells). The default ECO files are generated as mentioned below:

`top_eco_tempus.tcl` and `top_eco_innovus.tcl`; `blockA_eco_tempus.tcl` and `blockA_eco_innovus.tcl` and so on.

where the top ECO file contains all ECOs, which is a union of top and partition ECOs.

Notes:

- It is mandatory that the modules that are treated as partitions are of type FENCE.
- To avoid performing ECO in a given partition, you need to apply dont_touch attribute on that module.
- To perform top-level buffering only, you need to apply dont_touch on all the partitions.
- If set, this parameter must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_path_group_file_name

```
opt_signoff_path_group_file_name filename
```

Default: ""

Data_type: String

Allows you to pass a TCL file, which will be sourced on each Tempus client, after they loaded their respective saved session, which contains the path group definitions to be used by the tool when running in the High Capacity ECO mode. Those path groups can then be referred to `set_db opt_signoff_include_setup_path_groups` and `set_db opt_signoff_include_hold_path_groups`

applied on the manager session.

Note: This parameter is only applicable for the Tempus Paradime flow or Certus, and must be specified before `distribute_read_db`.

Related Commands

- `opt_signoff`

opt_signoff_pba_effort

```
opt_signoff_pba_effort {medium | high}
```

Default: `high`

Data_type: Enum

When high effort mode is specified, all the optimization engines will perform more aggressive timing closure or area/power recovery with respect to path-based (PBA) timing.

Note: If set, this attribute must be applied before `write_eco_opt_db` and `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_post_mask

```
opt_signoff_post_mask {true | false}
```

Default: `false`

Data_type: Boolean

Enables the timing closure engine to perform the following netlist changes without touching the base layer mask:

- Inserts the Gate Array buffers or an inverter pair in place of existing Gate Array filler cells.
- Resizes regular cells to Gate Array cells. It also resize Gate Array cells to Gate Array cells.
- Deletes a regular cell.

The software automatically identifies the Gate Array cells by checking which library instances have the same SITE name as the Gate Array filler cells specified by the user. Here, empty spaces will be

filled up by the Gate Array filler cells to ensure 100% placement density.

Note:

- This is supported by DRV, Setup, and Hold fixing.
- When `set_db add_tieoffs_cells {{<tieLo_name> <tieHi_name>}}` setting is used, the Metal ECO flow will insert the TieLo cells to connect dangling input pins, in case the existing TieLo cells in the floorplan are extremely far or too overloaded.

Related Commands

- `opt_signoff`

opt_signoff_power_opt_focus

```
opt_signoff_power_opt_focus {total | leakage | dynamic | leakageOnly | dynamicOnly}
```

Default: `total`

Data_type: Enum

Specifies whether a given netlist should be optimized only for leakage or dynamic power reduction, or more for leakage or dynamic power while still reducing the overall total power during total power optimization.

The following options are available:

- `total`: The power optimization engine focuses on getting the optimal total power in the design.
- `leakage`: The power optimization engine optimizes total power but focuses more on leakage power reduction than on dynamic power reduction.
- `dynamic`: The power optimization engine optimizes total power but focuses more on dynamic power reduction than on leakage power reduction.
- `leakageOnly`: The power optimization engine focuses only on leakage power reduction.
- `dynamicOnly`: The power optimization engine focuses only on dynamic power reduction.

Note:

- This attribute is honored by `opt_signoff -power` and also by `opt_signoff -drv/-setup/-hold` when the `opt_signoff_power_aware` attribute is set to `true`.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- opt_signoff

opt_signoff_post_sta_tcl

```
opt_signoff_post_sta_tcl filename
```

Default: ""

Data_type: String

Specifies the file containing the TCL code that will be executed immediately after signoff timing analysis per view.

Note: If set, this attribute must be applied before `write_eco_opt_db`.

Related Commands

- opt_signoff

opt_signoff_power_aware

```
opt_signoff_power_aware {true | false}
```

Default: `false`

Data_type: Boolean

Tempus ECO provides the ability to take power as part of the overall cost function when doing timing closure.

When this parameter is enabled before the start of an optimization session, the current optimization and any subsequent optimizations (if running in incremental mode) checks for power changes for all optimizers and transforms during any changes in the design.

In any of the above cases, the first priority remains getting best final timing but enabling this option will get there with possibly slighter ECO although resulting in lesser final power.

For example, while fixing a Setup timing path violation, by default, the tool chooses to size one instance to a high leakage cell, while with the `-opt_signoff_power_aware` mode, the tool tries to get to same timing by doing multiple sizing to non-high leakage cells.

Note :

- This feature requires power activity to be annotated on the netlist with the `report_power` command, either through external activity file (recommended) or using propagated activity based on some default toggling rate.
- This feature honors the `-opt_signoff_power_opt_focus` parameter, which means that you can decide to force the tool in reducing more the total or leakage or dynamic power increase.

Related Commands

- `opt_signoff`

opt_signoff_pre_sta_tcl

```
opt_signoff_pre_sta_tcl filename
```

Default: ""

Data_type: Boolean

Specifies the file containing the TCL code that will be executed immediately before signoff timing analysis per view.

Note: If set, this attribute must be applied before `write_eco_opt_db`.

Related Commands

- `opt_signoff`

opt_signoff_prefix

```
opt_signoff_prefix prefix
```

Default: ESO

Data_type: String

Specifies the prefix to be assigned to the new instances inserted.

Note: If set, this attribute must be applied before `opt_signoff`.

Related Commands

- opt_signoff

opt_signoff_read_eco_opt_db

```
opt_signoff_read_eco_opt_db dir_name
```

Default: ""

Data_type: String

Specifies the path of directory where previously generated database files are stored.

Related Commands

- opt_signoff

opt_signoff_read_irdrop_db

Default: ""

Data_type: String

Specifies the path of the directory where command, `debug_irdrop`, generated IR drop DB files are kept.

Related Commands

- opt_signoff

opt_signoff_read_smart_mmmc_db

```
opt_signoff_read_smart_mmmc_db <dirName>
```

Default: ""

Data_type: String

Points Tempus ECO to the ECO DB directory containing the additional compact timing information during the ECO fixing session. This attribute must be set before the design is loaded and must point to an ECO DB directory, which was previously generated through `write_eco_opt_db` using the setting, `set_db opt_signoff_create_smart_mmmc_db true`.

Note: Before `opt_signoff`, you still need to point to the ECO DB directory using the setting, `set_db opt_signoff_read_smart_mmmc_db <dirName>`.

Related Commands

- `opt_signoff`

opt_signoff_resize_inst

```
opt_signoff_resize_inst {true | false}
```

Default: `true`

Data_type: Boolean

Enables or disables resizing transform.

Related Commands

- `opt_signoff`

opt_signoff_retime

```
opt_signoff_retime {none |aocv | path_slew_propagation | aocv_path_slew_propagation | waveform_propagation | aocv_waveform_propagation}
```

Default: `path_slew_propagation`

Data_type: Enum

Selects the retime mode for path-based analysis.

For more details related to the parameters, see the `report_timing` command.

Related Commands

- `opt_signoff`

opt_signoff_retime_depth

```
-opt_signoff_retime_depth {default|infinite}
```

Default: `default`

Data_type: Enum

Retimes depth for exhaustive path-based analysis.

Related Commands

- opt_signoff

opt_signoff_retime_depth_early

```
-opt_signoff_retime_depth_early {default|infinite}
```

Default: `default`

Data_type: Enum

Retimes depth for exhaustive path-based early analysis.

Related Commands

- opt_signoff

opt_signoff_retime_mode

```
-opt_signoff_retime_mode {none|path|exhaustive}
```

Default: `path`

Data_type: Enum

Retimes mode for path-based analysis.

Related Commands

- opt_signoff

opt_signoff_route_congestion_threshold

```
opt_signoff_route_congestion_threshold threshold_value
```

Default: `0.7`

Data_type: Double

Allows to change the threshold value at which the software considers a local area for routing congestion buffering, to further allow any ECO change. This parameter accepts value between 0 (minimum) and 1 (maximum).

To tighten the routing congestion checks, apply a value between 0 and 0.7. To relax the routing congestion checks, apply a value between 0.7 and 1.

- *Min value*: `0`
- *Max value*: `1`

Note:

- This parameter is honored by DRV, Setup, Hold, Area, and Power optimization engines.
- If set, this parameter must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_routing_congestion_aware

```
opt_signoff_routing_congestion_aware {true | false}
```

Default: `false`

Data_type: Boolean

Checks routing congestion before adding buffers.

Related Commands

- `opt_signoff`

opt_signoff_select_drv_net_file

```
opt_signoff_select_drv_net_file {true | false}
```

Default: ""

Data_type: String

Specifies a file containing the list of nets to be optimized during DRV fixing. The fixing of max_transition, max_capacitance, SI glitch, SI Xtalk, and SI Slew violations occurs only on the specified nets and the other violating nets are ignored. The file specified includes one net per line.

Note:

- Wild cards and regular expressions are not supported in this file.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_select_drv_pin_file

```
opt_signoff_select_drv_pin_file
```

Default: ""

Data_type: String

Allows you to provide a list of instance pin, which is optimized during `opt_signoff -drv`. It is set before `write_eco_opt_db` for the High Capacity ECO flow, and also specified before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_select_hold_endpoints

```
opt_ signoff_select_hold_endpoints   file_name
```

Default: ""

Data_type: String

Specifies a file containing the list of selected endpoints for hold fixing.

Note: When including endpoints, the `opt_signoff_optimize_core_only` attribute must be set to `false`.

File format:

`<View> include/exclude <Endpoints>`

`<View>` can be specified as either viewName or V* (if the endpoints are to be included/excluded for all views)

`<Endpoints>` can be specified in three ways:

- As space-separated list of endpoint names: for example, E1 E2 E3.
- Slack range in nanosecond: `<minSlack> <maxSlack>`, where all endpoints with the slack greater than or equal to `minSlack` and slack less than or equal to `maxSlack` will be included/excluded, for example, -2.0 -1.0.
- E*, if all endpoints for the view need to be included/excluded.

Related Topics

- "Signoff ECO" chapter in the *Tempus Stylus User Guide*:
  - Optimization using Endpoint Control

Related Command

- `opt_signoff`

opt_signoff_select_setup_endpoints

`opt_ signoff_select_setup_endpoints` *`file_name`*

Default: `""`

Data_type: String

Specifies a file containing the list of selected endpoints for setup fixing.

Note: When including endpoints, the `opt_signoff_optimize_core_only` attribute must be set to `false`.

File format:

```
<View> include/exclude <Endpoints>
```

`<View>` can be specified as either viewName or V* (if the endpoints are to be included/excluded for all views)

`<Endpoints>` can be specified in three ways:

- As space-separated list of endpoint names: for example, E1 E2 E3.
- Slack range in nanosecond: `<minSlack> <maxSlack>`, where all endpoints with the slack greater than or equal to `minSlack` and slack less than or equal to `maxSlack` will be included/excluded, for example, -2.0 -1.0.
- E*, if all endpoints for the view need to be included/excluded.

Related Topics

- "Signoff ECO" chapter in the *Tempus Stylus User Guide*:
  - Optimization using Endpoint Control

Related Command

- `opt_signoff`

opt_signoff_setup_recovery

```
opt_signoff_setup_recovery {true | false}
```

Default: `false`

Data_type: Boolean

Specifies power-aware setup recovery for setup fixing.

Related Commands

- `opt_signoff`

opt_signoff_setup_target_slack

```
opt_signoff_setup_target_slack slack
```

Default: `0.0`

Data_type: Double

Specifies a target slack value in nanoseconds for Setup timing optimization.

Related Commands

- `opt_signoff`

opt_signoff_setup_target_slack_per_view

```
opt_signoff_setup_target_slack_per_view {viewName 1 slack1 viewName2 slack2 … }
```

Default=""

Data_type: String

When enabled, this attribute allows to force a specific setup target slack per view. If a given Setup active view is not listed in this attribute, the software will use the global Setup target slack set by the `opt_signoff_setup_target_slack` attribute for that view.

Note:

- This attribute is honored by Setup, Hold, Area, Power, and Leakage optimization engines.
- If set, this attribute must be applied before opt_signoff.

Related Commands

- `opt_signoff`

opt_signoff_setup_xtalk_delta_threshold

```
opt_signoff_setup_xtalk_delta_threshold value_in_ns
```

Default: `0.3 ns`

Data_type: Double

Fixes the nets with the crosstalk delta delay value equal to or greater than a specified threshold value for setup views.

Related Commands

- opt_signoff

opt_signoff_setup_xtalk_slack_threshold

```
opt_signoff_setup_xtalk_slack_threshold value_in_ns
```

Default: 1000.0 ns

Data_type: Double

Fixes the nets with the slack threshold value equal to or less than a specified threshold value for setup views.

Related Commands

- opt_signoff

opt_signoff_set_hold_endpoints_margin

```
opt_signoff_set_hold_endpoints_margin  file_name
```

Default: " "

Data_type: String

Specifies endpoint-specific margin for hold fixing.

Related Command

- opt_signoff

opt_signoff_set_setup_endpoints_margin

```
opt_signoff_set_setup_endpoints_margin file_name
```

Default: " "

Data_type: String

Specifies endpoint-specific margin for setup fixing.

Related Command

- opt_signoff

opt_signoff_skew_core_only

```
opt_signoff_skew_core_only  {true | false}
```

Default: false

Data_type: Boolean

When set to true, it does not allow the skewing engine to skew sequential elements that are part of an IO boundary timing path.

Note:

- This attribute is only honored by opt_signoff -setup.
- If set, this attribute must be applied before opt_signoff.

Related Command

- opt_signoff

opt_signoff_skip_drv_net_file

```
opt_signoff_skip_drv_net_file file_name
```

Default: " "

Data_type: String

Specifies a file containing the list of nets to be excluded during DRV fixing. The fixing of max_transition and max_capacitance violations occurs on nets other than specified nets. The file specified includes one net per line.

Note:

- Wild cards and regular expressions are not supported in this file.
- If set, this attribute must be applied before opt_signoff.

Related Command

- opt_signoff

opt_signoff_smart_mmmc_effort

```
opt_signoff_smart_mmmc_effort {low | medium | high}
```

Default: low

Data_type: Enum

Controls the trade-off between memory or runtime and timing QoR for timing or power ECO closure flow. This means that higher the effort, the lesser memory or runtime but lesser timing QoR while running opt_signoff.

- low: Is the recommended (default) setting and ensures there is no timing QoR loss.
- medium: Reduces memory and runtime significantly with minimal timing QoR loss.
- high: Reduces memory and runtime further when compared to the medium mode, but with additional timing QoR loss.

In the low effort mode, which is the default and recommended setting, there is no compromise on timing QoR.

The medium and high effort modes can be used for initial rounds of ECO timing/power ECO closure flow.

Note: This parameter must be set before the design is loaded.

Related Commands

- opt_signoff

opt_signoff_smart_mmmc_force_keep_hold_view_list

```
opt_signoff_smart_mmmc_force_keep_hold_view_list <string>
```

Default: " "

Data_type: String

List of hold views, which have to be retained after SmartMMMC view compaction. These views represent the minimum hold views retained by the system, on top of which additional views can be

retained.

Related Commands

- `opt_signoff`

opt_signoff_smart_mmmc_force_keep_setup_view_list

```
opt_signoff_smart_mmmc_force_keep_setup_view_list <string>
```

Default: " "

Data_type: String

List of setup views, which have to be retained after SmartMMMC view compaction. These views represent the minimum setup views retained by the system, on top of which additional views can be retained.

Related Commands

- `opt_signoff`

opt_signoff_smart_mmmc_max_view_count

```
opt_signoff_smart_mmmc_max_view_count {{setup <maxViewCount>} {hold <maxViewCount>} {drv <maxViewCount>}}
```

Default: " "

Data_type: String

Allows you to force the software to collect only a given number of views to cover Setup timing, Hold timing, and DRV. It acts as a limit to prevent smartMMMC from selecting more views than what is specified by the user. This attribute is especially useful in combination with `opt_signoff_smart_mmmc_reduce_clients` as it enables a user-defined limit on hardware resources for Tempus clients.

Note:

- This attribute is only honored by Tempus Paradime or Certus, and should be set before design loading.
- When `opt_signoff_smart_mmmc_max_view_count` is specified, `opt_signoff_smart_mmmc_effort_level` is ignored.

- When `opt_signoff_smart_mmmc_max_view_count` is specified, you need to provide `maxViewCount` for each Setup, Hold, and DRV field.
- `maxViewCount` should be equal or greater than 1 for setup and hold but can be 0 for drv only if you do not specify `set_analysis_view -drv` earlier.

Related Commands

- `opt_signoff`

opt_signoff_smart_mmmc_reduce_clients

```
opt_signoff_smart_mmmc_reduce_clients {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the SmartMMMC engine evaluates which views should be retained and available on Tempus clients. This helps to reduce the number of Tempus clients started by Tempus Paradime or Certus.

Default: `false`

Note:

- This attribute is only honored by Tempus Paradime or Certus, and must be set before design loading.
- The Tempus STA sessions must have been saved with `opt_signoff_create_smart_mmmc_db true`.
- SmartMMMC must be enabled through `set_distributed_mmmc_mode -enable_smart_mmmc`.

Related Commands

- `opt_signoff`

opt_signoff_swap_inst

```
opt_signoff_swap_inst {true | false}
```

Default: `true`

Data_type: Boolean

Enables/disables VT swap resizing. The setup/leakage optimizers do not honor this attribute and always perform VT-swapping.

Related Commands

- `opt_signoff`

opt_signoff_post_mask_gate_array_filler_cells

`opt_signoff_post_mask_gate_array_filler_cells {`*`list_of_Gate_Array_filler_cells_name`*`}`

Default: `""`

Data_type: String

Enables the software to list all the Gate Array filler cells that can be deleted or inserted back.

Note: This attribute is applicable in the Metal ECO mode only.

Related Commands

- `opt_signoff`

opt_signoff_post_mask_decap_cells

`opt_signoff_post_mask_decap_cells` *`list_of_decap_cells`*

Default: `""`

Data_type: String

Lists all the programmable decap cells that are allowed to be removed/reinserted in post-mask mode.

Note: This attribute is applicable in the post-mask mode only.

Related Commands

- `opt_signoff`

opt_signoff_post_mask_filler_cells

`opt_signoff_post_mask_filler_cells` *`list_of_filler_cells`*

Default: `""`

Data_type: String

Lists all the programmable filler cells that are allowed to be removed in post-mask mode.

Note: This attribute is applicable in the post-mask mode only.

Related Commands

- `opt_signoff`

opt_signoff_post_mask_logic_cells

`opt_signoff_post_mask_logic_cells` *`list_of_regular_cells`*

Default: `""`

Data_type: String

Lists all the programmable regular logic/ECO cells that are allowed to be inserted in place of filler/decap cells during optimization in post-mask mode.

Note: This attribute is applicable in the post-mask mode only.

Example:

The following set of commands allow Hold timing closure in post-mask mode using programmable filler cells:

```
set_db opt_signoff_post_mask true
set_db opt_signoff_post_mask_filler_cells {ln10_ecofiller40x … ln10_ecofiller4x}
set_db opt_signoff_post_mask_logic_cells {ln10_ecofillsgcap40x … ln10_ecofillsgcap4x}
set_db opt_signoff_post_mask_decap_cells {ln10_ecodlybt01 ln10_ecobuft04 …
ln10_ecoand02 … ln10_ecoinv04 …}
opt_signoff -hold
```

Related Commands

- `opt_signoff`

opt_signoff_touch_partition_list

```
opt_signoff_touch_partition_list string
```

Default = " "

Data_type: String

Selects a list of partition module names to be considered as touch only during optimizations. Any partition that is not specified using this parameter will be considered as don't_touch during optimization.

Notes:

- This parameter is exclusive to `opt_signoff_dont_touch_partition_list`.
- This parameter must be applied before `opt_signoff` and can be changed between each call of `opt_signoff`.

opt_signoff_upsize_in_power_opt

```
opt_signoff_upsize_in_power_opt {true | false}
```

Default: `false`

Data_type: Boolean

Allows the power optimization engine to perform cell upsizing in order to recover more power.

Note :

- This attribute is only honored by `opt_signoff -power`.
- If set, this attribute must be applied before `opt_signoff`.

Related Commands

- `opt_signoff`

opt_signoff_verbose

```
opt_signoff_verbose {true | false}
```

Default: `false`

Data_type: Boolean

Specifies whether the command should output log file reporting with verbosity or not.

Related Commands

- opt_signoff

opt_signoff_write_eco_opt_db

```
opt_signoff_write_eco_opt_db dir_name
```

Default: ecoTimingDB

Data_type: String

Specifies a path of directory where generated database files are kept.

Related Commands

- opt_signoff

read_hierarchical_def

```
read_hierarchical_def
[-help]
def_files <...>
[-add_blockage]
[-delete_blockages_over_partition_macro]
[-ignore_drcfill]
[-incremental_ilm_def]
[-preserve_shape]
[-rdl_def string]
[-rdl_orient orientation]
[-rdl_placement {x y}]
[-secondary {{suffix def_file_list}...}]
[-skip_pg]
[-skip_fillers]
[-skip_nets]
[-top_cell_orient orientation]
[-top_cell_placement {x y}]
[-top_scope]
[-transform_file string]
[-skip_routing {[pg|signal] [regular|special] [high_layer_routing]}... [-
to_layer routing_layer_name]]
[-use_top_def_die_area]
[-modules <vCell>+ | -hInsts <hInst>+]
```

Merges DEF data from various blocks to a single chip database. The `read_hierarchical_def` command provides the following features:

- Takes the TOP DEF file and a set of block DEF files and merges them into a single chip database.
- The block and TOP Verilog files (or their DEF files) can be specified in any order. However, if there are empty Verilog module stubs of blocks in some of the parent Verilog files, ensure that the parent Verilog files are specified prior to the child Verilog files.
- There is no need to specify the LEF files for the blocks.
- During the course of this command, TOP rows will be cut around the block boundary, and all block rows will be brought back as is.
- Partition definitions are created for each of the assembled block as per the DIE AREA statement in the block DEF.

- The wire at the block boundaries will be stitched with the electrically connected net's wires at the TOP.
- All orientations of blocks are supported for assembly. Repeated blocks are also supported. Non-R0 master is also supported.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `read_hierarchical_def` parameter. For a detailed description of the command and its parameters, use the `man` command: `man read_hierarchical_def`
`-add_blockage`	Specifies that the missing instances from the netlist should be converted into placement blockages.
`def_files <...>`	Specifies DEF files to be merged.
`-delete_blockages_over_partition_macro`	Deletes the top design placement and routing (both rectangular and rectilinear) blockages overlapping partitions modules.
`-hInsts <hInst>+`	Specifies the list of hierarchical instances for which physical data needs to be loaded. The tool then reads data from def file(s) associated to the specified hInstance(s).
`-ignore_drcfill`	Ignores the special routing with DRCFILL shape attribute.
`-incremental_ilm_def`	Incrementally flatten the ILM def.
`-modules <vCell>+`	Specifies the list of modules for which physical data needs to be loaded. The tool then reads data only from def file(s) associated to the specified module(s).

`-preserve_shape`	Preserves the shapes and geometries when the DEF data from various blocks is merged to a single chip database.
`-rdl_def` *string*	Specifies RDL def file.
`-rdl_orient` *orientation*	Specifies orientation for RDL def.
`-rdl_placement {`*x y*`}`	Specifies offset of RDL def.
`-secondary {{`*suffix def_file_list*`}...}`	Specifies the secondary DEF file(s) to load.
`-skip_fillers`	Skip merging filler and decap cells.
`-skip_nets`	Skips merging signal nets section.
`-skip_pg`	Skips merging routing on pg nets.
`-skip_routing {[pg|signal] [regular|special] [high_layer_routing]}...`	

	Skips merging routing based on the settings specified. This setting can be any combination of the three optional keywords from the following set: • `[pg|signal]`: Specifies the type of the net on which the routing to be skipped. `pg - pg net`, `signal - signal net`, `both` if not specified. • `[regular|special]`: Specifies the type of routing to be skipped. `regular - regular routing`, `special - special routing`, `both` if not specified. • `[high_layer_routing]`: Specifies that only the routing on high layers to be skipped, where the layers are calculated automatically. If not specified, the routing on all layers are skipped. The layers can be specified with `-to_layer` parameter. In that case, the routing on the layers from top to the specified layer are skipped. **Note**: You need to specify the `-to_layer` parameter explicitly if `high_layer_routing` is specified in `-skip_routing`; otherwise, `high_layer_routing` is not applicable. For example, you can specify multiple sets of arguments to this parameter, as shown below: `-skip_routing {{pg regular} {signal special high_layer_routing}}`
`-to_layer` *routing_layer_name*	Skips the routing of layers from top to the specified layer (inclusive) when the `high_layer_routing` option is specified for the `-skip_routing` parameter.
`-top_cell_orient` *orientation*	Specifies orientation for topcell.
`-top_cell_placement {`*x y*`}`	Specifies placement for topcell.

`-top_scope`	Merges routing on top scope nets only.
`-transform_file` *string*	Reports the final x and y offsets and orientation in the top-level of each LEF block that got replaced by DEF.
`-use_top_def_die_area`	Enables the use of TOP DEF DIE AREA.

Examples

Below is a sample template script that describes how to run Tempus ECO for DRV/Setup/Hold optimization on a hierarchical design in Stylus mode :

```
set_multi_cpu_usage -local_cpu 16

read_physical -lef {tech.lef stdCell.lef }

read_mmmc viewDefinition_stylus.tcl

read_netlist “top.v.gz blockA.v.gz"

init_design


set_db add_fillers_cells "list_of_filler"

read_hierarchical_def top.def.gz blockA.def.gz


read_spef -rc_corner corner_worst_CMAX “top_CMAX.spef.gz blockA_CMAX.spef.gz"

read_spef -rc_corner corner_worst_RMIN “top_RMIN.spef.gz blockA_RMIN.spef.gz"


set_db delaycal_enable_si true

set_db opt_signoff_partition_list_file partition.txt

set_db opt_signoff_allow_multiple_incremental true

set_db opt_signoff_optimize_replicated_modules true

set_db opt_signoff_read_eco_opt_db ecoTimingDB


set_db opt_signoff_eco_file_prefix DRV
```

```
opt_signoff  -drv

set_db opt_signoff_eco_file_prefix SETUP

opt_signoff  -setup

set_db opt_signoff_eco_file_prefix HOLD

opt_signoff  -hold
```

Related Commands

- opt_signoff
- read_spef

read_partition

```
read_partition
[-help]
-def <defFile>
-verilog <vFile>
```

The `read_partition` command allows you to load the Verilog/DEF files for a module that is currently treated as a black box (represented by LEF/LIB). This command is needed for hierarchical designs in order to assemble the top-level database with all the block-level databases. The tool will automatically create a fence for every partition loaded.

Note: It is mandatory that LEF files for every partitions be provided when loading the top-level database.

Parameters

`-def`	Specifies the DEF file name.
`-verilog`	Specifies the Verilog file name.

Examples

- The following command assembles the ptn1 block-level database to the top level database already present in the memory.

  ```
  tempus> read_partition -verilog ptn1.v -def ptn1.def
  ```

Related Commands

- `read_hierarchical_def`

write_eco_opt_db

```
write_eco_opt_db
[-help]
```

Saves an ECO Timing DB after performing timing analysis, in a non-distributed mode or in a distributed-MMMC session. In case several views are active at the same time, ECO Timing DB for each of them is saved. The data is saved in the directory pointed by the `opt_signoff_write_eco_opt_db` attribute.

Note: This command requires a 64-bit executable.

Examples

The following command saves an ECO Timing DB in the `mydb` directory.

```
tempus> set_db opt_signoff_write_eco_opt_db mydb

tempus> write_eco_opt_db
```

Related Commands

- `opt_signoff`

26

Timing Analysis Commands and Attributes

- check_boundary_model_consistency
- check_ipd_logic
- check_ldb_version
- check_timing
- check_timing_library
- check_timing_library_consistency
- commit_boundary_model_constraints
- compare_timing_context
- create_analysis_view
- create_basic_path_groups
- create_boundary_model
- create_constraint_mode
- create_delay_corner
- create_glitch_boundary_model
- create_ipd_configuration
- create_library_set
- create_opcond
- create_parasitic_coordinate_transform
- create_pg_net_group
- create_property_alias
- create_rc_corner
- create_timing_condition
- delete_analysis_view
- get_capacitance_unit
- get_clock_network_objects
- get_clock_to_clock_association

- get_constant
- get_interactive_constraint_modes
- get_propagated_clock
- get_resistance_unit
- get_socv_rc_variation_factor
- get_socv_reporting_nsigma_multiplier
- get_time_unit
- gui_show_timing_map
- list_libraries
- merge_timing_reports
- read_boundary_model
- read_ipd_scope
- read_path_descriptions
- read_sdf
- read_spef
- read_timing_context
- read_twf
- report_analysis_coverage
- report_analysis_summary
- report_analysis_views
- report_annotated_assertions
- report_annotated_check
- report_annotated_delay
- report_annotated_parasitics
- report_aocv_derate
- report_boundary_model_constraints
- report_case_analysis
- report_cell_instance_timing
- report_clock_gating_check
- report_clock_network_transitions
- report_clock_propagation

- report_clock_sense
- report_clock_timing
- report_clocks
- report_constraint
- report_cppr
- report_critical_insts
- report_critical_nets
- report_critical_pins
- report_design
- report_fanin
- report_fanout
- report_freq_violation
- report_inactive_arcs
- report_instance_library
- report_ipd_configuration
- report_ipd_crossings
- report_lib_arcs
- report_min_pulse_width
- report_mode
- report_nets
- report_parasitic_coordinate_transforms
- report_path_based_aocv_derate
- report_path_exceptions
- report_path_groups
- report_pg_net_group
- report_ports
- report_reference
- report_slack_histogram
- report_socv_interconnect_variation
- report_socv_library
- report_statistical_timing_derate_factors

- report_timing
- report_timing_context_clock_mapping
- report_timing_derate
- report_timing_lib
- report_timing_robustness
- report_timing_summary
- report_wire_load
- reset_instance_library
- reset_path_adjust
- reset_path_adjust_group
- reset_sdf_assertions
- reset_timing_derate
- reset_wire_load_mode
- reset_wire_load_model
- reset_wire_load_selection_group
- select_boundary_model_constraints
- set_analysis_view
- set_boundary_model_clock_mapping
- set_boundary_model_view_mapping
- set_default_view
- set_instance_library
- set_interactive_constraint_modes
- set_io_thresholds
- set_ipd_rail_voltage
- set_max_cap_per_freq
- set_max_cap_per_freq_transition
- set_max_transition_per_freq
- set_multi_input_switching_mode
- set_opcond
- set_path_adjust
- set_path_adjust_group

- set_socv_constraint_config
- set_socv_rc_variation_factor
- set_socv_reporting_nsigma_multiplier
- set_table_style
- set_threshold_voltage_group
- set_timing_context_clock_mapping
- set_timing_context_view_mapping
- set_timing_derate
- set_timing_library
- set_vt_skew_config
- set_vt_skew_derate
- set_wire_load_mode
- set_wire_load_model
- set_wire_load_selection_group
- timing_enable_boundary_model_eco
- timing Category Attributes
- update_analysis_view
- update_constraint_mode
- update_delay_corner
- update_io_latency
- update_library_set
- update_model_context
- update_opcond
- update_rc_corner
- update_timing
- update_timing_condition
- write_boundary_model
- write_global_slack_report
- write_ipd_scope
- write_ldb
- write_loop_break_sdc

- write_model_context
- write_path_descriptions
- write_sdc
- write_sdf
- write_timing_context
- write_timing_windows
- write_twf

check_boundary_model_consistency

```
check_boundary_model_consistency
[-help]
```

Performs several checks on loaded models.

When this command is run, the software performs the following checks:

- Global variables and modes set during the boundary model (BM) generation have the same settings in the loading run.
- If a boundary model sub-module is also instantiated outside of the BM, and the BM definition is used for the module.
- The view definition used to generate the BM is consistent with the loading run's view definition. (BM generation views can be a superset.)
- The loading run's active views are all active during the BM generation.
- Constant logic in the BM generation run has the same constants in the loading run.
- Power nets/voltages are consistent between BM generation and BM loading.
- Chip size is consistent between BM generation and BM loading.

A report file is written for each loaded boundary model. The default name of the report is ${auto_file_prefix} <cell_name>_bmodel_check.txt, written in to the $auto_file_dir. You can specify the name of the report file using the `read_boundary_model -report_file` command parameter.

Parameters

`-help`	Prints out the command usage.

Related Commands

- `read_boundary_model`

check_ipd_logic

```
check_ipd_logic
[-help]
[-outfile <filename>]
{ [-multi_pd_type <enum> [-verbose ]]}
```

Performs IPD logic checks and reports power domain crossings information based on data and clock path checks.

Parameters

`-help`	Prints out the command usage.
`-multi_pd_type <enum>`	Specifies the type of IPD logic check that needs to be performed. You can specify the following options: • `clock_divergence`: Enables similar crossings on the divergent clock check. • `clock_crossing`: Enables multiple PD crossings on clock path checks. • `data_crossing`: Enables multiple PD crossings on data path checks. • `all`: Enables all the crossing checks, except `cppr_aware`. • `cppr_aware`: Enables checkd for multiple power domains post the common point.
`-outfile <filename>`	Specifies the name of the output report file (suffix .rpt) to be generated.
`-verbose`	Specifies to generate detailed `report_timing` output for each check with .tcl suffix.

Examples

- The following command reports all the IPD logic checks in the design:

```
tempus> check_ipd_logic -multi_pd_type all -outfile ipd_logic_check_report -verbose
```

The output of `ipd_logic_check_report.rpt` is as shown below:

```
################################################
# Multiple Power Domain Crossings On Data Paths  #
################################################
1.
Beginpoint: ff1/D (clk1)
Endpoint: ff2/Q (clk1)
PD1->PD2 lvl_cell1/Y
PD2->PD3 lvl_cell2/Y
2.
Beginpoint: ff3/D (clk1)
Endpoint: ff4/Q (clk2)
PD1->PD2 lvl_cell3/Y
PD2->PD3 lvl_cell2/Y

##########################
# Clock Divergence Data  #
##########################
1.
Beginpoint: ff4/D (clk2)
Endpoint: ff5/Q (clk2)
PD1->PD2 lvl_cell5/Y (L) level_cell6/Y (C)

########################################
# Multiple Clock Power Domain Crossings  #
########################################

####################################
#Early Check Path Count Summary Output#
####################################

Multiple IPD Data Crossing : 2
-----------------------------------
Count            Power Domain
-----------------------------------
2              PD1->PD2->PD3

IPD Clock Divergence : 1
-----------------------------------
Count           Power Domain
-----------------------------------
1               PD1->PD2

Multiple IPD Clock Crossing : 0
-----------------------------------
Count           Power Domain
-----------------------------------
```

- The output of ipd_logic_check.tcl is as shown below:

```
##################################################
# Multiple Power Domain Crossings On Data Paths  #
##################################################
report_timing -through lvl_cell1/Y -through lvl_cell2/Y
report_timing -through lvl_cell3/Y -through lvl_cell2/Y

#########################
# Clock Divergence Data  #
#########################
report_timing -from ff4/D -clock_from clk1 -to ff5/Q -clock_to clk2

########################################
# Multiple Clock Power Domain Crossings  #
########################################
```

- The following command generates a report showing cppr_aware checks:

```
tempus> check_ipd_logic -multi_pd_type cppr_aware -outfile cppr_aware_report
```

cppr_aware_report.rpt

```
################################################
# Multiple Power Domain Crossings Paths  #
################################################
1.
Beginpoint : inst1/Q(clk1)
Endpoint : inst2/D(clk2)
PD1 -> PD2 cinst1/Y (L)
PD2 -> PD3 inst3/Y (D)
PD3 -> PD4 inst4/Y (D)
PD1 -> PD4 cins2/Y (C)
2.
Beginpoint : inst11/Q(clk21)
Endpoint : inst12/D(clk12)
PD1 -> PD2 cinst13/Y (L)
PD2 -> PD3 cinst123/Y (L)
PD3 -> PD4 inst314/Y (D)
PD4 -> PD5 inst414/Y (D)
PD5 -> PD3 inst42/Y (D)
PD1 -> PD2 cins246/Y (C)
PD2 -> PD3 cins26/Y (C)
#####################################
#Early Check Path Count Summary Output#
#####################################
Multiple Power Domain Paths : 2
-------------------------------------------------------------------------------
Count          |Power Domains
-------------------------------------------------------------------------------
1              |PD1->PD2(L) PD2->PD3->PD4(D) PD1->PD4(C)
1              |PD1->PD2->PD3(L) PD3->PD4->PD5->PD3(D) PD1->PD2->PD3(C)
```

The types of crossing in the path is indicated by the following abbreviations:

- C : Capture Path
- L : Launch Path
- D: Data Path

elated Commands

- `set_ipd_rail_voltage`
- `create_pg_net_group`
- `report_pg_net_group`
- `create_ipd_configuration`
- `report_ipd_configuration`
- `write_ipd_scope`
- `read_ipd_scope`

Related Information

- For more information, refer to the Inter Power Domain (IPD) Analysis chapter in the *Tempus Stylus User Guide*.

check_ldb_version

```
check_ldb_version
-library string
[-help]
```

Checks the library database version. If the library version is not updated, the details of missing features and recommendations for recompiling the library database are displayed.

Parameters

`-help`	Prints out the command usage.
`-library string`	Specifies the library database (LDB) file whose version is to be printed.

Examples

The following example displays the missing features/recommendations for the library database cell_w.db:

```
tempus > check_ldb_version -ldb cell_w.ldb
```

The output is as shown below:

```
LDB file compiled with tool version: v15.20 ((64bit) mm/dd/yyyy 08:40 (Linux 2.6.18-371.el5))
File Size is xxxxx
Source Tool: Innovus
```

check_timing

```
check_timing
[-help]
[-check_only warning_list]
[-clock_crossing_output_format {summary | csv}]
[-exclude_warnings warning_list]
[-include_warnings warning_list]
[-tcl_list]
[-type type_list]
[-verbose]
[-view view_name]
[> <file_name[.gz]>]
[>> <file_name[.gz]>]
[port_or_pin_list]
```

Performs a variety of consistency and completeness checks on the timing constraints specified for a design. Valid types are: `clocks`, `clock_clipping`, `constant_collision`, `endpoints`, `input`, and `loops`.

The checks include arrival time and external delay (or required time) for each clock in a multiple clock system. In addition, clock connectivity and data connectivity are checked to make sure the clock or data is propagated as expected. Clock gating points are also reported.

Use this command on generic or mapped netlists. The `check_timing` command considers constants applied to a pin while reporting warnings.

Use the `check_timing` command before using any commands for timing analysis. Warnings are displayed when problems occur. The Table below lists the `check_timing` warnings. For best results, re-run the `check_timing` command after resolving each warning. Redirecting the output to a file is the quickest run method.

Parameters

`-help`	Prints out the command usage.

`-check_only` *warning_list*	Allows you to specify warning types. For example, the following command lists the specified warnings only: `check_timing -check_only {uncons_endpoint clock_crossing}` This list can also include warnings that are not listed by default. Valid entries in *warning_list* are: clock_clipping_freq, clock_clipping_gate, clock_crossing, clock_expected, clock_gating_controlling_edge_unknown, clock_gating_no_data, clock_missing_at_sequential_output, clock_not_propagated, clocks_masked_by_another_clock, const_collision, data_check_multiple_reference_signal, data_check_no_reference_signal, ideal_clock_waveform, invalid_pll_configuration, loop, master_clk_edge_not_reaching, missing_clock_groups, missing_drive, multiple_clock, no_drive, no_gen_clock_source, no_input_delay, partial_input_delay, pulse_non-pulse_clock_merge, signal_level, uncons_endpoint, uncons_endpoints_due_to_case_analysis
`-clock_crossing_output_format {summary | csv}`	Specifies the format of the clock crossing report to be generated - as a summary or in csv format.
`{>` | `>>}` *file_name*	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. The *`filename`* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.

`-exclude_warnings` *warning_list*	Excludes the specified warning types when generating the report. Use this parameter to exclude any of the warnings included in the report by default. By default, the software includes the following warnings in the report: `clock_expected`, `loop`, `ideal_clock_waveform`, `master_clk_edge_not_reaching`, `partial_input_delay`, `missing_drive`, `no_input_delay`, `no_drive`, `no_gen_clock_source`, `uncons_endpoint`,`uncons_endpoints_due_to_case_analysis`. For a complete list of warnings and their descriptions, refer to Table
`-include_warnings` *warning_list*	Includes the specified warning types when generating the report. Use this parameter to add any of the warnings not included in the report by default. By default, the software reports all warnings except the following: `clock_clipping_gate`, `const_collision`, `clock_crossing`,`clock_not_propagated`, `missing_drive`, `multiple_clock`, `data_check_multiple_reference_signal`, `data_check_no_reference_signal`, `clock_gating_no_data`, `clock_gating_controlling_edge_unknown`, `clock_missing_at_sequential_output`, `signal_level`, and `clocks_masked_by_another_clock`. For a complete list of warnings and their descriptions, refer to Table.
port_or_pin_list	Lists pins or ports. For example: `check_timing -verbose [get_ports in*]`
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *filename* parameters are mutually exclusive; you cannot specify them together.
`-type` *type_list*	Limits the checks to the specified type. *Default*: All check types are reported.

	`clock_clipping`: Warns when there is a problem with clock gating that can result in clock clipping. Reports the following warnings: • clock_clipping_freq • clock_clipping_gate
	`clocks`: Reports the following clock warnings: • clock_crossing • clock_expected • clock_not_propagated • ideal_clock_waveform • master_clk_edge_not_reaching • no_gen_clock_source
	`clock_gating_inferred`: Warns when inferred clock gating checks are not analyzed by the software, even if a clock signal reaches and is used downstream the clock tree. Reports the following warnings. • clock_gating_controlling_edge_unknown • clock_gating_no_data
	`constant_collision`: Warns when there is a constant collision on a net connected to the pin or if there is a contradiction on a pin. Constant collision occurs when a net is driven simultaneously by conflicting values and a constant contradiction occurs when a constant constraint on a pin conflicts with the driven value.
	`endpoints`: For output ports, warns if no output delay constraint is applied to the port or if a specific (min/max rise/fall) output delay is missing. It also warns if an output delay constraint is applied with no clock information. For all endpoints (output ports and register data pins), it warns if any incoming signal is unconstrained. Also warns if a clock arrives where clock is not expected, or if multiple signals arrive at an endpoint.

	`inputs`: Reports the following input delay, arrival time assertion, and drive assertion warnings: • partial_input_delay • missing_drive • no_input_delay • no_drive
	`loops`: Warns when a combinational loop is detected. Reports the loop and the arc used to break the loop. The `-pin` option is ignored in this case. Also reports loops caused by generated clocks.
`-verbose`	Gives detailed information about the warnings. When specified, warnings are grouped and sorted by warning type. *Default:* Gives a summary of how many warnings of each type exists.
`-view`	Reports warning(s) that belong to the specified view.

Command Order

Use this command after setting all the constraints but before optimizing your design or generating any timing reports.

Warning Messages

The following table shows the `check_timing` warning messages.

check_timing Warning Messages

Warning	Description
`clock_clipping_gate`	**Warning**: Clock clipping possible due to wrong gate type or wrong trigger for data input.
`clock_clipping_freq`	**Warning**: Clock clipping possible due to incompatible clock signal and data signal frequencies.

`clock_crossing`	**Warning**: Clock domains interact. **Comments:** Checks clock interactions between multiple clock domains in the design. When you specify this parameter, the Timing Check Crossing Clocks report shows the interaction of clocks in the design and indicates whether the paths between the clocks are `False` or `Partial False`. If a clock launches one or more paths, which are captured by other clocks, the report lists all pairs of crossing clocks. • This includes inferred clock gating with different reference and signal clock. • If a path has been disabled using the `set_disable_timing` command, crossing clocks on that path are not reported. If all the paths between two clocks are false paths or they are exclusive/asynchronous clocks, the path is marked as `False`. The false path can be defined on clocks, for example, `set_false_path -clock_to CLK` or on pins or ports, such as, `-through buf1/Y`. If only part of paths are set as false paths or exclusive/asynchronous clocks, the path is marked as `Partial False`.
	• If a false path is defined only for one check type (say, setup), the crossing clocks are marked as `Partial False` if the simultaneous setup hold mode is set to `on`. The crossing clocks are marked `False` if the analysis type is setup and the simultaneous setup hold mode is set to `off`. • If only one edge (rise/fall) of a particular clock pair has false paths (say, using `-clock_fall`), then the path is marked as a `Partial False`.

`clock_expected`	**Warning:** Clock not found where clock is expected. **Comments:** Indicates that no clock signal is defined at the clock pin. See create_clock.
`clock_gating_controlling_edge_unknown`	**Warning:** Unknown controlling edge to perform clock gating check. **Comments:** Reports library cells with complex functions (MUX or AOI) where controlling edge of signal to perform clock gating check cannot be determined automatically. You need to specify correct edge by `-high`/`-low` option using the `set_clock_gating_check` command to resolve such warning. **Note:** The software does not report this warning when either: • `set_disable_clock_gating_check` command is applied on such pins, or • global settings to disable clock gating checks are on.
`clock_gating_no_data`	**Warning:** No data signal reaching clock gating enable pin **Comments:** Reports clock gating enable pins where no valid data signal arrives. You need to do either of the following: • specify valid data signal to reach enable pin of clock gating cells, or • remove any path exceptions or disabled checks that blocks valid data signal to reach enable pin **Note:** The software does not report this warning when either • `set_disable_clock_gating_check` is applied on such pins, or • global settings to disable clock gating checks are on.

`clocks_masked_by_another_clock`	**Warning:** Clock(s) in fanin cone are dropped due to another clock **Comments:** Reported timing pins on which creation of a regular or a generated clock causes clock(s) in fanins of such pins have been blocked or masked from propagating further. However, for pins having only master clock waveform on the fanin, this warning is not reported. You can use the detailed mode (`-verbose` option) to report names of all the incoming clock waveforms that have been blocked.
`clock_missing_at_sequential_output`	**Warning:** Found latency phase at sequential output on clock tree instead of clock phase **Comments:** Reports output pins of sequential elements that lie on source clock tree for the generated clocks created downstream and do not have any valid clock phase. This indicates possible missing clock related checks (e.g., clock gating) on downstream clock tree. You can add clocks on this pin or downstream clock tree only if no valid clock exists until downstream generated clock creation point.
`clock_not_propagated`	**Warning:** Clock not propagated **Comments:** Reports ideal clocks reaching the reference pin of a timing check.
`const_collision`	**Warning:** Constant collision **Comments:** See `set_case_analysis`.
`data_check_multiple_reference_signal`	**Warning:** Multiple clocked signals are reaching data check reference pin **Comments:** Reports on reference pins for non sequential timing check (data check) where more than one reference data arrives.

`data_check_no_reference_signal`	**Warning:** No clocked signal is reaching data check reference pin **Comments:** Reports reference pins for non-sequential timing check (data check) where no valid reference signal (data phase) arrives. You can do one of the following to resolve such warnings: • add valid reference signals to reach at such reference pins • remove path exceptions or disabled arcs that are blocking such reference signals
`ideal_clock_waveform`	**Warning:** Clock waveform is ideal. **Comments:** Reports clocks that are ideal and are not propagated. This warning type is reported by default (unless `-exclude_warning` or `-check_only` parameter is specified). You can use the `set_propagated_clock` command to avoid this warning.
`invalid_pll_configuration`	**Warning:** No valid path exists from output pin to feedback pin of PLL. **Comments**: Shows PLL (phase locked loop) check warnings. These checks allow you to identify and fix issues in the PLL flow.
`latch_fanout_clock_connectivity`	**Warning**: Latch and its fanout latch work on same clock. **Comments**: Checks for a latch and its fanout latch(es) that works on the same clock.
`loop`	**Warning:** Timing loop found in the design **Comments:** Indicates that there are timing loops in the design. It also includes loops caused by generated clocks.

`master_clk_edge_not_reaching`	**Warning:** Master clock edge does not reach the generated clock target. **Comments:** This warning means that rise/fall latency edge is not reaching to create generated clock root from the master clock. You can verify if the path that is seeing this warning is actually broken, or edge is somewhere disabled earlier on this path. There is a global `timing_enable_genclk_edge_based_source_latency` that controls how the software chooses generated clock source latency paths. Details of this global can be found at Tempus text command reference.
`missing_clock_groups`	**Warning**: Set clock group related checking **Comments:** Reports any potential issues with the defined clock grouping data.
`partial_input_delay`	**Warning:** Indicates that a rise or fall input delay assertion is not applied.
`missing_drive`	**Warning:** Missing specific (rise/fall) drive constraint **Comments:** Either a rise or fall drive constraint is applied and the other one is missing.
`multiple_clock`	**Warning:** Multiple clocks are reaching clock reference pin. **Comments:** Reports clock pins of sequential or clock gating cells where more than one clock waveform arrives. This warning is for information only and the software performs timing analysis with respect to all such clocks.
`no_input_delay`	**Warning:** No input delay assertion with or without `-clock` option specified. **Comments:** See `set_input_delay`.
`no_drive`	**Warning:** No drive constraint **Comments:** See `set_driving_cell` and `set_drive`.

`no_gen_clock_source`	**Warning:** No clock source found for generated clock. **Comments:** Indicates that the source pin of the clock generated by the `create_clock` command is not driven by a clock source.
`pulse_non-pulse_clock_merge`	**Warning**: Merge of pulse and non-pulse clock paths. **Comments**: Reports points where a pulse clock and a non-pulse clock merges.
`signal_level`	**Warning:** Driver pin voltage value do not match with sink pin voltage level **Comments:** Reports net driver/receiver pin pairs for which the pin voltage does not match.
`uncons_endpoint`	**Warning:** No constrained signal reaching pin. **Comments:** For output ports: Primary output port has no implicit timing check. Use `set_output_delay`, `set_max_delay`, or `set_min_delay` to create a valid constraint. For register data pins: There is no valid clock reaching the CK pin of the register data pin. Check for missing `create_clock` for the clock network. The endpoints are reported as unconstrained in both in early and late timing. This does not cover all the cases and scenarios because of which endpoint can become unconstrained. To cover all possible cases, you can use the `report_analysis_coverage` command.
`uncons_endpoints_due_to_case_analysis`	**Warning:** Unconstrained signal arriving at end point due to case analysis **Comments:** Reports unconstrained endpoints due to case analysis. To enable this warning, you must set the `timing_report_check_timing_unconstrained_endpoints_due_to_constants` attribute to `true`.

Examples

Here are some examples.

- Consider the following example:

```
in---------|>-----------|>---------out

buf1 buf2
```

The Timing Check Crossing Clocks reports False or Partial False in some of the scenarios given below:

`a) set_input_delay` with respect to CLK1 on in and set_output_delay with respect to CLK2 on out:

```
From Clock         Crossing Clocks
-----------------------------------------
CLK1               CLK2
```

`b) set_input_delay` with respect to CLK1 on in, set_output_delay with respect to CLK2 on out, and set_false_path -to CLK2 (clock-based false path):

```
From Clock         Crossing Clocks
-----------------------------------------
CLK1               CLK2  (False)
```

c) If the set_false_path becomes set_false_path -rise_to CLK2, the following is reported:

```
From Clock         Crossing Clocks
-----------------------------------------
CLK1               CLK2  (Partial False)
```

The following command displays the default summary report:

```
tempus> check_timing

--------------------------------------------------------------------------------
                              TIMING CHECK SUMMARY
--------------------------------------------------------------------------------
Warning                       Warning Description                     Number of
                                                                      Warnings
--------------------------------------------------------------------------------
ideal_clock_waveform          Clock waveform is ideal                        1
clock_expected                Clock not found where clock is expected        3
master_clk_edge_not_reaching  Master clock edge does not reach the
```

```
                              generated clock target                      2
no_drive                      No drive assertion                          8
no_input_delay                No input delay assertion with respect
                              to clock                                    4
uncons_endpoint               Unconstrained signal arriving at end point  5
-------------------------------------------------------------------------
```

- The following command displays a specified type of warning using the `-type` option if a timing loop is found in the design:

```
tempus>  check_timing -type loops

-------------------------------------------------------
                  TIMING CHECK SUMMARY
-------------------------------------------------------
Warning      Warning Description              Number of
                                              Warnings
-------------------------------------------------------
loop         Timing loop found in the design  0
-------------------------------------------------------
```

- The following command provides more information about problems and prints a summary using the `-verbose` option:

```
tempus> check_timing -verbose

-------------------------------------------------------------------------------
                            TIMING CHECK SUMMARY
-------------------------------------------------------------------------------
Warning                       Warning Description                       Number of
                                                                        Warnings
-------------------------------------------------------------------------------
ideal_clock_waveform          Clock waveform is ideal                       1
clock_expected                Clock not found where clock is expected       3
master_clk_edge_not_reaching  Master clock edge does not reach the
                              generated clock target                        2
no_drive                      No drive assertion                            8
no_input_delay                No input delay assertion with respect to clock  4
uncons_endpoint               Unconstrained signal arriving at end point    5
-------------------------------------------------------------------------------
----------------------------------------
TIMING CHECK DETAIL
----------------------------------------
Pin         Warning
-----------------------------------------------------
u99/CK      Clock not found where clock is expected
u5/CK       Clock not found where clock is expected
u8/CK       Clock not found where clock is expected
u8/Q        Master clock edge does not reach the target for generated clock GCLK4
```

```
CLK1        No drive assertion
in1         No drive assertion
in1         No input delay assertion with respect to clock
out1        Unconstrained signal arriving at end point
-----------------------------------------------------
-------------------------
TIMING CHECK IDEAL CLOCKS
-------------------------
Clock  Waveform
-------------------------
CLK1
-------------------------
```

- When `timing_report_check_timing_unconstrained_endpoints_due_to_constants` global variable is enabled, the case analysis warnings are reported as shown below:

```
tempus 11> check_timing
-------------------------------------------------------------------------------
                              TIMING CHECK SUMMARY
-------------------------------------------------------------------------------
Warning              Warning Description                              Number of
                                                                      Warnings
-------------------------------------------------------------------------------
clock_expected       Clock not found where clock is expected            4202
no_drive             No drive assertion                                 241
no_input_delay       No input delay assertion with respect to clock     160
uncons_endpoint      Unconstrained signal arriving at end point         145259
uncons_endpoints_due_to_case_analysis
                 >>>>Unconstrained signal arriving at end point due to case analysis
                                                                        5969437
-------------------------------------------------------------------------------
```

Related Commands

- report_timing

check_timing_library

```
check_timing_library
[-help]
[-check_power]
[-lib_cell string]
[-library string]
[-out_file string]
[-report_missing_power_only]
```

Checks the contents of the timing library and reports any inconsistencies in the log file.

You can use the `check_timing_library` command after importing the design and timing libraries.

Parameters

`-help`	Prints out the command usage.
`-check_power`	Checks if power constructs, such as, internal_power, leakage_power, or ecsm_power, are defined for the cells. **Note**: The `-check_power` parameter is optional. However, if you specify `-report_missing_power_only`, then you must specify `-check_power`. If you use the `-library` parameter, you do not have to specify the `-check_power` parameter, because when a library-specific cell is given as input, the software automatically checks for power constructs. *Default:* Only checks if timing is defined for the cells.
`-lib_cell string`	Specifies the name of the library cell. **Note**: The `-lib_cell` parameter is optional. However, if you specify the `-library` parameter, then you *must* specify `-lib_cell`.

`-library` *string*	Specifies the timing library in which to check a particular cell. You must specify the `-lib_cell` parameter with `-library`, to specify the particular library cell to check. **Note:** If you use the `-library` parameter, you do not have to specify the `-check_power` parameter, because when a library-specific cell is given as input, the software automatically checks for power constructs.
`-out_file` *string*	Saves the report output to the specified file.
`-report_missing_power_only`	Reports only those cells that do not have any defined internal_power (at the pin level), or leakage_power. You must specify the `-check_power` parameter with the `-report_missing_power_only` parameter.

Examples

- The following command checks if timing is defined for all cells that have been read into the design, and writes a report to the log file:
  ```
  check_timing_library
  ```
- The following command checks if timing is defined for all cells that have been read into the design, and writes a report to a file named `CheckTiming1`:
  ```
  check_timing_library -out_file CheckTiming1
  ```
- The following command checks if timing and power constructs are defined for all cells that have been read into the design, and writes the report to the log file:
  ```
  check_timing_library -check_power
  ```
- The following command checks if timing and power constructs are defined for all cell that have been read into the design, and writes the report to a file named `CheckTiming2`:
  ```
  check_timing_library -out_file CheckTiming2 -check_power
  ```
- The following command checks if timing is defined for all cells with the name `A01` in all timing libraries, and writes the report to a file named `CheckTiming3`:
  ```
  check_timing_library -out_file CheckTiming3 -lib_cell A01
  ```
- The following command checks if timing and power constructs are defined for all cells with the name `A01`, in all timing libraries, and writes the report to a file named `AllA01Check`:
  ```
  check_timing_library -out_file AllA01Check -lib_cell A01 -check_power
  ```

- The following command checks if timing and power constructs are defined for the cell with the name `A01` in the timing library `minFile1.lib`, and writes the report to a file named `A01Check`:

  ```
  check_timing_library -out_file A01Check -lib_cell A01 -library minFile1.lib
  ```

  Note: When you use the `-library` parameter, you do not have to specify the `-check_power` parameter, because when a library-specific cell is given as input, the software automatically checks for power constructs.

- The following command checks if the internal_power and/or leakage_power constructs are defined for all cells that have been read into the design, and reports only those cells that do not have the constructs defined. The software writes the report to a file named `MissingPower`:

  ```
  check_timing_library -out_file MissingPower -check_power -report_missing_power_only
  ```

check_timing_library_consistency

```
check_timing_library_consistency
[-help]
```

Checks for missing libraries for instances in the active view.

This command checks whether the library for the cell associated with the instance is present in the associated power domain library set of the instance in the active views.

Parameters

`-help`	Prints out the command usage.

Examples

- The following command displays a report showing missing libraries:
  ```
  check_timing_library_consistency
  ```

  The software displays the following information:

  ```
  Checking the Library binding for instance in active view 'vA'
  Pass. All instances have library definition in view 'vA'
  Checking the Library binding for instance in active view 'vB'
  No library found for instance 'U1', cell 'AND2X1', in powerdomain 'PD_BAD', of view 'vB'
  Fail. Missing Library for some instance found in view 'vB'
  Checking the Library binding for instance in active view 'vAB'
  Pass. All instances have library definition in view 'vAB'
  ```

commit_boundary_model_constraints

```
commit_boundary_model_constraints
[-help]
```

Allows the software to pull up a subset of constraints from a block run into the top boundary model session.

When this command is specified, the constraints are applied to instances of a boundary modeled block in a top level boundary model session. For glitch boundary models, glitch constraints, such as, `define_bundled_bus`, `set_glitch_threshold`, `set_cdb_binding`, and `set_quiet_attacker`, are pulled up.

Parameters

`-help`	Prints out the command usage.

compare_timing_context

```
compare_timing_context
[-help]
[-abs_thresh float]
[-instances string]
[-module string]
-out_file string
[-pct_thresh float]
-reference string
[-sort_by {abs | pct}]
-target string
[-type {clocks |  latency | arrival | required | slew | load | constant | glitch}]
[-verbose]
[-view string]
```

Compares two timing contexts generated by Tempus.

The contents of context like clocks, clock latency, arrival time, required, slew, load, constant, and glitch are compared. To perform context comparison, it is required to generate context using the `-include_text_format` parameter of the `write_timing_context` command.

Parameters

`-help`	Prints out the command usage.
`-abs_thresh` *float*	Specifies the threshold value to compare the context data. The absolute threshold value is specified in pico seconds (ps). *Default*: `0`
`-instances` *string*	Specifies the list of instances for which context comparison will be done.
`-module` *string*	Specifies the list of modules for which context comparison will be done.
`-out_file` *string*	Specifies the output file that will show the differences between reference and target context data.
`-pct_thresh` *float*	Specifies the percentage threshold in percent (%). If the difference is greater than both the specified `-abs_thresh` and `-pct_thresh` thresholds, the software considers the comparison as “Fail”. *Default*: `0`
`-reference` *string*	Specifies the reference path of the context directory.
`-sort_by {abs | pct`	Sorts the report based on the absolute tolerance or percent tolerance. By default, the report is sorted based on the percentage difference.
`-target` *string*	Specifies the name of target context directory used for comparison. **Note**: The missing entries in target or reference will be represented by “`-`”.
`-type {clocks | latency | arrival | required | slew | load | constant | glitch}`	Specifies the type of text data to be compared. The `latency` option includes comparison of the latency, uncertainty, or incremental delay values. By default, all the text data values are compared.
`-verbose`	Reports the failed as well as passed comparison data.
`-view` *string*	Specifies the MMMC analysis view for which context comparison needs to be done.

Context Comparison Report Sections and Examples

The following are the sections of the context comparison report along with examples:

Section 1: Comparison Summary

Shows the summary of total and failed entries/ports in reference and target context comparison. The header

contains the view, module, and instance for which the comparison is reported.

```
----------------------
Top View: view1
Module: Top
Top Instance: inst1
---------------------
------------------------------------------------------------
        Comparison Summary
------------------------------------------------------------

Section              Failed Entries   Total Entries   Failed Ports   Total Ports
Clocks               0                1               0              0
Constant Value       0                3               0              3
Latency              48               48              2              2
Arrival Time         0                0               0              0
Required             0                0               0              0
Input Transition     0                12              0              3
Output Load          0                4               0              1
Glitch               2                8               1              3
```

Section 2: Detailed Clock Comparison

This section provides the details of differences in clock definitions between the reference and target contexts.

```
-------------------------------------------------------------------------------------------------------
                        Clocks Comparison
-------------------------------------------------------------------------------------------------------
Clock       Ref Waveform        Ref Period      Target Waveform        Target Period     Status

-------------------------------------------------------------------------------------------------------
CK1         [0 0.000 0.500]      1.000          [0 0.000 0.500]        1.000             Pass

-------------------------------------------------------------------------------------------------------
```

Section 3: Detailed Constant Value Comparison

This section provides the details of differences in constants between the reference and target contexts.

```
-----------------------------------------------------
                    Constants Comparison
-----------------------------------------------------
Port         Ref Constant   Target Constant  Status
-----------------------------------------------------
prt1         0              1                Fail
-----------------------------------------------------
```

Section 4: Detailed Clock Latency Comparison

Provides details of differences in clock latency, pin-based uncertainty, and incremental delays between the reference and target contexts.

```
-------------------------------------------------------------------------------------
                    Clock Latency Comparison
-------------------------------------------------------------------------------------
```

```
Port   Clock     Type          Data        Ref     Target    Abs Diff  % Diff   Status
-------------------------------------------------------------------------------------
P1     TCK1(P)   latency       early_rise  10.12   10.23     0.11      1.08     Fail
P1     TCK1(P)   incr_delay    early_rise  0.2     0.2       0         0        Pass
P1     TCK1(P)   uncertainty   early_rise  0.1     0.2       0.1       100      Fail
P1     TCK1(P)   latency       late_rise   11.12   11.12     0         0        Pass
-------------------------------------------------------------------------------------
```

Section 5: Detailed Input Arrival Comparison

Provides details of differences in input arrivals between the reference and target contexts.

```
-------------------------------------------------------------------------------------
                    Input Arrival Comparison
-------------------------------------------------------------------------------------
Port           Clock          Data          Ref       Target    Abs Diff   % Diff   Status
-------------------------------------------------------------------------------------
test_scanpt    portclk1(P)    early_rise    1206.32   1214.33   8.0017     0.6633    Fail
test_scanpt    portclk0(P)    early_rise    1206.32   1214.33   8.0017     0.6633    Fail
test_scanpt    portclk1(P)    early_fall    1202.58   1210.53   7.9497     0.6610    Fail
test_scanpt    portclk0(P)    early_fall    1202.58   1210.53   7.9497     0.6610    Fail
-------------------------------------------------------------------------------------
```

Section 6: Detailed Required Time Comparison

This section provides the details of differences in required time between the reference and target contexts.

```
-------------------------------------------------------------------------------------
Required Time Comparison
-------------------------------------------------------------------------------------
Port         Clock     Data          Ref       Target     Abs Diff    % Diff      Status

-------------------------------------------------------------------------------------
tst_scopt   ekph1(P)  early_fall   4.2486    3.2270      -1.0216     -24.0456    Fail
tst_scopt   ekph1(P)  early_rise   -5.9780   -7.0736     -1.0956     18.3272     Fail
tst_scopt   emph1(N)  early_fall   -13.8087  -15.9599    -2.1512     15.5786     Fail
tst_scopt   emph1(N)  early_rise   -23.0335  -25.2794    -2.2459     9.7506      Fail
-------------------------------------------------------------------------------------
```

Section 7: Detailed Input Port Transition Comparison

Provides the details of differences in transition times on input ports between the reference and target contexts.

```
         Input Transition Comparison
-------------------------------------------------------------------------
Port     Data         Ref       Target    Abs Diff   % Diff    Status
-------------------------------------------------------------------------
clk2     early_rise   1.510     1.510     0.000      0.000     Pass
clk2     early_fall   0.772     0.772     0.000      0.000     Pass
-------------------------------------------------------------------------
```

Section 8: Detailed Output Port Load Comparison

Provides the details of differences in output port load between the reference and target contexts.

```
Output Load Comparison
```

```
-------------------------------------------------------------------------
Port     Data         Ref         Target      Abs Diff     % Diff    Status
-------------------------------------------------------------------------
Out      early_rise   171.374     171.374     0.000        0.00      Pass
Out      early_fall   171.374     171.374     0.000        0.000     Pass
-------------------------------------------------------------------------
```

Section 9: Detailed Glitch Comparison

Provides the details of differences in glitch value on the pins connected to the input ports between the reference and target contexts.

```
-------------------------------------------------------------------------------
                     Glitch Comparison
-------------------------------------------------------------------------------
Port      Pin Name     Data    Ref       Target     Abs Diff    % Diff     Status
-------------------------------------------------------------------------------
clk2      ff3/CK       VH      96.552    96.611     0.059       0.061      Fail
clk2      ff3/CK       VL      95.742    95.776     0.034       0.036      Fail
-------------------------------------------------------------------------------
```

Related Commands

- `read_timing_context`
- `write_timing_context`
- `report_timing_context_clock_mapping`

create_analysis_view

```
create_analysis_view
[-help]
-constraint_mode modeName
-delay_corner dcCornerObj
-name viewName
```

Creates an analysis view object that associates a delay calculation corner with a constraint mode. An analysis view object provides all of the information necessary to control a single multi-mode multi-corner analysis.

After creating analysis views, use the `set_analysis_view` command to specify which views to use for setup and hold optimization or timing analysis.

Use this command after creating constraint modes (`create_constraint_mode`) and delay calculation corners (`create_delay_corner`).

Parameters

`-help`	Prints out the command usage.
`-name` *viewName*	Specifies the name of the analysis view being defined.
`-constraint_mode` *modeName*	Specifies the name of the constraint mode to associate with this analysis view. A constraint mode groups a set of constraints to be used with any given analysis. To create the constraint mode, use the `create_constraint_mode` command.
`-delay_corner` *dcCornerObj*	Specifies the name of the delay calculation corner object to associate with this analysis view. A delay calculation corner object contains all of the information needed to control delay calculation for a specific analysis view. To create a delay calculation corner, use the `create_delay_corner` command.

Examples

- The following command creates an analysis view called `missionSlow` using the constraint mode `missionSetup` and the delay calculation corner `dcWCCOM`:

```
tempus> create_analysis_view
-name missionSlow
-constraint_mode missionSetup
-delay_corner dcWCCOM
```

- The following command creates an analysis view called `missionFast` using the constraint mode `missionHold` and the delay calculation corner `dcBCCOM`:

```
tempus> create_analysis_view
-name missionFast
-constraint_mode missionHold
-delay_corner dcBCCOM
```

Related Commands

- `set_analysis_view`
- `create_constraint_mode`
- `create_delay_corner`

create_basic_path_groups

```
create_basic_path_groups
[-help]
[-expanded]
[-reset]
```

Explicitly creates standard path groups irrespective of the design stage. The following path groups are created:

- reg2reg (register-to-register)
- reg2cgate (register-to-clock-gate)

Note: If a design does not have any reg2reg paths, or reg2cgate paths, this command will not create the corresponding path group(s).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_basic_path_groups` parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_basic_path_groups`.
`-expanded`	When this parameter is specified, the following path groups are created: • reg2reg • reg2cgate • in2reg • reg2out, and • in2out When this parameter is not specified, only the default path groups, reg2reg and reg2cgate, are created. **Note**: Timing optimization will consider these path groups in the same way as paths grouped using the `group_path` command.
`-reset`	Deletes all existing path groups. This is equivalent to running the `reset_path_group` command. **Note**: After deleting the existing path groups, new path groups, reg2reg and reg2cgate are created.

Examples

- The following command is used to create the standard path groups:

  ```
  @[DEV]tempus 4> create_basic_path_groups
  ```

  The software returns the following information:

  ```
  Created reg2reg path group

  Created reg2cgate path group
  ```

Related Topics

- group_path
- reset_path_group

create_boundary_model

```
create_boundary_model
[-help]
-dir string
[-exclude_constraints]
[-exclude_no_delay_ports]
[-include_full_netlist]
[-include_full_parasitics]
[-overwrite]
[-skip_constant_ports]
[-skip_constants]
[-skip_ports string]
[-validate_local_cpu integer]
[-absolute_tolerance float]]
```

Initiates the creation of a boundary model from a full design.

Parameters

`-help`	Prints out the command usage.
`-absolute_tolerance float`	Specifies the absolute tolerance value of 5ps during self validation.
`-dir string`	Specifies the directory for boundary model files.
`-exclude_constraints`	Does not include constraints in the generated model.
`-exclude_no_delay_ports`	When specified, the software skips the ports for which the `set_input_delay` constraint(s) is not applied.
`-include_full_netlist`	Keeps the full block netlist in a boundary model.
`-include_full_parasitics`	Allows you to write out RCDB parasitics for all the lower level blocks as part of the current boundary model. When specified, the software writes out full parasitics by running the `write_rcdb` command under the hood.
`-skip_constant_ports`	Ignores the timing endpoints driven exclusively by constant ports.
`-overwrite`	Allows overwrite of the existing directory.
`-skip_constants`	Does not include constant logic in the model.
`-skip_ports string`	Does not include fanin/fanout from the specified ports in the model.
`-validate_local_cpu integer`	Specifies the number of available local CPUs for boundary model validation.

Examples

- The following command creates a boundary model for the top cell being timed and writes the model files into the ./model_dir directory. Any existing directory contents are removed.

```
create_boundary_model -dir ./model_dir -overwrite
```

Related Commands and Variables

- read_boundary_model
- timing_full_context_flow

create_constraint_mode

```
create_constraint_mode
[-help]
-name modeName
-sdc_files {file1.sdc file2.sdc ...}
[-ilm_sdc_files {file1.sdc file2.sdc ...}]
[-tcl_variables {{var_name1 value1} {var_name2 value2} {var_name3 value3} ...}]
```

Associates a list of SDC constraint files with a specified constraint mode name, for multi-mode multi-corner analysis. This constraint mode name can be referred to later when creating analysis views. A constraint mode defines one of possibly many different functional, test, or Dynamic Voltage and Frequency Scaling (DVFS) modes of a design. SDC files can be shared by many different constraint modes, and the same constraint mode can be associated with multiple analysis views.

SDC files typically contain timing analysis information, such as the clock specifications, case analysis constraints, I/O timings, and path exceptions that make each mode unique. If the ILM is being used, the timing system utilizes the files specified with `-ilm_sdc_files` during the flatten ILM mode (`flatten_ilm`); otherwise, the system uses the files specified with `-sdc_files`.

Parameters

`-help`	Prints out the command usage.
`-ilm_sdc_files {file1.sdc file2.sdc ...}`	Specifies a Tcl list of ILM SDC files to be included in the constraint mode.
`-name modeName`	Specifies the name of the constraint mode to be created.
`-sdc_files {file1.sdc file2.sdc ...}`	Specifies a Tcl list of SDC files to be included in the constraint mode.
`-tcl_variables {{var_name1 value1} {var_name2 value2} {var_name3 value3} ...}`	
	Specifies a list of tcl variables and values to be used in the constraint mode.

Examples

- The following command groups the SDC files `io.sdc`, `mission1-clks.sdc`, and `mission1-except.sdc` to create a mode object named `missionSetup`:

```
tempus> create_constraint_mode -name missionSetup
   -sdc_files [list io.sdc mission1-clks.sdc mission1-except.sdc]
```

- The following command groups the SDC files `io.sdc`, `mission1-clks.sdc`, `mission1-except.sdc`, and `dont_use.sdc` to create a mode object named `missionHold`:

```
tempus> create_constraint_mode -name missionHold
  -sdc_files [list io.sdc mission1-clks.sdc mission1-except.sdc dont_use.sdc]
```

- The following command groups the SDC files `test-io.sdc`, `test-clks.sdc`, and `test-except.sdc` to create a mode object named `testHold`:

```
tempus> create_constraint_mode -name testHold
  -sdc_files [list test-io.sdc test-clks.sdc test-except.sdc]
```

Related Commands

- `update_constraint_mode`
- `create_analysis_view`

create_delay_corner

```
create_delay_corner
[-help]
[-early_estimated_worst_irdrop_factor <percent>]
-name delayCornerName
[-early_irdrop_files list_of_files]
[-early_rc_corner rcCornerObj]
[-early_temperature_files list_of_files]
[-early_timing_condition conditional_pairs]
[-late_estimated_worst_irDrop_factor percent]
[-late_irdrop_data files_or_directories]
[-late_irdrop_files list_of_files]
[-late_rc_corner rcCornerObj]
[-late_temperature_files list_of_files]
[-late_timing_condition conditional_pairs]
[-irdrop_data files_or_directories]
[-irdrop_files list_of_files | -irdrop_data files_or_directories]
[-pg_net_voltages voltage_pairs]
[-rc_corner rcCornerObj]
[-si_enabled <true | false>]
[-temperature_files list_of_files]
[-timing_condition conditional_pairs]
```

Creates a named delay calculation corner object that can be referenced later when creating an analysis view. A delay calculation corner provides all of the information necessary to control delay calculation for a specific view. Each corner contains information on the libraries to use, the operating conditions with which the libraries should be accessed, and the RC extraction parameters to use for calculating parasitic data. Delay corner objects can be shared by multiple top-level analysis views.

The `-early_*` and `-late_*` parameters should not be used to represent best and worst case corners. Instead use separate delay corners to define Best-Case and Worst-Case differences. Use the -early_* and -late_* parameters within a single delay calculation corner to control on-chip variation. In this latter situation different conditions or libraries will be used for the launch and capture paths of individual timing paths.

Note: A single delay calculation corner object specifies the delay calculation rules for the entire design. If a design includes power domains, the delay calculation corner can contain domain-specific subsections that specify the required operating condition information, and any necessary timing library rebinding for the power domain. Use the `update_delay_corner` command to add a power domain definition to a delay calculation corner.

Parameters

`-help`	Prints out the command usage.
`-early_estimated_worst_irDrop_factor` *`per cent`*	Specifies the early voltage derating factor. When this parameter is specified, the software performs delay calculations using the voltage = nomVolt * (1 – value). *Default*: 0.0
`-early_irdrop_data` *`files_or_directories`*	Specifies the IR drop files or directories to apply when calculating the early arrival times at a single delay corner.
`-early_irdrop_files` *`list_of_files`*	Specifies the list of IR drop files to apply when calculating early arrival times at a single power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. **Note**: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
`-early_rc_corner` *`rcCornerObj`*	Specifies the RC corner object to associate with the early corner object.
`-early_temperature_files` *`list_of_files`*	Specifies the name of the temperature file for temperature aware delay calculation for early corner.
`-early_timing_condition` *`conditional_pairs`*	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object.
`-irdrop_data` *`files_or_directories`*	Specifies the IR drop files or directories to apply to both the early and late delay calculation for the specified delay corner object.

`-irdrop_files` *list_of_files*	Specifies the list of IR drop files to apply to both early and late delay calculation for this power domain object. This parameter is used primarily to configure single-corner or Best-Case Worst-Case (BC-WC) analysis modes. **Note**: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
`-late_estimated_worst_irDrop_factor` *percent*	Specifies the late voltage derating factor. When this parameter is specified, the software performs delay calculations using the voltage = nomVolt * (1 – value). *Default*: 0.0
`-late_irdrop_data` *files_or_directories*	Specifies the IR drop files or directories to apply when calculating the late arrival times at a single delay corner.
`-late_irdrop_files` *list_of_files*	Specifies the list of IR drop files to apply when calculating late arrival times at a single power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. **Note**: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
`-late_temperature_files` *list_of_files*	Specifies the name of the temperature file for temperature aware delay calculation for late corner.
`-late_rc_corner` *rcCornerObj*	Specifies the RC corner object to associate with the late corner object.

`-late_timing_condition` *conditional_pairs*	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object. **Note**: The `-early_timing_condition` and `-late_timing_condition` parameters should be specified together.
`-name` *delayCornerName*	Specifies the delay calculation corner to which to add the power domain definition, or to which to update the delay corner or power domain attribute values.
`-pg_net_voltages` *voltage_pairs*	Specifies the power/ground net voltages to be used for both early/late timing analysis.
`-rc_corner` *rcCornerObj*	Specifies the new RC corner object to associate with this delay corner.
`-si_enabled {true | false}`	Controls signal integrity (SI) analysis. When set to `true`, the analysis of all MMMC views will include SI analysis by default. You can use this parameter to further determine which analysis views receive SI analysis and which do not. When this parameter is set to `false` on a delay corner, the software will disable SI analysis for all the views that reference that corner. By default (or when this parameter is set to `true`), SI analysis is computed for all the views related to that corner - whenever SI is globally enabled. (The global control of SI analysis is determined by the `delaycal_enable_si true` attribute setting.)
`-temperature_files` *list_of_files*	Specifies the name of the temperature file for temperature aware delay calculation for single corner.
`-timing_condition` *conditional_pairs*	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object.

Examples

- The following example specifies a list of conditional pairs of domains/supply sets and the timing conditions associated with the specified delay corner object:

```
create_delay_corner -name ccworst_setup\
-timing_condition {tc_1 PD1@tc_2 PD2@tc_2 PD3@tc_2 PD4@tc_2}
```

Related Commands

- update_delay_corner

create_glitch_boundary_model

```
create_glitch_boundary_model
[-help]
[-add_bound_lib_report]
[-bound_lib_report <string>]
-dir string
[-max_stages integer]
[-overwrite]
[-skip_constants]
[-skip_constant_ports]
[-skip_ports string]
```

Creates a boundary model for the current design for performing glitch analysis.

Parameters

`-help`	Prints out the command usage.
`-add_bound_lib_report`	Allows to write a report into the model directory as bound_libs.rpt.gz.
`-bound_lib_report <string>`	Specifies the name of a file to which a report will be written if the bound library/libraries for every instance is included in the model. When this parameter is not specified, a report will not be written.
`-dir string`	Specifies the directory where the model will be saved. If a directory does not exist, it will be created. If it already exists, and the `-overwrite` parameter is specified, then the existing directory will be deleted first - otherwise an error will be displayed.
`-max_stages integer`	Specifies the maximum number of levels of interface logic to keep from the boundary ports. All the attackers to any net along the interface logic path will be kept along with the attacker's driver and receiver. When this parameter is specified, the glitch at the input to the interface paths to the output pins will be kept and fed into the internal nets of the block. *Default*: 2
`-overwrite`	Allows the software to replace any existing directory contents with the generated boundary model.
`-skip_constants`	Allows the software to skip any logic that is constant for all the views during the forward tracing from input ports to find timing endpoints. In addition, the fanin logic is not traced for output ports that are constant in all the views.
`-skip_constant_ports`	Ignores the timing endpoints driven exclusively by constant ports.
`-skip_ports string`	Specifies primary ports of a block that should be skipped when tracing the interface paths to be included in the model. The ports can be specified as a port name or list of port names. The forward tracing for finding endpoints will not be done from skipped input ports, and the fanin tracing will not be done from the skipped output ports. Note that timing endpoints that can be reached from both a skipped input port and a non-skipped input port will be included in the model. in such a case, the skipped input port will be included in the model since it is a part of the fanin logic of the endpoint. **Note**: This parameter does not support collections.

Examples

- The following command creates a boundary model in the specified directory and overwrites any existing directory contents:

  ```
  create_glitch_boundary_model -dir model_dir -overwrite
  ```

- The following command creates a boundary model that keeps up to 6 stages of logic from the input ports and to the output ports of the design:

  ```
  create_glitch_boundary_model -dir model_dir -max_stages 6
  ```

- The following command creates a boundary model that does not include any logic constrained as constant:

  ```
  create_glitch_boundary_model -dir model_dir -skip_constants
  ```

- The following command creates a boundary model that does not include the fanout logic from input port “scan_in” or fanin logic to the output port “scan_out”:

  ```
  create_glitch_boundary_model -dir model_dir -skip_ports {scan_in scan_out}
  ```

Related Commands

- read_boundary_model
- create_boundary_model

create_ipd_configuration

```
create_ipd_configuration
[-help]
[-no_commit  | -load_constraints ]
[-no_commit  | -load_derates ]
[-setup  | -hold ]
[-setup  | -bounded_corners ]
[-hold  | -bounded_corners ]
[-min_bounded_corner  | -setup  | -bounded_corners  | -max_bounded_corner ]
[-max_bounded_corner  | -hold  | -bounded_corners  | -min_bounded_corner ]
[-min_bounded_corner  | -pd_list <string>]
[-max_bounded_corner  | -pd_list <string>]
[-hold  | -pd_list <string>]
[-setup  | -pd_list <string>]
[-bounded_corners  | -pd_list <string>]
```

Creates IPD combinations or the analysis views in both the bounded scope generation and the scope analysis session.

This command automatically generates the views based on the `set_ipd_rail_voltage` settings and loads the view definition for IPD analysis.

Parameters

`-help`	Prints out the command usage.
`-bounded_corners`	Creates the minimum/maximum bounding voltage combinations to be used for analysis. This parameter is used only in scope creation run.
`-load_constraints`	Loads the constraints stored in IPD scope during scope analysis run. (Applicable only in IPD scope analysis.)
`-load_derates`	Loads the derates stored in the IPD scope during scope analysis run. (Applicable in only IPD scope analysis.)
`-no_commit`	Creates the IPD view definition file named, ipdViewDefinition.tcl - but does not load it. By default, the IPD view definition is read in automatically. This parameter is mutually exclusive to the `-load_derates` and `-load_constraints` parameters.

Examples

- The following example shows usage of create_ipd_configuration command in scope analysis run.

  ```
  create_ipd_configuration -load_derates -load_constraints
  ```

  The `-load_constraints` and `-load_derates` parameters are optionally used in the scope analysis session to load the constraints and derates stored in the IPD scope. This IPD rail voltage specification, creates 16 (2^4) views corresponding to 4 PG nets.

- The following command creates bounding IPD views:

  ```
  tempus > set_ipd_rail_voltage -sweep { PGN1 {0.7 0.8} PGN2 {0.9 1.0} PGN3 {1.1 1.2} PGN4 {0.6 0.7} }
  create_ipd_configuration -bounded_corners
  ```

  This will create bounding IPD views with the following voltage combinations:

  - `minimum bounding view: PD1=0.8v PD2=1.0v PD3=1.2v PD4=0.7v`
  - `maximum bounding view: PD1=0.7v PD2=0.9v PD3=1.1v PD4=0.6v`

Related Commands

- `create_pg_net_group`
- `report_pg_net_group`
- `report_ipd_configuration`
- `write_ipd_scope`
- `read_ipd_scope`
- `report_ipd_crossings`

Related Information

- For more information, refer to the Inter Power Domain (IPD) Analysis chapter in the *Tempus Stylus User Guide*.

create_library_set

```
create_library_set
[-help]
[-library_side_file string]
-name libSetName
-timing {lib1.lib lib2.lib lib3.lib ...}
[-aocv string]
[-si {lib1.cdb lib2.cdb lib3.udn lib4.udn...}]
[-socv string]
```

Associates a TCL list of timing and cdB/UDN libraries with a specified library set name. Library sets allow a group of library files to be treated as a single entity so that higher-level descriptions (delay calculation corners) can simply refer to the library configuration by name. The same library set can be referenced multiple times by different delay calculation corners.

Note: You must have a set of timing libraries specified in the configuration file for the software to initialize properly. When configuring your multi-mode multi-corner environment, you can choose to specify these libraries in your own custom library set, or refer to them through the two library set definitions that are automatically initialized from the configuration file: `default_libs_min` and `default_libs_max`.

Note: The `create_library_set` command determines the priority of ECSM vs CCS data (if both are present). The software first checks for ECSM construct, if it is missing then CCS construct is loaded.

ⓘ The order in which you define timing libraries is important. The software considers the first library you specify in the list as the master library, with each successive library having a lower priority.

Parameters

`-help`	Prints out the command usage.
`-aocv string`	Specifies the aocv libraries to include in the library set.
`-library_side_file string`	Specifies the library side file.
`-name libSetName`	Specifies the name for the library set being created. This name is used to associate the library list with a specific delay calculation corner.

`-si {`*`lib1`*`.cdb `*`lib2`*`.cdb`*`lib3`*`.udn `*`lib4`*`.udn...}`	Specifies cdB libraries and/or user-defined noise (UDN) models to include in the library set. The cdB and/or UDN libraries are required for performing signal integrity analysis. **Note**: You can use this parameter for CCS-N or ECSM-N models, that is, for cdBs only.
`-socv `*`string`*	Specifies the SOCV libraries to include in the library set.
`-timing {`*`lib1`*`.lib `*`lib2`*`.lib `*`lib3`*`.lib...}`	Specifies the timing libraries to include in the library set. **Note:** Libraries intended for use in an ECSM-based multi-library interpolation must be grouped within an additional level of Tcl list structure to identify the intended usage to the library subsystem. Specify three timing libraries that are characterized at different voltages (typically the lower and upper voltage bounds, and a third intermediate point). All voltages required by the software will be interpolated from this set of libraries. The libraries in the set can differ only in the characterization voltage; they must be identical to each other in all other aspects. However, if you want to perform multi-temperature based library interpolation, you can use two library files that are characterized at different temperatures for each voltage value. See the examples for creating an ECSM-based multi-library set and multi-temperature based library interpolation in the Examples section.

The ECSM and CCS libraries can be very large and loading them may take significant time. You can convert the ASCII NLDM (Liberty .lib), ECSM, and CCS libraries to a binary version using the `write_ldb` command. For conversion of libraries to a binary version, refer to `write_ldb`.

Instead of the original ASCII libraries, you can specify these binary libraries in the configuration file:

- **Original MMMC setup file**:

Examples

- The following command creates a library set that associates timing libraries and cdB libraries with a nominal voltage of 1 volt with the library name IsCOM-1V:

```
tempus > create_library_set
-name IsCOM-1V -timing [list stdcell_F_1V.lib ram_F.lib pad.lib]
-si [list stdcell_F_2.cdb ram_F.cdb pad.cdb]
```

- The following command creates a library set that can be used to specify the 2-volt version of a core cell library, which could be associated later with a 2-volt MSMV power domain specified in the design:

```
tempus > create_library_set
-name IsCOM-2V
-timing [list stdcell_F_2V.lib]
```

- The following command creates an ECSM-based multi-library set called triLibFast. The libraries are grouped within an additional level of Tcl list structure to identify the intended usage to the library subsystem.

```
tempus > create_library_set
-name triLibFast
-timing [list
        [list stdcell_F_1V.lib stdcell_F_2V.lib stdcell_F_3V.lib]
        ]
        ram_F.lib pads.lib
        ]
```

- The following command performs multi-temperature based library interpolation for libraries characterized at 0.7, 1.0, and 1.1 volts, each of which is also characterized using -40 and -35 temperature values:

```
tempus > create_library_set
-name Trilib
-timing {{ stdcell_0.7_temp-40.lib stdcell_0.7_temp-35.lib stdcell_1.0_temp-40.lib
```

```
stdcell_1.0_temp-35.lib stdcell_1.1_temp-40.lib stdcell_1.1_temp-35.lib } mem.lib io.lib }
```

Related Commands

- update_library_set

create_opcond

```
create_opcond
[-help]
-name opcondName
[-process processValue]
[-temperature temperatureValue]
[-voltage voltageValue]
```

Creates a set of virtual operating conditions in the specified library without actually modifying the library. These virtual operating conditions can then be referenced by a delay calculation corner (`create_delay_corner`) as if they actually exist in the library.

Parameters

`-help`	Prints out the command usage.
`-name` *opcondName*	Specifies the name for the operating condition being created.
`-process` *processValue*	Specifies the process value for the operating condition.
`-temperature` *temperatureValue*	Specifies the temperature value for the operating condition.
`-voltage` *voltageValue*	Specifies the voltage value for the operating condition.

Examples

- The following command creates a virtual operating condition called `PVT1`:

```
tempus> create_opcond -name PVT1
   -process 1.0
   -voltage 1.2
   -temperature 120
```

- The following command creates a virtual operating condition called `PVT2`:

```
tempus> create_opcond -name PVT2
   -process 1.0
   -voltage 1.2
   -temperature 20
```

Related Commands

- create_delay_corner
- create_timing_condition
- update_timing_condition

create_parasitic_coordinate_transform

```
create_parasitic_coordinate_transform

[-help]
{-inst inst_name
[-x_block_origin value]
[-x_offset value]
[-y_block_origin value]
[-y_offset value]
[-orient {R0 R90 R180 R270 MY MX90 MX MY90}]
[-relative_to_parent]} | {-reset}
```

Determines accurate location information of nodes during SPEF reading.

In case of hierarchical designs, there can be multiple blocks in a design that are implemented separately. During Top level STA analysis, SPEFs of these blocks are stitched together with top for flat STA analysis. This works well if pin location of blocks is not required. In case of AOCV analysis, when spatial analysis is enabled, accurate location of all nodes (pin/net) in the design is required for calculating accurate derates in the design. This makes it necessary to determine accurate nodes location during SPEF reading. This can be achieved by transforming SPEF coordinates as per their physical locations.

Note:

- This command works with all possible SPEF and RCDB input combinations with `read_parasitics` command, which means for only RCDB(s) as input, mix of SPEF(s) and RCDB(s) as input and only SPEF(s) as input.
- If this command is not specified, the `read_spef` command tries to automatically find the orientation and origin of the hierarchical SPEF blocks and applies those identified transforms to the parasitic co-ordinates.
- The `create_parasitic_coordinate_transform` command settings remain in effect for all the subsequent `read_spef` and `read_parasitics` until they are reset using the `-reset` option.
- Whenever the block's origin (in block SPEF/DEF) is non-zero, the x and y coordinates need to be specified for applying the transform. The last two parameters, `-x_block_origin` and `-y_block_origin` in the following command use-model are only required to be specified for the case of non-zero block origin.

  ```
  create_parasitic_coordinate_transform -inst inst_name
  -x_offset x-offset of block origin in top-level DEF/SPEF
  -y_offset y-offset of block origin in top-level DEF/SPEF
  -orient orientation
  -x_block_origin x-coord of non-zero block origin
  -y_block_origin y-coord of non-zero block origin
  ```

  For example:

  - **For Non-Zero Origin Block**: Specify the block origin as shown below:

```
create_parasitic_coordinate_transform -inst eco_inst1
-x_offset 421.6
-y_offset 485
-orient R180
-x_block_origin 3.4
-y_block_origin 9.7
```

- **For zero origin block**: No need to specify the block origin as shown below:

```
create_parasitic_coordinate_transform
-inst controller
-x_offset 16.8
-y_offset 205
-orient R0
```

or

```
create_parasitic_coordinate_transform
-inst controller
-x_offset 16.8
-y_offset 205
-orient R0
-x_block_origin 0
-y_block_origin 0
```

Both the above commands are equivalent.

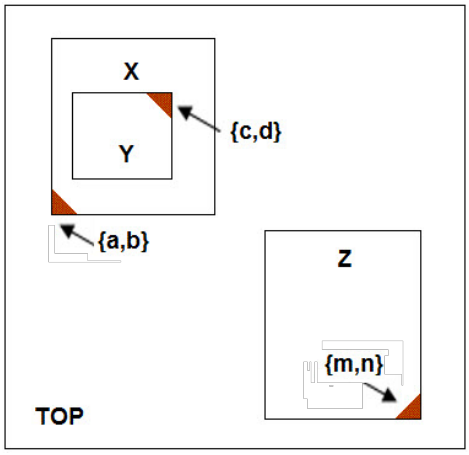
- The impact of this command is valid till first `read_spef` command is read (in case of MMMC, till all corners SPEFs have been read). If you want to read SPEF again, this command needs to be applied once again. These transformations will be applied during `read_spef` command.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man create_parasitic_coordinate_transform`
`-inst` *`inst_name`*	Indicates the hierarchical name of the instance for which SPEF is being read.
`-orient {r0 r90 r180 r270 my mx90 mx my90}`	Indicates orientation change of instances. The valid values are `R0`, `R90`, `R180`, `R270`, `MX`,`MX90`, `MY`, and `MY90`.
`-relative_to_parent`	When this parameter is specified, the software considers block offsets for the specified instance relative to the immediate parent. By default, the software considers the offset value relative to the top hinst.
`-reset`	Resets SPEF transform values.
`-x_block_origin` *`value`*	Specifies that in case of non-origin block, offset x-coordinate in microns
`-x_offset` *`value`*	Indicates the actual x-coordinate as exists in the physical floor plan, corresponding to the specified instance.
`-y_block_origin` *`value`*	Specifies that in case of non-origin block, offset y-coordinate in microns
`-y_offset` *`value`*	Indicates the actual y-coordinate as exists in the physical floor plan, corresponding to specified instance.

Examples

Consider the following hierarchical design where X and Z are two blocks inside TOP, and Y is another hierarchical block in X:



Here, X,Y, and Z have the following physical locations in the floor plan:

```
X - (a,b)
Y- (c,d)
Z- (m,n)
```

You can apply transformations to all blocks with respect to TOP, by using the following commands:

```
tempus> create_parasitic_coordinate_transform -inst X -x_offset a -y_offset b

tempus> create_parasitic_coordinate_transform -inst Z -x_offset m -y_offset n -orient MY

tempus> create_parasitic_coordinate_transform -inst X/Y -x_offset c -y_offset d -orient R180
```

These transformations will be applied when `read_spef` command is issued.

Related Commands

- `set_analysis_view`
- `redirect`
- `read_parasitics`
- `read_spef`
- `report_parasitic_coordinate_transforms`

create_pg_net_group

```
create_pg_net_group
[-help]
-name <name>
-pg_nets <net>
```

Allows you to group a set of PG nets. This helps in simplification of overall UI of the IPD flow by specifying sweeps for a PG net group rather than a PG net. Also, the IPD crossings between the PG nets within a PG net group are automatically ignored.

Parameters

`-help`	Prints out the command usage.
`-name <name>`	Specifies the PG net group name.
`-pg_nets <net>`	Specifies a list of power nets.

Examples

- In the following example, the PG nets pgn1, pgn2, and pgn3 are a part of the PG net group PG1.

  ```
  create_pg_net_group -name PG1 -pg_nets {pgn1 pgn2 pgn3}
  set_ipd_rail_voltage -sweep { PG1 {0.7 0.8} }
  ```

  The voltage values specified for PG1 are applicable for the PG nets that are part of the PG net group. The crossings between pgn1 and pgn2, pgn2 and pgn3, and pgn1 and pgn3 will be ignored automatically.

- In the following example, the PG nets pgn1, pgn2, and pgn3 are part of PG net group PG1:

  ```
  tempus > create_pg_net_group -name PG1 -pg_nets {pgn1 pgn2 pgn3}
  tempus > create_delay_corners -name dc1 -library_set libset1 -rc_corner rc1 -pg_net_voltages
  {PG1@0.7}
  ```

  The voltage values specified for PG1 are applicable for the PG nets that are part of this PG net group (defined using `create_delay_corner -pg_net_voltages`).

Related Commands

- `report_pg_net_group`
- `create_ipd_configuration`
- `report_ipd_configuration`
- `write_ipd_scope`
- `read_ipd_scope`
- `set_ipd_rail_voltage`

Related Information

- For more information, refer to the Inter Power Domain (IPD) Analysis chapter in the *Tempus Stylus User Guide*.

create_property_alias

```
create_property_alias
[-help]
-map <alias_propery_name_pairs>
-obj_type {pin port cell net clock lib_cell lib_pin design lib timing_path timing_point timing_arc
path_group lib_timing_arc si_victim si_attacker lib_pg_pin pg_pin pg_net}
```

Creates a property alias for the selected objects.

Parameters

`-help`	Prints out the command usage.
`-map <alias_propery_name_pairs>`	Specifies pair list of alias and property to be associate with that alias, such as, {alias1 prop1} or {{ialias1 prop1}{alias2 prop2}...}.
`-obj_type {pin port cell net clock lib_cell lib_pin design lib timing_path timing_point timing_arc path_group lib_timing_arc si_victim si_attacker lib_pg_pin pg_pin pg_net}`	Specifies the object type for which a property alias needs to be created.

create_rc_corner

```
create_rc_corner
[-help]
[-cap_table capTableFile]
-name rcCornerName
[-post_route_cap {capFactor1 capFactor2 capFactor3}]
[-post_route_clock_cap {capFactor1 capFactor2 capFactor3}]
[-post_route_clock_res {resFactor1 resFactor2 resFactor3}]
[-post_route_res {resFactor1 resFactor2 resFactor3}]
[-post_route_cross_cap {xcapFactor1 xcapFactor2 xcapFactor3}]
[-pre_route_cap capFactor]
[-pre_route_clock_cap capFactor]
[-pre_route_clock_res resFactor]
[-pre_route_res resFactor]
[-qrc_tech fileName]
[-temperature temperatureValue]
[-via_variation_file string]
```

Creates a named RC corner object that can be referenced later when creating a delay calculation corner object. An RC corner object provides the software with all of the information necessary to properly extract, annotate, and use the RCs for delay calculation.

Note: You must use the RC corner scaling attributes when running the software in multi-mode multi-corner analysis mode. Use the extract_rc Category Attributes to set the extraction mode for all RC corner objects in the design.

Note: In multi-mode multi-corner mode, only the default RC corners created by the software inherit the derating factor values defined in the configuration file. You can set different postroute scale factors for the postroute engine variants by specifying them as duplets and triplets. For example,

resFactor1 resFactor2 resFactor3

Where:

- Single value: If you specify one value, the scale factor applies to effort level `low`. Scale factor value of `1` is used for `medium` and `high` by default.
- Duplet: If you specify two values, the first value is used for effort level `low` and the second value is used for `medium`. Scale factor value of `1` is used for `high` by default.
- Triplet: If you specify three values, the first value is used for effort level `low`, the second value is used for `medium`, and the third value is used for `high`.

Note: If the `read_spef` command is specified without the `-rc_scale` parameter, for example, `"read_spef -rc_corner early_spef_${rc_corner}"`, then the software does not apply capacitance scale factor specified in `-post_route_cap` and `-post_route_cross_cap` parameters. If the `read_spef` command is specified with the `-rc_scale` parameter, for example, `"read_spef -rc_corner early_spef_${rc_corner} -rc_scale "`, then the software applies the capacitance scale factor. Here, no factor is specified in the `-rc_scale` option.

Parameters

`-help`	Outputs a brief description that includes type and default information for each create_rc_corner parameter. For a detailed description of the command and all of its parameters, use the man command: `man create_rc_corner`.
`-cap_table` *capTableFile*	Specifies the capacitance table to use for extraction when using this RC corner. *Default:* The Tempus System extraction system uses internal rules for extracting resistances and capacitances.
`-pre_route_cap` *capFactor*	Specifies the capacitance scale factor for RC Extraction in pre-route mode. *Default:* `1.0`
`-pre_route_clock_cap` *capFactor*	Specifies the scale factor for capacitance of clock nets in preroute mode. *Default:* `0` When this parameter is not specified, the value specified with the `-pre_route_cap` parameter is used for preroute extraction. When specified, this parameter overrides the value specified with the `-pre_route_cap` parameter for clock nets.
`-pre_route_clock_res` *resFactor*	Specifies the scale factor for resistance of clock nets in preroute mode. *Default:* `0` When this parameter is not specified, the value specified with the `-pre_route_res` parameter is used for preroute extraction. When specified, this parameter overrides the value specified with the `-pre_route_cap` parameter for clock nets.
`-pre_route_res` *resFactor*	Specifies the resistance scale factor for RC extraction in pre-route mode.

`-post_route_cap {capFactor1 capFactor2 capFactor3}`	Specifies the capacitance scale factor for RC extraction in post-route mode. *Default:* `1.0`
`-post_route_clock_cap {capFactor1 capFactor2 capFactor3}`	Specifies the scale factor for capacitance of clock nets in the postroute extraction mode. *Default:* `0` When this parameter is not specified, the value specified with the `-post_route_cap` parameter is used for postroute extraction. When specified, this parameter overrides the value specified with the `-post_route_cap` parameter.
`-post_route_clock_res {resFactor1 resFactor2 resFactor3}`	Specifies the scale factor for resistance of clock nets in the postroute extraction mode. *Default:* `0` When this parameter is not specified, the value specified with the `-post_route_res` parameter is used for postroute extraction. When specified, this parameter overrides the value specified with the `-post_route_res` parameter for clock nets.
`-post_route_res {resFactor1 resFactor2 resFactor3}`	Specifies the resistance scale factor for RC extraction in post-route mode.
`-post_route_cross_cap {xcapFactor1 xcapFactor2 xcapFactor3}`	Specifies the coupling capacitance scale factor for RC extraction in postroute mode. *Default:* `1.0`
`-name rcCornerName`	Specifies the name for the RC corner object being created. You can choose any name for an RC corner object, but it is recommended that you use a name that can be easily recognized as referring to a specific RC corner (for example, rc-min, rc-max, c-min, c-max, and so on).

`-qrc_tech` *fileName*	Specifies the name of the Quantus technology (`.tch`) file used by Quantus sign-off RC extraction. The technology file is generated by TechGen, a built-in functionality of Quantus. For most technologies you can get this file from technology vendors. For technologies that are not available through the technology vendors, you can enter the fabrication process information into an ASCII-format interconnect technology (ICT) input file. The ICT file is then used as input to IceCaps, which in turn generates the technology file.
`-temperature` *rcTemperatureValue*	Specifies the temperature, in units of Celsius, to use to derate resistance values. Use this parameter when you want to override the default temperature specified in the capacitance table or the Quantus technology file. **Note:** The software does not require that the temperature specified by the PVT operating condition at the delay calculation corner level be the same as the RC nominal or user-specified temperature value. *Default:* The Tempus system extraction system uses the default temperature of 25 degrees Celsius, unless it is overridden by the capacitance table or the Quantus technology file.

`-via_variation_file` *string*	Specifies a file for modeling via variation. The VIA variation file lists up all the VIA layer names with the corresponding multiplier. Each RC corner can have its own VIA variation file.
	
	For example:
	
	`create_rc_corner`
	
	`-name` *rcCornerName1* `\`
	
	`-via_variation_file` *viaVariationFileName1*
	
	`create_rc_corner`
	
	`-name` *rcCornerName2* `\`
	
	`-via_variation_file` *viaVariationFileName2*

Examples

- The following command creates an RC corner called `rc-cbest` that uses the capacitance table `myTech_4_bc.CapTbl`, and derates the resistance values based on the temperature of 50 Celsius:

```
create_rc_corner -name rc-cbest -cap_table myTech.4_bc.CapTbl -temperature 50
```

Related Commands

- `update_rc_corner`

create_timing_condition

```
create_timing_condition
[-help]
-library_sets string
-name timingConditionName
[-opcond string]
[-opcond_library string]
```

Creates a set of virtual operating conditions in the specified library.

Parameters

`-help`	Prints out the command usage. For a detailed description of the command and all of its parameters, use the `man` command: `man create_timing_condition`.
`-library_sets` *string*	Specifies a list of library sets to associate with the specified timing condition.
`-name` *timingConditionName*	Specifies a unique name for the timing condition.
`-opcond` *string*	Specifies a single name of an operating condition to associate with this timing condition.
`-opcond_library` *string*	Specifies the name of library to search for the named operating condition. **Notes**: • The opcond library must be a library name, and not a library file path. • In a trilib flow, where multiple library sets of different PVTs are used for opcond interpolation, the opcond library should be close to the operating condition.

Examples

- The following commands create a set of timing conditions for the defined list of library sets:

  ```
  create_library_set -name ls -timing [list typical.lib]
  create_timing_condition -name tc1 -library_sets [list ls]
  ```

- The following commands create a set of virtual operating and timing conditions on the defined library sets:

  ```
  create_opcond -name OC -process 1 -voltage 0.8 -temperature 25
  create_timing_condition -name tc1 -library_sets [list ls] -opcond OC
  ```

Related Commands

- update_timing_condition

delete_analysis_view

```
delete_analysis_view
[-help]
analysis_view_list
```

Deletes the specified analysis views.

Parameters

`-help`	Prints out the command usage.
`analysis_view_list`	Specifies the list of views for deleting.

Related Commands

- `create_analysis_view`
- `update_analysis_view`
- `set_analysis_view`
- `report_analysis_views`

get_capacitance_unit

`get_capacitance_unit`

Returns the capacitance unit (in picofarads) for the current session, which may be derived from the library file or set with the `set_library_unit -cap` parameter. This unit will be used while generating reports and for constraint settings.

Parameters

`-help`	Prints out the command usage.

Example

The following command returns the session capacitance unit:

```
get_capacitance_unit
```

```
1.000000pf
```

Related Commands

- `set_library_unit`

get_clock_network_objects

```
get_clock_network_objects
[-help]
[-clocks clock_list]
[-type {cell | pin}]
```

Returns the cells or pins that exist in a clock network of the specified clocks.

Parameters

`-help`	Prints out the command usage.
`-clocks` *`clock_list`*	Returns the cells or pins present in a clock network of the specified clock(s).
`-type {cell	pin}`	Returns the specified cells or pins in the clock network.

Examples

- The following command returns the pins present in the clock network of all the clocks:
  ```
  tempus > get_clock_network_objects
  ```
- The following command returns the pins that exist in the clock network of CLK and CLK2 clocks:
  ```
  tempus > get_clock_network_objects -clocks {CLK CLK2}
  ```
- The following command returns the cells that exist in the clock network of CLK clock.
  ```
  tempus > get_clock_network_objects -clocks {CLK} -type cell
  ```

get_clock_to_clock_association

```
get_clock_to_clock_association
[-help]
clock_list
[-allow_paths]
[-group_name group_name]
[-type {logically_exclusive | physically_exclusive | asynchronous}]
[-view viewName]
```

Reports clock to clock associations.

Parameters

`-help`	Prints out the command usage.
`clock_list`	Specifies a list of clocks that must contain two entries.
`-allow_paths`	Checks whether the two specified clocks are asynchronous and timing analysis is allowed between these clocks.
`-group_name` *`group_name`*	Reports clock to clock association for the specified clocks along with the clock group name.
`-type {logically_exclusive physically_exclusive asynchronous}`	Checks clock to clock association for the specified type of clocks. You can use one of the following options: • `logically_exclusive`: Reports clock group with clocks that do not have any functional paths between them, but coexist in the design at the same time. • `physically_exclusive`: Reports clock group with clocks that cannot exist in the design at the same time. • `asynchronous`: Reports clock group with asynchronous clocks. Asynchronous clocks have no specified phase relationship.
`-view` *`viewName`*	Generates a report for the specified analysis view.

Related Commands

- create_clock
- create_generated_clock
- set_clock_groups

get_constant

```
get_constant
[-help]
[-input_side | -output_side]
pin_port_obj
```

Queries the design database for the state of the specified pin or the state propagated through the combinational logic cone to that pin. The returned value is `0` for logic 0, `1` for logic 1, `X` for undefined, and `Z` for tristate.

Note: Constants on hierarchical ports are not computed.

The `get_constant` command returns the state for one pin only. If you want to know the state on all ports that have a constant set, use the `-type constant` option with the `report_ports` command. For example: `report_ports -type constant`. Use the `-tcl_list` option to process the list in Tcl.

Parameters

`-help`	Prints out the command usage.
`-input_side | -output_side`	Returns the constant that is set on the input or output part of bidirectional pins. *Default*: Returns the constant on the output part.
pin_port_obj	Specifies the name of the single pin to query for pin state. The pin must be an instance pin or primary I/O port.

Examples

- The following command sets the value of the specified pin(s) to either a 1 or 0 for use by the timing engine:
  ```
  set_case_analysis 0 J_block/zbuf0/A
  ```
- The following command queries the design database for the state of the specified pin or the state propagated through the combinational logic cone to that pin:
  ```
  get_constant J_block/zbuf0/A
  ```

Related Commands

- report_ports
- set_case_analysis
- report_case_analysis

get_interactive_constraint_modes

`get_interactive_constraint_modes`

Returns a list of the multi-mode multi-corner constraint modes that are in interactive constraint entry mode.

When the software is in interactive constraint entry mode (`set_interactive_constraint_modes`), any timing constraints you specify take effect immediately on all active analysis views that are associated with these constraint modes.

Note: Interactive constraint mode only works when the software is in multi-mode multi-corner timing analysis mode.

Related Commands

- `set_interactive_constraint_modes`

get_propagated_clock

```
get_propagated_clock
[-help]
[-clock clock_list] [-pin pin_list]
[> filename]
```

Returns the clock propagation mode, either `ideal` or `propagated`, for the specified clocks or pins.

Parameters

`-help`	Prints out the command usage.
`> filename`	Specifies the name of a file in which to save the report. You can add the `.gz` extension to the file name to generate a compressed report. **Note:** The file name must be the last argument in the list.
`-clock clock_list`	Specifies one or more clock waveforms for which you want to retrieve the clock propagation mode information.
`-pin pin_list`	Specifies one or more pins or ports for which you want to retrieve the clock propagation mode information. You cannot specify hierarchical pins.

Examples

- The following command gets the clock propagation mode information for all clock waveforms:
  ```
  get_propagated_clock -clock {*}
  ```

Related Commands

- `set_propagated_clock`

get_resistance_unit

```
get_resistance_unit
```

Returns the resistance unit for the current session.

Example

The following command returns the resistance unit for the current session:

```
get_resistance_unit
```

1.000000 kohm

get_socv_rc_variation_factor

```
get_socv_rc_variation_factor
[-help]
-view viewName
{-early | -late}
```

Displays the variation factor for interconnect delays.

Parameters

`-help`	Prints out the command usage.
`-early`	Displays the variation factor for early interconnect delays.
`-late`	Displays the variation factor for late interconnect delays.
`-view viewName`	Displays the SOCV interconnect variation factor for specified view.

Related Commands

- set_socv_rc_variation_factor

get_socv_reporting_nsigma_multiplier

```
get_socv_reporting_nsigma_multiplier
[-help]
-view viewName
{[-setup | -hold ]}
```

Allows you to query sigma multiplier setting for the specified view in setup or hold mode.

Parameters

`-help`	Prints out the command usage.
`-hold`	Returns the sigma multiplier for hold mode.
`-setup`	Returns the sigma multiplier for setup mode.
`-view viewName`	Returns the sigma multiplier value for the specified view. **Note**: Applies to active views only.

Examples

- The following command displays the sigma multiplier setting for the specified view:

```
tempus > set_socv_reporting_nsigma_multiplier -hold 2.5 -view default_emulate_view_hold
tempus > get_socv_reporting_nsigma_multiplier -view default_emulate_view_hold -hold
2.500000
```

Related Commands

- set_socv_reporting_nsigma_multiplier

get_time_unit

`get_time_unit`

Returns the current time unit (in nanoseconds) for the session derived from the library file or set with the `set_library_unit -time` parameter. This unit will be used during report generation and constraint settings.

Parameters

`-help`	Prints out the command usage.

Example

- The following command returns the time unit for the session:

```
get_time_unit


1.000000ns
```

Related Commands

- `set_library_unit`

gui_show_timing_map

```
gui_show_timing_map
[-help]
[-step float]
[-threshold float]
{-check_type {setup | hold} | -clear}
```

Displays the timing map in the Tempus GUI.

Parameters

`-help`	Prints out the command usage.
`-check_type {setup | hold}`	Specifies the check type - setup or hold.
`-clear`	Clears the timing map.
`-step float`	Specifies the step.
`-threshold float`	Specifies the slack threshold.

list_libraries

```
list_libraries
[-help]
[-power_domain string]
[-used]
[-used_analysis]
[-view string]
```

Lists all the libraries loaded in the session and used in the design.

Parameters

`-help`	Prints out the command usage.
`-power_domain string`	Specifies the power domain from which instances are to be selected.
`-used`	Prints out a list of libraries used in the design and all the other libraries, whose voltages match with those of the used libraries (like vt variant libraries). This list is useful for ECO optimization.
`-used_analysis`	Prints out a list of used libraries in the design.
`-view string`	Specifies the view name whose libraries will be listed.

Examples

Here are some examples.

- The following command reports all the libraries loaded in a session:

```
tempus> list_libraries

Library Library Library  Group Name Path
------- ------- -------  max fast /home/fetiming/cte/regressions/data/libs/tsmc18/fast.lib
min slow /home/fetiming/cte/regressions/data/libs/tsmc18/slow.lib
```

Related Commands

- current_design

merge_timing_reports

```
merge_timing_reports
[-help]
[-base_dir string]
-base_report string
[-command {report_constraint | report_timing | report_clock_timing}]
[-max_paths integer]
[-nworst integer]
[-text]
-view_dirs view_list
> file_name
```

Merges machine-readable/text timing reports from the specified analysis views into a single report file. The software merges the reports based on the slack value. The path with the most critical slack is listed first in the report.

The `merge_timing_reports` command can be used for both single CPU and distributed multi-mode multi-corner timing analysis.

Note: This command should only be used in DMMMC master script after reports are generated from the `distribute_views` command.

Parameters

`-help`	Prints out the command usage.
`-base_dir` *`string`*	Specifies the path to the output directory where the view-specific directories exist. The software looks in this directory for the base file name in the view specific directories, and then places the merged output in the specified redirection file. If it is not specified then software uses the base directory as the one specified with "distribute_read_design -outdir" option.
`-base_report` *`string`*	Specifies the base file name of the timing reports to merge.
`-command {report_constraint | report_timing | report_clock_timing}`	Merges the text timing reports from the specified file names. The generated timing reports can be from the commands `report_constraint`, `report_timing`, or `report_clock_timing`.
`file_name`	Specifies the name of the file in which to merge the timing reports.
`-max_paths` *`integer`*	Reports the top N worst paths that are picked from multiple views.
`-nworst` *`integer`*	Specifies to have at the most n paths per end point in a merged report.
`-text`	Generates merged report in the text format. **Note**: You can use this option only when the input reports are machine-readable.
`-view_dirs` *`view_list`*	Merges timing reports from the specified directories.

Examples

Here are some examples.

- The following command merges the machine-readable timing reports with the base file name `rt_setup1.rpt` in the analysis views `v1`, `v2`, and `v3` into a single machine readable timing report called `merged_report.rpt`:

  ```
  tempus > merge_timing_reports -view_dirs {v1 v2 v3} -base_report rt_setup1.rpt >
  merged_report.rpt
  ```

- The following command merges the machine-readable timing reports with the base file name rt_setup1.rpt in the analysis views v1, v2, and v3 into a single text timing report called `merged_report.rpt`:

  ```
  tempus > merge_timing_reports -view_dirs {v1 v2 v3} -base_report rt_setup1.rpt -text >
  merged_report.rpt
  ```

- The following commands merge text timing reports (from `report_timing` command) with the base file name `topPaths.rpt` in the analysis views `v1` and `v2` into a single timing report called `merged_topPaths.rpt`:

```
tempus > distribute_command {report_timing -max_paths 10 > topPaths.rpt}
tempus > merge_timing_reports -command report_timing -view_dirs {v1 v2} -base_report
topPaths.rpt -base_dir dmmmc_run_dir > merged_topPaths.rpt
```

- The following commands merge text timing reports (from `report_constraint` command) with the base file name `allViol.rpt` in the analysis views `v1` and `v2` into a single timing report called `merged_allViol.rpt`:

```
tempus > distribute_command {report_constraint -all_violators > allViol.rpt}
tempus > merge_timing_reports -command report_constraint -view_dirs {v1 v2} -base_report
allViol.rpt -base_dir dmmmc_run_dir > merged_allViol.rpt
```

Related Commands

- `report_timing`
- `report_constraint`
- `distribute_command`
- `distribute_views`

read_boundary_model

```
read_boundary_model
[-help]
[-check_contents]
[-instances string]
[-report_file string]
[-skip_version_check]
[-unconstrained_net_use_infinite_timing_windows]
[-clock_mapping_file string]
[-view_mapping_file string]
{-dir string}
```

Saves the directory information.

This command can be used for each cell boundary model generated by the block-level runs.

Parameters

-help	Prints out the command usage.
-check_contents	Checks for required files in the boundary model directory.

`-clock_mapping_file` *string*	Specifies the block to top-level clock mapping file. This command uses the mapping file created by the `set_boundary_model_clock_mapping` command.
	The basic syntax of `-clock_mapping_file` is:
	`Block_clock_name(view_name)`
	`Top_clock_name(view_name) instance_name`
	However, both `view_name` and `instance_name` are optional. If you want to have the same mapping for all instances and views then it is specified as follows:
	`Block_clock_name Top_clock_name`
	• If it is instance specific but not view specific then it is specified as follows:
	`Block_clock_name Top_clock_name instance_name`
	• If it is view specific but not instance specific then it is specified as follows:
	`Block_clock_name(view_name)`
	`Top_clock_name(view_name)`
	Note that no spaces are currently allowed between the clock name and the view name.

`-dir` *string*	Specifies the directory for boundary model.
`-instances` *string*	Specifies the instances for which the boundary model must be read.
`-report_file` *string*	Specifies the name of the boundary model check report file. After the boundary model is checked and the results are logged into this file. If this parameter is not specified, then by default the report gets written to <cell_name>_bmodel_check.txt file.
`-skip_version_check`	Does not check for Tempus version consistency.
`-unconstrained_net_use_infinite_timing_windows`	Forces unconstrained nets to infinite timing windows.
`-view_mapping_file` *string*	Uses the mapping file created by the `set_boundary_model_view_mapping` command.

Examples

- The following command loads the boundary model files from the ./model_dir directory:

  ```
  read_boundary_model -dir ./model_dir
  ```

- The following command loads the boundary model files from the ./model_dir directory:

  ```
  read_boundary_model -dir ./model_dir -clock_mapping_file clock.data
  ```

  The clock.data file will be read for user-defined updates for clock mappings from the boundary model clock names to the clocks defined in the constraints for the current session.

Related Commands and Variables

- create_boundary_model
- set_boundary_model_view_mapping
- set_boundary_model_clock_mapping

read_ipd_scope

```
read_ipd_scope
[-help]
[<dir_name>]
```

Loads the scope data in the scope analysis run.

Parameters

`-help`	Prints out the command usage.
<dir_name>	Specifies the scope directory containing the minimum/maximum IPD scopes.

Examples

- The following command shows the usage of `read_ipd_scope` in scope analysis session.

  ```
  tempus > read_ipd_scope bounded_corner_scope
  ```

Related Commands

- `create_pg_net_group`
- `report_pg_net_group`
- `create_ipd_configuration`
- `report_ipd_configuration`
- `write_ipd_scope`
- `set_ipd_rail_voltage`
- `report_ipd_crossings`

Related Information

- For more information, refer to the Inter Power Domain (IPD) Analysis chapter in the *Tempus Stylus User Guide*.

read_path_descriptions

```
read_path_descriptions
[-help]
filename
[-hpin]
[-path_type {end summary full full_clock end_slack_only summary_slack_only}]
[-retime {aocv ssta path_slew_propagation aocv_path_slew_propagation waveform_propagation}]
```

Reads the file created by the `write_path_descriptions` command.

This command reads the complete description of the paths specified in the file from another Tempus session and returns the data as a timing path collection. The paths description when read back can be used as a regular path collection for any reporting command or queried using the `get_property` command.

Parameters

`-help`	Prints out the command usage.
filename	Specifies the name of the input file.
`-hpin`	Prints all the hierarchical crossings of an arc under the "hpin" column in the `report_timing` table.
`-path_type {end | summary | full | full_clock | end_slack_only | summary_slack_only}`	Allows you to choose the format of the report based on the path type.
`-retime {aocv | ssta | path_slew_propagation | aocv_path_slew_propagation | waveform_propagation}`	Reanalyzes the specified set of paths using the specified analysis method.

Examples

- The following example command reads out the pathfile1.txt. The file contains details of the collection of timing paths with hierarchical pins (`-hpin`) format based on timing and exact path availability in the session in which the path description is read:

  ```
  read_path_descriptions pathfile1.txt -hpin
  ```

- The following command reads in a path descriptions file and queries the slack:

  ```
  tempus > set read_paths_coll [read_path_descriptions path_desc.txt]
  tempus > get_property $read_paths_coll slack
  -0.123
  ```

  The following command generates a timing report from the path descriptions file:

  ```
  tempus > report_timing [read_path_descriptions path_desc.txt] > paths.rpt
  ```

Related Commands

- write_path_descriptions
- get_property
- report_timing

read_sdf

```
read_sdf
[-help]
sdf_file_name
[-continue_on_error]
[-ilm_filter]
[-incremental]
[-overwrite_incremental_delay]
[-path instanceName]
[-persistent]
[-report_summary]
[-scale float]
[-strict_cond_matching]
[-view viewName]
[-sdf_field {min|typ|max} | [-early_sdf_field {min|typ|max}] [-late_sdf_field {min|typ|max}]]
```

Reads an OVI Standard Delay Format (SDF) file and annotates user-specified delay information from the SDF file into the timing system. You can use the `read_sdf` command to supply precalculated delays and timing check values from an external delay calculator, or from a previous Tempus System session.

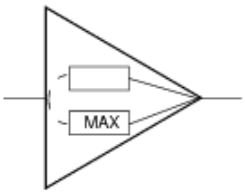
The `read_sdf` command can read version 2.1 and 3.0 of the SDF files, and can read both plain-text and GNU zipped (`.gz`) format files. The command and the Tempus System software do not support the Timing Environment section of the SDF specification.

The SDF file can provide one, two, or three different values for each timing arc delay or timing check value. These values are passed in the min, typ, or max SDF delay fields, represented as min:typ:max.

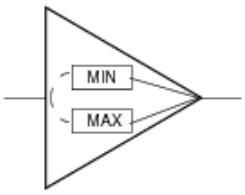
The Tempus System software provides multiple delay place holders to store the calculated or annotated delay information:

- In single-corner analysis mode, only max delays are available.
- In best-case worst-case mode, a min delay slot is provided for storing min delays used for hold analysis. A max delay slot stores delays used for setup analysis.
- In on-chip variation mode, the early delay slots store delays used for calculated early arrival times. The late delay slot stores delays used for calculated late arrival times.
- In multi-corner mode, an early and a late delay slot are provided for each multi-mode multi-corner analysis view.

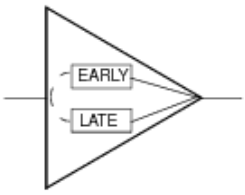
The `read_sdf` command provides parameters for controlling how the SDF min:typ:max triplet values are mapped into the Tempus System software's delay slots. The correct mapping depends on how the delay information in the SDF file was created, and what type of analysis you are performing.



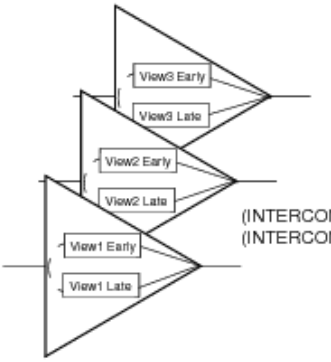
Delay slots in single-corner mode



Delay slots in best-case worst-case mode



Delay slots in on-chip variation mode



Delay slots in multi-corner mode

```
                                  min : typ : max    min : typ : max
(INTERCONNECT C1/Y RS/CK (0.123:0.456:0.789)(0.123:0.456:0.789))
(INTERCONNECT C1/Y RT/CK (0.123:0.456:0.789)(0.123:0.456:0.789))
                                  Rising delays      Falling delays
```

Standard SDF delay triplet

The timing system provides special handling for annotating incremental delays. These delays typically represent plus or minus offsets due to crosstalk effects. Incremental delay annotation to the Tempus System software can be done by:

- Importing an incremental SDF file (SDF has an internal `INCREMENT` keyword) using the `read_sdf` command.
- Tcl sourcing a file of `set_annotated_delay -net -delta_only` constraints.

The incremental delay annotations can be viewed in the `report_timing` report by adding the `incr_delay` option to the `report_timing -format` list, or to the `report_timing_format` attribute.

Parameters

`-help`	Prints out the command usage.

`-continue_on_error`	Continues to read the file if an error occurs. By default, when an error occurs, the `read_sdf` command stops reading the file. Use this parameter to attempt to continue reading the file, if possible. **Note:** Use caution if you continue, because some annotations could be missing due to the errors. Use the `report_annotated_parasitics` command to find missing annotations. If the file is corrupted, obtain a new SDF file.
`-early_sdf_field {min | typ | max}`	Annotates the early delay from the specified SDF field. In single-corner mode, all setup and hold analysis uses max delays. The software ignores any min/early delay information. In best-case worst-case mode, use this parameter to specify which SDF field to use for hold analysis. In on-chip variation mode, the software uses the early delay for calculation of fast paths.
`-ilm_filter`	Ignores ILM-related errors. By default, the `read_sdf` command does not ignore errors reported due to missing arcs in core ILM.
`-incremental`	Increments the total delay without affecting the delta delay. By default, when the SDF file contains the INCREMENT delay type, the `read_sdf` command will perform an adjustment on the delta delay slot which models SI cross-talk push-out/pull-in effects. When this option is used, the INCREMENT delays will be applied against the static base delay values instead.

`-late_sdf_field {min | typ | max}`	Annotates the late delay from the specified SDF field. In single-corner mode, use this parameter to specify which of the SDF triplet values to annotate for both setup and hold checks. In best-case worst-case mode, use this parameter to specify which SDF field to use for setup analysis. In on-chip variation mode, the software uses the late delay for the calculation of slow paths.
`-overwrite_incremental_delay`	When annotating an incremental SDF file, replaces any pre-existing delta delays with the new SDF values. The total delay will be equivalent to the original base delay, plus the new delta delay. If the delays in the incremental SDF file represent crosstalk delay push-outs from a signal integrity analysis, use the `-overwrite_incremental_delay` parameter to ensure that any stale data from a previous analysis is removed. By default, when an incremental SDF file (SDF has an internal `INCREMENT` keyword) is annotated, the delays are added to the base delay, and also stored separately as delta delays. If an additional incremental SDF file is annotated, the software adds the existing base and delta delays with the new values.
`-path` *instanceName*	Specifies the instance in the design to which the specified SDF file should be applied.
`-persistent`	Keeps the delays from certain SDF files persistently in use during various steps of the timing optimization . Use this parameter to support pre-timed structures, such as clock meshes.
`-report_summary`	Outputs the summary information of annotated delays.
`-scale` *float*	Overrides the automatic scaling of SDF delays to the library units by the value given. To disable all scaling, use a value of 1.0.

`-sdf_field {min | typ | max}`	Annotates the delay from the specified SDF slot. In single-corner and best-case worst-case modes, the software uses this value for both setup and hold analysis. In on-chip variation mode, the software uses the same value from the specified SDF slot for both the early and late delays of the specified arc.
sdf_file_name	Specifies the name of the SDF file that has the back annotation data.
`-strict_cond_matching`	Annotates only the conditions that are an exact syntax-to-syntax match between the SDF file and the `.lib` file. *Default:* Uses looser conditional matching rules between the `.lib` and SDF arc specifications.
`-view` *viewName*	Annotates the specified analysis view(s) with the delays from the SDF file. You can use this parameter only when you are in multi-mode multi-corner analysis mode.

Examples

- The following command annotates delay information from the SDF min field in to the min/early slots, and maps the SDF max field data to the max/late delay slots:

  ```
  read_sdf sdf_file_name
  ```

  Given the following SDF, the default annotation will be:

  ```
  (INTERCONNECT C1/Y RS/CK (0.123:0.456:0.789) (0.123:0.456:0.789))
  ```

  - In single-corner analysis mode, a delay of 0.789 ns is annotated to the max delay slot.
  - In best-case worst-case mode, a delay of 0.123 ns is annotated to the min delay slot, and a value of 0.789 ns is placed in the max delay slot.
  - In on-chip variation mode, a delay of 0.123 is used for early delays, and a value of 0.789 ns is used for late delays.
- The following command annotates the SDF typ field to both the early and late delay slots:

  ```
  read_sdf -sdf_field typ sdf_file_name
  ```
- The following command annotates the SDF typ field to the early delay slot, and the SDF max field to the late delay slot:

  ```
  read_sdf -early_sdf_field typ -late_sdf_field max sdf_file_name
  ```

- The following command annotates the SDF typ field to the late delay slot. The early delay slot is annotated with the default min SDF field:

  ```
  read_sdf -late_sdf_field typ sdf_file_name
  ```

Related Commands

- write_sdf
- set_annotated_delay
- report_timing
- report_annotated_parasitics

read_spef

```
read_spef
[-help]
file_name_list
[-decoupled]
[-extended]
[-idx]
[-keep_star_node_locations]
[-rc_corner list_of_rc_corners]
[-scale_rc]
[-skip_star_node_locations]
[-spef_field integer_list]
[-via_variation_file via_variation_file]
[-wire_variation_file wire_variation_file]
```

Loads resistors and capacitors for the interconnects in SPEF into the Tempus database to calculate delays or build a timing graph. Use this command to read in a SPEF file generated by a third-party extraction tool.

You can use this command to load a SPEF file, which is compressed (`.gzip`) format.

Note: This command supports multi-threaded SPEF reading scalable upto a maximum of four CPUs. The number of CPUs can be specified using the standard use model of Tempus.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `read_spef` parameter. For a detailed description of the command and all of its parameters, use the man command: `man read_spef`.
`-decoupled`	Grounds the coupling capacitances in the SPEF file and merges the capacitances on the same node. This parameter can be used only for static timing analysis and cannot be used for SI analysis.
`-extended`	Enables the reading of "`$llx=<value1> $lly=<value2> $urx=<value3> $ury=<value4> $LAYER=<number>`" to set the layer number for nodes of a resistance from Quantus extended SPEF file. It also reads in extended `*LAYER_MAP section`, if it is present. This parameter can also read in "`$l=<length> $w=<width> $si_w=<swidth> $lvl=<number>`" to set the width, silicon width, or layer for nodes of a resistance from StarRC extended SPEF file.
`file_name_list`	Specifies a SPEF file or list of SPEF files as input for flat chip or hierarchical SPEF stitching respectively.

`-idx`	Enables Inter-Die Extraction (IDX) SPEF reading. IDX SPEF models the Inter-Die capacitive impact in the form of coupling and ground capacitances. Individual die and IDX SPEFs need to be extracted by Quantus and in the extended SPEF (xSPEF) format. The `-extended` parameter is also required to be specified to read the xSPEF format. The software also supports multiple IDX SPEF reading. Flat die-level SPEFs need to be specified along with IDX SPEFs. With the `-idx` parameter, die-level and IDX SPEFs can be specified in any order.
`-keep_star_node_locations`	Enables the reading of the `*N` statements in the SPEF file. The `*N` statements define the location of the RC nodes.
`-rc_corner` *`list_of_rc_corners`*	Annotates the parasitics to the predefined RC corners for multi-corner analysis. You can specify a list of RC corners to be annotated. If you want to use SPEF information when calculating delays in multi-mode multi-corner analysis (MMMC) mode, you must annotate the parasitics for all RC corners in the design. A few examples are provided below to explain different scenarios of specifying rc corners are their corresponding corner indices and SPEF files. • `read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3 rcCorner4 rcCorner5} -spef_field {1 2 3 4 5} rc-spef.gz` This means that SPEF file can contain parasitics from N RC corners and names of RC corners in the RC corner list map to the index fields specified using the `-spef_field` option as shown above. Assuming that the SPEF file, `rc-spef.gz` has RC fields from the specified five RC corners, the above command would be interpreted as follows: 1st field in SPEF maps to RC corner, `rcCorner1` 2nd field in SPEF maps to RC corner, `rcCorner2` 3rd field in SPEF maps to RC corner, `rcCorner3` 4th field in SPEF maps to RC corner, `rcCorner4` , and 5th field in SPEF maps to RC corner, `rcCorner5` • `read_spef -rc_corner {rc_max rc_min} -spef_field {2 5} rc-spef.gz` Assuming `rc-spef.gz` in above example has RC fields from "N" RC corners, the above command would read SPEF data for two RC corners only and would be interpreted as follows: 2nd field in SPEF maps to RC corner, rc_max 5th field in SPEF maps to RC corner, `rc_min` • `read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3} -spef_field {1 1 3} rc-spef.gz` Assuming `rc-spef.gz` in the above example has RC fields from N RC

	corners, the above command would read SPEF data for 3 RC corners only and would be interpreted as follows: 1st field in SPEF maps to RC corner, `rcCorner1` 1st field in SPEF also maps to RC corner, `rcCorner2` 3rd field in SPEF maps to RC corner, `rcCorner3` • `read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3 rcCorner4 rcCorner5} -spef_field {1 2 3 4 5} {top.spef.gz block1.spef.gz block2.spef.gz}` In the above example, hierarchical design SPEF files are read in. It reads top level multi-field SPEF file followed by the multi-field SPEF files from `block1` and `block2` . **Note:** SPEF reading will not be triggered until SPEF files for all active RC corners are specified.
`-scale_rc`	Scales the SPEF read in for the specified corner as per the `postRoute_cap`, `postRoute_xcap`, and `postRoute_res` scale factors defined using the `create_rc_corner` and `update_rc_corner` commands.
`-skip_star_node_locations`	Ignores the *N statements in the SPEF file. The *N statements define the location of the RC nodes. **Note**: By default, the software ignores the *N statements while reading SPEF.
`-spef_field` *integer_list*	Specifies the field of the triplet values in spef to be loaded. **Note:** This parameter is relevant only when the `-rc_corner` parameter is specified.
`-via_variation_file` *via_variation_file*	

	Specifies the name of the file for modeling via variation. The contents of the file include all the via layer names with corresponding MEAN_SHIFT and STDDEV so that all the VIA resistance in SPEF can be mapped to their own variation multiplier. This option is used to specify the via_variation file only when the design has a single corner.
	For example,
	`read_spef`
	`-rc_corner rcCornerName \`
	`-via_variation_file` *viaVariationFileName*
	A sample via_variation file is provided below:
	`// corner_name : rcMax`
	`// wildcard to cover default`

```
object_type     : design
delay_type      : via

layer_name      : *

area            : 0.20 0.40 0.80

// no nominal for wildcard default
table_min_sigma : 0.80 0.90 0.95

table_max_sigma : 1.60 1.30 1.15

table_meanshift : 0.05 0.03 0.02

table_stdev     : 0.10 0.05 0.02

// per via layer data
object_type     : design
delay_type      : via
layer_name      : VIA1

Area            : 0.0004 0.0008 0.0016

table_min_sigma : 0.80 0.90 0.95

table_max_sigma : 1.60 1.30 1.15

table_meanshift : 0.015 0.015 0.012

table_stdev     : 0.214 0.150 0.117

object_type     : design

delay_type      : via

layer_name      : VIA2

area            : 0.0004 0.0008 0.0016

table_min_sigma : 0.80 0.90 0.95

table_max_sigma : 1.60 1.30 1.15

table_meanshift : 0.015 0.012 0.014

…..

……

table_stdev : 0.223 0.183 0.140
```

Note: Each line in the file lists the basic components of the statistical variation approach.

The values in the "area" line list all possible areas. In the above example, the area line lists three cases that can happen in the design. It can list different number of cases, for example one or four. The subsequent lines list the corresponding values matching to the areas defined in the area line.

`-wire_variation_file` *wire_variation_file*	When specified, the software loads the wire variation side file.

Examples

- The following command loads the SPEF file `TOPCHIP_SP.spef` for a flat chip methodology for RC corner, `corner1`:

  ```
  read_spef -rc_corner corner1 TOPCHIP_SP.spef
  ```

- The following command loads multiple SPEF files for hierarchical SPEF stitching:

  ```
  read_spef -rc_corner corner1 { PTN/results_conv/results_conv.spef
  tdsp_core/tdsp_core/tdsp_core.spef PTN/TOPCHIP_SP/TOPCHIP_SP.spef }
  ```

- The following commands a) setup the appropriate scale factors for the corners `RCmax` and `RCmin` b) scale the SPEF read for the defined corners `RCmax` and `RCmin`:

  ```
  === MMMC setup ====

  create_rc_corner -name RCmax -postRoute_cap 1.4 -post_route_xcap 1.4 -postRoute_res 1.4
  create_rc_corner -name RCmin -postRoute_cap 0.8 -post_route_xcap 0.8 -postRoute_res 0.8
  ...

  === run script===
  ...

  read_spef -rc_corner RCmax typSPEF.spef -scale_rc
  read_spef -rc_corner RCmin typSPEF.spef -scale_rc
  ```

- The following command loads a SPEF file `pnta-spef.gz` that has RC fields from 5 RC corners:

  ```
  read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3 rcCorner4 rcCorner5} -spef_field {1 2 3 4
  5} pnta-spef.gz
  ```

  The above command will be interpreted as follows:

  ```
  1st field in SPEF maps to RC corner rcCorner1
  2nd field in SPEF maps to RC corner rcCorner2
  3rd field in SPEF maps to RC corner rcCorner3
  4th field in SPEF maps to RC corner rcCorner4
  5th field in SPEF maps to RC corner rcCorner5
  ```

- The following command loads a SPEF file `pnta-spef.gz` that has RC fields from N RC corners:

  ```
  read_spef -rc_corner {rc_corner1 rc_corner2} -spef_field {2 5} pnta-spef.gz
  ```

  The above command will read SPEF data for 2 RC corners only and will be interpreted as follows:

  ```
  2nd field in SPEF maps to RC corner rc_corner1
  ```

```
5th field in SPEF maps to RC corner rc_corner2
```

- The following command loads a SPEF file `pnta-spef.gz` that has RC fields from N RC corners:

  ```
  read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3} -spef_field {1 1 3} pnta-spef.gz
  ```

  The above command will read SPEF data for 3 RC corners only and will be interpreted as follows:

  ```
  1st field in SPEF maps to RC corner rcCorner1
  1st field in SPEF maps to RC corner rcCorner2 as well.
  3rd field in SPEF maps to RC corner rcCorner3
  ```

- The following command loads hierarchical design SPEF files. The software reads the top level multi-field SPEF file followed by the multi-field SPEF file from `block1` and `block2`:

  ```
  read_spef -rc_corner {rcCorner1 rcCorner2 rcCorner3 rcCorner4 rcCorner5} -spef_field {1 2 3 4
  5} {top.spef.gz block1.spef.gz block2.spef.gz}
  ```

Related Commands

- create_rc_corner
- update_rc_corner
- report_annotated_parasitics

read_timing_context

```
read_timing_context
[-help]
[-clock_mapping_file clock_mapping_file]
[-instances instance_list]
[-skip_interface_paths]
[-view_map_file view_map_file]
[dir_name]
```

Reads the timing context of a block specified in the input directory.

Note: The `read_timing_context` command is not supported in DSTA. The context written from top level DSTA can be read in lower level STA runs.

Parameters

`-help`	Prints out the command usage.
dir_name	Specifies the name of the input directory to load context data.
`-clock_mapping_file` *clock_mapping_file*	Specifies the clock mapping file. This file contains user-provided information to guide clock mapping. The software inferred clock mapping will be overwritten by the user-specified clock mapping defined in the clock_mapping_file file.
`-instances` *instance_list*	Specifies the list of instances for which analysis needs to be performed. The instances provided should belong to a single cluster file, otherwise context reading will error out. When this parameter is not specified, the software performs context analysis of all the instances (belonging to a cluster) of a design.
`-skip_interface_paths`	Skips the interface paths.
`-view_map_file` *view_map_file*	Specifies the view mapping file. This file contains user-specified information to guide view mapping using the `set_timing_context_view_mapping` command. In case of MMMC flow with different view names in block and top STA, the equivalent block and top view names can be specified under the view_map_file file.

Examples

- The following command will read context models of all the instances of the `tdsp_core` design that are part of one cluster:
  ```
  read_timing_context context
  ```
- The following command will read context models corresponding to the `tdsp_core_inst1` instance of the `tdsp_core` design:
  ```
  read_timing_context context -instance {tdsp_core_inst1}
  ```
- The following command will read merged context model corresponding to the `tdsp_core_inst1` and `tdsp_core_inst2` instances of `tdsp_core` design - provided that both the instances are part of a single cluster:
  ```
  read_timing_context context -instance {tdsp_core_inst1 tdsp_core_inst2}
  ```
  The command will error out if these instances are part of different clusters.
- The following command will read context model of `tdsp_core` design with user-defined clock mapping and view mapping files:
  ```
  read_timing_context context -clock_mapping_file clkMap.txt -view_map_file viewMap.txt
  ```

Related Commands

- `write_timing_context`
- `set_timing_context_view_mapping`
- `report_timing_context_clock_mapping`
- `compare_timing_context`

read_twf

```
read_twf
[-help]
[filenames]
[-cell string]
[-prefix string]
[-quiet]
[-reset]
[-scope string]
[-skip_constant]
[-skip_timing_window]
[-strip_prefix string]
[-verbose]
[-view string]
```

Reads the external timing window file (TWF) for noise (Signal Integrity) and power calculation (Common Power Engine - CPE). This file is used to obtain clocks, constants, external load, slews, timing windows, and slack information. The external TWF specification is optional. In the absence of a TWF, this information is automatically derived using the Common Timing Engine (CTE), if timing constraints are read.

Note: If the `VOLTAGE_THRESHOLD` construct is missing from the TWF file header, the values 10 and 90 are automatically assigned as the minimum and maximum threshold values, respectively. For example:

```
VOLTAGE_THRESHOLD (10 90)
```

Parameters

`-help`	Prints out the command usage.
`-cell` *string*	Specifies the master cell for the timing window file.
filenames	Specifies the names of the timing window files.
`-prefix` *string*	Adds a prefix to the net or pin name.
`-quiet`	Skips information and warning messages.
`-reset`	Clears the previously read TWF files.
`-scope` *string*	Specifies the scope for the given hierarchical timing window file.
`-skip_constant`	Skips constant processing.
`-skip_timing_window`	Skips timing windows processing.
`-strip_prefix` *string*	Removes hierarchical path name.
`-verbose`	Prints the command progress details.
`-view` *string*	Specifies the view for the timing window file.

Examples

- The following command reads the timing window file TW.in for the view 'view01', while skipping the constant processing from the timing window file:

  ```
  tempus> read_twf TW.in -skipconst -view view01
  ```

- The following command clears the previously read TWF file:

  ```
  tempus> read_twf -reset
  ```

Related Commands

- `write_twf`

report_analysis_coverage

```
report_analysis_coverage
[-help]
[pin | port]
[-check_type check_type_list]
[-exclude_untested reason_list]
[-include_dynamic_checks]
[-include_parallel_check_arcs]
[-include_when_cond]
[-max_paths {type integer}]
[-nworst integer]
[-retime {aocv | ssta | path_slew_propagation | aocv_path_slew_propagation}]
[-sort {pin | refpin | checktype | slack | reason}]
[-tcl_list]
[-unsorted]
[-verbose check_status_list]
[-view view_name]
[> <filename[.gz]>]
[>> <filename[.gz]>]
```

Provides information about timing checks in the design.

For each type of timing check, the command reports the number of checks that meet constraints, violated constraints, or that are untested in the `SUMMARY` section of the report. The `DETAILS` section includes information about the signal pin, reference pin, check type, slack, and the reason for being untested.

When `timing_report_group_based_mode` is set to `true`, this command will report a check at end points for all the clocks. In verbose mode, the "clock" column will be displayed.

Use `check_timing` to look for missing constraints.

Parameters

`-help`	Prints out the command usage.

{> | >>} *file_name*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.

`-check_type` *check_type_list*	Reports checks that as specified in the *check_type_list*. Valid check types are: • clock_gating_hold • clock_gating_pulse_width • clock_gating_setup • clock_period • clock_separation • data_hold • data_setup • external_delay: • hold • no_change_hold • no_change_setup • path_delay • pulse_width • recovery • removal • setup • skew • time_borrow **Note:** If there are no specified timing checks in the design, the report shows "No timing checks found." *Default*: All (all types of check(s) are reported)

`-exclude_untested {`*`reason_list`*`}`	Excludes from the report endpoints that are unconstrained due to a specific reason. Valid reasons are: • `const`: Constant reaching signal/reference pin of any timing check. • `false_path`: Path disabled due to `set_false_path`. • `no_endpoint_clock`: Missing clock at the reference pin of the timing check. • `no_startpoint_clock`: Missing clock at the signal pin of the timing check. This may happen if the clock signal does not reach the launching register. • `unknown`: None of the known reasons are identified for an untested path. • `user_disable`: Checks arc disable due to `set_disable_timing`.
`-include_dynamic_checks`	Reports the dynamic checks (data_to_data, path_delay) on pins that are constrained by set_data_check, set_max_delay, or set_min_delay constraints. By default, these checks will not be reported and the command only reports the pins constrained by all library checks, set_output_delay or inferred clock gating checks. The dynamic checks, such as set_data_check or set_max_delay are not reported by default, when `-include_dynamic_checks` parameter is not specified.
`-include_parallel_check_arcs`	Reports all the parallel check arcs that have been merged.
`-include_when_cond`	Reports the when condition of check arcs.
`-max_paths {type` *`integer`*`}`	Enumerates the specified number of worst paths in the design, regardless of the endpoint.
`-nworst` *`integer`*	Enumerates the specified number of paths for each endpoint.

`-retime {aocv | ssta | path_slew_propagation | aocv_path_slew_propagation}`	Reanalyzes the set of paths using the specified method.
pin | *port*	Limits the report to timing checks on specified pins or ports.

`-sort {pin | refpin | checktype | slack | reason}`	Sorts the checks by the specified option while reporting detailed information using the `-verbose` option. The `-sort` parameter does not change the order of the columns.
	The valid sort options are: `pin`, `refpin`, `checktype`, `slack`, `reason`
	When sorting by pin name, if the pin names are the same, the software then attempts to sort the checks by slack value. If the slack values are the same, the software attempts to sort by timing check type.
	When sorting by timing check type, if the check types are the same, the software then attempts to sort by slack value. If the slack values are the same, the software attempts to sort by pin name.
	When sorting by slack, if the slack values are the same, the software then attempts to sort the checks by pin name. If the pin names are the same, the software attempts to sort by timing check type. When you sort by `slack`, the slack value is reported in order of least negative to most negative (worst slack last).
	You can use the following commands for debugging purposes and to improve readability of the report:
	`report_analysis_coverage -verbose untested -sort reason`
	`report_analysis_coverage -verbose violated -sort slack`
	Default: By default, checks are sorted by slack value (`slack`).

`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-unsorted`	Reports unsorted slacks with untested end points at the end of the report.
`-verbose` *`check_status_list`*	Limits the detailed information to checks as per the specified status. The valid status types are: `met`, `violated`, and `untested`. • `met`: Shows as a positive value in the Slack column. • `violated`: Shows as a negative value in the Slack column. • `untested`: Shows as UNTESTED in the Slack column and the reason is given in the Reason column. The Reason column will be blank when a Slack number is displayed - this indicates the check has been tested. You can reduce the number of lines in the DETAILS section by filtering out the met checks, as shown in the example below: `report_analysis_coverage -verbose {violated untested}` **Note**: The `-verbose` parameter supports multi-threading.
`-view` *`view_name`*	Generates a timing check report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner analysis mode.

Examples

- The following command shows the report output for pulse width checks for the specified view:

```
tempus > report_analysis_coverage   -check_type pulse_width   -view func_slow_RCMAX
```

```
--------------------------------------------------------------------------------
              TIMING CHECK COVERAGE SUMMARY (Analysis View:  func_slow_RCMAX)

--------------------------------------------------------------------------------
  Check Type       No. of   Met            Violated     Untested
                   Checks
--------------------------------------------------------------------------------
  PulseWidth       1541      1096 (71%)    42 (2%)      403 (26%)
--------------------------------------------------------------------------------
```

- The following command shows the report output for the specified view port:

```
tempus > @report_analysis_coverage port_pad_data_out
--------------------------------------------------------------------------------
                              TIMING CHECK COVERAGE SUMMARY
--------------------------------------------------------------------------------
Check Type                 No. of    Met          Violated      Untested
                           Checks
--------------------------------------------------------------------------------
ExternalDelay (Early)      16        16 (100%)    0 (0%)        0 (0%)
ExternalDelay (Late)       16        0 (0%)       16 (100%)     0 (0%)
--------------------------------------------------------------------------------
```

- The following command reports checks for graph-based analysis (GBA) paths:

```
tempus > report_analysis_coverage -retime path_slew_propagation
--------------------------------------------------------------------------------
                         TIMING CHECK COVERAGE SUMMARY
--------------------------------------------------------------------------------
Check Type              No. of    Met             Violated       Untested
                        Checks
--------------------------------------------------------------------------------
ClockPeriod             6         6 (100%)        0 (0%)         0 (0%)
ExternalDelay (Early)   29        26 (89%)        0 (0%)         3 (10%)
ExternalDelay (Late)    29        0 (0%)          26 (89%)       3 (10%)
Hold                    1741      1486 (85%)      37 (2%)        218 (12%)
PulseWidth              1541      1096 (71%)      42 (2%)        403 (26%)
Recovery                377       1 (0%)          3 (0%)         373 (98%)
Removal                 377       4 (1%)          0 (0%)         373 (98%)
Setup                   1741      951 (54%)       574 (32%)      216 (12%)
```

- The following command shows untested checks falling under the met status:

```
tempus > report_analysis_coverage -exclude_untested false_path  -verbose met
                      TIMING CHECK COVERAGE SUMMARY
-------------------------------------------------------------------------
Check Type               No. of    Met            Violated     Untested
                         Checks
-------------------------------------------------------------------------
ClockPeriod              6         6 (100%)        0 (0%)      0 (0%)
ExternalDelay (Early)    29        26 (89%)        0 (0%)      3 (10%)
ExternalDelay (Late)     29        0 (0%)         26 (89%)     3 (10%)
Hold                     1739      1486 (85%)     37 (2%)     216 (12%)
PulseWidth               1541      1096 (71%)     42 (2%)     403 (26%)
Recovery                 138       1 (0%)          3 (2%)     134 (97%)
Removal                  138       4 (2%)          0 (0%)     134 (97%)
Setup                    1741      950 (54%)     575 (33%)    216 (12%)
-------------------------------------------------------------------------
-------------------------------------------------------------------------
                           TIMING CHECK COVERAGE DETAILS
-------------------------------------------------------------------------
Pin                    Reference Pin           Check Type    Slack    Reason
-------------------------------------------------------------------------
RTS_INST/clr_flg/SI ^   RTS_INST/clr_flg/CK ^   Setup         0.001    -
```

- The following command generates a timing check report for the specified analysis view:

```
tempus > report_analysis_coverage  -view scan_slow_RCMAX
-------------------------------------------------------------------------
              TIMING CHECK COVERAGE SUMMARY (Analysis View:  scan_slow_RCMAX)

-------------------------------------------------------------------------
Check Type              No. of    Met         Violated   Untested
                        Checks
-------------------------------------------------------------------------
ClockPeriod             6         6 (100%)    0 (0%)     0 (0%)
ExternalDelay (Early)   29        26 (89%)    0 (0%)     3 (10%)
ExternalDelay (Late)    29        26 (89%)    0 (0%)     3 (10%)
Hold                   1741      458 (26%)    19 (1%)    1264 (72%)
PulseWidth             1541      1138 (73%)   0 (0%)     403 (26%)
Recovery                377       4 (1%)      0 (0%)     373 (98%)
Removal                 377       2 (0%)      0 (0%)     375 (99%)
Setup                  1741      1525 (87%)   0 (0%)      216 (12%)
```

```
-------------------------------------------------------------------------
```

- The following command filters from the report all endpoints that are unconstrained because of the reason `const`:

```
tempus> report_analysis_coverage -exclude_untested const -verbose untested

-------------------------------------------------------------------------
TIMING CHECK COVERAGE SUMMARY
-------------------------------------------------------------------------
Check Type             No. of     Met      Violated    Untested
                       Checks
-------------------------------------------------------------------------
Clock Gating Hold      1          0 (0%)    1 (100%)   0 (0%)
Clock Gating Setup     1          1 (100%)  0 (0%)     0 (0%)
ClockPeriod            28         24 (85%)  0 (0%)     4 (14%)
ExternalDelay (Early)  1          0 (0%)    0 (0%)     1 (100%)
ExternalDelay (Late)   1          0 (0%)    0 (0%)     1 (100%)
Hold                   13         8 (61%)   1 (7%)     4 (30%)
PulseWidth             28         24 (85%)  0 (0%)     4 (14%)
Setup                  13         8 (61%)   1 (7%)     4 (30%)
-------------------------------------------------------------------------

TIMING CHECK COVERAGE DETAILS
-------------------------------------------------------------------------------
Pin     Reference Pin      Check Type             Slack       Reason
-------------------------------------------------------------------------------
out1       -               ExternalDelay(Early)  UNTESTED    No data signal
out1       -               ExternalDelay(Late)   UNTESTED    No data signal
u99/CK ^   u99/CK ^        ClockPeriod           UNTESTED    No endpoint clock
u99/CK ^   u99/CK v        PulseWidth            UNTESTED    No endpoint clock
u99/CK v   u99/CK ^        PulseWidth            UNTESTED    No endpoint clock
u99/CK v   u99/CK v        ClockPeriod           UNTESTED    No endpoint clock
u99/D      u99/CK ^        Hold                  UNTESTED    No endpoint clock
u99/D      u99/CK ^        Setup                 UNTESTED    No endpoint clock
u8/CK ^    u8/CK ^         ClockPeriod           UNTESTED    No endpoint clock
u8/CK ^    u8/CK v         PulseWidth            UNTESTED    No endpoint clock
u8/CK v    u8/CK ^         PulseWidth            UNTESTED    No endpoint clock
u8/CK v    u8/CK v         ClockPeriod           UNTESTED    No endpoint clock
seg1/u9/D  seg1/u9/CK ^    Hold                  UNTESTED    No data signal
seg1/u9/D  seg1/u9/CK ^    Setup                 UNTESTED    No data signal
seg2/u9/D  seg2/u9/CK ^    Hold                  UNTESTED    No data signal
seg2/u9/D  seg2/u9/CK ^    Setup                 UNTESTED    No data signal
-------------------------------------------------------------------------------
```

  When the software is in multi-mode multi-corner timing analysis mode, You can specify the `-view` parameter to generate a timing check report for the specified analysis view only:

```
tempus 150> report_analysis_coverage -view view1
-------------------------------------------------
TIMING CHECK COVERAGE SUMMARY (Analysis View: view1)
```

```
-------------------------------------------------------
Check Type          No. of     Met       Violated      Untested
                    Checks
-------------------------------------------------------------

Clock Gating Hold     1         0 (0%)     1 (100%)     0 (0%)
Clock Gating Setup    1         1 (100%)   0 (0%)       0 (0%)
ClockPeriod           28       24 (85%)   0 (0%)       4 (14%)
ExternalDelay (Early) 1         0 (0%)     0 (0%)       1 (100%)
ExternalDelay (Late)  1         0 (0%)     0 (0%)       1 (100%)
Hold                  14        8 (57%)    1 (7%)       5 (35%)
PulseWidth            28       24 (85%)    0 (0%)       4 (14%)
Setup                 14        8 (57%)    1 (7%)       5 (35%)
-------------------------------------------------------------
```

Related Commands

- check_timing
- report_timing

report_analysis_summary

```
report_analysis_summary
[-help]
[-csv string]
[-merged_views]
[-retime {path_slew_propagation | aocv_path_slew_propagation}]
[-retime_mode {exhaustive}]
[-view viewName]
[> <filename[.gz]>]
[>> <filename[.gz]>]
[-early | -late ]
[-expanded_summary_clocks | -merged_groups]
```

Generates timing slack summary reports for specified analysis view(s). This command categorizes the slack summary in terms of groups - In2reg, Reg2reg, Reg2Out, and In2Out. A column "ALL" is displayed that regroups in2reg, in2out, reg2reg, and reg2out details.

By default, the slack is categorized in terms of worst negative slack (WNS), total negative slack (TNS), and the total number of violations in group-based mode.

The `report_analysis_summary` command is supported in path-based analysis (PBA) mode.

Parameters

`-help`	Prints out the command usage.
`{> | >>} filename`	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-csv string`	Specifies the name of the CSV output file.
`-early`	Generates report for hold analysis mode for early paths.
`-expanded_summary_clocks`	Generates report in terms of each possible clock domain pair separately.
`-late`	Generates report for setup analysis mode for late paths.
`-merged_groups`	Merges all the clock groups for a consolidated timing summary.
`-merged_views`	Generates a merged report across all the views.
`-retime {path_slew_propagation | aocv_path_slew_propagation}`	Reanalyzes the defined set of paths using the specified method. By default, the timing report shows a maximum of 2 million paths for retime PBA analysis.
`-retime_mode {exhaustive}`	Reanalyzes the specified set of paths using the specified retime mode.
`-view viewName`	Generates report for the specified analysis views only. By default, all the active views are analyzed.

Examples

- The `report_analysis_summary` command shows the following report output:

```
tempus> report_analysis_summary
```

```
REPORT ANALYSIS SUMMARY (View:default_emulate_view) (EL:Setup)
================================================================

REPORT ANALYSIS SUMMARY (View:default_emulate_view) (EL:Hold)
================================================================

GroupType:ClockGate
================================================================

All              IN2REG          IN2OUT          REG2REG          REG2OUT
-----------------------------------------------------------------------------------------------
WNS:TNS:#VIO    WNS:TNS:#VIO    WNS:TNS:#VIO    WNS:TNS:#VIO    WNS:TNS:#VIO    FromClk->ToClk
-----------------------------------------------------------------------------------------------
-5.614:-5.614:1  0.000:0.000:0  0.000:0.000:0  -5.614:-5.614:1 0.000:0.000:0  *->*
-----------------------------------------------------------------------------------------------

GroupType:Standard
===============================================================================================

All              IN2REG           IN2OUT          REG2REG          REG2OUT
-----------------------------------------------------------------------------------------------
WNS:TNS:#VIO     WNS:TNS:#VIO    WNS:TNS:#VIO    WNS:TNS:#VIO    WNS:TNS:#VIO    FromClk->ToClk
-----------------------------------------------------------------------------------------------
-8.897:-8.897:1  0.000:0.000:0   0.000:0.000:0  -8.897:-8.897:1 0.000:0.000:0  *->CLK_W_1
-----------------------------------------------------------------------------------------------
-0.571:-0.571:1  -0.571:-0.571:1 0.000:0.000:0   0.000:0.000:0  0.000:0.000:0  *->CLK_W_2
-----------------------------------------------------------------------------------------------
```

- The following command generates a consolidated summary of all the clock groups and for all the views:

```
tempus > report_analysis_summary  -merged_groups   -merged_views
REPORT ANALYSIS SUMMARY (EL:Setup)
=============================================================================================

GroupType:*
=============================================================================================

All                     IN2REG          IN2OUT        REG2REG                 REG2OUT
-----------------------------------------------------------------------------------------------
-----------------
WNS:TNS:#VIO            WNS:TNS:#VIO    WNS:TNS:#VIO  WNS:TNS:#VIO            WNS:TNS:#VIO
  FromClk->ToClk
-----------------------------------------------------------------------------------------------
-----------------
-22.239:-5171.932:604  -2.274:-5.220:4  0.000:0.000:0 -22.239:-5089.035:574  -5.034:-
77.678:26  *->*

REPORT ANALYSIS SUMMARY (EL:Hold)
=============================================================================================

GroupType:*
=============================================================================================
```

```
All                    IN2REG              IN2OUT          REG2REG              REG2OUT
------------------------------------------------------------------------------------------
------------
WNS:TNS:#VIO           WNS:TNS:#VIO        WNS:TNS:#VIO    WNS:TNS:#VIO         WNS:TNS:#VIO
 FromClk->ToClk
------------------------------------------------------------------------------------------
------------
-2.108:-46.905:37    -1.094:-14.238:19   0.000:0.000:0   -2.108:-32.668:18   0.000:0.000:0  *-
>*
```

- The following command reports retimed early paths for hold analysis mode:

```
tempus 149> report_analysis_summary  -retime path_slew_propagation  -early

REPORT ANALYSIS SUMMARY (View:func_slow_RCMAX) (EL:Hold)
========================================================================

GroupType:Standard
========================================================================

All                  IN2REG              IN2OUT          REG2REG              REG2OUT
------------------------------------------------------------------------------------------
---------
WNS:TNS:#VIO         WNS:TNS:#VIO        WNS:TNS:#VIO    WNS:TNS:#VIO         WNS:TNS:#VIO
 FromClk->ToClk
------------------------------------------------------------------------------------------
---------
-2.108:-46.804:37   -1.093:-14.151:19  0.000:0.000:0  -2.108:-32.654:18  0.000:0.000:0  *-
>m_clk

INFO: Out of 37 endpoints finally reported to be violating, 0 number of endpoints reported
Graph Based Analysis (GBA) bounded slack. Refer to message TCLCMD-1251 and TCLCMD-1377 in log
file for details regarding the reason for GBA bounding.
```

- The following command reports each possible clock domain pair separately:

```
tempus > report_analysis_summary -expanded_summary_clocks

REPORT ANALYSIS SUMMARY (View:func_slow_RCMAX) (EL:Setup)
========================================================================

GroupType:Async
========================================================================

All               IN2REG             IN2OUT          REG2REG          REG2OUT
------------------------------------------------------------------------------------------
--
WNS:TNS:#VIO      WNS:TNS:#VIO       WNS:TNS:#VIO    WNS:TNS:#VIO     WNS:TNS:#VIO    FromClk-
>ToClk
------------------------------------------------------------------------------------------
--
-0.618:-1.229:2  -0.618:-1.229:2   0.000:0.000:0   0.000:0.000:0   0.000:0.000:0  refclk->*
-0.618:-1.229:2  -0.618:-1.229:2   0.000:0.000:0   0.000:0.000:0   0.000:0.000:0  m_rcc_clk->*
```

```
-0.618:-1.229:2  -0.618:-1.229:2   0.000:0.000:0   0.000:0.000:0   0.000:0.000:0  m_ram_clk->*
```

Related Commands

- set_default_view
- set_analysis_view

report_analysis_views

```
report_analysis_views
-help
[-type {all | setup | hold | active}]
```

Generates a hierarchical report of the current multi-mode multi-corner configuration. By default, the `report_analysis_views` command generates a report of all defined views in the design. Use the `-type` parameter to generate a report on a specific type of analysis view.

Use this command after creating analysis views (`create_analysis_view`), and setting active setup and hold views (`set_analysis_view`) for the design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_analysis_views` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_analysis_views`.
`-type {all | setup | hold | active}`	Specifies the type of report to generate. • active: Generates a report of all active setup and hold views in the design. • all: Generates a report of all defined views in the design, including those that are currently inactive. • hold: Generates a report of all active hold views in the design. • setup: Generates a report of all active setup views in the design. *Default:* `all`

Examples

- The following command reports all the defined views in the design:
  ```
  tempus > report_analysis_views
  Multi-Mode Analysis View Report
  ____________________________________
  ```

```
+ ALL Views
   |
   + Analysis View:  func_slow_RCMAX
     |
     + Delay Calc Corner: delay_corner_slow_RCMAX
     |   |
     |   + early_timing_condition: default_mapping_tc_2
     |   |    |
     |   |    + library_sets: libset_slow
     |   |        |
     |   |        + timing: ../../libs/liberty/FreePDK45_lib_v1.0_worst.lib
                            ../../libs/MRO/LIB/pllclk.lib
                            ../../libs/MRO/LIB/ram_256x16A.lib
                            ../../libs/MRO/LIB/rom_512x16A.lib
     |   |
     |   + late_timing_condition: default_mapping_tc_1
     |   |    |
     |   |    + library_sets: libset_fast
     |   |        |
     |   |        + timing: ../../libs/liberty/FreePDK45_lib_v1.0_typical.lib
                            ../../libs/MRO/LIB/pllclk.lib
                            ../../libs/MRO/LIB/ram_256x16A.lib
                            ../../libs/MRO/LIB/rom_512x16A.lib
     |   |
     |   + early_rc_corner: corner_worst_RCMIN
     |   |    |
     |   |    + preRoute_res: 0.8
     |   |    |
     |   |    + postRoute_res: 0.8 0.8 0.8
     |   |    |
     |   |    + preRoute_cap: 0.8
     |   |    |
     |   |    + postRoute_cap: 0.8 0.8 0.8
     |   |    |
     |   |    + postRoute_xcap: 0.8 0.8 0.8
     |   |    |
     |   |    + preRoute_clkcap: 0
     |   |    |
     |   |    + postRoute_clkcap: 0 0 0
     |   |    |
     |   |    + preRoute_clkres: 0
     |   |    |
     |   |    + postRoute_clkres: 0 0 0
```

- The following command generates a hierarchical report for all the active setup views in the design:

```
tempus > report_analysis_views -type setup
Multi-Mode Analysis View Report
_________________________________
```

```
+ SETUP Views
   |
   + Analysis View:  func_slow_RCMAX
     |
     + Delay Calc Corner: delay_corner_slow_RCMAX
     |   |
     |   + early_timing_condition: default_mapping_tc_2
     |   |    |
     |   |    + library_sets: libset_slow
     |   |        |
     |   |        + timing: ../../libs/liberty/FreePDK45_lib_v1.0_worst.lib
                            ../../libs/MRO/LIB/pllclk.lib
                            ../../libs/MRO/LIB/ram_256x16A.lib
                            ../../libs/MRO/LIB/rom_512x16A.lib
     |   |
     |   + late_timing_condition: default_mapping_tc_1
     |   |    |
     |   |    + library_sets: libset_fast
|   |
     |   + early_rc_corner: corner_worst_RCMIN
     |   |    |
     |   |    + preRoute_res: 0.8
     |   |    |
     |   |    + postRoute_res: 0.8 0.8 0.8
|   + late_rc_corner: corner_worst_RCMAX
     |   |    |
     |   |    + preRoute_res: 1.2
|   + si_enabled: true
     |
     + Constraint Mode: functional_mode
     |   |
     |   + SDC Constraint Files: ../../design/rcvr_core.pr.tcl ../../design/rcvr_core.pr.sdc
```

- The following command generates a hierarchical report of the active setup analysis views in the design:

```
tempus> report_analysis_views -type setup
```

The software reports the following results:

```
Multi-Mode Analysis View Report
_______________________________

+ SETUP Views
|
+ Analysis View: default_emulate_view
|
+ Delay Calc Corner: default_emulate_delay_corner
| |
| |
| |
```

```
| + rc_corner: default_emulate_rc_corner
| | |
| | + preRoute_res: 1
| | |
| | + postRoute_res: 1
| | |
| | + preRoute_cap: 1
| | |
| | + postRoute_cap: 1
| | |
| | + postRoute_xcap: 1
| | |
| | + preRoute_clkcap: 0
| | |
| | + postRoute_clkcap: 0 0 0
| | |
| | + preRoute_clkres: 0
| | |
| | + postRoute_clkres: 0 0 0
| |
| + late_library_set: default_emulate_libset_max
| | |
| | + timing: /icd/ssv_pe_t2b/timing/pawang/unit_test_cases/unit_2/data/cell_w.lib
| | |
| | + si: /icd/ssv_pe_t2b/timing/pawang/unit_test_cases/unit_2/data/celtic-NDC.cdB
| |
| + early_library_set: default_emulate_libset_min
| | |
| | + timing: /icd/ssv_pe_t2b/timing/pawang/unit_test_cases/unit_2/data/cell_w.lib
| | |
| | + si: /icd/ssv_pe_t2b/timing/pawang/unit_test_cases/unit_2/data/celtic-NDC.cdB
| |
| + si_enabled: true
|
+ Constraint Mode: default_emulate_constraint_mode
| |
| + SDC Constraint Files: /dev/null
```

Related Commands

- create_analysis_view
- set_analysis_view

report_annotated_assertions

```
report_annotated_assertions
-type {load | transition}
[-list_annotated]
[-list_non_annotated]
[-view viewName]
[-max_lines number]
[-min | -max | -min_max]
obj_list
[{> | >>} file_name]
```

Reports information for annotated assertions, set using the `set_load` or `set_annotated_transition` command, according to the specified assertion type. You can select load or transition reporting.

Note: The `report_annotated_assertions` command is now obsolete. This command will continue to be supported in the current release, but will be removed in the next major release of the software.

Parameters

`{> | >>}` *file_name*	Writes the report to the specified file name. When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified filename. If the file already exists, the software concatenates the report to the end of the file.
`-list_annotated`	Lists the detailed information for the specified assertion types.
`-list_non_annotated`	Lists information for non-annotated assertions.
`-min | -max | -min_max`	Specifies to report minimum, maximum, or both values of the specified assertion type. When input assertions are applied using both -min and -max parameters, you can specify the `-min_max` parameter to report both the assertion values. In this case, you do not need to specify this command twice, that is, separately with -min and -max parameters. *Default*: max

`-max_lines` *number*	Controls the number of output lines reported by `-list_annotated` or `-list_non_annotated` parameter. By default, the specified net, pin, or port names are reported.
obj_list	Specifies the list of nets, pins, or ports to be reported.
`-type {load | transition}`	Specifies the assertion type, load or transition, to be reported.
`-view` *viewName*	Reports the annotations for the specified analysis view. In multi-mode multi-corner timing analysis mode, annotations are reported for the specified mode or corner.

Examples

- The following command reports all the transition annotated assertions for the specified view:

```
tempus > report_annotated_assertions  -type transition  -list_annotated  -view func_slow_RCMAX

Analysis View: func_slow_RCMAX
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
Assert type        Total    Annotated   Not Annotated    Percentage Annotated
--------------------------------------------------------------------------------
Transition (Max)   29097    0           29097            0.000
--------------------------------------------------------------------------------
```

- The following command reports details of both the minimum and maximum values of the specified assertion type:

```
tempus > report_annotated_assertions  -type load  -list_non_annotated  -min_max  -max_lines  5
 -view scan_slow_RCMAX

Analysis View: scan_slow_RCMAX
------------------------------
Load (Min:Max) Non
Annotated
----------
NetName
----------
tdigit[7]
tdigit[6]
tdigit[5]
tdigit[4]
tdigit[3]
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
Assert type      Total     Annotated    Not Annotated    Percentage Annotated
--------------------------------------------------------------------------------
Load (Min:Max)   8415      0            8415             0.000
--------------------------------------------------------------------------------
```

Related Commands

- set_load
- set_annotated_transition

report_annotated_check

```
report_annotated_check
[-help]
[-check_type check_type_list]
[-max_line number]
[instance_list]
[-list_annotated]
[-list_non_annotated]
[-view viewName]
[> | >> filename[.gz] | -tcl_list]
```

Reports coverage of annotated timing checks.

Parameters

`-help`	Prints out the command usage.
`-check_type` *check_type_list*	Allows you to specify the check type. The valid check types are `setup`, `hold`, `pulse_width`, `clock_period`, `clock_gating_setup`, `clock_gating_hold`, `clock_gating_pulse_width`, `data_setup`, `data_hold`, `recovery`, `removal`, `clock_separation`, `no_change_setup`, `no_change_hold`, and `skew`.
`>` *filename*	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.

`instance_list`	Specifies instance names for which annotations of check arcs are to be reported.
`-list_annotated`	Lists the detailed annotation information for the following timing checks: • setup • hold • recovery • removal • pulse-width • nochange-setup • clock-period • skew **Note**: If the software is in multi-mode multi-corner timing analysis mode, you need to specify the `-view` parameter.
`-list_non_annotated`	Lists the timing checks that do not have annotated delay information. The software checks for the following timing checks: • setup • hold • recovery • removal • pulse-width • nochange-setup • clock-period • skew **Note**: If the software is in multi-mode multi-corner timing analysis mode, you need to specify the `-view` parameter.
`-max_line` *number*	Controls the number of output lines reported by the `-list_annotated` or `list_non_annotated` parameter.

`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-view` *viewName*	Reports coverage of annotated timing checks for the specified analysis view. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode. *Default:* Reports annotated timing checks for the default analysis view.

Examples

- The following command reports the setup timing checks that do not have annotated delay information:

```
tempus > report_annotated_check  -check_type setup  -list_non_annotated  -max_line 6  -view
scan_slow_RCMAX
Analysis View: scan_slow_RCMAX
------------------------------------------------------------------------
NonAnnotated-Setup
                                        Checks

-------------------------------------------------------------------------------------------
SourceSignal               RefSignal              Type              Cond   View

-------------------------------------------------------------------------------------------
AB_INST/sp_0/D  ^     AB_INST/sp_0/CK  ^    setup_rising_clk_rise     -    scan_slow_RCMAX
AB_INST/sp_0/D  v     AB_INST/sp_0/CK  ^    setup_falling_clk_rise    -    scan_slow_RCMAX
AB_INST/sp_0/SE  ^    AB_INST/sp_0/CK  ^    setup_rising_clk_rise     -    scan_slow_RCMAX
AB_INST/sp_0/SE  v    AB_INST/sp_0/CK  ^    setup_falling_clk_rise    -    scan_slow_RCMAX
AB_INST/sp_0/SI  ^    AB_INST/sp_0/CK  ^    setup_rising_clk_rise     -    scan_slow_RCMAX
AB_INST/sp_0/SI  v    AB_INST/sp_0/CK  ^    setup_falling_clk_rise    -    scan_slow_RCMAX
-------------------------------------------------------------------------------------------
Check arc type  Total   Annotated  Not Annotated   Percentage Arcs Annotated
-------------------------------------------------------------------------
Setup arcs      1741    0          1741            0.000000
_
All arcs        1741    0          1741            0.000000
-------------------------------------------------------------------------
```

- The following command reports the detailed annotation information for the setup timing checks of the specified view:

```
tempus > report_annotated_check  -check_type  setup  -list_annotated  -max_line  6  -view
```

```
scan_slow_RCMAX  -tcl_list

Analysis View: scan_slow_RCMAX
{ banner { } } { Check arc Summary { { columns { { arc_type } { total } { total_ann } {
total_not_ann } { percentage_ann }  } } { row_1 { { Setup arcs } { 1741 } { 0 } { 1741 } {
0.000000 }  } } { row_2 { { _ }  } } { row_3 { { All arcs } { 1741 } { 0 } { 1741 } { 0.000000
}  } }  } }
```

- The `report_annotated_check` command shows the following report output in simultaneous setup and hold analysis mode:

```
tempus> report_annotated_check

--------------------------------------------------------------------------------

| Check arc type         | Total | Annotated | Not       |Percentage|
|                        |       |           | Annotated |Arcs      |
|                        |       |           |           |Annotated |
|------------------------+-------+-----------+-----------+----------|
| Setup arcs             | 1642  | 2         | 1640      | 0.121803 |
| Hold arcs              | 1642  | 1         | 1641      | 0.060901 |
| Recovery arcs          | 368   | 0         | 368       | 0.000000 |
| Removal arcs           | 367   | 1         | 366       | 0.272480 |
| Min pulse width arcs   | 1403  | 0         | 1403      | 0.000000 |
| Period arcs            | 6     | 0         | 6         | 0.000000 |
| Clock gating hold arcs | 2     | 0         | 2         | 0.000000 |
+------------------------+-------+-----------+-----------+----------+
| All arcs               | 5430  | 4         | 5426      | 0.073665 |
+--------------------------------------------------------------------+
```

- The following command lists down all the annotated checks in the `f_slow` view:

```
tempus> report_annotated_check -list_annotated -view f_slow

-----------------------------------------------------
Analysis View: f_slow

+-------------------------------------------------------------------------------------------------------
------+
| Setup Checks |                                |                         |
|--------------------------+------------------------+--------------------------+--------------------
------|
| SourceSignal         | RefSignal           | Delays           | Type                 |Cond
|View   |
|------------------------------+-------------------------------+------------------+------------
-------
| DTMF_IN/TDSP_CR/PROG| DTMF_IN/TDSP_CR/PROG| 0.500:0.500:0.500|
setup_rising_clk_rise|RN_EQ_1_AN|f_slow |
| BUS_MC_IN/dt_reg_2  | BUS_MC_IN/dt_reg_2  |                  |                      |
      |
| D ^                 | CK ^                |                  |                      |          |
```

```
     |
| DTMF_IN/TDSP_CR/PROG| DTMF_IN/TDSP_CR/PROG|
0.500:0.500:0.500|setup_falling_clk_rise|RN_EQ_1_AN|f_slow |
| BUS_MC_IN/dt_reg_2  | BUS_MC_IN/dt_reg_2  |                  |                       |          |
     |
| D v                 | CK ^                |                  |                       |          |
     |
+-------------------------------------------------------------------------------------------------
-------

+-----------------------------------------------------------------+
| Check arc type        | Total | Annotated | Not       | Percentage|
|                       |       |           | Annotated | Arcs      |
|                       |       |           |           | Annotated |
|-----------------------+-------+-----------+-----------+-----------|
| Setup arcs            | 1642  | 2         | 1640      | 0.121803  |
| Recovery arcs         | 368   | 0         | 368       | 0.000000  |
| Min pulse width arcs | 1403  | 0         | 1403      | 0.000000  |
| Period arcs           | 6     | 0         | 6         | 0.000000  |
+-----------------------+-------+-----------+-----------+-----------+
| All arcs              | 3419  | 2         | 3417      | 0.058497  |
+-----------------------------------------------------------------+
```

- The following command lists down all the annotated checks in the f_slow view:

```
tempus > report_annotated_check -list_annotated -view f_slow

-----------------------------------------------------
Analysis View: f_fast
+-------------------------------------------------------------------------------------------------
------+
| Hold Checks |                               |                        |
|-------------------------+-----------------------+-------------------------+--------------------
------|
| SourceSignal        | RefSignal           | Delays           | Type                 |Cond
|View   |
|-----------------------------+-------------------------------+------------------+------------
-------
| DTMF_INST/TEST_INST | DTMF_INST/TEST_INST | 0.400:0.400:0.400| hold_rising_clk_rise |          |
f_fast|
| RAM_INST/A[5] ^     | RAM_INST/CLK ^      |                  |                      |          |
     |
| DTMF_INST/TEST_INST | DTMF_INST/TEST_INST | 0.400:0.400:0.400  hold_falling_clk_rise|          |
f_fast|
| RAM_INST/A[5] v     | RAM_INST/CLK ^      |                  |                      |          |
     |

+-------------------------------------------------------------------------------------------------
------+

+-------------------------------------------------------------------------------------------------
```

```
------+
| Removal Checks |                                  |                          |
|------------------------+------------------------+--------------------------+--------------------
------|
| SourceSignal        | RefSignal           | Delays          | Type                  |Cond
|View    |
|-------------------------------+-------------------------------+------------------+------------
------|
| DTMF_IN/SPI_IN/dflag| DTMF_IN/SPI_IN/dflag|0.300:0.300:0.300|removal_rising_clk_rise|         |
f_fast |
| N ^                 | K ^                 |                 |                         |         |
      |
+-----------------------------------------------------------------------------------------------
------+
+-----------------------------------------------------------------------+

| Check arc type         | Total | Annotated | Not       | Percentage|
|                        |       |           | Annotated | Arcs      |
|                        |       |           |           | Annotated |
|----------------------+-------+-----------+-----------+------------|
| Hold arcs              | 1642  | 1         | 1641      | 0.060901 |
| Removal arcs           | 367   | 1         | 366       | 0.272480 |
| Clock gating hold arcs | 2     | 0         | 2         | 0.000000 |
+------------------------+-------+-----------+-----------+------------+
| All arcs               | 2011  | 2         | 2009      | 0.099453 |
+-----------------------------------------------------------------------+
```

Related Commands

- set_annotated_check
- reset_annotated_check

report_annotated_delay

```
report_annotated_delay
[-help]
[-ignore_tied_net_arcs]
[-list_annotated]
[-list_non_annotated]
[-view viewName]
[-max_line number]
[{> | >>} filename[.gz] | -tcl_list]
[pin_port_list]
```

Reports coverage of annotations on a design. These annotations include SDF delay arc annotations. Besides SDF annotations, the command also reports annotations that are set using the `set_annotated_delay` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_annotated_delay` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_annotated_delay`.
`> | >>` *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-ignore_tied_net_arcs`	Ignores timing arcs of power and ground nets.

`-list_annotated`	Lists detailed annotation information for the following annotated arcs: • annotated internal net arcs • annotated input port arcs • annotated output port arcs • annotated cell arcs
`-list_non_annotated`	Lists the arc information for the non-annotated arcs corresponding to the following types of arcs: • non-annotated internal net arcs • non-annotated input port arcs • non-annotated output port arcs • non-annotated cell arcs
`-max_line` *number*	Controls the number of output lines reported by `-list_annotated` or `-list_non_annotated` parameter.
pin_port_list	Specifies the pin/port list for which annotated delay is to be reported.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *filename* parameters are mutually exclusive; you cannot specify them together.
`-view` *viewName*	Generates the report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

- The following command reports the status of annotation for cell arcs, internal net arcs, net arcs from primary inputs, and net arcs to primary outputs, and all the timing arcs separately.
```
tempus > report_annotated_delay  -view scan_slow_RCMIN  -list_annotated  -ignore_tied_net_arcs
```

```
Analysis View: scan_slow_RCMIN
--------------------------------------------------------------------------------
Delay Arc Type       Total    Annotated    Not Annotated    Percentage Annotated
--------------------------------------------------------------------------------
Cell arcs            38365    0            38365              0.000000
Input net arcs       141      0            141                0.000000
Internal net arcs    20019    0            20019              0.000000
Output net arcs      29       0            29                 0.000000
--------------------------------------------------------------------------------
All timing arcs      58554    0            58554              0.000000
--------------------------------------------------------------------------------
```

- The following command reports the non-annotated input port arcs, internal net arcs, cell delay arcs, and output port arcs for the specified view:

```
tempus > report_annotated_delay -view scan_slow_RCMAX -list_non_annotated -max_line 3

Analysis View: scan_slow_RCMAX
--------------------------------------------------------------------------------
Delay Arc Type      Total   Annotated   Not Annotated   Percentage Annotated
--------------------------------------------------------------------------------
Cell arcs           38365   0            38365           0.000000
Input net arcs      141     0            141             0.000000
Internal net arcs   20027   0            20027           0.000000
Output net arcs     29      0            29              0.000000
--------------------------------------------------------------------------------
All timing arcs     58562   0            58562           0.000000
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
non-annotated input port arcs

--------------------------------------------------------------------------------
From                 To                  SdfCond        View
--------------------------------------------------------------------------------
reset  ^           AB_INST/p25A/A  ^                scan_slow_RCMAX
--------------------------------------------------------------------------------
non-annotated internal net

arcs

--------------------------------------------------------------------------------
From                  To                 SdfCond        View
--------------------------------------------------------------------------------
FE_C6_ar4/Z  ^     DT_MX_INST/p3A/B  ^              scan_slow_RCMAX
--------------------------------------------------------------------------------
non-annotated cell delay arcs

--------------------------------------------------------------------------------
```

```
From                    To                SdfCond        View
-------------------------------------------------------------------------------
FE_C6ar4/A  ^      FE_C6ar4/Z  ^                     scan_slow_RCMAX
-------------------------------------------------------------------------------
non-annotated output port arcs

-------------------------------------------------------------------------------
From                   To                    SdfCond      View
-------------------------------------------------------------------------------
FE_C63_port_out/Z  ^ port_data_out[13]  ^            scan_slow_RCMAX
-------------------------------------------------------------------------------
```

Related Commands

- set_annotated_delay
- report_nets

report_annotated_parasitics

```
report_annotated_parasitics
[-help]
[-include_nets <string>]
[-include_nets_file <string>]
[-view {viewName}]
[> {write_fileName}]
[>> {append_fileName}]
[{[-list_annotated] [-broken_nets][-list_extra_pin_net] [-floating_nets] [-no_driver_nets] [-
unloaded_nets]
     [-list_not_annotated] [-list_zero_cap_net] [-real_nets] [-supply_nets] [-max_missing
{max_missing_int}]}]
```

Reports the back-annotated parasitics of the design. By default only summary report is generated. The software groups all the nets by net-types and annotation-status (annotated or not-annotated).

Parameters

`-help`	Outputs a brief description that includes type and default information for the `report_annotated_parasitics` parameters. For a detailed description of the command and all of its parameters, use the `man` command: `man report_annotated_parasitics`.
`>> {append_fileName}`	Writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. **Note:** The file name must be the last argument in the list. *Default:* Report is displayed on standard output without being saved.
`-broken_nets`	Reports broken nets in the design.
`-list_extra_pin_net`	Generates a list of nets that have extra pins in the SPEF but not in the Verilog.
`-floating_nets`	Generates a list of floating nets.

`-include_nets_file` *string*	Accepts a file name, which contains a list of net names, as input. When this parameter is specified, the parasitic annotation summary is provided only for the listed nets. The net names in the file can be specified as one net per line.
`-include_nets` *string*	Specifies a list of net names for which annotation summary is required.
`-list_annotated`	Generates a list of annotated nets.
`-list_not_annotated`	Generates a list of not annotated nets.
`-list_zero_cap_net`	Generates a list of zero capacitance nets.
`-no_driver_nets`	Lists all the terminal nets that have no driver.
`-real_nets`	Lists all real nets.
`-supply_nets`	Lists all supply nets.
`-max_missing {max_missing_int}`	Limits the number of missing annotations to the number specified. *Default*: 1000
`-unloaded_nets`	Lists all the terminal nets that have no load.
`-view` *viewname*	Reports the back-annotated parasitics for the specified analysis view. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode. *Default*: Reports against the default analysis view
`> {`*write_fileName*`}`	Writes the report to the specified file name. If the file already exists, the software overwrites it.

Examples

- The following command generates a list of annotated and not-annotated nets:

```
tempus > report_annotated_parasitics -broken_nets -view func_slow_RCMAX -supply_nets -
max_missing 10 -floating_nets

# list syntax: type: net_name [nrCap Cap nrXCap XCap nrRes Res]
# unit: pF, Ohm

# No annotated supply net.

# No annotated dummy net (floating).
```

```
# Summary of Annotated Parasitics:
+-------------------------------------------------------------------------------+
|    Net Type          |    Count    |   Annotated (%)      | Not Annotated (%)   |
+----------------------+-------------+----------------------+---------------------+
| total                |       8751 |       8408 96.08%    |        343  3.92%   |
+----------------------+-------------+----------------------+---------------------+
| 0-term:floating (*)  |        336 |          0  0.00%    |        336 100.00%  |
| no drive (*)         |          2 |          0  0.00%    |          2 100.00%  |
| 1-term:no load       |          5 |          0  0.00%    |          5 100.00%  |
| real net (complete)  |       8408 |       8408 100.00%   |          0  0.00%   |
| real net (broken)    |          0 |          0  0.00%    |          0  0.00%   |
| zero capacitance net|          0 |          0  0.00%    |          0  0.00%   |
+-------------------------------------------------------------------------------+
Note for Net Types marked "(*)":  Such nets are never timed, but reported here for
informational purpose.

+-----------------------------------------------------------------+
| Annotated |    Res (KOhm)    |    Cap (pF)      |    XCap (pF)    |
+-----------+------------------+------------------+-----------------+
| Count     |        132172    |        140357    |         31664   |
| Value     |      730.4969    |       22.7070    |        6.2433   |
+-----------------------------------------------------------------+
```

- The following command reports a list of not-annotated dangling nets:

```
tempus > report_annotated_parasitics   -no_driver_nets   -unloaded_nets

# list syntax: type: net_name [nrCap Cap nrXCap XCap nrRes Res]
# unit: pF, Ohm

# List of total 2 not-annotated dangling net (no driver):
Not-annotated dangling net (no driver): [ARB_INST/ECO1net_2] [ 0 0 0 0 0 0 ]
Not-annotated dangling net (no driver): [ARB_INST/ECO1net_1] [ 0 0 0 0 0 0 ]

# List of total 5 not-annotated dangling net (no load):
Not-annotated dangling net (no load): [FE_OFN238_n_163] [ 0 0 0 0 0 0 ]
Not-annotated dangling net (no load): [scan_en] [ 0 0 0 0 0 0 ]
Not-annotated dangling net (no load): [int] [ 0 0 0 0 0 0 ]
Not-annotated dangling net (no load): [ARB_INST/UNCONNECTED1] [ 0 0 0 0 0 0 ]
Not-annotated dangling net (no load): [ARB_INST/ECO1net_24] [ 0 0 0 0 0 0 ]

# Summary of Annotated Parasitics:
+-------------------------------------------------------------------------------+
|    Net Type          |    Count    |   Annotated (%)      | Not Annotated (%)   |
+----------------------+-------------+----------------------+---------------------+
| total                |       8751 |       8408 96.08%    |        343  3.92%   |
+----------------------+-------------+----------------------+---------------------+
| 0-term:floating (*)  |        336 |          0  0.00%    |        336 100.00%  |
| no drive (*)         |          2 |          0  0.00%    |          2 100.00%  |
| 1-term:no load       |          5 |          0  0.00%    |          5 100.00%  |
| real net (complete)  |       8408 |       8408 100.00%   |          0  0.00%   |
| real net (broken)    |          0 |          0  0.00%    |          0  0.00%   |
```

```
| zero capacitance net|          0 |          0  0.00%   |          0  0.00%   |
+---------------------------------------------------------------------------------+
Note for Net Types marked "(*)":  Such nets are never timed, but reported here for
informational purpose.

+-----------------------------------------------------------------+
| Annotated |    Res (KOhm)  |   Cap (pF)     |    XCap (pF)    |
+----------+---------------+---------------+-----------------------------------+
| Count     |     132172     |    140357      |   31664         |
|  Value    |    730.4969    |    22.7070     |   6.2433        |
+-----------------------------------------------------------------+
```

- The following command generates a list of annotated and not-annotated nets:

```
report_annotated_parasitics -list_annotated -list_not_annotated -list_real_net -list_float_net
-list_nodriver_net \ + -list_noload_net -list_supply_net -max_missing 1000

# list syntax: type: net_name [nrCap Cap nrXCap XCap nrRes Res]
# unit: pF, Ohm
# List the first 82 of total 82 real nets:
Annotated Real Net: [seg3/w7] [ 10 0.0690723 25 0.0512442 9 219.251 ]
Annotated Real Net: [seg3/w5] [ 14 0.0373353 24 0.0265601 13 161.843 ]
:
:
Annotated Real Net: [CLK3] [ 2 0.00046867 0 0 1 18.872 ]
Annotated Real Net (broken): [CLK4] [ 1 0.0002379 0 0 2 0.0002 ]
# List the first 20 of total 20 not annotated real nets:
Not Annotated Real Net: [seg3/wout]
Not Annotated Real Net: [CLK44]
:
:
Not Annotated Real Net: [out1]
Not Annotated Real Net: [in1]
# No annotated supply net.
# List the first 4 of total 4 not annotated supply nets:
Not Annotated Supply Net: [gnd]
Not Annotated Supply Net: [GND]
Not Annotated Supply Net: [VDD]
Not Annotated Supply Net: [vdd]
# No annotated floating net.
# No not annotated floating net.
# No annotated no driver net.
# ist the first 9 of total 9 not annotated no driver dangling nets:
Not Annotated Dangling Net (no driver): [seg3/n8]
Not Annotated Dangling Net (no driver): [seg3/n111111]
:
:
Not Annotated Dangling Net (no driver): [in]
Not Annotated Dangling Net (no driver): [out5]
```

```
# List the first 8 of total 8 annotated no load dangling nets:
Annotated Dangling Net (no load): [seg3/n14] [ 1 0.00520893 0 0 0 0 ]
Annotated Dangling Net (no load): [seg2/n14] [ 1 0.00636874 0 0 0 0 ]
:
:
Annotated Dangling Net (no load): [n6] [ 2 0.1086 0 0 1 0.0001 ]
Annotated Dangling Net (no load): [n5] [ 2 0.1065 0 0 1 0.0001 ]
# List the first 16 of total 16 not annotated no load dangling nets:
Not Annotated Dangling Net (no load): [seg3/u14:QN]
Not Annotated Dangling Net (no load): [seg3/u9:QN]
:
:
Not Annotated Dangling Net (no load): [u2:QN]
Not Annotated Dangling Net (no load): [in4]
# Summary of Annotated Parasitics:
+-------------------------------------------------------------------+
| Net Type            | Count  | Annotated (%) | Not Annotated (%) |
+---------------------+--------+---------------+-------------------+
| total               | 135    | 90 66.7%      | 45 33.3%          |
+---------------------+--------+---------------+-------------------+
| no drive (*)        | 9      | 0 0.0%        | 9 100.0%          |
| 1-term:no load      | 24     | 8 33.3%       | 16 66.7%          |
| real net (complete) | 81     | 61 75.3%      | 20 24.7%          |
| real net (broken)   | 21     | 21 100.0%     | 0 0.0%            |
+-------------------------------------------------------------------+
Note for Net Types marked "(*)": Such nets are never timed, but reported here for
informational purpose.

+-------------------------------------------------+
| Annotated  | Res (KOhm) | Cap (pF) | XCap (pF) |
+------------+------------+----------+-----------+
| Count      | 982        | 1011     | 1152      |
| Value      | 20.0869    | 7.7289   | 5.4598    |
+-------------------------------------------------+
```

Related Commands

- read_sdf
- read_spef

report_aocv_derate

```
report_aocv_derate
[-help]
[-check_global_derating]
[-clock]
[-data]
[-list_cell_not_annotated]
[-mesh]
-out_file string
[-voltage float]
[-design | -cell string ] [ {-delay_corner <string> [-power_domain string] } | -max | -min]
[-early | -late]
[-rise | -fall]
```

Reports the AOCV derating tables from AOCV libraries. The command generates the output report in AOCV library format.

Parameters

`-help`	Prints out command usage.
`-cell` *string*	Reports the table for the specified library cell name. You can specify a regular expression corresponding to library cell names.
`-check_global_derating`	Reports the list of cells for which derating table or global derating table was not specified.
`-clock`	Reports derate tables read for clock paths.
`-delay_corner` *string*	Reports the derating tables from the specified delay corner. For a MMMC design, if a delay corner specific table is read using the `create_library_set -aocv` command, that derating table is reported. Otherwise the default table is reported. For a non-MMMC design, no details are reported. The `-delay_corner` and `-min`/`-max` parameters are mutually exclusive.
`-data`	Reports derate tables read for data paths.
`-design`	Reports the default cell or net derating tables.
`-early` | `-late`	Reports the early/late derating table.
`-list_cell_not_annotated`	Reports the list of cells for which derating table is not specified. This parameter reports an error when specified with `-voltage`, `-design`, or `-cell` parameters.
`-max`	Reports derate tables from max AOCV libraries.
`-mesh`	Reports derate tables read for mesh.
`-min`	Reports derate tables from min AOCV libraries.
`-out_file` *string*	Specifies the name of the output file. When this parameter is specified, all the derate tables will be reported.
`-power_domain` *string*	Reports derate tables from the specified power domain. This option should always be specified with delay corner.
`-rise` | `-fall`	Reports the rise/fall table for specified transition.
`-voltage` *float*	Reports derating tables for the specified voltage value.

Examples

Here are some examples.

- The following command reports derating tables for the specified cells:

```
tempus> report_aocv_derate -cell "BUF" -delay_corner default_emulate_delay_corner -out_file
report.lib

#content of file "report.lib" :

Cell BUF
LATE-DATA-RISE
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 8.000000 9.000000 18.000000 19.000000
1.180000 1.160000 1.140000 1.120000 1.100000 1.080000 1.060000 1.040000 1.020000

Cell BUF
LATE-DATA-FALL
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 8.000000 9.000000 18.000000 19.000000
1.180000 1.160000 1.140000 1.120000 1.100000 1.080000 1.060000 1.040000 1.020000

Cell BUF
LATE-CLOCK-RISE
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 8.000000 9.000000 18.000000 19.000000
1.180000 1.160000 1.140000 1.120000 1.100000 1.080000 1.060000 1.040000 1.020000

Cell BUF
LATE-CLOCK-FALL
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 8.000000 9.000000 18.000000 19.000000
1.180000 1.160000 1.140000 1.120000 1.100000 1.080000 1.060000 1.040000 1.020000
```

- The following command reports derating tables for the cells:

```
tempus > report_aocv_derate -clock -out_file aocv_clock_derate.rpt

Cell AN2D1
EARLY-CLOCK-RISE
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000 9.000000
10.000000
0.70000 0.850000 0.880000 0.890000 0.900000 0.910000 0.920000 0.9200000 0.930000 0.935000

Cell AN2D1
EARLY-CLOCK-FALL
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000 9.000000
10.000000
0.760090 0.830010 0.860060 0.880300 0.890200 0.901000 0.902000 0.910000 0.920000 0.9202594
Cell BUFF
EARLY-CLOCK-RISE
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000 9.000000
10.000000
0.842000 0.888258 0.908854 0.921954 0.929953 0.935137 0.940946 0.944305 0.942031 0.959500

Cell BUFF
```

```
EARLY-CLOCK-FALL
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000 9.000000
10.000000
0.739532 0.810305 0.840208 0.861346 0.880605 0.899462 0.903028 0.900408 0.912154 0.916708
* The following command reports derating tables for the cells in delay corner in mesh
report_aocv_derate -mesh -delay_corner dc_125c_rcworst -out_file
aocv_clock_delay_corner_dc_125c_rcworst.rpt
Cell AN2D1
EARLY-MESH-RISE

Cell AN2D1
EARLY-MESH-FALL
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000 9.000000
10.000000
0.739532 0.810305 0.840208 0.861346 0.880605 0.899462 0.903028 0.900408 0.912154 0.916708

Cell AN2D1
LATE-MESH-RISE
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000 9.000000
10.000000
0.842000 0.888258 0.908854 0.921954 0.929953 0.935137 0.940946 0.944305 0.942031 0.959500

Cell AN2D1
LATE-MESH-FALL
Stage 1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000 9.000000
10.000000
0.842000 0.888258 0.908854 0.921954 0.929953 0.935137 0.940946 0.944305 0.942031 0.959500
```

Related Commands

- timing_aocv_derate_mode

report_boundary_model_constraints

```
report_boundary_model_constraints
[-help]
```

Reports the boundary model constraints that are applied using the `select_boundary_model_constraints` command.

Parameters

`-help`	Prints out the command usage.

report_case_analysis

```
report_case_analysis
[-help]
[pins_and_ports_list]
[-all]
[-dir]
[-include_ref_name]
[-nosplit]
[-tcl_list]
[-verbose]
[-view view_name]
[> <filename[.gz]>]
[>> <filename[.gz]>]
[-propagated [-include_function]]
```

Generates a User Case Analysis table, which reports ports and pin that have a `set_case_analysis` constraint. These ports and pins are constant only for timing analysis. You can optionally generate a Logic Constant Value table, which reports ports and pins whose constant status is part of the design. Information from this table includes ports or pins with `set_logic` constraints (`set_logic_zero` and `set_logic_one`), netlist explicit constants, and output pins with constant output functions.

Parameters

`-help`	Prints out the command usage.

{> | >>} *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.
-all	Specifies that the Logic Constant Value table report pins or ports with set_logic constants, constants in the netlist, and output pins with a constant function. These output pins are from library cells or netlist modules.
-include_function	Displays the pin function of a pin. This must be specified with the -propagated parameter.
-include_ref_name	Displays the reference name of an instance in the report.
-no_split	Specifies that reports with long names will not be split into multiple lines. **Note:** The -no_split parameter works if set_table_style -no_frame_fix_width command is specified.
-dir	Specifies that the report include the direction of the pin or port, which is reported as input, output, or inout (internal pins may not have constant constraints).

pins_and_ports_list	Generates a report only for the pins and ports in the specified list. *Default:* Generates a report for all pins and ports in the design.
-propagated	Generates a report that includes all the constant pins, including pins with propagated constants. When specified, the report column header shows the constant values - the column header for constant values is `User case analysis value`. *Default:* Report shows pins specified with the `set_case_analysis` command.
-tcl_list	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *filename* parameters are mutually exclusive; you cannot specify them together.
-verbose	Generates a report in which each pin is listed on a separate line with a cause. If the pin is a propagated constant, a table is included that reports a chain of constants from an original cause to the reported pin. *Default:* All pins are reported in a table.
-view *view_name*	Generates a User Case Analysis table for the specified analysis view only. To specify this parameter, the software must be in multi-mode multi-corner (MMMC) analysis mode. *Default:* When in MMMC analysis mode, generates a table for all active analysis views

Examples

- The following command reports user case analysis values for all the analysis views:

```
tempus > report_case_analysis
Pin name      User       View Name
              case
              analysis
----------------------------------------
tst_mode      0          scan_fast_RCMIN
tst_mode      0          scan_slow_RCMIN
```

```
tst_mode      0          func_slow_RCMAX
scan_en       0          scan_fast_RCMIN
scan_en       0          scan_slow_RCMIN
```

- The following command reports all the constant pins, including pins with propagated constants - and long names are not split into multiple lines:

```
tempus > report_case_analysis -propagated -no_split
-----------------------------------------------
Pin name          Constant   View Name
                  value
-----------------------------------------------
AB_IS/se_0/D       0         func_slow_RCMAX
AB_IS/se_0/RN      1         func_slow_RCMAX
AB_IS/se_0/SE      0         func_slow_RCMAX
AB_IS/se_0/SI      0         func_slow_RCMAX
AB_IS/se_0/SN      1         func_slow_RCMAX
AB_IS/se_1/D       0         func_slow_RCMAX
AB_IS/se_1/RN      1         func_slow_RCMAX
AB_IS/se_1/SE      0         func_slow_RCMAX
```

- The following command reports the Logic Constant Value table report pins or ports with set_logic constants, constants in the netlist, and output pins with a constant function. These output pins are from library cells or netlist modules.

```
tempus > report_case_analysis -all -no_split
----------------------------------
Pin name                Logic
                        constant
                        value
----------------------------------
AB_IS/t0/Z              0
AB_IS/t1/Z              1
AB_IS/dma_reg/SE        0
AB_IS/pre_reg[1]/SE     0
AB_IS/ECO1reg[2]/SE     0
----------------------------------
```

- The following command reports the Logic Constant Value table of pins:

```
tempus > report_case_analysis -all -no_split -verbose
-------------------------------
Pin name              Logic
                      constant
                      value
-------------------------------
AB_IS/t0/Z            0
AB_IS/t1/Z            1
AB_IS/dma_reg/SE      0

Pin tst_mode 0 view scan_fast_RCMIN is caused by set_case_analysis 0 on pin tst_mode
```

```
Pin tst_mode 0 view scan_slow_RCMIN is caused by set_case_analysis 0 on pin tst_mode


Pin tst_mode 0 view scan_fast_RCMAX is caused by set_case_analysis 0 on pin tst_mode
```

- The following command reports user case analysis values for the specified view:

```
tempus > report_case_analysis -view scan_fast_RCMIN
-------------------------------------------
Pin name       User        View Name
               case
               analysis
               value
-------------------------------------------
tst_mode        0          scan_fast_RCMIN
scan_en         0          scan_fast_RCMIN
-------------------------------------------
```

- The following command reports pins that include the direction of the pin - as input:

```
tempus> report_case_analysis -dir

-----------------------------------------
Pin name          Dir      User case
                           Analysis value
-----------------------------------------
gscanmode        input     0
gscanenable      input     0
-----------------------------------------
```

- The following command report s all the constant pins, including pins with propagated constants:

```
tempus> report_case_analysis -propagated

---------------------------------------
Pin name                      Constant
                              value
---------------------------------------
clockandlatch/U8/TE           0
clockandlatch_dup1/U8/TE      1
clockandlatch_dup2/U8/TE      1
clockandlatch_dup3/U8/TE      0
---------------------------------------
```

- The following command reports details of each pin separately. The pin `crclocklatch/U8/TE` is reported because of a `set_case_analysis 0` assertion on pin gscan:

```
tempus> report_case_analysis -propagated -verbose crclocklatch/U8/TE

---------------------------------
Pin name               Constant
```

```
                          Value
---------------------------------
gscan                      0
FE_OC36_gscan/I            0
FE_OC36_gscan/ZN           1
FE_OC37_gscan/I            1
FE_OC37_gscan/ZN           0
FE_OC35_gscan/I            0
FE_OC35_gscan/Z            0
crclocklatch/U8/TE         0
---------------------------------
```

Related Commands

- set_case_analysis

report_cell_instance_timing

```
report_cell_instance_timing
[-help]
hinst_list
[-clock_timing]
[-tcl_list]
[-view view_name]
[> file_name]
[>> file_name]
[-early | -late]
```

Reports the following information about the timing context for all the pin and delay arcs of the specified instances:

- Arrival time
- Required time
- Slack
- Delays

Note: Internal pins (pins inside the instance boundary) are also reported.

This command uses the results of earlier timing analysis runs; therefore, you must run the `report_timing` command before using `report_cell_instance_timing`.

To analyze the setup (maximum) paths, you must use the `timing_analysis_check_type setup` attribute and run the `report_timing` command before using the `-late` parameter. Similarly, to analyze the hold (minimum) paths, you must use the `timing_analysis_check_type hold` attribute and run the `report_timing` command before using the `-early` parameter.

The `report_cell_instance_timing` timing report contains the following tables:

- The first table in the report contains information about the pins of the instance.
  The propagated slew is the slew that is propagated to the pin, based on the slew propagation mode.
- The second table contains information about the arcs through the instance.
  The slew out is the slew calculated at the arc output from the arc input slew, using the timing model.
- The third table, generated for sequential devices, contains information about the timing checks, if present, on the cell.
  Only checks that are from the library are shown in this table. Automatic checks, such as clock gating setup and hold, are not reported.

The *Phase* column of each table can contain the following symbols:

- `D` indicates it is a data signal in the clock domain.
- `C` indicates it is a clock signal in the clock domain.
- `P` indicates it is a positive phase.

- `N` indicates it is a negative phase.
- `L` indicates it is a latency phase.
- Asterisk (`*`) indicates there is a timing exception associated with the signal.

  Note: Currently, it is not possible to expand the asterisk to determine which exception is associated with the reported timing phase.

When the software is in multi-mode multi-corner timing analysis mode, the timing context information is reported for each active analysis view, with the view name listed in the *Phase* column.

Parameters

`-help`	Prints out the command usage.
`{> | >>} filename`	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. The *`filename`* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-clock_timing`	Specifies to report slack for clock style checks.
`-early` | `-late`	Reports the information for early (maximum) path (data hold/clock setup) or late (minimum) path (data setup/clock hold). *Default*: If you do not specify any of the parameters, the software uses the `-late` parameter.
`hinst_list`	Specifies a flat (non-hierarchical) cell instance or list of cell instances. Use curly braces (`{}`) to enclose the list and separate the instance names with white space.

`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-view` *`view_name`*	Specifies the analysis view in which to query the object attribute.

Examples

- The following command reports reports information about the timing context of the specified cell instance:

```
tempus >  report_cell_instance_timing FE_63_data_out_13 -view scan_fast_RCMIN

--------------------------------------------------------------------------------
                     Instance FE_63_data_out_13 of BUF_X32

--------------------------------------------------------------------------------
Pin    Dir   Propagated   Arrival  Required  Slack   Phase
                          Slew

 ------------------------------------------------------------------------------
A ^    IN    0.067        5.567    38.295    32.728  m_clk(D)(P)(scan_fast_RCMIN)
A v    IN    0.055        5.528    38.214    32.687  m_clk(D)(P)(scan_fast_RCMIN)
--------------------------------------------------------------------------------
                     Instance FE_63_data_out_13 of BUF_X32

--------------------------------------------------------------------------------
Arc            Slew
--------------------------------------------------------------------------------
From    To    In      Out     Load   Delay              Phase
--------------------------------------------------------------------------------
A ^     Z ^   0.067   0.248   0.012   0.303   m_clk(D)(P)(scan_fast_RCMIN)
A v     Z v   0.055   0.140   0.012   0.384   m_clk(D)(P)(scan_fast_RCMIN)
--------------------------------------------------------------------------------
```

- The following example reports information about the timing context of instance seg1/u9:

```
tempus > report_cell_instance_timing seg1/u9

-----------------------------------------------------
Instance seg3/u14 of DFF
-----------------------------------------------------
Pin   Dir  Propagated  Arrival  Required  Slack   Phase
           Slew
-------------------------------------------------------------------
CK ^  IN   0.062       0.621                     CLK_W_3(C)(P) *
```

```
CK v  IN   0.065                                    CLK_W_3(D)(P) *
D v   IN   0.003       1.815    2.465     0.650    CLK_W_3(D)(P)
CK ^  IN   0.062                                    CLK_W_3(D)(N) *
-------------------------------------------------------------------
Instance seg3/u14 of DFF
-----------------------------------------------
Arc Slew
-----------------------------------------------
From   To    In     Out    Load   Delay  Phase
-------------------------------------------------------
CK ^   QN ^  0.062  0.058  0.000  0.196  CLK_W_3(C)(P) *
CK ^   QN v  0.062  0.041  0.000  0.236  CLK_W_3(D)(P) *
CK ^   QN ^  0.062  0.000                CLK_W_3(D)(N) *
-------------------------------------------------------
Instance seg3/u14 of DFF
-----------------------------------------------
Pins Phase
-----------------------------------------------
Check   Sig   Ref   Slack   Delay  Adjust   Sig           Ref
-------------------------------------------------------------------------------
HOLD    D ^   CK ^  1.570  -0.129  0.000   CLK_W_3(D)(P)  CLK_W_3(C)(P) *
SETUP   D ^   CK ^  0.372   0.186  -2.000  CLK_W_3(D)(P)  CLK_W_3(C)(P) *
-------------------------------------------------------------------------------
```

- If a constant is asserted or propagated to the instance pin, it is reported in the Phase column of the pin table, as shown in the following example:

```
tempus > report_cell_instance_timing p5042A

-----------------------------------------------
Instance p5042A of MUX2HVTD1
-------------------------------------------------------
Pin    Dir   Propagated  Arrival  Required Slack  Phase
             Slew
-------------------------------------------------------
I0 ^   IN    0.032       0.127    8.583    8.455  EJ_TCK(D)(P)
I0 v   IN    0.021       0.127    8.568    8.441  EJ_TCK(D)(P)
S      IN                                        Constant 0
Z ^    OUT   0.020       0.168    8.624    8.455  EJ_TCK(D)(P)

--------------------------------
Instance p5042A of MUX2HVTD1
--------------------------------
Arc Slew
-----------------------------------------------
From   To  In       Out     Load   Delay Phase
-----------------------------------------------
I0 ^   Z ^  0.032   0.020   0.002  0.041 EJ_TCK(D)(P)
I1 ^   Z ^  0.060   0.002                EJ_TCK(D)(P)
-------------------------------------------------
```

- When the software is in multi-mode multi-corner analysis mode, the timing context information is reported for each active analysis view. The view name is reported in the Phase column, as shown in the following example:

```
tempus > report_cell_instance_timing FE_OFC304_EJ_TDO

-----------------------------------------------------
Instance FE_OFC304_EJ_TDO of BUFFHVTD8
-----------------------------------------------------
Pin    Dir   Propagated  Arrival Required  Slack  Phase
Slew
-------------------------------------------------------------------------------
I ^   IN     0.052       8.812   16.260    7.448  EJ_TCK(D)(N)(setup_view1)*
I v   IN     0.033       8.799   16.319    7.520  EJ_TCK(D)(N)(setup_view1)*
-------------------------------------------------------------------------------
```

Related Information

- report_nets
- report_timing

report_clock_gating_check

```
report_clock_gating_check
[-help]
[obj_list]
[-view view_name]
[{> | >>} file_name]
```

Reports information of all or specific clock gating checks of the current design. If you specify a list of pins, cells, or clocks, the clock gating check information is reported for the specified objects only. If you use the `report_clock_gating_check` command without an object list, all clock gating check information is reported.

The `Type` column of the generated report (see examples below) can contain one of the following abbreviations:

- `I` indicates that the clock gating check is inferred.
- `L` indicates that the clock gating check is from the library (integrated clock gating check).

Parameters

`-help`	Prints out the command usage.
`obj_list`	Specifies a list of pins, instances, or clocks. A collection of these can also be specified. • Pin: Any pin of a clock gating cell can be specified and the check information will be reported for that pin. • Instance: Any instance name of a clock gating cell can be specified and the check information will be reported for that instance. • Clock: Only a clock at a given clock (reference) pin of a cell can be specified for reporting the check information. Check information about the clocks at enable (signal) pin cannot be reported. If the *`obj_list`* parameter is not specified, all the clock gating check points (inferred and library) of the current design are reported.
`-view` *`view_name`*	Specifies a view for which clock gating check is to be reported. The default is worst of all the views.

{> | >>} *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file.

Examples

- The following command reports clock gating check details for the specified view:

```
tempus > report_clock_gating_check -view AV_bc_off
Clock Gating Check Report
--------------------------------------------------------------------------------
Instance         Enable  Clock  Type  Level  Setup   Setup   Hold    Hold   View
                                             Rise    Fall    Rise    Fall
--------------------------------------------------------------------------------
sub_inst2/LCLK   EN      A      I      H     0.000   0.000   0.000   0.000  AV_bc_off
--------------------------------------------------------------------------------
```

- The following command generates a clock gating check report for the specified collection:

```
tempus > report_clock_gating_check CLK
--------------------------------------------------------------------------------
Clock Gating Check Report
--------------------------------------------------------------------------------
Instance          Enable  Clock  Type   Level  Setup  Setup  Hold    Hold  View
                                             Rise   Fall   Rise    Fall
--------------------------------------------------------------------------------
sub_inst2/LCLK    EN      A      I      H      0.000  0.000  0.000   0.000 AV_wc_on
--------------------------------------------------------------------------------
```

- The following command generates a clock gating check report for the current design:

```
tempus > report_clock_gating_check
```

  The clock gating check report returned by this command is as follows:

```
------------------------------------------------------------------
Clock Gating Check Report
------------------------------------------------------------------
Instance  Enable  Clock  Type Level  Setup  Setup   Hold  Hold  View
                                     Rise   Fall    Rise  Fall
------------------------------------------------------------------
C_and_1    A       B     I     H     0.000  0.000   -     -     view1
------------------------------------------------------------------
```

report_clock_network_transitions

```
report_clock_network_transitions
[-help]
[-all]
[-clock string]
[-exclude_pins string]
[-threshold float]
[-verbose]
[-view string]
```

Reports all the clock tree pins that have valid rise/fall maximum transition values.

This command generates reports on a per clock basis. For virtual clocks, no pins will be reported.

Parameters

`-help`	Prints out the command usage.
`-all`	Reports all the pins having a valid transition value.
`-clock string`	Specifies a list of clocks.
`-exclude_pins string`	Does not report the specified pins.
`-threshold float`	Does not report pins that have at least one of transition values (rise or fall) greater than the specified threshold value.
`-verbose`	Specifies to include the library cell information.
`-view string`	Specifies the name of the view.

Examples

The following command reports all pins along with the library cell information for the specified clock:

```
report_clock_network_transitions -clock m_digit_clk -verbose -view scan_slow_RCMAX -all

Clock: m_digit_clk

View: scan_slow_RCMAX

Clock Network:

             Pin                                Lib Cell                  Rise       Fall

             ---                                --------                  ----       ----

             TST_CONTROL_INST/p214748365A90/ZN   FrPK45_lib_v1.0/OAI21_X1    0.545      0.193
```

m_digit_clk__L1_I0/A	FrPK45_lib_v1.0/BUF_X32	0.545	0.193
m_digit_clk__L1_I0/Z	FrPK45_lib_v1.0/BUF_X32	0.460	0.183
m_digit_clk__L2_I1/A	FrPK45_lib_v1.0/BUF_X16	0.462	0.181

report_clock_propagation

```
report_clock_propagation
[-help]
-clock clock_list
[-max_paths number of paths]
-to pin_port_list
[-verbose]
[-view viewName]
[> filename[.gz]]
[>> filename[.gz]]
```

This command provides the ability to determine clocks that do not propagate to a particular node in the network, and report the causes for such occurrences.

Parameters

`-help`	Prints out the command usage
`-clock clock_list`	List or collection of clocks that are to be reported.
`-max_paths number of paths`	Reports number of maximum clock paths per clock.
`-to pin_port_list`	Specifies the pin/port list.
`-verbose`	Generates a verbose report for clock propagation stop reason.
`-view viewName`	Generates a report for the specified analysis view.
`> filename[.gz] | >> filename[.gz]`	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. **Note**: To write a compressed report, add the .gz extension to the file name.

Examples

- The following command reports the specified clocks and pin details:

```
tempus > report_clock_propagation -clock clk1 -to f0/QN
Clock: clk1
Point: f0/QN
View:  view1
Reason 1: The clock 'clk1' propagation stops across the trigger arc:
 (f0/CK->f0/QN)
The partial clock path:
----------------------------------------
Source    Sink     ArcType      ArcSense
----------------------------------------
clk1      f0/CK    (net)          (net)
f0/CK     f0/QN    rising_edge    non_unate
----------------------------------------
```

- The following command shows that clock propagation stops due to an inactive timing arc from b12/A to b12/Y:

```
set_disable_timing -from A -to Y b12
report_clock_propagation -to u99/CK -clock CLK_W_4

Clock: CLK_W_4
Point: u99/CK
Reason 1: The clock 'CLK_W_4' propagation stops due to the inactive timing arc:
(b12/A->b12/Y)

The partial clock path:

-----------------------------------------------
Source    Sink    ArcType       ArcSense
-----------------------------------------------
CLK4      b11/A   (net)           (net)
b11/A     b11/Y   combinational   positive_unate
b11/Y     b12/A   (net)           (net)
b12/A     b12/Y   combinational   positive_unate
-----------------------------------------------
```

- The following command shows that clock propagation stops due to no clock source found:

```
report_clock_propagation -to launchff/Q -clock [get_clocks gen_clk]
report_clock_propagation -to seg3/u15/Q -clock CLK_W_5_gen


Clock: CLK_W_5_gen
Point: seg3/u15/Q
Reason 1: The generated clock ‘CLK_W_5_gen could not be created on its root pin ‘seg3/u15/Q’
```

```
because no clock source found for it
```

- The following command shows that clock propagation stops at the register clock pin as all the arcs from the clock pin of the register have been disabled:

```
set_disable_timing -from A -to Y seg3/u17_a
report_clock_propagation -to seg3/u14/CK -clock CLK_W_3

Clock: CLK_W_3
Point: seg3/u14/CK
Reason 1: The clock 'CLK_W_3' propagation stops due to the inactive timing arc:
(seg3/u17_a/A->seg3/u17_a/Y)

The partial clock path:
-----------------------------------------------------------
Source        Sink          ArcType         ArcSense
-----------------------------------------------------------
CLK3          seg3/u16/A    (net)           (net)
seg3/u16/A    seg3/u16/Y    combinational   positive_unate
seg3/u16/Y    seg3/u17/A    (net)           (net)
seg3/u17/A    seg3/u17/Y    combinational   positive_unate
seg3/u17/Y    seg3/u17_a/A  (net)           (net)
seg3/u17_a/A  seg3/u17_a/Y  combinational   positive_unate
-----------------------------------------------------------
```

Related Commands

- set_disable_timing

report_clock_sense

```
report_clock_sense
[-help]
```

Reports all clock sense settings in the design.

Parameters

`-help`	Prints out the command usage.

Related Commands

- `set_clock_sense`
- `reset_clock_sense`
- `get_pins`
- `get_ports`

report_clock_timing

```
report_clock_timing
[-help]
[-cppr_relative]
[-latency_greater_than]
-type [skew | interclock_skew | jitter | summary | latency | cppr_stage_count
      [ [-launch | -capture]
        [-rise | -fall]
        | [-histogram [-histogram_range interval size]]
        [-logic_level [-source {clock_root | generated_clock}]]
        ]
      ]
[-absolute_compare]
[-early | -late]
[-clock clock_list]
[-from_clock from_clock_list]
[-to_clock to_clock_list]
[-from from_list]
[-to to_list]
[-nworst worst_entries]
[-greater_than lower_limit]
[-view view_name]
[-verbose]
[-format column_list]
[-tcl_list]
[{> | >>} filename]
```

Generates a clock analysis report for the current design. The software generates a separate report for each specified clock or pair of clocks. You specify the clocks using the `-clock` parameter, and clock pairs using the `-from_clock` and `-to_clock` parameters. You can use the `-nworst` and `-greater_than` parameters with the `-from_clock` and `-to_clock` parameters to generate reports with skew to the specified sink pins. You can use the `-early` and `-late` parameters to specify the type of skew to be reported.

Note: To remove common path pessimism from the reported skew, use the `timing timing_analysis_cppr both` setting before using the `report_clock_timing` command. The report contains a separate column for common path adjustment values.

You can use the `report_clock_timing` command to perform MMMC based SSTA analysis. In MMMC mode, the command reports the latency and clock skew for all individual views. You can use the `report_clock_timing -view` parameter to report clock timing for a user-specified view.

Parameters

`-help`	Prints out the command usage.

{> | >>} *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.
-absolute_compare	Reports skews with values compared in absolute form.
-clock *clock_list*	Specifies the list of clocks to be reported. The software generates a separate report for each clock that you specify in the *clock_list*. *Default:* Skew report is generated for all the clocks in a design.
-cppr_relative	Reports the latency in skew reports from the CPPR common pin. When specified, the reports are printed with latency measured from branch point to the register's CK pin. The CPPR value is no longer required in reports.
-early	Uses hold skew to generate the report. Hold skew is the difference between minimum latency at the start point flip-flop and the maximum latency at the end point flip-flop. Default: Uses the setup skew.

`-format` *`column_list`*	Specifies which columns to display for path reports, and the order in which they should appear. For example: `-format {instance arc pin cell slew load}` The `-format` parameter can be used only for latency or skew reports (`-type skew`, `-type interclock_skew`, or `-type latency`), and if the `-verbose` parameter is specified.
`-from` *`from_list`*	Specifies the list of sequential device clock pins or ports to consider as from-pins in the generated report. If any pin in the *`from_list`* is not a sequential device clock pin, it is replaced with all the sequential device clock pins in the transitive fanout of the named pin. If there are no sequential pins in the transitive fanout of the pin, the software drops the pin from the list and generates a warning message.
`-from_clock` *`from_clock_list`*	
	Specifies the list of from-clocks to be used in generating the current interclock skew report. You can use this option only in an interclock skew report. *Default:* Report is generated considering all clocks as from-clocks.
`-greater_than` *`lower_limit`*	Reports skews with values greater than the *`lower_limit`* value.

`-histogram`	Generates a textual histogram of the latency report. The `-histogram` parameter can be used only when `-type latency` is specified. When specified in conjunction with the `-clock` parameter, the software limits the latency reporting to the specified clocks only. When specified with the `-greater_than` parameter, the software limits the latency reporting to clocks with a latency value that is greater than the specified value. **Note:** The `-histogram` parameter cannot be specified in conjunction with the following parameters: `-launch`,`-capture`, `-from`, `-to`, `-from_clock`, `-to_clock`, `-verbose`, `-nworst`.
`-histogram_range` *`interval_size`*	Generates the histogram report at the specified intervals. The interval size specified must be a positive integer or decimal values. For example, if the maximum latency value is 437, and you specify `-histogram_range 50`, the histogram would be generated with intervals of: 0-50, 50-100, 100-150, ... 400-450. **Note:** The unit of *`interval_size`* will be the same as that used for generating the timing report. This parameter can be used only when the `-histogram` parameter is specified.
`-late`	Uses setup skew to generate the report. Setup skew is the difference between the maximum latency at the start point flip-flop and minimum latency at the end point flip-flop.
`-latency_greater_than` *`lower_limit`*	Reports are filtered based on the specified -type option. When specified with: • `-type skew` or `interclock_skew`: Reports are filtered for latency greater than the provided value or depth. • `-type cppr_stage_count`: Reports are filtered for depth greater than the provided value.

`-launch	-capture`	When specified with `-type latency`, generates a report of either the launch latency values (`-launch`), or the capture latency values (`-capture`). **Note:** You can specify these parameters only with `-type latency`. *Default*: `-launch`
`-logic_level`	Reports the levels (the stage count of gates) between the clock root, or generated clock root, and the capturing register. The `-logic_level` parameter can be used only when `-type latency` is specified. The `-logic_level` parameter cannot be specified in conjunction with the following parameters: `-launch`,`-capture`, `-from`, `-to`, `-from_clock`, `-to_clock`, `-verbose`, `-nworst`.
`-nworst` *worst_entries*	Specifies the number of entries to be reported per clock domain. *Default:* `1`
`-rise	-fall`	Specifies active transition at sequential devices clock pins. The `-rise` and `-fall` parameters are mutually exclusive and cannot be specified together. **Note:** You can specify these parameters only with the `-type latency` parameter.
`-source {clock_root	generated_clock}`	Reports the latency from either the master clock, or the generated clock that was derived from it. The `-source` parameter can be used only when `-logic_level` is specified. The `-source` parameter cannot be specified in conjunction with the following parameters: `-launch`,`-capture`, `-from`, `-to`, `-from_clock`, `-to_clock`, `-verbose`, `-nworst`. *Default:* Reports the latency from the master clock (`-source clock_root`)

`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-to` *`to_list`*	Specifies the list or collection of sequential device clock pins or ports to be considered for reporting. If any pin in the *`to_list`* is not a sequential device clock pin, it is replaced with all the sequential device clock pins in the transitive fanout of the named pin. If there are no sequential pins in the transitive fanout of the pin, the software drops the pin from the list and generates a warning message.
`-to_clock` *`to_clock_list`*	Specifies the list of to-clocks to be used in generating the current interclock skew report. You can use this option only in an interclock skew report. *Default:* Report is generated considering all clocks as to-clocks.
`-type {skew | interclock_skew | jitter | summary | latency | cppr_stage_count}`	Specifies the type of report to generate. The `-type` options are described below: • `skew`: Reports the skew for all clocks, or for clocks that you specify using the -clock parameter. • `interclock_skew`: Reports the skew between all clocks (excluding same clock pair) in the design, or between clocks that you specify using the `-from_clock` and `-to_clock` parameters. **Note**: The `skew` option can be used to report the previous results that show the missed out same clock skew values (reported using the `interclock_skew` option). • `jitter`: Reports the difference in late and early arrival times of a clock phase at a clock end point. When specified, the software generates a

clock jitter report for one clock end point, for each clock in the design.
You can generate a jitter report for all clocks, or clocks specified with the `-clock` parameter. You can limit the report to a specific clock end point, using the `-to` parameter.
Note: The `report_clock_timing -type jitter` option can be used only when the software is in On-Chip Variation (OCV) timing analysis mode. Jitter analysis does not remove any common path pessimism.

- `summary`: Generates a summary report for each analyzed clock that includes the following information:
  - When setup skew is used (`-late`):
    - maximum launch latency
    - minimum capture latency
    - maximum setup skew
  - When hold skew is used (`-early`):
    - minimum launch latency
    - maximum capture latency
    - maximum hold skew
- `latency`: Reports the network and source latency values for all the clock pins in the design, or for clock pins that you specify using the `-clock` parameter.
- `cppr_stage_count`: The generated report is arranged in terms of difference of stages from common pin to CK pin. The reports are generated on the basis of logical CPPR points derived between each pair of phases reaching the launch/capture CK pins. Only same clock pairs (similar to type skew) are picked. For example,

```
Branch Point Difference Depth Clock Pin
----------------------------------------
-
```

	`inv2/Y`
	`5 r ffc1/CK`
	`4 1 f ffc3/CKN`

`-verbose`	Generates a report that contains the full clock path from the clock source to the sink pin (the clock pin of the flop or latch). You can specify the `-verbose` parameter only for skew or latency reports (`-type skew`, `-type interclock_skew`, or `-type latency`). When this parameter is specified in SOCV mode, the intermediate arrival times of pins are shown as "-" in the report output.
`-view` *view_name*	Generates a report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

- The following command reports skew details for the specified list of clocks:

```
tempus > report_clock_timing -clock clk1 -type skew
Clock: clk1
Analysis View: view1

Skew     Latency    CPPR      Clock Pin
-----------------------------------------
         0.000                r    f0/CK
0.000    0.000     -0.000     r    f1/CK
-----------------------------------------
```

- The following command uses hold skew to generate a report and shows details of skew between all the clocks:

```
tempus > report_clock_timing -early -type interclock_skew
Clock: clk2
Analysis View: view1

Skew     Latency   CPPR     Clock Pin
-----------------------------------------
         0.000              r    f0/CK
-0.000   0.000     0.000    r    f1/CK
-----------------------------------------
```

- The following command reports details of skew between all the clocks for the specified sequential device clock pins:

```
tempus > report_clock_timing -from_clock {refclk m_spi_clk} -type interclock_skew
Clocks: m_spi_clk -> m_clk
```

```
Analysis View: func_slow_RCMAX

Skew       Latency    CPPR      Clock Pin
-------------------------------------------------------------
                                TST_INST/p88/ZN
          0.580                 r SPI_INST/spi_sr_reg[7]/CK
                                TST_INST/p97/Z
-3.761    4.340       0.000     r TDSP_INST/as_reg/CK
-------------------------------------------------------------
```

- The following command uses hold skew (greater than 2) to generate a report and shows details of skew between all the clocks:

```
tempus > report_clock_timing -from {refclk m_spi_clk } -type skew -greater_than 2

Clock: m_clk
Analysis View: scan_fast_RCMIN

Skew     Latency   CPPR      Clock Pin
-----------------------------------------------------
         3.960               r    TDSP_INST/areg[0]/CK
2.735    1.225     -0.000    r    ROM_512x16_INST/CLK

Clock: m_clk
Analysis View: scan_slow_RCMIN

Skew    Latency   CPPR      Clock Pin
-----------------------------------------------------
        3.960               r    TDSP_INST/areg[0]/CK
2.735   1.225     -0.000    r    ROM_512x16_INST/CLK
-----------------------------------------------------
```

- The following command generates the histogram latency report at the specified interval range:

```
tempus > report_clock_timing -clock m_clk -histogram -type latency -histogram_range 50
[Delay Chart: TST_INST/p21/Z(m_clk)]
----------------------------------------
Min Delay :      1.118
Max Delay :      4.332
Count : 406
----------------------------------------
-----------------------------------------------------
Delay Range
Min            Max       Count    1        10     100
-----------------------------------------------------
0.000          50.000    406     |oooooo   |      |oooo
[Delay Chart: TST_INST/p21/Z(m_clk)]
----------------------------------------
Min Delay :      1.052
Max Delay :      3.995
Count : 406
----------------------------------------
-----------------------------------------------------
Delay Range
```

```
Min        Max       Count    1         10     100
-----------------------------------------------------
0.000      50.000    406     |oooooo    |      |oooo
-----------------------------------------------------
```

- The following command uses setup skew to generate the report and shows skew details for the specified list of clocks:

```
tempus > report_clock_timing -clock m_clk -late -type skew
Clock: m_clk
Analysis View: scan_fast_RCMIN

Skew       Latency     CPPR      Clock Pin
-------------------------------------------------------------
           3.960                 r    TDSP_INST/acc_reg[0]/CK
2.735      1.225       -0.000    r    ROM_512x16_0_INST/CLK

Clock: m_clk
Analysis View: scan_slow_RCMIN

Skew    Latency    CPPR      Clock Pin
-------------------------------------------------------------
        3.960                 r    TDSP_INST/acc_reg[0]/CK
2.735   1.225      -0.000     r    ROM_512x16_0_INST/CLK
-------------------------------------------------------------
```

- The following command reports hold skew details for the specified list of clocks:

```
tempus > report_clock_timing -clock m_clk -early -type skew
Clock: m_clk
Analysis View: scan_fast_RCMIN

Skew     Latency    CPPR     Clock Pin
-------------------------------------------------------------
         1.225                r    ROM_512x16_0_INST/CLK
2.765    3.990       0.000    r    TSP_CRE/PRG_BS/dt_reg[12]/CK

Clock: m_clk
Analysis View: scan_slow_RCMIN

Skew      Latency  CPPR      Clock Pin
-------------------------------------------------------------
          1.225               r    ROM_512x16_0_INST/CLK
2.765     3.990    0.000      r    TSP_CRE/PRG_BS/dt_reg[12]/CK
-------------------------------------------------------------
```

- The following command reports launch latency values for the specified list of clocks:

```
tempus > report_clock_timing -clock m_clk -capture -type latency
Clock: m_clk
    Analysis View: func_slow_RCMAX

          ---Latency---
      Source     Network    Total     Clock Pin
  -------------------------------------------------------------
      0.000      1.102      1.102     r    ROM_512x16_0_INST/CLK
```

```
    Clock: m_clk
    Analysis View: func_fast_RCMAX

          ---Latency---
      Source    Network     Total    Clock Pin
  ------------------------------------------------------------
     0.000      1.310       1.310    r    ROM_512x16_0_INST/CLK
```

- The following command reports latency details for the specified number of entries per clock domain:

```
tempus > report_clock_timing -clock m_clk -nworst 4 -type latency
Clock: m_clk
  Analysis View: func_slow_RCMAX

        ---Latency---
    Source     Network    Total     Clock Pin
-------------------------------------------------------------
    0.000      4.735      4.735     r    DM_IST/write_reg/CK
    0.000      4.735      4.735     r    DM_IST/top_flg_reg/CK
    0.000      4.735      4.735     r    DM_IST/read_reg/CK
    0.000      4.735      4.735     r    DM_IST/state_reg[3]/CK
-------------------------------------------------------------
```

- The following command reports latency details for all the levels between the generated clock root, and the capturing register.

```
tempus > report_clock_timing -source generated_clock -logic_level -type latency
Start point:refclk(refclk)
---------------------------------------------
      Delay     Count     End Point
      3.697         3     SPI/spr_108/CK r
      3.697         3     SPI/spr_107/CK r
      3.697         3     SPI/spr_105/CK r
      3.697         3     SPI/spr_103/CK r
      3.697         3     SPI/spr_101/CK r
      3.697         3     SPI/spr_100/CK r
      3.697         3     SPI/spr_104/CK r
      3.697         3     SPI/spr_102/CK r
      3.697         3     SPI/spr_106/CK r
      3.697         3     SPI/spr_109/CK r
      3.697         3     SPI/spr_110/CK r
---------------------------------------------
```

- The following command reports worst case skew for all the clocks in a design:

```
tempus > report_clock_timing -type skew -nworst 4


Clock: CLK_W_4
Skew    Latency   Clock Pin
---------------------------
        0.150 r   u99/CK
0.000   0.150 r   u55/CK
```

```
Clock: CLK_W_3
Skew   Latency   Clock Pin
-----------------------------
        0.682 r seg3/u9/CK
0.061   0.621 r seg3/u14/CK

Clock: CLK_W_2
Skew    Latency Clock Pin
-----------------------------
        0.682 r seg2/u9/CK
0.061   0.621 r seg2/u14/CK

Clock: CLK_W_1
Skew    Latency Clock Pin
-----------------------------
        0.682 r seg1/u9/CK
0.061   0.621 r seg1/u14/CK
```

Here '`r`' indicates the transition (r - rise, f - fall) at the reported register clock pin.

- The following command reports interclock skew with -from_clock, -to_clock, -from, and -to parameters defined:

```
tempus > report_clock_timing -type interclock_skew -nworst 10 -from CLK4 -to u5/CK -from_clock
CLK_W_4 -to_clock CLK_W_4_gen2

Clocks: CLK_W_4 -> CLK_W_4_gen2
Skew       Latency    Clock Pin
-----------------------------
                      CLK4
           0.180 r    u99/CK
                      u2/Q
                      u8/Q
-0.205     0.386 ri   u5/CK
```

Here '`r`' indicates the transition (`r` - rise, `f` - fall) at the reported register clock pin. The '`i`' refers to the 'inverted' polarity of the clock signal at the reported register clock pin.

- The following command reports skew for all clocks in the design including common path pessimism removal:

```
tempus > set_db timing_analysis_cppr both
```

```
tempus > report_clock_timing -type skew -nworst 10

Clock: CLK_W_4
Skew    Latency    CPPR    Clock Pin
---------------------------------------
        0.180              r u99/CK
0.000   0.120       -0.060 r u55/CK

Clock: CLK_W_3
Skew     Latency  CPPR    Clock Pin
---------------------------------------
         0.818            r seg3/u9/CK
0.139    0.496 -0.183     r seg3/u14/CK

Clock: CLK_W_2
Skew    Latency  CPPR     Clock Pin
---------------------------------------
        0.818             r seg2/u9/CK
0.139   0.496    -0.183   r seg2/u14/CK

Clock: CLK_W_1
Skew     Latency   CPPR     Clock Pin
---------------------------------------
         0.818              r seg1/u9/CK
0.139    0.496    -0.183    r seg1/u14/CK
```

- The following command generates a summary report for the clock:

```
tempus > report_clock_timing -type summary -nworst 10 -clock CLK_W_1


Clock: CLK_W_1
                  Skew    Latency    Clock Pin
-----------------------------------------------
Maximum launch latency:   0.818 r    seg1/u9/CK
Minimum capture latency:  0.000 r    u2/CK
Maximum skew:             0.818 r    seg1/u9/CK
                  0.322   0.496 r    seg1/u14/CK
-----------------------------------------------
```

- The following command reports the Network and source latency values for the clock CLK_W_1:

```
tempus > report_clock_timing -type latency -nworst 10 -clock CLK_W_1

Clock: CLK_W_1
---Latency---
Source Network Total Clock Pin
-----------------------------------------------
0.000 0.818 0.818 r seg1/u9/CK
0.000 0.000 0.000 r u2/CK
-----------------------------------------------
```

- The following command reports the Network and source latency values for the clock GCLK1, as well as the full clock paths:

```
tempus > report_clock_timing -type latency -verbose -clock CLK_W_1

Clock: CLK_W_1
---Latency---
Source Network Total Clock Pin
-------------------------------------------
0.000 0.818 0.818 r seg1/u9/CK

Clock latency Path
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Other End Path:
-------------------------------------------
Instance       Arc      Cell     Delay   Clock
Latency
-------------------------------------------
-            CLK1 ^       -       -       0.000
seg1/u16     A ^ -> Y ^  CLKBUF   0.111  0.111
seg1/u17     A ^ -> Y ^  CLKBUF   0.126  0.236
seg1/u17_a   A ^ -> Y ^  CLKBUF   0.107  0.343
seg1/and1    B ^ -> Y ^  AND      0.071  0.818
seg1/u9      CK ^        DFF      0.000  0.818
-------------------------------------------
```

- When the software is in multi-mode multi-corner timing analysis mode, the software generates a report for every active analysis view in the design:

```
tempus > report_clock_timing -type latency

Clock: EJ_TCK
Analysis View: hold_view1
---Latency---
Source Network Total Clock Pin
----------------------------------------
0.000 0.000 0.000 r cregister_q_reg_3_/CP

Clock: EJ_TCK
Analysis View: hold_view2
---Latency---
Source Network Total Clock Pin
----------------------------------------
0.000 0.000 0.000 r cregister_q_reg_3_/CP

Clock: EJ_TCK
Analysis View: setup_view1
---Latency---
Source Network Total Clock Pin
----------------------------------------
```

```
0.000 0.000 0.000 r cregister_q_reg_3_/CP

Clock: EJ_TCK
Analysis View: setup_view2
---Latency---
Source Network Total Clock Pin
-----------------------------------------
0.000 0.000 0.000 r cregister_q_reg_3_/CP
```

- Specify the -view parameter to generate a report only for a specific analysis view

```
tempus > report_clock_timing -view setup_view1 -type latency

Clock: EJ_TCK
Analysis View: setup_view1
---Latency---
Source    Network    Total    Clock       Pin
-----------------------------------------
0.000     0.000      0.000    r    cregister_q_reg_3_/CP

Clock: SI_ClkIn
Analysis View: setup_view1
---Latency---
Source   Network   Total    Clock        Pin
-----------------------------------------
0.000    0.000     0.000     r     cregister_q_reg_13_/CP
```

- The following command generates a textual histogram of the latency report:

```
tempus > report_clock_timing -type latency -histogram


[Delay Chart: CLK4(CLK_W_4_gen2)]
************************************
Min Delay : 0.778
Max Delay : 0.778
Count : 1
************************************
----------------------------------------
Delay    Range
Min    Max      Count   1
----------------------------------------
0.000  100.000 1       |o
```

- The following command generates the histogram report at intervals of 0.5:

```
tempus > report_clock_timing -type latency -histogram -histogram_range 0.5


[Delay Chart: CLK4(CLK_W_4_gen2)]
************************************
```

```
Min Delay : 0.778
Max Delay : 0.778
Count : 1
************************************
------------------------------------
Delay Range
Min    Max    Count   1
------------------------------------
0.500  1.000  1       |o
0.000  0.500  0
```

- The following command generates a latency report that includes the stage gate counts between the clock root or generated clock root, and the capturing register:

```
tempus > report_clock_timing -type latency -logic_level
************************************************
Start point:CLK2(CLK_W_4_gen1(CLK_W_3))
------------------------------------------------
      Delay         Count     End Point
      1.018             8     seg2/u9/CK r
      1.018             8     seg2/u3/CK r
      0.945             7     seg2/u14/CK r
************************************************
Start point:CLK3(CLK_W_4_gen2(CLK_W_4))
------------------------------------------------
      Delay         Count     End Point
      1.345             8     seg3/u6/CK r
      1.345             8     seg3/u1/CK r
      1.218             7     seg3/u12/CK r
```

- The following command generates a latency report starting from the generated clock, and includes the stage gate counts between the generated clock and the capturing register:

```
tempus > report_clock_timing -type latency -logic_level -source generated_clock
************************************************
Start point:CLK2(CLK_W_4_gen1(CLK_W_3))
------------------------------------------------
      Delay         Count     End Point
```

```
      0.818             8     seg2/u9/CK r
      0.818             8     seg2/u3/CK r
      0.745             7     seg2/u14/CK r
************************************************
Start point:CLK3(CLK_W_4_gen2(CLK_W_4))
------------------------------------------------
      Delay         Count     End Point
      0.845             8     seg3/u6/CK r
      0.845             8     seg3/u1/CK r
      0.718             7     seg3/u12/CK r
```

- The following command generates a latency report starting from the generated clock CLK_W_4_gen2, and includes the stage gate counts between the generated clock and the capturing register:

```
tempus > report_clock_timing -type latency -logic_level -source generated_clock -clock
CLK_W_4_gen2
************************************************
Start point:CLK3(CLK_W_4_gen2(CLK_W_4))
------------------------------------------------
      Delay         Count     End Point
      0.845             8     seg3/u6/CK r
      0.845             8     seg3/u1/CK r
      0.718             7     seg3/u12/CK r
```

- The following command uses the -format parameter to customize the generated latency report:

```
tempus > report_clock_timing -type latency -verbose -format {instance arc pin cell slew load user_derate}
Clock: CLK_W_4
---Latency---
Source     Network      Total      Clock Pin
-------------------------------------------------------------
0.000      0.152         0.152      r u55/CK
Clock latency Path
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Other End Path:
-------------------------------------------------------------
Instance    Arc           Pin     Cell   Slew    Load     User
                                                          Derate
-------------------------------------------------------------
```

```
-           CLK4 ^         CLK4     -       0.003   0.025    -
b11         A ^ -> Y ^     Y        BUF     0.131   0.011    1.000
b12         A ^ -> Y ^     Y        BUF     0.117   0.015    1.000
u55         CK ^           CK       DFF     0.117   0.015    1.000
-------------------------------------------------------------------
```

- The following command reports the worst case jitter for all clocks in the design:

```
tempus > report_clock_timing -type jitter -nworst 2

Clock: CLK_W_2

Jitter  Latency(Late)   Latency(Early) Clock Pin
-----------------------------------------------
0.060    0.180 r            0.120 r     u55/CK
0.060    0.180 r            0.120 r     u99/CK
Clock:   CLK_W_1

Jitter  Latency(Late)  Latency(Early)  Clock Pin
-----------------------------------------------
0.273   0.818 r        0.545           r seg1/u9/CK
0.273   0.818 r        0.545           r seg1/u3/CK
```

- The following command reports the worst case jitter for the clock CLK_W_1:

```
tempus > report_clock_timing -type jitter -clock CLK_W_1 -nworst 2

 Clock: CLK_W_1

      Jitter      Latency(Late)    Latency(Early)            Clock Pin

-------------------------------------------------------------------------------

       0.273         0.818      r            0.545      r    seg1/u9/CK

       0.273         0.818      r            0.545      r    seg1/u3/CK
```

- The following command reports the worst case jitter for the specified clock end point:

```
tempus > report_clock_timing -type jitter -to seg1/u3/CK

Clock: CLK_W_1

      Jitter      Latency(Late)    Latency(Early)            Clock Pin
-------------------------------------------------------------------------------

      0.273         0.818      r            0.545      r    seg1/u3/CK
```

Related Commands

- report_timing
- check_timing

report_clocks

```
report_clocks
[-description]
[-arrival_points]
[-phase_shift_table]
[-total_shift_table]
[-uncertainty_table]
[-adjustment_table]
[-delay_adjustment_table]
[-source_insertion]
[-insertion]
[-hierarchy]
[-groups]
[-clocks clk_signame | clk_signame_list]
[-view view_name]
[{> | >>} filename | -tcl_list]
```

Reports information on clocks created with the `create_clock` and `create_generated_clock` command. The report includes information about all clock waveforms, clock arrival points, and clock uncertainties.

Use the `-description`, `-arrival_points`, `-uncertainty_table`, and the `*_table` parameters to specify different parts of the report. If one of these options is supplied, only the section that the option corresponds to is reported. If other options are supplied, those sections are also reported.

Based on the current setup or hold analysis mode, the following parameters report only one of the late or early tables:

- `-phase_shift_table`
- `-total_shift_table`
- `-uncertainty_table`
- `-adjustment_table`
- `-delay_adjustment_table`

However, in simultaneous (setup and hold) mode both the tables are reported.

The `report_clocks` command only reports constraints with respect to clock waveforms. Use the `report_ports -type` command for constraints applied to clock pins.

Parameters

{> | >>} *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.
-adjustment_table	Reports two tables that detail the hold and setup cycle adjustments between clock waveforms as set by the set_multicycle_path command. **Note:** If specific ideal (or generated) clocks are requested with the -clocks *clock_name_list* option, then only the relationships between those clock signals are reported.
-arrival_points	Reports only the clock arrival points. A clock arrival point is the pin where the create_clock or create_generated_clock command has been asserted. **Note:** This parameter is obsolete and will be removed in a future release.
-clocks *clk_signame* | *clk_signame_list*	Requests information on a specific ideal (or generated) clock. If a list is given, only information about those clocks is reported. You can use wildcards in the names. *Default*: All clocks, except generated clocks, are reported. Use the -generated option to report generated clocks. **Note**: This parameter is obsolete and will be removed in a future release. Cadence recommends that you specify clock names without the -clocks parameter.

`-delay_adjustment_table`	Reports two tables that detail the early and late path delay adjustments between clock waveforms as set by the `set_max_delay` and `set_min_delay` commands. **Note:** If specific ideal (or generated) clocks are requested with the `-clocks` *`clock_name_list`* option, then only the relationships between those clock signals are reported.
`-description`	Reports only the description of the specified clock waveform(s).
`-generated`	Includes information on generated clocks in the report, such as source and master clock of the generated clock, and other information specified in the `create_generated_clock` constraints. The report uses the internal clock names created by the system, if you did not specify new clock names while creating these clocks. **Note**: This parameter is obsolete and will be removed in a future release. The software now reports generated clocks by default.
`-groups`	Reports the existing clock groups.
`-hierarchy`	Reports the inter-clock relationship in tree style format.
`-insertion`	Reports the network insertion delays specified on clock waveforms. The values in the table are in `min:typ:max` format.
`-phase_shift_table`	Reports the table that details the phase shift between clock waveforms. Based upon the current analysis mode, the table shows the phase shifts either in late or early mode. In case of simultaneous setup hold mode, both the tables are reported for phase shift in late and early mode. **Note:** If specific ideal (or generated) clocks are requested with the `-clocks` *`clock_name_list`* option, then only the relationships between those clock signals are reported.
`-source_insertion`	Reports the source insertion delays specified on clock waveforms.

`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-total_shift_table`	Based on the current analysis mode, reports (late or early mode) details and the relationship between clock waveforms. In case of simultaneous setup hold mode all the details are reported. **Note:** If specific ideal (or generated) clocks are requested with the `-clocks` *`clock_name_list`* option, then only the relationships between those clock signals are reported.
`-uncertainty_table`	Based on the current analysis mode, reports one of the tables (late mode or early mode) that detail the uncertainty between clock waveforms as specified using the `set_clock_uncertainty` command. In case of simultaneous setup hold mode both the tables are reported. The tables report only those clock waveforms for which uncertainty has been defined using the `set_clock_uncertainty` command. **Note:** If specific ideal (or generated) clocks are requested with the `-clocks` *`clock_name_list`* option, then only the relationships between those clock signals are reported.
`-view` *`view_name`*	Generates a clock report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

- The following command reports the clock arrival points, and details of the clock waveform(s):

```
tempus > report_clocks -arrival_points -description
```

```
-------------------------------------------------------------------------------------------------
                                        Clock Descriptions
-------------------------------------------------------------------------------------------------
                                                                  Attributes
-------------------------------------------------------------------------------------------------
Clock Name  Source               View          Period    Lead    Trail   Generated   Propagated
-------------------------------------------------------------------------------------------------
m_clk       TST_INST/p21/Z   scan_slow_RCMAX   40.000   0.000   20.000      n           y
m_clk       TST_INST/p21/Z   scan_fast_RCMAX   40.000   0.000   20.000      n           y
m_clk       TST_INST/p21/Z   func_fast_RCMAX    4.000   0.000    2.000      n           y
m_clk       TST_INST/p21/Z   func_slow_RCMAX    4.000   0.000    2.000      n           y
m_clk       TST_INST/p21/Z   scan_slow_RCMIN   40.000   0.000   20.000      n           y
m_clk       TST_INST/p21/Z   func_fast_RCMIN    4.000   0.000    2.000      n           y
m_clk       TST_INST/p21/Z   scan_fast_RCMIN   40.000   0.000   20.000      n           y
m_clk       TST_INST/p21/Z   func_slow_RCMIN    4.000   0.000    2.000      n           y
-------------------------------------------------------------------------------------------------
```

- The following command reports the clock arrival points, along with the hold and setup cycle adjustments between clock waveforms:

```
tempus > report_clocks -arrival_points  -description -clocks m_clk -adjustment_table
```

```
-------------------------------------------------------------------------------------------
                                        Clock Descriptions
-------------------------------------------------------------------------------------------
                                                              Attributes
-------------------------------------------------------------------------------------------
Clock   Source              View          Period   Lead    Trail   Generated   Propagated
Name
-------------------------------------------------------------------------------------------
m_clk   TST_INST/p21/Z   scan_slow_RCMAX   40.000   0.000   20.000      n           y
m_clk   TST_INST/p21/Z   func_fast_RCMIN    4.000   0.000    2.000      n           y
m_clk   TST_INST/p21/Z   scan_slow_RCMIN   40.000   0.000   20.000      n           y
m_clk   TST_INST/p21/Z   func_slow_RCMIN    4.000   0.000    2.000      n           y
m_clk   TST_INST/p21/Z   scan_fast_RCMIN   40.000   0.000   20.000      n           y
m_clk   TST_INST/p21/Z   func_fast_RCMAX    4.000   0.000    2.000      n           y
m_clk   TST_INST/p21/Z   scan_fast_RCMAX   40.000   0.000   20.000      n           y
```

```
m_clk   TST_INST/p21/Z   func_slow_RCMAX    4.000   0.000    2.000      n           y
      ------------------------------------------------------------------------------
      ----------------------------------------------------------------

                    Clock To Clock Cycle Adjustment (Late)

      ----------------------------------------------------------------
        From    To          View          L->L    L->T    T->L    T->T
      ----------------------------------------------------------------
       m_clk   m_clk   scan_slow_RCMAX   0.000   0.000   0.000   0.000
       m_clk   m_clk   func_fast_RCMIN   0.000   0.000   0.000   0.000
       m_clk   m_clk   scan_slow_RCMIN   0.000   0.000   0.000   0.000
       m_clk   m_clk   func_slow_RCMIN   0.000   0.000   0.000   0.000
       m_clk   m_clk   scan_fast_RCMIN   0.000   0.000   0.000   0.000
       m_clk   m_clk   func_fast_RCMAX   0.000   0.000   0.000   0.000
       m_clk   m_clk   scan_fast_RCMAX   0.000   0.000   0.000   0.000
       m_clk   m_clk   func_slow_RCMAX   0.000   0.000   0.000   0.000
      ----------------------------------------------------------------
      ----------------------------------------------------------------
```

- The following command reports details of generated clocks, and includes information about all the clock waveforms and clock arrival points:

```
tempus > report_clocks
------------------------------------------------------------
Clock Descriptions
------------------------------------------------------------
Attributes
------------------------------------------------------------
Clock Name  Source  Period  Lead   Trail  Generated Propagated
------------------------------------------------------------
CLK1        CLK1    20.000  0.000  10.000  n        y
CLK2        CLK1    10.000  0.000   5.000  n        y
------------------------------------------------------------
Generated-Clock Descriptions
-------------------------------------------------------------------------------------------------
Name        Generated   Master      Master-clock Invert Freq.      Duty-Cycle Edges Edge-Shift
            Source(pin) Source(pin)                 Multiplier
-------------------------------------------------------------------------------------------------
GCLK        u8/Q        CK1         CLK1         n      1/2         -          -      -
-------------------------------------------------------------------------------------------------
```

- The following command reports uncertainty information as per the defined assertions:

```
tempus > set_clock_uncertainty 5 CLK_W_1
tempus > set_clock_uncertainty 4 -from CLK_W_1 -to CLK_W_4_gen1
tempus > set_clock_uncertainty 6 -from CLK_W_2 -to CLK_W_4_gen2
tempus > report_clocks -uncertainty_table

-------------------------------------------------------
Clock To Clock Uncertainty (Late)
-------------------------------------------------------
From    To           L->L    L->T    T->L     T->T
-------------------------------------------------------
*       CLK_W_1      5.000  5.000   5.000     5.000
CLK_W_1 CLK_W_4_gen1 4.000  4.000   4.000     4.000
CLK_W_2 CLK_W_4_gen2 6.000  6.000   6.000     6.000
-------------------------------------------------------
Clock To Clock Uncertainty (Early)
-------------------------------------------------------------
From     To            L->L   L->T   T->L   T->T
-------------------------------------------------------------
*        CLK_W_1      -5.000 -5.000 -5.000 -5.000
CLK_W_1  CLK_W_4_gen1 -4.000 -4.000 -4.000 -4.000
CLK_W_2  CLK_W_4_gen2 -6.000 -6.000 -6.000 -6.000
-------------------------------------------------------------
```

- The following command reports the phase shift between clock waveforms for the defined clock:

```
tempus > create_clock -name "CLK_W_1" -add -period 60 -waveform {0 30}
tempus > report_clocks CLK_W_1 -phase_shift_table

-----------------------------------------------------
Clock To Clock Phase Shift (Late)
-----------------------------------------------------
From    To      L->L    L->T  T->L   T->T
-----------------------------------------------------
CLK_W_1 CLK_W_1 60.000  0.000 60.000 60.000
-----------------------------------------------------
-----------------------------------------------------
Clock To Clock Phase Shift (Early)
-----------------------------------------------------
From    To      L->L   L->T   T->L  T->T
-----------------------------------------------------
CLK_W_1 CLK_W_1 0.000 -60.000 0.000 0.000
-----------------------------------------------------
```

- You can use the following command to define uncertainty for a clock:
  Based on the current analysis mode, reports (late or early mode) details and the relationship between clock

waveforms.

```
tempus > set_clock_uncertainty 2 CLK_W_1
```

The following command reports (late or early mode) details and displays the changed shift values:

```
tempus > report_clocks -total_shift_table CLK_W_1

-----------------------------------------------------
Total Clock To Clock Phase Shift Required (Late)
-----------------------------------------------------
From    To      L->L    L->T  T->L   T->T
-----------------------------------------------------
CLK_W_1 CLK_W_1 58.000 -2.000 58.000 58.000
-----------------------------------------------------
-----------------------------------------------------
Total Clock To Clock Phase Shift Required (Early)
-----------------------------------------------------
From    To      L->L   L->T   T->L  T->T
-----------------------------------------------------
CLK_W_1 CLK_W_1 2.000 -58.000 2.000 2.000
-----------------------------------------------------
```

You can see that the uncertainty is subtracted from the phase shift.
Consider the following example path:

```
Path 1: MET Setup Check with Pin u2/CK
Endpoint: u2/D (v) checked with leading edge of 'CLK_W_1'
Beginpoint: CLK2 (v) triggered by trailing edge of 'CLK_W_2'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.188
+ Phase Shift 15.000
- Uncertainty 2.000
= Required Time 12.812
- Arrival Time 12.463
= Slack Time 0.349
Clock Fall Edge 12.500
= Beginpoint Arrival Time 12.500
-------------------------------------
Load  Slew    Delay   Arrival   Cell  Arc          Pin
                      Time
-----------------------------------------------------
0.034  0.003  12.500                  CLK2 v       CLK2
0.003  0.003  -0.037  12.463   BUF    A v -> Y v   u10999/Y
0.003  0.003  0.000   12.463   DFF    D v          u2/D
-----------------------------------------------------
```

Here the begin point CLK_W_2 to end point CLK_W_1 shows a phase shift value of 15.00. This means that if a path starts from the falling edge of CLK_W_2 , and ends at the rising edge of CLK_W_1, the phase shift is 15.00. In other words, when the timing (setup time) is reported, the required time at the end point begins with 15.00 (in addition to adjustments like setup time, time borrowing, uncertainty, etc). However, the arrival

time of this path starts at 12.500.

- When the software is in multi-mode multi-corner timing analysis mode, the software reports the clock information for each active analysis view:

```
tempus > report_clocks -total_shift_table -description

------------------------------------------------------------
Clock Descriptions
------------------------------------------------------------
Attributes
-------------------------------------------------------------------------
Clock Name Source  View       Period  Lead   Trail Generated Propagated
-------------------------------------------------------------------------
EJ_TCK     EJ_TCK  hold_view2 17.400  0.000  8.700  n         n
EJ_TCK     EJ_TCK  hold_view1 17.400  0.000  8.700  n         n
EJ_TCK     EJ_TCK  setup_view117.400  0.000  8.700  n         n
-------------------------------------------------------------------------
------------------------------------------------------------
Total Clock To Clock Phase Shift Required (Late)
------------------------------------------------------------
From      To        View        L->L   L->T   T->L    T->T
------------------------------------------------------------
EJ_TCK    EJ_TCK    hold_view2  17.400  0.000 17.400  17.400
EJ_TCK    EJ_TCK    hold_view1  17.400  0.000 17.400  17.400
EJ_TCK    EJ_TCK    setup_view2 17.400  0.000 17.400  17.400
-------------------------------------------------------------------------
```

- The following command reports the late or early mode details and displays the changed shift values for the specified view:

```
tempus > report_clocks -total_shift_table -description -view setup_view1 > clk_sv1

------------------------------------------------------------
Clock Descriptions
------------------------------------------------------------
Attributes
------------------------------------------------------------
Clock Name      Source  View         Period  Lead  Trail  Generated Propagated
-------------------------------------------------------------------------------
EJ_TCK          EJ_TCK  setup_view1  17.400  0.000 8.700  n         n
virt_in_EJ_TCK  -       setup_view1  17.400  0.000 8.700  n         n
-------------------------------------------------------------------------------
------------------------------------------------------------------
Total Clock To Clock Phase Shift Required (Late)
------------------------------------------------------------------
From            To      View        L->L     L->T   T->L     T->T
------------------------------------------------------------------
EJ_TCK           EJ_TCK setup_view1 17.400   0.000  17.400  17.400
virt_in_EJ_TCK   EJ_TCK setup_view1 17.400   0.000  17.400  17.400
------------------------------------------------------------------
```

- The `report_clocks` command output displays a column showing active or inactive clocks in the design:

```
------------------------------------------------------------
Clock Descriptions
------------------------------------------------------------
Attributes
------------------------------------------------------------
Clock Name    Source    Period    Lead    Trail    Generated    Propagated    Active
-------------------------------------------------------------------------------------
CLK_W_1       CLK1      20.000    0.000   10.000    n           y             y
CLK_W_2       CLK2      25.000    0.000   12.500    n           y             y
CLK_W_4_gen2  u8/Q      20.000    0.000   10.000    y           y             y
-------------------------------------------------------------------------------------
```

Related Information

- create_clock
- create_generated_clock
- set_clock_uncertainty

report_constraint

```
report_constraint
[-all_violators]
[-late]
[-early]
[-em_type {peak | rms | avg | all}]
[-check_type { pulse_width | clock_period | recovery | removal | clock_gating_setup |
clock_gating_hold | skew | pulse_clock_max_width | pulse_clock_min_width | electromigration |
rail_swing}]
[-clock]
[-connection_class]
[-data]
[-drv_fields {field_list}]
[-drv_output_format {txt|csv}]
[-drv_violation_type { max_capacitance | max_transition | max_fanout | min_capacitance |
min_transition | min_fanout | pulse_clock_max_transition | pulse_clock_min_transition}]
[-max_pins_per_drv <integer>]
[-retime {aocv | path_slew_propagation | aocv_path_slew_propagation}]
[-retime_mode {path | exhaustive}]
[-view viewName ]
[-worst_drv_per_net]
[pin_port_list]
[-verbose]
[{> | >>} filename[.gz]]
```

Reports constraint information of current design. The constraints information includes information such as whether or not constraint is violated, by how much amount, and the worst violator object.

The following timing checks are reported when you use the `report_constraint` command:

- All setup and hold checks
- DRV
- Clock and asynchronous checks including pulse_width, min_period, recovery, removal, clock_gating_setup, clock_gating_hold, skew

You can specify the design rule constraints on pins, ports, cells, or a design using the commands: `set_min_fanout`, `set_max_fanout`, `set_min_transition`, `set_max_transition`, `set_min_capacitance`, and `set_max_capacitance`.

The `report_constraint` command checks against the worst design rule constraint between the library constraint limit and the user-specified constraint value, when both are applied on the violating pin or port.

The `report_constraint` command supports multi-threaded reporting. You can make the following settings to enable multi-threaded reporting:

```
set_multi_cpu_usage -localCpu number_of_CPU
set_table_style -no_frame_fix_width [-nosplit]
```

By default, the `report_constraint` command reports only the mean slews in transition DRV checks. You can enable DRV checking to use mean as well as the n-sigma slew values (in SOCV mode), with the following setting:

```
set timing_report_max_transition_check_using_nsigma_slew true
```

Configurable DRV Reports

Tempus allows you to perform design rule violation (DRV) checks to ensure that the netlist meets the design rule constraints specified in the design library. To enable configurable DRV reporting, you can use the `timing_report_constraint_format` attribute. For example,

```
set timing_report_constraint_format {list of DRV fields}
report_constraint -drv_violation_type {list of DRV checks} \
        -all_violators | pin_or_port_list
```

You can also use the following command to specify DRV fields:

```
report_constraint -drv_violation_type {list of DRV checks} -drv_fields field_list
```

The field list specified with the `-drv_fields` parameter has higher precedence and will override the field list specified with the `timing_report_constraint_format` global variable. If DRV fields are not specified with this global variable (or with the `report_constraint -drv_fields` command), then the default set of DRV fields will be displayed. For a complete list of DRV fields, refer to the DRV Fields table.

For more information, refer to the Design Rule Violation (DRV) Reporting section in the *Tempus Stylus User Guide*.

Parameters

`{> | >>}` *`filename`*	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.

`-all_violators`	Reports the worst negative slack per violating pin against all the timing and design rule constraint checks. For timing checks, the violations are reported at timing endpoints. The DRV is checked and reported on leaf pins and ports. The supported check types are: • setup • hold • clock_gating_setup • clock_gating_hold • recovery • removal • clock_period • pulse_width • max_capacitance • max_transition • max_fanout • min_capacitance • min_transition • min_fanout • skew • pulse_clock_max_width • pulse_clock_min_width • pulse_clock_max_transition • pulse_clock_min_transition You can use the `-all_violators` parameter for reporting specific check types to generate report for that particular check type only. For example, you can use `-all_violators` with `-late` to report setup checks and path delays from set_max_delay.
`-check_type`	Reports setup or hold checks. The supported check types are: • `clock_gating_setup`: Reports only the paths that end at the clock_gating_setup timing check. • `clock_gating_hold`: Reports only the paths that end at the clock_gating_hold timing check. • `electromigration`: Reports signal EM analysis related violations. When

high density current passes through a thin metal wire, the high energy electrons exert forces on atoms causing them to migrate. Electromigration (EM) can drastically reduce the lifetime of a chip by increasing the resistivity of a metal wire or creating a short circuit between adjacent lines. When this option is specified, the software reports standard cell level electromigration violations.

- `recovery`: Reports paths that end at the recovery timing check.
- `removal`: Reports paths that end at the removal timing check.

  Note: The check types - recovery, removal, clock_gating_setup, and clock_gating_hold are also supported in AOCV mode.
- `pulse_width`: Reports only the paths that end at the pulse_width timing check.
  If you do not specify the `-all_violators` parameter, then the pulse_width option reports only one entry of the most critical pulse_width check. The DRV checks also report only the most critical entry.

  To report all the violated pulse_width checks, you need to specify `report_constraint -check_type pulse_width -all_violators` command (which is consistent with other check types of the `report_constraint` command).

  To report both violating and met entries, you can use the `report_min_pulse_width` command.
- `clock_period`: Reports only the paths that end at the clock_period timing check.
- `skew`: Reports skew checks.
  The `report_constraint` command performs skew checks by default. Skew checks are implemented as setup checks that are reported in the late analysis or simultaneous mode. These checks are not reported in the early mode. Skew checks are performed assuming the signal and reference clocks have the same frequency. If the frequencies of the signal and reference clocks are different, a warning message is displayed.
- `pulse_clock_max_width`: Reports the maximum pulse clock width checks set using the set using `set_pulse_clock_max_width` command.
- `pulse_clock_min_width`: Reports the minimum pulse clock width checks.
- `rail_swing`: Reports details related to the rail swing checks. For rail swing check settings, you can use the following attributes:

	○ `timing_rail_swing_checks_low_voltage_threshold` ○ `timing_rail_swing_checks_high_voltage_threshold`
`-clock`	Specifies that the DRV checks will be restricted to the clock network. If any pin is part of both the clock and data network, it will be reported as a clock.
`-connection_class`	Reports violations if the connection class attributes of the driving pin and receiving pin of a net are different.
`-data`	Specifies that the DRV check will be restricted to the data network. If any pin is part of both the clock and data network, it will be reported as a clock network pin. By default, the DRV checks are applied to both clock and data networks.
`-drv_fields {`*`field_list`*`}`	Allows only the specified fields to be included in the DRV report - in the same order as defined. For a complete list of DRV fields, refer to the DRV Fields table.
`-drv_output_format {txt | csv}`	Specifies that DRV output format – txt or CSV. The default value is “txt”.

`-drv_violation_type`	Reports DRV violations. The supported DRV check types are: • max_capacitance: Reports pins having the worst `max_capacitance` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual capacitance is greater than the `max_capacitance` value specified in library and timing constraints file. • min_capacitance: Reports pins having the worst `min_capacitance` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual capacitance is less than the `min_capacitance` value specified in the library and timing constraints file. • max_transition: Reports pins having the worst `max_transition` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual transition is greater than the `max_transition` value specified in the library and timing constraint file. **Note:** These checks are not reported in hold analysis mode. The `set_si_mode -enable_drv_with_delta_slew true` setting is required for max_transition DRV violations with SI slew. • min_transition: Reports pins having the worst `min_transition` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual transition is less than the `min_transition` value specified in the library and timing constraints file. • max_fanout: Reports pins having the worst `max_fanout` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual fanout is greater than the `max_fanout` value specified in the library and timing constraints file. • min_fanout: Reports pins having the worst `min_fanout` constraint. If you specify the `-all_violators` parameter, you can see all the pins whose actual fanout is less than the `min_fanout` value specified in the library and timing constraints file. • pulse_clock_max_transition: Reports maximum pulse clock transition checks. • pulse_clock_min_transition: Reports minimum pulse clock transition checks.
`-early`	Reports all of the hold checks, including regular, clock_gating_hold, and data-to-data hold checks. Also reports path delays from set_min_delay. **Note:** This parameter does not report removal checks.

`-em_type {peak | rms | avg | all}`	Reports DRV violations as per the specified EM type table lookup. When this parameter is specified, the software reads the em_max_toggle_rate table in the Liberty library for the specified EM current type (RMS, peak, or average).
`-late`	Reports all of the setup checks, including regular, clock_gating_setup, and data-to-data setup checks. Also reports path delays from set_max_delay. **Note:** This parameter does not report recovery checks.
`-max_pins_per_drv`	Reports the specified number of top worst violations per DRV category, such as, max_transition, max_capacitance. If this parameter is not specified, then by default 1000 entries per DRV category will be reported.
pin_port_list	Controls the output of DRV constraints. When specified, all DRV violations are reported for the specified pins or ports only.
`-retime {aocv | path_slew_propagation | aocv_path_slew_propagation}`	
	Reanalyzes the specified set of paths using the specified method. **Note**: The parameter supports setup/hold check types only. `aocv`: Re-evaluates the given set of graph-based analysis (GBA) paths by reverting back graph-based AOCV derate factors and applying path- based AOCV derates instead of less pessimistic derates. In this case the slew (and respective delays) values are worst that are propagated during graph-based analysis. `aocv_path_slew_propagation`: Re-evaluates the given set of graph-based analysis (GBA) paths by using path-based AOCV derates and delays using actual path- based propagated slew. `path_slew_propagation`: Re-evaluates the given set of graph-based analysis (GBA) paths by recalculating the delay values based upon actual propagated slew across the path. In this case the AOCV derates are the ones that are reported during graph-based analysis.

`-retime_mode {path | exhaustive}`	Allows the `report_constraint` command to be run in exhaustive path-based analysis (PBA) mode. In this mode, the software will exhaustively examine nworst paths of the selected endpoints and then select the true worst path to be reported - based on retimed slacks of all the examined paths. To enable exhaustive mode, the use model is given below: `report_constraint -all_violators -retime path_slew_propagation -retime_mode exhaustive` When you use this parameter, reporting can be run time intensive as the software needs to do an exhaustive search for true worst paths. Therefore, it is recommended to use this option in the later phases of sign-off stage when there are very few violations left for analysis. *Default*: `path`
`-verbose`	Generates a report that displays the full data path with the accompanying required time and slack calculation.
`-view` *viewName*	Generates a report containing endpoint and other constraint information for the specified analysis view. You can only specify this parameter when the software is in multi-mode multi-corner analysis mode. *Default:* Generates a report containing the worst endpoint and constraint information based on all of the analysis views.
`-worst_drv_per_net`	Reports only the worst DRV violation per net.

DRV Fields

Field Name	Report Column	Description
capacitance_actual	Actual Capacitance	Specifies the actual minimum or maximum capacitance at a pin or port. This field is used to determine the capacitance slack value.
capacitance_slack	Capacitance Slack	Specifies the max_cap/min_cap slack at a pin or port.

direction	Direction	Specifies the pin or port direction.
driver_max_capacitance	Driver Required Maximum Capacitance	Specifies the max_capacitance limit at the driver pin or port.
driver_max_fanout	Driver Required Maximum Fanout	Specifies the worst max_fanout limit on the driver pin or port.
driver_max_transition	Driver Required Maximum Transition	Specifies the max_transition limit at the driver pin or port.
driver_min_capacitance	Driver Required Minimum Capacitance	Specifies the min_capacitance limit at the driver pin or port.
driver_min_transition	Driver Required Minimum Transition	Specifies the min_transition limit at the driver pin or port.
driver_pin	Driver	Specifies the leaf driver pin or port name.
driver_ref_cell	Driver Libcell	Specifies the leaf driver reference cell or port.
driver_transition	Driver Actual Transition	Specifies the actual transition at driver pin or port.
drv_source	Source of DRV	Specifies the source of the DRV constraint which is used for the DRV check. Reports 'library_constraint' or 'user_constraint' depending on which DRV violation check is used.
edge	Edge	The signal transition direction on the net. Returns "rise" or "fall" if the DRV report corresponds to a rising or falling transition on the net.
fanout_actual	Actual Fanout	Specifies the actual minimum or maximum fanout at a pin or port. This field is used to determine the fanout slack value.
fanout_slack	Fanout Slack	Specifies the max_fanout/min_fanout slack at a pin or port.
hnet	Hierarchical Nets	Reports the hierarchical nets associated with violating pins.
lib_constraint	Library Constraint	Specifies the value of the library constraint on the violating pin. If not defined, reports as "NA".
max_capacitance	Required Capacitance	Specifies the maximum capacitance limit for a pin.

max_clock_frequency	Frequency	Specifies the frequency of the clock reported under the max_frequency_clock field.
max_fanout	Required Fanout	Specifies the maximum fanout limit for a pin.
max_frequency_clock	Maximum Frequency Clock	Specifies the name of the highest frequency clock constraining the pin. For clock network pins, the fastest clock reaching the pin will be reported. For data network pins, the fastest launch clock reaching the pin will be reported.
max_slack	Worst Setup Slack	Specifies the worst setup slack through at a pin or port.
max_transition	Required Transition	Specifies the maximum transition limit for a pin or port.
min_capacitance	Required Capacitance	Specifies the minimum capacitance limit for a pin.
min_fanout	Required Fanout	Specifies the minimum fanout limit for a pin.
min_slack	Worst Hold Slack	Specifies the worst hold slack through at a pin or port.
min_transition	Required Transition	Specifies the minimum transition limit for a pin or port.
net	Net	Specifies the top flat net connected to violating pin (net name at topmost hierarchy) flat net.
net_fanout	Net Fanout	Specifies the number of leaf level load pins of the net connected to a pin or port.
net_resistance	NetResistance	Resistance of the net connected to a pin or port.
net_type	Net Type	Specifies one or more of the following identifiers to indicate a special net: • ideal: the net is marked ideal using the set_ideal_network command • clock: the net is part of a clock network • constant: the net is tied to logic 1/0 respectively • multidriven: the net has multiple driver • dont_touch: the net is marked as dont_touch using set_dont_touch command • " -": None of the above

partition	Partition	Specifies the name of the SPEF hierarchy the violating pin appears in.
pin	Pin	Reports the pin or port on which DRV check is performed.
pin_type	Type	Specifies whether the pin/port is a data or a clock pin/port. Any clock network pin will be reported as CLOCK. Any pin that belongs to only data network will be reported as DATA. If a pin/port belongs to both data and clock networks, it will be reported as CLOCK.
receiver_max_transition	Receiver Required Maximum Transition	Specifies the max_transition limit at a receiver pin or port.
receiver_pin	Receiver	Specifies the leaf receiver pin name.
receiver_ref_cell	Receiver Libcell	Specifies the leaf reference cell name of a receiver pin.
receiver_transition	Receiver Actual Transition	Specifies the actual transition at a receiver pin or port.
ref_cell	Violating Libcell	Specifies the reference library cell the violating pin is attached to.
sequential	Sequential	Specifies if the pin belongs to a sequential cell or a combinational cell. For ports, the report will show "false".
total_capacitance	TotalCapacitance	Total capacitance of the net connected to a pin or port.
transition_actual	Actual Transition	Specifies the actual minimum/maximum transition at a pin or port. This can be base transition or base plus crosstalk-induced transition depending on the timing_max_transition_use_si_transition setting to false (default) or true, respectively.
transition_si	SI Transition	Specifies only the delta SI transition time at a pin or port. The total transition time comprises of the base transition time (transition value without SI crosstalk analysis) as well as delta SI transition time.

transition_sigma	Transition N-Sigma	Specifies the N-Sigma multiplier defined for transition with the `set_socv_reporting_nsigma_multiplier -transition` <*nsigma*> command. This field will be reported for SOCV STA, under the `timing_report_max_transition_check_using_nsigma_slew` to `true` setting. This is sensitive to views and -data and -clock specification. **Note**: This field will impact transition_actual and transition_slack computation for max_transition, min_transition, pulse_clock_max_transition, and pulse_clock_min_transition checks.
transition_slack	Transition Slack	Specifies the max_transition/min_transition slack at a pin/port pin/port.
user_constraint	User Constraint	Specifies user defined tran/cap/fanout limit value set by constraints as command or through SDC on the violating pin. If not defined, reports as "NA".
view	View	Name of the pertaining analysis view.

Examples

- The following command reports the worst negative slack per violating pin under pulse width checks:

```
tempus > report_constraint  FE_C9/A -all_violators -check_type pulse_width

# format : frame 0 : split 0
**WARN: (TCLCMD-1470):    Pins/ports are only supported with '-drv_violation_type' option

Check type : pulse_width
--------------------------
--------------------------------------------------------------------------------
                          SEQUENTIAL CLOCK PULSE WIDTH

--------------------------------------------------------------------------------
Pin                                Required     Actual Pulse    Slack      View Name
                                  Pulse Width      Width

--------------------------------------------------------------------------------
RM_25_TS_INST/RM_25_INST/CLK (high)  0.061          -0.788       -0.849     func_RCMAX
DGINST/out_reg[4]/CK (low)           2.017          1.514        -0.503     func_RCMAX
DGINST/out_reg[3]/CK (low)           2.017          1.514        -0.503     func_RCMAX
DGINST/out_reg[2]/CK (low)           2.017          1.519        -0.499     func_RCMAX
DGINST/out_reg[6]/CK (low)           2.017          1.519        -0.499     func_RCMAX
```

- The following command reports violations if the connection class attributes of the driving pin and receiving

pin of a net are different:

```
tempus > report_constraint  -connection_class -early -verbose  -view scan_slow_RCMAX

# format : frame 0 : split 0

min_delay/hold
---------------------------
Path 1: VIOLATED (-1.094 ns) Hold Check with Pin TP_CORE/data_out_reg[3]/CK->SI
               View: scan_slow_RCMAX
              Group: m_clk
         Startpoint: (R) scan_in2
              Clock: (R) m_clk
           Endpoint: (R) TP_CORE/data_out_reg[3]/SI
              Clock: (R) m_clk

                       Capture       Launch
         Clock Edge:+    0.000        0.000
        Src Latency:+    0.000        0.000
        Net Latency:+    4.571 (P)    0.000 (I)
            Arrival:=    4.571        0.000

               Hold:+   -0.052
        Uncertainty:+    0.600
      Required Time:=    5.118
       Launch Clock:=    0.000
        Input Delay:+    4.000
          Data Path:+    0.024
              Slack:=   -1.094

#-------------------------------------------------------------
# Pin                       Cell        Delay   Arrival    Edge
#                                       (ns)    (ns)
#-------------------------------------------------------------
scan_in2                   (arrival)     -      4.000       R
TP_CORE/data_out_reg[3]/SI SDFFR_X1    0.024   4.024       R
#-------------------------------------------------------------

Check type : connection_class
---------------------------
    **INFO: No connection_class information found for given parameters
```

- Consider a design having 3 sequential endpoints:
  - setup_pin: Having a setup check with slack equal to -9.000
  - recovery_pin: Having a recovery check with slack equal to -8.000
  - both_setup_recovery_pin: Having both setup check (slack = -6.000) and recovery check (slack = -7.000)
    In this case, the `report_constraint -late -all_violators` command will display the following output:

```
max_delay/setup
-------------------------------------------------
End Point                  Slack          Cause
-------------------------------------------------
setup_pin f                -9.000         VIOLATED
both_setup_recovery_pin    -6.000         VIOLATED
-------------------------------------------------
```

- The `report_constraint -check_type recovery -all_violators` command will display the following output:

```
Check type : recovery
-----------------------------------------------------
End Point                    Slack          Cause
-----------------------------------------------------
recovery_pin r               -8.000         VIOLATED
both_setup_recovery_pin      -7.000         VIOLATED
-----------------------------------------------------
```

- The `report_constraint -late -check_type recovery -all_violators` command displays the following output:

```
max_delay/setup
-----------------------------------------------------
End Point                  Slack           Cause
-----------------------------------------------------
setup_pin f                -9.000          VIOLATED
recovery_pin r             -8.000          VIOLATED
both_setup_recovery_pin    -7.000          VIOLATED
-----------------------------------------------------
```

- The `report_constraint` command displays the following output:

```
max_delay/setup
-------------------------------------------------
End Point             Slack                Cause
-------------------------------------------------
out2 r              -4111.834              VIOLATED
-------------------------------------------------
Check type : clock_period
---------------------------
No paths found
Check type : pulse_width
```

```
-------------------------------------------------------------
                        SEQUENTIAL CLOCK PULSE WIDTH
-------------------------------------------------------------
Pin             Required         Actual Pulse         Slack
                Pulse Width      Width
-------------------------------------------------------------
dff2/CK (low)   0.260            -670.800            -671.060
-------------------------------------------------------------
Check type : max_transition
---------------------------
Pin : sub1/dff1/D
max_transition : 4.500
Transition Time: 5114.060
-----------------------------------------------
slack: -5109.560 (VIOLATED)
Check type : max_capacitance
---------------------------
Pin : buf133/Y
max_capacitance: 0.000
-Capacitance : 0.423
-----------------------------
slack : -0.423 (VIOLATED)
Check type : max_fanout
---------------------------
Pin : specialClkGate/o
max_fanout : 0.557
fanout : 1.000
---------------------------
slack : -0.443 ( VIOLATED )
```

- The `report_constraint -all_violators` command displays the following output:

```
max_delay/setup

-----------------------------------------------------------
End Point               Slack               Cause
-----------------------------------------------------------
out2 r                  -4111.834           VIOLATED
dff3/D r                -4110.260           VIOLATED
lat2/D r                -3774.174           VIOLATED
```

```
dff2/D r                -3770.285          VIOLATED
out r                   -3161.115          VIOLATED
and1/B r                -2464.880          VIOLATED
dff1/D r                -2098.868          VIOLATED
sub2/bufH1/dff1/D r     -2098.821          VIOLATED
sub2/bufH2/dff1/D r     -2028.843          VIOLATED
sub1/dff1/D r           -2028.843          VIOLATED
dff5/D r                -1279.155          VIOLATED
-----------------------------------------------------
Check type : clock_period
---------------------------
No paths found
Check type : skew
---------------------------
No paths found
Check type : pulse_width
-------------------------------------------------------------
                            SEQUENTIAL CLOCK PULSE WIDTH
-------------------------------------------------------------
Pin                       Required      Actual Pulse     Slack
                          Pulse Width   Width
-------------------------------------------------------------
dff2/CK (low)              0.260       -670.800        -671.060
lat2/GN (low)              0.201       -670.800        -671.001
sub2/bufH1/dff1/CK (high)  0.167       -670.800        -670.968
dff1/CK (high)             0.167       -670.800        -670.968
sub2/bufH2/dff1/CK (high)  0.167       -670.800        -670.968
sub1/dff1/CK (high)        0.167       -670.800        -670.968
dff11/CK (high)            0.167       -670.800        -670.968
-------------------------------------------------------------------
Check type : max_transition
-------------------------------------------------------------------
Pin Name                Required       Actual         Slack
-------------------------------------------------------------------
and_parallel_1/A        4.500          5114.060      -5109.560
buf4/A                  4.500          5114.060      -5109.560
sub1/dff1/D             4.500          5114.060      -5109.560
Check type : max_capacitance
```

```
-------------------------------------------------------------
Pin Name                 Required        Actual      Slack
-------------------------------------------------------------
buf133/Y                 0.000          0.423       -0.423
buf11/Y                  0.000          0.105       -0.105
buf5/Y                   0.000          0.105       -0.105
dff11/Q                  0.000          0.105       -0.105
Check type : max_fanout
-------------------------------------------------------------
Pin Name                 Required        Actual      Slack
-------------------------------------------------------------
specialClkGate/o         0.557           1.000       -0.443
```

- The `report_constraint -all_violators -drv_violation_type min_capacitance` command reports the following:

```
-----------------------------------------------
Pin Name    Required     Actual       Slack
-----------------------------------------------
DIN        0.0400        0.0013       -0.0387
F2/Q       0.0400        0.0013       -0.0387
CLK1       0.0400        0.0051       -0.0349
```

- The `report_constraint -drv_violation_type min_capacitance` command outputs the following:

```
Check type : min_capacitance
----------------------------
Pin : F2/Q
Capacitance: 0.0013
- min_capacitance: 0.0400
-----------------------------------------------------
slack: -0.0387 ( VIOLATED )
```

- The `report_constraint -all_violators -drv_violation_type min_transition` command outputs the following:

```
-----------------------------------------------
Pin Name    Required      Actual      Slack
-----------------------------------------------
I0/A       0.0300        0.0001       -0.0299
I1/A       0.0300        0.0001       -0.0299
CLK0       0.0300        0.0001       -0.0299
CLK1       0.0300        0.0001       -0.0299
```

```
DIN         0.0300         0.0001        -0.0299
I4/S0       0.0300         0.0001        -0.0299
F1/DATA     0.0300         0.0001        -0.0299
SEL         0.0300         0.0001        -0.0299
```

- The `report_constraint -drv_violation_type min_transition` command outputs the following:

```
Check type : min_transition
---------------------------
Pin : F1/DATA


Transition Time:0.0001
- min_transition :0.0300
-----------------------------------------------------
slack : -0.0299 ( VIOLATED )
```

- The `report_constraint -all_violators -drv_violation_type min_fanout` command outputs the following:

```
-----------------------------------------------------
Pin Name    Required       Actual        Slack
-----------------------------------------------------
F5/Q        2.0000         0.0000        -2.0000
CLK0        2.0000         1.0000        -1.0000
F3/Q        2.0000         1.0000        -1.0000
F2/Q        2.0000         1.0000        -1.0000
F1/Q        2.0000         1.0000        -1.0000
SEL         2.0000         1.0000        -1.0000
CLK1        2.0000         1.0000        -1.0000
```

- The `report_constraint -drv_violation_type min_fanout` command outputs the following:

```
Check type : min_fanout
---------------------------
Pin : F5/Q
Fanout: 0.0000
- min_fanout: 2.0000
--------------------------------
slack : -2.0000 ( VIOLATED )
```

- The `report_constraints -check_type clock_period` command reports the following:

```
Check type : clock_period
-----------------------------------------------------------
End Point                Slack      Cause
```

```
-----------------------------------------------------------
ffo_ram/mm0/mm1/CLKB f   -0.027     VIOLATED
-----------------------------------------------------------
```

- The following example illustrates how to restrict the number of DRV violations to display for each of the specified DRV checks:

```
report_constraint -drv_violation_type {max_transition max_capacitance} -drv_fields
{max_transition transition_actual transition_slack max_capacitance capacitance_actual
capacitance_slack max_slack net  max_frequency_clock max_clock_frequency pin_type ref_cell
driver_pin pin} -all_violators -max_pins_per_drv 6
############################################################
# format : frame 0 : split 0
# Units: Time (1ns), Capacitance (1pf), Frequency (1MHz).
Check type : max_transition
---------------------------
-------------------------------------------------------------------------------------------------
------------------------
Required     Actual       Transition   Worst     Net    Maximum     Frequency   Pin
Violating    Driver    Pin
Transition   Transition   Slack        Setup           Frequency               Type
 Libcell
                                       Slack           Clock
-------------------------------------------------------------------------------------------------
------------------------
0.002        0.120        -0.118       NA        clk1   CK1         100.000     CLOCK
DFFRX2       clk1      ff1/CK
0.002        0.120        -0.118       NA        clk2   CK2         200.000     CLOCK
DFFRX2       clk2      ff2/CK
0.002        0.059        -0.057       0.894     n4     CK1         100.000     DATA    BUFX1
      buf2/Y    buf2/Y
0.002        0.059        -0.057       4.362     n4     CK1         100.000     DATA
 DFFRX2       buf2/Y    ff2/D
0.002        0.059        -0.057       0.894     n4     CK1         100.000     DATA    BUFX1
      buf2/Y    buf3/A
0.002        0.045        -0.043       3.840     n1     CK2         200.000     DATA    BUFX1
      buf0/Y    buf1/A

Check type : max_capacitance
---------------------------
-------------------------------------------------------------------------------------------------
------------------------
Required     Actual       Capacitance  Worst     Net    Maximum     Frequency   Pin
Violating    Driver    Pin
Capacitance  Capacitance  Slack        Setup           Frequency               Type
 Libcell
                                       Slack           Clock
-------------------------------------------------------------------------------------------------
------------------------
0.002        0.004        -0.002       0.894     n4     CK1         100.000     DATA    BUFX1
```

```
      buf2/Y    buf2/Y
0.002         0.002         -0.000         3.840     n1      CK2          200.000      DATA     BUFX1
      buf0/Y    buf0/Y
0.002         0.002         -0.000         3.840     n2      CK2          200.000      DATA     BUFX1
      buf1/Y    buf1/Y
0.002         0.002         -0.000         0.894     n3      CK1          100.000      DATA
 DFFRX2       ff1/Q     ff1/Q
```

- The following examples shows the pin-wise DRV report without the worst DRV per net:

```
set timing_report_constraint_format "max_transition transition_actual transition_slack
max_capacitance capacitance_actual capacitance_slack max_slack net  max_frequency_clock
max_clock_frequency pin_type ref_cell driver_pin pin"


report_constraint -drv_violation_type {max_transition max_capacitance} -all_violators
############################################################
# format : frame 0 : split 0
# Units: Time (1ns), Capacitance (1pf), Frequency (1MHz).
Check type : max_transition
---------------------------
-------------------------------------------------------------------------------------------------
---------------------
Required     Actual      Transition   Worst      Net    Maximum    Frequency   Pin
Violating   Driver    Pin
Transition   Transition  Slack        Setup             Frequency              Type    Libcell
                                      Slack             Clock
-------------------------------------------------------------------------------------------------
----------------------
0.002        0.120       -0.118       NA         clk1   CK1        100.000     CLOCK   DFFRX2
    clk1      ff1/CK
0.002        0.120       -0.118       NA         clk2   CK2        200.000     CLOCK   DFFRX2
    clk2      ff2/CK
0.002        0.059       -0.057       0.894      n4     CK1        100.000     DATA    BUFX1
     buf2/Y    buf2/Y
0.002        0.059       -0.057       4.362      n4     CK1        100.000     DATA    DFFRX2
    buf2/Y    ff2/D
0.002        0.059       -0.057       0.894      n4     CK1        100.000     DATA    BUFX1
     buf2/Y    buf3/A
0.002        0.045       -0.043       3.840      n1     CK2        200.000     DATA    BUFX1
     buf0/Y    buf1/A
0.002        0.045       -0.043       3.840      n1     CK2        200.000     DATA    BUFX1
     buf0/Y    buf0/Y
0.002        0.044       -0.042       3.840      n2     CK2        200.000     DATA    BUFX1
     buf1/Y    buf1/Y
0.002        0.044       -0.042       3.840      n2     CK2        200.000     DATA    DFFRX2
    buf1/Y    ff1/D
0.002        0.038       -0.036       0.894      n3     CK1        100.000     DATA    BUFX1
     ff1/Q     buf2/A
0.002        0.038       -0.036       0.894      n3     CK1        100.000     DATA    DFFRX2
```

```
    ff1/Q     ff1/Q
0.002        0.031       -0.029       1.208      out2   CK2        200.000     DATA    DFFRX2
    ff2/Q     ff2/Q
0.002        0.030       -0.028       0.894      out1   CK1        100.000     DATA    BUFX1
     buf3/Y    buf3/Y

Check type : max_capacitance
---------------------------
-------------------------------------------------------------------------------------------
---------
Required    Actual      Capacitance  Worst   Net   Maximum   Frequency  Pin    Violating
Driver   Pin
Capacitance Capacitance Slack        Setup         Frequency            Type   Libcell
                                     Slack         Clock
-------------------------------------------------------------------------------------------
----------
0.002        0.004       -0.002       0.894   n4    CK1       100.000    DATA   BUFX1
 buf2/Y   buf2/Y
0.002        0.002       -0.000       3.840   n1    CK2       200.000    DATA   BUFX1
 buf0/Y   buf0/Y
0.002        0.002       -0.000       3.840   n2    CK2       200.000    DATA   BUFX1
 buf1/Y   buf1/Y
0.002        0.002       -0.000       0.894   n3    CK1       100.000    DATA   DFFRX2
ff1/Q    ff1/Q
```

- The following command example shows pins per net with the worst DRV slack:

```
report_constraint -drv_violation_type {max_transition max_capacitance} -all_violators -
worst_drv_per_net -max_pins_per_drv 5

################################################
# format : frame 0 : split 0
# Units: Time (1ns), Capacitance (1pf), Frequency (1MHz).
Check type : max_transition
---------------------------
-------------------------------------------------------------------------------------------
--------------------
Required     Actual      Transition   Worst     Net    Maximum     Frequency   Pin
Violating   Driver   Pin
Transition   Transition  Slack        Setup           Frequency               Type    Libcell
                                      Slack           Clock
-------------------------------------------------------------------------------------------
--------------------
0.002        0.120       -0.118       NA        clk1   CK1         100.000     CLOCK   DFFRX2
    clk1     ff1/CK
0.002        0.120       -0.118       NA        clk2   CK2         200.000     CLOCK   DFFRX2
    clk2     ff2/CK
0.002        0.059       -0.057       0.894     n4     CK1         100.000     DATA    BUFX1
     buf2/Y   buf2/Y
0.002        0.045       -0.043       3.840     n1     CK2         200.000     DATA    BUFX1
     buf0/Y   buf1/A
```

```
0.002        0.044       -0.042       3.840     n2     CK2          200.000     DATA    BUFX1
     buf1/Y   buf1/Y

Check type : max_capacitance

---------------------------
-------------------------------------------------------------------------------------------
-----------------
Required     Actual      Capacitance   Worst    Net   Maximum    Frequency   Pin    Violating
 Driver   Pin
Capacitance  Capacitance Slack         Setup          Frequency              Type   Libcell
                                       Slack          Clock
-------------------------------------------------------------------------------------------
-----------------
0.002        0.004       -0.002        0.894    n4    CK1        100.000     DATA   BUFX1
 buf2/Y   buf2/Y
0.002        0.002       -0.000        3.840    n1    CK2        200.000     DATA   BUFX1
 buf0/Y   buf0/Y
0.002        0.002       -0.000        3.840    n2    CK2        200.000     DATA   BUFX1
 buf1/Y   buf1/Y
```

- The following command reports the hierarchical nets associated with the violating pins:

```
tempus > report_constraint -drv_violation_type max_transition -drv_fields {pin net hnet}

Check type : max_transition

---------------------------

     +----------------------------------------+

     | Pin         | Net         | Hierarchical |

     |             |             | Net          |

     |-------------+-------------+--------------|

     | data_bidi2 | data_bidi2 | data_bidi2   |

     +----------------------------------------+
```

Related Commands

- set_min_fanout
- set_min_fanout
- set_min_transition
- set_max_transition
- set_min_capacitance
- set_max_capacitance
- report_timing

report_cppr

```
report_cppr
[-help]
[-check_type {setup | hold | clock_gating_setup | clock_gating_hold | clock_gating_pulse_width |
data_setup | data_hold | recovery | removal | pulse_width | clock_period | clock_separation | skew
| no_change_setup | no_change_hold | time_borrow}]
-from pin_or_port
-to pin_or_port
[-from_clock clkname]
[-to_clock clkname]
[-early | -late]
[-view view_name]
[> filename]
```

Reports the clock reconvergence pessimism value at the common branching point of the early and late paths in the clock network of the specified data path.

The `report_cppr` command reports separate values for both opening and closing edges for paths with latches as destinations. If both source and destination flops are triggered by multiple clocks, then one report is generated for each separate clock.

When the software is in multi-mode multi-corner analysis mode, `report_cppr` reports the results for each active analysis view.

To use the `report_cppr` command, you must first set the following attributes:

- `timing_cppr_transition_sense`
- `timing_cppr_threshold_ps`

For more details, see timing Category Attributes.

Parameters

`-help`	Prints out the command usage.

`-check_type {setup | hold | clock_gating_setup | clock_gating_hold | clock_gating_pulse_width | data_setup | data_hold | recovery | removal | pulse_width | clock_period | clock_separation | skew | no_change_setup | no_change_hold | time_borrow}`	Reports the CPPR for the paths that end at the specified timing check. The following check types are reported when the software is in setup analysis mode: `setup`, `pulse_width`, `clock_period`, `clock_gating_setup`, `clock_gating_pulse_width`, `data_setup`, `recovery`, skew, `clock_separation`, `time_borrow`, and `no_change_setup`. The following check types are reported when the software is in hold analysis mode: `hold`, `clock_gating_hold`, `data_hold`, `removal`, and `no_change_hold`. If the software is in simultaneous setup and hold analysis mode, all of the check types listed above are reported.
`> filename`	Writes the report to the specified file name. If the file already exists, the software overwrites it. The *`filename`* parameter must be the last argument in the list. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-early`	Reports the hold analysis clock re-convergence pessimism value. You can specify this parameter when the software is in hold analysis mode, or in simultaneous setup and hold analysis mode (`set_db timing_enable_simultaneous_setup_hold_mode true`). For more details, see timing Category Attributes.
`-from` *`pin_or_port`*	Reports the clock re-convergence pessimism value for paths starting from the specified clock pin of the source flop, or from the specified port.
`-from_clock` *`clkname`*	Reports the clock re-convergence pessimism value for paths with the specified triggering clock. Use this parameter when there is more than one clock triggering the source flop.

`-late`	Reports the setup analysis clock re-convergence pessimism value. You can specify this parameter when the software is in setup analysis mode, or in simultaneous setup and hold analysis mode (`set_db timing_enable_simultaneous_setup_hold_mode true`). For more details, see timing Category Attributes.
`-to` *pin_or_port*	Reports the clock re-convergence pessimism value for paths ending at the specified clock pin of the destination flop, or at the specified port.
`-to_clock` *clkname*	Reports the clock re-convergence pessimism value for paths with the specified capturing clock. Use this parameter when there is more than one clock capturing the destination flop.
`-view` *view_name*	Reports the clock re-convergence pessimism values for paths in the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

- The following command reports the clock re-convergence pessimism value for paths starting from the specified clock pin and ending at the specified destination pin:

```
tempus > report_cppr   -from clk   -to out2


Start Clock Pin: clk (Triggered by rising  edge of clock CLK)
End Clock Pin: out2 (Triggered by rising  edge of clock CLK)
Analysis Type: Setup
Analysis View: AV_wc_on
Common Branch Point : CLK
Launching Edge Type At Common Point : Rising
Capturing Edge Type At Common Point : Rising
Pessimism Threshold Value :    0.040
      -------------------------------------
       Arrival    Late   Early   Pessimism
         Times
      -------------------------------------
```

```
      Rise     0.000   0.000     0.000
      Fall     0.000   0.000     0.000
      -------------------------------------
Edge Selection Mode : Normal
Edge Selection : Match, Using Rise
CPPR Adjustment :   0.000
```

- The following command reports reports the hold analysis clock re-convergence pessimism value:

```
tempus > report_cppr   -from clk   -to out2   -early

Start Clock Pin: clk (Triggered by rising  edge of clock CLK)
End Clock Pin: out2 (Triggered by rising  edge of clock CLK)
Analysis Type: Hold
Analysis View: AV_wc_on
Common Branch Point : CLK
Launching Edge Type At Common Point : Rising
Capturing Edge Type At Common Point : Rising
Pessimism Threshold Value :    0.040
      -------------------------------------
       Arrival    Late   Early   Pessimism
        Times
      -------------------------------------
      Rise     0.000   0.000     0.000
      Fall     0.000   0.000     0.000
      -------------------------------------
Edge Selection Mode : Normal
Edge Selection : Match, Using Rise
CPPR Adjustment :   0.000
```

- The following command reports the setup analysis clock re-convergence pessimism value:

```
tempus > report_cppr   -from clk   -to out2   -late

Start Clock Pin: clk (Triggered by rising  edge of clock CLK)
End Clock Pin: out2 (Triggered by rising  edge of clock CLK)
Analysis Type: Setup
Analysis View: AV_wc_on
Common Branch Point : CLK
Launching Edge Type At Common Point : Rising
Capturing Edge Type At Common Point : Rising
Pessimism Threshold Value :    0.040
```

```
      -------------------------------------
       Arrival    Late   Early   Pessimism
        Times
      -------------------------------------
      Rise     0.000   0.000     0.000
      Fall     0.000   0.000     0.000
      -------------------------------------
Edge Selection Mode : Normal
Edge Selection : Match, Using Rise
CPPR Adjustment :   0.000
```

- When the software is in multi-mode multi-corner timing analysis mode, you can specify the `-view` parameter to report the CPPR values for a specific analysis view:

```
tempus > report_cppr -from CLK3 -to seg3/u14/CK

Start Clock Pin: CLK3 (Triggered by rising edge of clock CLK_W_3)
End Clock Pin: seg3/u14/CK (Triggered by rising edge of clock CLK_W_3)
Analysis Type: Setup
Analysis View: setup_view1
Common Branch Point : seg3/u17_c/Y
Launching Edge Type At Common Point : Rising
Capturing Edge Type At Common Point : Rising
Pessimism Threshold Value : 0.020
-------------------------------------
Arrival   Late    Early   Pessimism
Times
-------------------------------------
Rise      0.549   0.366   0.183
Fall      0.400   0.267   0.133
-------------------------------------
Edge Selection Mode : Normal
Edge Selection : Match, Using Rise
CPPR Adjustment : 0.183
```

- The following report shows sigma and mean values:

```
tempus > report_cppr -from FF01/CP -to FF02/CP


Start Clock Pin: FF01/CP (Triggered by rising edge of clock CK)
End Clock Pin: FF02/CP (Triggered by rising edge of clock CK)
Analysis Type: Setup
Analysis View: view1
Common Branch Point : CK
Launching Edge Type At Common Point : Rising
Capturing Edge Type At Common Point : Rising
```

```
Pessimism Threshold Value : 0.020
+-------------------------------------------------------------------------------+
| Arrival | Late  | Late  | Early | Early | Pessimism | Pessimism | Pessimism |
| Times   | Mean  | Sigma | Mean  | Sigma | Mean      | Sigma     |           |
|---------+-------+-------+-------+-------+-----------+-----------+-----------|
| Rise    | 0.000 | 0.000 | 0.000 | 0.000 | 0.000     | 0.000     | 0.000     |
| Fall    | 0.000 | 0.000 | 0.000 | 0.000 | 0.000     | 0.000     | 0.000     |
+-------------------------------------------------------------------------------+
Edge Selection Mode : Normal
Edge Selection : Match, Using Rise
CPPR Adjustment : 0.000 (+3.00S) | 0.000 | 0.00
```

Related Commands and Global Variables

- `timing_cppr_transition_sense`
- `timing_cppr_threshold_ps`
- `timing_enable_simultaneous_setup_hold_mode`

report_critical_insts

```
report_critical_insts
[-help]
[-cost_type {worst_slack | total_slack | path_count | start_end_point_count}]
[-estimate]
[-include_sequential]
[-max_insts value]
[-max_slack slack]
[-out_file string]
[>> file ]
```

Reports instances in critical paths where the slack is less than the target slack.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_critical_insts`parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_critical_insts` .
`-cost_type`	Specifies the criteria for choosing the critical instance. You can use the `-max_slack` and `-cost_type` parameters together to filter the paths to only those with slack worse than the specified *`slack`* value, then report the filtered set of paths based on the specified cost. *Default:* `worst_slack` Select one of the following options: `worst_slack`: Sets the criteria as the worst slack of all points that pass through an instance. `total_slack`: Sets the criteria as a sum of the slack of the end points. `path_count`: Sets the criteria as the sum of the total paths through the instance. `start_end_point_count`: Sets the criteria as the sum of the total start and end points connected to the instance.
`-estimate`	Generates a critical instance report based on the fast algorithm (linear-complexity) when cost type =`total_slack` to give a slightly pessimistic estimation that may include some false paths. Use this parameter in early design phase. *Default:* `false`

file	Writes the report to the specified file name. If the file already exists, the software overwrites it. **Note:** The file name must be the last argument in the list. *Default:* Displays the results in Tempus console window without being saved.
-include_sequential	Includes the sequential instances group in the output. *Default:* false (Reports combinatorial instances only.)
-max_insts *value*	Specifies the number of worst cost instances to report. *Default:* 20
-max_slack *slack*	Specifies the lower bound of the slack range. Only paths with slack worse than the specified *slack* value are reported. You can use the -max_slack and -cost_type parameters together to filter the paths to only those with slack worse than the specified *slack* value, then report the filtered set of paths based on the specified cost. *Default:* 0
-out_file *string*	Writes the report into the specified output file. Default: stdout

Command Order

Use this command after the timing graph is built by time_design/opt_design .

Examples

- The following command reports the critical instance information including sequential instances group for 5 worst cost instances:

```
tempus> report_critical_instance -sequential_insts -name_length 42 -max_insts 5
```

  The following report is generated:

```
# ----------------------------------------
# Inst:combinatorial  Cell        Cost
# ----------------------------------------
p0042A82620           CKNHVTD4   -0.124
p0009A84488           CKND2HVTD2 -0.124
p0009A84792           IND2D1     -0.124
p0012A85164           CKNXHVTD1  -0.124
# ----------------------------------------
# Inst:sequential     Cell              Cost
# ----------------------------------------
dcache_/ram256        vl1rwb_wsz10_hz8 -0.124
dcache_data3/ram10    vl1rwb_wsz32_hz8 -0.120
dcache_tagram2/ram256 vl1rwb_wsz24_hd8 -0.120
# ----------------------------------------
```

- The following command reports the critical instance information based on the worst slack of all points that pass through an instance:

```
tempus> report_critical_instance -cost_type worst_slack
# ----------------------------------------
# Inst:combinatorial  Cell        Cost
# ----------------------------------------
I3                    BUFX2       -0.336
I10                   AND2X1      -0.336
I1                    BUFX2       -0.117
I2                    BUFX2       -0.117
----------------------------------------
```

- The following command reports the critical instance information based on the worst slack of all points that pass through an instance, for paths with slack worse than -0.2:

```
tempus> report_critical_instance -cost_type worst_slack -max_slack -0.2
# ------------------------------------------------
# Inst:combinatorial    Cell          Cost
# ------------------------------------------------
I3                      BUFX2       -0.336
I10                     AND2X1      -0.336
------------------------------------------------
```

- The following command reports the critical instance information based on the sum of the slack of the end points:

```
tempus> report_critical_instance -cost_type total_slack
# ------------------------------------------------------
# Inst:combinatorial    Cell       Cost    Start  End
# ------------------------------------------------------
I3                      BUFX2     -0.336    1     1
I10                     AND2X1    -0.336    2     1
I1                      BUFX2     -0.117    0     1
I2                      BUFX2     -0.117    0     1
------------------------------------------------------
```

- The following command reports the critical instance information based on the sum of the total paths through the instance:

```
tempus> report_critical_instance -cost_type path_count
# ------------------------------------------------
# Inst:combinatorial   Cell       Cost    Start  End
# ------------------------------------------------
I10                    AND2X1      4       2     1
I3                     BUFX2       2       1     1
I1                     BUFX2       2       0     1
I2                     BUFX2       2       0     1
------------------------------------------------
```

- The following command reports the critical instance information based on the sum of the total start and end points connected to the instance:

```
tempus> report_critical_instance -cost_type start_end_point_count
# ------------------------------------------------------
# Inst:combinatorial    Cell       Cost    Start  End
# ------------------------------------------------------
I10                     AND2X1      2       1     1
I3                      BUFX2       2       1     1
I1                      BUFX2       2       1     1
I2                      BUFX2       2       1     1
------------------------------------------------------
```

- The following command reports the critical instance information based on the sum of the total start and end points connected to the instance, for paths with slack worse than -0.2:

```
tempus> report_critical_instance -cost_type start_end_point_count -max_slew -0.2
# ------------------------------------------------
# Inst:combinatorial    Cell       Cost    Start  End
```

```
# ------------------------------------------------------
I10                     AND2X1      2       1     1
I3                      BUFX2       2       1     1
------------------------------------------------------
```

Related Commands

- report_timing

report_critical_nets

```
report_critical_nets
[-help]
[-max_slack double]
[-number integer]
[-view viewName]
-out_file fileName
```

Generates a file containing a list of nets which have critical slack for the currently specified (setup or hold) timing analysis mode.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_critical_nets` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_critical_nets`
`-max_slack double`	Reports nets with slack less than the specified slack value.
`-number integer`	Restricts the number of critical nets to be reported to the first *integer* - most critical nets.
`-out_file fileName`	Specifies the name of report file.
`-view viewName`	Generates the report based on the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Command Order

Use this command after the timing graph is built.

Examples

- The following command reports nets with slack less than the specified slack value:

```
tempus > report_critical_nets -max_slack -1
Marking critical nets with target slack = -0.1ps.
Critical nets number = 7144.
```

- The following command generates a maximum slack report for the specified number of critical nets:

```
tempus > report_critical_nets -max_slack -1 -number 5
Marking critical nets with target slack = -0.1ps.
Critical nets number = 5.
```

- The following command generates a report named `critnet` that lists critical nets:

```
report_critical_nets -out_file critnet
```

report_critical_pins

report_critical_pins
`[-help]`
`-out_file` *`fileName`*

Performs slack analysis and writes the names of all critical terminals to the specified file. As a result, deterioration of setup slack can be avoided while the Innovus software repairs hold time violations.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_critical_pins` parameter. For a detailed description of the command and all of its parameters, use the man command:`man report_critical_pins`
`-out_file` *`fileName`*	Specifies the name of report file.

Command Order

Use this command after the timing graph is built by `time_design/opt_design` .

Examples

- The following attribute and command write the setup critical terminals into a file called `critTerm` :

```
timing_analysis_check_type setup
report_critical_pins -out_file critTerm
```

report_design

```
report_design
[-help]
[-view viewName]
[{> | >>} filename | -tcl_list]
[-early | -late]
```

Reports the minimum and/or maximum operating conditions and active design rules for the current design.

Both minimum and maximum operating conditions are available for on-chip variation analysis mode.

When simultaneous setup hold analysis mode is set to `on`, the minimum or maximum conditions can be reported using the `-early`/`-late` parameters.

Parameters

`-help`	Prints out the command usage.
`{> | >>} filename`	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. The *`filename`* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.

`-early	-late`	The `-early` parameter reports minimum operating condition in `best_case_worst_case` or `ocv` analysis mode (`timing_analysis_type` attribute). The `-late` parameter reports maximum operating condition. The `-late` and `-early` parameters are mutually exclusive.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. **Note:** The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify these parameters together.
`-view` *`viewName`*	Reports view specific operating conditions.

Examples

- The following command reports the operating conditions related to the view, `scan_slow_RCMAX`, along with the design rules:

```
report_design -view scan_slow_RCMAX


View : scan_slow_RCMAX
-------------------------------------------------------------
    Design Property             Value
-------------------------------------------------------------
    Design Name                 dtmf_recvr_core
-------------------------------------------------------------
                                MAX               MIN
    Operating Condition Name    TypTyp_0.8V_25C   TypTyp_0.8V_25C
        Process                 1.000             1.000
        Voltage                 0.800             0.800
        Temperature             25.000            25.000
        Tree Type               balanced          balanced
 -------------------------------------------------------------

      -------------------------
         Design Rule     Value
      -------------------------
       Max Transition       NA
       Min Transition       NA
       Max Capacitance      NA
       Min Capacitance      NA
       Max Fanout           NA
       Min Fanout           NA
      -------------------------
```

Related Commands

- `report_timing`
- `write_db`

report_fanin

```
report_fanin
-to pin_port_net_list
[-trace_through {case_disable | user_disable | all}]
[-pin_levels numOfPinLevels]
[-view viewName]
[-nosplit]
[{> | >> } filename]
```

Allows a cone traversal that is not tied to the timing graph, that is, not blocked by `set_disable_timing` and case analysis. By default, the trace matches the timing view and the reports generated using the `all_fanin` and `all_fanout` commands. The reports indicate whether disabled or case analysis arcs were traversed.

Parameters

`> | >> filename`	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file.
`-nosplit`	Specifies that reports with long names will not be split into multiple lines.
`-pin_levels numOfPinLevels`	Specifies the desired depth of the output.
`-trace_through {case_disable | user_disable | all}`	Allows traversal through disabled arcs. You can specify one of the following options: • `case_disable`: Arcs which are disabled because of set_case_analysis are traversed but because of set_disable_timing are not traversed • `user_disable`: Arcs disabled because of set_disable_timing are traversed but because of set_case_analysis are not traversed • `all`: Trace through all arcs
`-to pin_port_net_list`	Specifies a list or collection of pins, ports, or nets. A mixed list or heterogeneous collection is also allowed.

`-view` *viewName*	Specifies disabled arcs in the specified view.

Examples

- The following command generates a report of the specified collection of pins along with the depth of the output for the specified view:

```
tempus > report_fanin  -to  FE_C674_ts_4  -pin_levels 3  -view  scan_fast_RCMAX

Fanin Network for View: scan_fast_RCMAX
Fanin Network for Pin: FE_C674_ts_4/A
--------------------------------------------------------------------------------------------------
-
Source Pin              Sink Pin                Cell Type   Timing Sense   Arc        Case

                                                                           Disabled   Analysis


--------------------------------------------------------------------------------------------------
-
RE_INST/FE_O6_t_4/Z     FE_C674_ts_4            (net)       -               -         -
--------------------------------------------------------------------------------------------------
-
RE_INST/FE_OFC106_4/A   RE_INST/FE_O6_t_4/Z     BUF_X8      positive_unate  -          -
--------------------------------------------------------------------------------------------------
-
RE_INST/g6253/Z         RE_INST/FE_OFC106_4/A   (net)
--------------------------------------------------------------------------------------------------
-
```

- The `report_fanin -to h0/A` command shows the following output:

```
tempus > report_fanin -to h0/A
Fanin Network for Pin: h0/A
-------------------------------------------------------------------
Source Pin   Sink Pin  Cell Type   Timing Sense    Arc       Case
                                                   Disabled  Analysis
-------------------------------------------------------------------
u1/Y         h0/A      (net)       -                -         -
-------------------------------------------------------------------
u1/A         u1/Y      BUFX2       positive_unate   -         -
-------------------------------------------------------------------
in           u1/A      (net)       -                -         -
-------------------------------------------------------------------
```

Related Commands

- set_disable_timing
- set_case_analysis
- all_fanin
- all_fanout

report_fanout

```
report_fanout
{-from pin_port_net_list | -clock_tree}
[-trace_through {case_disable | user_disable | all}]
[-pin_levels numOfPinLevels]
[-view viewName]
[-nosplit]
[{> | >> } filename]
```

Allows a cone traversal that is not tied to the timing graph, that is, not blocked by `set_disable_timing` and case analysis. By default, the trace matches the timing view and the reports generated using the `all_fanin` and `all_fanout` commands. The reports indicate whether disabled or case analysis arcs were traversed.

Parameters

`> | >> filename`	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file.
`-clock_tree`	Reports the fanout of the source pin, one after the other for all the clocks in the design.
`-nosplit`	Specifies that reports with long names will not be split into multiple lines.
`-pin_levels numOfPinLevels`	Specifies the desired depth of the output.
`-trace_through {case_disable | user_disable | all}`	Allows traversal through disabled arcs. You can specify one of the following options: • `case_disable`: Arcs which are disabled because of `set_case_analysis` are traversed but because of set_disable_timing are not traversed. • `user_disable`: Arcs disabled because of set_disable_timing are traversed but because of set_case_analysis are not traversed. • `all`: Trace through all the arcs.

`-from` *pin_port_net_list*	Specifies a list or collection of pins, ports, or nets. A mixed list or heterogeneous collection is also allowed.
`-view` *viewName*	Specifies disabled arcs in the specified view.

Examples

- The following command reports details of a collection of pins at the desired depth of pin levels for the specified view:

```
tempus > report_fanout -from FE_C674_ts_4/A -pin_levels 4 -view scan_fast_RCMIN

Fanout Network for View: scan_fast_RCMIN
Fanout Network for Pin: FE_C674_ts_4/A
-------------------------------------------------------------------------------
Source Pin       Sink Pin           Cell Type  Timing Sense      Arc        Case
                                                                Disabled   Analysis
-------------------------------------------------------------------------------
FE_C674_ts_4/A   FE_C674_ts_4/Z     BUF_X32    positive_unate    -          -
-------------------------------------------------------------------------------
FE_C674_ts_4/Z   MX_INST/p3A/B     (net)         -               -          -
FE_C674_ts_4/Z   FE_O402_ts_4/A    (net)         -               -          -
FE_C674_ts_4/Z   CS_IST/p21A/A3    (net)         -               -          -
-------------------------------------------------------------------------------
MX_INST/p3A/B    MX_INST/p3A/Z     MUX2_X2     positive_unate    -          -
FE_O402_ts_4/A   FE_O402_ts_4/Z    BUF_X16     positive_unate    -          -
CS_IST/p21A/A3   TSP_INST/p2A/ZN   NOR4_X1     negative_unate    -          -
-------------------------------------------------------------------------------
MX_INST/p3A/Z    FE_OFC105_n_94/A  (net)       -                 -          -
FE_O402_ts_4/Z   M_128x16_ST/A[4]  (net)       -                 -          -
TSP_INST/p2A/ZN  TST_CL/p3/A       (net)       -                 -
-------------------------------------------------------------------------------
```

- The following command reports details of a collection of pins at the desired depth of pin levels for disabled arcs:

```
tempus > report_fanout -from FE_C674_ts_4/A -pin_levels 2 -trace_through case_disable

Fanout Network for View: func_slow_RCMAX
Fanout Network for Pin: FE_C674_ts_4/A
-------------------------------------------------------------------------------
Source Pin       Sink Pin         Cell Type   Timing Sense      Arc        Case
                                                               Disabled   Analysis
-------------------------------------------------------------------------------
FE_C674_ts_4/A   FE_C674_ts_4/Z   BUF_X32     positive_unate    -          -
-------------------------------------------------------------------------------
FE_C674_ts_4/Z   MX_INST/p3A/B   (net)           -             -          -
```

```
FE_C674_ts_4/Z   FE_O402_ts_4/A  (net)            -             -          -
FE_C674_ts_4/Z   CS_IST/p21A/A3  (net)            -             -          -
-------------------------------------------------------------------------------
```

- Suppose arc seg1/u1 is disabled due to following constraint:
  `tempus> set_disable_timing -from A -to Y seg1/u1`
  `tempus> report_fanout -from IN1` (command will show the following output):

```
Fanout Network for Port: IN1
-------------------------------------------------------------------------
Source Pin  Sink Pin    Cell Type   Timing Sense   Arc       Case
                                                   Disabled  Analysis
-------------------------------------------------------------------------
IN1         seg1/u1/A  (net)        -              -         -
-------------------------------------------------------------------------
```

- The following command will show the output below:
  `tempus> report_fanout -from IN1 -trace_through user_disable`

```
Fanout Network for Port: IN1
-------------------------------------------------------------------------
Source Pin    Sink Pin   Cell Type  Timing Sense    Arc       Case
                                                    Disabled  Analysis
-------------------------------------------------------------------------
IN1           seg1/u1/A  (net)      -               -         -
-------------------------------------------------------------------------
seg1/u1/A     seg1/u1/Y  CLKBUF     positive_unate  Y         -
-------------------------------------------------------------------------
seg1/u1/Y     seg1/u2/A  (net)      -               -         -
-------------------------------------------------------------------------
```

Related Commands

- set_disable_timing
- set_case_analysis
- all_fanin
- all_fanout

report_freq_violation

```
report_freq_violation
[-help]
[-detailed]
[-em_type {avg | peak | rms | all}]
[-excinstfile string]
[-include_insts_file string]
[-outfile string]
[-reportformat integer]
[-selinstfile string]
[-slew{min | max}]
[-sort {name | freq | minDiff | maxDiff | minFreq | maxFreq}]
[-tablescale double]
[-view string]
```

Runs electromigration (EM) analysis based on the frequency table defined in the Liberty library and generates a frequency violation report.

You must run the following commands prior to setting the `report_freq_violation` command:

- `write_tcf` *activityFile*
- `read_activity_file -format TCF` *activityFile*
- `propagate_activity`
- `read_activity_file -set_net_freq true -format TCF activityFile`
- `set_analysis_view -setup default_view_setup -hold default_view_hold`
- `set_db timing_analysis_check_type hold`

Note: If activity files are not defined, then the frequency can also be calculated on a pin by the clock period. There will also be a default frequency of 500M (toggle count 1e+9) if there is no clock phase on this pin.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_freq_violation` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_freq_violation`.
`-detailed`	Generates a detailed report containing information for instances with violations.
`-em_type {avg | peak | rms | all}`	Specify the EM type. *Default*: `avg`

`-excinstfile <string>`	Specifies an instance file that lists the names of instances to be excluded from the frequency violation report. **Note:** The instance file is a text file that contains a list of instances. Each instance name appears in one line in the file. The comments begin with "#". Leading or ending spaces in instance names are ignored.
`-include_insts_file string`	Specifies a file that lists the names of instances to be included in the frequency violation report. **Note:** The instance file is a text file that contains a list of instances. Each instance name appears in one line in the file. The comments begin with "#". Leading or ending spaces in instance names are ignored.
`-outfile fileName`	Specifies the report file for the violation data.
`-reportformat integer`	Specifies a format for the violation report. The report details vary according to the selected report format. You can specify report formats 0, 1, or 2. For more information on the report formats, see Report Format section below. *Default*: 0
`-selinstfile string`	Specifies the selected instance file name.
`-slew {min | max}`	Specifies the minimum or maximum slew value. *Default*: *min*
`-sort {name | freq | minDiff | maxDiff | minFreq | maxFreq}`	Sorts the instance-based frequency data by name, freq, minDiff, maxDiff, minFreq, and maxFreq. By default, the violation report is not sorted. You can use one of the following options: ◦ `name`: Name of a pin or arc. ◦ `freq`: Frequency value of a pin or arc. ◦ `minDiff`: Difference between freq and minFreq of a pin or arc. ◦ `maxDiff`: Difference between freq and maxFreq of a pin or arc. ◦ `minFreq`: Minimum frequency value of a pin or arc as defined in the frequency table. ◦ `maxFreq`: Maximum frequency value of a pin or arc as defined in the frequency table.
`-tablescale double`	Specifies the scale factor for EM table value. *Default*: `1.0`
`-view viewName`	Specifies the view for which to generate the frequency violation report. Use this parameter when working with multi-mode multi-corner (MMMC) designs.

Report Format

The report formats are described below.

- Format `0`
  The following command generates a report providing details of `minFreq`, `Freq`, `Tran`, `Cap`, `Remark`, and `Arc/Pin` and directs the output to report file `summaryViol.rpt`:

  ```
  report_freq_violation -outfile summaryViol.rpt -reportformat 0
  ```

  For example:

  ```
  minFreq    Freq    Tran     Cap         Remark    Arc/Pin
  5.985      1.009   0.0356   0.003103              tt0/data/r3/add_27/U1_1_6/Y
  ```

- Format `1`
  When `report_freq_violation -reportformat` is set to `1`, the report provides details as shown in the following example:

  ```
  Status   Remark   freq   minFreq   maxFreq   Arc/Pin
  [P]      *        1.009  5.985     5.995     tt0/data/r3/add_27/U1_1_6/Y
  ```

- Format `2`
  A detailed report is generated when `report_freq_violation -reportformat` is set to `2`, as shown in the following example:

  ```
  Status  Remark  freq   minFreq  maxFreq    minTran    maxTran    minCap         maxCap     Arc
  /Pin
  [P]     *       1.009  5.985    5.995      3.560e-02  3.790e-02  3.103e-03      3.103e-
  03  tt0/data/r3/add_27/U1_1_6/Y
  ```

  Where:

`Status`	Arc or pin status - `F` (fail), `V` (violation), `P` (pass), or `?` (unknown)
`Remark`	Indicates whether the pin is a clock
`freq`	Working frequency of the arc or pin
`minFreq`	Minimum frequency value taken from the frequency table
`maxFreq`	Maximum frequency value taken from the frequency table
`minTran`	Minimum slew value

`maxTran`	Maximum slew value
`minCap`	Minimum capacitance load
`maxCap`	Maximum capacitance load
`Arc/Pin`	Name of the arc or pin

Examples

- The following command reports instances that violate frequency constraints and lists them in the `freqViol.rpt` report file:

```
report_freq_violation -outfile freqViol.rpt
```

Related Commands

- `write_tcf`
- `read_activity_file`
- `propagate_activity`
- `set_analysis_view`

report_inactive_arcs

```
report_inactive_arcs
[-help]
[instance_port_obj]
[-exclude_type {const snipped disable library_disable global_disable conditional_disable
library_mode power_mode_disable}]
[-include_net_arcs]
[-include_when_cond]
[-type {const snipped disable library_disable global_disable conditional_disable library_mode
power_mode_disable}]
[-view view_name]
[> <filename[.gz]>]
[>> <filename[.gz]>]
[-delay_arcs_only | -check_arcs_only]
```

Reports information about disabled timing and timing check arcs in the design. Reports all arcs disabled due to user-specified exceptions such as `set_disable_timing` or `set_case_analysis`, as well as information about arcs disabled by constant propagation during timing analysis, snipped loop arcs, and arcs disabled in the timing library.

Parameters

`-help`	Prints out the command usage.
instance_port_obj	Reports disabled timing arcs for the specified list of instances or ports. If you specify a list of instances, the software reports all disabled arcs of the specified cells. If you specify a list of ports, the software reports all disabled arcs directly connected to the ports. *Default:* Reports all disabled net and cell arcs.
`-check_arcs_only`	Lists only disabled check arcs. The `check_arcs` are disabled by using the `set_disable_timing` command. *Default*: Both disabled timing delay arcs and check arcs are listed.
`-delay_arcs_only`	Lists only disabled timing delay arcs.

`-exclude_type {const snipped disable library_disable global_disable conditional_disable library_mode power_mode_disable}`	Excludes the disabled arcs that belong to the specified category of disabled arcs. The valid options are: const, snipped, disable, library_disable, global_disable, conditional_disable, library_mode, and power_mode_disable.
`-include_net_arcs`	Includes disabled net arcs in the report.
`-include_when_cond`	Shows the when_cond of arcs in the output report.
`-type {const snipped disable library_disable global_disable conditional_disable library_mode power_mode_disable}`	Lists only the disabled arcs that belong to the specified category of disabled arcs. The `-type` options restrict the report to only those arcs that are disabled due to a particular timing analysis step, such as constant propagation, specified disables, or loop arc disables. The valid options are: • `const`: Propagated constant -- arcs disabled due to a constant value. The constant value can apply to one of the arc nets/pins, to side inputs which disable a condition for the arc, to nets that disable an active mode for the arc, or disable a default conditional arc. • `snipped`: Loop snipped arcs -- arcs automatically disabled by the Tempus software as part of a feedback loop. • `disable`: Defined by using the `set_disable_timing` command. • `library_disable`: Delay arc disabled due to library disables. These disables may be caused by the following conditions: ◦ Disables specified using the `set_disable_timing` command. ◦ The arc is an asynchronous arc and the attribute `timing_enable_preset_clear_arcs` is set to `false`. • `global_disable`: Lists the arcs that are disabled through user-specified global variable settings.

	• `conditional_disable`: Arcs disabled if a `when` condition is specified. There is no constant on sink or source pin and arc is disabled due to the `when` condition. • `library_mode`: Displays library mode of disabled arcs. • `power_mode_disable`: Allows reporting the arcs for which power mode is disabled.
`-view` *view_name*	Generates the report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.
`{>` | `>>}` *filename*	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.

Examples

- The following command lists disabled check arcs for the specified view:

```
tempus > report_inactive_arcs -check_arcs_only -view sn_fast_RMAX
-------------------------------------------------------------------------------------------
From           To          ArcType           Sense      Reason              View
-------------------------------------------------------------------------------------------
BT/se_0/CK   BT/se_0/D    hold_rising       non_unate  constant disabled   sn_fast_RMAX
BT/se_0/CK   BT/se_0/D    setup_rising      non_unate  constant disabled   sn_fast_RMAX
BT/se_0/RN   BT/se_0/RN   min_pulse_width   non_unate  constant disabled   sn_fast_RMAX
BT/se_0/CK   BT/se_0/RN   removal_rising    non_unate  constant disabled   sn_fast_RMAX
-------------------------------------------------------------------------------------------
```

- The following command reports all disabled arcs in the design:

```
tempus> report_inactive_arcs > Report
```

  The following example shows the report generated:

```
-----------------------------------------------
Flags :   const                propagated constant
          snipped              loop snipped arcs
          disable              set_disable_timing
          disable_clock_gating set_disable_clock_gating
          library_disable      set_disable_cell_timing
          Missing_Phase        No Arc Phase Data
From         To            DisableType ArcType              Reason
-------------------------------------------------------------------------
u8/CK ^      u8/D ^        const       hold_rising_clk_rise  u8/D = 1
u8/CK ^      u8/D v        const       hold_falling_clk_rise u8/D = 1
seg3/u12/A ^ seg3/u12/Y v disable     combinational         User Disable
-------------------------------------------------------------------------
```

- When the software is in multi-mode multi-corner analysis mode, the report generated lists the disabled arcs by active analysis view, as shown in the following example:

```
-----------------------------------------------
Flags : const propagated constant
snipped loop snipped arcs
disable set_disable_timing
disable_clock_gating set_disable_clock_gating
library_disable set_disable_cell_timing
Missing_Phase No Arc Phase Data
From        To                DisableType ArcType  Reason         View
-------------------------------------------------------------------------------

gscanmode ^ FE_O305_gmode/I ^  const       net_arc  gscanmode = 0  setup_view2
gscanmode v FE_O305_gmode/I v  const       net_arc  gscanmode = 0  setup_view2
gscanmode ^ FE_O305_gmode/I ^  const       net_arc  gscanmode = 0  hold_view1
```

```
gscanmode v FE_O305_gmode/I v  const       net_arc  gscanmode = 0  hold_view1
gscanmode ^ FE_O305_gnmode/I ^ const       net_arc  gscanmode = 0  hold_view2
-------------------------------------------------------------------------------
```

Related Commands

- set_disable_timing
- set_case_analysis

report_instance_library

```
report_instance_library
-out_file fileName
[-instance instanceName]
[-view viewName | -delay_corner delayCornerName]
[-power_domain domainName]
[-early | -late]
```

Reports library related information for an instance(s). The report prints the following information about an instance:

- Instance name
- Cell type
- View name or delay corner
- Power domain name
- Library/Libset (early/late)
- Library file (early/late)
- Operating conditions
- Binding Status : Prints out the status for the matched voltage library available. If the user setting is `True`, then the binding status is not displayed.

Parameters

`-delay_corner` *delayCornerName*	Specifies the delay corner from which an instance library is to be selected. If no delay corner is specified, then all the instance libraries will be printed.
`-early` | `-late`	Specifies the library set (early/late) from which a library instance is to be printed. If either of these options are not specified then both the early and late library sets are used.
`-out_file` *fileName*	Specifies the name of the output file. If no file is specified, the report will be printed on standard output file.
`-instance` *instanceName*	Specifies the instance(s) for which the report is to be printed. If no instance is provided the report will print information of all the instances. The value of instance can be a list of regular expressions.

`-power_domain` *domainName*	Specifies the power domain from which instance(s) are to be selected. **Note**: This parameter should be specified with `-delay_corner` or `-view`.
`-view` *viewName*	Specifies the view from which instance library is to be selected.

Examples

- The following command displays the below information about instance `LIN_CONV_INST/F6A` for the specified view:

```
report_instance_library -instance LIN_CONV_INST/F6A -early -view scan_slow_RCMAX

Instance              : LIN_CNV_INST/F6A
Cell                  : INV_X2
Analysis View         : scan_slow_RCMAX
Power Domain          : Default
Library/Libset(early) : F_lib_v1.0/libset_slow
Library File(early)   : /libs/F_lib_v1.0_worst.lib
Binding Status(early) : All PG Pins Voltage Match
Op Cond(early)        : P->1.000000, V->0.800000, T->25.000000 (TypTyp_0.8V_25C)
Cell Type             : gateCell
```

- The following command displays the below information about instance `FE_C674` for the specified delay corner:

```
report_instance_library -delay_corner delay_corner_slow_RCMAX -late

Instance              : FE_C674
Cell                  : BUF_X32
Delay Corner          : delay_corner_slow_RCMAX
PowerDomain           : Default
Library/Libset(late)  : F_lib_v1.0/libset_fast
Library File(late)    : /libs/F_lib_v1.0_typic
Binding Status(late)  : All PG Pins Voltage Match
Op Cond(late)         : P->1.000000, V->0.800000, T->25.000000 (TypTyp_0.8V_25C)
Cell Type             : gateCell
```

Related Commands

- check_timing_library_consistency

report_ipd_configuration

```
report_ipd_configuration
[-help]
[-all]
```

Reports the rail sweep information and the software created voltage combinations, that is, the IPD views.

This command reports and indicates the valid voltage combinations along with the excluded voltage combinations.

Additionally:

- IPD views are named automatically as “ipd_view<count>”.
- “Excluded” column reports “yes” against the excluded voltage combinations - based on `set_ipd_rail_voltage -exclude_voltage_signatures` settings.

Parameters

`-help`	Prints out the command usage.
`-all`	Reports all the valid and excluded configurations. By default, the view exclusion status is not printed.

Examples

- A sample report is shown below, when the following command is used:

```
report_ipd_configuration


POWER RAIL/GROUP MAPPING
------------------------
Mapping Power rail/group
------------------------
PG1 VDD
PG2 VDDR
PG3 VDDF
VOLTAGE SWEEP
-----------------------
Power Sweep
Rail
-----------------------
PG1 { 0.9 }
```

```
PG2 { 1.1 1.2 }
PG3 { 0.8 0.9 }
VOLTAGE SIGNATURES
----------------------------------------
PG1    PG2    PG3   View       Excluded
----------------------------------------
0.9    1.1    0.8   ipd_view1  no
0.9    1.2    0.8   ipd_view2  no
0.9    1.1    0.9   ipd_view3  no
0.9    1.2    0.9   ipd_view4  yes
```

- Sample report when `-all` parameter is used:

```
tempus > report_ipd_configuration -all

POWER RAIL/GROUP MAPPING
------------------------
Mapping Power rail/group
------------------------
PG1 VDD
PG2 VDDR
PG3 VDDF
VOLTAGE SWEEP
----------------------
Power Sweep
Rail
----------------------
PG1 { 0.9 }
PG2 { 1.1 1.2 }
PG3 { 0.8 0.9 }
VOLTAGE SIGNATURES
---------------------------------------
PG1    PG2   PG3   View        Excluded
---------------------------------------
0.9    1.1   0.8   ipd_view1   no
0.9    1.2   0.8   ipd_view2   no
0.9    1.1   0.9   ipd_view3   no
0.9    1.2   0.9   ipd_view4   yes
---------------------------------------
```

Related Commands

- `set_ipd_rail_voltage`
- `create_pg_net_group`
- `report_pg_net_group`
- `create_ipd_configuration`
- `write_ipd_scope`
- `read_ipd_scope`
- `report_ipd_crossings`

Related Information

- For more information, refer to the Inter Power Domain (IPD) Analysis chapter in the *Tempus Stylus User Guide*.

report_ipd_crossings

```
report_ipd_crossings
[-help]
[-all]
[-cppr_aware]
[-from <pin_list>]
[-from_power_domain <string>]
[-out_file <file_name>]
[-to <pin_list>]
[-to_power_domain <string>]
```

Reports the IPD crossing information summary on the design. This command also provides details on the pins and instances that are part of these crossings.

Parameters

`-help`	Prints out the command usage.
`-all`	Reports detailed crossing information.
`-cppr_aware`	Reports all the IPD crossings with CPPR awareness.
`-from <pin_list>`	Specifies the source pin for which crossing is to be reported.
`-from_power_domain <string>`	Specifies the source power domain for which crossing is to be reported.
`-out_file <file_name>`	Specifies the output file name - in case the output is to be redirected to a file.
`-to <pin_list>`	Specifies the sink pin for which crossing is to be reported.
`-to_power_domain <string>`	Specifies the sink power domain for which crossing is to be reported.

Examples

- The following output is generated by the `report_ipd_crossings` command:

```
tempus > report_ipd_crossings
-------------------------------------------------------------------------------
IPD          Inferred    Inferred    User       Excluded    Excluded    Active
Crossing     IPD         IPD         Excluded   due to      due to      IPD
             Crossings   Crossings              Same        Same        Crossings
             (Clock)     (Data)                 Voltage     Supply
                                                            Net
-------------------------------------------------------------------------------
AO -> PD1    1           1           0          0           0           2
PD1 -> AO    0           1           0          0           0           1
-------------------------------------------------------------------------------
```

- The following command reports detailed crossing information:

```
tempus > report_ipd_crossings -all
-------------------------------------------------------------------------------------
Source    Sink     Object   IPD        IPD        Source Crossing    Sink Crossing
Power     Power    Type     Crossing   Crossing   Pin                Pin
Domain    Domain            Type       Element
-------------------------------------------------------------------------------------
AO        PD1      Net      Inferred   Data       i4/Y               sub_inst2/LIN/A
AO        PD1      Net      Inferred   Clock      i3/Y               sub_inst2/LCLK/A
PD1       AO       Cell     Inferred   Data       sub_inst2/LOUT/A   sub_inst2/LOUT/Y
-------------------------------------------------------------------------------------
```

- The following command reports details of the specified source power domain:

```
tempus > report_ipd_crossings -from_power_domain PD1
-------------------------------------------------------------------------------------------------
-----------
Source    Sink     Object   IPD        IPD        Worst     Worst    Source Crossing    Sink
Crossing
Power     Power    Type     Crossing   Crossing   Setup     Hold     Pin                Pin

Domain    Domain            Type       Element    Slack     Slack

-------------------------------------------------------------------------------------------------
-----------
PD1       AO       Cell     Inferred   Data       -1.4127   1.2917   sub_inst2/LOUT/A
```

```
sub_inst2/LOUT/Y
-------------------------------------------------------------------------------------------------
-----------
```

- The following command reports details of the specified sink crossing pin:

```
tempus > report_ipd_crossings -to sub_inst2/LOUT/Y
-------------------------------------------------------------------------------
Source   Sink     Object   IPD        IPD        Source Crossing    Sink Crossing
Power    Power    Type     Crossing   Crossing   Pin                Pin
Domain   Domain            Type       Element
-------------------------------------------------------------------------------
PD1      AO       Cell     Inferred   Data       sub_inst2/LOUT/A   sub_inst2/LOUT/Y
-------------------------------------------------------------------------------
```

- The following command reports crossings between specified pins:

```
tempus > report_ipd_crossings -from inst3/A -to inst3/Y
-----------------------------------------------------------------------------------------
Source Power     Sink Power   Object   IPD        IPD       Source Crossing  Sink Crossing
Domain           Domain       Type     Crossing   Crossing  Pin              Pin
                                       Type       Element
-----------------------------------------------------------------------------------------
PD_A2VCC_A0VSS   PD_VDD_VSS   Cell     Inferred   Data      inst3/A          inst3/Y
-----------------------------------------------------------------------------------------
```

- The following command reports crossings between specified power domains:

```
tempus > report_ipd_crossings -from_to_power_domain PD_A0VCC_A0_VSS -to_power_domain
PD_VDD_VSS
-----------------------------------------------------------------------------------------
Source Power      Sink Power    Object  IPD       IPD       Source Crossing Sink Crossing
Domain            Domain        Type    Crossing  Crossing  Pin             Pin
                                        Type      Element
-----------------------------------------------------------------------------------------
PD_A0VCC_A0VSS    PD_VDD_VSS    Net     Inferred  Data      inst1/n[3]      inst5/A
PD_A0VCC_A0VSS    PD_VDD_VSS    Net     Inferred  Data      inst1/n[4]      inst5/A
-----------------------------------------------------------------------------------------
```

Related Commands

- set_ipd_rail_voltage
- create_pg_net_group
- report_pg_net_group

- `create_ipd_configuration`
- `report_ipd_configuration`
- `write_ipd_scope`
- `read_ipd_scope`

Related Information

- For more information, refer to the Inter Power Domain (IPD) Analysis chapter in the *Tempus Stylus User Guide*.

report_lib_arcs

```
report_lib_arcs
[-help]
-arc <string>
[-fall]
[-nldm]
[-quiet]
[-rise]
[-lvf  [-early]]
[-lvf  [-late]]
[-moments  [-mean_shift]]
[-moments  [-std_dev]]
[-moments  [-skewness]]
[-input_net_transition <float> | -constrained_pin_transition <float>]
[-input_net_transition <float> | -related_pin_transition <float>]
[-total_output_net_capacitance <float> | -constrained_pin_transition <float>]
[-total_output_net_capacitance <float> | -related_pin_transition <float>]
[-constrained_pin_transition <float> | -delay ] [-constrained_pin_transition <float> | -transition
]
[-related_pin_transition <float> | -delay ]
[-related_pin_transition <float> | -transition ]
```

Reports the library arc data. The command also returns the data in a Tcl object, which can be used in scripts.

This command uses a collection of arcs as input, which can be queried using the `get_lib_arcs` command.

The units for the input and output data can be obtained using the `get_time_unit` and `get_capacitance_unit` commands.

Parameters

`-help`	Prints out the command usage.
`-arc <string>`	Specifies the arc(s) for which the report is to be printed.
`-constrained_pin_transition <float>`	Specifies the constrained pin slew (valid for constraint arcs) at which the parameters need to be reported. The slew units must be specified using the `get_time_unit` command.
`-delay`	Specifies if delay is to be reported.

`-early`	Specifies if early LVF data is to be reported. This parameter can be used only in conjunction with the `-lvf`.
`-fall`	Specifies if details of fall tables are to be reported.
`-input_net_transition <float>`	Specifies the slew at which the parameters need to be reported. The slew units must be specified using the `get_time_unit` command.
`-late`	Specifies if late LVF data is to be reported. This parameter can be used only in conjunction with the `-lvf`.
`-lvf`	Specifies if LVF data is to be reported.
`-mean_shift`	Specifies if the mean shift data is to be reported. This parameter can be used only in conjunction with the `-moments`.
`-moments`	Specifies if moments data is to be reported.
`-nldm`	Specifies if NLDM data is to be reported.
`-quiet`	Disables output to be displayed on the screen.
`-related_pin_transition <float>`	Specifies the related pin slew (valid for constraint arcs) at which the parameters need to be reported. The slew units must be specified using the `get_time_unit` command.
`-rise`	Specifies if details of rise tables are to be reported.
`-skewness`	Specifies if skewness data is to be reported. This parameter can be used only in conjunction with the `-moments`.
`-std_dev`	Specifies if standard deviation data is to be reported. This parameter can be used only in conjunction with the `-moments`.
`-total_output_net_capacitance <float>`	Specifies the load at which the parameters need to be reported. The load must be specified using the `get_capacitance_unit` command.
`-transition`	Specifies if the output slew is to be reported.

Examples

- The following command reports the library arc data containing mean shift rise tables:

```
tempus > report_lib_arcs -arc [get_lib_arcs -from cell_w/DFF/CK -to cell_w/DFF/Q] -
input_net_transition 0.08 /
       -total_output_net_capacitance 0.055 -mean_shift -rise -delay -moments -nldm

Units of Report:- Time: '1000ps', Capacitance: '1000fF'.
input_net_transition:0.080000, total_output_net_capacitance:0.055000
-------------------------------------------------------------------
| Index | Timing Arc | Value(s) |
| ------------------------------------------------------------------------------|
|       | Cell       | Source/Sink Pin | Timing Sense | Timing Type | cell_rise |
-------------------------------------------------------------------------------
| 1     | cell_w/DFF | CK -> Q         | non_unate    | rising_edge | 0.258444  |
-------------------------------------------------------------------------------
{{cell_rise 0.258444279432} {ocv_mean_shift_cell_rise NA}}
```

- The following command reports the library arc data containing fall tables for moments data:

```
tempus > report_lib_arcs -arc [get_lib_arcs -from cell_w/BUF/A -to cell_w/BUF/Y] /
    -input_net_transition 0.055 -total_output_net_capacitance 0.15 -fall -delay -moments -nldm

Units of Report:- Time: '1000ps', Capacitance: '1000fF'.
input_net_transition:0.055000, total_output_net_capacitance:0.150000
-------------------------------------------------------------------------------------
| Index | Timing Arc                                                    | Value(s)   |
| ------------------------------------------------------------------------------------|
|       | Cell       | Source/Sink Pin | Timing Sense   | Timing Type  | cell_fall  |
-------------------------------------------------------------------------------------
| 1     | cell_w/BUF | A -> Y          | positive_unate | combinational| 0.129744   |
-------------------------------------------------------------------------------------
----------
{{cell_fall 0.129} {ocv_std_dev_cell_fall NA} {ocv_mean_shift_cell_fall NA}
{ocv_skewness_cell_fall NA}}
```

- The following command reports the library arc data containing rise tables data with output slew:

```
tempus > report_lib_arcs -arc [get_lib_arcs -from cell_w/DFF/CK -to cell_w/DFF/Q] -
input_net_transition 0.08 /
     -total_output_net_capacitance 0.055 -mean_shift -rise -delay -moments -nldm -transition

Units of Report:- Time: '1000ps', Capacitance: '1000fF'.
input_net_transition:0.080000, total_output_net_capacitance:0.055000
-------------------------------------------------------------------------------------------
-----
| Index | Timing Arc                                                                |
Value(s)  |
| ------------------------------------------------------------------------------------------
```

```
-----
|       | Cell       | Source/Sink Pin | Timing Sense | Timing Type | rise_transition |
cell_rise |
-------------------------------------------------------------------------------------------
-----
| 1     | cell_w/DFF | CK -> Q         | non_unate    | rising_edge | 0.21692         |
0.258444  |
-------------------------------------------------------------------------------------------
-----
{{cell_rise 0.258} {rise_transition 0.216} {ocv_mean_shift_cell_rise NA}
{ocv_mean_shift_rise_transition NA}}
```

This command also returns the data in a Tcl object that can be parsed and used in scripts. A 3-Dimensional Tcl List will return the following information:

{{TableName Value} {TableName Value}} {{TableName Value} {TableName Value}}

Here, the first list (left column) indicates the Table-Value pair in one arc. The second list indicates the various tables present per arc. And the third list (last column) indicates the list of total arcs in the collection.

- The following command reports the slew and capacitance load:

```
tempus > set items [report_lib_arcs -arc [ get_lib_arcs -from sc_cmoslp/AO22X/clk -to
sc_cmoslp/AO22X /out1] /
     -input_net_transition 0.003 -total_output_net_capacitance 0.021]


Units of Report:- Time: '1000ps', Capacitance: '1000fF'.
input_net_transition:0.003, total_output_net_capacitance:0.021
-------------------------------------------------------------------------------------------
-----------------------------------------
| Index | Timing Arc
                                | Value(s) |
|-----------------------------------------------------------------------------------------
-----------------------------------------
|       | Cell            | Source/Sink Pin | Timing Sense   | Timing Type   | rise_transition
| fall_transition | cell_rise | cell_fall |
-------------------------------------------------------------------------------------------
-----------------------------------------
| 1     | sc_cmoslp/AO22X | clk -> out1     | positive_unate | combinational | 0.20**
| 0.30**          | 1.00**    | 1.20**    |
-------------------------------------------------------------------------------------------
-----------------------------------------
| 2     | sc_cmoslp/AO22X | clk -> out1     | negative_unate | combinational | 0.20**
| 0.30**          | 1.00**    | 1.20**    |
-------------------------------------------------------------------------------------------
-----------------------------------------
```

```
** --> Scalar Value{{cell_rise 0.999} {rise_transition 0.199} {cell_fall 1.199}
{fall_transition 0.299}} {{cell_rise 0.999} {rise_transition 0.199} {cell_fall 1.199}
{fall_transition 0.299}}

tempus 5 > puts $items
{{cell_rise 0.999} {rise_transition 0.199} {cell_fall 1.199} {fall_transition 0.299}}
{{cell_rise 0.999} {rise_transition 0.199} {cell_fall 1.199} {fall_transition 0.299}}
```

Related Commands

- get_lib_arcs
- get_time_unit
- get_capacitance_unit

report_min_pulse_width

```
report_min_pulse_width
[-help]
[-clock <object_list>]
[-net]
[-path_type {summary | short | full_clock}]
[-pins <object_list>]
[-tcl_list]
[-verbose]
[-view <view_name>]
[-violation_only]
[-waveform_aware_type {regular | waveform_aware | both}]
[> <filename[.gz]>]
[>> <filename[.gz]
```

Reports a set of minimum pulse width violations on pins and ports on the clock network. The report shows the required pulse width, the actual pulse width, and by how much the constraint is violated or met. This is reported separately for sequential elements and instances on the clock tree.

You can customize pulse width reporting format by using the `report_timing_format` attribute.

The report header shows clock edge information to make it more intuitive for clock style checks.

Parameters

`-help`	Prints out the command usage.
`-clock <object_list>`	Specifies a set of clocks to for which to report the minimum pulse width.

{> | >>} *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.
-net	Reports the net arcs. This parameter can be specified with the -path_type full_clock option only.
-path_type {summary | short | full_clock}	Specifies the type of report to generate. *Default:* summary • summary: Generates a summary report for each pulse width check that shows the required width, the actual width, and the resulting slack. • short: Generates a path report for each pulse width check that lists the latency of each edge of the pulse as a single lumped value, instead of reporting a detailed path report. • full_clock: Generates a path report for each pulse width check that shows the detailed pin-to-pin path for each edge of the pulse. **Note:** The -path_type full_clock parameter may cause an increased runtime. In that case you can use the -path_type short parameter.
-pins *object_list*	Specifies a set of ports and pins on the clock network. If this list is not specified, then the report contains all the pins and ports in the current design.

`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.
`-verbose`	Provides detailed information about minimum pulse width calculations. **Note:** This parameter is obsolete. The parameter still works in this release, but to ensure compatibility with future releases, use the `-path_type` parameter instead.
`-view` *`view_name`*	Generates the report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.
`-violation_only`	Reports only the negative slack minimum pulse width violations.
`-waveform_aware_type {regular | waveform_aware | both}`	Reports the waveform-aware pulse width checks. You can specify the following options: • `waveform_aware`: Reports only the waveform-aware pulse width checks. • `both`: Reports waveform-aware as well as regular pulse width checks. • `regular`: Reports regular pulse width checks. This is the default setting.

Examples

- The following command reports minimum pulse width for the specified clock and view:

```
tempus > report_min_pulse_width -clock m_spi_clk -view slow_RCMIN
--------------------------------------------------------------------------------
                          SEQUENTIAL CLOCK PULSE WIDTH

--------------------------------------------------------------------------------
Pin                        Required       Actual Pulse  Slack    View Name
                           Pulse Width    Width
--------------------------------------------------------------------------------
SP_IN/bit_reg[0]/CK (low)  1.900          1.596         -0.304    slow_RCMIN
SP_IN/bit_reg[3]/CK (low)  1.900          1.596         -0.304    slow_RCMIN
```

```
SP_IN/bit_reg[2]/CK (low)  1.900          1.596         -0.304    slow_RCMIN
SP_IN/bit_reg[1]/CK (low)  1.900          1.596         -0.304    slow_RCMIN
```

- The following command generates a summary report for each pulse width check that shows the required and actual width, and negative slack minimum pulse width violations.

```
tempus > report_min_pulse_width -clock ram_clk -violation_only -path_type summary
------------------------------------------------------------------------
                     SEQUENTIAL CLOCK PULSE WIDTH
------------------------------------------------------------------------
Pin                      Required    Actual Pulse   Slack    View Name
                                     Pulse Width    Width

------------------------------------------------------------------------
RAM_256x16/CLK (high)    0.061       -0.788         -0.849   slow_RCMAX
------------------------------------------------------------------------
```

- The following command reports net arcs, along with detailed information about minimum pulse width calculations:

```
tempus > report_min_pulse_width -clock ram_clk -net -verbose

SEQUENTIAL CLOCK PULSE WIDTH
----------------------------
Path 1: VIOLATED (-0.849 ns) PulseWidth Check with Pin RAM_256x16/CLK->CLK
               View: func_slow_RCMAX
         Startpoint: (R) RAM_256x16/CLK
              Clock: (R) ram_clk
           Endpoint: (F) RAM_256x16/CLK
              Clock: (F) ram_clk

                       Capture       Launch
         Clock Edge:+    2.000        0.000
        Src Latency:+    0.000        0.000
        Net Latency:+    0.552 (P)    2.940 (P)
            Arrival:=    2.552        2.940

         PulseWidth:-    0.061
        Uncertainty:-    0.400
      Required Time:=    2.091
       Launch Clock:=    2.940
          Data Path:+    0.000
              Slack:=   -0.849
     Timing Path:

#-------------------------------------------------------------
# Pin                     Cell          Delay    Arrival   Edge
#                                       (ns)     (ns)
#-------------------------------------------------------------
```

```
CTRL_INST/p5A89/ZN        (arrival)      -       0.000     R
ram_clk_L1_I0/A           BUF_X32       0.000    0.000     R
ram_clk_L1_I0/Z           BUF_X32       1.897    1.897     R
RAM_256x16/CLK           ram_256x16A    1.043    2.940     R
#-------------------------------------------------------------
     Other End Path:

#-------------------------------------------------------------
# Pin                    Cell          Delay    Arrival   Edge
#                                      (ns)       (ns)
#-------------------------------------------------------------
CTRL_INST/p5A89/ZN       (arrival)       -      2.000     F
ram_clk_L1_I0/A           BUF_X32    0.000      2.000     F
ram_clk_L1_I0/Z           BUF_X32    0.669      2.669     F
RAM_256x16/CLK         ram_256x16A  -0.117      2.552     F
```

- The `report_min_pulse_width` will generate a summary report as shown below:

```
report_min_pulse_width

-----------------------------------------------------
SEQUENTIAL CLOCK PULSE WIDTH
-----------------------------------------------------
Pin                Required     Actual Pulse   Slack
                   Pulse Width  Width
-----------------------------------------------------
seg3/u14/CK (high) 0.169        0.860          0.691
seg3/u3/CK (high)  0.169        0.875          0.706
seg3/u9/CK (high)  0.169        0.875          0.706
seg3/u3/CK (low)   0.195        1.083          0.888
seg3/u9/CK (low)   0.195        1.083          0.888
-----------------------------------------------------
```

- The following command reports sequential clock pulse width details:

```
report_min_pulse_width -path_type short

SEQUENTIAL CLOCK PULSE WIDTH
----------------------------
Path 1: MET PulseWidth Check with Pin seg3/u14/CK
Ending Clock Edge: seg3/u14/CK (v) checked with trailing edge of 'CLK_W_3'
Beginning Clock Edge: seg3/u14/CK (^) triggered by leading edge of 'CLK_W_3'
Other End Arrival Time 1.503
- PulseWidth 0.169
+ Phase Shift 0.000
= Required Time 1.334
- Arrival Time 0.621
= Slack Time 0.714
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.621
= Beginpoint Arrival Time 0.621
----------------------------------------
```

```
Pin         Cell     Delay  Arrival  Edge
                            Time
-----------------------------------------
seg3/u14/CK DFF              0.621   ^
-----------------------------------------
Path 2: MET PulseWidth Check with Pin seg3/u3/CK
Ending Clock Edge: seg3/u3/CK (v) checked with trailing edge of 'CLK_W_3'
Beginning Clock Edge: seg3/u3/CK (^) triggered by leading edge of 'CLK_W_3'
Other End Arrival Time 1.578
- PulseWidth 0.169
+ Phase Shift 0.000
= Required Time 1.409
- Arrival Time 0.682
= Slack Time 0.727
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.682
= Beginpoint Arrival Time 0.682
---------------------------------------
Pin        Cell    Delay  Arrival   Edge
                          Time
---------------------------------------
seg3/u3/CK DFF             0.682     ^
---------------------------------------
```

Related Commands

- report_analysis_coverage
- report_timing
- set_min_pulse_width
- reset_min_pulse_width

report_mode

```
report_mode
[-help]
inst_list
[-view viewName]
```

Reports the status of library modes associated with a specified instance. The report includes the status of each mode and its condition, for each mode group defined in the timing library.

Parameters

`-help`	Prints out the command usage.
`inst_list`	Generates a report for the specified instances.
`-view` *`viewName`*	Reports the status of library modes in the specified active analysis view. Default: Reports the status of library modes in all active analysis views.

Examples

- The following command activates the tst_mode_disable library mode for instances of i_ram:
  ```
  tempus> set_mode tst_mode_disable [get_cells i_ram]
  ```
- The following command sets the active setup and hold analysis views:
  ```
  tempus> set_analysis_views -setup {v1 v2} -hold {v3}
  ```
- The following command reports the status of all modes associated with m4k_top:
  ```
  tempus> report_mode m4k_top

  ------------------------------------------------
  Mode Group tets_mode of m4k_top
  ------------------------------------------------
  Mode Name         Status    Condition     View Name
  ------------------------------------------------
  test_mode_enable  ACTIVE    TST_BYPASS    View1
  test_mode_disable INACTIVE  !(TST_BYPASS) View2
  ------------------------------------------------
  ```
- The following command reports the status of all modes in the analysis view `View1`:
  ```
  tempus> report_mode m4k_top -view View1
  ```

```
-------------------------------------------
Mode Group tets_mode of m4k_top
-------------------------------------------
Mode Name        Status  Condition  View Name
-------------------------------------------
test_mode_enable ACTIVE  TST_BYPASS View1
-------------------------------------------
```

Related Commands

- set_analysis_view
- set_mode

report_nets

```
report_nets
[-help]
[-min_fanout int]
[-max_fanout int]
{-net list_of_net_name_or_id | -pin list_of_pin_name_or_id}
[-tcl_list]
[-hier]
[[{> | >>} filename] | -tcl_list]
```

Reports the net information on the current module. The information includes the net name, the number of source pins, sink pins and bidirectional pins on the net, wire capacitance, and total capacitance.

Electrical information such as pin capacitance, and wire capacitance with which the net is associated are also included.

Use the `-pin` option to report the net that is connected to the specified pin (or port). Both `-net` and `-pin` cannot be specified at the same time. The net (or pin) names can be hierarchical names that are unique relative to the top cell.

This command only searches for the defined nets and does not modify any object in database.

Parameters

`-help`	Prints out the command usage.
`{> | >>} filename`	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. The *`filename`* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-hier`	Reports all of the nets in the design hierarchy. Without this argument, only the nets in the current level of hierarchy are reported.

`-max_fanout` *int*	Reports the net whose number of fanouts is less than or equal to the maximum number specified by *integer*.
`-min_fanout` *int*	Reports the net whose number of fanouts is greater than or equal to the minimum number specified by *integer*.
`-net` *list_of_net_name_or_id*	Reports the set of nets on the list of net names. This argument cannot be specified at the same time as `-pin`. If neither `-net` or `-pin` is specified, all nets are listed by default. **Note:** This argument also reports the capacitance of the net and the input pins.
`-pin` *list_of_pin_name_or_id*	Reports the set of nets to which the pins on the list *list_of_pin_name_or_id* are connected. This argument cannot be specified at the same time as `-net`.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for extracting information from the report in a Tcl program.

Examples

- The following command reports details of the specified set of nets:

```
tempus > report_nets  -net FE_data_out_1

Net Name                 : FE_data_out_1
Number of Sources        : 1
Number of Sinks          : 1
Number of Bidis          : 0
Source of parasitics     : Detailed Spef
Net Capacitance          :   0.000
Total Capacitance        :   0.001
------------------------------------------------------------------------------------------------------------------
                                                Source

------------------------------------------------------------------------------------------------------------------
                 Pin                                    R     Driver Load         Slew

------------------------------------------------------------------------------------------------------------------
          Name       Dir     Cell      Cap     Model     Model   Ctotal   Rise    Fall
```

```
  Phase
-------------------------------------------------------------------------------------------------
------------------
FE33_data_out_1/Z    OUT    BUF_X16   0.000   Unknown   CTOTAL    1.000   0.072   0.069
 clk(D)(P)(slow_RCMAX)
FE33_data_out_1/Z    OUT    BUF_X16   0.000   Unknown   CTOTAL    1.000   0.072   0.069
 clk(D)(P)(fast_RCMAX)
-------------------------------------------------------------------------------------------------
------------------
                                            Sinks for pin FE33_data_out_1/Z

-------------------------------------------------------------------------------------------------
-----------------
                  Pin             R       Delay           Slew

-------------------------------------------------------------------------------------------------
------------------
             Name       Dir    Cell      Cap    Parameters   Rise   Fall   Rise   Fall
 Phase
-------------------------------------------------------------------------------------------------
------------------
FE_C583_data_out_1/A    IN     BUF_X32   0.000       -       0.000  0.000  0.072  0.069
 clk(D)(P)(slow_RCMAX)
FE_C583_data_out_1/A    IN     BUF_X32   0.000       -       0.000  0.000  0.072  0.069
 clk(D)(P)(fast_RCMAX)
-------------------------------------------------------------------------------------------------
------------------
```

- The following command reports nets whose number of fanouts is less than or equal to the specified maximum number 20:

```
tempus > report_nets -pin FE_F5_datain_7/A -max_fanout 20

Net Name                 : ds_datain[7]
Number of Sources        : 1
Number of Sinks          : 1
Number of Bidis          : 0
Source of parasitics     : Detailed Spef
Net Capacitance          :   0.000
Total Capacitance        :   0.001
-------------------------------------------------------------------------------------------------
----------------
                                         Source

-------------------------------------------------------------------------------------------------
----------------
                               Pin                                    R     Driver Load
Slew
-------------------------------------------------------------------------------------------------
----------------
```

```
Name                  Dir     Cell     Cap     Model     Model   Ctotal    Rise    Fall
 Phase
-------------------------------------------------------------------------------------------
----------------
DATA_MUX_INST/F6A     OUT     INV_X8   0.000   Unknown   CTOTAL    1.000   0.190   0.365
 clk(D)(P)(slow_RCMAX)
-------------------------------------------------------------------------------------------
----------------
-------------------------------------------------------------------------------------------
----------------
                              Sinks for pin DATA_MUX_INST/F6A

-------------------------------------------------------------------------------------------
-----------
                           Pin                                R       Delay           Slew

-------------------------------------------------------------------------------------------
-----------
        Name       Dir   Cell      Cap    Parameters   Rise   Fall   Rise   Fall
 Phase
-------------------------------------------------------------------------------------------
-----------
FE_F5_datain_7/A   IN    BUF_X16   0.000       -       0.000  0.000  0.190  0.365   clk(D)(P)
(slow_RCMAX)
FE_F5_datain_7/A   IN    BUF_X16   0.000       -       0.000  0.000  0.190  0.365   clk(D)(P)
(fast_RCMAX)
-------------------------------------------------------------------------------------------
-----------
```

- Control the `report_nets` table fields, such as setting the minimum and maximum column width using the `set_table_style` command. The table names are `report_nets_source_table`, `report_nets_sink_table`, `report_nets_summary_table`, `report_nets`, and `report_nets_header`.
- The following command sets an indentation of six columns for the table and disables printing of the first column:

  ```
  set_table_style -name report_nets_source_table -indent 6 -max_widths {0,}
  report_nets
  ```

- The format of the `report_nets` report is the same for all the examples, as shown in the first example. The following command reports information about net named `in`, as shown in the following example:

  ```
  report_nets -net n5

  Net Name : n5
  Number of Sources : 1
  Number of Sinks : 1
  Number of Bidis : 0
  Source of parasitics : Wireload model(.lib None Generated)
  ```

```
Net Capacitance : 0.000
Total Capacitance : 0.011
---------------------------------------------------------------------
Source
---------------------------------------------------------------------
Pin R Driver Load Slew
---------------------------------------------------------------------
Name  Dir  Cell  Cap   Model   Model  Ctotal  Rise   Fall  Phase
---------------------------------------------------------------------------
u5/Q  OUT  DFF   0.000 Unknown CTOTAL 1.000   0.003  0.003 CLK_W_4_gen2(D)(N)
u5/Q  OUT  DFF   0.000 Unknown CTOTAL 1.000   0.003  0.003 CLK_W_4_gen1_inv(D)(P)
---------------------------------------------------------------------------
---------------------------------------------------------------------
Sinks for pin u5/Q
---------------------------------------------------------------------
Pin R Delay Slew
---------------------------------------------------------------------
Name   Dir  Cell  Cap    Parameters Rise   Fall   Rise   Fall   Phase
-------------------------------------------------------------------------------
u7/A  IN   BUF    0.011  -          0.000  0.000  0.003  0.003   CLK_W_4_gen2(D)(N)
u7/A  IN   BUF    0.011  -          0.000  0.000  0.003  0.003   CLK_W_4_gen1_inv(D)(P)
-------------------------------------------------------------------------------
```

- The following command reports the net to which the pin `interesting_pin` is connected:

  ```
  report_nets -pin interesting_pin
  ```

- The following command reports all the nets in the current module:

  ```
  report_nets -net *
  ```

- The following command reports all nets with a fanout greater than 16:

  ```
  report_nets -net * -min_fanout 16
  ```

Related Commands

- report_path_exceptions
- report_timing

report_parasitic_coordinate_transforms

```
report_parasitic_coordinate_transforms
[-help]
[-user]
```

Reports the transformations applied to the hierarchical SPEF instances to translate the hierarchical locations given in the SPEF files to the flat locations for the design.

Parameters

`-help`	Prints a brief description that includes type and default information for each `report_parasitic_coordinate_transforms` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_parasitic_coordinate_transforms`.
`-user`	Reports user-specified transforms.

Examples

The following command reports user-specified transforms:

```
create_parasitic_coordinate_transform -inst ARB_INST -x_offset  16.8  -y_offset 20.5 -orient R0 -x_block_origin 0 -y_block_origin 0

report_parasitic_coordinate_transforms -user

#cell      x_offset     y_offset   orient   x_block_origin    y_block_origin   source
 hinst
arb        16.8         20.5       R0       0                 0                User
 ARB_INST
```

Related Commands

- create_parasitic_coordinate_transform

report_path_based_aocv_derate

```
report_path_based_aocv_derate
  [-help]
  [-aocv_derate_only]
  [-delay_corner delayCornerName]
  [-derate_mode {worst_slack | worst_derate}]
  [-early]
  [-include_clock_derate]
  [-late]
  [-max_slack val]
  [-max_paths integer | -max_points integer]
  [-nworst worst_entries]
  [-retime {aocv | aocv_path_slew_propagation}]
  [-retime_mode {path | exhaustive}]
  [> filename[.gz]]
  [>> filename[.gz]]
```

Parameters

`-help`	Prints out the command usage.
`-aocv_derate_only`	Allows AOCV reporting only.
`-delay_corner` *delayCornerName*	Specifies name of the delay corner.
`-derate_mode` {`worst_slack` | `worst_derate`}	Specifies the derating mode. *Default*: `worst_derate`
`-early`	Specifies derates for early paths.
`-include_clock_derate`	Includes the clock derate values.
`-late`	Specifies derates for late paths.
`-max_paths` *integer*	Reports derates based on the specified number of paths.
`-max_points` *integer*	Reports derates based on the specified number of endpoints.
`-max_slack` *val*	Reports the upper bound value of the reported slack.
`-nworst` *worst_entries*	Reports the derates based on the specified number of worst paths per end point.
`-retime {aocv | aocv_path_slew_propagation}`	Reanalyzes the specified set of paths using the specified method. Default: `aocv`
`-retime_mode {path | exhaustive}`	Defines the retime mode as `path` or `exhaustive`.
`>` *filename[.gz]*	Redirects output to the specified file.
`>>` *filename[.gz]*	Redirects output to the specified file.

Examples

- The following command reports derates based on the given number of paths for the specified delay corner:

```
tempus > report_path_based_aocv_derate -delay_corner DC_wc -max_paths 1

INFO: Path Based Analysis (PBA) performed on total '1' paths
set_timing_derate -late -delay_corner DC_wc -data -rise 1.600 [get_cells { i0 }]
set_timing_derate -late -delay_corner DC_wc -data -rise 1.000 [get_cells { i1 }]
set_timing_derate -late -delay_corner DC_wc -data -rise 1.000 [get_cells { sub_inst1/i1 }]
```

- The following command reports clock derates for the specified delay corner:

```
tempus 8> report_path_based_aocv_derate -delay_corner DC_wc -include_clock_derate

INFO: Path Based Analysis (PBA) performed on total '4' paths
set_timing_derate -late -delay_corner DC_wc -data -rise 1.600 [get_cells { i0 }]
set_timing_derate -late -delay_corner DC_wc -data -rise 1.000 [get_cells { i1 }]
set_timing_derate -late -delay_corner DC_wc -data -fall 1.014 [get_cells { i2 }]
set_timing_derate -early -delay_corner DC_wc -clock -rise 0.820 [get_cells { i3 }]
```

Related Commands

- report_timing

report_path_exceptions

```
report_path_exceptions
[-help]
[-all]
[-early | -late]
[-ignored]
[-view view_name]
[{> | >>} filename | -tcl_list]
```

Generates a report about path exceptions specified using the `set_false_path`, `set_min_delay`, `set_max_delay` and `set_multicycle_path` commands.

By default, the `report_path_exceptions` command only reports activated path exceptions. To report only the path exceptions that have been ignored by timing analysis, specify the `-ignored` parameter.

If more than one path exception is applied on a timing path, multiple path exceptions that match a given path are prioritized. The adjustment on the path honors the path exception with the highest priority.

When the software is in multi-mode multi-corner timing analysis mode, the report generates path exception information for every active analysis view.

The `report_path_exceptions` command output also includes 2 columns that show the file name and line number. The report output shows the exact file name and line number of the applied/ignored constraints. The compressed constraints will be decompressed first and then shown as individual constraints as written in the SDC file.

Parameters

`-help`	Prints out command usage.
`-all`	Reports all the applied and ignored path exceptions.

{> | >>} *filename*	When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *filename* parameter must be the last argument in the list. The *filename* and -tcl_list parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the .gz extension to the file name. *Default:* Report is displayed on standard output without being saved.
-early | -late	Generates the path exception report for either early paths or late paths. *Default:* Generates a report based on the current timing analysis mode.

-ignored	Reports only the path exceptions that have been ignored by timing analysis. This parameter can be used for debugging constraints that may be ineffective or incorrectly entered. A path exception may be ineffective if it is covered by, or overridden by, another path exception. For example:
	
	`set_false_path -to OUT`
	`set_multicycle_path -to OUT 2`
	
	In the above situation, the set_multicycle_path command would be ignored because the false path has a higher priority.

-tcl_list	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The -tcl_list and *filename* parameters are mutually exclusive; you cannot specify them together.

`-view` *view_name*	Generates a path exception report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

- The following command generates an exception report of late paths for the specified view:

```
tempus > report_path_exceptions -view scan_slow_RCMAX -late

From                To                   Late    View Name
-----------------------------------------------------------------
scan_se                                  add 1   scan_slow_RCMAX
tst_mode            refclk               false   scan_slow_RCMAX
SPI/bit_reg[3]/RN   SPI/bit_reg[1]/RN    false   scan_slow_RCMAX
-----------------------------------------------------------------
```

- The following commands shows that a false path has priority over cycle addition on the same path:

```
tempus > set_false_path -from seg1/u3/q -to seg1/u9/d
tempus > set_multicycle_path -from seg1/u3/Q -to seg1/u9/D 2
tempus > report_path_exception -ignored

-------------------------------------------------------
From        To           Early            Late
-------------------------------------------------------
seg1/u3/Q   seg1/u9/D    false ignored    false ignored
seg1/u3/Q   seg1/u9/D    -                add 1 ignored
seg1/u3/Q   seg1/u9/D    add 0 ignored    -
-------------------------------------------------------
```

  The following command shows how edges are handled:

```
tempus > set_false_path -rise_from {IN1} -through {OUT1}
tempus > set_false_path -rise_from {IN1 IN2} -fall_to {OUT1}
tempus > set_false_path -rise_from {IN1}
tempus > report_path_exceptions -ignored

-------------------------------------------------------------------
From         Through     To       Early            Late
-------------------------------------------------------------------
^ IN1 IN2    -           v OUT1   false ignored    false ignored
^ IN1        OUT1        -        false ignored    false ignored
-------------------------------------------------------------------

tempus > report_path_exceptions -early
---------------------------------------
From       Through    To   Early
---------------------------------------
^ IN1      -          -    false
---------------------------------------
```

```
tempus > report_path_exceptions -late
----------------------------------------
From    Through  To   Late
----------------------------------------
^ IN1   -        -    false
----------------------------------------
```

When the software is in multi-mode multi-corner timing analysis mode, the software generates path exception information for every active analysis view:

```
tempus > set_analysis_view -setup {setup1 setup2} -hold {hold1 hold2}
tempus > report_path_exceptions

-----------------------------------------
From               Early   Late     View
                                    Name
-----------------------------------------
test_mode reset   false            setup2
test_mode reset   false            setup1
test_mode reset           false    hold1
test_mode reset           false    hold2
-----------------------------------------
```

You can specify the `-view` parameter to generate a report only for a specific active analysis view:

```
tempus > report_path_exceptions -view setup1

-------------------------------------------
From        Early    Late      View
                              Name
-------------------------------------------
test_mode   reset    false     setup1
-------------------------------------------
```

Related Commands

- report_timing

report_path_groups

```
report_path_groups
[-help]
[-name group_name]
[{> | >>} filename | -tcl_list]
```

Generates a report that lists the names of all path groups with the corresponding paths.

Parameters

`-help`	Prints out the command usage.
`{> | >>} filename`	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. The *`filename`* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-name group_name`	Limits the report to the specified path group. *Default*: All path groups are reported.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together.

Examples

- The following command generates a report that shows details of the specified path group:

```
tempus > set_path_group -name path1 -through CF_PS_PD_142/NS
tempus > report_path_groups -name path1
-----------------------------------------
Through           Group Name   View Name
-----------------------------------------
CF_PS_PD_142/NS   path1        setup_gs_mC
-----------------------------------------
```

- The following command shows a sample report:

```
tempus> report_path_groups
---------------------------------
From    To         Group Name
---------------------------------
CLK1    seg1/u3/D  path1
---------------------------------
```

Related Commands

- set_clock_groups
- reset_clock_groups
- reset_path_group
- get_path_groups
- group_path

report_pg_net_group

```
report_pg_net_group
[-help]
[-expand]
[-group <name>]
```

Reports the output of the `create_pg_net_group` command settings. The output indicates the PG net groups created in the design and the PG nets associated with these groups.

Parameters

`-help`	Prints out the command usage
`-group <name>`	Specifies a list of power nets associated with the specified group.

Examples

- The following command shows the report output of the previously created group PG1:

```
tempus > report_pg_net_group

create_pg_net_group  -name PG1  -pg_nets  {VDD_top VDD1 VDDalive VSS}
```

- The following command shows the PG nets associated with the PG net group PG1:

```
tempus> report_pg_net_group  -group PG1

VDD1   VDD_top  VDDalive  VSS
```

Related Commands

- `create_pg_net_group`
- `create_ipd_configuration`
- `report_ipd_configuration`
- `write_ipd_scope`
- `read_ipd_scope`
- `set_ipd_rail_voltage`
- `report_ipd_crossings`

Related Information

- For more information, refer to the Inter Power Domain (IPD) Analysis chapter in the *Tempus Stylus User Guide*.

report_ports

```
report_ports
[-type {[input] | [source_insertion] | [insertion] | [clock_root]
| [uncertainty] | [arrival] | [required] | [external] | [clk_arrival]
| [port_cap] | [fanout_load] | [fanout_load_limit] | [drive_resistance]
| [drive_cell] | [slew_time] | [slew_limit] | [constant] | external_detail
| drive_resistance_detail}]
[-include_pins]
[-pins port_name_list]
[-view view_name]
[> filename | -tcl_list]
```

Reports timing constraints on ports. By default, the command reports all timing constraints on all ports of the current module.

The direction (*DIR*) column of the generated report can contain one of the following abbreviations:

- `IN` indicates that the pin is an input pin.
- `OUT` indicates that the pin is an output pin.
- `INTL` indicates that the pin is an internal pin (a pin inside the instance boundary).

The *Clock Name* column of the report can contain the following symbols:

- `D` indicates it is a data signal in the clock domain.
- `C` indicates it is a clock signal in the clock domain.
- `P` indicates it is a positive phase.
- `N` indicates it is a negative phase.
- Asterisk (`*`) indicates there is a timing exception associated with the signal.

  Note: Currently, it is not possible to expand the asterisk to determine which exception is associated with the reported timing phase.

Parameters

`> filename`	Writes the report to the specified file name. If the file already exists, the software overwrites it. The *filename* parameter must be the last argument in the list. The *filename* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-include_pins`	Reports timing constraints on instance pins, as well as on ports. *Default:* Reports timing constraints on ports only.
`-pins` *port_name_list*	Specifies the pins or ports to be reported. Pins can be specified by name, or by object ID in a Tcl list.
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions, and also for customizing report generation. The `-tcl_list` and *filename* parameters are mutually exclusive; you cannot specify them together.
`-type {`*assertion_type* | *assertion_type_list*`}`	Generates the report for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.
`-view` *view_name*	Specifies the view name.

Examples

- The following command reports details of the specified list of pins:

```
tempus > report_ports -pins scan_en
-------------------------------------------------------------------------------------------
                                      Early          Late
-------------------------------------------------------------------------------------------
Pin Name   Dir   Assertion   Clock Name                    Rise    Fall    Rise    Fall
-------------------------------------------------------------------------------------------
```

```
scan_en    IN    input       m_clk(D)(P)(scan_fast_RCMIN)  4.000   4.000   4.000   4.000
scan_en    IN    input       m_clk(D)(P)(scan_slow_RCMAX)  4.000   4.000   4.000   4.000
-------------------------------------------------------------------------------------------
Pin Name   Di    Assertion   Value  View
-------------------------------------------------
scan_en    IN    constant    0      func_slow_RCMAX
scan_en    IN    constant    0      func_fast_RCMAX
-------------------------------------------------
```

- The following command reports details of the specified list of pins: Reports timing constraints on instance pins, as well as on ports.

```
tempus > report_ports -include_pins
-------------------------------------------------------------------------------
                                    Early          Late
-------------------------------------------------------------------------------
Pin Name    Dir   Assertion    Clock Name          Rise   Fall   Rise   Fall
-------------------------------------------------------------------------------
INT/p97/Z   OUT   clock_root   clk(sn_fast_RMIN)   -      -      -      -
INT/p97/Z   OUT   clock_root   clk(sn_slow_RCMIN)  -      -      -      -
INT/p97/Z   OUT   clock_root   clk(sn_fast_RCMAX)  -      -      -      -
INT/p97/Z   OUT   clock_root   clk(sn_slow_RCMAX)  -      -      -      -
```

- The following command shows a port report, as shown in the following example:

```
tempus > report_ports

-------------------------------------------------------------------------
Early Late
-------------------------------------------------------------------------
Pin     Dir     Assertion          Clock Name   Rise    Fall   Rise  Fall
Name
-------------------------------------------------------------------------
CLK1    IN      clock_root          CK2(C)(P)
CLK2    IN      clock_root          CLK2(C)(P)
CLK3    IN      clock_root          CLK3(C)(P)
IN1     IN      drive_resistance    *(D)(P)      2.000  2.000  2.000  2.000
IN2     IN      drive_resistance    *(D)(P)      2.000  2.000  2.000  2.000
IN3     IN      drive_cell          *(D)(P)      INV    INV    INV    INV
OUT1    OUT     external            GCLK1(C)(P)  1.000  1.000  1.000  1.000
OUT2    OUT
OUT3    OUT     derived_clock       GCLK4(C)(P)
-------------------------------------------------------------------------
Pin   Dir    Assertion               Value
Name
-----------------------------------------------------------
CLK3   IN   slew_limit              ( 0.900 : )
```

```
CLK3   IN   port_cap_limit          ( 0.020 : 0.900 )
CLK3   IN   fanout_load_limit       5.000
CLK2   IN   slew_limit              ( 0.900 : )
CLK2   IN   port_cap_limit          ( 0.020 : 0.900 )
CLK2   IN   fanout_load_limit       5.000
CLK1   IN   slew_limit              ( 0.900 : )
CLK1   IN   port_cap_limit          ( 0.020 : 0.900 )
CLK1   IN   fanout_load_limit       5.000
IN3    IN   slew_limit              ( 0.900 : )
IN3    IN   port_cap                ( 1.500 : : 2.600 )
IN3    IN   port_wire_cap           ( 1.200 : : 3.000 )
IN3    IN   port_cap_limit          ( 0.020 : 0.900 )
IN3    IN   fanout_load_limit       5.000

IN2    IN   slew_limit              ( 0.900 : )
IN2    IN   port_cap                ( 1.500 : : 2.600 )
IN2    IN   port_wire_cap           ( 1.200 : : 3.000 )
IN2    IN   port_cap_limit          ( 0.020 : 0.900 )
IN2    IN   fanout_load_limit       5.000
IN1    IN   slew_limit              ( 0.900 : )
IN1    IN   port_cap                ( 1.500 : : 2.600 )
IN1    IN   port_wire_cap           ( 1.200 : : 3.000 )
IN1    IN   port_cap_limit          ( 0.020 : 0.900 )
IN1    IN   fanout_load_limit       5.000
OUT3   OUT  slew_limit              ( 0.900 : )
OUT3   OUT  port_cap_limit          ( 0.020 : 0.900 )
OUT2   OUT  slew_limit              ( 0.900 : )
OUT2   OUT  port_cap_limit          ( 0.020 : 0.900 )
OUT1   OUT  slew_limit              ( 0.900 : )
OUT1   OUT  port_cap_limit          ( 0.020 : 0.900 )
```

- The following command shows `source_insertion` and `uncertainty` assertion type options as shown in the following example:

```
tempus > report_ports -type clock_root source_insertion input -pin clk in
tempus > report_ports -type {uncertainty source_insertion} -pin CLK1

-----------------------------------------------------------------
Early Late
-----------------------------------------------------------------
Pin    Dir  Assertion     Clock Rise    Fall  Rise   Fall
Name                     Name
-----------------------------------------------------------------
CLK1   IN   source_insertion *(C)(P)  -0.500 -0.500 -0.500 -0.500
-----------------------------------------------------------------

Pin    Dir  Assertion    Value
Name
-------------------------------------------------------------
CLK1  IN   uncertainty  0.400(T) 0.400(T) 0.400(T) 0.400(T)
```

```
-------------------------------------------------------------
```

- The following command shows `clock_root` and `uncertainty` assertion type options as shown in the following example:

```
tempus > report_ports -type {input uncertainty clock_root}

-------------------------------------------------------------------
Early Late
-------------------------------------------------------------------
Pin    Dir  Assertion   Clock Name    Rise   Fall   Rise   Fall
Name
-------------------------------------------------------------------
CLK1   IN   clock_root  CLK_W_1(C)(P)
in1    IN   input       @(D)(P)       1.000  1.000  1.000  1.000
-------------------------------------------------------------------

Pin   Dir  Assertion    Value
Name
-------------------------------------------------------------
CLK1  IN   uncertainty  0.400(T) 0.400(T) 0.400(T) 0.400(T)
-------------------------------------------------------------
```

Related Commands

- `report_clocks`
- `report_nets`
- `report_timing`

report_reference

```
report_reference
[-help]
[-flat]
[-nosplit]
[-view]
[> file_name]
[>> file_name]
```

Reports reference cell usage statistics.

The command reports the unit area, library name, and total count of the reference cells. By default, the report is sorted in the order of number of instances.

Parameters

`-help`	Prints out the command usage.
`-flat`	Disables printing of hierarchical instances.
`-nosplit`	Generates the report in no split mode.
`-view`	Generates the report for the specified analysis view.
`> file_name`	Specifies the name of write mode file name.
`>> file_name`	Specifies the name of the append mode file name.

Examples

- The following command displays the report for the specified view:

```
report_reference -view func_slow_RCMIN

Design  : dtmf_recvr_core

Attributes :

c   -  combinational (standard cell)

sq  -  sequential (standard cell)

m   -  macro

h   -  hierarchical

b   -  blackbox


  Reference        Library              Unit Area         Count        Total Area
Attributes

--------------------------------------------------------------------------------------------
```

```
----
  AND3_X4          FrPK45_lib_v1.0       1.330            8            10.640           c
  SDFF_X1          FrPK45_lib_v1.0       6.916            10           69.160           sq
  AOI22_X4         FrPK45_lib_v1.0       2.128            9            19.152           c
  NAND3_X1         FrPK45_lib_v1.0       1.064            87           92.568           c
```

report_slack_histogram

```
report_slack_histogram
[-help]
[-compact]
[-early]
[-in_file string]
[-late]
[-max_slack float]
[-min_slack float]
[-out_file string]
[-plus_non_violating]
[-retime {aocv | ssta | path_slew_propagation | aocv_path_slew_propagation}]
[-retime_mode {path | exhaustive}]
[-step float]
[-target_slack float]
[-view string]
```

Generates a histogram of end-point slacks.

Parameters

`-help`	Prints out the command usage.
`-compact`	Specifies the compact column report format. This format removes any histogram rows, which have a count of 0.
`-early`	Specifies early mode analysis for computing histogram.
`-in_file` *string*	Specifies input filename to be used for generating histogram. When the `-in_file` parameter is specified, the `report_slack_histogram` command generates reports using data from the specified file. *Default*: The software will generate this data with default max_paths limit.
`-late`	Specifies late mode analysis for computing histogram.
`-max_slack` *float*	Specifies the upper slack limit for the generated report. *Default:* 0
`-min_slack` *float*	Specifies the lower slack limit for the generated report. *Default:* Worst negative slack
`-out_file` *string*	Specifies the result filename. *Default:* Displays the report on the Tempus console.
`-plus_non_violating`	Specifies to include the non-violating slack values.
`-retime {aocv | ssta | path_slew_propagation | aocv_path_slew_propagation}`	Re-analyzes the specified set of paths using the specified method.
`-retime_mode {path | exhaustive}`	Re-analyzes the specified set of paths using the specified method.
`-step` *float*	Specifies the precision for slack histogram. *Default:* `0.5ns`
`-target_slack` *float*	When specified, the slack histogram is centered around the defined target slack value.
`-view` *string*	Writes out slack histograms for the specified view. If you do not specify this option, the command outputs details for all the views.

Examples

- The following command reports the lower slack limits along with the non-violating slack values:

```
report_slack_histogram -compact -min_slack -10 -plus_non_violating

-------------------------------------------
| Slack Range (ns)     |  Count  |     Sum  |
+----------------------+---------+----------+
| ( -9.949 ~  -9.449]  |     15  |       15 |
| ( -9.449 ~  -8.949]  |     12  |       27 |
| ( -8.949 ~  -8.449]  |     18  |       45 |
| ( -8.449 ~  -7.949]  |     16  |       61 |
-------------------------------------------
```

- The following command reports the upper slack limits for the specified view:

```
report_slack_histogram -max_slack -20 -step 0.5 -view func_fast_RCMAX

-------------------------------------------
| Slack Range (ns)     |  Count  |     Sum  |
+----------------------+---------+----------+
| (-21.182 ~ -21.000]  |      7  |        7 |
| (-21.000 ~ -20.500]  |      2  |        9 |
| (-20.500 ~ -20.000]  |      5  |       14 |
-------------------------------------------
```

- The following command shows a report for the specified target slack value:

```
report_slack_histogram -target_slack -21

-------------------------------------------
| Slack Range (ns)     |  Count  |     Sum  |
+----------------------+---------+----------+
| (-22.239 ~ -22.000]  |      7  |        7 |
| (-22.000 ~ -21.500]  |      2  |        9 |
| (-21.500 ~ -21.000]  |      5  |       14 |
| (-21.000 ~ -20.500]  |      2  |       16 |
| (-20.500 ~ -20.000]  |      3  |       19 |
-------------------------------------------
```

Related Commands

- write_global_slack_report

report_socv_interconnect_variation

```
report_socv_interconnect_variation
[-help]
[-delay_corner string]
[-power_domain string]
```

Reports the RC variation multiplier data provided in the SOCV files.

Parameters

`-help`	Prints out the command usage.
`-delay_corner string`	Specifies the name of the delay corner.
`-power_domain string`	Specifies the name of the power domain.

Examples

- The following command generates the below output for the specified delay corner:

```
report_socv_interconnect_variation -delay_corner delay_corner_FUNC_PVT_ss_0p72_m40_rcmax

Delay Corner: delay_corner_FUNC_PVT_ss_0p72_m40_rcmax

-------------------------------------------------------------------------------------------------
-----------------------------------------------
| Power Domain                                          | Voltage  | Data   | Data   |
Data   | Data   | Clock  | Clock  | Clock  | Clock  |
|                                                       | (in V)   | Rise   | Rise   |
Fall   | Fall   | Rise   | Rise   | Fall   | Fall   |
|                                                       |          | Early  | Late   |
Early  | Late   | Early  | Late   | Early  | Late   |
-------------------------------------------------------------------------------------------------
-----------------------------------------------
| -                                                     | 0.72     | -      | -      |
-      | -      | -      | -      | -      | -      |
-------------------------------------------------------------------------------------------------
-----------------------------------------------
| PD_WIFICORE                                           | 0.72     | -      | -      |
-      | -      | -      | -      | -      | -      |
-------------------------------------------------------------------------------------------------
-----------------------------------------------
```

```
| PD_MXRAMWRAP_VCCD                                  | 0.72     | -      | -      |
-      | -      | -      | -      | -      | -      |
-------------------------------------------------------------------------------------------------
-----------------------------------------------
```

- The following command generates the below output for the specified power domain:

```
report_socv_interconnect_variation -power_domain _internal_VDD_0P9_BT2XTAL_GND_BT2XTAL_

Delay Corner: delay_corner_FUNC_PVT_ss_0p72_m40_rcmax

-------------------------------------------------------------------------------------------------
-------------------------------
| Power Domain                           | Voltage  | Data   | Data   | Data   | Data   |
Clock  | Clock  | Clock  | Clock  |
|                                        | (in V)   | Rise   | Rise   | Fall   | Fall   |
Rise   | Rise   | Fall   | Fall   |
|                                        |          | Early  | Late   | Early  | Late   |
Early  | Late   | Early  | Late   |
-------------------------------------------------------------------------------------------------
-------------------------------
| _internal_VDD_0P9_BT2XTAL_GND_BT2XTAL_ | 0.72     | -      | -      | -      | -      |
-      | -      | -      | -      |
-------------------------------------------------------------------------------------------------
-------------------------------
```

report_socv_library

```
report_socv_library
[-help]
[-categories string]
[-coverage_verbose]
[-list_not_annotated]
[-library_set string | { -delay_corner string [ -power_domain string] }| { -view string [-
power_domain string]}]
[-libraries string | -lib_cells string]
{-coverage}
```

Reports the coverage of SOCV library(s) loaded in the design.

Parameters

`-help`	Prints out the command usage.
`-categories` *string*	Allows you to control what types of table data to check delay arcs, transition arcs, and check arcs. *Default:* `all`
`-coverage`	Controls for reporting coverage library data.
`-coverage_verbose`	Generates a detailed report table per cell.
`-delay_corner` *string*	Checks all the libraries defined in the specified library set.
`-lib_cells` *string*	Specifies the library cells. The allowed format is \u201clibrary/cell\u201d name or pattern or collection. This will typically be done to drill-down debugging of problems.
`-libraries` *string*	Allows you to select the library. If `-library_set`, `-delay_corner`, or \u2013view is not specified then all libraries matching the library name across the MMMC configuration will get checked.
`-library_set` *string*	Checks all the libraries defined in the specified library set.
`-list_not_annotated`	Allows you to control the types of table data to check delay arcs, transition arcs, and check arcs. *Default*: `all`
`-power_domain` *string*	Checks the specified power domain.
`-view` *string*	Checks all the delay corners associated with the specified view.

Examples

- The following command displays the SOCV configuration summary report, which shows the coverage type for the specified libraries:

```
report_socv_library -coverage -delay_corner delay_corner_slow_RCMIN -categories
transition

|-------------------------------------------------------------------------------------------|
|                                  SOCV Configuration Summary                               |
|-------------------------------------------------------------------------------------------|
```

```
|Library Set|Libraries         |Arc       |Cells           |Cells           |Cells           |
|-------------------------------------------------------------------------------------------|
|           |                  |Type      |Full Coverage   |Partial Coverage|No Coverage     |
|-------------------------------------------------------------------------------------------|
|libset_fast|FrPD45_lib_v1.0|transition| 0(0.00%)       | 0(0.00%)       | 104(100.00%)   |
|           |-------------------------------------------------------------------------------|
|           |pllclk            |transition| 0(0.00%)       | 0(0.00%)       | 1(100.00%)     |
|           |-------------------------------------------------------------------------------|
|           |USERLIB           |transition| 0(0.00%)       | 0(0.00%)       | 1(100.00%)     |
|-------------------------------------------------------------------------------------------|
|libset_slow|FrPD45_lib_v1.0|transition| 0(0.00%)       | 0(0.00%)       | 104(100.00%)   |
|           |-------------------------------------------------------------------------------|
|           |pllclk            |transition| 0(0.00%)       | 0(0.00%)       | 1(100.00%)     |
|           |-------------------------------------------------------------------------------|
|           |USERLIB           |transition| 0(0.00%)       | 0(0.00%)       | 1(100.00%)     |
```

- The following command displays the SOCV configuration summary report, which shows the coverage library data:

```
report_socv_library -coverage -coverage_verbose
|-------------------------------------------------------------|
|                  SOCV Configuration Summary                 |
|-------------------------------------------------------------|
|Library Set|Libraries         |Lib Cell    |Types     |Status    |
|-------------------------------------------------------------|
|libset_slow|FrPD45_lib_v1.0|XOR2_X2     |delay     |none      |
|           |                  |            |----------------------|
|           |                  |            |transition|none      |
|           |                  |            |----------------------|
|           |                  |-----------------------------------|
|           |                  |XOR2_X1     |delay     |none      |
|           |                  |            |----------------------|
|           |                  |            |transition|none      |
           |                  |-----------------------------------|
```

```
|           |-------------------------------------------------|
|           |USERLIB           |ram_256x16A |delay     |none   |
|           |                  |            |----------------|
|           |                  |            |transition|none |
|           |                  |            |----------------|
|           |                  |            |constraint|none |
|           |                  |            |----------------|
|           |                  |---------------------------|
|           |-------------------------------------------------|
|           |USERLIB           |rom_512x16A |delay     |none   |
|           |                  |            |----------------|
|           |                  |            |transition|none |
|           |                  |            |----------------|
|           |                  |            |constraint|none |
|           |                  |            |----------------|
```

report_statistical_timing_derate_factors

```
report_statistical_timing_derate_factors
[-help]
[-derate_cell]
[-derate_cell_check]
[-derate_net]
[-max_slack float]
[-path_based_ssta]
[-path_group groupname_list]
[-ssta_view ssta_view_name]
[-sta_view sta_view_name]
[-type {arrival_time_based | slack_time_based}]
[-early | -late ]
[-clock_factor | -data_factor]
[-max_paths integer | -max_points integer]
```

Reports design-specific OCV factors. The scaling factor (hold or setup) is computed either by using the arrival time or slack time approach.

Parameters

`-clock_factor | -data_factor`	Specifies if the user-defined factor value is for clock or data. If you specify the `-clock_factor` parameter, then the `-data_factor` parameter is calculated by the software, and vice versa. These parameters are applicable to type `slack_time_based`. **Note:** The `-clock_factor` parameter and the `-data_factor` parameter are mutually exclusive. You cannot specify them together.
`-derate_cell`	Applies user-specified scaling factors on cell delays.
`-derate_cell_check`	Applies user-specified scaling factors on timing checks.
`-derate_net`	Applies user-specified scaling factors on net delays.

`-late	-early`	Specifies early or late mode analysis. Default is `late`.
`-max_paths`	Reports the specified number of worst paths in the design, regardless of the endpoint.
`-max_points` *integer*	Reports the worst path that is found to each endpoint up to the number specified by the `-max_points` option.
`-max_slack` *float*	Reports only those paths with slack equal to or less than the specified float value.
`-path_based_ssta`	Enables generation of the statistical OCV factor using path based SSTA analysis.
`-path_group` *groupname_list*	Considers only the path contained in the groups in the specified *groupname_list* for computing the statistical timing derate factor.
`-ssta_view` *ssta_view_name*	Specifies view for SSTA run.
`-sta_view` *sta_view_name*	Specifies view for STA run.
`-type arrival_time_based	slack_time_based`	Specifies if the OCV factor needs to be generated using the arrival time or slack time approach. In the arrival time approach, the SSTA arrival time values are used to compute the statistical OCV factor for clock and data. In the slack time approach, the STA and SSTA slacks are used to generate the statistical OCV factor for either clock or data depending on whether you specify the factor for data or clock. *Default:* `arrival_time_based` You can use the `-type arrival_time_based` parameter with the following parameters only: • `-late` • `-early` • `-path_based_ssta`

	You can use the `-type slack_time_based` parameter with the following parameters only: • `-ssta_view` • `-sta_view` • `-late` • `-early` • `-clock_factor` • `-data_factor` • `-derate_cell` • `-derate_net` • `-derate_cell_check` • `-max_paths` • `-max_slack`

Examples

- The following command enables SOCV analysis mode:

  ```
  tempus> set_db timing_analysis_socv true
  ```

  The following command reports OCV factors that are generated using the arrival time:

  ```
  tempus> report_statistical_timing_derate_factors -type arrival_time_based
  ------------------------------------------------------
  p out waveforms (MEM=2971.51 CPU=0:00:00.0 REAL=0:00:00.0)

      ------------------------------------------------
          SSTA Recommended Timing Derate Factors
      ------------------------------------------------
       min early (D)     avg     max late (C)     avg
                        early                     late
                         (D)                      (C)
      ------------------------------------------------
       1.000 (ff1/D)    1.000    1.000 (ff1/CP)   1.000
      ------------------------------------------------
  ```

```
-----------------------------------------------
    SSTA Recommended Timing Derate Factors
-----------------------------------------------
  min early (C)    avg     max late (D)    avg
                  early                    late
                   (C)                     (D)
-----------------------------------------------
 1.000 (ff1/CP)   1.000   1.000 (ff1/D)   1.000
-----------------------------------------------
```

- The following example computes the factors for setup analysis using the arrival time based approach:
  `report_statistical_timing_derate_factors -type arrival_time_based -late`
- The following example computes the data factor for setup analysis using the slack time based approach applying a user defined clock factor of 0.95:
  `report_statistical_timing_derate_factors -type slack_time_based -ssta_view viewname -sta_view viewname -clock_factor 0.95`
- The following example computes the clock factor for hold analysis using the slack time based approach applying a user defined data factor of 1.05:
  `report_statistical_timing_derate_factors -type slack_time_based -ssta_view viewname -sta_view viewname -data_factor 1.05 -early`

report_timing

```
report_timing
[-help]
[-analysis_summary_csv_extended_file string]
[-analysis_summary_csv_file string]
[-analysis_summary_file string]
[-check_clocks]
[-check_type {setup hold clock_gating_setup clock_gating_hold clock_gating_pulse_width data_setup
data_hold recovery removal pulse_width clock_period clock_separation skew no_change_setup
no_change_hold max_delay min_delay}]
[-debug {unconstrained | time_borrow | cppr_point | constraint_arc| context | wire_variation |
vt_skew}]
[-derate_summary]
[-gui]
[-fields column_list]
[-group groupname_list]
[-hpin]
[-max_delay float]
[-max_slack float]
[-min_slack float]
[-nets]
[-output_format {text |  csv | gtd | binary}]
[-path_exceptions {applied | ignored | all}]
[-path_type {endpoint | summary | full | full_clock}]
[-retime {aocv | path_slew_propagation | aocv_path_slew_propagation}]
[-retime_delaycal_pins pin_name]
[-retime_format {manual | retime_compare | retime_replace}]
[-retime_mode {path | exhaustive}]
[-skip_io_paths]
[-split_delay]
[-unique_pins]
[-view viewName]
[> <filename[.gz]>]
[>> <filename[.gz]>]
[-late | -early]
[-rise | -fall]
[-begin_end_pair | [[ -max_paths integer] [-nworst integer]]]
[-unconstrained | -point_to_point]
[-collection | -tcl_list ]
[-not_through object_list | -not_rise_through object_list | -not_fall_through object_list]
[ [-from object_list | -from_rise object_list | -from_fall object_list] [-through object_list | -
through_rise object_list | -through_fall object_list] [-to object_list | -to_rise object_list | -
to_fall object_list] ]
```

Generates a timing report that provides information about the various paths in a design.

The reports typically contain data on the delay through the entire path. The start node and the end node of each

path is identified.

The timing report contains the following information:

- The slack times of the arriving signal at the end node
- The start node
- The associated transitions
- The signal required times and the actual signal arrival times
- Summary of propagated vs ideal status of launching and capturing clocks
- Any phase shifts applied when evaluating timing checks
- Any CPPR values applied to timing check evaluation

The `report_timing` command output accommodates both the instance pin names and net names in the same column. Since both instance pin names and net names can be long, to save screen space alternating instance pin and net names (depending on whether the row displays gate delay or net delay) are reported in a single column.

When the `timing_report_timing_header_detail_info` timing attribute is set to `extended`, the software generates timing reports with additional header information, that includes the following details:

- Analysis mode - best-case worst-case (BCWC) or on-chip variation (OCV)
- Active views (displayed only in MMMC mode)
- Clock re-convergence pessimism (CPPR)
- Latch slack mode
- Specified clock propagation value
- Delay calculation
- Specified delay calculation engine

When the `timing_report_group_based_mode` attribute is set to `true`, the paths are grouped by clock domain. In this mode, the `report_timing` command will have the following four types of groups:

- Clock gating group: This is an internal group that reports the clock gating check type paths.
- Asynchronous group: This is also an internal group that reports the asynchronous check type paths.
- Groups for all clocks in a design: These are internal groups that are classified on the basis of each reference clock. The paths belonging to each reference clock are reported separately under each clock name's group. These groups do not honor the clock gating/asynchronous check types. Suppose there are three clocks C1, C2, and C3 in a design, then three groups will be reported for each of these clocks. The endpoint sharing the same reference clock will be part of the same group. In case the endpoint belongs to multiple groups then it will be reported separately in each group.
- Default group: This is an internal group that reports combinational maximum or minimum delay paths.
- User-defined groups: These are user-defined path groups that can be created by users with single or

multiple constraints. These groups honor all the check types and are reported by default.

By default, the `timing_report_group_based_mode` attribute is set to `false`.

Note: This attribute does not honor user-defined path groups, so the paths will still be grouped by different clocks.

The `report_timing` command supports multi-threaded reporting. You can make the following settings to enable multi-threaded reporting:

`set_multi_cpu_usage -localCpu` *`number_of_CPU`*

`set_table_style -no_frame_fix_width [-nosplit]`

Note: When the `timing_allow_input_delay_on_clock_source` attribute is enabled and the `set_input_delay` command is applied to a clock input, the specified delay value will be considered as the source clock latency for clock paths and as regular data path arrival time for clock source paths.

You can use a one-step method for generating merged MMMC view reports from DMMMC master shell. After running the `distribute_views` command, you can use the `report_timing` command to generate reports collectively from all the active distributed views (that is, views available for interactive DMMMC analysis). The merged output report is arranged as per the slack/criticality order across active views.

When the `timing_enable_wire_variation` attribute is set to `true`, the timing reports show wire variation details in the below columns:

- `retime_slew_wire_stddev`
- `retime_delay_wire_stddev`
- `arrival_wire_stddev`

For more details on wire variation, refer to the "Wire Variation Analysis" chapter in the *Tempus Stylus User Guide*.

- Parameters
- Group Path Reporting for User-Defined Groups
- Use of Informational Flags
- Retime Analysis Report Header Flags
- Report Timing Column List
- Reporting Timing Checks Between Various Types of Clocks
- Reports Sections
- Examples
- report_timing -from latch/D Settings
- Related Information

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_timing` parameter. For a detailed description of the command and all of its parameters, use the man command: man `report_timing`.
`{> | >>}` *`filename`*	When specified with `>`, writes the report to the specified file name. If the file already exists, the software will overwrite it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software will concatenate the report at the end of the file. The *`filename`* parameter must be the last argument in the list. The *`filename`* and `-tcl_list` parameters are mutually exclusive; you cannot specify them together. **Note:** To write a compressed report, add the `.gz` extension to the file name. **Note:** The last argument specified with the `report_timing` command is not considered a filename. *Default:* Report is displayed on standard output (not saved).
`-analysis_summary_csv_extended_file` *`string`*	Generates the CSV file to be used for merging. *Data_type*: string, optional
`-analysis_summary_csv_file` *`string`*	Generates the analysis summary CSV file. *Data_type*: string, optional
`-analysis_summary_file` *`string`*	Generates the analysis summary file. *Data_type*: string, optional

`-begin_end_pair`	Reports all the violating paths between unique source and target register pairs. To report other paths, you can specify this parameter with the `-max_slack` parameter. The paths are not sorted by slack - paths with the same source pins are reported adjacent to each other. When specified with the `-from`, `-through`, or `-to` parameter, the software only reports the unique register pair paths that match the path description. In merged group-based reporting mode (that is, when `timing_report_backward_compatible_max_paths_reporting` attribute is set to `false`), the `report_timing -begin_end_pair` command `report_timing -begin_end_pair` will always report the paths grouped by path_groups irrespective of `timing_report_group_based_mode` attribute setting. You can use the `timing_report_begin_end_pair_max_path_limit` attribute to specify the maximum paths limit. When the number of maximum paths to be reported by the `-begin_end_pair` parameter exceeds the specified limit (default is 200000), a warning message will be displayed. Also, some of the paths may not be reported. **Note**: You cannot specify this parameter with the `-max_paths`, `-nworst`, `-unique_pins`, and `-min_slack` parameters. *Data_type*: bool, optional
`-check_clocks`	Generates reports based on timing paths on the clock network instead of the standard timing to data endpoints. This report includes clock paths that end at the reference end of a check or a clock gating end point. *Default*: Data paths and all the other clock paths (for example, a clock path ending at a D pin of a register) are reported. *Data_type*: bool, optional
`-check_type` *check_type*	Reports only the paths that end at the specified

timing check. You can specify the following options:

`setup`, `hold`, `clock_gating_setup`, `clock_gating_hold`, `clock_gating_pulse_width`, `data_setup`, `data_hold`, `recovery`, `removal`, `pulse_width`, `clock_period`, `clock_separation`, `skew`, `no_change_setup`, `no_change_hold`, `max_delay`, and `min_delay`.

- The clock style checks are: `pulse_width`, `clock_gating_pulse_width`, `clock_period`, `skew`, and `clock_separation`.
- In setup analysis mode, the reported check types will be: `setup`, `skew`, `pulse_width`, `clock_period`, `clock_gating_setup`, `clock_gating_pulse_width`, `data_setup`, `recovery`, `clock_separation`, and `no_change_setup`.
- In hold analysis mode, the reported check types will be: `hold`, `clock_gating_hold`, `data_hold`, `removal`, and `no_change_hold`.

If the software is in simultaneous setup and hold analysis mode, all of the check types listed above will be reported.

When the `max_delay` and `min_delay` options are specified, the timing report will display details of timing paths for which the `set_max_delay` or `set_min_delay` constraints are defined.

By default, the following checks are reported:

- `setup`
- `hold`
- `clock_gating_setup`
- `clock_gating_hold`
- `data_setup`
- `data_hold`
- `recovery`

	• `removal` • `no_change_setup` • `no_change_hold` Do not use this parameter with the `-unconstrained`, `-early`, or `-late` parameters. **Note:** The `report_timing` command does not perform skew checks by default. *Data_type*: enum, optional
`-collection`	Returns a collection of timing paths, which is useful for performing Tcl queries on selected timing reports. To generate a collection of paths analyzed by Path-Based Analysis (PBA), you can use the `-retime` parameter. **Note**: This parameter is not supported in DMMMC mode. *Data_type*: bool, optional

`-debug {unconstrained | time_borrow | cppr_point | constraint_arc| context | wire_variation | vt_skew}`	You can specify the following options for debugging: • `unconstrained`: Determines the reasons for a path to be unconstrained. The report output will list all the applicable arcs/constraints that could not constrain the paths, along with the reasons. • `time_borrow`: Reports time borrowing calculations. Time borrowing is the amount of time borrowed by a previous logic. • `cppr_point`: Reports the CPPR point information for the reported paths. • `constraint_arc`: Reports check arc details, such as, constraint arc delay, constraint pins (along with the related pin slews), skewness, user derate, annotation, and when_cond. • `context`: Reports the launch and capture path interface timing for each instance and indicates the block port that shows latency in the MIM flow. • `wire_variation`: Reports the corresponding slack shift details at the end of each timing path report. • `vt_skew`: Reports voltage threshold (VT) skew analysis details. *Data_type*: string, optional
`-derate_summary`	Generates a summary table for AOCV derating of launch and capture paths. *Data_type*: bool, optional
`-fields` *`column_list`*	Formats the report according to the specified *`column_list`*. The *`column_list`* specifies the columns to be displayed in the timing report and the order in which they appear. The valid field columns are: adjustment, abs_delay_err, abs_transition_err, annotation, aocv_adj_stages, aocv_derate, aocv_weight, arc, arrival, arrival_mean,

arrival_sigma, cell, delay, delay_mean, delay_sigma, direction, edge, fanin, fanout, flags, hpin, incr_delay, instance, instance_location, latch_window, load, module, net, pct_delay_err, pct_slew_err, phase, phys_info, pin, pin_load, pin_location, power_domain, required, retime_delay, retime_delay_mean, retime_delay_sigma, retime_mis_delta, retime_mis_derate, retime_slew, retime_transition_mean, retime_transition_sigma, stage_count, transition, transition_mean, transition_sigma, socv_derate, spice_arrival, spice_delay, spice_slew, stolen, timing_point, total_derate, user_derate, voltage, vt_skew_derate, when_cond, wire_load, wlmodel

For example:

```
-fields {hpin cell delay required arrival required edge}
```

See Report Timing Column List for details on the above options.

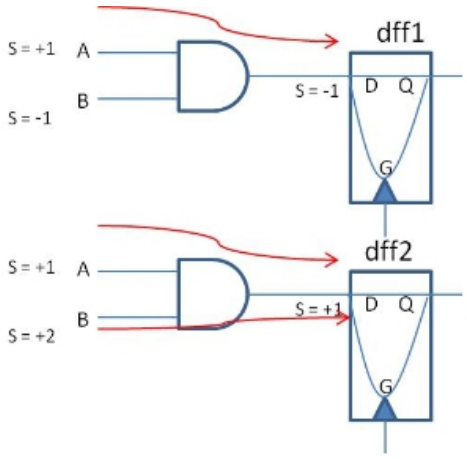
Few notes:

- The default net format (with the `-net` parameter) for the full path will be:
  ```
  {hpin edge net cell delay arrival required}
  ```
- The `required` column will not be displayed, when the `-unconstrained` parameter is specified.
- The `adjustment` column is automatically added to the timing report. To disable this feature you can set the `timing_report_generated_clock_info` attribute to `false`. By default this global variable is set to `true`.
- You can use a combination of the `-fields` and `-tcl_list` parameters to integrate the timing reports into your Tcl scripts. You can use the `-from` parameter to limit the number of paths reported, and to find specific paths in the design.

	• The `-format` option cannot be used with the `-path_type summary` option. • You can manage the retiming columns with the `timing_report_retime_formatting_mode` global variable. • If you specify the `report_timing -split_delay` parameter, then the timing report format will change to: `{timing_point flags edge cell power_domain fanout transition delay arrival}` • Informational tags are displayed for reporting additional details with the `flags` option. For details on the '`Flags`' column, refer to the section on Use of Informational Flags. *Default*: `{timing_point flags arc edge cell power_domain fanout transition delay arrival}` *Data_type*: string, optional
`-from | -from_rise | -from_fall` *`object_list`*	Reports paths starting from the object(s) specified in the *`object_list`*. The supported objects are pins, ports, and clocks. Using `-from_rise` (or `-from_fall`) parameter specifies that the rising (or falling) edge of the signals on the objects in the *`object_list`* are the start of the paths. This parameter also reports on source clock waveform(s). You can use this parameter with the `-through` and `-to` parameters for specifying particular paths in the design. *Data_type*: string, optional
`-group` *`groupname_list`*	Reports paths contained in the groups specified in the *groupname_list*. *Data_type*: string, optional
`-gui`	Displays the timing report in a GUI window. *Data_type*: bool, optional
`-hpin`	Prints all the hierarchical crossings of an arc under the `hpin` column. *Data_type*: bool, optional

`-late	-early`	Reports for early paths (hold checks) or late paths (setup checks). If you specify a check type, setup or hold, with the `timing_analysis_check_type` attribute, the `report_timing` command reports the specified check by default. **Note:** The `-early` and `-late` parameters are not mutually exclusive; you can specify them together. *Default*: `-late` *Data_type*: bool, optional
`-max_delay` *float*	Specifies the path delay limit for unconstrained paths. *Data_type*: string, optional
`-max_paths` *integer*	Reports the specified number of worst paths in the design, with maximum of nworst (`-nworst` parameter) paths to any single endpoint (default 1). When `timing_report_group_based_mode` (group-based mode) attribute is set to `true`, the `-max_paths` parameter specifies the number of worst paths reported per clock group. *Default*: 1 *Data_type*: int, optional

`-max_slack` *float*	Reports only those paths with slack less than the specified maximum slack value. The `-max_slack` parameter reports paths that fall within the specified range. A positive slack value indicates that timing was met and a negative slack value indicates a timing violation. When the `-retime_mode exhaustive` option is specified, the `-max_slack` value is set to `0`. The `-max_slack` parameter cannot be used with the `-unconstrained` option. *Data_type*: string, optional

`-min_slack` *float*	Reports only those paths whose slack is greater than the specified minimum slack value. You can generate a timing report showing paths with slack greater than the specified slack using the `-max_paths` parameter. For example, to report all paths with slack greater than 2ns, you can use the following command: `min_slack 2.0 -max_paths 1000` In this case the maximum number of paths reported will be 1000. When the `-min_slack` parameter is specified, the total numbers of paths processed will be as determined by the `-max_paths`/`-nworst` settings and only those paths satisfying the `-min_slack` criteria will be reported. The software will not process paths beyond the specified `max_paths`/`nworst` limit, even if the `min_slack` criteria is satisfied. The `-min_slack` cannot be used with the `-unconstrained` parameter. In the below diagram, if `-min_slack` is 0 and `-max_paths 4 -nworst 2` is specified, then only 3 paths will be reported; path with slack value of -1 will be ignored. Here, even if there is another path with slack that is greater than or equal to 2, then it will not be reported because worst max_paths have been processed.  *Data_type*: string, optional

`-nets`	Adds a row to the "Timing Point" field column to display the net names. **Note**: The `-split_delay` and `-nets` parameters can be used together. *Data_type*: bool, optional
`{-not_through | -not_rise_through | -not_fall_through}` *object_list*	You can specify one of the following options: • `-not_through`: Reports paths that do not traverse through the specified nets, ports, or pins of a cell. By default, this parameter will be ignored if an object has already been specified. In this case the parameter shows the following behavior: ◦ If a collection of all the input ports has been specified (using the `all_inputs` command), then the report will show paths to the inout part, and paths starting from the internal part of inout will be excluded. ◦ If a collection of all the output ports has been specified (using the `all_outputs` command), then the report will show paths starting from the internal part of inout, and paths ending at inout part will be excluded. ◦ If any other collection (not specified with `all_inputs` or `all_outputs` command) is specified, then paths to both the ports will not be reported. • `-not_rise_through`: Does not report paths with rise transition. • `-not_fall_through`: Does not report paths with fall transition. *Data_type*: string, optional

`-nworst` *integer*	Specifies the maximum number of paths which can be reported for a particular endpoint (the total number of paths reported is specified by the `-max_paths` parameter). When `timing_report_group_based_mode` attribute is set to `true` (group-based mode), the `-nworst` parameter specifies the maximum number of paths for a particular endpoint per clock group. You can use the `-nworst` parameter to report all the checks at an endpoint (or use `-check_type`) to report a specific check. Default: 1 *Data_type*: int, optional
`-output_format {text | csv | gtd | binary}`	Generates a detailed timing report in the specified format - text, CSV, GTD, or binary. *Data_type*: enum, optional

`-path_exceptions {applied | ignored | all}`	Includes information of path exceptions applied and considered for the reported path. You can specify the following options: • `applied`: Reports the honored exception on the reported path. • `ignored`: Reports the ignored exceptions on the reported path. • `all`: Reports both applied and ignored set of exceptions on the reported. **Note:** Analyzing all the exceptions for a path will cause increase in runtime. If multiple from, to, or through pins are specified, the exception report for that path will show only those pins that are part of the current path. **Note**: If a path is already unconstrained, then all further path exceptions applied on that particular path will be reported as ignored exceptions. The `-path_exceptions` parameter also reports -start/-end options with multi-cycle paths. Without the start or end specification, by default, the setup check is relative to the end clock and the hold check is relative to the start clock. The `report_timing -path_exceptions all` command will report all the non-default -start/-end specifications. You cannot use this parameter with the following parameters: • `-point_to_point` • `-collection` • `-path_type` • `-tcl_list` Also, you cannot use this parameter with block-based SSTA. **Note:** Analyzing all the exceptions for a path will increase the runtime. *Data_type*: enum, optional

`-path_type {endpoint | summary | full | full_clock}`	The `-path_type` parameter allows you to choose the report format by path type. You can specify one of the following options: • `endpoint`: Generates an endpoint report for each path containing the ID, slack, endpoint, and group view. This option generates a very fast report. • `summary`: Generates a summary report for each path consisting of a start point, endpoint, cause, slack, arrival time, required time, and phase. The -fields option does not have any impact on this report format. • `full`: Generates a report that displays the full path with accompanying required time and slack calculation. This is the default path type. You can control the report format using the `-fields` option. • `full_clock`: If the reported path ends at a timing check, the software reports the full clock path (Other End Path), in addition to the full data path (Timing Path). *Data_type*: enum, optional

`-point_to_point`	Traces the worst delay path between a pair of from-to pins. The paths reported with this parameter only include the cumulative delay of the path. You must specify either the `-from` or `-to` parameter, to define the begin and end points of the path. If you do not specify a `-from` pin, the standard definition of the begin point of a path applies. If you specify a `-from` pin, it is treated as the begin point of the path. If you specify a `-from` pin that is not a valid begin point, the arrival time of the first pin starts from 0. If you do not specify a `-to` pin, the standard definition of the end point applies. If you specify a `-to` pin, it is treated as the end point, and only those paths ending in `-to` pins are reported. If you specify more than one -to pin, the worst path to each pin-to-pin is reported. When from-to pins are specified, the software will trace across the trigger arcs and report a path, if there is a clock reaching the CK pin of the trigger arc. However, if there is a begin or end point present between the `-from` and `-to` pins, the software will not trace beyond it. If there is no clock reaching the CK pin of the trigger arc, the software cannot trace across the trigger arc, and no path is reported. Additionally, clock path delay is included only when the clock is in propagated mode. **Note:** The `-point_to_point` and `-through` parameters are mutually exclusive; you cannot specify them together. **Note**: The `-point_to_point` parameter supports a single `-from` and `-to` pin; a list of pins is not supported. *Data_type*: bool, optional

`-retime_delaycal_pins` *pin_name*	Reports the PBA delay calculation details for timing arcs that fanin to the user-specified to-pin list. You can specify multiple pins in the pin list. The delay calculation report will be printed at the end of each timing path for the arcs, which are retimed in that path. The PBA delay calculation report format is similar to the `report_delay_calculation` output in GBA mode, however, the results may be vary because this output is based on path-based analysis. When a cell output pin list is provided, the generated report is for the cell arcs; the cell input pin list generates the net arc report. This parameter is supported for PBA reporting only. *Data_type*: string, optional
`-retime_format {manual | retime_compare | retime_replace}`	Allows you to set the format of timing reports for retime related fields. You can specify one of the following options: • `manual`: Does not change the `report_timing` format. • `retime_compare`: If GBA columns are present in the report format, then the corresponding retime options are added. If PBA columns are already present, then there is no change in the format. • `retime_replace`: If GBA columns are present in the report format, then the GBA columns are replaced by the corresponding PBA columns. If PBA columns are already in the report, then there is no change in the format. **Note**: The timing report format can be controlled with the `timing_report_retime_formatting_mode` attribute. The `report_timing` command setting has precedence over this attribute. *Data_type*: enum, optional

`-retime_mode {path | exhaustive}`	Reanalyzes the specified set of paths using the specified method. You can use one of the following options: • `path`: Reanalyzes the specified set of paths. By default, the software uses the `-retime_mode path` setting, when the `-retime path_slew_propagation` option is specified. • `exhaustive`: Enables exhaustive path-based analysis mode. In this mode, the software will exhaustively examine nworst paths of the selected endpoints and then select the true worst path to be reported, based on retimed slacks of all the examined paths. When you use this option, reporting can be runtime intensive as the software needs to run an exhaustive search for true worst paths. Therefore, it is recommended to use this option in the later phases of the signoff stage when few violations are left for analysis. *Default*: `path` *Data_type*: enum, optional
`-retime {aocv | path_slew_propagation | aocv_path_slew_propagation}`	Reanalyzes the specified set of paths using path-based analysis (1 path per endpoint). **Note:** This feature is not meant to be run on all the paths of a design. You can specify the following options: • `aocv`: Computes AOCV stage count based on specific paths and applies the AOCV derate values to GBA (graph-based analysis) delays. This option can be used with the following parameters only: ◦ `-rise/-fall` ◦ `-early/-late` ◦ `-check_type` ◦ `-max_paths` ◦ `-begin_end_pair`

  - `-from/-from_rise/-from_fall`
  - `-through/-through_rise/ -through_fall`
  - `-to/-to_rise/-to_fall`
  - `-rise/-fall`
  - `-early/-late`
  - `-check_type`
  - `-max_paths`
  - `-begin_end_pair`
  - `-from/-from_rise/-from_fall`
  - `-through/-through_rise/ -through_fall`
  - `-to/-to_rise/-to_fall`
  - `-path_exceptions`
  - `-path_type`
  - `-max_slack`
  - `-min_slack`
  - `-view`
  - `-derate_summary`
  - `-nworst`
- `path_slew_propagation`: Re-evaluates the given set of graph-based analysis (GBA) paths by recalculating the delay values based upon actual propagated slew across the path. In this case the AOCV derates are the ones that are reported during graph-based analysis.

The default timing analysis uses worst-case slew propagation to calculate the delays at each stage. As a result, the timing analysis results can be pessimistic.

Note: This feature should be run only on paths that do not meet the slack margin criteria to help identify the actual failing paths from those that are simply failing due to the conservative delay calculation.

The signal integrity incremental delays can be recomputed for the entire path. You can use the following command to perform path-based signal integrity analysis:

```
report_timing -retime path_slew_propagation
```

In the `report_timing` command output, the `Retime Incr Delay` column reports recomputed signal integrity delays.

The `-retime` parameter also supports retiming a collection of paths. This parameter does not retime an already retimed path collection. This feature works in graph-based analysis (GBA) mode for path collections. You can use the following use model to retime collections:

```
report_timing $collection -retime
path_slew_propagation
```

A path collection stores context dependent data, that is, a delay property in GBA will reflect delay data of the timing path report generated in GBA mode. The same property in PBA will reflect the retime delay data of the timing path report in PBA mode.

The delay column in timing reports is context dependent, when the `timing_report_retime_formatting_mode retime_replace` attribute is used.

Note: For memory optimization purposes, both the delays will not be stored in the same path collection. You can use this setting for comparison purposes.

- `aocv_path_slew_propagation`: Re-evaluates the given set of graph-based analysis (GBA) paths by using path-based AOCV derates and delays using actual path- based propagated slew. This option is a combination of `-aocv` and `-path_slew_propagation` options.

For retime report header details, see Retime Analysis Report Header Flags.

Data type: enum, optional

`-skip_io_paths`	Specifies whether the input/output paths are to be ignored. *Data_type*: bool, optional
`-rise | -fall`	Reports the path with the specified edge on the endpoint. If an endpoint is specified using the `-to_rise` (or `-to_fall`) parameter, the `-rise` (or `-fall`) parameter is ignored and paths with edge specified by `-to_rise` (or `-to_fall`) are reported. The `-rise` option applies to endpoints only. *Data_type*: bool, optional
`-split_delay`	Adds a row for the net arc. **Note**: The `-split_delay` and `-nets` parameters can be used together. *Data_type*: bool, optional
`-tcl_list`	Produces the report in Tcl list format instead of a tabular format. This is useful for integrating timing with custom Tcl functions and also for customizing report generation. The `-tcl_list` and *`filename`* parameters are mutually exclusive; you cannot specify them together. *Data_type*: bool, optional
`-through | -through_rise | -through_fall` *`object_list`*	Reports paths that pass through pin(s) or net(s) specified by the *object_list*. Any number of `-through` pins or nets can be specified. Using the `-through_rise` (or `-through_fall`) parameters specifies that the paths go through the rising (or falling) edge of the signals on the pins/nets in the *object_list*. The *object_list* is a logical OR function. The resulting path may pass through any of the pins or nets in the `-through` *object_list*. To force the report to pass through multiple pins or nets, separate `-through` statements are needed. You can use these parameters with the `-net` parameter. *Data_type*: string, optional

`-to	-to_rise	-to_fall` *object_list*	Reports paths leading to the pin(s) specified by the *object_list*. The pins in the *object_list* can be either pins on the design boundary (ports) or pins on an instance. Only one list of -to pins can be specified per report. Using `-to_rise` (or `-to_fall`) specifies that the rising (or falling) edge of the signals on the pins in *object_list* are at the end of the paths. This parameter also reports on target clock waveform(s). The `-to_rise` option applies to endpoints as well as intermediate pins. *Data_type*: string, optional
`-unconstrained`	Reports only the unconstrained paths (paths with no slack). Each signal arriving at the path end node which does not have a matching required time, results in an unconstrained path. The `report_timing` command without the `-unconstrained` parameter reports only constrained paths. If no constrained path is found, there may be unconstrained paths or paths may not exist. **Note**: The `-min_slack` and `-max_slack` cannot be specified with the `-unconstrained` parameter. See Examples for the conditions under which a path is reported as constrained or unconstrained. *Data_type*: bool, optional
`-unique_pins`	Reports paths that pass through unique set of pins. When this parameter is specified, in group-based mode for a given begin-end point pair, the software will consider all different launch clock paths for reporting unique paths. If for a given begin-end pair and a given capture clock, there is a possibility of different launch clock paths, then each such path will be reported. **Note**: Using this parameter may significantly affect runtime and memory usage, if the number of reported paths is high.

`-view viewName`	Generates report for the specified analysis view. By default, all the active views are analyzed. You must be in multi-mode multi-corner (MMMC) analysis mode in order to use this parameter. To create a view, you use the `create_analysis_view` command. To set an active view for analysis, you use the `set_analysis_view` command. The `report_timing` command performs block-based analysis in MMMC mode. By default, the command reports the worst endpoint(s) across all views. You can use the `-view` parameter to report for individual views. *Default:* Reports the worst path to each endpoint across all the analysis views. *Data_type*: bool, optional

Group Path Reporting for User-Defined Groups

- A user-defined group is created with the same name as any of the defined reference clocks and with the same constraint as for the "to" clock. For example, `group_path -name CLK -to CLK`.
  In this case only one group named `CLK` will be displayed in the timing report. This will be the same default group with reference clock `CLK`. Note that default groups - clock gating and asynchronous checks - will not be honored for this group.
- A user-defined group is created with the same name as any of the reference clocks but with different constraint as specified for the "to" clock. E.g., `group_path -name CLK -from pin1 -to CLK`.
  In this case the two groups named `CLK` will be displayed in the timing report. One group will be the same as the default group with reference clock `CLK` (which will not honor clock gating or asynchronous checks) and the other will be the user-defined path group with name `CLK`, which will honor all the checks.

Use of Informational Flags

Informational tags are used providing additional details. The following informational tags are supported:

Flags	Description
(i)	Net is ideal
(P)	Instance is preserved / don’t_touch

(p)	Instance is preserved / don't_touch but may be resized

The output format uses the flag keys on the timing report and adds a description at the end of the report for reference. A sample output is as shown below:

```
#------------------------------------------------------------------------------
# Timing Point  Delay (ns) Latch  Window   Arrival   Arc   Flags   Adjustment
# (ns)
#------------------------------------------------------------------------------
f0/CP           -           -    1.000000            CP       -      -
f0/Q            0.254100    -    1.254100            CP->Q    -      -
b32/Z           0.598446    -    1.852546            I->Z     -      -
...
b4/Z            0.086940    -    2.502181            I->Z     -      -
f1/D            0.000000    -    2.502181            D        <<<    -
#------------------------------------------------------------------------------

(P) : Instance is preserved
(p) : Instance is preserved but may be resized
(i) : Net is ideal.
(b) : Timing paths are broken.
```

Retime Analysis Report Header Flags

AOCV(default)	Indicates that the selected retime analysis includes recalculation of path depths and AOCV derates. The AOCV retime is enabled with the `-retime aocv` or `aocv_path_slew_propagation` parameter. The AOCV stage counting mode (specified using the `timing_aocv_analysis_mode` attribute) is indicated in parenthesis. The stage counting modes are given below: • `launch_capture (default) (Default)` • `combine_launch_capture (Combine)` • `separate_clock_data (Separate)` • `clock_only (Clock)`

Corr	Indicates that the current SOCV accuracy mode considers correlation of variation between stages. Starting with SOCV accuracy mode `medium` (specified with the `delaycal_socv_accuracy_mode` attribute), it is assumed that some portion of the variation is not completely uncorrelated random variation, so the software tracks a correlated and uncorrelated component for each of the applicable moments.
Data Clock	Indicates the portion of the path that has been retimed. By default, only the data path is retimed. If the `timing_disable_retime_clock_path_slew_propagation` attribute is set to `false`, then the clock path will also be retimed, and “Clock” will be added to the retime status.
EWM	Indicates that equivalent waveform model is used (`delaycal_equivalent_waveform_model` attribute).
GBA COLL	Shows a collection of paths for which retime analysis is not performed. This can be controlled with the When the `report_timing -path_type exhaustive -collection` command. By default, the `report_timing -collection` parameter does not show GBA and PBA slack values separately.
GP	Indicates that graph-based PBA mode settings are enabled.
GS	Indicates that GBA skewness mode settings are enabled.
PBA COLL	Shows a collection of paths for which retime analysis is performed while creating the collection. This can be controlled with the When the `report_timing -path_type exhaustive -collection` command. By default, the `report_timing -collection` parameter does not show GBA and PBA slack values separately.

Path-Slew	Indicates the selected retime analysis is set to use path correct slew rates to reduce the pessimism from slew merging. This can be controlled with the `report_timing -retime` parameter. The path slews are recalculated in `path_slew_propagation` and `aocv_path_slew_propagation` modes.
PG	Indicates that group-based mode reporting is enabled when N paths are reported for every group.
RC	Indicates that RC variation is enabled with the `timing_socv_rc_variation_mode` attribute setting, and a SOCV format RC variation side file library is available.
SI	Indicates that SI analysis was enabled during retime analysis. The path-based analysis will recompute SI effects using path correct victim timing windows.
SOCV(Ultra)	Indicates that SOCV analysis is enabled and has been incorporated into the retiming. The SOCV analysis mode (specified with the `delaycal_socv_accuracy_mode` attribute) is shown in parenthesis. The accuracy modes are mapped as follows: • `low` `<not shown>` • `medium` `Medium` • `high` `High` • `ultra` `Ultra`

Spatial (CS)	Indicates that spatial derating side files have been loaded and an explicit spatial derate factor will be calculated. This is not shown for AOCV 2D derate tables, where the spatial and depth derates are combined. The GBA spatial derate mode (specified with the `timing_spatial_derate_distance_mode` attribute) is shown in parenthesis with the possible values given below: • `chip_size (CS)` • `bounding_box (BB)` The GBA spatial derate mode does not affect the path-based analysis bounding box calculation.
WP	Indicates that waveform propagation is enabled (using the the `delaycal_equivalent_waveform_model` attribute). The path-based retime will also use this delay calculation accuracy setting when recomputing delays and transitions for the path.

Examples of GBA/PBA Collection Reports

- When the `report_timing -path_type exhaustive -collection` command is specified, the following command generates a GBA collection report as shown below:

```
tempus > report_timing $tim_coll ; set tim_coll [report_timing -path_type exhaustive -
collection]

**WARN: (TCLCMD-1248):  Since no '-max_slack' has been specified with exhaustive '-
retime_mode', tool will assume it's default value in this case which is '0'
**WARN: (TCLCMD-1251):  The PBA exhaustive mode '-nworst' limit has been exhausted for some of
the endpoints. Refer to verbose log file of current session for more details.
INFO: Path Based Analysis (PBA) performed on total '3' paths
INFO: Max depth 3 reached for end point 'htb/CG_REBAL_HIER_IN21/RC_IC_IN_1/E' during Path
Based Analysis

Path 1: VIOLATED Clock Gating Setup Check with Pin htb/CG_REBAL_HIER_IN21/RC_IC_IN_1/CK
Endpoint:   htb/CG_REBAL_HIER_IN21/RC_IC_IN_1/E (v) checked with  leading edge of 'clk'
Beginpoint: htb/mc_vl_reg_2/Q                              (v) triggered by  leading edge of
```

```
'clk'
Common point: SPINE_CELL_BBF1/Y
Path Groups: {clock_gating_default}
Analysis View: sg_05v_15c_sigmax
Retime Analysis {GBA COLL}
```

- When the `report_timing -path_type exhaustive -collection` command is specified, the following command shows a PBA collection report:

```
tempus > report_timing $tim_coll ; set tim_coll [report_timing -path_type exhaustive -
collection]

**WARN: (TCLCMD-1248):  Since no '-max_slack' has been specified with exhaustive '-
retime_mode', tool will assume it's default value in this case which is '0'
INFO: Path Based Analysis (PBA) performed on total '530' paths
INFO: Max depth 131 reached for end point 'htb/CG_REBAL_HIER_IN21/RC_IC_IN_1/E' during Path
Based Analysis

Path 1: VIOLATED Clock Gating Setup Check with Pin
htb/RC_CG_REBALANCE_HIER_INST2989/RC_IC_IN_1/CK
Endpoint:   htb/RC_CG_REBALANCE_HIER_INST2989/RC_IC_IN_1/E (v) checked with  leading edge of
'clk'
Beginpoint: htb/mc_vl_reg_2/Q                              (^) triggered by  leading edge of
'clk'
Common point: SPINE_CELL_BBF1/Y
Closest CPPR Driver Pin: { Launch: SPINE_CELL_BBF1/Y Capture: SPINE_CELL_BBF1/Y }
Path Groups: {clock_gating_default}
Analysis View: sg_05v_15c_sigmax
Retime Analysis {PBA COLL}
```

VT Skew Reports

The following informational tags are supported for VT skew reports:

Flags	Description	Command Settings
VT-Skew	Shows VT skew details.	`set timing_enable_vtskew_derate_mode true`

PBA	Shows that the current path has undergone VT skew analysis. However, all the paths might not be considered VT skew candidates - even if VT skew analysis mode is enabled.	`set_vt_skew_config -unique_vt_mode {none | slow | fast | nominal}`
PU	Indicates unique VT skew for PBA. Displayed when unique VT mode is enabled.	`set_vt_skew_config -unique_vt_mode {slow | fast | nominal}`
GU	Indicates unique VT skew for GBA. Displayed when unique VT mode is enabled.	

A sample report is shown below:

```
Path 1: MET Hold Check with Pin CRE_IN/EXE_IN/top_reg[11]/CP
Endpoint:   CRE_IN/EXE_IN/top_reg[11]/D            (v) checked
with  leading edge of 'm_clk'
Beginpoint: TDSP_CORE_INST/DATA_BUS_MACH_INST/data_out_reg[11]/Q (v) triggered
by  leading edge of 'm_clk'
Path Groups: {m_clk}
Analysis View: view2
Retime Analysis {Data Path-Slew SOCV(High)-Corr SI Vt-Skew (PBA GU PU)}
Retime Vt-Skew Analysis {VT_group_1@fast VT_group_2@fast VT_group_3@fast }
Retime Vt-Skew Stats { Dominant_VT_group: NA Launch_Clock_VTGroups: 2 Capture_
Clock_VTGroups: 1 Total_Clock_VTGroups: 2}
Other End Arrival Time         0.569769 (+3.00S) |  0.531640 |  0.012710
+ Hold                        -0.035400 (+3.00S) | -0.035400 |  0.000000
+ Phase Shift                  0.000000
- CPPR Adjustment              0.051262 (+3.00S) |  0.000100 |  0.017054
= Required Time                0.462027 (+3.00S) |  0.496140 |  0.011371(-)
  Arrival Time                 0.769317 (-3.00S) |  0.803430 |  0.011371
  Slack Time                   0.307290 (-3.00S) |  0.307290 |  0.000000
= Slack Time(original)         0.356572 (-3.00S) |  0.356572 |  0.000000
     Clock Rise Edge                    0.000000
     = Beginpoint Arrival Time          0.000000 (-3.00S) |  0.000000 |  0.000000
```

Report Timing Column List

Option	Description

adjustment	Reports generated clock adjustment values. When reporting the detailed path for a generated clock, there are circumstances where the arrival time needs to be adjusted when a `create_generated_clock` assertion is encountered along the path. The adjustment argument can be used to provide a clearer indication of why a "jump" in the arrival time is seen in the path report. There are two primary reasons for generated clock-related adjustment to occur. • If the `timing_enable_generated_clock_edge_based_source_latency` attribute is set to `false`, it is possible that the edge-to-edge requirement specified by the generated clock cannot be satisfied by an existing logic path. This can happen when the source clock to generated clock polarity of the constraint is non-inverting, but the clock path itself constrains an inverting path. In this case, it would be normal to see a half-cycle of adjustment as the software compensates for the path versus constraint mismatch. If the above mentioned attribute is set to true, the software handles the path versus constraint mismatch by zeroing the source latency of the generated clock, and forcing alignment of the source and generated clock. As a result, no further adjustment is required, or reported. • In cases where `create_generated_clock -edge_shift` is specified, the adjustment column will show the amount of shift specified.
abs_delay_err	Reports the absolute delay difference of Tempus from Spice.
abs_transition_err	Reports the absolute slew difference of Tempus from Spice.

annotation	Reports RC parasitic annotations for net arcs, and delay annotations for either gate or net arcs. The following annotations are reported: • SDF: SDF back-annotation. The annotation status reflects the gate arc, not the interconnect arc. • SPEF: This entry appears if parasitic extraction has been done or annotated using SPEF. • LumpedRC: Lumped RC annotation • DlyAssert: Assertion set with set_annotated_delay command • WLM: Wire load model back-annotation • <none>: All others
aocv_adj_stages	Shows up "Aocv Adjust Stages" column. This column reports the AOCV adjustment stages that indicate the total effective stage count.
aocv_derate	Reports the graph-based AOCV derating factor used for cells/nets in AOCV mode. In retime mode, this column will report graph-based derating for cells/nets before the AOCV branch point and path-based derating.
aocv_weight	Reports AOCV weight values for the path elements.
arc	Reports the arc as described by the from pin, from pin edge, to pin, and to pin edge. For example, the arc from the rising edge of pin A to the falling edge of pin Z is reported as `A -> Z`.
arrival	Reports the arrival time on the pin.
arrival_mean	Reports mean of the arrival time on a pin.
arrival_sigma	Reports standard deviation of the arrival time on a pin.
cell	Reports the cell name of the given pin's instance.
delay	Reports the arc delay. If the stolen slack at the arc output pin (output of transparent latches) is not zero, the stolen column is also displayed along with this column.

delay_mean	Reports the mean of each arc's delay.
delay_sigma	Reports the standard deviation of each arc's delay. **Note**: In the absence of correlated and uncorrelated components of variation, the individual delay sigma values will not be summed up.
direction	Reports the pin direction (IN, OUT).
edge	Reports the edge of a pin (^=R, v=F).
fanin	Reports the number of source nodes of a net connected to a timing pin.
fanout	Reports the number of sinks of a net connected to a timing pin.
flags	Indicates the size_only, dont_use, dont_touch, and ideal attributes. When the 'flags' option is specified, a capacitance group entry (if it exists) will be displayed under the "Flags" column. This value indicates the input capacitance group being used for delay calculations – C1C2 (2-piece model) or C1CN (n-piece model).
hpin	Reports the hierarchical name for a given pin. **Note**: The hpin, pin, and timing_point will print the same values in the report output.
incr_delay	Reports the incremental delay of a pertaining arc. The software reads the incremental delay from an incremental SDF file using the `read_sdf` command.
instance	Reports the hierarchical name of a given pin's instance.
instance_location	Reports the location (x,y) of an instance. If a design is not placed, this column will be blank. This option displays the `instance` column automatically.

latch_window	Reports the available window for latch to sample the data.
load	Reports the total capacitance load on a given pin.
module	Reports the module name of the corresponding block hierarchy. This hierarchical module name will be reported only when the `-hpin` parameter of the `report_timing` command is specified.
net	Reports the hierarchical name of a net connected to a given pin.
pct_delay_err	Reports the percent delay difference of Tempus from Spice.
pct_slew_err	Reports the percent slew difference of Tempus from Spice.
phase	Reports the phase name of a pin.
phys_info	Shows minimum/maximum assigned layer, and non-default rule name information.
pin	Reports the reference name for the given pin.
pin_load	Reports the pin load data. The pin load refers to the sum of pin capacitance of all the pins connected to the current net.
pin_location	Reports the location (x,y) of a pin. If a design is not placed, this column will be blank.
power_domain	Reports power domain details of the specified path instance.
required	Reports the required time on a pin.
retime_delay	Reports the recalculated arc delay based on the path being reanalyzed with the specified retime method.
retime_delay_mean	Reports the mean of the recalculated arc delay on the path being reanalyzed with the specified retime method.
retime_mis_delta	Reports MIS delta delay in PBA mode for MIS-aware timing arcs in the path.

retime_mis_derate	Reports MIS derate values (in PBA mode), when derate-based advanced multi-input switching (AMIS-Derate) analysis is enabled.
retime_delay_sigma	Reports the standard deviation of each parameter of an arc's retimed delay.
retime_slew	Reports the recalculated slew based on the path being reanalyzed with the specified retime method.
retime_slew_mean	Reports the mean of each parameter of an arc sink pin's retimed slew.
retime_slew_sigma	Reports the standard deviation of each parameter of an arc sink pin's retimed slew.
stage_count	Displays the aggregate stage count of a specific instance in graph-based analysis (GBA) mode, and the actual stage count in path-based analysis (PBA) mode.
transition	Reports the propagated slew at the given pin.
transition_mean	Reports mean of each parameter of an arc sink pin's slew.
transition_sigma	Reports the standard deviation of each parameter of an arc sink pin's slew.
socv_derate	Reports the SOCV or the spatial derating factor applied on the cells or nets in SOCV analysis mode.
spice_arrival	Reports the Spice arrival time at the timing point.
spice_delay	Reports the Spice delay of a timing arc.
spice_slew	Reports the Spice slew of a timing point.
stolen	Reports the slack stolen (or the time given to the previous stage) at the given pin. If the slack is not zero and the `delay` column is specified, then this column is displayed by default (but only visible on the output of transparent latches).
timing_point	Reports the hierarchical names for the given points.

total_derate	Reports the total derating factor applied on the corresponding cell or nets that reflect the combined impact of all the derates (OCV/AOCV/spatial) applied on each stage of the timing path. **Note**: By default, the software will take into account the negative delays and keep the SOCV derate column in sync with the user-derate column. The software multiplies the negative delay by (2.0 - derating factor) to make it more pessimistic. However, reporting will always show derates based on the positive delay. The timing report output will show similar values for the SOCV Derate and Total Derate columns, but there will be no changes in the slack values.
user_derate	Reports the derating scale factors set with the `set_timing_derate` command. **Note:** The timing system cannot provide an accurate derating factor for the lumped gate and wire delay when the `set_timing_derate` factors for `-cell_delay` and `-net_delay` are not equivalent. In this case, an asterisk (*) is reported in the SSI Derate column.
voltage	Returns the voltage that is used during delay calculations.
vt_skew_derate	Returns voltage threshold (VT) skew analysis details.
when_cond	Reports the "when" condition of the arc specified in the library.
wire_load	Reports the wire load.
wlmodel	Reports the kind of parasitic used by a pin or net, such as SPEF. If the wire load model is derived from a Liberty file, the information is reported in the following format: `Wireload model (.lib` *`model_name library_name`*`)`

Reporting Timing Checks Between Various Types of Clocks

The following table summarizes the timing checks and the min/max delay checks that are performed based on the types of launching and capturing clocks:

- NZ (Non Zero Period Clock - Regular Clock)
- Z (Zero Period Clock - Regular clock whose period is equal to zero)
- @ (Asynchronous Clock - Clock which triggers the default input delay and the input delay without a clock)
- CPD (Combinational Path Delay)

Launching Clock	Capturing Clock	Timing Checks	Path Delay Checks
@	@	NO	CPD
Z	@	NO	CPD
NZ	@	NO	CPD
@	Z	YES	SPD
@	NZ	YES	SPD
NZ	Z	NO	SPD
Z	NZ	YES	SPD
Z	Z	YES	SPD
NZ	NZ	YES	SPD

Reports Sections

Path Summary Report

```
tempus > report_timing -path_type summary
```

A sample path summary report is shown below:

```
Path 1: VIOLATED (-16.913 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[30]/CK->D
Group: m_clk
Startpoint: (R) TDSP_CORE_INST/DECODE_INST/ir_reg[9]/CK
Clock: (R) m_clk
Endpoint: (F) CRE_IN/EXE_IN/p_reg[30]/D
Clock: (R) m_clk

                Capture       Launch
Clock Edge:+    4.000         0.000
Src Latency:+   0.000         0.000
Net Latency:+   4.206 (P)     4.101 (P)
Arrival:=       8.206         4.101

Setup:-          1.251
Uncertainty:-    0.400
```

```
Cppr Adjust:+     0.000
Required Time:=   6.555
Launch Clock:=    4.101
Data Path:+      19.367
Slack:=         -16.913
```

Advanced Summary Section

The ‘advanced summary’ section shows specific information, such as, slack analysis, SI pessimism. This section provides cell-by-cell information on the path as well as nets, hierarchical pins, exceptions, and other aspects of the timing path.

A sample advanced summary report is shown below.

```
Path 1: MET (2285 ps) Setup Check with Pin rt/mul_73_17/retime_s5_16_reg/CK->D
. . .
Exceptions/Constraints:
path_adjust       370      scripts/constraints.sdc_line_57
input_delay      1001      scripts/constraints.sdc_line_27

Slack Analysis(3):
Stage(-3): (2123.60 ps) rt/mul_73_17/retime_s2_16_reg/D
Stage(-2): (2210.60 ps) rt/mul_73_17/retime_s3_16_reg/D
Stage(-1): ( 348.60 ps) rt/mul_73_17/retime_s4_16_reg/D
Stage( 0): (2285.30 ps) rt/mul_73_17/retime_s5_16_reg/D
Stage( 1): (-816.00 ps) rt/mul_73_17/retime_s6_9_reg/D
Stage( 2): (1283.90 ps) o6[15]
. . .
```

Note: These reports may contain content that has substantial runtime impact (such as meta-data, etc.).

Path Description Section

The path description section shows the cell-by-cell delay, slew, arc, exception information, and informational flag legends.

A sample path description section is shown below.

```
#-------------------------------------------------------------------------------
# Timing Point            Arc     Edge    Cell     Load     Trans   Delay  Arrival
#                                                  (fF)     (ps)    (ps)   (ps)
#-------------------------------------------------------------------------------
FF_I1_reg[0]/CK        <<< -      R       -          -      150     -       0
FF_I1_reg[0]/Q            CK->Q   R       DFFHQX4    45.4   201     334     334
mul_26_28/Fn0755D13176/Y  A->Y    F       INVX2      15.7   103     68      402
mul_26_28/Fn0755D13172/Y  A->Y    R       INVX4      53.0   223     130     532
mul_26_28/n0617DTtn0525/Y B->Y    F       NAND2X1    10.9   169     108     640
mul_26_28/n0634Dtn0548/Y  A1N->Y  F       AOI2BB2X4  19.3   164     232     872
mul_26_28/Fn0612D/Y       A->Y    R       INVX2      18.5   183     119     992
. . .
mul_26_43/n0014Dtn0331/Y  A->Y    F       NAND2X2     17.3   147    94      4168
mul_26_43/Fn0013D16832/Y  A->Y    R       INVX1       5.6    145    97      4265
mul_26_43/n0078Dtn0331/Y  A1N->Y  R       OAI2BB1X2  18.9   201     203     4468
```

```
mul_26_43/n0064Dtn0331/Y  A->Y    R       XOR2X4      10.6    86    225    4693
mul_26_43/n0025Dtn0331/S  CI->S   F       ADDFHX4     5.2     82    269    4962
FF_O2_reg[0]/D          <<< -     F       DFFHQX2     -       -       0    4962
#-------------------------------------------------------------------------------
```

By default, the timing reports show the ‘path summary’ and ‘path description’ section. However, the default can be changed by using the `report_timing -path_type` parameter.

Endpoint Reports

In addition to detailed reports, endpoint reports can also be generated. These reports allow you to include additional endpoint related information that can be useful in debugging or identifying certain issues. For example, an endpoint report can show the number of buffers and inverters on a path. A sample report is as shown:

```
tempus > report_timing -path_type endpoint
#-----------------------------------------------------
# ID    Slack   Endpoint           Group   View
#       (ns)
#-----------------------------------------------------
   1   -16.913  CRE_IN/p_rg[30]/D  m_clk   def_em_view1
#-----------------------------------------------------
```

The endpoint reports and detailed reports are mutually exclusive.

Clock Tracing Reports

Optionally, the report includes the clock tracing section that helps in debugging issues pertaining to clocks used by the startpoint and/or endpoint of a path. Clock tracing traces the source of the clock and the detailed path taken to reach the launch and/or capture elements’ clock pin. A sample report is shown below.

```
> report_timing -view func1@wc_cworst -path_type full_clock
                -from ff1_reg -to ff3_reg
                -fields {-fanout}
Path 1: MET (405 ps) Setup Check with Pin ff3_reg/CK->D
          View: func1@wc_cworst
          Group: ck2
     Startpoint: (R) ff1_reg/CK
          Clock: (R) ck1
       Endpoint: (R) ff3_reg/D
          Clock: (R) ck2

                Capture        Launch
Clock Edge:+       1000             0
Src Latency:+         0             0
Net Latency:+         0 (I)         0 (I)
Arrival:=          1000             0

Setup:-              61
Required Time:=     939
Launch Clock:-        0
Data Path:-         595
Slack:=             405
```

```
#-------------------------------------------------------------
# Timing Point   Arc   Edge  Cell   Load   Trans   Delay Arrival
#                                    (fF)   (ps)    (ps)  (ps)
#-------------------------------------------------------------
clk1             -      R     (port)  0.9      0    0       0
clk1_buf1/Y      A->Y   R     BUFX4   0.9     24   60       60
clk1_buf2/Y      A->Y   R     BUFX4   0.0      0   67       127
ff1_reg/CK       -      R     DFFX1     -      0    0       127
#-------------------------------------------------------------
ff1_reg/QN <<<   CK->QN R     DFFX1   0.6     23  335       462
ff1_i2_and/Y     B->Y   R     AND2X2  1.2     45  133       595
ff3_reg/D <<<    -      R     DFFX4     -      -    0       595
#-------------------------------------------------------------
clk2             -      R    (port)   0.9      0    0      1000
clk2_buf1/Y      A->Y   R     BUFX4   2.8     31   64      1164
clk2_buf2/Y      A->Y   F     INVX4   2.8     22   24      1188
clk2_buf3/Y      A->Y   R     INVX4   0.0      0   14      1102
ff3_reg/CK       -      R     DFFX4   -        -    0      1102
#-------------------------------------------------------------
```

PBA Header

- In PBA, EPBA, and lPBA mode, the timing report shows PBA header for columns (by default):

```
report_timing -retime path_slew_propagation -path_type full_clock

Path 1: VIOLATED Late External Delay Assertion
Endpoint:   o5    (^) checked with  leading edge of 'CLK'
Beginpoint: ff3/Q (^) triggered by  leading edge of 'CLK'
Analysis View: an_view_setup
Retime Analysis {Data Clock Path-Slew Spatial(CS) SOCV}
Retime SOCV Analysis { Distance(um) : 27.653 }
                              NSigma           |  Mean    |  Sigma  |  Stddev
Other End Arrival Time         0.0000 (-3.00S)  |  0.0000  |  0.0000 |  0.0000
- External Delay               2.5000
+ Phase Shift                  0.5000
+ CPPR Adjustment              0.0000 (+3.00S)  |  0.0000  |  0.0000 |  0.0000
- Uncertainty                  0.5000
= Required Time               -2.5000 (-3.00S)  | -2.5000  |  0.0000 |  0.0000
- Arrival Time                 18.4496 (+3.00S) |  18.4486 |  0.0004 |  0.0004
= Slack Time                  -20.9496 (-3.00S) | -20.9486 |  0.0004 |  0.0004
= Slack Time(original)        -20.9484 (-3.00S) | -20.9484 |  0.0000 |  0.0000
     Clock Rise Edge                     0.0000
     + Source Insertion Delay            1.0000 (+3.00S)   |  1.0000 |  0.0000 |  0.0000
     = Beginpoint Arrival Time           1.0000 (+3.00S)   |  1.0000 |  0.0000 |  0.0000
```

```
      ------------------------------------------------------------------------------------------
-----------------------------------------------
      Pin            Phase                      Edge  Delay    Delay Mean  Delay Stddev  Retime
Delay  Retime Delay  Retime Delay  Arrival Time

          Mean          Stddev
      ------------------------------------------------------------------------------------------
-----------------------------------------------
      clk           CLK(C)(P)(an_view_setup)*   ^     -        -           -             -
        -             -             1.0000
      clk_L1_I0/A   CLK(C)(P)(an_view_setup)*   ^     0.0004   0.0004      0.0000        0.0014
       0.0005         0.0003        1.0014
      clk_L1_I0/Y   CLK(C)(P)(an_view_setup)*   ^     0.0600   0.0600      0.0000        0.0600
       0.0600         0.0000        1.0613
      clk_L2_I0/A   CLK(C)(P)(an_view_setup)*   ^     0.0001   0.0001      0.0000        0.0004
       0.0002         0.0001        1.0613
      clk_L2_I0/Y   CLK(C)(P)(an_view_setup)*   ^     0.0604   0.0604      0.0000        0.0604
       0.0604         0.0000        1.1217
      ff3/CLK       CLK(C)(P)(an_view_setup)*   ^     0.0001   0.0001      0.0000        0.0011
       0.0001         0.0003        1.1215
      ff3/Q         CLK(D)(P)(an_view_setup)*   ^     17.3092  17.3092     0.0000
 17.3091       17.3091       0.0000        18.4306
      o5            CLK(D)(P)(an_view_setup)    ^     0.0182   0.0182      0.0000        0.0197
       0.0183         0.0005        18.4496
      ------------------------------------------------------------------------------------------
-----------------------------------------------
```

Examples

- The following command reports paths leading to the pin(s) specified by the *object_list*:

```
tempus > report_timing -to CRE_INST/EXE_INST/p_reg[31]/D

Path 1: VIOLATED Setup Check with Pin CRE_INST/EXE_INST/p_reg[31]/CK
Endpoint: CRE_INST/EXE_IN/p_reg[31]/D (v) checked with leading edge of 'm_clk'
Beginpoint: CRE_INST/DE_INST/ir_reg[10]/Q (^) triggered by leading edge of 'm_clk'
Path Groups:  {m_clk}
Other End Arrival Time          5.272
- Setup                         1.298
+ Phase Shift                   4.000
```

```
- Uncertainty                    0.400
= Required Time                  7.574
- Arrival Time                   38.446
= Slack Time                    -30.871
Clock Rise Edge                  0.000
+ Clock Network Latency (Prop)   5.348
= Beginpoint Arrival Time        5.348
-------------------------------------------------------------------------------------------
Instance                       Arc           Cell        Delay   Arrival Time  Required Time
-------------------------------------------------------------------------------------------
CRE_INST/DE_INST/ir_reg[10]    CK ^          -           -       5.348         -25.523
CRE_INST/DE_INST/ir_reg[10]    CK ^ -> Q ^   SDFFS_X2    0.639   5.987         -24.884
CRE_INST/DE_INST/FE_prt_2      A ^ -> Z ^    BUF_X16     1.477   7.464         -23.407
CRE_IN/GL_IN/lsh_47/F0D26      A ^ -> ZN v   INV_X4      1.833   9.297         -21.574
CRE_IN/GL_IN/lsh_47/p0088D     A2 v -> ZN v  AND2_X2     1.170   10.467        -20.404
CRE_IN/GL_IN/lsh_47/g9132      A1 v -> ZN v  AND2_X2     0.336   10.803        -20.068
CRE_IN/GL_IN/lsh_47/FE_661     A v -> Z v    BUF_X32     0.341   11.144        -19.727
CRE_IN/GL_IN/p0171A            A1 v -> ZN ^  NAND2_X4    0.230   11.375        -19.497
CRE_IN/GL_IN/p0089D            A3 ^ -> ZN ^  AND3_X2     0.385   11.760        -19.111
CRE_IN/GL_IN/p0053D            A1 ^ -> ZN v  NAND2_X4    0.417   12.176        -18.695

-------------------------------------------------------------------------------------------
```

- The following command shows a report similar to the first with the addition of net arc information:

```
tempus > report_timing -to CRE_INST/EXE_INST/p_reg[31]/D -net

Path 1: VIOLATED Setup Check with Pin CRE_INST/EXE_INST/p_reg[31]/CK
Endpoint:   CRE_INST/EXE_INST/p_reg[31]/D (v) checked with  leading edge of 'm_clk'
Beginpoint: CRE_INS/DDE/ir_reg[10]/Q (^) triggered by  leading edge of 'm_clk'
Path Groups: {m_clk}
Other End Arrival Time          5.272
- Setup                         1.298
+ Phase Shift                   4.000
- Uncertainty                   0.400
= Required Time                 7.574
- Arrival Time                 38.446
= Slack Time                  -30.871
Clock Rise Edge                 0.000
+ Clock Network Latency (Prop)  5.348
= Beginpoint Arrival Time       5.348
-------------------------------------------------------------------------------------------
Pin                        Edge  Net                     Cell     Delay   Arrival   Required

                                                                          Time      Time
```

```
-------------------------------------------------------------------------------------------
CRE_INS/DDE/ir_reg[10]/CK   ^    m_clk__L19_N6          -         -      5.348    -25.523
CRE_INS/DDE/ir_reg[10]/Q    ^    CRE_INS/DDE/FE_9_ad    SDFFS_X2  0.639  5.987    -24.884
CRE_INS/DDE/OFC3_prt_add/A  ^    CRE_INS/DDE/FE_9_ad    BUF_X16   0.000  5.987    -24.884
CRE_INS/DDE/OFC3_prt_add/Z  ^    port_address[2]        BUF_X16   1.477  7.464    -23.407
CRE_IN/GL/lsh_47/Fn96/A     ^    port_address[2]        INV_X4    0.045  7.509    -23.362
CRE_IN/GL/lsh_47/Fn96/ZN    v    CRE_IN/GL/ls_47/n_3    INV_X4    1.789  9.297    -21.574
CRE_IN/GL/lsh_47/p8D/A2     v    CRE_IN/GL/ls_47/n_3    AND2_X2   0.000  9.298    -21.573
CRE_IN/GL/lsh_47/p8D/ZN     v    CRE_IN/GL/ls_47/n_16   AND2_X2   1.170  10.467   -20.404
CRE_IN/GL/lsh_47/g32/A1     v    CRE_IN/GL/ls_47/n_16   AND2_X2   0.000  10.468   -20.404
CRE_IN/GL/lsh_47/g32/ZN     v    CRE_IN/GL/ls_47/FE_61  AND2_X2   0.336  10.803   -20.068
CRE_IN/GL/lsh_47/FE_661/A   v    CRE_IN/GL/ls_47/FE_61  BUF_X32   0.000  10.803   -20.068
CRE_IN/GL/lsh_47/FE_661/Z   v    CRE_IN/GL/n_661        BUF_X32   0.341  11.144   -19.727
CRE_IN/GL/p0171A/A1         v    CRE_IN/GL/n_661        NAND2_X4  0.000  11.144   -19.727
CRE_IN/GL/p0171A/ZN         ^    CRE_IN/GL/n_75         NAND2_X4  0.230  11.375   -19.497
CRE_IN/GL/p0089D/A3         ^    CRE_IN/GL/n_75         AND3_X2   0.000  11.375   -19.496
CRE_IN/GL/p0089D/ZN         ^    CRE_IN/GL_IN/n_223     AND3_X2   0.385  11.760   -19.111
-------------------------------------------------------------------------------------------
```

- The `report_timing` command accepts `-collection` as an argument. For example,

  ```
  report_timing [report_timing -collection]
  set rpt [report_timing -collection]
  report_timing $rpt
  ```

- The following command displays the worst late path to each violating endpoint that has a slack less than -1.0:

  ```
  report_timing -max_slack -1.0
  ```

- The following command displays all the late paths that end at port `out[2]` and that have negative slack up to a maximum of 1000 worst paths. If there are more than 1000 paths, only the 1000 worst paths are reported:

  ```
  report_timing -to out[2] -max_paths 1000 -max_slack 0.0
  ```

- The following command reports the worst late path that starts at `in[0]` and ends at `out[1]`:

  ```
  report_timing -from in[0] -to out[1]
  ```

- The following command displays the three worst paths that start at `in[1]` and end at `out[3]`. With reconvergent fanout, more than one path may exist:

  ```
  report_timing -from in[1] -to out[3] -max_paths 3
  ```

- The following command displays the ten worst paths. Only the paths between the specified pins are enumerated.With reconvergent fanout, more than one path may exist:

  ```
  report_timing -from i102/Z -to i123/A -max_paths 10
  ```

- The following command enumerates the worst ten paths through the given two pins, starting at the beginning points in the design and ending at the endpoints, similar to using the `-from` option:

  ```
  report_timing -through i102/Z -through i123/A -max_paths 10
  ```

- The following command reports the ten worst paths through blocks A and C, or blocks B and C. The path only has to satisfy one element in a through list, but all through lists must be satisfied. The example can be thought of as ((A or B) and C):

  ```
  report_timing -through {A/*B/*} -through {C/*} -max_paths 10
  ```

- The following command forces a path through pin A and pin B. Use the syntax shown in the example. Do not use `-through {A B}`:

  ```
  report_timing -through A -through B
  ```

- The following command displays the clock path through the clock root to Launch flop:

  ```
  report_timing -through aBC_bs/bs_mex1/r2_00_q1_reg_10_/CK -to .... -path_type full_clock
  ```

- You will not see the path for the launch clock if you specify the following command because the report_timing command starts reporting from the CK pin:

  ```
  report_timing -from aBC_bs/bs_mex1/r2_00_q1_reg_10_/CK
  ```

- The first command displays the path leading to the `CK` pin.

- The following command shows the report generated using the `-unconstrained` parameter:

  ```
  report_timing -unconstrained
  ---------------------------------------------------------------------------
  Path 1:Endpoint:   RAM_256x16_IN/g417/A1 (^) (unconstrained output)
  Beginpoint: SP_CRE/DE_IN/ir_reg[9]/Q (v) triggered by  leading edge of 'm_clk'
  Arrival Time                   23.760
  + Clock Network Latency (Prop)  5.348
  = Beginpoint Arrival Time       5.348
  ---------------------------------------------------------------------------
  Instance                    Arc            Cell       Delay   Arrival
                                                                Time
  ---------------------------------------------------------------------------
  SP_CRE/DE_IN/ir_reg[9]      CK ^           -          -       5.348
  SP_CRE/DE_IN/ir_reg[9]      CK ^ -> Q v    SDFFS_X2   0.687   6.035
  SP_CRE/DE_IN/OFC186_pt_ad   A v -> Z v     BUF_X16    0.527   6.562
  SP_GLE_IN/p8211A            A3 v -> ZN ^   NOR3_X1    0.929   7.492
  SP_GLE_IN/p8090A            A2 ^ -> ZN v   NAND2_X1   0.361   7.852
  SP_GLE_IN/p6871A            A4 v -> ZN ^   NOR4_X2    1.873   9.725
  SP_GLE_IN/p785133           A1 ^ -> ZN v   NAND2_X4   0.528   10.253
  SP_GLE_IN/p7333D            A2 v -> ZN ^   NOR2_X4    3.227   13.480
  SP_GLE_IN/p7A6115           B1 ^ -> ZN v   AOI222_X1  1.576   15.055
  ```

```
SP_GLE_IN/p850085          A2 v -> ZN ^   NAND3_X4    1.098   16.154
OFC539_tsp_data_out        A ^ -> Z ^     BUF_X16     1.302   17.456
SM_MUX_IN/p9662A497        B1 ^ -> ZN v   AOI22_X1    0.792   18.248
SM_MUX_IN/OFC17_n_23       A v -> ZN ^    INV_X4      3.115   21.363
SM_MUX_IN/OFC18_n_23       A ^ -> ZN v    INV_X4      0.338   21.700
SM_MUX_IN/C594_OFN18_n_23  A v -> Z v     BUF_X32     0.867   22.567
SM_MUX_IN/Fp9546A496       A v -> ZN ^    INV_X1      0.247   22.815
OFC16_ds_datain_0          A ^ -> Z ^     BUF_X8      0.944   23.759
RAM_256x16_IN/g417         A1 ^           AOI22_X1    0.001   23.760
-------------------------------------------------------------------------
```

- The following command sets a false path from `CRE_IN/DE_IN/ir_reg[9]/Q` to `CRE_IN/EXE_IN/p_reg[31]/D`:

```
tempus > set_false_path -from CRE_IN/DE_IN/ir_reg[9]/Q  -to CRE_IN/EXE_IN/p_reg[31]/D
tempus > set_false_path -through CRE_IN/DE_IN/ir_reg[11]/Q
tempus > report_timing -from CRE_IN/DE_IN/ir_reg[11]/Q -to CRE_IN/EXE_IN/p_reg[25]/D -
unconstrained -path_exceptions all
```

```
Path 1:Endpoint:   CRE_IN/EXE_IN/p_reg[31]/D (^) (unconstrained output)
Beginpoint: CRE_IN/DE_IN/ir_reg[10]/Q (^) triggered by  leading edge of 'm_clk'
Arrival Time                   39.024
+ Clock Network Latency (Prop)  5.348
= Beginpoint Arrival Time       5.348
-------------------------------------------------------------------------
Instance                         Arc           Cell      Delay  Arrival
Time
-------------------------------------------------------------------------
CRE_IN/DE_IN/ir_reg[10]          CK ^          -         -      5.348
CRE_IN/DE_IN/ir_reg[10]          CK ^ -> Q ^   SDFFS_X2  0.639  5.987
CRE_GLE_INST/p0089D              A3 ^ -> ZN ^  AND3_X2   0.385  11.760
CRE_GLE_INST/p0053D              A1 ^ -> ZN v  NAND2_X4  0.417  12.176
CRE_INST/MPY_32/FE_RC_514_0      A v -> ZN ^   INV_X32   0.271  12.448
CRE_INST/MPY_32/p0258A           A2 ^ -> ZN v  NAND2_X4  0.200  12.648
CRE_INST/MPY_32/Fp0229A          A v -> ZN ^   INV_X8    0.460  13.108
CRE_INST/MPY_32/p0287A           A1 ^ -> ZN v  NAND2_X1  0.261  13.369
CRE_INST/MPY_32/p0073D           A2 v -> ZN ^  NAND2_X4  0.320  13.689
CRE_INST/MPY_32/p0137D           A1 ^ -> ZN v  NAND2_X1  0.215  13.904
CRE_INST/MPY_32/p0041D           A2 v -> ZN ^  NAND2_X4  1.214  15.118
CRE_INST/MPY_32/Fp2263A          A ^ -> ZN v   INV_X4    0.968  34.647
CRE_IN/EXE_IN/n1456D             B1 ^ -> ZN v  AOI22_X1  0.355  38.297
CRE_IN/EXE_IN/n1510D             A v -> ZN ^   INV_X8    0.728  39.024
CRE_IN/EXE_IN/p_reg[31]          D ^           SDFF_X2   0.000  39.024
-------------------------------------------------------------------------
Applied exceptions:
-------------------------------------------------------------------------------------------------
-
From                         Through                      To                           Late
```

```
-------------------------------------------------------------------------------------------
-
CRE_IN/DE_IN/ir_reg[10]/CK   -                          CRE_IN/EXE_IN/p_reg[31]/D    false
-                            CRE_IN/EXE_IN/n1510D/A     -                            false
-------------------------------------------------------------------------------------------
```

- For hold paths, if the path has a setup multi-cycle path then those exceptions will also be reported because hold slacks get impacted by inferred path exceptions. For example:

```
tempus > set_multicycle_path 2 -setup -from CRE_IN/DE_IN/ir_reg[9]/CK
tempus > report_timing -from CRE_IN/DE_IN/ir_reg[10]/CK
```

With multi-cycle path:

```
Path 1: VIOLATED (-26.872 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[31]/CK->D Group:m_clk
Startpoint:   (R) CRE_IN/DE_IN/ir_reg[l0]/CK Clock: (R) m_clk
Endpoint:     (F) CRE_IN/EXE_IN/p_reg[31]/D
Clock:        (R)m_clk
N-Sigma:      3.000

                 Capture    Launch
Clock Edge:+     8.000      0.000
Src Latency:+    0.000      0.000
Net Latency:+    5.272 (P)  5.348 (P)
Arrival:=       13.272      5.348

Setup:-           1.298
Uncertainty:-     0.400
Cppr Adjust:+     0.001
Required Time:=  11.575
Launch Clock:=    5.348
Data Path:+      33.099
Slack:=         -26.872
```

Without multi-cycle path:

```
Path 1: VIOLATED (-30.872 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[31]/CK->D Group:m_clk
Startpoint: (R) CRE_IN/DE_IN/ir_reg[l0]/CK Clock: (R) m_clk
Endpoint:   (F) CRE_IN/EXE_IN/p_reg[31]/D
Clock:      (R)m_clk
N-Sigma: 3.000
                 Capture    Launch
Clock Edge:+     4.000      0.000
Src Latency:+    0.000      0.000
Net Latency:+    5.272 (P)  5.348 (P)
Arrival:=        9.272      5.348

Setup:-          1.298
```

```
Uncertainty:-    0.400
Cppr Adjust:+    0.001
Required Time:=  7.575
Launch Clock:=   5.348
Data Path:+     33.099
Slack:=        -30.872
```

- The following command displays the timing report for analysis view `setup_view1`:

```
tempus > report_timing -view setup_view1

Path 1: VIOLATED (-30.660 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[31]/CK->D
Group: m_clk
Startpoint: (R) CRE_IN/DE_IN/ir_reg[11]/CK
Clock: (R) m_clk
Endpoint: (F) CRE_IN/EXE_IN/p_reg[31]/D
Clock: (R) m_clk
N-Sigma: 3.000

                Capture      Launch
Clock Edge:+    4.000        0.000
Src Latency:+   0.000        0.000
Net Latency:+   5.272 (P)    5.300 (P)
Arrival:=       9.272        5.300

Setup:-         1.298
Uncertainty:-   0.400
Cppr Adjust:+   0.001
Required Time:= 7.575
Launch Clock:=  5.300
Data Path:+     32.935
Slack:=        -30.660

#-------------------------------------------------------------------------------------------
-
# Timing Point                Flags   Arc     Edge  Cell      Fanout   Trans   Delay   Arrival

#                                                                      (ns)    (ns)     (ns)
#-------------------------------------------------------------------------------------------
-
CRE_IN/DE_IN/ir_reg[11]/CK     -      CK      R     (arrival)  4       0.139       -    5.300

CRE_IN/DE_IN/ir_reg[11]/Q      -      CK->Q   F     SDFFS_X2   1       0.125   0.651    5.951

CRE_IN/MPY_32/p0258A/ZN        -      A2->ZN  R     NAND2_X4   3       0.397   0.460   12.676

CRE_IN/MPY_32/Fp0229A/ZN       -      A->ZN   F     INV_X8     5       0.217   0.254   12.929

CRE_IN/MPY_32/Fp2263A/ZN       -      A->ZN   F     INV_X4     6       1.134   0.966   34.437

CRE_IN/MPY_32/inc_ad4/p0A/ZN   -      A1->ZN  R     NAND2_X1   2       0.388   0.333   36.458
```

```
CRE_IN/MPY_32/FE_RC_18_0/ZN    -      A1->ZN  F     NAND2_X4   1       0.231   0.177   37.166

CRE_IN/EXE_IN/n1456D/ZN        -      B1->ZN  R     AOI22_X1   1       1.069   0.734   37.899

CRE_IN/EXE_IN/p_reg[31]/D      -      D       F     SDFF_X2    1       0.413   0.000   38.235

#-------------------------------------------------------------------------------------------
-
```

- The following command shows report when `-point_to_point` parameter is used:

```
tempus > report_timing -to CRE_IN/EXE_IN/p_reg[31]/D -from CRE_IN/DE_IN/ir_reg[11]/Q -
point_to_point

Path 1:
Startpoint: (F) CRE_IN/DE_IN/ir_reg[11]/Q
Clock: (R) m_clk
Endpoint: (R) CRE_IN/EXE_IN/p_reg[31]/D
Clock:
N-Sigma: 3.000

                 Capture      Launch
Src Latency:+    0.000        0.000
Net Latency:+    0.000 (I)    0.000 (I)
Arrival:=        0.000        0.000

Phase Shift:+    0.000
Launch Clock:=   0.000
Data Path:+     32.862

#-------------------------------------------------------------------------------------------
-------
# Timing Point                      Flags  Arc     Edge  Cell       Fanout  Trans  Delay
Arrival
#                                                                              (ns)    (ns)
(ns)
#-------------------------------------------------------------------------------------------
------
CRE_IN/DE_IN/ir_reg[11]/Q          <<<    Q       F     (arrival)  1       0.125   0.000
 0.000
CRE_IN/DE_IN/FE_OFC192_ir_11_/Z    -      A->Z    F     BUF_X16    25      0.385   0.498
 0.498
CRE_IN/MY_32_IN/p0258A/ZN          -      A2->ZN  R     NAND2_X4   3       0.397   0.460
 6.724
CRE_IN/MY_32_IN/Fp0229A/ZN         -      A->ZN   F     INV_X8     5       0.217   0.254
 6.978
CRE_IN/MY_32_IN/Fp2263A/ZN         -      A->ZN   F     INV_X4     6       1.134   0.966
28.486
CRE_IN/MY_32_IN/FE_RC_1689_0/ZN    -      A1->ZN  F     NAND2_X4   1       0.705   0.243
31.313
```

```
CRE_IN/EXE_IN/n1456D/ZN            -      B1->ZN    F     AOI22_X1   1       1.085   0.355
32.135
CRE_IN/EXE_IN/n1510D/ZN            -      A->ZN     R     INV_X8     1       0.609   0.727
32.862
#-------------------------------------------------------------------------------------------
-----
```

- The following command reports pin names and net names in the same column alternatively:

```
tempus > report_timing -fields {load slew delay arrival cell pin_load fanout timing_point} -
net

Path 1: VIOLATED (-37.450 ns) Setup Check with Pin CRE/EXE_IN/p_reg[31]/CK->D
Group: m_clk
Startpoint: (R) CRE/DE_IN/ir_reg[10]/CK
Clock: (R) m_clk
Endpoint: (F) CRE/EXE_IN/p_reg[31]/D
Clock: (R) m_clk

                   Capture      Launch
Clock Edge:+       4.000        0.000
Src Latency:+      0.000        0.000
Net Latency:+      6.337 (P)    6.408 (P)
Arrival:=          10.337       6.408

Setup:-            1.298
Uncertainty:-      0.400
Cppr Adjust:+      0.001
Required Time:=    8.640
Launch Clock:=     6.408
Data Path:+       39.681
Slack:=          -37.450

#-------------------------------------------------------------------------------------
#  Load  Trans   Delay  Arrival  Cell     Pin Load  Fanout  Timing Point
#  (pf)   (ns)    (ns)     (ns)
#-------------------------------------------------------------------------------------
0.005  0.145       -    6.408  (arrival)  0.003      4      CRE/DE_IN/ir_reg[10]/CK
0.001  0.145   0.703    7.112  SDFFS_X2   0.000      1      CRE/DE_IN/ir_reg[10]/Q
0.001      -       -        -  (net)      0.000      1      CRE/DE_IN/O13_pt_add_2
0.081  0.108   1.835    8.947  BUF_X16    0.021      45     CRE/DE_IN/OF3_prt_add_2/Z
0.081      -       -        -  (net)      0.021      45     port_address[2]
0.022  2.592   2.147   11.094  INV_X4     0.009      16     CRE_GLU_IN/ls_47/Fn26/ZN
0.022      -       -        -  (net)      0.009      16     CRE_GLU_IN/ls_47/n_3
0.005  1.541   1.404   12.497  AND2_X2    0.003       5     CRE_GLU_IN/ls_47/p08D/ZN
0.005      -       -        -  (net)      0.003       5     CRE_GLU_IN/ls_47/n_16
0.001  0.221   0.403   12.900  AND2_X2    0.000       1     CRE_GLU_IN/ls_47/g932/ZN
0.001      -       -        -  (net)      0.000       1     CRE_GLU_IN/ls_47/FE
0.003  1.069   0.403   46.090  INV_X8         -       1     CRE/EXE_IN/n1510D/ZN
0.003      -       -        -  (net)      0.002       1     CRE/EXE_IN/n_680
0.003  0.413   0.000   46.090  SDFF_X2        -       1     CRE/EXE_IN/p_reg[31]/D
#-------------------------------------------------------------------------------------
```

- The following command reports paths through unique set of pins:

```
tempus > Report_timing -nworst 2 unique_pins

Path 1: VIOLATED (-30.645 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[25]/CK->D
Group: m_clk
Startpoint: (R) CRE_IN/DE_IN/ir_reg[11]/CK
Clock: (R) m_clk
Endpoint: (F) CRE_IN/EXE_IN/p_reg[25]/D
Clock: (R) m_clk

                 Capture       Launch
Clock Edge:+     4.000         0.000
Src Latency:+    0.000         0.000
Net Latency:+    5.274 (P)     5.300 (P)
Arrival:=        9.274         5.300

Setup:-          1.282
Uncertainty:-    0.400
Cppr Adjust:+    0.001
Required Time:=  7.592
Launch Clock:=   5.300
Data Path:+     32.937
Slack:=        -30.645

#-------------------------------------------------------------------------------------------
# Timing Point                 Flags  Arc     Edge  Cell       Fanout  Trans   Delay  Arrival

#                                                                      (ns)    (ns)     (ns)

#-------------------------------------------------------------------------------------------
CRE_IN/DE_IN/ir_reg[11]/CK     -      CK      R     (arrival)  4       0.139       -    5.300

CRE_IN/DE_IN/ir_reg[11]/Q      -      CK->Q   F     SDFFS_X2   1       0.139   0.651    5.952

CRE_IN/DE_IN/FE_OFC111_/Z      -      A->Z    F     BUF_X16    25      0.125   0.498    6.450

CRE_IN/GL_IN/lsh_47/Fn05/ZN    -      A->ZN   R     INV_X4     21      0.386   2.155    8.604

CRE_IN/Fp0074D/ZN              -      A->ZN   R     INV_X8     1       0.190   0.183   13.635

CRE_IN/p0052D/ZN               -      A1->ZN  F     NAND2_X1   1       0.151   0.146   13.781

CRE_IN/EXE_IN/p0136A/ZN        -      B1->ZN  R     AOI22_X1   1       0.525   0.741   37.840

CRE_IN/EXE_IN/p0059A/ZN        -      A->ZN   F     INV_X4     1       0.792   0.398   38.237

CRE_IN/EXE_IN/p_reg[25]/D      -      D       F     SDFF_X2    1       0.372   0.000   38.237

#-------------------------------------------------------------------------------------------
```

```
Path 2: VIOLATED (-30.574 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[25]/CK->D
Group: m_clk
Startpoint: (R) CRE_IN/DE_IN/ir_reg[11]/CK
Clock: (R) m_clk
Endpoint: (F) CRE_IN/EXE_IN/p_reg[25]/D
Clock: (R) m_clk

Capture       Launch
Clock Edge:+      4.000        0.000
Src Latency:+     0.000        0.000
Net Latency:+     5.274 (P)    5.300 (P)
Arrival:=         9.274        5.300

Setup:-           1.282
Uncertainty:-     0.400
Cppr Adjust:+     0.001
Required Time:=   7.592
Launch Clock:=    5.300
Data Path:+      32.866
Slack:=         -30.574

#-------------------------------------------------------------------------------------------
# Timing Point                Flags  Arc     Edge  Cell       Fanout  Trans   Delay  Arrival
#                                                                      (ns)    (ns)     (ns)

#-------------------------------------------------------------------------------------------
CRE_IN/DE_IN/ir_reg[11]/CK     -      CK      R     (arrival)       4  0.139       -    5.300

CRE_IN/DE_IN/ir_reg[11]/Q      -      CK->Q   F     SDFFS_X2        1  0.139   0.651    5.952

CRE_IN/DE_IN/FE_OFCr_11_/Z     -      A->Z    F     BUF_X16        25  0.125   0.498    6.450

CRE_IN/GL_IN/lh_47/p08D/ZN     -      A1->ZN  R     AND2_X2         5  3.448   1.144    9.748

CRE_IN/Fp0161D4462/ZN          -      A->ZN   F     INV_X32         3  0.423   0.132   12.690

CRE_IN/p0161D/ZN               -      A1->ZN  F     NOR2_X4         4  0.421   0.238   13.277

CRE_IN/p0278A/ZN               -      A1->ZN  R     NAND2_X1        1  0.208   0.286   13.562

CRE_IN/M16_IN/ml_14/Fn5D/ZN    -      A->ZN   F     INV_X32        28  1.763   0.760   16.037

CRE_IN/Fp3A/ZN                 -      A->ZN   F     INV_X4          6  2.661   0.966   34.366

CRE_IN/p3A/ZN                  -      B1->ZN  R     AOI22_X1        1  0.407   0.853   36.608

CRE_IN/EXE_IN/p9A/ZN           -      A->ZN   F     INV_X4          1  0.792   0.398   38.166

CRE_IN/EXE_IN/p_reg[25]/D      -      D       F     SDFF_X2         1  0.372   0.000   38.166
```

```
#-------------------------------------------------------------------------------------------
```

- The following command reports all the violating paths (or having slack greater than the value specified with `-max_slack`) between unique source and target register pairs:

```
tempus > report_timing  -to  CRE_IN/EXE_IN/p_reg[25]/D -begin_end_pair -max_slack 10

Path 1: VIOLATED (-30.645 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[25]/CK->D
Group: m_clk
Startpoint: (R) CRE_IN/DE_IN/ir_reg[11]/CK
Clock: (R) m_clk
Endpoint: (F) CRE_IN/EXE_IN/p_reg[25]/D
Clock: (R) m_clk

                Capture      Launch
Clock Edge:+    4.000        0.000
Src Latency:+   0.000        0.000
Net Latency:+   5.274 (P)    5.300 (P)
Arrival:=       9.274        5.300

Setup:-         1.282
Uncertainty:-   0.400
Cppr Adjust:+   0.001
Required Time:= 7.592
Launch Clock:=  5.300
Data Path:+    32.937
Slack:=       -30.645

#-------------------------------------------------------------------------------------------
# Timing Point                 Flags  Arc     Edge  Cell       Fanout  Trans   Delay  Arrival

#                                                                      (ns)    (ns)     (ns)
#-------------------------------------------------------------------------------------------
CRE_IN/DE_IN/ir_reg[11]/CK     -      CK      R     (arrival)    4     0.139       -    5.300
CRE_IN/DE_IN/ir_reg[11]/Q      -      CK->Q   F     SDFFS_X2     1     0.139   0.651    5.952
CRE_IN/DE_IN/FE_O92_ir_11_/Z   -      A->Z    F     BUF_X16     25     0.125   0.498    6.450
CRE_IN/MP_32_IN/p0258A/ZN      -      A2->ZN  R     NAND2_X4     3     0.377   0.460   12.676
CRE_IN/MP_32_IN/Fp0229A/ZN     -      A->ZN   F     INV_X8       5     0.397   0.254   12.929
CRE_IN/MP_32_IN/p0287A/ZN      -      A1->ZN  R     NAND2_X1     1     0.217   0.310   13.239
CRE_IN/EXE_IN/p0136A/ZN        -      B1->ZN  R     AOI22_X1     1     0.525   0.741   37.840
CRE_IN/EXE_IN/p0059A/ZN        -      A->ZN   F     INV_X4       1     0.792   0.398   38.237
CRE_IN/EXE_IN/p_reg[25]/D      <<<    D       F     SDFF_X2      1     0.372   0.000   38.237
#-------------------------------------------------------------------------------------------
```

- The following command shows a path slew propagation report:

```
tempus > report_timing -retime -path_slew_propagation

Path 1: VIOLATED (-15.415 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[27]/CK->D
Retime Analysis:  Data Path-Slew
Group: m_clk
Startpoint: (R) CRE_IN/DE_IN/ir_reg[9]/CK
```

```
Clock: (R) m_clk
Endpoint: (F) CRE_IN/EXE_IN/p_reg[27]/D
Clock: (R) m_clk

                  Capture      Launch
Clock Edge:+      4.000        0.000
Src Latency:+     0.000        0.000
Net Latency:+     4.206 (P)    4.101 (P)
Arrival:=         8.206        4.101

Setup:-            1.201
Uncertainty:-      0.400
Cppr Adjust:+      0.000
Required Time:=    6.605
Launch Clock:=     4.101
Data Path:+        17.919
Slack:=           -15.415
Slack(original):= -16.743

#-------------------------------------------------------------------------------------------
-
# Timing Point                   Flags  Arc     Edge  Cell       Fanout Trans   Delay  Arrival

#                                                                          Retime  Retime
(ns)
#-------------------------------------------------------------------------------------------
-
CRE_IN/DE_IN/ir_reg[9]/CK         -     CK      R     (arrival)   4     0.102       -    4.101

CRE_IN/DE_IN/ir_reg[9]/Q          -     CK->Q   F     SDFFS_X2    1     0.117   0.609    4.710

CRE_IN/DE_IN/FE_86_port_add_1/Z   -     A->Z    F     BUF_X16    44     0.180   0.362    5.072

CRE_IN/CRE_GLU_IN/lsh_47/F4/ZN    -     A->ZN   R     INV_X4     11     1.381   0.835    5.907

CRE_IN/MP_32/M16_IN/m14/p4D/ZN    -     A2->ZN  R     NAND2_X4    4     0.369   0.308   11.530

CRE_IN/MP_32/M16_IN/m14/FpD/ZN    -     A->ZN   F     INV_X32    14     0.155   0.154   11.684

CRE_IN/EXE_IN/n0385D/ZN           -     B1->ZN  R     AOI22_X1    1     0.440   0.371   21.650

CRE_IN/EXE_IN/FE_P0_n_650/Z       -     A->Z    R     BUF_X16     1     0.078   0.294   21.944

CRE_IN/EXE_IN/n0439D/ZN           -     A->ZN   F     INV_X8      1     0.059   0.077   22.020

CRE_IN/EXE_IN/p_reg[27]/D         -     D       F     SDFF_X2     1     0.059   0.000   22.020

#-------------------------------------------------------------------------------------------
-
INFO: Path Based Analysis (PBA) performed on total '1' paths
```

- The following command generates the PBA delay calculation report for the net arc terminating at the specified pin:

```
tempus > report_timing -to CRE_IN/EXE_IN/p_reg[31]/D  -retime  -path_slew_propagation -
retime_delaycal_pins  CRE_IN/EXE_IN/p_reg[31]/D

Path 1: VIOLATED (-28.273 ns) Setup Check with Pin CRE_IN/EXE_IN/p_reg[25]/CK->D
Retime Analysis:  Data Path-Slew SI
Group: m_clk
Startpoint: (R) CRE_IN/DE_IN/ir_reg[11]/CK
Clock: (R) m_clk
Endpoint: (F) CRE_IN/EXE_IN/p_reg[25]/D
Clock: (R) m_clk

                 Capture       Launch
Clock Edge:+     4.000        0.000
Src Latency:+    0.000        0.000
Net Latency:+    5.274 (P)    5.300 (P)
Arrival:=        9.274        5.300

Setup:-             1.254
Uncertainty:-       0.400
Cppr Adjust:+       0.001
Required Time:=     7.620
Launch Clock:=      5.300
Data Path:+        30.593
Slack:=           -28.273
Slack(original):=  -30.645
RDC PBA Report
-------------------------------------
From pin  : CRE_IN/EXE_IN/p0059A/ZN
Voltage   : 0.8 V
Cell      : INV_X4
Library   : PK45_lib_v1.0
To pin    : CRE_IN/EXE_IN/p_reg[25]/D
Voltage   : 0.8 V
Cell      : SDFF_X2
Library   : PK45_lib_v1.0
Base or SI: SI delay
Delay type: net

              Rise   Rise      Fall   Fall      Slew
Thresholds:   Delay  Slew      Delay  Slew      Derate
From pin  :   50     10->90    50     90->10    1
To pin    :   50     10->90    50     90->10    1
-------------------------------------------------------------
From pin waveforms
-------------------------------------------------------------
Fall Waveform :  { 0 0.8 3.4584e-10 0  }
-------------------------------------------------------------
```

```
To pin waveforms
------------------------------------------------------------
Fall Waveform :  { 0 0.8 3.4584e-10 0  }
------------------------------------------------------------

------------------------------------------------------------
RC Summary for net CRE_IN/EXE_IN/n_686
------------------------------------------------------------
Number of capacitance : 7
Net capacitance       : 0.00051675 pF
Total rise capacitance: 0.00299075 pF
Total fall capacitance: 0.00298075 pF
Number of resistance  : 6
Total resistance      : 10.8929 Ohm
------------------------------------------------------------
Underated Delays              Fall
------------------------------------------------------------
Net delay                  : 3.25295e-05 ns
------------------------------------------------------------
Transitions                   Fall
------------------------------------------------------------
Input transition           : 0.276672 ns
Output transition          : 0.276672 ns
------------------------------------------------------------
INFO: Path Based Analysis (PBA) performed on total '1' paths
```

- The following command generates a report that shows timing paths for which `set_max_delay` constraints are defined:

```
report_timing -check_type max_delay -max_paths 10 > 11

Path 1: MET (0.721 ns) Path Delay Check with Pin ff2/D
          Group: CK1
     Startpoint: (R) ff1/Q
          Clock: (R) CK1
       Endpoint: (R) ff2/D
          Clock: (R) CK1

                    Capture         Launch
    Path Delay:+     1.000          -
    Src Latency:+    0.000          0.000
    Net Latency:+    0.010 (P)      0.010 (P)
       Arrival:=     1.010          0.010

      Setup:-           0.100
      Required Time:=   0.910
      Launch Clock:=    0.010
      Data Path:+       0.179
          Slack:=       0.721
      ---------------------------------------------------
      Pin     Edge  Direction  Phase        Arrival  Delay
```

```
                                          Time
      -----------------------------------------------------
      ff1/CK  ^     IN         CK1(C)(P) *  0.010    -
      ff1/Q   ^     OUT        CK1(D)(P) *  0.013    0.003
      buf4/Y  ^     OUT        CK1(D)(P) *  0.023    0.010
      buf5/Y  ^     OUT        CK1(D)(P) *  0.033    0.010
      and1/Y  ^     OUT        CK1(D)(P) *  0.179    0.146
      buf6/Y  ^     OUT        CK1(D)(P) *  0.189    0.010
      ff2/D   ^     IN         CK1(D)(P) *  0.189    0.000
      -----------------------------------------------------

Path 2: MET (0.987 ns) Path Delay Check with Pin out1
          Group: default
     Startpoint: (R) ff2/Q
          Clock: (R) CK1
       Endpoint: (R) out1
          Clock:

                       Capture         Launch
     Path Delay:+       1.000          -
    Src Latency:+       0.000          0.000
    Net Latency:+       0.000 (I)      0.010 (P)
        Arrival:=       1.000          0.010

      Output Delay:-    0.000
     Required Time:=    1.000
      Launch Clock:=    0.010
     Data Path:+        0.003
         Slack:=        0.987
      -----------------------------------------------------
      Pin     Edge  Direction  Phase        Arrival  Delay
                                          Time
      -----------------------------------------------------
      ff2/CK  ^     IN         CK1(C)(P) *  0.010    -
      ff2/Q   ^     OUT        CK1(D)(P)    0.013    0.003
      out1    ^     OUT        CK1(D)(P) *  0.013    0.000
      -----------------------------------------------------
```

report_timing -from latch/D Settings

The `report_timing -from` [register/latch]/CK command settings can be defined in line with the path reported in full_clock path report (as reported path might be clock->lat/CK->lat/Q->ff2/D). So the `report_timing -from` CK-pin can be very intuitive.

However, in latch case scenarios where there are multiple D pins on a latch, Tempus supports `-from` latch/D pin, which is generally used for debug purposes. When propagating across CK->Q arc, the software will perform worst casing across all the D->Q arcs and report the worst path possible. The `report_timing -from` latch/D2 command can be used to propagate arrival only across D2->Q arc, so that no worst casing occurs. And path report or slack with respect to a particular D pin of latch can be attained.

For example:

- The following command uses lat/G (clock pin) , lat/D and lat/D2 (2 data pins) , lat/Q (output/begin pin), and shows the below report:

```
report_timing -to ff2/D -path_type full -fields {hpin edge delay arrival arc} -net -from
lat/D2
------------------------------------------------------------------------
Path 1: MET Setup Check with Pin ff2/CK
Endpoint:   ff2/D (v) checked with trailing edge of 'CK1'
Beginpoint: lat/Q (v) triggered by trailing edge of 'CK1'
Path Groups: {CK1}
Other End Arrival Time          1.400
- Setup                         0.110
+ Phase Shift                   2.500
+ CPPR Adjustment               0.000
= Required Time                 3.790
- Arrival Time                  1.773
= Slack Time                    2.017
     Clock Fall Edge                 1.250
     + Clock Network Latency (Prop)  0.120
     = Beginpoint Arrival Time       1.370
      ------------------------------------------------
      Pin       Edge  Delay  Time   Arrival  Arc
                           Given  Time
                           To
                           Start
                           Point
      ------------------------------------------------
      lat/G     v     -      -      1.370    G v
      lat/Q     v     0.150  0.193  1.713    D2 v -> Q v
      buf9/A    v     0.000  -      1.713    -
      buf9/Y    v     0.030  -      1.743    A v -> Y v
      buf10/A   v     0.000  -      1.743    -
      buf10/Y   v     0.030  -      1.773    A v -> Y v
      ff2/D ->  v     0.000  -      1.773    -
      ------------------------------------------------
```

Related Information

- report_nets
- report_ports
- set_multi_cpu_usage
- set_table_style
- set_input_delay
- set_max_delay
- set_min_delay
- create_analysis_view

report_timing_context_clock_mapping

```
report_timing_context_clock_mapping
[-help]
[-format <text|csv>]
[> <fileName>]
[>> <fileName>]
```

Generates clock mapping report.

By default, Tempus generates the clock mapping report when the `read_timing_context` command is used. The `report_timing_context_clock_mapping` command can be used to generate clock mapping report after `read_timing_context`. This command can generate clock mapping reports in either text format or CSV (helps in easier post-processing of a report).

Parameters

`-help`	Prints out the command usage.
`-format <text | csv>`	Writes the report in text or csv formats, default is `text`.
`> <fileName>`	Writes the report to the specified file name.
`>> <fileName>`	Writes the report to the specified filename.

Examples

- The given below example shows the `report_timing_context_clock_mapping` command output sample CSV report:

```
# View V1
# Inst I1
# Block Clock, Block Clock Period, Block Clock Waveform, Top Clock, Top Clock Period, Top
Clock Waveform, Port, Comment
# Block clocks mapped to top clocks on clock pins/ports:
BCLK1, 10, {0 5}, TCLK1, 10, {0 5}, CPORT1, auto mapped
BCLK2, 10, {0 5}, TCLK2, 20, {0 10}, CPORT2, user mapped
# Block Clocks NOT Mapped to top clocks on clock pin/ports:
BCLK3, 10, {0 5}, NA, NA, NA, CPORT3, No possible match by parameter
# Top clocks NOT mapped to block clocks on clock pins/ports:
NA, NA, NA, TCLK3, 10, {0 5}, CPORT4, No possible match by parameter
# Virtual block clocks NOT mapped to top clocks:
VCLK1, 10, {0 5}, NA, NA, NA, NA, NA
# Block clocks mapped to top clocks on data ports:
VCLK2, 10, {0 5}, TCLK4, 10, {0 5}, DPORT1, auto mapped
# Block clocks NOT mapped to top clocks on data ports:
VCLK2, 10, {0 5}, NA, NA, NA, DPORT2, No possible match by parameter
# Top clocks NOT mapped to block clocks on data ports:
NA, NA, NA, TCLK5, 10, {0 5}, DPORT3, No possible match by parameter
```

Related Commands

- `read_timing_context`
- `write_timing_context`
- `compare_timing_context`

report_timing_derate

```
report_timing_derate
[help]
[-delay_corner delayCornerName]
[-include_inherited]
[collection]
[{> | >>} filename[.gz]]
```

Reports delay scaling or derating factors for early and late clock and data paths in the current design.

By default, the `report_timing_derate` output shows pulse_width_check and clock_period_check details for instance objects (see `set_timing_derate`).

If you specify this command without any parameters, reports are generated for global derating factors for each instance, net, or cell for which derating is asserted.

The following reports are also generated:

- In best case worst case, maximum and minimum corner
- Early values from minimum corner and late values from maximum corner in on-chip variation mode
- In MMMC mode, specified corners, or for all corners when corner is not specified
- Power domains (if present in the design)
- Derates for pulse width and clock period checks. Also, the report will show mean/sigma values, when the `set_timing_derate -mean/-sigma` parameters are specified.

Parameters

`> | >> filename`	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. **Note:** To write a compressed report, add the `.gz` extension to the file name.
`-delay_corner delayCornerName`	Reports derating factors for the specified delay calculation corner only. *Default:* Reports derating factors for all delay calculation corners.

`-include_inherited`	Reports the object names from which the derating factor values are inherited. When specified, the report displays the name of the cell for each instance from which derate values are inherited.
collection	Specifies the list of cells, nets, or library cells.

Examples

- The following command reports derating factors for delay corner `dly_cnr_slow_RX`:

```
tempus > report_timing_derate -delay_corner dly_cnr_slow_RX -include_inherited

---- Report Timing Derate for DC Corner: dly_cnr_slow_RX

---------------------------------Clock---------------------------------Data---------------
                            Rise             Fall             Rise               Fall
                         Early    Late     Early    Late     Early    Late    Early    Late
-----------------------------------------------------------------------------------------

Cell Delay               0.900    1.100    0.900    1.100    0.820    1.010   0.820    1.010
Net Delay Static         0.900    1.100    0.900    1.100    0.820    1.010   0.820    1.010
Net Delay Dynamic        0.900    1.100    0.900    1.100    0.820    1.010   0.820    1.010
-----------------------------------------------------------------------------------------
```

- The following commands display global derating factors:

```
tempus > set_timing_derate 1.2
tempus > set_timing_derate -clock -early 1.1
tempus > set_timing_derate -data -late 1.2
tempus > report_timing_derate

---- Report Timing Derate for DC Corner: def_emul_delay_corner

                      -------------Clock-------------   ----------- Data-------------
                            Rise             Fall             Rise             Fall
                      Early    Late     Early    Late     Early   Late     Early   Late
-----------------------------------------------------------------------------------------

Cell Delay            1.100    1.200    1.100    1.200    1.200   1.200    1.200   1.200
Net Delay Static      1.100    1.200    1.100    1.200    1.200   1.200    1.200   1.200
Net Delay Dynamic     1.100    1.200    1.100    1.200    1.200   1.200    1.200   1.200
Cell Check            --       --       --       --       --      --       --      --
Retain delay          --       --       --       --       --      --       --      --
pulse_width_check     --       --       --       --       --      --       --      --
clock_period          --       --       --       --       --      --       --      --
```

- The following commands display instance derating factors:

```
tempus > set_timing_derate -cell_delay -clock -late 1.3 [get_cells {m_clk_LI89}]
tempus > set_timing_derate -cell_check -late 1.4 [get_cells {m_clk_LI89}]
```

```
tempus > report_timing_derate

                          -------------Clock-----------------    ---------------Data-------
                               Rise           Fall              Rise            Fall
                           Early  Late    Early   Late     Early   Late     Early    Late
                          ------------------------------------------------------------------

Cell (leaf): m_clk_LI89
Cell Delay                  --     1.300   --      1.300    --      --       --       --
Cell Check                  --     1.400   --      1.400    --      1.400    --       1.400
pulse_width_check           --     --      --      --       --      --       --       --
clock_period_check          --     --      --      --       --      --       --       --
```

- The following commands display hierarchical derating factors:

```
tempus > set_timing_derate 1.1 -late [get_cells  {TD_DS_INST}]
tempus > set_timing_derate 1.2 -early [get_cells {TD_DS_INST/TDSP_MUX}]
tempus > set_timing_derate -cell_check 1.2 [get_cells {TD_DS_INST}]
tempus > report_timing_derate -include_inherited

---- Report Timing Derate for DC Corner: def_emul_delay_corner

                          -------------Clock-----------------    ---------------Data----------
---------
                              Rise                  Fall              Rise              Fall
                           Early     Late      Early     Late     Early     Late    Early
Late
                          ------------------------------------------------------------------
---------

Cell Delay                 1.200     1.200     1.200     1.200     1.200     1.200     1.200
  1.200
Net Delay Static           1.200     1.200     1.200     1.200     1.200     1.200     1.200
  1.200
Net Delay Dynamic          1.200     1.200     1.200     1.200     1.200     1.200     1.200
  1.200
Cell Check                    --        --        --        --        --        --        --
     --
Retain delay                  --        --        --        --        --        --        --
     --
pulse_width_check             --        --        --        --        --        --        --
     --
clock_period                  --        --        --        --        --        --        --
     --

Cell (hier): TD_DS_INST
Cell Delay                1.200     1.100     1.200     1.100     1.200     1.100     1.200
  1.100
inherited            dt_rvr_cre       --dt_rvr_cre        --dt_rvr_cre        --dt_rvr_cre
   --
Net Delay Static          1.200     1.100     1.200     1.100     1.200     1.100     1.200
  1.100
```

```
inherited            dt_rvr_cre       --dt_rvr_cre        --dt_rvr_cre        --dt_rvr_cre
   --
Net Delay Dynamic         1.200     1.100     1.200     1.100     1.200     1.100     1.200
  1.100
inherited            dt_rvr_cre       --dt_rvr_cre        --dt_rvr_cre        --dt_rvr_cre
   --
Cell Check                1.200     1.200     1.200     1.200     1.200     1.200     1.200
  1.200
inherited                    --        --        --        --        --        --        --
     --
pulse_width_check            --        --        --        --        --        --        --
     --
clock_period_check           --        --        --        --        --        --        --
     --
inherited                    --        --        --        --        --        --        --
     --
pulse_width_check            --        --        --        --        --        --        --
     --
clock_period_check           --        --        --        --        --        --        --
     --
```

- The following commands display best case worst case derating factors:

```
tempus > set_timing_derate -cell_delay -clock -late 1.2 [get_cells {f2}] -min
tempus > set_timing_derate -cell_delay -late 1.3 [get_cells {f2}] -max
tempus > report_timing_derate

------------------------Report Timing Derate for MIN corner----------------------
                  ---------------Clock-------------  -----------Data-------------
                  Early                Late           Early             Late
                  ------------------------------------------------------------
Cell Delay        --                   --             --                --
Net Delay Static  --                   --             --                --
Net Delay Dynamic --                   --             --                --
Cell Check        --                   --             --                --
Retain delay      --                   --             --                --


Cell (leaf): f2
Cell Delay        --                   1.200000       --                --


------------------------Report Timing Derate for MAX corner----------------------
                  ---------------Clock-------------  -----------Data-------------
                  Early                Late           Early             Late
                  ------------------------------------------------------------
Cell Delay        --                   --             --                --
Net Delay Static  --                   --             --                --
```

```
Net Delay Dynamic --                --              --                --
Cell Check        --                --              --                --
Retain delay      --                --              --                --


Cell (leaf): f2
Cell Delay        --              1.300000          --              1.300000

-------------------------------------------------------------------------------
```

- The following example shows a report that is displayed when mean/sigma derates are applied in the SOCV mode:

```
tempus > set_timing_derate 1.11 -mean -delay_corner def_emul_delay_corner [get_lib_cells
{INV_X32}]
tempus > set_timing_derate 1.11 -sigma -delay_corner def_emul_delay_corner [get_lib_cells
{INV_X32}]
tempus > report_timing_derate [get_lib_cells {INV_X32}]

---- Report Timing Derate for DC Corner: def_emul_delay_corner

                     ------------------------Clock-----------------------          -----------
-----------Data------------------
                            Rise                        Fall                        Rise
              Fall
                     Early        Late          Early         Late         Early        Late
         Early        Late
                    ---------------------------------------------------------------------------
---------------------------------

LibraryCell: FPD4_lib_v1.0/INV_X32
Cell_delay           1.110:1.110  1.110:1.110    1.110:1.110   1.110:1.110  1.110:1.110
1.110:1.110  1.110:1.110    1.110:1.110
Input_switching           --          --             --             --           --
 --           --             --
Retain_delay              --          --             --             --           --
 --           --             --
Input_switching           --          --             --             --           --
 --           --             --
Cell Check                --          --             --             --           --
 --           --             --
pulse_width_check
clock_period_check

-------------------------------------------------------------------------------------------------
---------------------------------
```

Related Commands

- set_timing_derate

report_timing_lib

```
report_timing_lib
[-help]
[-out_file <string>]
[[[-all | -pin | -threshold | -sibling]
[cell_name]
[-by_lib ]] | -library <string>]
```

Reports the contents of the timing library.

Parameters

`-help`	Prints out the command usage.
`-all`	Reports all the pin properties, siblings of a cell, the libraries where they are defined, and the voltage.
`-by_lib`	Organizes the generated report by libraries. If you do not specify this parameter, the `report_timing_lib` command prints out cells with the same name together. Use this parameter to support multi-supply multi-voltage feature of the software, where you can define same cell names in more than one library with different voltage values and timing data.
`cell_name`	Specifies the individual cell name to be reported.
`-library` *`string`*	Reports a specified library. If you do not specify this parameter, all the library names are reported.
`-pin`	Reports the pin properties, such as the pin name, direction, capacitance, max load, max fanout, and fanout load.
`-threshold`	Reports the input, output, and slew thresholds of a cell.
`-sibling`	Reports sibling cells, the libraries where they are defined, and the voltage.
`-out_file` *`string`*	Specifies the name of the file to which the results are written.

Command Order

Use this command after importing the design when the timing library is also loaded.

Examples

- The following command organizes the generated report by libraries and displays all the cell information:

```
tempus > report_timing_lib  -by_lib  -all

Cell: DLH_X1      Library (libset_slow): FrP45_lib_v1.0         0.80000 V.
InThR=50          InThF=50          OutThR=50       OutThF=50
SlewLowThR=10     SlewLowThF=10     SlewUpThR=90     SlewUpThF=90
SlewMLowThR=10    SlewMLowThF=10    SlewMUpThR=90    SlewMUpThF=90

Pin      Dir   Pin Cap   Rise Cap   Fall Cap   Max Load   Max Tran   FO Load  Max FO
D        In    0.001178   0.001180   0.001177      -       5.000000      -        -
G        In    0.000734   0.000739   0.000730      -       5.000000      -        -
Q        Out   0.000000   0.000000   0.000000   0.008000   5.000000      -        -

Sibling cell                      Library                            Voltage
DLH_X1                            FrP45_lib_v1.0(libset_slow)    0.80000
DLH_X1                            FrP45_lib_v1.0(libset_fast)    0.80000
```

- The following command reports the input, output, and slew thresholds of the given cell:

```
tempus > report_timing_lib -by_lib -threshold

Cell AND3_X1

InThR=50          InThF=50          OutThR=50       OutThF=50

SlewLowThR=10     SlewLowThF=10     SlewUpThR=90     SlewUpThF=90

SlewMLowThR=10    SlewMLowThF=10    SlewMUpThR=90    SlewMUpThF=90
```

- The following command organizes the generated report by libraries and reports the pin properties:

```
tempus > report_timing_lib -by_lib -pin

Cell AND3_X4

Pin      Dir   Pin Cap   Rise Cap   Fall Cap   Max Load   Max Tran   FO Load  Max  FO

A1       In    0.000390   0.000394   0.000385      -       5.000000     -        -

A2       In    0.000524   0.000532   0.000517      -       5.000000     -        -

A3       In    0.000355   0.000362   0.000347      -       5.000000     -        -

ZN       Out   0.000000   0.000000   0.000000   0.032000   5.000000     -        -
```

- The following command reports the cell information in the timing library, and writes the results to the file `yz.dmp.lib`:

```
report_timing_lib -out_file yz.dmp.lib
```

Related Commands

- read_libs

report_timing_robustness

```
report_timing_robustness
[-help]
-check_type {setup | hold | setup_CGS | hold_CGH}
[-opt_list string]
[-path_group string]
-run_time_limit integer
-target_PTR float
[-view string]
```

Reports the PTR score of a design.

The Parametric Timing Robustness (PTR) analysis calculates the probability of an entire chip meeting timing requirements, instead of relying on the probabilities on individual paths to compute the robustness of a chip. This command reports the percentage of parts (to be manufactured), which meet timing at all the endpoints.

The PTR score of a design, which is a statistical measure of the timing slack for the whole design, is based on the following steps:

- Enumerates all the instances in a design
- Identifies paths that are critical for PTR of a design
- Computes the PTR-based on Monte-Carlo analysis

Parameters

`-help`	Prints out the command usage.
`-check_type {setup | hold | setup_CGS | hold_CGH}`	Specifies the check type.
`-opt_list string`	Specifies the PTR options list.
`-path_group string`	Specifies the user-defined path groups created using the `group_path` command.
`-run_time_limit integer`	Specifies the runtime limit. Unit in minutes.
`-target_PTR float`	Specifies the target PTR score.
`-view string`	Specifies the single view corresponding to the specified check type.

Examples

- The following examples show the calculated PTR value and the report output:

```
report_timing_robustness -check_type setup -path_group R2R -target_PTR 0.8 -run_time_limit 15
collecting paths and reporting PTR ......
Final PTR 0.89735

report_timing_robustness -check_type setup -path_group R2R -target_PTR 0.95 -run_time_limit 15
collecting paths and reporting PTR ......
Final PTR 0.91295

set P1 [report_timing_robustness -check_type setup -path_group R2R -target_PTR 0.95 -
run_time_limit 15]
collecting paths and reporting PTR ......
Final PTR 0.91295

report_property $P1
-------------------------------------
property             | value
-------------------------------------
slack_mean           | 0.961705
slack_ptr            | 0.912950
slack_quantile       | -2.085161
slack_skewness       | -0.668439
slack_stddev         | 0.682927
-------------------------------------
```

Related Commands

- report_timing
- group_path

report_timing_summary

```
report_timing_summary
[-help]
[-checks {setup | hold | drv | clock}]
[-expand_clocks {launch_capture | capture_only}]
[-expand_views]
[-groups groupname_list]
[-sort_by {name | wns | tns | fep}]
[-views viewName]
```

Generates reports that provide details summarized by the violation type - setup, hold, or DRV. You can sort the rows by WNS, TNS, FEP, or by view name.

Parameters

`-help`	Prints out the command usage.
`-checks {setup | hold | drv | clock}`	Generates a timing report for the specified check types. The valid values are `setup`, `hold`, `drv`, and `clock`. *Default*: `setup hold drv clock`
`-expand_clocks {launch_capture | capture_only}`	Includes per capture or launch/capture clock group details.
`-expand_views`	Includes per analysis view details.
`-groups groupname_list`	Generates a timing report for the specified path group(s). By default, all the path groups are analyzed.
`-sort_by {name | wns | tns | fep}`	Defines the order in which the report will be sorted - views, clocks, or path groups. The valid values are: • `name`: Places the items in alphabetical order. • `wns`: Sorts the output by WNS. • `tns`: Sorts the output by TNS. • `fep`: Sorts the output by FEP. *Default*: `name`
`-views viewName`	Generates a timing report for the specified analysis views only. By default all the active views are analyzed.

Examples

- The following command generates a summary report by violation type for the specified view:

```
tempus >  report_timing_summary -views scan_slow_RCMIN

# SETUP                     WNS    TNS   FEP
#-----------------------------------------------
 View : scan_slow_RCMIN   4.922  0.000     0
    Group : Async        15.286  0.000     0
    Group : in2reg       15.726    0.0     0
#-----------------------------------------------
# HOLD                      WNS     TNS   FEP
#-----------------------------------------------
 View : scan_slow_RCMIN  -0.409  -1.354    15
    Group : Async         3.504   0.000     0
    Group : in2reg       -0.409  -1.354    15
#-------------------------------------------------
# DRV                          WNS     TNS   FEP
#-------------------------------------------------
 View : scan_slow_RCMIN
    Check : max_transition   -2.648  -5.786     7
    Check : min_transition      N/A     N/A     0
 View : scan_slow_RCMIN
    Check : max_transition   -2.648  -5.786     7

#-------------------------------------------------------------
# Clock checks                                WNS    TNS   FEP
#-------------------------------------------------------------
 View : scan_slow_RCMIN
    Check : min_pulse_width (endpoints)   17.548  0.000     0
    Check : min_period                    38.493  0.000     0

#-------------------------------------------------------------
```

- The following command generates a summary report as per launch/capture clock group:

```
tempus >  report_timing_summary -views scan_slow_RCMIN -expand_clocks  launch_capture

# SETUP                                     WNS    TNS   FEP
#-------------------------------------------------------------
 View : scan_slow_RCMIN                   4.922  0.000     0
```

```
    Group : Async                          15.286  0.000     0
    Group : in2reg                         15.726    0.0     0
       Clock : refclk->m_rcc_clk           15.726  0.000     0
       Clock : m_rcc_clk->m_rcc_clk        15.726  0.000     0

    Group : reg2reg                         4.922    0.0     0
       Clock : m_rcc_clk->m_rcc_clk        36.191  0.000     0
       Clock : m_clk->m_rcc_clk             4.922  0.000     0

    Group : in2out                            N/A    N/A     0
    Group : reg2out                        31.987  0.000     0
       Clock : m_clk->refclk               31.987  0.000     0
       Clock : m_digit_clk->refclk         35.831  0.000     0

#------------------------------------------------------------
# HOLD                                      WNS     TNS   FEP
#------------------------------------------------------------
 View : scan_slow_RCMIN                   -0.409  -1.354    15
    Group : Async                          3.504   0.000     0
    Group : in2reg                        -0.409  -1.354    15
       Clock : refclk->m_rcc_clk          22.583   0.000     0
       Clock : m_rcc_clk->m_rcc_clk       22.583   0.000     0

    Group : reg2reg                           N/A     N/A     0
    Group : in2out                            N/A     N/A     0
    Group : reg2out                         2.499   0.000     0
       Clock : m_clk->refclk                7.064   0.000     0
       Clock : m_digit_clk->refclk          2.499   0.000     0

#------------------------------------------------------------
# DRV                           WNS     TNS   FEP
#------------------------------------------------------------
 View : scan_slow_RCMIN
    Check : max_transition   -2.648  -5.786     7
 View : scan_slow_RCMIN
    Check : max_transition   -2.648  -5.786     7

#------------------------------------------------------------
```

```
# Clock checks                                  WNS    TNS   FEP
#-------------------------------------------------------------
 View : scan_slow_RCMIN
    Check : min_pulse_width (endpoints)   17.548  0.000     0
       Clock : m_rcc_clk                  17.896  0.000     0
       Clock : m_ram_clk                  18.270  0.000     0
#-------------------------------------------------------------
```

- The following example shows a sample `report_timing_summary` command output:

```
# SETUP NSIGMA WNS TNS FEP
#-----------------------------------------
View : ALL * -0.021 -18.719 5715
Group : Async * -0.003 -0.016 21
Group : ClockGate * -0.015 -0.920 393
Group : in2out * N/A N/A 0
Group : reg2out * N/A N/A 0
Group : in2reg * N/A N/A 0
Group : reg2reg * -0.021 -17.783 5301
#-----------------------------------------
# DRV NSIGMA WNS TNS FEP
#-----------------------------------------
View : ALL *
Check : max_transition * -0.028 -22.700 9356
Check : min_transition * N/A N/A 0
Check : max_capacitance * N/A N/A 0
Check : min_capacitance * N/A N/A 0
Check : max_fanout * N/A N/A 0
Check : min_fanout * N/A N/A 0
#-----------------------------------------
```

Related Commands

- report_timing

report_wire_load

```
report_wire_load
[-help]
[-instances name_or_collection_of_hierarchical_instances]
[-ports name_or_collection_of_ports]
[-late | -early]
[> | >> filename]
```

Reports the wireload name, library, wire load mode and selection type for all hierarchical instances in the design.

Note: To use the wireload model functionality, you must enable the `enable_wire_load_model_support` attribute using the following command:

```
set_var enable_wire_load_model_support 1
```

Parameters

`-help`	Prints out the command usage.
`>|>> filename`	When specified with `>`, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with `>>`, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. The *`filename`* parameter must be the last argument in the list. **Note:** To write a compressed report, add the `.gz` extension to the file name. *Default:* Report is displayed on standard output without being saved.
`-instances name_or_collection_of_hierarchical_instances`	Specifies the name of an instance or collection for which the `report_wire_load` command reports the wireload name, library, wire load mode, and selection type. The `report_wire_load` command also supports wildcard names such as U*.
`-late | -early`	Reports wireload models in setup or hold analysis mode. By default, the software reports wireload models for the mode that you set using the command.

`-ports` *name_or_collection_of_ports*	Reports the wireload details for the specified ports.

Examples

- The following command reports the wireload details for the specified ports.

```
report_wire_load -ports clk1 -early

---------------------------------------------------------------------------------------------
-----

Design Name      top_chip

Wire Load Mode   enclosed


Port     Module      WireLoadModel    Library                              SelctionType

---------------------------------------------------------------------------------------------
-----

clk1     top_chip    MACRO_15         UX8L_wide1_1.1V_TYP_primitive_mvt    automatic-by-area

---------------------------------------------------------------------------------------------
-----
```

Related Commands

- set_wire_load_model
- reset_wire_load_model

reset_instance_library

```
reset_instance_library
[-help]
```

Resets the settings/binding done using the `set_instance_library` command.

Parameters

`-help`	Prints out the command usage.

Related Commands

- `set_instance_library`

reset_path_adjust

```
reset_path_adjust
[-help]
-path_adjust_group path_adjust_group_name
-view string
[-setup | -hold]
```

Removes the specified slack adjustment values for timing paths that are set using the set_path_adjust command.

Parameters

-help	Prints out the command usage.
-hold	Resets the adjust value for early paths.
-path_adjust_group *path_adjust_group_name*	Specifies the name of path adjust group that defines the path for which the adjustment will be applied.
-setup	Resets the adjust value for late paths.
-view *string*	Specifies the view for which adjustment will be reset.

Related Commands

- set_path_adjust

reset_path_adjust_group

```
reset_path_adjust_group
[-help]
[-name group_name | -all ]
```

Removes all path adjust groups that are set using the `set_path_adjust_group` command for setting path slack adjustment values.

Parameters

`-help`	Prints out the command usage.
`-all`	Removes all the path adjust groups.
`-name group_name`	Removes the named path adjust group completely.

Related Commands

- `set_path_adjust_group`
- `set_path_adjust`

reset_sdf_assertions

`reset_sdf_assertions`

Resets the SDF delays to the values defined in the library. When you perform timing analysis, the software uses the SDF values defined with the `read_sdf` command.

You can use this command to discard the delay values specified with the SDF files, and use the delays defined in the library.

Note: You do not need to read in the SDF delays every time you perform delay calculations. The software uses the SDF files specified earlier in the flow.

Parameters

`-help`	Prints out the command usage.

Related Commands

- `read_sdf`

reset_timing_derate

```
reset_timing_derate
[-help]
[-delay_corner delayCornerName]
[-increment]
[obj_list]
```

Resets derate factors specified on a design or a list of instances (cells, net, or library cells). All the derating factors that are set globally or on specific instances are reset to a default value of `1.0`.

Derating factors for early or late paths in the design are set using the `set_timing_derate` command.

Parameters

`-help`	Prints out the command usage.
`-delay_corner` *delayCornerName*	Specifies the delay corner or a list of delay corners on which reset derating is to be applied.
`-increment`	Resets the incremental derate values that are set using the `set_timing_derate` command. The `set_timing_derate` command saves two different values – the “base” derate and the “incremental” derate. When the `-incremental` parameter is used, the software populates the incremental derates. The `reset_timing_derate` command will reset the incremental derate values only.
obj_list	Specifies the list of cells, nets, or library cells to reset. If the *obj_list* contains hierarchical cells then all cells within the hierarchical cells and across lower hierarchies have reset derate factors.

Examples

- The following example resets all the object-specific and global derating factors:
  ```
  reset_timing_derate *
  ```
- The following example resets only global derating factors:

```
reset_timing_derate [current_design]
```

- The following example resets derating factor values for the specified delay corner:
  ```
  reset_timing_derate -delay_corner delayCornerName
  ```
- The following example resets the global derating values for the specified delay corner:
  ```
  reset_timing_derate -delay_corner delayCornerName [current_design]
  ```

Related Commands

- set_timing_derate

reset_wire_load_mode

`reset_wire_load_mode`

Resets the wire-load mode to the default mode of `top`.

Note: To use the wireload model functionality, you must enable the `enable_wire_load_model_support` attribute using the following command:

```
set_var enable_wire_load_model_support 1
```

Related Commands

- `set_wire_load_mode`

reset_wire_load_model

```
reset_wire_load_model
[-help]
[{ obj_list }]
```

Resets wireload models for the specified list of objects. The list of objects can include designs, hierarchical cells, or ports.

When objects are not specified, the software removes the wireload model specified using the `set_wire_load_model` command from the current instance only.

Note: To use the wireload model functionality, you must enable the `enable_wire_load_model_support` attribute using the following command:

```
set_var enable_wire_load_model_support 1
```

Parameters

`-help`	Outputs a brief description that includes type and default information for each `reset_wire_load_model` parameter. For a detailed description of the command and all of its parameters, use the man command: `man reset_wire_load_model`.
`obj_list`	Specifies the list of instances or ports for which to remove the wireload models.

Examples

- The following command removes the wireload model for `clk1`:

  ```
  reset_wire_load_model clk1
  ```

Related Commands

- `set_wire_load_model`

reset_wire_load_selection_group

```
reset_wire_load_selection_group
[-help]
[{ obj_list }]
```

Removes the wireload selection group for the specified list of objects. The list of objects can include designs, hierarchical cells, or ports. When objects are not specified, wireload selection groups are removed for all objects.

Note: To use the wireload model functionality, you must enable the `enable_wire_load_model_support` attribute using the following command:

```
set_var enable_wire_load_model_support 1
```

Parameters

`-help`	Prints out the command usage.
obj_list	Specifies the list of object names for which to remove the wireload selection groups.

Examples

- The following command removes the wireload selection group for `clk1`:

  ```
  reset_wire_load_selection_group clk1
  ```

Related Commands

- `set_wire_load_selection_group`

select_boundary_model_constraints

```
select_boundary_model_constraints
[-help]
[-block string]
[-define_bundled_bus string]
[-instance string]
[-set_instance_library <string>]
[-set_quiet_attacker string]
```

Applies boundary model constraints to specific blocks/instances.

Parameters

`-help`	Prints out the command usage.
`-block` *string*	Specifies the block for which the constraints will be applied.
`-define_bundled_bus` *string*	Applies the `define_bundled_bus` constraints.
`-instance` *string*	Specifies the instance for which the constraints will be applied.
`-set_instance_library` <string>	Applies the instance library constraints defined with the `set_instance_library` command.
`-set_quiet_attacker` *string*	Applies the `set_quiet_attacker` constraints.

Related Commands

- `report_boundary_model_constraints`
- `define_bundled_bus`
- `set_quiet_aggressor`
- `set_instance_library`

set_analysis_view

```
set_analysis_view
[-help]
[-drv string]
[-dynamic string]
[-hold string]
[-leakage string]
[-setup string]
[-update_timing]
```

Defines the analysis views to use for setup and hold analysis and optimization.You must define at least one setup and one hold analysis view. The "active" views represent the different design variations that will be analyzed. These views can be changed throughout the flow to utilize different subsets of views. The libraries and data are loaded into the system that are required to support the selected set of active views.

When the `set_analysis_view` command is issued, it will cause a full timing reset. During a timing reset, all the interconnect parasitics, delays, and timing slacks will be recalculated.

The order in which you specify the views with the `-setup` and `-hold` parameters is important. By default, the first views defined in the `-setup` and `-hold` lists are the default views. The default views serve two purposes:

- Provide the library data that is used during the design initialization process.
- Indicate to parts of the application, that are not MMMC-aware, which view's information to use for a particular analysis step (in most cases, the default view is the first setup or hold view). You can use the `set_default_view` command to change the default view to any currently active view without incurring a timing reset. This control is not frequently used since most applications either support MMMC or provide a direct control for selecting the appropriate view to use for that specific analysis.

Using the `set_default_view` command does not affect software performance because it only uses views that are already active in the design. If you use the `set_analysis_view` command to change the default views, the existing timing and delay calculation data is reset.

Use this command after creating multiple analysis views (`create_analysis_view`).

Note: The `set_analysis_view` command does not work in an incremental manner for setting parameters. Each invocation of this command must include the full specification of all the active views required for analysis.

Parameters

`-help`	Prints out the command usage.

`-drv` *string*	Selects specific views for DRV analysis, e.g., max_transition and max_capacitance checking. When this parameter is specified, additional analysis views are specified for DRV analysis, so the timing system efficiently computes the required data for these checks. This methodology is efficient with respect to memory and performance, because all the views as full Setup views are not included. The following guidelines should be followed when selecting DRV views: • Any setup views that require DRV analysis should not be specified as DRV views. The setup views are automatically considered for DRV. • If a subset of hold-only views require DRV analysis, they should be specified as DRV views as well. The timing system will augment the timing information normally computed for Hold views with additional data required for the DRV analysis. • Additional views that are not active setup or hold views can be specified as DRV only views. The system will efficiently compute the required data for performing accurate DRV checks.
`-dynamic` *string*	Sets a view as dynamic view. Delay calculation is required to provide transition times. Delay calculation should not perform SI analysis if the delay corner is only required for the dynamic power. The timer is not required to propagate required times or perform slack computations.
`-hold` *list_of_views*	Sets the active views for hold analysis.
`-leakage` *string*	Sets a view as leakage view. The timing libraries must be loaded, if not already in use. The Timer constant propagation is required, whereas delay calculation is not required.
`-setup` *list_of_views*	Sets the active views for setup analysis.

`-update_timing`	Reloads the configuration described by the MMMC view and mode objects. When the timing attribute `timing_defer_mmmc_obj_updates` is set to `true`, this parameter is required to update the MMMC timing environment. The `-update_timing` parameter may be specified without any `-setup` or `-hold` arguments. In this case, the data associated with the current set of setup and/or hold views will be loaded. If no active setup or hold views exist, then using this parameter alone may display an error. The `-update_timing` parameter may also be used in conjunction with the `-setup` and `-hold` options. In this case, the newly specified setup/hold view configuration will be loaded with the updated MMMC object information.

When you issue the `set_analysis_view` command, the software will perform a timing reset, so that only the new active view information is loaded up. The software requires the `-setup` and `-hold` view parameters to be explicitly specified for each `set_analysis_view` command run. If these parameters are not specified, the software issues an error message. Any optional settings, such as, `-leakage` or `-dynamic` views will be reset if they are not specified with the new `set_analysis_view` command.

Note: It is recommended to use the same power view for both leakage and dynamic, that is, both the `-dynamic` and `-leakage` parameters should have the same user-specified view settings.

Examples

- The following command sets `missionWCCOM` and `mission2WCCOM` as the active views for setup analysis, and `missionBCCOM` and `testBCCOM` as the active views for hold analysis:

  ```
  set_analysis_view -setup {missionWCCOM mission2WCCOM} -hold {missionBCCOM testBCCOM}
  ```

Related Commands

- `create_analysis_view`
- `set_default_view`

set_boundary_model_clock_mapping

```
set_boundary_model_clock_mapping
[-help]
-block_clock <string>
-instance <string>
-top_clock <string>
 [-view <string>]
```

Provides the boundary model view mapping data.

Parameters

`-help`	Prints out the command usage.
`-block_clock <string>`	Specifies the block view.
`-instance <string>`	Specifies the name of the instance.
`-top_clock <string>`	Specifies the name of the top clock.
`-view <string>`	Specifies the name of the view.

Related Commands

- read_boundary_model
- set_boundary_model_view_mapping

set_boundary_model_view_mapping

```
set_boundary_model_view_mapping
[-help]
-block_view <string>
-top_view <string>
```

Provides the boundary model view mapping data.

Parameters

`-help`	Prints out the command usage.
`-block_view <string>`	Specifies the name of the block view.
`-top_view <string>`	Specifies the name of the top view.

Examples

- The following command specifies the block view name as `bview`, and the corresponding view name at the top level is `tview`:

  ```
  set_boundary_model_view_mapping -block_view bview -top_view tview
  ```

Related Commands

- `read_boundary_model`
- `set_boundary_model_clock_mapping`

set_default_view

```
set_default_view
[-help]
{-setup newDefaultSetupView | -hold newDefaultHoldView}
```

Temporarily changes the default active analysis view to a different active view.

Some Tempus applications can function only on a single analysis view at a time. By default, these single-view applications use the default analysis view. If an application or flow step does not provide a native option for specifying which view to use, the `set_default_view` command can be used to temporarily change the default view to a different active view that is better suited to that flow step.

Note: Using the `set_default_view` command does not affect software performance because it only uses views that are already active in the design. If you use the `set_analysis_view` command to change the default views, the existing timing, delay calculation, and RC data is reset.

Use this command after defining active views for the design (`set_analysis_view`).

Parameters

`-help`	Prints out the command usage.
`-hold` *`newDefaultHoldView`*	Changes the default active hold analysis view to the specified analysis view.
`-setup` *`newDefaultSetupView`*	Changes the default setup analysis view to the specified analysis view.

Examples

- The following command sets `missionWCCOM`, `mission2WCCOM`, and `missionSlow` as the active views for setup analysis, and `missionBCCOM` and `testBCCOM` as the active views for hold analysis. The default active setup view is `missionWCCOM`, and the default active hold view is `missionBCCOM`.

  ```
  set_analysis_view -setup {missionWCCOM mission2WCCOM missionSlow} -hold {missionBCCOM testBCCOM}
  ```

- The following command changes the default active setup view to `missionSlow`:

  ```
  set_default_view -setup missionSlow
  ```

- The following command changes the default active hold view to `testBCCOM`:

  ```
  set_default_view -hold testBCCOM
  ```

Related Commands

- create_analysis_view

set_instance_library

```
set_instance_library
[-help]
-library string
-instance string
[-early  | -late]
```

Binds a cell library to an instance. When specified, the software overrides the default binding mechanism for a cell from the library set.

Note: The `set_instance_library` command cannot be used at the command prompt - this command can be specified in the ViewDefinition file. For more details, refer to the *Tempus User Guide*.

Parameters

`-help`	Prints out the command usage.
`-early`	Uses the library binding for early timing analysis.
`-instance <string>`	Specifies the instance for which the library binding needs to be done.
`-late`	Uses the library binding for late timing analysis.
`-library <string>`	Specifies the library file path.

Examples

The following commands specify the instance for which the library binding will be done:

- `set_instance_library -library newcell.lib -instance {i1}`
- `set_instance_library -library mysil.lib -instance {i1 i2}`
- `set_instance_library -library {new.lib mysil.lib} -instance {i1} -early`
- `set_instance_library -library {mynew.lib mysil.lib} -instance {i1 i2} -late`

Related Commands

- `reset_instance_library`

set_interactive_constraint_modes

```
set_interactive_constraint_modes
[-help]
{constraint_mode_list}
```

Puts the software into interactive constraint entry mode for the specified multi-mode multi-corner constraint mode objects. Any timing constraints that you specify after this command take effect immediately on all active analysis views that are associated with the specified constraint modes. By default, no constraint modes are considered active.

The software stays in interactive mode until you exit by specifying the `set_interactive_constraint_modes` command with an empty list:

```
set_interactive_constraint_modes { }
```

All new assertions are saved in the SDC files of the specified constraint modes when you save the design (`write_db`).

Use the `get_interactive_constraint_modes` command to return a list of the constraint mode objects in constraint entry mode.

Note: Interactive constraint mode only works when the software is in multi-mode multi-corner timing analysis mode. In min/max analysis mode, constraints are always accepted interactively.

Parameters

`-help`	Prints out the command usage.
constraint_mode_list	Specifies the constraint mode objects to update with the new assertions.

Examples

- The following commands put the software into interactive constraint entry mode, and apply the `set_propagated_clock` assertion on all views in the current session that are associated with the constraint mode func1:

  ```
  tempus> set_interactive_constraint_modes func1
  tempus> set_propagated_clock [all_clocks]
  ```

  When you specify `write_db` command, the updated constraints file for the func1 constraint mode will include the `set_propagated_clock` constraints.

- The following commands put the software into interactive constraint entry mode, and apply the `set_propagated_clock` assertion on all active analysis views in the current session.

  ```
  tempus> set_interactive_constraint_modes [all_constraint_modes -active]
  tempus> set_propagated_clock [all_clocks]
  ```

  When you specify the `write_db` command, the `set_propagated_clock` assertion will be saved persistently as part of a new .sdc file.

Related Commands

- `get_interactive_constraint_modes`
- `get_db`
- `write_db`

set_io_thresholds

```
set_io_thresholds
[-help]
[-slew_lower_threshold_pct pctValue]
[-slew_upper_threshold_pct pctValue]
[-input_threshold_pct pctValue]
[-output_threshold_pct pctValue]
[-slew_lower_threshold_pct_rise pctValue]
[-slew_lower_threshold_pct_fall pctValue]
[-slew_upper_threshold_pct_rise pctValue]
[-slew_upper_threshold_pct_fall pctValue]
[-input_threshold_pct_rise pctValue]
[-input_threshold_pct_fall pctValue]
[-output_threshold_pct_rise pctValue]
[-output_threshold_pct_fall pctValue]
```

Sets the threshold trip-points used for delay calculation to and from the primary I/Os. The `set_io_thresholds` command does not affect the interpretation of threshold settings in the Liberty timing libraries; the effect is strictly limited to the I/O ports.

If you do not specify I/O thresholds with the `set_io_thresholds` command, delay calculation assumes that the I/O thresholds are equivalent to those of the logic gates to which they are connected.

The `set_io_thresholds` command supports separate rise and fall slew thresholds as normally found in Liberty timing libraries.

Note: Some third-party applications derive the I/O threshold values from the first Liberty timing library in the library search path. The software currently does not support this methodology. Explicit setting of I/O thresholds with the `set_io_thresholds` command should be used to complete a compatible analysis environment.

Parameters

`-help`	Prints out the command usage.
`-input_threshold_pct` *pctValue*	Specifies the slew input threshold value.
`-input_threshold_pct_fall` *pctValue*	Specifies the measurement point for a falling signal that will be used for calculating the interconnect delay from a primary input to internal logic. *Value:* `0 - 100` *Default:* `50`

`-input_threshold_pct_rise` *pctValue*	Specifies the measurement point for a rising signal that will be used for calculating the interconnect delay from a primary input to internal logic. *Value:* `0 - 100` *Default:* `50`
`-output_threshold_pct` *pctValue*	Specifies the slew output threshold value.
`-output_threshold_pct_fall` *pctValue*	Specifies the measurement point for a falling signal that will be used for calculating the interconnect delay from internal logic to a primary output. *Value:* `0 - 100` *Default:* `50`
`-output_threshold_pct_rise` *pctValue*	Specifies the measurement point for a rising signal that will be used for calculating the interconnect delay from internal logic to a primary output. *Value:* `0 - 100` *Default:* `50`
`-slew_lower_threshold_pct` *pctValue*	Specifies the slew lower threshold value.
`-slew_lower_threshold_pct_fall` *pctValue*	Specifies the lower trip-point for a falling signal that will be used to measure transition times. *Value:* `0 - 100` *Default:* `20`
`-slew_lower_threshold_pct_rise` *pctValue*	Specifies the lower trip-point for a rising signal that will be used to measure transition times. *Value:* `0 - 100` *Default:* `20`
`-slew_upper_threshold_pct_fall` *pctValue*	Specifies the upper trip-point for a falling signal that will be used to measure transition times. *Value:* `0 - 100` *Default:* `80`

`-slew_upper_threshold_pct_rise` *pctValue*	Specifies the upper trip-point for a rising signal that will be used to measure transition times. *Value:* `0` - `100` *Default:* `80`
`-slew_upper_threshold_pct` *pctValue*	Specifies the slew lower rise threshold value.

Examples

- The following command specifies trip-points for measuring delay and transition times:

```
set_io_thresholds -slew_lower_threshold_pct_rise 10

-slew_upper_threshold_pct_fall 80
-slew_upper_threshold_pct_rise 90
-slew_lower_threshold_pct_fall 20
-input_threshold_pct_rise 40
-input_threshold_pct_fall 70
-output_threshold_pct_rise 60
-output_threshold_pct_fall 30
```

set_ipd_rail_voltage

```
set_ipd_rail_voltage
[-help]
[-exclude_voltage_signatures <List of combinations to be excluded>]
[-add  | -sweep <List of power rails and corresponding voltage sweep list>]
```

Specifies the PG net voltage and excludes those voltage combinations that do not require analysis. When power intent is specified through CPF/IEEE-1801, the `set_ipd_rail_voltage` specification refers to the primary power net of power domain.

Parameters

`-help`	Prints out the command usage.
`-add`	Specified with the `-exclude_voltage_signatures` parameter to exclude voltage combinations incrementally.
`-exclude_voltage_signatures <List of combinations to be excluded>`	Specifies the list of voltage combinations to be excluded from all the possible combinations of voltage.
`-sweep <List of power rails and corresponding voltage sweep list>`	Specify the list of power rails and their corresponding voltage values.

Examples

- The following command is used to specify the PG nets in the design and the corresponding voltages (in volts):

  ```
  tempus > set_ipd_rail_voltage -sweep { PGN1 {0.7 0.8} PGN2 {0.9 1.0} PGN3 {1.1 1.2} PGN4 {0.6 0.7} }
  ```

- The following command removes a particular combination from the analysis:

  ```
  tempus > set_ipd_rail_voltage -exclude_voltage_signatures { {PGN1 0.7 PGN2 0.9 PGN3 1.2 PGN4 0.6} }
  ```

- The following command excludes more combinations:

  ```
  tempus > set_ipd_rail_voltage -exclude_voltage_signatures { {PGN1 0.7 PGN2 0.9 PGN3 1.2 PGN4 0.7} } -add
  ```

Related Commands

- `create_pg_net_group`
- `report_pg_net_group`
- `create_ipd_configuration`
- `report_ipd_configuration`
- `write_ipd_scope`
- `read_ipd_scope`
- `report_ipd_crossings`

Related Information

- For more information, refer to the Inter Power Domain (IPD) Analysis chapter in the *Tempus Stylus User Guide*.

set_max_cap_per_freq

```
set_max_cap_per_freq
[-help]
-cap {cap1 cap2 …}
[-force]
-freq {freq1 freq2 …}
[-lib lib_name]
-pins {cell pin}
```

Adds a maximum capacitance per frequency table to the existing `max_capacitance` statement in the timing library. Timing analysis uses this table to detect violations and the software attempts to repair violations.

The timing engine derives the frequency of an instance from the clock domain to which it is connected (the fastest clock domain, if there is more than one). If the instance is a clock instance (that is, part of the clock tree), the actual clock frequency is used. If the instance is part of the combinational logic (that is, a flip-flop, or the logic connected to the output of a flip-flop), the frequency used equals one-half of the associated clock domain frequency because a flip-flop output can only toggle at one-half the rate of the input clock. For example, a clock of 1.0 Mhz has 1,000,000 rise and fall transitions per second; however the output of a flip-flop connected to a 1.0 Mhz clock can only have 500,000 rise and fall transitions per second.

Timing analysis computes the frequency directly from the timing graph.

You can use this command any time after loading the timing library files (that is, after importing or restoring the design).

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_max_cap_per_freq`parameter. For a detailed description of the command and all of its parameters, use `man set_max_cap_per_freq`.
`-cap {`*`cap1cap2 …`*`}`	Specifies a list of capacitance values that must be greater than or equal to 0. You must specify the same number of -freq and -cap values. The software derives the maximum capacitance from a linear interpolation (or extrapolation) for a given frequency value using two frequency values in the table. For interpolation, the software uses the closest value above the frequency, and the closest value below the frequency. For extrapolation, the software uses the two closest values to the frequency. *Type:* Float, specified in units of pF
`-force`	Computes the maximum capacitance from the interpolated `-freq` and `-cap` values only, and ignores any previously defined `max_capacitance` values for these pins in the library. *Default:* Computes the maximum capacitance using the worst case (minimum value) between the library`max_capacitance` values and the interpolated `-cap`values.
`-freq {`*`freq1freq2 …`*`}`	Specifies a list of at least two frequency values that must be greater than or equal to 0. You must specify the same number of -freq and -cap values, and the frequency values must be specified in ascending order. *Type:* Float, specified in units of Hz
`-lib` *`libName`*	Modifies only the matching cell-pins in the specified library. *Default:* Modifies the matching cell-pins in all libraries
`-pins {`*`cellpin`*`}`	Adds the new capacitance constraints to the specified set of cells and pins. You can specify output or inout pins. The asterisk (*) wildcard character can be used in a cell or pin name. If a table associated with the pin already exists, it is replaced with the new table. For analysis and optimization, the software uses the most restrictive (the lowest capacitance) value from either the table or the .lib max_capacitance. If you also specify the -force parameter, the max_capacitance value is ignored. The final value can be further modified using .sdc set_max_capacitance statements.

Command Order

Use this command any time after loading the timing library files (that is, after importing or restoring the design).

Examples

Assume all output pins of a library have a `max_capacitance` of 3.5 pF from the library file.

- The following command sets the frequency dependent `max_capacitance` to 3.0 pF at a frequency of 100 Mhz, and to 1.5 pF at a frequency of 200 Mhz, for all output or inout pins for all cells in the library named `fast`:

  ```
  tempus>  set_max_cap_per_freq -lib fast -freq {100e6 200e6} -cap {3.0 1.5} -pins {* *}
  ```

  As a result, for a net at 300 Mhz, an and2 cell output pin has an extrapolated `max_capacitance` value of 0 pF (which means the cell cannot be used at that frequency). An extrapolation to 50 Mhz results in a `max_capacitance` value of 3.75 pF. However, the 3.75 pF value is greater than the existing `.lib max_capacitance` limit of 3.5 pF; therefore timing analysis uses the 3.5 pF value.

- The following command applies the same constraints to all libraries with cell names matching `dff*` that have a pin named `Qbar`:

  ```
  tempus>  set_max_cap_per_freq -force -freq {100e6 200e6} -cap {3.0 1.5} -pins {dff* Qbar}
  ```

  Note: The original `.lib max_capacitance` limit is ignored because the `-force` parameter is specified; therefore extrapolation to 50 Mhz results in a final `max_capacitance` constraint value of 3.75 pF.

Related Commands

- `set_max_transition_per_freq`

set_max_cap_per_freq_transition

```
set_max_cap_per_freq_transition
[-help]
-cap { {cap11 cap12 cap13…} {cap21 cap22 cap23} {cap31 cap32 cap33…}}
[-force]
-freq {freq1 freq2 freq3…}
[-input_pin inputpin]
[-lib libName]
-pins {cell outputpin}
-transition {freq1 freq2 ...}
```

Sets the maximum capacitance of a pin in the design. Considers both the input transition time and the output frequency. If the maximum capacitance exceeds this value, the `report_constraint -drv_violation_type max_capacitance` command displays a capacitance violation.

You can use this command any time after loading the timing library files (that is, after importing or restoring the design).

Parameters

`-help`	Outputs a brief description that includes type and default information for each`set_max_cap_per_freq_transition` parameter. For a detailed description of the command and all of its parameters, use `man set_max_cap_per_freq_transition`.
`-cap {{cap11cap12 cap13…} {cap21 cap22 cap23…} {cap31 cap32 cap33…}}`	Specifies a table of capacitance values with the maximum output pin capacitances associated with `-freq` and `-transition` values. The total number of `-cap` values must be the total number `-freq` values times the total number of `-transition` values. The `-freq` and `-transition` values must be specified before `-cap`. The capacitance values are floats in units of pico farad and all values must be greater than or equal to zero. The capacitance table uses frequency as the first index and the input transition as the second index. For example, `cap11`,`cap12`, and `cap13` are for `freq1`, while `c12`, `c22`, and `c32` correspond to the input transition `tran2`. The maximum capacitance is derived from a linear interpolation (or extrapolation) for a given frequency value and a transition value using two frequency values and two transition values n the table (closest above and below for interpolation, the two closest for extrapolation. *Type:* Float, specified in units of pF

`-force`	Computes the maximum capacitance from the interpolated `-freq, -transition,` and `-cap` values only, and ignores any previously defined`max_capacitance` values for these pins in the library. *Default:* (`-force` not specified) Computes the maximum capacitance using the worst case (minimum value) between the library `max_capacitance` values and the interpolated `-cap` values.
`-freq {`*`freq1freq2 freq3…`*`}`	Specifies a list of at least two frequency values with the toggle frequencies at the output pin. Frequency values must be increasing, and the total number of frequency values cannot exceed 255. *Type:* Float, specified in units of Hz. All values must be equal to or greater than `0`.
`-input_pin` *`inputpin`*	Specifies the input pin name corresponding to the input transition values in as specified using the `-transition` parameter. If the *`-inputpin`* parameter is not specified, then `-transition` parameter values correspond to all the input pins.
`-lib` *`libName`*	Specifies the library name from inside a `.lib` file to update the cell-pins. *Default:* If a library name is not specified, modifies the matching cell-pins in all libraries
`-pins {`*`cell`* `output`*`pin`*`}`	Adds the new capacitance constraints to the specified set of cells and pins. You can specify output or inout pins; input pins are ignored. The asterisk (`*`) wildcard character can be used in a cell or pin name. If there is already a table associated with the same input-output pair (input pins are specified using `-input_pin`), it is replaced with the new constraints; otherwise, it is added to the output maximum capacitance table list. For analysis and optimization, the software uses the most restrictive (the lowest capacitance) value from either the table or the `.libmax_capacitance`. If you also specify the `-force` parameter, the `max_capacitance` value is ignored. The final value can be further modified using `.sdc set_max_capacitance` statements.
`-transition {freq1 freq2 ...}`	Specifies a list of at least two transition values with the input transition times. Transition times must be increasing in value, and the total number of transition times cannot exceed 255. *Type:* Float, specified in units of nanoseconds, and all values must be equal to or greater than `0`.

Examples

- The following command establishes a maximum capacitance table for all pins and on all the cells in the libraries:

```
set_max_cap_per_freq_transition -force -pins {* *} -freq {1e8 1.25e8 1.5e8} \
-transition {450 475 500} -cap {{0.022 0.017 0.012} {0.02 0.014 0.01} {0.018 0.013 0.008}}
```

  The table is shown below:

```
          450       475       500
1e8       0.022     0.017     0.012
1.25e8    0.02      0.014     0.01
1.5e8     0.018     0.013     0.008
```

Related Commands

- set_max_cap_per_freq
- set_max_transition_per_freq
- report_constraint

set_max_transition_per_freq

```
set_max_transition_per_freq
[-help]
[-force]
-freq <string>
[-lib <string>]
-pins <string>
-transitions <string>
```

Adds a maximum transition per frequency table to the existing max_transition statement in the timing library. Timing analysis uses this table to detect violations, and the software attempts to repair those violations.

Use this command any time after loading the timing library files (that is, after importing or restoring the design).

The timing engine derives the frequency of an instance from the clock domain to which it is connected (the fastest clock domain, if there is more than one). If the instance is a clock instance (that is, part of the clock tree), the actual clock frequency is used. If the instance is part of the combinational logic (that is, a flip-flop, or the logic connected to the output of a flip-flop), the frequency used equals one-half of the associated clock domain frequency because a flip-flop output can only toggle at one-half the rate of the input clock. For example, a clock of 1.0 Mhz has 1,000,000 rise and fall transitions per second; however the output of a flip-flop connected to a 1.0 Mhz clock can only have 500,000 rise and fall transitions per second.

Timing analysis computes the frequency directly from the timing graph. Use the `check_ac_limits -toggle` command to write the effective frequency onto the associated net in the database.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_max_transition_per_freq` parameter. For a detailed description of the command and all of its parameters, use `manset_max_transition_per_freq`.
`-force`	Computes the maximum transition from the interpolated `-freq` and `-transition` values only and ignores any previously defined `max_transition` values for these pins in the library. *Default:* Uses the worst case (minimum value) between the `.libmax_transition` values and the interpolated `-transition` values.
`-freq` *string*	Specifies a list of at least two frequency values that must be greater than or equal to 0. You must specify the same number of `-freq` and `-transition` values, and the frequency values must be specified in ascending order. *Type:* Float, specified in units of Hz
`-lib` *string*	Modifies only the matching cell-pins in the specified library. *Default:* Modifies the matching cell-pins in all libraries.
`-pins` *string*	Adds the new transition constraints to the specified set of cells and pins. You can specify input, output, or inout pins. The asterisk (`*`) wildcard character can be used in a cell or pin name. If a table associated with the pin already exists, it is replaced with the new table. For analysis and optimization, the software uses the most restrictive (the lowest transition) value from either the table or the `.lib max_transition`. If you also specify the -force parameter, the `max_transition` value is ignored. The final value can be further modified using `.sdc set_max_transition` statements.
`-transitions` *string*	Specifies a list of transition values that must be greater than or equal to 0. You must specify the same number of `-freq` and `-transition` values. The software derives the maximum transition limit from a linear interpolation (or extrapolation) for a given frequency value using two frequency values in the table. For interpolation, the software uses the closest value above the frequency, and the closest value below the frequency. For extrapolation, the software uses the two closest values to the frequency. *Type:* Float, specified in units of ns

Examples

Assume all output pins of a library have a `max_transition` value of 0.9 ns from the library file.

- The following command sets the frequency dependent `max_transition` to 1.0 ns at a frequency of 100 Mhz, and to 0.7 ns at a frequency of 200 Mhz, for pins matching `in*`of cells matching `*` (that is, all cells) in the library named `slow`:

  ```
  set_max_transition_per_freq -lib slow -freq {100e6 200e6} -transitions {1.0 0.7} -pins {* in*}
  ```

  As a result, at 300 Mhz, an `and2` cell input pin named `in1` has an extrapolated `max_transition` value of 0.4ns. An extrapolation to 50 Mhz results in a `max_transition` value of 1.15 ns. However, the 1.15 ns value is greater than the existing `.lib max_transition` value of 0.9 ns; therefore timing analysis uses the 0.9 value.

- The following command applies the same constraints to all libraries with cell names matching `dff*` that have a pin named `clk`:

  ```
  set_max_transition_per_freq -force -freq {100e6 200e6} -transitions {1.0 0.7} -pins {dff* clk}
  ```

  Note: The original `.lib max_transition` value is ignored because the `-force` parameter is specified; therefore an extrapolation to 50 Mhz results in a final `max_transition` limit of 1.15 ns.

Related Commands

- `set_max_cap_per_freq`
- `report_constraint`
- `check_ac_limits`

set_multi_input_switching_mode

```
set_multi_input_switching_mode
[-help]
[-disable_lib_cells <string>]
[-mis_alignment_mode {0 | 1}]
[-switching_alignment_factor <float>]
[-view <string>]
```

Configures multi-input switching (MIS) mode settings.

You can enable the MIS feature using the `timing_analysis_multi_input_switching_mode` attribute.

For reporting advanced MIS in delay calculation and delta delay in timing reports, you can use the following options:

- `report_delay_calculation -mis`
- `report_timing -fields retime_mis_delta`
- `timing_report_fields {retime_mis_delta}`

Parameters

`-help`	Prints out the command usage.
`-disable_lib_cells <string>`	Enables or disables the multi-input switching (MIS) mode support for the specified library cells. *Default*: " "
`-mis_alignment_mode {0 | 1}`	Controls the MIS alignment mode. *Default*: 0
`-switching_alignment_factor <float>`	Allows to configure the relative alignment of side inputs switching with respect to the primary inputs. The switching alignment factor is applied for all the instances for which MIS analysis is to be performed. This parameter accepts values between -1 to 1. These are described below: • 1: Side inputs switch at the earliest arrival. • 0: Side inputs switch at the same time as actual input. • -1: All the side inputs switch at the same time and align at the completion of the output pin transition. *Default*: 0 **Note**: Setting the `-switching_alignment_factor` to 1 can be pessimistic if a side input switches much earlier than the main input. You can use the `-mis_alignment_mode` parameter to reduce pessimism in this mode.
`-view <string>`	Specifies the name of the view.

Examples

- The following commands disable MIS support for the specified library cells:

```
set_multi_input_switching_mode -disable_lib_cells [get_libs_cell aoi*]
set_multi_input_switching_mode -disable_lib_cells [get_libs_cell lib_0.510v/aoi*]
```

Related Commands

- `report_timing`
- `report_delay_calculation`
- `write_spice_deck`

Related Information

- "Advanced Multiple-Input Switching Analysis" section of Analysis and Reporting chapter in the *Tempus Stylus User Guide*.

set_opcond

```
set_opcond
[-help]
[-forceLibrary libName]
[-library libName]
[-min minOpCond [-minLibrary minLib]]
[-max maxOpCond [-maxLibrary maxLib]]
[<opCond <opCond>>]
[-powerDomain string]
```

Sets the operating temperature, process, or voltage conditions for the design. The operating conditions are contained in the timing library, which are normally read in during design import.

The `set_opcond` command can use a virtual operating condition that was created with the multi-mode multi-corner `create_opcond` command.

Parameters

`-help`	Prints out the command usage.
`-forceLibrary` *libName*	Specifies the library in which the *opCond* will be set. Do not use this parameter with the `-min`, `-max`, or `-powerDomain` parameters. Use this parameter to specify different operating conditions for different libraries in a single session.

`-library` *libName*	Specifies the library in which the operating condition (*opcond*) is defined. You can define a single operating condition from a library, and the software uses the PVT characteristics associated with that library for every library in the design. When you use this parameter in multiple `set_opcond` commands, the software uses the value specified in the last command. If the `-library` parameter is not specified, the *opCond* in the first library read is used.
	For example, consider the following commands:
	`set_opcond WCCOM -library wcTestLib`
	`set_opcond BCCOM -library bcTestLib`
	`set_opcond WCCOM -library wcTestLib1`
	`set_opcond BCCOM -library bctestLib1`
	`set_opcond -library wcTestLib2`
	`set_opcond -library bcTestLib2`
	In this case, the software uses the last value defined (`bcTestLib2`). Therefore, the operating conditions in the `bcTestLib2` library are used for both setup and hold analysis.

`-max` *maxOpCond*	Specifies the operating condition normally used for setup analysis.
`-maxLibrary` *maxLib*	Specifies the library in which the *maxOpCond* is defined. If the `-maxLibrary` parameter is not specified and `-max` is used, the *maxOpCond* from the first library listed in the maximum libraries is used.
`-min` *minOpCond*	Specifies the operating condition normally used for hold analysis.
`-minLibrary` *minLib*	Specifies the library in which the *minOpCond* is defined. If the `-minLibrary` parameter is not specified and `-min` is used, the *minOpCond* from the first library listed in the minimum libraries is used.
<opCond <opCond>>	Specifies the operating condition used for setup and hold analysis. You can specify a virtual operating condition created with the multi-mode multi-corner create_op_cond command.
`-powerDomain` *string*	Specifies that the *opCond* is used for the power domain.

Command Order

Use this command after importing the design and reading the libraries.

Examples

- The following command sets the operating condition of the design to `WCCOM` defined in the `slowLib` timing library. The worst case (WCCOM) operation condition will be used for both setup and hold analysis:
  ```
  tempus> set_opcond WCCOM -library slowLib
  ```
- The following Tcl command sets the operating condition to `WCCOM` defined in the `slowlib` library for setup analysis and sets the operating condition to `BCCOM` defined in the `fastlib` library for hold analysis:
  ```
  tempus> set_opcond -max WCCOM -maxLibrary slowlib -min BCCOM -minLibrary fastlib
  ```
- The following example specifies the `slow` operating condition defined in the `slow` library for setup analysis and the `fast` operating condition defined in the `fast` library for hold analysis:
  ```
  tempus> set_opcond -maxLibrary slow -max slow -minLibrary fast -min fast
  ```
- The following example sets the timing libraries to associate the fast library (Min Libraries) to setup and the slow library (Max Libraries) to hold:
  ```
  tempus> set_timing_library -max Min -min Max
  ```

  However, this means that setup analysis uses the `fast` library (defined in the Min Libraries list) with a `slow` operating condition defined in the `slow` library, and hold analyses uses the `slow` library (defined in the Max Libraries list) but with a `fast` operating condition defined in the `fast` library.
  To get a consistent result, reset the `set_opcond` command to:
  ```
  tempus> set_opcond -maxLibrary fast -max fast -minLibrary slow -min slow
  ```

  Now setup analysis uses the `fast` library (defined in the Min Libraries list) with a `fast` operating condition defined in the `fast` library, and hold analyses uses the `slow` library (defined in the Max Libraries list) but with a `slow` operating condition defined in the `slow` library.

Related Commands

- `create_opcond`

set_path_adjust

```
set_path_adjust
[-help]
adjust_value
[-path_adjust_group path_adjust_group_name]
[-view string]
[-setup | -hold]
```

Defines the slack adjustment value for timing paths. The defined adjustment value is added to the slack of a path specified by the path adjust group, referred to by the `-path_adjust_group` parameter. You can specify a path adjust group by using the `set_path_adjust_group` command.

If multiple path adjustment values are applicable on the same pin, which is on the path specified by multiple `set_path_adjust_group` constraints, then the smallest adjustment value will be applied.

Note: The `set_path_adjust` command is not allowed to be used inside the SDC constraints file. It is only allowed in the software run script.

Parameters

`-help`	Prints out the command usage.
adjust_value	Specifies the value of slack adjustment.
`-hold`	Specifies to apply the adjust value to early paths.
`-path_adjust_group` *path_adjust_group_name*	Specifies the name of path adjust group that defines the path for which the adjustment will be applied.
`-setup`	Specifies to apply the adjust value to late paths. **Note**: If you do not specify `-setup` or `-hold` parameter, the software sets the adjust value to both setup and hold analysis.
`-view` *string*	Specifies a list of analysis views for which adjustment will be applied. This parameter accepts multiple views as argument.

Examples

- The following commands define the slack adjustment values for the specified views:

```
tempus> set_path_adjust 0.44 -path_adjust_group AG1 -view view1 -hold
tempus> set_path_adjust 0.44 -path_adjust_group AG2 -view view2 -setup
tempus> set_path_adjust 0.44 -path_adjust_group AG3 -view view3
```

Related Commands

- set_path_adjust_group

set_path_adjust_group

```
set_path_adjust_group
[-help]
[-fall]
[-fall_through through_list]
-name path_adjust_group_name
[-rise]
[-rise_through through_list]
[-through through_list]
[-from from_list | -rise_from from_list | -fall_from from_list]
[-to to_list | -rise_to to_list | -fall_to to_list]
```

Allows you to specify the path that is used by the `set_path_adjust` command for setting path slack adjustment values.

The `set_path_adjust_group` command is not allowed to be used inside the SDC constraints file. It is only allowed in the tool run script.

If path group definitions are matching/overlapping (when the `-from`/`-through`/`-to` parameters are used), then path group resetting is applied. The path group overwriting and prioritization rules are similar to path exceptions.

For more details, refer to `group_path`.

Parameters

`-help`	Prints out the command usage
`-fall`	Applies `set_path_adjust_group` at the falling edge of the signal on the path endpoints. *Data_type*: string, optional
`-fall_from from_list`	Applies `set_path_adjust_group` at the falling edge of the signal on the from pins. *Data_type*: string, optional
`-fall_through through_list`	Applies `set_path_adjust_group` at the falling edge of the signal on the through pins. *Data_type*: string, optional
`-fall_to to_list]`	Applies `set_path_adjust_group` at the falling edge of the signal on the to pins. *Data_type*: string, optional
`-from from_list`	Specifies the list of pins, instances, or clocks that are the start of the paths. *Data_type*: string, optional

`-name` *path_adjust_group_name*	Specifies the name of the path adjust group. *Data_type*: string, optional
`-rise`	Applies `set_path_adjust_group` at the rising edge of the signal on the path endpoints. *Data_type*: string, optional
`-rise_from` *from_list*	Applies `set_path_adjust_group` at the rising edge of the signal on the from pins. *Data_type*: string, optional
`-rise_through` *through_list*	Applies `set_path_adjust_group` at the rising edge of the signal on the through pins. *Data_type*: string, optional
`-rise_to` *to_list*	Applies `set_path_adjust_group` at the rising edge of the signal on the to pins. *Data_type*: string, optional
`-through` *through_list*	Specifies the list of pins or instances where the paths go through. *Data_type*: string, optional
`-to` *to_list*	Specifies the list of pins, instances, or clocks where the path ends. *Data_type*: string, optional

Examples

The following commands specify the adjusted path groups:

- `set_path_adjust_group -name AG1 -from in1 -through buf1/A -to out1`
- `set_path_adjust_group -name AG2 -from [get_clocks CK1] -through buf1/A -to [get_clocks CK2]`
- `set_path_adjust_group -name AG3 -from [get_clocks {CK1 CK2}] -to [get_clocks {CK1 CK2}]`

Related Commands

- `reset_path_adjust_group`
- `set_path_adjust`
- `group_path`

set_socv_constraint_config

```
set_socv_constraint_config
[-help]
[-hold_folding float]
{-libs <string>}
[-nochange_hold_folding float]
[-nochange_setup_folding float]
[-setup_folding float]
```

Enhances the built-in sigma pessimism of check values for setup and hold constraints in the library. The folding factor attribute records the amount of on-chip-variation pessimism added during cell characterization to the nominal setup/hold tables in the libraries. The folding factor attributes are library level attributes.

Notes

- The `set_socv_constraint_config` command does not reset the timing; so it should be used before the `report_timing` command.
- To change the setting later in the flow, you need to use the `update_timing -full` command.
- You cannot use this command, if any of the following attributes are specified:
  `timing_library_setup_sigma_multiplier`
  `timing_library_hold_sigma_multiplier`

Parameters

`-help`	Prints out the command usage.
`-hold_folding` *float*	Specifies a number of additional sigma units added on top of the existing nominal hold value. *Min* = -100 *Max* = 100
`-libs` *string*	Specifies a list of library names (not library paths) or collection objects.
`-nochange_hold_folding` *float*	Specifies a number of additional sigma units added on top of the existing nominal hold value for no-change timing arcs. *Min* = -100 *Max* = 100
`-nochange_setup_folding` *float*	Specifies a number of additional sigma units added on top of the existing nominal setup value for no-change timing arcs. *Min* = -100 *Max* = 100
`-setup_folding` *float*	Specifies a number of additional sigma units added on top of the existing nominal setup value. *Min* = -100 *Max* = 100

Examples

- The following command is used when an additional margin of 10 sigma units is applied to the nominal hold tables in the library collection test.lib:

```
set_socv_constraint_config -hold_folding 10 -libs [get_libs test.lib]
```

Related Commands

- `report_timing`
- `update_timing`

set_socv_rc_variation_factor

```
set_socv_rc_variation_factor
[-help]
value
[-early]
[-late]
[-views list_of_views]
```

Defines the value of interconnect delay variation factor.

This command applies global variation as percentage of the nominal delay.

To enable this feature, you can set the `timing_socv_rc_variation_mode` attribute to `true`.

Parameters

`-help`	Prints out the command usage
value	Specifies the value of the interconnect delay variation factor to be used.
`-early`	Specifies the variation factor for early interconnect delays.
`-late`	Specifies the variation factor for late interconnect delays.
`-views` *list_of_views*	Apply the SOCV interconnect variation factor to the specified list of analysis views.

Related Commands

- `get_socv_rc_variation_factor`

set_socv_reporting_nsigma_multiplier

```
set_socv_reporting_nsigma_multiplier
[-help]
[-views list_of_views]
{[-setup float] [-hold float]}
[-transition float]
```

Controls n-sigma value for reporting purposes.

This command supports n-sigma specification per view per setup/hold combination.

Parameters

`-help`	Prints out the command usage.
`-hold float`	Specifies the sigma multiplier for hold mode.
`-setup float`	Specifies the sigma multiplier for setup mode.
`-transition float`	Specifies the sigma multiplier to use for transition related DRV checks – including max_transition, min_transition, pulse_clock_max_transition, and pulse_clock_min_transition. When this parameter is not specified, the sigma multiplier specified for setup will be used for transitions. When the `-transition` parameter is specified, you can set the `timing_report_max_transition_check_using_nsigma_slew` attribute to `true` to report transition related DRV checks, such as, max_transition, min_transition, pulse_clock_max_transition, and pulse_clock_min_transition. This setting enables the `report_constraint` command to use mean and sigma slew values for reporting transition values in DRV checks.
`-views list_of_views`	Applies the sigma multiplier factor to the specified list of analysis views. **Note**: The specified views should be active.

Examples

- The following command applies the sigma multiplier to the specified view in setup mode :
  ```
  tempus > set_socv_reporting_nsigma_multiplier -setup 2.5 -view std_max_setup
  ```
- The following command applies the sigma multiplier to the specified view in for hold mode:
  ```
  tempus > set_socv_reporting_nsigma_multiplier -hold 3.0 -view std_min_hold
  ```

Related Commands

- get_socv_reporting_nsigma_multiplier

set_table_style

```
set_table_style
-help
-name string
[-reverse_rows]
[-major_sort integer]
[-minor_sort integer]
[-max_widths integer]
[-min_widths integer]
[-indent integer]
-frame | -no_frame_fix_width [-nosplit]
```

Disables printing of specific columns and allows for specifying the minimum and maximum size of each column in a report. It allows for reversing the data, for controlling the left indent, and for controlling the sorting of the columns.

Note: Do *not* include a blank space between commas "*,*" when specifying widths. If you are specifying a value for the width(s), make sure there is *no* leading or trailing blank space.

The `-frame`, `-no_frame`, and `-no_frame_fix_width` parameters work in a global manner. If you specify one of these parameters, it will affect all reports that can be formatted with this command. You cannot specify them with the -name parameter, to change the format of a specific report.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_table_style` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_table_style`.
`-frame`	Specifies that the table columns have a fixed frame or border, based on the minimum and maximum width settings. Any data that is longer than the maximum width for its column is wrapped within the same column. This is the default format for tables.
`-indent` *integer*	Specifies the number of spaces to leave on the left for that particular table.
`-major_sort` *integer*	Specifies the column to be used in the major sort. If the column number is negative, it reverses the sort.

`-max_widths` *integer*	Specifies the maximum width for each table column in the report specified by the `-name` parameter. **Note**: You cannot specify `-max_widths` parameter without `-name`. You can separate each specified width value with a comma (,). For example, if you specify: -max_widths `{10,20,30}` The first column will have a maximum width of 10, the second column will have a maximum width of 20, and the third column will have a maximum width of 30. The fourth and fifth columns will remain the default width spacing. Commas are also specified as column "placeholders", when you want to change the spacing of a column without changing the spacing of the preceding column. For example, if you specify: -max_widths {,,20,30} The third column will have a maximum width of 20, the fourth column will have a maximum width of 30, and the first, second, and fifth columns will remain the default width spacing. If you specify: `-max_widths {20,,30}` The first column will have a maximum width of 20, the third column will have a maximum width of 30, and the second, fourth and fifth columns will remain the default width spacing. Each column resizes itself to be between the range of the maximum and minimum widths specified for that column, based on the data in that column. Overruns are wrapped within the same column, unless otherwise specified. *Default:* `{50,50,50,50,50}`

`-min_widths` *integer*	Specifies the minimum width for each table column in the report specified by the `-name` parameter. **Note**: You cannot specify `-min_widths` parameter without `-name`. You can separate each specified width value with a comma (,). For example, if you specify: -min_widths `{7,10,15}` The first column will have a minimum width of 7, the second column will have a minimum width of 10, and the third column will have a minimum width of 15. The fourth and fifth columns will remain the default width spacing. Commas are also specified as column "placeholders", when you want to change the spacing of a column without changing the spacing of the preceding column. For example, if you specify: -min_width {,,10,15} The third column will have a minimum width of 10, the fourth column will have a minimum width of 15, and the first, second, and fifth columns will remain the default width spacing. If you specify: -min_width {7,,10} The first column will have a minimum width of 7, the third column will have a width of 10, and the second, fourth and fifth columns will remain the default width spacing. Each column resizes itself to be between the range of the maximum and minimum widths specified for that column, based on the data in that column. Overruns are wrapped within the same column, unless otherwise specified. *Default:* `{5,5,5,5,5}`
`-minor_sort` *integer*	Specifies the column to be used in the minor sort. It applies to *all* rows that have the same values in the `major_sort` column. If the column number is negative, the software reverses the sort.

`-name` *string*	Changes the formatting for the specified table. See Table for a list of table names you can use, and the commands with which they work.
`-no_frame_fix_width`	Specifies that the table columns have a fixed minimum width, but no maximum width (the data does not wrap). Column data that fits within the column's minimum width will left-justify and pad its location to fill the specified column space. Column data that is longer than the column's minimum width will take as much space in the table as needed. If a column is overwritten by the data from the preceding column, its data will begin on a new line at the correct column location. *Default:* `-frame`
`-nosplit`	Specifies that each column follows the preceding column without splitting to a new line, if the data in one column overruns the next column. The wrapping behavior for data is controlled by the size of the display window. This format is useful for machine parsing the report. **Note:** You must specify `-no_frame_fix_width` in order to use this parameter.
`-reverse_rows`	Reverses the data rows. This is useful, for example, if you want to trace back from an end point and the default report traces forward from the end point.

The timing report formatting depends on the specified `set_table_style` parameters. These are described below:

- When `set_table_style -no_frame_fix_width` parameter is specified, the timing report displays long inst/hpin names and the rest of the column values are shown on the next line.
- When `set_table_style -no_frame_fix_width -nosplit` is specified, the timing report displays long inst/hpin names and rest of the column values are shown on the same line, because `-nosplit` table style is specified. In this case, other column headers (beyond inst/pin) may not be aligned properly.
- The following settings are specified:

  ```
  set_table_style -no_frame_fix_width -nosplit
  set_table_style -name report_timing -max_width {50,50,,,,,}
  ```

  The generated report will show properly aligned column headers and end-point/begin-points are not split.
- The following settings generate a report displaying longer (than above reports) inst/pin names:

  ```
  set_table_style -no_frame_fix_width -nosplit
  ```

```
set_table_style -name report_timing -max_width {120,120,,,,,}
```

In this report output, the column headers are aligned properly and are not split.

List of Table Names

Table Name	Formats Reports Generated by
report_cell_instance_delays_arcs	report_cell_instance_timing
report_cell_instance_delays_checks	report_cell_instance_timing
report_cell_instance_delays_pins	report_cell_instance_timing
report_inactive_arcs	report_inactive_arcs
report_nets	report_nets
report_net_header	report_nets
report_net_sink_table	report_nets
report_net_source_table	report_nets
report_net_summary_table	report_nets
report_path_exceptions	report_path_exceptions
report_port	report_ports
report_timing	report_timing
report_timing_summary	report_timing

Examples

- The following command resizes the first column to a maximum range of 70 and specifies a default width of 50 for the second column. The default for the maximum width is 50 and the default for the minimum width is 5:

  ```
  tempus> set_table_style -name report_timing -max_widths {70,}
  ```

  Note: Do *not* include a blank space between commas "," when specifying widths. If you are specifying a value for the width(s), make sure there is *no* leading or trailing blank space.

- The following command specifies a default width of 50 for the first and second column:

  ```
  tempus> set_table_style -name report_timing -max_widths {,}
  ```

- The following sample timing report shows the table style generated with the `-frame` parameter (set_table_style -frame):
  tempus> report_timing -format {cell arc delay arrival required instance}

```
Path 1: MET Setup Check with Pin seg3/u14/CK
Endpoint: seg3/u14/D (^) checked with leading edge of 'CLK_W_3'
Beginpoint: seg3/u9/Q (^) triggered by leading edge of 'CLK_W_3'
Path Groups: {CLK_W_3}
Other End Arrival Time 0.621
- Setup 0.186
+ Phase Shift 2.000
= Required Time 2.434
- Arrival Time 2.062
= Slack Time 0.372
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.682
= Beginpoint Arrival Time 0.682
+-------------------------------------------------------------------------+
| Cell    | Arc          | Delay    | Arrival  | Required  | Instance  |
|         |              |          | Time     | Time      |           |
|------+-------------+-------+---------+----------+--------------------|
|         | CK ^         |          | 0.682    | 1.054     | seg3/u9   |
| DFF     | CK ^ -> Q ^  | 0.185    | 0.867    | 1.240     | seg3/u9   |
| BUF     | A ^ -> Y ^   | 0.073    | 0.940    | 1.312     | seg3/u10  |
| INV     | A ^ -> Y v   | 0.317    | 1.257    | 1.629     | seg3/u11  |
| INV     | A v -> Y ^   | 0.402    | 1.659    | 2.031     | seg3/u12  |
| BUF     | A ^ -> Y ^   | 0.403    | 2.062    | 2.434     | seg3/u13  |
| DFF     | D ^          | 0.000    | 2.062    | 2.434     | seg3/u14  |
+-------------------------------------------------------------------------+
```

- The following sample timing report shows the table style generated with the `-no_frame_fix_width` parameter (`set_table_style -no_frame_fix_width`):

```
tempus> report_timing -format {cell arc delay arrival required instance}

Path 1: MET Setup Check with Pin seg3/u14/CK
Endpoint: seg3/u14/D (^) checked with leading edge of 'CLK3'
Beginpoint: seg3/u9/Q (^) triggered by leading edge of 'CLK3'
Path Groups: {CLK3}
Other End Arrival Time 0.621
- Setup 0.186
+ Phase Shift 2.000
= Required Time 2.434
- Arrival Time 2.062
= Slack Time 0.372
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.682
= Beginpoint Arrival Time 0.682
```

```
-------------------------------------------------------------------
Cell      Arc           Delay    Arrival    Required    Instance
                                 Time       Time
-------------------------------------------------------------------
          CK ^                   0.682       1.054        seg3/u9
DFF       CK ^ -> Q ^   0.185     0.867       1.240        seg3/u9
BUF       A ^ -> Y ^    0.073     0.940       1.312        seg3/u10
INV       A ^ -> Y v    0.317     1.257       1.629        seg3/u11
INV       A v -> Y ^    0.402     1.659       2.031        seg3/u12
BUF       A ^ -> Y ^    0.403     2.062       2.434        seg3/u13
DFF       D ^           0.000     2.062       2.434        seg3/u14
-------------------------------------------------------------------
```

- The following sample timing report shows the table style generated with the `-no_frame_fix_width` and `-nosplit` parameters (`set_table_style -no_frame_fix_width -nosplit`):

```
tempus> report_timing -format {cell arc delay arrival required instance}

Path 1: MET Setup Check with Pin seg3/u14/CK
Endpoint: seg3/u14/D (^) checked with leading edge of 'CLK3'
Beginpoint: seg3/u9/Q (^) triggered by leading edge of 'CLK3'
Path Groups: {CLK3}
Other End Arrival Time 0.621
- Setup 0.186
+ Phase Shift 2.000
= Required Time 2.434
- Arrival Time 2.062
= Slack Time 0.372
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.682
= Beginpoint Arrival Time 0.682
-------------------------------------------------------------------
Cell      Arc           Delay    Arrival    Required   Instance
                                 Time       Time
-------------------------------------------------------------------
CK ^                              0.682      1.054      seg3/u9
DFF       CK ^ -> Q ^   0.185     0.867      1.240      seg3/u9
BUF       A ^ -> Y ^    0.073     0.940      1.312      seg3/u10
INV       A ^ -> Y v    0.317     1.257      1.629      seg3/u11
INV       A v -> Y ^    0.402     1.659      2.031      seg3/u12
BUF       A ^ -> Y ^    0.403     2.062      2.434      seg3/u13
DFF       D ^           0.000     2.062      2.434      seg3/u14
-------------------------------------------------------------------
```

Related Commands

- report_cell_instance_timing
- report_inactive_arcs
- report_nets
- report_path_exceptions
- report_ports
- report_timing

set_threshold_voltage_group

```
set_threshold_voltage_group
[-help]
[-hinst <string>]
-name <group_name>
[-override]
{-libraries <library list> | -lib_cells <lib cell list>}
```

Groups timing libraries into different voltage threshold (Vth) classes (e.g., HVT, LVT, ULVT).

This command can be used to define (or redefine) the threshold voltage groups associated with the specified timing library cells. You can group entire libraries into a single group, or group by per library cell specification.

Parameters

`-help`	Prints out the command usage.
`-hinst <string>`	Specifies the hierarchical instances.
`-lib_cells <lib cell list>`	Specifies a list of library cells to be defined in the VT group.
`-libraries <library list>`	Specifies a list of libraries to be defined in the VT group.
`-name <group_name>`	Specifies the name of the threshold voltage group being defined.
`-override`	Allows you to override Liberty defined VT groups. If the Liberty timing library already has native `threshold_voltage_group` attributes at the library or library cell level, then you can use the -override parameter to customize it with the specified threshold voltage group.

Examples

- The following commands define library threshold voltage groups if they do not preexist in the .lib:

```
set_threshold_voltage_group -name SVT  -libraries stdcell_vt_1p62V_cworst
set_threshold_voltage_group -name LVT  -libraries stdcell_2vt_1p62V_cworst
set_threshold_voltage_group -name ULVT -libraries stdcell_vlvt_1p62V_cworst
set_threshold_voltage_group -name ELVT -libraries stdcell_flvt_1p62V_cworst
```

Related Commands

- set_vt_skew_derate
- set_vt_skew_config

Related Topics

For more information, refer to the "Voltage Threshold Skew Modeling and Analysis" section of Analysis and Reporting chapter in the *Tempus User Guide*.

set_timing_context_clock_mapping

```
set_timing_context_clock_mapping
[-help]
-block_clock block_clock_name
[-instance string]
-top_clock top_clock_name
[-top_view view_name]
[-view view_name]
```

Defines user clock mapping in context analysis. The auto-inferred clock mappings will be overridden by user-specified mappings. A text file containing the `set_timing_context_clock_mapping` commands is specified using the `read_timing_context -clock_mapping_file` parameter.

The `set_timing_context_clock_mapping` command should not be specified directly in the Tempus shell.

Parameters

`-help`	Prints out the command usage.
`-block_clock` *block_clock_name*	Specifies the name of the clock from the block-level analysis. The software maps the specified clock with the clock specified using the `-top_clock` parameter.
`-instance` *string*	Specifies the list of block instances for which the clock mapping is to be applied. When this parameter is not specified, the clock mapping will be applied to all the instances for which context analysis is to be performed.
`-top_clock` *top_clock_name*	Specifies the name of top level clock.
`-top_view` *view_name*	Specifies the name of the clock from top-level analysis. The software maps the specified clock with the clock specified using the `-block_clock` parameter.
`-view` *view_name*	Specifies the MMMC analysis view for which the top-block clock mapping is to be applied.

Examples

- In the following example, the block clock BCLK is mapped to top clock TCLK in block view B_view:
  ```
  set_timing_context_clock_mapping -top_clock TCLK -block_clock BCLK -view B_view
  ```
- In the following example, the block clock BCLK is mapped to both top clocks TCLK1 and TCLK2 in block view B_view:
  ```
  set_timing_context_clock_mapping -block_clock BCLK -top_clock TCLK1 -view B_view
  set_timing_context_clock_mapping -block_clock BCLK -top_clock TCLK2 -view B_view -add
  ```
- In the following example, the block clock BCLK is mapped only to TCLK2 in block view B_view:
  ```
  set_timing_context_clock_mapping -block_clock BCLK -top_clock TCLK1 -view B_view
  ```

```
set_timing_context_clock_mapping -block_clock BCLK -top_clock TCLK2 -view B_view
```

Related Commands

- set_timing_context_view_mapping
- read_timing_context
- write_timing_context

set_timing_context_view_mapping

```
set_timing_context_view_mapping
[-help]
[-block_clock_suffix block_clock_suffix]
-top_view topViewName
-block_view blockViewName
```

Provides mapping between the top and block STA views. This command can be used only if the top and block STA view names differ.

A text file containing a set of `set_timing_context_view_mapping` commands can be provided to the `read_timing_context -view_map_file` parameter.

Parameters

`-help`	Prints out the command usage.
`-block_clock_suffix` *`block_clock_suffix`*	Specifies the suffix for block clock names. The block clocks with the specified suffix in the `block_view` name will only be mapped to the top clocks from the `top_view`.
`-block_view blockViewName`	Specifies the name of the view in the block-level analysis. The software maps the specified view with the `-top_view` view.
`-top_view topViewName`	Specifies the name of the view in the top-level analysis. The software maps the specified view with the `-block_view` view.

Examples

- The following example shows how a view mapping file can be setup:

```
set_timing_context_view_mapping -top_view T1_view -block_view B1_view
set_timing_context_view_mapping -top_view T2_view -block_view B2_view
```

Related Commands

- `set_timing_context_clock_mapping`
- `read_timing_context`
- `write_timing_context`

set_timing_derate

```
set_timing_derate
[-help]
[obj_list]
[value]
[-add]
[-cell_check]
[-cell_delay]
[-clock]
[-clock_period_check]
[-corner]
[-data]
[-dynamic]
[-early float]
[-fall]
[-increment]
[-input_switching]
[-late float]
[-mean]
[-min | -max | -delay_corner delayCornerName]
[-multiply]
[-net_delay]
[-power_domain string]
[-pulse_width_check]
[-retain_delay]
[-rise]
[-sigma]
[-static]
[-statistical]
[-transition]
```

Sets delay scaling or derating factors for early and late paths in the current design. You use this command to perform scaled on-chip variation around best-case and worst-case corners. The timing scaling factors affect the delay values generated in the timing reports. You can set scaling factors for data paths, clock paths, minimum and maximum operating conditions.

If you do not specify the `-clock` or `-data` parameter, the software applies the scaling factors to both clock and data paths. If you specify only one of the two parameters, `-data` or `-clock`, the software uses the last defined value for the other.

Similarly, if you do not specify the `-min` or `-max` parameter, the software applies the scaling factor to both minimum and maximum operating conditions. You should only use the `-min` or `-max` parameter for best-case worst-case analysis. If you specify them in on-chip variation (OCV) mode, the software will use the `-min -early` derating factor for early path scaling, and the `-max -late` derating factor for late path adjustments.

When there are separate max and min libraries specified, only those `set_timing_derate` constraints are honored that are specified with max library. The constraints specified with min library are ignored.

In MMMC mode, for delay corners having different late and early library sets, only those `set_timing_derate` constraints are honored that are specified with late library set of the delay corner. The constraints specified with early library set are ignored.

Note: The `set_timing_derate` command can only be used on the command line. You cannot import it as part of an SDC file.

Note: If the `set_timing_derate` constraint is added incrementally in SI mode, an explicit `update_timing -full` command is required (before `report_timing` or `report_constraint`) for SI full accuracy.

Parameters

`-help`	Prints out the command usage.
`-add`	Incrementally adds or subtracts derates. You can also apply derates on specific leaf instances. This parameter also support nets objects.
`-cell_check`	Specifies scaling on timing checks. Use the `-cell_check` option with the `-early` option to specify scaling on hold arcs and use with the `-late` option to specify scaling on setup arcs. When you specify this parameter, derating factors are also applied to user constraints such as `set_output_delay`, `set_data_check`, and `set_clock_gating_check`. The `-cell_check` parameter by default does not apply check derate to output external delay (e.g., `set_output_delay`). To enable this feature, you can set the `timing_apply_check_derate_to_external_output_delay` timing attribute to `true`. When set to `true`, this attribute applies cell derate to the top boundary port.
`-cell_delay`	Specifies scaling on cell delays. If neither the `-cell_delay` nor the `-net_delay` option is specified, the specified scaling factor applies to both types of delays.
`-clock`	Applies the derating factors to clock paths only.
`-clock_period_check`	Specifies derate values for clock period checks.

`-corner`	Applies derating on cells or nets that are modeled as corner data elements. The specified derating factor is applied to both the mean and sensitivities of the delays (corner objects can have propagated sensitivities). A cell with no timing arc data in a library that has any sensitivity information with respect to any parameter defined in SPDF are considered as a corner object. All cells that do not qualify are considered to be statistical objects. A net with parasitic data that has no sensitivity information with respect to any parameter defined in SPDF is considered as a corner object. All nets that do not qualify are considered to be statistical objects.
`-data`	Applies the derating factors to data paths only.
`-delay_corner` *`delayCornerName`*	Applies the derating factor to the specified delay calculation corner or a list of delay corners. **Note**: The timing derate can be applied to corners associated with active views only.
`derate_value`	Specifies the derating value to be used.
`-dynamic`	Applies the specified derating factor to the delta portion of the net and cell delays. The delay typically models SI crosstalk adjustments from detailed RC analysis or annotation. You can use this parameter in conjunction with the `-net_delay` or `-cell_delay` parameter to target the derate to desired components of the overall stage delay. The delay calculation controls allow for the SI crosstalk impact to be apportioned to the cell delay and/or net delay. You should coordinate the use of `-dynamic` derate with your delay calculation configuration. For more information, you can refer to the following si attributes: • `si_delay_delta_annotation_mode` • `si_delay_separate_on_data` You cannot specify `-dynamic` with `-mean` or `-sigma` parameters.

`-early` *float*	Specifies that the derating factor applies to early paths. Early paths are launching clock during hold analysis, receiving clock during setup analysis and datapath during hold analysis. **Note**: To use this parameter, you must specify `-delay_corner`.
`-fall`	Applies the derating factor to falling delay and check arcs based on the transition direction at the arc's sink pin. This parameter can be used to selectively apply timing derates for the fall arcs (cell, net, or check arcs).
`-increment`	Incrementally adds the derate value to the total derate value in all the modes - OCV, AOCV, and SOCV mode. The `set_timing_derate` command saves two different values – the "base" derate and the "incremental" derate. When the `-incremental` parameter is specified, the software populates the incremental derates. **Notes**: • This parameter is used to specify incremental derates for leaf cell instances or library cell objects only. • `-increment` and `-multiply` are mutually exclusive parameters. • `-increment`/`-multiply` are supported with existing parameters, such as, `-late`/`-early`, `-rise`/`-fall`, `-mean`/`-sigma`. • Unlike base transition derate, the derate values with `-increment`/`-multiply` can be greater or smaller than 1. • Base transition derate values will be limited to: Late >= 1.0 and < 100.0, Early <= 1.0 and > 0.0. By default, a limit check on total derate value will be performed during transition for delay calculation. If the total transition derate is out of range, a warning message will be issued.

	By default, the incremental derates are overwritten. To disable this behavior, you can use the following global variables: • `timing_derate_incremental_adjust_additive_mode`: Incremental derates are added to each other to obtain the final incremental derate value. • `timing_derate_incremental_multiply_accumulative_mode`: Incremental derates are multiplied with each other to obtain the final incremental derate value. For more information, see timing category attributes. **Note**: The `-increment` parameter is mutually exclusive with `-pulse_width_check`, `-clock_period_check`, and `-input_switching` parameters.
`-input_switching`	Considers the given derate as simultaneous switching input (SSI) derate that is applicable to early paths only.
`-late` *float*	Specifies that the derating factor applies to late paths. Late paths are receiving clock during hold analysis, launching clock during setup analysis and data path during setup analysis. **Note**: To use this parameter, you must specify `-delay_corner`.
`-max`	Applies the derating factor to the maximum operating condition during best-case worst-case analysis. This parameter is ignored during on-chip variation analysis. **Note:** You cannot use the `-max` parameter when the software is in multi-mode multi-corner (MMMC) mode. You must use the `-delay_corner` parameter instead.

`-mean`	Controls nominal derating. You can apply separate derating the mean components of the delay. This parameter can be used in the SOCV analysis flow to specify a derating factor for the mean component of delays. For cell delays and checks, the `-mean` parameter is supported on library cell and instance objects. For net delays, it is supported as a global level derate factor. The `-multiply` and `-add` option can be used with the `-mean` parameter to provide an incremental adjustment to a previously specified mean base derate.
`-min`	Applies the derating factor to the minimum operating condition during best-case worst-case analysis. This parameter is ignored during on-chip variation analysis. **Note:** You cannot use the `-min` parameter when the software is in multi-mode multi-corner (MMMC) mode. You must use the `-delay_corner` parameter instead.
`-multiply`	Incrementally multiplies derates. You can also apply derates on specific leaf instances. This parameter also support nets objects.
`-net_delay`	Specifies scaling on net delays.
`object_list`	Specifies the list of instances, library cells, or nets to which the derating will be applied.
`-power_domain` *`string`*	Allows you to specify a specific power domain for the timing derate factors to be applied.
`-pulse_width_check`	Specifies derate values for pulse width checks. **Note:** The `-pulse_width_check` and `-clock_period_check` parameters are mutually exclusive. To apply check derating to pulse width checks, set the `timing_enable_derating_for_pulse_width_checks` attribute to `true`. By default, the attribute is set to `false`.

`-retain_delay`	Specifies scaling on retain constructs. If you specify both `-retain_delay` and `-cell_delay` parameters, the software uses the retain_delay value for retain construct scaling. If you do not specify the `-retain_delay` parameter, retain constructs are scaled in the same manner as cell delays. You cannot specify `-retain_delay` with `-mean` or `-sigma` parameters.
`-rise`	Applies the derating factor to rising delay and check arcs based on the transition direction at the arc's sink pin. This parameter can be used to selectively apply timing derates for the rise arcs (cell, net, or check arcs).
`-sigma`	Controls derating on the variation component. You can apply separate derating to the sigma components of the delay. This parameter can be used in the SOCV analysis flow to specify a derating factor for the variation component of delays. For cell delays and checks, the `-sigma` parameter is supported on library cell and instance objects. For net delays, it is supported as a global level derate factor. The `-multiply` and `-add` options can be used with the `-sigma` parameter to provide an incremental adjustment to a previously specified sigma base derate.
`-static`	Applies the specified derating factor to the non-delta portion of net delays only. You can specify `-net_delay` or `-cell_delay` parameter with the `-static` parameter. You cannot specify `-static` with `-mean` or `-sigma` parameters.

`-statistical`	Applies derating on cells or nets that are modeled as statistical data elements. The specified derating factor is applied to both the mean and sigma of delays. If both `set_timing_derate -corner` and `-statistical` options are not specified, the derating factor is applied on variation and corner objects equally. This is equivalent to issuing to two separate `set_timing_derate` commands, one with `-statistical` option and the other with `-corner` option.
`-transition`	Specifies scaling on transitions. When specified, the software allows derate transition and stretch waveforms. For incremental derates applied to the base derate (without the `-increment` parameter), you can use the following parameters with the `-transition` parameter: • `-increment` parameter adds to the base derate to obtain the final derate. • `-multiply` parameter multiplies the incremental derate with the base derate to obtain the final derate value (`-increment` parameter is not required). • Validity of final derate value (same limits as base transition derate) is applied only during timing analysis, and not at the time of constraint processing. **Note**: You can use the `-transition` parameter with the following parameters: • `-delay_corner` • `-early/-late` • `-rise/-fall` • `-mean/-sigma` • `-increment` • `-multiply`

Examples

- The following command (1) can be used when the software is in on-chip variation analysis mode to add pessimism to both the setup and hold analyses by making the early paths 20 percent faster:

```
set_db timing_analysis_type ocv
    ...
(1) set_timing_derate -early 0.8 -late 1.0
```

- The following example uses two `set_timing_derate` commands to set the scaling factors independently for the min and max corners. When the software is in best-case worst-case analysis mode:
  - Command (1) adds pessimism to the setup analysis by making the latching clock path 20 percent faster.
  - Command (2) adds pessimism to the hold analysis by making the latching clock path 10 percent slower.

```
(1) set_timing_derate -max -early 0.8 -late 1.0
(2) set_timing_derate -min -early 1.0 -late 1.1
```

- When the software is in on-chip variation analysis mode, there is only one active delay corner; therefore, the `-min` and `-max` parameters should not be used. The following command sequence will generate a warning:

```
set_db timing_analysis_type ocv
...
set_timing_derate -min -early 0.8 -late 0.9
set_timing_derate -max -early 1.1 -late 1.2
```

  In this situation, the software uses `0.8` for derating early paths, and `1.2` for derating late paths.

- The following command sets the early path scaling factor of 0.85 for instance `MULT_9` cell delays:

```
set_timing_derate -early -cell_delay 0.85 [get_cells {MULT_9}]
```

- The following command sets the early path scaling factor of 0.75 for the net named `MUX2/Z`:

```
set_timing_derate -net_delay -early 0.75 [get_nets {MUX2/Z}]
```

- The following command specifies power domain `PD1` for derating instances:

```
set_timing_derate -power_domain PD1 -cell_delay 1.1
```

- In the following example, a design having max library as 'slow' and min library as 'fast', only the first two `set_timing_derate` constraints are honored:

```
set_timing_derate -late 1.2 [get_lib_cells {slow/BUFX1}] -cell_delay
set_timing_derate -early 0.8 [get_lib_cells {slow/BUFX1}] -cell_delay
set_timing_derate -late 1.1 [get_lib_cells {fast/BUFX1}] -cell_delay
set_timing_derate -early 0.9 [get_lib_cells {fast/BUFX1}] -cell_delay
```

- Consider the following library sets, in MMMC mode:

```
create_library_set -name libset1 -timing slow.lib
create_library_set -name libset2 -timing fast.lib
create_delay_corner -name corner1 -late_library_set libset1 -early_library_set libset2
```

From the following `set_timing_derate` constraints the first and last are honored:

```
set_timing_derate -late 1.2 -cell_delay [get_lib_cells slow/*] -delay_corner corner1
set_timing_derate -early 2.3 -cell_delay [get_lib_cells slow/*] -delay_corner corner1
set_timing_derate -late 3.4 -cell_delay [get_lib_cells fast/*] -delay_corner corner1
set_timing_derate -early 4.5 -cell_delay [get_lib_cells fast/*] -delay_corner corner1
```

- The following command can be used when the software is in on-chip variation analysis mode to add pessimism to both the setup and hold analyses by making the early paths 20 percent faster:

  ```
  set_db timing_analysis_type ocv
  ...
  set_timing_derate -early 0.8 -late 1.0
  ```

  This command adds pessimism to the setup analysis by making the latching clock path 20 percent faster.
- The following example uses two `set_timing_derate` commands to set the scaling factors independently for the min and max corners. When the software is in best-case worst-case analysis mode:
  - The following command sets the early path scaling factor of 0.85 for instance MULT_9 cell delays:

    ```
    set_timing_derate -early -cell_delay 0.85 [get_cells {MULT_9}]
    ```
  - The following command sets the early path scaling factor of 0.75 for the net named MUX2/Z:

    ```
    set_timing_derate -net_delay -early 0.75 [get_nets {MUX2/Z}]
    ```
- The following command specifies power domain PD1 for derating instances:

  ```
  set_timing_derate -power_domain PD1 -cell_delay 1.1
  ```
- The following command shows library object level support for clock (and pulse width) check options:

  ```
  set_timing_derate -delay_corner -clock_period_check 1.05 [object_list]
  ```
- The following command sets an increment derate of 0.5 on mean and 0.8 on sigma of the cell:

  ```
  set_timing_derate -increment 0.8 -sigma  MP_32_INST/p11A
  set_timing_derate -increment 0.5 -mean  MP_32_INST/p11A
  ```

  To increment the sigma derate further by 0.9, you can use the following command:

  ```
  set_timing_derate -increment 0.9 -sigma  MP_32_INST/p11A
  ```

  The resultant sigma derate can be calculated as 1+0.9=1.9.

  When the `timing_derate_incremental_adjust_additive_mode` global variable is set to `true`, the resultant sigma derate will be 1+0.8+0.9=2.7.

Related Commands

- report_timing_derate
- reset_timing_derate

set_timing_library

```
set_timing_library
[-help]
[-max {Max | Min}]
[-min {Max | Min}]
```

Sets the timing libraries to be used in setup analysis (late-path delay calculation) or hold analysis (early-path delay calculation).

The default setting, if this command is not used, is that only maximum libraries will be used for setup analysis (late-path delay calculation) and only minimum libraries will be used for hold analysis (early-path delay calculation).

Use this command only after loading all the timing libraries.

Note: This command will become obsolete in the next major release as the software moves to using the Multi-Mode/Multi-Corner (MMMC) architecture for design import.

Parameters

`-help`	Prints out the command usage.
`-max {Max | Min}`	Specifies the Max or Min libraries to be used in setup analysis.
`-min {Max | Min}`	Specifies the Max or Min libraries to be used in hold analysis.

Examples

- The following command sets the timing library to `Max` used for both setup and hold analysis (early-path delay calculation):

  ```
  tempus> set_timing_library -max Max -min Max
  ```

- The following script shows how to set a Min library in setup analysis and a Max library in hold analysis for reporting the detail data paths that do not meet timing in a report file:

  ```
  tempus> set_timing_library -max Min -min Max
  tempus> set_db timing_analysis_check_type setup
  tempus> report_timing -max_points 10 > setupTiming.rpt
  tempus> set_db timing_analysis_check_type hold
  tempus> report_timing -max_points 10 > holdTiming.rpt
  ```

- The set_timing_library `-max Min -min Max` command sets the Min libraries for setup analysis and the Max libraries for hold analysis.
- The `set_db timing_analysis_check_type setup` command sets the timing analysis mode to `setup`.
- The `report_timing -max_points 10 > setupTiming.rpt` command runs setup analysis using

the `Min` Libraries and writes the result to the `setupTiming.rpt` file.

- The `set_db timing_analysis_check_type hold` command sets the timing analysis mode to `hold`.
- The `report_timing -max_points 10 > holdTiming.rpt` command runs `hold` analysis using `Max` Libraries and writes the output to the `holdTiming.rpt` file.

set_vt_skew_config

```
set_vt_skew_config
[-help]
[-analysis_mode {setup | hold | both}]
[-clocks_only]
[-correlated_vt_groups <corr_group>]
[-exclude <exclude>]
[-exclude_nominal]
[-include_only <include_only>]
[-unique_vt_mode {none | slow | fast | nominal}]
```

Allows customization of the combinations of Vth classes to be analyzed. By default, the software will analyze all the possible combinations.

This command is not cumulative as each invocation will reset any options previously set by its default value unless that option is re-specified.

Parameters

`-help`	Prints out the command usage.
`-analysis_mode {setup | hold | both}`	Specifies the analysis mode for which the configuration is to be used.
`-clocks_only`	Allows the software to consider only launch and capture clock VTs post CPPR point when enumerating Vth combinations. By default, the software will consider Vth classes on the data path as well.
`-correlated_vt_groups <corr_group>`	Specifies the groups for correlation. By default, all Vth groups are considered uncorrelated. If a set of Vth groups is specified as correlated then they will be considered as one entity when determining the combinations of Vth groups. All the correlated VT's will use the same (fast, slow, nominal) derate factor.
`-exclude <exclude>`	Specifies the configurations to be excluded. When this parameter is specified, the software excludes some combinations of Vth classes from the full elaborated set of combinations. The argument to this parameter is a Tcl list-of-lists as described for `-include_only`. The `-exclude` expression is a secondary filter which acts against the the current included set of enumerations – either all by default, or a subset configured by `-include_only`.

`-exclude_nominal`	Determines whether to enumerate fast, slow, nominal combinations or just fast, slow.
`-include_only <include_only>`	Specifies the configurations to be included. You can specify only the Vth combinations of interest. By default, all the possible combinations are analyzed. The argument to this parameter is a Tcl list-of-lists structure, which associates a Vth group with the set of analyses to consider for that group. **Note**: The options for `-include_only` and `-exclude` parameters is a Tcl list-of-lists that provides a way to customize the Vth enumerations that are analyzed. The format is a Vth group name followed by an enum list of combinations of fast, slow, or nominal. If a Vth group is missing from the specification a value of 'all' will be assumed.
`-unique_vt_mode {none | slow | fast |nominal}`	Allows the software to consider only the launch and capture clock VTs post CPPR point when enumerating Vth combinations. By default, the software will consider Vth classes on the data path as well. The unique Vt analysis mode is applicable only during the path-based analysis (PBA) step.

Examples

- The following command defines the derate mode to use in both GBA and PBA when the path has unique VT:

  ```
  set_vt_skew_config -unique_vt_mode slow
  ```

  Setting a unique Vt mode is required for the GBA unique Vt flow.

Related Commands

- `set_threshold_voltage_group`
- `set_vt_skew_derate`

Related Topics

For more information, refer to the "Voltage Threshold Skew Modeling and Analysis" section of Analysis and Reporting chapter in the *Tempus User Guide*.

set_vt_skew_derate

```
set_vt_skew_derate
[-help]
[-delay_corner <string>]
[-fast <float>]
[-hold]
[-mean]
[-setup]
[-sigma]
[-slow <float>]
[-threshold_voltage_group <group name>]
```

Assigns fast and slow derate factors to specific threshold voltage groups.

Parameters

`-help`	Prints out the command usage.
`-delay_corner <string>`	Specifies the list of delay corners.
`-fast <float>`	Specifies the derate factor to use when evaluating both the early and late paths for a speed-up due to Vth skew shift. This derate value is expected to be less than 1.0.
`-hold`	Specifies that the derate is applied to the hold component.
`-mean`	Specifies that the derate is applied to the mean component. The specified derate only applies to mean delays in SOCV analysis. If the `-mean` or `-sigma` parameter is not specified, the derate applies to both the delay components.
`-setup`	Specifies that the derate is applied to the setup component.
`-sigma`	Specifies that the derate is applied to the sigma component. The specified derate only applies to the delay variation component in SOCV analysis.
`-slow <float>`	Specifies the derate factor to use when evaluating both the early and late paths for a slow-down due to Vth skew shift. This derate value is expected to be greater than 1.0.
`-threshold_voltage_group <group name>`	Specifies the threshold voltage group that is either predefined in Liberty or defined using the `set_voltage_threshold_group` command.

Examples

- The following commands define the VT skew derate factors, and customize the different path combinations to be analyzed in PBA mode:

```
set_vt_skew_derate -threshold_voltage_group SVT  -fast 0.90 -slow 1.10
set_vt_skew_derate -threshold_voltage_group LVT  -fast 0.91 -slow 1.11
set_vt_skew_derate -threshold_voltage_group ULVT -fast 0.92 -slow 1.12
set_vt_skew_derate -threshold_voltage_group ELVT -fast 0.93 -slow 1.13
```

Related Commands

- set_threshold_voltage_group
- set_vt_skew_config

Related Topics

For more information, refer to the "Voltage Threshold Skew Modeling and Analysis" section of Analysis and Reporting chapter in the *Tempus User Guide*.

set_wire_load_mode

```
set_wire_load_mode
[-help]
{top | enclosed | segmented}
```

Controls how the software searches through the design hierarchy to find the appropriate wire load model for a net, or a hierarchical part of a net.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_wire_load_mode` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_wire_load_mode`.
*`{top	enclosed	segmented}`*	Specifies the wire-load model. You can specify one of the following options. • `enclosed` • `segmented` • `top`

	`enclosed`: Selects the wire load model using an area lookup table for the module that is at the lowest level in the design hierarchy and contains the entire net. When you set the mode to `enclosed`, the software uses the area of the smallest block enclosing the net to select the wire load model. In the `enclosed` mode, the software uses the following order of precedence to search the wire load model in: • The inner-most block enclosing the net on which you set the wire load selection group using the `set_wire_load_selection_group` command, if the `auto_wire_load_selection` global variable is enabled. Or: The inner-most block enclosing the net on which you set the wire load model using the `set_wire_load_model` command. • Each higher level hierarchical block for a `set_wire_load_selection_group` assertion, all the way up to the top-level block, if the `auto_wire_load_selection` global variable is enabled. Or: Each higher level hierarchical block for a `set_wire_load_model` assertion, all the way up to the top-level block. • The `default_wire_load_selection` field in the first timing library. Or, The `default_wire_load` field in the first timing library. **Note:** If even the default values are not present in the timing library, you will not be able to use the wireload model functionality.
	`segmented`: Selects the wire load model in a way similar to the `enclosed` mode except that each hierarchical segment of the net is treated separately. In the `segmented` mode: • A segment is defined by pins (including hierarchical pins) connected to a particular net. • When counting the drivers and receivers, only the pins connected to the segment are considered. This information along with the wire load models determine the parasitic information for the segment. • The parasitics of each segment sum up to form the parasitics for the entire net. **Note:** Each segment may have different wire load models.

	`top`: Selects the wire load model in a way similar to `enclosed` and `segmented` except that all nets use the wire load selection or wire load models defined by the top-level (largest) block enclosing the net. **Note:** If the first library read in does not define the default wire load mode, the software uses `top` as the default mode.

Examples

- The following command sets the wire load mode to `enclosed`:

```
tempus > set_wire_load_mode enclosed
```

Related Commands

- `reset_wire_load_mode`

set_wire_load_model

```
set_wire_load_model
[-help]
[obj_list]
[-library library_name]
[-max]
[-min]
-name wireload_model
```

Specifies the wire load model to be used from the technology library, and sets the wire load model on the design or on a hierarchical object (instance). If the list of instances is not specified, the wire load model is set on the current instance. Otherwise, the wire load model is set on all the specified instances or ports.

Use wire load models for estimating delays before the actual wire loads are back annotated. A technology library contains different wire load models, previously computed based on the analysis of several designs differing in area.

Parameters

`-help`	Prints out the command usage.
`-library` *library_name*	Specifies the technology library to use when searching for the name of the wire load model. You can specify a library name, or a collection of libraries. *Default:* All technology libraries are searched.
obj_list	Specifies the list of objects - designs, ports, or hierarchical instances.
`-max`	Sets the wire load model only for the maximum PVT (process, voltage, temperature) corner. *Default*: The specified wire load model applies to all three PVT corners if neither the `-min` nor the `-max` parameter is specified.
`-min`	Sets the wire load model only for the minimum PVT (process, voltage, temperature) corner. *Default*: The specified wire load model applies to all three PVT corners if neither the `-min` nor the `-max` parameter is specified.
`-name` *wireload_model*	Specifies the name of the wire load model in the library.

Examples

- The following command sets the `COM3_LV_WLM_0.0` wire load model from the `COM3_LV_WIRE` library on the current module for all PVT corners:

  ```
  tempus>  set_wire_load_model -library COM3_LV_WIRE -name COM3_LV_WLM_0.0
  ```

- The following command sets the `B1X1` wire load from the `lca500kv` library on the `clk1` port for the minimum PVT corner:

  ```
  tempus>  set_wire_load_model -library lca500kv -min -name B1X1 clk1
  ```

- The following command sets the `3M_TO_4500K`wire load from the `c13tsmc_slow2s108v125c` library on the current module for the minimum PVT corner:

  ```
  tempus>  set_wire_load_model -library c13tsmc_slow2s108v125c -min -name 3M_TO_4500K
  ```

Related Commands

- `report_wire_load`
- `reset_wire_load_model`

set_wire_load_selection_group

```
set_wire_load_selection_group
[-help]
group_name
[hinst_list]
[-library library_name]
[-max]
[-min]
```

Overrides the wire load selection table in the specified library.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_wire_load_selection_group` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_wire_load_selection_group`
group_name	Specifies the name of the wire load selection group.
`-library` *library_name*	Specifies the name of the technology library to search for the wire load selection table. When this parameter is not specified, the software searches all the libraries.
hinst_list	Specifies the list of instances.
`-max`	Controls the choice of a wire load selection table only for the PVT max corner. *Default*: Selects the table for all PVT corners if neither the `-min` nor the `-max` parameter is specified.
`-min`	Controls the choice of a wire load selection table only for the minimum PVT corner. *Default*: Selects the table for all PVT corners if neither the `-min` nor the `-max` parameter is specified.

Examples

- The following command selects `TLM` as the selection table used by the library:

  ```
  set_wire_load_selection_group TLM
  ```

- The following command selects `ASPEC1` as the selection table used by the `sm333s` library; only the minimum PVT corner is affected:

  ```
  set_wire_load_selection_group -min -library sm333s ASPEC1
  ```

Related Commands

- `reset_wire_load_selection_group`

timing_enable_boundary_model_eco

```
timing_enable_boundary_model_eco
[-help]
```

Enables all the boundary model settings that are required to be set for proper working of the ECO flow.

This command should be used after all the STA settings have been made. You can use this command in the `create_boundary_model` and `read_boundary_model` command sessions.

Parameters

`-help`	Prints out the command usage.

Related Commands

- `create_boundary_model`
- `read_boundary_model`

timing Category Attributes

- timing_all_registers_filter_clock_pins_by_clock
- timing_all_registers_identify_macros_include_is_macro_cell
- timing_all_registers_include_icg_cells
- timing_allow_input_delay_on_clock_source
- timing_analysis_aocv
- timing_analysis_aging
- timing_analysis_async_checks
- timing_analysis_check_type
- timing_analysis_clock_gating
- timing_analysis_clock_propagation_mode
- timing_analysis_clock_source_paths
- timing_analysis_cppr
- timing_analysis_engine
- timing_analysis_enable_inter_power_domain

- timing_analysis_enable_robustness_mode
- timing_analysis_graph_pba_mode
- timing_analysis_multi_input_switching_mode
- timing_analysis_precision_ps
- timing_analysis_self_loops_paths_no_skew
- timing_analysis_socv
- timing_analysis_type
- timing_aocv_analysis_mode
- timing_aocv_chip_size
- timing_aocv_core_size
- timing_aocv_derate_mode
- timing_aocv_stage_count_update_on_timing_reset
- timing_apply_check_derate_to_external_output_delay
- timing_apply_default_primary_input_assertion
- timing_apply_exceptions_to_data_check_related_pin
- timing_apply_setup_hold_exceptions_to_data_check_related_pin
- timing_cap_unit
- timing_case_analysis_for_icg_propagation
- timing_case_analysis_for_sequential_propagation
- timing_case_analysis_propagation
- timing_check_timing_report_all_checks
- timing_check_timing_signal_level_high_to_low_threshold
- timing_check_timing_signal_level_low_to_high_threshold
- timing_clock_phase_propagation
- timing_clock_source_paths_unconstrained_mark_clock_used_as_data
- timing_clock_source_use_driving_cell
- timing_clock_uncertainty_from_to_precedence
- timing_collection_all_fanin_fanout_traversal_mode
- timing_collection_result_display_limit
- timing_collection_variable_assignment_compatibility
- timing_context_apply_default_primary_input_assertion

- timing_context_apply_port_sdc_exceptions
- timing_context_clock_mapping_percentage_tolerance
- timing_context_clock_mapping_check_waveform
- timing_context_continue_on_invalid_module_list
- timing_context_clock_phase_based_clock_mapping
- timing_context_data_phase_based_clock_mapping
- timing_context_port_based_clock_mapping
- timing_constraint_disable_min_max_input_delay_worst_casing
- timing_constraint_enable_detailed_report_invalid_begin_end_points
- timing_constraint_enable_report_invalid_begin_end_points
- timing_constraint_enable_search_path
- timing_constraint_enable_separate_multicycle_data_checks
- timing_constraint_enable_logging
- timing_constraint_load_minimal_set_for_automated_eco
- timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency
- timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency
- timing_constraint_path_delay_exclude_unconstrained_endpoints
- timing_constraint_path_delay_include_clock_pin_endpoints
- timing_constraint_warn_for_timing_derate_exceeding_max_limit
- timing_continue_on_error
- timing_constraint_update_io_latency_averaging_mode
- timing_cppr_opposite_edge_mean_scale_factor
- timing_cppr_opposite_edge_sigma_scale_factor
- timing_cppr_opposite_edge_sigma_scale_factor_net
- timing_cppr_opposite_edge_sigma_scale_factor_cell
- timing_cppr_propagate_thru_latches
- timing_cppr_remove_clock_to_data_pessimism
- timing_cppr_self_loop_mode
- timing_cppr_skip_clock_reconvergence
- timing_cppr_skip_clock_reconvergence_for_unmatched_clocks
- timing_cppr_threshold_ps

- timing_cppr_transition_sense
- timing_create_clock_default_propagated
- timing_default_opcond_per_lib
- timing_defer_mmmc_obj_updates
- timing_derate_aocv_dynamic_delays
- timing_derate_aocv_reference_point
- timing_derate_dynamic_compatibility
- timing_derate_incremental_adjust_additive_mode
- timing_derate_enable_incremental_multiplication_mode
- timing_derate_incremental_multiply_accumulative_mode
- Examples
- timing_derate_negative_delay_backward_compatibility
- timing_derate_spatial_distance_unit
- timing_derate_voltage_scaling_mode
- timing_disable_bus_contention_check
- timing_disable_clock_period_checks
- timing_disable_constant_propagation_for_sequential_cells
- timing_disable_drv_report_on_constant_nets
- timing_disable_floating_bus_check
- timing_disable_genclk_combinational_blocking
- timing_disable_inferred_clock_gating_checks
- timing_disable_inout_output_side_timing_checks
- timing_disable_internal_inout_cell_paths
- timing_disable_internal_inout_net_arcs
- timing_disable_lib_pulse_width_checks
- timing_disable_library_data_to_data_checks
- timing_disable_library_tieoffs
- timing_disable_netlist_constants
- timing_disable_model_parasitics
- timing_disable_nochange_checks
- timing_disable_non_sequential_checks

- timing_disable_output_as_clock_port
- timing_disable_parallel_arcs
- timing_disable_pulse_width_same_edge_si_cppr_mode
- timing_disable_report_header_info
- timing_disable_retime_clock_path_slew_propagation
- timing_disable_sdf_retain_arc_merging
- timing_disable_skew_checks
- timing_disable_test_signal_arc
- timing_max_transition_use_si_transition
- timing_model_netlist_exclude_dummy
- timing_disable_timing_model_latch_inferencing
- timing_disable_tristate_disable_arcs
- timing_disable_user_data_to_data_checks
- timing_driving_cell_override_library
- timing_drv_analysis_ignore_latency_phases
- timing_enable_aocv_slack_based
- timing_enable_case_analysis_conflict_warning
- timing_enable_clock_to_clock_clock_gating_check
- timing_enable_clock_phase_based_rise_fall_derating
- timing_enable_data_through_clock_gating
- timing_enable_db_attribute_query
- timing_enable_derating_for_pulse_width_checks
- timing_enable_early_late_data_slews_for_setuphold_mode_checks
- timing_enable_genclk_divide_by_inherit_parent_duty_cycle
- timing_enable_genclk_source_path_register_limit
- timing_enable_generated_clock_edge_based_source_latency
- timing_enable_get_obj_escaped_name_backward_compatible
- timing_enable_get_objects_regexp_compatibility
- timing_enable_hier_context_unmapped_clock_analysis
- timing_enable_hierarchical_get_nets_support
- timing_enable_individual_distance_inter_die_analysis

- timing_enable_latch_thru_mode
- timing_enable_latency_through_clock_gating
- timing_enable_mmmc_loop_breaking
- timing_enable_model_constraint_warnings
- timing_enable_multi_drive_net_reduction_with_assertions
- timing_enable_multi_frequency_latch_analysis
- timing_enable_multicycle_data_check_compatibility
- timing_enable_path_delay_to_unconstrained_endpoints_compatibility
- timing_enable_pessimistic_cppr_for_reconvergent_clock_paths
- timing_enable_power_ground_constants
- timing_enable_preset_clear_arcs
- timing_enable_pulse_latch
- timing_enable_si_cppr
- timing_enable_simultaneous_setup_hold_mode
- timing_enable_spatial_derate_mode
- timing_enable_timing_window_pessimism_removal
- timing_enable_tristate_clock_gating
- timing_enable_uncertainty_for_clock_checks
- timing_enable_uncertainty_for_pulse_width_checks
- timing_enable_vtskew_derate_mode
- timing_enable_wire_variation
- timing_enable_zero_delay_analysis_mode
- timing_extract_model_aocv_mode
- timing_extract_model_case_analysis_in_library
- timing_extract_model_check_arcs_as_lvf
- timing_extract_model_consider_design_level_drv
- timing_extract_model_disable_cycle_adjustment
- timing_extract_model_enable_combinational_arc_to_clock_source_on_output_ports
- timing_extract_model_enable_output_pin_attributes
- timing_extract_model_enable_unconstrained_port_modelling
- timing_extract_model_exhaustive_validation_dir

- timing_extract_model_exhaustive_validation_mode
- timing_extract_model_gating_as_nochange_arc
- timing_extract_model_half_cycle_non_interface_paths
- timing_extract_model_ideal_clock_latency_arc
- timing_extract_model_include_applied_load_in_characterization_range
- timing_extract_model_include_applied_slew_in_characterization_range
- timing_extract_model_non_borrowing_latch_path_as_setup
- timing_extract_model_slew_propagation_mode
- timing_extract_model_synchronous_clocks_as_skew_checks
- timing_extract_model_write_clock_checks_as_arc
- timing_extract_model_write_clock_checks_as_scalar_tables
- timing_extract_model_write_function_for_combinational_arcs
- timing_extract_model_write_min_max_clock_tree_path
- timing_extract_model_write_lvf
- timing_full_context_flow
- timing_generate_normalized_driver_waveform
- timing_generated_clocks_inherit_ideal_latency
- timing_generated_clocks_allow_nested_assertions
- timing_get_of_objects_hier_compatibility
- timing_hierarchical_context_continue_on_top_block_clock_mismatch
- timing_hier_obj_name_compatibility
- timing_ignore_clock_based_path_delay_on_data_checks
- timing_ignore_lumped_rc_assertions
- timing_io_use_clock_network_latency
- timing_ipd_allow_paths_for_asynchronous_clocks
- timing_library_build_async_deassert_arc
- timing_library_convert_async_setuphold_to_recrem
- timing_library_generated_clock_use_group_name
- timing_library_hold_constraint_corner_sigma_multiplier
- timing_library_hold_sigma_multiplier
- timing_library_ignore_combinational_arc_mismatch_tables

- timing_library_infer_async_pins_from_timing_arcs
- timing_library_infer_cap_range_from_ccs_receiver_model
- timing_library_infer_cap_range_from_ecsm_receiver_model
- timing_library_interpolate_drv_values
- timing_library_infer_socv_from_aocv
- timing_library_read_ccs_noise_data
- timing_library_read_without_ecsm
- timing_library_read_without_power
- timing_library_read_without_sensitivity
- timing_library_scale_aocv_to_socv_to_n_sigma
- timing_library_setup_constraint_corner_sigma_multiplier
- timing_library_setup_sigma_multiplier
- timing_library_term_voltage_from_lib_pin
- timing_library_use_two_piece_receiver_cap
- timing_library_zero_negative_timing_check_arcs
- timing_multi_frequency_clock_rounding_factor
- timing_normalized_driver_waveform_clip_linear_part
- timing_normalized_driver_waveform_weight_factor
- timing_nsigma_multiplier
- timing_null_collection_return_compatibility
- timing_path_based_enable_bounding_box_for_io_paths
- timing_path_based_enable_closest_common_pin_driver_for_bounding_box
- timing_path_based_epba_bounded_mode
- timing_path_based_enable_report_launch_clock_path
- timing_path_based_enable_verbose_mode
- timing_path_based_enable_exhaustive_depth_bounded_by_gba
- timing_path_based_exhaustive_enable_design_coverage
- timing_path_based_exhaustive_max_paths_limit
- timing_path_based_ipba_endpoints_max_limit
- timing_pba_exhaustive_path_nworst_limit
- timing_path_based_report_analysis_summary_max_paths_limit

- timing_pll_clock_use_driving_cell
- timing_prefix_module_name_with_library_generated_clock
- timing_propagate_latch_data_uncertainty
- timing_property_arrival_window_enable_tcl_dict_format
- timing_property_arrival_clocks_consider_clock_source_data_phase
- timing_property_arrival_clocks_consider_data_phase
- timing_property_arrival_clocks_consider_latency_phase
- timing_property_return_null_collection_with_quiet
- timing_rail_swing_checks_high_voltage_threshold
- timing_rail_swing_checks_low_voltage_threshold
- timing_rcdb_allow_mismatch_option
- timing_recompute_sdf_in_setuphold_mode
- timing_reduce_multi_drive_net_arcs
- timing_reduce_multi_drive_net_arcs_threshold
- timing_report_analysis_summary_csv_extended_new_format
- timing_report_arrival_property_worstcase_mode
- timing_report_begin_end_pair_max_path_limit
- timing_report_check_timing_unconstrained_endpoints_due_to_constants
- timing_report_clock_pin_as_begin_point
- timing_report_constraint_extended_cell_em_flow
- timing_report_constraint_enable_extended_drv_format
- timing_report_constraint_format
- timing_report_constraint_rise_fall_clock_period_check
- timing_report_constraint_report_all_drv_violation
- timing_report_constraint_use_infinity_slack_for_unconstrained
- timing_report_default_frequency_for_unconstrained_nets
- timing_report_disable_max_paths_per_group
- timing_report_drv_enable_clock_source_as_clock
- timing_report_drv_enable_frequency_per_view
- timing_report_drv_per_frequency_per_input_slew
- timing_report_drv_enable_ghz_notation_for_drv_fields

- timing_report_drv_enable_slew_threshold_scaling
- timing_report_drv_per_frequency
- timing_report_drv_use_worst_timing_slack
- timing_report_enable_cppr_point
- timing_report_enable_em_index_clipping_report
- timing_report_enable_flag_field_symbols
- timing_report_enable_clock_to_for_unconstrained_paths
- timing_report_enable_lead_trail_to_rise_fall_map
- timing_report_enable_markers
- timing_report_enable_max_capacitance_drv_for_constant_nets
- timing_report_enable_max_path_limit_warning
- timing_report_enable_qpin_as_start_for_begin_end_pair
- timing_report_enable_report_clock_timing_across_clock_pin
- timing_report_enable_si_debug
- timing_report_enable_verbose_ssta_mode
- timing_report_enable_unique_pins_multiple_capture_clock_paths
- timing_report_enable_write_twf_third_party_for_input_pins
- timing_report_fields
- timing_report_generated_clock_info
- timing_report_group_based_mode
- timing_report_limit_max_path_count
- timing_report_max_transition_check_using_nsigma_slew
- timing_report_precision
- timing_report_property_fastest_clock_consider_data_phase
- timing_report_pulse_width_matching_launch_capture_paths
- timing_report_redirect_message_types
- timing_report_retime_formatting_mode
- timing_report_skip_constraint_loop_check
- timing_report_skip_icg_from_all_fanin_fanout_collection
- timing_report_socv_summary_mean_sigma
- timing_report_timing_header_detail_info

- timing_report_unconstrained_paths
- timing_report_unconstrained_path_early_late_header
- timing_report_use_receiver_model_capacitance
- timing_report_use_worst_parallel_cell_arc
- timing_resolve_driver_conflicts
- timing_scaling_for_negative_checks
- timing_scaling_for_negative_delays
- timing_sdf_adjust_negative_setuphold
- timing_sdf_enable_setuphold_scond_ccond
- timing_self_loop_paths_no_skew_max_depth
- timing_self_loop_paths_no_skew_max_slack
- timing_set_clock_source_to_output_as_data
- timing_socv_preserve_variation_with_annotations
- timing_socv_rc_variation_mode
- timing_socv_sigma_spatial_multiplier
- timing_socv_view_based_nsigma_multiplier_mode
- timing_spatial_derate_chip_size
- timing_spatial_derate_distance_mode
- timing_suppress_escape_characters
- timing_suppress_ilm_constraint_mismatches
- timing_time_unit
- timing_use_clock_pin_attribute_for_clock_net_marking
- timing_use_latch_early_launch_edge
- timing_use_incremental_si_transition
- timing_use_latch_time_borrow
- timing_use_verilog_for_model_netlist
- timing_waveform_aware_pulse_width_checks_high_voltage_level
- timing_waveform_aware_pulse_width_checks_low_voltage_level
- timing_write_sdf_no_escape_backslash

To set the value of any root attribute, use the `set_db` command:

```
set_db attribute_name value
```

To get the current value of any root attribute, use the `get_db` command:

```
get_db attribute_name
```

To get the current value of all attributes in the `check_drc` category, use the following command:

```
get_db -category check_drc
```

timing_all_registers_filter_clock_pins_by_clock

```
timing_all_registers_filter_clock_pins_by_clock {true | false}
```

Default: `false`

Data_type: Boolean

Reports clock pins filtered based on specified clocks.

When set to `true`, the software will filter and return only those clock pins where at least one clock is from the specified list of clocks.

This attribute applies only when any of the `all_registers` command parameters, `-clock`, `-rise_clock`, `-fall_clock`, is specified.

timing_all_registers_identify_macros_include_is_macro_cell

```
timing_all_registers_identify_macros_include_is_macro_cell {true | false}
```

Default: `true`

Data_type: Boolean

Enables to include cells as macro cells.

When set to `true`, the software includes cells as macro cells if the `is_macro_cell` property (see `get_property`) is set to `true` in the library.

When set to `false`, the `all_registers` command will not return cells for which the `is_macro_cell` property is `true`.

timing_all_registers_include_icg_cells

```
timing_all_registers_include_icg_cells {true | false}
```

Default: `true`

Data_type: Boolean

Includes integrated clock gating (ICG) cell information from the `all_registers` report output.

When set to `false`, the attribute does not allow the `all_registers` command report output to display ICG cell details.

timing_allow_input_delay_on_clock_source

```
timing_allow_input_delay_on_clock_source {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, this attribute allows the software to apply input delay constraints on a pin where a clock was previously asserted.

timing_analysis_aocv

```
timing_analysis_aocv {true | false}
```

Default: `false`

Data_type: Boolean

Sets the analysis mode to advanced on-chip variation (AOCV) analysis.

timing_analysis_aging

```
timing_analysis_aging {true | false}
```

Default: `false`

Data_type: Boolean

Enables timing-aware aging analysis.

The aging feature must be enabled before the timing updates are done.

Note: You cannot disable this feature in the same session later.

timing_analysis_async_checks

```
timing_analysis_async_checks {async | no_async | async_only | auto}
```

Default: `auto`

Data_type: Enum

Reports timing violations for asynchronous timing checks, including recovery and removal, and other checks on pins defined as preset or clear in the Liberty file.

The following options are supported:

- `async`: Reports timing checks on pins that are defined as preset or clear in the Liberty file.
- `no_async`: Reports timing violations while ignoring the two time arc attributes.
- `async_only`: Reports only recovery and removal checks.

The following setting corresponds to `set timing_disable_recovery_removal_checks true` in Tempus legacy.

```
set_db timing_analysis_async_checks no_async
```

The following setting corresponds to `set timing_disable_recovery_removal_checks false` in Tempus legacy.

```
set_db timing_analysis_async_checks async
```

timing_analysis_check_type

```
timing_analysis_check_type {setup | hold}
```

Default: `setup`

Data_type: Enum

Checks the design for setup or hold violations in the current analysis.

Note: When simultaneous setup/hold mode analysis is on, this settiing does not have any impact. You can use the `report_timing -late/-early` parameter to report setup/hold timing reports.

timing_analysis_clock_gating

```
timing_analysis_clock_gating {true | false}
```

Default: `true`

Data_type: Boolean

Enables clock gating checks.

timing_analysis_clock_propagation_mode

```
timing_analysis_clock_propagation_mode {sdc_control | forced_ideal}
```

Default: `sdc_control`

Data_type: Enum

Specifies how to determine whether clock end points are interpreted as having either ideal or propagated clock timing.

- `sdc_control`: Reports the timing violations considering clock tree and enables propagating clocks to report clock latency. You must set the `set_propagated_clock` constraint to propagate the clocks.
  Note: By default, CTS will instruct the timer to make all of the clock roots propagated. In this case, you do not need to set the `set_propagated_clock` constraint.
- `forced_ideal`: Does not report the timing violations considering the clock tree.

timing_analysis_clock_source_paths

```
timing_analysis_clock_source_paths {true | false}
```

Default: `true`

Data_type: Boolean

Reports combinational paths from the clock source to the timing checks on data pins.

When set to `false`, the clock portion of the path is considered as ideal (due to the SDC constraints or the `timing_analysis_clock_propagation_mode forced_ideal` attribute setting), the delay calculation software treats the clock nets as ideal nets when calculating the delays and slews along the data path.

timing_analysis_cppr

```
timing_analysis_cppr {both | none | setup | hold}
```

Default: `none`

Data_type: Enum

Removes pessimism from clock paths that have a portion of the clock network in common between the clock source and clock destination paths.

The pessimism is introduced when the timing analysis tools assume that the common path has different delay values for two different paths in case of on-chip variation. The pessimism can also be because of clock reconvergence in the clock network. The pessimism effects all analysis modes including single, best case-worst case (bcwc), and on-chip variation.

- `none`: Disables removal of clock reconvergence pessimism.
- `both`: Enables removal of clock reconvergence pessimism for both setup and hold modes.
- `setup`: Enables removal of clock reconvergence pessimism in setup mode only.
- `hold`: Enables removal of clock reconvergence pessimism in hold mode only.

timing_analysis_engine

```
timing_analysis_engine {statistical | static}
```

Default: `static`

Data_type: Enum

Allows you to specify the analysis engine - Statistical or Static - for performing timing analysis.

timing_analysis_enable_inter_power_domain

```
timing_analysis_enable_inter_power_domain {true | false}
```

Default: `false`

Data_type: Boolean

Enables IPD analysis.

For more information, refer to the Inter Power Domain (IPD) Analysis chapter in the *Tempus Stylus User Guide*.

timing_analysis_enable_robustness_mode

```
timing_analysis_enable_robustness_mode {true | false}
```

Default: `false`

Data_type: Boolean

Enables PTR analysis.

The Parametric Timing Robustness (PTR) analysis calculates the probability of an entire chip meeting timing requirements, instead of relying on the probabilities on individual paths to compute the robustness of a chip.

timing_analysis_graph_pba_mode

```
timing_analysis_graph_pba_mode {true | false}
```

Default: `false`

Data_type: Boolean

When specified, the software performs partial PBA analysis during GBA to reduce GBA pessimism.

timing_analysis_multi_input_switching_mode

```
timing_analysis_multi_input_switching_mode int
```

Default: 0

Data_type: Integer

Sets the multiple-input switching (MIS) mode.

You can specify one of the following options below:

- `0`: Simultaneous input switching is not considered.
- `1`: MIS analysis considers cells for which multiple inputs of a cell are tied together (tied-inputs).
- `2`: MIS analysis considers both the tied-inputs and inputs of cells switching independent to each other.
- 3: Enables derate-based MIS. When MIS-derate is enabled, Tempus will automatically disable AMIS-simulation.

timing_analysis_precision_ps

```
timing_analysis_precision_ps {0.1 | 0.01}
```

Default: `0.01`

Data_type: Double

Allows you to set the delay precision value (in 0.1ps or 0.01ps) for the `write_sdf` command.

The `write_sdf` command shows a double digit value after the decimal point by default. You can use this attribute for more accuracy.

timing_analysis_self_loops_paths_no_skew

```
timing_analysis_self_loops_paths_no_skew {true | false}
```

Default: `false`

Data_type: Boolean

Checks for self loop paths without skew.

When set to `true`, the software eliminates clock skew due to clock uncertainty for a path starting and ending at the same register. If the clock skew is not eliminated, the timing for such paths is pessimistic.

timing_analysis_socv

```
timing_analysis_socv {true | false}
```

Default: `false`

Data_type: Boolean

Enables SOCV Analysis.

timing_analysis_type

```
timing_analysis_type {single | best_case_worst_case | ocv}
```

Default: `ocv`

Data_type: Enum

Sets the timing analysis type to single, best case worst case or on-chip variation.

In this mode, the software calculates the delay for one path based on maximum operating condition while calculating the delay for another path based on minimum operating condition for setup or hold checks.

- `single`: Scales the delay values based on one operating condition.
- `best_case_worst_case`: Checks the design for two extreme operating conditions. The software uses the maximum delays for all paths during setup checks and minimum delays for all paths during hold checks.
- `ocv`: Calculates the delay for one path based on maximum operating condition while calculating the delay for another path based on minimum operating condition for setup or hold checks.

If you read in one library set and do not specify any timing analysis type, the software uses `single` by default. If you read in two library sets and do not specify any timing analysis type, the software uses `best_case_worst_case` by default.

For more information, see Timing Analysis Modes in the *Tempus User Guide*.

timing_aocv_analysis_mode

```
timing_aocv_analysis_mode {launch_capture | clock_only | separate_data_clock |
combine_launch_capture}
```

Default: `launch_capture`

Data_type: Enum

Controls handling of AOCV analysis modes.

- `launch_capture`: AOCV analysis will be performed for both data and clock paths, and the AOCV derating

factor will be calculated based on both clock and data path stages.

- `clock_only`: AOCV analysis will be performed for clock paths only, and the AOCV derating factor will be calculated based on the clock path stage depth only.
- `separate_data_clock`: The software calculates the AOCV stage count for clock paths and data paths separately. Both clock and data paths have related AOCV derating factors.
- `combine_launch_capture`: The software calculates AOCV stage count using combined depth for launch and capture paths. This is achieved by summation of path depth on the launch and capture paths.

timing_aocv_chip_size

`timing_aocv_chip_size` *`double`*

Default: 1e+30

Data_type: double

Specifies the diagonal length of the chip, in microns. The software will automatically calculate the length from the SPEF file, if it is loaded.

The chip and core sizes specified with the `timing_aocv_chip_size` and `timing_aocv_core_size` attributes take priority over chip and core size values derived from the SPEF file, or through placement. When the SPEF file is not loaded, or placement information is not available, then the following cases are considered:

- If you specify `timing_aocv_chip_size` but not `timing_aocv_core_size`, the software considers the core size to be equal to the `timing_aocv_chip_size` value.
- If you specify `timing_aocv_core_size` but not `timing_aocv_chip_size`, the software ends the AOCV analysis and generates an error message.
- If both `timing_aocv_chip_size` and `timing_aocv_core_size` are not specified, the software ends the AOCV analysis and generates an error message.

timing_aocv_core_size

`timing_aocv_core_size` *`real_number`*

Default: 1e+30

Data_type: double

Specifies the diagonal length of the core area, in microns. The software will automatically calculate the length from the SPEF file, if it is loaded.

The chip and core sizes specified with the `timing_aocv_chip_size` and `timing_aocv_core_size` attributes take priority over chip and core size values derived from the SPEF file, or through placement. When the SPEF file is

not loaded, or placement information is not available, then the following cases are considered:

- If you specify `timing_aocv_chip_size` but not `timing_aocv_core_size`, the software considers the core size to be equal to the `timing_aocv_chip_size` value.
- If you specify `timing_aocv_core_size` but not `timing_aocv_chip_size`, the software ends the AOCV analysis and generates an error message.
- If both `timing_aocv_chip_size` and `timing_aocv_core_size` are not specified, the software ends the AOCV analysis and generates an error message.

timing_aocv_derate_mode

```
timing_aocv_derate_mode {aocv_multiplicative | aocv_additive}
```

Default: `aocv_multiplicative`

Data_type: Enum

Controls the AOCV derating mode.

When set to aocv_multiplicative, the derating factor will be calculated as AOCV derating * OCV derating, which is set using the `set_timing_derate` command.

When set to aocv_additive, the derating factor will be calculated as AOCV derating + OCV derating values.

When you use this attribute, the `report_timing` command shows the `total_derate` column in the timing report output, which allows you to view and cross-check the calculated total derate factor.

timing_aocv_stage_count_update_on_timing_reset

```
timing_aocv_stage_count_update_on_timing_reset {true | false}
```

Default: `false`

Data_type: Boolean

When set to true, the AOCV stage counts are re-calculated when timing is reset.

timing_apply_check_derate_to_external_output_delay

```
timing_apply_check_derate_to_external_output_delay {true | false}
```

Default: `false`

Data_type: Boolean

Applies check derate to output external delay.

The `set_timing_derate -cell_check` parameter by default does not apply check derate to output external delay (using the `set_output_delay` command). To enable this feature, you can set the `timing_apply_check_derate_to_external_output_delay` timing attribute to `true`. The attribute applies cell (check) derate to the top boundary port.

timing_apply_default_primary_input_assertion

```
timing_apply_default_primary_input_assertion {true | false}
```

Default: `true`

Data_type: Boolean

When set to `true`, primary input and bidirectional ports that do not have an explicit arrival time specified (through the set_input_delay constraint), are provided a default arrival time. The arrival is referenced to an internal default clock that is indicated by the @ sign in various timing reports.

If a `set_input_delay` constraint is assigned to the input port, but the `-clock` parameter was not specified, the software assumes the constraint to be with reference to the default clock.

When set to `false`, inputs ports with no explicit `set_input_delay` are unconstrained for sequential timing analysis.

timing_apply_exceptions_to_data_check_related_pin

```
timing_apply_exceptions_to_data_check_related_pin {true | false}
```

Default: `true`

Data_type: Boolean

Controls if false path assertions which block the data path to the related (-from) reference pin of the data-to-data check disable the data-to-data check.

When set to `false`, the data-to-data remains valid when a false path assertion blocks the data path to the related (-from) pin.

When set to `true`, any false path assertion which blocks the data path to the related (-from) pin of the data-to-data check also causes the check to be disabled.

This attribute has no effect on the data-to-data checks with a reference clock, where an exception blocks the clock reference signal at the reference pin. The data-to-data check will always be disabled in this situation.

Examples

- In the following example, the data-to-data check is disabled:

  ```
  set timing_apply_exceptions_to_data_check_related_pin true
  set_data_check -from portA -to portB
  ```

```
set_false_path -to portA
```

- In the following example, the data-to-data check is valid. This is the default situation:

  ```
  set timing_apply_exceptions_to_data_check_related_pin false
  set_data_check -from portA -to portB
  set_false_path -to portA
  ```

- In this case, the data check is always disabled and is not affected by this attributes' settings:

  ```
  set_data_check -from portA -to portB
  set_false_path -to [get_clocks ClkA]
  ```

  where the path to `portA` is clocked by `ClkA`.

timing_apply_setup_hold_exceptions_to_data_check_related_pin

```
timing_apply_setup_hold_exceptions_to_data_check_related_pin {true | false}
```

Default: `true`

Data_type: Boolean

Disables the early signal data-to-data paths, when false path is applied on the late reference path.

When set to `true`, the `set_false_path -setup` parameter settings on the reference path disables early signal d2d path, and vice versa. If both the `-setup` and `-hold` parameters are set, the signal early/late d2d path is disabled, irrespective of this attribute's settings.

When set to `false`, the `set_false_path` exceptions set on d2d reference path do not affect data path slack. When the `set_false_path` command is set on the reference d2d path with only `-setup` (or only `-hold`) parameter, the signal path of d2d check is not affected by these exceptions.

timing_cap_unit

```
timing_cap_unit string
```

Default: " "

Data_type: string

Specifies the capacitance units to be used for timing constraints and timing reports. The default unit for capacitance is inferred from the timing libraries. You can override this inference by explicitly setting the value to 1pf or 1ff. Setting this variable explicitly is the same as executing the `set_library_unit -cap` command to set the capacitance units.

timing_case_analysis_for_icg_propagation

```
timing_case_analysis_for_icg_propagation {false | require_seq_prop | always | case_only |
constant_only}
```

Default: `false`

Data_type: Enum

Controls whether constants, which propagate to the inputs of integrated clock gating (ICG) cells, are further propagated to the output pins of these cells. The ICG cells typically contain an internal register or latch structure which synchronizes the devices' enable signal. If a constant propagates to this enable, it can be selectively propagated through the internal sequential element to the related output pins.

This attribute setting can uniquely control the handling of sequential propagation through ICG cells, or it can work in conjunction with the more general `timing_case_analysis_for_sequential_propagation` attribute settings.

The constants can be present in the analysis due to several different reasons, broadly breaking down to netlist constants and case analysis. The netlist constants come from connections to tie-hi or tie-lo cells, a 1'b0/1'b1 assignment in the netlist, or explicit power rail connections. The constants derived from these sources are considered to be immutable. The constants due to case analysis come strictly from the `set_case_analysis` constraint and are considered to be temporary - configuring the analysis to align to a certain mode. You may choose to handle constants from any source in a similar manner or customize the handling based on the constant type.

If you choose to allow propagation of only a specific type of constant, the output pin's state function must resolve to a constant by only considering inputs pins with the required constant type. Otherwise, the constant will not be propagated.

The propagation of constants to the output of a sequential device may affect how the timing checks on related input pins are analyzed. If all the possible transitions on a given input pin are unable to change the constant state of the output pin, the timing checks on the input pin will be disabled by default.

Changing the state of this attribute will change where constants are present in the analysis, typically resulting in sets of timing arcs and timing checks being enabled or disabled. As a result, a full timing update, including a fresh delay calculation, will be initiated when this setting is toggled.

You can use one of the following options:

Option	Description
`false`	Disables sequential propagation from any constant source (default setting).
`require_seq_prop`	When specified, the software will defer ICG sequential propagation handling to what is more generally set for all sequential devices, via either the `timing_case_analysis_for_sequential_propagation` attribute setting. For all the other options, the state of these external settings is not consequential.
`always`	Allows sequential propagation from any constant source - netlist or constraint-based.

`constant_only`	Allows sequential propagation only for constants that are due to the propagation of netlist-based constants.
`case_only`	Allows sequential propagation only for constants that are due to the propagation of constraint-based constants from `set_case_analysis` settings.

timing_case_analysis_for_sequential_propagation

```
timing_case_analysis_for_sequential_propagation {false | true | case_only | constant_only}
```

Default: `false`

Data_type: Enum

Controls whether constants, which propagated to the inputs of sequential devices like registers or flip-flops, are propagated to the output pins of those devices. This attribute will also affect the propagation of constants through integrated clock gating (ICG) cells - depending on the setting of the related attribute `timing_case_analysis_for_icg_propagation`. Refer to the documentation for this attribute for more information on ICG handling.

The constants can be present in the analysis due to several different reasons - broadly breaking down to netlist constants and case analysis. The netlist constants come from connections to tie-hi or tie-lo cells, a 1'b0/1'b1 assignment in the netlist, or explicit power rail connections. The constants derived from these sources are considered to be immutable. The constants due to case analysis come strictly from the `set_case_analysis` constraint and are considered to be temporary - configuring the analysis to align to a certain mode. You may choose to handle constants from any source in a similar manner or customize the handling based on the constant type.

If you choose to allow propagation of only a specific type of constant, the output pin's state function must resolve to a constant by only considering inputs pins with the required constant type. Otherwise, the constant will not be propagated. The propagation of constants to the output of a sequential device may affect how the timing checks on related input pins are analyzed. If all the possible transitions on a given input pin are unable to change the constant state of the output pin, the timing checks on the input pin will be disabled by default.

Changing the state of this attribute will change where constants are present in the analysis, typically resulting in sets of timing arcs and timing checks being enabled or disabled. As a result, a full timing update, including a fresh delay calculation, will be initiated when this setting is toggled. You can use one of the following options:

Option	Description
`false`	Disables sequential propagation from any constant source (default setting).
`true`	Allows sequential propagation from any constant source - netlist or constraint-based.
`constant_only`	Allows sequential propagation only for constants that are due to the propagation of netlist-based constants.

`case_only`	Allows sequential propagation only for constants that are due to the propagation of constraint-based constants from the `set_case_analysis` command.

timing_case_analysis_propagation

`timing_case_analysis_propagation {true | false}`

Default: `true`

Data_type: Boolean

Enables case analysis propagation.

The `timing_case_analysis_propagation` attribute is the top-level control for enabling or disabling all case propagation in the current session. When set to `true`, case propagation will be enabled and other attributes can then be used to selectively control case propagation in specific circumstances. The `timing_case_analysis_for_sequential_propagation` attribute variable can be used to control whether case is propagated across sequential devices. Propagation through integrated clock-gating (ICG) cells can be controlled with the `timing_case_analysis_for_icg_propagation` attribute.

When set to `false`, the software disables case analysis propagation.

timing_check_timing_report_all_checks

`timing_check_timing_report_all_checks {true | false}`

Default: `false`

Data_type: Boolean

Enables reporting of all the checks performed by the `check_timing` command.

When set to `true`, the `check_timing` command reports all the checks performed, even if there is no violation(s) for the corresponding check type.

When set to `false`, only the check types that have any violations will be reported.

timing_check_timing_signal_level_high_to_low_threshold

`timing_check_timing_signal_level_high_to_low_threshold` *double*

Default: `0.0`

Data_type: Double

Sets the threshold value for warning the signal level difference between driver and receiver voltages, when the driver is at a higher voltage than the receiver voltage.

The software shows the signal level warnings (`check_timing signal_level`) that report the net driver/receiver pin pairs for which the pin voltage does not match. The signal level warning messages can be filtered based on these threshold values.

timing_check_timing_signal_level_low_to_high_threshold

```
timing_check_timing_signal_level_low_to_high_threshold double
```

Default: `0.0`

Data_type: Double

Sets the threshold value for warning the signal level difference between driver and receiver voltages, when the driver is at a lower voltage than the receiver voltage.

The software shows the signal level warnings (`check_timing signal_level`) that report the net driver/receiver pin pairs for which the pin voltage does not match. The signal level warning messages can be filtered based on these threshold values.

timing_clock_phase_propagation

```
timing_clock_phase_propagation {positive | negative | both}
```

Default: `both`

Data_type: Enum

Allows you select the appropriate clock phases at register clock pins when both positive and negative phases of the same clock signal are seen in the clock network. For example, a register whose clock pin is fed by a two-input XOR gate where one input is a clock port and the other input is a data port. When the value of the data signal of the XOR gate is unknown, the XOR gate can see both positive and negative phases of the clock signal.

`both`	Selects both positive and negative clock phases at the register clock pins.
`positive`	Selects positive clock phases.
`negative`	Selects negative clock phases.

timing_clock_source_paths_unconstrained_mark_clock_used_as_data

```
timing_clock_source_paths_unconstrained_mark_clock_used_as_data {true | false}
```

Default: `true`

Data_type: Boolean

The `is_clock_used_as_data` property (by using the `get_property` command) returns a value of `true` for unconstrained clock source network.

timing_clock_source_use_driving_cell

```
timing_clock_source_use_driving_cell  {true | false}
```

Default: `true`

Data_type: Boolean

When set to `false`, delay calculation in propagated clock mode will use an idealized driver model at the `create_clock` root pin or port when it calculates delays for the first stage of the clock tree. The idealized model is a ramp input with the transition time set by the system default transition, which is normally 0ns.

When this variable is set to `true`, the drive characteristics of the cell output pin that `create_clock` is asserted on will be used to calculate the first stage clock tree delays and slews. The input slew used for this driver model is whatever slew has been propagated to the cell's input pins from upstream logic. The arrival time of the clock at the root pin is not modified in any way.

If you wish to specify the input slews to be used for the clock driver cell, that is, not use the upstream propagated slews - you can use the `set_annotated_transition` constraint on the cell's input pins to control this. Setting an input transition of 0ns on the input pins can be useful when trying to correlate analysis results with other applications which do not use the full propagated transition times.

timing_clock_uncertainty_from_to_precedence

```
timing_clock_uncertainty_from_to_precedence {true | false}
```

Default: `false`

Data_type: Boolean

Sets the clock uncertainty precedence.

When set to `true`, the software gives higher precedence to clock-to-clock uncertainty.

timing_collection_all_fanin_fanout_traversal_mode

```
timing_collection_all_fanin_fanout_traversal_mode {hierarchical | flat}
```

Default: `flat`

Data_type: Enum

Controls how the software processes netlists during `all_fanin`/`all_fanout` command processing.

timing_collection_result_display_limit

```
timing_collection_result_display_limit integer
```

Default: `100`

Limits the number of objects of a collection to be displayed in the output report.

When set to an integer value, all the commands, such as, `get_cells`, `add_to_collection`, and `query_objects` commands, that display collection objects in the output report will be impacted.

When set to `-1`, all the elements in the collection will be displayed in the output.

timing_collection_variable_assignment_compatibility

```
timing_collection_variable_assignment_compatibility {true | false}
```

Default: `true`

Data_type: Boolean

Disables printing the names of the output of `get_*` commands.

When set to `false`, the software prints the `get_*` command output on the shell. This attribute should be used for debugging purposes only.

timing_context_apply_default_primary_input_assertion

```
timing_context_apply_default_primary_input_assertion {true | false}
```

Default: `false`

Data_type: Boolean

Propagates the phase at block level for unmapped clocks.

When set to `true`, the nets in the fanout of data ports with unmapped clocks are considered to have infinite timing windows during SI context analysis.

When set to `false`, the unmapped top clock arrivals on data ports are dropped during context analysis.

timing_context_apply_port_sdc_exceptions

```
timing_context_apply_port_sdc_exceptions {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software will consider path exceptions on block constraints in the hierarchical context flow.

timing_context_clock_mapping_percentage_tolerance

`timing_context_clock_mapping_percentage_tolerance` *double*

Default : `0.0`

Data_type: Double

Controls the tolerance value of period and waveform in clock mapping.

timing_context_clock_mapping_check_waveform

`timing_context_clock_mapping_check_waveform {true | false}`

Default: `true`

Data_type: Boolean

Controls the waveform checking.

timing_context_continue_on_invalid_module_list

`timing_context_continue_on_invalid_module_list {true | false}`

Default: `false`

Data_type: Boolean

Enables to continue timing analysis without generating timing context.

When set to `true`, the software will continue with timing analysis without generating timing context, when all the specified modules/instances are invalid.

timing_context_clock_phase_based_clock_mapping

```
timing_context_clock_phase_based_clock_mapping
{true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the port-based clock mapping search is enhanced by additionally attempting to map originally unmapped top clocks.

Such unmapped top clock on data port is to be mapped to block clock if top clock was mapped to block clock as a

clock on clock port.

Note: This attribute can be used when `timing_context_port_based_clock_mapping` is set to `true`.

timing_context_data_phase_based_clock_mapping

```
timing_context_data_phase_based_clock_mapping {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the port-based clock mapping search () is enhanced by additionally trying to map originally unmapped top clocks.

Such unmapped top clock on data port is to be mapped to block clock if top clock was mapped to block clock on other data ports.

Note: This attribute can be applied when `timing_context_port_based_clock_mapping` is set to `true`.

timing_context_port_based_clock_mapping

```
timing_context_port_based_clock_mapping {true | false}
```

Default: `false`

Data_type: Boolean

Enables clock mapping based on block ports.

When set to `true`, clock mapping is done depending solely on clock property of the top and block clock for each clock and data port separately. For enabling N-to-1 or 1-to-N clock mapping, this attribute should be set `true`.

When set to `false`, only 1-to-1 clock mapping is done. In such cases, the following rules are used to determine clock mapping:

- Rule1: Clocks with matching property (period/duty cycle)
- Rule2: Clocks with matching name
- Rule3: Clocks present on maximum number common ports
- Rule4: Any one of the matching clocks

For example,

- Consider the following illustration:



When set to `true`, the following mapping will be seen:

Block Port	Block Clock	Mapped top clock
P1	CLK	CLK, TCLK

When set to `false`, CLK will be mapped to CLK according to Rule 2.

Notes:

- The context_data property of block port can be accessed only when this attribute is set `true`.
- When set to `true`, the software may increase the pessimism as the worst latency/arrival will be picked for block context analysis.
- If there are multiple block clocks that can be mapped to same top clock on the same block port, then Tempus does not perform auto mapping (when set to `true`). Whereas for the `false` setting, clock mapping is done depending on the above mentioned rules.

timing_constraint_disable_min_max_input_delay_worst_casing

```
timing_constraint_disable_min_max_input_delay_worst_casing {true | false}
```

Default: `false`

Data_type: Boolean

Disables the override of the maximum value (set using the `set_input_delay -max` parameter) from the specified `-min` value.

timing_constraint_enable_detailed_report_invalid_begin_end_points

```
timing_constraint_enable_detailed_report_invalid_begin_end_points {true | false}
```

Default: `false`

Data_type: Boolean

Enables detailed reporting of invalid begin/end points specified with the `set_false_path`/`set_multicycle_path` constraints.

When set to `false`, the software checks only the structural end points.

timing_constraint_enable_report_invalid_begin_end_points

```
timing_constraint_enable_report_invalid_begin_end_points {true | false}
```

Default: `false`

Data_type: Boolean

Enables reporting of invalid begin points/end points specified with the `set_false_path` or `set_multicycle_path` constraint.

timing_constraint_enable_search_path

```
timing_constraint_enable_search_path {true | false}
```

Default: `false`

Data_type: Boolean

Enables search of constraint files from the search path.

timing_constraint_enable_separate_multicycle_data_checks

```
timing_constraint_enable_separate_multicycle_data_checks {true | false}
```

Default: `false`

Data_type: Boolean

Controls multicycle path constraint application on data checks.

timing_constraint_enable_logging

```
timing_constraint_enable_logging {true | false}
```

Default: `false`

Data_type: Boolean

Enables dumping of constraint commands in the log file.

Note: The impact of this attribute is visible only when the constraints are loaded using the `read_sdc` command or the view definition file.

timing_constraint_load_minimal_set_for_automated_eco

`timing_constraint_load_minimal_set_for_automated_eco {true | false}`

Default: `false`

Data_type: Boolean

Skips loading the SDC constraints that are not required in the Tempus session, when the `opt_signoff` command is used. The timing data is obtained from the ECO timing database - this attribute reduces the TAT for designs with complex constraint mode files.

timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency

`timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency {true | false}`

Default: `false`

Data_type: Boolean

Excludes check arc delays from slack computation.

When set to `true`, the software excludes check arc delays from slack computation for paths constrained by the `set_max_delay`/`set_min_delay` `-ignore_clock_latency` command.

timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency

`timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency {true | false}`

Default: `false`

Data_type: Boolean

Excludes the input/output delays from slack computation.

When set to `true`, the software excludes the input/output delays from slack computation for paths constrained by the `set_max_delay`/`set_min_delay` `-ignore_clock_latency` command.

timing_constraint_path_delay_exclude_unconstrained_endpoints

`timing_constraint_path_delay_exclude_unconstrained_endpoints {true | false}`

Default: `false`

Data_type: Boolean

When path delay exceptions are asserted (using the `set_min_delay` or `set_max_delay` commands) with a complete `-from` and `-to` specification, the exception can be applied between any two data path pins. If the constraint assertion includes only a -from and/or -through argument, then a set of valid endpoints must be

established to define the constrained paths. This attribute allows you to configure the set of valid endpoints (inferred -to pins) that are selected.

When set to `false`, the set of allowed endpoints is simply all of the endpoints in the combinational fanout of the `-from` and/or `-through` arguments - essentially, wherever the path tracing stops. The path delay constraint will be applied as if there were a -to argument specified for each endpoint.

When set to `true`, the software uses different rules to determine the valid set of inferred -to endpoints. Only constrained pins in the fanout of -from and/or -through arguments are considered as valid -to endpoints for the constraint. A constrained endpoint will have some type of timing check - such as Setup or Hold already in force on that pin. Additionally, each transition at the endpoints will be examined to determine if each is individually constrained. The path exception will only be applied to the edges that are constrained. As an example, if a register preset or a clear pin was a potential endpoint, then only the edge constrained by the recovery or removal check will have the path delay applied. To apply the path delay constraint to unconstrained endpoints while in this mode, you must specify them explicitly with the `-to`, `-rise_to`, or `-fall_to` parameters of the `set_min_delay` and `set_max_delay` constraints.

timing_constraint_path_delay_include_clock_pin_endpoints

```
timing_constraint_path_delay_include_clock_pin_endpoints {true | false}
```

Default: `false`

Data_type: Boolean

Considers register clock pins as end-points for path delay constraints applied upstream.

When set to `true`, the `set_max_delay` command considers unconstrained paths ending at the register clock pins.

timing_constraint_warn_for_timing_derate_exceeding_max_limit

```
timing_constraint_warn_for_timing_derate_exceeding_max_limit double
```

Default: `100.0`

Value Range: 0 to 100

Data_type: Double

Sets a limit on derate values. If a larger derate value is applied, the derates are still processed. However, the software issues a warning.

Note: It is recommended to specify derate values within the acceptable value range, because large derate values may cause errors during processing.

timing_continue_on_error

`timing_continue_on_error {true | false}`

Default: `false`

Data_type: Boolean

When set to `true`, the software continues to process if an error occurs when any constraints are used, or during timing analysis, or when a library is read. This attribute does not have any effect on other commands, except constraints or commands related to timing analysis.

When set to `false`, if an error occurs, during timing analysis, library reading, or defined constraints, the software stops processing and exits.

timing_constraint_update_io_latency_averaging_mode

`timing_constraint_update_io_latency_averaging_mode {rise_fall | rise_only}`

Default: `rise_fall`

Data_type: Enum

Controls the analysis to update IO latency for both rise and fall, or for rise only. You can set one of the options - `rise_fall`, `rise_only`.

timing_cppr_opposite_edge_mean_scale_factor

`timing_cppr_opposite_edge_mean_scale_factor` *`double`*

Default: 1.0

Value Range: 0 to 1

Data_type: Double

Controls the mean delay component for CPPR credit for opposite transition edges at a common pin.

When this attribute is specified, the software provides a finer level configuration of the CPPR credit for reducing Common Path Pessimism Removal (CPPR) in timing analysis. The software allows a certain amount of CPPR credit in the case of opposite edge transitions to be scaled to a user-specified value, that is between zero credit or full credit provided by the existing controls.

You can use `timing_cppr_opposite_edge_sigma_scale_factor` attribute for adjustment of the sigma component of the CPPR credit. In SOCV analysis mode, you can use both of these attributes to separately control the adjustment of the mean and sigma components of the CPPR credit. In regular static-timing analysis (STA) mode (without SOCV), only the mean scale factor is considered for CPPR adjustment scaling.

timing_cppr_opposite_edge_sigma_scale_factor

```
timing_cppr_opposite_edge_sigma_scale_factor double
```

Default: 1.0

Value Range: 0 to 1

Data_type: Double

Controls sigma delay component for CPPR credit for opposite transition edges at common pin.

timing_cppr_opposite_edge_sigma_scale_factor_net

```
timing_cppr_opposite_edge_sigma_scale_factor_net double
```

Default: `1.0`

Data_type: Double

Controls the net sigma delay component for CPPR credit for opposite transition edges at the common pin.

The scale factors are applicable only when the common pin has mismatched transitions for the launch and capture paths.

timing_cppr_opposite_edge_sigma_scale_factor_cell

```
timing_cppr_opposite_edge_sigma_scale_factor_cell double
```

Default: `1.0`

Data_type: Double

Controls the cell sigma delay component for CPPR credit for opposite transition edges at the common pin.

The scale factors are applicable only when the common pin has mismatched transitions for the launch and capture paths.

timing_cppr_propagate_thru_latches

```
timing_cppr_propagate_thru_latches {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the CPPR of a timing path crossing a latch (or latches), for example, ff1 -> lat1 -> ff2, is calculated using the common point of the origin device (ff1) and the capture device at the end of the path (ff2).

When set to `false`, the segment-based CPPR calculation is performed, considering the common point of latch

(lat1) and the capture device (ff2). This CPPR setting is considered only when the latch thru analysis mode is enabled and a latch thru path exists.

timing_cppr_remove_clock_to_data_pessimism

```
timing_cppr_remove_clock_to_data_pessimism {true | false}
```

Default: `false`

Data_type: Boolean

When set to true, removes clock re-convergence pessimism (CRP) for clock source paths.

timing_cppr_self_loop_mode

```
timing_cppr_self_loop_mode {true | false}
```

Default: `true`

Data_type: Boolean

When set to `true` in case of self-loop paths, computes CPPR adjustment by taking the difference between early and late clock arrival time of the common point; the clock pin of the flop. A self-loop path is a path that goes from a clock pin of a flop and ends at the data pin of the same flop. However detecting self-loop paths requires runtime, and is design dependant.

When set to `false`, does not detect self-loop paths. In this case, the software computes CPPR adjustment by using the common point as the output pin driving the clock pin of the flop. This might result in better runtime, but less accurate results. The loss of accuracy is the difference between early and late delay of the interconnect portion driving the clock pin.

timing_cppr_skip_clock_reconvergence

```
timing_cppr_skip_clock_reconvergence {true | false}
```

Default: `false`

Data_type: Boolean

Specifies the branch point to use for computing clock path pessimism removal (CPPR) adjustment when there is reconvergence in the clock tree.

When set to `false`, the software uses the branch point closest to the register clock pins where the clocks reconverge. When set to `true`, the software uses the farthest branch point from the register clock pins (that is, the branch point closest to the clock root pin where the clocks diverge).

timing_cppr_skip_clock_reconvergence_for_unmatched_clocks

```
timing_cppr_skip_clock_reconvergence_for_unmatched_clocks {true | false}
```

Default: `false`

Data_type: Boolean

Allows the software to skip re-convergence of unmatched clocks.

When set to `true`, the software uses the farthest branch point from the register clock pins when the launching and capturing clocks are different - the re-converge points are ignored and not searched.

timing_cppr_threshold_ps

```
timing_cppr_threshold_ps double
```

Default: `20.0` ps

Data_type: Double

Specifies the maximum amount of pessimism that clock path pessimism removal (CPPR) analysis is allowed to leave in the path. The unit is pico seconds. Setting this attribute to a specified value means that all the paths might be reported without having their pessimism removed. This attribute can be used to speed up CPPR analysis run time. The larger the value of the attribute, the faster the run time.

timing_cppr_transition_sense

```
timing_cppr_transition_sense {normal | same_transition | same_transition_expanded}
```

Default: `normal`

Data_type: Enum

Specifies the transition sense of the launching and capturing clocks at the common node, to calculate clock path pessimism removal (CPPR).

- `normal`: The CPPR analysis computes the CPPR value even if the launching and capturing clock transitions at the common node are in opposite directions. If the transition sense at the common node are in opposite directions, the pessimism is the minimum of the difference between late and early arrival times for rise and fall clock transitions.
- `same_transition`: The CPPR analysis computes the CPPR value only if the launching clock transition is same as the capturing clock transition at the common node. If the transitions are different, the CPPR value is zero.
- `same_transition_expanded`: The CPPR analysis computes the CPPR value only if the launching clock

transition is the same as the capturing clock transition at the common node. If the transitions are different, the software searches until it finds a common point where the transitions are the same, and calculates the CPPR value from that point.

timing_create_clock_default_propagated

```
timing_create_clock_default_propagated {true | false}
```

Default: `false`

Data_type: Boolean

Enables clocks to be created in propagated mode.

By default, all clocks, created using `create_clock` and `create_generated_clock` commands, are in ideal mode.

When the attribute is set to `true`, all subsequently created regular clocks, derived clocks, and library generated clocks are in propagated mode.

There is no effect on the functionality of `set_propagated_clock` and `set_clock_latency` commands.

timing_default_opcond_per_lib

```
timing_default_opcond_per_lib {true | false}
```

Default: `true`

Data_type: Boolean

Determines the default operating condition behavior when there is no explicit user operating condition.

When set to `true`, each timing library uses its own internally-defined default operating condition specification.

When set to `false`, the default operating condition of the master library (that is, the first library in the library search list) is used to set the PVT operation point for the entire design. All timing libraries are accessed and derated to that operating point.

timing_defer_mmmc_obj_updates

```
timing_defer_mmmc_obj_updates {true | false}
```

Default: `false`

Data_type: Boolean

By default, the multi-mode multi-corner (MMMC) update commands, `update_constraint_mode` and `update_analysis_view`, actively load new data upon execution. Since

all the timing information is reloaded with each update command, executing a sequence of updates can result in multiple reloads and a subsequent degradation of performance and turn-around-time.

When set to `true`, the software allows a sequence of MMMC updates to be accumulated before new data is loaded and analyzed.

If the `set_analysis_view -update_timing` parameter is specified, the software indicates that the MMMC object updates are complete and new configuration can be loaded and analyzed.

For example:

```
update_constraint_mode -name ioTiming -sdc_files {postCtsIO.sdc}

update_constraint_mode -name coreTiming -sdc_files {postCtsCore.sdc}

set_analysis_view -update_timing
```

timing_derate_aocv_dynamic_delays

```
timing_derate_aocv_dynamic_delays {true | false}
```

Default: `true`

Data_type: Boolean

Controls AOCV derating of SI-induced delays.

When set to `false`, AOCV derates will not be applied on dynamic, SI-induced delays, that is, a value of `false` ensures AOCV derating is only applied to the static component of the delay.

When set to `true`, AOCV derating is applied to both static and dynamic arc delay components.

timing_derate_aocv_reference_point

```
timing_derate_aocv_reference_point {true | false}
```

Default: `1`

Data_type: Integer

Allows specification of reference point for AOCV factors. As per the current AOCV library definitions, the AOCV reference point is always expected to be set at `1.0`.

timing_derate_dynamic_compatibility

```
timing_derate_dynamic_compatibility {true | false}
```

Default: `true`

Data_type: Boolean

Controls how the `set_timing_derate` command factors that are not specified, using either `-static` or `-dynamic` options, are applied to the delay.

When this attribute is set to `1`, the behavior will be compatible with previous releases. In this mode, the static and dynamic components will be summed up before applying derating.

When set to 0, the static and dynamic components will be derated separately and then combined. A setting of 0 is equivalent to using separate derate assertions with the `set_timing_derate -dynamic` and `-static` options explicitly specified.

timing_derate_incremental_adjust_additive_mode

```
timing_derate_incremental_adjust_additive_mode {true | false}
```

Default: `false`

Data_type: Boolean

Allows the software to accumulate incremental derate values instead of overwriting them.

When set to `false`, the successive application of incremental derating will overwrite any existing derate value with the new value.

When set to `true`, the successive applications of the incremental derates will be accumulated into a final derate value.

timing_derate_enable_incremental_multiplication_mode

```
timing_derate_enable_incremental_multiplication_mode {true | false}
```

Default: `false`

Controls the overwrite or accumulative mode settings when the `set_timing_derate -multiply` parameter is specified.

When set to `false`, the latest `set_timing_derate -multiply` command will overwrite the previous constraints at the same level.

When set to `true`, all the values in history will be accumulatively multiplied, when multiple `set_timing_derate -multiply` settings are made on the same object.

timing_derate_incremental_multiply_accumulative_mode

```
timing_derate_incremental_multiply_accumulative_mode {true | false}
```

Default: `false`

Data_type: Boolean

Controls the overwrite or accumulative mode settings when the `set_timing_derate -multiply` parameter is specified. You can use this attribute when `timing_derate_enable_incremental_multiplication_mode` is set to `true` (that is, the `-multiply` setting must be enabled).

When set to `false`, the last specified `set_timing_derate -multiply` command will overwrite the previous constraints at the same level.

When set to `true`, all the values saved in history will be accumulatively multiplied, when multiple `-multiply` settings are made on the same object.

Examples

- The following example shows that in overwrite mode, the multiplicative adjustment will be updated to 0.5:

```
set_global timing_derate_enable_incremental_multiplication_mode true
set_global timing_derate_incremental_multiply_accumulative_mode false
set_timing_derate -multiply 1.2
set_timing_derate -multiply 0.5
```

- The following example shows that in accumulative mode, the multiplicative adjustment will be updated to 1.2*0.5=0.6:

```
set_global timing_derate_enable_incremental_multiplication_mode true
set_global timing_derate_incremental_multiply_accumulative_mode true
set_timing_derate -multiply 1.2
set_timing_derate -multiply 0.5
```

timing_derate_ocv_reference_point

```
timing_derate_ocv_reference_point {true | false}
```

Default: `1`

Data_type: Integer

Allows specification of reference point for OCV factors. As per the current OCV library definitions, the OCV reference point is always expected to be set at `1.0`.

timing_derate_negative_delay_backward_compatibility

```
timing_derate_negative_delay_backward_compatibility {true | false}
```

Default: `true`

Data_type: Boolean

Controls the effective derate calculation for negative delays. When effective derate is requested for negative

delays, the value will be converted according to the negative delay scaling rule, e.g., 2 - positiveVal.

timing_derate_spatial_distance_unit

```
timing_derate_spatial_distance_unit {default | 1um | 1nm}
```

Default: `default`

Data_type: Enum

Specifies distance units in the SOCV and AOCV library formats.

This attribute supports 3 distance units - `default`, `1um`, and `1nm`. The default value is the default units that are derived from the current system units for distance (µm/nm overrides the default).

Note: The `timing_derate_spatial_distance_unit` attribute should be set before a design is loaded.

timing_derate_voltage_scaling_mode

```
timing_derate_voltage_scaling_mode first_library | snap_to_nearest | interpolate
```

Default: `first_library`

Data_type: Enum

Controls how timing derates are computed when the analysis voltage is not at the characterization voltage of any of the currently loaded timing libraries. When the timing analysis environment is configured for interpolation, the MMMC library set will contain a list of timing libraries at different voltage characterizations points.

You can specify the following options:

`first_library`	Derates associated with the first library in the timing library list will be used.
`snap_to_nearest`	Allows the software to choose one set of derates from the library closest to the required analysis voltage.
`interpolate`	Allows the timer to perform interpolation of the derates associated with the two bounding reference libraries.

Consider the following example:

```
create_library_set -name ls_interp_1.50V_1.80V
-timing [list stdcell_1.50V.lib stdcell_1.75V.lib stdcell_1.80V.lib]
```

In this example, by default (`first_library`) the derates associated with the library stdcell_1.50V.lib will be used.

When set to `snap_to_nearest`, if the analysis voltage is 1.70V, the derates from the library stdcell_1p75V.lib will be used.

When set to `interpolate`, if the analysis voltage is 1.60V, then interpolation of derates from the libraries

stdcell_1.50V.lib and stdcell_1.75V.lib will be used.

If interpolation mode is selected and the analysis voltage is beyond either the smallest or largest voltage, then the library derates will not be interpolated. The derates associated with the nearest bounding library will be used. If the analysis voltage is 1.90V, then only the derates from stdcell_1.80V.lib will be used.

If interpolation mode is enabled, but not all the bounding libraries have derate values, a default derate value of 1.0 will be assumed for the missing derates and used for interpolation. A TA-960 warning message will also be issued.

timing_disable_bus_contention_check

```
timing_disable_bus_contention_check {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, disables propagation of maximum delay along three state disable timing arcs and minimum delay along three state enable arcs. If you know that bus contention conditions do not occur in your design, disable these checks by setting these attributes to `true`.

Checks for setup and hold violations in three-state bus designs. Checks the worst delay path to ensure that the latched signals are stable and not unknown values - X. Ensures proper timing checks for bus contention.

timing_disable_clock_period_checks

```
timing_disable_clock_period_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, disables timing model clock period checks during timing analysis.

timing_disable_constant_propagation_for_sequential_cells

```
timing_disable_constant_propagation_for_sequential_cells {true | false}
```

Default: `false`

Data_type: Boolean

Controls constant propagation across sequential cells.

When set to `true`, the software disables constant propagation across sequential cells.

When set to `false`, the software enables constant propagation across sequential cells.

timing_disable_drv_report_on_constant_nets

```
timing_disable_drv_report_on_constant_nets {true | false}
```

Default: `false`

Data_type: Boolean

Disables DRV checks for nets that are disabled by constant propagation:
When the attribute is set to true, the `report_constraint` command does not report DRV violations for constant nets.

timing_disable_floating_bus_check

```
timing_disable_floating_bus_check {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, disables propagation of minimum delay through three state disable timing arcs and maximum delay through three state enable arcs. These checks are only valid during floating bus conditions.

timing_disable_genclk_combinational_blocking

```
timing_disable_genclk_combinational_blocking
{true | false}
```

Default: `false`

Data_type: Boolean

Disables blocking of the latency path for a generated clock.

When set to `true`, allows the latency path to traverse through the upstream generated clock.

When set to `false`, if two back-to-back generated clocks are on the same path, and if the downstream generated clock is set using the `create_generated_clock -combinational` parameter, then the latency path for this generated clock is blocked by the upstream clock. The downstream generated clock is created with zero latency.

timing_disable_inferred_clock_gating_checks

```
timing_disable_inferred_clock_gating_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, disables clock gating checks that are inferred on combinational elements in the clock path. Explicit clock gating checks that are described in the timing library are not affected by this attribute.

timing_disable_inout_output_side_timing_checks

```
timing_disable_inout_output_side_timing_checks {true | false}
```

Default: `false`

Data_type: Boolean

Disables timing checks on both the input and output sides of the bidirectional pin.

When set to `false`, the timing checks on both the input and output sides of the bidirectional pin are analyzed.

timing_disable_internal_inout_cell_paths

```
timing_disable_internal_inout_cell_paths {true | false}
```

Default: `true`

Data_type: Boolean

When set to `false`, enables internal bidirectional feedback paths that are completely contained in one instance.

timing_disable_internal_inout_net_arcs

```
timing_disable_internal_inout_net_arcs {true | false}
```

Default: `true`

Data_type: Boolean

When set to `true`, this attribute disables internal bidirectional feedback paths that span multiple instances.

When set to `false`, this attribute enables internal bidirectional feedback paths, which are feedback paths that you might not want to use during analysis and optimization.

This attribute applies only to paths that span multiple instances. The attribute has no impact on internal feedback paths that are completely contained in one instance.

timing_disable_lib_pulse_width_checks

```
timing_disable_lib_pulse_width_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, disables timing model pulse width checks during timing analysis.

timing_disable_library_data_to_data_checks

```
timing_disable_library_data_to_data_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software disables the timing arcs between any data-to-clock or clock-to-clock checks - when timing_type attribute is set to one of the following:

- `non_seq_setup_rising`
- `non_seq_setup_falling`
- `non_seq_hold_rising`
- `non_seq_hold_falling`

This attribute affects non-sequential checks only, where a reference pin of the timing check is a clock pin. The data-to-data checks (where the reference pin of the timing check is a data pin) are not affected by this setting. To disable data-to-data checks you can use the `timing_disable_library_data_to_data_checks` attribute.

timing_disable_library_tieoffs

```
timing_disable_library_tieoffs {true | false}
```

Default: `false`

Data_type: Boolean

Disables constant functions in a library.

When set to `false`, the software considers constant propagation from library tie high/low cells prior to those from

logical netlist and constraints. There may be cases where programmable tie cells are used whose value can be changed later in the flow, assuming the programmable tie cells are modelled with constant output functions.

When set to `true`, the library constants are not propagated during timing analysis or timing optimization.

timing_disable_netlist_constants

```
timing_disable_netlist_constants {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, ignores constants defined in the Verilog netlist. This includes all uses of the in-place constant notation, such as `1'b0`, and the use of power and ground nets. Constants from `set_case_analysis` assertions and from tie-hi and tie-lo cells are still recognized.

When set to `false`, the software recognizes all constants.

timing_disable_model_parasitics

```
timing_disable_model_parasitics {true | false}
```

Default: `0`

Data_type: int

Disables saving parasitics data into a model.

When set to `true`, the software will not save parasitics data into the boundary model, so the generated boundary models will occupy a smaller disk space. However, while reading this boundary model at the top level, the original block parasitics (which was used while generating the model) should be read along with the top level parasitics.

When set to `false`, the parasitic data is not stored in the boundary models. Hence, the user does not need to read the block parasitics, since it is read from the boundary models. By default, this attribute is set to `false`.

timing_disable_nochange_checks

```
timing_disable_nochange_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, disables no change timing model checks during timing analysis.

timing_disable_non_sequential_checks

```
timing_disable_non_sequential_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software disables the timing arcs between any data-to-clock or clock-to-clock checks.

This attribute affects non-sequential checks only, where a reference pin of the timing check is a clock pin. The data-to-data checks (where the reference pin of the timing check is a data pin) are not affected by this setting. To disable data-to-data checks you can use the `timing_disable_library_data_to_data_checks` attribute.

timing_disable_output_as_clock_port

```
timing_disable_output_as_clock_port{true | false}
```

Default: `false`

Data_type: Boolean

Allows a clock path to be considered as a data path.

When set to `true`, the software allows a clock source path, leading to an output or bidi port, to be treated as a data path, irrespective of constraints, such as, `set_output_delay`, `set_data_check`, or `set_max_delay`, asserted on that port.

When set to `false`, all the clock source paths, leading to output ports, are considered as valid clock end points. Any unconstrainted output port may not be considered as data end point.

timing_disable_parallel_arcs

```
timing_disable_parallel_arcs {true | false}
```

Default: `true`

Data_type: Boolean

Enables the software to automatically perform reduction of parallel delay arcs and check arcs to improve efficiency.

When set to `false`, parallel delay and check arc reduction will be disabled.

timing_disable_pulse_width_same_edge_si_cppr_mode

```
timing_disable_pulse_width_same_edge_si_cppr_mode {true | false}
```

Default: `true`

Data_type: Boolean

Enables SI CPPR credit up to the last divergent pin for pulse width checks for zero width pulse clocks. By default, the software credits minimum of rise and fall transition CPPR up to the common pin as the transitions mismatch at the common pin.

When set to `true`, the software computes base delay CPPR up to the common pin as CPPR credit for the path. The software then finds an alternate common timing pin in the clock network where the transition for launch and capture match. If a match is found, the software includes the SI delay up to this pin in the CPPR credit as well.

timing_disable_report_header_info

```
timing_disable_report_header_info {true | false}
```

Default: `false`

Data_type: Boolean

Controls whether timing reports are generated using a common report header.

When set to `false`, the software generates timing reports with a report header that includes the following information:

- Tempus software version number
- Platform on which the software is being run
- Time when the report was generated
- Command used to generate the report

Any report-specific banner information is included in the same header box, but listed after the common header information.

When set to `true`, the software generates backward-compatible timing reports without the additional header information.

timing_disable_retime_clock_path_slew_propagation

```
timing_disable_retime_clock_path_slew_propagation {true | false}
```

Default: `true`

Data_type: Boolean

When set to `true`, path based analysis is performed on data paths only and the clock segments are not considered.

When set to `false`, analysis is performed on both data and clock paths.

timing_disable_sdf_retain_arc_merging

```
timing_disable_sdf_retain_arc_merging {true | false}
```

Default: `false`

Data_type: Boolean

Controls handling of retain arcs in SDF2.1.

When set to `false`, the Retain information is merged into SDF 2.1, as Retain represents a valid minimum path timing for a cell.

When set to `true`, the Retain arc merging will be disabled.

timing_disable_skew_checks

```
timing_disable_skew_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, disables library skew checks for timing analysis.

timing_disable_test_signal_arc

```
timing_disable_test_signal_arc {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, timing analysis will not analyze the signal arcs coming from or going to the test pin. Such pins are marked in the timing library with either the `test_scan_in`, the `test_scan_enable` attribute for the input pin, the `test_scan_clock` attribute for the input clock pin, or the `test_output_only` attribute for the output pin.

The following are the equivalent Liberty and TLF pin attributes:

Liberty and TLF Pin Attributes

Type	Liberty	TLF
enable	`test_scan_enable`	`SCAN_PINTYPE` (ENABLE)

input	`test_scan_in`	`SCAN_PINTYPE` (INPUT)
input clock	`test_scan_clock`	`SCAN_PINTYPE` (CLOCK)
output	`test_output_only`	`SCAN_PINTYPE` (OUTPUT)

timing_max_transition_use_si_transition

```
timing_max_transition_use_si_transition {true | false}
```

Default: `false`

Data_type: Boolean

Enables reporting of noisy slew in DRV flows.

timing_model_netlist_exclude_dummy

```
timing_model_netlist_exclude_dummy {true | false}
```

Default: `false`

Data_type: Boolean

Excludes the dummy cell definitions that are written to the model netlist.

timing_disable_timing_model_latch_inferencing

```
timing_disable_timing_model_latch_inferencing {true | false}
```

Default: `true`

Data_type: Boolean

Controls whether latch behavior is inferred for cell descriptions tagged with the Liberty `timing_model_type` attribute, including all values: `abstracted`, `extracted`, or `qtm`.

When set to `false`, the software infers latch behavior from the timing check and timing delay arc relationships in the model's description.

When set to `true`, latch inferencing does not happen for cells with the `timing_model_type` attribute. Explicit Liberty latch modelling constructs must be coded into the description in order to model latch behavior.

This attribute does not affect Liberty timing model cells that do not contain the `timing_model_type` attribute. Standard cell core libraries generally do not have this attribute specified; therefore, they typically are not be affected by this attribute setting.

timing_disable_tristate_disable_arcs

```
timing_disable_tristate_disable_arcs {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, disables all 0/1->Z transitions of tristate arcs during timing analysis. By default, all tristate arcs are enabled.

timing_disable_user_data_to_data_checks

```
timing_disable_user_data_to_data_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, disables data to data checks that are created by the `set_data_check` command. This attribute does not affect non-sequential timing checks described in the timing library.

timing_driving_cell_override_library

```
timing_driving_cell_override_library {true | false}
```

Default: `false`

Data_type: Boolean

Controls the selection of library cells specified using the `set_driving_cell`command.

When set to `true`, a driving cell will be picked up from the Max Library for max drive cell adjustment and Min Library for min drive cell adjustment, irrespective of the library specified with the `set_driving_cell -library` parameter.

When set to `false`, the libraries are selected as specified with the `set_driving_cell` command.

timing_drv_analysis_ignore_latency_phases

```
timing_drv_analysis_ignore_latency_phases {true | false}
```

Default: `false`

Data_type: Boolean

Enables ignoring latency phases in DRV analysis.

timing_enable_aocv_slack_based

```
timing_enable_aocv_slack_based {0 | 1}
```

Default: `false`

Data_type: Boolean

Enables AOCV analysis using slack-based method.

When set to `true`, the software will try to remove excess pessimism from the graph-based analysis (GBA) AOCV stage counts. The paths to endpoints, which are under the specified slack criteria (default 0.0 ns), are considered when computing stage counts. This allows the stage counts and subsequent AOCV derates to be aligned with the failing paths – and not unnecessarily punished by derates coming from related short paths that meet timing requirements.

Note: This setting is not recommended for signoff timing analysis in Tempus. For signoff, path-based analysis (PBA) is the recommended flow for removing excess GBA pessimism.

timing_enable_case_analysis_conflict_warning

```
timing_enable_case_analysis_conflict_warning {true | false}
```

Default: `true`

Data_type: Boolean

Displays a warning when propagated and asserted constant values are not the same when a full timing update is performed.

When set to `true`, a report file, named `CTE_constant_mismatch.rpt`, is generated that contains details of pins and the reason for constant mismatch. This report is over-written each time a full timing update takes place.

timing_enable_clock_to_clock_clock_gating_check

```
timing_enable_clock_to_clock_clock_gating_check {true | false}
```

Default: `false`

Data_type: Boolean

Enables clock-to-clock clock gating checks.

When set to `true`, the software performs clock to clock gating checks at gating elements. For clock to clock gating

checks, the frequency of a clock arriving at the reference side of a check should be equal to or greater than the frequency of the clock arriving at the signal side of the check.

timing_enable_clock_phase_based_rise_fall_derating

```
timing_enable_clock_phase_based_rise_fall_derating {true | false}
```

Default: `false`

Data_type: Boolean

Determines the interpretation of edge-specific derating factors for clock paths, specified using the `set_timing_derate -rise/-fall` parameters.

When set to `false`, the `set_timing_derate -rise/-fall` parameter refer to the direction of the transition from the perspective of each individual output pin along the clock path.

When set to `true`, the `-rise/-fall` parameter refers to which edge (or phase) of the clock is triggering the transition as viewed from the source of the clock where the clock waveform is defined.

timing_enable_data_through_clock_gating

```
timing_enable_data_through_clock_gating {true | false}
```

Default: `true`

Data_type: Boolean

When set to `false`, blocks signals arriving on the enable of the clock-gating check.

When set to `true`, allows both data and clock signals at the enable pin of a clock-gating check to propagate forward past the check.

timing_enable_db_attribute_query

```
timing_enable_db_attribute_query {true | false}
```

Default: `true`

Data_type: Boolean

Allows to query DB attributes with the `get_property` command.

timing_enable_derating_for_pulse_width_checks

`timing_enable_derating_for_pulse_width_checks {true | false}`

Default: `false`

Data_type: Boolean

Controls whether the cell check derates, set by `set_timing_derate -cell_check` command, can be applied for pulse width checks.

When set to `true`, the pulse width checks honor cell check derate factors.

timing_enable_early_late_data_slews_for_setuphold_mode_checks

`timing_enable_early_late_data_slews_for_setuphold_mode_checks {true | false}`

Default: `false`

Data_type: Boolean

Controls calculated delays.

When set to `true`, the software allows use of both the minimum and maximum data slews for setup and hold checks. By default, the minimum or maximum slew values for clock pins are considered, and for data pins only the maximum slew value is considered while calculating the setup/hold timing check values.

The `timing_enable_early_late_data_slews_for_setuphold_mode_checks` attribute enables an eight-way timing check for:

- Early/late libraries
- Clock/data pins
- Min/max slews

You can use this attribute in simultaneous setup or hold mode only. To enable this mode, you need to set `timing_enable_simultaneous_setup_hold_mode` attribute to `true`. In this mode, you can use the `report_delay_calculation` command (`-from` and `-to` parameters) to report all the setup/hold check values corresponding to minimum/maximum combination of clock and data slews.

timing_enable_genclk_divide_by_inherit_parent_duty_cycle

`timing_enable_genclk_divide_by_inherit_parent_duty_cycle{true | false}`

Default: `false`

Data_type: Boolean

Controls inheritance of duty cycle from master clock for generated clocks with divide by option.

timing_enable_genclk_source_path_register_limit

```
timing_enable_genclk_source_path_register_limit {true | false}
```

Default: `false`

Data_type: Boolean

Limits generated clock source latency path to traverse across one register only.

When set to `false`, the software allows master clock latency path to traverse across any number of sequential devices to reach the generated clock target pin.

When set to `true`, the software restricts the master clock latency path for a generated clock to traverse exactly one sequential device.

timing_enable_generated_clock_edge_based_source_latency

```
timing_enable_generated_clock_edge_based_source_latency {true | false}
```

Default: `true`

Data_type: Boolean

Controls how the software chooses generated clock source latency paths. Certain specifications of the `create_generated_clock`constraint imply a cause-and-effect relationship between the edges of the master clock and those of the generated clock.

When set to `true`, the software checks whether a path being analyzed has the correct cause-effect relationship consistent with the specified constraint. Paths that are found to be inconsistent are ignored. Only the paths that are in agreement with the required edge relationship are considered for source latency. If no such path exists, the software uses a source latency of `0` ns. This selection is done independently for each edge of the generated clock; therefore the result might be different for each edge.

The edge-to-edge sufficiency checks are performed assuming a Rise-to-Rise, Rise-to-Fall relationship between the master clock and the generated clock with an even number `-divide_by` factor. For the odd `-divide_by` factors, the sufficiency check is performed assuming a Rise-to-Rise, Fall-to-Fall relationship. The sufficiency checking is not performed for `-multiply_by` options.

When set to `false`, the software does not check paths for the correct cause-effect relationship.

timing_enable_get_obj_escaped_name_backward_compatible

```
timing_enable_get_obj_escaped_name_backward_compatible {true | false}
```

Default: `false`

Data_type: Boolean

Enables printing of escape characters in the `get_object_name` command output.

timing_enable_get_objects_regexp_compatibility

```
timing_enable_get_objects_regexp_compatibility {true | false}
```

Default: `false`

Data_type: Boolean

Defines the pattern search scope to boundaries deduced from hierarchy separation characters specified in the search pattern.

When set to `false`, the commands with `-regexp` parameter will search at the top level only (unless used with the `-hier` parameter).

When set to `true`, the commands will search the specified pattern within the hierarchies based upon the given pattern hierarchy (similar to pattern search without the `-regexp` parameter). The `-hier` along with the `-regexp` parameter shows the same behavior similar to when the command is used without the `-regexp` option - only the leaf level object name is compared to the pattern match.

Examples

Command behavior when this attribute is set to `false`:

```
get_pins -regexp seg2/.*/QN
**WARN: (TCLCMD-513): The software could not find a matching object of the specified type for the
pattern '.*/.*'
**ERROR: (TCLCMD-917): Cannot find 'pins' that match 'seg2/.*.*/QN'

get_pins -regexp seg2/.*/QN -hier
seg2/u3/QN seg2/u9/QN seg2/u14/QN

get_pins -regexp .*/QN -hier
seg1/u3/QN seg1/u9/QN seg1/u14/QN seg2/u3/QN seg2/u9/QN seg2/u14/QN seg3/u3/QN seg3/u9/QN
seg3/u14/QN u2/QN u5/QN u8/QN
```

Command behavior when this attribute is set to `true`:

```
get_pins -regexp seg2/.*/QN
seg2/u3/QN seg2/u9/QN seg2/u14/QN

get_pins -regexp seg2/.*/QN -hier
**WARN: (TCLCMD-513): The software could not find a matching object of the specified type for the
pattern '.*/.*'
**ERROR: (TCLCMD-917): Cannot find 'pins' that match 'seg2/.*.*.*.*.*/QN'

get_pins -regexp .*/QN -hier
seg1/u3/QN seg1/u9/QN seg1/u14/QN seg2/u3/QN seg2/u9/QN seg2/u14/QN seg3/u3/QN seg3/u9/QN
seg3/u14/QN u2/QN u5/QN u8/QN
```

timing_enable_hier_context_unmapped_clock_analysis

```
timing_enable_hier_context_unmapped_clock_analysis {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, for unmapped block clocks, the software enables timing analysis based on the block SDC constraints set on block ports.

timing_enable_hierarchical_get_nets_support

```
timing_enable_hierarchical_get_nets_support {true | false}
```

Default: `false`

Data_type: Boolean

Enables the hierarchical segment support for nets that are specified using the `get_nets` command.

When set to `false`, the `get_nets` command will report only flat nets.

timing_enable_individual_distance_inter_die_analysis

```
timing_enable_individual_distance_inter_die_analysis {true | false}
```

Default: `false`

Data_type: Boolean

Enables individual inter-die calculation in 3DIC analysis.

timing_enable_latch_thru_mode

```
timing_enable_latch_thru_mode {true | false}
```

Default: `false`

Data_type: Boolean

Enables the latch through analysis mode in which timing paths can propagate across latches, depending on their arrival time with respect to the latch transparency window.

When set to `true`, the arrival and required times will be propagated across a latch if it is in transparent mode. The reporting will show the entire flow through the path, rather than individual segments. This allows the overall path slack to be seen through upstream borrowing latch stages. The timing analysis, timing optimization, and reporting will consider the sequence of borrowing latch stages as a single path. As a result, the timing optimization reaches the target slack by viewing the entire path.

This feature provides better visibility into the overall context of latch path timing issues - for both the stage and path-level slacks. Additionally, this improves usability for debugging and manual ECOs, and shows better quality of result for Tempus ECO timing closure.

timing_enable_latency_through_clock_gating

```
timing_enable_latency_through_clock_gating {true | false}
```

Default: `true`

Data_type: Boolean

Allows propagation of latency phases across clock gating elements.

When set to `true`, the software allows propagation of clock latency phase across clock gating elements.

When set to `false`, the software suppresses propagation of clock latency across clock gating elements from the enable pin. The enable pin of the clock gating element should not be in the clock network for any other clocks. This is not recommended because the enable path (through the 'inferred' clock gates, such as, AND, NAND, OR) is either for clock latency or data paths. If the gated clock check is met, then the enable should not pass through to the output.

In case of ICGs, there is no arc from the EN to the CLKOUT on ICGs that have an incoming enable signal latched. In case of ICG/AND, using the enable path may not be correct for determining latency paths. If no latency paths are found, then the generated clocks definition will need to be re-checked.

timing_enable_mmmc_loop_breaking

```
timing_enable_mmmc_loop_breaking {true | false}
```

Default: `true`

Data_type: Boolean

When set to `true`, loop breaking is handled independently per analysis view, according to the following rules in order of precedence:

- If an analysis view does not contain a combinational loop, it does not inherit any loop breaking done for other views.
- If a loop exists, and the user has supplied specific case analysis and `set_disable_timing` constraints to break the loop at a certain place, the specified loop break is honored.

- If a loop is still detected, the software automatically selects a point to break the loop, and applies the loop break consistently to any views where the loop exists.

When set to `false`, if a combinational loop is detected in any analysis view, the timing system breaks the loop by disabling an arc along the looping path. These combinational loop breaks are reflected in all analysis views, even if those views by themselves do not contain these looping paths.

Note: Cadence recommends that all combinational loops detected by the software be evaluated, to ensure that the loop is broken at the most ideal location. Automatic loop breaking decisions are made based on how the design is traversed. Small changes in the design can influence where loop breaking decisions are made.

To reduce risk and the need to re-evaluate loop-breaking decisions, it is highly recommended that once combinational loop breaks have been evaluated, the user add constraints to ensure that the loop is always broken in that location.

timing_enable_model_constraint_warnings

```
timing_enable_model_constraint_warnings integer
```

Default: `0`

Data_type: Integer

Allows the software to report ignored warnings generated from boundary modeled blocks while processing constraints. These warnings can be reviewed later for debugging purposes.

When set to `0`, the software suppresses the constraint warnings.

timing_enable_multi_drive_net_reduction_with_assertions

```
timing_enable_multi_drive_net_reduction_with_assertions {none | delay | all}
```

Default: `none`

Data_type: Enum

Allows reduction of multi-drive networks when constraint assertions are present on the network elements:

The following options are available:

- `none`: Reduction is not performed on portions of the network where constraints have been asserted.
- `delay`: Reduction of the multi-drive network will still be applied if only `set_annotated_delay` and `set_annotated_transition` constraints have been applied. Other constraints present on the multi-drive network can still prevent full reduction.
- `all`: The software ignores the presence of any assertions on the multi-drive network and forces reduction to occur.

timing_enable_multi_frequency_latch_analysis

```
timing_enable_multi_frequency_latch_analysis {true | false}
```

Default: `false`

Data_type: Boolean

Enables multi-frequency latch timing analysis of the latch time borrowing for when a data signal coming to a latch is controlled by a clock with a frequency different to a clock of the latch enabling signal.

timing_enable_multicycle_data_check_compatibility

```
timing_enable_multicycle_data_check_compatibility {true | false}
```

Default: `false`

Data_type: Boolean

Enables checking for SDC compatible data checks when `set_multicycle_path -start` parameter is specified.

timing_enable_path_delay_to_unconstrained_endpoints_compatibility

```
timing_enable_path_delay_to_unconstrained_endpoints_compatibility {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software does not constrain paths with path delay - if the path ends at an unconnected input pin of a combination cell.

timing_enable_pessimistic_cppr_for_reconvergent_clock_paths

```
timing_enable_pessimistic_cppr_for_reconvergent_clock_paths{true | false}
```

Default: `false`

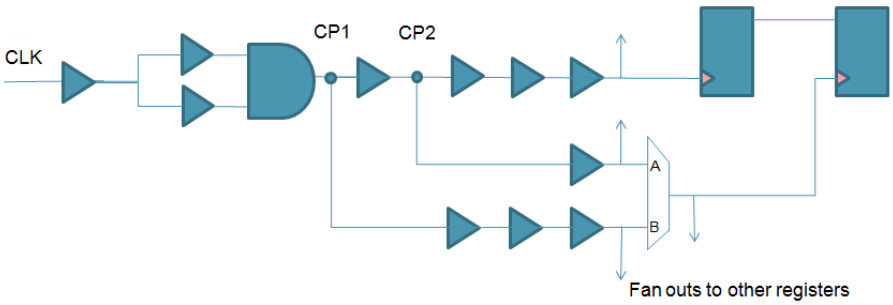
Data_type: Boolean

When set to `true`, the software will consider all combinations of launch and capture clock path pairs to identify all possible common pins, and then selects the pin with minimum CPPR adjustment over all such common pins as the common pin.

When set to `false`, the default, the software will choose the CPPR common pin based on the commonality of the

worst launch and worst capture path only.

Consider the following example,



In the above diagram, both capture paths are considered along with the single launch path, and the common pin is chosen as the pin with the least CPPR adjustment. The pin CP1 is chosen as a common pin and CPPR is reported up to this pin. The same common pin, CP1, will be used for CPPR adjustment whether the capture clock path goes through the mux A pin or the mux B pin. When this attribute is set to false, CPPR common pin is chosen by the commonality in the worst-case launch and worst-case capture paths. so CP2 is chosen as the common pin and CPPR is reported up to this pin.

timing_enable_power_ground_constants

```
timing_enable_power_ground_constants {true | false}
```

Default: `false`

Data_type: Boolean

Determines if case analysis is inferred from power and ground rail connections/nets, e.g., VDD and GND.

When set to `true`, the software recognizes the constants that come from power and ground rail connections.

When set to `false`, the software ignores these constants. This provides the most intuitive behavior with logical style netlist (and other tools that do not read physical data).

timing_enable_preset_clear_arcs

```
timing_enable_preset_clear_arcs {true | false}
```

Default: `false`

Data_type: Boolean

Determines whether the timing arcs are created to model the transition to active state (assertion) of the preset or clear pin, and the subsequent transition of the output to a controlled state.

When set to `true`, the software builds the arcs and the timer automatically analyzes paths through them (enable

clock propagation through preset clear pins).

When set to `false`, the software does not build these arcs, because they are associated with analyzing the reset mode timing paths. Filtering them prevents unwanted interaction with functional mode timing analysis.

Note: The `timing_enable_preset_clear_arcs` attribute can also impact delay calculation results of a design. When an arc is disabled, the software will calculate delay using the linear table lookup. When an arc is not disabled, the delay calculations are made based on the quadratic table lookup.

Note: This attribute should be set to `true` when reset checks are performed on a dedicated reset SDC.

timing_enable_pulse_latch

```
timing_enable_pulse_latch {true | false}
```

Default: `true`

Data_type: Boolean

Enables pulse-latch analysis.

When set to `true`, the pulse-latch analysis mode provides several additional pulse-clock specific constraint specifications and reporting capabilities.

When in pulse-latch mode, you can use the following pulse clock constraints:

- `set_pulse_clock_max_width`
- `set_pulse_clock_min_width`
- `set_pulse_clock_max_transition`
- `set_pulse_clock_min_transition`

Additionally, in this mode the `report_constraint` command supports the following pulse clock check types:

- `pulse_clock_max_width`
- `pulse_clock_min_width`
- `pulse_clock_max_transition`
- `pulse_clock_min_transition`

timing_enable_si_cppr

```
timing_enable_si_cppr {true | false}
```

Default: `true`

Data_type: Boolean

Enables more accurate CPPR analysis when incremental delays are present.

When set to `true`, the incremental delay is not included in the CPPR calculation during setup analysis, but it is included in the CPPR calculation during hold analysis.

When set to `false`, the incremental delay is included in both setup and hold CPPR calculations.

timing_enable_simultaneous_setup_hold_mode

```
timing_enable_simultaneous_setup_hold_mode {true | false}
```

Default: `true`

Data_type: Boolean

Controls whether setup and hold checks are analyzed separately, or together on the same timing graph.

When set to `false`, switches the system from analyzing setup slacks to analyzing hold slacks.

When set to `true`, setup and hold slacks are analyzed concurrently. As a consequence, both `-early` and `-late` queries for reporting commands such as `report_timing` can be satisfied immediately, without additional calculation. Simultaneous mode is useful for immediately understanding the consequences to both setup and hold slacks from what-if operations and manual ECOs to the design.

Note: Simultaneous setup and hold mode can be used only for timing analysis; it cannot be used for any physical implementation steps.

Note: This attribute must always be true while running SI analysis.

timing_enable_spatial_derate_mode

```
timing_enable_spatial_derate_mode {true | false}
```

Default: `true`

Data_type: Boolean

Enables the spatial derate mode in SOCV analysis.

timing_enable_timing_window_pessimism_removal

```
timing_enable_timing_window_pessimism_removal{true | false}
```

Default: `false`

Data_type: Boolean

Removes pessimism in SI impact calculation. As a result of elongated arrival windows of attackers and victims,

there may be pessimism in SI impact calculation due to a difference in the minimum-maximum arrivals of common clock paths shared between attackers and victims. This pessimism can be removed by adjusting attacker/victim arrival window with the difference in minimum-maximum delay of common clock path between victim and attacker.

When set to `true` (when CPPR is enabled), the software adjusts the timing windows of attackers/victims when the victim is located either on a data path or a clock path during path-based analysis.

For adjustment on victim nets lying on clock paths, full path path-based analysis can be enabled by setting the `timing_disable_retime_clock_path_slew_propagation` attribute to `false`.

timing_enable_tristate_clock_gating

```
timing_enable_tristate_clock_gating {true | false}
```

Default: `false`

Data_type: Boolean

The timing system infers gated-clock checks in the design where a data and clock path intersect at a combinational device. When this attribute is set to `true`, inferred gated-clock checks are added when clock and data signals converge through the tristate enable and data input of tristate buffers. By default (a setting of `false`), gated-clock checks are not added for these situations.

timing_enable_uncertainty_for_clock_checks

```
timing_enable_uncertainty_for_clock_checks {true | all | false | none | pulse_width | period |
skew}
```

Default: `true`

Data_type: Boolean

Considers clock uncertainty when performing clock checks.

When set to `true`, the `set_clock_uncertainty` command supports clock uncertainty for period, pulse width, skew, and separation checks.

Note: This attribute overrides the `timing_enable_uncertainty_for_pulse_width_checks` setting.

timing_enable_uncertainty_for_pulse_width_checks

```
timing_enable_uncertainty_for_pulse_width_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, considers clock uncertainty when performing minimum pulse width checks. Clock uncertainty is considered during checks of both user-defined minimum pulse widths (using the `.lib`).

Note: Specifying a value of `true` affects the following commands that take minimum pulse width checks into account:

- `report_timing -check_type pulse_width`
- `report_timing -check_clocks`
- `report_min_pulse_width`
- `report_analysis_coverage`

timing_enable_vtskew_derate_mode

```
timing_enable_vtskew_derate_mode {true | false}
```

Default : `false`

Data_type: Boolean

Enables voltage threshold (VT) skew analysis.

When set to `true`, the software analyzes the effects of VT skew in a multi-VT environment.

Due to variations in the manufacturing process, the voltage threshold (Vth) of a given Vth class (e.g., SVT, LVT, ULVT) can show some variations that might result in cells of that class operating either faster or slower than the nominal value. The Vth skew analysis automates the process of selecting which Vth permutations need to be run, which ensures a robust analysis and optimizes the software to run faster and efficiently.

timing_enable_wire_variation

```
timing_enable_wire_variation {true | false}
```

Default: `false`

Data_type: Boolean

Enables wire variation analysis in path-based analysis mode.

timing_enable_zero_delay_analysis_mode

```
timing_enable_zero_delay_analysis_mode
{true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software will use zero delays during timing analysis.

timing_extract_model_aocv_mode

```
timing_extract_model_aocv_mode {graph_based | path_based | none}
```

Default: none

Data_type: Enum

Allows you to specify AOCV derating mode.

The valid values are described below:

- `graph_based`: Specifies that the delays in the timing model are derated using the graph-based stage counts.
- `path_based`: Specifies that the delays in the timing model are derated using the total path stage count of worst path between those pin pairs.
- `none`: Specifies that the delays in the timing model are under-rated base delays.

Note: Before setting this attribute to `graph_based or path_based` mode, it is mandatory that the AOCV libraries are read in and AOCV analysis is enabled.

timing_extract_model_case_analysis_in_library

```
timing_extract_model_case_analysis_in_library {true | false}
```

Default: `true`

Data_type: Boolean

Enables the software to model the constants as a function in the extracted timing model.

When set to `true`, the software models constants as functions in the library if these propagate to the output port.

When set to `false`, instead of modeling constants as functions inside the library, these are written as `set_case_analysis` in the constraints file (*.asrt).

timing_extract_model_check_arcs_as_lvf

```
timing_extract_model_check_arcs_as_lvf {true | false}
```

Default: `true`

Data_type: Boolean

Enables check arcs to be modeled as LVF arcs. By default, the check arcs will be written as nSigma numbers.

The software allows you to write mean+/-nSigma values for constraint arcs, while writing separate mean and sigma values for delay and transition data. This can be enabled by setting `timing_extract_model_check_arc_as_lvf` attribute to `false` and `timing_extract_model_write_lvf` attribute to `true`.

timing_extract_model_consider_design_level_drv

```
timing_extract_model_consider_design_level_drv{true | false}
```

Default : `true`

Data_type: Boolean

Considers `set_max_capacitance` and `set_max_transition` DRV constraints, that have been applied on the current design during model extraction.
When set to `true`, these constraints applied at the design level will be considered during model extraction. However, the maximum capacitance and maximum transition from various DRV constraints applied/inferred on the ports is considered.

When set to `false`, the software extracts the worse-case DRV values and the constraints applied on the current design will be ignored.

timing_extract_model_disable_cycle_adjustment

```
timing_extract_model_disable_cycle_adjustment {true | false}
```

Default: `false`

Data_type: Boolean

Controls cycle adjustment for multi-cycle paths.

When set to `false`, if the multi-cycle paths specified in the SDC at the block-level cannot be pushed out to the ports (since there are other paths going through these paths), then the Tempus software adjusts the delay/check numbers
in the ETM to take into account the cycle adjustment.

When set to `true`, the cycle adjustment for the multi-cycle paths in ETM arc delay are disabled.

timing_extract_model_enable_combinational_arc_to_clock_source_on_output_ports

```
timing_extract_model_enable_combinational_arc_to_clock_source_on_output_ports {true | false}
```

Default : `false`

Data_type: Boolean

When set to `true`, the software models the combinational arc ending on an end point with clock asserted on it.

timing_extract_model_enable_output_pin_attributes

```
timing_extract_model_enable_output_pin_attributes String
```

Default: `none`

Data_type: String

Specifies the attributes to be enabled on the output pin.

timing_extract_model_enable_unconstrained_port_modelling

```
timing_extract_model_enable_unconstrained_port_modelling

{true | false}
```

Default: `false`

Data_type: Boolean

Enables extraction of unconstrained paths to the output ports.

When set to `false`, for unconstrained output ports, which have no inbound timing, there will be no arcs modelled for the paths.

When set to `true`, the software will extract dummy arcs inbound to unconstrained output ports that will show the following characteristics:

- Early delay/slew of last stage connected to the output ports will be modelled.
- Related pins will be specified as unique unconstrained internal pins.
- Extracted slew values will be modelled as unidimensional *_transition tables that are dependent on the output capacitance and real propagated slews inside a block.
- Extracted unidimensional delays will be displayed in the cell_* tables.

For more information, refer to "Timing Model Generation" chapter in the *Tempus Stylus User Guide.*

timing_extract_model_exhaustive_validation_dir

```
timing_extract_model_exhaustive_validation_dir {true | false}
```

Default: `{}`

Data_type: String

Controls the location of files to be stored in the ETM Extremity Validation mode.

When set to `false`, additional files generated during ETM Extremity Validation (but not generated during normal validation) will be stored in the current working directory.

When set to `true`, the location to place these files can be user-specified.

timing_extract_model_exhaustive_validation_mode

```
timing_extract_model_exhaustive_validation_mode{true | false}
```

Default: `false`

Data_type: Boolean

Enables extracted timing model (ETM) validation at the minimum and maximum indices of the slew load indices.

When set to `false`, the ETM validation is done only for the input slews/output-loads that have been asserted on the design during timing model extraction.

When set to `true`, the software uses the “Extremity-Validation” (EV-ETM) methodology that allows ETM to model the timing arcs in the design for a range of discrete input slews/output-loads, thus providing a more comprehensive validation.

timing_extract_model_gating_as_nochange_arc

```
timing_extract_model_gating_as_nochange_arc {true | false}
```

Default: `true`

Data_type: Boolean

Controls the conversion of clock gating checks to nochange arcs.

When set to `false`, the software disables conversion of clock gating checks to nochange arcs in the extracted timing model.

timing_extract_model_half_cycle_non_interface_paths

```
timing_extract_model_half_cycle_non_interface_paths {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software models half cycle non-interface paths as MPW (minimum pulse width) checks in ETM.

timing_extract_model_ideal_clock_latency_arc

```
timing_extract_model_ideal_clock_latency_arc{true | false}
```

Default: `true`

Data_type: Boolean

When set to `true`, the latency arcs for generated clocks that are defined ideal are extracted with reference as master clock, during ETM extraction.

When set to `false`, arcs are not extracted for generated clocks that are defined ideal during ETM extractions.

timing_extract_model_include_applied_load_in_characterization_range

```
timing_extract_model_include_applied_load_in_characterization_range {true | false}
```

Default: `true`

Data_type: Boolean

Specifies that the actual load visible to the timer that will be included in the load characterization range for path end points.

timing_extract_model_include_applied_slew_in_characterization_range

```
timing_extract_model_include_applied_slew_in_characterization_range {true | false}
```

Default: `true`

Data_type: Boolean

Specifies that the actual timer slew must be included in the characterization range timing model.

timing_extract_model_non_borrowing_latch_path_as_setup

```
timing_extract_model_non_borrowing_latch_path_as_setup {true | false}
```

Default: `false`

Data_type: Boolean

Models the non-borrowing interface latch paths as setup checks.

timing_extract_model_slew_propagation_mode

```
timing_extract_model_slew_propagation_mode {worst_slew | path_based_slew}
```

Default: `worst_slew`

Data_type: Boolean

Specifies the type of slew propagation to use for generating extracted timing model (ETM).

- `worst_slew`: Propagates the worst slew of all the incoming arcs at a converging point for extracting the arcs.
- `path_based_slew`: Propagates the actual slew for the path elements for extracting the arcs.

timing_extract_model_synchronous_clocks_as_skew_checks

```
timing_extract_model_synchronous_clocks_as_skew_checks {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software enables modelling of skew checks between synchronous clocks on the design boundary.

timing_extract_model_write_clock_checks_as_arc

```
timing_extract_model_write_clock_checks_as_arc{true | false}
```

Default : `false`

Data_type: Boolean

When set to `true`, the clock checks (min_period and min_pulse_width) are modelled as arcs in the extracted timing models (ETM). Clock check arcs will be modelled as a single dimensional table with constrained and reference pin as clock pin itself.

When set to `false`, clock checks are modeled as scalar numbers in the extracted ETM.

timing_extract_model_write_clock_checks_as_scalar_tables

```
timing_extract_model_write_clock_checks_as_scalar_tables {true | false}
```

Default: `false`

Data_type: Boolean

Allows the extracted timing model to write the minimum pulse width or minimum period checks as scalar arcs.

timing_extract_model_write_function_for_combinational_arcs

```
timing_extract_model_write_function_for_combinational_arcs {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software writes the function attribute on ports with combinational arcs.

timing_extract_model_write_min_max_clock_tree_path

```
timing_extract_model_write_min_max_clock_tree_path {true | false}
```

Default: `true`

Data_type: Boolean

Writes the minimum/maximum worst latency for each clock source.

timing_extract_model_write_lvf

```
timing_extract_model_write_lvf {true | false}
```

Default: `false`

Data_type: Boolean

Generates extracted timing model (ETM) containing Liberty Variation Format (LVF) model.

When set to `true`, this attribute enables writing separate mean and variation tables for all the delay/constraint and transition tables.

When set to `false`, the variation data will be merged with the mean values. The mean +/- nsigma values of delay/constraint/transition data will be written in the ETM.

timing_full_context_flow

```
timing_full_context_flow integer
```

Default: `0`

Data_type: Integer

Enables the full chip timing context mode. This variable captures the estimated slews and timing windows as part of the timing context in the boundary model.

When set to 1, the full chip timing analysis (STA or DSTA) run saves the timing context that is used by the block scope timing later in the flow.

timing_generate_normalized_driver_waveform

```
timing_generate_normalized_driver_waveform {0 | 1}
```

Default: `true`

Data_type: Boolean

Generates the analytical normalized driver waveform for timing library if the waveform is not specified in the library. The waveform is generated by weighed mean of linear and exponential waveform.

timing_generated_clocks_inherit_ideal_latency

```
timing_generated_clocks_inherit_ideal_latency {false | true | actual_partial_latency_path |
ideal_partial_latency_path}
```

Default: `false`

Data_type: Boolean

When the `set_clock_latency` command is used to define network latency for ideal clock mode at intermediate pins of a clock network, Tempus considers the latency. The `timing_generated_clocks_inherit_ideal_latency` global variable allows including the closest applied network latency for ideal inheritance also. You can use the following options:

- `false`: Uses only the source latency of generated clocks, if it exists.
- `true`: Allows the generated clocks to inherit the ideal network latency of the parent clock.
- `actual_partial_latency_path`: Uses the network latency of the master clock asserted closest to the target pin and propagated delays for the path from this point to the assertion point of the generated clock.
- `ideal_partial_latency_path`: Uses the network latency of the master clock asserted closest to the target pin and ideal delays for the path from this point to the assertion point of the generated clock.

Note: If the `set_clock_latency -source` parameter is used to specify latency of a generated clock, the software will completely override any inheritance.

timing_generated_clocks_allow_nested_assertions

```
timing_generated_clocks_allow_nested_assertions {true | false}
```

Default: `false`

Data_type: Boolean

Controls clock latency handling for nested generated clocks that are created on the same timing pin as the master clock.

When set to `true`, the software handles generation of multiple nested generated clocks that are asserted on the same pin, such that the child clock inherits the clock latency value of its parent clock.

When set to `false`, the nested generated clocks asserted on the same pin do not inherit the latency values and are created with 0ns master clock latency. Any full clock timing path reported using these clocks will show the clock path up to the target pin of the generated clock.

timing_get_of_objects_hier_compatibility

```
timing_get_of_objects_hier_compatibility {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the `get_pins -of_objects` parameter will return leaf pins connected to the specified objects.

When set to `false`, the `get_pins -of_objects` parameter will return hierarchical pins and leaf pins connected to the specified objects. In order to return leaf pins only when the attribute is set to `false`, the `-leaf` option must be specified along with the `get_pins -of_objects` parameter.

When set to `true`, the `get_cells -objects` parameter returns cells connected to the net at the same level of hierarchy as the net itself. Hierarchical ports will be returned if the net is connected to a hierarchical module.

timing_hierarchical_context_continue_on_top_block_clock_mismatch

```
timing_hierarchical_context_continue_on_top_block_clock_mismatch {true | false}
```

Default: `false`

Data_type: Boolean

Allows loading of the timing context model with unmapped clocks.

When set to `true`, the software allows the script to process even if there are any mismatches between the top and block-level clocks. The mismatches are reported in the *timing_context_clock_mapping_data.rpt* file, which shows data that allows you to determine if corrections or additions are required to the clock mapping file.

When set to `false`, the software will show an error if any of the top-level clocks cannot be matched with a block-level clock.

timing_hier_obj_name_compatibility

```
timing_hier_obj_name_compatibility {true | false}
```

Default: `true`

Data_type: Boolean

Controls how hierarchical delimiters are interpreted in the search pattern, when the `-hier` parameter is used with the `get_*`collection commands.

When set to `true`, the software interprets the hierarchical delimiter as part of the search pattern itself, and tries to match names that include the delimiter. For example, if you specify:

```
get_cells -hier L1/R*
```

The software searches at every level of the hierarchy for instances with the name `L1/R*`, completely ignoring the hierarchy separator:

When set to `false`, hierarchical delimiters are interpreted as hierarchy separators. For example, if you specify:

```
get_cells -hier L1/R*
```

The software searched for instances with names matching `R*` in the hierarchy level below `L1`:

```
TL2/L1/RX1 TL2/L1/RX2
```

timing_ignore_clock_based_path_delay_on_data_checks

```
timing_ignore_clock_based_path_delay_on_data_checks {true | false}
```

Default: `false`

Data_type: Boolean

Disables clock-based path delay application on data checks.

When set to `true`, the Tempus software enables the data checks to be reported by disabling the applied path delay. The "Data Check Setup" details are displayed in the timing report output.

When set to `false`, the timing reports display the "Path Delay" details. By default, this attribute is set to `false`.

timing_ignore_lumped_rc_assertions

```
timing_ignore_lumped_rc_assertions {true | false}
```

Default: `false`

Data_type: Boolean

When set to `false`, the `set_load` and `set_resistance` values override the actual extracted representation. A warning message is issued stating that the specified constraints on internal nets will override extracted parasitics on those nets.

When set to `true`, instructs the software to ignore `set_load` and `set_resistance` assertions on internal nets in the design. Parasitic data for internal nets normally should be provided by Tempus RC extraction, or external SPEF annotation.

Using `set_load` to specify I/O boundary loading on ports is supported by either setting of this attribute.

timing_io_use_clock_network_latency

```
timing_io_use_clock_network_latency {always | ideal}
```

Default: `ideal`

Data_type: Enum

Controls whether network latency of a reference clock is added or not to the data arrival time on the port. When set to `ideal`, the `set_input_delay` and the `set_output_delay` constraints will not add the network latency of the reference clock to the data arrival time on the port if the clock is in `propagated` mode.

When this attribute is set to `always`, network latency is added to the data arrival time on the port regardless of the clock propagation mode.

timing_ipd_allow_paths_for_asynchronous_clocks

```
timing_ipd_allow_paths_for_asynchronous_clocks {true | false}
```

Default: `true`

Data_type: Boolean

When set to `true`, the software will ensure that the paths between the asynchronous clocks are allowed to be treated as IPD (inter-power domain) paths.

For more information, refer to the "Inter Power Domain (IPD) Analysis" chapter in the *Tempus Stylus User Guide*.

timing_library_build_async_deassert_arc

```
timing_library_build_async_deassert_arc {true | false}
```

Default: `true`

Data_type: Boolean

Controls whether input to output arcs from the pre-set or clear pins transitioning to an active state are included, when the timing system is
initialized.

timing_library_convert_async_setuphold_to_recrem

```
timing_library_convert_async_setuphold_to_recrem {true | false}
```

Default: 1

Data_type: Integer

Controls whether single-edged setup and hold checks on asynchronous pins in the Liberty library are inferred as recovery and removal checks.
This attribute provides support for older modeling styles in legacy libraries.

timing_library_generated_clock_use_group_name

```
timing_library_generated_clock_use_group_name {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software uses the generated_clock group name when creating a generated clock from a library-generated clock group. When set to `false`, the software uses the pin name when creating the generated clock.

For example, assume the cell for instance `a/b` in the timing library contained the following generated clock group:

```
generated_clock(genclk2) {
clock_pin : clk1;
master_pin : "int/clk";
multiplied_by : 2;
duty_cycle : 50.0;
}
```

By default (`false`), the software creates a generated clock with the following name:

```
a/b/clk1
```

If the `timing_prefix_module_name_with_library_generated_clock` attribute is set to `true`, and you set this attribute to `true`, the software creates a generated clock with the following name:

```
a/b/genclk2
```

If the `timing_prefix_module_name_with_library_generated_clock` attribute is set to `false`, and you set this attribute to `true`, the software creates a generated clock with the following name:

```
genclk2
```

timing_library_hold_constraint_corner_sigma_multiplier

```
timing_library_hold_constraint_corner_sigma_multiplier double
```

Default: `0.0`

Data_type: Double

Applies user control multiplier to generate a constraint table using sigma values applied for hold arc.

timing_library_hold_sigma_multiplier

```
timing_library_hold_sigma_multiplier double
```

Default: 0.0

Data_type: Double

User control multiplier to generate constraint table applied for hold arc.

timing_library_ignore_combinational_arc_mismatch_tables

```
timing_library_ignore_combinational_arc_mismatch_tables

{true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software will give preference to timing_type over arc tables. This will change infer arcs based on the timing_type (liberty format) or timing tables settings. A warning message will be displayed when the preference (timing_type or arc tables) is changed.

timing_library_infer_async_pins_from_timing_arcs

```
timing_library_infer_async_pins_from_timing_arcs {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, a pin is marked clear if it has timing arcs with type “clear” to output pins. The sequential block is not required to be present in the cell for marking a pin as clear.

timing_library_infer_cap_range_from_ccs_receiver_model

```
timing_library_infer_cap_range_from_ccs_receiver_model {true | false}
```

Default: `false`

Data_type: Boolean

Infers the rise or fall of capacitance range values from CCS receiver capacitance model.

When set to `false`, the rise/fall CCS capacitance range values are not inferred. The NLDM pin capacitance range values defined in the library are honored.

timing_library_infer_cap_range_from_ecsm_receiver_model

```
timing_library_infer_cap_range_from_ecsm_receiver_model {true | false}
```

Default: `false`

Data_type: Boolean

Infers the rise or fall of NLDM pin capacitance range from the ECSM capacitance range while calculating the total loading capacitance during switching power computation. If the attribute is set to `true` and the capacitance range is not defined in the library, the rise/fall capacitance range is inferred from the ECSM receiver capacitance models. The capacitance range values are derived either from the pin-level ECSM cap models or the timing-level ECSM cap models.

When this attribute is set to `false`, the rise/fall capacitance range values are not inferred, and the library values are honored.

timing_library_interpolate_drv_values

```
timing_library_interpolate_drv_values {true | false}
```

Default: `false`

Data_type: Boolean

Allows the software to use a range of trilib DRV values for performing delay calculations.

timing_library_infer_socv_from_aocv

```
timing_library_infer_socv_from_aocv {true | false}
```

Default: `false`

Data_type: Boolean

Controls the transformation of AOCV data.

This attribute infers sensitivity data from AOCV libraries, and must be set before loading any timing library data.

timing_library_read_ccs_noise_data

```
timing_library_read_ccs_noise_data{true | false}
```

Default: `true`

Data_type: Boolean

This attribute can be used when the CCS library contains noise models, and SI glitch analysis is being performed.

timing_library_read_without_ecsm

```
timing_library_read_without_ecsm {true | false}
```

Default: `false`

Data_type: Boolean

Disables loading ECSM data.

When set to `true`, the software allows you to change the mode of use (of ECSM timing model) if the timing library contains NLDM, ECSM, or CCS data in the same file.

Note: This attribute does not disable CCS data loading.

timing_library_read_without_power

```
timing_library_read_without_power {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, this attribute allows the software to read timing libraries without power information.
When set to `false`, the software reads power data also.

Note: The `get_db` command shows the default value as `false`. However, this global variable is set to `true` by default in Tempus.

timing_library_read_without_sensitivity

```
timing_library_read_without_sensitivity {true  | false}
```

Default: `false`

Data_type: Boolean

Disables loading of ECSM timing sensitivity data.

timing_library_scale_aocv_to_socv_to_n_sigma

```
timing_library_scale_aocv_to_socv_to_n_sigma <n_sigma value>
```

Default: 3.0

Data_type: Double

AOCV derates are expected to be derived based on 3-sigma variation.

timing_library_setup_constraint_corner_sigma_multiplier

```
timing_library_setup_constraint_corner_sigma_multiplier double
```

Default: `0.0`

Data_type: Double

Applies user control multiplier to generate a constraint table using sigma values applied for setup arc.

timing_library_setup_sigma_multiplier

```
timing_library_setup_sigma_multiplier double
```

Default: 0.0

Data_type: Double

Applies user control multiplier to generate constraint table using sigma values applied for setup arc.

timing_library_term_voltage_from_lib_pin

```
timing_library_term_voltage_from_lib_pin int
```

Default: 1

Data_type: Integer

Enables the use of library pin voltage for reporting the term voltage.

timing_library_use_two_piece_receiver_cap

```
timing_library_use_two_piece_receiver_cap {true | false}
```

Default: `false`

Data_type: Boolean

Enables the use of 2-piece receiver capacitance constructs, if a library contains both the N-piece and 2-piece receiver capacitance construct.

When set to `true`, if a library has 2-piece and multi-piece pin capacitance, then the software gives priority to 2-piece models.

When set to `false`, the software reads the N-piece receiver capacitance constructs.

timing_library_zero_negative_timing_check_arcs

```
timing_library_zero_negative_timing_check_arcs {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software checks for negative values in the timing arcs.

timing_multi_frequency_clock_rounding_factor

`timing_multi_frequency_clock_rounding_factor` *double*

Default: `1e-05`

Data_type: Double

Rounds off irrational numbers so that they can be read appropriately.

timing_normalized_driver_waveform_clip_linear_part

`timing_normalized_driver_waveform_clip_linear_part {0 | 1}`

Default: `false`

Data_type: Boolean

When set to 1, the linear ramp component in the normalized driver waveform is saturated to the maximum value, while generating the normalized driver waveform.

timing_normalized_driver_waveform_weight_factor

`timing_normalized_driver_waveform_weight_factor` *double*

Default: 0.5

Data_type: Double

Defines the value of the weight of linear components in the algorithm for generation of normalized driver waveform.

When set to `1`, the waveform will be purely linear. When set to `0`, the waveform will be purely exponential. You can specify values within the range of 0 to 1.

timing_nsigma_multiplier

`timing_nsigma_multiplier` *double*

Default: 3.0

Data_type: Double

Controls the sigma multiplier to be used in SOCV mode.

timing_null_collection_return_compatibility

```
timing_null_collection_return_compatibility {true | false}
```

Default: `false`

Data_type: Boolean

Allows you to migrate previous release scripts to the new use model.

When the attribute is set to `true`, the software returns "NA" values that are consistent with the previous release. The nil return values are returned as an empty collection. This allows the return value to be used directly by collection iterators, such as `foreach_in_collection`.

The default value is `false`, which is consistent with the current release.

Note: This compatibility option may be removed from future releases. You should update your scripts to the new use model to avoid future problems.

timing_path_based_enable_bounding_box_for_io_paths

```
timing_path_based_enable_bounding_box_for_io_paths {true | false}
```

Default : `false`

Data_type: Boolean

Enables the bounding box mode for input-output paths.

timing_path_based_enable_closest_common_pin_driver_for_bounding_box

```
timing_path_based_enable_closest_common_pin_driver_for_bounding_box {true | false}
```

Default : `false`

Data_type: Boolean

Enables bounding box calculation using closest common pin driver from the clock mesh.

timing_path_based_epba_bounded_mode

```
timing_path_based_epba_bounded_mode {true | false}
```

Default: `false`

Data_type: Boolean

Enables the IPBA mode with exhaustive PBA analysis (run using the `report_timing -retime_mode exhaustive` command).

When set to `true`, the software ensures that all the paths to the violating endpoints are covered for PBA, and the slack is the worst PBA slack reported across all the paths to the endpoint. For any endpoint that is considered for IPBA analysis, the complete fanin cone (all the timing paths ending at that endpoint) is evaluated for any possible PBA violation.

timing_path_based_enable_report_launch_clock_path

```
timing_path_based_enable_report_launch_clock_path {true | false}
```

Default: `true`

Data_type: Boolean

When set to `false`, the software skips the reporting launch clock path in PBA path_type full report.

timing_path_based_enable_verbose_mode

```
timing_path_based_enable_verbose_mode value
```

Default: `1`

Data_type: Enum

Controls printing of verbose messages during and at the end of path-based analysis (PBA).

You can specify one of the three values - 0, 1, and 2. These are described below:

- `0`: Verbose messages are not reported during exhaustive PBA analysis. For default PBA analysis, the path count will be reported.
- `1`: Reports the final message for PBA path count and max depth reached during exhaustive PBA. Progress messages are not reported.
- `2`: Reports multiple progress messages after regular intervals (in addition to the final message for PBA path count and max depth).

This attribute impacts messages when the `report_timing` and `report_constraint` commands are used in PBA mode.
For example,

- The following commands print messages during and after PBA analysis:

  ```
  set_db timing_path_based_enable_verbose_mode 2
  report_timing -retime path_slew_propagation -retime_mode exhaustive -max_paths 500000 -nworst 1
  ```

The following progress messages are displayed during PBA analysis:

```
INFO: Path Based Analysis (PBA) currently performed on '108455' paths
INFO: Path Based Analysis (PBA) currently performed on '217574' paths
INFO: Path Based Analysis (PBA) currently performed on '327511' paths
INFO: Path Based Analysis (PBA) currently performed on '437472' paths
....
```

The following messages are displayed at the end of PBA analysis mentioning the PBA path count and max depth reached during exhaustive PBA:

```
INFO: Path Based Analysis (PBA) performed on total '2660526' paths
INFO: Max depth 9460 reached for end point 'top/hier1/hier2/flop/d' during Path Based Analysis
```

timing_path_based_enable_exhaustive_depth_bounded_by_gba

```
timing_path_based_enable_exhaustive_depth_bounded_by_gba
{true | false}
```

Default: `true`

Data_type: Boolean

Enables the timing report to bound PBA results by GBA slacks of the respective leftover nworst paths, whenever exhaustive PBA depth for an endpoint is exhausted.

When set to `true`, during exhaustive PBA, if you specify exhaustive depth as N, and an endpoint has nworst paths greater than N, then the timing report will show the worst path corresponding to worst slack out of worst PBA slack (out of first N paths) and GBA slack (for the (N+1)th path). Here, the variable N refers to exhaustive PBA depth and can be set using the `timing_pba_exhaustive_path_nworst_limit` attribute.

When set to `false`, during exhaustive PBA, if you specify exhaustive depth as N, and an endpoint has nworst paths greater than N, then the software will ignore the paths with depth greater than N, which may result in reporting an optimistic PBA slack for the endpoint reaching the depth.

timing_path_based_exhaustive_enable_design_coverage

```
timing_path_based_exhaustive_enable_design_coverage
{true | false}
```

Default: `false`

Data_type: Boolean

Enables exhaustive path-based analysis (PBA) to automatically pick up and analyze all the violating endpoints in the design, irrespective of the specified maximum paths.

When set to `true`, exhaustive PBA analysis automatically picks up and analyzes the violating design endpoints instead of N endpoints (specified with the `report_timing -max_paths` parameter). After analyzing the violating endpoints, the software reports the top N worst endpoints - depending on the specified value of N.

When set to `false`, exhaustive PBA analysis restricts analysis (using the `-max_paths` parameter) to the specified number of N endpoints.

This attribute is valid for both EPBA as well as infinite depth path-based analysis (IPBA).

Note: When this attribute is enabled, there may be an increase in run time because the software will analyze all the violating endpoints and analysis will not be restricted to the specified N endpoints.

timing_path_based_exhaustive_max_paths_limit

`timing_path_based_exhaustive_max_paths_limit` *integer*

Default: `2000000`

Data_type: Integer

Specifies the maximum number of paths that can be internally analyzed during exhaustive path-based analysis (PBA).

If the number of candidate paths during exhaustive PBA is greater than the specified max paths limit, then the software will skip path-based analysis of these paths beyond this limit. The paths exceeding the specified max paths limit will then be included as GBA in the final report output so that real violations are not missed.

timing_path_based_ipba_endpoints_max_limit

timing_path_based_ipba_endpoints_max_limit *integer*

Default : `100000`

Data_type: Integer

Specifies the maximum number of worst critical GBA endpoints, which will be picked for IPBA analysis. The rest of the GBA endpoints will report GBA slack.

This attribute should be set to a number higher than the total number of GBA violating endpoints, in order to cover all the violating endpoints for IPBA.

To ensure that all the GBA violations are considered for IPBA analysis, the `report_timing -max_paths` parameter should be set equal to or greater than the total number of GBA violating endpoints.

timing_pba_exhaustive_path_nworst_limit

`timing_pba_exhaustive_path_nworst_limit <value>`

Default: `10000`

Data_type: Integer

Specifies the maximum number of paths per endpoint (nworst) that will be re-timed in exhaustive path-based analysis (PBA).

In exhaustive PBA mode, the value specified with the `report_timing -nworst` parameter is used only during the final reporting. The value specified with the `timing_pba_exhaustive_path_nworst_limit` attribute (and not the `report_timing -nworst` value) is used to specify the maximum number of paths to be re-timed per endpoint.

If the limit specified by `timing_pba_exhaustive_path_nworst_limit` is reached and `timing_path_based_enable_exhaustive_depth_bounded_by_gba` attribute is set to:

- `true`: The worst of the current PBA slack is used and N+1th GBA path is reported.
- `false`: The current PBA slack is reported - which may be optimistic.

timing_path_based_report_analysis_summary_max_paths_limit

`timing_path_based_report_analysis_summary_max_paths_limit` *integer*

Default: `2000000`

Data_type: Integer

Specifies the limit on the maximum paths that can be reported in PBA mode with the `report_analysis_summary` command.

timing_pll_clock_use_driving_cell

`timing_pll_clock_use_driving_cell {true | false}`

Default : `false`

Data_type: Boolean

Uses delay calculation of all the enabled arcs to the PLL clock output to compute the transition.

When set to `true`, the delay calculation of all enabled arcs to the PLL clock output will be used to compute the transition. If no driving arcs are available in the model or if they are disabled, the same idealized transition will be used.

When set to `false`, the software considers the PLL clock output to be electrically isolated from the input-side reference clock, and creates the output clock with an idealized transition. You can use the `set_annotated_transition` command to customize the output clock transition.

timing_prefix_module_name_with_library_generated_clock

```
timing_prefix_module_name_with_library_generated_clock {true | false}
```

Default: `true`

Data_type: Boolean

When set to `true`, the software appends the instance name to the clock pin name when creating a generated clock. When set to `false`, the software uses only the clock pin name when creating a generated clock.

For example, assume the cell for instance `a/b` in the timing library contained the following generated clock group:

```
generated_clock(genclk2) {
clock_pin : clk1;
master_pin : "int/clk";
multiplied_by : 2;
duty_cycle : 50.0;
}
```

By default (`true`), the software creates a generated clock with the following name:

```
a/b/int/clk
```

If you set the attribute to `false`, the software creates a generated clock with the following name:

```
int/clk
```

If you set this attribute to `false`, and the `timing_library_generated_clock_use_group_name` attribute is set to `true`, the software creates a generated clock with the following name:

```
genclk2
```

timing_propagate_latch_data_uncertainty

```
timing_propagate_latch_data_uncertainty {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, uses the clock phase associated with a flush latch's data pin as the "from" clock phase for downstream uncertainty timing calculations

When set to `false`, the downstream uncertainty calculations are based on the clock phase of the flush latch's clock signal.

timing_property_arrival_window_enable_tcl_dict_format

```
timing_property_arrival_window_enable_tcl_dict_format {true | false}
```

Default : `false`

Data_type: Boolean

Allows the software to print the new format of the `arrival_window` property (see `get_property` command).

timing_property_arrival_clocks_consider_clock_source_data_phase

```
timing_property_arrival_clocks_consider_clock_source_data_phase {true | false}
```

Default: `true`

Data_type: Boolean

Considers the clock source data phases.

The `get_property` and `report_property` commands support the `arrival_clocks` collection property on pin objects. The `arrival_clocks` property considers data phases, clock source data phases, and latency phases.

timing_property_arrival_clocks_consider_data_phase

```
timing_property_arrival_clocks_consider_data_phase {true | false}
```

Default: `true`

Data_type: Boolean

Returns the pure data phase.

The `get_property` and `report_property` commands support the `arrival_clocks` collection property on pin objects. The `arrival_clocks` property considers data phases, clock source data phases, and latency phases.

timing_property_arrival_clocks_consider_latency_phase

```
timing_property_arrival_clocks_consider_latency_phase {true | false}
```

Default: `true`

Data_type: Boolean

Returns the latency phase.

The `get_property` and `report_property` commands support the `arrival_clocks` collection property on pin objects. The `arrival_clocks` property considers data phases, clock source data phases, and latency phases.

timing_property_return_null_collection_with_quiet

```
timing_property_return_null_collection_with_quiet {true | false}
```

Default: `false`

Data_type: Boolean

Allows you to query values of all the properties.

When set to `true`, the software enables returning values of properties of all the objects, when the `get_property -quiet` parameter is specified.

timing_rail_swing_checks_high_voltage_threshold

```
timing_rail_swing_checks_high_voltage_threshold double
```

Default: `0.95`

Data_type: Double

Specifies the voltage threshold values for high rail swing checks.

timing_rail_swing_checks_low_voltage_threshold

```
timing_rail_swing_checks_low_voltage_threshold double
```

Default: `0.05`

Data_type: Double

Specifies the voltage threshold values for low rail swing checks.

timing_rcdb_allow_mismatch_option

```
timing_rcdb_allow_mismatch_option {true | false}
```

Default: `true`

Data_type: Boolean

Disables errors while reading boundary models.

When set to `true`, the software disables errors when reading the boundary model RCBD for cases where the contents do not match the boundary model netlist.

timing_recompute_sdf_in_setuphold_mode

```
timing_recompute_sdf_in_setuphold_mode {true | false}
```

Default: `false`

Data_type: Boolean

Controls the recomputing of SDF delays when the software is in simultaneous setup and hold analysis mode.

When the software is not in simultaneous setup and hold analysis mode, only one corner of data is stored at a time; therefore additional delay calculation is required for `write_sdf` command. When the software is in simultaneous setup and hold analysis mode, both sets of data are in memory at the same time, and additional delay calculation is not necessary.

If set to `false`, `write_sdf` will write out the delays from memory, instead of calling delay calculation.

Note: This attribute must be used with `timing_enable_simultaneous_setup_hold_mode true`.

timing_reduce_multi_drive_net_arcs

```
timing_reduce_multi_drive_net_arcs {true | false}
```

Default: `true`

Data_type: Boolean

Controls whether multi-drive or mesh networks are reduced to simpler, single driver models.

When set to `true`, the nets that meet the drivers' receivers arc threshold (defined using `timing_reduce_multi_drive_net_arcs_threshold` attribute), are reduced. The net arcs are created only from one driver or from the drivers for which you have set the constraints.

If you specify constraints on the net driven by parallel buffers, it might result in increased run time and memory degradation. By specifying timing constraints through nets, the constraints are moved to the drivers. This leads to the creation of net arcs from all the drivers, thus disabling net arc reduction. In such cases, you can select one driver of the net and use that driver to specify the constraints.

When set to `false`, the number of net arcs created for timing analysis for nets driven by parallel buffers are not reduced. If a net is driven by `n` parallel buffers and is connected to `m` sink pins, then the total number of net arcs created is `n*m`. For example, if `n` is `100` and `m` is `20000`, the number of net arcs created for timing analysis will be two million. This may result in increased run time and memory usage.

timing_reduce_multi_drive_net_arcs_threshold

```
timing_reduce_multi_drive_net_arcs_threshold int
```

Default: `10000`

Data_type: Integer

Sets a threshold number that is used to trigger the reduction of timing arcs of nets driven by parallel buffers. The reduction takes place if the number of arcs is greater than the threshold value and if

the `timing_reduce_multi_drive_net_arcs` attribute is set to `true`.

timing_report_analysis_summary_csv_extended_new_format

`timing_report_analysis_summary_csv_extended_new_format {true | false}`

Default: `false`

Data_type: Boolean

Keeps the analysis summary CSV extended report in sync with the summary timing report. These reports are available when the respective `report_timing` parameters are specified:

- `-analysis_summary_csv_extended_file`
- `-analysis_summary_file`

When set to `true` (and when the `report_timing -analysis_summary_csv_extended_file` parameter is specified), the column name sequence will be from fromClk to toClk, that is, fromClk will be printed first, and then toClk. Also, the timing report will show "-", if toClk or fromClk is not present (and when `report_analysis_summary -expanded_summary_clocks` is set to `false`).

When set to `false`, the missing toClk or fromClk column will be represented as “*”.

timing_report_arrival_property_worstcase_mode

`timing_report_arrival_property_worstcase_mode` *`Integer`*

Default: `0`

Data_type: Integer

Differentiates the clock and data phases based on the arrival_* properties (see `get_property` command) . The accepted values are 0, 1, 2, or 3.

timing_report_begin_end_pair_max_path_limit

`timing_report_begin_end_pair_max_path_limit` *`integer`*

Default: `2000000`

Data_type: Integer

Sets a limit on the maximum number of paths that can be reported when the `report_timing -begin_end_pair` parameter is specified.

Note: Higher maximum paths limit may lead to high memory usage and runtime.

timing_report_check_timing_unconstrained_endpoints_due_to_constants

```
timing_report_check_timing_unconstrained_endpoints_due_to_constants {true | false}
```

Default: `false`

Data_type: Boolean

Enables the software to flag unconstrained signals arriving at the end points due to case analysis.

When set to `false`, the `check_timing` command output will show warnings for the unconstrained endpoints.

When set to `true`, the command output will also show warnings for unconstrained endpoints due to case analysis under the subcategory `uncons_endpoints_due_to_case_analysis`. For more details, see `check_timing`.

timing_report_clock_pin_as_begin_point

```
timing_report_clock_pin_as_begin_point {true | false}
```

Default: `false`

Data_type: Boolean

Allows the software to report clock pins as begin points.

timing_report_constraint_extended_cell_em_flow

```
timing_report_constraint_extended_cell_em_flow {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software enables extended Cell EM flow.

timing_report_constraint_enable_extended_drv_format

```
timing_report_constraint_enable_extended_drv_format {true | false}
```

Default: `false`

Data_type: Boolean

Enables the software to report data as per clock/data and rise/fall phases.

timing_report_constraint_format

```
timing_report_constraint_format string
```

Default: " "

Data_type: String

Specifies the DRV fields that will be reported with the `report_constraint` command. For valid format column options, refer to `report_constraint -drv_fields`.

If DRV fields are not specified with this attribute (or with `report_constraint -drv_fields`), then the default set of DRV fields will be displayed. By default, the constraint report will display the pin names, the required and actual DRC values, and the view names.

Note: The reporting fields specified with the `report_constraint -drv_fields` parameter will override the attribute settings.

timing_report_constraint_rise_fall_clock_period_check

```
timing_report_constraint_rise_fall_clock_period_check {true | false}
```

Default: `false`

Data_type: Boolean

Enables reporting of rise-to-rise and fall-to-fall clock period checks separately for the `report_constraint` command.

When set to `false`, the software reports the rise-to-rise and fall-to-fall clock period checks under the same table in the report. With the `false` setting, when the `report_constraint -check_type clock_period` option is specified, the software reports violations that are the worst amongst all the rise-fall transitions for each point.

timing_report_constraint_report_all_drv_violation

```
timing_report_constraint_report_all_drv_violation {true | false}
```

Default: `true`

Data_type: Boolean

Enables reporting of all the design rule violations (DRV).

When set to `false`, the software reports the worst 1000 DRV violations per DRV check (specified with the `report_constraint -all_violators -drv_violation_type` command).

When set to `true`, the software will report all the DRV violations. You can use the `report_constraint -max_pins_per_drv` parameter to specify the maximum number of DRV violations to be reported.

Note: The `-max_pins_per_drv` parameter has higher precedence over the `timing_report_constraint_report_all_drv_violation` attribute setting.

timing_report_constraint_use_infinity_slack_for_unconstrained

`timing_report_constraint_use_infinity_slack_for_unconstrained {true | false}`

Default: `false`

Data_type: Boolean

Reports the timing slacks of unconstrained paths through the DRV violating pin as “INFINITY”. By default, slacks of unconstrained paths are reported as * (asterisk).

timing_report_default_frequency_for_unconstrained_nets

`timing_report_default_frequency_for_unconstrained_nets` *`value`*

Default:1e+09

Defines the frequency for unconstrained nets to be used in maximum capacitance DRV reporting.

timing_report_disable_max_paths_per_group

`timing_report_disable_max_paths_per_group {true | false}`

Default: `false`

Data_type: Boolean

Controls the interpretation of the `report_timing -max_paths` parameter when you request reporting across a set of path groups. By default (`false` setting), the specified `-max_paths` will be reported per path group. With the `true` setting, a total of the specified `-max_paths` will be reported across all of the specified path groups.

For Stylus flow cross-tool compatibility – you should specify a setting of `true`. For backward compatibility with the legacy UI flow – you should use a value of `false`.

For example,

When set to `false`:

Legacy UI: `report_timing -path_group {reg2reg in2reg} -nworst 2 ...`
Stylus UI: `report_timing -group {reg2reg in2reg} -nworst 2 ...`

The `report_timing` command will return up to 2 paths for the reg2reg group and also up to 2 paths for the in2reg group.

When set to `true`:

Legacy UI: `set timing_report_disable_max_paths_per_group true`
Stylus UI: `set_db timing_report_disable_max_paths_per_group true`

A total of 2 paths will be reported across both path groups. The paths may come entirely from one or the other group – or both, based on the slack value.

timing_report_drv_enable_clock_source_as_clock

```
timing_report_drv_enable_clock_source_as_clock {true | false}
```

Default: `false`

Data_type: Boolean

Enables the clock DRV for pure clock source path instead of data DRV.

timing_report_drv_enable_frequency_per_view

```
timing_report_drv_enable_frequency_per_view {true | false}
```

Default: `true`

Data_type: Boolean

Enables view-based frequency DRV flow.

When set to `true`, in the MMMC DRV flow, the software performs frequency computation for design pins as per the clock period of specific analysis views reaching a pin. In case of non-activity based flow, the frequency is computed as per the fastest clock of all the views.

When set to `false`, in the propagate activity flow, the software performs frequency computation for design pins from propagated activity on that pin.

For other flows. the software performs frequency computation for design pins as per the smallest clock period (largest clock frequency) from all the active analysis views.

timing_report_drv_per_frequency_per_input_slew

```
timing_report_drv_per_frequency_per_input_slew {true | false}
```

Default: `false`

Data_type: Boolean

Enables frequency dependent DRV lookup from a library.

When set to `true`, the software will enable the `report_constraint` command to honor max_transition or

max_capacitance limit after worst-casing due to the following cases:

- One-dimensional frequency DRV commands, such as, `set_max_transition_per_freq` or `set_max_cap_per_freq`.
- Two-dimensional maximum transition or capacitance frequency/electro-migration tables.
- Other frequency independent DRV limit sources, e.g., `set_max_transition` from the SDC or library limit.

timing_report_drv_enable_ghz_notation_for_drv_fields

```
timing_report_drv_enable_ghz_notation_for_drv_fields {true | false}
```

Default: `false`

Data_type: Boolean

Enables reporting of clock frequency in GigaHertz (Ghz). The default frequency unit is MegaHertz (MHz).

timing_report_drv_enable_slew_threshold_scaling

```
timing_report_drv_enable_slew_threshold_scaling {true | false}
```

Default: `false`

Data_type: Boolean

Enables the transition constraints scaling as per the slew threshold value.

timing_report_drv_per_frequency

```
timing_report_drv_per_frequency
{true | false}
```

Default: `true`

Data_type: Boolean

Improves reporting of maximum transition violations as per the worst limit defined using the `set_max_transition` and `set_max_transition_per_freq` commands, or the library specifications. This behavior applies to minimum/maximum transition and capacitance limits.

In MMMC designs, for reporting transition and capacitance violations, the software uses the worst transition limit specified in the transition limit sources, for example, library specifications, or `set_max_transition` constraints.

For frequency-based transition specifications, the transition violations are reported only on the basis of the transition limit specified with frequency-based specifications. The `set_max_transition` specifications are ignored by the software, if the values are pessimistic.

timing_report_drv_use_worst_timing_slack

```
timing_report_drv_use_worst_timing_slack
{true | false}
```

Default: `false`

Data_type: Boolean

Enables reporting of the worst DRV slack across all the views for configurable DRV report.

When set to `true`, the software shows a non-default special case report, where timing slacks for MMMC flow is the worst among all active views.

timing_report_enable_cppr_point

```
timing_report_enable_cppr_point { true | false}
```

Default: `false`

Data_type: Boolean

Enables the timing report to output CPPR point details.

When set to `true`, the `report_timing` command output shows the CPPR point field.

timing_report_enable_em_index_clipping_report

```
timing_report_enable_em_index_clipping_report {true | false}
```

Default: `false`

Data_type: Boolean

Enables index clipping reporting for EM (electro-migration) DRV (design rule violation) reports.

For more information, refer to the Cell EM Analysis and ECO chapter in the *Tempus Stylus User Guide*.

timing_report_enable_flag_field_symbols

```
timing_report_enable_flag_field_symbols  {true | false}
```

Default: `false`

Data_type: Boolean

Enables reporting of Instance/Net related bits in the `report_timing` `Flags` column.

timing_report_enable_clock_to_for_unconstrained_paths

`timing_report_enable_clock_to_for_unconstrained_paths {true | false}`

Default : `false`

Data_type: Boolean

Enables unconstrained path reporting when the `report_timing` `-to_clock` parameter is specified.

timing_report_enable_lead_trail_to_rise_fall_map

`timing_report_enable_lead_trail_to_rise_fall_map {true | false}`

Default: `false`

Data_type: Boolean

Enables reporting of rise/fall edge times, instead of lead/trail edge times of a clock (reported using the `report_clocks` command).

timing_report_enable_markers

`timing_report_enable_markers{true | false}`

Default: `true`

Data_type: Boolean

Enables display of markers in reports.

When set to `true`, the `report_timing` output format displays markers that identify pins specified using the `-from`, `-to`, or `-through` parameters. The markers, displayed as '`->`', show up in the 'timing point' or ' hpin' column (if the 'timing point' column is not available) of the timing report. This feature provides better readability. When set to `false`, the timing report does not display the markers.

timing_report_enable_max_capacitance_drv_for_constant_nets

`timing_report_enable_max_capacitance_drv_for_constant_nets {true | false}`

Default: `false`

Data_type: Boolean

When set to `true`, this attribute will perform max_capacitance design rule violation check on constant elements as well.

timing_report_enable_max_path_limit_warning

```
timing_report_enable_max_path_limit_warning {true | false}
```

Default: `false`

Data_type: Boolean

Determines if the number of reported paths for a particular clock group equals the specified maximum paths limit.

When set to`true`, the software will issue a warning message in case the number of reported paths for a clock group is equal to the given maximum paths limit. This applies to both graph- and path-based reporting modes.

This attribute is supported in clock group-based non-statistical reporting flows only. The clock group-based reporting mode can be enabled by setting `timing_report_group_based_mode` attribute to `true`.

timing_report_enable_qpin_as_start_for_begin_end_pair

```
timing_report_enable_qpin_as_start_for_begin_end_pair
{true | false}
```

Default : `false`

Data_type: Boolean

Enables reporting of multiple paths with different q-pins having the same clock pin.

timing_report_enable_report_clock_timing_across_clock_pin

```
timing_report_enable_report_clock_timing_across_clock_pin {true | false}
```

Default: `false`

Data_type: Boolean

Enables fanout tracing across clock pins if any combinational arc is present.

timing_report_enable_si_debug

```
timing_report_enable_si_debug {true | false}
```

Default: `false`

Data_type: Boolean

Allows the software to capture signal integrity (SI) related details for each stage of a path being analyzed during path-based analysis in the form of attributes.

timing_report_enable_verbose_ssta_mode

```
timing_report_enable_verbose_ssta_mode {true | false}
```

Default: `false`

Data_type: Boolean

Enables reporting of sigma/multiplier for the timing report (`report_timing` command) header section in SSTA or SOCV mode.

timing_report_enable_unique_pins_multiple_capture_clock_paths

```
timing_report_enable_unique_pins_multiple_capture_clock_paths {true | false}
```

Default: `false`

Data_type: Boolean

Enables reporting of different paths with respect to the reference clocks in the unique pins.

timing_report_enable_write_twf_third_party_for_input_pins

```
timing_report_enable_write_twf_third_party_for_input_pins {true | false}
```

Default : `false`

Data_type: Boolean

When set to `true`, the software writes timing information for input pins in the third-party TWF file.

timing_report_fields

```
timing_report_fields string
```

Default: `{instance arc cell delay arrival required}`

Data_type: String

Specifies a `report_timing` format. See `report_timing -fields` for more information.

Valid format columns are:

adjustment, abs_delay_err, abs_transition_err, annotation, aocv_adj_stages, aocv_derate, aocv_weight, arc, arrival, arrival_mean, arrival_sigma, cell, delay, delay_mean, delay_sigma, direction, edge, fanin, fanout, flags, hpin, incr_delay, instance, instance_location, latch_window, load, module, net, pct_delay_err, pct_slew_err, phase, phys_info, pin, pin_load, pin_location, power_domain, required, retime_delay, retime_delay_mean, retime_delay_sigma, retime_mis_delta, retime_slew, retime_transition_mean, retime_transition_sigma, stage_count, transition, transition_mean, transition_sigma, socv_derate, spice_arrival, spice_delay, spice_slew, stolen, timing_point, total_derate, user_derate, voltage, vt_skew_derate, when_cond, wire_load, wlmodel

timing_report_generated_clock_info

`timing_report_generated_clock_info {true | false}`

Default: `true`

Data_type: Boolean

When set to `true` (the default), generated clock information is automatically added to the report if generated clocks are encountered in either the launching or latching clock paths. The generated clock information column is displayed if the `Delay' field exists. If the `Delay' field does not exist, then the generated clock adjustment column is added to the report only when explicitly specified using the `report_timing -fields` command.

timing_report_group_based_mode

`timing_report_group_based_mode {true | false}`

Default: `false`

Data_type: Boolean

Groups paths by clock domain.

When set to `true`, the behavior of the following reporting commands will change as follows:

`report_timing`

The behavior of the following `report_timing` parameters changes as follows:

- `-max_points`: In this mode you are not required to specify this parameter. If specified, the parameter is mapped to `-max_paths` option.
- `-max_paths M`: Reports `M` paths per clock group.

- `-nworst N`: Limits the number of paths per endpoint per clock group. By default, the value of `-nworst` is `1`.

The `report_timing` command will have the following four types of groups (if design does not have user-defined path groups):

- Asynch group: Recovery/removal checks.
- Clock gating group: Inferred and library clock gating checks.
- Default group: Combinational max delay or min delay paths.
- Clock Groups: Groups for all design clocks. For a design with three clocks, three groups will be reported, one group for each clock. Endpoints sharing the same reference clock will be part of the same group. If an endpoint belongs to multiple groups then it will be reported separately in each group.

Note: If user-defined path groups are specified, the software will override the path groups described above.

`report_min_pulse_width`

The `report_min_pulse_width` command reports pulse width slacks of all the clocks for a pin. In summary mode, the "clock" column will be displayed. In verbose mode (with `path_type` option), the behavior will be similar to the `report_timing` output in group-based mode.

`report_constraint`

The `report_constraint`command will report the following checks:

- For max_delay/setup and min_delay/hold type checks paths will be reported for each clock group separately.
- The behavior of pulse width check types will be the same as `report_min_pulse_width` command.

`report_analysis_coverage`

The `report_analysis_coverage` command will report a check at end points for all the clocks. In verbose mode, the "clock" column will be displayed.

timing_report_limit_max_path_count

`timing_report_limit_max_path_count` *value*

Default : `2000000`

Data_type: integer

Sets the maximum number of violations (of data/clock checks) to be reported.

timing_report_max_transition_check_using_nsigma_slew

```
timing_report_max_transition_check_using_nsigma_slew {true | false}
```

Default: `false`

Data_type: Boolean

Enables the `report_constraint` command to use mean and sigma slew values for reporting transition values in DRV checks.

When set to `true`, in the SOCV flow the `report_constraint` command uses the mean plus n-sigma slew values for DRV checks.

When set to `false`, the `report_constraint` command will report only the mean slews.

timing_report_precision

```
timing_report_precision integer
```

Default: `3`

Data_type: Integer

Controls the number of digits appearing after the decimal point in the `report_timing`, `report_nets`, `report_cell_instance_timing`, and `report_clocks` reports.

timing_report_property_fastest_clock_consider_data_phase

```
timing_report_property_fastest_clock_consider_data_phase
```

```
{true | false}
```

Default : `false`

Data_type: Boolean

Considers the data signals while determining the fastest clock arriving on any given pin.

When set to `false`, the clocks arriving in clock phase form will be considered for determining the fastest clock.

When set to `true`, the clocks arriving in clock phase as well as data phase form will be considered.

timing_report_pulse_width_matching_launch_capture_paths

```
timing_report_pulse_width_matching_launch_capture_paths {true | false}
```

Default: `false`

Data_type: Boolean

Reports pulse width paths that have matching launch and capture paths - the count of pins and the pins must match exactly. The paths that do not meet this condition will not be reported.

When set to `true`, the software will impact the minimum pulse width report that is generated with the `report_min_pulse_width` command along with the `-path_type short` and `-path_type full_clock` options.

timing_report_redirect_message_types

```
timing_report_redirect_message_types
{none | info}
```

Default: `none`

Data_type: Enum

Redirects reporting messages to a report file.

When set to `info`, the path count and max depth (displayed when the `report_timing` command is run in PBA mode) will be appended to the final report output.

When set to `none`, the reporting messages are not redirected to a report file.

timing_report_retime_formatting_mode

```
timing_report_retime_formatting_mode {manual | retime_compare | retime_replace}
```

Default: `manual`

Data_type: Enum

Manages the re-timing fields in the reporting format. Based on the given settings, this attribute automatically replaces (or adds) the default columns with the respective re-timing columns. For example, in case of delay column, 'Retime Delay' will be added or replaced automatically.

timing_report_skip_constraint_loop_check

```
timing_report_skip_constraint_loop_check {true | false}
```

Default: `true`

Data_type: Boolean

Allows you to skip loop checking when the `check_timing` command is used.

When set to `true`, the software skips reporting of loops in case the timing graph inside a loop is broken. This may

happen if some of the commands, such as, `create_clock`, `set_input_delay`, or `set_output_delay` are defined inside a loop.

Note: When this attribute is set to `true`, the `check_timing` command will report lesser loops than the ones reported in the CTE_loops.rpt report - which reports all the loops.

timing_report_skip_icg_from_all_fanin_fanout_collection

```
timing_report_skip_icg_from_all_fanin_fanout_collection {true | false}
```

Default : `false`

Data_type: Boolean

When set to `true`, the software skips ICG clock pins from the `all_fanin` (or `all_fanout`) command collection, when queried with startpoints/endpoints.

timing_report_socv_summary_mean_sigma

```
timing_report_socv_summary_mean_sigma {true | false}
```

Default: `false`

Data_type: Boolean

Prints the report header in new format when the `report_timing` command is run in SOCV mode. The new header contains nsigma, mean, and sigma details.

timing_report_timing_header_detail_info

```
timing_report_timing_header_detail_info {default | extended}
```

Default: `default`

Data_type: Enum

Controls the timing report (generated using the `report_timing` command) header format. You can specify one of the following options:

- When set to `default`, the software generates timing reports with a report header that includes the following information:
  - Tempus software version number
  - Platform on which the software is being run

  - Time when the report was generated
  - Command used to generate the report
- When set to `extended`, the software generates timing reports with additional header information given below:
  - The analysis mode - best case worst case or on chip variation
  - The active views (displayed only in MMMC mode)
  - The clock reconvergence pessimism (CPPR)
  - The latch slack mode
  - The specified clock propagation value
  - The specified delay calculation engine

Note: You can use this attribute when `timing_disable_report_header_info` is set to `false`.

timing_report_unconstrained_paths

```
timing_report_unconstrained_paths {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the `report_timing` command reports unconstrained paths if the software cannot find any constrained paths to report.

When set to `false`, the `report_timing` command reports only the constrained paths.

Note: The behavior of `timing_report_unconstrained_paths` attribute is not the same as the behavior of `report_timing -unconstrained` command. If there are several paths to the same endpoint from different begin points, including constrained and unconstrained paths, the `report_timing -unconstrained` parameter always reports unconstrained paths, even if a constrained path can be reported. Whereas the `timing_report_unconstrained_paths` attribute reports constrained paths but not unconstrained paths.

timing_report_unconstrained_path_early_late_header

```
timing_report_unconstrained_path_early_late_header {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, this software provides information in the timing report header that indicates whether an unconstrained path is a late or early path.

timing_report_use_receiver_model_capacitance

```
timing_report_use_receiver_model_capacitance {true | false}
```

Default: `false`

Data_type: Boolean

Enables the max_capacitance violations based on dynamic pin capacitance.

When set to `true`, the software checks for both the input level and timing arc level dynamic pin capacitance tables for worst-case pin capacitance table for given input slew. If conditional pin capacitance tables (or timing_arcs) exist, the worst-case is used among all the conditions.

This attribute performs static pin capacitance inference after delay calculation, so the delay computation is not impacted. The pin capacitance is used only for reporting and max_cap violations.

timing_report_use_worst_parallel_cell_arc

```
timing_report_use_worst_parallel_cell_arc {true | false}
```

Default: `false`

Data_type: Boolean

When set to `false`, setting the `-nworst` parameter reports several paths by selecting parallel arcs in the library cell between two pins.

When set to `true`, only a single worst path is reported by selecting the worst delay arc among all parallel arcs of a cell. For example, for reporting timing through A to Y pins (where the cell is a 2-input cell), the parallel arc is applicable for the same input and output pin. So the worst arc for A->Y is chosen from all arcs of A->Y.

timing_resolve_driver_conflicts

```
timing_resolve_driver_conflicts {conservative | aggressive}
```

Default: `aggressive`

Data_type: Enum

Resolves constant propagation results on nets with multiple drivers.

When the attribute is set to `conservative` mode, the net values are as follows:

- If all the drivers are 1, the net will be driven to value `1`
- If all the drivers are 0, the net will be driven to value `0`
- In other cases, the net value will be `Undefined`

When the attribute is set to `aggressive` mode, the net values are as follows:

- If some of the drivers are 0, the net value will be `0`
- If some of the drivers are 1, but none are 0, then the net value will be `1`
- In other cases, the net value will be `Undefined`

timing_scaling_for_negative_checks

```
timing_scaling_for_negative_checks {default | divider | multiplier}
```

Default: `default`

Data_type: Enum

Modifies the scaling of the negative timing analysis check value. This variable does not affect the positive checks. You can specify the following options:

- `default`: Negative check is multiplied by (2.0 - derating factor), where the derating factor is the value that you set using the `set_timing_derate` command.
- `divider`: Divides the negative check value by the derating factor.
- `multiplier`: Multiplies the negative check value by the derating factor.

timing_scaling_for_negative_delays

```
timing_scaling_for_negative_delays {default | divider | multiplier}
```

Default: `default`

Data_type: Enum

Modifies the scaling of the negative delay value to be used during timing analysis. This attribute does not affect the positive delay. You can specify the following options:

- `default`: The negative delay is multiplied by (2.0 - derating factor), where the derating factor is the value that you set using the `set_timing_derate` command.
- `divider`: The software divides the negative delay value by the derating factor.
- `multiplier`: The negative delay value is multiplied by the derating factor.

Currently, the constraints are used during timing in the flattened mode, so the internal ILM instances are seen instead of the LEF pins of the ILMs. Therefore, reading the bounding box constraints can cause errors without this variable set. If you want to see the LEF pins of the ILM in GUI, the design must be in the unflattened mode.

timing_sdf_adjust_negative_setuphold

```
timing_sdf_adjust_negative_setuphold {true | false}
```

Default: `false`

Data_type: Boolean

Controls how the setup and hold check values are adjusted when both are initially negative in the SDF file.

When set to `true`, the software sets the smaller check delay value to a negated larger check delay value. For example, if the initial setup/hold value is equal to -1/-2. then the value -2 from the setup is smaller and it is reset to a negated value of the larger one (-1 from hold). So the adjusted setup/hold value is equal to -1/1.

When set to `false`, both the negative setup and hold check values are set to 0.

The command order for using this attribute is as follows:

- `read_sdf`
- `timing_sdf_adjust_negative_setuphold {true | false}`
- `write_sdf`

The timing report (`report_timing`) results are not impacted.

timing_sdf_enable_setuphold_scond_ccond

```
timing_sdf_enable_setuphold_scond_ccond {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, enables proper generation of SDF (Standard Delay Format) with scond and ccond qualifiers on `SETUPHOLD` and `RECREM` timing checks.

Since some SDF readers do not accept scond and ccond qualifiers this attribute is set to `false` by default.

timing_self_loop_paths_no_skew_max_depth

```
timing_self_loop_paths_no_skew_max_depth int
```

Default: 10

Data_type: Integer

Identifies self-loop paths at the specified depth.

For example:

```
set_db timing_self_loop_paths_no_skew_max_depth 20
```

Allows the timer traverse 20 timing arcs (10 stages of logic) before abandoning the search for a self loop.

timing_self_loop_paths_no_skew_max_slack

```
timing_self_loop_paths_no_skew_max_slack value
```

Default: `0.0`

Data_type: Double

Identifies self-loop paths at the specified threshold.

For example:

```
set_db timing_self_loop_paths_no_skew_max_slack 1.0
```

In this case, all timing endpoints with a worst slack less than `1.0` are considered for self loop skew removal.

timing_set_clock_source_to_output_as_data

```
timing_set_clock_source_to_output_as_data {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, causes a clock source path leading to an output or bidi port to be treated as a data path if there is `set_output_delay` or `set_data_check` assertion on the port.

timing_socv_preserve_variation_with_annotations

```
timing_socv_preserve_variation_with_annotations {true | false}
```

Default: `false`

Data_type: Boolean

Preserves the sigma value computed by delay calculation along with the annotation from SDC.

In the clock mesh flow, it is possible to preserve sigma values even after applying annotation on slew/delay through SDC.

When this attribute is set to `true`, the software accepts the sigma value computed by delay calculation (for nets that have delay/slew annotations from SDC, e.g., clock-mesh), along with the annotation from SDC.

timing_socv_rc_variation_mode

```
timing_socv_rc_variation_mode  {true | false}
```

Default: `true`

Data_type: Boolean

Enables interconnect variation mode in SOCV mode.

timing_socv_sigma_spatial_multiplier

```
timing_socv_sigma_spatial_multiplier {true | false}
```

Default: `1.0`

Data_type: Double

Provides a fractional percentage multiplier, which allows you to select some (or all, none) of the spatial derate to be applied.

timing_socv_view_based_nsigma_multiplier_mode

```
timing_socv_view_based_nsigma_multiplier_mode {true | false}
```

Default: `true`

Data_type: Boolean

Enables setting of view-based separate setup and hold sigma multipliers.

By default or with an explicit true setting, the timing system is configured to support different N-sigma multiplier values for Setup vs. Hold – and for different analysis views. The specific N-sigma multipliers can be set using the `set_socv_reporting_nsigma_multiplier` command.

timing_spatial_derate_chip_size

```
timing_spatial_derate_chip_size value_in_microns
```

Default: `1e+30`

Data_type: Double

Allows you to define the chip size.

The chip size is the diagonal length of a rectangle that is formed by the chip's length and width. You can obtain the diagonal distance by calculating the square-root of the Length^2 + Width^2. The diagonal distance should always be specified in units of microns.

The chip size can be picked in the following ways (in order of highest to lowest precedence):

- When set to a value other than `1e+30`, the specified value will have the highest precedence.
- If the physical floorplan information is available, the chip size will be derived automatically from the floorplan.
- If the SPEF parasitic files containing the coordinate information are loaded, then the chip size will be automatically derived.

timing_spatial_derate_distance_mode

```
timing_spatial_derate_distance_mode {chip_size}
```

Default: `chip_size`

Data_type: Enum

Allows you to select the graph-based analysis (GBA) spatial mode - `chip_size`.

This attribute can be used when the spatial derate mode is enabled using the `timing_enable_spatial_derate_mode true` setting.

timing_suppress_escape_characters

```
timing_suppress_escape_characters {true | false}
```

Default: `true`

Data_type: Boolean

Suppresses the reporting of escape characters in timing object names.

When the attribute is set to `true`, all the reporting and collection commands report the timing objects names (e.g., pins, ports, or instances) without the escape characters.

timing_suppress_ilm_constraint_mismatches

```
timing_suppress_ilm_constraint_mismatches {true | false}
```

Default: `false`

Data_type: Boolean

When set to `true`, the software suppresses all error and warning messages related to objects not found when loading SDC constraint files for the ILM flow.

In the current ILM flow, SDC constraints originally specified against the complete flat netlist for the design do not get filtered when being applied against the ILM view of the design. Constraints that reference parts of the design that were pruned cause warnings and errors during constraint loading.

timing_time_unit

```
timing_time_unit string
```

Default: `none`

Data_type: String

Specifies the timing units to be used for timing constraints and timing reports. The default units for timing are inferred from the timing libraries. You can override this inference by explicitly setting the value to `none`, `1ns`, `1ps`, `10ps`, or `100ps`. Setting this variable explicitly is the same as executing the `set_library_unit -time` command to set the time units.

timing_use_clock_pin_attribute_for_clock_net_marking

```
timing_use_clock_pin_attribute_for_clock_net_marking {true | false}
```

Default: `false`

Data_type: Boolean

Allows the propagation of clocks to pins with clock attributes regardless of the presence of check arcs or trigger arcs at the pins.

timing_use_latch_early_launch_edge

```
timing_use_latch_early_launch_edge {true | false}
```

Default: `true`

Data_type: Boolean

Enables the latch use early launch edge feature.

During timing analysis when time borrowing and clock path pessimism removal (CPPR) is enabled, the given time is calculated using both borrow time and CPPR time. This may result in unnecessary timing violations.

Since CPPR is an accurate means of pessimism removal, you are required to set the `timing_use_latch_early_launch_edge` attribute to `false` so that CPPR is applied to all the paths in the design.

When set to `true`, the launch clock path of timing paths starting at a latch is timed in early mode for setup analysis, and for late mode for hold analysis. No CPPR credit is given to the timing path.

When set to `false`, the behavior is like for any other path. The launch clock path of timing paths starting at a latch is timed in late mode for setup analysis, and early mode for hold analysis. The CPPR credit is given to the path according to the attribute CPPR settings.

timing_use_incremental_si_transition

```
timing_use_incremental_si_transition {true | false}
```

Default: `false`

Data_type: Boolean

Enables usage of incremental slew during DRV violation reporting.

timing_use_latch_time_borrow

```
timing_use_latch_time_borrow {true | false}
```

Default: `true`

Data_type: Boolean

When set to `false`, does not consider time borrowing during timing analysis. Time borrowing is the amount of time borrowed by a previous logic.

timing_use_verilog_for_model_netlist

```
timing_use_verilog_for_model_netlist int
```

Default: `0`

Data_type: Boolean

Allows you to choose whether to write the scope and boundary model netlists in Verilog format instead of binary.

When set to `true`, the software will use Verilog format for saving model nets.

timing_waveform_aware_pulse_width_checks_high_voltage_level

```
timing_waveform_aware_pulse_width_checks_high_voltage_level double
```

Default: 0.95

Data_type: Double

Specifies the high voltage level to be used for waveform-aware pulse width checks.

timing_waveform_aware_pulse_width_checks_low_voltage_level

```
timing_waveform_aware_pulse_width_checks_low_voltage_level double
```

Default: `0.05`

Data_type: Double

Specifies the low voltage level to be used for waveform-aware pulse width checks.

timing_write_sdf_no_escape_backslash

```
timing_write_sdf_no_escape_backslash {true | false}
```

Default: `false`

Data_type: Boolean

Supports various patterns of the escape character.

For example, consider the following input Verilog expression:

\a/b\/c\/d

When the attribute is set to `true`, the `write_sdf` command removes the first escape character and outputs `a/b\/c\/d` in the SDF file.

When set to `false`, the command removes all the escape characters and outputs `a/b/c/d`.

update_analysis_view

```
update_analysis_view
[-help]
-name existingAnalysisViewName
{[-constraint_mode newModeName]
[-delay_corner newDelayCornerObj]
[-latency_file string]
[-power_modes powerModesList]}
```

Updates the attribute values of the specified existing analysis view.

ⓘ You can update an analysis view before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

Parameters

`-help`	Prints out the command usage.
`-constraint_mode` *newModeName*	Specifies the name of the new constraint mode to associate with this analysis view.
`-delay_corner` *newDelayCornerObj*	Specifies the name of the new delay calculation corner to associate with this analysis view.
`-latency_file` *string*	Specifies the latency file name.
`-name` *existingAnalysisViewName*	Specifies the name of the existing analysis view to update.
`-power_modes` *powerModesList*	Specifies list of power modes to use with this view. The modes must exist in the power intent file.

Examples

- The following command creates an analysis view called `missionSlow` using the constraint mode `missionSetup` and the delay calculation corner `dcWCCOM`:

  ```
  tempus> create_analysis_view
  -name missionSlow
  -constraint_mode missionSetup
  -delay_corner dcWCCOM
  ```

  The following command changes the delay calculation corner associated with the analysis view `missionSlow` to `dcMax`:

  ```
  tempus> update_analysis_view -name missionSlow -delay_corner dcMax
  ```

Related Commands

- `create_analysis_view`
- `set_analysis_view`

update_constraint_mode

```
update_constraint_mode
[-help]
[-name modeName]
{[[-sdc_files {file1.sdc file2.sdc ...}]
[-ilm_sdc_files {file1.sdc file2.sdc ...}]
[-tcl_variables {{var_name1 value1} {var_name2 value2} {var_name3 value3} ...}]
[-reset_latency_files ]] }
```

Updates the SDC constraint file information for the specified existing constraint mode object.

Note: The `update_constraint_mode` command overwrites the previous constraint mode settings. However, by default, this command updates a constraint mode without affecting other constraint modes, including interactive constraints, such as, `set_dont_use` and `set_dont_touch`.

ⓘ You can update a constraint mode object before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

Parameters

`-help`	Prints out the command usage.
`-ilm_sdc_files {file1.sdc file2.sdc ...}`	Specifies the new SDC constraint files for the ILM flow to associate with this constraint mode.
`-name modeName`	Specifies the name of the existing constraint mode object to update.
`-reset_latency_files`	Cleans up latency files of all related views.
`-sdc_files {file1.sdc file2.sdc ...}`	Specifies the new SDC constraint files to associate with this constraint mode.
`-tcl_variables {{var_name1 value1} {var_name2 value2} {var_name3 value3} ...}`	Specifies a list of TCL variables and values to be used in the constraint mode.

Examples

- The following command groups the SDC files io.sdc, mission1-clks.sdc, and mission1-except.sdc to create a mode object named `missionSetup`:

  ```
  tempus> create_constraint_mode -name missionSetup
  -sdc_files [list io.sdc mission1-clks.sdc mission1-except.sdc]
  ```

  The following command changes the SDC files associated with the constraint mode missionSetup to io.sdc, mission3-clks.sdc, and mission3-except.sdc:

  ```
  tempus> update_constraint_mode -name missionSetup
  -sdc_files {io.sdc mission3-clks.sdc mission3-except.sdc}
  ```

Related Commands

- `create_constraint_mode`

update_delay_corner

```
update_delay_corner
[-help]
-name delayCornerName
[-early_estimated_worst_irDrop_factor percent]
[-early_irdrop_data files_or_directories]
[-early_irdrop_files list_of_files]
[-early_rc_corner rcCornerObj]
[-early_temperature_files list_of_files]
[-early_timing_condition conditional_pairs]
[-late_irdrop_data files_or_directories]
[-late_irdrop_files list_of_files]
[-late_rc_corner rcCornerObj]
[-late_temperature_files list_of_files]
[-late_timing_condition conditional_pairs]
[-late_estimated_worst_irDrop_factor percent]
[-irdrop_data files_or_directories]
[-irdrop_files list_of_files]
[-pg_net_voltages voltage_pairs]
[-rc_corner rcCornerObj]
[-si_enabled {true | false}]
[-temperature_files list_of_files]
[-timing_condition conditional_pairs]
```

Modifies the parameters of an existing delay calculation corner object. You can use the `update_delay_corner` command to add or change attribute values for an existing delay calculation corner object, or for an existing power domain definition.

You can update a delay calculation corner object before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

You also can use the `update_delay_corner` command to add a power domain definition to the specified delay calculation corner. Instances contained within an MSMV power domain often require a different library or operating condition that used for the default power domain. Power domain definitions within a delay calculation corner object can be used to assign specific libraries and PVT settings per power domain.
Note: Use separate delay calculation corners to define major PVT corner differences. Use the `-early_*` and `-late_*` parameters within a single delay calculation corner to control on-chip-variation.

The `-early_*` and `-late_*` parameters can be specified separately, to update a delay corner object with early or late information. If you only specify one of the `-early_*` or `-late_*` parameters, the software uses the original information pointer in place of the missing early/late parameter.

For example, if you initially created a delay corner object using:

```
create_delay_corner
-library_set libsNominal
...
```

Use this command after creating at least one delay calculation corner (`create_delay_corner`).

Parameters

`-help`	Prints out the command usage.
`-early_estimated_worst_irDrop_factor` *`per cent`*	Specifies the early voltage derating factor. When this parameter is specified, the software performs delay calculations using the voltage = nomVolt * (1 – value). *Default*: 0.0
`-early_irdrop_data` *`files_or_directories`*	Specifies the IR drop files or directories to apply when calculating the early arrival times at a single delay corner.
`-early_irdrop_files` *`list_of_files`*	Specifies the list of IR drop files to apply when calculating early arrival times at a single power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. **Note**: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
`-early_rc_corner` *`rcCornerObj`*	Specifies the RC corner object to associate with the early corner object.
`-early_temperature_files` *`list_of_files`*	Specifies the name of the temperature file for temperature aware delay calculation for early corner.
`-early_timing_condition` *`conditional_pairs`*	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object.
`-irdrop_data` *`files_or_directories`*	Specifies the IR drop files or directories to apply to both the early and late delay calculation for the specified delay corner object.

`-irdrop_files` *list_of_files*	Specifies the list of IR drop files to apply to both early and late delay calculation for this power domain object. This parameter is used primarily to configure single-corner or Best-Case Worst-Case (BC-WC) analysis modes. **Note**: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
`-late_estimated_worst_irDrop_factor` *percent*	Specifies the late voltage derating factor. When this parameter is specified, the software performs delay calculations using the voltage = nomVolt * (1 – value). *Default*: 0.0
`-late_irdrop_data` *files_or_directories*	Specifies the IR drop files or directories to apply when calculating the late arrival times at a single delay corner.
`-late_irdrop_files` *list_of_files*	Specifies the list of IR drop files to apply when calculating late arrival times at a single power domain. This parameter is used primarily when configuring the multi-mode multi-corner environment for on-chip-variation (OCV) analysis. **Note**: This parameter is obsolete and will be removed in a future release. To ensure compatibility with future releases, you should update your scripts.
`-late_temperature_files` *list_of_files*	Specifies the name of the temperature file for temperature aware delay calculation for late corner.
`-late_rc_corner` *rcCornerObj*	Specifies the RC corner object to associate with the late corner object.

`-late_timing_condition` *conditional_pairs*	Specifies pair list of domains/supply sets and timing conditions to associate with this delay corner object. **Note**: The `-early_timing_condition` and `-late_timing_condition` parameters should be specified together.
`-name` *delayCornerName*	Specifies the delay calculation corner to which to add the power domain definition, or to which to update the delay corner or power domain attribute values.
`-pg_net_voltages` *voltage_pairs*	Specifies the power/ground net voltages to be used for both early/late timing analysis.
`-rc_corner` *rcCornerObj*	Specifies the new RC corner object to associate with this delay corner.
`-si_enabled {true | false}`	Controls signal integrity (SI) analysis. When set to `true`, the analysis of all MMMC views will include SI analysis by default. You can use this parameter to further determine which analysis views receive SI analysis and which do not. When this parameter is set to `false` on a delay corner, the software will disable SI analysis for all the views that reference that corner. By default (or when this parameter is set to `true`), SI analysis is computed for all the views related to that corner - whenever SI is globally enabled. (The global control of SI analysis is determined by the `delaycal_enable_si true` attribute setting.)
`-temperature_files` *list_of_files*	Specifies the name of the temperature file for temperature aware delay calculation for single corner.

`-timing_condition` *`conditional_pairs`*	Specifies pair list of domains/supply sets and timing conditions to associate with the defined delay corner object. Some of the conditional pairs are explained below: • timing_condition: Specifies the default timing condition which applies to all cells not covered by any other timing condition. • power_domain_list@timing_condition: The named timing condition applies to the list of all power domains (defined in a power intent file). • supply_set_list@timing_condition: The named timing condition applies to all blocks tied to the specified supply set. • hinst@timing_condition: The named timing condition applies to the specified hierarchical instance. The timing conditions must be defined with the `create_timing_condition` command.

Examples

- The following example specifies a list of conditional pairs that are domains/supply sets and the timing conditions associated with the specified delay corner object:

```
update_delay_corner -name ccworst_setup\
-timing_condition {tc_1 PD1@tc_2 PD2@tc_2 PD3@tc_2 PD4@tc_2}
```

Related Commands

- `create_delay_corner`
- `create_rc_corner`
- `create_timing_condition`

update_io_latency

```
update_io_latency
[-help]
[-adjust_source_latency]
-verbose
```

Provides an automated means of balancing IO and Core clock latencies after the clock network has been added to the design so that pre and post clock tree skew between the clocks is maintained as closely as possible. This results in minimum change of inbound and outbound IO slacks.

This command performs the following actions:

- Measures and calculates ideal average latency for each Core clock.
- Calculates preCTS skew between IO and Core clocks.
- Asserts `set_propagated_clock` on all clock root pins or ports to put clock trees in propagated mode.
- Measures and calculates propagated mode average latency for each Core clock.
- Adjusts `set_clock_latency` on IO clocks network latency to minimize pre to post CTS IO-to-Core clock skew.

In case a single IO clock interacts with multiple Core clocks, the IO clock will be adjusted by the overall average latency of all the related Core clocks combined.

To be effective, this command should only be used after clock trees have been implemented and before the set_propagated_clock command is issued. This allows the command to see both the ideal and propagated state of the clock trees.

Parameters

`-help`	Prints out the command usage.
`-adjust_source_latency`	Adjusts the source latency of core clock.
`-verbose`	Provides additional information to the console as adjustments are made to each IO clock.

Related Commands

- `set_propagated_clock`

update_library_set

```
update_library_set
[-help]
[-aocv string]
[-library_side_file string]
-name libSetName
[-si string]
[-socv string]
[-timing string]
```

Updates an existing specified library set.

Note: You can edit a library set before (or after) the multi-mode multi-corner view definition file is loaded into the design. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode resets the timing, delay calculation, and RC data.

Parameters

`-help`	Prints out the command usage.
`-aocv string`	Specifies a list of new AOCV libraries to include in the specified library set.
`-library_side_file string`	Specifies the library side file.
`-name libSetName`	Specifies the name of an existing library set to be updated. The library set must have been defined using the `create_library_set` command.
`-si string`	Specifies a list of new libraries and user-defined noise (UDN) models to include in the library set.
`-socv string`	Specifies a list of new SOCV libraries to be included in the library set.
`-timing string`	Specifies a list of new timing libraries to include in the specified library set. Note: The ECSM and CCS libraries can be very large and loading them may take significant time. You can convert the ASCII NLDM (Liberty .lib), ECSM, and CCS libraries to a binary version using the write_ldb command. For conversion of libraries to a binary version, refer to `write_ldb`. Instead of the original ASCII libraries, you can specify the following binary libraries in the configuration file: `create_library_set -name libName -timing "AAA.lib BBB.ecsm CCC.ccs"`

Examples

- The following command creates a library set that associates timing libraries with a nominal voltage of 1 volt with the library name IsCOM-1V:

  ```
  tempus>  create_library_set -name IsCOM-1V -timing [list stdcell_F_1V.lib ram_F.lib pad.lib]
  ```

  The following command changes the timing libraries associated with the library set IsCOM-1V:

  ```
  tempus>  update_library_set -name IsCOM-1V -timing [list stdcell_CVF_1V.lib ram_CVF.lib
  pad.lib]
  ```

Related Commands

- create_library_set

update_model_context

```
update_model_context
[-help]
-dir string
[-instances string]
```

Allows the software to read in the context-only boundary models and perform timing analysis without the need to setup a new STA run with a different boundary model.

Parameters

`-help`	Prints out the command usage.
`-dir string`	Specifies the directory for the model context.
`-instances string`	Specifies the instances for context update.

Examples

- The following example shows that commands that are used to update the timing context of a boundary model:

```
read_boundary_model -dir dir_derate_nominal
…
set_timing_derate -early 1
set_timing_derate -late 1
update_timing
update_model_context -dir dir_derate_extreme
set_timing_derate -early 0.8
set_timing_derate -late 1.2
update_timing
```

Related Commands

- write_model_context
- create_boundary_model
- read_boundary_model

update_opcond

```
update_opcond
[-help]
-name opcondName
[-process processValue]
[-temperature temperatureValue]
[-voltage voltageValue]
```

Updates a set of virtual operating conditions in the specified library that were created using the create_opcond command.

Parameters

-help	Prints out the command usage.
-name *opcondName*	Specifies the name of the operating condition to be updated.
-process *processValue*	Specifies the process value for the operating condition.
-temperature *temperatureValue*	Specifies the temperature value for the operating condition.
-voltage *voltageValue*	Specifies the voltage value for the operating condition.

Related Commands

- set_opcond
- create_opcond

update_rc_corner

```
update_rc_corner
[-help]
[-temperature temperatureValue]
[-cap_table newcapTableFile]
-name existingRcCornerName
[-post_route_res {resFactor1 resFactor2 resFactor3}]
[-post_route_cap {capFactor1 capFactor2 capFactor3}]
[-post_route_cross_cap {xcapFactor1 xcapFactor2 xcapFactor3}]
[-post_route_clock_res {resFactor1 resFactor2 resFactor3}]
[-post_route_clock_cap {capFactor1 capFactor2 capFactor3}]
[-pre_route_clock_res resFactor]
[-pre_route_clock_cap capFactor]
[-pre_route_res resFactor]
[-pre_route_cap capFactor]
[-qrc_tech newFileName]
[-via_variation_file <string>]
```

Adds or changes attribute values for the specified existing RC corner object.

> You can edit an RC corner object before the multi-mode multi-corner definition file is loaded into the design, or after. However, after the software processes the file, any change to an existing object when the software is in multi-mode multi-corner mode results in the timing, delay calculation, and RC data being reset.

You can set different postroute scale factors for the postroute engine variants by specifying them as duplets and triplets. For example,

resFactor1 resFactor2 resFactor3

Where:

- Single value: If you specify one value, the scale factor applies to effort level `low`. Scale factor value of 1 is used for `medium` and `high` by default.
- Duplet: If you specify two values, the first value is used for effort level `low` and the second value is used for `medium`. Scale factor value of `1` is used for `high` by default.
- Triplet: If you specify three values, the first value is used for effort level `low`, the second value is used for `medium`, and the third value is used for `high`.

Note: If the `read_spef` command is specified without the `-rc_scal`e parameter, for example, `"read_spef -rc_corner early_spef_${rc_corner}"`, then the software does not apply capacitance scale factor specified in `-post_route_cap` and `-post_route_cross_cap` parameters. If the `read_spef` command is specified with the `-rc_scale` parameter, for example, `"read_spef -rc_corner early_spef_${rc_corner} -rc_scale "`, then the software applies the capacitance scale factor. Here, no factor is specified in the `-rc_scale` option.

Parameters

`-help`	Prints out the command usage.
`-cap_table` *newCapTableFile*	Specifies a new capacitance table to use for extraction when using this RC corner.
`-pre_route_cap` *capFactor*	Specifies the capacitance scale factor for RC Extraction in pre-route mode.
`-pre_route_clock_cap` *capFactor*	Specifies the scale factor for capacitance of clock nets in preroute mode. Default: `0` When this parameter is not specified, the value specified with the `-pre_route_cap` parameter is used for preroute extraction. When specified, this parameter overrides the value specified with the `-pre_route_cap` parameter for clock nets.
`-pre_route_clock_res` *resFactor*	Specifies the scale factor for resistance of clock nets in preroute mode. Default: `0` When this parameter is not specified, the value specified with the `-pre_route_res` parameter is used for preroute extraction. When specified, this parameter overrides the value specified with the `-pre_route_cap` parameter for clock nets.
`-pre_route_res` *resFactor*	Specifies the resistance scale factor for RC Extraction in pre-route mode.
`-post_route_cap {`*capFactor1capFactor2 capFactor3*`}`	Specifies the capacitance scale factor for RC Extraction in post-route mode.

`-post_route_clock_cap {`*`capFactor1capFactor2 capFactor3`*`}`	Specifies the scale factor for capacitance of clock nets in the postroute extraction mode. Default: `0` When this parameter is not specified, the value specified with the `-post_route_cap` parameter is used for postroute extraction. When specified, this parameter overrides the value specified with the `-post_route_cap` parameter.
`-post_route_clock_res {`*`resFactor1 resFactor2 resFactor3`*`}`	Specifies the scale factor for resistance of clock nets in the postroute extraction mode. Default: `0` When this parameter is not specified, the value specified with the `-post_route_res` parameter is used for postroute extraction. When specified, this parameter overrides the value specified with the `-post_route_res` parameter for clock nets.
`-post_route_res {`*`resFactor1 resFactor2 resFactor3`*`}`	Specifies the resistance scale factor for RC extraction in post-route mode.
`-name` *`existingRcCornerName`*	Specifies the name of the existing RC corner object to update.
`-qrc_tech` *`newFileName`*	Specifies a new QRC technology (`.tch`) file which is used to perform sign-off RC extraction.
`-temperature` *`newTempValue`*	Specifies a new temperature, in units of Celsius, to use to derate resistance values.
`-post_route_cross_cap {`*`xcapFactor1 xcapFactor2 xcapFactor3`*`}`	Specifies the cross-coupling capacitance scale factor for RC extraction in postroute mode.

`-via_variation_file` *string*	Specifies a file for modeling via variation. The VIA variation file lists up all the VIA layer names with the corresponding multiplier. Each RC corner can have its own VIA variation file.
	For example:
	`update_rc_corner`
	`-name rcCornerName1 \`
	`-via_variation_file viaVariationFileName1`
	`update_rc_corner`
	`-name rcCornerName2 \`
	`-via_variation_file viaVariationFileName2`

Examples

- The following command creates an RC corner called `rc-cbest` that uses the capacitance table `myTech_bc.CapTbl`, and derates the resistance values based on the temperature of 50 Celsius:
  ```
  create_rc_corner -name rc-cbest -cap_table myTech.4_bc.CapTbl -temperature 50
  ```
- The following command changes the capacitance table used by RC corner `rc-cbest` to `myCap.2_bc.capTbl`, and the temperature by which resistance values are derated to `25` Celsius:
  ```
  update_rc_corner -name rc_cbest -cap_table myCap.2_bc.capTbl -temperature 25
  ```

Related Commands

- `create_rc_corner`

update_timing

```
update_timing
[-help]
[-full]
```

Updates timing for the current design.

Parameters

`-help`	Prints out the command usage.
`-full`	Runs a full timing update for the current design overriding the default behavior of running incremental timing updates, where only a minimal amount of computation is necessary to update the timing analysis information. Note that for full SI accuracy during reporting, an explicit `update_timing -full` command is required after incrementally applying constraints such as, `set_timing_derate`, `set_input_delay`, `set_max_delay`, `set_clock_groups`, or `create_clock` (these constraints only affect SI delay calculation). **Note**: It is not required to run the `update_timing` command without the `-full` parameter because timing is automatically updated by the `report_timing` and `report_constraint` commands - except after incrementally applying constraints that only affect SI delay calculation.

Related Commands

- `check_timing`
- `report_timing`
- `report_constraint`

update_timing_condition

```
update_timing_condition
[-help]
[-library_sets string]
-name timingConditionName
[-opcond string]
[-opcond_library string]
```

Updates the virtual operating conditions (set using `create_timing_condition` command) with the new settings.

Parameters

`-help`	Prints out the command usage
`-library_sets` *string*	Specifies list of library sets to associate with the specified timing condition.
`-name` *timingConditionName*	Specifies a unique name for the timing condition. This name must exist or an error is issued.
`-opcond` *string*	Specifies a single name of an operating condition to associate with the timing condition.
`-opcond_library` *string*	Specifies a name of the library to search for the defined operating condition.

Examples

- The following example shows the `update_timing_condition` command performs an update on the specified timing condition:

```
create_timing_condition -name TC1 -library_sets {ls1 ls2} -opcond oc1
update_timing_condition -name TC1 -library_sets {ls3}; # TC1 now only has libset ls3
```

- The following commands result in identical results. However, the first command creates the bin TC2 and the second command updates both the options:

```
create_timing_condition -name TC2 -library_sets {ls1 ls2} -opcond oc1
update_timing_condition -name TC2 -library_sets {ls1 ls2} -opcond oc1
```

Related Commands

- `create_timing_condition`

write_boundary_model

```
write_boundary_model
[-help]
-dir string
-output_dir string
[-overwrite]
```

Allows you to export files - Verilog, SPEF, TWF, and SDC - from a boundary model.

This command cannot be run when there is an active design in the session. So the expected usage is to start a Tempus session, and then immediately run one or more export commands, such as:

```
write_boundary_model -dir HCELL1_model -output_dir HCELL1_export -overwrite
write_boundary_model -dir HCELL2_model -output_dir HCELL2_export -overwrite
```

After the export is complete, the session is still usable for other commands.

Parameters

`-help`	Prints out the command usage.
`-dir string`	Specifies the boundary model directory.
`-output_dir string`	Specifies the target directory for exported boundary model files.
`-overwrite`	Allows you to overwrite the existing target directory.

write_global_slack_report

```
write_global_slack_report
[-help]
[-early]
[-late]
[-rise]
[-fall]
[-include_ports]
[-max_slack val]
[-min_slack val]
[-view view_name]
[> | >> file_name]
[pin_port_list]
```

Reports early and late slacks on every instance pin in the design.

This command computes and reports the required time and slack values of the specified pins.

Parameters

`-help`	Prints out the command usage.
`> | >> filename`	Specifies the output file name. When specified with >, writes the report to the specified file name. If the file already exists, the software overwrites it. When specified with >>, writes the report to the specified file name. If the file already exists, the software concatenates the report to the end of the file. **Note:** To write a compressed report, add the .gz extension to the file name.
`-early/-late`	Specifies the mode for which slack is to be reported.
`-fall/-rise`	Specifies the transition for which slack is to be reported.
`-include_ports`	Reports slack information for ports. By default, the slack information is reported for pins only.
`-max_slack val`	Reports pins that have early and late slack values less than the specified value. Type: Float
`-min_slack val`	Reports pins that have early and late slack values less than the specified value. *Type:* `Float` **Note**: This parameter is obsolete and will be removed in a future release. Cadence recommends that you use the `-max_slack` parameter.
pin_port_list	Specifies the list of pins or ports for which slack information is to be reported.
`-view view_name`	Reports slack information for the specified view.

Examples

- The following command reports the early and late slacks on every instance pin in the design:

```
tempus > write_global_slack_report > out.rpt

# Unit: 1ns
# <late_rise> <late_fall> <early_rise> <early_fall> <view_name> <hpin_name>
# ------------------------------------------------------------------------
*             *          *            *            (null)      u2/Q
*             *          *            *            (null)      u8/Q
*             *          *            *            (null)      b11/A
*             *          *            *            (null)      b11/Y
0.823         0.347      0.115        0.501        (null)      u10999/A
0.823         0.347      0.115        0.501        (null)      u10999/Y
# ------------------------------------------------------------------------
```

- The following command reports slack for a pin collection:

```
tempus> set x [get_pins -hier */A]


0x14 <---- 'x' stores Pointer for pin collection
```

- The following command will generate a global slack report for pin collection as defined in "x":

```
tempus> write_global_slack_report $x


# Unit: 1ns
# <late_rise> <late_fall>  <early_rise>  <early_fall>  <view_name> <hpin_name>
# ------------------------------------------------------------------------
19.622       19.742        0.321         0.106         (null)      seg1/u1/A
19.622       19.742        0.321         0.106         (null)      seg1/u2/A
18.018       18.362        1.876         1.486         (null)      seg1/u10/A
18.018       18.362        1.876         1.486         (null)      seg1/u11/A
18.362       18.018        1.486         1.876         (null)      seg1/u12/A
18.018       18.362        1.876         1.486         (null)      seg1/u13/A
25.845       25.883       -0.934        -1.053         (null)      seg2/u1/A
25.845       25.883       -0.934        -1.053         (null)      seg2/u2/A
# ------------------------------------------------------------------------
```

- The following example shows slack calculations, when you run the `report_timing` command for the same hpin.

```
tempus > write_global_slack_report DT_IN/RE_CO/r697_reg_2/D

# Unit: 1ns
# <late_rise> <view_name> <late_fall> <view_name> <early_rise>
<view_name> <early_fall> <view_name> <hpin_name>
# ------------------------------------------------------------------------
12.641 dtmf_view_setup 12.127 dtmf_view_setup * *
DT_IN/RE_CO/r697_reg_2/D
```

```
tempus > report_timing -to DT_IN/RE_CO/r697_reg_2/D -late -fall

Path 1: MET Setup Check with Pin DT_IN/RE_CO/r697_reg_2/CKN
Endpoint: DT_IN/RE_CO/r697_reg_2/D (v) checked with trailing
edge of 'vclk2'
Beginpoint: DT_IN/RE_CO/r697_reg_2/Q (v) triggered by trailing
edge of 'vclk2'
Path Groups: {vclk2}
Analysis View: dtmf_view_setup
Other End Arrival Time 7.015
- Setup 0.600
+ Phase Shift 14.000
- Uncertainty 0.250
= Required Time 20.164
- Arrival Time 8.037
= Slack Time 12.127
Clock Fall Edge 7.000
+ Clock Network Latency (Prop) 0.015
= Beginpoint Arrival Time 7.015
+-------------------------------------------------------------------------------+
| Instance               | Arc          | Cell    | Delay | Arrival | Required|
|                        |              |         |       | Time    | Time |
|----------------------------------------+------------+---------+------+------|
| DT_IN/RE_CO/r697_reg_2 | CKN v        |         |       | 7.015   | 19.142  |
| DT_IN/RE_CO/r697_reg_2 | CKN v -> Q v | SDFFNX1 | 0.710 | 7.725   | 19.852  |
| DT_IN/RE_CO/i_6594     | B v -> Y v   | MX2X1   | 0.312 | 8.037   | 20.164  |
| DT_IN/RE_CO/r697_reg_2 | D v          | SDFFNX1 | 0.000 | 8.037   | 20.164  |
+-------------------------------------------------------------------------------+

tempus > report_timing -to DT_IN/RE_CO/r697_reg_2/D -late -rise


Path 1: MET Setup Check with Pin DT_IN/RE_CO/r697_reg_2/CKN
Endpoint: DT_IN/RE_CO/r697_reg_2/D (^) checked with trailing
edge of 'vclk2'
Beginpoint: DT_IN/RE_CO/r697_reg_2/Q (^) triggered by trailing
edge of 'vclk2'
Path Groups: {vclk2}
Analysis View: dtmf_view_setup
Other End Arrival Time 7.015
- Setup 0.204
+ Phase Shift 14.000
- Uncertainty 0.250
= Required Time 20.560
- Arrival Time 7.919
= Slack Time 12.641
Clock Fall Edge 7.000
+ Clock Network Latency (Prop) 0.015
= Beginpoint Arrival Time 7.015
```

Instance	Arc	Cell	Delay	Arrival Time	Required Time
DT_IN/RE_CO/r697_reg_2	CKN v			7.015	19.656
DT_IN/RE_CO/r697_reg_2	CKN v -> Q ^	SDFFNX1	0.628	7.643	20.284
DT_IN/RE_CO/i_6594	B ^ -> Y ^	MX2X1	0.276	7.919	20.560
DT_IN/RE_CO/r697_reg_2	D ^	SDFFNX1	0.000	7.919	20.560

write_ipd_scope

```
write_ipd_scope
[-help]
[-context_only]
[-exclude <string>]
[-include <string>]
[-scope_data <string>]
[[[-bounded_corners ] [-include_derates ] [-include_constraints ] [-overwrite ] [<dir_name>]] | [[-early_structural_checks <string>] [-out_file_prefix <string>] [-power_domain_count_threshold <integer>]] | [-write_all_paths <string>]]
```

Writes out the IPD scope in the scope creation run. The scope is generated in the directory location specified as the command argument.

This command is used to write out the IPD Scope in the scope creation run. The scope is generated in the directory location specified as command argument - that provides options to control whether constraints and derates are to be written out in the scope or not.

Parameters

`-help`	Prints out the command usage.
`-context_only`	Re-generates constants that will be a part of context data.
`-early_structural_checks <string>`	Specifies the early check types to be performed. The valid values are: • clock_divergence • multiple_pd_data_paths • multiple_pd_clock_paths • multiple_pd_paths
`-exclude <string>`	Specifies the netlist exclusions. The supported options are clock, port, async, scanchain, and <$list_of_PD_combinations>.
`-include <string>`	Specifies the netlist filter inclusions - the supported options are: • equivalent_power_domains • internal_power_domains • timing_disabled_network • equivalent_supply_voltage

`-include_constraints`	Allows IPD scope to be written with victim constraints.
`-include_derates`	Allows IPD scope to be written with timing derates.
`<dir_name>`	Specifies the output directory containing partial Verilog and partial SPEF, or marking data, and context data.
`-bounded_corners`	Generates a bounded corner scope based on the minimum/maximum voltage combination analyzed in the scope creation run. By default, the software writes out the maximum voltage combination details in the scope, if this parameter is not specified in the bounded corner run.
`-scope_data <string>`	Specifies the scope data to be written from minimum or maximum voltage combination.
`-out_file_prefix <string>`	Specifies the outfile for structural checks.
`-overwrite`	Overwrites the already present directory's content with the new content.
`-power_domain_count_threshold <integer>`	Specifies the threshold for the power domain count reported using the `multiple_pd_paths` option.
`-write_all_paths <string>`	Writes all the IPD paths and uses the name of the path descriptions' file.

Examples

- The following command generates a bounded corner scope based on the minimum/maximum voltage combination analyzed in the scope creation run:

  ```
  tempus > write_ipd_scope -bounded_corner -overwrite bounded_corner_scope
  ```

  This command writes out the derates and constraints in the IPD scope, and overwrites an existing scope generated with same directory name argument.

Related Commands

- `create_pg_net_group`
- `report_pg_net_group`
- `create_ipd_configuration`
- `report_ipd_configuration`
- `read_ipd_scope`
- `set_ipd_rail_voltage`
- `report_ipd_crossings`

Related Information

- For more information, refer to the Inter Power Domain (IPD) Analysis chapter in the *Tempus Stylus User Guide*.

write_ldb

```
write_ldb
[-help]
ldb_file
-library string
```

Reads and generates compiled libraries.

You can use the `write_ldb` command to run timing and signal integrity analysis with binary timing libraries.

The ECSM and CCS libraries can be very large and loading them may take significant time. You can convert the ASCII NLDM (Liberty .lib), ECSM, and CCS libraries to a binary version using the `write_ldb` command. When loading the design, the reading of the timing libraries will be faster because it reads the binary versions.

The following example illustrates the conversion of libraries to a binary version:

```
write_ldb -library AAA.lib -outfile AAA.db
write_ldb -library BBB.ecsm -outfile BBB.db
write_ldb -library CCC.ccs -outfile CCC.db
```

Instead of the original ASCII libraries, you can specify these binary libraries in the configuration file:

- **Original MMMC setup file**: `create_library_set -name libName -timing "AAA.lib BBB.ecsm CCC.ccs"`
- **New MMMC setup file**: `create_library_set -name libName -timing "AAA.db BBB.db CCC.db"`

Notes

- The software does not support concatenated Liberty library (multiple libraries in the same `.lib` file) files during library database (LDB) conversion. To use these files, you are required to separate out individual libraries before converting to individual `.ldb` files. This feature may be available in future releases.
- The software does not support library database conversion for Timing Library Format (TLF) files.

Parameters

`-help`	Prints out the command usage.
`ldb_file`	Specifies the name of the generated compiled library file.
`-library` *`string`*	Specifies the name of the input library file.

Examples

- The following example illustrates the conversion of libraries to LDB:

```
write_ldb -library A.lib A.ldb
```

Related Commands

- check_ldb_version

write_loop_break_sdc

```
write_loop_break_sdc
[-help]
out_file
-in_file string
```

Generates SDC file containing constraints for combinational loop breaking. The command selects the disable arc (s) with flag set to 1 from the third-party report file, and generates the equivalent SDC commands in an output file.

You can load the output SDC file at any time during an Tempus session. To load the file, use the read_sdc command.

Parameters

`-help`	Prints out the command usage.
out_file	Specifies the name of the output SDC file.
`-in_file` *string*	Specifies the disabled time arcs report that you generate using third-party tool.

Command Order

Use this command after generating the disable arcs report.

Examples

The following command generates an `etsCombLoopBreak.sdc` file when the input is a `disable_arcs.rpt` file:

```
tempus > write_loop_break_sdc -in_file disable_arcs.rpt -out_file etsCombLoopBreak.sdc
```

The contents of `disable_arcs.rpt` file are as follows:

Cell or Port	From	To	Sense	Flag Reason

:				
uchip_core/u_s5_power	oLpcSd_4_	orig_oSmAcpiSd_4_	*	1
:				

The contents of output file `etsCombLoopBreak.sdc` are as follows:

```
:

set_disable_timing -from {oLpcSd_4_} -to {orig_oSmAcpiSd_4_}
                [get_cells {uchip_core/u_s5_power}]
```

:

write_model_context

```
write_model_context
[-help]
-dir string
[-overwrite]
-reference_dir string
```

Allows the software to create a single complete boundary model and a timing context-only boundary model, when several boundary models for a block need to be created with different timing settings (such as, derates, timing global variables).

The timing context-only boundary models do not have any netlist or parasitic information and provide only timing context data to perform timing analysis using the boundary models. Thus, these can be created with a much smaller run time using the `write_model_context` command.

Parameters

`-help`	Prints out the command usage.
`-dir string`	Specifies the directory for the model context.
`-overwrite`	Allows the software to overwrite the existing directory.
`-reference_dir string`	Specifies the directory path of the original model.

Examples

- The following example shows the commands that are used to create two boundary models. The first boundary model `dir_derate_nominal` is created using the `create_boundary_model` command and is a complete boundary model.

```
set_timing_derate -early 1
set_timing_derate -late 1
update_timing
create_boundary_model -dir dir_derate_nominal -overwrite
```

  The second boundary model `dir_derate_extrem` is a context-only boundary model that is created using the `write_model_context` command. This model provides the timing context for the `dir_derate_nominal` boundary model:

```
set_timing_derate -early 0.8
set_timing_derate -late 1.2
update_timing
write_model_context -dir dir_derate_extreme -overwrite -reference_dir dir_derate_nominal
```

- The following example shows the commands that are used to read the complete boundary model. The context can be updated using the `update_model_context` command to update the timing context of the boundary model:

```
read_boundary_model -dir dir_derate_nominal
…
set_timing_derate -early 1
set_timing_derate -late 1
update_timing
update_model_context -dir dir_derate_extreme
set_timing_derate -early 0.8
set_timing_derate -late 1.2
update_timing
```

Related Commands

- `update_model_context`
- `create_boundary_model`
- `read_boundary_model`

write_path_descriptions

```
write_path_descriptions
[-help]
var_name
-out_file string
```

Writes out the path descriptions of all the timing paths specified in the variable (path collection) to an ASCII file.

The output file contains a set of `report_timing` commands for each timing path in the given path collection. The file also includes a complete description of each timing path captured through timing points along with the clock and edge information.

The path description of a given collection of paths can be read back and the same path configuration can be recreated in another Tempus session. This allows to process and compare the timing, attributes, and other characteristics of the specified paths.

Parameters

`-help`	Prints out the command usage.
var_name	Specifies the object collection of timing paths. The input must be a single collection. If you need to write out path descriptions of collections generated with different analysis types or path collections, a single collection can be created by appending all the collections using the `append_to_collection` command.
`-out_file` *string*	Redirects output to the specified file.

Examples

- The following command writes out descriptions of late GBA paths - when the `-path_type` parameter is not specified:

```
set path_coll [report_timing -collection]
write_path_descriptions $path_coll > PathSpecFile.txt
```

  The output file PathSpecFile.txt contains:

```
report_timing -net -path_type full_clock -late -from_rise A -through_rise B -through_rise C \
       -to_rise D -view slow_v -clock_from clk1 -edge_from lead -clock_to clk2 -edge_to lead
```

- The following command writes out descriptions of late PBA paths when the `-path_type full` option is specified:

```
set path_coll [report_timing -collection -retime  path_slew_propagation -path_type full]
write_path_descriptions $path_coll > PathSpecFile.txt
```

  The output file PathSpecFile.txt contains:

```
report_timing -net -path_type full_clock -late -from_rise A -through_rise B -through_rise C \
       -to_rise D -view slow_v -clock_from clk1 -edge_from lead -clock_to clk2 -edge_to lead
```

  Note: The output does not show retime details. You should use the `read_path_descriptions -retime` parameter to analyze retime collections that are defined with the `write_path_descriptions` command.

- The following command writes out descriptions of both the early and late GBA paths:

```
set path_coll [report_timing -collection]
set path_coll_early [report_timing -collection -early]
append_to_collection path_coll $path_coll_early
write_path_descriptions $path_coll > PathSpecFile.txt
```

  The output file PathSpecFile.txt contains:

```
report_timing -net -path_type full_clock -late -from_rise A -through_rise B -through_rise C \
       -to_rise D -view slow_v -clock_from clk1 -edge_from lead -clock_to clk2 -edge_to lead
report_timing -net -path_type full_clock -early -from_rise A through_rise B -to_rise D -view \
       slow_v -clock_from clk1 -edge_from lead -clock_to clk2 -edge_to lead
```

Related Commands

- read_path_descriptions
- report_timing
- append_to_collection

write_sdc

```
write_sdc
[-help]
[-clocks <string>]
[-commands <string>]
[file_name]
[-exclude_annotated <string>]
[-path_adjust_only]
[-path_group_only]
[-pins <string>]
[-sdc]
[-view <string>]
[-vt_skew_derate_only]
```

Generates the timing constraints file in a third-party compiler format.

Parameters

`-help`	Prints out the command usage.
`-clocks <string>`	Specifies the name of the clocks to be included in the SDC file.
`-commands <string>`	Specifies the name of the commands to be included in the SDC file.
`file_name`	Specifies the name of the constraints file. By default, the constraint file is generated in the current directory. To save to a different directory, you need to specify the complete path.
`-exclude_annotated` *`string`*	Generates the constraints file excluding annotated constraints. You can specify one of the following options: • `delays` • `checks` • `transitions`
`-path_adjust_only`	Exports `set_path_adjust_group` and `set_path_adjust` constraints.
`-path_group_only`	Exports `group_path` constraints.
`-pins <string>`	Specifies the name of the pins to be included in the SDC file.
`-strict_sdc`	Produces a SDC file where functions to manipulate collections are expanded. For example, the `remove_from_collection` `all_inputs`, `all_outputs`, `all_clocks`, and filter commands are replaced by the resulting list of objects. **Note:** When you specify the `-strict_sdc` parameter, a warning is issued stating that using the parameter produces a constraint file containing SDC standard constraints only. This may cause some design constraints to be dropped.
`-view` *`viewName`*	Specifies a view name to obtain the desired SDC.
`-vt_skew_derate_only`	When specified, the software will check the voltage threshold (VT) skew configuration, and report the VT skew derate details as per the voltage groups defined.

Command Order

Use this command after reading in a timing constraints file.

Examples

- The following command generates constraints file `newc` and saves the generated file in the directory `new`:

  ```
  tempus > write_sdc new/newc
  ```

- Specifying the following constraints will have new entries in the output constraints file:

  ```
  tempus > set_annotated_delay -from u1/A -to u1/Y -cell 2
  tempus > set_annotated_check 2 -setup -from f0/CK -to f0/D
  ```

  The `write_sdc` output will be as follows:

  ```
  set_annotated_delay -cell 2 -from [get_pins {u1/A}] -to [get_pins {u1/Y}]
  set_annotated_check -setup -from [get_pins {f0/CK}] -to [get_pins {f0/D}] 2
  ```

- For the following constraints in the design :

  ```
  tempus > set_annotated_delay -from u1/A -to u1/Y -cell 2
  tempus > set_annotated_transition 2 u1/A
  tempus > set_annotated_check 2 -setup -from f0/CK -to f0/D
  ```

  The `write_sdc -exclude_annotated delays` output will be as follows:

  ```
  set_annotated_transition 2 [get_pins {u1/A}]
  set_annotated_check -setup -from [get_pins {f0/CK}] -to [get_pins {f0/D}] 2
  ```

  The `write_sdc -exclude_annotated {delays transitions}` command outputs the following:

  ```
  set_annotated_check -setup -from [get_pins {f0/CK}] to [get_pins {f0/D}] 2
  ```

Related Commands

- `read_sdc`
- `write_loop_break_sdc`

write_sdf

```
write_sdf
[sdf_file]
[-gate_level_sim_model]
[-adjust_setuphold_for_zero_hold_slack]
[-base_delay]
[-cell_timing {all | none | nochecks}]
[-collapse_internal_pins]
[-infer_condelse]
[-delimiter char]
[-delta_delay]
[-edges {edged | library | noedge | check_edge}]
[-exclude_cells]
[-exclude_cells_keep_io_nets]
[-exclude_cells_keep_top_level_nets]
[-exclude_disabled_arcs]
[-exclude_disabled_modes]
[-exclude_whatif_arcs]
[-zero_negative_delays]
[-ideal_clock_network]
[-include_cells string]
[-include_variation]
[-infer_condelse]
[-interconnect {all | none | noport | noinport | nooutport}]
[-map_file {file1 file2 ...]
[-recrem [merge_always | split | merge_when_paired]]
[-map_removal_to_hold]
[-max_view viewName]
[-min_clock_si_threshold number]
[-min_data_si_threshold number]
[-min_period_edges {posedge | negedge | both | none}]
[-min_view viewName]
[-multi_drive_net_syntax string]
[-no_derate]
[-no_escape]
[-exclude_negative_checks]
[-path name]
[-precision non_neg_integer]
[-process]
[-recompute_delaycal]
[-recompute_parallel_arcs]
[-scale float]
[-setuphold [merge_always | split | merge_when_paired]]
```

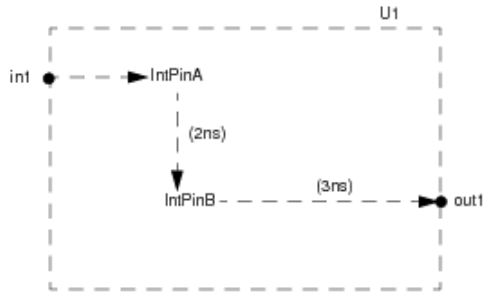
```
[-split_recrem]
[-split_setuphold]
[-target_application {sta | verilog}]
[-gate_level_sim_model]
[-temperature]
[-transform_out_to_out_arcs]
[-typical_view viewName]
[-version { 2.1 | 3.0 }]
[-view viewName]
[-voltage]
[-worst_case_illegal_cond_arcs string]
```

Writes delays to a Standard Delay Format (SDF) file. The `write_sdf` command can generate both plain-text and GNU zipped (.gz) SDF files.

By default, the `write_sdf` command uses the delays already cached by the system and writes only values of the triplets.

Parameters

`-help`	Prints out the command usage.
`-gate_level_sim_model`	Writes out the gate-level simulation (GLS) SDF file.
`--adjust_setuphold_for_zero_hold_slack`	Adjusts the hold timing check to meet non-negative slack values.
`-base_delay`	Writes out base delay information in the SDF file.
`-cell_timing {all | none | nochecks}`	Specifies which cell delays and timing checks to write out. *Default*: `all` • `all` - Writes all cell delays and timing checks to the SDF file. This is the default. • `nochecks` - Disables the writing of timing checks only. • `none` - Prevents cell delays and timing checks from being written into the SDF file.

`-collapse_internal_pins`	Collapses arcs across internal pins to a single primary input to primary output arc.
	
	Use the `-collapse_internal_pins` parameter when the SDF file being generated is intended for use by an application that does not support the same internal pin structure in its leaf cell libraries (for example, Cadence VXL and NCsim Verilog simulation products). Do *not* use this parameter to generate an SDF file intended for use within the Tempus flow, or by third party tools that utilize the same Liberty timing models.
	
	Default: Arcs between internal pins in a timing model are written to the SDF file as explicit annotations to and from the internal pins internal pins.
	
	 
	
	For example, assume the following arc between internal pins:
	
	The following SDF would be generated by default:
	`(U1/in1 U1/IntPinA (1::1) (1::1))`
	
	`(U1/IntPinA U1/IntPinB (2::2) (2::2))`
	
	`(U1/IntPinB U1/out1 (3::3) (3::3))`
	
	If you specify `-collapse_internal_pins`, the resulting SDF would show a single arc with the merged delays:
	`(U1/in1 U1/out1 (6::6) (6::6))`

`-infer_condelse`	Writes `CONDELSE` constructs with the default value when a `COND` construct is written. **Note:** Use this argument with the `-version 3.0` option.

`-delimiter` *char*	Specifies the hierarchy divider character to use in the output file. This option sets the hierarchy divider of only the `write_sdf` command. If omitted, the default hierarchy divider character (`/`) is used as the `write_sdf` default.
`-delta_delay`	Writes out delta delay values to the SDF file.
`-edges {edged | library | noedge | check_edge}`	Specifies the edges values. • `check_edge` - Keeps edge specifiers on timing check arcs but does not add edge specifiers on delay arcs. • `edged` - Keeps edge specifiers on timing check arcs and delay arcs. • `library` - Assumes that the *`sdf_edges`* attributes are specified in the `.lib` file. The default *`sdf_edges`* value of a timing arc is `noedge`. • `noedge` - Does not keep edge specifiers. *Default*: `edged` Refer to Table for full qualifications of the possible values for the edges option.
`-exclude_cells`	Specifies a list of leaf-cell or hierarchical instances whose timing data will be excluded from the the SDF (Standard Delay Format) file. If you specify a hierarchical instance, the timing data for all leaf-cells below the level of hierarchy will be omitted from the SDF file.
`-exclude_cells_keep_io_nets`	Restricts the filtering to just the annotations which are completely enclosed by the cell. Annotations to or from the cell will be preserved. By default, when the `-exclude_cells` parameter is used, any annotations within the excluded cell and any annotations to or from the excluded cell are filtered from the SDF.

`-exclude_cells_keep_top_level_nets`	Allows you to configure the handling of top-level interconnect annotations to an instance that is excluded from the SDF using the `write_sdf -exclude_cells` parameter. When this parameter is specified, the interconnect arcs which span to the ports of the instance will be included in the SDF. By default, no interconnect arcs to the instance are included. The delay annotated will be the worst-case delays of the top-level nets to/from the first level sink/source pins within the block that traverse a given input/output port of the instance, respectively.
`-exclude_disabled_arcs`	Excludes user set disabled arcs in the Standard Delay Format (SDF) file. If the user sets this option, the disabled arcs set by commands `set_mode`, `set_disable_timing`, and `set_case_analysis` are excluded.
`-exclude_disabled_modes`	Excludes timing arcs that have been disabled due to an active library mode set by the `set_mode` command, from appearing in the SDF file.
`-exclude_whatif_arcs`	Excludes what-if arcs in the SDF file.
`sdf_file`	Specifies the name of the SDF output file.
`-zero_negative_delays`	Converts all negative interconnect and `IOPATH` delays to zero.
`-ideal_clock_network`	Marks delays of delay arcs, that are on the ideal clock networks, as 0 in the SDF file.
`-include_cells` *`string`*	Specifies a list of leaf-cell or hierarchical instances to be only included in the SDF file.
`-include_variation`	When the selected analysis type is SOCV, the `-include_variation` parameter can be used to export SDF delays and check values, including the variation component. By default, the SDF delays and check values will not have the mean + N*sigma value.
`-infer_condelse`	Writes CONDELSE constructs with the default value when a COND construct is written.

`-interconnect {all | none | noport | noinport | nooutport}`	Specifies which interconnect delays to write. • `all` - Writes all `INTERCONN` delays into the SDF file. This is the default if the `-interconnect` option is not specified. • `none` - No `INTERCONN` delays are written into the SDF file. • `noport` - No top level port `INTERCONN` delays are written into the SDF file. • `nooutport` - Disables only the output port `INTERCONN` delays. *Default*: `all`
`-map_file {file1 file2 ...}`	Specifies a list of map files. The map files can be used to determine what is written into the SDF files. For example, a map file can provide delay information on internal pins which are not defined in the timing library. Syntax for a map file (for a cell) is as follows: • A template section (the text to output) that reference variables of the form $<number> • A variable definition section that tells the expression to evaluate to obtain the value

`-recrem [merge_always | split | merge_when_paired]`	Determines how `RECOVERY and REMOVAL` timing checks are written. • `merge_always` - Always writes combined `RECREM` checks, even if either a `RECOVERY`, or a `REMOVAL`, check does not exist. For example, if a `RECOVERY` check exists, but a `REMOVAL` check does not, the software writes a combined `RECREM` check in the SDF file as follows: RECREM (*value*) (*value*) () () • `split` - Splits the `RECREM` checks into a `RECOVERY` check and a `REMOVAL` check. • `merge_when_paired` - Writes combined `RECREM` checks only when both a `RECOVERY` and a `REMOVAL` check exist. For example, if both a `RECOVERY` and a `REMOVAL` check exist, the software writes a combined `RECREM` check in the SDF file as follows: RECREM (*value*) (*value*) (*value*) (*value*) However, if a `RECOVERY` check exists, but a `REMOVAL` check does not, the software only writes the `RECOVERY` check in the SDF file: RECOVERY (*value*) (*value*) *Default:* `merge_always` **Note**: The `RECREM` sum should be zero or greater than zero. A negative value will be adjusted to enforce this condition. This is an SDF v3.0 requirement (the software may issue a warning).

`-map_removal_to_hold`	Converts the SDF v2.1 unsupported `REMOVAL` (and the `REMOVAL` portion of the `RECREM` constructs) to `HOLD` checks when used with the `-version 2.1` option. *Default*: Not to write out the unsupported checks. Converts the `REMOVAL` checks to `HOLD` checks when combined with the `-version 3.0` option.The `RECREM` checks are split into a `RECOVERY` check and a `HOLD` check. This option automatically splits the `RECREM` checks. *Default*: To maintain the version 3.0 checks.
`-max_view` *viewName*	Uses the late delay from the specified analysis view to populate the SDF max slot, when in multi-mode multi-corner analysis mode.
`-min_clock_si_threshold` *number*	Specifies the absolute minimum delta delay change threshold on the clock path to be written into the SDF file. Time unit is in seconds. *Default*: 1e-20
`-min_data_si_threshold` *number*	Specifies the absolute minimum delta delay change threshold on the data path to be written into the SDF file. Time unit is in seconds. *Default*: 1e-20
`-min_period_edges {posedge | negedge | both | none}`	Specifies how to write out minimum `PERIOD` checks. Two check arcs are always created for the `min_period` check when you read in a `.lib` file. The following options let you keep or remove the positive edge, the negative edge, both edges, or no edges from the PERIOD statement in the `.sdf` file. • `posedge` - Writes out one positive edge `PERIOD` check. • `negedge` - Writes out one negative edge `PERIOD` check. • `both` - Writes out both the positive edge and the negative edge `PERIOD` checks. This is the default. • `none` - Writes out one `PERIOD` check without any edge specifier.
`-min_view` *viewName*	Uses the early delay from the specified analysis view to populate the SDF `min` slot, when in multi-mode multi-corner analysis mode.

`-multi_drive_net_syntax` *string*	Specifies the multi-drive nets as ports or interconnects. You can use the `interconnect` and `port` options if you need to define the net delays of multi-driven nets (clock mesh) as interconnects or ports respectively in the SDF.
`-no_derate`	Writes non-derated delays to the Standard Delay Format (SDF) file. By default, the `set_timing_derate` command writes scaled delays to the SDF file. When the `-no_derate` parameter is specified, the `write_sdf` command removes all the applied derates.
`-no_escape`	Writes out object names without escaping hierarchical dividers. Normally, when writing out a SDF file, the hierarchical divider (/) is escaped because it is a special character.
`-exclude_negative_checks`	Converts all negative timing check values to `0.0`.
`-path` *name*	Specifies the sub-block name.
`-precision` *non_neg_integer*	Controls the number of digits appearing after the decimal point in the output SDF file. The default precision is the value of the `report_precision`.

`-process`	Specifies one or a combination of the following process names that will appear in the header of the generated SDF file:
	• `best`
	• `typical`
	• `worst`
	Use the colon delimiter to specify multiple process names. For example:
	`-process best:typical:worst`
	In the above example, the header of the generated SDF file will contain:
	`(PROCESS "best:typical:worst")`

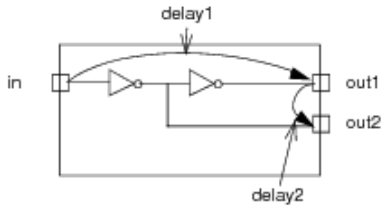
`-recompute_delaycal`	Instructs the software to recompute any necessary data required for a complete SDF file before exporting. By default, only the existing data that has been computed for STA is exported to the SDF. You must use this parameter to ensure that the SDF file is complete for general use in other applications.

`-recompute_parallel_arcs`	Instructs the software to temporarily recompute delays with separate delay calculation for parallel arcs. The software normally operates in a high performance mode which optimizes the handling of multiple timing arcs between input and output pin pairs. By default, the SDF file generated will contain the same delay information for all of these arcs. To have the SDF recalculated with explicit pin pair data, you should use the option `-recompute_parallel_arcs` to cause delays for each parallel arc to be explicitly calculated. It is recommended to use this option for generating SDF for functional simulation applications. **Note**: When you use the `write_sdf -recompute_parallel_arcs` option timing analysis is performed again. You need to obtain a new delay calculation to generate the SDF, and another delay calculation to put the system back to where it was before `write_sdf` was specified. Since delay calculation is run again, timing analysis will be reset. **Note**: The `-recompute_parallel_arcs` parameter is obsolete. This parameter still works in this release, but to ensure compatibility with future releases update your scripts to use the `-recompute_delaycal` parameter instead.
`-scale` *float*	Specifies a multiplier to the delay values used in the SDF file. The delay values are scaled, but the units are the same. *Default*: `1`

`-setuphold [merge_always | split | merge_when_paired]`	Determines how `SETUP and HOLD` timing checks are written. • `merge_always` - Always writes combined `SETUPHOLD` checks, even if either a `SETUP` or a `HOLD`, check does not exist. For example, if a `SETUP` check exists, but a `HOLD` check does not, the software writes a combined `SETUPHOLD` check in the SDF file as follows: SETUPHOLD (*value*) (*value*) () () • `split` - Splits the `SETUPHOLD` timing checks into separate `SETUP` and `HOLD` checks. • `merge_when_paired` - Writes combined `SETUPHOLD` checks only when both a `SETUP` and a `HOLD` check exist. For example, if both a `SETUP` and a `HOLD` check exist, the software writes a combined `SETUPHOLD` check in the SDF file as follows: SETUPHOLD (*value*) (*value*) (*value*) (*value*) However, if a `SETUP` check exists, but a `HOLD` check does not, the software only writes the `SETUP` check in the SDF file: SETUP (*value*) (*value*) *Default:* `merge_always`
`-split_recrem`	Splits the `RECREM` checks into a `RECOVERY` check and a `REMOVAL` check. *Default*: To write the combined `RECREM` check. **Note:** Use this option only with the `version 3.0` option.

`-split_setuphold`	Splits the `SETUPHOLD` timing checks into separate `SETUP` and `HOLD` checks. *Default*: To write the combined `SETUPHOLD` checks. **Note**: In SDC v3.0, the negative values are illegal in `SETUP`, `HOLD`, `RECOVERY`, and `REMOVAL` checks. Therefore, the negative values are converted to zero automatically with the `-split_setuphold` and `-split_recrem` parameters.
`-target_application { sta | verilog}`	Allows customization of the SDF output for use with STA or Verilog applications. You can select the following options: • `sta`: You can use the `sta` option to configure the SDF output for use with static-timing analysis (STA) applications. This includes sign-off timing analysis with products, such as Cadence Tempus, as well as for working with implementation flows that use STA for internal timing analysis. The `sta` option preserves maximum accuracy and generates SDF output that matches the structure of the Liberty timing libraries. This option is equivalent to the following options: `write_sdf -interconnect all, -cell_timing all, -edges edged, -setuphold merge_when_paired`, and `-recrem merge_when_paired`. • `verilog`: You can use the `verilog` option to generate SDF output that is compatible with Cadence Verilog-XL or Verilog-NC simulators. This option is equivalent to the following options: `write_sdf -zero_negative_delays -edges check_edge, -setuphold merge_when_paired, -recrem merge_when_paired, -min_period_edges -none, -collapse_internal_pins`, and `-recompute_parallel_arcs`. When the `-target_application` option is not specified then by default SDF output is configured for use with STA applications.
`-gate_level_sim_model`	Writes out the GLS SDF file.

`-temperature`	Defines the temperature values that will appear in the header of the generated SDF file.
	Use the colon delimiter to specify multiple temperature values. For example:
	`-temperature -40.0:25.0:125`
	In the above example, the header of the generated SDF file will contain:
	`(TEMPERATURE -40.0:25.0:125)`

`-transform_out_to_out_arcs`	Writes out all output-to-output timing arcs as input-to-output arcs. Input-to-output arc delays are computed by adding the delay from the input pin to one of the output pins plus the delay from that output pin to the other output pin. For instance, in the following figure, the delay between `in` and `out2` would be the sum of `delay1` and `delay2`. 
`-typical_view` *viewName*	Uses the late delay from the specified analysis view to populate the SDF `typ` slot, when in multi-mode multi-corner analysis mode.
`-version {2.1 | 3.0}`	Specifies whether to generate the SDF V2.1 or V3.0. The default SDF version is 3.0. Used in combination with the `-map_removal_to_hold` option.
`-view` *viewName*	Uses the early and late delays for the specified analysis view to populate the `min` and `max` SDF triplet slots, when in multi-mode multi-corner analysis mode. The SDF `typ` slot remains empty (*earlyVal* `::` *lateVal*). In multi-mode multi-corner analysis mode, if you want to use SPEF information when calculating the delays, you must use `read_spef -rc_corner` to import the SPEF files for each RC corner in the design.

`-voltage`	Defines the voltage values that will appear in the header of the generated SDF file.
	Use the colon delimiter to specify multiple voltage values. For example:
	`-voltage 1.980:1.800:1.620`
	In the above example, the header of the generated SDF file will contain:
	`(VOLTAGE 1.980:1.800:1.620)`

`-worst_case_illegal_cond_arcs` *string*	Specifies the sdf_cond - all, none, delays, or checks - for the arcs. When this parameter is specified, the `write_sdf` command detects illegal SDF condition syntax coming from the Liberty library and generates an unconditional annotation in the SDF file that worst-cases the delays of the related timing arcs. This prevents downstream tools (such as, Verilog simulators) from complaining about illegal syntax in the SDF file.

Definition of -edges Option Values

	edged Value	**libraryValue**	**noedgeValue**

IOPATH from a unate input pin to output pin	No edge specifier	No edge specifier	No edge specifier
IOPATH from a non-unate input pin to output pin	Edge specifier at the input pin	Edge specifier at the input pin when `sdf_edges` is ei	No edge specifier When merging edged paths, the m

		either "both_edges" or "start_edge"	aximum values are used for MAX and TYP fields; minimum values are used

			for MIN fields.
IOPATH of a clock/enable pin to data out pin	Edge specifier at the input pin	Edge specifier at the input pin when `sdf_edges` is either "	No edge specifier

		`both_edges"` or `"start_edge"`	
Timing Checks	Edge specifiers match the check arcs specified in the timing library	Overridden by value of the `sdf_ed`	Only the edge specifiers of the referen

		ges attribute For both_edges, the timing checks will contain edges specified in the	cepin are written

cifieronbothstartingandendingpins

For `start_edge`, the starting pin

(for HOLD type check) or ending pin (for SETUP type check) is edge-sp

			ecified For `end_edge`, the ending pin (for HOLD stylecheck) or the starti		

			ngpin (for SETUP type check) is edge-specified The timing checks a		

redundant or combined appropriately. The start_edge and end_

edge are interpreted as first and second events, respectively, of the Veril

		o g - X L' s $ h ol d () a n d $ s et u p s y st e m ta s k s	

Examples

- The following command writes out an SDF 2.1 compliant output file named *my.sdf* with 4 digits after the decimal point:

  ```
  tempus> write_sdf -version 2.1 -precision 4 my.sdf
  ```

  The following SDF file output shows a comparison between the output of `-edges edged` with `-edges noedge`:

  ```
  (HOLD (posedge D) (posedge CK) (-0.14:-0.14:-0.14))
  (HOLD (negedge D) (posedge CK) (-0.06:-0.06:-0.06))
  (SETUP (posedge D) (posedge CK) (0.22:0.22:0.22))
  (SETUP (negedge D) (posedge CK) (0.41:0.41:0.41))
  ```

  When the `-edges` parameter is set to `noedge`, only the edge specifiers of the clock pin will be written, as shown below:

  ```
  (HOLD D (posedge CK) (-0.14:-0.06:-0.06))
  (SETUP D (posedge CK) (0.22:0.41:0.41))
  ```

- The following command writes out the delta delay values for the analysis view to the specified SDF file:

  ```
  write_sdf -delta_delay -view view1 tst.sdf
  ```

- The following command writes non-derated delays to the specified SDF file:

  ```
  write_sdf tst.sdf -no_derate
  ```

Related Commands

- `read_sdf`

write_timing_context

```
write_timing_context
[-help]
[dir_name]
[-instances instance_list]
-module module_name
[-overwrite]
[-skip_interface_paths]
```

Writes the timing context model of the specified lower level modules/instances in the given output directory. Within this directory, Tempus creates sub-directories for each of the specified modules and writes the context model of each module within the corresponding sub-directory.

Parameters

`-help`	Prints out the command usage.
dir_name	Specifies the name of the output directory.
`-instances` *instance_list*	Specifies the instance list. If not specified, then context of all the instances of modules (specified with the `-module` parameter) will be generated.
`-module` *module_name*	Specifies the name of the module. You can use this parameter to write timing context information or the required lower level block modules. The timing context information for multiple modules can be written by specifying more than one module name.
`-overwrite`	Allows the software to overwrite any existing context directory data. If the context output directory already exists, the software will issue an error.
`-skip_interface_paths`	Skips the interface paths. However, the context data impacting only the internal reg-reg paths is preserved. During context analysis, the interface paths will be unconstrained. In DSTA, it is required to provide the `-skip_interface_paths` parameter. If not specified, a warning message will be issued.

Examples

- The following command will write the context model of block module `tdsp_core` and `results_conv` in respective sub-directories under the context directory `context/tdsp_core context/results_conv`:

  ```
  write_timing_context -module {tdsp_core results_conv} context
  ```

  Since the `-instance` parameter is not specified with the `write_timing_context` command, the output directory will contain context models of all the instances of `tdsp_core` and `results_conv` modules.

- The following command will write context for tdsp_core_inst1 instance of tdsp_core module and all the instances of results_conv module.

  ```
  write_timing_context -module {tdsp_core results_conv} -instances {tdsp_core_inst1} context
  ```

Related Commands

- `read_timing_context`
- `set_timing_context_view_mapping`
- `report_timing_context_clock_mapping`
- `compare_timing_context`

write_timing_windows

```
write_timing_windows
[-help]
out_file
[-pin]
[-power_compatible]
[-ssta_sigma_multiplier value]
[-view viewName]
[-voltage_threshold {lower_threshold upper_threshold}]
```

Generates a Timing Window File (TWF). The TWF file mainly contains the earliest and the latest possible arrival times that a signal may arrive on a net or a pin.

Parameters

`-help`	Prints out the command usage.
out_file	Writes the TWF file to the specified file name. The *filename* parameter must be the last argument in the list. **Note:** To write a compressed file, add the `.gz` extension to the file name.
`-pin`	Generates a TWF for each pin instead of each net. *Default*: Generates a TWF for the crosstalk analysis tool, which contains timing windows on each net.
`-power_compatible`	Generates a timing windows file that is suitable for power analysis.
`-ssta_sigma_multiplier` *value*	Specifies a sigma multiplier between `0` and `3`. The value that you specify increases the arrival window by that amount and worst slack value is generated. The transition and impedance values are not impacted.

`-view` *viewName*	Generates a TWF for the specified multi-mode multi-corner view only. You must be multi-mode multi-corner analysis mode to use this parameter. If the `-view` parameter is not specified, then by default the first active view will be used to generate timing windows.
`-voltage_threshold` {*lower_thresholdupper_threshold*}	Specifies voltage threshold values for generating a Timing Window File (TWF). All the slews in the TWF file will be scaled with these values. The lower threshold value must be less than the higher threshold value. *Value range:* `0` to `100`

Examples

- The `write_timing_windows -voltage_threshold {20 80}` command outputs the following in the TWF file:

```
(VOLTAGE_THRESHOLD 20 80)
```

- When active clocks are set using the `set_active_clocks` command, the `write_timing_windows` command will not display windows for inactive clocks in the output. The example report is shown below:

```
(TIMING_WINDOWS
(HEADER
(VERSION "0.1")
(DATE "Tue Apr 2 23:07:58 2013")
(DESIGN "top")
(DELIMITERS "/[]")
(VOLTAGE_THRESHOLD 20 80)
(PROCESS 1.000 1.000 1.000)
(VOLTAGE 1.012 1.012 1.012)
(TEMPERATURE 125.000 125.000 125.000)
(OPERATING_CONDITIONS DEFAULT DEFAULT DEFAULT)
(TIME_SCALE 1.000E-12)
(CAP_SCALE 1.000E-12)
(RESISTANCE_SCALE 1.000E+00)
(DEFAULT_INPUT_SLEW 0.000000 0.000000 0.000000 0.000000)
(PROGRAM "novus" "v13.10-dev.festa ((64bit) 04/02/2013 23:24 (Linux 2.6))" "Cadence")
)
(WAVEFORM "Iclk1" 10000.000000 (POSEDGE 0.000000) (NEGEDGE 5000.000000))
(WAVEFORM "Iclk2" 15000.000000 (POSEDGE 0.000000) (NEGEDGE 7500.000000))
(WAVEFORM "Iclk3" 15000.000000 (POSEDGE 0.000000) (NEGEDGE 7500.000000))
(WAVEFORM "Iclk4" 10000.000000 (POSEDGE 0.000000) (NEGEDGE 5000.000000))
(CAUSED_BY "Iclk1" RISE
```

```
(NET "in1" 0:200 120:120 * 9200 0:200 120:120 * 9200 D)
)
(CAUSED_BY NULL
(NET "sel" 0:200 120:120 * * 0:200 120:120 * * D)
)
(CAUSED_BY "Iclk4" RISE
(NET "clk1" 0:200 120:120 * * * 120:120 * * D)
)
(CAUSED_BY "Iclk4" FALL
(NET "clk1" * 120:120 * * 5000:5200 120:120 * * D)
)
(CAUSED_BY "Iclk1" RISE
(NET "clk1" 0:200 120:120 * * * 120:120 * * D)
)
(CAUSED_BY "Iclk1" FALL
(NET "clk1" * 120:120 * * 5000:5200 120:120 * * D)
)
(CAUSED_BY "Iclk1" RISE
(NET "clk1" 0:200 120:120 * * 5000:5200 120:120 * * C)
)
(CAUSED_BY "Iclk4" RISE
(NET "clk1" 0:0 120:120 * * 5000:5000 120:120 * * C)
)
(CAUSED_BY NULL
(NET "clk2" 0:200 120:120 * * 0:200 120:120 * * D)
)
(CAUSED_BY "Iclk1" RISE
(NET "out1" 1200:2600 12.7:12.7 * 1300:7400 1200:2600 10.8:10.8 * 1300:7400 D)
)
(CAUSED_BY "Iclk4" RISE
(NET "out1" 300:800 12.7:12.7 * 400:9200 300:800 10.8:10.8 * 400:9200 D)
```

Related Commands

- write_twf
- read_twf

write_twf

```
write_twf
-help
outfile
[-exclude_instances string]
[-format string]
[-include_instances string]
[-include_lib_cells string]
[-pin]
[-power_compatible]
[-ssta_sigma_multiplier float]
[-third_party_file_format string]
[-view string]
[-voltage_threshold string]
```

Generates a timing window file (TWF). The TWF file mainly contains the earliest and the latest possible arrival times that a signal may arrive on a net or a pin.

The format of a timing window file consists of eight pairs of data as given below:

```
(NET <netname>

minRTW:maxRTW minRt:maxRt minRDr:maxRDr minRSlk:maxRSlk minFTW:maxFTW minFt:maxFt minFDr:maxFDr
minFSlk:maxFSlk)
```

Where:

- `minRTW`: Minimum rise arrival time
- `maxRTW`: Maximum rise arrival time
- `minRt`: Minimum rise transition time
- `maxRt`: Maximum rise transition time
- `minRDr`: Minimum rise source resistance
- `maxRDr`: Maximum rise source resistance
- `minRSlk`: Minimum rise slack time
- `maxRSlk`: Maximum rise slack time
- `minFTW`: Minimum fall timing window
- `maxFTW`: Maximum fall timing window
- `minFt`: Minimum fall transition time
- `maxFt`: Minimum fall transition time
- `minFDr`: Minimum fall source resistance
- `maxFDr`: Maximum fall source resistance

- `minFSlk`: Minimum fall slack time
- `maxFSlk`: Maximum fall slack time

Parameters

`-help`	Prints out the command usage.
`-exclude_instances` *`string`*	Specifies a list of hierarchical instances to be excluded from the TWF file.
`-format` *`string`*	Specifies the name of the format to be used for TWF generation. **Note**: This parameter supports third-party TWF generation.
`-include_instances` *`string`*	Specifies a list of hierarchical instances to be included in the TWF file.
`-include_lib_cells` *`string`*	Specifies the library cells that require the input timing window.
`outfile`	Specifies the name of the output file. The *`out_file`* parameter must be the last argument in the list. To write a compressed file, add the `.gz` extension to the file name. **Note**: This parameter supports third-party TWF generation.
`-pin`	Generates a TWF for each pin instead of each net. *Default*: Generates a TWF for the crosstalk analysis tool, which contains timing windows on each net.
`-power_compatible`	Generates a timing window file (TWF) that is suitable for Power Analysis.
`-ssta_sigma_multiplier` *`float`*	Specifies a sigma multiplier between `0` and `3`. The value that you specify increases the arrival window by that amount and worst slack value is generated. The transition and impedance values are not impacted.
`-third_party_file_format` *`string`*	Specifies the name of the third-party file format to be used during TWF generation. You can specify <ir_drop> or <jitter> format option. **Note**: This parameter supports third-party TWF generation.
`-view` *`string`*	Specifies name of the analysis view. **Note**: This parameter supports third-party TWF generation.

`-voltage_threshold` *string*	Specifies voltage threshold values for generating a Timing Window File (TWF). All the slews in the TWF file will be scaled to these values. The lower threshold value must be less than the higher threshold value. *Value range:* `0` to `100`

Examples

- The `write_twf -voltage_threshold {20 80}` command writes the following in the TWF file:

  ```
  (VOLTAGE_THRESHOLD 20 80)
  ```

- The following example shows third-party TWF report with IR drop option:

  ```
    I m_clk__I10
  S A 0.508 0.509 0.175 0.195
  S Z 0.077 0.064 0.054 0.054
  K Z -- -- -- --
  T Z 1 -- -- 2.353 2.280 3 f
  T Z 1 0.374 0.268 2.353 2.280 3 r
  ```

  Third-party TWF report with Jitter option is shown below:

  ```
     I m_clk__I10
  T Z 1 -- -- 2.353 2.280 3 f
  T Z 1 0.374 0.268 2.353 2.280 3 r
  ```

  Where I - Instance, S - slew data, and K - slack data.

Related Commands

- `read_twf`
- `write_timing_windows`

27

Distributed Static Timing Analysis Commands

- distribute_launch_deferred_clients
- distribute_partition
- distribute_print_client_usage
- distribute_print_partition_stats
- distribute_start_clients

distribute_launch_deferred_clients

```
distribute_launch_deferred_clients
[-help]
```

Launches the clients and waits for them to become available.

When the `distribute_start_clients -defer_launch` parameter is specified, the clients are not launched until the launch command is run.

The `distribute_launch_deferred_clients` command can be run at any point after `distribute_start_clients -defer_launch` option is specified, but should be specified no later than immediately after the `distribute_partition` command. Most commands after this point will require clients to be running.

Parameters

`-help`	Prints out the command usage.

Related Commands

- `distribute_start_clients`
- `distribute_partition`

distribute_partition

```
distribute_partition
[-help]
[-block_list string]
[-cell string]
[-cell_nohier string]
[-exclude_block_list string]
[-instance string]
[-instance_nohier string]
```

Controls the method by which design partitioning is performed, and determines the scope of DSTA. This command tells the master process to start using the clients for partitioning. It is recommended to run the command after the parasitics are read in, thereby, enabling constraints, timing updates and reports to be shared between all the clients.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `distribute_partition` parameter. For a detailed description of the command and all of its parameters, use the man command: `man distribute_partition`.
`-block_list` *string*	Specifies the list of blocks for partitioning. It lists the blocks that are to be analyzed in the incremental flow. This enables block-scope analysis in DSTA, therefore, only the specified blocks are analyzed in the run and all the other blocks are ignored.
`-cell` *string*	Specifies a list of instances of cells for partitioning.
`-cell_nohier` *string*	Specifies a list of instances of cells for partitioning excluding hierarchical children.
`-exclude_block_list` *block_names*	Specifies the list of blocks to exclude from the DSTA analysis in the incremental flow. This enables top-scope analysis in DSTA. All blocks that are not in this list are analyzed, along with the top-level of the design.
`-instance` *string*	Specifies a list of instances for partitioning.
`-instance_nohier` *string*	Specifies a list of instances for partitioning, excluding hierarchical children.

Examples

The following command specifies to perform a full-flat (non-incremental) flow:

```
dsta> distribute_partition
```

Related Commands

- `distribute_launch_deferred_clients`
- `distribute_start_clients`

distribute_print_client_usage

```
distribute_print_client_usage
[-help]
```

Reports CPU time, real time, current memory, and peak memory for all the clients. This command must be used instead of `report_resource` which is not capable of running in DSTA mode.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `distribute_print_client_usage` parameter. For a detailed description of the command and all of its parameters, use the man command: `man distribute_print_client_usage`.

Related Commands

- `distribute_start_clients`
- `distribute_launch_deferred_clients`
- `distribute_partition`

distribute_print_partition_stats

```
distribute_print_partition_stats
[-help]
[-all]
[-counts]
[-resources]
[-tcl_list]
```

Provides an estimate of the client memory requirements after partitioning, based on the instance count for each partition.

This command prints the partition statistics and estimated resource requirements after partitioning.

Parameters

`-help`	Prints out the command usage.
`-all`	Prints out instance and net counts by partition, overlap statistics, and resource requirements.
`-counts`	Prints out the instance and net counts by partition and overlap statistics.
`-resources`	Prints out the estimated memory and temp file size requirement by partition. This is the default behavior.
`-tcl_list`	Returns the requested statistics in a TCL list form. This parameter is supported with -resources.

Examples

- The following command shows the client resources:

```
distribute_print_partition_stats -resources

Active analysis views: 1
Partition 0 Estimated Memory: 50 GB. Estimated /tmp space: 60 GB.
Partition 1 Estimated Memory: 40 GB. Estimated /tmp space: 48 GB.
Partition 2 Estimated Memory: 50 GB. Estimated /tmp space: 60 GB.
Partition 3 Estimated Memory: 40 GB. Estimated /tmp space: 48 GB.
```

- The following command shows the instance count for each partition:

```
distribute_print_partition_stats -counts

Partition Stats:
Partition 0 instance-count: 13654609 extended instance-count: 4331968 (clustered:
4063874) outside clock instance-count: 320793
Partition 1 instance-count: 12983662 extended instance-count: 1153506 (clustered:
1045248) outside clock instance-count: 738683
Partition 2 instance-count: 14100713 extended instance-count: 4345271 (clustered:
4170905) outside clock instance-count: 308583
Partition 3 instance-count: 12959154 extended instance-count: 1166173 (clustered:
1064585) outside clock instance-count: 757964
Partition 0 net-count: 15498000 extended net-count: 1288107 (clustered: 1137204)
outside clock net-count: 428712
Partition 1 net-count: 14730101 extended net-count: 345115 (clustered: 297528)
outside clock net-count: 787647
Partition 2 net-count: 15970893 extended net-count: 1293759 (clustered: 1191587)
outside clock net-count: 419663
Partition 3 net-count: 14751488 extended net-count: 350045 (clustered: 306878)
outside clock net-count: 807687

Total nets: 59753864
Total partitioned nets: 59751874
Total instances: 52467900
Total partitioned instances: 52467832
Primary instance overlap: 2.34488%
Full instance overlap: 27.3563%(clustered: 26.113%)
```

  When `-all` is specified, both the instance counts as well as client resources will be printed.

Related Commands

- distribute_partition
- distribute_start_clients
- distribute_launch_deferred_clients
- distribute_partition

distribute_start_clients

```
distribute_start_clients
[-help]
[-client_nowait]
[-count integer]
[-defer_launch | -minHost integer]
[-no_log]
[-no_logv]
[-outdir string]
[-overwrite]
[-startup_script <string>]
```

Starts Tempus on distributed clients. This command starts the client processes. It requests LSF or SGE for a machine and starts to create a copy of Tempus on it. These client machines will be given tasks to do by the master machine.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `distribute_start_clients` parameter. For a detailed description of the command and all of its parameters, use the man command: `man distribute_start_clients`.
`-client_nowait`	Specifies whether the master machine must wait for all the clients to be launched. When this parameter is specified, the master machine will continue to process, and will not wait for all the clients to be launched. However, the master will wait until partitioning occurs. **Note**: You cannot specify this parameter with the `-minHost` or `-defer_launch` parameter.
`-count` *integer*	Specifies the number of clients. This parameter is used to set the number of clients when this value is not specified by the `set_multi_cpu_usage` command.
`-defer_launch`	Defers the launch of any clients unless the `distribute_launch_deferred_clients` command is issued explicitly. **Note**: This parameter cannot be used with the `-minHost` parameter.
`-minHost` *integer*	Specifies the minimum number of available remote hosts (servers) that can be accepted.
`-no_log`	Does not allow the software to create a client log file.
`-no_logv`	Does not allow the software to create a verbose client log file.
`-outdir` *string*	Specifies the base name of the client output directory.
`-overwrite`	Overwrites an existing .log file (and associated .cmd and .logv files), and does not add an integer to make the files unique.
`-startup_script <string>`	When specified, the software will source the flowtool file on clients for attribute and object initialization.

Examples

- The following command starts client processes on 4 distributed clients:

```
dsta> distribute_start_clients -count 4 -outdir run1_output_dir
```

Related Commands

- distribute_launch_deferred_clients
- distribute_partition

28

Timing Constraint Commands

- create_clock
- create_generated_clock
- current_design
- current_instance
- get_current_constraint_mode
- group_path
- reset_annotated_check
- reset_annotated_delay
- reset_annotated_transition
- reset_aocv_stage_weight
- reset_case_analysis
- reset_clock
- reset_clock_exclusivity
- reset_clock_gating_check
- reset_clock_groups
- reset_clock_latency
- reset_clock_sense
- reset_clock_transition
- reset_clock_tree_latency
- reset_clock_uncertainty
- reset_data_check
- reset_disable_clock_gating_check
- reset_disable_timing
- reset_drive
- reset_driving_cell
- reset_generated_clock
- reset_ideal_latency

- reset_ideal_network
- reset_ideal_transition
- reset_input_delay
- reset_load
- reset_max_capacitance
- reset_max_fanout
- reset_max_time_borrow
- reset_max_transition
- reset_min_capacitance
- reset_min_fanout
- reset_min_pulse_width
- reset_min_transition
- reset_mode
- reset_output_delay
- reset_path_exception
- reset_path_group
- reset_pll_timing
- reset_propagated_clock
- reset_property
- reset_pulse_clock_max_transition
- reset_pulse_clock_max_width
- reset_pulse_clock_min_transition
- reset_pulse_clock_min_width
- reset_resistance
- reset_sense
- set_active_clocks
- set_annotated_check
- set_annotated_delay
- set_annotated_transition
- set_aocv_interface_path_offset
- set_aocv_stage_weight
- set_aocv_thresholds

- set_case_analysis
- set_clock_exclusivity
- set_clock_gating_check
- set_clock_groups
- set_clock_latency
- set_clock_sense
- set_clock_transition
- set_clock_uncertainty
- set_data_check
- set_disable_clock_gating_check
- set_disable_timing
- set_dont_touch
- set_dont_touch_network
- set_dont_use
- set_drive
- set_driving_cell
- set_false_path
- set_fanout_load
- set_ideal_latency
- set_ideal_network
- set_ideal_transition
- set_input_delay
- set_input_transition
- set_load
- set_logic_one
- set_logic_zero
- set_max_capacitance
- set_max_delay
- set_max_fanout
- set_max_time_borrow
- set_max_transition
- set_min_capacitance

- set_min_delay
- set_min_fanout
- set_min_pulse_width
- set_min_transition
- set_mode
- set_multicycle_path
- set_normalized_driver_waveform_lib
- set_output_delay
- set_pll_timing
- set_propagated_clock
- set_property
- set_pulse_clock_max_transition
- set_pulse_clock_max_width
- set_pulse_clock_min_transition
- set_pulse_clock_min_width
- set_resistance
- set_sense
- write_annotated_transition
- write_load
- Path Exception Priorities - Overview
  - Priority Rules on the Basis of Path Exception Type
  - Priority Rules on the Basis of Path Exception Command Reference Points
  - Priority Rules on the Basis of -start/-end (only for set_multicycle_path)
  - Path Exception Priority Rules Table
  - Examples of Path Exception Priorities
- Bidirectional Pin Defaults
- Cycle Accounting Rules - Data Checks
  - General Single-Cycle Timing
  - Zero-Cycle Timing
  - Zero-Cycle Timing with Multi-Cycle Adjust
  - set_multicycle_path -start

Path Exception Priorities - Overview

This section describes the path exception priority rules, which are followed by Tempus to determine the effective path exceptions to be applied on a particular path if multiple path exceptions are applied to it. The path exception priority rules are explained as per the path exception command type and various command parameters. The path exception commands are:

- `set_false_path`
- `set_max_delay`
- `set_min_delay`
- `set_multicycle_path`

Note: The priority rules (as explained in the following sections) are applicable to `set_path_adjust_group` and `group_path` commands also.

Priority Rules on the Basis of Path Exception Type

If the types of path exception commands are different, the priority order (in decreasing priority) is as follows:

1. `set_false_path`
2. `set_min_delay`/`set_max_delay`
3. `set_multicycle_path`

For example, `set_false_path -from A` is given priority over `set_multicyle_path -from A 2`.

Priority Rules on the Basis of Path Exception Command Reference Points

The path exception commands use parameters to specify reference points, such as, `-from`/`-through`/`-to`/`-clock_from`/`-clock_to`.

If there is more than one exception of a given type, for example the `set_multicycle_path` command, the path exception that is more specific has higher priority. A path exception has higher priority if it specifies more of the `-from`/`-through`/`-to` parameters. For example, the `-from -to` options will have priority over the `-from` option.

Here is an example of path exception priorities for `-from`/`-through`/-to parameters:

Priority	Path Exception
1 (highest)	`set_multicycle_path -from` *pins* `-through` *pins* `-to` *pins*

2	`set_multicycle_path -from pins -to pins`
3	`set_multicycle_path -from pins -through pins -to clocks`

If the path has the same number of reference points:

- `-from` and `-to` option has priority over the `-from` option
- `-from` option has priority over the `-to` option
- `-to` option has priority over the `-through` option

For example:

i. `set_multicycle_path -from A 2` is given priority over `set_multicyle_path -through B 4`.

ii. `set_false_path -from A` is given priority over `set_false_path -through B`.

2. If there are more than one exceptions of a given type with the same reference point, the path exception that is more specific has a higher priority. A path exception is more specific if it specifies a longer path than the other. For example, the '`-from -to`' parameters will have a higher priority over `-from`.

3. If a path exception differs only in terms of number of `-through` commands specified, the exception with a more constraining value will be honored.

Priority Rules on the Basis of -start/-end (only for set_multicycle_path)

For the `set_multicycle_path` constraint, if the reference points are the same and both the constraints differ only in `-start`/`-end`, the precedence rule is as follows:

- Command with `-start` will be higher priority, whereas `-end` will be lower priority.

Path Exception Priority Rules Table

The following table shows the priorities for path exceptions applied to the same path:

Path Exception Priorities

Priority	Path Exception

1. (Highest)	`set_false_path`
	`set_false_path -from pin_list`
	`set_false_path -to pin_list`
	`set_false_path -through pin_list`

2.	`set_max_delay -from pin_list`

3.	`set_max_delay -to pin_list`
4.	`set_max_delay -through pin_list`
5.	`set_max_delay -from clkwave_name`
6.	`set_max_delay -to clkwave_name`
7.	`set_max_delay` (The most constraining adjustment has the higher priority over less constraining adjustments.)
8.	`set_multicycle_path -from pin_list -to pin_list`
9.	`set_multicycle_path -from pin_list`
10.	`set_multicycle_path -to pin_list`
11.	`set_multicycle_path -through pin_list`
12.	`set_multicycle_path -from clkwave_name`
13.	`set_multicycle_path -to clkwave_name`
14. (Lowest)	`set_multicycle_path` (The most constraining adjustment has the higher priority over less constraining adjustments.)

Note: To check for applied/ignored path exceptions, you can use the following command:

```
report_timing -path_exceptions <applied | ignored | all>
```

To report all the path exceptions that have been ignored by timing analysis, you can use the following command:

```
report_path_exceptions [-ignored]
```

Examples of Path Exception Priorities

The following are examples of the path exception priorities for a path starting at `A`, going through `B`, and ending with `C`. The pairs of examples are ordered such that the highest priority constraint in one pair has precedence over the highest priority constraint in the next pair.

- In the following pair of constraints, the false path overrides the cycle addition for the path from `A` through `B` to `C`. For all other paths from `A`, the cycle addition applies.

  ```
  set_false_path -from A -through B -to C
  set_multicycle_path -from A 2
  ```

- In the following pair, the first constraint is applied (two cycles added) because the `-from` option has priority over the `-to` option:

  ```
  set_multicycle_path -from A 3
  ```

```
set_multicycle_path -to C 2
```

- In the following pair, the first constraint is applied (two cycles added) because it specifies a `-through` pin as well as a `-to` pin:

  ```
  set_multicycle_path -through A -to B 3
  set_multicycle_path -to B 2
  ```

- In the following pair, the first constraint is applied (two cycles added) because the `-from` option has precedence over the `-to` option:

  ```
  set_multicycle_path -from CLKA 3
  set_multicycle_path -to CLKB 2
  ```

- In the following pair, the second constraint is applied because the second constraint overwrites the first one:

  ```
  set_multicycle_path -from A 2
  set_multicycle_path -from A 3
  ```

Bidirectional Pin Defaults

Bidirectional pins have both an input pin and an output pin type. This section explains the default pin type that the system uses for bidirectional pins.

Defaults for Pin (or Port) Constraints

The following defines the pin and port types that are used in the next table.

Bidirectional Pin (or Port) Type Definitions

Pin Type	Type 1	Type 2
Blackbox pins	out	in
Instance pins	out	in
Netlist (primary) ports	in	out
Hierarchical ports	in	out

Bidirectional Pin Type Defaults

Pin or Port Constraint	Type	Hierarchical Port
`set_input_delay`	1	ignored
`set_output_delay`	2	ignored
`set_clock_latency`	1	ignored
`create_generated_clock`	1	used

set_input_transition	1	ignored
set_driving_cell	1	ignored
set_drive	1	ignored

Defaults for Port Constraints

The following table defines the port types that are used in the next table.

Bidirectional Port Type Definitions

Pin Type	Type 1	Type 2
Netlist (primary) ports	in	out
Hierarchical ports	in	out

Bidirectional Port Type Defaults

Port Constraint	Type	Hierarchical Port
set_fanout_load	2	ignored
set_load	2	ignored
set_max_transition	2	ignored

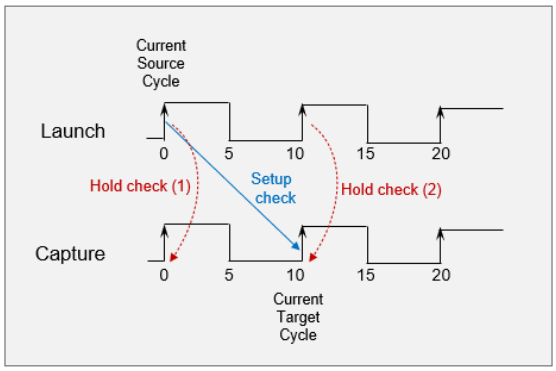
Cycle Accounting Rules - Data Checks

The following cycle accounting rules apply in the given below scenarios.

General Single-Cycle Timing

- When the launch and capture root clock edges align, the Setup capture edge is adjusted so that a full clock cycle is allowed for the path.
- When launch and capture Setup edges do not align, then the most constraining pair of root edges defines the cycle time The launch and capture Hold edges are derived from the adjusted Setup edges:
    - Any multi-cycle adjustment to Setup is done before deriving the Hold edges.
    - Two Hold rules are checked and the most constraining edge pair is used (often they are the same).
        - Data launched from the current cycle at the source cannot be captured by the previous cycle at the target.
        - Data from the next cycle at the source cannot be captured by the current cycle at the target.
- Hold edges typically align - allowing full CPPR credit - including SI to be given.

The following diagram illustrates this behavior:



Zero-Cycle Timing

The zero-cycle Setup paths launch and capture data off of the same cycle. The zero-cycle Setup paths are obtained from the `set_data_check` constraint and non-sequential type library checks.

The following report outputs shows setup and hold data check output comparison:

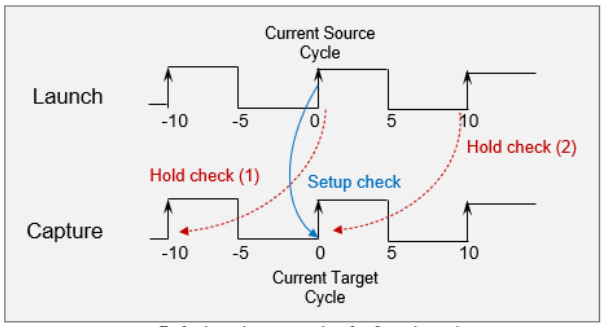
```
Path 1: MET Data To Data Setup Check with Pin ref
Endpoint:   data (v) checked with  leading edge of 'PH10'
Beginpoint: RB/Q (v) triggered by  leading edge of 'PH10'
Other End Arrival Time          6.110
- Data Check Setup              3.000
+ Phase Shift                   0.000
+ CPPR Adjustment               0.150
= Required Time                 3.260
- Arrival Time                  3.260
= Slack Time                    0.000

     ----------------------------------------
     Timing Point  Edge  Cell     Delay  Arrival
                                         Time
     ----------------------------------------
     clk           ^     -        -      0.000
     C1/Y          ^     BUFX1    0.260  0.260
     ...
     data ->       v     -        0.000  3.260
     ----------------------------------------
     ----------------------------------------
     Timing Point  Edge  Cell     Delay  Arrival
                                         Time
     ----------------------------------------
     clk           ^     -        -      0.000
     C1/Y          ^     BUFX1    0.110  0.110
     ...
     ref           ^     -        0.000  6.110
     ----------------------------------------
```

```
Path 1: MET Data To Data Hold Check with Pin ref
Endpoint:   data (v) checked with  leading edge of 'PH10'
Beginpoint: RB/Q (v) triggered by  leading edge of 'PH10'
Other End Arrival Time          6.260
+ Data Check Hold               3.000
+ Phase Shift                 -10.000
- CPPR Adjustment               0.100
= Required Time                -0.840
  Arrival Time                  3.110
  Slack Time                    3.950

     ----------------------------------------
     Timing Point  Edge  Cell     Delay  Arrival
                                         Time
     ----------------------------------------
     clk           ^     -        -      0.000
     C1/Y          ^     BUFX1    0.110  0.110
     ...
     data ->       v     -        0.000  3.110
     ----------------------------------------
     ----------------------------------------
     Timing Point  Edge  Cell     Delay  Arrival
                                         Time
     ----------------------------------------
     clk           ^     -        -      0.000
     C1/Y          ^     BUFX1    0.260  0.260
     ...
     ref           ^     -        0.000  6.260
     ----------------------------------------
```

The following diagram shows the default cycle accounting for a zero-cycle path:



Default cycle-accounting for 0-cycle path

In this case:

- The adjusted Hold edges mostly yield positive slack.
- The SI CPPR (50ps) is credited to Setup, but not Hold due to the Hold launch and capture edges not aligning.

Zero-Cycle Timing with Multi-Cycle Adjust

The following command will move the Hold launch back one cycle - making the Hold checks zero-cycle as well:

```
set_multicycle_path -1 -hold -to [get_ports data]
```

```
Path 1: MET Data To Data Setup Check with Pin ref
Endpoint:   data (v) checked with  leading edge of 'PH10'
Beginpoint: RB/Q (v) triggered by  leading edge of 'PH10'
Other End Arrival Time          6.110
- Data Check Setup              3.000
+ Phase Shift                   0.000
+ CPPR Adjustment               0.150
= Required Time                 3.260
- Arrival Time                  3.260
= Slack Time                    0.000

     ----------------------------------------
     Timing Point  Edge  Cell     Delay  Arrival
                                         Time
     ----------------------------------------
     clk           ^     -        -      0.000
     C1/Y          ^     BUFX1    0.260  0.260
     ...
     data ->       v     -        0.000  3.260
     ----------------------------------------
     ----------------------------------------
     Timing Point  Edge  Cell     Delay  Arrival
                                         Time
     ----------------------------------------
     clk           ^     -        -      0.000
     C1/Y          ^     BUFX1    0.110  0.110
     ...
     ref           ^     -        0.000  6.110
     ----------------------------------------
```

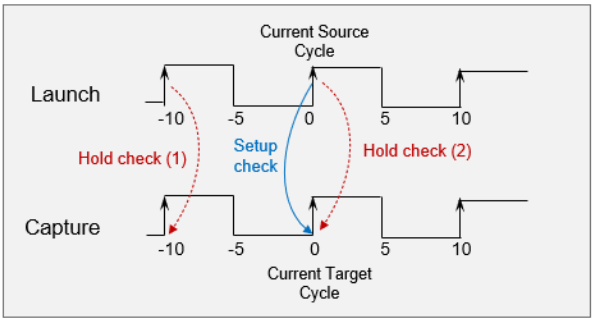
```
Path 1: VIOLATED Data To Data Hold Check with Pin ref
Endpoint:   data (v) checked with  leading edge of 'PH10'
Beginpoint: RB/Q (v) triggered by  leading edge of 'PH10'
Other End Arrival Time          6.260
+ Data Check Hold               3.000
+ Phase Shift                 -10.000
- CPPR Adjustment               0.150
- Cycle Adjustment            -10.000
= Required Time                 9.110
  Arrival Time                  3.110
  Slack Time                   -6.000

     ----------------------------------------
     Timing Point  Edge  Cell     Delay  Arrival
                                         Time
     ----------------------------------------
     clk           ^     -        -      0.000
     C1/Y          ^     BUFX1    0.110  0.110
     ...
     data ->       v     -        0.000  3.110
     ----------------------------------------
     ----------------------------------------
     Timing Point  Edge  Cell     Delay  Arrival
                                         Time
     ----------------------------------------
     clk           ^     -        -      0.000
     C1/Y          ^     BUFX1    0.260  0.260
     ...
     ref           ^     -        0.000  6.260
     ----------------------------------------
```

The current syntax does not restrict the multi-cycle application to only data-to-data checks.

Since both the Setup and Hold are zero-cycle, both receive the full CPPR credit - including SI.

The following diagram illustrates accounting for zero-cycle path when the above mentioned command is specified.



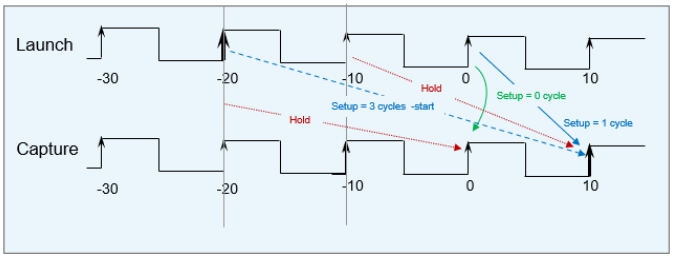
Accounting for 0-cycle path with: `set_multicycle_path -1 -hold`

set_multicycle_path -start

When the `set_multicycle_path -start` parameter is used to adjust data-to-data paths, there might be some compatibility issues with third-party software.

You can set the `timing_enable_sdc_compatible_data_check_mcp` attribute to `true` to enable checking for SDC compatible data checks. When the `-start` parameter is specified, the parameter reverses the reference clock by one clock cycle.

The following diagram shows Tempus cycle accounting when `set_multicycle_path -start 3` command is specified.



create_clock

```
create_clock
-period period_value
[-name clock_name]
[-waveform edge_list]
[-add]
[-comment string]
[source_objects]
```

Creates a clock object and defines its waveform in the current design. If you do not specify any source_objects, but you specify the `-name` *clock_name* argument, a virtual clock is created. The new clock has an ideal clock latency, and the software does not assume any propagation delay through the clock network.

The clock that you create using the `create_clock` command cannot have multiple pulses per period. You can specify multiple constraints on the same pin using the `-add` parameter.

If one of the source_objects is already the source of a clock, the source is removed from that clock. If you need to specify multiple clocks on the same source for simultaneous analysis, you must specify the `-add` parameter, to add this clock to the existing clock.

For example, if you specify the following command sequence, the `PH2` clock waveform overwrites the `PH1` waveform.:

```
create_clock -name PH1 ... [get_ports sysclk]

create_clock -name PH2 ... [get_ports sysclk]
```

To have both clocks applied, you must specify the following command sequence:

```
create_clock -name PH1 ... [get_ports sysclk]

create_clock -name PH2 -add ... [get_ports sysclk]
```

You can create a collection of clocks, which is a group of objects referenced by a string identifier using the get_clocks command.

Note: If the `create_clock` constraint is added incrementally in SI mode, an explicit `update_timing -full` command is required (before `report_timing` or `report_constraint`) for SI full accuracy.

Parameters

`-add`	Specifies multiple clocks on the same source for simultaneous analysis with different clock waveforms. You must specify the `-name` parameter with this parameter.
`-comment` *string*	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
`-name` *clock_name*	Specifies the string name of the ideal clock being created. If you do not specify this option, the clock gets the same name as the first clock source that you specify using the `-source_objects` option. If you do not specify any source_objects, you must specify the `-name` *clock_name* option, which creates a virtual clock not associated with a port or pin.
`-period` *period_value*	Specifies the value of the ideal clock in library time units.
source_objects	Specifies the list of pins or ports associated with an ideal clock waveform of the clock. If you do not specify this option, you must specify the `-name` *clock_name* option, which creates a virtual clock not associated with a port or pin. If you specify a clock on a pin that already has a clock, the new clock replaces the old one.
`-waveform` *edge_list*	Specifies the rise and fall edge times of the clock waveforms in library time units over an entire clock period. The first value in the edge list is the time at which the first transition on the clock occurs, which is the first rising transition after time zero, and edges are monotonically increasing. The numbers should represent one full clock period. If you do not specify the `-waveform` *edge_list* option, a default waveform is assumed, and the leading edge is placed at `0` and the trailing edge is placed at the midpoint of the period, such that a symmetric clock is generated.

Examples

- The following command creates an `ideal_clk1` clock on port `clkA` with a period of `5`, a rise at `0`, and a fall at `2.5`:
  ```
  create_clock [get_ports {clkA}] -name ideal_clk1 -period 5 -waveform {0 2.5}
  ```
- The following command creates an `ideal_clk2` clock on port `clkB` with a period of `10`, a rise at `1`, and a fall at `9`:
  ```
  create_clock [get_ports {clkB}] -name ideal_clk2 -period 10 -waveform {1 9}
  ```

Related Commands

- all_clocks
- get_clocks
- get_pins
- get_ports
- set_clock_latency
- report_clocks
- set_clock_uncertainty
- set_input_delay
- set_output_delay

create_generated_clock

```
create_generated_clock
[-help]
target_pin_list
[-add]
[-comment string]
[-duty_cycle percent]
[-edge_shift edge_shift_list]
[-invert]
[-master_clock source_clock_name]
[-name <clock_name>]
-source source_pin
{[-multiply_by factor]
[-divide_by factor]
[-edges edge_list]
[-combinational]}
```

Creates a new clock signal from the clock waveform of a given pin in the design, and binds it with the pins or hierarchical pins in the *target_pin_list* argument. Whenever the source clock changes, the derived clock(s) change automatically.

You can generate a new clock waveform using one of the three ways:

- Multiply the frequency of the source clock
- Divide the frequency of the source clock
- Select the edges of the source clock to be mapped to the edges of the new clock

Given below are the SDC semantics to be used when the `create_generated_clock` command is defined:

- The edges are interpreted as alternating rise and falling edges.
- Each edge must be greater than and equal to its previous edge.
- Less than 3 edges are not allowed.
- Legal edge numbers start at “1”, edge 0 is undefined.
- Legal assertion of period 0 will be -edges {1 1 1 }.

If incorrect clock definitions are specified, an error will occur and the clock will not be generated.

When the `create_generated_clock` command is specified without the `-master_clock` parameter, and the software finds multiple clocks reaching the specified source pin, then the regular clocks are picked ahead of the generated clocks. In case if multiple regular clocks are present, the master clock is picked based on alphabetical sorting order of the clock names.

When this constraint is set on a pin, the software will stop the propagation of clock phases from all upstream clocks, including the generated clock’s own master clock. The special latency-only phases of the master clock may still be propagated to fulfill the requirements of other downstream generated clocks, which are used only for latency path calculations.

Note: A `create_generated_clock` assertion on a pin overrides any existing `create_generated_clock` assertion if both the constraints have the same source pin. In the following command sequence, the second assertion overrides the

first:

```
create_generated_clock -name GCK1 -source CK -divide_by 2 U_FD1/Q
create_generated_clock -name GCK2 -source CK -multiply_by 2 U_FD1/Q
```

To have both assertions applied, you must specify the `-add` parameter. For example:

```
create_generated_clock -name GCK1 -source CK -divide_by 2 U_FD1/Q
create_generated_clock -name GCK2 -add -master_clock CLK -source CK -multiply_by 2    U_FD1/Q
```

Parameters

`-help`	Prints out the command usage.
`-add`	Models multiple generated clocks on the same source when multiple clocks must fan into the source pin. Ideally, one generated clock must be specified for each clock that fans into the master pin. Specify this parameter with the `-name` and `-master_clock` parameters. You can use the `-add` parameter to specify a different clock name for each generated clock, when used with the `-master_clock` option. Subsequently, you can use this clock name for setting other constraints, such as, `set_false_path` or `set_input_delay`.
`-combinational`	Restricts the path search for latency to only paths which do not cross sequential devices, or the source pins of other generated clocks. When you specify `-combinational`, if there are both sequential and combinational latency paths, the software only considers the combinational paths for the generated clock. If there are no combinational latency paths for the generated clock, the generated clock does not inherit latency from its parent clock hierarchy. By default, when calculating the latency of a generated clock, the software searches for the longest (and shortest) paths from the parent waveform to the generated clock point. All paths are considered, including those which traverse sequential elements such as flip-flops, registers, and latches. The default behavior is intended for cases where there is a cascade of generated clocks, and some of the clocks are not explicitly defined. The `-combinational` option can be used in conjunction with the `-multiply_by`, `-divide_by`, or `-edges` options.
`-comment` *string*	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
`-divide_by` *integer*	Determines the frequency of the new clock by dividing the frequency of the source clock by *dfactor*. For example, if *dfactor=5*, the new frequency is 1/5 of the source frequency, and the new clock period is 5 times the source clock period.

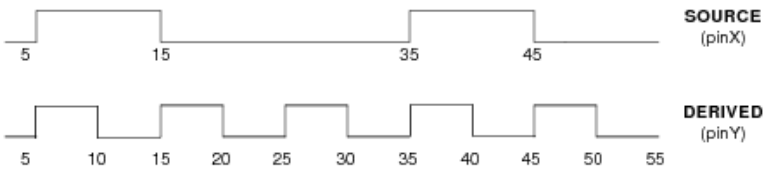
`-duty_cycle` *percent*	Sets the duty cycle (high pulse width or clock period). The number (*percent*) is between `0` and `100`. For example, if *percent=50*, the high pulse width of the new clock waveform is the same as its low pulse width. **Note:** The `-duty_cycle` option can only be used with the `-multiply_by` option, otherwise it will be ignored with a warning.
`-edges` *edge_list*	Selects a list of edges from the source clock that form the edges of the derived clock. See the frequency division example below.
`-edge_shift` *edge_shift_list*	Specifies the amount of shift for each edge in the *edge_list* option.
`-invert`	Inverts the source clock. Use with the `-source` option and the `-multiply_by` or `-divide_by` options.
`-master_clock` *source_clock_name*	Generates a target clock from the specified clock name. If you specify the `-master_clock` option and multiple signals arrive at the source pin, the clock specified by this option is used to generate the target clock. Use the `-add` parameter along with`-master_clock` parameter to add a clock to the existing clock when you want to specify multiple clocks on the same source for simultaneous analysis.
`-multiply_by` *integer*	Determines the frequency of the new clock by multiplying the frequency of the source clock by *mfactor*. For example, if *mfactor=4*, the new frequency is 4 times the source frequency, and the new clock period is 1/4 of the source clock period.
`-name` *clock_name*	Specifies the name for the generated clock. *Default*: The system creates a name for you. Always specify a name for the generated clock. Some commands issue an error if you use the system-generated name.
`-source` *source_pin*	Specifies the name of the source pin from which the generated clock is to be derived - with respect to the master clock containing the master clock as well as the generated clock definition.
target_pin_list	Specifies the list of pins where the generated clock assertion is applied. The clock signal propagating downstream from each of these pins becomes associated with the generated clock.

Examples

- The following command generates the derived clock waveform, as shown in the figure below:

```
tempus > create_generated_clock -name mult_3 -source pinX -multiply_by 3 -duty_cycle 50 pinY
```

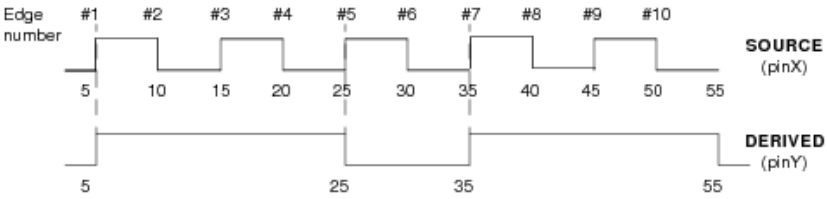
Figure: **Derived Clock (Frequency Multiplication)**



- The following command shows one way to create a divide-by-3 clock by using edge numbers of the source waveform to specify the new waveform, as shown in the figure below:

  ```
  tempus > create_generated_clock -name genclk -source pinX -edges {1 5 7} pinY
  ```

Figure: **Derived Clock (Frequency Division)**

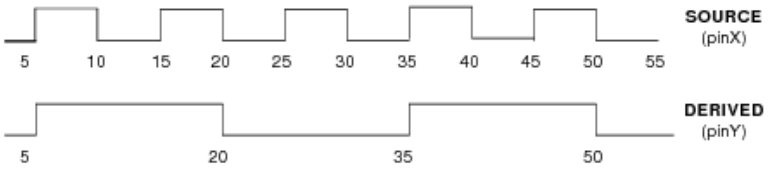


- The following command uses the `-divide_by` option to get the same clock period; the difference is in the duty cycle:

  ```
  tempus > create_generated_clock -name div_3 -source pinX -divide_by 3 pinY
  ```

  With the -edges option, the above example creates a clock with a duty cycle of 67 percent (20/30). However, using the -divide_by 3 option preserves the 50 percent duty cycle of the source clock as shown in the figure below:

Figure: **Derived Clock (Frequency Division, 50 Percent Duty-cycle)**



- Using the following commands, when there is more than 1 source clock, the tool creates the generated clock based on the source clock specified with the -master option: `create_clock -name c1 -period 10 [get_ports {ck} ] -add` `create_clock -name c2 -period 10 [get_ports {ck} ] -add` `create_generated_clock -name newclk -source ck -divide_by 2 -master_clock c1 reg/Q -add`

  Using the `report_clocks` command, the output waveform of the newclk clock is a divide-by-2 of the c1 clock and is reported as shown in the example below.

Example: report_clocks after using the -master option

```
tempus > create_clock -name CK1 -period 20 [get_ports {CLK1} ] -add
tempus > create_clock -name CK2 -period 10 [get_ports {CLK1} ] -add
tempus > create_generated_clock -name GCLK -source CLK1 -divide_by 2 u8/Q -add -master_clock CK2

tempus > report_clocks
------------------------------------------------------------
Clock Descriptions
```

```
------------------------------------------------------------
Attributes
------------------------------------------------------------
Clock Name  Source  Period   Lead   Trail   Generated   Propagated
------------------------------------------------------------------
CK1         CLK1    20.000   0.000  10.000  n           n
CK2         CLK1    10.000   0.000  5.000   n           n
------------------------------------------------------------------

Generated-Clock Descriptions
-------------------------------------------------------------------------------------
Name  Generated   Master      Master-clock Invert Freq.      Duty-Cycle Edges Edge-Shift
      Source(pin) Source(pin)                   Multiplier
-------------------------------------------------------------------------------------
GCLK    u8/Q      CLK1        CK2          n     1/2         -          -     -
```

Related Commands

- create_clock
- get_clocks
- report_clocks
- reset_generated_clock

current_design

```
current_design
[-help]
[design_name]
```

Changes the current directory in the design hierarchy to the specified design and returns the path to the design.

If no design is specified, the command returns the top-level design.

Parameters

`-help`	Prints out the command usage
design_name	Specifies the name of the top design.

Related Commands

- set_max_transition
- set_min_capacitance
- set_min_transition
- list_libraries

current_instance

```
current_instance
[-help]
[hinst]
```

Sets the instance given by *hinst* to be the current instance. Design objects referenced by subsequent commands can be found relative to this instance. All searches for design objects are started in this instance. If design objects are referenced hierarchically, *hinst* is used as the root (top) of the hierarchy.

Parameters

-help	Prints out the command usage.
hinst	Specifies the name of the instance to become the current instance. If *hinst* is not given, the current instance is set to the top instance of the hierarchy.

Examples

- The following command sets the current instance to MAC/MULTI, sets a constant on a register pin inside the hierarchical instance and then later sets the current instance back to top:

```
tempus> current_instance MAC/MULT1
tempus> set_case_analysis 0 [get_pins {reg6/SE}]
...
tempus> current_instance
```

Related Commands

- set_case_analysis

get_current_constraint_mode

```
get_current_constraint_mode
[-help]
```

Determines the constraint mode that is being currently read.

This command can be used inside SDC constraint files which are shared across multiple constraint modes to conditionalize how the constraints are interpreted for specific modes.

group_path

```
group_path
[-help]
[-comment string]
-name path_group_name
-from from_list]
[-to to_list]
[-through through_list]]
```

Groups paths in a design, and identifies them with a path group name. You can use the path group names for reporting.

You cannot use `group_path` statements in the SDC file when the software is in MMMC mode. In this case a warning is issued. However, the software supports the group_path command in MMMC mode as a command outside of the SDC file. The path groups are considered to be global in nature and apply to all the views, and are not supported on MMMC mode-specific basis. The path groups are persistent through a saveDesign to restoreDesign sequence. They are saved in the `<saveDir>/mmmc/pathgroup.sdc` file of a MMMC save directory.

Note that if a path can be included in multiple path groups, it will only be included in the path group with the most specific definition (`-from ... -through ... -to ...` is more specific than `-from ... -to ...` is more specific than `-from ...`).

Parameters

`-help`	Prints out the command usage.
`-comment` *string*	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
`-from` *from_list*	Specifies a list of pins, instances, or clocks that are the start of the paths. When you specify a clock object, all paths that are triggered by the clock are included in the path group.
`-name` *path_group_name*	Specifies the name of the path group. If the path_group_name already exists, the specified paths are added to the existing path group.
`-through` *through_list*	Specifies the pins or instances where the paths go through. When you specify multiple through list, the paths that go through the objects in the specified order are added to the path group.
`-to` *to_list*	Specifies a list of pins, instances, or clocks where the path ends. You must specify valid endpoints in the *to_list* or else the software ignores the constraint. When you specify a clock object, all paths that are checked by the clock are included in the path group. If you specify an instance name in the *to_list*, all the data input pins of the instance are added to the path group.

Examples

- The following example groups paths that start from specific pins and end at the D pin of any instances whose name start with the string ff. The example also reports the specified path groups:

  ```
  tempus > group_path -name path1 -from {ff1/CLK ff2/CLK} -to {ff*/D}
  tempus > report_timing -path_group path1
  ```

- The following command groups paths that go through specific points.

  ```
  tempus > group_path -name path2 -through an1/Z -through an2/Z -through an3/Z
  ```

- `If path group definitions are matching/overlapping (when the -from/-through/-to parameters are used), then path group resetting is applied. The path group overwriting and prioritization rules are similar to path exceptions.`

  The following examples show how path group resetting is applied.

  If the timing path from u1/CK to u2/CK is part of both the path groups, then the path will be reported in the second path group only (that is, the most recent matching path group).

  ```
  group_path -name g2 -from u1/CK,u4/Y -to u2/CK
  group_path -name g1 -from u1/CK -to u2/CK
  ```

  In the following example, the path groups are not overlapping and have different `-from`/`-thru`/`-to` and map to the same path, then the path will be reported only in the first path group (that is, the most specific matching path group):

  ```
  group_path -name g2 -from u1/CK -to u2/CK
  group_path -name g1 -to u2/CK
  ```

  In this case, the path will be reported only in the first path group (that is, the most specific matching path group).

Related Commands

- `report_timing`
- `get_path_groups`
- `reset_path_group`
- `report_path_groups`
- `create_basic_path_groups`

reset_annotated_check

```
reset_annotated_check
[-help]
[-clock [rise | fall]]
[-cond sdf_condtion]
[-setup | -recovery]
[-hold | -removal]
[-nochange_high | -nochange_low]
[-rise] [-fall]
{[[-from from_pins][-to to_pins]] | -all}
[obj_list]
```

Removes annotated timing check values that you had set by using the `set_annotated_check` command or by SDF annotation.

Note: The `reset_annotated_check` command does not reset the timing check values set by SDF annotations.

Parameters

`-help`	Prints out the command usage.
`-all`	Resets all annotated timing check values in the design.
`-clock [rise | fall]`	Specifies the direction of the related pin of the check to reset. *Default:* Resets checks against both edges of the clock
`-cond sdf_condtion`	Resets all annotated timing check values for which a condition is specified.
`-fall`	Restricts the reset to only affect the falling edge of the constrained pins. *Default:* Checks on both rising and falling signals are reset
`-from from_pins`	Specifies the beginning instance pin of the timing arcs for which the annotated check is reset. This is usually the reference (for example, clock pin) of the constraint. The selected timing arcs are those that have their source pins specified using this option and the sink pins specified using the `-to` option. If this option is not specified, only timing arcs whose sink pins are specified using the `-to` option are selected. You cannot use this parameter with `obj_list` parameter.
`-hold | -removal`	Resets annotations on the hold or removal check type.
`-nochange_high | -nochange_low`	Resets the annotations on the specified timing check type.
`obj_list`	Resets the annotated checks on specified list or collection of cell instances.

`-rise`	Restricts the reset to only affect the rising edge of the constrained pins. *Default:* Checks on both rising and falling signals are reset
`-setup | -recovery`	Resets annotations on the setup or recovery check type.
`-to` *to_pins*	Specifies the terminating instance pin of the timing arcs for which the annotated check is reset. This is usually the constrained pin (data pin) of the check. The selected timing arcs are those that have their sink pins specified using this option and the source pins specified using the `-from` option. If this option is not specified, only timing arcs whose source pins are specified using the `-from` option are selected. You cannot use this parameter with *obj_list* parameter.

Examples

- The following command resets all the timing check annotations in the design:
  ```
  tempus > reset_annotated_check -all
  ```
- The following command resets all timing check annotations on a particular cell:
  ```
  tempus > reset_annotated_check [get_cells BLK/BR1]
  ```
- The following command resets specific checks on a specific instance:
  ```
  tempus > reset_annotated_check -setup -hold [get_cells BLK/BR2]
  ```
- The following command resets checks on a specific timing check arc:
  ```
  tempus > reset_annotated_check -from BLK/BR1/CK -to BLK/BR1/D
  ```

Related Commands

- `set_annotated_check`

reset_annotated_delay

```
reset_annotated_delay
[-help]
{[-all] | [[-from from_pins] [-to to_pins]] | [object_list]}
```

Resets annotated delay assertions that have been previously specified using the `set_annotated_delay` command. When you reset an annotated delay, the system uses the delays which are calculated from the original libs for the arc.

Parameters

`-help`	Prints out the command usage.
`-all`	Resets all annotated delay assertions in the design.
`-from` *from_pins*	Specifies timing arcs for which the delay is reset. The selected timing arcs are those that have their source pins specified using this option and the sink pins specified using the `-to` option. If this option is not specified, only timing arcs whose sink pins are specified using the `-to` option are selected.
object_list	Resets the annotated delays on specified list or collection of cell instances.
`-to` *to_pins*	Specifies timing arcs for which the delay is reset. The selected timing arcs are those that have their sink pins specified using this option and the source pins specified using the `-from` option. If this option is not specified, only timing arcs whose source pins are specified using the `-from` option are selected.

Examples

- The following command resets all the annotated delay assertions in the design:
  ```
  reset_annotated_delay -all
  ```
- The following command resets all the `set_annotated_delay` assertions applied on pins (either `from` or `to`) of instance `f0`:
  ```
  reset_annotated_delay f0
  ```
- The following command resets the `set_annotated_delay` assertions with the `from` and `to` pin pair combination:
  ```
  reset_annotated_delay -from u1/Y -to u2/A
  ```

Related Commands

- `set_annotated_delay`

reset_annotated_transition

```
reset_annotated_transition
[-help]
{-all | port_or_pin_list}
```

Resets annotated transitions previously asserted by the `set_annotated_transition` command.

Parameters

`-help`	Prints out the command usage.
`-all`	Resets all annotated transitions in the design.
`port_or_pin_list`	Specifies a list of specific pins or primary ports to reset the transition.

Examples

- The following commands reset the `set_annotated_transition` constraint on an internal pin `pad/Y0`.

  ```
  reset_annotated_transition pad/Y0
  ```

  or

  ```
  reset_annotated_transition [get_pins {pad/Y0}]
  ```

  You can also specify a collection as the pin argument. For example:

  ```
  reset_annotated_transition [get_pins pad/*]
  ```

- The following commands reset the `set_annotated_transition` constraint set on a port `in`:

  ```
  reset_annotated_transition in
  ```

  or

  ```
  reset_annotated_transition [get_ports {in}]
  ```

  Any `set_input_transition` constraint applied on the same port is not removed. This is applicable only for input ports, as `set_input_transition` cannot be applied on output ports.

- The following command resets all the `set_annotated_transition` constraints:

  ```
  reset_annotated_transition -all
  ```

Related Commands

- set_annotated_transition

reset_aocv_stage_weight

```
reset_aocv_stage_weight
[-help]
port_list
[-early]
[-late]
```

Resets the existing stage weight values set for the specified ports.

Parameters

`-help`	Prints out the command usage.
`-early`	Resets weight for early paths.
`-late`	Resets weight for late paths.
`port_list`	Specifies a list or collection of input, output, and bidi ports for which stage weight values are to be reset.

Examples

- The following example resets the stage weight values of ports `clock1` and `out0`:
  reset_aocv_stage_weight [get_ports {clock1 out0}]

Related Commands

- `set_aocv_stage_weight`

reset_case_analysis

```
reset_case_analysis
[-help]
port_pin_list
```

Removes the assertions set by the set_case_analysis command.

Parameters

`-help`	Prints out the command usage.
`port_pin_list`	Specifies the list of ports or pins for which the assertions are to be removed.

Examples

- The following commands sets the assertion to 0 and then resets them:

```
set_case_analysis 0 CM1/S0
get_property [get_pins CM1/S0] constant_value
```

Returns a value of 0

```
reset_case_analysis [get_pins CM1/S0]
get_property [get_pins CM1/S0] constant_value
```

Returns a value of NA

Related Commands

- set_case_analysis

reset_clock

```
reset_clock
[-help]
{-all | clock_list}
```

Removes previously created clock assertions.

Parameters

`-help`	Prints out the command usage.
`-all`	Removes all clocks in the design.
`clock_list`	Specifies the list of clocks for which the clock assertion is removed.

Examples

- The following commands reset a list of clocks and all clocks, and display the results using the `report_clocks` command:

```
tempus > report_clocks

# Generates the following report:

------------------------------------------------------------------
Clock Descriptions
------------------------------------------------------------------
Attributes
------------------------------------------------------------------
Clock Name   Source   Period   Lead   Trail   Generated   Propagated
------------------------------------------------------------------
CK1          CLK1     20.000   0.000  10.000   n          n
CK2          CLK1     10.000   0.000  5.000    n          n
------------------------------------------------------------------
Generated-Clock Descriptions
-------------------------------------------------------------------------------------------
Name    Generated   Master      Master-clock Invert Freq.     Duty-Cycle   Edges   Edge-Shift
        Source(pin) Source(pin)                 Multiplier
-------------------------------------------------------------------------------------------
GCLK4   u8/Q        CLK4        CLK_W_4      n      1/2        -            -       -


tempus> reset_clock [get_clocks CK1]
tempus> report_clocks
------------------------------------------------------------------
Clock Descriptions
------------------------------------------------------------------
Attributes
------------------------------------------------------------------
Clock Name   Source   Period   Lead   Trail   Generated   Propagated
------------------------------------------------------------------
CK2          CLK1     10.000   0.000  5.000   n           n
------------------------------------------------------------------
------------------------------------------------------------------
Generated-Clock Descriptions
-------------------------------------------------------------------------------------------
Name   Generated    Master      Master-clock  Invert  Freq.      Duty-Cycle  Edges  Edge-Shift
       Source(pin)  Source(pin)                       Multiplier
-------------------------------------------------------------------------------------------
GCLK4  u8/Q         CLK4        CLK_W_4       n       1/2        -           -      -
```

Related Commands

- `create_clock`
- `report_clocks`

reset_clock_exclusivity

```
reset_clock_exclusivity
[-help]
pin_name
```

Allows to reset the `set_clock_exclusivity` constraints applied on pins.

Parameters

`-help`	Prints out the command usage.
`pin_name`	Specifies the name of pins for which clock exclusivity constraints need to be reset.

Related Commands

- `set_clock_exclusivity`

reset_clock_gating_check

```
reset_clock_gating_check
[-help]
[-setup]
[-hold]
[-rise]
[-fall]
[-high | -low]
[obj_list]
```

Resets the specified setup and hold values for clock gating checks to the default values.

Parameters

`-help`	Prints out the command usage.
`-high | -low`	Resets the non-controlling value (high or low) of the clock.
`-hold`	Resets clock gating hold values.
`-rise | -fall`	Resets the specified setup (or hold) value to the rising (or falling) signals on the specified pins.
`-setup`	Resets clock gating setup values.
obj_list	Specifies a list of clocks, instances, or pins for which clock gating checks are to be reset to default values. If the *obj_list* parameter is not specified, the clock gating check is reset on the current design.

Related Commands

- set_clock_gating_check

reset_clock_groups

```
reset_clock_groups
[-help]
{-physically_exclusive | -logically_exclusive | -asynchronous}
{-name name_list | -all}
```

Resets clock groups that have been set using the `set_clock_groups` command.

Parameters

`-help`	Prints out the command usage.
`-all`	Removes all physically exclusive, logically exclusive, or asynchronous clock groups in the design. Removes all the physically exclusive, logically exclusive, or asynchronous clock groups in the design. This parameter should be used with one of the parameters: `-logically_exclusive`, `-physically_exclusive`, or `-asynchronous` at one time, and should not be used with the `-name` parameter.
`-asynchronous`	Specifies the asynchronous clock groups to be removed. This parameter must be used with either `-all` or `-name` parameter.
`-logically_exclusive`	Specifies the logically exclusive clock groups to be removed. This parameter must be used with either `-all` or `-name` parameter.
`-name` *`name_list`*	Specifies the previously defined names of clock groups to be removed. This parameter should be used with one of the parameters: `-logically_exclusive`, `-physically_exclusive`, or `-asynchronous` at one time.
`-physically_exclusive`	Specifies all the physically exclusive clock groups to be removed. This parameter must be used with either `-all` or `-name` parameter.

Examples

- The following command resets all physically exclusive clock groups in the design:
  ```
  reset_clock_groups -physically_exclusive -all
  ```
- The following command resets physically exclusive clock group `CLKGRP0`:
  ```
  reset_clock_groups -physically_exclusive -name CLKGRP0
  ```

Related Commands

- set_clock_groups

reset_clock_latency

```
reset_clock_latency
[-help]
[-source]
[-clock clock_list]
obj_list
```

Resets assertions made by previous `set_clock_latency` commands.

Parameters

`-help`	Prints out the command usage.
`-clock` *`clock_list`*	Specifies the list of clocks with which associated `set_clock_latency` constraints, at the pin level, are to be removed. **Note:** This parameter cannot be used with clocks specified using the *`obj_list`* parameter.
`obj_list`	Specifies a list of pins, ports, or clock objects for which the assertions are to be removed.
`-source`	Removes the source latency.

Examples

- The following commands show the use of the `reset_clock_latency` command to remove a leaf-level `set_clock_latency` assertion, after the clock tree has been put into propagated mode:

  ```
  set_clock_latency 3.141 [get_pins BLK/BR2/CK]
  report_timing -to BLK/BR2/D -path_type full_clock
  ```

  The following report is generated:

  ```
  Path 1: MET Setup Check with Pin BLK/BR2/CK
  Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
  Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
  Other End Arrival Time 3.141 <---
  - Setup 0.191
  + Phase Shift 10.000
  = Required Time 12.950
  - Arrival Time 0.249
  = Slack Time 12.701
  ...
  ```

```
set_propagated_clock [get_ports tclk]
report_timing -path_type full_clock -net -to BLK/BR2/D
```

The following report is generated:

```
Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 3.141 <--------
- Setup 0.191
+ Phase Shift 10.000
= Required Time 12.950
- Arrival Time 0.594
= Slack Time 12.356
...
```

In this example, setting the root of the clock tree to propagated mode is not sufficient to override the `set_clock_latency` assertion at the clock endpoint. You must use the `reset_clock_latency` command to remove the assertion, so that the register sees propagated clock timing.

```
reset_clock_latency [get_pins BLK/BR2/CK]
report_timing -path_type full_clock -net -to BLK/BR2/D
```

The following report is generated:

```
Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 0.345 <--------
- Setup 0.186
+ Phase Shift 10.000
= Required Time 10.159
- Arrival Time 0.588
= Slack Time 9.571
```

- The following commands show that `reset_clock_latency` is only effective when it is applied on the same object for which you had set the assertion using the `set_clock_latency` command:

  Latency here is specified on a pin object:

  ```
  set_clock_latency 3.141 [get_pins BLK/BR2/CK]
  report_timing -path_type full -net -to BLK/BR2/D
  ```

  The following report is generated:

  ```
  Path 1: MET Setup Check with Pin BLK/BR2/CK
  Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
  Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
  Other End Arrival Time 3.141 <-
  - Setup 0.191
  + Phase Shift 10.000
  ```

```
= Required Time 12.950
- Arrival Time 0.249
= Slack Time 12.701
```

Resetting of the latency is done on a clock object and has no effect on downstream latencies specified on pin objects.
In this example, a `reset_clock_latency` on the clock waveform that is driving BLK/BR2/CK will have no effect.

```
reset_clock_latency [get_clocks WAVE]
report_timing -path_type full -net -to BLK/BR2/D
```

The following report is generated:

```
Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 3.141 <-
- Setup 0.191
+ Phase Shift 10.000
= Required Time 12.950
- Arrival Time 0.249
= Slack Time 12.701
```

Related Commands

- `set_clock_latency`

reset_clock_sense

```
reset_clock_sense
[-help]
[-all]
| [[-clocks clock_list] pin_or_port_list]]
```

Resets clock sense constraint(s) specified using the `set_clock_sense` command.

Parameters

`-help`	Prints out the command usage.
`-all`	Resets all the specified `clock_sense` constraints.
`-clocks` *clock_list*	Specifies the list of clocks for which the clock sense constraints are to be removed. The clocks must have previously been defined using the `set_clock_sense -clock` parameter.
pin_or_port_list	Specifies the list or collection of cell output pins for which the clock sense constraints will be removed.

Examples

- The following command resets all the previously defined `clock_sense` constraints from a design:

  ```
  tempus > reset_clock_sense -all
  ```

Related Commands

- `set_clock_sense`

reset_clock_transition

```
reset_clock_transition
[-help]
clock_list
```

Removes the clock transition assertions you had set on the specified list of clock waveforms.

Parameters

`-help`	Prints out the command usage.
`clock_list`	Specifies a list of clock waveforms for which the clock transition assertions are removed. The *`clock_list`* variable can be a Tcl list or collection of clock waveforms.

Examples

- The following commands set the clock transition assertions, and resets them and displays the results using the `report_timing` command:

```
set_clock_transition 0.123 [get_clocks WAVE]
report_timing -net -to BLK/BR2/D

# Generates the following report

Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 0.000
- Setup 0.192
+ Phase Shift 10.000
= Required Time 9.808
- Arrival Time 0.249
= Slack Time 9.559
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
+-------------------------------------------------------------------------+
| Instance   | Cell    | Arc       | Pin     | Slew  | Delay | Arrival |
|            |         |           |         |       |       | Time    |
|------------+---------+-------------+-------+-------+-------+---------|
|            |         | tclk ^    | tclk    | 0.123 |       | 0.000 |
| CG/BC1     | BUFX2   |           | A       | 0.123 | 0.000 | 0.000 |
```

```
| CG/BC1     | BUFX2  | A ^ -> Y ^ | Y       | 0.123 | 0.000 | 0.000 |
| TC1        | BUFX2  |            | A       | 0.123 | 0.000 | 0.000 |
| TC1        | BUFX2  | A ^ -> Y ^ | Y       | 0.123 | 0.000 | 0.000 |
| CG1        | AND2X4 |            | B       | 0.123 | 0.000 | 0.000 |
| CG1        | AND2X4 | B ^ -> Y ^ | Y       | 0.123 | 0.000 | 0.000 |
| BLK/blkint | BUFX2  |            | A       | 0.123 | 0.000 | 0.000 |
| BLK/blkint | BUFX2  | A ^ -> Y ^ | Y       | 0.123 | 0.000 | 0.000 |
| BLK/BR1    | DFFHQX1|            | CK      | 0.123 | 0.000 | 0.000 |

reset_clock_transition [get_clocks WAVE]
report_timing -net -to BLK/BR2/D

# Generates the following report:

Path 1: MET Setup Check with Pin BLK/BR2/CK
Endpoint: BLK/BR2/D (v) checked with leading edge of 'WAVE'
Beginpoint: BLK/BR1/Q (v) triggered by leading edge of 'WAVE'
Other End Arrival Time 0.000
- Setup 0.191
+ Phase Shift 10.000
= Required Time 9.809
- Arrival Time 0.249
= Slack Time 9.560
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
+-------------------------------------------------------------------+
| Instance | Cell      | Arc         | Pin   | Slew  | Delay | Arrival |
|                                    |                       |   Time  |
|------------+---------+-------------+-------+-------+-------+---|
|            |         | tclk ^      | tclk  | 0.000 |       | 0.000 |
| CG/BC1     | BUFX2   |             | A     | 0.000 | 0.000 | 0.000 |
| CG/BC1     | BUFX2   | A ^ -> Y ^  | Y     | 0.000 | 0.000 | 0.000 |
| TC1        | BUFX2   |             | A     | 0.000 | 0.000 | 0.000 |
| TC1        | BUFX2   | A ^ -> Y ^  | Y     | 0.000 | 0.000 | 0.000 |
| CG1        | AND2X4  |             | B     | 0.000 | 0.000 | 0.000 |
| CG1        | AND2X4  | B ^ -> Y ^  | Y     | 0.000 | 0.000 | 0.000 |
| BLK/blkint | BUFX2   |             | A     | 0.000 | 0.000 | 0.000 |
| BLK/blkint | BUFX2   | A ^ -> Y ^  | Y     | 0.000 | 0.000 | 0.000 |
| BLK/BR1    | DFFHQX1 |             | CK    | 0.000 | 0.000 | 0.000 |
---------------------------------------------------------------------
```

Related Commands

- set_clock_transition

reset_clock_tree_latency

```
reset_clock_tree_latency
[-help]
 [obj_list]
```

Resets network clock latency set using the `set_clock_latency` command in the fanout of the specified clock or pins and ports. The `reset_clock_tree_latency` command resets the latency until the root pins of clocks and generated clocks.

Use this command after building the clock tree and setting the clock network from the root to propagated mode using the `set_propagated_clock` command.

Note: You can use the `reset_clock_tree_latency` command only on command line and command script. You cannot specify it in an SDC file.

Parameters

`-help`	Prints out the command usage.
`obj_list`	Specifies list of objects - pins, ports, or clocks for which latency will be reset.

Related Commands

- `set_clock_latency`

reset_clock_uncertainty

```
reset_clock_uncertainty
[-help]
[-setup]
[-hold]
[-half_cycle_jitter | -full_cycle_jitter]
{  {-from | -rise_from | -fall_from} clksig_from_list
   {-to | -rise_to | -fall_to} clksig_to_list
 | pin_or_clock_list
}
```

Removes the assertions that were made by previous `set_clock_uncertainty` commands.

Parameters

`-help`	Prints out the command usage.
`{-from | -rise_from | -fall_from}` *`clksig_from_list`*	Removes the assertions for uncertainty value for the specified list of launching clocks. **Note:** You must specify one of these parameters with one of the `-to`, `-rise_to`, or `-fall_to` parameters. You cannot use these parameters when pins are specified.
	• `-fall_from`: Removes the uncertainty value only from the falling edge of the launching clocks.
	• `-from`: Removes the uncertainty value from both the rising and falling edges of the launching clocks.
	• `-rise_from`: Removes the uncertainty value only from the rising edge of the launching clocks.
`-half_cycle_jitter | -full_cycle_jitter`	Removes or resets the clock source jitter assertion(s) of the specified type.
`pin_or_clock_list`	Removes the target uncertainty assertions from the specified list of clocks or pins. The uncertainty value is removed from the registers clocked by the target waveform, or register pins in the fanout of the specified pin. You can specify a list of clock names, or clock root pin or port names relative to the current module. You also can specify a collection of clocks, or clock root pins or ports. **Note:** You cannot use this parameter when any of the `-from` and `-to` parameter pairs are specified.

`-setup/-hold`	Specifies whether the clock uncertainty is removed for setup or hold checks. *Default*: Removes clock uncertainty for both setup and hold check.
`{-to | -rise_to | -fall_to}` *clksig_to_list*	Removes the uncertainty value from the specified list of capture clocks. **Note:** You must specify one of these parameters with one of the `-from`, `-rise_from`, or `-fall_from` parameters. You cannot use these parameters when pins are specified.
	• `-fall_to`: Removes the uncertainty value only from the falling edge of the capture clocks.
	• `-rise_to`: Removes the uncertainty value only from the rising edge of the capture clocks.
	• `-to`: Removes the uncertainty value from both the rising and falling edges of the capture clocks.

Examples

The following commands sets and resets the clock uncertainty assertions and displays the results using the `report_timing` command:

- ```
  set_clock_uncertainty 0.345 -from [get_clocks VCLK] -to [get_clocks WAVE]report_timing -net -from tin
  # Generates the following report:
  Path 1: MET Setup Check with Pin TR1/BR1/CK
  Endpoint: TR1/BR1/D (v) checked with leading edge of 'WAVE'
  Beginpoint: tin (v) triggered by trailing edge of 'VCLK'
  Other End Arrival Time 0.000
  - Setup 0.192
  + Phase Shift 10.000
  - Uncertainty 0.345 <---
  = Required Time 9.463
  - Arrival Time 5.877
  = Slack Time 3.586
  ```

- ```
  reset_clock_uncertainty -from [get_clocks VCLK] -to [get_clocks WAVE]report_timing -net -from tin
  # Generates the following report:
  Path 1: MET Setup Check with Pin TR1/BR1/CK
  Endpoint: TR1/BR1/D (v) checked with leading edge of 'WAVE'
  Beginpoint: tin (v) triggered by trailing edge of 'VCLK'
  Other End Arrival Time 0.000
  ```

```
- Setup 0.192
+ Phase Shift 10.000
= Required Time 9.808
- Arrival Time 5.877
= Slack Time 3.931
```

Related Commands

- set_clock_uncertainty

reset_data_check

```
reset_data_check
[-help]
{-from | -rise_from | -fall_from} pin_or_port_list
{-to | -rise_to | -fall_to} pin_or_port_list
[-setup]
[-hold]
[-clock clock_object]
```

Removes specified data-to-data checks that you have specified using the `set_data_check` command.

Parameters

`-help`	Prints out the command usage.
`-clock` *clock_object*	Removes the data check with a related (`-from`) pin event that is triggered by the specified clock. Use this parameter only if the original `set_data_check` constraint also used the `-clock` parameter to specify an explicit clock relationship.
`-from` |`-rise_from` | `-fall_from` *pin_or_port_list*	Specifies the related pins of the data-to-data checks to be removed. The *pin_or_port_list* list can contain either object IDs or hierarchical names relative to the current module. Leaf design cells (instances) can also be specified. Use only one `-from`, `-rise_from`, or `-fall_from` parameter per command. By default, the `-from` parameter removes checks for both the rising and the falling edges. Using `-rise_from` parameter removes checks at the rising edge of the signal on the from pins. Using `-fall_from` parameter removes checks at the falling edge of the signal on the from pins.
`-setup`/`-hold`	Specifies whether the data check to be removed is for setup or hold check. If you do not specify either `-setup` or `-hold` option, both setup and hold checks are removed.

`-to	-rise_to	-fall_to` *`pin_or_port_list`*	Specifies the constrained pins of data-to-data checks to be removed. The *`pin_or_port_list`* list can contain either object IDs or hierarchical names relative to the current module. Leaf design cells (instances) can also be specified. Use only one `-to`, `-rise_to`, or `-fall_to` parameter per command. By default, the `-to` parameters removes both the rising and the falling edges. Using `-rise_to` parameter removes the rising edge of the signal on the to pins. Using `-fall_to` parameter removes the falling edge of the signal on the to pins.

Examples

- The following commands set and reset the data checks from `port30` to `port28`:

```
set_data_check 0.345 -rise_from port30 -to port28
report_timing -rise_from port30 -to port28
```

The following report is displayed:

```
Path 1: VIOLATED Data To Data Setup Check with Pin port30
Endpoint: port28 (v) checked with leading edge of 'PH1'
Beginpoint: U2/Q (v) triggered by leading edge of 'PH1'
Other End Arrival Time 0.465
- Data Check Setup 0.345
+ Phase Shift 0.000
= Required Time 0.120
- Arrival Time 0.309
= Slack Time -0.189
```

```
reset_data_check -rise_from port30 -to port28
report_timing -rise_from port30 -to port28
```

The following information is displayed:

```
No constrained timing paths with given description found.
Paths may be unconstrained (try '-unconstrained' option) or may not exist.
```

Related Commands

- `set_data_check`

reset_disable_clock_gating_check

```
reset_disable_clock_gating_check
[-help]
pin_or_inst_list
```

Enables clock gating checks for the specified instances or pins.

Parameters

`-help`	Prints out the command usage.
`pin_or_inst_list`	Specifies the list of disabled pins or instances that are to be enabled.

Related Commands

- `set_disable_clock_gating_check`

reset_disable_timing

```
reset_disable_timing
[-help]
[-from pin_name -to pin_name]
obj_list
```

Restores timing arcs that were disabled using the `set_disable_timing` command.

Parameters

`-help`	Prints out the command usage.
`-from` *pin_name* `-to` *pin_name*	Specifies the arc(s) that are to be restored. Specify the `-from` and `-to` options together. You can specify the `-from` and `-to` parameters when the *obj_list* contains instance or library cell objects. Do not specify the `-from` and `-to` parameters when the *obj_list* includes instance pin or library cell pin objects. The `-from` and `-to` options are optional. If not specified, all cell arcs found in the instance or library cell objects are restored.
obj_list	Specifies the instance, instance pin, library arc, library cell, or library cell pin objects. If the *obj_list* includes instance pin or library cell pin objects, all cell arcs attached to the specified pin objects are restored. If the *obj_list* contains instance or library cell objects, and the `-from` and `-to` parameters are not specified, all arcs found in the specified instance or library cell objects are restored.

Examples

- The following commands sets and resets the timing arcs and displays the resulting reports:

```
tempus > set_disable_timing -from A -to Y [get_cells b1]
tempus > report_inactive_arcs

-------------------------------------------------------------
Flags :              const                propagated constant
                     snipped              loop snipped arcs
                     disable              set_disable_timing
                     disable_clock_gating set_disable_clock_gating
                     library_disable      set_disable_cell_timing
                     Missing_Phase        No Arc Phase Data
From       To       DisableType  ArcType         Reason
-------------------------------------------------------------
b1/A ^     b1/Y ^   disable      combinational   User Disable
b1/A v     b1/Y v   disable      combinational   User Disable
-------------------------------------------------------------
```

- The following commands disable the timing report header:

```
tempus > set_db timing_disable_report_header_info true
tempus > report_inactive_arcs

-------------------------------------------------------------------------
Flags :                  const                    propagated constant
                         snipped                  loop snipped arcs
                         disable                  set_disable_timing
                         disable_clock_gating     set_disable_clock_gating
                         library_disable          set_disable_cell_timing
                         Missing_Phase            No Arc Phase Data
From       To        DisableType    ArcType                 Reason
-------------------------------------------------------------------------
u8/CK ^    u8/D ^    const          hold_rising_clk_rise    u8/D = 1
u8/CK ^    u8/D v    const          hold_falling_clk_rise   u8/D = 1
u8/CK ^    u8/D ^    const          setup_rising_clk_rise   u8/D = 1
u8/CK ^    u8/D v    const          setup_falling_clk_rise  u8/D = 1
-------------------------------------------------------------------------
```

- The following commands disable the timing arcs from CK to Q for instance U8, then reset the assertions using the instance reference:

```
tempus > set_disable_timing -from CK -to Q [get_cells u8]
tempus > report_inactive_arcs
```

```
-------------------------------------------------------------------
Flags :                 const                 propagated constant
                        snipped               loop snipped arcs
                        disable               set_disable_timing
                        disable_clock_gating  set_disable_clock_gating
                        library_disable       set_disable_cell_timing
                        Missing_Phase         No Arc Phase Data
From       To       DisableType     ArcType       Reason
-------------------------------------------------------------------
u8/CK ^    u8/Q ^   disable         rising_edge   User Disable
u8/CK ^    u8/Q v   disable         rising_edge   User Disable
-------------------------------------------------------------------
```

- The following commands disable the timing arcs from CK to Q for instance U8, then reset the assertions using the instance reference:

```
tempus > reset_disable_timing u8
tempus > report_inactive_arcs

-------------------------------------------------------------------------
Flags :                   const                 propagated constant
                          snipped               loop snipped arcs
                          disable               set_disable_timing
                          disable_clock_gating  set_disable_clock_gating
                          library_disable       set_disable_cell_timing
                          Missing_Phase         No Arc Phase Data
From      To       DisableType     ArcType                   Reason
-------------------------------------------------------------------------
u8/CK ^   u8/D ^   const           hold_rising_clk_rise      u8/D = 1
u8/CK ^   u8/D v   const           hold_falling_clk_rise     u8/D = 1
u8/CK ^   u8/D ^   const           setup_rising_clk_rise     u8/D = 1
u8/CK ^   u8/D v   const           setup_falling_clk_rise    u8/D = 1
-------------------------------------------------------------------------
```

Related Commands

- set_disable_timing
- report_inactive_arcs

reset_drive

```
reset_drive
[-help]
port_list
```

Resets the drive previously asserted with the `set_drive` command on input ports. The `reset_drive` command removes assertions for all rise/fall, minimum/maximum combinations for which the assertion exists.

Parameters

`-help`	Prints out the command usage.
`port_list`	Specifies a list of input or bidirectional ports for which the drive resistance is reset.

Related Commands

- `set_drive`

reset_driving_cell

```
reset_driving_cell
[-help]
[-rise]
[-fall]
[-min]
[-max]
port_list
```

Resets the driving cell assertions applied with the `set_driving_cell` command for the given rise/fall, minimum/maximum combination on ports.

If `-rise`, `-fall`, `-min`, or `-max` parameters are not specified, the `reset_driving_cell` command resets the assertion for all rise/fall, minimum/maximum combinations for which the assertion exists.

Parameters

`-help`	Prints out the command usage.
`-rise`	Specifies the rising drive assertions.
`-fall`	Specifies the falling drive assertions.
`-min`	Specifies the early drive assertion.
`-max`	Specifies the late drive assertion.
`port_list`	Specifies a list of ports for which the driving cell assertions are reset.

Related Commands

- `set_driving_cell`

reset_generated_clock

```
reset_generated_clock
[-help]
{-all | clock_list}
```

Removes generated clock assertions previously created using the `create_generated_clock` command.

To use the `reset_generated_clock` command you must specify the `set_interactive_constraint_modes` constraints for the applicable modes. You can also include these commands in the SDC constraint file - applicable only for the views that include that SDC.

Note: The generated clocks may originate from the library files dotlib - you can use `reset_generated_clock` from the command line, or add it to all the SDC files in use.

Parameters

`-help`	Prints out the command usage.
`-all`	Removes all the generated clocks in the design.
`clock_list`	Specifies the list of clocks (in collection format) for which the generated clock assertions are to be removed.

Examples

- Here are some examples.
  - The following command removes the generated clock assertions on all the generated clocks `RC/Q`:

```
tempus > report_clocks

# The following report is displayed:

--------------------------------------------------------------------------
                                Clock Descriptions
--------------------------------------------------------------------------
                                                          Attributes
--------------------------------------------------------------------------
Clock Name  Source     Period     Lead     Trail    Generated   Propagated
--------------------------------------------------------------------------
CK1         CLK1       20.000     0.000    10.000     n           n
CK2         CLK1       10.000     0.000    5.000      n           n
GCLK        u8/Q       20.000     0.000    10.000     y           y
--------------------------------------------------------------------------

Generated-Clock Descriptions
-------------------------------------------------------------------------------
Name  Generated    Master      Master-clock  Invert  Freq.      Duty-Cycle Edges Edge-Shift
```

```
      Source(pin)  Source(pin)                        Multiplier
-------------------------------------------------------------------------------
GCLK  u8/Q         CK1         CLK1           n      1/2        -          -     -
-------------------------------------------------------------------------------

tempus > set_interactive_constraint_modes [all_constraint_modes -active]
tempus > reset_generated_clock [get_clocks GCLK]
tempus > report_clocks

# The following report is displayed:
-------------------------------------------------------------------------
                         Clock Descriptions
-------------------------------------------------------------------------
                                                  Attributes
-------------------------------------------------------------------------
Clock Name   Source   Period   Lead    Trail    Generated     Propagated
-------------------------------------------------------------------------
CK1          CLK1    20.000   0.000   10.000   n             n
CK2          CLK1    10.000   0.000   5.000    n             n
GCLK         u8/Q    20.000   0.000   10.000   y             y
-------------------------------------------------------------------------
Generated-Clock Descriptions
-------------------------------------------------------------------------------
Name  Generated   Master      Master-clock Invert  Freq.      Duty-Cycle Edges Edge-Shift
      Source(pin) Source(pin)                     Multiplier
-------------------------------------------------------------------------------
```

Related Commands

- create_generated_clock
- report_clocks
- set_interactive_constraint_modes

reset_ideal_latency

```
reset_ideal_latency
[-help]
[-min]
[-max]
[-rise]
[-fall]
pin_or_port_list
```

Resets the ideal latency constraints, specified using the `set_ideal_latency` command, on instance pins or ports.

Parameters

`-help`	Prints out the command usage.
`-fall`	Resets the ideal latency for the falling edge.
`-max`	Resets the ideal latency for the maximum corner.
`-min`	Resets the ideal latency for the minimum corner.
`pin_or_port_list`	Resets a list of objects - instance pins, ports, or nets.
`-rise`	Resets the ideal latency for the rising edge.

Related Commands

- `set_ideal_latency`

reset_ideal_network

```
reset_ideal_network
[-help]
obj_list
```

Resets the ideal network constraints, specified using the `set_ideal_network` command, on pins, ports, or nets.

Note: The `reset_ideal_network` command applies to all the active views and is not view/mode-based.

Parameters

`-help`	Prints out the command usage.
`obj_list`	Resets a list of objects - instance pins, ports, or nets.

Related Commands

- `set_ideal_network`

reset_ideal_transition

```
reset_ideal_transition
[-help]
[-min]
[-max]
[-rise]
[-fall]
pin_or_port_list
```

Resets the ideal transition constraint, specified using the `set_ideal_transition` command, on instance pins or ports.

Parameters

`-help`	Prints out the command usage.
`-fall`	Resets the ideal transition for the falling edge.
`-max`	Resets the ideal transition for the maximum corner.
`-min`	Resets the ideal transition for the minimum corner.
`pin_or_port_list`	Resets a list of objects - instance pins, ports, or nets.
`-rise`	Resets the ideal transition for the rising edge.

Related Commands

- `set_ideal_transition`

reset_input_delay

```
reset_input_delay
[-help]
[-clock clk_name]
[-clock_fall]
[-rise] [-fall]
[-max] [-min]
[-level_sensitive]
pin_or_port_list
```

Resets previously specified input delay assertions.

Parameters

`-help`	Prints out the command usage.
`-clock` *clock_name*	Specifies the name of the clock. *Default*: The asynchronous (`@`) clock. If you specify multiple clocks, an error occurs and the constraint is ignored. When you specify the `-clock` parameter, the constraints specified for a pin with matching `-clock` parameter are reset. If you do not specify the `-clock` parameter, all the constraints applied on the specified pin, with or without the `-clock` parameter specification, are reset.
`-clock_fall`	Removes the assertions for the delay that is relative to the falling edge of the clock. You must specify the `-clock` parameter with this parameter. Default: Removes the delay assertions relative to the falling edge.
`-fall`	Resets the input delay for the falling edge at the input port. If both the `-rise` and `-fall` options are not specified, then the input delay is reset on both the edges.
`-level_sensitive`	Resets input delay that is launched by a positive level sensitive latch. When you specify the `-level_sensitive` parameter, the command resets all the `set_input_delay` constraints specified with the `-level_sensitive` parameter. If `-level_sensitive` parameter is not specified, the constraints specified for the same pin, with or without the `-level_sensitive` parameter, are reset. **Note:** You must specify the `-clock` parameter with this parameter.
`-max`	Removes the delay assertions that refer to the setup analysis. If you do not specify the `-max` parameter, the input delay assertions for both setup and hold analysis are removed.

`-min`	Removes the delay assertions that refer to the hold analysis. If you do not specify the `-min` parameter, the input delay assertions for both setup and hold analysis are removed.
`pin_or_port_list`	Specifies a single or multiple pins or ports for which the input delay is reset. To specify multiple pins, enclose the list with curly braces (`{}`) and separate the pin information with white space.
`-rise`	Resets the input delay for the rising edge at the input port. If both `-rise` and `-fall` options are omitted, the input delay is reset on both the edges.

Related Commands

- `set_input_delay`

reset_load

```
reset_load
[-help]
port_net_list
```

Resets the existing `set_load` assertions specified on nets or ports.

Parameters

`-help`	Prints out the command usage.
`port_net_list`	Specifies a list of nets or ports in the top cell for which assertions are reset.

Related Commands

- `set_load`

reset_max_capacitance

```
reset_max_capacitance
[-help]
obj_list
```

Resets maximum capacitance limit specified (using the the `set_max_capacitance` command) on the objects in the object list. The valid objects in the object list are primary ports, clocks, and current design.

Parameters

`-help`	Prints out the command usage.
`obj_list`	Specifies the list of ports, modules, and/or clock waveforms for which assertions are reset.

Related Commands

- `set_max_capacitance`

reset_max_fanout

```
reset_max_fanout
[-help]
obj_list
```

Resets maximum fanout limit specified (using the `set_max_fanout` command) on the objects in the object list. The valid objects in the object list are input and bidirectional ports and current design.

Parameters

`-help`	Prints out the command usage.
`obj_list`	Specifies the list of ports or modules for which assertions are reset.

Related Commands

- `set_max_fanout`

reset_max_time_borrow

```
reset_max_time_borrow
[-help]
obj_list
```

Resets the specified maximum time borrow limit of the specified objects.

Parameters

`-help`	Prints out the command usage.
`obj_list`	Specifies the list of pins, instances, or clocks for which the time borrow limit is reset.

Related Commands

- `set_max_time_borrow`

reset_max_transition

```
reset_max_transition
[-help]
obj_list
```

Resets the maximum slew (transition) limit specified using the `set_max_transition` command (from the SDC or the Tempus command line) on the objects in the given object list. The valid objects in the object list are primary ports, clocks, and current design.

The maximum transition limit that is set by the user overrides the dotlib value so that sharper slews are implemented on the timing sensitive nets, such as clocks. A violation due to the `set_max_transition` setting does not impact the timing accuracy.

Note: For command-line usage of this command, you need to use `set_interactive_constraint_modes`, for example, `set_interactive_constraint_modes [all_constraint_modes -active]`

Parameters

`-help`	Prints out the command usage.
`obj_list`	Specifies the list (or collection) of ports, modules, and/or clock waveforms for which assertions are to be reset. For example, `reset_max_transition [get_ports *]`

Related Commands

- `set_max_transition`

reset_min_capacitance

```
reset_min_capacitance
[-help]
obj_list
```

Resets minimum capacitance limit specified (using the the `set_min_capacitance` command) on objects in the object list. The valid objects in the object list are primary ports, clocks, and the current design.

Parameters

`-help`	Prints out the command usage.
`obj_list`	Specifies the list of ports, modules, and/or clock waveforms for which assertions are reset.

Related Commands

- `set_min_capacitance`

reset_min_fanout

```
reset_min_fanout
[-help]
obj_list
```

Resets minimum fanout limit specified (using the `set_min_fanout` command) on the objects in the object list. The valid objects in the object list are the inputs, bi-directional ports, and the current design.

Parameters

`-help`	Prints out the command usage.
`obj_list`	Specifies the list of ports or modules for which assertions are reset.

Related Commands

- `set_min_fanout`

reset_min_pulse_width

```
reset_min_pulse_width
[-help]
obj_list
[-high]
[-low]
[-waveform_aware_type {absolute | source_width_ratio | all}]
```

Resets minimum pulse width constraint on the specified input ports of the top cell and the specified modules.

Parameters

`-help`	Prints out the command usage.
`-low`	Resets the minimum pulse width checks that are applied to low signal levels only.
`-high`	Resets the minimum pulse width checks that are applied to high signal levels only.
obj_list	Specifies a list of pins or clock waveforms for which the minimum pulse width is to be reset. If the list is not specified, the minimum pulse width check applies to all the pins in the clock tree of the current design. If a clock waveform is specified, all the pins driven by the clock are affected.
`-waveform_aware_type {absolute | source_width_ratio | all}`	Resets the regular and waveform-aware pulse width assertions.

Related Commands

- `set_min_pulse_width`

reset_min_transition

```
reset_min_transition
[-help]
obj_list
```

Resets minimum slew (transition) limit specified (using the `set_min_transition` command) on the objects in the object list. The valid objects in the object list are primary ports, clocks, and current design.

Parameters

`-help`	Prints out the command usage.
`obj_list`	Specifies the list of ports, modules, and/or clock waveforms for which assertions are reset.

Related Commands

- `set_min_transition`

reset_mode

```
reset_mode
[-help]
[-type cell]
mode_list
inst_list
```

Resets the Liberty timing library modes for a specified instance.

Library modes are set using the set_mode command. By default, all library modes in the timing library are active. If you specify one or more library modes using the `set_mode` command, only the timing arcs consistent with the specified mode(s) are active for the instance. All other library modes become inactive. When you reset the library mode for the instance, all library modes become active again.

Parameters

`-help`	Prints out the command usage.
`inst_list`	Specifies the instances to which to reset the library modes.
`mode_list`	Specifies the name of the library modes to reset.
`-type cell`	Resets the library mode at the instance level. **Note:** Setting the library mode at the instance level is the only form of `set_mode` that is currently supported. This parameter exists to support constraints coming from external sources.

Examples

- The following command specifies that only the timing arcs consistent with the `test_A` library mode are active for instances of b1:
  The test_A library mode is made inactive.

  ```
  tempus > set_mode test_mode [get_cells BUF]
  ```

  The following command reports the status of all library modes associated with BUF 'b1' . It shows that test_A is active and test_B is inactive.

  ```
  tempus > report_mode b1
  -------------------------------
  Mode Group test_mode of b1
  -------------------------------
  Mode     Status    Condition
  Name
  -------------------------------
  test_B   Inactive  !(Y)
  test_A   ACTIVE    Y
  -------------------------------
  ```

  The following command resets the library mode for instances of b1 :

  ```
  tempus > reset_mode test_A b1
  ```

  If you generate a report of the library mode status for b1, it shows that both modes are now active:

  ```
  tempus > report_mode b1
  -----------------------------
  Mode Group test_mode of b1
  -----------------------------
  Mode    Status   Condition
  Name
  -----------------------------
  test_B  ACTIVE    !(Y)
  test_A  ACTIVE    Y
  -----------------------------
  ```

Related Commands

- set_mode
- report_mode

reset_output_delay

```
reset_output_delay
[-help]
port_or_pin_list
[-clock clock_name]
[-clock_fall]
[-fall]
[-level_sensitive]
[-max]
[-min]
[-rise]
```

Resets previously specified output delay assertions.

Parameters

`-help`	Prints out the command usage.
`-clock` *clock_name*	Specifies the name of the clock. *Default*: The asynchronous (`@`) clock. If you specify multiple clocks, an error occurs and the constraint is ignored. When you specify the `-clock` parameter, the constraints specified for a pin with matching `-clock` parameter are reset. If you do not specify the `-clock` parameter, all the constraints applied on the specified pin, with or without the `-clock` parameter specification, are reset.
`-clock_fall`	Removes the assertions for the delay that is relative to the falling edge of the clock. You must specify the -clock parameter with this parameter. *Default:* Removes the delay assertions relative to the rising edge.
`-fall`	Resets the output delay for the falling edge at the output port. If both `-rise` and `-fall` options are omitted, the output delay is reset on both the edges.
`-level_sensitive`	Resets output delay that is launched by a positive level sensitive latch. When you specify the `-level_sensitive` parameter, the command resets all the `set_output_delay` constraints specified with the `-level_sensitive` parameter. If `-level_sensitive` parameter is not specified, the constraints specified for the same pin, with or without the `-level_sensitive` parameter, are reset. **Note:** You must specify the `-clock` parameter with this parameter.

`-max`	Removes the delay assertions that refer to the setup analysis. If you do not specify the `-max` parameter, the output delay assertions for both setup and hold analysis are removed.
`-min`	Removes the delay assertions that refer to the hold analysis. If you do not specify the `-min` parameter, the output delay assertions for both setup and hold analysis are removed.
pin_or_port_list	Specifies a single or multiple pins or ports for which the output delay is reset. To specify multiple pins, enclose the list with curly braces (`{}`) and separate the pin information with white space.
`-rise`	Resets the output delay for the rising edge at the output port. If both `-rise` and `-fall` options are omitted, the output delay is reset on both the edges.

Examples

- The following example sets the output delays for the output port Z. The output signal is required at Z at 2 units before the rising edge of clock Clk1, and at 2.5 units before falling edge of clock Clk2:

```
tempus > set_output_delay 2 Z -clock Clk1
tempus > set_output_delay 2.5 Z -clock Clk2 -clock_fall -add_delay
```

- The following command will reset the output delay for the rising edge of clock Clk2:

```
tempus > reset_output_delay Z -clock Clk2

report_timing -to_rise Z -clock_to Clk2
#################################################################
# Generated by: Cadence Tempus 18.11-s094_1
# OS: Linux x86_64(Host ID noi-rakeshp)
# Generated on: Thu Apr 18 09:50:52 2019
# Design: top
# Command: report_timing -to_rise Z -clock_to Clk2
#################################################################
No constrained timing paths with given description found.
Paths may be unconstrained (try '-unconstrained' option) or may not exist.

report_timing -to_fall Z -clock_to Clk2
#################################################################
Path 1: VIOLATED Late External Delay Assertion
Endpoint: Z (v) checked with trailing edge of 'Clk2'
Beginpoint: dff3/Q (v) triggered by leading edge of 'Clk'
Path Groups: {vclk2}
Other End Arrival Time 2.000
- External Delay 2.500
+ Phase Shift 0.000
```

```
= Required Time -0.500
- Arrival Time 0.165
= Slack Time -0.665
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.052
= Beginpoint Arrival Time 0.052
------------------------------------------------------------
Instance Arc Cell Delay Arrival Required
Time Time
------------------------------------------------------------
dff3 CK ^ - - 0.052 -0.613
dff3 CK ^ -> Q v DFFHQX1 0.113 0.165 -0.500
- Z v - 0.000 0.165 -0.500
------------------------------------------------------------
```

- The following command resets the output delay for clock Clk2 with respect to the falling edge:

  ```
  tempus > reset_output_delay Z -clock Clk2 -clock_fall
  ```

- The following command will reset all the output delays with respect to all the edges for both the clocks - Clk1 and Clk2:

  ```
  tempus > reset_output_delay Z
  ```

Related Commands

- set_output_delay

reset_path_exception

```
reset_path_exception
[-help]
[-exact]
[-type {false_path | multicycle | path_delay}]
[-rise]
[-fall]
[{-from | -rise_from | -fall_from} from_list]
[{-through | -rise_through | -fall_through} through_list]
[{-to | -rise_to | -fall_to} to_list]
[-setup | -hold]
[-all]
```

Removes any previously set path exceptions (or path exception of the specified type) for the given paths. To remove all path exceptions, use the `-all` option with the `reset_path_exception` command.

Parameters

`-help`	Prints out the command usage.
`-all`	Removes all path exceptions. This option cannot be used with the `-from`, `-through` and `-to` options.
`-exact`	Forces exact matching of the constraints. When this parameter is specified, the reset settings (-from/-through/-to) match with the original constrained path settings.
`-fall`	Applies the assertion at the falling edge of the signal on the path endpoints.
`{-from | -rise_from | -fall_from}` *from_list*	Specifies pins at the start of the path exceptions that are to be reset.
`-rise`	Applies the assertion at the rising edge of the signal on the path endpoints.
`{-through | -rise_through | -fall_through}` *through_list*	Specifies the pins that the path to be reset goes through.
`{-to | -rise_to | -fall_to}` *to_list*	Specifies pins at the end of the path exceptions that are to be reset.
`-setup | -hold`	Resets only the setup (or hold) paths. The `-setup`/`-hold` parameters should be used to reset maximum or minimum delay that is set using the `set_max_delay` and `set_min_delay` commands. *Default*: Both `-setup` and `-hold` are reset.
`-type {false_path | multicycle | path_delay}`	Removes only path exceptions of the specified type. *Default*: All path exception assertions are reset.

Examples

- In the example below the rising path from R1 to R2 has three different path exceptions asserted on it - they are listed in priority from lowest to highest. As shown in the reports below, the false path is the highest precedence and so no path is reported.

```
tempus > set_multicycle_path -from R1/CK -rise_to R2/D 2
tempus > set_max_delay 3 -from R1/CK -rise_to R2/D
tempus > set_false_path -from R1/CK -rise_to R2/D

tempus > report_timing -from R1/CK -rise_to R2/D
```

  The following information is displayed:

```
No constrained timing paths with given description found.
Paths may be unconstrained (try '-unconstrained' option) or may not exist.

tempus > report_path_exceptions
```

  The following report is displayed:

```
---------------------------------------
From      To        Early        Late
---------------------------------------
R1/CK     ^ R2/D    false        false
---------------------------------------
```

  To report ignored path exceptions, you can use the following command:

```
tempus > report_path_exception -ignored

-------------------------------------------
From     To       Early      Late
-------------------------------------------
R1/CK   ^ R2/D               delay 3 ignored
R1/CK   ^ R2/D    add 0 ignored
R1/CK   ^ R2/D               add 1 ignored
-------------------------------------------
```

- In this example, the false path exception is removed from the path. This causes the max path delay assertion to become active - as it is the next highest precedence.

```
tempus > reset_path_exception -from R1/CK -rise_to R2/D -type false_path
tempus > report_path_exceptions
```

  The following report is displayed:

```
-------------------------------------
From      To       Early     Late
-------------------------------------
R1/CK    ^ R2/D             delay 3
R1/CK    ^ R2/D    add 0
-------------------------------------

tempus > report_timing -from R1/CK -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Setup Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.097
+ Path Delay 3.000
= Required Time 2.903
- Arrival Time 0.243
= Slack Time 2.660
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
-------------------------------------------------------------------
Load    Slew    Delay    Arrival    Cell    Arc           Pin
                         Time
-------------------------------------------------------------------
0.026   0.003            0.000              CK ^          R1/CK ->
0.011   0.063   0.187    0.187      DFF     CK ^ -> Q ^   R1/Q
0.003   0.042   0.056    0.243      BUF     A ^ -> Y ^    B1/Y
0.003   0.042   0.000    0.243      DFF     D ^           R2/D ->
-------------------------------------------------------------------
```

- In this example, the set_max_delay exception is removed, leaving only the set_multicycle_path exception as active.

```
tempus > reset_path_exception -from R1/CK -rise_to R2/D -type path_delay
tempus > report_timing -from R1/CK -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Setup Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.097
+ Phase Shift 20.000
+ Cycle Adjustment 20.000 <----- Multi-Cycle timing
= Required Time 39.903
- Arrival Time 0.243
= Slack Time 39.660
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
-------------------------------------------------------------------
Load    Slew     Delay    Arrival    Cell   Arc           Pin
                          Time
-------------------------------------------------------------------
0.026   0.003             0.000             CK ^          R1/CK ->
```

```
0.011   0.063    0.187    0.187      DFF    CK ^ -> Q ^   R1/Q
0.003   0.042    0.056    0.243      BUF    A ^ -> Y ^    B1/Y
0.003   0.042    0.000    0.243      DFF    D ^           R2/D ->
-------------------------------------------------------------------
```

- In this example, the multi-cycle path assertion is removed, and the analysis reverts back to single-cycle timing

```
tempus > reset_path_exception -rise_to R2/D -type multicycle
tempus > report_path_exceptions
```

None reported

```
tempus > report_timing -from R1 -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Setup Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.097
+ Phase Shift 20.000 <--- Single-cycle timing
= Required Time 19.903
- Arrival Time 0.243
= Slack Time 19.660
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
-------------------------------------------------------------------
Load    Slew    Delay    Arrival    Cell    Arc           Pin
                         Time
-------------------------------------------------------------------
0.026   0.003   0.000                       CK ^          R1/CK ->
0.011   0.063   0.187   0.187       DFF     CK ^ -> Q ^   R1/Q
0.003   0.042   0.056   0.243       BUF     A ^ -> Y ^    B1/Y
0.003   0.042   0.000   0.243       DFF     D ^           R2/D ->
-------------------------------------------------------------------
```

- The example below shows how the `-hold` and `-setup` options can be used to control the effect of issuing `reset_path_exception`. The example will issue multi-cycle path constraints against both setup and hold modes, and then reset only the setup side.

```
tempus > set_multicycle_path 3 -setup -rise_from [get_clocks PH1] -rise_through U2/A
tempus > set_multicycle_path 2 -hold -rise_from [get_clocks PH1] -rise_through U2/A
tempus > report_timing -late -from R1 -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Setup Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
```

```
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.097
+ Phase Shift 20.000
+ Cycle Adjustment 40.000 <-- 3 cycles
= Required Time 59.903
- Arrival Time 0.243
= Slack Time 59.660
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
-------------------------------------------------------------------
Load     Slew     Delay    Arrival   Cell     Arc           Pin
                           Time
-------------------------------------------------------------------
0.026   0.003    0.000                        CK ^          R1/CK ->
0.011   0.063    0.187     0.187     DFF      CK ^ -> Q ^   R1/Q
0.003   0.042    0.056     0.243     BUF      A ^ -> Y ^    B1/Y
0.003   0.042    0.000     0.243     DFF      D ^           R2/D ->
-------------------------------------------------------------------
tempus > report_timing -early -from R1 -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Hold Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
+ Hold -0.046
+ Phase Shift 0.000
- Cycle Adjustment 0.000 <- Adjusted 2 cycles
= Required Time -0.046
Arrival Time 0.243
Slack Time 0.289
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
-------------------------------------------------------------------
Load    Slew     Delay     Arrival     Cell     Arc           Pin
                           Time
-------------------------------------------------------------------
0.026   0.003   0.000                          CK ^          R1/CK ->
0.011   0.063   0.187      0.187       DFF      CK ^ -> Q ^   R1/Q
0.003   0.042   0.056      0.243       BUF      A ^ -> Y ^    B1/Y
0.003   0.042   0.000      0.243       DFF      D ^           R2/D ->
-------------------------------------------------------------------
tempus > reset_path_exception -setup -rise_from [get_clocks CLK_W_1] -rise_through B1/A
tempus > report_timing -late -from R1 -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Setup Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
- Setup 0.097
+ Phase Shift 20.000 <-- Reset to single-cycle timing
= Required Time 19.903
- Arrival Time 0.243
= Slack Time 19.660
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
-------------------------------------------------------------------------
Load     Slew     Delay     Arrival     Cell     Arc      Pin
                            Time
-------------------------------------------------------------------------
0.026   0.003    0.000                            CK ^           R1/CK ->
0.011   0.063    0.187     0.187       DFF        CK ^ -> Q ^    R1/Q
0.003   0.042    0.056     0.243       BUF        A ^ -> Y ^     B1/Y
0.003   0.042    0.000     0.243       DFF        D ^            R2/D ->
-------------------------------------------------------------------------
tempus > report_timing -early -from R1 -rise_to R2/D
```

The following report is displayed:

```
Path 1: MET Hold Check with Pin R2/CK
Endpoint: R2/D (^) checked with leading edge of 'CLK_W_1'
Beginpoint: R1/Q (^) triggered by leading edge of 'CLK_W_1'
Path Groups: {CLK_W_1}
Other End Arrival Time 0.000
+ Hold -0.046
+ Phase Shift 0.000
- Cycle Adjustment 40.000 <-- Still 2 cycle adjustment
= Required Time -40.046 (from new setup edges)
Arrival Time 0.243
Slack Time 40.289
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.000
= Beginpoint Arrival Time 0.000
-------------------------------------------------------------------------
Load     Slew      Delay     Arrival     Cell     Arc           Pin
                             Time
-------------------------------------------------------------------------
0.026    0.003     0.000                            CK ^          R1/CK ->
0.011    0.063     0.187     0.187       DFF        CK ^ -> Q ^   R1/Q
0.003    0.042     0.056     0.243       BUF        A ^ -> Y ^    B1/Y
0.003    0.042     0.000     0.243       DFF        D ^           R2/D ->
-------------------------------------------------------------------------
```

- Consider the following `set_false_path` constraint:

  ```
  tempus > set_false_path -from {u1/Y u2/Y} -to {u3/Y}
  ```

  When you specify `reset_path_exception -from {u1/Y}` command without the `-exact` parameter, the command outputs the following:

  ```
  tempus > set_false_path -from {u2/Y} -to {u3/Y}
  ```

  The command `reset_path_exception -from {u1/Y} -exact` outputs the following:

  ```
  tempus > set_false_path -from {u1/Y u2/Y} -to {u3/Y}
  ```

  In this case (when `-exact` parameter is used), the path exception is not reset because differential matching is not allowed.

Related Topics

- Path Exception Priorities

Related Commands

- `set_false_path`
- `report_path_exceptions`

reset_path_group

```
reset_path_group
[-help]
[-name group_name]
[-all]
```

Removes the specified path groups or completely removes all the groups.

Parameters

`-help`	Prints out the command usage.
`-all`	Removes all path groups.
`-name` *`group_name`*	Removes the named group completely.

Examples

- The following command removes the group named GRPA:
  ```
  tempus > reset_path_group -name GRPA
  ```

Related Commands

- `group_path`

reset_pll_timing

```
reset_pll_timing
[-help]
inst
-pll_output_clock clock name or ID
-pll_output_pin output pin name
```

Resets the phase locked loop (PLL) that is used to adjust the delay on the PLL output clock waveform.

Parameters

`-help`	Prints out the command usage.
inst	Specifies the instance on which we will search reference pin, feedback pin, or output pin.
`-pll_output_clock` *clock name or ID*	Specifies the PLL output clock.
`-pll_output_pin` *output pin name*	Specifies the PLL output pin.

Related Commands

- `set_pll_timing`

reset_propagated_clock

```
reset_propagated_clock
[-help]
pin_or_clock_list
```

Removes the propagated clock assertion for the specified clock waveforms or pins.

The ideal or propagated status of clock endpoints in the fanout of the specified pin/clock list is determined by the remaining clock latency propagated clock assertions in the clock network.

When new analysis views are used, the new corners do not have the source latency updated as per the clock propagation (`update_io_latency`) settings, so a reset and re-calculation is required.

Note: You must specify `set_interactive_constraint_modes` constraint prior to running the `reset_propagated_clock` command.

Parameters

`-help`	Prints out the command usage.
`pin_or_clock_list`	Specifies a list of pins or clocks for which you want to reset the clock propagation mode to ideal mode. The *`pin_or_clock_list`* parameter can be a collection, however, no hierarchical pins are allowed in the list.

Examples

- The following command sets the `CLK1` clock waveform to `propagated` mode:

  ```
  tempus > set_propagated_clock CLK1
  ```

- The following command resets the `CLK1` clock waveform back to `ideal` mode:

  ```
  tempus > reset_propagated_clock CLK1
  ```

- The following commands set the `CLK1` clock to `propagated` mode and then reports them:

  ```
  tempus > set_propagated_clock [get_clocks CLK1]
  tempus > report_clocks
  ```

  The following report is displayed:

  ```
  ------------------------------------------------------------------------
                             Clock Descriptions
  ------------------------------------------------------------------------
                                                          Attributes
  ------------------------------------------------------------------------
  Clock Name      Source   Period    Lead    Trail    Generated   Propagated
  ------------------------------------------------------------------------
  CLK1            CLK1      20.000   0.000    10.000     n           y
  CLK2            CLK2      25.000   0.000    12.500     n           y
  CLK3            CLK3      10.000   0.000    6.000      n           y
  GCLK3           u8/Q      20.000   0.000    10.000     y           y
  ------------------------------------------------------------------------
  Generated-Clock Descriptions
  ------------------------------------------------------------------------------
  Name  Generated     Master     Master-clock  Invert  Freq.       Duty-Cycle  Edges  Edge-Shift
        Source(pin)   Source(pin)                      Multiplier
  ------------------------------------------------------------------------------
  GCLK3  u8/Q         CLK3       CLK3          n       1/2          -           -     -
  ------------------------------------------------------------------------------
  ```

- The following commands reset the `CLK1` clock to `ideal` mode and then reports them:

  ```
  tempus > reset_propagated_clock [get_clocks CLK1]
  tempus > report_clocks
  ```

  The following report is displayed:

  ```
  ------------------------------------------------------------------------
  ```

```
                              Clock Descriptions
-------------------------------------------------------------------------
                                                        Attributes
-------------------------------------------------------------------------
Clock Name    Source   Period   Lead    Trail    Generated      Propagated
-------------------------------------------------------------------------
CLK1          CLK1     20.000   0.000   10.000   n              n
CLK2          CLK2     25.000   0.000   12.500   n              y
CLK3          CLK3     10.000   0.000   6.000    n              y
GCLK3         u8/Q     20.000   0.000   10.000   y              y
-------------------------------------------------------------------------
Generated-Clock Descriptions
-------------------------------------------------------------------------
Name    Generated    Master       Master-clock   Invert  Freq.       Duty-Cycle   Edges   Edge-
Shift
        Source(pin)  Source(pin)                         Multiplier
-------------------------------------------------------------------------
GCLK3   u8/Q         CLK3         CLK3            n      1/2         -            -       -
-------------------------------------------------------------------------
```

Related Commands

- create_clock
- get_propagated_clock
- report_clocks
- set_clock_latency
- set_propagated_clock
- update_io_latency
- set_interactive_constraint_modes

reset_property

```
reset_property
[-help]
[-obj_type {object_type}]
{obj_list}
property_name
```

Resets user-defined properties on objects set using the `set_property` command.

Parameters

`-help`	Prints out the command usage.
obj_list	Specifies a list of objects for which the property is to be reset.
`-obj_type {`*object_type*`}`	Specifies the type of object on which the property is to be reset.
property_name	Specifies the name of the property for which the user-defined property is to be reset.

Examples

- The following command resets user-defined property `slk` from pins `C1/A`:

  ```
  tempus> reset_property [get_pins C1/A] slk
  ```

Related Commands

- `get_property`
- `set_property`
- `define_property`

reset_pulse_clock_max_transition

```
reset_pulse_clock_max_transition
[-help]
obj_list
[-transitive_fanout]
[-rise]
[-fall]
```

Resets the maximum pulse clock transition constraint specified (using the `set_pulse_clock_max_transition` command) on the pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

`-help`	Prints out the command usage.
`-fall`	Resets transition limit value for falling transition.
`obj_list`	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design - for which the transition value is to be reset.
`-rise`	Resets transition limit value for rising transition.
`-transitive_fanout`	Specifies the transition limit value for transitive fanout.

Related Commands

- `set_pulse_clock_max_transition`

reset_pulse_clock_max_width

```
reset_pulse_clock_max_width
[-help]
obj_list
[-transitive_fanout]
```

Resets the maximum pulse clock width constraint specified (using the `set_pulse_clock_max_width` command) on the pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

`-help`	Prints out the command usage.
`obj_list`	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design - for which the maximum pulse clock width is to be reset.
`[-transitive_fanout]`	Specifies the pulse width value for transitive fanout.

Related Commands

- `set_pulse_clock_max_width`

reset_pulse_clock_min_transition

```
reset_pulse_clock_min_transition
[-help]
obj_list
[-transitive_fanout]
[-rise]
[-fall]
```

Resets the minimum pulse clock transition constraint specified (using the `set_pulse_clock_min_transition` command) on the pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

`-help`	Prints out the command usage.
`-fall`	Resets transition limit value for falling transition.
`obj_list`	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design - for which the minimum pulse clock transition value is to be reset.
`-rise`	Resets transition limit value for rising transition.
`-transitive_fanout`	Specifies the transition limit value for transitive fanout.

Related Commands

- `set_pulse_clock_min_transition`

reset_pulse_clock_min_width

```
reset_pulse_clock_min_width
[-help]
obj_list
[-transitive_fanout]
```

Resets the minimum pulse clock width constraint specified (using the `set_pulse_clock_min_width` command) on the pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

`-help`	Prints out the command usage.
`obj_list`	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design - for which the minimum pulse clock width is to be reset.
`[-transitive_fanout]`	Specifies the pulse width value for transitive fanout.

Related Commands

- `set_pulse_clock_min_width`

reset_resistance

```
reset_resistance
[-help]
net_list
```

Resets the existing `set_resistance` assertions specified on nets.

Parameters

`-help`	Prints out the command usage.
`net_list`	Specifies a list of nets for which the wire resistance is reset.

Related Commands

- `set_resistance`

reset_sense

```
reset_sense
[-help]
[-all | [[-clocks clock_list] pin_or_port_list]]
```

Resets clock sense constraint(s) specified using the `set_sense` command.

Parameters

`-help`	Prints out the command usage.
`-all`	Resets all the specified `clock_sense` constraints.
`-clocks` *`clock_list`*	Specifies the list of clocks for which the clock sense constraints are to be removed. The clocks must have previously been defined using the `set_sense -clocks` parameter.
`pin_or_port_list`	Specifies the list or collection of cell output pins or ports for which the clock sense constraints will be removed.

Examples

- The following command resets all the previously defined `clock_sense` constraints from a design:

  ```
  tempus > reset_sense -all
  ```

Related Commands

- `set_sense`

set_active_clocks

```
set_active_clocks
[-help]
active_clock_list | all_clocks
```

Defines a list of active clocks in the design.

When the `set_active_clocks` command is specified, only the specified clocks are made active while others remain inactive. Any clock created after using the `set_active_clocks` command is active by default. The following commands are impacted:

- `report_clocks` command output displays a new column showing active or inactive clocks in the design.
- `get_clocks` command will report all clocks including inactive clocks.
- `write_timing_windows` command will not display windows for inactive clocks in the output report.
- `get_property` command allows you to query on active clocks using the `is_clock_active` property.

Parameters

`-help`	Prints out the command usage.
*`active_clock_list	all_clocks`*	Specifies a list of active clocks.

Examples

- The following command makes the specified clocks active while others remain inactive (only `Iclk1` is active, while `Iclk2` is inactive):

  ```
  tempus > set_active_clocks Iclk1
  ```

Related Commands

- `report_clocks`
- `get_clocks`
- `get_property`
- `write_timing_windows`

set_annotated_check

```
set_annotated_check
[-help]
value
[-clock clock_check]
[-cond sdf_condtion]
[-fall]
-from from_pins
[-incremental]
[-max]
[-min]
[-rise]
-to to_pins
{-setup | -hold | -recovery | -removal | -nochange_high | -nochange_low}
```

Annotates the setup, hold, recovery, or removal timing check value between two or more pins of a cell in the current design.

Parameters

`-help`	Prints out the command usage.
value	Specifies the timing check value.
`-clock` *clock_check*	Specifies clock rising or falling. Default: Sets checks for both clock rise and fall. Use the *clock_check* argument to specify rise or fall.
`-cond` *sdf_condition*	Specifies the condition only if the library has a condition attached to the specified timing check arc. The *sdf_expression* must match the syntax of the condition specified in the library.
`-fall`	Specifies the data fall transition for the timing check. Default: Sets both `-rise` and `-fall`.
`-from` *from_pins*	Specifies start points of the timing arcs for which checks are annotated. Use the *from_pins* argument to specify a list of leaf cell clock pins.
`-incremental`	Sets the incremental checks.
`-max`	Specifies the maximum timing check for both data rise and data fall transitions. Use this option only if the design uses minimum and maximum (`min-max`) operating conditions.

`-min`	Specifies the minimum timing check for both data rise and data fall transitions. Use this option only if the design uses minimum and maximum (`min-max`) operating conditions.
`-rise`	Specifies the data rise transition for the timing check. Default: Sets both rise and fall.
`-setup | -hold | -recovery | -removal | -nochange_high | -nochange_low`	Specifies the timing check type. Make sure there is a corresponding timing arc between the *`from_pins`* and the *`to_pins`* arguments.
`-to` *`to_pins`*	Specifies endpoints of the timing arcs for which checks are annotated. Use the *`to_pins`* argument to specify a list of leaf cell clock pins.

Examples

- The following commands annotates a `nochange check` between data low and clock low. The nochange margin before the clock is `1.3` and the nochange margin after the clock is `2.4`:

```
set_annotated_check -nochange_low 1.3 -clock fall -fall -from {I1/CP} -to {I1/EN}
set_annotated_check -nochange_low 2.4 -clock fall -rise -from {I1/CP} -to {I1/EN}
```

- The following command annotates a 2.0 setup time between the CP clock pin and the A data pin of the <*instance_name*> instance:

```
tempus > set_annotated_check -setup 2.0 -from <instance_name>/CP -to <instance_name>/A
```

Related Commands

- `reset_annotated_check`
- `set_annotated_delay`

set_annotated_delay

```
set_annotated_delay
[-help]
delay_value
[-cond expression]
[-delta_only]
[-fall]
[-increment]
[-max]
[-min]
[-rise]
{-net | -cell }
{[-from from_pins] [-to to_pins]}
```

Annotates delay to timing arcs. If you specify the `set_annotated_delay` command without any parameters, the delay value specified is assumed to include both the base delay and any delta (cross-talk) delay.

Parameters

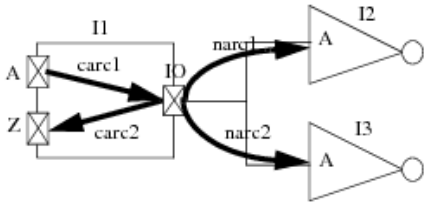
`-help`	Prints out the command usage.
`-cond` *expression*	Specifies the sdf condition of the selected timing arcs. *Default*: The delay applies to all the timing arcs selected using the `-from` and the `-to` options. The expression is ignored for net arcs. **Note:** You cannot specify the `-cond` parameter with `-net`. If specified, an error is displayed.
value	Specifies the delay value to be annotated to the selected timing arcs.
`-delta_only`	Populates the incremental delay slot with the delay value, so that you can differentiate between the base net delay and a delay offset that is due to crosstalk. The software reports the incremental (or delta) delay value separately in the timing report using `report_timing -format incr_delay`, and maintains the separate value for CPPR calculations. If the base delay was previously calculated or annotated from the SDF, the `-delta_only` parameter adds the specified delay to the existing base delay. **Note:** This parameter only can be used with the `-net` parameter; you cannot specify it with the `-cell` parameter.
`-from` *from_pins*	Specifies timing arcs to which the delay applies. The selected timing arcs are those that have their source pins specified using this option and the sink pins specified using the `-to` option. If this option is not specified, only timing arcs whose sink pins are specified using the `-to` option are selected. **Note:** You cannot specify hierarchical pins. If specified, an error message is displayed.
`-increment`	Increments the specified delay value on top of the existing base delay value.

`-min	-max`	Specifies the annotated delay for a particular operating corner. You can apply the `-min` parameter to the minimum operating corner and the `-max` parameter to the maximum operating corner.
`-net	-cell`	Specifies whether the delay should be annotated to cell arcs or net arcs.
`-rise/-fall`	Specifies the transition to which the delay applies. For cell arc, this is the output transition. *Default*: The delay applies to both transitions.
`-to` *to_pins*	Specifies timing arcs to which the delay applies. The selected timing arcs are those that have their sink pins specified using this option and the source pins specified using the `-from` option. If this option is not specified, only timing arcs whose source pins are specified using the `-from` option are selected. **Note:** You cannot specify hierarchical pins. If specified, an error message is displayed.

Examples

- The following example shows the usage of `-cell` and `-net` options:

  Example Figure to Show How to Use the -cell and -net Options

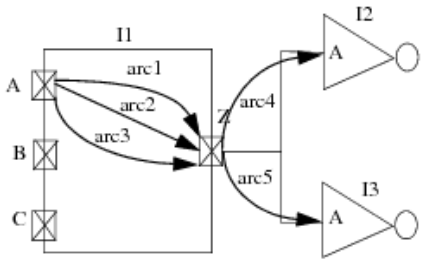
  

  - Use the `-cell` option to select the cell arc `carc2` only:
    set_annotated_delay 3.0 -cell -from {I1/IO}
  - Use the `-net` arc option to select net arcs `narc1` and `narc2` only:
    set_annotated_delay 3.0 -net -from {I1/IO}

- This TLF description is used for the following examples:

```
CELL (NAND3 . . . . // arc 1 Path( A => Z 10 01 DELAY(ioDelayRiseModel0) SLEW(SlopeRiseModel0) )
Path( A => Z 01 10 DELAY(ioDelayFallModel0) SLEW(SlopeFallModel0) ) // arc2 Path( A => Z 10 01
COND(B & C) SDF_COND(B & C) DELAY(ioDelayRiseModel1) SLEW(SlopeRiseModel1) ) Path( A => Z 01 10
COND(B & C) SDF_COND(B & C) DELAY(ioDelayFallModel1) SLEW(SlopeFallModel1) ) arc3 Path( A => Z 10
01 COND(~B | ~C) SDF_COND(!B | !C) DELAY(ioDelayRiseModel2) SLEW(SlopeRiseModel2) ) Path( A => Z 01
10 COND(~B | ~C) SDF_COND(!B | !C) DELAY(ioDelayFallModel2) SLEW(SlopeFallModel2) ) . . . }
```

  Example Figure to Show How to Use the -from and -to Options



- The following command annotates a delay of `3.0` to arcs `arc1`, `arc2`, and `arc3` for the rise and fall transition at the pin I1/Z, for the two corners `{min typ max}`: `set_annotated_delay 3.0 -from A -to Z -cell`
- The following command annotates a delay of 2.5 to net arc `arc4` for both the rising and the falling transition and for the two corners `{min typ max}`: `set_annotated_delay 2.5 -from I1/Z -to I2/A -net`
- The following command annotates a delay of 1.5 to `arc2` for the rising transition and the min corner:
  `set_annotated_delay 1.5 -from I1/A -to I1/Z -rise -min -cond "B & C" -cell`
- The following commands annotate a delay of 3.0 to `arc1` and `arc2`, and a delay of 2.4 to the `arc3`:
  `set_annotated_delay 3.0 -from I1/A -to I1/Z -cell set_annotated_delay 2.4 -from I1/A -to I1/Z -cond {!B | !C} -cell`
- The following commands first annotate a delay of 3.0 to `arc1`, `arc2` and `arc3`, then overwrite the value set by the first `set_annotated_delay` command: `set_annotated_delay 2.4 -from I1/A -to I1/Z -cond {!B | !C} -cell set_annotated_delay 3.0 -from I1/A -to I1/Z -cell`

Related Commands

- `reset_annotated_delay`
- `set_annotated_check`

set_annotated_transition

```
set_annotated_transition
[-help]
[-rise | -fall]
[-min | -max]
value
pin_list
```

Sets the transition time to be annotated on specified ports or pins in the current design. The transition time that you specify using this command overrides the transition time calculated internally within the software. You can use this command to set the transition time of internal cell instance pins.

Parameters

`-help`	Prints out the command usage.
`-min | -max`	Specifies whether the transition value is set for minimum or maximum transition time. *Default:* Transition value is set for both transition times.
pin_list	Specifies a list of pins or ports to be annotated with the transition time that you specify using the *value* parameter.
`-rise | -fall`	Specifies whether the transition value is set for rising or falling transition time. *Default*: Transition value is set for both transitions.
value	Specifies the transition time to be annotated to the selected pins or ports.

Examples

- The following command annotates a transition time value of 0.6 on port A:
  ```
  tempus > set_annotated_transition 0.6 [get_ports A]
  ```
- The following command annotates a transition time value of 0.7 on pin I2/A:
  ```
  tempus > set_annotated_transition 0.7 [get_pins I2/A]
  ```
- The following command annotates a transition value of 0.7 on pin I2/A for rising transition time:
  ```
  tempus > set_annotated_transition -rise 0.7 [get_pins I2/A]
  ```
- The following command annotates a transition value of 0.7 on pin I2/A for rising transition time and minimum pvt corner:
  ```
  tempus > set_annotated_transition -rise -min 0.7 [get_pins I2/A]
  ```

Related Commands

- set_input_transition
- get_pins
- get_ports
- report_ports
- reset_annotated_transition

set_aocv_interface_path_offset

```
set_aocv_interface_path_offset
[-help]
derate_offset_value
[lib_cell_list]
[-view view_name]
[-delay_corner dc_corner_name]
[-cell]
[-net]
-power_domain power_domain_name_list
```

You can perform path-based inter power domain (IPD) timing analysis by specifying derate offset value for an entire power domain(s) and specific library cell(s) on paths crossing multiple power domains. This feature is supported for path-based analysis (PBA) mode only.

The `set_aocv_interface_path_offset` command allows you to specify the derate offset value for path segments crossing multiple power domains. The software adds the derate offset value to the AOCV derate value if the combined (AOCV + IPD offset) derate number causes pessimistic PBA timing slack value.

Parameters

`-help`	Prints out the command usage.
`-cell`	It is a boolean option. When the -cell option is specified, intent is to apply “derate offset” only for that cell objects. In absence of these options, derate offset value will be applied to both cell and net type of objects
`-delay_corner` *dc_corner_name*	Specifies a list of delay corners.
derate_offset_value	Specifies value of derate offset.
lib_cell_list	Specifies a list of library cells for a specific interface.
`-net`	It is a boolean option. When the -net option is specified, intent is to apply “derate offset” only for that net objects. In absence of these options, derate offset value will be applied to both cell and net type of objects
`-power_domain` *power_domain_name_list*	Specifies a list of power domains.
`-view` *view_name*	Specifies a list of views.

Examples

- The following example specifies the offset derate value of `-0.6` for the power domain `{PD3 PD4}`:

```
set_aocv_interface_path_offset -0.6 -power_domain {PD3 PD4} -delay_corner V1_PM1_dc
```

set_aocv_stage_weight

```
set_aocv_stage_weight
[-help]
stage_weight
port_list
[-early]
[-late]
[-input]
[-output]
```

While performing advanced on-chip variation (AOCV) analysis of a block in standalone mode, the command allows you to specify the stage weight seen at the port(s) when it is modeled as part of the top level design. The applied stage weights affect the internal weight count of the block for the IO paths. The stage weights can be applied only on ports.

You can view stage weights in the `report_timing` output under the `Stage Weighting` column.

The `set_aocv_stage_weight` command can be used in multi-mode multi-corner mode. The command can be used as part of SDC file.

Parameters

`-help`	Prints out the command usage.
`-early`	Specifies weight for early paths.
`-input`	Specifies input weight for the ports.
`-late`	Specifies weight for late paths.
`-output`	Specifies output weight for port.
`port_list`	Specifies a list or collection of input, output, and bidi ports.
`stage_weight`	Specifies value of stage weight.

Examples

- The following example specifies the stage weight value as `25.5` for port `out0`:

  ```
  tempus > set_aocv_stage_weight 25.5 out0
  ```

Related Commands

- `reset_aocv_stage_weight`
- `report_timing`

set_aocv_thresholds

```
set_aocv_thresholds
[-help]
[-min_stage_count_setup <int>]
[-min_stage_count_hold <int>]
[-reset]
[-slack_pruning_threshold <float>]
-view <list_of_views>
```

Limits the minimum AOCV stage depth in graph-based analysis (GBA) mode.

Parameters

`-help`	Prints out the command usage
`-min_stage_count_setup <int>`	Applies the AOCV stage count floor, during Setup analysis, so that all the instances are guaranteed to have a stage depth no less than the specified value. A value of '0' will remove the assertion.
`-min_stage_count_hold <int>`	Applies the AOCV stage count floor, during Hold analysis, so that all the instances are guaranteed to have a stage depth no less than the specified value. A value of '0' will remove the assertion.
`-reset`	Removes/sets to default all of the AOCV threshold assertions for specified views.
`-slack_pruning_threshold <float>`	Controls the slack threshold (in two-pass/slack-based AOCV analysis mode), at which paths are removed from consideration for the next phase of AOCV stage counting.
`-view <list_of_views>`	Applies the specified thresholds to the list of analysis views.

set_case_analysis

```
set_case_analysis
[-help]
{0 | 1 | zero | one | rising | falling | rise | fall | non_switching}
pin_or_port_list
```

Sets constant or transitional values to a list of pins or ports for use by the timing engine. The specified constants or transitional values are valid only during timing analysis and do not alter the netlist. If there is a conflict between the specified `set_case_analysis` value and the logical value from the netlist, the value from the `set_case_analysis` is used in timing analysis.

Parameters

`-help`	Prints out the command usage.
`0 | 1 | zero | one`	Specifies a logic value of `0` or `1`, which is propagated through combinational logic to disable or enable parts of logic.
`non_switching`	Marks the nets that are in a constant, non-transitioning state. This marking can be used when nets are considered static without the additional impact of logic state injection into the analysis, e.g., when modes are merged into a single constraint mode. The `report_cell_instance_timing` and `report_case_analysis` command reports show non-switching as constant values.
`rising | falling | rise | fall`	Specifies a valid transition value to be considered by the timing engine. For example, if a rising value is specified at a certain pin or port, only the rising signal transition from the pin or port is considered for timing analysis and the opposite transition type (falling) is ignored.
`pin_or_port_list`	Specifies a list of ports or pins on which to apply the constant or transitional value. Specify multiple pins by enclosing them with curly braces `{}`and separating the pin names with white space. The *`pin_or_port_list`* argument can be a collection.

Examples

- The following command sets the value of pin i4/A to `1` for use by the timing engine:
  ```
  tempus > set_case_analysis 1 i4/A
  ```
- The following example shows that the pins P1/P2/B and P1/P3/CI are considered only for a rising transition. The falling transition on these pins is ignored:
  ```
  tempus > set_case_analysis rising {P1/P2/B P1/P3/CI}
  ```

Related Commands

- check_timing -type constant_collision
- report_case_analysis
- report_timing
- set_disable_timing
- set_false_path

set_clock_exclusivity

```
set_clock_exclusivity
[-help]
pin_name
[-exclude_opposite_polarity]
[-exclude_related_input_pins]
[-exclude_same_polarity]
[-group clock_list]
[-inputs string]
```

Controls filtering of mutually-exclusive clocks at strategic points in the design, e.g., the output of clock multiplexers.

Parameters

`-help`	Prints out the command usage.
`-exclude_opposite_polarity`	Isolates positive and negative phases of the clocks in the same group. By default, the positive and negative phases of the clocks in the same group that arrive at a strategic point will not be isolated from each other.
`-exclude_related_input_pins`	Allows comparison of clocks going through the same input pins of the specified gate.
`-exclude_same_polarity`	Allows comparison of clocks going through the same launch or capture paths.
`-group` *clock_list*	Defines a set of clocks that are exclusive to the clocks defined in all other groups.
`-inputs` *string*	Specifies a list or collection of input pins for which clock exclusivity is to be recognized.
pin_name	Specifies the name of the pin.

Examples

- The following command will cause downstream analysis from the mux to isolate interactions between the groups:

  ```
  tempus > set_clock_exclusivity -group {CLK1 CLK2} -group {CLK3 CLK4} U1/Y
  ```

  This assertion will cause downstream analysis from pin `U1/Y` to isolate interactions between the groups. The groups `{CLK1 CLK2}` and `{CLK3 CLK4}` are allowed, but all other combinations are considered logically exclusive.

Related Commands

- create_clock

set_clock_gating_check

```
set_clock_gating_check
[-help]
[object_list]
[-fall]
[-hold hold_value]
[-rise]
[-setup setup_value]
[-high | -low]
```

Specifies or overrides the default setup and hold values for clock gating checks.

Parameters

`-help`	Prints out the command usage.
`-high | -low`	Specifies that the non-controlling value of the clock is high or low. Timing analysis performs the check on the non-controlling value of the clock. By default, the software determines the non-controlling value of the clock using information from the gate's logic. For example, for OR gates the non-controlling value is low. For complex gates, such as XOR and MUX, the clock is never controlling and it does not perform checks unless you specify either the `-high` option or the `-low` option. The specified value takes precedence over the default values. This option can only be used with pins or instances. **Note**: The clock pin value is controlling if it causes the gate output to be a constant (`0` or `1`).
`-hold hold_value`	Specifies the hold value. The default is `0.0`.
`obj_list`	Specifies a list of clocks, instances, or pins. If a clock is specified, the check applies to all the clock gating gates driven by this clock. If an instance is specified, the check applies to all input pins of the instance. If the *`obj_list`* argument is not specified, the clock gating check is applied to the current design.
`-rise | -fall`	Applies the specified setup (or hold) value to the rising (or falling) signals on the specified pins. If the clock is specified or the *`obj_list`* variable is not specified, the value applies to rising or falling, setup or hold. If neither option is specified, the default is both rising and falling.
`-setup setup_value`	Specifies the setup value. The default is `0.0`.

Note: The clock gating constraint is not propagated along the clock tree.

The setup check is performed with respect to the edge of the clock signal that changes the state of the clock pin from controlling to non-controlling. For example, for AND and NAND gates, the setup check is performed with respect to the rising edge of the clock input. For OR and NOR gates, the setup check is performed with respect to falling edge of the clock input.

The hold check is performed with respect to the edge of the clock signal that changes the state of the clock pin from non-controlling to controlling. For example, for AND and NAND gates, the hold check is performed with respect to the

falling edge of the clock input. For OR and NOR gates, the hold check is performed with respect to rising edge of the clock input.

If the gate logic in unknown, setup and hold checks are performed with respect to both the rising edge and the falling edge of the clock signal and the timing analyzer uses the worst case values.

Priority Rules for Determining Setup or Hold Values

Timing analysis uses the following priority rules to determine the setup (or hold) value to use for a particular clock gating check:

1. Use the clock gating check assertion on the data pin if it exists
2. Otherwise, use the clock gating check assertion on the clock pin if it exists
3. Otherwise, use the clock gating check assertion on the clock waveform for the clock pin if it exists
4. Otherwise, use the attribute clock gating check assertion if it exists
5. If no clock gating check assertion is present, use the default setup (or hold) value of `0.0`.

These rules are demonstrated in the examples below.

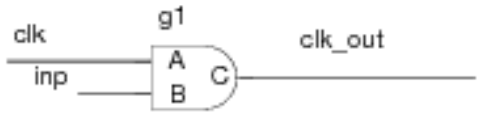
Note: This command has no effect on a clock gating check defined as a non-sequential setup or hold check in the library and reported as `ClockGatingSetup` or `ClockGatingHold`.

Examples

Here are some examples.

- The following example shows the priority rules for multiple clock gating check constraints for the gated clock shown in figure below:

  Gated Clock

  Tcl Script for set_clock_gating_check

```
tempus > create_clock -name CLK1 -period 12 -waveform {0 6} clk
...
# clock-gating timing check assertions:
# On ideal clock waveform
set_clock_gating_check -setup 0.44 [get_clocks {CLK1}]
# On clock pin
set_clock_gating_check -hold 0.11 [get_pins {g1/A}]
# On data pin
set_clock_gating_check -setup 0.66 -rise [get_pins {g1/B}]
```

```
set_clock_gating_check -hold 0.22 -fall [get_pins {g1/B}]
```

Given the previous constraints, the following timing margins are used for slack calculation:

- `SETUP g1/B rise: 0.66` -- from assertion on data pin `g1/B` (rule 1)
- `SETUP g1/B fall: 0.44` -- from assertion on ideal clock `CLK1` (rule 3)
- `HOLD g1/B rise: 0.11` -- from assertion on clock pin `g1/A` (rule 2)
- `HOLD g1/B fall: 0.22` -- from assertion on data pin `g1/B` (rule 1)

Related Commands

- `create_clock`
- `reset_clock_gating_check`

set_clock_groups

```
set_clock_groups
[-help]
[-name name]
[-comment string]
[-logically_exclusive] | [-physically_exclusive] | [[-asynchronous]
[-allow_paths]]
[-group clock_list]
```

Defines clock groups with specified clock definitions. The software assigns all clocks that are defined after you use the `set_clock_groups` command to a default clock group. The software does not apply clock groupings on a master clock to the generated child clocks.

To report the defined clock groups, you can use the following command:

```
report_clocks -groups <name>
```

If there is an overlap in the clock group specification, the software uses the following precedence to resolve the behavior:

1. Physically exclusive
2. Asynchronous
3. Logically exclusive

The software drops any explicit false path assertions between clocks that you have set as either physically or logically exclusive or asynchronous using the `set_clock_groups` command.

An alternative to setting the clocks as physically or logically exclusive using the `set_clock_groups` command is to use the multi-mode mechanism. Generally in an implementation flow, sequential setting of active clocks cannot be used. For concurrent analysis of different modes, you can use an multi-mode based approach to drive IPO instead of using the `set_clock_groups` command.

Note: If the `set_clock_groups` constraint is added incrementally in SI mode, an explicit `update_timing -full` command is required (before `report_timing` or `report_constraint`) for SI full accuracy.

Parameters

`-help`	Prints out the command usage.
`-allow_paths`	Allows paths between asynchronous clock groups to be traced in timing analysis. These paths are blocked by default. When you use this option, the software still considers the paths to be asynchronous for signal integrity considerations. Signal integrity analysis does not support the `-allow_paths` parameter. Use the `-allow_paths` parameter with the `-asynchronous` parameter.
`-asynchronous`	Defines clock group with asynchronous clocks. Asynchronous clocks have no specified phase relationship. The software considers asynchronous nets to have infinite timing window overlap for signal integrity considerations and considers these as false paths for timing analysis. Use the `-allow_paths` parameter to trace these paths in timing analysis.
`-comment` *`string`*	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
`-group` *`clock_list`*	Defines a set of clocks that are exclusive or asynchronous to the clocks defined in all other groups. If you define only one group, then all clocks not in the group implicitly belong to a default clock group. The software automatically adds any new clocks that you define after the `set_clock_groups` command to this default clock group.
`-logically_exclusive`	Defines clock group with clocks that do not have any functional paths between them, but coexist in the design at the same time. The software considers aggressor interactions between these clocks for signal integrity considerations, but considers these as false paths for timing analysis. **Note:** The `-logically_exclusive` parameter does not have any impact on SI analysis. For example, clocks selected via a clock mux select.
`-name` *`name`*	Specifies the names of the clock groups to be added.
`-physically_exclusive`	Defines clock group with clocks that cannot exist in the design at the same time. The software does not consider aggressor interactions between clock groups that are physically exclusive for signal integrity considerations. From the perspective of SI analysis, all attackers that are physically exclusive with the victim net are ignored for both glitch and noise-on-delay analysis. Attacker nets that belong to physically exclusive groups cannot attack together. In this case, all sets of attackers belonging to all the physically exclusive groups are evaluated and the set of nets that has the highest impact on SI analysis is chosen. For timing analysis, the software considers these clock groups as false paths. Example: multiple clocks defined on the same source pin

Examples

Here are some examples.

- The following commands assert two different clocks on the same clock root port, and uses `set_clock_groups` to isolate them from each other:

```
tempus > create_clock -name SYSCLK -period 10 [get_ports sclk]
tempus > create_clock -name SCANCLK -period 100 -add [get_ports sclk]
tempus > set_clock_groups -name PHY_EX_SCLK -physically_exclusive -group {SYSCLK} -group {SCANCLK}
```

- The following command declares a single bus clock to be asynchronous to all of the other on board clocks in the design (which remain in the default clock group):

```
tempus > set_clock_groups -name ASYNCH_BUS -asynchronous -group {BUSCLK}
```

  Assume a design with four clocks CLK1, CLK2, CLK3, and CLK4.

- The following command defines CLK1 physically exclusive to CLK2, CLK3 as well as CLK4.

```
tempus > set_clock_groups -name group1 -group CLK1 -physically_exclusive
```

- The following command defines CLK1 and CLK2 to be physically exclusive to CLK3 and CLK4.

```
tempus > set_clock_groups -name group2 -group {CLK1 CLK2} -physically_exclusive
```

- The following command defines CLK1 and CLK2 as physically exclusive to CLK3

```
tempus > set_clock_groups -name group3 -group {CLK1 CLK2} -group {CLK3} -physically_exclusive
```

- The following command defines the following:

```
tempus > set_clock_groups -name group4 -group {CLK1} -group {CLK2} -group {CLK3} -
physically_exclusive
```

  - CLK1 physically exclusive to CLK2 and CLK3.
  - CLK2 physically exclusive to CLK1 and CLK3.
  - CLK3 physically exclusive to CLK1 and CLK2.

- The following example shows that the clock group {clkA clkB} is asynchronous only to clock group {clkC clkD}:

```
tempus > set_clock_groups -asynchronous -group {clkA clkB} -group {clkC clkD}
```

  If an extra clock clkE is defined in the design, then timing check will be allowed in between the first group to clkE and second
  group to clkE.

- In the following example since only one clock group is specified with the command `set_clock_groups` command, so the specified clock group is considered to be asynchronous to all the other clocks in the design:

```
tempus > set_clock_groups -asynchronous -group {clkA clkB}
tempus > set_clock_groups -asynchronous -group {clkC clkD}
```

Related Commands

- `create_clock`
- `report_clocks`
- `reset_clock_groups`

set_clock_latency

```
set_clock_latency
[-help]
<value>
<obj_list>
[-clock <clock_list>]
[-clock_gate]
[-fall]
[-max]
[-min]
[-rise]
[-source [-early ] [-late ] [-dynamic <jitter_val>]]
```

Specifies ideal internal clock value and external clock arrival delay.

Clock value has two components:

- Source value:
  - Specified using the `-source` parameter and is the delay from the external clock generator to the clock port of the design.
  - Applied when the clock is either ideal or propagated.
  - `set_clock_latency` set on a clock port has higher precedence than `set_clock_latency` set on a clock waveform.
  - Specified on clock waveforms with both the `set_input_delay` command and the `set_output_delay` command.
- Network value (clock tree delay):
  - Default value type and is the delay from the clock port to the register.
  - Applied only when the clock is ideal. When the clock is propagated, this delay is replaced by the actual clock tree delays.
  - Specified on a clock waveform or a clock port is ignored if the clock is propagated.
  - `set_clock_latency` set on a timing pin has higher precedence than `set_clock_latency` set on a waveform.
  - If there are multiple `set_clock_latency` and `set_propagated_clock` assertions between the clock waveform, clock root and clock endpoint, the closest assertion to the clock endpoint takes precedence.
  - Specified on internal circuit pins or hierarchical ports and used in the fanout cone of the network value pin, when a clock is ideal.

Use the `set_clock_latency` command in two ways:

- To specify clock value for a clock waveform:
  - Use the following syntax for source value associated with a clock signal:
    `set_clock_latency [-source] [-early | -late] [-min ][-max] value obj_list`
  - Use the following syntax for network value associated with a clock signal:

```
set_clock_latency [-min] [-max] value obj_list
```

You cannot specify the `-late` and `-early` parameters for network value.

Specifying value on a port or pin overrides any previous value specified on the port pin or waveform. The *obj_list* argument is the list of the clock waveform names as specified by the create_clock command.

- To specify clock value on clock pins:
  - Use the following syntax for source delay on a clock port:

    ```
    set_clock_latency -source [-early | -late] [-rise] [-fall]
    insertion_delay obj_list
    ```

  - Use the following syntax for network delay on a clock pin:

    ```
    set_clock_latency [-min] [- max] [-rise] [-fall]
    insertion_delay -pin obj_list
    ```

When a clock is ideal, the last specification along a path is used. This is useful when you want to intentionally skew the clock tree and model the physical hierarchical clock tree.

Parameters

`-help`	Prints out the command usage.
`-clock` *clock_list*	Associates clock value constraints, placed at the pin level, with a specific clock. This makes it possible to specify a different value per clock. **Note:** You cannot use the `-clock` parameter with clock waveform objects.
`-clock_gate`	Allows setting the local clock value on pin or port objects. This setting overwrites the existing value of clock gating cells and is used when performing timing checks against the specified pins or ports.
`-dynamic` *jitter_val*	Specifies the portion of source clock latency value that can have cycle-to-cycle variation. When calculating CPPR adjustment this portion is removed from CPPR adjustment for different clock edge checks (e.g., setup), but it remains as part of cppr adjustment for same clock edge checks (e.g., hold).
`-early | -late`	Specifies clock arrival time with respect to the early or the late time of the clock signal. In setup analysis, launch path is the late path and capture path is the early path. In hold analysis, launch path is the early path and the capture path is the late path. If neither parameter is specified, the default is both early and late. Use these parameters only with the `-source` parameter. You cannot use these parameters with network value. Specify the clock value at the given operating corner. **Note:** The `-min` and `-max` parameters refer to the operating corner and the `-early` and `-late` parameters refer to the clock arrival time at the source. Use the `-early` and `-late` parameters with the `-source` parameter to specify the four different clock latencies.
value	Specifies the value of the latency, in library units, as a floating point.

obj_list	Specifies a list of clock waveform names or a list of pins to associate with the clock value. The pin can be a port or an instance pin. Setting source value on a pin is ignored.
-max	Specifies the clock value for a particular operating corner. Apply the -max parameter to the maximum operating corner.
-min	Specifies the clock value for a particular operating corner. Apply the -min parameter to the minimum operating corner.
-rise | -fall	Specifies the -rise or -fall parameter to the waveform at register clock pins for ideal value and to the waveform at the source for source value. Ideal network value does not change polarity when crossing negative unate arcs.
-source	Specifies the source latency. If you do not use the -source parameter, the network latency will be applied. You can use the timing_generated_clocks_inherit_ideal_latency attribute to include the closest applied network latency for ideal inheritance. However, when the -source parameter is used, the software will completely override any inheritance.

Examples

- The following set of commands enable you to create a clock using the create_clock command and apply latency to the created clock using the set_clock_latency command:

```
tempus > create_clock CLK2 -period 20 [get_ports {clkA clkB clkD}]
tempus > set_clock_latency -source 2 {clkB clkD} # Define port settings
tempus > set_clock_latency -rise 5 {clkB clkD} # Define port settings
tempus > set_clock_latency -early -late -source CLK1 # Define waveform settings
tempus > set_clock_latency 3 -early -late -source CLK1 # Define waveform settings
```

Related Commands

- reset_clock_latency
- create_clock
- set_clock_transition
- set_clock_uncertainty
- set_input_delay
- set_output_delay
- set_propagated_clock

set_clock_sense

```
set_clock_sense
[-help]
[-positive | -negative | -stop_propagation | -logical_stop_propagation | -stoptype_list]
[-clocks clock_list]
pin_or_port_list
```

Defines the phases of a clock or clock source that can propagate beyond a port or instance pin. The clocks can be selectively or entirely blocked through the pin to all downstream logic. Additionally, data relating to clocks can be blocked through the pin.

Note: The `set_clock_sense` command is now obsolete. This command will continue to be supported in the current release, but will be removed in the next major release of the software.

Parameters

`-help`	Prints out the command usage.
`-clocks clock_list`	Specifies a list of clocks passing through a specified point to which the constraint applies. When this parameter is not specified, the filtering of clock phase will be applicable to all the clocks passing through that specified point. This parameter must be used with a specified pin name and one of the parameters : `-positive`, `-negative`, `-stop_propagation`, `-logical_stop_propagation`, or `-stop`.
`-logical_stop_propagation`	Stops the propagation of specified clocks passing through the specified pins.
`-negative`	Specifies that positive phase filtered clocks pass through pins in the *pin_list*.
pin_or_port_list	Specifies the list or collection of cell output pins. The unateness specified by the set_clock_sense command is applied to clocks propagating through the specified point.
`-positive`	Specifies that negative phase filtered clocks pass through pins in the *pin_list*.
`-stop_propagation`	Stops the propagation of specified clocks and clock source data paths passing through the specified pins.

`-stop` *`type_list`*	Applies clock sense to object types specified in the *`type_list`*. The type list options are - `all`, `clock`, `data`, `clock_source_data`, `default_data`. These are explained below: • `all`: Stops all phases from propagating. This is equivalent to specifying both the `clock` and `data` options. • `clock`: Stops clock phases launched by clock(s) specified in the *`clock_list`* from propagating, and is equivalent to existing `-logical_stop_propagation`. • `data`: Stops all data phases launched by clock(s) specified in *`clock_list`*, starting from the pins/ports specified in *`pin_or_port_list`*. The objects in the *`pin_or_port_list`* can be selected anywhere in the design - clock/data network. • `clock_source_data`: Stops only clock source data phases from propagating. The combination of `clock_source_data` and `clock` is equivalent to the existing `-stop_propagation` option. • `default_data`: Stops only default data phases (representing constrained paths) from propagation. This option cannot be specified along with the *`-clock clock_list`*. The `-stop` parameter extends the capability of the `set_clocks_sense` command to selectively block data timing phases. This parameter provides a more generalized use model for selecting among all types of paths that should be filtered. The `-stop` parameter is mutually exclusive with the `-positive`, `-negative`, `-stop_propagation`, and `-logical_stop_propagation` options.

Examples

- The following command stops the propagation of clock CLK1 through the pin CLKBUF1/Z:
  ```
  tempus > set_clock_sense -logical_stop_propagation -clocks CLK1 CLKBUF1/Z
  ```
- The following command allows the positive phase clock CLK1 to pass through CLKBUF1/Z but filters out the negative phase clock CLK1:
  ```
  tempus > set_clock_sense -positive -clocks CLK1 CLKBUF1/Z
  ```
- The following command stops clock phases starting from pin1:
  ```
  tempus > set_clock_sense pin1 -stop {clock}
  ```
  or
  ```
  tempus > set_clock_sense pin1 -logical_stop_propagation
  ```
- The following command stops data phases based only on CK1 and CK2 starting from pin1:
  ```
  tempus > set_clock_sense pin1 -stop {data} -clocks {CK1 CK2}
  ```
- The following command stops only the clock source data phases starting from pin1:
  ```
  tempus > set_clock_sense pin1 -stop {clock_source_data}
  ```
- The following command stops the clock and clock source data from pin1:
  ```
  tempus > set_clock_sense pin1 -stop {clock clock_source_data}
  ```
  or
  ```
  tempus > set_clock_sense pin1 -stop_propagation
  ```

Related Commands

- reset_clock_sense

set_clock_transition

```
set_clock_transition
[-help]
[-rise]
[-fall]
[-min]
[-max]
[-min |-max]
transition_time
clock_list
```

Specifies the transition time of sequential endpoints of the ideal clock network.

This command only affects ideal clock networks, so typically it is used before the clock network is implemented. If no clock transition is explicitly specified for an ideal clock, the system's default transition time (often close to 0.0) is used. For propagated clocks, the calculated transition times are used, so you can set propagated clocks only after the final clock tree is constructed.

This command only affects sequential clock endpoints, typically the clock pin of registers in the combinational fanout of the clock. Any data endpoints that are also in the combinational fanout of the clock will not be affected. The software will always use the propagated transition times. You can consider turning off clock-source data paths until the clock tree is built or, you can use the `set_ideal_network` command for additional control.

Parameters

`-help`	Prints out the command usage.
`clock_list`	Specifies a list of clocks only with ideal mode in the design, which affects the slew times on all register clock pins in the transitive fanout of the specified clocks. Specified `set_clock_transition` values on register clock pins in the fanout of a clock with propagated latency are ignored.
`-rise | -fall`	Specifies the rising or falling edge of the register clock pins on which the slew time is asserted.
`-min | -max`	Specifies the clock slew time for minimum or maximum process corner slew time.
`transition_time`	Specifies the slew time (transition time) of the clock pins.

Examples

- The following commands specify a `2.0` rise slew time and a `1.0` fall slew time for register clock pins clocked by `CLKA`:

```
tempus > set_clock_transition -rise 2.0 [get_clocks CLKA]
tempus > set_clock_transition -fall 1.0 [get_clocks CLKA]
```

Related Commands

- report_clocks
- set_clock_latency
- set_clock_uncertainty
- set_input_delay
- set_propagated_clock

set_clock_uncertainty

```
set_clock_uncertainty
[-help]
uncertainty_value
[-setup]
[-hold]
[-half_cycle_jitter | -full_cycle_jitter]
[-rise | -fall]
{  {-from | -rise_from | -fall_from} clksig_from_list
   {-to | -rise_to | -fall_to} clksig_to_list
 | clock_or_clock_pin
}
```

Specifies the clock uncertainty (skew) on the clock network.

If you set multiple constraints with the `set_clock_uncertainty` command, the constraints set with the `-from*` *`clksig_from_list`*/`-to*` *`clksig_from_list`* parameters take precedence over simple constraints set on clock objects.

Clock uncertainty set on pins takes highest precedence and overrides clock based uncertainty.

The `set_clock_uncertainty` command can also support clock uncertainty for period, pulse width, skew, and separation checks. To enable this feature, you need to set `timing_enable_uncertainty_for_clock_checks` attribute to `true`. By default, the attribute is set to `false`. For more details, see timing Category Attributes.

Parameters

`-help`	Prints out the command usage.
`uncertainty_value`	Specifies the uncertainty value.
`-from | -rise_from | -fall_from` *`clksig_from_list`*	Applies the uncertainty value to the specified list of launching clocks. **Note:** You must specify one of these parameters with one of the `-to`, `-rise_to`, or `-fall_to` parameters. You cannot use these parameters when *`clock_or_clock_pin`* is specified.
	• `-fall_from`: Applies the uncertainty value only to the falling edge of the launching clocks.
	• `-from`: Applies the uncertainty value to both the rising and falling edges of the launching clocks.
	• `-rise_from`: Applies the uncertainty value only to the rising edge of the launching clocks.

`-half_cycle_jitter` *value* | `-full_cycle_jitter` *value*	Specifies the clock source jitter for half-cycle and for full-cycle paths. These parameters can accept different values, that can be specified in the same constraint. **Note**: These parameters can be used on clocks only. You cannot use them when any of the `-from` and `-to` parameter pairs are specified.
clock_or_clock_pin	Specifies a list of clocks or pins for target uncertainty. The uncertainty value is applied to the registers clocked by the target waveform, or register pins in the fanout of the specified pin. You can specify a list of clock names, or clock root pin or port names relative to the current module. You also can specify a collection of clocks, or clock root pins or ports. **Note:** You cannot use this parameter when any of the `-from` and `-to` parameter pairs are specified.
`-rise` | `-fall`	Specifies the rising or falling edge at the capture clock pin on which the uncertainty is applied. *Default*: Applies uncertainty on both edges **Note:** You must use these parameters with the `-to` and `-from` parameters.
`-setup/-hold`	Specifies whether the clock uncertainty applies to setup or hold checks. *Default*: Applies to both setup and hold check
`-to` | `-rise_to` | `-fall_to` *clksig_to_list*	Applies the uncertainty value to the specified list of capture clocks. **Note:** You must specify one of these parameters with one of the `-from`, `-rise_from`, or `-fall_from` parameters. You cannot use these parameters if *clock_or_clock_pin* is specified.
	• `-fall_to`: Applies the uncertainty value only to the falling edge of the capture clocks.
	• `-rise_to`: Applies the uncertainty value only to the rising edge of the capture clocks.
	• `-to`: Applies the uncertainty value to both the rising and falling edges of the capture clocks.

Examples

- The following command gives all paths starting from registers clocked by `clk1` to registers clocked by the rising edge of `clk2` a hold uncertainty of `2.0`:

  ```
  tempus > set_clock_uncertainty 2.0 -from clk1 -to clk2 -rise -hold
  ```

  In case of multiple constraints, the actual uncertainty applied by the software depends on which constraint matches the specific path in question.

- The following command sets an uncertainty on the `clk_in` clock root pin that is applied to all the register pins in the fanout of `clk_in`:

  ```
  tempus > set_clock_uncertainty 1.2 clk_in
  ```

- The following command specifies an inter-clock uncertainty of `0.2` between clock waveforms `clk1` and `clk2`:

  ```
  tempus > set_clock_uncertainty 0.2 -from clk1 -to clk2
  ```

- The following command specifies a target uncertainty of `0.5` for all registers clocked by the clock waveform `ideal_clk1`:

  ```
  tempus > set_clock_uncertainty 0.5 ideal_clk1
  ```

Related Commands

- `report_clocks`
- `set_clock_latency`
- `set_clock_transition`
- `reset_clock_uncertainty`

set_data_check

```
set_data_check
[-help]
<check_value>
[-clock <clock_object>]
[-hold]
[-setup]
{-from <pin_or_port_list> | -rise_from <pin_or_port_list> | -fall_from <pin_or_port_list>}
{-to <pin_or_port_list> | -rise_to <pin_or_port_list> | -fall_to <pin_or_port_list>}
```

Sets data-to-data checks using the specified values of setup and hold time. Data-to data checks are non-sequential checks that constrain a data signal with respect to another data signal. The reference (related) delay is defined by the `-from` pin/port and the target(s) by the `-to` pin/port. Note that these need not to be on the same component.

In a normal sequential check, a data signal is constrained with respect to a clock signal. In a sequential setup, the clock is incremented by a cycle between launch (`-from`) and capture (`-to`). In contrast, a data-to-data setup check is performed on the same edge as the launch. A clock can be specified to refine the triggering condition of the reference signal. You can use the `set_multicycle_path` constraint to adjust the cycle accounting rules.

Parameters

`-help`	Prints out the command usage.
`-clock` *clock_object*	Specifies the name of the clock that triggers the data event for the reference (`-from`) pin. In the case where there are multiple data events triggered by different clock phases arriving at the `-from` pin, the `-clock` parameter allows you to specify which data event should be used for the "related pin" side of the timing check.
`{-from <pin_or_port_list> | -rise_from <pin_or_port_list> | -fall_from <pin_or_port_list>}`	Specifies the pins or ports that are set as related pins for data-to-data checks. The *pin_or_port_list* argument can contain either object IDs or hierarchical names relative to the current module. Use only one `-from`, `-rise_from`, or `-fall_from` parameter per command. By default, the `-from` parameter checks both the rising and the falling edges. Using `-rise_from` parameter checks at the rising edge of the signal on the from pins. Using `-fall_from` parameter checks at the falling edge of the signal on the from pins. **Note:** You cannot use the `-from` parameter to specify multiple objects.
`-setup | -hold`	Specifies whether the data check value is for setup or hold check. If you do not specify either `-setup` or `-hold` option, the value applies to both setup and hold.

{-to <*pin_or_port_list*> | -rise_to <*pin_or_port_list*> | -fall_to <*pin_or_port_list*>}	Specifies the pins or ports that are set as constrained pins for data-to-data checks. The *pin_or_port_list* argument can contain either object IDs or hierarchical names relative to the current module. Use only one -to, -rise_to, or -fall_to parameter per command. By default, the -to parameter constrains both the rising and the falling edges. Using -rise_to parameter constrains the rising edge of the signal on the to pins. Using -fall_to parameter constrains the falling edge of the signal on the to pins.
check_value	Specifies the value of the setup or hold time for the check.

Examples

- The following example demonstrates how the set_data_check command can be used to constrain signals: Consider two control signals, say XE and SE, going to a macro block. Both of these are derived from the same state machine, and hence launched from the same clock domain. You want XE to reach the macro earlier than SE. The set_data_check constraint can meet this design intent, by applying data checks on non-sequential signals, without setting path specific limits using the following command:

  ```
  tempus > set_data_check -from [get_pins {sub/SE}] -to [get_pins {sub/XE}] -setup 0.5
  ```

  The set_data_check command establishes data-to-data checks using similar ideas of setup and hold time as in sequential checks, with the -from parameter denoting the reference signal (the clock in a traditional setup/hold timing check), and the -to parameter indicating the signal being constrained.

Related Commands

- reset_data_check

set_disable_clock_gating_check

```
set_disable_clock_gating_check
[-help]
pin_or_inst_list
```

Disables clock gating checks for the specified instances or pins.

The `set_disable_clock_gating_check` command has a character limit for the details that are logged into the SDC file (the `write_sdc` command output). This helps to prevent SDC file errors.

Parameters

`-help`	Prints out the command usage.
`pin_or_inst_list`	Specifies the list of pins or instances that are to be disabled.

Examples

- The following command disables clock gating checks for the specified instance:
  ```
  tempus > set_disable_clock_gating_check [get_cells clockgate1]
  ```

Related Commands

- `reset_disable_clock_gating_check`

set_disable_timing

```
set_disable_timing
[-help]
[-from pin_name -to pin_name]
obj_list
```

Disables timing propagation through the specified arcs, or a collection of instance-specific arcs that are created using the `get_arcs` command.

Parameters

`-help`	Prints out the command usage.
`-from pin_name -to pin_name`	Specifies the arc that is to be disabled. Specify the `-from` and `-to` options together. You can specify the `-from` and `-to` parameters when the *`obj_list`* contains instance or library cell objects. Do not specify the `-from` and `-to` parameters when the *`obj_list`* includes instance pin or library cell pin objects. The `-from` and `-to` options are optional. If not specified, all cell arcs found in the instance or library cell objects are disabled.
`obj_list`	Specifies the instance, instance pin, library cell, library cell pin objects, collection of library arcs, or collection of timing arcs. If the *`obj_list`* includes instance pin or library cell pin objects, all cell arcs attached to the specified pin objects are disabled. If the *`obj_list`* contains instance or library cell objects, and the `-from` and `-to` parameters are not specified, all arcs found in the specified instance or library cell objects are disabled.

Examples

Here are some examples.

- The following command disables the arcs that lead to the output pins of the `i1`, `i2`, and `i3` instances. Note that timing checks at the inputs of `i1`, `i2`, and `i3` are still active.

  ```
  tempus > set_disable_timing [get_cells {i1 i2 i3}]
  ```

- The following command disables the arc from pin `B` to pin `Z` in the `i1` and `i2` instances, and the arc from pin `A` to pin `Z` in the `i3` instance:

  ```
  tempus > set_disable_timing -from B -to Z [get_cells {i1 i2}]
  tempus > set_disable_timing -from A -to Z [get_cells {i3}]
  ```

- The following example disables timing check arcs from pin `CLK` to pin `D` of the `ff1` instance:

  ```
  tempus > set_disable_timing -from CLK -to D [get_cells ff1]
  ```

- The following example disables the outgoing cell arcs from instance input pin `i1/B`:

```
tempus > set_disable_timing [get_pins i1/B]
```

- The following example disables all arcs found in any instances whose reference cell is `AN2`:

```
tempus > set_disable_timing [get_lib_cells AN2]
```

- The following example disables a collection of timing arcs:

```
tempus > set arcs [get_lib_arcs -from typical/MX2X1/S0 -to typical/MX2X1/Y]
tempus > set_disable_timing [index_collection $arcs 3]
```

- The following example disables arcs created using `filter_collection` on library timing arcs:

```
tempus > set arc [filter_collection $arcs {when == "((A)&(!(B)))"}]
tempus > set_disable_timing $arc
```

Related Commands

- `report_timing`
- `reset_disable_timing`

set_dont_touch

```
set_dont_touch
[-help]
obj_list
[{true | false}]
```

Prevents the specified object from being modified during optimization. However, the specified object can be moved during optimization.

When the `set_dont_touch` command is set to `true`, the specified nets cannot be buffered or become part of the logic cone restructuring. However, you can resize the instances connected to this net because the `set_dont_touch` attribute does not affect the connected instances.

Parameters

`-help`	Prints out the command usage.
`{true | false}`	Sets or removes the `set_dont_touch` attribute for the specified list of objects. *Default:* `true`
`obj_list`	Specifies the collection or list of instances, library cells, nets, or sub-modules.

Examples

- The following command prevents the specified instance from being modified:
  ```
  tempus > set_dont_touch [get_cells {TWA/FF1}]
  ```
- The following commands prevent the specified net from being modified and check the setting with `get_property`:
  ```
  tempus > set_dont_touch clk1_net
  tempus > get_property [get_nets clk1_net] dont_touch true
  ```
- The following commands can be used to prevent any ECO inside RESULTS_CONV_INST hierarchical instance:
  ```
  tempus > set_dont_touch [get_cells RESULTS_CONV_INST/*] true
  tempus > set_dont_touch [get_nets RESULTS_CONV_INST/*] true
  ```

  The `set_dont_touch RESULTS_CONV_INST/* true` command will return `[get_cells RESULTS_CONV_INST/*]` only. This is because the software looks into the first match (type of object). If a match is not found, then the successive objects are searched.

Related Commands

- get_cells
- get_lib_cells
- get_nets
- get_property

set_dont_touch_network

```
set_dont_touch_network
[-help]
obj_list
[-no_propagate]
```

Prevents the combinational path connected to the given clocks, pins, or ports from being modified.

Parameters

`-help`	Prints out the command usage.
`-no_propagate`	Preserves all the nets driven by the specified pin up to the first leaf-cell input (combinational or sequential). If this parameter is omitted, then preserving stops at the first sequential cell in the network.
`obj_list`	Specifies the objects - clocks, pins, or ports - to be preserved.

Examples

- The following command prevents the combinational path connected to the specified pin from being modified:
  ```
  set_dont_touch_network [get_pins and1/z]
  ```

Related Commands

- `get_pins`

set_dont_use

```
set_dont_use
[-help]
lib_cell_list
[true | false]
```

Prevents the specified design, hierarchical module, or library cells from being used during optimization. However, optimization is allowed to size out don't use cells if instances are already there in the initial netlist.

Parameters

`-help`	Prints out the command usage.
`lib_cell_list`	Specifies the list of library cells.
`true | false`	Sets or removes the `set_dont_use` attribute for the specified object list. Default: `true`

Examples

- The following command resets all the previously defined `clock_sense` constraints from a design:

  ```
  tempus > reset_clock_sense -all
  ```

Related Commands

- `set_clock_sense`

set_drive

```
set_drive
[-help]
[-rise] [-fall]
[-max] [-min]
resistance_value
port_list
```

Sets the drive resistance on the specified input and/or bidirectional ports of the top cell.

The drive resistance models the cell driving the port. This resistance is used to compute the load dependent delay and the slew at the specified ports. The load-dependent delay is added to the arrival times.

The load-dependent delay and the slew are computed by multiplying the drive resistance by the wire capacitance of the net connected to the port.

Use the `report_ports` command to view drive resistances asserted on ports.

Parameters

`-help`	Prints out the command usage.
`-fall`	Specifies that the drive resistance is to be used for the falling transition at the specified ports. By default, if neither the `-rise` nor the `-fall` option are specified, the drive resistance applies to both the rising transition and the falling transition.
`-min`	Specifies that the drive resistance is to be used to compute the early path delay. By default, if neither the `-min` option nor the `-max` option is specified, the drive resistance applies to both the early path delay and the late path delay.
`-max`	Specifies that the drive resistance is to be used to compute the late path delay. By default, if neither the `-min` option nor the `-max` option is specified, the drive resistance applies to both the early path delay and the late path delay.
`port_list`	Specifies a list of input or bidirectional ports on which the drive resistance is set.
`resistance_value`	Specifies the value of the drive resistance. This value must be greater than or equal to zero. The drive resistance is in units found in the target technology library.
`-rise`	Specifies that the drive resistance is to be used for the rising transitions at the specified ports. By default, if neither the `-rise` option nor the `-fall` option are specified, the drive resistance applies to both the rising transition and the falling transition.

Examples

- The following command sets a drive resistance of 2.0 for the early path, the late path, the rising and the falling transitions at the in1 and in2 ports and shows how drive resistances are reported using the report_ports command as shown in the example below:

```
tempus > set_drive 2.0 [get_ports {in1 in2}]
tempus > report_ports -type drive_resistance -pin [get_ports {in1 in2}]
```

  The following report is displayed:

```
-------------------------------------------------------------------
Early Late
-------------------------------------------------------------------
Pin     Dir   Assertion       Clock   Rise   Fall   Rise   Fall
Name                          Name
-------------------------------------------------------------------
in1     IN    drive_resistance *(D)(P) 2.000   2.000  2.000  2.000
in2     IN    drive_resistance *(D)(P) 2.000   2.000  2.000  2.000
-------------------------------------------------------------------
```

Related Commands

- all_inputs
- get_ports
- report_ports
- reset_drive
- set_driving_cell
- set_input_transition
- set_load

set_driving_cell

```
set_driving_cell
[-help]
[-library library_name]
-lib_cell cell_name
[-pin pin_name]
[-from_pin from_pin_name]
[-rise]
[-fall]
[-min]
[-max]
[-multiply_by factor]
[-no_design_rule]
[-input_transition_rise rise_slew_value]
[-input_transition_fall fall_slew_value]
port_list
```

Models the drive capability of an external driver connected to the input port. Use this command only for timing analysis. The command does not affect the electrical properties of the design, which means that the capacitance of the output pin of the driving cell does not add to the total capacitance of the input port.

To model the drive capability of the external driver, the timing analyzer computes an offset to the arrival time of an input, and changes the slew time used to compute the delay for the next cell. The timing analyzer computes the offset by subtracting the zero load delay from the total delay of the cell. The `set_driving_cell` command uses the offset value to identify a cell from the library to drive the specified input port(s).

This command overrides the `set_drive` command per port or per net, if it is the last command applied. Also, if the `set_driving_cell` command is set for a port, the `set_input_transition` command is ignored for that port. If you do not define drive using the `set_driving_cell` command, the software uses the default value defined in the configuration file. The default value is `0.1ps`.

Parameters

`-help`	Prints out the command usage.
`-fall`	Annotates only the falling drive assertion. The default is to annotate both rise and fall delays.
`-from_pin` *from_pin_name*	Specifies the name of the input port of the driver cell that has an arc to the output port of the driver cell, which is connected to the input port of the design. This option is required only if there are multiple timing arcs in the driver cell from its input ports to its output ports, and the arcs from those pins have different drive characteristics. If you do not specify the `-from_pin` option, the default is to use the first timing arc found on the library cell.

`-input_transition_fall` *fall_slew_value*	Specifies the slew value for the falling signal at the input of the drive cell.
`-input_transition_rise` *rise_slew_value*	Specifies the slew value for the rising signal at the input of the drive cell.
`-lib_cell` *cell_name*	Specifies the name of the driver cell. This parameter supports wildcard patterns and collections.
`-library` *library_name*	Specifies the name of the library that contains the driving cell.
`-max`	Annotates only the late drive assertion. The default is to annotate both early and late.
`-min`	Annotates only the early drive assertion. The default is to annotate both early and late.
`-multiply_by` *factor*	Specifies a multiplier for the drive.
`-no_design_rule`	Does not allow transfer of design rules from the driving cell to the port. If you do not specify this option, the design rules (such as `max_capacitance`) of the library pin are applied to the port.
`-pin` *pin_name*	Specifies the name of the output port of the driver cell that drives the signal at the input port of the design. This option is required if there are multiple output ports on the driver. If there is only one output port on the driver cell, then using this option is not necessary.
port_list	Specifies the list of ports driven by the driver cell.
`-rise`	Specifies only the rising drive assertion is annotated.The default is to annotate both rise and fall delays.

Examples

- The following command sets 'AND' cell (arc B -> Y) as the driving cell for the input port IN1:

```
tempus > set_driving_cell -lib_cell AND -from_pin B -pin Y IN1
```

Related Commands

- `reset_driving_cell`
- `all_inputs`
- `report_ports`
- `set_drive`
- `set_input_transition`

set_false_path

```
set_false_path
[-help]
[-hold | -setup]
[-rise]
[-fall]
{[{-from | -rise_from | -fall_from} from_list]
[{-through | -rise_through | -fall_through} through_list]
[{-to | -rise_to | -fall_to} to_list]}
```

Identifies false paths in a design, and breaks or disables specific instance timing arcs in a design.

To break combinational loops, use the `set_disable_timing` command instead of the `set_false_path` command.

The `set_false_path` command differs from the `set_disable_timing` command as follows:

- `set_false_path` command selectively disables the arrival time information depending on the precedence rules.
- `set_false_path` command does not affect constant values.
- `set_disable_timing` command physically snips a timing arc such that neither arrival times nor constant values propagate through it.
- `set_disable_timing` command is applied in all cases to valid timing arcs.

The `set_false_path -from fpin` command disables all timing paths whose source pin is an `fpin`. Similarly, the `set_false_path -to tpin` command disables all timing paths whose sink pin is a `tpin`. Using the `set_false_path -from fpin -to tpin` command disables all timing paths whose source is `fpin` and sink is `tpin`.

Specify a group of pins by enclosing the group with curly braces (`{}`) within one `set_false_path` command.

To set a false path for any path beginning at `p1` or `p2` and terminating at `t1` or `t2`, group the pins as follows:
`set_false_path -from {p1 p2} -to {t1 t2}`

Note: This method of pin grouping reduces the number of exceptions. Having large numbers of exceptions adversely impacts the run time of the timing analysis commands.

Parameters

`-help`	Prints out the command usage.

`{-from | -rise_from | -fall_from} from_list`	Specifies a list of timing paths start points. The from_list argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. The from_list argument can be a collection. The valid start points are: • Input port • Bidirectional port • Clock pin of sequential cell Clock pin of sequential cell is one of the following: • Structural clock pin that is part of latch or flip-flop. • Inferred clock pin in cells like macro cells. A pin is an inferred clock pin if it is associated with one of the following: • Reference end of setup check • Reference end of hold check • Reference end of removal check • Reference end of recovery check • Minimum period check • Minimum pulse width check • Pin associated with set_input_delay Specify one of the following:
	`-from`: Applies the false path constraint to all paths originating from the from_list list. If a clock is specified as the -from object, all primary inputs and registers clocked by that clock are used as start points. By default, the -from option applies the constraint to both the rising and the falling edges.
	• `-rise_from`: Applies the false path constraint at the rising edge of the signal on the from pins.
	• `-fall_from`: Applies the false path constraint at the falling edge of the signal on the from pins.
`-hold/-setup`	Sets false paths for either setup or hold analysis. **Note**: If you do not specify -setup or -hold, the software sets false paths for both setup and hold analysis.
`-rise | -fall`	Applies the assertion at the rising edge of the signal on the path endpoints. Applies the assertion at the falling edge of the signal on the path endpoints.

`{-through | -rise_through | -fall_through} through_list`	Specifies a list of timing paths through points. The `through_list` argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. The *through_list* argument can be a collection. You can specify one of the following options: • `-through`: When you use this parameter without using the `-from` or `-to` options, the false path constraint is set for all paths that go through the specified pins. When you use it with both the `-from` and `-to` options, any path starting at any pin on the -from list and going through any point on the -through pin list and going to any point on the -to list is affected by the constraint. By default, the `-through` parameter applies constraints to both the rising and falling edges. • `-rise_through`: Applies the assertion at the rising edge of a signal on the through pins. • `-fall_through`: Applies the assertion at the falling edge of a signal on the through pins. **Note**: The `-through` parameter does not support clock paths.
`{-to| -rise_to | -fall_to} to_list`	Specifies a list of timing paths end points. The to_list argument can contain either object IDs or hierarchical names relative to the current module. The pins can be on intermediate hierarchical boundaries. The to_list argument can be a collection. The valid end points are: • Output port • Bidirectional port • Data pin of sequential cell Data pin of sequential cell is one of the following: • Structural data pin that is part of latch or flip-flop. • Inferred data pin in cells like macro cells. A pin is an inferred data pin if it is associated with one of the following: • Signal end of setup check • Signal end of hold check • Signal end of non-sequential setup check • Signal end of non-sequential hold check • Signal end of removal check • Signal end of recovery check • Pin associated with set_external_delay • Data pin of clock gating cell Select from one of the following:

	`-to`: Applies the false path constraint to all paths ending in the -to pin list. If a clock is specified as the -to object, all primary outputs and registers clocked by that clock are used as endpoint. By default, the -to option applies the constraint to both the rising and the falling edges.
	`-rise_to`: Applies the false path constraint at the rising edge of the signal on the to pins.
	`-fall_to`: Applies the false path constraint at the falling edge of the signal on the to pins.

Examples

- The following command disables hold checking from `ff1` to `ff2`:

  ```
  tempus > set_false_path -hold -from ff1 -to ff2
  ```

- The following command sets all timing paths from `ff1/CK` to `D` input of `ff2` that pass through `u1/Y` and one or more of `u2/Y` and `u3/Y` as false paths:

  ```
  tempus > set_false_path -from ff1/CK -through {u1/Y} -through {u2/Y u3/Y} -to ff2/D
  ```

- The following command sets all timing paths from the rise of `ff1/CK` to `ff2/D` through one or more of `u1/Y` and `u4/Y` as false paths:

  ```
  tempus > set_false_path -rise_from ff1/CK -through {u1/Y u4/Y} -to ff2/D
  ```

Related Topics

- Path Exception Priorities

Related Commands

- `set_disable_timing`
- `set_max_delay`
- `set_min_delay`
- `set_multicycle_path`
- `reset_path_exception`
- `report_path_exceptions`

set_fanout_load

```
set_fanout_load
[-help]
value
port_list
```

Specifies the fanout load on a set of ports.

The library instance pins can have fanout loads defined in the technology library. The fanout does not have specific units so it is assumed that a fanout as used by the constraints is consistent with the fanout defined in the technology libraries. If a pin or port has no fanout attribute, either from the constraints or from the technology library, then the fanout load will be zero. The fanout load is used only for design rule checking and does not impact the timing results.

Parameters

`-help`	Prints out the command usage.
value	Specifies the fanout load value for the port or ports.
port_list	Specifies the list of ports for which the fanout load is applied.

Examples

- The following command sets the fanout load of port `out` to `2`:

  ```
  tempus > set_fanout_load 2 out
  ```

- The following command sets the fanout load of ports `in1` and `in2` to `1`:

  ```
  tempus > set_fanout_load 1 {in1 in2}
  ```

- The following command sets the fanout load of element `3` of port `out` to `12`:

  ```
  tempus > set_fanout_load 12 { out[3] }
  ```

- The following command sets the fanout load of all the ports to `2`:

  ```
  tempus > set_fanout_load 2 [get_ports *]
  ```

Related Commands

- `report_nets`
- `report_ports`
- `set_max_fanout`
- `report_constraint`

set_ideal_latency

```
set_ideal_latency
[-help]
[-min]
[-max]
[-rise]
[-fall]
value
pin_or_port_list
```

Specifies the ideal latency to use for input or output leaf-cell pins and top-level ports that are part of an ideal network. These set a point delay on the specified object.

Note: You should specify `set_ideal_latency` with `set_ideal_network` to view the ideal latency result.

Parameters

`-help`	Prints out the command usage.
`-fall`	Specifies the ideal latency for the falling edge.
value	Specifies the ideal latency value.
`-max`	Specifies the ideal latency for the maximum corner.
`-min`	Specifies the ideal latency for the minimum corner.
pin_or_port_list	Specifies a list of objects - instance pins or ports.
`-rise`	Specifies the ideal latency for the rising edge.

Examples

- The following command adds an ideal latency of 1.5ns to the input pin A of cell FE_OFC413_scan_se:
  `tempus > set_ideal_latency 1.5 FE_OFC413_scan_se/A`

  Please note that the above command will only be valid if the pin is part of an ideal network.

  Now, this ideal latency value can be observed in the output of the following `report_timing` command:
  `tempus > report_timing -though FE_OFC413_scan_se/A`

```
Path 1: MET Setup Check with Pin RESULTS_CONV_INST/r697_reg[14]/CK
Endpoint: RESULTS_CONV_INST/r697_reg[14]/SE (v) checked with trailing edge of 'm_rcc_clk'
Beginpoint: scan_se (v) triggered by leading edge of 'm_ram_clk'
Path Groups: {m_rcc_clk}
Analysis View: scan_fast_RCMIN
Other End Arrival Time          20.679
- Setup                         1.123
+ Phase Shift                   0.000
+ Cycle Adjustment             40.000
- Uncertainty                   0.400
= Required Time                59.156
- Arrival Time                  8.000
= Slack Time                   51.156
Clock Rise Edge                      0.000
+ Input Delay                        4.000
+ Ideal Latency                      1.500
= Beginpoint Arrival Time            5.500
-------------------------------------------------------------
Pin                      Cell   Load    Slew   Delay Arc
                                                  Annotation
-------------------------------------------------------------
scan_se                         0.000   0.000         Ideal
FE_OFC413_scan_se/Z   BUF_X32   0.000   0.000  0.000  Ideal
FE_OFC247_scan_se/A   BUF_X16   0.000   0.000  0.000  Ideal
FE_OFC247_scan_se/Z   BUF_X16   0.000   0.000  0.000  Ideal
FE_OFC248_scan_se/A   BUF_X16   0.000   0.000  0.000  Ideal
FE_OFC248_scan_se/Z   BUF_X16   0.000   0.000  0.000  Ideal
FE_OFC249_scan_se/A   BUF_X8    0.000   0.000  0.000  Ideal
-------------------------------------------------------------
```

Related Commands

- reset_ideal_latency
- report_timing
- set_ideal_network

set_ideal_network

```
set_ideal_network
[-help]
obj_list
[-no_propagate]
```

Identifies driver pins or ports as sources of an ideal network. By default, all pins and nets in the transitive fanout of the specified port or driver pin are marked as ideal. Ideal network marking normally propagates through combinational cells and stops at sequential cells.

This command is not view/mode-based - it applies to all the active views.

The `set_ideal_network` command sets 0 slew for bi-directional pins. This ensures that unnecessary DRV violations do not occur.

Parameters

`-help`	Prints out the command usage.
`-no_propagate`	Prevents propagation through transitive fanout.
`obj_list`	Specifies a list of objects - instance pins, ports, or nets.

Examples

- The following command marks the fanout of the input port scan_se as an ideal network:

```
tempus > set_ideal_network scan_se
```

  This can be observed in the output of below mentioned report_timing command where the whole fanout of 'scan_se' is considered as ideal :

```
tempus > report_timing -from scan_se

Path 1: MET Setup Check with Pin RESULTS_CONV_INST/r697_reg[12]/CK
Endpoint: RESULTS_CONV_INST/r697_reg[12]/SE (v) checked with trailing edge of 'm_rcc_clk'
Beginpoint: scan_se (v) triggered by leading edge of 'm_ram_clk'
Path Groups: {m_rcc_clk}
Analysis View: scan_fast_RCMIN
Other End Arrival Time 20.700
- Setup 1.122
+ Phase Shift 0.000
+ Cycle Adjustment 40.000
- Uncertainty 0.400
= Required Time 59.178
- Arrival Time 4.000
= Slack Time 55.178
Clock Rise Edge 0.000
+ Input Delay 4.000
= Beginpoint Arrival Time 4.000
-------------------------------------------------------------------
Pin                  Cell      Load   Slew   Delay     Arc
                                                      Annotation
-------------------------------------------------------------------
scan_se ->                     0.000  0.000            Ideal
FE_OFC413_scan_se/Z  BUF_X32   0.000  0.000  0.000     Ideal
FE_OFC247_scan_se/Z  BUF_X16   0.000  0.000  0.000     Ideal
FE_OFC248_scan_se/Z  BUF_X16   0.000  0.000  0.000     Ideal
FE_OFC249_scan_se/Z  BUF_X8    0.000  0.000  0.000     Ideal
FE_OFC250_scan_se/Z  BUF_X16   0.000  0.000  0.000     Ideal
FE_OFC251_scan_se/Z  BUF_X4    0.000  0.000  0.000     Ideal
RES_CONV_IN/r697/SE  SDFF_X2   0.000  0.000  0.000     Ideal
-------------------------------------------------------------------
```

Related Commands

- reset_ideal_network

set_ideal_transition

```
set_ideal_transition
[-help]
[-min]
[-max]
[-rise]
[-fall]
value
pin_or_port_list
```

Specifies the ideal transition to use for input or output leaf-cell pins and top-level ports that are part of an ideal network. These set a point delay on the specified object.

Note: You should specify `set_ideal_transition` with `set_ideal_network` to view ideal transition result.

Parameters

`-help`	Prints out the command usage.
`-fall`	Specifies the ideal transition for the falling edge.
`-max`	Specifies the ideal transition for the maximum corner.
`-min`	Specifies the ideal transition for the minimum corner.
`pin_or_port_list`	Specifies a list of objects - instance pins or ports.
`-rise`	Specifies the ideal transition for the rising edge.
`value`	Specifies the ideal transition time.

Examples

Here are some examples.

- The following command specifies the ideal transition of 0.5ns on pin scan_se:
  ```
  tempus > set_ideal_transition 0.5 scan_se
  ```

  Please note that this ideal transition would only be applicable if the pin is part of an ideal network, otherwise this value would be ignored.

  This transition will be propagated through the ideal network as shown in the output of the following `report_timing` command:
  ```
  tempus > report_timing -from scan_se

  Path 1: MET Setup Check with Pin RESULTS_CONV_INST/r697_reg[14]/CK
  Endpoint: RESULTS_CONV_INST/r697_reg[14]/SE (v) checked with trailing edge of 'm_rcc_clk'
  ```

```
Beginpoint: scan_se (v) triggered by leading edge of 'm_ram_clk'
Path Groups: {m_rcc_clk}
Analysis View: scan_fast_RCMIN
Other End Arrival Time 20.699
- Setup 1.277
+ Phase Shift 0.000
+ Cycle Adjustment 40.000
- Uncertainty 0.400
= Required Time 59.022
- Arrival Time 4.000
= Slack Time 55.022
Clock Rise Edge 0.000
+ Input Delay 4.000
= Beginpoint Arrival Time 4.000
-------------------------------------------------------------
Pin                   Cell      Load     Slew Delay     Arc  Annotation
-------------------------------------------------------------
scan_se ->                       0.000   0.500          Ideal
FE_OFC413_scan_se/Z   BUF_X32    0.000   0.500  0.000   Ideal
FE_OFC247_scan_se/Z   BUF_X16    0.000   0.500  0.000   Ideal
FE_OFC248_scan_se/Z   BUF_X16    0.000   0.500  0.000   Ideal
FE_OFC249_scan_se/Z   BUF_X8     0.000   0.500  0.000   Ideal
FE_OFC250_scan_se/Z   BUF_X16    0.000   0.500  0.000   Ideal
FE_OFC251_scan_se/Z   BUF_X4     0.000   0.500  0.000   Ideal
RESULT_CONV_INST/SE   SDFF_X2    0.000   0.500  0.000   Ideal
```

Related Commands

- reset_ideal_transition

set_input_delay

```
set_input_delay
[-help]
[-clock clock_name]
[-clock_fall]
[-rise] [-fall]
[-max] [-min]
[-add_delay]
[-network_latency_included]
[-source_latency_included]
[-reference_pin pin_name]
[-level_sensitive]
value
pin_or_port_list
```

Defines the arrival time relative to a clock edge on input ports or internal input pins. This input path delay models the delay from an external register to an input port of the module.

If the input delay is not specified on an input port, it is assumed to be zero.

Note: If the `set_input_delay` constraint is added incrementally in SI mode, an explicit `update_timing -full` command is required (before `report_timing` or `report_constraint`) for SI full accuracy.

Parameters

`-help`	Prints out the command usage.
`-add_delay`	Adds delay information to the existing input delay. The additional delay information can be for multiple paths relative to different clocks or clock edges leading to an input port or internal input pins. If you do not use the `-add_delay` option, the software uses the last `set_input_delay` related to a pin when there are multiple `set_input_delay` statements for the same pin.
`-clock` *clock_name*	Specifies that the delay is relative to the rising edge of the given clock. If you do not specify the `-clock` parameter, the delay is relative to time zero. To specify a delay relative to the falling edge of the clock, use the -clock_fall option with the -clock option. When you specify multiple clocks, an error occurs and the constraint is ignored.
`-clock_fall`	Specifies that the delay is relative to the falling edge of the clock. You must specify the -clock parameter with this parameter. *Default:* Delay is relative to the rising edge. You cannot specify the `-clock_fall` parameter without the `-clock` parameter.
value	Specifies the value for the path delay.

`-fall`	Specifies that the input delay refers to the falling edge of the signal at specified ports or pins. If you do not specify the `-fall` option, the input delay applies to both the edges.
`-level_sensitive`	Models input delay that is launched by a positive level sensitive latch. Use the `-clock` parameter to model a positive level sensitive latch, or the `-clock_fall` parameter to model a negative level sensitive latch. You cannot specify the `-level_sensitive` parameter without the `-clock` parameter.
`-max`	Specifies that the delay refers to the setup analysis. If you do not specify the `-max` parameter, the input delay applies to both setup and hold analysis. **Note**: If for a specific clock the maximum delay is less than the minimum delay, then during analysis the software makes the maximum delay equal to the minimum delay.
`-min`	Specifies that the delay refers to the hold analysis. If you do not specify the `-min` parameter, the input delay applies to both setup and hold analysis.
`-network_latency_included`	Specifies that the clock network latency should not be added to the input delay value. If you do not specify this option, the clock network latency of the related clock is added to the input delay value. This parameter has no effect if the specified clock is in propagated mode.
pin_or_port_list	Specifies a list of input port or pin names. The pin_or_port_list argument can be a collection.
`-reference_pin` *pin_name*	Adds source and network latency of the specified reference pin to the input delay value in propagated mode. A reference pin is a leaf pin in the fanout cone of the clock source of the clock that you specify using the `-clock` parameter. You cannot use the `-reference_pin` parameter to specify multiple pins. If specified, the command issues a warning and accepts the first pin. You cannot use the `-reference_pin` parameter with the `-network_latency_included` or `-source_latency_included` parameters. You cannot use the `-reference_pin` parameter to specify hierarchical pins.
`-rise`	Specifies that input delay refers to the rising edge of the signal at the specified ports or pins. If you do not specify the `-rise` option, the input delay applies to the both edges.

`-source_latency_included`	Specifies that the clock source latency should not be added to the input delay value. With different delay values for `-rise/-fall` and `-min/-max`, only the last specified combination of `-network_latency_included` and `-source_latency_included` parameters is honored. The specified combination can be one of the following: a) Either `-network_latency_included` or `-source_latency_included` b) Both of these c) None of these

Examples

- The following command sets an input delay of 1.0 for all inputs in the current design relative to the rising edge of the `CLK1` clock:

```
tempus > set_input_delay -clock CLK1 1.0 [all_inputs]
```

- The following command sets an input delay of 2.0 for the input `IN1` port relative to the falling edge of the `CLK1` clock:

```
tempus > set_input_delay -clock CLK1 2.0 -clock_fall { IN1 }
```

- The following example shows an input `IN1` port that is constrained by both the rising edge and the falling edge of the same `CLK1` clock. The `-add_delay` option prevents the second `set_input_delay` statement from overwriting the first `set_input_delay`:

```
tempus > set_input_delay -clock CLK1 1.0 { IN1 }
tempus > set_input_delay -clock CLK1 -clock_fall -add_delay 2.0 { IN1 }
```

- When multiple `-min` and `-max` values are specified relative to the same clock using the `-add_delay` option at the same port or pin, the worst case value for each case is used. For the `-min` case the smallest value is retained, and for the `-max` case the largest value is retained. For example, the input delay value of 1.0 is stored for the shortest path and the value of 4.0 is stored for the longest path in the following example:

```
tempus > set_input_delay -min -clock CLK1 1.0 { IN1 }
tempus > set_input_delay -min -clock CLK1 2.0 -add_delay { IN1 }
tempus > set_input_delay -max -clock CLK1 3.0 -add_delay { IN1 }
tempus > set_input_delay -max -clock CLK1 4.0 -add_delay { IN1 }
```

Related Commands

- all_inputs
- create_clock
- report_ports
- set_load
- set_output_delay
- report_path_exceptions

set_input_transition

```
set_input_transition
[-help]
[-min]
[-max]
[-rise]
[-fall]
value
port_list
```

Specifies the slew time for a port on the top level module. This command is useful only in specifying a hard ramp signal as if the port were driven by a very powerful driver. The command does not change the arrival time of an input, but changes the slew time used to compute the delay of the cell on the sink of a net. If you do not define transition using the `set_input_transition` command, the software uses the default input port transition defined in the configuration file. The default value is `0.1ps`. You can also set the `delaycal_input_transition_delay attribute` to override the default value.

The `set_input_transition` command specification for clock signals does not impact setup/hold timing calculations.

Note: The `set_input_transition` command does not support internal pins. For specifying transition on internal pins use the `set_annotated_transition` command.

Note: The most recent command settings (of `set_driving_cell`, `set_input_transition`, or `set_drive`) that are applied to a port will be considered - the software will override the previous commands.

The `set_input_threshold` command should be issued after the `set_lib_binding_by_voltage` command.

Parameters

`-help`	Prints out the command usage.
`-fall | -rise`	Specifies that the slew time should be applied to the rising edge or the falling edge of the signal (data or clock). If neither time is specified, the default is to use both.
`-min | -max`	Indicates whether the slew times are with respect to the early (hold) or the late (setup) time checks of the data signal. If neither option is specified, the default is to use both.
port_list	Specifies the list of ports for which the slew time is specified. **Note:** You cannot use the `set_input_transition` command to specify transition on output ports. **Note:** When you use both `set_input_transition` and `set_annotated_transition` commands for specifying transition on input ports, the worst value is used as the transition value.
value	Specifies the transition time.

Examples

- The following command specifies the slew time of `0.5` on port `input3`.

```
set_input_transition 0.5 input3
```

Related Commands

- `report_ports`
- `set_driving_cell`
- `set_drive`
- `set_clock_transition`

set_load

```
set_load
[-help]
[-max] [-min]
[-pin_load]
[-wire_load]
[-subtract_pin_load]
capacitance_value
port_or_net_list
```

Sets the specified capacitance on the defined ports (or nets) of the top cell.

The specified capacitance overrides the capacitance from the wire load models and SPEF.

You can use the `report_ports` command to view capacitance of asserted ports. The pin load appears in the report as `port_cap` in the assertion column and the wire load appears as `port_wire_cap` in the assertion column.

In the timing report (generated using the `report_timing` command), the load value for ports is displayed as a sum of the `set_load` value on a port and the existing wire capacitance obtained from the SPEF file, and pin capacitance obtained from the library files.

Parameters

`-help`	Prints out the command usage.
capacitance_value	Specifies the value for the capacitance. The capacitance is in units from the target technology library.
`-min`	Specifies the minimum capacitance. The minimum capacitance is used to compute best-case delays in BcWc analysis mode and early path delays in OCV analysis mode.
`-max`	Specifies the maximum capacitance. The maximum capacitance is used to compute worst-case delays in BcWc analysis mode and late path delays in OCV mode. It is also used to check the maximum capacitance design rule.
port_or_net_list	Specifies a list of ports or nets in the top cell.
`-pin_load`	Specifies the external pin capacitance for a port. You can use this parameter for ports only. For ports, the asserted capacitance is either the external pin capacitance when the `-pin_load` parameter is specified, or the external wire capacitance when the `-wire_load` parameter is specified, or the total external capacitance when neither option is specified.

`-subtract_pin_load`	This parameter can be used only if nets are specified in the object list. When this parameter is specified, the total net capacitance value will be the same as set by the `set_load` *capacitance_value object_list* command. When this parameter is not specified, the asserted load value will only change the wire capacitance value from the SPEF. The total net capacitance value will be the `set_load` capacitance value plus the pin capacitance from the library file.
`-wire_load`	Interprets the specified capacitance as the external wire capacitance of the port. Use this option only for ports. Use this option with the *`port_net_list`* argument if the list contains only ports.

Examples

Here are some examples.

- The following command sets an external minimum pin capacitance of 1.5 units on the `in1 in2` ports:
  ```
  tempus > set_load 1.5 -min -pin_load [get_ports {in1 in2}]
  ```
- The following command sets an external minimum wire capacitance of 1.2 units on the `in1 in2` ports:
  ```
  tempus > set_load 1.2 -min -wire_load [get_ports {in1 in2}]
  ```
- The following command sets an external maximum pin capacitance of 2.6 units on the `in1 in2` ports:
  ```
  tempus > set_load 2.6 -max -pin_load [get_ports {in1 in2}]
  ```
- The following command sets an external maximum wire capacitance of 3.0 units on the `in1 in2` ports:
  ```
  tempus > set_load 3.0 -max -wire_load [get_ports {in1 in2}]
  ```
- The following command reports the capacitances set on the ports, as shown in the example below:
  ```
  tempus > report_ports -pin [get_ports {in1}]

  ------------------------------------------------------------
  Early Late
  ------------------------------------------------------------
  Pin   Dir Assertion       Clock   Rise   Fall  Rise  Fall
  Name                      Name
  ------------------------------------------------------------
  in1   IN drive_resistance *(D)(P) 2.000  2.000  2.000  2.000
  ------------------------------------------------------------
  ------------------------------------------------------
  Pin    Dir   Assertion     Value
  Name
  ------------------------------------------------------
  in1   IN    port_cap      ( 1.500 : : 2.600 )
  in1   IN    port_wire_cap ( 1.200 : : 3.000 )
  ------------------------------------------------------
  ```

Related Commands

- all_outputs
- report_ports
- report_timing
- set_drive
- set_resistance

set_logic_one

```
set_logic_one
-help
pin_or_port_list
```

Sets ports or pins to logic 1 in the design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_logic_one` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_logic_one`.
`pin_or_port_list`	Specifies the list of the ports or pins that are to be set to logic 1.

Examples

- The following command sets the specified port to logic 1:

  ```
  tempus > set_logic_one [get_ports {din[1]}]
  ```

Related Commands

- `get_ports`

set_logic_zero

```
set_logic_zero
-help
pin_or_port_list
```

Sets ports or pins to logic 0 in the design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_logic_zero` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_logic_zero`.
`pin_or_port_list`	Specifies the list of the ports or pins that are to be set to logic 0.

Examples

- The following command sets the specified port to logic 0:

  ```
  tempus > set_logic_zero [get_ports {din[0]}]
  ```

Related Commands

- `get_ports`

set_max_capacitance

```
set_max_capacitance
[-help]
limit
obj_list
[-clock_path]
[-data_path]
[-fall]
[-override]
[-rise]
```

Sets the maximum capacitance limit on the specified ports of the top cell, the specified designs, or clock waveforms.

When set on a port, the total capacitance of the net connected to that port must be less than or equal to the specified limit. Maximum capacitance limits also come from library objects, which is the default maximum capacitance limit of libraries and the maximum capacitance limit on library cell pins. Pins in the design inherit the limits from libraries.

Note: The limit set on the top cell applies to its ports. Thus, if maximum capacitance limits are set on the top cell and its ports, the most constraining limit will apply to the ports.

When limits are set on a clock waveform, you can use `-clock_path`, `-data_path`, `-rise`, and `-fall` parameters to provide more specifications.

The `set_max_capacitance` constraint on clock objects will be honored only by the `report_constraint` command for DRV violations. The constraint will not be used in optimizations.

Parameters

`-help`	Prints out the command usage.
`-clock_path`	Applies the max capacitance constraint only to pins in the clock network of the clocks defined in the object list.
`-data_path`	Applies the max capacitance constraint only to data path pins, where the path is launched by the clocks defined in the object list.
`-fall`	Applies the max capacitance constraint only to falling capacitance.
`-rise`	Applies the max capacitance constraint only to rising capacitance.
`limit`	Specifies the value for the minimum capacitance limit. The limit must be greater than or equal to zero. The limit is in capacitance units from the target technology library.
`obj_list`	Specifies a list of objects - ports, instance pins, library cell pins, designs, and/or clock waveform.

`-override`	Overrides the capacitance limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting.

Examples

Here are some examples.

- The following example sets a maximum capacitance limit of `1.2` units on all the ports of the top cell and a maximum capacitance limit of `0.5` units on the port named `d_out`:

```
tempus > set_max_capacitance 1.2 [get_ports *]
tempus > set_max_capacitance 0.5 d_out
```

- The following example uses the `report_ports` command to show the maximum capacitance limit set on ports:

```
tempus > set_max_capacitance 0.9 [get_ports *]
tempus > report_ports -type port_cap_limit -pin [get_ports *]
```

  The following report is displayed:

```
----------------------------------------
Pin    Dir  Assertion       Value
Name
----------------------------------------
CLK2   IN   port_cap_limit  ( : 0.900 )
CLK1   IN   port_cap_limit  ( : 0.900 )
in2    IN   port_cap_limit  ( : 0.900 )
in1    IN   port_cap_limit  ( : 0.900 )
out2   OUT  port_cap_limit  ( : 0.900 )
out1   OUT  port_cap_limit  ( : 0.900 )
----------------------------------------
```

Related Commands

- `reset_max_capacitance`
- `get_ports`
- `report_ports`
- `set_max_transition`
- `set_max_fanout`
- `report_constraint`

set_max_delay

```
set_max_delay
[-help]
[-combinational_from_to | -ignore_clock_latency]
[{-from |-rise_from | -fall_from} from_list]
[-no_segment]
[{-through | -rise_through | -fall_through} through_list]
[{-to | -rise_to | -fall_to} to_list]
[-comment string]
[-rise]
[-fall]
value
```

Specifies a maximum delay for a timing path. The `set_max_delay` command takes precedence over other path exceptions specified for the same path, except for the `set_false_path` command.

Use either the `-from`, `-to`, or `-through` option with the `set_max_delay` command to identify the paths. Specifying both the `-from` and `-to` options identifies a path with specific starting and ending points.

When you specify a sequential cell name for starting point of the timing path, the valid path starts from the clk pin. This means the latency and delay from clk to output Q is also included in the delay. For path segmentation, you can specify the path to start from Q output of the register by adding /Q after the cell name. In this case, path from clk to Q is not considered in the calculation.

Use only one `-from`, `-rise_from`, or `-fall_from`, `-to`, `-rise_to`, or `-fall_to` parameter per command. You can use multiple `-through` parameters in the same command.

Note: If the `set_max_delay` constraint is added incrementally in SI mode, an explicit `update_timing -full` command is required (before `report_timing` or `report_constraint`) for SI full accuracy.

Parameters

`-help`	Prints out the command usage.

`-combinational_from_to`	Defines a path defined by -from/to pins as combinational. When this parameter is used, the path between the -`from`/-`to` pins is treated as purely combinational, that is, both the source/capture clock latencies are excluded from slack computation. The presence of -`from`/`-to` pins is required for this option to take effect; the -`from`/`-to` pins can be sequential or combinational start/end pins. The `-combinational_from_to` parameter will be deprecated in a future release. This parameter will continue to work in the current release, but for compatibility with future releases you should migrate your scripts to use the `-ignore_clock_latency` parameter. You can use the following attributes to match the semantics between the `-ignore_clock_latency` and `-combinational_from_to` parameters: • `timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency` • `timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency`
`-comment` *string*	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
`-fall`	Apply the assertion at the falling edge of the signal on the path endpoints.

`-from |-rise_from |-fall_from` *`from_list`*	Specifies a list of timing paths start points. The *`from_list`* argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The *`from_list`* argument can be a collection. The valid start points are: • Input port • Bidirectional port • Clock pin of sequential cell Clock pin of sequential cell is one of the following: • Structural clock pin that is part of latch or flip-flop. • Inferred clock pin in cells like macro cells. A pin is an inferred clock pin if it is associated with one of the following: • Reference end of setup check • Reference end of hold check • Reference end of removal check • Reference end of recovery check • Minimum period check • Minimum pulse width check • Pin associated with `set_input_delay` Specify one of the following:
	`-from`: Applies the maximum delay constraint to all paths originating from the *`from_list`* list. If a clock is specified as the `-from` object, all primary inputs and registers clocked by that clock are used as start points. By default, the `-from` option applies the constraint to both the rising and the falling edges.
	`-rise_from`: Applies the maximum delay constraint at the rising edge of the signal on the from pins.
	`-fall_from`: Applies the maximum delay constraint at the falling edge of the signal on the from pins.
`-no_segment`	Specifies the constraint as non-segmenting. When specified, the software allows path delay checks without path segmentation.

`-ignore_clock_latency`	When this parameter is specified, the software does not count the launch/capture clock latency delays into the timing path delay. The `-ignore_clock_latency` parameter is an industry-standard construct introduced in SDC 2.1 to control the handling of clock-related timing with respect to the `set_max_delay`/`set_min_delay` constraints.
`-rise`	Applies the assertion at the rising edge of the signal on the path endpoints.
`-through | -rise_through | -fall_through` *`through_list`*	Specifies a list of timing paths through points. The *`through_list`* argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The *`through_list`* argument can be a collection. Specify one of the following:
	`-through`: When you use this parameter without using the `-from` or `-to` options, all paths that go through the specified pins have the specified delay. When you use it with both the `-from` and `-to` options, any path starting at any pin on the `-from` list *and* going through any point on the `-through` pin list *and* going to any point on the `-to` list is affected by the constraint. By default, the `-through` option applies the constraint to both the rising and the falling edges.
	`-rise_through`: Applies the assertion at the rising edge of the signal on the through pins.
	`-fall_through`: Applies the assertion at the falling edge of the signal on the through pins.

`-to| -rise_to	-fall_to` *`to_list`*	Specifies a list of timing paths end points. The *`to_list`* argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The *`to_list`* argument can be a collection. The valid end points are: • Output port • Bidirectional port • Data pin of sequential cell Data pin of sequential cell is one of the following: • Structural data pin that is part of latch or flip-flop. • Inferred data pin in cells like macro cells. A pin is an inferred data pin if it is associated with one of the following: • Signal end of setup check • Signal end of hold check • Signal end of non-sequential setup check • Signal end of non-sequential hold check • Signal end of removal check • Signal end of recovery check • Pin associated with `set_external_delay` • Data pin of clock gating cell Select from one of the following:
	`-to`: Applies the maximum delay constraint to all paths ending in the `-to` pin list. If a clock is specified as the `-to` object, all primary outputs and registers clocked by that clock are used as endpoint. By default, the `-to` option applies the constraint to both the rising and the falling edges.
	`-rise_to`: Applies the maximum delay constraint at the rising edge of the signal on the to pins.
	`-fall_to`: Applies the maximum delay constraint at the falling edge of the signal on the to pins.
`value`	Specifies the maximum delay value.

Examples

- The following command specifies that all paths from `in1` to `out1` must have delays less than `10` units:

```
tempus > set_max_delay 10 -from in1 -to out1
```

Related Topics

- Path Exception Priorities

Related Commands

- `create_clock`
- `set_false_path`
- `set_input_delay`
- `set_min_delay`
- `set_multicycle_path`
- `set_output_delay`
- `reset_path_exception`
- `report_path_exceptions`

set_max_fanout

```
set_max_fanout
[-help]
limit
obj_list
[-override]
```

Sets the maximum fanout load limit constraint on the specified instance pins, library cell pins, ports of the top cell, clock waveforms, and/or specified modules. When pins of cell instances have their own maximum constraint defined in the library, the software will pick the more aggressive value of the two.

When set on a port, the fanout load of the net connected to the port must be less than or equal to the specified limit value to meet the constraint. The fanout load of a net is the sum of the fanout loads of the pins driven by the net. When set on a module, the limit applies to all the nets of that module. Libraries and library cell pins may also have the maximum fanout load limit. In this case the most constraining limit is applied to the nets.

Use the `report_ports -type fanout_load_limit` command to view the maximum fanout load limits set on ports of the top cell.

Fanout load limit can be asserted on input ports, output pins, bidirectional pins, or ports and library pins. You cannot apply the constraint on input pins and output ports.

Parameters

`-help`	Prints out the command usage.
limit	Specifies the value of the maximum fanout load limit constraint. This is a float number and it must be greater than or equal to zero.
obj_list	Specifies a list of objects - input ports, instance output pins, designs, or library cell output pins.
`-override`	Overrides the maximum fanout limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting.

Examples

- The following example sets a maximum fanout load limit of `5.0` units on all the ports of the top cell:

  ```
  tempus > set_max_fanout 5.0 [get_ports *]
  ```

- The following example shows a report of the maximum fanout load limit set on ports using the `report_ports` command:

  ```
  tempus > set_max_fanout 5.0 [get_ports *]
  tempus > report_ports -type fanout_load_limit -pin [get_ports *]
  ```

  The following report is displayed:

  ```
  ----------------------------------------
  ```

```
Pin     Dir   Assertion           Value
Name
-----------------------------------------
CLK2    IN    fanout_load_limit   5.000
CLK1    IN    fanout_load_limit   5.000
in2     IN    fanout_load_limit   5.000
in1     IN    fanout_load_limit   5.000
-----------------------------------------
```

- The following example sets a maximum fanout load limit of `5.0` units on all the ports of the top cell:

  ```
  tempus > set_max_fanout 5.0 [get_ports *]
  ```

- The following example shows a report of the maximum fanout load limit set on ports using the `report_ports` command, as shown in the example below:

  ```
  tempus > set_max_fanout 5.0 [get_ports *] report_ports -type fanout_load_limit -pin [get_ports *]
  ```

Related Commands

- `reset_max_fanout`
- `get_ports`
- `report_ports`
- `set_fanout_load`
- `set_max_capacitance`
- `set_max_transition`
- `report_constraint`

set_max_time_borrow

```
set_max_time_borrow
[-help]
value
obj_list
[-interface]
```

Specifies the maximum time that can be borrowed by one stage from the next logic stage following a latch to meet timing constraints. Time borrowing is sometimes called cycle stealing. Time borrowing is performed automatically when performing timing analysis.

Use the `set_max_time_borrow` assertion on pins or waveforms. If the assertion is placed on a clock waveform, all latches triggered by this clock signal get the specified maximum time borrow limit. If both the clock waveform arriving at the clock pin of a latch and the clock pin itself have time borrow limit constraints, the maximum borrow limit on the latch is decided by the assertion on the latch clock pin.

Parameters

`-help`	Prints out the command usage.
`-interface`	Specifies the IO paths for which maximum time will be borrowed.
value	Specifies the maximum amount of time that can be borrowed. The minimum value allowed is `0`, which disables time borrowing. The maximum allowed is equal to the pulse width minus the setup time: Max = pulse width - setup
obj_list	Specifies the list of instance pins, instances, or clocks on which the borrow value is applied. The *object*`_list` argument can be a collection.

Examples

Here are some examples.

- `tempus > set_max_time_borrow 2 [get_cells {i1, i2, i3}]`
- `tempus > set_max_time_borrow 1.7 I4/g`
- `tempus > set_max_time_borrow 2.5 [get_clocks CLK1]`

Related Commands

- `get_clocks`
- `get_cells`
- `get_pins`
- `reset_max_time_borrow`

set_max_transition

```
set_max_transition
[-help]
[-clock_path]
[-data_path]
[-rise]
[-fall]
[-override]
transition_limit
object_list
```

Sets the maximum slew time limit (transition) on the specified ports of the top cell, and/or clock waveforms.

When set on a port, the slew time at the port must be less than or equal to the specified limit. Maximum transition limits on pins can come from library objects, which are the default maximum transition of the libraries or maximum transition on library cell pins. In this case, the most constraining limit, which is the smallest, will apply on pins and ports.

Use the `report_ports -type slew_limit` command to view the maximum transition limits set on ports of the top cell.

When limits are set on a clock waveform, you can use `-clock_path`, `-data_path`, `-rise`, and `-fall` parameters to provide more specifications.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_max_transition` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_max_transition`.
`-clock_path`	Applies the max transition constraint only to pins in the clock network of the clocks defined in the object list.
`-data_path`	Applies the max transition constraint only to data path pins, where the path is launched by the clocks defined in the object list.
`-fall`	Applies the max transition constraint only to falling transitions.
`object_list`	Specifies the list of objects - ports, instance pins, library cell pins, clock waveforms, or current design.
`-rise`	Applies the max transition constraint only to rising transitions.
`transition_limit`	Specifies the value of the maximum slew time limit (maximum transition), which must be greater than or equal to zero. The maximum slew time limit is in time units in the current session.

`-override`	Overrides the transition limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting.

Examples

Here are some examples.

- The following command sets a maximum transition limit of `0.9` units on all the ports of the top cell:

  ```
  tempus > set_max_transition 0.9 [get_ports *]
  ```

- The following example shows a report of the maximum transition limit set on ports using the `report_ports` command, as shown in the example below:

  ```
  tempus > set_max_transition 0.9 [get_ports *]
  tempus > report_ports -type slew_limit -pin [get_ports *]
  ```

  The following report is displayed:

  ```
  Pin Name    Dir     Assertion       Value
  in1         IN      slew_limit     ( : 0.9000 )
  in2         IN      slew_limit     ( : 0.9000 )
  out1        OUT     slew_limit     ( : 0.9000 )
  out2       OUT      slew_limit      ( : 0.9000 )
  out3       OUT      slew_limit      ( : 0.9000 )
  out4       OUT      slew_limit      ( : 0.9000 )
  ```

- The following command sets the maximum transition limit of 0.001 units on pins in the `all_clocks` collection:

  ```
  tempus > set_max_transition 0.001 -clock_path [all_clocks]
  ```

- The following command sets the transitions to the specified value for your entire design:

  ```
  tempus > set_max_transition value [current_design]
  ```

- The following command impacts report_contraint output as shown below:

  ```
  set_max_transition 0.01 TDP_COE_INST/TDP_COE_GL_INST/p82
  ```

  The following command shows output as shown below:

  ```
  report_constraint -all_violators -drv_violation_type max_transition

  ##################################################################
  # format : frame 0 : split 0

  Check type : max_transition

  ------------------------------------------------------------------------
  Pin Name                          Required Actual  Slack    View
  ------------------------------------------------------------------------
  ```

```
TDP_COE_INST/TDP_COE_GL_INST/p82  0.010    1.138   -1.128   func_slow_RCMAX
--------------------------------------------------------------------------------
```

Related Commands

- get_ports
- report_ports
- set_max_capacitance
- report_constraint

set_min_capacitance

```
set_min_capacitance
[-help]
[-clock_path]
[-data_path]
[-rise]
[-fall]
[-override]
capacitance_limit
obj_list
```

Sets the minimum capacitance limit on the specified ports of the top timing module and/or clock waveforms.

When set on a port, the total capacitance of the net connected to that port must be greater than or equal to the specified limit. When set on a module, the limit applies to all the nets of that module. The total capacitance of each net of the module must be greater than or equal the specified limit. The minimum capacitance limit also comes from library objects, which is the default minimum capacitance limit of libraries and minimum capacitance limit on library cell pins. Pins in the design inherit the limits from libraries. When limits are set on a module and the limit is inherited from libraries, the most constraining limit, which is the largest limit, is used.

Note: The limit set on the top timing module applies to its ports. Thus, if minimum capacitance limits are set on the top timing module and its ports, the most constraining limit will apply to the ports.

Use the `report_ports -type port_cap_limit` command to view the minimum capacitance limits set on ports.

When limits are set on a clock waveform, you can use `-clock_path`, `-data_path`, `-rise`, and `-fall` parameters to provide more specifications.

The `set_min_capacitance` constraint on clock objects will be honored only by the `report_constraint` command for DRV violations. The constraint will not be used in optimizations.

Note: This command syntax is applicable only when the flow-compatible mode is on.

Parameters

`-help`	Prints out the command usage.
`-clock_path`	Specifies all pins that are in the network of the specific clocks in the object list.
`-data_path`	Specifies all pins that are in the data paths launched by the specific clocks in the object list.
`-fall`	Specifies falling capacitance.
`-rise`	Specifies rising capacitance for all selected pins.
capacitance_limit	Specifies the value for the minimum capacitance limit. The limit must be greater than or equal to zero. The limit is in capacitance units from the target technology library.
obj_list	Specifies the list of objects - ports, instance pins, library cell pins, modules, and/or clock wave forms.
`-override`	Overrides the minimum capacitance limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting.

Examples

- The following example sets a minimum capacitance limit of 0.02 units on all the ports of the top timing module and a minimum capacitance limit of 0.05 units on the top timing module:

  ```
  tempus > set_min_capacitance 0.02 [get_ports *]
  tempus > set_min_capacitance 0.05 [current_design]
  ```

  The limit used for ports is 0.05, which is the most restrictive limit set on the top timing module (0.05) and the limit set on ports (0.02).

- The following example uses the `report_port` command to show the minimum capacitance limit set on ports:

  ```
  tempus > set_min_capacitance 0.02 [get_ports *]
  tempus > report_ports -type port_cap_limit -pin [get_ports *]
  ```

- The following command sets the minimum capacitance limit of 0.001 units on pins in the `all_clocks` collection (see example below):

  ```
  tempus > set_min_capacitance 0.001 -data_path [all_clocks]
  tempus > report_ports -type port_cap_limit -pin [get_ports *]
  ```

  The following report is displayed:

  ```
  -------------------------------------------------
  Pin      Dir    Assertion         Value
  Name
  -------------------------------------------------
  ```

```
CLK2     IN      port_cap_limit    ( 0.020 : 0.900 )
CLK1     IN      port_cap_limit    ( 0.020 : 0.900 )
in2      IN      port_cap_limit    ( 0.020 : 0.900 )
in1      IN      port_cap_limit    ( 0.020 : 0.900 )
out2     OUT     port_cap_limit    ( 0.020 : 0.900 )
out1     OUT     port_cap_limit    ( 0.020 : 0.900 )
-------------------------------------------------
```

Related Commands

- report_ports
- reset_min_capacitance
- set_max_capacitance
- set_max_transition
- set_min_transition
- report_constraint

set_min_delay

```
set_min_delay
[-help]
[-combinational_from_to | -ignore_clock_latency]
[{-from |-rise_from | -fall_from} from_list]
[-no_segment]
[{-through | -rise_through | -fall_through} through_list]
[{-to | -rise_to | -fall_to} to_list]
[-rise]
[-fall]
[-comment string]
value
```

Specifies a minimum delay for a timing path. The `set_min_delay` command takes precedence over other path exceptions specified for the same path, except for `set_false_path` command specifications.

Use either the `-from`, `-to`, or `-through` option with the `set_min_delay` command to identify the paths. Specifying both the `-from` and `-to` options identifies a path with specific starting and ending points. If a clock is specified as the `-to` object, all primary outputs and registers clocked by that clock are used as an endpoint.

When you specify a sequential cell name for starting point of the timing path, the valid path starts from the clk pin. This means the latency and delay from clk to output Q is also included in the delay. For path segmentation, you can specify the path to start from Q output of the register by adding /Q after the cell name. In this case, path from clk to Q is not considered in the calculation. For example, in the following set of commands, the valid path starts from different points:

```
set_min_delay 0.66 -from TR1/BR1 -to BLK/BR1/D
```

```
set_min_delay 0.66 -from TR1/BR1/Q -to BLK/BR1/D
```

For the first command, the valid path starts from clk, and for the second command the valid path starts from the Q output. In the second case, the path from clk to Q is not considered. For an example of the results generated for paths with and without path segmentation, see Examples.

Use only one `-from`, `-rise_from`, or `-fall_from`, `-to`, `-rise_to`, or `-fall_to` parameter per command. You can use multiple `-through` parameters in the same command.

Parameters

`-help`	Prints out the command usage.

-combinational_from_to	Defines a path defined by -from/to pins as combinational. When this parameter is used, the path between the -from/-to pins is treated as purely combinational, that is, both the source/capture clock latencies are excluded from slack computation. The presence of -from/-to pins is required for this option to take effect; the -from/-to pins can be sequential or combinational start/end pins. The -combinational_from_to parameter will be deprecated in a future release. This parameter will continue to work in the current release, but for compatibility with future releases you should migrate your scripts to use the -ignore_clock_latency parameter. You can use the following attributes to match the semantics between the -ignore_clock_latency and -combinational_from_to parameters: • timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency • timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency
-comment *string*	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
-fall	Applies the assertion at the falling edge of the signal on the path endpoints.

`-from |-rise_from	-fall_from` *`from_list`*	Specifies a list of timing paths start points. The *`from_list`* argument can contain either object IDs or hierarchical names relative to the current module or clock names. The *`from_list`* argument can be a collection. The valid start points are: • Input port • Bidirectional port • Clock pin of sequential cell Clock pin of sequential cell is one of the following: • Structural clock pin that is part of latch or flip-flop. • Inferred clock pin in cells like macro cells. A pin is an inferred clock pin if it is associated with one of the following: • Reference end of setup check • Reference end of hold check • Reference end of removal check • Reference end of recovery check • Minimum period check • Minimum pulse width check • Pin associated with set_input_delay Specify one of the following:
	`-from`: Applies the minimum delay constraint to all paths originating from the *`from_list`* list. If a clock is specified as the `-from` object, all primary inputs and registers clocked by that clock are used as start points. By default, the `-from` option applies the constraints to both the rising and the falling edges.
	`-rise_from`: Applies the minimum delay constraint at the rising edge of the signal on the from pins.
	`-fall_from`: Applies the minimum delay constraint at the falling edge of the signal on the from pins.
`-no_segment`	Specifies the constraint as non-segmenting. When specified, the software allows path delay checks without path segmentation.

`-ignore_clock_latency`	When this parameter is specified, the software does not count the launch/capture clock latency delays into the timing path delay. The `-ignore_clock_latency` parameter is an industry-standard construct introduced in SDC 2.1 to control the handling of clock-related timing with respect to the `set_max_delay`/`set_min_delay` constraints.
`-rise`	Applies the assertion at the rising edge of the signal on the path endpoints.
`-through | -rise_through | -fall_through` *`through_list`*	Specifies a list of timing paths through points. The *`through_list`* argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The *`through_list`* argument can be a collection. Specify one of the following:
	`-through`: When you use this parameter without using the `-from` or `-to` options, all paths that go through the specified pins have the specified delay. When you use it with both the `-from` and `-to` options, any path starting at any pin on the `-from` list *and* going through any point on the `-through` pin list *and* going to any point on the `-to` list is affected by the constraint. By default, the `-through` option applies the constraint to both the rising and the falling edges.
	`-rise_through`: Applies the assertion at the rising edge of the signal on the through pins.
	`-fall_through`: Applies the assertion at the falling edge of the signal on the through pins.

`-to | -rise_to | -fall_to` *to_list*	Specifies a list of timing paths end points. The *to_list* argument can contain either object IDs or hierarchical names relative to the current module or clock names. The pins can be on intermediate hierarchical boundaries. The *pin_or_clock_list* argument can be a collection. The valid end points are: • Output port • Bidirectional port • Data pin of sequential cell Data pin of sequential cell is one of the following: • Structural data pin that is part of latch or flip-flop. • Inferred data pin in cells like macro cells. A pin is an inferred data pin if it is associated with one of the following: • Signal end of setup check • Signal end of hold check • Signal end of non-sequential setup check • Signal end of non-sequential hold check • Signal end of removal check • Signal end of recovery check • Pin associated with `set_external_delay` • Data pin of clock gating cell Select from one of the following:
	`-to`: Applies the minimum delay constraint to all paths ending in the `-to` pin list. If a clock is specified as the `-to` object, all primary outputs and registers clocked by that clock are used as endpoint. By default, the `-to` option applies the constraint to both the rising and the falling edges.
	`-rise_to`: Applies the minimum delay constraint at the rising edge of the signal on the to pins.
	`-fall_to`: Applies the minimum delay constraint at the falling edge of the signal on the to pins.
value	Specifies the minimum delay value.

Examples

- The following command specifies a minimum delay of `2.5` for paths from `in1` to `out1`:

```
tempus > set_min_delay 2.5 -from in1 -to out1
```

- The following commands show the timing report output without path segmentation:

```
tempus > set_min_delay 3 -from f0 -to f1/D
tempus > report_timing -early -from f0 -to f1/D

Path 1: VIOLATED (-2.927 ns) Path Delay Check with Pin f1/D
               View: view1
              Group: clk2
         Startpoint: (R) f0/CK
              Clock: (R) clk1
           Endpoint: (F) f1/D
              Clock: (R) clk2

                       Capture       Launch
        Path Delay:+    3.000            -
       Src Latency:+    0.000        0.000
       Net Latency:+    0.000 (I)    0.000 (I)
           Arrival:=    3.000        0.000

              Hold:+    0.043
     Required Time:=    3.043
      Launch Clock:=    0.000
         Data Path:+    0.116
             Slack:=   -2.927

#-------------------------------------------------------------------------------
# Timing Point  Flags  Arc     Edge  Cell         Fanout  Trans   Delay  Arrival
#                                                        (ns)    (ns)     (ns)
#-------------------------------------------------------------------------------
  f0/CK         <<<    CK      R     (arrival)         1  0.000       -    0.000
  f0/QN         -      CK->QN  F     LDN_FDPQB_1       1  0.000   0.074    0.074
  b0/X          -      A->X    F     LDN_BUF_1         1  0.024   0.042    0.116
  f1/D          <<<    D       F     LDN_FDPQB_1       1  0.017   0.000    0.116
#-------------------------------------------------------------------------------
```

- The following commands show the timing report output with path segmentation:

```
tempus > set_min_delay 3 -from f0/QN -to f1/D
tempus > report_timing -early -from f0/QN -to f1/D

Path 1: VIOLATED (-3.001 ns) Path Delay Check with Pin f1/D
               View: view1
              Group: clk2
         Startpoint: (F) f0/QN
              Clock:
           Endpoint: (F) f1/D
              Clock: (R) clk2

                       Capture       Launch
        Path Delay:+    3.000            -
```

```
          Src Latency:+    0.000        0.000
          Net Latency:+    0.000 (I)    0.000 (I)
              Arrival:=    3.000        0.000

                 Hold:+    0.043
       Required Time:=     3.043
        Launch Clock:=     0.000
           Data Path:+     0.042
               Slack:=    -3.001

#-------------------------------------------------------------------------
#Timing Point  Flags  Arc   Edge  Cell         Fanout  Trans   Delay   Arrival
#                                                 (ns)    (ns)     (ns)
#-------------------------------------------------------------------------
 f0/QN         <<<    QN    F     (arrival)         1  0.024   0.000    0.000
 b0/X          -      A->X  F     LDN_BUF_1         1  0.024   0.042    0.042
 f1/D          <<<    D     F     LDN_FDPQB_1       1  0.017   0.000    0.042
#-------------------------------------------------------------------------
```

Related Topics

- Path Exception Priorities

Related Commands

- set_input_delay
- set_output_delay
- set_false_path
- set_max_delay
- set_multicycle_path
- reset_path_exception
- report_path_exceptions

set_min_fanout

```
set_min_fanout
[-help]
limit
obj_list
[-override]
```

Sets the minimum fanout load limit constraint on the specified input ports of the top cell.

When set on a port, the fanout load of the net connected to the port must be greater than or equal to the specified limit value to meet the constraint. The fanout load of a net is the sum of the fanout loads of the pins driven by the net. When set on a module, the limit applies to all the nets of that module. The libraries and library cell pins may also have the minimum fanout load limit. In this case the most constraining limit is applied to the nets.

The constraint will not be used in optimizations.

Fanout load limit can be asserted on input ports, output pins, bidirectional pins, or ports and library pins. The constraint cannot be applied on input pins and output ports.

Parameters

`-help`	Prints out the command usage.
`limit`	Specifies the value of the minimum fanout load limit constraint. The specified value must be greater than or equal to zero.
`obj_list`	Specifies a list of objects - ports, instance pins, modules, or library cell pins.
`-override`	Overrides the minimum fanout limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting.

Examples

- The following example sets a minimum fanout load limit of 0.001 units on all the ports of the top cell:

  ```
  tempus > set_min_fanout 0.001 [get_ports *]
  ```

- The following example shows a report of the minimum fanout load limit set on ports using the report_ports command, as shown in the following example:

  ```
  tempus > set_min_fanout 5.0 [get_ports *]
  tempus > report_ports -type fanout_load_limit -pin [get_ports *]
  ```

  The following report is displayed:

  ```
  ----------------------------------------
  Pin      Dir  Assertion           Value
  Name
  ```

```
-----------------------------------------
CLK2      IN   fanout_load_limit   5.000
CLK1      IN   fanout_load_limit   5.000
in2       IN   fanout_load_limit   5.000
in1       IN   fanout_load_limit   5.000
-----------------------------------------
```

Related Commands

- get_ports
- report_ports
- reset_min_fanout
- set_fanout_load
- set_max_capacitance
- set_max_transition
- report_constraint

set_min_pulse_width

```
set_min_pulse_width
[-help]
[-low]
[-high]
[-waveform_aware_type {absolute | source_width_ratio}]
value
object_list
```

Selects the most restrictive value when multiple minimum pulse width checks are set.

Note: The `timing_disable_lib_pulse_width_checks` attribute has no effect on pulse width checks created by the `set_min_pulse_width` command.

Parameters

`-help`	Prints out the command usage.
`-high`	Applies the minimum pulse width check to high signal levels only. If neither the `-low` or `-high` option is specified, the value is applied to both low and high signal levels.
list	Specifies a list of pins or clock waveforms to which the minimum pulse width check is to be applied. If the list is not specified, the minimum pulse width check applies to all the pins in the clock tree of the current design. If a clock waveform is specified, all the pins driven by the clock are affected.
`-low`	Applies the minimum pulse width check to low signal levels only. If neither the `-low` or `-high` option is specified, the value is applied to both low and high signal levels.
object_list	Specifies list of objects: pins, ports, or clocks.
value	Specifies a non-negative floating minimum pulse width check value. No default value is assumed for minimum pulse width checks.
`-waveform_aware_type {absolute | source_width_ratio}`	Specifies the type of waveform-aware pulse width check to apply as `absolute` or `source_width_ratio`.

Examples

- The following command sets a minimum pulse width, both low and high, check of `1.0` to the clock network of the current design:

  ```
  tempus > set_min_pulse_width 1.0
  ```

- The following command sets a minimum pulse width low check of `2.0` to all the pins clocked by the `CLK1` clock:

  ```
  tempus > set_min_pulse_width -low 2.0 [get_clocks CLK1]
  ```

- The following command sets a minimum pulse width `-high` check of `1.0` to the `CP` clock pin of register `ff1`, as shown in the following example:

```
tempus > set_min_pulse_width 0.5 u55/CK -low
tempus > set_min_pulse_width 1.0 u55/CK -high
tempus > report_min_pulse_width -pin u55/CK

-------------------------------------------------------
SEQUENTIAL CLOCK PULSE WIDTH
-------------------------------------------------------
Pin                 Required     Actual Pulse    Slack
                    Pulse Width  Width
-------------------------------------------------------
u55/CK (low)        0.500        4.200           3.700
u55/CK (high)       1.000        5.800           4.800
-------------------------------------------------------

tempus > report_timing -check_type pulse_width -to u55/CK

Path 1: MET User PulseWidth Check with Pin u55/CK
Ending Clock Edge: u55/CK (^) checked with leading edge of 'CLK4'
Beginning Clock Edge: u55/CK (v) triggered by trailing edge of 'CLK4'
Path Groups: {CLK4}
Other End Arrival Time 0.152
- PulseWidth 0.500
+ Phase Shift 10.000
= Required Time 9.651
- Arrival Time 5.951
= Slack Time 3.700
Clock Fall Edge 6.000
+ Clock Network Latency (Prop) -0.049
= Beginpoint Arrival Time 5.951
-------------------------------------------------
Load   Slew    Delay  Arrival  Cell  Arc  Pin
                      Time
-------------------------------------------------
0.015  0.019   5.951  DFF      CK    v    u55/CK ->
-------------------------------------------------
```

Related Commands

- report_analysis_coverage
- report_min_pulse_width
- report_timing
- reset_min_pulse_width

set_min_transition

```
set_min_transition
[-help]
[-clock_path]
[-data_path]
[-rise]
[-fall]
[-override]
limit
obj_list
```

Sets the minimum slew time limit (transition) on the specified ports of the top timing module and/or clock waveforms.

When set on a port, the slew time at the port must be greater than or equal to the specified limit. When set on a module, the limit applies to all the pins in the module and its ports if the module is the top timing module. The minimum transition limits on pins can come from library objects, which are the default minimum transition of the libraries or minimum transition on library cell pins. In this case, the most constraining limit, which is the largest, will apply on pins and ports.

Use the `report_ports -type slew_limit` command to view the minimum transition limits set on ports of the top timing module.

When limits are set on a clock waveform, you can use `-clock_path`, `-data_path`, `-rise`, and `-fall` parameters to provide more specifications.

The `set_min_transition` constraint on clock objects will be honored only by the `report_constraint` command for DRV violations. The constraint will not be used in optimizations.

Note: This command syntax is applicable only when the flow-compatible mode is on.

Parameters

`-help`	Prints out the command usage.
`-clock_path`	Specifies all pins that are in the network of the specific clocks in the object list.
`-data_path`	Specifies all pins that are in the data paths launched by the specific clocks in the object list.
`-fall`	Specifies falling capacitance.
`obj_list`	Specifies the list of objects - ports, instance pins, library cell pins, modules, and/or clock waveforms.
`-override`	Overrides the minimum transition limit from other sources, such as, library or constraints. This parameter can be used to force a specific constraint value for a specified list of instance pins/port/library cell pins - this overrides the maximum/minimum values set prior to this setting.
`-rise`	Specifies rising capacitance for all selected pins.
`limit`	Specifies the value of the minimum slew time limit (minimum transition). The minimum slew time limit is in time units from the target technology library. Transition limit value (float, required) Min=-1000000 Max=1000000

Examples

- The following example sets a minimum transition limit of 0.9 units on all the ports of the top timing module, and a minimum transition limit of 0.7 units on the top timing module:

```
tempus > set_min_transition 0.9 [get_ports *]
tempus > set_min_transition 0.7 [current_design]
```

  The minimum limit that applies to ports is 0.9, which is the largest constraints value.

- The following example shows a report of the minimum transition limit set on ports using the `report_ports` command, as shown in the example below:

```
tempus > set_min_transition 0.9 [get_ports *]
tempus > report_ports -type slew_limit -pin [get_ports *]
```

  The following report is displayed:

```
-----------------------------------------
Pin    Dir   Assertion    Value
Name
-----------------------------------------
CLK2   IN    slew_limit   ( 0.900 : )
```

```
CLK1    IN     slew_limit   ( 0.900 : )
in2     IN     slew_limit   ( 0.900 : )
in1     IN     slew_limit   ( 0.900 : )
out2    OUT    slew_limit   ( 0.900 : )
out1    OUT    slew_limit   ( 0.900 : )
-----------------------------------------

tempus > report_constraint -drv_violation_type min_transition

Check type : min_transition
---------------------------
Pin : CLK4

Transition Time: 0.003
- min_transition: 0.900
-----------------------------------------------------
slack : -0.897 ( VIOLATED )
```

Related Commands

- set_max_capacitance
- reset_min_transition
- report_ports
- report_constraint

set_mode

```
set_mode
[-help]
[-type cell]
mode_list
inst_list
```

Specifies the Liberty timing library modes to be active for a specific instance.

A library mode is a subset of active timing arcs that is defined in the timing library. Library modes are similar to conditional timing arcs, but do not require the application of constants to set. Instead of changing a cell's mode by applying constants to its input pins, you can use the `set_mode` command to refer to sets of active arcs by name (such as `READ` or `WRITE`, for a memory cell).

By default, all library modes in the timing library are active. If you specify one or more library modes using the `set_mode` command, only the timing arcs consistent with the specified mode(s) are active for the instance. All other library modes become inactive.

To reset the library modes for an instance, use the `reset_mode` command.

Parameters

`-help`	Prints out the command usage.
`inst_list`	Specifies the instances to which to apply the library mode.
`mode_list`	Specifies the name of the library mode value defined in the timing library. You can specify one or more mode values, but only one mode value per defined mode group in the Liberty library.
`-type cell`	Sets the library mode at the instance level. **Note:** Setting the library mode at the instance level is the only form of `set_mode` that is currently supported. This parameter exists to support constraints coming from external sources.

Examples

- The following command specifies that only the timing arcs consistent with the test_A library mode are active for instances of b1 (The test_A library mode is made inactive.):

  ```
  tempus > set_mode test_mode [get_cells b1]
  ```

  The following command reports the status of all library modes associated with b1 . It shows that test_A is active and test_B is inactive.

  ```
  tempus > report_mode b1
  --------------------------------
  Mode Group test_mode of b1
  --------------------------------
  Mode     Status     Condition
  Name
  --------------------------------
  test_B   Inactive   !(Y)
  test_A   ACTIVE      Y
  --------------------------------
  ```

  If you specify the `report_inactive_arcs` command, the report shows the disabled arcs for the b1 instance are from the `test_mode` library mode:

  ```
  ------------------------------------------------------
  Flags :  const propagated constant
           snipped loop snipped arcs
           disable set_disable_timing
           disable_clock_gating set_disable_clock_gating
           library_disable set_disable_cell_timing
           Missing_Phase No Arc Phase Data
  From     To       DisableType   ArcType                 Reason
  ------------------------------------------------------------------
  u8/CK ^   u8/D ^   const        hold_rising_clk_rise    u8/D = 1
  u8/CK ^   u8/D v   const        hold_falling_clk_rise   u8/D = 1
  u8/CK ^   u8/D ^   const        setup_rising_clk_rise   u8/D = 1
  u8/CK ^   u8/D v   const        setup_falling_clk_rise  u8/D = 1
  ------------------------------------------------------------------
  ```

Related Commands

- reset_mode
- report_mode
- report_inactive_arcs
- get_cells

set_multicycle_path

```
set_multicycle_path
[-help]
<number_of_cycles>
[-comment <string>]
[-fall]
[-fall_through <through_list>]
[-hold]
[-rise]
[-rise_through <through_list>]
[-setup]
[-through <through_list>]
[-start  | -end ]
[-from <from_list> | -rise_from <from_list> | -fall_from <from_list>]
[-to <to_list]> | -rise_to <to_list> | -fall_to <to_list]>]
```

Specifies multicycle paths between specific timing paths in a design or between clock domains. The multicycle paths are timing paths that require more than one clock period for execution. By default, all the paths are considered as one cycle paths for setup and hold check evaluation. The `set_multicycle_path` command allows you specify additional cycles for the paths.

Parameters

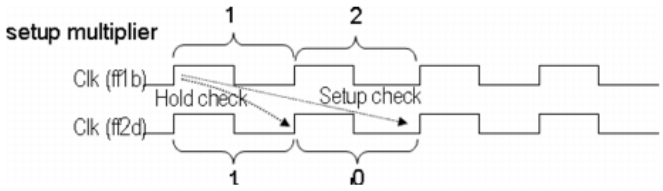
`-help`	Prints out the command usage.
number_of_cycles	Specifies the number of cycles allowed for the path.
`-comment` *string*	Allows you to insert comments. The comments are read but are ignored with a one-time warning message. They are not used during timing analysis reporting nor are they written out during saving of constraints.
`-fall`	Applies the assertion at the falling edge of the signal on the path endpoints.
`[-fall_through <through_list>]`	Applies the assertion at the falling edge of the signal on the through pins

`-from <from_list> | -rise_from <from_list> | -fall_from <from_list>`	Specifies the clocks, ports, pins, or sequential instances that are at the start of the multicycle paths. The port direction can be either input or inout. Pins must be valid path startpoints such as sequential instance clock pins, latch data pins or pins on which input delays have been specified. Pins can be on intermediate hierarchical boundaries. The *from_list* variable can contain either collection IDs or object names. If the `-from` option is used without the `-to` option, all paths originating from the `from_list` are multicycle paths. Use either the `-from`, `-to`, or `-through` option with the `set_multicycle_path` command to identify the paths. Specifying both the `-from` and `-to` options identifies a path with specific starting and ending points. Use only one `-from`, `-rise_from`, or `-fall_from` option per command.
`-hold | -setup`	Specifies whether the multicycle adjustment applies to setup/hold check. The `-setup` parameter applies multicycles to setup check. The hold check is affected but the hold multiplier does not change. The `-hold` parameter applies multicycles to hold check only. *Default:* If you do not specify `-setup` or `-hold` parameter, the software considers the specified multicycle value for setup check and uses 0 for the hold check.
`-rise`	Applies the assertion at the rising edge of the signal on the path endpoints.
`-rise_through <through_list>`	Applies the assertion at the rising edge of the signal on the through pins.
`-start | -end`	Specifies whether the multicycle adjustment is to be applied relative to the starting clock edge for the path or relative to the ending clock edge for the path. Use these options for multiple clocks designs. By default, the setup check is relative to the end clock and the hold check is relative to the start clock. **Note:** The `-start` and `-end` parameters are mutually exclusive; you cannot specify them together. You can set the `timing_enable_sdc_compatible_data_check_mcp` global variable to `true` to enable checking for SDC compatible data checks. When the `-start` parameter is specified, the parameter reverses the reference clock by one clock cycle.

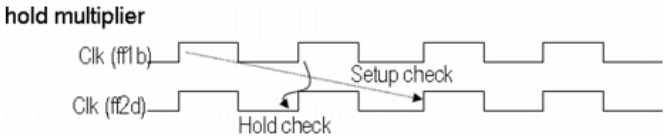
`-through <through_list>`	Applies the assertion to both the rising and the falling edges. This parameter specifies the ports, pins, instances, or nets that the path goes through. The pins can be on intermediate hierarchical boundaries. The *through_list* variable can contain either collection IDs or object names. When used without the `-from` or `-to` option, all paths that go through the `-through` pins are multicycle paths. When used with both the `-from` and `-to` options, any path starting at any pin on the `-from` list and going through any point on the `-through` pin list and going to any point on the `-to` list is affected by the assertion. You can use only one `-from` and `-to` option, but you can specify several `-through` options in the same command. By default, the `-through` option applies the assertion to both the rising and the falling edges. Using the `-rise_through` option applies the assertion at the rising edge of the signal on the through pins. Using the `-fall_through` option applies the assertion at the falling edge of the signal on the through pins.
`-to <to_list]> | -rise_to <to_list> | -fall_to <to_list]>`	Specifies the clocks, ports, pins, or sequential instances that are at the end of the multicycle paths. The port direction can be either output or inout. Pins must be valid path end-points such as sequential instance data pins or pins on which output delays have been specified. Pins can be on intermediate hierarchical boundaries. The *to_list* variable can contain either collection IDs or object names. If the `-to` option is used without the `-from` option, all paths ending in the `-to` pin list are multicycle paths. Use only one `-to` option per command. Use only one `-to`, `-rise_to`, or `-fall_to` option per command.

Examples

- The following command sets all paths between ff1b + ff2d to two cycles for setup:
  ```
  set_multicycle_path -setup 2 -from {ff1b} -to {ff2d}
  ```

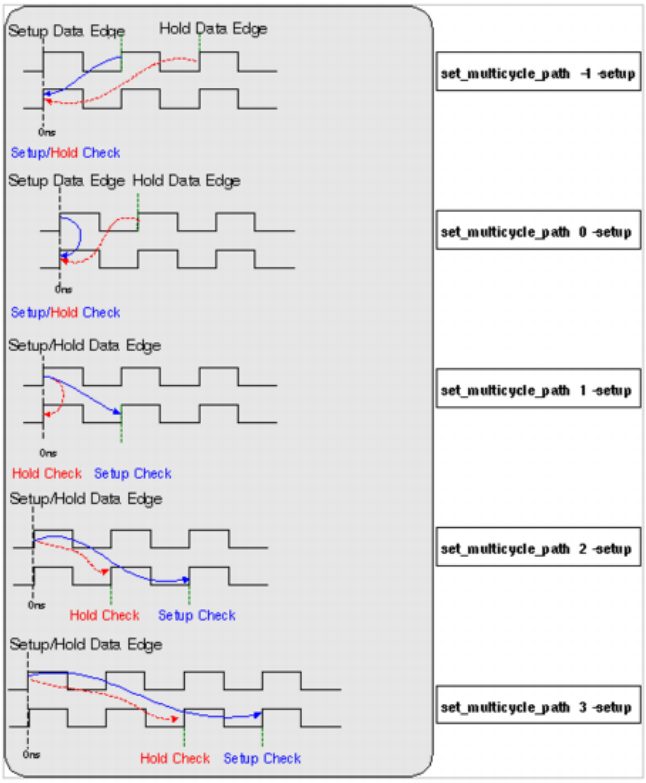


- The following command moves hold check to start clock preceding edge:
  ```
  set_multicycle_path -hold 1 -from {ff1b} -to {ff2d}
  ```



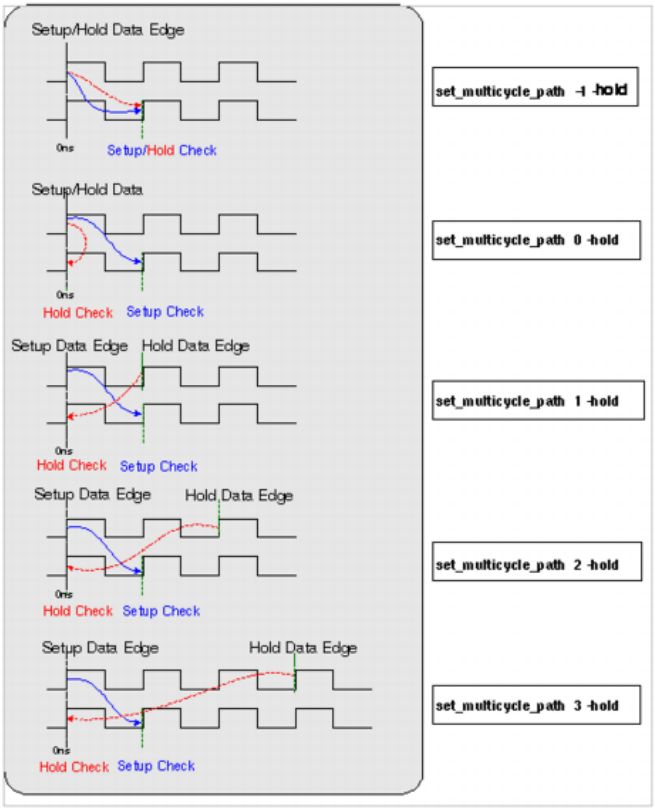
- The following commands show setup multiplier values from -1 to 3:

```
set_multicycle_path -1 -setup
set_multicycle_path 0 -setup
set_multicycle_path 1 -setup
set_multicycle_path 2 -setup
set_multicycle_path 3 -setup
```



- The following commands show hold multiplier values from -1 to 3:

```
set_multicycle_path -1 -hold
set_multicycle_path 0 -hold
set_multicycle_path 1 -hold
set_multicycle_path 2 -hold
set_multicycle_path 3 -hold
```



Related Topics

- Path Exception Priorities

Related Commands

- `set_false_path`
- `set_max_delay`
- `set_min_delay`
- `reset_path_exception`
- `report_path_exceptions`

set_normalized_driver_waveform_lib

```
set_normalized_driver_waveform_lib
[-help]
[-fall_waveform string]
[-rise_waveform string]
-ref_lib string
-libs string
```

Specifies normalized driver waveform for a library. The normalized driver waveform given in the reference library is applied to all the specified libraries.

Note: This command must be specified after the design is loaded. For this command to take effect, you should set the `timing_generate_normalized_driver_waveform` attribute to `false` before loading libraries.

Parameters

`-help`	Prints out the command usage.
`-fall_waveform` *string*	Specifies the name of the fall normalized driver waveform.
`-libs` *string*	Specifies a list of space separated libraries to which normalized driver waveform is to be applied.
`-ref_lib` *string*	Specifies the reference library from which normalized driver waveform is to be selected.
`-rise_waveform` *string*	Specifies the name of the rise normalized driver waveform.

Examples

- The following example shows that normalized driver waveform given in $libDir/libs/ref.lib will be applied to all the cells in `$libDir/1.lib $testDir/2.lib $libDir/libs/3.lib`:

  ```
  tempus > set_normalized_driver_waveform_lib -libs "$libDir/1.lib $testDir/2.lib $libDir/libs/3.lib"
  -ref_lib $libDir/libs/ref.lib
  ```

set_output_delay

```
set_output_delay
[-help]
-clock clock_name
[-clock_fall]
[-rise]
[-fall]
[-max]
[-min]
[-add_delay]
[-network_latency_included]
[-source_latency_included]
[-reference_pin pin_name]
[-level_sensitive]
[-group_path group_name]
value
pin_or_port_list
```

Specifies the data required time on output ports, hierarchical ports, or internal output pins.

Parameters

`-help`	Prints out the command usage.
`-add_delay`	Adds delay information to the existing output delay. The additional delay information can be for multiple paths relative to different clocks or clock edges leading to an input port or internal input pins. If you do not use the `-add_delay` option, the software uses the last `set_output_delay` related to a pin when there are multiple `set_output_delay` statements for the same pin.
`-clock` *clock_name*	Specifies that the delay is relative to the specified clock. You cannot use the `set_output_delay` command without the `-clock` parameter. Such constraints are accepted but are ignored during timing analysis. When you specify multiple clocks, an error occurs and the constraint is ignored.
`-clock_fall`	Specifies that the delays are with respect to the falling edge of the clock waveform. If you do not specify the `-clock_fall` option, the delay is with respect to the rising edge of the clock waveform. You cannot specify the `-clock_fall` parameter without the `-clock` parameter.
value	Specifies the value for the path delay.

`-fall`	Specifies that the output delay refers to the falling edge of the signal at specified ports or pins. If you do not specify the `-fall` option, the output delay applies to both the edges.
`-group_path` *group_name*	Specifies the group name where paths ending at the specified ports or pins are to be added. The `-group_path` parameter is not honored in multi-mode multi-corner (MMMC) designs. If specified, a warning is issued and the constraints are applied as if the `-group_path` parameter is not specified.
`-level_sensitive`	Models output delay that is captured by a positive level sensitive latch. For setup checks, the timing system checks the arrival time at the output port with the opening edge of the latch. The timing system treats any arrival time exceeding the opening edge as negative slack. For hold checks, the timing system performs the checks with respect to the closing edge of the latch. Use the `-clock` parameter to model a positive level sensitive latch, or the `-clock_fall` parameter to model a negative level sensitive latch. You cannot use the `-level_sensitive` parameter without the `-clock` parameter.
`-max`	Specifies that the delay refers to the setup analysis. If you do not specify the `-max` parameter, the output delay applies to both setup and hold analysis.
`-min`	Specifies that the delay refers to the hold analysis. If you do not specify the `-min` parameter, the output delay applies to both setup and hold analysis.
`-network_latency_included`	Specifies that the clock network latency should not be added to the output delay value. If you do not specify this option, the clock network latency of the related clock is added to the output delay value. This parameter has no effect if the specified clock is in propagated mode.
pin_or_port_list	Specifies a list of output port or pin names. The pin_or_port_list argument can be a collection.
`-reference_pin` *pin_name*	Adds source and network latency of the specified reference pin to the output delay value in propagated mode. A reference pin is a leaf pin in the fanout cone of the clock source of the clock that you specify using the `-clock` parameter. You cannot specify multiple pins for `-reference_pin` parameter. The command issues a warning and accepts the first pin. You cannot use the `-reference_pin` parameter with the `-network_latency_included` or `-source_latency_included` parameters. You cannot use the `-reference_pin` parameter to specify hierarchical pins.
`-rise`	Specifies that output delay refers to the rising edge of the signal at the specified ports or pins. If you do not specify the `-rise` option, the output delay applies to the both edges.

`-source_latency_included`	Specifies that the clock source latency should not be added to the output delay value. With different delay values for `-rise/-fall` and `-min/-max`, only the last specified combination of `-network_latency_included` and `-source_latency_included` parameters is honored. The specified combination can be one of the following: a) Either `-network_latency_included` or `-source_latency_included` b) Both the parameters c) None of these

Examples

- The following example sets the input and output delays for the bidirectional INOUT2 port.The input signal arrives at INOUT2 at 4.5 units after the falling edge of CLKB. The output signal is required at INOUT2 at 2.5 units before the rising edge of CLKD:

```
tempus > set_input_delay 4.5 -clock CLKB -clock_fall { INOUT2 }
tempus > set_output_delay 2.5 -clock CLKD { INOUT2 }
```

- The following example shows how to use the -group_path option to add ports into a named group:

```
tempus > set_output_delay 2.5 -max -clock CLK -group_path busC {busC[*]}
```

Related Commands

- `all_outputs`
- `create_clock`
- `report_ports`
- `reset_output_delay`
- `set_input_delay`
- `set_load`
- `report_path_exceptions`

set_pll_timing

```
set_pll_timing
[-help]
inst
-feedback_pin <feedback pin name>
-output_clock <clock name or ID>
-output_pin <output pin name>
[-reference_edge {rise | fall | both}]
-reference_pin <reference pin name>
```

This command identifies phase locked loop (PLL) insertion delays and the associated generated clock(s) that require adjustment. To remove the effect of clock tree latency, PLLs can be used to adjust the delay on the PLL output clock waveform so that the effective edges of the clock at the clock tree endpoints are aligned with the reference edges of the input clock to the PLL.

When you use the `set_pll_timing` command, the software uses the settings to automatically calculate and apply a delay offset to a PLL output clock to compensate for internal clock tree latency.

The `set_pll_timing` command works in conjunction with a generated clock definition to provide the correct clock timing at the output of the PLL. To use this command, you must first assert a generated clock on the `set_pll_timing -output_pin`. The `-source` option of the `create_generated_clock` command must also match the `-reference_pin` specified by the `set_pll_timing` command.

Parameters

`-help`	Prints out the command usage.
inst	Specifies the instance name or the object ID of a PLL cell, on the basis of which the reference pins, feedback pins, or the output pins will be searched.
`-feedback_pin <`*feedback pin name*`>`	Specifies the name of the PLL feedback pin.
`-output_clock <clock name or ID>`	Specifies the generated clock on the PLL output.
`-output_pin <output pin name>`	Specifies the PLL clock output pin name.
`-reference_edge {rise | fall | both}`	Defines adjustment edge(s). When this parameter is specified, the software selects the edges of the clocks at the reference and feedback pins that need to be aligned. If the `rise` or `fall` option is specified, then the clock latency from that reference edge to the related edge at the feedback pin will be used. In this case, both the rise and fall edges will use the same latency adjustment. If both of these options are specified, then the software will measure separate latency adjustments for both the rise and fall edges. *Default*: `rise`
`-reference_pin <reference pin name>`	Specifies the PLL reference pin name.

Examples

Here are some examples.

- The following example shows the defined clocks in a sample design:

```
tempus > create_clock [get_ports {clk_in}] -name CLK -period 6 -waveform {0 3}
tempus > create_generated_clock -name GCLK1 -source [get_pins {mypll/CLK}] -multiply_by 1 -
master_clock [get_clocks {CLK}] -add [get_pins {mypll/GCLK}]
```

  The following `report_timing` command shows the insertion delay values at various points in the timing report:

```
tempus > report_timing -from_rise mypll/GCLK -to_rise mypll/CEN -point_to_point

Path 1:Endpoint: mypll/CEN (^) (unconstrained output)

Beginpoint: mypll/GCLK (^) triggered by leading edge of 'GCLK1'
Arrival Time 0.496
Analysis View: default_analysis_view_setup
------------------------------------------------------------
Pin           Cell      Delay   Arrival   Generated Clock
                                Time      Adjustment
```

```
-------------------------------------------------------------
mypll/CLK                          0.112
mypll/GCLK ->  PLL        0.133    0.246     GCLK1 Adj. = 0.000
clkTreeL0/Y    CLKBUFX1   0.139    0.385
clkTreeL1_1/Y  CLKBUFX1   0.118    0.503
mypll/CEN ->   PLL       -0.007    0.496
-------------------------------------------------------------
```

From the above table, it can be seen that the rise insertion delay of clock CLK at pin mypll/CLK is 0.112ns.

Now, the following command will adjust the source latency of clock GCLK1 such that the edges at mypll/CLK and mypll/CEN are aligned i.e. the clock's rising edge should reach mypll/CEN at 0.112ns only:

```
tempus > set_pll_timing [get_cells mypll] -reference_pin CLK -feedback_pin CEN -output_pin GCLK -
output_clock [get_clocks GCLK1]
```

- The following `report_timing` command shows the source latency of GCLK clock as -0.138ns, and the clock reaches mypll/CEN pin at 0.112ns (same as on mypll/CLK):

```
tempus > report_timing -from_rise mypll/GCLK -to_rise mypll/CEN -point_to_point

Path 1:Endpoint: mypll/CEN (^) (unconstrained output)
Beginpoint: mypll/GCLK (^) triggered by leading edge of 'GCLK1'
Arrival Time 0.112
Analysis View: default_analysis_view_setup
+ Source Insertion Delay 0.112
- PLL Feedback Delay 0.250
= Beginpoint Arrival Time -0.138
-------------------------------------------------------------------
Pin             Cell       Delay    Arrival    Generated Clock
                                    Time       Adjustment
-------------------------------------------------------------------
mypll/GCLK ->                       -0.138    GCLK1 Pll Adj. = 0.000
clkTreeL0/Y     CLKBUFX1    0.139    0.001
clkTreeL1_1/Y   CLKBUFX1    0.118    0.120
mypll/CEN ->    PLL        -0.007    0.112
-------------------------------------------------------------------
```

Related Commands

- create_generated_clock
- create_clock
- reset_pll_timing

set_propagated_clock

```
set_propagated_clock
[-help]
obj_list
```

Puts the `propagated_clock` assertion on the specified pin, port, or clock object. This will cause all clock endpoints in the fanout of the specified object to receive propagated clock timing unless there is a `set_clock_latency` with higher precedence. If there are multiple `set_clock_latency` and `set_propagated_clock` assertions between the clock waveform, clock root and clock endpoint, the closest assertion to the clock endpoint takes precedence.

There are two methods of clock propagation: depending on whether the clock is ideal or propagated.

- When a clock is ideal, the delay from a clock port to a register clock pin (network insertion delay) comes from design constraints using the `set_clock_latency` command.
- When a clock is propagated, the network insertion delay is computed from the actual gates and interconnects in the clock network.

To set the propagated_clock property on all clock waveform objects, use the following command:
`set_propagated_clock [all_clocks]`

When `set_propagated_clocks [all_clocks]` command is asserted, it will apply to all the currently defined clock waveform objects. This setting will not apply to clocks created later in the script. Whether all the clock endpoints receive propagated or ideal timing depends on the precedence of `set_clock_latency` in the fanout.

The `set_propagated_clock [all_clocks]` command puts the `propagated_clock` assertion on all clocks except virtual clocks. When you put the assertion on a real clock that is used to define I/O timing (with `set_input_delay -clock` or `set_output_delay -clock`), the software does not take the specified network latency into account. Instead, the software calculates the network latency, which changes the I/O timing. You can change this behavior by using the following attribute:

```
set_db timing_io_use_clock_network_latency always
```

You can use this command after creating the clock with the `create_clock` command.

Parameters

`-help`	Prints out the command usage.
`obj_list`	Specifies the objects on which you want to put the `propagated_clock` assertion. The *`pin_or_clock_list`* argument can be a collection. When a pin is specified, it affects the propagation mode for all the registers in the transitive fanout (TFO) of the pin.

Examples

- The following command puts the propagated clock assertion on the clock waveform CLK1:

  ```
  tempus > set_propagated_clock CLK1
  ```

- The following command puts the propagated clock assertion on all clock waveforms:

```
tempus > set_propagated_clock [all_clocks]
```

Related Commands

- all_clocks
- get_propagated_clock
- report_clocks
- reset_propagated_clock
- set_clock_latency
- set_clock_uncertainty

set_property

```
set_property
[-help]
[-obj_type {object_type}]
{obj_list}
property_name
property_value
[-quiet]
```

Sets user-defined properties on objects.

The `set_property` command needs a property to be defined using the `define_property` command.

Parameters

`-help`	Prints out the command usage.
obj_list	Specifies a list of objects on which the property is to be applied.
`-obj_type {`*object_type*`}`	Specifies the type of object on which the property is to be defined.
property_name	Specifies the name of the property.
property_value	Specifies the value of the property.
`-quiet`	Specifies to exit quietly without issuing an error.

Examples

- The following command applies a user-defined property `slk` to pins `C1/A`:

  ```
  tempus > set_property [get_pins C1/A] slk 2
  ```

- The following command applies aocv_weight property to library arcs:

  ```
  tempus > set_property [get_lib_arcs -of_objects slow/INVX1] aocv_weight 8
  ```

Related Commands

- `get_property`
- `report_property`
- `define_property`
- `reset_property`

set_pulse_clock_max_transition

```
set_pulse_clock_max_transition
[-help]
limit
obj_list
[-transitive_fanout]
[-rise]
[-fall]
```

Sets the maximum pulse clock transition constraint on the specified pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

`-help`	Prints out the command usage.
`-fall`	Sets transition limit value for falling transition.
`obj_list`	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design.
`-rise`	Sets transition limit value for rising transition.
`limit`	Specifies the transition limit value.
`-transitive_fanout`	Specifies the transition limit value for transitive fanout.

Examples

- The following example sets the max pulse clock transition constraint of 0.1ns on the input pins of all the instances of the lib cell PULSE_GEN_00:

  ```
  tempus > set_pulse_clock_max_transition 0.1 PULSE_GEN_00
  ```

- This constraint can be observed on the input pin 'A' of an instance 'pg_inst' of this lib cell using the following command:

  ```
  tempus > report_constraint pg_inst/A -drv_violation_type pulse_clock_max_transition

  Fragmented output:

  Check type : pulse_clock_max_transition_rise
  ----------------------------
  ---------------------------------------------
  Pin Name Required Actual Slack
  ---------------------------------------------
  pg_inst/A 0.100 0.030 0.070
  ```

```
Check type : pulse_clock_max_transition_fall
---------------------------
---------------------------------------------
Pin Name Required Actual Slack
---------------------------------------------
pg_inst/A 0.100 0.030 0.070
```

Related Commands

- get_clocks
- get_cells
- reset_pulse_clock_max_transition

set_pulse_clock_max_width

```
set_pulse_clock_max_width
[-help]
width
obj_list
[-transitive_fanout]
```

Sets the maximum pulse clock width constraint on the specified pulse generator cells, pulse generator library cells, clocks, and/or current design.

To report the maximum pulse clock width constraint, you can use the `report_constraint -check_type pulse_clock_max_width` command.

Parameters

`-help`	Prints out the command usage.
`obj_list`	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design.
`width`	Specifies the pulse width value.
`-transitive_fanout`	Specifies the pulse width value for transitive fanout.

Examples

- The following example sets the max pulse clock width constraint of 1ns on the pulse generator instance 'pg_inst':

  ```
  tempus > set_pulse_clock_max_width 1 pg_inst
  ```

  This constraint can be observed using the following command:

  ```
  tempus > report_constraint pg_inst/Y -check_type pulse_clock_max_width
  ```

  Fragmented output:

  ```
  ---------------------------------------------
  Pin Required Actual Pulse Slack Clock Type
  Pulse Width Width
  ---------------------------------------------
  pg_inst/Y (high) 1.000 0.366 0.634 PG
  ---------------------------------------------
  ```

Related Commands

- get_clocks
- get_cells
- reset_pulse_clock_max_transition
- reset_pulse_clock_max_width

set_pulse_clock_min_transition

```
set_pulse_clock_min_transition
[-help]
limit
obj_list
[-transitive_fanout]
[-rise]
[-fall]
```

Sets the minimum pulse clock transition constraint on the specified pulse generator cells, pulse generator library cells, clocks, and/or current design.

Parameters

`-help`	Prints out the command usage.
`-fall`	Sets transition limit value for falling transition.
obj_list	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design.
`-rise`	Sets transition limit value for rising transition.
limit	Specifies the transition limit value.
`-transitive_fanout`	Specifies the transition limit value for transitive fanout.

Examples

- The following example sets the min pulse clock transition constraint of 0.05ns on the input pins of all the instances of the lib cell PULSE_GEN_00:

  ```
  tempus > set_pulse_clock_min_transition 0.05 PULSE_GEN_00
  ```

  This constraint can be observed on the input pin 'A' of an instance 'pg_inst' of this lib cell using the following command:

  ```
  tempus > report_constraint pg_inst/A -drv_violation_type pulse_clock_min_transition
  ```

  Fragmented output:

  ```
  Check type : pulse_clock_min_transition_rise
  ----------------------------
  -------------------------------------------
  Pin Name Required Actual Slack
  -------------------------------------------
  pg_inst/A 0.050 0.030 -0.020

  Check type : pulse_clock_min_transition_fall
  ----------------------------
  -------------------------------------------
  Pin Name Required Actual Slack
  -------------------------------------------
  pg_inst/A 0.050 0.030 -0.020
  ```

Related Commands

- reset_pulse_clock_min_transition
- get_clocks
- get_cells

set_pulse_clock_min_width

```
set_pulse_clock_min_width
[-help]
pulse_width
object_list
[-transitive_fanout]
```

Sets the minimum pulse clock width constraint on the specified pulse generator cells, pulse generator library cells, clocks and/or current design.

When you set this constraint on a clock, the settings are applied to all the pulse generators that are driven by the clock. When this constraint is used for a design, the software applies the constraint to all the clock networks in the design.

The specified minimum pulse width is expected to be larger than the pulse width value. The width is defined when a clock object is created and its waveform is defined in the current design (using the `create_clock -waveform` parameter).

In case of conflicting minimum width constraints, the more restrictive constraint is used. When applied on a non-pulse generator object, or a non-clock target, this constraint has no effect.

Note: You must specify `set_interactive_constraint_modes` constraint prior to running the `set_pulse_clock_min_width` command.

Parameters

`-help`	Prints out the command usage.
`object_list`	Specifies a list of objects - pulse generator cells, pulse generator library cells, clocks, or current design.
`pulse_width`	Specifies the minimum pulse width value.
`-transitive_fanout`	Specifies the pulse width value for transitive fanout.

Examples

- The following example sets the min pulse clock width constraint of 0.3 ns on the pulse generator instance `pg_inst`:

  ```
  tempus > set_pulse_clock_min_width 0.3 pg_inst
  ```

  This constraint can be observed using the following command:
  ```
  tempus > report_constraint pg_inst/Y -check_type pulse_clock_min_width
  ```

  Fragmented output:

  ```
  ------------------------------------------
  Pin Required Actual Pulse Slack Clock Type

  Pulse Width Width
  ------------------------------------------
  pg_inst/Y (high)    0.300 0.366 0.066 PG
  ------------------------------------------
  ```

Related Commands

- `report_constraint`
- `reset_pulse_clock_min_width`
- `set_pulse_clock_max_width`
- `report_min_pulse_width`
- `set_interactive_constraint_modes`
- `timing_enable_uncertainty_for_pulse_width_checks`

set_resistance

```
set_resistance
-help
[-max]
[-min]
value
net_list
```

Sets the wire resistance value on the specified nets of the top cell, which is used to compute the net delay of the specified nets. The resistance that you specify using this command overrides the resistance from wire load models and SPEF.

Note: In the best-case worst-case (BC-WC) analysis mode, the minimum resistance is used to compute the early path delay and the late path delay for the minimum corner. The maximum resistance is used to compute the early path delay and the late path delay for the maximum corner.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `set_resistance` parameter. For a detailed description of the command and all of its parameters, use the man command: `man set_resistance`.
`-max`	Specifies that the asserted resistance is the maximum resistance. *Default*: If the `-max` (or `-min`) parameter is not specified, the resistance value is used as both minimum and maximum resistance.
`-min`	Specifies that the asserted resistance is the minimum resistance. *Default*: If the `-min` (or `-max`) parameter is not specified, the resistance value is used as both minimum and maximum resistance.
net_list	Specifies a list of nets on which the wire resistance is set.
resistance_value	Specifies the value for the wire resistance. This value (in units) is found in the target technology library.

Examples

- The following example set the maximum wire resistance of `2.0` units on the net `n1`:

  ```
  tempus > set_resistance -max 2.0 [get_nets n1]
  ```

- The following example set the minimum wire resistance of `1.2` units and a maximum wire resistance of `2.0` units on the net `n1`:

  ```
  tempus > set_resistance 1.2 -min [get_nets n1]
  tempus > set_resistance 2.0 -max [get_nets n1]
  ```

Related Commands

- reset_resistance
- report_net
- report_timing
- set_drive
- set_load

set_sense

```
set_sense
[-help]
pin_or_port_list
[-clocks clock_list]
{[-positive | -negative | -stop_propagation | -clock_leaf]
[-type type_list]}
```

Selects which phase of the clock to filter at the specified point.

This command accepts timing arcs collection as a valid argument.

Parameters

`-help`	Prints out the command usage.
`-clocks` *clock_list*	Specifies a list of clocks passing through a specified point to which the constraint applies. When this parameter is not specified, the filtering of clock phase will be applicable to all the clocks passing through that specified point. This parameter must be used with a specified pin name and one of the parameters : `-positive`, `-negative`, `-stop_propagation`, or `-clock_leaf`.
`-clock_leaf`	When specified, the software propagates clock phases to the specified pin.
`-negative`	Specifies that positive phase filtered clocks pass through pins in the *pin_list*. **Note**: The `-positive`, `-negative` and `-stop_propagation` parameters are mutually exclusive.
pin_or_port_list	Specifies the list or collection of cell output pins. The unateness specified by the `set_sense` command is applied to clocks propagating through the specified point.
`-positive`	Specifies that negative phase filtered clocks pass through pins in the *pin_list*. **Note**: The `-positive`, `-negative` and `-stop_propagation` parameters are mutually exclusive.
`-stop_propagation`	Stops the propagation of the specified clocks passing through the specified pins, or any phases that are specified by the `-type` parameter. **Note**: The `-positive`, `-negative` and `-stop_propagation` parameters are mutually exclusive.

`-type` *`type_list`*	Applies clock sense to object types specified in the *`type_list`*. The type list options are - `all`, `clock`, `data`, and `clock_source_data`. These are explained below: • `all`: Stops all phases from propagating. This is equivalent to specifying both the `clock` and `data` options. • `clock`: Stops clock phases launched by clock(s) specified in the *`clock_list`* from propagating. • `data`: Stops all data phases launched by clock(s) specified in *`clock_list`*, starting from the pins/ports specified in *`pin_or_port_list`*. The objects in the *`pin_or_port_list`* can be selected anywhere in the design - clock/data network. • `clock_source_data`: Stops only clock source data phases from propagating. **Notes**: • The `-type clock` option should be used with the `-stop_propagation` parameter. When the `-type` parameter is not specified, then `set_sense -stop_propagation` will stop only the clock phases. • The `-type data` option should be used with the `-stop_propagation` and `-clock` parameters. • The `-type data` option is not supported with both `-positive` and `-negative` parameters. • The `write_sdc` command will automatically add `-type` clock when `set_sense` is specified with the `-stop_propagation` parameter (without `-type`).

Examples

- The following command allows the positive phase clock CLK1 to pass through CLKBUF1/Z and filters out the negative phase clock CLK1:
  ```
  tempus > set_sense -positive -clocks CLK1 CLKBUF1/Z
  ```
- The following command stops clock phases starting from pin1:
  ```
  tempus > set_sense pin1 -type {clock}
  ```
- The following command stops data phases based only on CK1 and CK2 starting from pin1:
  ```
  tempus > set_sense pin1 -type {data} -clocks {CK1 CK2}
  ```
- The following command stops only the clock source data phases starting from pin1:
  ```
  tempus > set_sense pin1 -type {clock_source_data}
  ```
- The following command stops the clock and clock source data phases:
  ```
  tempus > set_sense pin1 -type {clock clock_source_data}
  ```
  or
  ```
  tempus > set_sense pin1 -stop_propagation
  ```

Related Commands

- reset_sense

write_annotated_transition

```
write_annotated_transition
[-help]
out_file
[-direct]
[-exclude_ports]
[-include_output_pins]
[-max]
[-min]
[-view string]
```

Writes the value of transitions on each pin to an output file. You can either set the transitions using the `set_annotated_transition` command or the software computes them internally.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_annotated_transition` parameter.
`-direct`	Refers the ports directly for which the annotated transition will be applied in the output SDC. This option removes the dependency of using the `get_ports` and `get_pins` commands.
`-include_output_pins`	Writes the slews for input and output pins, and the output slew for bi-directional pins. If this option is not specified, the software writes the slew for input pins and the input slew of bi-directional pins.
`-min/-max`	Specifies whether to write out minimum or maximum transition. *Default:* Both minimum and maximum transitions are written out.
`-exclude_ports`	Skips writing the transition values for the primary ports (input and output).
`out_file`	Specifies the name of the output file.
`-view` *`string`*	Writes the transitions for the specified analysis view only. You can specify this parameter only when the software is in multi-mode multi-corner timing analysis mode.

Examples

Here are some examples.

- The following command generates the output file `tran.rpt`:

  ```
  tempus > write_annotated_transition -file tran.rpt
  ```

  The following file is generated:

```
set_annotated_transition -rise -min 0.0278 [get_ports {sig_out4000}]
set_annotated_transition -fall -min 0.0275 [get_ports {sig_out4000}]
set_annotated_transition -rise -max 0.0278 [get_ports {sig_out4000}]
set_annotated_transition -fall -max 0.0275 [get_ports {sig_out4000}]
set_annotated_transition -rise -min 0.001 [get_pins {buf_in2000/A}]
set_annotated_transition -fall -min 0.001 [get_pins {buf_in2000/A}]
set_annotated_transition -rise -max 0.001 [get_pins {buf_in2000/A}]
set_annotated_transition -fall -max 0.001 [get_pins {buf_in2000/A}]
set_annotated_transition -rise -min 0.001 [get_pins {buf_in2001/A}]
set_annotated_transition -fall -min 0.001 [get_pins {buf_in2001/A}]
set_annotated_transition -rise -max 0.001 [get_pins {buf_in2001/A}]
set_annotated_transition -fall -max 0.001 [get_pins {buf_in2001/A}]
set_annotated_transition -rise -min 0.0453 [get_pins {buf_out5000/A}]
set_annotated_transition -fall -min 0.0418 [get_pins {buf_out5000/A}]
set_annotated_transition -rise -max 0.0453 [get_pins {buf_out5000/A}]
set_annotated_transition -fall -max 0.0418 [get_pins {buf_out5000/A}]
```

- The following command generates the minimum transitions in the output file, `min_tran.rpt`:

  ```
  tempus > write_annotated_transition -min -file min_tran.rpt
  ```

  The following file is generated:

  ```
  set_annotated_transition -rise -min 0.0278 [get_ports {sig_out4000}]
  set_annotated_transition -fall -min 0.0275 [get_ports {sig_out4000}]
  set_annotated_transition -rise -min 0.001 [get_pins {buf_in2000/A}]
  set_annotated_transition -fall -min 0.001 [get_pins {buf_in2000/A}]
  set_annotated_transition -rise -min 0.001 [get_pins {buf_in2001/A}]
  set_annotated_transition -fall -min 0.001 [get_pins {buf_in2001/A}]
  set_annotated_transition -rise -min 0.0453 [get_pins {buf_out5000/A}]
  set_annotated_transition -fall -min 0.0418 [get_pins {buf_out5000/A}]
  ```

- The following command generates the maximum transitions in the output file, `max_tran.rpt`:

  ```
  tempus > write_annotated_transition -max -file max_tran.rpt
  ```

  The following file is generated:

  ```
  set_annotated_transition -rise -max 0.0278 [get_ports {sig_out4000}]
  set_annotated_transition -fall -max 0.0275 [get_ports {sig_out4000}]
  set_annotated_transition -rise -max 0.001 [get_pins {buf_in2000/A}]
  set_annotated_transition -fall -max 0.001 [get_pins {buf_in2000/A}]
  set_annotated_transition -rise -max 0.001 [get_pins {buf_in2001/A}]
  set_annotated_transition -fall -max 0.001 [get_pins {buf_in2001/A}]
  ```

```
set_annotated_transition -rise -max 0.0453 [get_pins {buf_out5000/A}]
set_annotated_transition -fall -max 0.0418 [get_pins {buf_out5000/A}]
```

Related Commands

- set_annotated_transition
- get_ports
- get_pins

write_load

```
write_load
[-help]
load_file
[-direct]
[-exclude_zero_cap]
[-include_pin_cap]
[-wire_load]
```

Writes the capacitive loading on each net to an output file. By default, the capacitance value does not include the pin capacitance. You can either set the load using the `set_load` command or the software uses the load defined in the SPEF file.

Note: The command is currently not supported in multi-mode multi-corner mode.

Parameters

`-help`	Prints out the command usage.
`-direct`	Refers the ports directly for which the capacitive loading will be written. This option removes the dependency of using the `get_nets` command.
`-exclude_zero_cap`	Excludes the nets not annotated in the design while writing out the capacitive loading on each net to the output file.
load_file	Specifies the name of the output file.
`-help`	Outputs a brief description that includes type and default information for each `write_load` parameter. For a detailed description of the command and all of its parameters, use the man command: `man write_load`.
`-include_pin_cap`	Includes the pin capacitance while writing out the capacitive loading on each net to the output file.
`-wire_load`	Adds option '`-wire_load`' to "`set_load`" (that is, `set_load -wire_load`) only on the output (out) and bi-directional (inout) ports in the output file.

Examples

- The following command writes the capacitive load in the report file `load.rpt`:

  ```
  tempus > write_load -file load.rpt
  ```

  The following report is generated:

  ```
  set_load 1e-05 [get_nets {sig_out4000}]
  set_load 0.00011 [get_nets {sig_in1000}]
  set_load 0.00011 [get_nets {sig_in1001}]
  set_load 0.00015 [get_nets {node_out6000}]
  ```

Related Commands

- set_load
- get_nets

29

Timing Debug Commands

- analyze_paths_by_bottleneck
- analyze_paths_by_clock_domain
- analyze_paths_by_critical_false_path
- analyze_paths_by_default_path_group
- analyze_paths_by_drv
- analyze_paths_by_hier_port
- analyze_paths_by_hierarchy
- analyze_paths_by_view
- create_histogram_png
- create_path_category
- delete_path_category
- gui_gtd_highlight_timing_report
- gui_open_gtd
- load_failure_report
- read_path_categories
- read_timing_debug_report
- write_category_summary
- write_histogram_view
- write_path_categories
- write_path_list_summary
- write_text_timing_report
- zoom_highlighted_timing_path

analyze_paths_by_bottleneck

```
analyze_paths_by_bottleneck
[-help]
{-category n | -delta_delay {worst|total}}
[-path_num m]
[-max_slack float]
[-min_slack float]
[-master master_category_name]
```

Creates categories according to the bottleneck analysis of the critical paths and fixes the violation according to the bottleneck information. Bottleneck Analysis detects the instances that are often involved in the critical paths in the design. Any change in timing for these instances impacts multiple critical paths in the design. Therefore you can use this information to change your design (floorplan, placement, constraints etc.) around these instances. The bottleneck analysis computes the number of occurrences that an instance appears in the paths that you display in the Timing Debug window, and provides the following output:

- Colors most used instance based on number of occurrences.
- Creates *n* number of path categories for the instances with the highest number of occurrences. *n* is the number you specify using the `-category` parameter.

Users can also perform bottleneck analysis of the critical paths based on incremental delay. Categories will be created based on the worst *m* paths with:

- The biggest worst `incr_delay` contribution
- The biggest total `incr_delay` contribution

The command can be applied wherever the user loads the machine readable timing report.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `analyze_paths_by_bottleneck` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man analyze_paths_by_bottleneck`
`-category` *n*	Specifies the number of categories to be created for most occurring instances.
`-delta_delay {worst|total}`	Specifies that the categories are created based on worst `incr_delay` contribution or total `incr_delay` contribution.
`-master` *master_category_name*	Specifies that the analysis should be performed only for the specified `master` category.
`-max_slack` *float*	Specifies the max slack value of the critical paths for bottleneck analysis based on incr_delay.
`-min_slack` *float*	Specifies the min slack value of the critical paths for bottleneck analysis based on incr_delay.
`-path_num` *m*	Specifies the number of critical paths to be analyzed when performing bottleneck analysis based on incr_delay.

Examples

- The following command creates path categories based on bottleneck analysis of the critical paths on the instances:
  ```
  analyze_paths_by_bottleneck -category 2
  ```
  The following categories are created: `BOT_instance1` and `BOT_instance2`. You can change the number of categories.
- The following command creates path categories based on bottleneck analysis of the critical 20 paths on the worst incr_delay:
  ```
  analyze_paths_by_bottleneck -path_num 20 -delta_delay worst
  ```
  The following category is created: `BOT_SI_WORST`.

Related Commands

- create_path_category
- delete_path_category

analyze_paths_by_clock_domain

```
analyze_paths_by_clock_domain
[-help]
[-include_edges]
[-include_views]
[include_mode]
[-master master_category_name]
```

To find the possible timing issues in clock combinations, the command creates categories according to launch clocks - capture clock combinations.

Following categories are created:

- Paths with clock fully contained in a single domain, clk1.
- Paths with clocks starting one clock domain and ending at another, clk1-> clk2.

The command can be applied wherever the user loads the machine readable timing report.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `analyze_paths_by_clock_domain` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man analyze_paths_by_clock_domain`
`-include_edges`	Considers leading or trailing edge of the clock while defining categories.
`-include_mode`	Considers constraint mode while defining categories.
`-include_views`	Considers analysis views while defining categories.
`-master` *`master_category_name`*	
	Specifies that the analysis should be performed only for the specified master category.

Examples:

- The following command creates path categories based on clock domain analysis of the critical paths, and considers the edges of the clock:

  ```
  analyze_paths_by_clock_domain -include_edges
  ```

- The following command creates path categories based on clock domain analysis of the critical paths, and considers the analysis views:

  ```
  analyze_paths_by_clock_domain -include_views
  ```

Related Commands

- create_path_category
- delete_path_category

analyze_paths_by_critical_false_path

```
analyze_paths_by_critical_false_path
[-help]
-critical_false_paths
{false_path_only | exclude_false_path}
[-master master_category_name]
```

Identifies violating false paths, which are paths that appear as violations but cannot be exercised due to the structure of the netlist, and creates a category that contains them. With this command the user can, in advance, categorize the critical false paths together to analyze them.

The command can be applied wherever the user loads the machine readable timing report.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `analyze_paths_by_critical_false_path` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man analyze_paths_by_critical_false_path`
`-critical_false_paths`	Specifies which paths to analyze. • `false_path_only` Includes only false paths when creating the category. • `exclude_false_paths` Excludes false paths when creating the category. *Default*: `false_path_only`
`-master` *`master_category_name`*	Specifies that the analysis should be performed only for the specified master category.

Examples:

- The following command creates path categories based on the critical false paths:
  `analyze_paths_by_critical_false_path -critical_false_path\ false_path_only`
- The following category is created: `default_report_false_path_only`

Related Commands

- create_path_category
- delete_path_category

analyze_paths_by_default_path_group

```
analyze_paths_by_default_path_group
[-help]
[-master master_category_name]
[-predefined]
```

Categorizes the timing paths based on basic path groups. The basic path groups are fundamental to timing analysis and help the user easily observe the timing information of different basic path groups. For instance, if user is only concerned about the paths from registers to registers, the user can analyze the paths in the category REG -> REG. The basic path groups categories are:

- Register to register
- Input to register
- Register to output
- Input to output

The following categories are created:

- `REG->REG`
- `IN->REG`
- `REG->OUT`
- `IN->OUT`

Timing debug can also convert user defined path groups into path categories. The command can be applied wherever the user loads the machine readable timing report.

Note: If your design does not have paths belonging to a type of path group, the software will not create that category.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `analyze_paths_by_default_path_group` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man analyze_paths_by_default_path_group`
`-master` *`master_category_name`*	Specifies that the analysis should be performed only for the specified master category.
`-predefined`	Creates path categories for user defined path groups.

Examples

- The following command create path categories based on basic path groups:
  `analyze_paths_by_default_path_group`
  The following categories are created: REG->REG, IN->REG, REG->OUT, IN->OUT.
- The following command create path categories based on predefined path group "critical" :

```
group_path -name critical -from {ff*/CLK } -to {ff*/D} #create predefined path group
analyze_paths_by_default_path_group -predefined
```

  The category "critical" will be created based on the predefined path group name.

Related Commands

- `create_path_category`
- `delete_path_category`

analyze_paths_by_drv

```
analyze_paths_by_drv
[-help]
[[-load_cap_file capacitance_file] | [-generate_cap_file fileName]]
[[-load_transition_file transition_file] | [-generate_transition_file fileName]]
[-load_fanout_file fanout_file | [-generate_fanout_file fileName]]
[-master master_category_name]
```

Reads or generates a report containing max transition, max capacitance, and max fanout violations. Paths that are affected by the selected DRV type are grouped into a category. This command can be used if the user wants to debug the timing issues caused by DRV.

The command can be applied wherever the user loads the machine readable timing report.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `analyze_paths_by_drv` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man analyze_paths_by_drv`
`-load_cap_file` *`capacitance_file`*	
	Reads a file containing a list of paths containing max capacitance violations.
`-load_fanout_file` *`fanout_file`*	
	Reads a file containing a list of paths containing max fanout violations.
`-load_transition_file` *`transition_file`*	
	Reads a file containing a list of paths containing max transition violations.
`-generate_cap_file` *`fileName`*	
	Generates a file containing a list of paths containing max capacitance violations.
`-generate_fanout_file` *`fileName`*	
	Generates a file containing a list of paths containing max fanout violations.
`-generate_transition_file` *`fileName`*	
	Generates a file containing a list of paths containing max transition violations.
`-master` *`master_category_name`*	
	Specifies that the analysis should be performed only for the specified master category.

Examples:

The following command generates DRV report files, then creates path categories based on the critical paths which are affected by DRV:

```
analyze_paths_by_drv -generate_cap_file capViolation.rpt

-generate_transition_file tranViolation.rpt
```

```
-generate_fanout_file fanoutViolation.rpt
```

Related Commands

- create_path_category
- delete_path_category

analyze_paths_by_hier_port

```
analyze_paths_by_hier_port
[-help]
[-generate_hier_file InstanceName_list]
[-load_hier_file file_name]
[-master master_category_name]
[-out_file]
```

Helps you to automatically group paths based on each hierarchical port of a given instance. You can use this command to perform the timing results analysis of a hierarchical design. For example, while looking at the setup/hold time at the same time of each port.

The command can be applied wherever the user loads the machine readable timing report.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `analyze_paths_by_hier_port` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man analyze_paths_by_hier_port`
`-generate_hier_file` *`InstanceName_list`*	
	Generates hier ports fanin/fanout file when the database is available based on the given InstanceName_list. Then uses that file to create the categories.
`-load_hier_file` *`file_name`*	
	Uses a previously saved hier ports fanin/fanout file when the database is not available.
`-master` *`master_category_name`*	
	Specifies that the timing paths should be categorized only for the specified master category.
`-out_file`	Specifies the generated hier ports fanin/fanout file name.

Example

The following command categorizes paths based on hierarchical port of the instance `DTMF_INST/TDSP_CORE_INST`:

```
analyze_paths_by_hier_port -generate_hier_file DTMF_INST/TDSP_CORE_INST
```

Related Commands

- create_path_category
- delete_path_category

analyze_paths_by_hierarchy

```
analyze_paths_by_hierarchy
[-help]
[-master master_category_name]
{-floorplan_file floorplan_file [-include_macros]} | {-use_current_floorplan  [-all] [-types type_list] [-port_to_port] [-port_to_macro] [-port_to_bbox] [-port_to_hinst] [-port_to_stdcell] [-port_to_inst_group] [-macro_to_port] [-macro_to_macro] [-macro_to_bbox] [-macro_to_hinst] [-macro_to_stdcell] [-macro_to_inst_group] [-bbox_to_port] [-bbox_to_macro] [-blackbox_to_blackbox] [-bbox_to_hinst] [-bbox_to_stdcell] [-bbox_to_inst_group] [-hinst_to_port] [-hinst_to_macro] [-hinst_to_bbox] [-hinst_to_hinst] [-hinst_to_stdcell] [-hinst_to_inst_group] [-stdcell_to_port] [-stdcell_to_macro] [-stdcell_to_bbox] [-stdcell_to_hinst] [-stdcell_to_stdcell] [-stdcell_to_inst_group] [-inst_group_to_port] [-inst_group_to_macro] [-inst_group_to_bbox] [-inst_group_to_hinst] [-inst_group_to_stdcell] [-inst_group_to_inst_group]} | {-partitions partition_list | -macros macro_list}
```

Creates categories according to the hierarchical characteristics of the paths. User can use either a user specified floorplan file or current floorplan file of the design in memory and can create categories according to the hierarchy of the design. This way, the paths can be narrowed down to small group.

Use Floorplan Plan:

The software automatically extracts partition-boundary information from guides, regions, and fences in the floorplan file. Optionally you can specify option to sort paths based on the macros in the floorplan file.

- All paths fully contained in a single partition, Ptn1.
- Paths starting in one partition and ending in another partition, Ptn1–>Ptn2.
- Paths spanning several partitions, Ptn1–> Ptn2–>Ptn3.
- Top-level inputs to partitions, INPUT–>Ptn1.
- Paths from partitions to top-level outputs, Ptn1–>OUTPUT.

Use Current Floorplan:

This method categorizes paths based on the current floor plan of the design in memory. It will rely on the grouped/ungrouped as a basis for what level to view the hierarchy as well as what levels to create timing categories.

Grouped/ungrouped status of an Hinst means whether or not its children are shown on the floor plan view.

It categorizes paths starting and ending on/within these hierarchical entities:

- Port
- Hard Macro (ungrouped)
- Black Box (ungrouped)
- Hinst (regular, partition, Black Blob, or Flex Model) (highest grouped)
- Stdcell (ungrouped)

Parameters

`-help`	Outputs a brief description that includes type and default information for each `analyze_paths_by_hierarchy` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man analyze_paths_by_hierarchy`
`-all`	Creates categories of paths within all types *Default:* All types are categorized.
`-bbox_to_hinst`	Creates categories of paths from black box to hierarchical instance.
`-bbox_to_inst_group`	Creates categories of paths from black box to instance group.
`-bbox_to_port`	Creates categories of paths from black box to port.
`-bbox_to_macro`	Creates categories of paths from black box to macro.
`-blackbox_to_blackbox`	Creates categories of paths from black box to black box.
`-bbox_to_stdcell`	Creates categories of paths from black box to standard cell.

`-floorplan_file floorplan_file`	Specifies the name of the floorplan file. *Default:* Uses the current floorplan file.
`-hinst_to_bbox`	Creates categories of paths from hierarchical instance to black box.
`-hinst_to_port`	Creates categories of paths from hierarchical instance to port.
`-hinst_to_macro`	Creates categories of paths from hierarchical instance to macro.
`-hinst_to_hinst`	Creates categories of paths from hierarchical instance to hierarchical.
`-hinst_to_inst_group`	Creates categories of paths from hierarchical instance to instance group.
`-hinst_to_stdcell`	Creates categories of paths from hierarchical instance to standard cell.
`-include_macros`	Includes macros in addition to guides and fences for creating categories. *Default:* Macros are not considered.
`-inst_group_to_bbox`	Creates categories of paths from instance group to black box.
`-inst_group_to_hinst`	Creates categories of paths from instance group to hierarchical instance.
`-inst_group_to_inst_group`	Creates categories of paths from instance group to instance group.
`-inst_group_to_macro`	Creates categories of paths from instance group to macro.
`-inst_group_to_port`	Creates categories of paths from instance group to port.
`-inst_group_to_stdcell`	Creates categories of paths from instance group to standard cell.
`-macro_to_bbox`	Creates categories of paths from macro to black box.
`-macro_to_port`	Creates categories of paths from macro to port.
`-macro_to_macro`	Creates categories of paths from macro to macro.
`-macro_to_hinst`	Creates categories of paths from macro to hierarchical instance.

`-macro_to_inst_group`	Creates categories of paths from macro to instance group.
`-macro_to_stdcell`	Creates categories of paths from macro to standard cell.
`-macros` *`macro_list`*	Specifies the names of the macros to be used. Use this parameter with the `-partitions` parameter.
`-master` *`master_category_name`*	Specifies that the timing paths should be categorized only for the specified master category.
`-partitions` *`partition_list`*	Specifies the names of the partitions to consider for extracting hierarchical information.
`-port_to_bbox`	Creates categories of paths from port to black box.
`-port_to_port`	Creates categories of paths from port to port.
`-port_to_macro`	Creates categories of paths from port to macro.
`-port_to_hinst`	Creates categories of paths from port to hierarchical instance.
`-port_to_inst_group`	Creates categories of paths from port to instance group.
`-port_to_stdcell`	Creates categories of paths from port to std cell.
`-stdcell_to_bbox`	Creates categories of paths from standard cell to black box.
`-stdcell_to_port`	Creates categories of paths from standard cell to port.
`-stdcell_to_macro`	Creates categories of paths from standard cell to macro.
`-stdcell_to_hinst`	Creates categories of paths from standard cell to hierarchical instance.
`-stdcell_to_inst_group`	Creates categories of paths from standard cell to instance group.
`-stdcell_to_stdcell`	Creates categories of paths from standard cell to standard cell.
`-types type_list`	Creates categories of paths within the type_list. Valid types are: `port`, `macro`, `bbox`, `hinst`, `stdcell`
`-use_current_floorplan`	Uses the current floorplan.

Examples:

- By default, it categorizes all paths inter/intra the 5 hierarchical entities by following command:
  `analyze_paths_by_hierarchy -use_current_floorplan -all`
- The following command categorizes all paths from "port" or to "port":
  `analyze_paths_by_hierarchy -use_current_floorplan -type port`

Related Commands

- `create_path_category`
- `delete_path_category`

analyze_paths_by_view

```
analyze_paths_by_view
[-help]
[-prefix string]
[-master master_category_name]
```

Categorizes the timing paths based on views that you create during multi-mode multi-corner analysis. By arranging the timing information of different views quickly, the command helps in analyzing timing paths.

The command can be applied wherever the user loads the machine readable timing report.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `analyze_paths_by_view` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man analyze_paths_by_view`
`-master` *`master_category_name`*	Specifies that the timing paths should be categorized only for the specified master category.
-prefix *`string`*	Adds a prefix to the category name.

Examples:

The following command categorizes paths based on views that you created during mmmc analysis :

```
analyze_paths_by_view
```

Related Commands

- create_path_category
- delete_path_category

create_histogram_png

```
create_histogram_png
[-help]
[-height integer]
-name string
[-width integer]
```

Dumps out the timing histogram image from global timing debug in the PNG format without bringing up the GUI. Earlier, this was done when the GUI was displayed by right-clicking the histogram window and selecting Snapshot. But that worked only when the snapshot was displayed in the GUI. This command has eliminated the necessity of bringing up the GUI to dump out the timing histogram.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `create_histogram_png` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man create_histogram_png`
`-height integer`	Specifies the height of the PNG image. *Minimum:* `1` *Maximum:* `10000`
`-name string`	Specifies the name of the PNG file.
`-width integer`	Specifies the width of the PNG image. *Minimum:* `1` *Maximum:* `10000`

create_path_category

```
create_path_category
[-help]
[-capture_latency string]
[-check_type string] [-clock string]
[-comment string] [-critical_false_path string]
[-delay_of_any_inst string]
[-delay_of_any_inst_not_of_lib_cell string]
[-delay_of_any_inst_not_of_name string]
[-delay_of_any_inst_of_name string]
[-delay_of_any_inst_of_lib_cell string]
[-delay_of_any_net string]
[-delta_delay_of_every_net string]
[-delta_delay_of_any_net string]
[-delay_of_any_net_not_of_name string]
[-delay_of_any_net_of_name string]
[-delay_of_every_inst string]
[-delay_of_every_inst_of_lib_cell string]
[-delay_of_every_inst_not_of_lib_cell string]
[-delay_of_every_inst_not_of_name string]
[-delay_of_every_inst_of_name string]
[-delay_of_every_net string]
[-delay_of_every_net_not_of_name string]
[-delay_of_every_net_of_name string]
[-file string]
[-from_lib_cell string]
[-from_clock string]
[-from_clock_edge string]
[-from_inst string]
[-from_inst_pointer string]
[-from_pdomain string]
[-from_pin string]
[-from_port string]
[-from_port_pointer string]
[-launch_latency string]
[-master string]
[-name string]
[-not_from_cell string]
[-not_from_clock string]
[-not_from_clock_edge string]
[-not_from_inst string]
```

[-not_from_pdomain *string*]
[-not_from_pin *string*]
[-not_from_port *string*]
[-not_through_lib_cell *string*]
[-not_through_inst *string*]
[-not_through_net *string*]
[-not_through_pdomain *string*]
[-not_through_pin *string*]
[-not_through_port *string*]
[-not_to_lib_cell *string*]
[-not_to_clock *string*]
[-not_to_clock_edge *string*]
[-not_to_inst *string*]
[-not_to_pdomain *string*]
[-not_to_pin *string*]
[-not_to_port *string*]
[-number_of_insts *string*]
[-number_of_insts_not_of_name *string*]
[-number_of_insts_not_of_lib_cell *string*]
[-number_of_insts_of_lib_cell *string*]
[-number_of_insts_of_name *string*]
[-overwrite]
[-sdc *string*]
[-skew *string*]
[-slack *string*]
[-through_lib_cell *string*]
[-through_inst *string*]
[-through_net *string*]
[-through_pdomain *string*]
[-through_pin *string*]
[-through_port *string*]
[-to_lib_cell *string*]
[-to_clock *string*]
[-to_clock_edge *string*]
[-to_inst *string*]
[-to_inst_pointer *string*]
[-to_pdomain *string*]
[-to_pin *string*]
[-to_port *string*]
[-to_port_pointer *string*]
[-total_delay_of_insts *string*]
[-total_delay_of_insts_not_of_name *string*]
[-total_delay_of_insts_not_of_lib_cell *string*]

```
[-total_delay_of_insts_of_lib_cell string]
[-total_delay_of_insts_of_name string]
[-total_delay_of_level_of_chain_of_inst_type string]
[-total_delay_of_nets string]
[-total_delay_of_nets_not_of_name string]
[-total_delay_of_nets_of_name string]
[-total_delay_of_path string]
[-total_delta_delay_of_path string]
[-uncertainty string]
[-view string]
```

This command creates path categories using conditions for grouping paths according to analyzed timing results. The command can be applied wherever the user loads the machine readable timing report.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `create_path_category` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man create_path_category`
`-check_type {Setup Check | Hold Check | Clock Gating Setup Check | Library Clock Gating Setup Check | Clock Gating Hold Check | Library Clock Gating Hold Check | External Delay Assertion | Latch Borrowed Time Check | PulseWidth Check | ClockPeriod Check | Recovery Check | Removal Check}`	Indicates that the check type matches the specified option.

`-clock {both | launch | capture}`	Helps you search the Clock Tree instead of the data path. If this option is not specified, the user searches the data path by default. You can use one of the following values: • `Both`: searches on launch or capture clock paths. • `Launch`: searches only on launch clock paths • `Capture`: searches only on capture clock paths **Note:** Supports simultaneous criteria on data path.
`-comment {`*`commentText`*`}`	Specifies a comment that is associated with the category.
`-critical_false_path`	*Type:* String, optional
`-master` *`existing_category_name`*	Specifies that the new category should be created as a nested category of the specified master category. The master category should be an existing category. *Default:* If you do not specify this parameter, the category is created as a new category at the top level.
`-name` *`group_name`*	Specifies the name of the category to be created.
`-overwrite`	Overwrites any predefined category with the same name in your category list.
`-sdc {`*`file_name line_number`*`}`	Creates a category based on the SDC constraint as per the specified SDC file name and line number. Creating a path category based on an SDC constraint helps you to analyze the affect of an SDC constraint by grouping the affected paths together.

`-file string`	Specifies the file name. *Type:* String, optional
`{-from_inst | -from_inst_pointer | -from_pin | -from_lib_cell | -from_clock | -from_port| -from_port_pointer}` *string*	Indicates that the path must be from the specified instance, pin, library cell, clock, or port pattern.
`-from_pdomain` *string*	Indicates that the path must be from the specified power domain.
`-from_clock_edge`	Indicates that the path must be from the specified clock edge.
`-not_from_inst | -not_from_pin | -not_from_cell | -not_from_clock | -not_from_port` *string*	Indicates that the path must not be from the specified instance, pin, cell, clock, or port pattern.
`-not_from_pdomain` *string*	Indicates that the path must not be from the specified power domain.
`-through_inst | -through_pin | -through_lib_cell | -through_net | -through_port` *string*	Indicates that the path must be through the specified instance, pin, net, library cell, or port pattern.
`-through_pdomain` *string*	Indicates that the path must be through the specified power domain.
`-not_to_clock_edge`	*Type:* String, optional
`-slack` *float*	Indicates that the slack of the path must match the specified *float* value.
`-not_from_clock_edge`	*Type:* String, optional
`{-not_through_inst | -not_through_pin | -not_through_net | -not_through_port | -not_through_pdomain}` *string*	Indicates that the path must not be through the specified instance, pin, power domain, net, or port pattern.
`-not_through_lib_cell` *string*	Indicates that the path must not be through the specified library cell.
`{-to_inst | -to_inst_pointer | -to_pin | -to_lib_cell | -to_clock | -to_port | -to_port_pointer | -to_pdomain}` *string*	Indicates that the path must be the specified instance, pin, clock, power domain, library cell, or port pattern.

`-not_to_inst	-not_to_pin	-not_to_lib_cell	-not_to_clock	-not_to_port	-not_to_pdomain` *string*	Indicates that the path must not be to the specified instance, pin, clock, library cell, power domain, or port pattern.
`-skew` *float*	Indicates that the skew of the path must match the specified *float* value.
`-launch_latency` *float*	Indicates that the launch latency of the path must match the specified *float* value.
`-capture_latency` *float*	Indicates that the capture latency of the path must match the specified *float* value.
`-uncertainty` *float*	Indicates that the uncertainty of the path must match the specified *float* value.
`-number_of_insts` *integer*	Indicates that the number of instances in the data segment (with launch and capture clock paths) must match the specified *integer* value.
`-number_of_insts_of_name` *string_plus_integer*	Indicates that the number of instances matching the specified instance name match the specified *integer* value.
`-number_of_insts_not_of_name` *string_plus_integer*	Indicates that the number of instances not matching the specified instance name match the specified *integer* value.
`-number_of_insts_of_lib_cell` *string*	Indicates that the number of instances matching the specified library cell match the specified *integer* value.
`-number_of_insts_not_of_lib_cell` *string*	Indicates that the number of instances not matching the specified library cell match the specified *integer* value.
`-to_clock_edge`	*Type:* String, optional
`-total_delay_of_insts` *float*	Indicates that the total delay of instances matches the specified *float* value.

-total_delay_of_level_of_chain_of_inst_type	*Type:* String, optional
-total_delay_of_path *float*	Indicates that the total delay of path matches the specified *float* value.
-total_delta_delay_of_path *float*	Indicates that the total incremental delay of path matches the specified *float* value.
-total_delay_of_nets *float*	Indicates that the total delay of nets matches the specified *float* value.
-total_delay_of_insts_of_name *string_plus_float*	Indicates that the total delay of instances matching the specified name matches the specified *float* value.
-total_delay_of_insts_not_of_name *string_plus_float*	Indicates that the total delay of instances not matching the specified name matches the specified *float* value.
-total_delay_of_insts_not_of_lib_cell *string*	Indicates that the total delay of instances not matching the specified library cell matches the specified *float* value.
-total_delay_of_insts_of_lib_cell *string*	Indicates that the total delay of instances matching the specified library cell matches the specified *float* value.
-total_delay_of_nets_of_name *string_plus_float*	Indicates that the total delay of nets matching the specified name matches the specified *float* value.
-total_delay_of_nets_not_of_name *string_plus_float*	Indicates that the total delay of nets not matching the specified name matches the specified *float* value.
-delay_of_any_inst *float*	Indicates that the delay of any instance in the design matches the specified *float* value.
-delay_of_any_inst_not_of_lib_cell *float*	Indicates that the total delay of of any instance in the design not matching the specified library cell matches the specified *float* value.

`-delay_of_any_net` *`float`*	Indicates that the delay of any net in the design matches the specified *`float`* value.
`-delta_delay_of_any_net` *`float`*	Indicates that the incremental delay of any net in the design matches the specified *`float`* value.
`-delta_delay_of_every_net` *`float`*	Indicates that the incremental delay of every net in the design matches the specified *`float`* value.
`-delay_of_every_inst` *`float`*	Indicates that the delay of every instance in the specified library cell matches the specified *`float`* value.
`-delay_of_every_inst_not_of_lib_cell` *`float`*	Indicates that the delay of every instance in the design not matching the specified library cell matches the specified *`float`* value.
`-delay_of_every_net` *`float`*	Indicates that the delay of every instance in the design matches the specified *`float`* value.
`-delay_of_any_inst_of_lib_cell` *`float`*	Indicates that the delay of any instance in the specified library cell matches the specified *`float`* value.
`-delay_of_any_inst_of_name` *`string_plus_float`*	Indicates that the delay of any instance in the design matching the specified name matches the specified *`float`* value.
`-delay_of_any_inst_not_of_name` *`string_plus_float`*	Indicates that the delay of any instance in the design not matching the specified name matches the specified *`float`* value.
`-delay_of_any_net_of_name` *`string_plus_float`*	Indicates that the delay of any net in the design matching the specified name matches the specified *`float`* value.
`-delay_of_any_net_not_of_name` *`string_plus_float`*	Indicates that the delay of any net in the design not matching the specified name matches the specified *`float`* value.

`-delay_of_every_inst_of_name` *`string_plus_float`*	Indicates that the delay of every instance in the design matching the specified name matches the specified *`float`* value.
`-delay_of_every_inst_of_lib_cell` *`string`*	Indicates that the delay of every instance in the design matching the specified name matches the specified *`float`* value.
`-delay_of_every_inst_not_of_name` *`string_plus_float`*	Indicates that the delay of every instance in the design not matching the specified name matches the specified *`float`* value.
`-delay_of_every_net_of_name` *`string_plus_float`*	Indicates that the delay of every net in the design matching the specified name matches the specified *`float`* value.
`-delay_of_every_net_not_of_name` *`string_plus_float`*	Indicates that the delay of every net in the design not matching the specified name matches the specified *`float`* value.
`-view`	*Type:* String, optional

Examples:

- The following command categorizes paths whose source clock is `vclk1`:

  ```
  create_path_category -name vclk1 -from_clock vclk1
  ```

  The category `vclk1` is created.

- The following command categorizes paths whose skew is larger than 0.2:

  ```
  create_path_category -name worst_skew -skew {> 0.2}
  ```

  The category `worst_skew` is created.

Related Commands

`create_path_category`

delete_path_category

```
delete_path_category
[-help]
category_name
```

Deletes the selected category name.

The command can be applied when categories are created in Global Timing Debug.

Parameter

`-help`	Outputs a brief description that includes type and default information for each `delete_path_category` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man delete_path_category`
`category_name`	Specifies the name of the category to be deleted.

Examples:

The following command deletes the category `worst_paths`:

```
delete_path_category worst_paths
```

Related Commands

`create_path_category`

gui_gtd_highlight_timing_report

```
gui_gtd_highlight_timing_report
[-help]
[-append]
[-color_index integer]
[-in_file filename]
[-no_arrow]
[-paths path_number | -all]
```

Highlight the selected paths in the user interface.

Parameter

`-help`	Outputs a brief description that includes type and default information for each `gui_gtd_highlight_timing_report` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man gui_gtd_ighlight_timing_report`
`-all`	Highlights all the paths. The option is of type boolean and required.
`-append`	Appends a path to the current highlight.
`-color_index integer`	Specifies the `color_index` of the path from 1 to 8. The option is of type int and optional with minimum value 1, and maximum value 8.
`-in_file filename`	Name of the file. The option is of type string and optional.
`-no_arrow`	Removes the highlighted arrow. The option is of type boolean and optional.
`-paths path_number`	Specifies the number of the path. The option is of type string and required.

Related Commands

`gui_highlight_pin_connection`

gui_open_gtd

```
gui_open_gtd
[-help]
```

Opens the global timing debugger in the common user interface mode for accessing the GTD files report.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `gui_open_gtd` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man gui_open_gtd`

load_failure_report

```
load_failure_report
[-help]
-file filename
```

Loads the failure report generated during `opt_signoff`. This failure report contains various reasons that limited timing closure during automated signoff ECO. This command reads the Failure Catalog File (FCF) in the format used by the Failure Catalog Browser (FCB).

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `load_failure_report` parameter. For a detailed description of the command and all its parameters, use the man command: `man load_failure_report`
`-file filename`	Specifies the name of the file describing the failure category. This is a mandatory parameter.

Related Commands

`opt_signoff`

read_path_categories

```
read_path_categories
[-help]
in_file filename
```

Loads the path category file in the Timing Debug form. The path category file contains category definitions that you saved in an earlier timing debug analysis. To save path categories, use the `write_path_categories` command.

Parameter

`-help`	Outputs a brief description that includes type and default information for each `read_path_categories` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_path_categories`
`in_file` *`filename`*	Specifies the name of the category file.

Example

The following command loads the path category file `category.cat`:

```
read_path_categories in_file category.cat
```

Related Commands

`write_path_categories`

read_timing_debug_report

```
read_timing_debug_report
[-help]
[-name report_name]
[{-proto [-slack_margin number]  [-num_path number]}]
[-num_path]
timing_report_filename
```

Loads the violation report in Tempus for debugging timing results.

> The violation report that you specify using the `read_timing_debug_report` command is in the output binary format, which you can generate through the `binary` value of the `report_timing -output_format` command parameter.

Note: The violation report that you specify in this command has a special machine readable format. Use the `report_timing -output_format gtd` command to generate the machine readable violation report.

Parameter

`-help`	Outputs a brief description that includes type and default information for each `read_timing_debug_report` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man read_timing_debug_report`
`timing_report_filename`	Specifies the name of the timing debug timing_report_file.
`-name`	Specifies the report name to load the machine readable timing report `timing_report_files` incrementally.
`-num_path`	Specifies the number of paths to be reported in the detailed path violations.
`{-proto [-slack_margin number]  [-num_path number]}`	
	Creates flex model categories and displays the top path of the top 8 (by default) categories. This option can be used for design that has flexModels. Use `-additional_slack_past_wns` to report all paths with slack worst than (WNS and additional specified slack value). Use `-num_path` to control the number of violation paths.

Examples:

- The following command loads the machine readable timing report `top.btarpt`:

  ```
  read_timing_debug_report top.btarpt
  ```

- The following commands load two machine readable timing reports `top1.btarpt` and `top2.btarpt` together:

  ```
  read_timing_debug_report -name default_report top1.btarpt
  read_timing_debug_report -name default_report top2.btarpt
  ```

  GTD will first load the report `top1.btarpt`, then append `top2.btarpt` incrementally to the current report.
  Note: The `name` can be specified by user with any string, it is not the timing_report_file name.

Related Commands

report_timing

write_category_summary

```
write_category_summary
[-help]
out_file string
[[-sort_low_to_high | -sort_high_to_low ]
[-sort_by_tns | -sort_by_wns ]]
[-csv | {-no_frame_fix_width [-no_split ]}]
```

Writes a text file containing the following information:

- Category name
- Total number of paths
- Number of passing paths
- Number of failing paths
- Worst negative slack
- Total negative slack

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_category_summary` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_category_summary`
`-sort_low_to_high`	Specifies that the report in ascending order, should specify option `-sort_by_tns` or `-sort_by_wns`.
`-sort_by_tns`	The TNS value specified by the sorting key through the option `-sort_low_to_high` or `-sort_high_to_low`.
`-sort_by_wns`	The WNS value specified by the sorting key through the option `-sort_low_to_high` or `-sort_high_to_low`.
`-csv`	Reports entries in csv format.

`-sort_high_to_low`	Specifies that the report in descending order, should specify option `-sort_by_tns` or `-sort_by_wns`.
`-no_frame_fix_width`	Specifies that the table columns have a fixed minimum width, but no maximum width (the data does not wrap). Column data that fits within the column's minimum width will left-justify and pad its location to fill the specified column space. Column data that is longer than the column's minimum width will take as much space in the table as needed. If a column is overwritten by the data from the preceding column, its data will begin on a new line at the correct column location. *Default:* `-frame`
`-no_split`	Specifies that each column follows the preceding column without splitting to a new line, if the data in one column overruns the next column. The wrapping behavior for data is controlled by the size of the display window. This format is useful for machine parsing the report. **Note:** You must specify `-no_frame_fix_width` in order to use this parameter.
`out_file` *fileName*	Specifies the name of the category report file. *Default*: `category.rpt`

Examples

The following command writes a category summary file `category.rpt`:

```
write_category_summary out_file category.rpt
```

write_histogram_view

```
write_histogram_view
[-help]
file_name string
[-type string]
```

Saves (dumps) the image of the histogram shown in *Global Timing Debug* automatically during the process.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_histogram_view` parameter. For a detailed description of the command and all of its parameters, use the man command: man `write_histogram_view`.
`file_name` *`string`*	Specifies the name of the image file to save the histogram.
`-type` *`string`*	Specifies the type of file (image format) to be saved. Options are `bmp/jpg/gif/tif/csv`. *Default*: `bmp`

write_path_categories

```
write_path_categories
[-help]
out_file filename
```

Saves the selected categories in a path category file. You can use the category file to load saved category definitions in a new timing debug session. To load the saved category definitions, use the `read_path_categories` command.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_path_categories` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_path_categories`
`out_file` *`filename`*	Specifies the name of the category file.

Example

The following command saves the path category file `category.cat`:

```
write_path_categories out_file category.cat
```

Related Commands

`read_path_categories`

write_path_list_summary

```
  write_path_list_summary
[-help]
[-category string]
[-compress]
[-csv]
-report fileName
```

Writes the Path List categorized summary into a text and/or CSV file.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `write_path_list_summary` parameter. For a detailed description of the command and all its parameters, use the `man` command: `man write_path_list_summary`
`-category string`	Specifies the category of paths to be stored into the CSV file.
`-compress`	Compresses the output file in the `.gzip` file format.
`-csv`	Reports entries in the CSV format.
`-report fileName`	Specifies the name of the CSV output file.

write_text_timing_report

```
write_text_timing_report
[-help]
[-machine_readable file.mtarpt |
-category category_name]
[-format{{column1 width1} {column2 width2} ...} ]
[-path_spec path_specification]
[-summary] |
report_file file.tarpt
```

Writes a text file containing information about paths that have been loaded from a `.mtarpt` file from Timing Debug, either with the `read_timing_debug_report` command or in the *Display/Generate Timing Report* form in the GUI.

Paths are selected as follows:

- If only one `.mtarpt` file is loaded and no other parameters are specified, then the tool writes all the paths contained in the loaded `.mtarpt` file.
- If there are several `.mtarpt` files loaded, you must specify the `.mtarpt` file from which the tool can select the paths, except if a category is specified.

You can use this report for performing timing analysis.

If only one machine-readable report file is loaded, then the tool writes all paths from that file into the output report. If more than one file is loaded, then the tools issues an error message.

Note: The generated report cannot be read back into the timing debug environment.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `write_text_timing_report` parameter. For a detailed description of the command and all of its parameters, use the `man` command: `man write_text_timing_report`
`-category` *`category_name`*	
	Specifies the category of paths to write to the text file. You cannot specify `-machine_readable` if you specify this parameter.
`-format{{`*`column1 width1`*`} {`*`column2 width2`*`} ...}`	
	Formats the report according to the specified column name and its width. The column-width pair specifies the column to display in the report and the width of that column. Multiple column names can be specified.
`-machine_readable` *`file.mtarpt`*	
	Specifies the name of the `.mtarpt` file to read and convert to a text file. You cannot specify this parameter if you specify `-category`.
`-path_spec` *`path_specification`*	
	Writes a single number or path range separated by blank space. A path range is two numbers separated by a minus "-" sign. You can use this parameter only if a single file is specified or loaded.
`report_file` *`file.tarpt`*	
	Specifies the name of the text output file.
`-summary`	Writes the file header only. Use this parameter if you do not need a detailed table.

Examples

- The following command loads machine-readable report `top.mtarpt`:

  ```
  load_timing_debug_report top.mtarpt
  ```

- The following command reads the `top.mtarpt` report and reports paths 1 through 50 in the `top_tarpt.txt` file.

  ```
  write_text_timing_report -machine_readable top.mtarpt -path_spec {1-50}
  report_file top_tarpt.txt
  ```

Related Commands

`read_timing_debug_report`

zoom_highlighted_timing_path

```
  zoom_highlighted_timing_path
[-help]
[-previous | -next]
```

Enables zooming into highlighted timing paths within the Timing Debug form in the Layout view, and supports sequential traversal when multiple paths are highlighted. This command reduces the number of steps required to view and navigate timing paths in the Layout view. The command is very useful for zooming into highlighted timing paths in large designs.

Parameters

`-help`	Outputs a brief description that includes the type and default information for each `zoom_highlighted_timing_path` parameter. For a detailed description of the command and all its parameters, use the man command: `man zoom_highlighted_timing_path`
`-next`	Enables zooming into the next highlighted timing path.
`-previous`	Enables zooming into the previous highlighted timing path.

30

Timing Modeling Commands

- Extracted Timing Model - ETM
- Prototype Timing Model - PTM

Extracted Timing Model - ETM

- check_timing_model
- merge_model_timing
- report_timing_model
- write_timing_model

check_timing_model

```
check_timing_model
[-absolute_cap_tolerance float]
[-absolute_tolerance float]
[-absolute_tolerance_optimistic float]
[-absolute_tolerance_pessimistic float]
[-absolute_transition_tolerance float]
-compare_file string
[-ignore_view]
-out_file string
[-percentage_cap_tolerance float]
[-percentage_tolerance float]
[-percentage_tolerance_optimistic float]
[-percentage_tolerance_pessimistic float]
[-percentage_transition_tolerance float]
-reference_file string
```

Compares two reports that contain interface timing characteristics generated by the `report_timing_model` command.

This command compares the interface timing report generated by using the `report_timing_model` command on the original netlist with the interface timing report generated by using the `report_timing_model` command on the model extracted for the same design.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `check_timing_model` parameter. For a detailed description of the command and all of its parameters, use the man command: `man check_timing_model`.

`-absolute_cap_tolerance` *float*	Specifies the tolerance value for capacitance (as an absolute value) while comparing two report files. This parameter specifies the permissible difference, as an absolute value, between the capacitance values of two report files. If the difference in the value of capacitance exceeds the absolute tolerance value specified, the comparison status is set to `FAIL`. **Note:** When this parameter is not specified, the software uses the absolute value specified with the `-absolute_tolerance` parameter. If the `-absolute_tolerance` parameter is not specified, the value defaults to `0`. When the -percentage_cap_tolerance and -absolute_cap_tolerance parameters are used together, the comparison status is set to FAIL only when the difference in the value of capacitance exceeds the specified absolute as well as percentage tolerance values.
`-absolute_tolerance` *float*	Specifies the absolute tolerance value. The tolerance value is used when there is a difference between the timing characteristics of the reference file (`-reference_file`) and the file that is being compared (`-compare_file`) to determine the `PASS` or `FAIL` status of the comparison. If the difference in the value of a timing characteristic exceeds the absolute tolerance value, the comparison status is set to `FAIL`. **Note:** When used with the `-percentage_tolerance` option, the difference in the value of a timing characteristic should exceed both absolute and percentage tolerance values for the comparison result status of an arc to be `FAIL`. *Default*: `0`

`-absolute_tolerance_optimistic` *float*	Specifies the absolute tolerance limit for flagging an optimistic failure. An optimistic path violating absolute as well as percentage tolerance limit will be flagged as FAIL in the report output. *Default*: 15ps
`-absolute_tolerance_pessimistic` float	Specifies the absolute tolerance limit for flagging a pessimistic failure. A pessimistic path violating absolute as well as percentage tolerance limit will be flagged as FAIL in report output. *Default*: 15ps
`-absolute_transition_tolerance` *float*	Specifies the tolerance value for transition time at ports (as an absolute value) while comparing two report files. This parameter specifies the permissible difference, as an absolute value, between the transition time at ports of two report files. If the difference in the transition time exceeds the absolute tolerance value specified, the comparison status is set to `FAIL`. **Note:** When this parameter is not specified, the software uses the absolute value specified with the `-absolute_tolerance` parameter. If the `-absolute_tolerance` parameter is not specified, the value defaults to `0`. When the `-percentage_transition_tolerance` and `-absolute_transition_tolerance` parameters are used together, the comparison status is set to `FAIL` only when the difference in the value of transition exceeds the specified absolute as well as percentage tolerance values.

`-compare_file` *string*	Specifies the name of the report file that contains the interface timing characteristics extracted from the model created using the `write_timing_model` command. The timing characteristics of this file are compared with the timing characteristics of the reference file that you specify with the `-reference_file` option.
`-ignore_view`	Compares non-MMMC design `report_timing_model` output report with the MMMC design output report (of `report_timing_model` command). When you specify this parameter, the software ignores the view information while comparing the reports. So you do not require to create dummy views for timing model validation flow.
`-out_file` *string*	Specifies the name of the output file in which the comparison results will be generated. This file contains the `PASS` or `FAIL` status for the timing data comparison.

`-percentage_cap_tolerance` *value*	Specifies the tolerance value for capacitance (as a percentage) while comparing two report files. This parameter specifies the permissible difference, in percentage, between the capacitance values of two report files. If the percentage difference in the value of capacitance exceeds the percentage tolerance value specified, the comparison status is set to `FAIL`. **Note:** When this parameter is not specified, the software uses the percentage value specified with the `-percentage_tolerance` parameter. If the `-percentage_tolerance` parameter is not specified, the value defaults to `0`. ⓘ When the `-percentage_cap_tolerance` and `-absolute_cap_tolerance` parameters are used together, the comparison status is set to `FAIL` only when the difference in the value of capacitance exceeds the specified absolute as well as percentage tolerance values.
`-percentage_tolerance` *float*	Specifies the percentage tolerance value for comparing the timing data of two report files. The percentage tolerance value is used when there is a difference between the timing characteristics of the reference file (`-reference_file`) and the file that is being compared (`-compare_file`) to determine the `PASS` or `FAIL` status of the comparison. If the percentage difference in the value of a timing characteristic exceeds the percentage tolerance value, the comparison status is set to `FAIL`. **Note:** When used with the `-absolute_tolerance` option, the difference in the value of a timing characteristic should exceed both percentage and absolute tolerance values for the comparison result status of an arc to be `FAIL`. *Default*: `0`

`-percentage_tolerance_optimistic` *float*	Specifies the percentage tolerance limit for flagging an optimistic failure. An optimistic path violating percentage as well as absolute tolerance limit will be flagged as FAIL in the report output. Default: 3%
`-percentage_tolerance_pessimistic` *float*	Specifies the percentage tolerance limit for flagging a pessimistic failure. A pessimistic path violating percentage as well as absolute tolerance limit will be flagged as FAIL in the report output. Default: 3%
`-percentage_transition_tolerance` *float*	Specifies the tolerance value for transition time at ports (as a percentage) while comparing two report files.This parameter specifies the permissible difference, in percentage, between the transition time at ports of two report files. If the percentage difference in the transition time exceeds the percentage tolerance value specified, the comparison status is set to `FAIL`. **Note:** When this parameter is not specified, the software uses the percentage value specified with the `-percentage_tolerance` parameter. If the `-percentage_tolerance` parameter is not specified, the value defaults to `0`. When the `-percentage_transition_tolerance` and `-absolute_transition_tolerance` parameters are used together, the comparison status is set to `FAIL` only when the difference in the value of transition exceeds the specified absolute as well as percentage tolerance values.

`-reference_file` *string*	Specifies the name of the reference file that is used for comparison. This is the report file that was generated by using the `write_timing_model` command on the original design. The file specified with the `-compare_file` option is compared with this file.

Examples

- The following command compares the timing characteristics of the `sample_compare.rpt` file with the `sample_reference.rpt` file and generates the comparison results in the `sample_output.rpt` file:

```
tempus > check_timing_model -reference_file sample_reference.rpt -compare_file
sample_compare.rpt
-out_file sample_output.rpt
#################################################################
# check_timing_model report
# Design : netlist = topcell =
# Version : 07.10-dev.indfe_cm
# Analysis Mode : setup
# Date : Wed Mar 14 15:56:59 IST 2007
#################################################################
#Section 1: Worst-case Arc-Delay
#Tolerance Percent: 0.0%
#Tolerance Absolute: 0.0
#NOTE: The various columns list values in the form 'X[Y]', where:
# X = value that is compared.
# Y = reference value.

#NOTE2: If a comparison fails (i.e. status = FAIL), then the clauses
# (i.e. 'X[Y]' pairs) that caused the failure are identified with
# a '*' in front of them.
#From   To    Arc-Type   Transition   Arc-Delay     abdDiff   %Diff  Status
#################################################################
```

```
in      CLK3   setup      fall/rise   0.134[0.134]   0.000   0.000%  PASS
in      CLK3   setup      rise/rise  -0.005[-0.005]  0.000  -0.000%  PASS
#################################################################
#Section 2: Pin-capacitances
#Tolerance: 0.0%
#NOTE: The various columns list values in the form 'X[Y]', where:
# X = value that is compared.
# Y = reference value.

#NOTE2: If a comparison fails (i.e. status = FAIL), then the clauses
# (i.e. 'X[Y]' pairs) that caused the failure are identified with
# a '*' in front of them.
#
#Pin C(total)[rise]  C(total)[fall] Status
#################################################################
clk   0.002[0.002]  0.002[0.002]    PASS
in    0.002[0.002]  0.002[0.002]    PASS
out   0.000[0.000]  0.000[0.000]    PASS
#################################################################
#Section 3: Transition-times
#Tolerance: 0.0%
#NOTE: The various columns list values in the form 'X[Y]', where:
# X = value that is compared.
# Y = reference value.
#
#NOTE2: If a comparison fails (i.e. status = FAIL), then the clauses
# (i.e. 'X[Y]' pairs) that caused the failure are identified with
# a '*' in front of them.
#
#Pin   Rise        Fall         Status
#################################################################
clk   0.000[0.000] 0.000[0.000] PASS
in    0.000[0.000] 0.000[0.000] PASS
out   0.056[0.056] 0.051[0.051] PASS
```

```
#################################################################
#Section 4: Design Rules
#Tolerance: 0.0%
#NOTE: The various columns list values in the form 'X[Y]', where:
# X = value that is compared.
# Y = reference value.

#NOTE2: If a comparison fails (i.e. status = FAIL), then the clauses
# (i.e. 'X[Y]' pairs) that caused the failure are identified with
# a '*' in front of them.
#
#Pin    MaxCap    MinCap    MaxTrans    MaxFanout    FanoutLoad    Status
#################################################################################
clk     NA[NA]     NA[NA]    NA[NA]      NA[NA]       NA[NA]         PASS
in      NA[NA]     NA[NA]    NA[NA]      NA[NA]       NA[NA]         PASS
out     NA[NA]     NA[NA]    NA[NA]      NA[NA]       NA[NA]         PASS

#################################################################################
# SUMMARY
#
# Check-Type Passed Failed N/A Total
#################################################################
Worst-Slacks 2 0 0 2
Capacitances 3 0 0 3
Trans-Times 3 0 0 3
Design-Rules 3 0 0 3
```

Related Commands

- report_timing_model
- write_timing_model

merge_model_timing

```
merge_model_timing
[-help]
-input_library_file string
[-keep_separate_arc]
[-merged_library_file string]
-mode_group string
-modes string
[-pin_cap_tolerance float]
[-tolerance float]
```

Merges extracted timing models (ETM) for various analysis views into a single output.

The `merge_model_timing` command should be the last command in the run script. Also, this command should be run separately in a Tempus session when a design is not loaded.

When the DRV values are different in the libraries that are merged, then Tempus saves the worst value of the DRV with a warning message.

If the DRV values are not equal, the greater value is stored for the following:

- Minimum transition
- Minimum capacitance
- Minimum fanout

The smaller value is stored for the following, in case of unequal DRVs:

- Maximum transition
- Maximum capacitance
- Maximum fanout
- Fanout load

Parameters

`-keep_separate_arc`	Enables creating separate timing arcs with respective modes.
`-input_library_file` *`string`*	Specifies input library files that are to be merged.
`-modes` *`string`*	Specifies the mode names required for merging the libraries. The number of mode names should be equal to the number of input libraries.
`-mode_group` *`string`*	Specifies the name of the mode group.
`-merged_library_file` *`string`*	Specifies the name of the merged library output file name. By default, the output file name is `merge.lib`.
`-pin_cap_tolerance` *float*	Specifies the pin capacitance tolerance for making tolerance comparison for library merging.
`-tolerance` *`float`*	Specifies the tolerance value for making tolerance comparison for library merging.

Examples

- The following command merges library files `model1.view1.lib` and `model1.view1.lib.ref` into a single output file:

```
tempus >  merge_model_timing -input_library_file "model1.view1.lib
model1.view1.lib.ref"
-modes "v1 v2" -mode_group v1_v2
```

Related Topics

- See "Extracted Timing Models" in the *Tempus User Guide*.

Related Commands

- `write_timing_model`

report_timing_model

```
report_timing_model
[-help]
model_timing_report_file
[-in_ports string]
[-nworst integer]
[-out_ports string]
[-slew_propagation {worst | path_based}]
[-type {arc | slack}]
[-verbose]
[-view string]
-setup string | -hold string
```

Writes the interface timing characteristics of the design in the specified timing model report file.

The report file contains the following sections:

- `Worst-Case Arc/Slack`: Contains details about the extracted arcs and the slack or delay across them depending on the argument specified with the `-type` option.
- `Transition Time`: Contains information about transition time at ports.
- `Capacitance`: Contains the capacitance values for the ports.
- `Design Rules`: Contains the design rules applied to the ports.

The command reports unconstrained arcs delays by default.

Parameters

`-help`	Outputs a brief description that includes type and default information for each `report_timing_model` parameter. For a detailed description of the command and all of its parameters, use the man command: `man report_timing_model`.
`-hold` *string*	Writes the paths corresponding to early analysis.
model_timing_report_file	Specifies the name of the interface timing model report file that will be generated with the interface timing characteristics of the loaded design.

`-in_ports` *string*	Restricts the `report_timing_model` report to the input ports specified with this parameter. As a result, the software considers the timing paths related to the specified input ports only, while generating the report. If this parameter is not specified, all the input ports in the design are considered while generating the report.
`-nworst` *integer*	Specifies the number of paths to be enumerated for each end point. **Note:** The extracted model will contain greater number of setup/hold paths per end point because it will have lesser end points compared to the original netlist. Therefore, it is recommended that you use a greater `-nworst` value such as `40` while writing a timing report for the extracted model to derive better correlation.
`-out_ports` *string*	Restricts the `report_timing_model` report to the output ports specified with this parameter. As a result, the software considers the timing paths related to the specified output ports only, while generating the report. If this parameter is not specified, all the output ports in the design are considered while generating the report.
`-setup` *string*	Writes the paths corresponding to late analysis.
`-slew_propagation {worst | path_based}`	Sets the slew propagation mode to use for dumping the timing characteristics of the design.
`-type {arc | slack}`	Determines whether to report arc delay values (`arc`) or the worst-slack values (`slack`).
`-verbose`	Prints the command progress details.
`-view` *string*	Specifies a view name. The report can be generated for a specific view.

Examples

- The following command generates a detailed report of the worst-slack values for the specified view:

```
tempus > report_timing_model rep2.lib -verbose -type slack -view FN_2max
```

```
# Section 1 : Worst Case Slacks
# Analysis Mode : setup
# Description : worst case slacks
#
# From    To                    Arc Type                          Transition   Slack
 View
-------------------------------------------------------------------------------
-----------
BTN_clk   wrf2_clk_o           max_sequential_uncons_arrival   fall/fall   9.615
 FN_2max
BTN_clk   wrf2_clk_o           max_sequential_uncons_arrival   rise/rise   0.000
 FN_2max
dvWe_GI   gci_aon_clk          setup                           fall/rise   38.838
FN_2max
dvWe_GI   gci_aon_clk          setup                           rise/rise   37.977
FN_2max
m_PBG    {wl_gpio_out_por[0]}  max_sequential_uncons_arrival   rise/fall   6.767
 FN_2max
-------------------------------------------------------------------------------
-----------
```

- The following command generates a detailed report of the arc delay values for the specified view:

```
tempus > report_timing_model  rep3.lib   -verbose  -type arc   -view  FN_2max
-------------------------------------------------------------------------------
-----------
# Section 1 : Worst Case Arcs
# Analysis Mode : setup
# Description : worst case arcs
#
# From   To    Arc Type    Transition   Arc-Delays    View
-------------------------------------------------------------
# Section 2 : Transition time
# Analysis Mode : setup
```

```
# Description : Actual transition times on the ports
# View : FN_2max
#
# Port            RiseTran    FallTran
---------------------------------------
RF_TI2G_GP        0.0040      0.0040
RF_TI5G           0.0040      0.0040
mux_ft_v          0.0040      0.0040
dvadr_GI[10]      0.0040      0.0040
dvadr_GI[9]       0.0040      0.0040
dvadr_GI[8]       0.0040      0.0040
dvadr_GI[7]       0.0040      0.0040
dvadr_GI[6]       0.0040      0.0040
```

- The following command extracts the interface timing characteristics of the currently loaded design and writes out the report file named `sample.rpt` with the arc delay information:

```
tempus > report_timing_model -type arc sample.rpt

------------------------------------------------------------------------
# Section 1 : Worst Case Arc
# Analysis Mode : setup
# Description : worst case arcs
#
# From To Arc Type Transition Arc-Delays
------------------------------------------------------------------------
in CLK3 setup fall/rise 0.134
in CLK3 setup rise/rise -0.005
------------------------------------------------------------------------
# Section 2 : Transition time
# Analysis Mode : setup
# Description : Actual transition times on the ports
#
# Port RiseTran FallTran
------------------------------------------------------------------------
in 0.000 0.000
clk 0.000 0.000
```

```
out 0.056 0.051
------------------------------------------------------------------------
# Section 3 : Capacitance
# Analysis Mode : setup
# Description : Total Cap on the ports
#
# Port CTotal(Rise) CTotal(Fall)
------------------------------------------------------------------------
in 0.002 0.002
clk 0.002 0.002
out 0.000 0.000
------------------------------------------------------------------------
# Section 4 : Design Rules
# Analysis Mode : setup
# Description : Design Rules on the ports
#
# Port MaxCap MinCap MaxTrans MaxFanout Fanout Limit
------------------------------------------------------------------------
in     NA     NA    NA        NA        NA
clk    NA     NA    NA        NA        NA
out    NA     NA    NA        NA        NA
```

- In MMMC mode, the `report_timing_model` command displays the following report output:

```
report_timing_model -type arc sample.rpt
```

```
------------------------------------------------------------------------------
# Section 1 : Worst Case Arcs
# Analysis Mode : setup
# Description : worst case arcs
#
# From   To           Arc Type      Transition   Arc-Delays    View
------------------------------------------------------------------------------
vclk2    tdigit[0]    max_sequential rise/fall   2.676   mission+worst-rcMax
vclk2    tdigit[0]    max_sequential rise/rise   2.831   mission+worst-rcMax
vclk2    tdigit[1]    max_sequential rise/fall   2.413   mission+worst-rcMax
vclk2    tdigit_flag  max_sequential rise/rise   2.58    mission+worst-rcMax
vclk2    tdigit[0]    max_sequential rise/fall   2.676   test+worst-rcMax
```

```
vclk2    tdigit[0]    max_sequential rise/rise   2.831   test+worst-rcMax
vclk2    tdigit[1]    max_sequential rise/fall   2.413   test+worst-rcMax
-------------------------------------------------------------------------
# Section 2 : Transition time
# Analysis Mode : setup
# Description : Actual transition times on the ports
# View : default
#
# Port                   RiseTran   FallTran
-------------------------------------------------------------------------
reset                     0.120     0.120
int                       0.120     0.120
port_pad_data_in[15]      0.120     0.120
port_pad_data_in[0]       0.120     0.120
scan_en                   0.120     0.120
scan_clk                  0.120     0.120
scan_in_1                 0.120     0.120
scan_in_2                 0.120     0.120
test_mode                 0.120     0.120
spi_data                  0.120     0.120
spi_fs                    0.120     0.120
refclk                    0.120     0.120
vcom                      1.025     1.138
-------------------------------------------------
# Section 3 : Capacitance
# Analysis Mode : setup
# Description : Total Cap on the ports
# View : default
#
# Port               CTotal(Rise) CTotal(Fall)
-------------------------------------------------
reset                    3.078     3.078
port_pad_data_in[0]      3.078     3.078
scan_en                  3.078     3.078
scan_clk                 3.078     3.078
scan_in_1                3.078     3.078
vcop                     0.007     0.007
-------------------------------------------------
# Section 4 : Design Rules
# Analysis Mode : setup
# Description : Design Rules on the ports
# View : default
#
```

```
# Port                MaxCap  MinCap  MaxTrans  MaxFanout  Fanout  Limit
-------------------------------------------------------------------------
reset                   NA      NA      2.700     15.000     NA
int                     NA      NA      2.700     15.000     NA
port_pad_data_in[15]    NA      NA      2.700     15.000     NA
port_pad_data_in[14]    NA      NA      2.700     15.000     NA
port_pad_data_in[13]    NA      NA      2.700     15.000     NA
port_pad_data_in[12]    NA      NA      2.700     15.000     NA
```

Related Commands

- check_timing_model
- write_timing_model

write_timing_model

```
write_timing_model
[-help]
model_file
[-absolute_tolerance_validation float]
[-absolute_tolerance_optimistic_validation float]
[-absolute_tolerance_pessimistic_validation float]
[-assertions constraint filename]
[-cell_name cell_name]
[-clock_transitions {clk_slew1 clk_slew2...}]
[-gain integer]
[-include_aocv_weights]
[-include_power_ground]
[-include_power]
[-input_transitions {input_slew1 input_slew2...}]
[-lib_name lib_name]
[-max_num_loads value]
[-max_num_transitions value]
[-output_loads {output_load1 output_load2...}]
[-percent_tolerance_validation float]
[-percent_tolerance_optimistic_validation float]
[-percent_tolerance_pessimistic_validation float]
[-precision integer]
[-resolution float]
[-tolerance float]
[-validation_dir string]
[-verilog_shell_file filename]
[-verilog_shell_module top_module_name]
[-view string]
```

Writes the interface timing characteristics of the design in the specified timing model output file.

Parameters

`-help`	Prints out the command usage.
model_file	Specifies the name of the output file.

`-absolute_tolerance_validation` *float*	Specifies the absolute tolerance value to be used during the validation of extracted ETM. The integer value specified with this parameter is read as ns/ps depending upon the time unit. You can use this parameter when the `-validation_dir` is used with model extraction. For more details refer to `check_timing_model -absolute_tolerance` command. *Default*: 3ps
`-absolute_tolerance_optimistic_validation` *float*	Specifies the absolute tolerance limit for flagging an optimistic failure. An optimistic path violating the absolute as well as percentage tolerance limit will be flagged as FAIL in the `check_timing_model` command output report. You can use this parameter when the `-validation_dir` is used with model extraction. *Default*: 15ps
`-absolute_tolerance_pessimistic_validation` *float*	Specifies the absolute tolerance limit for flagging a pessimistic failure. A pessimistic path violating the absolute as well as percentage tolerance limit will be flagged as FAIL in the `check_timing_model` command output report. You can use this parameter when the `-validation_dir` is used with model extraction. *Default*: 15ps
`-assertions` *constraint filename*	Writes out the constraints for model, which are used in the validation flow.
`-cell_name <`*cell_name*`>`	Specifies the cell name used in the timing model

`-clock_transitions {<clk_slew1 clk_slew2>...}`	Specifies a range of slew values used at all clock pins.
`-gain <integer>`	Represents an increase in the number of delay arcs if a pin is removed.
`-include_aocv_weights`	Writes out the stage weights on individual arcs.
`-include_power`	Writes information about power consumed by a block.
`-include_power_ground`	Writes out the power ground information for pins.
`-input_transitions {<input_slew1 input_slew2>...}`	Specifies a range of slew values used at all the data input ports.
`-lib_name <lib_name>`	Specifies the library name used in the timing model.
`-max_num_loads <value>`	Specifies the maximum number of output load values.
`-max_num_transitions <value>`	Specifies the maximum number of input slews or clock slews.
`-output_loads {<output_load1 output_load2>...}`	Specifies a range of load values used at all output ports.
`-percent_tolerance_validation float`	Specifies the percentage tolerance value to be used during the validation of extracted ETM. For more details refer to the `check_timing_model -percent_tolerance` command details. You can use this parameter when the `-validation_dir` is used with model extraction. *Default*: 3%

`-percent_tolerance_optimistic_validation` *float*	Specifies the percentage tolerance limit for flagging an optimistic failure. An optimistic path violating percentage as well as absolute tolerance limit will be flagged as FAIL in the `check_timing_model` command output report. You can use this parameter when the `-validation_dir` is used with model extraction. *Default*: 3%
`-percent_tolerance_pessimistic_validation` *float*	Specifies the percentage tolerance limit for flagging a pessimistic failure. A pessimistic path violating percentage as well as absolute tolerance limit will be flagged as FAIL in the `check_timing_model` command output report. You can use this parameter when the `-validation_dir` is used with model extraction. *Default*: 3%
`-post_check`	Checks the generated ETM library using Valus.
`-precision` *integer*	Specifies the floating point precision used for the timing model
`-resolution` *float*	Specifies the magnitude of delay difference that can be ignored in extraction
`-tolerance` *float*	Specifies the accuracy level of extractor in percent
`-validation_dir <`*string*`>`	Enables the validation of generated ETM for corresponding view and displays number of failures at the end of validation. It is mandatory to specify the name of validation directory with this option (`-validation_dir valDir`), this contains the write_model_timing, check_timing_model and summary reports.
`-verilog_shell_file` *filename*	Generates a Verilog shell file that instantiates the timing model

`-verilog_shell_module` *`top_module_name`*	Specifies the top module name of the Verilog shell file.
`-view` *`string`*	Specifies the view for which model needs to extracted.

Examples

- The following command builds a `.lib` model (`block.lib`) for the block:

  ```
  tempus > write_timing_model block.lib
  ```

- The following command creates the timing model for the `tst_model` cell belonging to the `tst_lib` library, and writes the model into the `tst.lib` liberty file:

  ```
  tempus > write_timing_model -cell_name tst_model -library_name tst_lib tst.lib
  ```

- The following command validates the generated ETM, where the pessimistic and optimistic path violation tolerance is set to 1%:

  ```
  write_timing_mode tst.lib -cell_name block_1  -validation_dir -
  percent_tolerance_validation 1 -percent_tolerance_optimistic_validation 1
      \ -percent_tolerance_pessimistic_validation 1
  ```

- The following command includes details related to the power ground (of the pins) and power consumed by a block:

  ```
  write_timing_mode tst.lib -cell_name block_1 -include_power -include_power_ground
  ```

Related Commands

- `check_timing_model`

Prototype Timing Model - PTM

- create_ptm_constraint_arc
- create_ptm_delay_arc
- create_ptm_drive_type

- create_ptm_generated_clock
- create_ptm_load_type
- create_ptm_model
- create_ptm_path_type
- create_ptm_port
- report_ptm_model
- save_ptm_model
- set_ptm_design_rule
- set_ptm_global_parameter
- set_ptm_port_drive
- set_ptm_port_load

create_ptm_constraint_arc

```
create_ptm_constraint_arc
-setup
-hold
-from clock_port_name ]
[-to port_name ]
-edge { rise | fall }
-path_type pathTypeName
- path_factor value
[-value constraint_value ]
```

Sets the timing check from the clock input port to the data input port.

To create a new timing check arc which does not already exist in the timing model, the clock port that you specify as `-from` should be created as a clock type by the `create_ptm_port` command.

Note: The `create_ptm_constraint_arc` command can be specified only after you create a PTM model using the `create_ptm_model` command.

Parameters

`-edge { rise | fall }`	Specifies which transition on the clock input is relevant for the timing check arc definition.
`-from` *`clock_port_name`*	Specifies the name of the clock port, which can be an input, bidirectional, or internal port. **Note:** This parameter allows you to specify a pattern for port names using wildcard characters.
`-hold`	Specifies that the check arc to be created is of `hold` type.
`-path_type` *`pathTypeName`*	Specifies the reference time arc created using the `create_ptm_path_type` command. This parameter is mutually exclusive to the `-value` parameter currently available. Either the `-value` parameter or the `-path_type` parameter must be specified.
`-path_factor` *`value`*	Used as an additional scaling value for the delay/slew tables. *Default*: `1.0`
`-setup`	Specifies that the check arc to be created is of `setup` type.

`-to` *port_name*	Specifies the name of the input or bidirectional port which will be the signal pin for check arc. If you do not specify a port name, the software sets the timing check for all input and bidirectional data ports. **Note:** This parameter allows you to specify a pattern for port names using wildcard characters.
`-value` *constraint_value*	Specifies a value for the timing check.

Example

- The following command sets the setup timing check value of `0.3` with respect to rising edge at the clock port `clock_in` to the data input port `data_in`:

```
tempus> create_ptm_constraint_arc -setup -value 0.3 -from clock_in -to data_in -edge rise
```

- The following command specifies the reference check arc by specifying the path type as `SETUP_TYPE1` and path factor as `1.03`:

```
tempus> create_ptm_constraint_arc -from clock_gen_1 -to data_in -path_type SETUP_TYPE1 -path_factor 1.03
```

Related Commands

- `create_ptm_port`

create_ptm_delay_arc

```
create_ptm_delay_arc
-from port_name
-path_type pathTypeName
-path_factor value
-to port_name
[-edge { rise | fall }]
-value delay_value
```

Sets the delay for combinational or sequential paths from input ports to the output ports. If you do not specify the drive type on the output port, a constant timing table is created. In a constant timing table, delay is constant. It does not vary with respect to input slew and output capacitance.

To create a new sequential timing arc which does not already exist in the timing model, the port you specify as `-from` should be created as a clock type by the `create_ptm_port` command.

Note: The `create_ptm_delay_arc` command can be specified only after you create a PTM model using the `create_ptm_model` command.

Parameters

`-edge rise | fall`	Specifies the edge on the clock input relevant to the sequential delay arc definition. By default, the software uses a rising clock transition.
`-from` *port_name*	Specifies the name of the input port or the clock port. **Note:** This parameter allows you to specify a pattern for port names using wildcard characters.
`-path_type` *pathTypeName*	Specifies the reference time arc created using the `create_ptm_path_type` command. This parameter is mutually exclusive to the `-value` parameter. Either the `-value` parameter or the `-path_type` parameter must be specified.
`-path_factor` *value*	Used as an additional scaling value for the delay/slew tables. *Default*: `1.0`
`-to` *port_name*	Specifies the name of the output port or the bidirectional port. **Note:** This parameter allows you to specify a pattern for port names using wildcard characters.
`-value` *delay_value*	Specifies the value for the delay.

Example

- The following command defines a sequential delay arc with respect to the rising edge at the clock port `clock_in` to the output port `data_out` and specifies the delay value as `1.0`:

```
tempus> create_ptm_delay_arc -from clock_in -to data_out -edge rise -value 1.0
```

- The following command specifies the reference check arc by specifying the path type as `SETUP_TYPE1` and path factor as `1.03`:

```
tempus> create_ptm_path_type SEQ_TYPE -type sequential -lib_cell DFFHQX1 -
relative_pin CK -target_pin Q
tempus> create_ptm_delay_arc -from ptm_clk -to ptm_reg_out -path_type SEQ_TYPE
```

Related Commands

- `create_ptm_port`

create_ptm_drive_type

```
create_ptm_drive_type
-lib_cell cell_name
[-input_slew_fall value ]
[-input_slew_rise value ]
[-input_pin pin_name ]
[-output_pin pin_name ]
drive_type_name
```

Creates drive types to specify the drive strength equivalent to existing library cells. Optionally, you can also specify the rise or fall transition value(s) at the driver port using this command.

This command can be specified only after you create a PTM model using the `create_ptm_model` command.

Note: Use the `set_ptm_port_drive` command to associate the drive type to existing ports.

Parameters

drive_type_name	Specifies the name of the drive type to use for the timing model.
`-lib_cell` *cell_name*	Specifies the reference cell which will be used to determine the drive value.
`-input_pin` *pin_name*	Specifies the input pin of the timing arc for the reference cell specified with the `-lib_cell` parameter. The combinational arcs starting from this pin will be used to create the drive type. When only one of `-input_pin`/`-output_pin` parameter is specified, the tool chooses the first random combinational arc at the specified pin. When none of these options are specified, the tool chooses the first random combinational arc.
`-input_slew_fall` *value*	Specifies the fall transition value.
`-input_slew_rise` *value*	Specifies the rise transition value.

`-output_pin` *pin_name*	Specifies the output pin of the timing arc for the reference cell specified with the `-lib_cell` option. The combinational arcs ending to this pin will be used to create the drive type. When only one of `-input_pin`/`-output_pin` parameter is specified, the tool chooses the first random combinational arc at the specified pin. When none of these options are specified, the tool chooses the first random combinational arc.

Example

- The following command defines the drive type name as `stdDrive` with drive resistance of library cell `INVX1`:

  ```
  tempus> create_ptm_drive_type -lib_cell INVX1 stdDrive
  ```

- The following command defines the drive type name as `stdDrive` with drive resistance of the library cell `BUF1`, and input and output pins as `B` and `Y`, respectively:

  ```
  tempus> create_ptm_drive_type -lib_cell BUF1 -input_pin B -output_pin Y stdDrive
  ```

Related Commands

- `create_ptm_model`
- `set_ptm_port_drive`

create_ptm_generated_clock

```
create_ptm_generated_clock
-source clock_pin
[-divide_by value | -multiply_by value ]
clock_name
```

Creates an internal pin on a timing model or uses an existing internal pin (defined by the `create_ptm_port` command), and creates a generated clock on this internal pin.

The following points apply to the usage of this command:

- The internal pin created with this command will be treated as a clock port. This internal pin can, therefore, be used to create timing check arcs using the `create_ptm_constraint_arc` command and sequential delay arcs using the `create_ptm_delay_arc` command.
- The latency of the generated clock is set to `0`.
- Any timing arc related to the internal pin must include the clock latencies defined on both: the internal pin and the master pin.
- The generated clock appears in the timing model generated by the `save_ptm_model` command.
- The internal pin and the generated clock are persistent, that is, they cannot be deleted using PTM commands.

Parameters

clock_name	Specifies the name of the generated clock defined on the internal pin.
-divide_by *value* |- multiply_by *value*	Specifies the frequency division or multiplication factor. When you use the `-divide_by` option, the generated clock period is equal to the master clock period multiplied by the value specified with the `-divide_by` option. For example, if the value is `2`, the generated clock period is two times the master clock period. When you use the `-multiply_by` option, the generated clock period is equal to the master clock period divided by the value specified with the `-multiply_by` option. For example, if the value is `3`, the generated clock period is one-third the master clock period.

`-source` *`clock_pin`*	Specifies the source from which the derived clock is generated. The source can be an existing clock port or the one created by using the `create_ptm_port` command, and a clock should be available on this port. This clock port is set at the master pin for the generated clock.

Example

- The following command creates a generated clock with the name `clock_gen` on internal pin `clock_gen` (also create the pin if it does not already exist), using the clock port `clock_in` as source pin for the generated clock, and the frequency division factor of `2`:

```
tempus> create_ptm_generated_clock -source clock_in -divide_by 2 clock_gen
```

Related Commands

- `create_ptm_constraint_arc`
- `create_ptm_delay_arc`

create_ptm_load_type

```
create_ptm_load_type
-lib_cell lib_cell_name
[-input_pin lib_pin_name]
load_type_name
```

Creates a load type, which is identified by a unique *load_type_name*. Typically, the load type represents a capacitance value from a library input or bidirectional pin. You can create multiple load types for a PTM model provided you use different `create_ptm_load_type` commands for each load type.

Note: The load type can be associated with PTM ports by using the `set_ptm_port_load` command.

Parameters

`-lib_cell` *lib_cell_name*	Specifies the reference cell which will be used to derive the load characteristics.
`-input_pin` *lib_pin_name*	Specifies the name of the input or bidirectional pin which will be used to derive the load characteristics. **Note:** When this option is not used, the software randomly selects an input or bidirectional pin from the library cell.
load_type_name	Specifies a unique name for the load type.

Example

- The following command defines a load type called `stdLoad` for the `INVX1` cell and the input pin `A`:

  ```
  tempus> create_ptm_load_type -lib_cell INVX1 -input_pin A stdLoad
  ```

Related Commands

- `set_ptm_port_load`

create_ptm_model

`create_ptm_model` *`model_name`*

Initiates the creation of a Prototype Timing Model (PTM) model. A PTM model contains preliminary information about the block, such as ports, delay arcs, and timing check arcs.

Note: You must save the existing model details using the `save_ptm_model` command before creating another model. This means that you cannot generate multiple models at the same time.

Parameters

`model_name`	Specifies a name for the PTM model.

Example

- The following command creates a PTM model named `sample_model`:

  ```
  tempus> create_ptm_model sample_model
  ```

Related Commands

- `report_ptm_model`
- `save_ptm_model`

create_ptm_path_type

```
create_ptm_path_type
[-help]
-type arc_type
-lib lib_name
-lib_cell lib_cell_name
-relative_pin pin_name
-target_pin pin_name
path_type_name
```

Uniquely identifies the worst time arc of the given type between a pin pair for a given library cell when `-target_pin` and `-relative_pin` has been specified. If only either of `-target_pin` or `-relative_pin` is specified, the software finds out the worst library arc seen with type as provided with `-type` parameter. If no pin is specified, then the first worst arc with type as specified with `-type` parameter is selected. The created path type name is used later with the `create_ptm_constraint_arc` and `create_ptm_delay_arc` commands to create the reference time arc inside the PTM model. Newly created arc will inherit the rise/fall delay/ slew LUTs after applicable scale specified by `-path_factor` in `create_ptm_constraint_arc`/`create_ptm_delay_arc` and the unateness property from the reference arc.

Parameters

`-help`	Prints out the command usage.
`path_type_name`	Specifies the name to uniquely identify a reference timing arc specification (path type) which can be used for multiple PTM creation later.
`-type` *`arc_type`*	Provides the arc type from which the reference library is to be picked up. This reference library will be used as the reference arc for this path type. Possible type values are: `setup`, `hold`, `combinational`, and `sequential`.
`-lib` *`lib_name`*	Specifies the reference library name.
`-lib_cell` *`lib_cell_name`*	Specifies the reference cell name.
`-relative_pin` *`pin_name`*	Specifies the relative pin (that is, reference pin for check arcs and source pin for delay arcs) for picking the reference arc.
`-target_pin` *`pin_name`*	Specifies the target pin (that is, signal pin for check arcs and sink pin for delay arcs) for picking the reference arc.

Example

- The following command specifies the reference check arc from relative pin `CLK1` to the target pin `D1`:

```
tempus> create_ptm_path_type SETUP_TYPE1 -lib_cell DFFX1 \
-relative_pin CLK -target_pin D1 -type setup
```

Related Commands

- `create_ptm_constraint_arc`
- `create_ptm_delay_arc`

create_ptm_port

```
create_ptm_port
[-help]
[-clock]
[port_list]
[-type {output | inout | input | internal | clock}]
```

Creates a PTM port for the current model.

Parameters

`-help`	Prints out the command usage.
`-clock`	Specifies that the provided port is a clock port. The software allows clock creation at an internal pin of PTM with another internal pin as a source.
port_list	Specifies a port names of the type specified with the `-type` option.
`-type {output | inout | input | internal | clock}`	Specifies the type of port to create. You can use one of the following values to specify the type of port: • `output` • `inout` • `input` • `internal` • `clock`

Examples

- The following command creates a bidirectional port named `bidiA`:
  ```
  tempus > create_ptm_port -type inout bidiA
  ```

Related Commands

- create_ptm_load_type

report_ptm_model

```
report_ptm_model
[-path_types]
[-ports]
[-delay_param]
[-arcs]
[-arc_type]
```

Generates a report on ports, arcs, and global delay parameter details for the current PTM model. This command must be used after the `create_ptm_model` command and before the `save_ptm_model` command.

Parameters

`-arcs`	Displays the timing arc details for the current model, such as the input port, output port, and the delay value associated with the timing arc.
`-arc_type`	Reports the type of arc. When specified, the command output displays the `Arc Type` column.
`-delay_param`	Displays the global delay parameter details set using the `set_ptm_global_parameter` command.
`-path_types`	Reports only the path type details. *Default*: Includes the details of path types created.
`-ports`	Displays the port details from the current PTM model that were set using PTM commands.

Example

- The following command displays a report on the arcs of the `model1` PTM model:

```
tempus> report_ptm_model -arcs
ModelName  : model1
Arc                 From        To         Delay      PathType   PathFactor
---                 ----        --         -----      --------   ----------
refclk_to_spi_fs    refclk      spi_fs     0.3        -          -
spi_fs_to_scan_se   spi_fs      scan_se    1.0        -          -
```

- The following command displays a report on the type of arc of the `model1` PTM model.

```
report_ptm_model -arc_type
ModelName  : model1

DriveType    Cell
---------    ----
stdDrive     pllclk
PathType     Cell       Arc Type
--------     ----       --------
 No path type defined.
```

- The following command displays a report on the global delay parameters of the model1 PTM model:

```
report_ptm_model -delay_param
ModelName  : model1
DelayParameter       Delay
--------------       -----
setup                -
hold                 -
clk_to_output        -
```

- The following command displays a report on the port details of the `model1` PTM model:

```
report_ptm_model -ports
ModelName  : model1
DriveType    Cell
---------    ----
stdDrive     pllclk

Port         Direction  BusRange   LoadValue  LoadType   DriveValue  DriveType
----         ---------  --------   ---------  --------   ----------  ---------
refclk       clock      -          -          -          -           -
scan_se      out        -          -          -          -           -
```

```
spi_fs       in          -           -           -           -           -
```

- The following command displays a report on the global delay parameters of the `dma` PTM model:

```
tempus> report_ptm_model -delay_param


ModelName : dma
DelayParameter Delay
-------------- -----
setup 1.2
hold 1.5
clk_to_output 1.6
```

- The following command reports the path type details for the SETUP_TYPE1 path type.

```
tempus> create_ptm_model <name>
:
tempus> create_qtm_port -type clock CLOCK
tempus> create_qtm_port -type input IN
:
tempus> create_ptm_path_type SETUP_TYPE1 -lib_cell typ_1.2/DFFX1 \
-relative_pin CLK -target_pin D1
tempus> create_ptm_path_type HOLD_TYPE1 -lib_cell typ_1.2/DFF2X1 \
-relative_pin CLK -target_pin D2
tempus> create_ptm_constraint_arc -from clock_gen_1 -to data_in -setup \
-edge fall -path_type SETUP_TYPE1
```

Related Commands

- create_ptm_model
- save_ptm_model

save_ptm_model

```
save_ptm_model
[-help]
[-output filename_prefix]
[-library_cell]
```

Saves the PTM model including the timing arc specifications in the .lib library file format. The file name contains the PTM cell with the pin and bus definitions, and various constraints and delay arcs that are created by `create_ptm_*` command set.

Parameters

`-help`	Prints out the command usage.
`-library_cell`	Specifies the reference cell which will be used to derive the model characteristics.
`-output` *filename_prefix*	Specifies the output file name. If a file name is not specified, the model name defined using the `create_ptm_model` command is used.

Examples

- The following command specifies a PTM model file name:

  ```
  tempus > save_ptm_model -output ptm.lib
  ```

Related Commands

- `create_ptm_model`
- `report_ptm_model`

set_ptm_design_rule

```
set_ptm_design_rule
{-max_transition <max_tran> | -max_capacitance <max_cap> | -max_fanout <max_fanout>}
[-port port_list]
```

Specifies a value for the `max_transition`, `max_capacitance`, and `max_fanout` design constraints. These values can be applied to all the ports or a selected list of ports.

Parameters

`-max_capacitance` *max_cap*	Specifies the `max_capacitance` value for the ports in the PTM model.
`-max_fanout` *max_fanout*	Specifies the `max_fanout` value for the ports in the PTM model.
`-max_transition` *max_tran*	Specifies the `max_transition` value for the ports in the PTM model.
`-port` *port_list*	Specifies a list of PTM port names on which the value specified with the `-max_capacitance`, `-max_fanout`, or `-max_transition` parameters should be applied. **Note:** This parameter allows you to specify a list of patterns for port names using wildcard characters.

Example

- The following commands specify a global `max_transition` value of `1.0` for all ports except `trigger`, which will use the `max_transition` value of `1.1`:

```
tempus> set_ptm_design_rule -max_transition 1.0
tempus> set_ptm_design_rule -max_transition 1.1. -port trigger
```

set_ptm_global_parameter

```
set_ptm_global_parameter
-param {setup | hold | clk_to_output}
-value real_number
```

Defines global values for setup check arcs, hold check arcs, or sequential delay arcs across the PTM model. This global value is added to the arc specific delay value. As a result, the final arc's delay is calculated as a sum of the global parameter value and the arc delay specified using the `create_ptm_constraint_arc` or the `create_ptm_delay_arc` command.

Note: The global parameters must be specified before defining the first arc in the current PTM model.

Parameters

`-param {setup | hold | clk_to_output}`	Specifies the name of a global parameter, which can be one of the following: • `setup` for setup check arcs. • `hold` for hold check arcs. • `clk_to_output` for sequential delay arcs.
`-value real_number`	Defines a value for the specified global parameter name.

Example

- The following command specifies `setup` as the global parameter and assigns the value `1.2` to it:

  ```
  tempus> set_ptm_global_parameter -param setup -value 1.2
  ```

Related Commands

- `create_ptm_constraint_arc`
- `create_ptm_delay_arc`

set_ptm_port_drive

```
set_ptm_port_drive
-type drive_type
[-input_slew_rise rise_slew_value]
[-input_slew_fall fall_slew_value]
port_list
```

Sets the drive for the output and bidirectional ports. The *drive_type* specifies the type of cell within the model as drivers on the interface nets. You cannot specify two different drivers for two timing arcs ending at the same output port.

Parameters

port_list	Specifies the name of the output or bidirectional port. **Note:** This parameter allows you to specify a list of patterns for port names using wildcard characters.
-input_slew_fall *fall_slew_value*	Overrides the fall transition value specified with the -input_slew_fall option of the create_ptm_drive_type command.
-input_slew_rise *rise_slew_value*	Overrides the rise transition value specified with the -input_slew_rise option of the create_ptm_drive_type command.
-type *drive_type*	Specifies the drive type to use for the timing model.

Examples

- The following command sets the stdDrive drive to be used on the output data port data_out:

  ```
  tempus> set_ptm_port_drive -type stdDrive data_out
  ```

Related Commands

- create_ptm_drive_type

set_ptm_port_load

```
set_ptm_port_load
-type load_type_name | -value real_number
[-multiply_by multiply_factor]
port_list
```

Defines a load for PTM model ports by either specifying a capacitance value or by using the load type defined by the `create_ptm_load_type` command.

Parameters

`-type` *load_type_name*	Specifies the load type name created by the `create_ptm_load_type` command.
`-multiply_by` *multiply_factor*	Specifies the multiplication factor that will be used to calculate the load on the specified ports, as shown: `Load on port(s) = {(load_value) * (factor)}` Where: • *load_value* is the capacitance value derived from the cell name specified with the `-lib_cell` option of the `create_ptm_load_type` command when you use the `-type` option. • *factor* is the multiplication factor.
`-value` *real_number*	Specifies the value to be used as load/capacitance for the list of ports specified. The value specified with this option overrides the load value that is derived by the `-type` option. **Note:** This option cannot be used with the `-multiply_by` option.
port_list	Specifies the list of input or bidirectional ports that exist in the PTM model defined by the `create_ptm_port` command. **Note:** This parameter allows you to specify a list of patterns for port names using wildcard characters.

Example

- The following command defines a load for the PTM model port `data_in_2` by specifying a capacitance value of `0.15`:

  ```
  tempus> set_ptm_port_load -value 0.15 data_in_2
  ```

- The following command specifies a load type called `stdLoad` and a multiplication factor of `2` for the `data_in` port:

  ```
  tempus> set_ptm_port_load -type stdLoad -multiply_by 2 data_in
  ```

Related Commands

- `create_ptm_load_type`
- `create_ptm_port`

Index

A B C D E F G H I J L M O P Q R S T U V W Z

A

add_fillers_cells
add_fillers_check_trim_rule
add_fillers_double_height_filler_insertion
add_fillers_enable_restore_filler_flow
add_fillers_prefix
add_pg_netlist_power_intent
add_route_vias_auto
add_route_vias_ndr_only
add_thru_substrate_insts
add_to_collection
all_clocks
all_fanin
all_fanout
all_inputs
all_instances
all_outputs
all_registers
analyze_ir_paths
analyze_jitter
analyze_paths_by_bottleneck
analyze_paths_by_clock_domain
analyze_paths_by_critical_false_path
analyze_paths_by_default_path_group
analyze_paths_by_drv
analyze_paths_by_hier_port
analyze_paths_by_hierarchy
analyze_paths_by_view
append_to_collection
auto_file_dir
auto_file_prefix

B

bindKey

C

checkDesign
check_ac_limit_additional_individual_violation_report
check_ac_limit_avg_recovery
check_ac_limit_check_thermal_aware_em
check_ac_limit_current_file
check_ac_limit_current_scale_factor
check_ac_limit_current_scale_table
check_ac_limit_default_freq_for_unconstrained_nets
check_ac_limit_delta_temperature
check_ac_limit_delta_temperature_layer_list
check_ac_limit_detailed
check_ac_limit_effort_level
check_ac_limit_em_cdf_percentage
check_ac_limit_em_limit_scale_factor
check_ac_limit_em_limit_scale_table
check_ac_limit_em_res_width
check_ac_limit_em_temperature
check_ac_limit_em_temperature_layer_list
check_ac_limit_em_threshold
check_ac_limit_enable_seb
check_ac_limit_env_temperature
check_ac_limit_extraction_tech_file
check_ac_limit_force_hold_view
check_ac_limit_handle_pin_obs_via
check_ac_limit_ict_em_models
check_ac_limit_ircx_models
check_ac_limit_lifetime
check_ac_limit_lifetime_rating_factor_file
check_ac_limit_max_error
check_ac_limit_method
check_ac_limit_min_peak_duty_ratio
check_ac_limit_min_peak_freq
check_ac_limit_net_file
check_ac_limit_nets
check_ac_limit_out_file
check_ac_limit_peak_td_method
check_ac_limit_read_detail_delta_temperature_file
check_ac_limit_report_db
check_ac_limit_report_unannotated_shapes
check_ac_limit_seb_lifetime
check_ac_limit_seb_table
check_ac_limit_seb_temperature
check_ac_limit_selected
check_ac_limit_simulation_net
check_ac_limit_simulation_net_file
check_ac_limit_skip_category_mode
check_ac_limit_skip_net
check_ac_limit_skip_net_file
check_ac_limit_toggle
check_ac_limit_top_scope_ignore_block_internal_nets_on_boundary_path
check_ac_limit_use_db_freq
check_ac_limit_use_qrc_tech
check_ac_limit_use_rms_delta_t
check_ac_limit_view
check_ac_limits
check_boundary_model_consistency
check_design_across_hierarchy
check_design_report_async_pins
check_design_report_scan_pins
check_design_threshold_fanout
check_flow
check_ipd_logic
check_ldb_version
check_multi_cpu_usage

check_netlist
check_power_domains
check_power_vias
check_timing
check_timing_library
check_timing_library_consistency
check_timing_model
clear_active_logic_view
clear_all_rulers
clone_msv_gate
cmd_file
commit_boundary_model_constraints
commit_module_model
commit_power_intent
compare_oa_cellview
compare_timing_context
compress_special
connect
connect_hpin
connect_net
connect_pin
convert_dbu_to_um
convert_um_to_dbu
copy_collection
create_active_logic_view
create_aging_stress_profile
create_analysis_view
create_basic_path_groups
create_boundary_model
create_clock
create_constraint_mode
create_delay_corner
create_flow
create_flow_step
create_generated_clock
create_glitch_boundary_model
create_gui_shape
create_hinst
create_histogram_png
create_hnet
create_hport
create_ilm_from_files
create_inst
create_ipd_configuration
create_library_set
create_marker
create_metric_page
create_metric_table
create_metric_table_heading
create_module
create_net
create_oa_lib
create_oa_lib_property
create_opcond
create_parasitic_coordinate_transform
create_path_category
create_pg_net_group
create_port
create_property_alias
create_ptm_constraint_arc
create_ptm_constraint_arc
create_ptm_delay_arc
create_ptm_drive_type
create_ptm_generated_clock
create_ptm_load_type
create_ptm_path_type
create_ptm_port
create_rc_corner
create_ruler
create_snapshot
create_timing_condition
current_design
current_instance
cut_power_domain_by_overlaps

D

db_hinst_connection
decompress_special
decrypt
define_attribute
define_bundled_bus
define_feature
define_glitch_holding_resistance
define_metric
define_parameter
define_proc
define_proc_arguments
define_property
define_variables
delete_analysis_view
delete_cell_obs
delete_dangling_ports
delete_floating_constants
delete_floating_nets
delete_flow_config
delete_inst
delete_markers
delete_metric
delete_metric_header
delete_metric_page
delete_metric_run
delete_metric_table
delete_metric_table_heading
delete_nets
delete_obj
delete_path_category
delete_power_intent
delete_routes
delete_thru_substrate_insts
deselect_obj
design_backside_bottom_routing_layer
design_backside_top_routing_layer
design_bottom_routing_layer
design_compressed_pg_db
design_cong_effort
design_ideal_hold_fixing
design_merge_trim_shapes
design_optimization_density_screen_margin
design_pessimistic_mode
design_process_node
design_tech_node
design_top_routing_layer

design_trim_grid_group
disconnect
disconnect_hpin
disconnect_net
disconnect_pin
display_temperature_result
distribute_command
distribute_design_views
distribute_launch_deferred_clients
distribute_partition
distribute_print_client_usage
distribute_print_partition_stats
distribute_read_design
distribute_start_clients
distribute_views
distributed_mmmc_disable_reports_auto_redirection

E

eco_add_ports
eco_add_repeater
eco_batch_mode
eco_check_logical_equivalence
eco_client_tempus_executable
eco_def_file_list
eco_delay_calculation_effort
eco_delete_repeater
eco_drv_view_list
eco_dynamic_view
eco_hold_view_list
eco_honor_dont_touch
eco_honor_dont_use
eco_honor_fixed_status
eco_honor_fixed_wires
eco_honor_power_intent
eco_leakage_view
eco_lef_file_list
eco_prefix
eco_preserve_hpin_function
eco_refine_place
eco_select_partition_list
eco_setup_view_list
eco_si_effort
eco_update_cell
eco_update_only_layers
eco_update_timing
edit_flow
encrypt
encrypt_thermal_file
end_flow
etric_capture_design_image_route_drc
eval_legacy
extract_parasitics
extract_rc_cap_filter_mode
extract_rc_cerebrus_license_only
extract_rc_compress_rcdb
extract_rc_coupled
extract_rc_coupling_cap_threshold
extract_rc_lef_tech_file_map
extract_rc_local_cpu
extract_rc_pvs_fill
extract_rc_qrc_cmd_file
extract_rc_qrc_cmd_type
extract_rc_qrc_run_mode
extract_rc_qrc_stream_map_file
extract_rc_relative_cap_threshold
extract_rc_signoff_stream_layer_map
extract_rc_total_cap_threshold
extract_rc_tsv_subckt_file
extract_rc_turbo_reduce
extract_rc_write_def_options_for_signoff_extract

F

filter_collection
flatten_ilm
flow_branch
flow_caller_data
flow_db_directory
flow_error_errorinfo
flow_error_message
flow_error_write_db
flow_exclude_time_for_init_flow
flow_exit_when_done
flow_feature_values
flow_features
flow_footer_tcl
flow_header_tcl
flow_hier_path
flow_history
flow_log_directory
flow_log_prefix_generator
flow_mail_on_error
flow_mail_to
flow_metrics_file
flow_metrics_snapshot_parent_uuid
flow_metrics_snapshot_uuid
flow_overwrite_db
flow_plugin_canonical_current
flow_plugin_names
flow_plugin_steps
flow_post_db_overwrite
flow_remark
flow_report_directory
flow_run_tag
flow_schedule
flow_starting_db
flow_startup_directory
flow_status_file
flow_step_canonical_current
flow_step_check_tcl
flow_step_current
flow_step_last
flow_step_last_msg
flow_step_last_status
flow_step_next
flow_summary_tcl
flow_template_feature_definition
flow_template_tools
flow_template_type
flow_template_version
flow_top

flow_user_templates
flow_verbose
flow_working_directory
flow_write_db_snapshot
flow_write_db_snapshot_exclude_time
flow_yamllint_exec
flowtool_exit_timeout
flowtool_extra_arguments
flowtool_metrics_qor_excel
flowtool_metrics_qor_html
flowtool_metrics_qor_text
flowtool_metrics_qor_vivid
flowtool_predict_full_names
flowtool_schedule_flow_immediate
flowtool_summary_tcl

G

get_aging_activity
get_aging_analysis_mode
get_all_layers
get_arcs
get_capacitance_unit
get_cells
get_clock_network_objects
get_clocks
get_common_ui_map
get_computed_shapes
get_cpf_user_attributes
get_current_constraint_mode
get_db
get_db_display_limit
get_default_switching_activity
get_designs
get_distribute_host
get_distributed_variables
get_dynamic_power_simulation
get_edge
get_feature
get_flow_config
get_generated_clocks
get_ilm_type
get_legacy
get_lib_arcs
get_lib_cells
get_lib_pg_pins
get_lib_pins
get_libs
get_message
get_metric
get_metric_alias
get_metric_config
get_metric_definition
get_metric_header
get_module_view
get_multi_input_switching_mode
get_nets
get_oa_default_rule_lib
get_oa_design_data
get_obj_in_area
get_object_name
get_path_groups
get_pg_nets
get_pg_pins
get_pins
get_ports
get_power
get_preference
get_propagated_clock
get_property
get_rect
get_socv_rc_variation_factor
get_socv_reporting_nsigma_multiplier
get_time_unit
get_transform_shapes
get_via_pillars
get_visible_nets
get_wire_variation_mode
group_path
gui_add_marker
gui_add_ui
gui_attach_to_cursor
gui_bind_key
gui_clear_highlight
gui_clear_scan_chains
gui_clear_spare_cells
gui_create_user_bundle_net
gui_ctd_close
gui_ctd_get_title
gui_ctd_get_window_id
gui_ctd_highlight
gui_ctd_open
gui_ctd_set_title
gui_ctd_show_skew_group
gui_ctd_write_histogram
gui_ctd_write_view
gui_ctd_zoom
gui_delete_objs
gui_delete_ui
gui_delete_user_bundle_net
gui_deselect
gui_dim_foreground
gui_find_ui
gui_fit
gui_get_edge_number
gui_get_objects_from_schematic
gui_get_schematic
gui_get_ui
gui_get_visible_bumps
gui_get_visible_net_groups
gui_gtd_highlight_timing_report
gui_hide
gui_highlight
gui_highlight_pin
gui_list_marker
gui_move_cursor
gui_open_gtd
gui_open_schematic
gui_pan
gui_pan_center
gui_pan_page
gui_redraw

gui_remove_marker
gui_report_ruler
gui_schematic_add_selected
gui_schematic_load_view
gui_schematic_write_view
gui_select
gui_select_highlighted
gui_set_power_rail_display
gui_set_power_rail_layers_nets
gui_set_tool
gui_set_ui
gui_set_visible_bumps
gui_set_visible_net_groups
gui_set_visible_nets
gui_show
gui_show_buffer_tree
gui_show_edge_number
gui_show_markers
gui_show_scan_chains
gui_show_spare_cells
gui_show_timing_map
gui_show_user_bundle_nets
gui_verbose
gui_view_box
gui_write_picture
gui_zoom

H

handle_old_lib_with_signal_level
help

I

ilm_keep_async
ilm_keep_flatten
ilm_keep_high_fanout_ports
ilm_keep_inst_in_sdc
ilm_keep_loopback
ilm_max_num_insts
ilm_max_num_registers
ilm_slack_driven
index_collection
init_delete_floating_hnets
init_design
init_flow
init_hier_cell
init_keep_empty_modules
init_ml
init_oa_tech_lib
init_power_intent_files
init_read_netlist_allow_port_mismatch
init_read_netlist_allow_undefined_cells
init_sync_relative_path
init_timing_enabled
is_attribute
is_collection_object
is_common_ui_mode
is_feature
is_flow

J

jedai_catalog_token
jedai_catalog_url

L

list_libraries
load_failure_report
log_file
log_verbose_type
logv_file

M

man
merge_model_timing
merge_noise_reports
merge_timing_reports
merge_vtm
metric_advanced_url_endpoint
metric_capture_3d_hotspots
metric_capture_depth
metric_capture_design_image
metric_capture_design_image_blockages_exclude_layers
metric_capture_design_image_blockages_threshold
metric_capture_design_image_power_intent
metric_capture_max_drc_markers
metric_capture_min_count
metric_capture_overwrite
metric_capture_pba_tns_histogram
metric_capture_per_view
metric_capture_reg2reg_metrics
metric_capture_timing_path_groups
metric_capture_timing_paths
metric_capture_tns_histogram
metric_capture_tns_histogram_buckets
metric_capture_tns_histogram_max_slack
metric_capture_tns_histogram_paths
metric_capture_vth_metrics
metric_capture_vth_per_power_domain
metric_capture_wns_average_paths
metric_category_default
metric_current_run_id
metric_enable
metric_summary_metrics
monitor_distributed_hosts

O

oa_allow_analysis_only
oa_allow_bit_connection
oa_allow_tech_update
oa_bindkey_file
oa_cell_view_dir
oa_convert_diagonal_path_to

oa_cut_rows
oa_display_resource_file_in_library
oa_enclose_quick_abstract_pins
oa_full_path
oa_inst_placed_if_none
oa_lock
oa_merge_trim
oa_non_rectilinear_shapes
oa_push_pin_constraint
oa_read_system_reserved_layers
oa_text_purpose
oa_update_mode
oa_use_virtuoso_color
oa_write_cds_fixed_vias
oa_write_net_voltage
opt_signoff
opt_signoff_add_load
opt_signoff_allow_skewing
opt_signoff_buffer_cell_list
opt_signoff_clock_cell_list
opt_signoff_create_smart_mmmc_db
opt_signoff_disable_geometry_checks
opt_signoff_drv_margin
opt_signoff_em_type
opt_signoff_fix_cell_em
opt_signoff_fix_data_drv
opt_signoff_fix_hold_allow_setup_degrade
opt_signoff_fix_hold_allow_setup_tns_degrade
opt_signoff_fix_hold_with_margin
opt_signoff_fix_max_cap
opt_signoff_fix_osus_glitch
opt_signoff_fix_slack_in_range_lower_limit
opt_signoff_fix_xtalk
opt_signoff_fix_xtalk_preserve_setup
opt_signoff_hold_target_slack_per_view
opt_signoff_hold_xtalk_slack_threshold
opt_signoff_include_hold_path_groups
opt_signoff_inter_partition
opt_signoff_intra_partition
opt_signoff_legal_only
opt_signoff_max_cap_margin
opt_signoff_max_opt_runtime
opt_signoff_max_runtime
opt_signoff_max_transition_margin
opt_signoff_optimize_core_only
opt_signoff_optimize_sequential_cells
opt_signoff_path_group_file_name
opt_signoff_post_mask
opt_signoff_post_mask_filler_cells
opt_signoff_post_mask_logic_cells
opt_signoff_power_aware
opt_signoff_pre_sta_tcl
opt_signoff_read_eco_opt_db
opt_signoff_resize_inst
opt_signoff_retime_depth
opt_signoff_retime_mode
opt_signoff_routing_congestion_aware
opt_signoff_select_drv_pin_file
opt_signoff_select_setup_endpoints
opt_signoff_set_setup_endpoints_margin
opt_signoff_setup_target_slack
opt_signoff_setup_xtalk_delta_threshold
oa_display_resource_file
oa_drc_fill_purpose
oa_full_layer_list
oa_hybrid_rows
oa_lib_create_mode
oa_logic_only_import
oa_new_lib_compress_level
oa_pin_purpose
oa_quick_abstract_for_custom_cells
oa_silently_ignore_unsupported_vias
oa_tie_net
oa_use_virtuoso_bindkey
oa_view_sub_type
oa_write_mask_data_locked
oa_write_relative_path
opt_signoff_add_inst
opt_signoff_allow_multiple_incremental
opt_signoff_along_route_buffering
opt_signoff_check_type
opt_signoff_clock_max_level
opt_signoff_delete_inst
opt_signoff_dont_touch_partition_list
opt_signoff_eco_file_prefix
opt_signoff_enable_high_capacity_eco
opt_signoff_fix_clock_drv
opt_signoff_fix_glitch
opt_signoff_fix_hold_allow_setup_optimization
opt_signoff_fix_hold_variation_aware
opt_signoff_fix_ir_drop
opt_signoff_fix_max_transition
opt_signoff_fix_si_slew
opt_signoff_fix_slack_in_range_upper_limit
opt_signoff_fix_xtalk_preserve_hold
opt_signoff_hold_target_slack
opt_signoff_hold_xtalk_delta_threshold
opt_signoff_ignore_drv_checks
opt_signoff_include_setup_path_groups
opt_signoff_inter_partition_exclusive
opt_signoff_keep_tmp_files
opt_signoff_load_cell_list
opt_signoff_max_local_density
opt_signoff_max_paths
opt_signoff_max_slack
opt_signoff_nworst
opt_signoff_optimize_replicated_modules
opt_signoff_partition_list_file
opt_signoff_pba_effort
opt_signoff_post_mask_decap_cells
opt_signoff_post_mask_gate_array_filler_cells
opt_signoff_post_sta_tcl
opt_signoff_power_opt_focus
opt_signoff_prefix
opt_signoff_read_smart_mmmc_db
opt_signoff_retime
opt_signoff_retime_depth_early
opt_signoff_route_congestion_threshold
opt_signoff_select_drv_net_file
opt_signoff_select_hold_endpoints
opt_signoff_set_hold_endpoints_margin
opt_signoff_setup_recovery
opt_signoff_setup_target_slack_per_view
opt_signoff_setup_xtalk_slack_threshold

opt_signoff_skew_core_only
opt_signoff_skip_drv_net_file
opt_signoff_smart_mmmc_effort
opt_signoff_smart_mmmc_force_keep_hold_view_list
opt_signoff_smart_mmmc_force_keep_setup_view_list
opt_signoff_smart_mmmc_max_view_count
opt_signoff_smart_mmmc_reduce_clients
opt_signoff_swap_inst
opt_signoff_touch_partition_list
opt_signoff_upsize_in_power_opt
opt_signoff_verbose
opt_signoff_write_eco_opt_db

P

parse_proc_arguments
place_3d_use_existing_bump_tsv
place_cell_edge_spacing
place_check_report_fixed_inst_on_blockage
place_design_enable_3d
place_design_refine_macro
place_detail_allow_border_pin_abut
place_detail_allow_single_height_row_symmetry_x
place_detail_check_route
place_detail_color_aware_legal
place_detail_context_aware_legal
place_detail_honor_inst_pad
place_detail_iraware_max_drive_strength
place_detail_m3_stripe_shrink
place_detail_pad_fixed_insts
place_detail_pgftv_insertion_cell_list
place_detail_swap_eeq_cells
place_detail_use_diffusion_transition_fill
place_detail_use_gate_array_filler_groups
place_global_adaptive_subd1
pop_snapshot_stack
power_add_simulation
power_analysis_temperature
power_average_rise_fall_cap
power_block_independent_peakpower
power_bulk_pins
power_capacity
power_clock_source_as_clock
power_comprehensive_automapping
power_compress_reports
power_constant_override
power_corner
power_create_driver_db
power_current_generation_method
power_db_name
power_decap_cell_list
power_default_frequency
power_default_slew
power_default_supply_voltage
power_detailed_view_current_only
power_disable_clock_gate_clipping
power_disable_leakage_scaling
power_disable_static
power_distribute_switching_power
power_distributed_setup
power_domain_based_clipping
power_dynamic_glitch_filter
power_dynamic_vectorless_ranking_methods
power_enable_auto_mapping
power_enable_disk_mapping
power_enable_duty_propagation_with_global_activity
power_enable_dynamic_current_slew_load_interpolation
power_enable_dynamic_scaling
power_enable_flop_state_propagation
power_enable_force_propagation
power_enable_generated_clock
power_enable_input_net_power
power_enable_mt_state_propagation
power_enable_multithread_reports
power_enable_pba_for_tempus_pi
power_enable_power_target_flow
power_enable_rtl_dynamic_vector_based
power_enable_scan_report
power_enable_single_cycle_scheduling
power_enable_slew_based_ccs_pin_cap
power_enable_stable_clock_gating
power_enable_stable_flop_scheduling
power_enable_state_propagation
power_enable_tempus_pi
power_enable_xp
power_enhanced_blackbox_avg
power_enhanced_blackbox_max
power_equivalent_annotation
power_event_based_leakage_power
power_external_load_config_file
power_extraction_tech_file
power_fanout_limit
power_from_x_transition_factor
power_from_z_transition_factor
power_generate_current_for_rail
power_generate_leakage_power_map_based_on_calculated_leakage
power_generate_static_report_from_state_propagation
power_grid_libraries
power_handle_glitch
power_handle_transport_glitch
power_handle_tristate
power_hier_delimiter
power_honor_combinational_logic_on_clock_net
power_honor_negative_energy
power_honor_net_activity
power_hybrid_analysis
power_ignore_control_signals
power_ignore_data_phase_for_clock
power_ignore_end_toggles_in_profile
power_ignore_glitches_at_same_time_stamp
power_ignore_inout_pin_cap
power_ignore_macro_leakage_scale_for_temperature
power_ignore_unconstraint_net_with_timing_data
power_include_file
power_include_initial_x_transitions
power_include_sequential_clock_pin_power
power_intent_allow_back_to_back_isolation
power_intent_allow_nested_default_domain
power_intent_allow_power_domain_min_gap_zero
power_intent_allow_shifter_voltage_mismatch
power_intent_assume_iso_enable_pin_is_always_on
power_intent_check_all_nets_for_domain_crossing
power_intent_do_not_use_top_domain_for_port_voltage

power_intent_honor_power_domain_for_domain_crossing_route
power_intent_honor_power_domain_for_intra_domain_route
power_intent_include_dot_lib_related_pg_pin
power_intent_use_cpf_global_connect_for_always_on_buffer
power_intent_use_cpf_global_connect_for_shifter
power_intent_use_effective_domain_for_iso_shifter_insertion
power_ir_derated_timing_view
power_leakage_scale_factor_for_temperature
power_lib_files
power_library_preference
power_macro_toggle_pin_percentage
power_match_state_for_logic_x
power_memory_current_scaling_method
power_merge_switched_net_currents
power_method
power_min_leaf_count
power_multi_scenario_simulation
power_multibit_flop_toggle_behavior
power_multibit_flop_toggle_pin_percentage
power_off_pg_bulk_leakage
power_off_pg_nets
power_output_current_data_prefix
power_output_dir
power_partition_count
power_partition_twf
power_pin_based_twf
power_pre_simulation_empty_period
power_pre_simulation_period
power_pre_simulation_power_exclude_period
power_precision
power_quit_on_activity_coverage_threshold
power_read_rcdb
power_relax_arc_match
power_report_black_boxes
power_report_clock_instance_switching_info
power_report_idle_instances
power_report_instance_switching_info
power_report_instance_switching_list
power_report_library_usage
power_report_missing_bulk_connectivity
power_report_missing_input
power_report_missing_nets
power_report_scan_chain_statistics
power_report_statistics
power_report_time_display_fraction_digits
power_report_twf_attributes
power_reuse_flop_ranking_data
power_scale_to_sdc_clock_frequency
power_scan_chain_activity
power_scan_chain_name_pattern
power_scan_control_file
power_scan_multi_bit_flop_chain_type
power_settling_buffer
power_show_cell_usage_stats
power_smart_window
power_split_bus_power
power_start_time_alignment
power_state_dependent_leakage
power_stateprop_ignore_unannot_pins
power_static_multi_mode_scenario_file
power_static_netlist
power_switching_power_on_rise_only
power_thermal_input_file
power_to_x_transition_factor
power_to_z_transition_factor
power_transition_factor_based_duty
power_transition_time_method
power_twf_delay_annotation
power_twf_load_cap
power_unified_power_switch_flow
power_use_cell_leakage_power_density
power_use_fastest_clock_for_dynamic_scheduling
power_use_lef_for_missing_cells
power_use_only_ddv_current
power_use_physical_partition
power_use_twf_stable_randomized_arrival_delay
power_use_zero_delay_vector_file
power_vector_based_multithread
power_vector_profile_mode
power_worst_case_vector_activity
power_worst_step_size
power_worst_window_count
power_worst_window_coverage
power_worst_window_reports
power_worst_window_size
power_worst_window_type
power_write_bbox
power_write_db
power_write_default_pti_files
power_write_dynamic_currents
power_write_profiling_db
power_write_simulation_db
power_write_static_currents
power_x_count_transition_using_3_states
power_x_transition_factor
power_z_transition_factor
power_zero_delay_vector_toggle_shift
print_message
program_major_version
program_name
program_short_name
program_version
propagate_activity
push_snapshot_stack
puts

Q

query_objects
query_power_data

R

range_collection
read_activity_map_file
read_boundary_model
read_db
read_db_directory
read_db_file_check
read_db_stop_at_design_in_memory
read_db_tool_name

read_db_version
read_def
read_eco
read_flow
read_hierarchical_def
read_ilm
read_instance_voltage
read_ipd_scope
read_libs
read_markers
read_metric
read_metric_config
read_metric_html_package
read_mmmc
read_netlist
read_oa
read_partition
read_path_categories
read_path_descriptions
read_physical
read_physical_check_antenna
read_physical_check_mask_shifts
read_physical_context_data
read_physical_extend_polygon_cell_shapes
read_physical_merge_multi_port_to_single_port
read_physical_must_join_all_ports
read_power_intent
read_power_rail_results
read_rcdb
read_sdc
read_sdf
read_spef
read_temperature
read_timing_context
read_timing_debug_report
read_twf
redirect
register_trigger
related documents, list of
relink_db_files
remove_from_collection
rename_obj
rename_snapshot
reportDelayCalculation
report_aging_activity
report_aging_stress_profile
report_analysis_coverage
report_analysis_summary
report_analysis_views
report_annotated_assertions
report_annotated_check
report_annotated_delay
report_annotated_parasitics
report_aocv_derate
report_area
report_boundary_model_constraints
report_bundled_bus
report_case_analysis
report_cell_edge_type
report_cell_instance_timing
report_clock_gating_check
report_clock_network_transitions
report_clock_sense
report_clock_timing
report_clocks
report_congestion
report_constraint
report_cppr
report_critical_insts
report_critical_nets
report_critical_pins
report_dangling_ports
report_density_map
report_design
report_disk
report_fanin
report_fanout
report_floating_nets
report_flow
report_freq_violation
report_glitch_thresholds
report_hidden_usage
report_ilm_status
report_inactive_arcs
report_inst_power
report_instance_cdb
report_instance_library
report_ipd_configuration
report_ipd_crossings
report_isolation
report_jitter
report_lib_arcs
report_markers
report_messages
report_metal_stack
report_metric
report_min_pulse_width
report_mode
report_mutex_condition
report_net
report_net_parasitics
report_net_wires
report_netlist_statistics
report_noise
report_noise_lib_pin
report_noise_propagation
report_obj
report_obj_display_limit
report_object_property_names
report_parameter
report_parasitic_coordinate_transforms
report_partition_boundary_ports
report_path_groups
report_pg_net_group
report_pin_density_map
report_ports
report_power_domains
report_power_rail_results
report_property
report_ptm_model
report_rcdb
report_reference
report_resource
report_shifters

report_slack_histogram
report_socv_library
report_timing
report_timing_derate
report_timing_model
report_timing_summary
report_unit_parasitics
report_voltage
report_voltage_swing
reset_annotated_check
reset_annotated_transition
reset_case_analysis
reset_clock_exclusivity
reset_clock_groups
reset_clock_sense
reset_clock_tree_latency
reset_data_check
reset_disable_clock_gating_check
reset_drive
reset_flow
reset_glitch_derate
reset_ideal_network
reset_ilm
reset_instance_library
reset_max_capacitance
reset_max_time_borrow
reset_metric_config
reset_min_capacitance
reset_min_pulse_width
reset_mode
reset_output_delay
reset_path_adjust_group
reset_path_group
reset_pll_timing
reset_propagated_clock
reset_pulse_clock_max_transition
reset_pulse_clock_min_transition
reset_resistance
reset_sdf_assertions
reset_servers
reset_wire_load_mode
reset_wire_load_selection_group
route_special_allow_non_preferred_direction_route
route_special_block_pin_connect_ring_pin_corners
route_special_connect_broken_core_pin
route_special_core_pin_length
route_special_core_pin_max_via_scale
route_special_core_pin_refer_to_follow_pin
route_special_core_pin_site_rail_width
route_special_core_pin_stop_route
route_special_extend_nearest_target
route_special_layer_non_preferred_direction_cost
route_special_pad_pin_min_via_size
route_special_pad_pin_split_from_center
route_special_pg_pin_as_signal
route_special_secondary_pin_rail_width
route_special_split_long_via
route_special_target_search_distance
route_special_via_connect_to_shape
route_special_welltap_as_endcap
run_metric_category
report_socv_interconnect_variation
report_statistical_timing_derate_factors
report_timing_context_clock_mapping
report_timing_lib
report_timing_robustness
report_unannotated_nets
report_via_def
report_voltage_scaling
report_wire_load
reset_annotated_delay
reset_aocv_stage_weight
reset_clock
reset_clock_gating_check
reset_clock_latency
reset_clock_transition
reset_clock_uncertainty
reset_db
reset_disable_timing
reset_driving_cell
reset_generated_clock
reset_ideal_latency
reset_ideal_network
reset_input_delay
reset_load
reset_max_fanout
reset_max_transition
reset_metric_run_id
reset_min_fanout
reset_min_transition
reset_noise_lib_pin
reset_path_adjust
reset_path_exception
reset_physical_context_data
reset_power_activity
reset_property
reset_pulse_clock_max_width
reset_pulse_clock_min_width
reset_sdc
reset_sense
reset_timing_derate
reset_wire_load_model
resume
route_special_avoid_over_core_row_layer
route_special_block_pin_route_with_pin_width
route_special_core_pin_ignore_obs
route_special_core_pin_length_as_inst
route_special_core_pin_merge_limit
route_special_core_pin_reference_macro
route_special_core_pin_snap_to
route_special_endcap_as_core
route_special_jog_threshold_ratio
route_special_layer_preferred_direction_cost
route_special_pad_pin_split
route_special_pad_ring_use_lef
route_special_secondary_pin_max_gap
route_special_signal_pin_as_pg
route_special_target_number
route_special_time_limit
route_special_via_through_to_closest_ring
run_flow

S

save_ptm_model
script_search_path
select_obj
set_active_clocks
set_aging_activity
set_analysis_view
set_annotated_delay
set_annotated_transition
set_aocv_stage_weight
set_boundary_model_clock_mapping
set_case_analysis
set_cell_edge_spacing
set_cell_power_domain
set_clock_gating_check
set_clock_sense
set_clock_uncertainty
set_compress_level
set_data_check
set_db_verbose
set_default_view
set_disable_timing
set_distributed_hosts
set_dont_touch
set_dont_use
set_driving_cell
set_false_path
set_flow_config
set_flowkit_write_db_args
set_ideal_latency
set_ideal_transition
set_ilm_type
set_input_transition
set_instance_temperature_file
set_io_thresholds
set_layer_preference
set_library_unit
set_limited_access_feature
set_logic_one
set_max_cap_per_freq
set_max_capacitance
set_max_fanout
set_max_transition
set_message
set_metric_alias
set_min_capacitance
set_min_fanout
set_min_transition
set_model_priority
set_multi_cpu_usage
set_multicycle_path
set_noise_lib_pin
set_object_color
set_outbound_report
set_path_adjust
set_power_def_files
set_power_lib_files
set_preference
set_propagated_clock
set_ptm_design_rule
set_ptm_port_drive
schedule_flow
select_boundary_model_constraints
selected
set_advanced_variation_mode
set_aging_analysis_mode
set_annotated_check
set_annotated_glitch
set_aocv_interface_path_offset
set_aocv_thresholds
set_boundary_model_view_mapping
set_cdb_binding
set_cell_edge_type
set_clock_exclusivity
set_clock_groups
set_clock_transition
set_clock_value
set_compress_level
set_db
set_db_verbose
set_disable_clock_gating_check
set_distribute_mmmc_mode
set_distributed_variables
set_dont_touch_network
set_drive
set_dynamic_power_simulation
set_feature
set_flowkit_read_db_args
set_glitch_derate
set_ideal_network
set_ilm
set_input_delay
set_instance_library
set_interactive_constraint_modes
set_ipd_rail_voltage
set_lib_binding_by_voltage
set_license_check
set_load
set_logic_zero
set_max_cap_per_freq_transition
set_max_delay
set_max_time_borrow
set_max_transition_per_freq
set_metric
set_metric_header
set_min_delay
set_min_pulse_width
set_mode
set_module_view
set_multi_input_switching_mode
set_mutex_attacker
set_normalized_driver_waveform_lib
set_op_cond
set_output_delay
set_path_adjust_group
set_power_include_file
set_power_output_dir
set_proc_verbose
set_property
set_ptm_global_parameter
set_ptm_port_load

set_pulse_clock_max_transition
set_pulse_clock_max_width
set_pulse_clock_min_transition
set_pulse_clock_min_width
set_quiet_aggressor
set_resistance
set_selectively_expanded_modules
set_selectively_expanded_modules
set_sense
set_socv_constraint_config
set_socv_rc_variation_factor
set_socv_reporting_nsigma_multiplier
set_table_style
set_threshold_voltage_group
set_timing_context_clock_mapping
set_timing_context_view_mapping
set_timing_derate
set_timing_library
set_twf_attribute
set_value
set_via_pillars
set_via_variation_mode
set_virtual_clock_network_parameters
set_vt_skew_config
set_vt_skew_derate
set_wire_load_mode
set_wire_load_model
set_wire_load_selection_group
sizeof_collection
soft_stack_size_limit
sort_collection
sort_db
source
source_continue_on_error
source_echo_filename
source_verbose
source_verbose_line_length_limit
summaryReport
suspend
switch_top_cell

T

tcl_error_info
tcl_report_header_with_value
tech_db_units
tech_finfet_grid_direction
tech_finfet_grid_offset
tech_finfet_grid_pitch
tech_inst_mask_shift_layers
tech_mfg_grid
tempus
time_info
timing_all_registers_filter_clock_pins_by_clock
timing_all_registers_identify_macros_include_is_macro_cell
timing_all_registers_include_icg_cells
timing_allow_input_delay_on_clock_source
timing_analysis_aging
timing_analysis_aocv
timing_analysis_async_checks
timing_analysis_check_type
timing_analysis_clock_gating
timing_analysis_clock_propagation_mode
timing_analysis_clock_source_paths
timing_analysis_cppr
timing_analysis_enable_inter_power_domain
timing_analysis_enable_robustness_mode
timing_analysis_engine
timing_analysis_graph_pba_mode
timing_analysis_multi_input_switching_mode
timing_analysis_precision_ps
timing_analysis_self_loops_paths_no_skew
timing_analysis_socv
timing_analysis_type
timing_aocv_analysis_mode
timing_aocv_chip_size
timing_aocv_core_size
timing_aocv_derate_mode
timing_aocv_stage_count_update_on_timing_reset
timing_apply_check_derate_to_external_output_delay
timing_apply_default_primary_input_assertion
timing_apply_exceptions_to_data_check_related_pin
timing_apply_setup_hold_exceptions_to_data_check_related_pin
timing_cap_unit
timing_case_analysis_for_icg_propagation
timing_case_analysis_for_sequential_propagation
timing_case_analysis_propagation
timing_check_timing_report_all_checks
timing_check_timing_signal_level_high_to_low_threshold
timing_check_timing_signal_level_low_to_high_threshold
timing_clock_phase_propagation
timing_clock_source_paths_unconstrained_mark_clock_used_as_data
timing_clock_source_use_driving_cell
timing_clock_uncertainty_from_to_precedence
timing_collection_result_display_limit
timing_collection_variable_assignment_compatibility
timing_constraint_disable_min_max_input_delay_worst_casing
timing_constraint_enable_detailed_report_invalid_begin_end_points
timing_constraint_enable_logging
timing_constraint_enable_report_invalid_begin_end_points
timing_constraint_enable_search_path
timing_constraint_enable_separate_multicycle_data_checks
timing_constraint_load_minimal_set_for_automated_eco
timing_constraint_path_delay_exclude_check_delay_from_ignore_clock_latency
timing_constraint_path_delay_exclude_io_delay_from_ignore_clock_latency
timing_constraint_path_delay_exclude_unconstrained_endpoints
timing_constraint_path_delay_include_clock_pin_endpoints
timing_constraint_warn_for_timing_derate_exceeding_max_limit
timing_context_apply_default_primary_input_assertion
timing_context_apply_port_sdc_exceptions
timing_context_continue_on_invalid_module_list
timing_context_port_based_clock_mapping
timing_continue_on_error
timing_cppr_opposite_edge_mean_scale_factor
timing_cppr_opposite_edge_sigma_scale_factor
timing_cppr_propagate_thru_latches
timing_cppr_remove_clock_to_data_pessimism
timing_cppr_self_loop_mode
timing_cppr_skip_clock_reconvergence
timing_cppr_skip_clock_reconvergence_for_unmatched_clocks
timing_cppr_threshold_ps
timing_cppr_transition_sense
timing_create_clock_default_propagated

timing_default_opcond_per_lib
timing_derate_aocv_dynamic_delays
timing_derate_dynamic_compatibility
timing_derate_incremental_multiply_accumulative_mode
timing_derate_ocv_reference_point
timing_derate_voltage_scaling_mode
timing_disable_clock_period_checks
timing_disable_drv_report_on_constant_nets
timing_disable_genclk_combinational_blocking
timing_disable_inout_output_side_timing_checks
timing_disable_internal_inout_net_arcs
timing_disable_library_data_to_data_checks
timing_disable_netlist_constants
timing_disable_non_sequential_checks
timing_disable_pulse_width_same_edge_si_cppr_mode
timing_disable_retime_clock_path_slew_propagation
timing_disable_skew_checks
timing_disable_timing_model_latch_inferencing
timing_disable_user_data_to_data_checks
timing_drv_analysis_ignore_latency_phases
timing_enable_boundary_model_eco
timing_enable_clock_phase_based_rise_fall_derating
timing_enable_db_attribute_query
timing_enable_early_late_data_slews_for_setuphold_mode_checks
timing_enable_genclk_source_path_register_limit
timing_enable_get_obj_escaped_name_backward_compatible
timing_enable_hierarchical_get_nets_support
timing_enable_latch_thru_mode
timing_enable_mmmc_loop_breaking
timing_enable_multi_drive_net_reduction_with_assertions
timing_enable_multicycle_data_check_compatibility
timing_enable_pessimistic_cppr_for_reconvergent_clock_paths
timing_enable_preset_clear_arcs
timing_enable_si_cppr
timing_enable_spatial_derate_mode
timing_enable_tristate_clock_gating
timing_enable_uncertainty_for_pulse_width_checks
timing_enable_wire_variation
timing_extract_model_aocv_mode
timing_extract_model_consider_design_level_drv
timing_extract_model_enable_output_pin_attributes
timing_extract_model_exhaustive_validation_dir
timing_extract_model_gating_as_nochange_arc
timing_extract_model_ideal_clock_latency_arc
timing_extract_model_include_applied_slew_in_characterization_range
timing_extract_model_slew_propagation_mode
timing_extract_model_write_clock_checks_as_arc
timing_extract_model_write_function_for_combinational_arcs
timing_full_context_flow
timing_generated_clocks_allow_nested_assertions
timing_get_of_objects_hier_compatibility
timing_hierarchical_context_continue_on_top_block_clock_mismatch
timing_ignore_lumped_rc_assertions
timing_ipd_allow_paths_for_asynchronous_clocks
timing_library_convert_async_setuphold_to_recrem
timing_library_hold_constraint_corner_sigma_multiplier
timing_library_ignore_combinational_arc_mismatch_tables
timing_library_infer_cap_range_from_ccs_receiver_model
timing_library_infer_socv_from_aocv
timing_library_read_ccs_noise_data
timing_library_read_without_power
timing_library_scale_aocv_to_socv_to_n_sigma
timing_defer_mmmc_obj_updates
timing_derate_aocv_reference_point
timing_derate_incremental_adjust_additive_mode
timing_derate_negative_delay_backward_compatibility
timing_derate_spatial_distance_unit
timing_disable_bus_contention_check
timing_disable_constant_propagation_for_sequential_cells
timing_disable_floating_bus_check
timing_disable_inferred_clock_gating_checks
timing_disable_internal_inout_cell_paths
timing_disable_lib_pulse_width_checks
timing_disable_library_tieoffs
timing_disable_nochange_checks
timing_disable_output_as_clock_port
timing_disable_report_header_info
timing_disable_sdf_retain_arc_merging
timing_disable_test_signal_arc
timing_disable_tristate_disable_arcs
timing_driving_cell_override_library
timing_enable_aocv_slack_based
timing_enable_case_analysis_conflict_warning
timing_enable_data_through_clock_gating
timing_enable_derating_for_pulse_width_checks
timing_enable_genclk_divide_by_inherit_parent_duty_cycle
timing_enable_generated_clock_edge_based_source_latency
timing_enable_hier_context_unmapped_clock_analysis
timing_enable_individual_distance_inter_die_analysis
timing_enable_latency_through_clock_gating
timing_enable_model_constraint_warnings
timing_enable_multi_frequency_latch_analysis
timing_enable_path_delay_to_unconstrained_endpoints_compatibility
timing_enable_power_ground_constants
timing_enable_pulse_latch
timing_enable_simultaneous_setup_hold_mode
timing_enable_timing_window_pessimism_removal
timing_enable_uncertainty_for_clock_checks
timing_enable_vtskew_derate_mode
timing_enable_zero_delay_analysis_mode
timing_extract_model_case_analysis_in_library
timing_extract_model_disable_cycle_adjustment
timing_extract_model_enable_unconstrained_port_modelling
timing_extract_model_exhaustive_validation_mode
timing_extract_model_half_cycle_non_interface_paths
timing_extract_model_include_applied_load_in_characterization_range
timing_extract_model_non_borrowing_latch_path_as_setup
timing_extract_model_synchronous_clocks_as_skew_checks
timing_extract_model_write_clock_checks_as_scalar_tables
timing_extract_model_write_min_max_clock_tree_path
timing_generate_normalized_driver_waveform
timing_generated_clocks_inherit_ideal_latency
timing_hier_obj_name_compatibility
timing_ignore_clock_based_path_delay_on_data_checks
timing_io_use_clock_network_latency
timing_library_build_async_deassert_arc
timing_library_generated_clock_use_group_name
timing_library_hold_sigma_multiplier
timing_library_infer_async_pins_from_timing_arcs
timing_library_infer_cap_range_from_ecsm_receiver_model
timing_library_interpolate_drv_values
timing_library_read_without_ecsm
timing_library_read_without_sensitivity
timing_library_setup_constraint_corner_sigma_multiplier

timing_library_setup_sigma_multiplier
timing_library_use_two_piece_receiver_cap
timing_max_transition_use_si_transition
timing_multi_frequency_clock_rounding_factor
timing_normalized_driver_waveform_weight_factor
timing_null_collection_return_compatibility
timing_path_based_enable_exhaustive_depth_bounded_by_gba
timing_path_based_enable_verbose_mode
timing_path_based_exhaustive_max_paths_limit
timing_pba_exhaustive_path_nworst_limit
timing_propagate_latch_data_uncertainty
timing_property_return_null_collection_with_quiet
timing_rail_swing_checks_low_voltage_threshold
timing_recompute_sdf_in_setuphold_mode
timing_reduce_multi_drive_net_arcs_threshold
timing_report_clock_pin_as_begin_point
timing_report_constraint_extended_cell_em_flow
timing_report_constraint_report_all_drv_violation
timing_report_default_frequency_for_unconstrained_nets
timing_report_drv_enable_clock_source_as_clock
timing_report_drv_enable_ghz_notation_for_drv_fields
timing_report_drv_per_frequency
timing_report_drv_use_worst_timing_slack
timing_report_enable_em_index_clipping_report
timing_report_enable_markers
timing_report_enable_max_path_limit_warning
timing_report_enable_report_clock_timing_across_clock_pin
timing_report_enable_unique_pins_multiple_capture_clock_paths
timing_report_enable_write_twf_third_party_for_input_pins
timing_report_generated_clock_info
timing_report_limit_max_path_count
timing_report_precision
timing_report_pulse_width_matching_launch_capture_paths
timing_report_retime_formatting_mode
timing_report_skip_icg_from_all_fanin_fanout_collection
timing_report_timing_header_detail_info
timing_report_unconstrained_paths
timing_report_use_worst_parallel_cell_arc
timing_scaling_for_negative_checks
timing_sdf_adjust_negative_setuphold
timing_self_loop_paths_no_skew_max_depth
timing_set_clock_source_to_output_as_data
timing_socv_rc_variation_mode
timing_socv_view_based_nsigma_multiplier_mode
timing_spatial_derate_distance_mode
timing_suppress_ilm_constraint_mismatches
timing_use_clock_pin_attribute_for_clock_net_marking
timing_use_latch_early_launch_edge
timing_use_verilog_for_model_netlist
timing_waveform_aware_pulse_width_checks_low_voltage_level
timing_library_term_voltage_from_lib_pin
timing_library_zero_negative_timing_check_arcs
timing_model_netlist_exclude_dummy
timing_normalized_driver_waveform_clip_linear_part
timing_nsigma_multiplier
timing_path_based_enable_bounding_box_for_io_paths
timing_path_based_enable_report_launch_clock_path
timing_path_based_exhaustive_enable_design_coverage
timing_path_based_ipba_endpoints_max_limit
timing_prefix_module_name_with_library_generated_clock
timing_property_arrival_window_enable_tcl_dict_format
timing_rail_swing_checks_high_voltage_threshold
timing_rcdb_allow_mismatch_option
timing_reduce_multi_drive_net_arcs
timing_report_analysis_summary_csv_extended_new_format
timing_report_constraint_enable_extended_drv_format
timing_report_constraint_format
timing_report_constraint_use_infinity_slack_for_unconstrained
timing_report_disable_max_paths_per_group
timing_report_drv_enable_frequency_per_view
timing_report_drv_enable_slew_threshold_scaling
timing_report_drv_per_frequency_per_input_slew
timing_report_enable_cppr_point
timing_report_enable_flag_field_symbols
timing_report_enable_max_capacitance_drv_for_constant_nets
timing_report_enable_qpin_as_start_for_begin_end_pair
timing_report_enable_si_debug
timing_report_enable_verbose_ssta_mode
timing_report_fields
timing_report_group_based_mode
timing_report_max_transition_check_using_nsigma_slew
timing_report_property_fastest_clock_consider_data_phase
timing_report_redirect_message_types
timing_report_skip_constraint_loop_check
timing_report_socv_summary_mean_sigma
timing_report_unconstrained_path_early_late_header
timing_report_use_receiver_model_capacitance
timing_resolve_driver_conflicts
timing_scaling_for_negative_delays
timing_sdf_enable_setuphold_scond_ccond
timing_self_loop_paths_no_skew_max_slack
timing_socv_preserve_variation_with_annotations
timing_socv_sigma_spatial_multiplier
timing_spatial_derate_chip_size
timing_suppress_escape_characters
timing_time_unit
timing_use_incremental_si_transition
timing_use_latch_time_borrow
timing_waveform_aware_pulse_width_checks_high_voltage_level
timing_write_sdf_no_escape_backslash

U

ui_precision
ui_precision_capacitance
ui_precision_derating
ui_precision_power
ui_precision_sensitivities
ui_precision_timing
unflatten_ilm
uniquify
uniquify_filename
unlock_oa_design
update_analysis_view
update_constraint_mode
update_delay_corner
update_glitch
update_inst
update_inst_name
update_io_latency
update_library_set
update_metric
update_metric_definition

update_model_context
update_oa_lib
update_opcond
update_partition
update_power_domain
update_rc_corner
update_timing
update_timing_condition

V

view_last
view_log
view_next
vt_swap_out_dir
vt_swap_out_file
vt_swap_to_fix_filler_violations

W

write_annotated_transition
write_boundary_model
write_category_summary
write_db
write_db_auto_save_user_globals
write_db_create_read_file
write_db_include_metal_fill_rules
write_db_restrict_under_batch_mode
write_def
write_def_lef_out_version
write_eco
write_eco_opt_mode
write_exclude_nets
write_flow
write_flow_template
write_global_slack_report
write_ilm
write_ipd_scope
write_ldb
write_lef_library
write_load
write_metric
write_metric_config
write_ml_design
write_ml_library
write_ml_timing_graph
write_ml_timing_path
write_mmmc
write_model_context
write_netlist_full_pin_out
write_netlist_port_association_style
write_noise_eco
write_oa_techfile
write_parasitics
write_path_categories
write_path_descriptions
write_path_list_summary
write_physical_context_data
write_power_constraints
write_power_intent
write_preserves
write_rcdb
write_sdc
write_sdf
write_set_load
write_spice_deck
write_tcf
write_text_timing_report
write_timing_context
write_timing_model
write_timing_windows
write_to_gif
write_twf
write_via_defs

Z

zoomTo
zoom_highlighted_timing_path